



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

ELECTRÓNICA DIGITAL. UN LIBRO PARA SU ESTUDIO

Tomás Pollán Santamaría
Departamento de Ingeniería Electrónica y Comunicaciones
Universidad de Zaragoza. tpollan@unizar.es

RESUMEN

Prensas Universitarias de Zaragoza ha publicado un texto de «electrónica digital», distribuido (por razones de tamaño, precio y utilización) en tres partes: Sistemas combinacionales, Sistemas secuenciales y Microelectrónica digital. Pretende ser un libro para el estudio, para el esfuerzo individual que cada estudiante ha de realizar para «aprehender», es decir, para «hacer suyos» los conocimientos y la forma de razonar propios de esta disciplina. El resultado ha sido un manual muy «completo» y puesto al día, que procura recorrer todo el camino que lleva desde los conceptos y métodos básicos hasta los sistemas complejos, situándose en una perspectiva de diseño digital actual (en que los circuitos se realizan, en tamaño «micro», sobre dispositivos programables o ASICs). Particular interés tienen capítulos no habituales, como los referentes al problema del ruido, al significado y requisitos del sincronismo, a la modulación en anchura de pulsos, a VHDL, al test de fabricación,...

1. PRESENTACIÓN

En octubre de 1994 fue publicada por Prensas Universitarias de Zaragoza, en la Colección de Textos Docentes, la primera edición de esta ELECTRÓNICA DIGITAL que, desde el año 1982 y en sucesivas versiones, había sido impresa en forma de apuntes por capítulos. Habiéndose agotado la primera edición, abordé la tarea de escribir un nuevo texto con la finalidad de actualizar sus contenidos (tal como requiere el acelerado y constante ritmo de evolución de esta materia y, en particular, de sus componentes electrónicos) y de mejorar en lo posible la organización y el tratamiento de los mismos.

La presente edición, por motivo de número de páginas (y, también, por razones de precio y de secuencia de utilización), se ofrece en tres tomos: el primero con el subtítulo de *sistemas combinacionales*, con el de *sistemas secuenciales* el segundo y el último relativo a la *microelectrónica digital*. No es que esta división sea realmente operativa en cuanto al diseño digital; de hecho todo sistema digital de una cierta complejidad es secuencial y todo diseño digital actual es microelectrónico. Pero de alguna forma había que dividir la materia y la división adoptada es útil en relación con la progresión en el estudio de la misma.

Este texto pretende ser un libro para el estudio tanto de la Electrónica Digital como de los Sistemas Lógicos (en el caso de los dos primeros volúmenes, el tercero es más específico de *microelectrónica*), que se pone a disposición de los estudiantes de tales asignaturas y de todas las personas interesadas en ellas. Para facilitar su utilización por parte de quienes se encuentren interesados en los sistemas lógicos, pero no en su realización electrónica, se ha procurado separar en capítulos distintos dichos aspectos.

No es un libro de consulta; existe un gran número y una amplia diversidad de libros de consulta disponibles, algunos de los cuales vienen reseñados en la bibliografía, al final del texto. Ni tampoco pretende ser una guía para la preparación de las clases por parte del profesor, aunque seguramente pueda proporcionar una ayuda eficaz en tal sentido.

El texto ha sido elaborado y contrastado con la intención de configurar un libro para el estudio personal, para el esfuerzo individual que cada estudiante ha de realizar para «aprehender», es decir, para «hacer suyos» los conocimientos y la forma de razonar propios de una materia o disciplina:

[*aprender* → *aprehendere* → *apropiarse* → *adueñarse* → *ser dueño de*]

Desde tal perspectiva se han ordenado los diversos conceptos y se ha desarrollado la exposición de los mismos, intercalando en dicha exposición numerosos ejemplos que permitan introducir, aclarar o aplicar los correspondientes conceptos. De ahí, también, la reiteración, casi repetitiva, de algunos conceptos e ideas en diversos lugares del texto, con el fin de insistir en su comprensión y asimilación; el detalle con que se analizan los ejemplos que sirven de introducción a algunos temas; la propia presentación del texto, destacando con negrita expresiones, variables y valores booleanos,...

La primera página de cada capítulo, además del índice, contiene un breve resumen y justificación de su contenido, a modo de invitación a su estudio y, también, para establecer la perspectiva y finalidad del capítulo en el contexto global de los sistemas digitales.

En las secciones 5, 6 y 7 de esta comunicación se describe, con un cierto detalle, el contenido de cada uno de los volúmenes. Además, aprovechando la disponibilidad de espacio, en la sección 2 se afirma, a modo de declaración de principios, la información como concepto referencial de todo el texto: la electrónica como técnica de manejo de la información; y, en el mismo sentido, las secciones 3 y 4 inciden en dicho «manejo de la información» en su perspectiva histórica y en su perspectiva simbólica.

Material complementario para el estudio y utilización de este texto se encontrará en la página www.unizar.es/euitiz/digital.htm: en ella se dispondrá de copia actualizada de los capítulos; ampliaciones, comentarios, observaciones y erratas; hojas de ejercicios clasificadas por temas; colecciones de transparencias; etc., que pueden ser utilizadas, copiadas y distribuidas libremente en los términos de la licencia de Documentación Libre GNU (referencia <http://gugs.sindominio.net/licencias/fdl-es.html>).

La página WEB de Prensas Universitarias de Zaragoza (editorial propia de la Universidad de Zaragoza) es: <http://puz.unizar.es/index.html>.

2. LA INFORMACIÓN COMO PIEDRA ANGULAR

Un punto de referencia básico en la redacción del texto lo constituye el concepto de información y de su manejo, la consideración finalista de los sistemas lógicos como herramienta conceptual para el procesamiento de la información y de la electrónica como técnica eficaz para el tratamiento de la información. Ha sido escrito con el deseo expreso de transmitir al conjunto del texto tal referencia básica a la información y a su procesamiento, como objeto y objetivo propio de la electrónica digital.

La electrónica es la técnica de manejo de la información más eficaz que, hoy por hoy, posee el hombre; la electrónica maneja la información codificada en señales eléctricas y utiliza, para ello, dispositivos que aprovechan las propiedades de los electrones.

Desde la perspectiva del desarrollo histórico de la electrónica, podemos identificar tres grandes áreas de aplicación en el manejo de la información:

➤ Telecomunicación: enviar la información lejos, tanto en el espacio (comunicación por ondas) como en el tiempo (almacenamiento de la información en un soporte material para reproducirla posteriormente).

- Automatización: utilizar la información para controlar procesos; para ello, aparte de las propias operaciones a efectuar sobre la información, se necesitan sensores (capaces de convertir en señales eléctricas las magnitudes físicas que afectan al proceso) y efectores (capaces de traducir las señales en acciones, en definitiva en otro tipo de magnitudes físicas).
- Informatización: procesar la información en sí misma para darle una nueva forma o para obtener nueva información a través de combinar varias informaciones.

Estos tres ámbitos de actuación sobre la información coinciden con las tareas que la electrónica ha ido abordando, sucesivamente, en su desarrollo a lo largo del siglo XX. También coinciden con las tres especialidades de la ingeniería dedicadas a la electrónica: telecomunicaciones, electrónica industrial (control de procesos) e informática.

¿Por qué la información? ¿Qué interés tiene para nosotros la información?

La información es una componente de la actividad humana; es, probablemente, la componente más intrínseca de la actividad del hombre. Junto con los materiales y la energía, la información está presente, como parte integrante y necesaria, en las diversas acciones de los humanos (incluso podemos distinguir actuaciones en las que no intervengan, externamente, los otros dos componentes, materia y energía, como es la simple reflexión o pensamiento).

Pero, además, materiales, energía e información representan escalones sucesivos en la macrohistoria socio-económica del hombre. Durante el proceso histórico del devenir humano, el hombre tuvo que ocuparse, en un primer y muy prolongado período, de los materiales que satisficieran sus necesidades, que le permitieran sobrevivir y vivir cada vez mejor, que le aportaran comodidades y, también, que le sirvieran para confeccionar útiles e instrumentos que facilitasen y aumentasen la eficacia de sus acciones.

En segundo lugar, el hombre se preocupó de que «otros» trabajasen por él, de desarrollar formas de complementar y de suplir su trabajo y su esfuerzo, hasta poder aprovechar las más diversas fuentes de energía y disponer de mecanismos que permitieran utilizar la energía externa para obtener los productos y servicios que le interesaban. Un hito relevante de este proceso de aprovechamiento de la energía lo constituye la máquina de vapor, que da lugar a un período histórico conocido como revolución industrial («mudanza en el estado de las cosas» producida por la utilización eficaz de la energía en los procesos de producción).

Hoy día, el hombre se encuentra con la posibilidad de utilizar recursos externos para manejar la información, para transmitirla, recogerla y ampliarla y, también, para utilizar, en lugar del propio hombre, la información. Lo que hasta hace poco parecía patrimonio específico del hombre, la captación, el procesamiento y la utilización eficaz de informaciones complejas en forma versátil, ha pasado a ser también dominio de las máquinas y de las técnicas. Por ello nos encontramos en un nuevo período de «mudanza en el estado de las cosas» que podemos nombrar como revolución informacional.

Precisamente, porque la información se ha «externalizado» del hombre, la hemos descubierto como concepto significativo. Hasta ahora había pasado desapercibida como parte intrínseca y consustancial a la actividad humana. La información, como otros conceptos inherentes a la actividad del hombre (el tiempo y la energía), se pone de manifiesto cuando se «exterioriza», cuando el hombre la ve fuera de sí, cuando la manejan las máquinas.

3. RECONOCER COMO HEMOS LLEGADO HASTA AQUÍ

Conviene resaltar tres aspectos que, desde el principio, distinguen a la electrónica: su carácter aplicado (no es una disciplina teórica o de «investigación básica» sino de manejo efectivo de la información), la rápida difusión de sus productos, su relación directa con la gente (con las personas comunes, más allá de los profesionales o especialistas) y, con ello, su influencia en la vida cotidiana.

Con el transistor (a mitad del siglo XX) comienza un proceso continuo de reducción de la electrónica: todo se hace más pequeño, más corto, más rápido: se reduce no sólo el tamaño, sino también el consumo y el coste; también se hace más pequeño el tiempo de respuesta de los circuitos, el tiempo de desarrollo de los sistemas y el tiempo de su difusión pública. Ello ha permitido construir y poner rápidamente en nuestras manos sistemas electrónicos cada vez más complejos y potentes, de tamaño, consumo y costes muy reducidos.

A la electrónica la minimización «le viene de familia» (el electrón es diminuto y muy veloz, en cuanto a sus efectos) y el resultado es que, al hacerse tan pequeña y tan rápida, la electrónica se ha metido por todos los rincones de nuestra vida y de nuestra sociedad y ha promovido esa «mudanza en el estado de las cosas» que caracteriza nuestro presente: la revolución informacional.

La electrónica de la primera mitad del siglo XX se dedicó a la telecomunicación, en su doble aspecto: espacial y temporal; desarrolló la radiodifusión y la grabación del sonido (en discos mecánicos, cintas magnéticas y bandas ópticas de las películas sonoras), mejoró ampliamente la telefonía e inició la transmisión de imágenes (televisión).

A partir de los años 40, la electrónica aborda el control de procesos. La penetración en la industria de los sistemas de control electrónicos se ve favorecida por la introducción de dispositivos electrónicos de control de energía y por la posibilidad de abordar tareas complejas gracias a los circuitos integrados; de forma que, a partir de los años 70, la electrónica pasa a controlar todo tipo de proceso industrial y, desde los años 80, se incorpora masivamente dentro de los productos resultantes de la fabricación industrial.

A la vez, en esta segunda mitad del siglo XX, muy poquito a poco al principio pero de forma espectacular en el último cuarto de siglo, la electrónica ha ido asumiendo otra vertiente más abstracta y genérica: operar con la información en sí misma, representarla y manejarla a través de símbolos, lo que hoy en día entendemos por procesar la información.

El camino hacia la informatización lo habían abierto dos precursores distantes entre sí:

- Georges Boole, matemático inglés, que tuvo éxito en su empeño de construir un modelo matemático del pensamiento humano (de la forma de razonar), estableciendo las bases de la lógica proposicional (la forma de combinar proposiciones), a través de una estructura matemática que, andando el tiempo, sería conocida como álgebra de Boole (*Una investigación de las leyes del pensamiento*, publicado en 1854).[1]
- Claude E. Shannon, ingeniero norteamericano, que, al desarrollar un modelo matemático para tratar con las redes de múltiples conmutadores propias de la telefonía, identifica la interconexión de conmutadores como álgebra booleana y pone de manifiesto que también lo es el sistema binario de numeración (*Un análisis simbólico de los relés y circuitos de conmutadores*, publicado en 1938).[2]

Sobre las bases conceptuales que establecen Boole y Shannon se edifica la electrónica digital (soporte instrumental del procesamiento de la información), que alcanza su mayoría de edad en los años 70, cuando los circuitos integrados permiten configurar sistemas informáticos potentes y reducir su coste, hasta llegar (en los años 80) al microprocesador que hace viables los computadores personales.

Pero la electrónica propia de la informatización (la electrónica digital) no se limita a la configuración de sistemas propiamente informáticos sino que, desde sus inicios, se dedica también al control de procesos y, en buena medida, desplaza a la electrónica anterior (analógica). El microprocesador resuelve muy eficazmente el control de procesos industriales y la integración de circuitos de aplicación específica (ASICs) permite miniaturizar controles sumamente sofisticados para el interior de los productos fabricados en tales procesos.

Asimismo, hoy en día, la electrónica digital ha invadido y renovado el ámbito de las comunicaciones y los sistemas digitales han abierto nuevas alternativas (con extraordinarias prestaciones) en cuanto a almacenamiento de sonido e imagen, en cuanto a telefonía por microondas y, también, en radio y televisión.

4. LO SIMBOLICO FRENTE A LO ANÁLOGO

Las señales eléctricas son el soporte material de la información; según la manera de codificar la información (de representarla en forma de señales eléctricas) aparecen dos tipos de electrónica: la analógica y la digital.

La analógica representa las cantidades «por analogía cuantitativa» (a mayor cantidad, mayor tensión) según una relación de proporcionalidad directa, mientras que la digital utiliza símbolos a través de un proceso de codificación abstracta.

El nombre de «analógica» deriva de que la representación se hace «por analogía»: los valores de la señal eléctrica son «análogos en cantidad» a los de la magnitud física: hay una relación directa en términos de cantidad, una relación de proporcionalidad. El nombre de «digital» le viene de que utiliza dígitos: representa la información mediante «palabras» formadas por varios dígitos, a través de una codificación: es una representación simbólica que requiere un proceso de abstracción.

Un sensor adecuado transforma directamente la correspondiente magnitud física en tensión eléctrica analógica, pero se requiere una codificación posterior para que la señal resultante de la medida sea trasladada a la palabra binaria (al conjunto de señales) que corresponde a su representación digital. Por medio hay un código que establece la relación entre cada símbolo y la cantidad de tensión analógica que representa, cantidad que, además, depende de la posición (valor relativo) del símbolo en la palabra binaria.

Es indudable que se asume una complicación, un esfuerzo adicional, al pasar de la representación analógica a la digital. En la utilización digital de símbolos hay un esfuerzo intermedio importante que no resulta obvio: la representación en dígitos requiere una transformación cualitativa, una conversión abstracta en símbolos que, según el lugar que ocupan, representan cantidades diferentes.

¿Qué ganamos con ello?

- a) Precisión: los valores, una vez expresados en símbolos, están claramente identificados con absoluta precisión; en cuanto a tensiones analógicas, al utilizar éstas todo el rango de valores de tensión, dos valores próximos tendrán dificultades para diferenciarse mientras que, en el caso digital, corresponderán a dos palabras binarias diferentes (y su expresión en tensiones empleará para cada dígito dos valores distantes).

- b) Fortaleza frente a perturbaciones (frente al «ruido» electromagnético): las tensiones digitales utilizadas corresponderán a dos valores distantes mientras que las analógicas recorren todo el rango de valores, de forma que la más mínima perturbación modificará el valor que representan.
- c) Fortaleza frente a derivas o faltas de precisión de los circuitos: al operar con las señales eléctricas cualquier etapa analógica causará un cierto grado de error (una mínima desviación de tensión o un pequeño fallo de precisión) que, al actuar en un rango continuo supondrá un error en el valor de la magnitud resultante; la separación entre los valores de tensión que corresponden a los símbolos digitales anula el efecto de tales desviaciones.
- d) Capacidad de cálculo: la representación simbólica permite utilizar los mecanismos de cálculo propios del correspondiente sistema de numeración (en el caso digital, el cálculo en el sistema binario).
- e) Capacidad de razonamiento (de combinar proposiciones): el razonamiento es propio de la representación simbólica y de la combinación de símbolos (a través de las reglas de la lógica).

La electrónica digital, al trasladarnos al mundo de los símbolos, aporta precisión y fortaleza y nos transfiere al plano de lo abstracto que es el ámbito del cálculo y del razonamiento.

En tal sentido importa reiterar que «todo esto» (es decir, el procesamiento de la información, codificada en símbolos binarios) comenzó con los trabajos de Georges Boole y de Claude E. Shannon. En su búsqueda, junto a muchos otros investigadores de su época, de un modelo para expresar matemáticamente el pensamiento humano, esto es, la forma en que el hombre razona, Georges Boole nos legó un «lenguaje formal» (la lógica proposicional) que permite combinar proposiciones y una estructura matemática (el álgebra de Boole) que soporta dicho lenguaje. Claude E. Shannon nos mostró que dicha estructura matemática también soporta los cálculos numéricos en sistema binario y, además, puede ser materializada por medio de conmutadores.

De la disponibilidad de un «lenguaje» y de un «sistema de numeración» capaces de ser ejecutados por unos «dispositivos físicos» y de la forma de configurar adecuadamente tales dispositivos para realizar las correspondientes tareas de cálculo, deducción, decisión y control trata este libro; en las tres secciones siguientes se relata el detalle de su contenido.

5. PRIMER VOLUMEN: I. SISTEMAS COMBINACIONALES [3]

La base matemática de la electrónica digital la constituye el álgebra de Boole, cuyas funciones expresan todas las correspondencias entre las variables de los sistemas digitales. Por ello, resulta adecuado comenzar por el estudio del álgebra booleana, sus operaciones y teoremas (cap. 1) y la forma de construir y simplificar las funciones booleanas (cap. 2).

Todo ello con referencia a las tres álgebras de Boole de dos elementos cuyo isomorfismo es la base de la electrónica digital (cap. 1): la lógica proposicional (lenguaje formal para razonar), el sistema binario (sistema de numeración para calcular) y el álgebra de conmutadores (componentes físicos para construir las operaciones booleanas). La diversidad de representaciones de una función booleana (cap. 2) es la base para su construcción circuital, siendo sumamente útiles los procedimientos de simplificación de la función para reducir el tamaño del circuito.

Las funciones booleanas pueden agruparse en bloques o módulos que realizan operaciones globales de interés genérico: bloques combinacionales. De un lado (cap. 3), interesan los bloques que efectúan operaciones aritméticas entre dos números binarios y, dentro de esta perspectiva numérica, interesa ampliar el código binario para representar (con los únicos dos símbolos disponibles, el 0 y el 1) números negativos y números con parte decimal.

Por otra parte (cap. 4), son útiles los bloques que facilitan la distribución de la información y la selección de posibilidades (multiplexores, demultiplexores y decodificadores) y, también, los que simplemente trasladan la misma información de un código a otro (codificadores); además, estos tipos de bloques (distribuidores y codificadores) pueden construirse mediante configuraciones reticulares de sus conexiones, lo cual simplifica en gran medida su diseño y fabricación.

Los bloques combinacionales constituyen «piezas de diseño» que facilitan la división de un sistema digital en partes y permiten configurarlo por ensamblaje de tales módulos.

Se trata siempre de construir conjuntos de funciones booleanas, lo cual se complica cuando el número de sus entradas es alto: las estructuras matriciales (cap. 5) permiten abordar tal complejidad. Existen tres estructuras conceptualmente simples ROM, PAL y PLA, que facilitan la descripción y construcción de bloques combinacionales de muchas entradas; tales configuraciones sirven, además, para conformar circuitos integrados programables, disponibles para insertar (programar) en su interior el conjunto de funciones booleanas de un codificador concreto o de un bloque combinacional específico propio de un diseño particular.

Una vez recorridos «los cimientos y el almacén de piezas» de los sistemas lógicos combinacionales (las funciones y los módulos que hacen viable su diseño), conviene recordar que la materia sobre la que trabajan es la información y que ésta se encuentra codificada en palabras binarias de «ceros» y «unos», existiendo múltiples posibilidades de codificación (cap. 6). Conviene, asimismo, tomar conciencia de la posibilidad de error (principalmente en la transmisión y en la conservación de la información) y conocer la existencia de códigos capaces de detectar e, incluso, de corregir los errores.

Hasta aquí (capítulos del 1 al 6) se ha tratado de los sistemas lógicos sin referencia a la electrónica que permite construirlos físicamente; también es preciso conocer y comprender la tecnología (y a ello van dedicados los capítulos 7, 8, 9 y 10).

Se denomina puerta lógica a la realización física de una operación booleana. Las puertas con diodos (cap. 7), además de su utilidad como puertas individuales, sirven para apreciar la necesidad de buen acoplo en tensión (requisito inexcusable para conectar una puerta lógica a la siguiente, ya que lo que se transmite es una tensión eléctrica). Por otra parte, las puertas lógicas habituales son de tipo inversor, construidas con interruptores según el álgebra de conmutadores, y el transistor NMOS es un excelente interruptor.

La combinación de interruptores de los dos tipos complementarios, utilizando transistores NMOS y PMOS, permite anular el consumo estático de las puertas lógicas y reducir su resistencia de salida, configurando puertas de características cuasi-ideales; es por ello la tecnología digital predominante: familia lógica CMOS (cap. 8).

La tecnología CMOS ofrece una muy amplia diversidad de configuraciones (cap. 9), tanto en variedad de puertas complementarias como en otros tipos de puertas para aplicaciones específicas: las puertas de transmisión facilitan la configuración de multiplexores y la desconexión (estado de alta impedancia) y las puertas pseudoNMOS permiten construir estructuras matriciales de muchas entradas y, también, bloques programables tipo ROM, PAL y PLA.

Como puede apreciarse el texto opta por los transistores MOS y, en concreto, por la tecnología CMOS como forma de realización física de los circuitos digitales; tal opción se fundamenta en dos razones:

- los transistores MOS se corresponden, casi idealmente, con los interruptores propios del álgebra de conmutadores, base conceptual para construir físicamente los sistemas digitales
- y, además, la integración CMOS es actualmente la forma habitual de realización de circuitos integrados digitales. A partir de los años 80, la tecnología CMOS relegó a un segundo plano a las tecnologías bipolares y, por sus excelentes características funcionales, se ha impuesto como la tecnología propia de la electrónica digital (siendo previsible que su actual predominio se mantenga, al menos, en las próximas dos décadas).

No obstante, el apéndice A3 presenta, en forma resumida, la configuración de las puertas lógicas con transistores bipolares y las características propias de la familia TTL; y el apartado 10.1 describe la evolución de los circuitos integrados digitales y la situación actual en cuanto a utilización de las diversas series.

Cerrando la parte referida a la realización electrónica de los sistemas lógicos, se presenta (cap. 10) la evolución histórica y el panorama general de las familias lógicas integradas, junto con las características a tener en cuenta a la hora de utilizarlas y el importante problema del «ruido» electromagnético que afecta a los circuitos digitales.

Además, el primer volumen incluye tres capítulos complementarios dedicados a aspectos tecnológicos (numerados como T1, T2 y T3).

En el primero de ellos (cap. T1) se explica, en forma conceptual, breve y sencilla pero con adecuada profundidad, el comportamiento de los semiconductores, sus dos tipos N y P y la unión PN con sus características de discriminación de polaridad y aislamiento eléctrico (cuando se encuentra en polarización negativa) y se desarrolla un modelo operativo de funcionamiento de los transistores MOS. Es un capítulo auxiliar muy útil para quienes no dominen (o, simplemente, necesiten repasar) la teoría de semiconductores.

En el segundo (cap. T2), se presenta el proceso de fabricación de circuitos integrados CMOS: las etapas básicas de dicho proceso, las diversas operaciones físico-químicas que se aplican y el detalle de los sucesivos pasos que conforman el proceso de integración. Además, se incluye un epígrafe con los datos numéricos de los parámetros físicos de las regiones que forman parte de un circuito integrado CMOS (concentraciones de impurezas y de portadores, movilidad de los mismos, resistividad, capacidad por unidad de área,...) para facilitar una idea precisa de la configuración atómica y de las propiedades eléctricas de cada zona semiconductor, conductora o aislante.

Asimismo, dentro de los aspectos tecnológicos, se analiza con mayor detenimiento el problema del «ruido» en los circuitos digitales (cap. T3), detallando las causas físicas de las interferencias electromagnéticas, los mecanismos que las producen y transmiten en relación con los circuitos digitales y las formas de evitar o reducir los efectos de tales interferencias.

Tres apéndices describen, respectivamente, el método de simplificación Quine-McCluskey (que es la base de los algoritmos informáticos de simplificación de funciones booleanas), la propagación rápida de acarreo (para evitar largos tiempos de respuesta en los sumadores de números binarios de muchos dígitos) y las puertas lógicas con transistores bipolares (abarcando tanto las puertas discretas como la familia lógica TTL).

6. SEGUNDO VOLUMEN: II. SISTEMAS SECUENCIALES [4]

Los sistemas secuenciales son sistemas digitales que incorporan memoria de su pasado; ésta se consigue mediante realimentación en las propias funciones booleanas.

La memoria presenta dos aspectos (cap. 11): la necesidad de recordar la evolución anterior del sistema y el almacenamiento de datos para su posterior utilización; en ambos casos, el biestable es la célula básica capaz de almacenar un dígito. El conjunto de variables de estado contiene la información que el sistema secuencial necesita sobre su pasado y los grafos de estado son una herramienta adecuada para representar su comportamiento.

El proceso de diseño secuencial (cap. 12), a partir del grafo de estados, de su simplificación (si es posible) y de la asignación de una palabra binaria a cada uno de ellos (codificación), consiste en dedicar un biestable a cada variable de estado y establecer las funciones que controlan dichos biestables (evolución del estado) y las funciones de las variables de salida (activación de las salidas).

El sincronismo, como división del tiempo en unidades discretas, facilita el diseño de los sistemas secuenciales y les confiere una gran seguridad de funcionamiento. Para ello es necesario introducir una señal de reloj, cuyos flancos señalarán el paso de una unidad de tiempo a la siguiente, y disponer de biestables síncronos que solamente cambian en dichos flancos (cap. 13); la combinación de biestables con funciones previas en configuración PAL da lugar a circuitos integrados programables (CPLDs), capaces de admitir la inserción (por programación) en su interior de un sistema secuencial completo.

El diseño de sistemas secuenciales con biestables síncronos (cap. 14) pasa también por establecer las funciones de evolución del estado (que actúan sobre los biestables que contienen las variables de estado) y las funciones de activación de las salidas. El diseño síncrono es intrínsecamente necesario en sistemas de procesamiento o transmisión serie y es altamente conveniente en todo tipo de diseño digital de una cierta complejidad.

El sincronismo ofrece facilidad de diseño al referirlo a unidades de tiempo discretas y numeradas pero, sobre todo, seguridad de funcionamiento para evitar los espurios (*glitches*) y errores debidos a los diferentes tiempos de propagación. Para ello es preciso comprender en profundidad su significado conceptual y los requisitos que el sincronismo impone (cap. 15); tales requisitos se reflejan sobre los biestables, en forma de condiciones de diseño y tiempos funcionales que han de ser respetados y sobre la señal de reloj, cuya verticalidad, simultaneidad y no-contaminación han de ser aseguradas.

Un registro es un conjunto de biestables y sirve para almacenar una palabra binaria (capítulos 11 y 13). Los contadores (cap. 16) son un tipo especial de registros que evolucionan entre números binarios consecutivos (pasan de un número al siguiente o viceversa) y que sirven para contar pulsos y para dividir frecuencias; aunque son circuitos secuenciales, debido a que suelen tener muchos estados (si bien muy ordenados entre sí), su diseño requiere una metodología propia.

Los contadores son bloques secuenciales sumamente útiles en el diseño de sistemas de medida y de control, debido a la gran variedad de aplicaciones que ofrecen (cap. 17): conteo y control de número de objetos y de sucesos, medida de tiempos (herramienta específica para manejar el tiempo), medida de frecuencias y de número de revoluciones,...

Una aplicación más sofisticada de los contadores, la modulación de anchura de pulsos, merece particular atención (cap. 18); la información reflejada sobre amplitudes de tensión no resulta apropiada para los métodos digitales y, en cambio, hay procedimientos digitales que facilitan un buen manejo de información expresada en anchura de pulsos de amplitud fija. Los pulsos de anchura modulada permiten realizar con procedimientos básicamente digitales tareas que, en principio, parecen más propias de la electrónica analógica como el control de potencia, la conversión digital-analógica y la recíproca de analógica a digital, la disponibilidad de potenciómetros para controlar la amplitud de las señales,...

Algunos sistemas digitales necesitan almacenar en su memoria grandes cantidades de datos, para ello los registros se agrupan en amplios bloques, denominados memorias (cap. 19); tales bloques (RAM) presentan aspectos novedosos relativos a la organización de la transferencia de la información por medio de buses (de datos, de direcciones y de control). El bus de direcciones genera un amplio campo de posiciones numeradas para situar los registros de los bloques de memoria: mapa de memoria.

La arquitectura basada en la utilización de buses permite agrupar múltiples bloques de memoria (algunos de ellos de solo lectura ROM) y, también, incorporar dentro de la memoria los periféricos de entrada y de salida (cap. 20); para ello, es preciso situar circuitualmente cada bloque en un segmento del mapa de memoria y dotar de adecuada fuerza a las señales que son transmitidas por los buses.

Además, este segundo volumen incluye dos capítulos complementarios dedicados a aspectos tecnológicos, referidos a circuitos auxiliares, complementarios de los propiamente digitales y necesarios, respectivamente, para la gestión del tiempo (cap. T4) y para la relación con el mundo exterior, básicamente analógico (cap. T5). El primero describe los circuitos temporizadores que permiten la determinación de intervalos de duración dada: osciladores para producir ondas de reloj y monostables para pulsos de anchura dada. El segundo trata de los circuitos que facilitan la comunicación de los sistemas digitales con su entorno físico analógico: conversores de información digital a analógica y viceversa, de tensiones analógicas a códigos digitales.

Dos pequeños apéndices describen los contadores asíncronos contruidos directamente con biestables T en serie, y los bloques de memoria dinámica, que ofrecen amplias capacidades de memoria pero necesitan refrescar continuamente la información almacenada.

7. TERCER VOLÚMEN: III. MICROELECTRÓNICA DIGITAL [5]

El diseño digital actual se desarrolla en forma «micro»: la microelectrónica digital alude a la configuración del circuito electrónico completo, resultante de un diseño específico, en el interior de un solo circuito integrado. Lo cual supone pasar de la interconexión de circuitos integrados estándar a la construcción de un circuito integrado específico.

Se trata de insertar nuestro propio diseño completo en un único circuito integrado y para ello disponemos de dos alternativas:

- programar nuestro diseño sobre un circuito integrado programable
- o, fabricar dicho diseño como circuito integrado específico, ASIC.

Cinco capítulos desarrollan el proceso de diseño y «construcción circuital» de un sistema digital en forma microelectrónica (que es la forma actual de llevar a cabo tal diseño). Los dos primeros tratan de las dos opciones antes indicadas para obtener un circuito integrado con un diseño específico completo (programación o fabricación del diseño); ambas opciones utilizan el mismo esquema de desarrollo del proceso de diseño, el mismo tipo de herramientas y las mismas consideraciones metodológicas a tener en cuenta.

El primer capítulo (cap. 21) presenta las diversas posibilidades de programación y fabricación que permiten obtener un circuito integrado con un diseño específico completo; ambas opciones utilizan el mismo esquema de desarrollo del proceso de diseño, el mismo tipo de herramientas y las mismas consideraciones metodológicas a tener en cuenta.

El segundo (cap. 21) repasa la configuración de macroceldas y CPLDs (dispositivos programables ya introducidos en el 2º volumen) y describe, en detalle, la configuración y posibilidades de los circuitos integrados programables de tipo avanzado (FPGAs).

El diseño microelectrónico se desarrolla habitualmente con la ayuda de un lenguaje de descripción circuital, de los cuales el más utilizado es el VHDL (cap. 23). VHDL es un lenguaje de descripción y simulación de sistemas en general y, desde el punto de vista de la microelectrónica, interesa la parte que puede ser sintetizada por compiladores digitales, es decir, que puede ser traducida a un conjunto de puertas y biestables para configurar un circuito digital.

La microelectrónica permite abordar sistemas digitales muy complejos y resulta conveniente considerar las pautas o «formas de actuar» que orienten y faciliten el diseño complejo (cap. 24); frente a lo grande y complicado la estrategia más eficaz de los limitados cerebros humanos es la división en partes, una división que no ha de perder la visión de globalidad: fraccionar con estructura.

Además, la microelectrónica exige plantear el problema del test de fabricación (cap. 25); los vectores o métodos de comprobación de que un circuito integrado se ha fabricado bien son responsabilidad del diseñador, aunque su aplicación efectiva corresponda al fabricante.

También es necesario conocer el soporte material que permite la programación o fabricación de un circuito específico, estudiar con un poco de detalle la tecnología que permite «materializar» un diseño particular. Los cuatro capítulos que (al igual que en los dos volúmenes anteriores) se agrupan bajo la denominación de «aspectos tecnológicos» pretenden presentar un modelo básico conceptual y operativo de la tecnología, que facilite la comprensión de sus posibilidades, sus prestaciones, sus limitaciones y sus exigencias (los requisitos que la tecnología impone sobre el diseño).

El capítulo T6 desarrolla un modelo funcional del comportamiento de los transistores MOS que permite una representación gráfica de la distribución de carga en el canal de dichos transistores y, sobre ella, efectuar cálculos relativos a tensiones, intensidades, tiempos y consumos. A partir de dicho modelo, se estudian en detalle las características del inversor CMOS (cap. T7), como prototipo de las puertas inversoras, y se analizan las puertas pseudoNMOS y las puertas de transmisión (cap. T8), como alternativas de interés para determinadas configuraciones; asimismo, se introduce la lógica dinámica.

El capítulo T9 se dedica a cuestiones diversas que completan el panorama de un circuito integrado global: entradas y salidas, densidad de integración, reglas de diseño físico,...

Tres apéndices incluyen, respectivamente, algunos ejemplos de diseño VHDL de sistemas de control, la presentación del simulador SPICE como herramienta de simulación eléctrica del comportamiento y características de las puertas o módulos digitales y la posibilidad de incluir etapas analógicas complementarias dentro del mismo integrado digital: ASIC mixto.

8. RUEGOS Y AGRADECIMIENTOS

El autor agradece a todos los compañeros del departamento de ingeniería electrónica y comunicaciones de la Universidad de Zaragoza sus aportaciones y su ayuda cotidiana, y a quien utilice este libro para su personal esfuerzo de estudio la confianza que deposita en su labor como profesor; asimismo, agradece a los profesores y estudiantes de la especialidad de Electrónica Industrial de la Escuela Universitaria de Ingeniería Técnica Industrial de Zaragoza la amistad, colaboración, ilusión y esfuerzo que comparten día a día.

También agradecerá efusivamente cualquier sugerencia o propuesta que contribuya a mejorar el texto, a clarificar las ideas que expresa o a facilitar su estudio y, a la vez, ruega que se le avise de cualquier error, errata o incorrección que se detecte.

9. BIBLIOGRAFÍA

[1] G. Boole, “An investigation of the laws of thought”, Walton and Maberley, London, 1854. (Reprinted by Dover Books, New York, 1954). Hay traducción española: “Investigación sobre las leyes del pensamiento”, Editorial Paraninfo, Madrid, 1982.

[2] Claude E. Shannon, “A symbolic analysis of relay and switching circuits”, Transactions American Institute of Electrical Engineers, vol. 57, pp. 713-723, March 1938.

[3] T. Pollán, “Electrónica Digital. I. Sistemas Combinacionales”, Prensas Universitarias de Zaragoza, Colección Textos Docentes nº 97, Universidad de Zaragoza, 2003.

[4] T. Pollán, “Electrónica Digital. II. Sistemas Secuenciales”, Prensas Universitarias de Zaragoza, Colección Textos Docentes nº 102, Universidad de Zaragoza, 2004.

[5] T. Pollán, “Electrónica Digital. III. Microelectrónica”, Prensas Universitarias de Zaragoza, Colección Textos Docentes nº 105, Universidad de Zaragoza, 2004.



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

Este texto pretende ser un libro para el estudio tanto de la Electrónica Digital como de los Sistemas Lógicos, que se pone a disposición de los estudiantes de tales asignaturas y de todas las personas interesadas en ellas.

No es un libro de consulta; existe gran número y amplia diversidad de libros de consulta disponibles, algunos de los cuales vienen reseñados en la bibliografía (al final del 2º volumen). Tampoco pretende ser una guía para la preparación de clases por parte del profesor, aunque seguramente pueda proporcionar una ayuda eficaz en tal sentido.

El presente texto ha sido elaborado y contrastado con la intención de configurar un libro para el estudio personal, para el esfuerzo individual que cada estudiante ha de realizar para «aprehender», es decir, para «hacer suyos» los conocimientos y la forma de razonar propios de una materia o disciplina.

[*aprender* → *apprehendere* → *apropiarse* → *adueñarse* → *ser dueño de*]

[*comprender* → *comprehendere* → *abarcarse* → *tener dentro* → *hacer suyo*]

Desde tal perspectiva se han ordenado los diversos conceptos y se ha desarrollado la exposición de los mismos, intercalando en dicha exposición numerosos ejemplos que permitan introducir, aclarar o aplicar los correspondientes conceptos. De ahí, también, la reiteración, casi repetitiva, de algunos conceptos e ideas en diversos lugares del texto, con el fin de insistir en su comprensión y asimilación; el detalle con que se analizan los ejemplos que sirven de introducción a algunos temas; la propia presentación del texto, destacando con negrita expresiones, variables y valores booleanos, ...

Un punto de referencia básico en mi actividad como profesor de Tecnología Electrónica lo constituye el concepto de información y de su manejo, la consideración finalista de los sistemas lógicos como herramienta conceptual para el procesamiento de la información y de la electrónica como la técnica más eficaz de que disponemos para el tratamiento de la información. Deseo y espero haber logrado transmitir al conjunto del texto tal referencia básica a la información y a su procesamiento, como objeto y objetivo propio de la electrónica digital.

En tal sentido importa destacar que «todo esto» (es decir, el procesamiento de la información, codificada en símbolos binarios, a que se dedica este libro) comenzó con los trabajos de Georges Boole (*An investigation of the laws of thought*, 1854) y de Claude E. Shannon (*A symbolic analysis of relay and switching circuits*, 1938).

En su búsqueda, junto a muchos otros investigadores de su época, de un modelo para expresar matemáticamente el pensamiento humano, esto es, la forma en que el hombre razona, Georges Boole nos legó un «lenguaje formal» (la lógica proposicional) que permite combinar proposiciones y una estructura matemática (el álgebra de Boole) que soporta dicho lenguaje. Claude E. Shannon nos mostró que dicha estructura matemática también soporta los cálculos numéricos en sistema binario y, además, puede ser materializada por medio de conmutadores.

De la disponibilidad de un «lenguaje» y de un «sistema de numeración» capaces de ser ejecutados por unos «dispositivos físicos» y de la forma de configurar adecuadamente tales dispositivos para realizar las correspondientes tareas de cálculo, deducción, decisión y control trata este libro.

En octubre de 1994 fue publicada por Prensas Universitarias de Zaragoza, en la Colección de Textos Docentes, la primera edición de esta ELECTRÓNICA DIGITAL que, desde el año 1982 y en sucesivas versiones, había sido impresa en forma de apuntes por capítulos. Habiéndose agotado esta edición, he abordado la tarea de escribir un nuevo texto con la finalidad de actualizar sus contenidos (tal como requiere el acelerado y constante ritmo de evolución de esta materia y, en particular, de sus componentes electrónicos) y de mejorar en lo posible la organización y el tratamiento de los mismos.

La presente edición, por motivo de número de páginas, se ofrece en tres tomos: este primero con el subtítulo de *sistemas combinacionales*, *sistemas secuenciales* el segundo y *microelectrónica* el tercero de ellos. No es que esta división sea realmente operativa en cuanto al diseño digital; de hecho todo sistema digital de una cierta complejidad es secuencial y su realización microelectrónica es la forma de construirlo conforme a la tecnología actual. Pero de alguna forma había que dividir la materia y la división adoptada es útil en relación con la progresión en el estudio de la misma

Para facilitar su utilización por parte de quienes se encuentren interesados en los sistemas lógicos, pero no en su realización electrónica, he procurado separar en capítulos distintos dichos aspectos. En tal sentido, los seis primeros capítulos de este volumen se dedican a los sistemas lógicos como tales, sin incluir ningún concepto electrónico y pueden ser estudiados independientemente del resto de los capítulos.

Al final del capítulo inicial (capítulo 0), en las páginas que van de la 21 a la 26, se encuentra una descripción resumida de los contenidos de los diversos capítulos; he considerado que era el lugar adecuado para comentar el esquema conceptual de la Electrónica Digital tal como lo trata este texto, una vez que el lector se ha hecho cómplice (eso espero) de mi visión de la electrónica como técnica de la información y ha asumido la distinción entre electrónica analógica y electrónica digital.

Material complementario para el estudio y utilización de este texto se encontrará en la página www.unizar.es/euitiz/digital.htm: en ella se dispondrá de copia actualizada de los capítulos; ampliaciones, comentarios, observaciones y erratas; hojas de ejercicios clasificadas por temas; colecciones de transparencias; etc., que pueden ser utilizadas, copiadas y distribuidas libremente en los términos de la licencia de Documentación Libre GNU (referencia <http://gugs.sindominio.net/licencias/fdl-es.html>).

Agradezco a todos los compañeros del departamento de ingeniería electrónica y comunicaciones de la Universidad de Zaragoza sus aportaciones y su ayuda cotidiana, y a quien utilice este libro para su personal esfuerzo de estudio la confianza que deposita en mi labor de profesor.

Ruego que se me avise de cualquier error, errata o incorrección que se detecte en el texto y, sobre todo, agradeceré efusivamente cualquier sugerencia o propuesta que contribuya a mejorarlo, a clarificar las ideas que expresa o a facilitar su estudio.

Tomás Pollán Santamaría tpollan@unizar.es
Profesor de tecnología electrónica en la EUITIZ
Departamento de ing. electrónica y comunicaciones

Universidad de Zaragoza.



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

0 LA ELECTRÓNICA COMO TÉCNICA DE LA INFORMACIÓN CODIFICACIÓN DIGITAL <> REPRESENTACIÓN ANALÓGICA

- 0.1. Objeto de la electrónica
- 0.2. Un breve paseo por la historia
- 0.3. Lo análogo y lo simbólico: dos electrónicas diferentes
- 0.4. Panorama general de la electrónica digital

*La historia de las cosas ayuda mucho en la comprensión de las cosas mismas,
ayuda a entrar en la esencia misma de las cosas, a no quedarnos en la superficie,
a formular su porqué y para qué, sus relaciones y sus limitaciones.*

¿A qué se dedica la electrónica? ¿Cuál es su objeto propio?

¿Cómo ha sido el desarrollo histórico de la electrónica? ¿Qué tareas ha desempeñado a lo largo del mismo? ¿Qué aspectos caracterizan a la electrónica?

¿Por qué dos electrónicas: analógica y digital? ¿Cuáles son las formas, métodos y maneras de cada una de ellas?

¿Qué contenidos conforman la electrónica digital? ¿Cómo están organizados en este texto y por qué?

*Éstas son las preguntas que intenta responder esta introducción,
los aspectos que desea dejar claros este capítulo previo.*

[Esta primera página en cada capítulo pretende ser una introducción, justificación y resumen del propio capítulo; en este caso, no tiene mucho sentido escribir una «introducción de la introducción» y, por ello, ha quedado reducida a un conjunto de preguntas.]

0.1. Objeto de la electrónica

¿A qué se dedica la electrónica? ¿Cuál es el objeto de esta disciplina?

La denominación de «electrónica» resulta equívoca, ya que no expresa ni la finalidad ni el objeto propios de esta materia, sino que alude a lo que utiliza: dispositivos basados en el comportamiento del electrón. Es, pues, una nominación instrumental, pero no funcional ni finalista: dice lo que la electrónica usa pero no indica lo que hace, ni sus objetivos propios.

La palabra electrónica pone de manifiesto el aprovechamiento de las propiedades de los electrones: estas partículas son diminutas (10^{-12} cm), sus efectos son muy rápidos (300.000 Km/s) y llevan carga eléctrica (capaces, por tanto, de ser portadores de señales eléctricas). Aprovechando tales propiedades, la electrónica utiliza dispositivos (llamados electrónicos) conectados formando circuitos que operan con señales eléctricas.

También las señales eléctricas presentan, de por sí, buenas propiedades: son fáciles de transmitir, de detectar y medir, de amplificar, de transformar, de combinar entre sí,... Existe, además, una amplia diversidad de transductores que realizan la transferencia entre otras magnitudes físicas y tensiones eléctricas (sensores) y viceversa (efectores).

La electrónica emplea dispositivos electrónicos, los cuales actúan sobre señales eléctricas. Pero eso no explica la finalidad propia de la electrónica, ni tampoco aclara el interés y la importancia de esta disciplina ni su utilidad: pocas personas y pocas entidades adquirirían o usarían equipos electrónicos con el simple objetivo o por el mero placer de manipular señales eléctricas.

Lo que realmente maneja la electrónica es información: las señales eléctricas interesan como portadoras de información y la electrónica opera eficientemente con ellas en su finalidad propia de manejar la información.

Una definición adecuada, a la vez finalista e instrumental, podría ser la siguiente: la electrónica es una técnica de manejo de la información, codificada en señales eléctricas, utilizando dispositivos que aprovechan las propiedades de los electrones.

¿Cómo se maneja la información? ¿Qué tareas interesa hacer con ella?

Desde la perspectiva del desarrollo histórico de la electrónica, podemos identificar tres grandes áreas de aplicación en el manejo de la información:

➤ Telecomunicación: enviar la información lejos, tanto en el espacio (comunicación por ondas) como en el tiempo (almacenamiento de la información en un soporte material para reproducirla posteriormente).

➤ Automatización: utilizar la información para controlar procesos; para ello, aparte de las propias operaciones a efectuar sobre la información, se necesitan sensores (capaces de convertir en señales eléctricas las magnitudes físicas que afectan al proceso) y efectores (capaces de traducir las señales eléctricas en acciones, en definitiva en otro tipo de magnitudes físicas).

➤ Informatización: procesar la información en sí misma para darle una nueva forma o para obtener nueva información a través de combinar varias informaciones.

Estos tres ámbitos de actuación sobre la información coinciden con las tareas que la electrónica ha ido abordando, sucesivamente, en su desarrollo a lo largo del siglo XX. También coinciden con las tres especialidades de la ingeniería dedicadas a la electrónica: telecomunicaciones, electrónica industrial (control de procesos) e informática.

¿Por qué la información? ¿Qué interés tiene para nosotros la información?

La información es una componente de la actividad humana; es, probablemente, la componente más intrínseca de la actividad del hombre. Junto con los materiales y la energía, la información está presente, como parte integrante y necesaria, en las diversas acciones de los humanos (incluso podemos distinguir actuaciones en las que no intervengan, externamente, los otros dos componentes, materia y energía, como es la simple reflexión o pensamiento).

Pero, además, materiales, energía e información representan escalones sucesivos en la macrohistoria socio-económica del hombre.

Durante el proceso histórico del devenir humano, el hombre tuvo que ocuparse, en un primer y muy prolongado período, de los materiales que satisficieran sus necesidades, que le permitieran sobrevivir y vivir cada vez mejor, que le aportaran comodidades y, también, que le sirvieran para confeccionar útiles e instrumentos que facilitasen y aumentasen la eficacia de sus acciones.

En segundo lugar, el hombre se preocupó de que «otros» trabajasen por él, de desarrollar formas de complementar y de suplir su trabajo y su esfuerzo, hasta poder aprovechar las más diversas fuentes de energía y disponer de mecanismos que permitieran utilizar la energía externa para obtener los productos y servicios que le interesaban. Un hito relevante de este proceso de aprovechamiento de la energía lo constituye la máquina de vapor, que da lugar a un período histórico conocido como revolución industrial («mudanza en el estado de las cosas» producida por la utilización eficaz de la energía en los procesos de producción).

Hoy día, el hombre se encuentra con la posibilidad de utilizar recursos externos para manejar la información, para transmitirla, recogerla y ampliarla y, también, para utilizar, en lugar del propio hombre, la información. Lo que hasta hace poco parecía patrimonio específico del hombre, la captación, el procesamiento y la utilización eficaz de informaciones complejas en forma versátil, ha pasado a ser también dominio de las máquinas y de las técnicas.

Por ello nos encontramos en un nuevo período de «mudanza en el estado de las cosas» que podemos nombrar como revolución informacional.

Precisamente, porque la información se ha «externalizado» del hombre, la hemos descubierto como concepto significativo. Hasta ahora había pasado desapercibida como parte intrínseca y consustancial a la actividad humana.

La información, como otros conceptos inherentes a la actividad del hombre (el tiempo y la energía), se pone de manifiesto cuando se «exterioriza», cuando el hombre la ve fuera de sí, cuando la manejan las máquinas.

La electrónica es, hoy por hoy, la base técnica más eficaz que dispone el hombre para manejar la información y tal eficacia le viene de las propiedades de los electrones y de las buenas características de las señales eléctricas: por ello, manejamos la información en forma de señales eléctricas y utilizamos para ello dispositivos electrónicos.

0.2. Un breve paseo por la historia

La electrónica ha ido abordando sucesivamente los tres grandes ámbitos de tareas de manejo de la información antes descritos (telecomunicación, automatización e informatización) y lo ha hecho en el mismo orden en que los hemos enumerado.

La electrónica emerge desde la electricidad que fue la gran protagonista de la física del siglo XIX. Los precursores del mundo eléctrico (Ampere, Coulomb, Faraday, Gauss, Henry, Kirckoff, Ohm, Volta,...), muchos de ellos a caballo entre los siglos XVIII y XIX, fueron sentando las bases de la electricidad y el magnetismo y las tradujeron en multitud de leyes parciales.

A partir de estos conocimientos dispersos, Maxwell realiza una gran síntesis unificadora, el electromagnetismo (1868), en la cual, al entrelazar la electricidad con el magnetismo, aparece un fenómeno novedoso: las ondas electromagnéticas. Dieciocho años más tarde (1886), Herz logra producir y detectar dichas ondas en su laboratorio y, muy pronto, Marconi las aprovecha para la comunicación sin cable.

Son tiempos en que ya se había desarrollado la comunicación a distancia (telégrafo: Morse, 1837, y teléfono: Bell, 1876) y la conservación del sonido en soporte material (gramófono: Edison, 1877).

La electricidad había ofrecido tres tipos de recursos tecnológicos: una energía fácilmente transportable, nuevas y mejores máquinas y la comunicación a gran distancia a través de cable. La búsqueda de comunicación a distancia sin cable condujo a la electrónica; en los albores del siglo XX (1901) Marconi logra que sus mensajes de puntos y rayas (código morse) crucen el atlántico y Auvrey (1906) consigue modular las ondas, de forma que el sonido cabalge sobre ellas.

La electrónica aparece cuando Lee de Forest introduce el primer dispositivo amplificador: el triodo (1907). Las experiencias anteriores de telecomunicación (relacionadas con el telégrafo inalámbrico) estaban «pidiendo a gritos» un buen amplificador de señales: de la energía que el emisor lanza en todas las direcciones (ya de por sí pequeña) la parte que recoge un receptor lejano es mínima; poder amplificar la energía recibida modifica cualitativamente la eficiencia de la transmisión.

Lee de Forest aporta un dispositivo que, aprovechando el comportamiento de los electrones, sirve para amplificar señales eléctricas, iniciando la electrónica de válvulas cuya aplicación y desarrollo ocupa la primera mitad del siglo XX. Una década después (Pittsburg 1921), la radiodifusión comienza sus primeros pasos y pronto proliferarían las emisoras de radio y los receptores radiofónicos serían de uso común en los países desarrollados.

Conviene resaltar estos aspectos que, desde el principio, distinguen a la electrónica: su carácter aplicado (no es una disciplina teórica o de investigación sino de manejo efectivo de la información), la rápida difusión de sus productos, su relación directa con la gente (con las personas comunes, más allá de los profesionales o especialistas) y, con ello, su influencia en la vida cotidiana.

Un nuevo rasgo entra en escena y acelera el desarrollo y la utilidad de la electrónica en la segunda mitad del siglo XX: un continuado proceso de miniaturización. En 1949 los trabajos del equipo científico de la Bell Telephone condujeron a la fabricación del primer transistor, reduciendo el tamaño del dispositivo amplificador en un factor cercano a 100.

Con el transistor comienza un proceso continuo de reducción de la electrónica: todo se hace más pequeño, más corto, más rápido: Se reduce no sólo el tamaño, sino también el consumo y el coste; también se hace más pequeño el tiempo de respuesta de los circuitos (con lo cual trabajan mucho más rápido y mejor), el tiempo de desarrollo de los sistemas (desde que se tiene una idea funcional hasta que el equipo está disponible) y el tiempo de su difusión pública (de su utilización por la gente).

Este proceso de miniaturización ha continuado en la década de los 60 con el desarrollo de los circuitos integrados, cuya densidad de integración ha ido progresivamente en aumento. Ello ha permitido construir y poner rápidamente en nuestras manos sistemas electrónicos cada vez más complejos y potentes, de tamaño, consumo y costes muy reducidos.

A la electrónica la minimización «le viene de familia» (el electrón es diminuto y muy veloz, en cuanto a sus efectos) y el resultado es que, al hacerse tan pequeña y tan rápida, la electrónica se ha metido por todos los rincones de nuestra vida y de nuestra sociedad y ha promovido esa «mudanza en el estado de las cosas» que caracteriza nuestro presente: la revolución informacional.

La electrónica de la primera mitad del siglo XX se dedicó a la telecomunicación, en su doble aspecto: espacial y temporal; desarrolló la radiodifusión y la grabación del sonido (en discos mecánicos, cintas magnéticas y bandas ópticas de las películas sonoras), mejoró ampliamente la telefonía e inició la transmisión de imágenes (televisión).

A partir de los años 40, la electrónica aborda el control de procesos. Los intereses bélicos, en torno a la segunda guerra mundial impulsaron la investigación y desarrollo de sistemas electrónicos de control (para control de tiro y, también, para localización de aviones por medio del radar).

La penetración en la industria de los sistemas de control electrónicos se ve favorecida por la introducción de dispositivos electrónicos de control de energía (tiristores y triacs) y por la posibilidad de abordar tareas complejas gracias a los circuitos integrados; de forma que, a partir de los años 70, la electrónica pasa a controlar todo tipo de proceso industrial y, desde los años 80, se incorpora masivamente dentro de los productos resultantes de la fabricación industrial.

A la vez, en esta segunda mitad del siglo XX, muy poquito a poco al principio pero de forma espectacular en el último cuarto de siglo, la electrónica ha ido asumiendo otra vertiente más abstracta y genérica: operar con la información en sí misma, representarla y manejarla a través de símbolos, lo que hoy en día entendemos por procesar la información.

El camino hacia la informatización lo habían abierto dos precursores distantes entre sí:

- Georges Boole, matemático inglés, que tuvo éxito en su empeño de construir un modelo matemático del pensamiento humano (de la forma de razonar), estableciendo las bases de la lógica proposicional (la forma de combinar proposiciones), a través de una estructura matemática que, andando el tiempo, sería conocida como álgebra de Boole (*Una investigación de las leyes del pensamiento*, publicado en 1854).
- Claude E. Shannon, ingeniero norteamericano, que, al desarrollar un modelo matemático para tratar con las redes de múltiples conmutadores propias de la telefonía, identifica la interconexión de conmutadores como álgebra booleana y pone de manifiesto que también lo es el sistema binario de numeración (*Un análisis simbólico de los relés y circuitos de conmutadores*, publicado en 1938).

Sobre las bases conceptuales que establecen Boole y Shannon se edifica la electrónica digital (soporte instrumental del procesamiento de la información), que alcanzará su mayoría de edad en los años 70, cuando los circuitos integrados permitan configurar sistemas informáticos potentes y reducir su coste, hasta llegar (en los años 80) al microprocesador que hace viables los computadores personales.

Pero la electrónica propia de la informatización (la electrónica digital) no se limita a la configuración de sistemas propiamente informáticos sino que, desde sus inicios, se dedica también al control de procesos y, en buena medida, desplaza a la electrónica anterior (analógica). El microprocesador resuelve muy eficazmente el control de procesos industriales y la integración de circuitos de aplicación específica (ASICs) permite miniaturizar controles sumamente sofisticados para el interior de los productos fabricados en tales procesos.

Asimismo, hoy en día, la electrónica digital ha invadido y renovado el ámbito de las comunicaciones y los sistemas digitales han abierto nuevas alternativas (con extraordinarias prestaciones) en cuanto a almacenamiento de sonido e imagen, en cuanto a telefonía por microondas y, también, en radio y televisión.

0.3. Lo análogo y lo simbólico: dos electrónicas diferentes

Las señales eléctricas son el soporte material de la información; según la manera de codificar la información (de representarla en forma de señales eléctricas) aparecen dos tipos de electrónica: la analógica y la digital.

La analógica representa las cantidades «por analogía cuantitativa» (a mayor cantidad, mayor tensión) según una relación de proporcionalidad directa, mientras que la digital utiliza símbolos a través de un proceso de codificación abstracta.

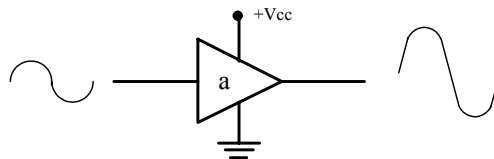
Tomemos el ejemplo de un termómetro analógico y otro digital; el primero representa la temperatura mediante una columna de mercurio (a mayor temperatura mayor longitud de la columna), en cambio el digital proporciona unos signos numéricos (que es preciso interpretar por referencia a un determinado código). Si observamos que la columna de mercurio ha aumentado, sabemos que ha subido la temperatura; en cambio, entre una indicación digital de 19° y otra de 23° no es obvio cuál de ellas indica mayor temperatura (solamente la interpretación de los símbolos, a través del código numérico, permite resolver la comparación).

0.3.1. Electrónica analógica

La electrónica analógica representa los valores de una magnitud física mediante una tensión, a través de una relación de proporcionalidad $V(t) = k \cdot M(t)$; se utiliza una sola tensión, una constante de proporcionalidad relaciona la tensión con el valor que representa y el rango de valores de la tensión es continuo, entre dos valores extremos $V_{\max}$ y $V_{\min}$.

Para trasladar el valor de la magnitud física a la correspondiente tensión se necesita un sensor adecuado (transductor magnitud física $\rightarrow$ señal eléctrica).

El dispositivo básico de la electrónica analógica es el amplificador, el cual suministra una tensión de salida proporcional a la tensión de entrada $V_o = a \cdot V_i$, a expensas de recibir una energía eléctrica desde una fuente de alimentación V_{CC} .



El amplificador se construye con dispositivos amplificadores (triódos o transistores), enmarcados en un circuito de polarización (ubicación en el punto de operación adecuado para que reciban la energía de la fuente de alimentación).

Pueden diseñarse muy diversas etapas amplificadoras (de tensión, de intensidad o de ambas) y, con ellas, puede construirse un amplificador de muy alta ganancia y características ideales, el amplificador operacional, que permite configurar bloques correspondientes a operaciones aritméticas (sumadores, restadores, comparadores, integradores, derivadores,...); tales bloques constituyen los módulos básicos para el control de procesos (automatización).

Asimismo, razonando en el espectro de frecuencias, con la correspondiente ayuda de condensadores y bobinas, las etapas amplificadoras pueden transformarse en filtros, sintonizadores, demoduladores, moduladores, amplificadores de antena,... que son los módulos básicos para la comunicación por ondas (telecomunicación).

Las matemáticas propias de la electrónica analógica son las que corresponden a la proporcionalidad, las operaciones lineales (suma, resta, producto por una constante, derivada, integral). Más específicamente, el control de procesos se basa conceptualmente en la realimentación y en la teoría derivada de ella que recibe el nombre de regulación automática, mientras que la telecomunicación utiliza el dominio de la frecuencia, apoyándose en la descomposición en serie de Fourier (espectro de frecuencias).

0.3.2. Electrónica digital

Los sistemas digitales representan cada valor de una magnitud física mediante un conjunto de dígitos o cifras, cada uno de los cuales admite varias posibilidades o símbolos. Por ejemplo, en el sistema de numeración decimal, cada dígito tiene diez posibilidades (0, 1, 2, 3, 4, 5, 6, 7, 8 y 9).

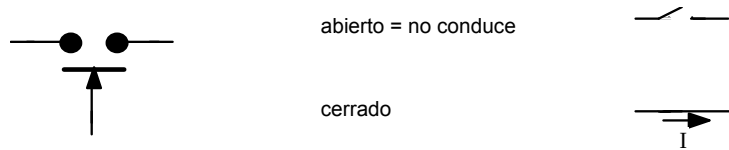
La electrónica digital es binaria, es decir, cada dígito admite solamente dos posibilidades, que solemos expresar con los símbolos **0** y **1** (también, con los símbolos **L** y **H**), de forma que el sistema de numeración que le es propio es el sistema de base 2 (binario).

La electrónica digital representa los valores de una magnitud física mediante **m** señales eléctricas, cada una de las cuales admite solamente dos valores de tensión que corresponden a los símbolos **0** y **1**; para expresar un solo valor de la magnitud se necesitan **m** señales, la relación se establece mediante un proceso abstracto de codificación y el rango de cada señal es discontinuo, reducido a dos únicos valores $V_{\max}$ y $V_{\min}$ (por ejemplo, 0V y 5V).

Para trasladar el valor de la magnitud a la correspondiente representación digital es preciso utilizar el sensor que transforma la magnitud en tensión analógica, seguido de un conversor analógico-digital que aplique el correspondiente código.

Por ejemplo, un sensor de temperatura aplicado a una temperatura de 17° puede proporcionar una tensión de 4,25 V (factor de proporcionalidad 1/4) y el conversor analógico-digital debe transformar dicha tensión en **10001** (que corresponde al número 17 en binario), que, en realidad, serán cinco tensiones en los terminales de salida del conversor: 5V 0V 0V 0V 5V.

El dispositivo básico de la electrónica digital es el conmutador o interruptor (con dos estados: abierto $\equiv$ no conduce y cerrado $\equiv$ conduce).



Al igual que en el caso del amplificador, el interruptor puede construirse con transistores (también podría hacerse con triodos); para amplificadores se utiliza la zona lineal de operación del transistor mientras que para interruptores se emplean los extremos de dicha zona: corte ($I = 0$) y saturación ($V = 0$).

Con interruptores (transistores) se construyen puertas lógicas (capaces de efectuar operaciones booleanas individuales) y agrupando dichas puertas se configuran funciones booleanas que son la base relacional de las variables digitales. Determinados conjuntos de funciones booleanas de utilidad general se agrupan en bloques combinacionales que, junto con los bloques con memoria (biestables, registros y contadores) constituyen los módulos básicos para el diseño digital. Los biestables provienen de establecer realimentación dentro de una función booleana y con ellos se configuran los registros y los contadores. En definitiva, todo en los sistemas digitales son funciones booleanas, las cuales se componen de conjuntos de puertas lógicas, construidas con interruptores.

Por ello, la matemática propia de la electrónica digital es el álgebra de Boole: las funciones booleanas expresadas como combinación de operaciones del álgebra booleana. Complementariamente, los grafos de estado son una herramienta auxiliar apropiada para describir el comportamiento de los circuitos digitales con memoria.

0.3.3. Lo análogo frente a lo simbólico

El nombre de «análoga» deriva de que la representación (de la magnitud física en tensión eléctrica) se hace «por analogía»: los valores de la señal eléctrica son «análogos en cantidad» a los de la magnitud física: hay una relación directa en términos de cantidad, una relación de proporcionalidad. El nombre de «digital» le viene de que utiliza dígitos: representa la información mediante «palabras» formadas por varios dígitos, a través de una codificación: es una representación simbólica que requiere un proceso de abstracción.

Un sensor adecuado transforma directamente la correspondiente magnitud física en tensión eléctrica análoga, pero se requiere una codificación posterior para que la señal resultante de la medida sea trasladada a la palabra binaria (al conjunto de señales) que corresponde a su representación digital. Por medio hay un código que establece la relación entre cada símbolo y la cantidad de tensión análoga que representa, cantidad que, además, depende de la posición del símbolo en la palabra binaria (cada dígito presenta diferente valor relativo).

Es indudable que se asume una complicación, un esfuerzo adicional, al pasar de la representación análoga a la digital. En la utilización digital de símbolos hay un esfuerzo intermedio importante que no resulta obvio: la representación en dígitos requiere una transformación cualitativa, una conversión abstracta en símbolos que, según el lugar que ocupan, representan cantidades diferentes.

¿Qué ganamos con ello?

- a) *Precisión*: los valores, una vez expresados en símbolos, están claramente identificados con absoluta precisión; en cuanto a tensiones analógicas, al utilizar éstas todo el rango de valores de tensión (entre dos extremos), dos valores próximos tendrán dificultades para diferenciarse mientras que, en el caso digital, corresponderán a dos palabras binarias diferentes (y su expresión en tensiones empleará para cada dígito dos valores distantes).

Por ejemplo, utilizando el intervalo de 0 a 5 V para representar análogicamente las temperaturas de 0 a 100°, sendas temperaturas de 19° y 20° corresponderán a valores de tensión de 0,95 V y 1,00 V, que resultan muy próximos entre sí, mientras que sus palabras digitales serán **10011** y **10100**, claramente diferentes, y el conjunto de tensiones que corresponde a su representación digital serán, respectivamente:

5V 0V 0V 5V 5V y 5V 0V 5V 0V 0V.

- b) *Fortaleza frente a perturbaciones (frente al ruido electromagnético)*: las tensiones digitales utilizadas corresponderán a dos valores distantes (por ejemplo 0V y 5V) mientras que las tensiones analógicas recorren todo el rango de valores (entre 0V y 5V), de forma que la más mínima perturbación modificará el valor que representan.
- c) *Fortaleza frente a derivas o faltas de precisión de los circuitos*: al operar con las señales eléctricas cualquier etapa analógica causará un cierto grado de error (una mínima desviación de tensión o un pequeño fallo de precisión) que, al actuar en un rango continuo supondrá un error en el valor de la magnitud resultante; la separación entre los valores de tensión que corresponden al **0** y al **1** digitales anula el efecto de tales desviaciones.
- d) *Capacidad de cálculo*: la representación simbólica permite utilizar los mecanismos de cálculo propios del correspondiente sistema de numeración (en el caso digital, el cálculo en el sistema binario).
- e) *Capacidad de razonamiento (de combinar proposiciones)*: el razonamiento es propio de la representación simbólica y de la combinación de símbolos (a través de las reglas de la lógica).

La electrónica digital, al trasladarnos al mundo de los símbolos, aporta precisión y fortaleza y nos transfiere al plano de lo abstracto que es el ámbito del cálculo y del razonamiento.

0.4. Panorama general de la electrónica digital

El estudio de los sistemas digitales suele dividirse en dos grandes partes dedicadas, respectivamente, a los sistemas combinacionales y a los sistemas secuenciales; ambos se diferencian entre sí por la existencia de memoria en los segundos, mientras que en los primeros las salidas son función directa de los valores presentes en sus entradas.

0.4.1. Sistemas combinacionales

La base matemática de la electrónica digital la constituye el álgebra de Boole, cuyas funciones expresan todas las correspondencias entre las variables de los sistemas digitales. Por ello, resulta adecuado comenzar por el estudio del álgebra booleana, sus operaciones y teoremas (cap. 1) y la forma de construir y simplificar las funciones booleanas (cap. 2).

Todo ello con referencia a las tres álgebras de Boole de dos elementos cuyo isomorfismo es la base de la electrónica digital (cap. 1): la lógica proposicional (lenguaje formal para razonar), el sistema binario (sistema de numeración para calcular) y el álgebra de conmutadores (componentes físicos para construir las operaciones booleanas). La diversidad de representaciones de una función booleana (cap. 2) es la base para su construcción circuital, siendo sumamente útiles los procedimientos de simplificación de la función para reducir el tamaño del circuito.

Las funciones booleanas pueden agruparse en bloques o módulos que realizan operaciones globales de interés genérico: bloques combinacionales. De un lado (cap. 3), interesan los bloques que efectúan operaciones aritméticas entre dos números binarios y, dentro de esta perspectiva numérica, interesa ampliar el código binario para representar (con los únicos dos símbolos disponibles, el 0 y el 1) números negativos y números con parte decimal.

Por otra parte (cap. 4), son útiles los bloques que facilitan la distribución de la información y la selección de posibilidades (multiplexores, demultiplexores y decodificadores) y, también, los que simplemente trasladan la misma información de un código a otro (codificadores); además, estos tipos de bloques (distribuidores y codificadores) pueden construirse mediante configuraciones reticulares de sus conexiones, lo cual simplifica en gran medida su diseño y fabricación.

Los bloques combinacionales constituyen «piezas de diseño» que facilitan la división de un sistema digital en partes y permiten configurarlo por ensamblaje de tales módulos.

Se trata siempre de construir conjuntos de funciones booleanas, lo cual se complica cuando el número de sus entradas es alto: las estructuras matriciales (cap. 5) permiten abordar tal complejidad. Existen tres estructuras conceptualmente simples ROM, PAL y PLA, que facilitan la descripción y construcción de bloques combinacionales de muchas entradas; tales configuraciones sirven, además, para conformar circuitos integrados programables, disponibles para insertar (programar) en su interior el conjunto de funciones booleanas de un codificador concreto o de un bloque combinacional específico propio de un diseño particular.

Una vez recorridos «los cimientos y el almacén de piezas» de los sistemas lógicos combinacionales (las funciones y los módulos que hacen viable su diseño), conviene recordar que la materia sobre la que trabajan es la información y que ésta se encuentra codificada en palabras binarias de «ceros» y «unos», existiendo múltiples posibilidades de codificación (cap. 6). Conviene, asimismo, tomar conciencia de la posibilidad de error (principalmente en la transmisión y en la conservación de la información) y conocer la existencia de códigos capaces de detectar e, incluso, de corregir los errores.

Hasta aquí (capítulos del 1 al 6) se ha tratado de los sistemas lógicos sin referencia a la electrónica que permite construirlos físicamente; también es preciso conocer y comprender la tecnología (y a ello van dedicados los capítulos 7, 8, 9 y 10).

Se denomina puerta lógica a la realización física de una operación booleana. Las puertas con diodos (cap. 7), además de su utilidad como puertas individuales, sirven para apreciar la necesidad de buen acople en tensión (requisito inexcusable para conectar una puerta lógica a la siguiente, ya que lo que se transmite es una tensión eléctrica). Por otra parte, las puertas lógicas habituales son de tipo inversor, construidas con interruptores según el álgebra de conmutadores, y el transistor NMOS es un excelente interruptor.

La combinación de interruptores de los dos tipos complementarios, utilizando transistores NMOS y PMOS, permite anular el consumo estático de las puertas lógicas y reducir su resistencia de salida, configurando puertas de características cuasi-ideales; es por ello la tecnología digital predominante: familia lógica CMOS (cap. 8).

La tecnología CMOS ofrece una muy amplia diversidad de configuraciones (cap. 9), tanto en variedad de puertas complementarias como en otros tipos de puertas para aplicaciones específicas: las puertas de transmisión facilitan la configuración de multiplexores y la desconexión (estado de alta impedancia) y las puertas pseudoNMOS permiten construir estructuras matriciales de muchas entradas y, también, los bloques programables tipo ROM, PAL y PLA.

Como puede apreciarse este texto opta por los transistores MOS y, en concreto, por la tecnología CMOS como forma de realización física de los circuitos digitales; tal opción se fundamenta en dos razones:

- los transistores MOS se corresponden, casi idealmente, con los interruptores propios del álgebra de conmutadores, que es la base conceptual para construir físicamente los sistemas digitales
- y, además, la integración CMOS es, actualmente, la forma habitual de realización de circuitos integrados digitales. A partir de los años 80, la CMOS relegó a un segundo plano a las tecnologías bipolares y, por sus excelentes características funcionales, se ha impuesto como la tecnología propia de la electrónica digital (siendo previsible que su actual predominio se mantenga, al menos, en las próximas dos décadas).

No obstante, el apéndice A3 presenta, en forma resumida, la configuración de las puertas lógicas con transistores bipolares y las características propias de la familia TTL; y el apartado 10.3 describe la evolución de los circuitos integrados digitales y la situación actual en cuanto a utilización de las diversas series.

Cerrando la parte referida a la realización electrónica de los sistemas lógicos, se presenta (cap. 10) la evolución histórica y el panorama general de las familias lógicas integradas, junto con las características a tener en cuenta a la hora de utilizarlas y el importante problema del ruido electromagnético que afecta a los circuitos digitales.

Además, este primer volumen incluye tres capítulos complementarios dedicados a aspectos tecnológicos (numerados como T1, T2 y T3).

En el primero de ellos (cap. T1) se explica, en forma conceptual, breve y sencilla pero con adecuada profundidad, el comportamiento de los semiconductores, sus dos tipos N y P y la unión PN con sus características de discriminación de polaridad y aislamiento eléctrico (cuando se encuentra en polarización negativa) y se desarrolla un modelo operativo de funcionamiento de los transistores MOS.

Después de la teoría de semiconductores, se presenta el proceso de fabricación de circuitos integrados CMOS (cap. T2): las etapas básicas de dicho proceso, las diversas operaciones físico-químicas que se aplican y el detalle de los sucesivos pasos que conforman el proceso de integración. Además, se incluye un epígrafe con los datos numéricos de los parámetros físicos de las regiones que forman parte de un circuito integrado CMOS (concentraciones de impurezas y de portadores, movilidad de los mismos, resistividad, capacidad por unidad de área,...) para facilitar una idea precisa de la configuración atómica y de las propiedades eléctricas de cada zona semiconductor, conductora o aislante.

Asimismo, dentro de los aspectos tecnológicos, se analiza con detenimiento el problema del ruido en los sistemas digitales (cap. T3), detallando las causas físicas de las interferencias electromagnéticas, los mecanismos que las producen y transmiten en relación con los circuitos digitales y las formas de evitar o reducir sus efectos.

0.4.2. Sistemas secuenciales

Los sistemas secuenciales son sistemas digitales que incorporan memoria de su pasado; ésta se consigue mediante realimentación en las propias funciones booleanas.

La memoria presenta dos aspectos (cap. 11): la necesidad de recordar la evolución anterior del sistema y el almacenamiento de datos para su posterior utilización; en ambos casos, el biestable es la célula básica capaz de almacenar un dígito. El conjunto de variables de estado contiene la información que el sistema secuencial necesita sobre su pasado y los grafos de estado son una herramienta adecuada para representar el comportamiento del sistema (la evolución del estado).

El proceso de diseño secuencial (cap. 12), a partir del grafo de estados y de la asignación de una palabra binaria a cada uno de ellos (codificación), consiste en dedicar un biestable a cada variable de estado y establecer las funciones que controlan dichos biestables (evolución del estado) y las variables de salida (activación de las salidas).

El sincronismo, como división del tiempo en unidades discretas, facilita el diseño de los sistemas secuenciales y les confiere una gran seguridad de funcionamiento. Para ello es necesario introducir una señal de reloj, cuyos flancos señalarán el paso de una unidad de tiempo a la siguiente, y disponer de biestables síncronos que solamente cambian en dichos flancos (cap. 13); la combinación de biestables con funciones previas en configuración PAL da lugar a circuitos integrados programables, capaces de admitir la inserción (por programación) en su interior de un sistema secuencial completo.

El diseño de sistemas secuenciales con biestables síncronos (cap. 14) pasa también por establecer las funciones de evolución del estado (que actúan sobre los biestables que contienen las variables de estado) y las funciones de activación de las salidas. El diseño síncrono es intrínsecamente necesario en sistemas de procesamiento o transmisión serie (en los cuales a cada dígito le corresponde un intervalo de tiempo definido por el reloj) y es altamente conveniente en todo tipo de diseño digital de una cierta complejidad.

El sincronismo ofrece facilidad de diseño al referirlo a unidades de tiempo discretas y numeradas pero, sobre todo, seguridad de funcionamiento para evitar las aleas y errores debidos a los diferentes tiempos de propagación. Para ello es preciso comprender en profundidad su significado conceptual y los requisitos que el sincronismo impone (cap. 15); tales requisitos se reflejan sobre los biestables, en forma de condiciones de diseño y tiempos funcionales que han de ser respetados y sobre la señal de reloj, cuya verticalidad, simultaneidad, y no-contaminación han de ser aseguradas.

Un registro es un conjunto de biestables y sirve para almacenar una palabra binaria (capítulos 11 y 13). Los contadores (cap. 16) son un tipo especial de registros que evolucionan entre números binarios consecutivos (pasan de un número al siguiente o viceversa) y que sirven para contar pulsos y para dividir frecuencias; aunque son circuitos secuenciales, debido a que suelen tener muchos estados (si bien muy ordenados entre sí), su diseño requiere una metodología propia.

Los contadores son bloques secuenciales sumamente útiles en el diseño de sistemas de medida y de control, debido a la gran variedad de aplicaciones que ofrecen (cap. 17): conteo y control de número de objetos y de sucesos, medida de tiempos (los contadores son la herramienta específica para manejar el tiempo), medida de frecuencias y de número de revoluciones,...

Una aplicación más sofisticada de los contadores, la modulación de anchura de pulsos, merece particular atención (cap. 18); la información reflejada sobre amplitudes de tensión no resulta apropiada para los métodos digitales y, en cambio, hay procedimientos digitales que facilitan un buen manejo de información expresada en anchura de pulsos de amplitud fija. Los pulsos de anchura modulada permiten realizar con procedimientos básicamente digitales tareas que, en principio, parecen más propias de la electrónica analógica como el control de potencia, la conversión digital-analógica y la recíproca de analógica a digital, la disponibilidad de potenciómetros para controlar la amplitud de las señales,...

Algunos sistemas digitales necesitan almacenar en su memoria grandes cantidades de datos, para ello los registros se agrupan en amplios bloques, denominados memorias (cap. 19); tales bloques (RAM) presentan aspectos novedosos relativos a la organización de la transferencia de la información por medio de buses (de datos, de direcciones y de control). El bus de direcciones genera un amplio campo de posiciones numeradas para situar los registros de los bloques de memoria: mapa de memoria.

La arquitectura basada en la utilización de buses permite agrupar múltiples bloques de memoria (algunos de ellos de solo lectura ROM) y, también, incorporar dentro de la memoria los periféricos de entrada y de salida (cap. 20); para ello, es preciso situar circuitalmente cada bloque en un segmento del mapa de memoria y dotar de adecuada fuerza a las señales que son transmitidas por los buses.

Dos capítulos, dedicados a aspectos tecnológicos, se refieren a circuitos auxiliares, complementarios de los propiamente digitales y necesarios, respectivamente, para la gestión del tiempo (cap. T4) y para la relación con el mundo exterior, básicamente analógico (cap. T5). El primero describe los circuitos temporizadores que permiten la determinación de intervalos de duración dada: osciladores para producir ondas de reloj y monostables para pulsos de anchura dada. El segundo trata de los circuitos que facilitan la comunicación de los sistemas digitales con su entorno físico analógico: conversores de información digital a analógica y viceversa, de tensiones analógicas a códigos digitales.

0.4.3. Microelectrónica

El diseño digital actual se desarrolla en forma «micro»: la microelectrónica digital alude a la configuración del circuito electrónico completo, resultante de un diseño específico, en el interior de un solo integrado. Lo cual supone pasar de la interconexión de circuitos integrados estándar a la construcción de un circuito integrado específico.

Se trata de insertar nuestro propio diseño completo en un único circuito integrado (que no será de tipo estándar, disponible en catálogo, sino de «aplicación específica») y para ello disponemos de dos alternativas:

- programar nuestro diseño sobre un circuito integrado programable
- o, fabricar dicho diseño como circuito integrado específico, ASIC.

El estudio de la microelectrónica debe atender a dos aspectos diferenciados:

- por un lado, el proceso de diseño de un circuito particular (para una aplicación específica) y las posibilidades de programación o de fabricación del mismo,
- y, de otro, la tecnología como soporte material sobre el cual se va a configurar el circuito resultante de nuestro diseño.

En primer lugar, se ha de conocer cómo llevar a cabo el proceso de diseño de un sistema digital para conseguir configurarlo, en el interior de un circuito integrado: insertarlo sobre un dispositivo programable o fabricarlo como integrado específico. Las dos opciones (programación o fabricación del diseño) utilizan el mismo esquema de desarrollo del proceso de diseño, el mismo tipo de herramientas y las mismas consideraciones metodológicas a tener en cuenta.

Es razonable comenzar estudiando (cap. 21) las diversas posibilidades de programación y fabricación que permiten obtener un circuito integrado con un diseño específico completo; y considerar luego, en mayor detalle (cap. 22), la configuración y prestaciones de los circuitos integrados programables de tipo avanzado (FPGAs).

El diseño microelectrónico se desarrolla habitualmente con la ayuda de un lenguaje de descripción circuital, de los cuales el más utilizado es el VHDL (cap. 23). VHDL es un lenguaje de descripción y simulación de sistemas en general y, desde el punto de vista de la microelectrónica, interesa la parte que puede ser sintetizada por compiladores digitales, es decir, que puede ser traducida a un conjunto de puertas y biestables para configurar un circuito digital. Complementariamente, se detallan en un apéndice (ap. A6) diez ejemplos de diseño VHDL de sistemas de control, incluyendo en ellos máquinas algorítmicas (dedicadas a cálculos numéricos) y diseños «amplios» de sistemas habituales.

La microelectrónica permite abordar sistemas digitales muy complejos y resulta muy adecuado considerar las pautas o «formas de actuar» que orienten y faciliten el diseño complejo (cap. 24); frente a lo grande y complicado la estrategia más eficaz de los limitados cerebros humanos es la división en partes, una división que no ha de perder la visión de globalidad: fraccionar con estructura.

Además, la microelectrónica exige plantear el problema del test de fabricación (cap. 25); los vectores o métodos de comprobación de que un circuito integrado se ha fabricado bien son responsabilidad del diseñador, aunque su aplicación efectiva corresponda al fabricante.

También es necesario conocer el soporte material que permite la programación o fabricación de un circuito específico, estudiar con un poco de detalle la tecnología que permite «materializar» un diseño particular. Cuatro capítulos, de los etiquetados como «aspectos tecnológicos», presentan un modelo básico conceptual y operativo de la tecnología, que facilita la comprensión de sus posibilidades, sus prestaciones, sus limitaciones y sus exigencias (los requisitos que la tecnología impone sobre el diseño).

El primero de ellos (cap. T6) desarrolla un modelo funcional del comportamiento de los transistores MOS que permite una representación gráfica de la distribución de carga en el canal de dichos transistores y, sobre ella, efectuar cálculos relativos a tensiones, intensidades, tiempos y consumos.

A partir de dicho modelo, se estudian en detalle las características del inversor CMOS (cap. T7), como prototipo de las puertas inversoras, y se analizan las puertas pseudoNMOS y las puertas de transmisión (cap. T8), como alternativas de interés para determinadas configuraciones; asimismo, se introduce la lógica dinámica.

Por último (cap. T9), se consideran cuestiones diversas que completan el panorama de un circuito integrado global: entradas y salidas, densidad de integración, reglas de diseño físico,..., así como (ap. A7), la posibilidad de incluir etapas analógicas complementarias dentro del mismo integrado digital (ASICs mixtos) y el simulador SPICE (ap. A8) como eficaz herramienta de simulación eléctrica del comportamiento y características de las puertas o módulos digitales.

1 ÁLGEBRAS DE BOOLE DE 2 ELEMENTOS

OPERACIONES BOOLEANAS Y PUERTAS LÓGICAS

- 1.1. Estructura de Álgebra de Boole
- 1.2. Casos de interés de Álgebras de Boole binarias
- 1.3. Representación gráfica de las operaciones booleanas: puertas lógicas
- 1.4. Operaciones unitarias
- 1.5. Las operaciones booleanas en lenguaje de descripción circuital: VHDL

La finalidad de la Electrónica Digital es procesar la información. Para ello utiliza las operaciones definidas por George Boole en su «investigación sobre las leyes del pensamiento», publicada en 1854. En una época de triunfo de las matemáticas en la tarea de «modelizar» el mundo físico, George Boole dio también forma matemática a la combinación de proposiciones; Boole introdujo, a la vez, un lenguaje formal (la lógica proposicional) y una estructura matemática (el álgebra de Boole) capaz de representar y de validar tal lenguaje.

Casi un siglo después, en 1938, al estudiar los complejos circuitos de relés que se utilizaban en la comunicación telefónica, Claude E. Shannon demostró que las operaciones booleanas son aptas para describir los circuitos con conmutadores y, también, para expresar cálculos en el sistema de numeración de base 2. Shannon estableció la posibilidad de utilizar la misma estructura matemática (el álgebra de Boole) como soporte de un sistema de numeración y cálculo (el sistema binario) y proporcionó una forma de «construir» las operaciones del álgebra booleana mediante la conexión de dispositivos físicos (los conmutadores).

Boole y Shannon fijaron los «cimientos» conceptuales para el procesamiento digital de la información. Gracias a ellos disponemos de un lenguaje formalizado capaz de expresar la combinación de proposiciones, de un sistema de numeración capaz de soportar cálculos aritméticos y de una forma de «materializar» (es decir, de construir máquinas capaces de ejecutar) tanto el lenguaje como el sistema de numeración.

La base matemática que soporta todo esto corresponde a la estructura de álgebra de Boole de dos elementos (el 0 y el 1): álgebra booleana binaria. Las «máquinas digitales», aunque solamente «saben» trabajar con el 0 y el 1 (una lógica dual muy limitada), son capaces de manejar, a más alto nivel (por programación), la lógica difusa, las redes neuronales, la inferencia matemática, la inteligencia artificial,...

Este tema presenta un resumen general de los conceptos fundamentales del álgebra de Boole y de sus operaciones, considerando en particular las tres álgebras binarias citadas: la lógica proposicional, el sistema de numeración con base 2 y el álgebra de conmutadores. Además, se expone la representación gráfica de las operaciones booleanas mediante puertas lógicas, como esquema para describir (y forma de construir) los circuitos digitales, y se introducen las operaciones «unitarias» que permiten expresar, con sólo una de ellas, todo el álgebra booleana y, por lo mismo, permiten construir cualquier circuito digital con un solo tipo de puertas.

1.1. Estructura de Álgebra de Boole

El álgebra de Boole es una estructura matemática definida sobre un conjunto de elementos $\{a, b, c, \dots\}$ por tres operaciones con las propiedades siguientes:

- la complementación o negación, $\bar{a}$, con propiedad de involución, $\bar{\bar{a}} = a$
- la operación "o", $a + b$, asociativa y conmutativa
- la operación "y", $a \cdot b$, también asociativa y conmutativa
- siendo estas dos últimas operaciones distributivas entre sí

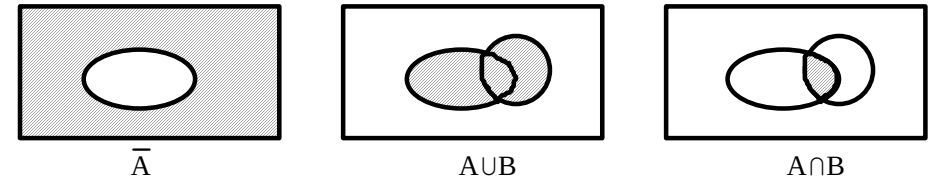
$$a \cdot (b + c) = (a \cdot b) + (a \cdot c)$$

$$a + (b \cdot c) = (a + b) \cdot (a + c)$$

- y existiendo dos elementos únicos, 0 y 1, tales que $\bar{0} = 1$ $\bar{1} = 0$ y

$a + 0 = a$	$a + 1 = 1$	$a + \bar{a} = 1$
$a \cdot 0 = 0$	$a \cdot 1 = a$	$a \cdot \bar{a} = 0$

Un ejemplo característico de álgebra booleana lo constituye el conjunto partes de un conjunto dado (conjunto de todos sus subconjuntos) con las operaciones de complementación, unión e intersección; tales operaciones pueden representarse gráficamente mediante los diagramas de Venn.



En relación con el cuerpo de los números reales, contrastando sus operaciones aritméticas de suma y resta con las operaciones "o" e "y" booleanas, el álgebra de Boole presenta las siguientes diferencias:

- La propiedad distributiva es doble; no sólo de $\cdot$ respecto a $+$, sino también de $+$ respecto a $\cdot$.
- No existen elementos inversos respecto a las operaciones "o" e "y" y por ello no están definidas las operaciones inversas (como son la resta y división aritméticas).
- Existe, en cambio, el elemento complementario.

[Considérese el diferente papel que desempeña el elemento complementario en relación con los elementos inversos: - complementario $a + \bar{a} = 1$ $a \cdot \bar{a} = 0$
- inversos $a + (-a) = 0$ $a \cdot (1/a) = 1$].

A partir de los axiomas que definen el álgebra de Boole pueden deducirse directamente, los siguientes teoremas operativos:

- Dualidad : toda expresión booleana sigue siendo válida si se efectúan, a la vez, los siguientes cambios: $a \leftrightarrow \bar{a}$ + (operación "o") $\leftrightarrow \cdot$ (operación "y") $0 \leftrightarrow 1$
- Idempotencia: $a + a = a$ $a \cdot a = a$
- Absorción: $a + a \cdot b = a$ $a + \bar{a} \cdot b = a + b$
 $a \cdot (a + b) = a$ $a \cdot (\bar{a} + b) = a \cdot b$
- de Morgan: $\overline{a + b} = \bar{a} \cdot \bar{b}$ $\overline{a \cdot b} = \bar{a} + \bar{b}$
- de Consenso: $a \cdot b + \bar{a} \cdot c + b \cdot c = a \cdot b + \bar{a} \cdot c$
 $(a + b) \cdot (\bar{a} + c) \cdot (b + c) = (a + b) \cdot (\bar{a} + c)$

Como simple ejemplo de demostración de teoremas, se incluye a continuación la correspondiente a los teoremas de idempotencia:

$$a + a = a \cdot 1 + a \cdot 1 = a \cdot (1 + 1) = a \cdot 1 = a$$

$$a \cdot a = (a + 0) \cdot (a + 0) = a + 0 \cdot 0 = a + 0 = a$$

[En el primer paso, se hace uso de los axiomas $a \cdot 1 = a$, $a + 0 = a$; en el segundo, se aplica la propiedad distributiva; en el tercer paso, se utilizan los axiomas $a + 1 = 1$, $a \cdot 0 = 0$ y en el cuarto, se emplean los mismos axiomas que en el primer paso.]

El teorema de Dualidad se deduce de que todos los axiomas siguen siendo válidos si se aplican sobre ellos dichos cambios (los tres a la vez) y, por tanto, tales cambios pueden generalizarse a cualquier expresión booleana.

[Debe tenerse en cuenta que la Dualidad ha de ser aplicada a ambos miembros de una expresión booleana y no solamente a uno de ellos:

$$\text{sea la función } y = a + \bar{c} \cdot b$$

$$\text{no es válido hacer } y = a + \bar{c} \cdot b = \bar{a} \cdot (c + \bar{b}) \quad \text{sino } \bar{y} = \bar{a} \cdot (c + \bar{b}) \cdot]$$

El álgebra booleana más simple y de mayor interés práctico es la definida sobre un conjunto de sólo dos elementos, que necesariamente han de ser el 0 y el 1:

negación	operación "o"	a+b	operación "y"	a.b
$\bar{0} = 1$	$0 + 0$	0	$0 \cdot 0$	0
	$0 + 1$	1	$0 \cdot 1$	0
$\bar{1} = 0$	$1 + 0$	1	$1 \cdot 0$	0
	$1 + 1$	1	$1 \cdot 1$	1

En la operación "o" predomina el valor 1, en el sentido de que si un operando tiene dicho valor 1, el resultado también es 1; mientras que para la operación "y" el valor que prevalece es el 0.

Una operación compuesta muy frecuente en el álgebra binaria es la denominada "**o-exclusiva**" $a \oplus b$, que corresponde a la función «ser diferente» o «desigualdad»; la denominación "**o-exclusiva**" deriva de que esta operación coincide con la "o", salvo en el caso 11 que adopta el valor 0:

$$0 \oplus 0 = 0 ; 0 \oplus 1 = 1 ; 1 \oplus 0 = 1 ; 1 \oplus 1 = 0$$

$$a \oplus b \text{ vale } 1 \text{ cuando } a \neq b \text{ y vale } 0 \text{ cuando } a = b,$$

$$\text{es decir, } a \oplus b = 1 \text{ si } a = 0 \text{ y } b = 1 \text{ o, también, si } a = 1 \text{ y } b = 0$$

$$\text{o sea, } a \oplus b = \bar{a} \cdot b + a \cdot \bar{b}.$$

La expresión $\bar{a} \cdot b + a \cdot \bar{b}$ representa la función "**o-exclusiva**" en términos de operaciones booleanas básicas, en la forma de «suma de productos»; también puede expresarse dicha operación como «producto de sumas», en la forma siguiente:

$$\begin{aligned} a \oplus b &= \bar{a} \cdot b + a \cdot \bar{b} = \overline{\overline{\bar{a} \cdot b + a \cdot \bar{b}}} = \overline{\overline{\bar{a} \cdot b} \cdot \overline{a \cdot \bar{b}}} = \overline{(\overline{\bar{a} \cdot b}) \cdot (\overline{a \cdot \bar{b}})} = \overline{(\overline{\bar{a}} + \overline{b}) \cdot (\overline{a} + \overline{\bar{b}})} = \overline{a + b + \bar{a} + \bar{b}} = \\ &= \overline{a + b + \bar{a} + \bar{b}} = \overline{(a + b) \cdot (\bar{a} + \bar{b})} = \overline{(a + b) \cdot (a + b)} = \overline{(a + b)} = \bar{a} + \bar{b} \end{aligned}$$

1.2. Casos de interés de Álgebras de Boole binarias

1.2.1. Lógica Proposicional

Entenderemos por proposición toda frase (afirmación o negación) que admite la asignación de valores verdad (1) y mentira (0). La lógica proposicional trata de la combinación de proposiciones para formar proposiciones compuestas; en tal sentido, deduce nuevas proposiciones a partir de las iniciales.

Las funciones básicas de la lógica proposicional son: negación (**no a**), disyunción (**a ó b**), conjunción (**a y b**), implicación (**a $\Rightarrow$ b**) y equivalencia (**a $\equiv$ b**). Las tres primeras operaciones coinciden, respectivamente, con las operaciones básicas booleanas (negación, "o" e "y") y las otras dos pueden expresarse en la siguiente forma:

- la implicación $a \Rightarrow b$ equivale a la expresión $a \cdot \bar{b} = 0$
 (suceder **a** y que no suceda **b** es falso)
 o a la expresión dual $\bar{a} + b = 1$
 (o no sucede **a** o necesariamente sucede **b**).
- la equivalencia $a \equiv b$ equivale a la expresión $a \cdot b + \bar{a} \cdot \bar{b} = 1$
 (**a** y **b** son, a la vez, ciertas o son ambas falsas).

La lógica proposicional constituye un lenguaje formalizado, capaz de configurar decisiones lógicas a partir de la combinación de proposiciones; tal lenguaje puede ser expresado mediante las funciones del álgebra booleana de dos elementos.

Ejemplos:

a) A Dios rogando y con el mazo dando: $rD \cdot dm = 1$
 (rD : rogar a Dios; dm : dar con el mazo).

b) O me toca la lotería o me pego un tiro $lot + tir = 1$
 (lot : tener suerte en la lotería; tir : pegarme un tiro).

c) En el país de los ciegos el tuerto es rey: $pc.t \Rightarrow R$ $pc \cdot t \cdot \bar{R} = 0$
 o también $\bar{pc} + \bar{t} + R = 1$
 (pc : estar en el país de los ciegos; t : ser tuerto; R : ser rey).

d) Todo hombre es mortal $h \Rightarrow m$ $h \cdot \bar{m} = 0$
 Juan es hombre $j \Rightarrow h$ $J \cdot \bar{h} = 0$
 Luego Juan es mortal $j \Rightarrow m$? $j \cdot \bar{m} = 0$?
 (Efectuando la operación "o" de las dos premisas $h \cdot \bar{m} + J \cdot \bar{h} = 0$
 y por el teorema de consenso $h \cdot \bar{m} + \bar{h} \cdot J + \bar{m} \cdot J = 0$
 para lo cual han de ser nulos todos los «sumandos» $J \cdot \bar{m} = 0$).

e) p es una condición necesaria y suficiente para q : $p \equiv q$ $p \cdot q + \bar{p} \cdot \bar{q} = 1$.

f) Ser aragonés y ser tozudo es una misma cosa : $a \equiv t$ $a \cdot t + \bar{a} \cdot \bar{t} = 1$
 (todos los aragoneses son tozudos y todas las personas tozudas son aragonesas).

g) Me casaré con una persona joven e inteligente o que tenga fortuna y no ronque
 $c = j \cdot i + f \cdot \bar{r}$
 (c : decisión de casarme; j : ser joven; i : inteligente; f : rico; r : roncar).

h) La alarma de un automóvil suena cuando la llave de contacto está puesta y no lo está el cinturón de seguridad y, también, con el cinturón de seguridad puesto si la puerta del conductor está abierta

$a = ll \cdot \bar{c} + c \cdot p$
 (a : alarma; ll : llave de contacto puesta; c : cinturón de seguridad puesto;
 p : puerta del conductor cerrada).

i) En una oficina trabajan cuatro personas y han decidido que la musica ambiental estará activada solamente cuando lo deseen tres de ellas

$$m = p1.p2.p3 + p1.p2.p4 + p1.p3.p4 + p2.p3.p4$$

(m : suena la música;

$p1, p2, p3, p4$: la correspondiente persona -1,2,3,4- desea escuchar música).

1.2.2. Sistema binario de numeración

El sistema de numeración en base 2 emplea solamente los dígitos **0** y **1**, siendo 2^n el valor relativo de la cifra que ocupa el lugar n (contado de derecha a izquierda, partiendo de 0).

El esquema operativo de los cambios de base binario-decimal es el siguiente:

$$e d c b a \text{ (base 2)} = a \cdot 2^0 + b \cdot 2^1 + c \cdot 2^2 + d \cdot 2^3 + e \cdot 2^4 \text{ (base 10)}$$

Ejemplo:

$$10011000100101_2 = 1 \times 1 + 0 \times 2 + 1 \times 4 + 0 \times 8 + 0 \times 16 + 1 \times 32 + 0 \times 64 + 0 \times 128 + 0 \times 256 + 1 \times 512 + 1 \times 1024 + 0 \times 2048 + 0 \times 4096 + 1 \times 8192 = 9765_{10}$$

		Ejemplo:		cocientes	restos
$n^0_{(10)}$	2				
r_1	c_1	2		9765	1
	r_2	c_2	2	4882	0
		r_3	c_3	2441	1
			r_4	1220	0
			r_5	610	0
				305	1
			r_6	152	0
				76	0
				38	0
				19	1
				9	1
				4	0
				2	0
				1	1

$$n^0 \text{ (base 10)} = 1 r_6 r_5 r_4 r_3 r_2 r_1 \text{ (base 2)}$$

$$9765_{10} = 10011000100101_2$$

Cualquier operación de un sistema de numeración puede realizarse mediante un algoritmo que contenga solamente sumas, restas y comparaciones. Ahora bien, en el caso del sistema binario estas tres operaciones pueden construirse con funciones booleanas sencillas; véase a continuación para dos números binarios de 1 solo dígito:

suma:

a	b	suma	acarreo
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

acarreo = «me llevo»

$\text{suma} = \bar{a}.b + a.\bar{b}$

$= a \oplus b$

$\text{acarreo} = a.b$

resta:

a	b	resta	acarreo
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

acarreo = «me llevo»

$\text{resta} = \bar{a}.b + a.\bar{b}$

$= a \oplus b$

$\text{acarreo} = \bar{a}.b$

comparación :

a	b	igual	mayor	menor
0	0	1	0	0
0	1	0	0	1
1	0	0	1	0
1	1	1	0	0

$\text{igual} = \bar{a}.\bar{b} + a.b$

$= \overline{a \oplus b}$

$\text{mayor} = a.\bar{b}$

$\text{menor} = \bar{a}.b$

Obviamente las anteriores expresiones booleanas para la suma, resta y comparación de dos dígitos pueden extenderse a números binarios de más de 1 dígito; precisamente en el capítulo 3 dedicado a bloques aritméticos se estudian sumadores, restadores y comparadores para números binarios de n dígitos.

La expresión de estas tres operaciones aritméticas en términos de operaciones booleanas implica que cualquier cálculo en sistema binario puede realizarse algorítmicamente mediante operaciones del álgebra de Boole de dos elementos.

Sea **dcba** un número binario (base 2): **a** es la cifra de las unidades; el valor relativo de **b** es 2 y podemos referirnos a tal cifra como «dosenas»; **c** puede ser nominada como «cuatrenas» ya que su valor relativo es 4 y **d** cuyo valor relativo es 8 puede ser llamada «octenas».

Habrán 16 números de 4 dígitos, desde el **0000** al **1111** (del 0 al 15 en base 10). Como regla general, toda la numeración digital comienza por 0 = **000...**, no por 1, de forma que n dígitos o bits contienen 2^n posibilidades que irán de 0 a $2^n - 1$.

1.3.3. Álgebra de Conmutadores

Un conmutador o interruptor es un sistema de dos estados:

- cerrado, permitiendo el paso a su través
- abierto, interrumpiendo dicho paso.

Consideremos un conjunto de interruptores controlados de alguna forma y asignemos a cada interruptor una letra **A, B, C,...**:

- el símbolo **A** representa a todos aquellos interruptores, relacionados entre sí, que se encuentran en todo momento en el mismo estado;
- a su vez, se denotan con $\bar{A}$ aquellos interruptores que, en todo momento, se encuentran en estado contrario al que tienen los interruptores **A** en dicho momento; la negación o inversión se realiza mediante algún mecanismo que obliga a un interruptor a adoptar el estado contrario al de otro interruptor.

La operación "**o**" corresponde a conectar dos interruptores en paralelo mientras que la operación "**y**" supone disponerlos en serie:



A partir de estas operaciones y del concepto de negación indicado puede describirse cualquier combinación de interruptores, por compleja que sea la malla que dichos interruptores configuren.

Ejemplo:

Consideremos el caso de dos interruptores eléctricos conmutados: al accionar cualesquiera de ellos cambia el estado (encendida-apagada) de la lámpara conectada al conjunto; su tabla de verdad puede ser la siguiente:

A	B	luz
0	0	1
0	1	0
1	0	0
1	1	1

$$\text{luz} = \bar{A} \cdot \bar{B} + A \cdot B$$

Esta función corresponde a la combinación de conmutadores de la figura a, pero también puede construirse en la forma representada en la figura b que es el esquema circuital correspondiente a los interruptores conmutados comerciales.

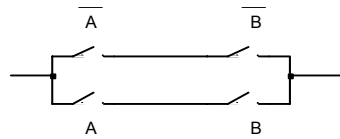


figura a

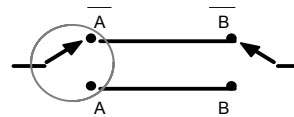


figura b

El álgebra de conmutadores puede realizarse físicamente mediante dispositivos muy diversos: mecánicos, fluidicos, neumáticos, relés, electrónicos..., que constituyen diversas formas de materializar el álgebra de Boole de dos elementos.

Existe un isomorfismo en cuanto a estructura matemática entre el sistema binario de numeración, la lógica proposicional y el álgebra de conmutadores, como álgebras booleanas que son los tres. De dicho isomorfismo se deduce que *el hombre dispone de mecanismos físicos capaces de realizar cálculos numéricos en el sistema binario y de automatizar decisiones lógicas por combinación de proposiciones en el marco de la lógica proposicional.*

Este isomorfismo estructural entre los conmutadores como dispositivos físicos, el sistema binario como sistema de numeración en el que realizar cálculos y la lógica proposicional como lenguaje formal en el que desarrollar deducciones proporciona a las técnicas digitales su magnífica capacidad para procesar información.

1.3. Representación de las operaciones booleanas: puertas lógicas

Las operaciones básicas del álgebra de Boole (negación, "o", "y") se representan gráficamente en la forma que sigue:



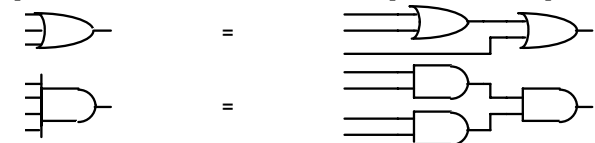
En el primer caso, es el círculo el que expresa directamente la negación o inversión; a veces se utiliza dicho círculo sin más para indicar una negación, como por ejemplo en las siguientes figuras:



En el caso de una entrada de habilitación **E**, la presencia de un círculo en la misma indica que se activa con **0**, en lugar de con **1** (la habilitación se produce cuando **E = 0**); si es una entrada de reloj **CK**, el círculo previo indica que dicho reloj actúa en su flanco negativo (bajadas ↓), en lugar del flanco positivo (subidas ↑).

Los símbolos que representan gráficamente a las operaciones booleanas reciben el nombre de puertas lógicas. En tal sentido las tres primeras figuras son, respectivamente, un inversor, una puerta "o" y una puerta "y". También se emplea el nombre de puerta lógica para designar la realización física de tales operaciones, en particular cuando se construyen con dispositivos electrónicos.

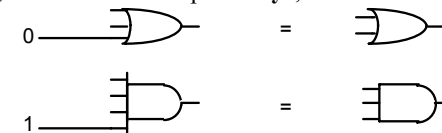
La propiedad asociativa permite utilizar puertas "o" y puertas "y" de tres o más entradas, que expresan la realización sucesiva de la operación correspondiente.



Habida cuenta de que en la operación "o" predomina el valor **1**, las puertas "o" quedan «trabadas» con valor **1** en la salida si una de sus entradas se fija a **1**; lo mismo sucede en las puertas "y" respecto al valor **0**.



En cambio, la forma de «anular» una entrada de una puerta "o" consiste en conectarla a valor **0** y, en el caso de una puerta "y", a valor **1**.

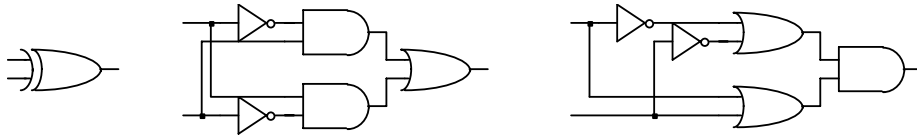


Una puerta "y" de dos entradas **a** **b** puede ser utilizada para «controlar el paso» de una de ellas **a**, permitiéndolo o interrumpiéndolo según que el valor de la otra entrada **b** sea **1** ó **0**; la puerta "y" actúa como interruptor controlado por **b**, que «deja pasar» la señal **a** cuando **b** = **1** y no le permite el paso (salida **0**) cuando **b** = **0**.



Una puerta "o-exclusiva" (función «ser diferente» o «desigualdad») se representa con el símbolo de la figura y equivale a la combinación de cinco puertas básicas:

$$a \oplus b = \bar{a}.b + a.\bar{b} = (\bar{a} + \bar{b}).(a + b)$$



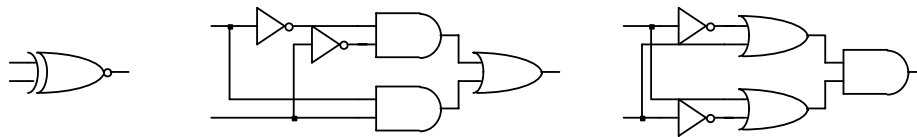
La puerta **o-exclusiva** puede ser utilizada como «inversor de **a** controlado por **b**»:

- cuando **b** = **0**, $y = a \oplus 0 = \bar{a}.0 + a.\bar{0} = a$, la señal **a** pasa directamente,
- para **b** = **1**, $y = a \oplus 1 = \bar{a}.1 + a.\bar{1} = \bar{a}$, la señal **a** pasa en forma invertida.

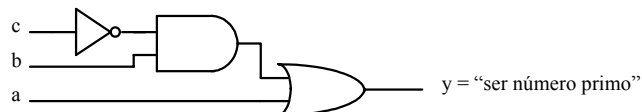


La negación de la puerta "o-exclusiva" se denomina "y-inclusiva" (porque coincide con la operación "y" incluyendo el caso **00**) y corresponde a la función «igualdad»: da resultado **1** cuando los dos operandos son iguales **00**, **11**.

$$a \otimes b = \bar{a}.\bar{b} + a.b = (\bar{a} + b).(a + \bar{b})$$



Veamos una función booleana sencilla: para números binarios de 3 dígitos **bca** (del 0 al 7) la condición "ser número primo" afecta a todos los impares (**a** = **1**) y al número 2 (**010**), resultando la función siguiente: $y = a + \bar{c}.b.\bar{a} = a + \bar{c}.b$



1.4. Operaciones unitarias

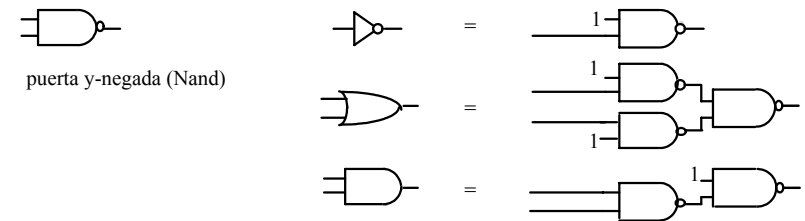
Aunque en la definición del álgebra de Boole se establecen tres operaciones básicas, existe un tipo especial de operaciones que podemos llamar «unitarias» porque con sólo una de ellas se puede realizar toda el álgebra; las principales operaciones unitarias son la operación "y-negada" (Nand) y la operación "o-negada" (Nor).

- operación "y-negada" (Nand): $a * b = \overline{a.b}$

$$\bar{a} = \overline{a.1} = a * 1$$

$$a + b = \overline{\bar{a}.\bar{b}} = \overline{a * b} = (a * 1) * (b * 1)$$

$$a.b = \overline{\overline{a.b}} = \overline{a * b} = (a * b) * 1$$

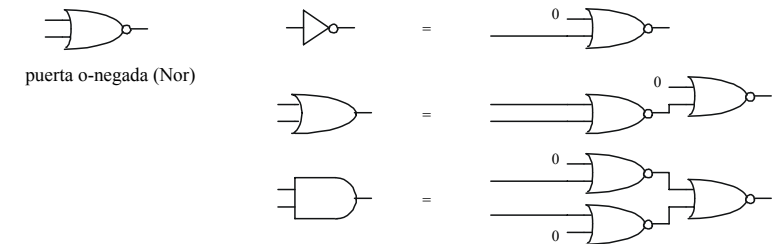


- operación "o-negada" (Nor): $a \Delta b = \overline{a + b}$

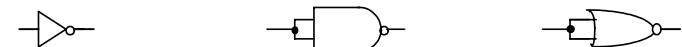
$$\bar{a} = \overline{a + 0} = a \Delta 0$$

$$a + b = \overline{\overline{a + b}} = \overline{a \Delta b} = (a \Delta b) \Delta 0$$

$$a.b = \overline{\overline{a + b}} = \overline{a \Delta b} = (a \Delta 0) \Delta (b \Delta 0)$$



Comentario: la inversión puede expresarse según la figura siguiente, pero (aunque «en el papel» no hay diferencia con la anterior) técnicamente es peor solución porque supone una «carga» de dos entradas (con doble intensidad o capacidad de entrada).



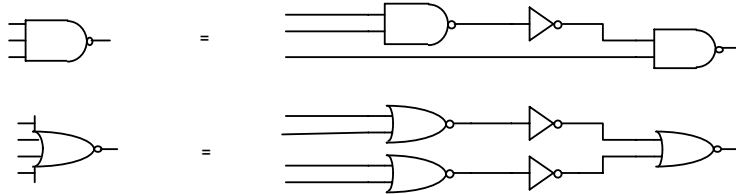
También otras operaciones como $\bar{a}.b$ o $\bar{a} + b$ son unitarias, pero no son utilizadas por no poseer la propiedad conmutativa.

Las operaciones "**y-negada**" y "**o-negada**" son conmutativas pero no asociativas:

$$(a * b) * c \neq a * (b * c)$$

$$(a \Delta b) \Delta c \neq a \Delta (b \Delta c)$$

Por ello, las correspondientes puertas "**o-negada**" e "**y-negada**" de tres o más entradas no equivalen directamente a la realización sucesiva de tales operaciones con dos entradas; es preciso añadir una inversión intermedia en cada paso (tal inversión restituye la propiedad asociativa al conjunto, al negar la operación "**o/y-negada**" y convertirla en operación "**o/y**"):



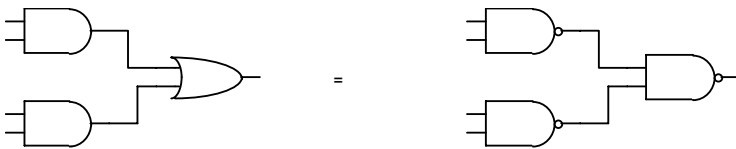
El álgebra de Boole realizada con una de estas operaciones unitarias es muy uniforme (solamente utiliza un tipo de puertas lógicas), pero menos potente por carecer de la propiedad asociativa. Esta carencia determina la ineludible necesidad de colocar los correspondientes paréntesis:

- las expresiones $a * b * c$ y $a \Delta b \Delta c$ carecen de sentido, pues no indican el orden en que deben aplicarse las dos operaciones contenidas en ellas;
- las puertas de tres o más entradas deben expresarse como **Nand(a,b,c)** y **Nor(a,b,c)**, respectivamente.

La operación "**y-negada**" se acomoda bien a expresiones del tipo «suma de productos», $a.b + c.d$, ya que pueden convertirse directamente en la forma que sigue:

$$a.b + c.d = \overline{\overline{a.b + c.d}} = \overline{(\overline{a.b}) \cdot (\overline{c.d})} = (a * b) * (c * d)$$

[En las dos expresiones, el número de puertas y la conexión entre ellas son idénticos; basta sustituir directamente todas las puertas "**o**" e "**y**" por puertas "**y-negada**".]



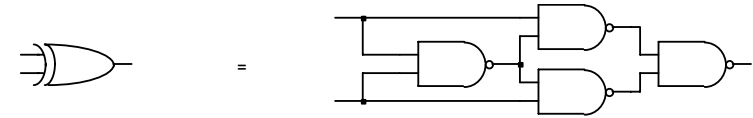
En cambio, la operación "**o-negada**" se acomoda bien a expresiones del tipo «producto de sumas», $(a+b) \cdot (c+d)$:

$$(a + b).(c + d) = \overline{\overline{(a + b).(c + d)}} = \overline{(\overline{a + b}) + (\overline{c + d})} = (a \Delta b) \Delta (c \Delta d)$$



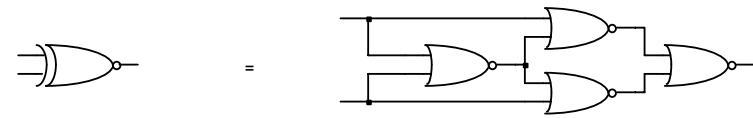
La operación "**o-exclusiva**" (*ser diferentes*) puede construirse con solamente cuatro puertas "**y-negada**" mediante la siguiente transformación:

$$\begin{aligned} a \oplus b &= \overline{a}.b + a.\overline{b} = \overline{a}.b + a.\overline{b} + a.\overline{a} + b.\overline{b} = a.(\overline{a} + \overline{b}) + b.(\overline{a} + \overline{b}) = \\ &= a.(\overline{a}.b) + b.(\overline{a}.b) = a.(a * b) + b.(a * b) = [a * (a * b)] * [b * (a * b)] \end{aligned}$$



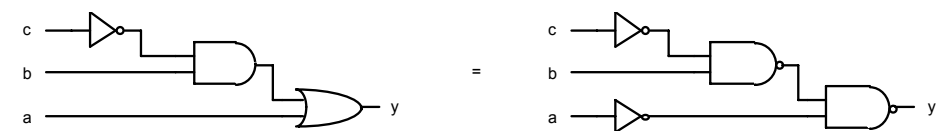
Y, del mismo modo la operación "**y-inclusiva**" (*igualdad*) puede construirse con solamente cuatro puertas "**o-negada**" mediante la transformación:

$$\begin{aligned} a \otimes b &= (\overline{a} + b).(a + \overline{b}) = (\overline{a} + b).(a + \overline{b}).(\overline{a} + \overline{a}).(\overline{b} + \overline{b}) = (a + \overline{a}.b).(b + \overline{a}.b) = \\ &= [a + (a \Delta b)] \cdot [b + (a \Delta b)] = [a \Delta (a \Delta b)] \Delta [b \Delta (a \Delta b)] \end{aligned}$$



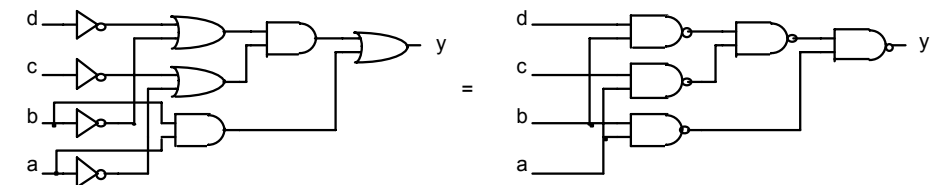
Veamos un par de ejemplos de transformación de expresiones booleanas con operaciones básicas a expresiones con puertas "**y-negada**":

$$y_1 = \text{"ser número primo de 3 dígitos"} = a + \overline{c}.b = a + \overline{c}.b = \overline{\overline{a + \overline{c}.b}} = \overline{\overline{a} \cdot (\overline{\overline{c}.b})} = \overline{a * (\overline{c} * b)}$$



Por último, sea la siguiente función y_2 :

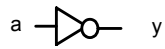
$$\begin{aligned} y_2 &= \overline{d}.c + \overline{d}.a + c.\overline{b} + b.a + \overline{b}.a = \overline{d}.(\overline{c} + a) + b.(\overline{c} + a) + b.a = \\ &= (\overline{c} + a).(\overline{d} + b) + b.a = \overline{\overline{(\overline{c} + a).(\overline{d} + b) + b.a}} = \overline{[(\overline{c} + a).(\overline{d} + b)] \cdot (\overline{b}.a)} = \\ &= [(\overline{c} + a) * (\overline{d} + b)] * (b * a) = [(\overline{c} + a) * (\overline{d} + b)] * (b * a) = \\ &= [(\overline{c}.a) * (\overline{d}.b)] * (b * a) = [(c * a) * (d * b)] * (b * a) \end{aligned}$$



1.5 Las operaciones booleanas en lenguaje de descripción circuital: VHDL

Actualmente el diseño de circuitos digitales no se expresa en forma gráfica sino en forma de texto, a través de lenguajes de descripción circuital; el más utilizado de tales lenguajes es el VHDL.

A fin de desarrollar una aproximación gradual al diseño con lenguajes de descripción circuital, se incluye aquí la forma de describir las operaciones lógicas en VHDL; asimismo, al final del capítulo 4 se incluye la descripción de bloques combinacionales en VHDL.



- a) $y \leq \text{not } a$;
b) $y \leq '1'$ when $a = '0'$ else $'0'$;



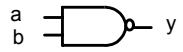
- a) $y \leq a \text{ or } b$;
b) $y \leq a$ when $b = '0'$ else $'1'$;



- a) $y \leq a \text{ and } b$;
b) $y \leq a$ when $b = '1'$ else $'0'$;



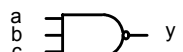
- a) $y \leq a \text{ xor } b$;
b) $y \leq \text{not } a$ when $b = '1'$ else a ;
c) $y \leq '0'$ when $a = b$ else $'1'$;



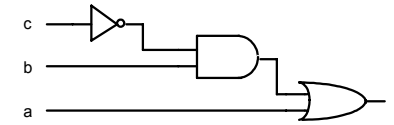
- a) $y \leq a \text{ nand } b$;
b) $y \leq '0'$ when $(a = '1')$ and $(b = '1')$ else $'1'$;



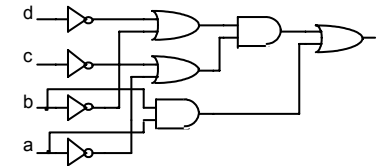
- a) $y \leq a \text{ nor } b$;
b) $y \leq '1'$ when $(a = '0')$ and $(b = '0')$ else $'0'$;



- a) $y \leq \text{not } (a \text{ and } b \text{ and } c)$;
 $y \leq '0'$ when $(a = '1')$ and $(b = '1')$ and $(c = '1')$ else $'1'$;



$y \leq a \text{ or } (b \text{ and not } c)$;



$y \leq ((\text{not } d \text{ or not } b) \text{ and } (\text{not } c \text{ or not } a)) \text{ or } (b \text{ and } a)$;

$y \leq \text{not } ((d \text{ and } b) \text{ or } (c \text{ and } a)) \text{ or } (b \text{ and } a)$;

2 FUNCIONES BOOLEANAS Y SU SIMPLIFICACION

- 2.1. Funciones Lógicas
- 2.2. Simplificación de funciones booleanas: mapas de Karnaugh
- 2.3. Ejercicios de síntesis y simplificación de funciones booleanas
- 2.4. Decodificadores y multiplexores; otras formas de configurar funciones

El álgebra de Boole permite expresar, en forma de funciones matemáticas, tanto la realización de cálculos en el sistema binario como la adopción de decisiones a través de la combinación de proposiciones.

Cantidades y cualidades pueden ser representadas por conjuntos de «ceros» y «unos», es decir, mediante palabras binarias cuyos dígitos pueden adoptar solamente los valores 0 y 1; cada dígito o «bit» corresponde a una variable.

Una función booleana establece una dependencia entre una variable de salida "y" y un conjunto de variables de entrada "a b c...": una correspondencia entre el conjunto de valores de las variables de entrada y el valor de la variable de salida.

Las funciones booleanas son «multiformes», es decir, pueden representarse de muy diversas formas: desde el mero enunciado textual que expresa las especificaciones o requisitos que definen la función, hasta su forma algebraica como operaciones entre variables, pasando por su tabla funcional (o «tabla de verdad») que detalla, en forma de listado, el valor de la función para cada conjunto de valores de las entradas.

Precisamente el diseño del circuito digital correspondiente a una función booleana consiste en el «cambio de forma» de la misma, a partir de su enunciado, construyendo su tabla funcional y extrayendo de ella la forma algebraica de la función; dicha expresión algebraica puede ser trasladada directamente a un esquema de puertas lógicas que conforma el circuito digital de dicha función.

En este proceso resulta de mucha importancia la simplificación de la expresión algebraica de la función, de forma que contenga el menor número de términos y el menor número de variables posible. Al reducir la expresión algebraica disminuye el tamaño, la complejidad y el coste (y, en muchos casos, aumenta la velocidad) del circuito digital que permite «obtener» tal función. Con esta finalidad, los «mapas de Karnaugh» constituyen una eficaz herramienta gráfica de simplificación «a mano», mientras que el método de Quine-McCluskey proporciona las bases algorítmicas para programar la simplificación sobre un computador.

También es posible configurar el circuito digital de una función booleana sin llegar a su expresión algebraica, directamente desde su tabla funcional. Para ello pueden utilizarse dos bloques digitales de tipo «selector»: el decodificador y el multiplexor; ambos incluyen todas las posibilidades de valores de sus variables de entrada y permiten activar cada una de dichas posibilidades.

2.1. Funciones Lógicas

Dentro del Álgebra de Boole de 2 elementos, una función booleana o función lógica es una expresión de operaciones booleanas enlazando variables que solamente pueden adquirir los valores 0 y 1. Una función booleana es una aplicación que a cada conjunto de valores booleanos de sus variables le asigna un y sólo un valor booleano.

La primera de las dos definiciones anteriores es de tipo «descriptivo»: describe la forma algebraica de una función booleana; mientras que la segunda es de tipo «conceptual»: identifica la función como correspondencia entre el conjunto de valores de las variables y el valor booleano de la variable dependiente.

En una función **f** designaremos con el nombre de *variables de entrada* x_i al conjunto de sus variables propias y denominaremos *variable de salida* **y** a la variable dependiente o resultado de la propia función $y = f(x_i)$.

De acuerdo con las definiciones anteriores, las funciones lógicas pueden representarse en dos formas diferentes:

- por su *expresión algebraica* o *fórmula booleana*, como expresión de las operaciones que ligan a sus variables;
- por su *tabla operativa* o *tabla de verdad*, expresando en forma de tabla la correspondencia entre la variable de salida y cada combinación posible de valores de sus variables de entrada.

También puede expresarse una función en forma de enunciado o texto que manifiesta las especificaciones o requisitos que dan lugar a dicha función y en forma gráfica como circuito digital o esquema de puertas lógicas que «produce» los valores de salida de la función al recibir los correspondientes valores en sus entradas.

El proceso de síntesis o «construcción digital» de una función parte del enunciado o especificaciones de la misma, para configurar la «tabla de verdad» de la función y obtener, a través de ella, su expresión algebraica; una vez simplificada, dicha expresión puede ser directamente trasladada a un esquema de puertas como representación gráfica del circuito digital que «hace efectiva» dicha función.

enunciado → tabla funcional → expresión algebraica → esquema de puertas

Dada una función de **m** variables, cada una de las posibles combinaciones de valores de dichas **m** variables recibe el nombre de *vector de entrada*; el número total de vectores de entrada será 2^m y tal será el número de filas que ha de tener la tabla funcional completa.

Para cada vector de entrada podemos construir un *término mínimo*, formado por el producto booleano (operación "y") de las **m** variables de entrada, estando cada una de ellas afirmada si su valor en el vector de entrada es 1 y negada cuando vale 0.

Un término mínimo da resultado **1** al asignar a sus variables los valores del vector de entrada que le corresponde y, en cambio, para cualquier otro vector de entrada da resultado **0**.

Así pues, tal como están contruidos, los términos mínimos poseen la propiedad de «seleccionar» o «filtrar» a su propio vector de entrada: a cada vector de entrada le corresponde un término mínimo y cada término mínimo es un *discriminador* o *selector* del vector de entrada al que corresponde.

Ejemplos para 4 variables (*a, b, c, d*):

vector de entrada 1001	término mínimo	$a.\bar{b}.\bar{c}.d$	$1.0.0.1 = 1$
vector de entrada 1100	término mínimo	$a.b.\bar{c}.\bar{d}$	$1.1.0.0 = 1$
vector de entrada 0010	término mínimo	$\bar{a}.\bar{b}.c.\bar{d}$	$0.0.1.0 = 1$
vector de entrada 0000	término mínimo	$\bar{a}.\bar{b}.\bar{c}.\bar{d}$	$0.0.0.0 = 1$
vector de entrada 1111	término mínimo	$a.b.c.d$	$1.1.1.1 = 1$

número de vectores de entrada y de términos mínimos posibles: $2^4 = 16$

Dualmente se construye el *término máximo* correspondiente a un vector de entrada mediante la suma booleana (operación "o") de sus variables, afirmadas cuando su valor es **0** y negadas cuando valen **1**. Un término máximo da resultado **0** al asignar a sus variables los valores del vector de entrada que le corresponde y adopta el valor **1** para cualquier otro vector.

A cada vector de entrada le corresponden un término máximo y cada término máximo es, asimismo, un *selector* o *discriminador* del vector al que corresponde.

Ejemplos para 4 variables (*a, b, c, d*):

vector de entrada 1001	término máximo	$\bar{a} + b + c + \bar{d}$	$\bar{1} + 0 + 0 + \bar{1} = 0$
vector de entrada 1100	término máximo	$\bar{a} + \bar{b} + c + d$	$\bar{1} + \bar{1} + 0 + 0 = 0$
vector de entrada 0010	término máximo	$a + b + \bar{c} + d$	$0 + 0 + \bar{1} + 0 = 0$
vector de entrada 0000	término máximo	$a + b + c + d$	$0 + 0 + 0 + 0 = 0$
vector de entrada 1111	término máximo	$\bar{a} + \bar{b} + \bar{c} + \bar{d}$	$\bar{1} + \bar{1} + \bar{1} + \bar{1} = 0$

número de vectores de entrada y de términos máximos posibles: $2^4 = 16$

Se denomina **forma canónica** de una función booleana a su expresión como suma (operación "o") de términos mínimos de sus variables; para construirla a partir de su tabla funcional bastará tomar todos aquellos términos mínimos que corresponden a vectores de entrada que hacen la función igual a **1** y sumarlos.

Habida cuenta de la propiedad de filtrado o selección que tienen los términos mínimos, dicha suma de términos mínimos asigna resultado **1** para aquellos vectores de entrada cuyo término mínimo se encuentra presente en la misma y resultado **0** para todos los demás vectores de entrada.

Ejemplo: función "ser número primo" para números de 3 dígitos.

c	b	a	ser nº primo	
0	0	0	0	← $c + b + a$
0	0	1	1	← $\bar{c}.\bar{b}.a$
0	1	0	1	← $\bar{c}.b.\bar{a}$
0	1	1	1	← $\bar{c}.b.a$
1	0	0	0	← $\bar{c} + b + a$
1	0	1	1	← $c.\bar{b}.a$
1	1	0	0	← $\bar{c} + \bar{b} + a$
1	1	1	1	← $c.b.a$

Para obtener la forma canónica de esta función hemos de tomar los términos mínimos de los vectores de entrada que dan resultado **1**.

$$y = \bar{c}.\bar{b}.a + \bar{c}.b.\bar{a} + \bar{c}.b.a + c.\bar{b}.a + c.b.a \quad \text{forma canónica}$$

$$\text{simplificando}$$

$$= \bar{c}.a + \bar{c}.b + c.a = a + \bar{c}.b$$

La **forma canónica dual** de una función booleana corresponde a su expresión como producto (operación "y") de términos máximos; para construirla se toman todos aquellos términos máximos que corresponden a vectores de entrada que hacen la función igual a **0** y se realiza el producto de ellos.

La propiedad de filtrado o selección que tienen los términos máximos asegura que su producto asigna resultado **0** para aquellos vectores de entrada cuyo término máximo está presente y resultado **1** para todos los demás vectores de entrada.

En el ejemplo anterior (función "ser número primo" para números de 3 dígitos), para obtener la forma canónica dual tomamos los términos máximos de los vectores de entrada que dan resultado **0**.

$$y = (c + b + a) . (\bar{c} + b + a) . (\bar{c} + \bar{b} + a) \quad \text{forma canónica dual}$$

$$\text{simplificando}$$

$$= (b + a) . (\bar{c} + a)$$

La forma canónica de una función es única, salvo en lo que se refiere al orden de los términos y al de las variables en los mismos; igualmente es única la forma canónica dual. En cambio, una misma función puede tener expresiones simplificadas muy diversas.

Dos funciones de m variables diferirán en los términos mínimos que contenga su forma canónica; el número de funciones diferentes de m variables coincide con el de combinaciones posibles de sus 2^m términos mínimos: 2^k siendo $k = 2^m$.

El número de vectores de entrada, y consiguientemente el de funciones posibles, crece fuertemente al hacerlo el número de variables de la función. Hay 16 funciones de 2 variables, las funciones de 4 variables tendrán 16 vectores de entrada, siendo posibles $2^{16} = 65.536$ funciones y para 6 variables existen 64 vectores de entrada con un número de funciones posibles 2^{64} superior a 10^{18} (trillones de ellas).

2.2. Simplificación de funciones booleanas: mapas de Karnaugh

La aplicación de los teoremas del álgebra permite simplificar las funciones, reduciendo el número de puertas necesarias para su configuración; en concreto, los teoremas más útiles para la simplificación son los de idempotencia y absorción:

$$\begin{array}{llll} a + a = a & a + \bar{a} = 1 & a + a.b = a & a + \bar{a}.b = a + b \\ a . a = a & a . \bar{a} = 0 & a.(a + b) = a & a.(\bar{a} + b) = a.b \end{array}$$

y, en ocasiones, el teorema de consenso: $a.b + \bar{a}.c + b.c = a.b + \bar{a}.c$

Ejemplo: $Y = \bar{d}.c.\bar{b}.\bar{a} + \bar{d}.c.\bar{b}.a + \bar{d}.\bar{c}.\bar{b}.\bar{a} + \bar{d}.\bar{c}.\bar{b}.a + \bar{d}.c.\bar{b}.\bar{a} + \bar{d}.c.\bar{b}.a + \bar{d}.c.\bar{b}.\bar{a}$
($x + \bar{x} = 1$ aplicado a las siguientes parejas de términos: 1º y 2º $x=a$; 3º y 4º $x=b$; 5º y 6º $x=a$; 7º y 4º $x=c$, utilizando, en este último caso, también, $x=x+\bar{x}$)

$$Y = \bar{d}.c.\bar{b} + \bar{d}.\bar{c}.\bar{a} + \bar{d}.c.\bar{b} + \bar{d}.\bar{b}.\bar{a}$$

($d + \bar{d} = 1$ aplicado a los términos 1º y 3º y sacando factor común en los otros dos)

$$Y = c.\bar{b} + \bar{d}.\bar{a}.\bar{c} + b$$

(aplicando el teorema de Morgan al paréntesis del segundo término)

$$Y = c.\bar{b} + \bar{d}.\bar{a}.\bar{c} + b$$

(y, finalmente, el teorema de absorción $x + \bar{x}.z = x + z$ aplicado a ambos términos)

$$Y = c.\bar{b} + \bar{d}.\bar{a}.$$

La aplicación directa de teoremas booleanos para simplificar las funciones requiere una cierta habilidad, cuyos resultados dependen de la complejidad de la función y de la experiencia e intuición de quien la realiza.

Existen métodos de simplificación que aportan una formulación sistemática del proceso y que aseguran la máxima simplificación; los más utilizados de ellos son el método gráfico de los *mapas de Karnaugh*, que se describe a continuación, y el método algorítmico de *Quine-McCluskey* (cuya descripción se encuentra en el apéndice A1).

Los mapas de Karnaugh son el método habitual de simplificación cuando se hace «a mano» y el número de variables de la función es pequeño (no superior a 6). Para mayor número de variables se recurre a la ayuda del computador, con programas de simplificación automática que suelen estar basados en el algoritmo de Quine-McCluskey.

La simplificación de una función por medio de los *mapas de Karnaugh* se realiza dibujando su tabla de operación en un diagrama bidimensional según la estructura siguiente:

		B A			
C		0 0	0 1	1 1	1 0
	0				
	1				

Mapa de Karnaugh para 3 variables

		B A			
D C		0 0	0 1	1 1	1 0
	0 0				
	0 1				
	1 1				
	1 0				

Mapa de Karnaugh para 4 variables

		<u>C B A</u>								
<u>E D</u>		0 0 0	0 0 1	0 1 1	0 1 0		1 1 0	1 1 1	1 0 1	1 0 0
	0 0									
	0 1									
	1 1									
	1 0									

Mapa de Karnaugh para 5 variables

<u>F E D</u>	<u>C B A</u>							
	0 0 0	0 0 1	0 1 1	0 1 0	1 1 0	1 1 1	1 0 1	1 0 0
0 0 0								
0 0 1								
0 1 1								
0 1 0								
1 1 0								
1 1 1								
1 0 1								
1 0 0								

Mapa de Karnaugh para 6 variables

La estructura de los mapas de Karnaugh aprovecha las propiedades del *código Gray*, en el que dos números o vectores sucesivos difieren únicamente en el valor de una variable.

	Binario	Gray
0	0	0
1	1	1
2	10	11
3	11	10
4	100	110
5	101	111
6	110	101
7	111	100
8	1000	1100
9	1001	1101
15	1111	1000

Una forma sencilla de generar el *código Gray* para números sucesivos a partir del 0 es la representada en el siguiente esquema:

0	1	11	10	110	111	101	100	1100	1101	...
---	---	----	----	-----	-----	-----	-----	------	------	-----

Las líneas verticales señalan la necesidad de añadir un nuevo dígito y actúan «a manera de espejo», de forma que tras cada línea vertical los números empiezan por un primer dígito adicional 1 y el resto de sus dígitos son iguales a los del número que se encuentra en posición simétrica respecto a dicha línea vertical, «simetría especular», añadiendo, en su caso, los ceros que fueran necesarios.

El código Gray aparece con frecuencia en los sistemas digitales por sus prestaciones en relación con la simplificación y con la seguridad funcional, derivadas ambas de la propiedad de que dos números sucesivos solamente difieren en el valor de un dígito (son iguales salvo en un bit).

Los términos mínimos correspondientes a dos vectores sucesivos, según el código Gray, son simplificables entre sí pues difieren solamente en el valor de una de sus variables. Por ello, los mapas de Karnaugh tienen la propiedad de que dos cuadros adyacentes se pueden simplificar entre sí.

El proceso de simplificación en un mapa de 4 variables consiste en agrupar los cuadros para formar rectángulos que contengan un número de cuadros potencia de 2: 1, 2, 4, 8; los cuadros contenidos en cada uno de dichos rectángulos son simplificables entre sí y conducen a un solo término:

<u>DC</u>	<u>B A</u>			
	0 0	0 1	1 1	1 0
0 0				
0 1		1	1	
1 1		1	1	
1 0				

$$y = C.A$$

<u>DC</u>	<u>B A</u>			
	0 0	0 1	1 1	1 0
0 0			1	1
0 1				
1 1				
1 0			1	1

$$y = \bar{C}.B$$

<u>DC</u>	<u>B A</u>			
	0 0	0 1	1 1	1 0
0 0		1		
0 1		1		
1 1		1		
1 0		1		

$$y = \bar{B}.A$$

<u>DC</u>	<u>B A</u>			
	0 0	0 1	1 1	1 0
0 0	1			1
0 1				
1 1				
1 0	1			1

$$y = \bar{C}.\bar{A}$$

(téngase en cuenta que el diagrama conecta consigo mismo por sus bordes vertical y horizontal)

DC	BA			
	00	01	11	10
00				
01				
11	1	1		
10				

$$y = D.C.\bar{B}$$

DC	BA			
	00	01	11	10
00			1	
01			1	
11				
10				

$$y = \bar{D}.B.A$$

En el caso de 5 variables, el mapa de Karnaugh está conformado por dos hojas de dimensión 4 x 4 y son simplificables entre sí aquellos rectángulos que se encuentran en posición simétrica respecto a la separación de las dos hojas:

ED	CBA							
	000	001	011	010	110	111	101	100
00								
01	1	1					1	1
11	1	1					1	1
10								

$$y = D.\bar{B}$$

Para 6 variables, el mapa de Karnaugh consta de 4 hojas de dimensión 4 x 4 e igualmente la simplificación entre hojas actúa por simetría especular:

FED	CBA							
	000	001	011	010	110	111	101	100
000								
001				1	1			
011				1	1			
010								
110								
111				1	1			
101				1	1			
100								

$$y = D.B.\bar{A}$$

Si bien en todos los casos anteriores se ha aplicado la simplificación a cuadros con valor **1**, lo cual conduce a términos en forma de productos, *términos producto*, por dualidad el mismo procedimiento de simplificación puede aplicarse en forma análoga a cuadros con valor **0**, dando lugar a *términos suma*.

Dada una función y representada su tabla de verdad en forma de mapa de Karnaugh la expresión más simple de dicha función como suma de términos producto se obtiene agrupando todos los cuadros de valor **1** en el menor número de rectángulos simplificables y expresando la suma de los términos que corresponden a dichos rectángulos.

Dualmente, la forma más simple de tal función como producto de términos suma se obtiene agrupando los cuadros con valor **0** en el menor número de rectángulos y expresando el producto de los términos duales que les corresponden.

Ejemplo:

Sea la función "ser número primo" en el caso de números binarios de 5 dígitos *edcba*.

ed	cba							
	000	001	011	010	110	111	101	100
00	0	1	1	1	0	1	1	0
01	0	0	1	0	0	0	1	0
11	0	0	0	0	0	1	1	0
10	0	1	1	0	0	1	0	0

Recorriendo los cuadros con valor 1 para formar rectángulos simplificables desde la izquierda hacia la derecha y de arriba hacia abajo :

$$y = \bar{e}.\bar{d}.a + \bar{e}.\bar{d}.\bar{c}.b + \bar{e}.\bar{c}.b.a + \bar{d}.\bar{c}.a + d.c.\bar{b}.a + e.c.b.a$$

Simplificando en forma de producto de sumas, para lo cual han de recorrerse los cuadros con valor 0 formando rectángulos simplificables:

$$y = (b + a).(\bar{d} + c + b).(\bar{d} + a).(\bar{e} + \bar{d} + c).(\bar{e} + a).(\bar{c} + a).(e + \bar{d} + \bar{c} + \bar{b}).(\bar{e} + d + \bar{c} + b)$$

Aunque esta segunda (producto de sumas) parece más extensa que la anterior (suma de productos), ambas contienen el mismo número de variables (22 variables).

En aquellos casos en que algún vector de entrada no puede presentarse nunca o bien cuando no importa (*da igual*) el valor que adquiera la función booleana para algún vector de entrada, se anota con **X** el valor que corresponde a tales vectores de entrada. El símbolo **X** (*don't care*) expresa que es indiferente el valor que la función pueda tener y, posteriormente, se utiliza en lugar de **X** el valor **0** o el valor **1** según interese en el proceso de simplificación.

Consideremos las nueve cifras decimales codificadas en binario (código **BCD**) y la misma función booleana "ser número primo" aplicada a ellas.

El número binario para representar las cifras decimales, 0 - 9, ha de disponer de 4 dígitos **dcba**; son números primos las siguientes cifras: 1, 2, 3, 5 y 7. El mapa de Karnaugh correspondiente a la función que determina si una cifra decimal **BCD** es número primo o no será el siguiente:

		B A			
D C		0 0	0 1	1 1	1 0
	0 0	0	1	1	1
	0 1	0	1	1	0
	1 1	X	X	X	X
	1 0	0	0	X	X

El símbolo **X** en un cuadro indica que resulta indiferente el valor booleano que adopte la función para el correspondiente vector de entrada; en este caso ello es debido a que no van a presentarse entradas con valor decimal superior a 9.

Los cuadros con valor **X** son tomados como **1** o como **0** según interese a efectos simplificativos. De esta forma podemos agrupar los cuadros con valor **1** en dos cuadrados (asignando valor **1** a los dos cuadros con valor **X** de la última fila), cuyos términos producto son, respectivamente:

$$y = \bar{d}.a + \bar{c}.b = \overline{\overline{d}.a + \bar{c}.b} = \overline{(\bar{d}.a) . (\bar{c}.b)} = (\bar{d} * a) * (\bar{c} * b)$$

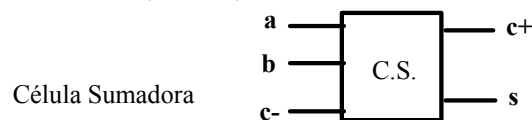
2.3. Ejercicios de síntesis y simplificación de funciones booleanas

2.3.1. Expresar en forma de funciones booleanas la suma aritmética de dos números binarios de varios dígitos

La suma aritmética de dos bits requiere dos funciones booleanas, una para expresar el dígito resultante y la otra para el posible dígito de *acarreo* o *arrastre* (*carry*):

1+1 = 10 ; resultado **0** y «me llevo» **1**.

Para configurar un sumador de números de varios dígitos, podemos proceder modularmente, empleando una celda para cada dígito: dicha celda ha de tener, además de las entradas correspondientes a los dos dígitos de ambos números, una entrada adicional de acarreo que reciba el arrastre (*me llevo*) resultante de la suma de los dígitos anteriores.



Se indica con la notación **c+** el acarreo resultante de la suma y con **c-** el acarreo anterior, que participa como sumando; la tabla funcional será la siguiente:

c-	b	a	suma: s	acarreo: c+
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

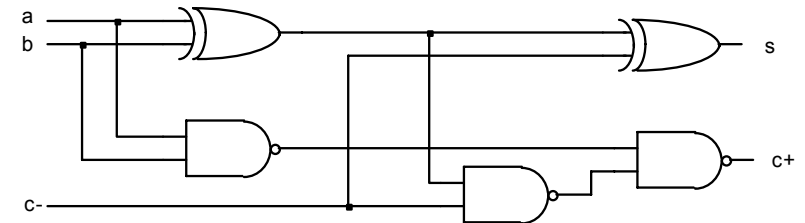
suma:

$$s = \bar{c}_- . (\bar{b}.a + b.\bar{a}) + c_- . (\bar{b}.\bar{a} + b.a) = \bar{c}_- . (b \oplus a) + c_- . (b \otimes a) = \bar{c}_- . (b \oplus a) + c_- . (\overline{b \oplus a}) = c_- \oplus (b \oplus a)$$

arrastre:

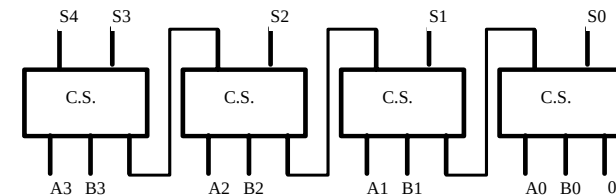
$$c_+ = \bar{c}_- . b.a + c_- . (\bar{b}.a + b.\bar{a} + b.a) = b.a + c_- . (\bar{b}.a + b.\bar{a}) = b.a + c_- . (b \oplus a) = (b * a) * [c_- * (b \oplus a)]$$

Para configurar estas funciones se requieren 3 puertas "y-negada" y 2 puertas "o-exclusiva" conectadas según la figura siguiente:



Si se desea utilizar únicamente puertas "y-negada" serán necesarias 11 puertas (ya que una puerta "o-exclusiva" puede configurarse con 4 puertas "y-negada").

Conectando **n** de estas células sumadoras «en cadena» (salida **c+** unida a entrada **c-** de la celda siguiente) se obtendrá un sumador de dos números de **n** dígitos.



2.3.2. Sean dos números binarios de dos dígitos; deducir las tres funciones que realizan la comparación entre ambos números (mayor, menor e igual), expresándolas mediante operaciones "y-negada" (Nand) e inversores

Sean los dos números: **a1a0** y **b1b0**; la forma sistemática de sintetizar las funciones de comparación entre ellos requiere escribir la tabla de verdad de las tres funciones, pero resulta más sencillo acudir a un razonamiento directo:

- para comparar dos números hay que comenzar por la cifra más significativa y, en el caso de que ambos dígitos sean iguales, irse desplazando hacia la derecha efectuando la comparación cifra a cifra
- dos dígitos son iguales cuando ambos valen **0** o ambos valen **1**: $\bar{a}. \bar{b} + a.b$.

Los dos números propuestos serán iguales cuando tengan iguales su primer y su segundo dígito:

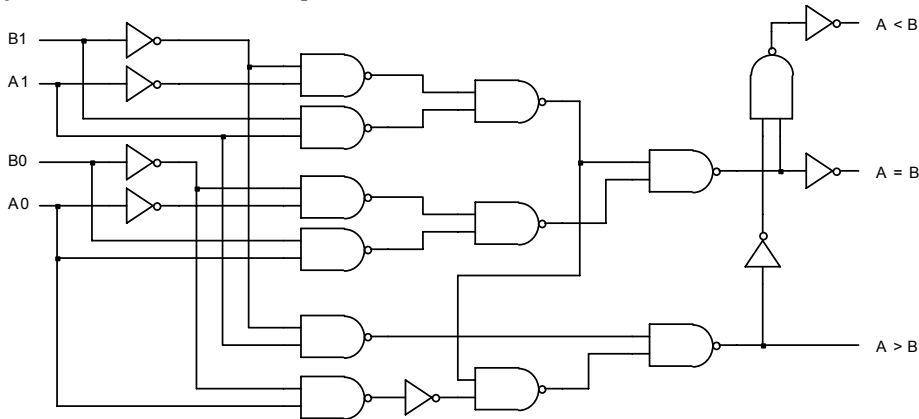
$$y_1 = (\bar{a}_1.\bar{b}_1 + a_1.b_1).(\bar{a}_0.\bar{b}_0 + a_0.b_0) = \overline{[(\bar{a}_1 * \bar{b}_1) * (a_1 * b_1)] * [(a_0 * \bar{b}_0) * (a_0 * b_0)]}$$

función que requiere 7 puertas "y-negada" y 5 inversores.

El número **A** será mayor que el **B** en caso de que el dígito **a1** sea **1** y el **b1** sea **0**, o también en caso de que, siendo ambos dígitos iguales, **a0** tenga valor **1** y **b0** sea **0**:

$$y_2 = a_1.\bar{b}_1 + (\bar{a}_1.\bar{b}_1 + a_1.b_1).(a_0.\bar{b}_0) = (a_1 * \bar{b}_1) * \overline{[(\bar{a}_1 * \bar{b}_1) * (a_1 * b_1)] * (a_0 * \bar{b}_0)}$$

función que necesita otras 7 puertas "y-negada" y otros 5 inversores, pero 3 de las puertas y 4 inversores coinciden con puertas utilizadas en la función anterior.



La función **A < B** ha sido construida a partir de las dos anteriores (**A=B** y **A>B**): dicha función debe adoptar el valor **1** cuando las otras dos valgan **0** (es decir cuando no sea ni **A=B** ni **A>B**):

$$y_3 = \overline{y_1.y_2} = \overline{y_1 + y_2} = y_1 \Delta y_2 = \overline{y_1 * y_2}$$

Se necesitan en total 12 puertas "y-negada" (Nand) y 8 inversores.

2.3.3. Sean dos números binarios de tres dígitos; expresar la función booleana que corresponde a $A \leq B$ y generalizarla para números de 5 dígitos

Sean los dos números: **a2a1a0** y **b2b1b0**; **A** será menor o igual que **B**

- si el dígito más significativo de **A** **a2** es **0** y el de **B** **b2** es **1**
- o también, si dichos dígitos son iguales y el siguiente dígito de **A** **a1** es **0** y el de **B** **b1** es **1**
- o también, si son iguales **a2** y **b2** y, también, **a1** y **b1** y, además, el dígito de las unidades de **A** **a0** es **0** y el de **B** **b0** es **1** o bien son iguales **a0** y **b0**.

$$y = \bar{a}_2.b_2 + (\bar{a}_2.\bar{b}_2 + a_2.b_2).[\bar{a}_1.b_1 + (\bar{a}_1.\bar{b}_1 + a_1.b_1).(\bar{a}_0.b_0 + a_0.\bar{b}_0 + a_0.b_0)] = \bar{a}_2.b_2 + (\bar{a}_2.\bar{b}_2 + a_2.b_2).[\bar{a}_1.b_1 + (\bar{a}_1.\bar{b}_1 + a_1.b_1).(\bar{a}_0 + b_0)]$$

La expresión anterior se puede simplificar, aplicando el teorema de absorción:

$$y = \bar{a}_2.b_2 + (\bar{a}_2 + b_2).[\bar{a}_1.b_1 + (\bar{a}_1 + b_1).(\bar{a}_0 + b_0)]$$

Y su generalización para números de cinco dígitos, **a4a3a2a1a0** y **b4b3b2b1b0**, es directa:

$$\bar{a}_4.b_4 + (\bar{a}_4 + b_4).(\bar{a}_3.b_3 + (\bar{a}_3 + b_3).(\bar{a}_2.b_2 + (\bar{a}_2 + b_2).[\bar{a}_1.b_1 + (\bar{a}_1 + b_1).(\bar{a}_0 + b_0)]))$$

De la misma forma esta función puede generalizarse para números de más dígitos.

2.3.4. El Concejo Municipal de una localidad pequeña está formado por un alcalde con dos votos, un secretario con otros dos votos y tres delegados de barrio con un voto cada uno. Los acuerdos se toman por mayoría simple, pero el voto en contra simultáneo de los tres delegados supone un veto al acuerdo. Sintetizar y simplificar la función booleana que expresa el resultado de las votaciones

Dada la complejidad de la función (32 vectores de entrada) y para obtener su expresión simplificada, se expresa la tabla de verdad directamente en forma de mapa de Karnaugh:

d1 d2 d3									
a	s	0 0 0	0 0 1	0 1 1	0 1 0	1 1 0	1 1 1	1 0 1	1 0 0
		0 0	0	0	0	0	0	0	0
	0 1	0	0	1	0	1	1	1	0
	1 1	0	1	1	1	1	1	1	1
	1 0	0	0	1	0	1	1	1	0

En el anterior mapa de Karnaugh todos los cuadrados con valor **1** pueden agruparse de 4 en 4 y así resultan los siguientes términos producto, comenzando de arriba hacia abajo y de izquierda a derecha:

$$y = s.d_2.d_3 + a.s.d_1 + a.s.d_2 + a.d_2.d_3 + s.d_1.d_2 + a.d_1.d_2 + s.d_1.d_3 + a.d_1.d_3 + a.s.d_3$$

y agrupando términos análogos:

$$\begin{aligned} y &= s.(d_2.d_3 + d_1.d_2 + d_1.d_3) + a.s.(d_1 + d_2 + d_3) + a.(d_2.d_3 + d_1.d_2 + d_1.d_3) = \\ &= a.s.(d_1 + d_2 + d_3) + (s + a).(d_2.d_3 + d_1.d_2 + d_1.d_3) \end{aligned}$$

Solución a la que podría haberse llegado por simple razonamiento directo:

Se aprobará un acuerdo cuando voten a favor el alcalde y el secretario y uno cualquiera de los delegados, o también cuando lo hagan el alcalde o el secretario y dos delegados.

Esta frase constituye la lectura en lógica proposicional de la función anterior.

2.3.5. En una instalación se controla la presión, la temperatura y la intensidad eléctrica consumida, de forma que debe activarse una alarma cuando alguno de estos parámetros sobrepase un valor límite detectado por un transductor con salida digital ("1" por encima de dicho valor y "0" por debajo). Se controla también la tensión que alimenta la instalación, de forma que la alarma también se active cuando ésta sea inferior a un valor mínimo. Construir la función booleana que debe accionar la alarma

<u>V I</u>	<u>T P</u>			
	0 0	0 1	1 1	1 0
0 0	1	1	1	1
0 1	1	1	1	1
1 1	1	1	1	1
1 0	0	1	1	1

El mapa de Karnaugh de esta función está completamente cubierto por el valor 1 salvo en una posición; por ello, resulta más apropiada la síntesis de la función por términos máximos, pues solamente es preciso recoger uno de ellos.

$$\text{"alarma"} = \overline{V} + I + T + P$$

es decir, la alarma suena cuando la tensión está por debajo del valor umbral o cuando la intensidad, la temperatura o la presión están por encima del valor límite.

2.3.6. Sea la función booleana siguiente, expresada en forma de suma de productos; obtener la expresión algebraica de dicha función en forma de producto de sumas

$$y = \overline{d}.\overline{c}.\overline{b} + \overline{d}.\overline{b}.a + d.\overline{c}.b + d.b.\overline{a}$$

Se trata, en primer lugar, de obtener el mapa de Karnaugh de dicha función; para ello tomamos cada término producto, identificamos las casillas que le corresponden y anotamos en ellas el valor **1**:

- al término $\overline{d}.\overline{c}.\overline{b}$ le corresponden las dos casillas en que $d = 0$, $c = 0$ y $b = 0$, es decir, las dos primeras casillas horizontales del mapa;
- al término $\overline{d}.\overline{b}.a$ le corresponden las dos casillas en que $d = 0$, $b = 0$ y $a = 0$, es decir, las dos primeras casillas verticales;
- al término $d.\overline{c}.b$ le corresponden las dos casillas en que $d = 1$, $c = 0$ y $b = 1$, es decir, las dos últimas casillas horizontales del mapa;
- al término $d.b.\overline{a}$ le corresponden las dos casillas en que $d = 1$, $b = 1$ y $a = 0$, es decir, las dos últimas casillas verticales del mapa.

Se completa el mapa de Karnaugh con «ceros» en las casillas vacías, resultando el de la figura siguiente:

<u>d c</u>	<u>b a</u>			
	0 0	0 1	1 1	1 0
0 0	1	1	0	0
0 1	1	0	0	0
1 1	0	0	0	1
1 0	0	0	1	1

Los «ceros» pueden ser agrupados en 3 cuadrados, que dan lugar a una expresión algebraica en forma de producto de sumas más reducida que la expresión inicial en suma de productos:

$$y = (d + \overline{b}).(\overline{d} + b).(\overline{c} + a)$$

2.3.7. La combinación ganadora de la «primitiva» del jueves día 23 de enero de 2003 puede expresarse mediante la función booleana siguiente; ¿cuáles son los números que forman dicha combinación ganadora escritos en base 10 (decimal)?

$$y = (f + \bar{d} + c).(\bar{f} + \bar{d} + b).(\bar{f} + \bar{d} + \bar{c}).(\bar{c} + \bar{b} + \bar{a}). \\ .(e + d).(\bar{e} + b).(d + a).(b + a).$$

Recuérdese que la lotería primitiva utiliza los números del 1 al 49 y deben seleccionarse 6 de dichos números para formar una combinación; a estos efectos, el número 0 y los números superiores a 49 no interesan (valor X).

Se trata de obtener la tabla de dicha función para seleccionar sobre ella los vectores de entrada que hacen la función igual a 1; la forma más rápida de obtener dicha tabla funcional consiste en rellenar el correspondiente mapa de Karnaugh, a partir de los términos suma de la función (las casillas que corresponden a estos vectores suma deben tener valor 0, la casilla 0 y las superiores a 49 deben tener valor X y el resto valor 1).

		C B A							
F E D		0 0 0	0 0 1	0 1 1	0 1 0	1 1 0	1 1 1	1 0 1	1 0 0
	0 0 0	X	0	0	0	0	0	0	0
	0 0 1	0	0	0	0	1	0	1	0
	0 1 1	0	0	0	0	1	0	0	0
	0 1 0	0	0	1	0	0	0	0	0
	1 1 0	0	0	X	X	X	X	X	X
	1 1 1	X	X	X	X	X	X	X	X
	1 0 1	0	0	1	1	0	0	0	0
	1 0 0	0	0	0	0	0	0	0	0

Los vectores de entrada correspondientes a casillas con valor 1 son los siguientes:

001101 001110 010011 011110 101010 101011

números binarios que, expresados en base 10, corresponden a los siguientes números decimales:

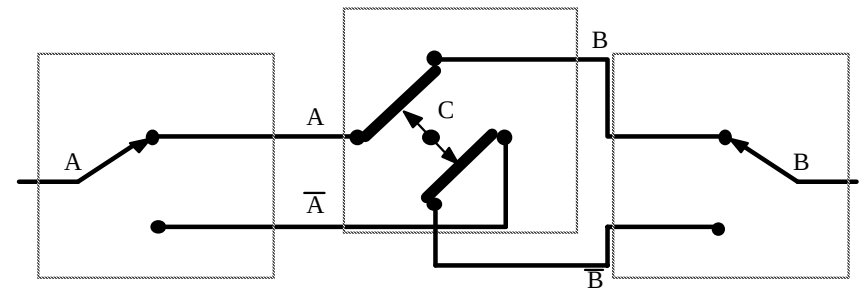
13 14 19 30 42 43

2.3.8. Se desea controlar una lámpara mediante tres interruptores conmutados, de forma que actuando sobre cualquiera de ellos se pueda cambiar el estado de la lámpara. Determinar la función booleana necesaria

Suponemos que con todos los interruptores en la posición 0 la lámpara está apagada, si pasa uno cualquiera de ellos a la posición 1 la lámpara se enciende, si son dos los que pasan a posición 1 la lámpara se apaga y, finalmente, con los tres interruptores en posición 1 la lámpara se encontrará encendida.

C	B	A	luz	
0	0	0	0	
0	0	1	1	$y = \bar{c}.b.a + \bar{c}.b.\bar{a} + c.\bar{b}.\bar{a} + c.b.a$
0	1	0	1	
1	0	0	1	
0	1	1	0	
1	0	1	0	$= \bar{c}.(\bar{b}.a + b.\bar{a}) + c.(\bar{b}.\bar{a} + b.a)$
1	1	0	0	
1	1	1	1	$= c \oplus (b \oplus a)$

La función anterior puede realizarse con interruptores mecánicos en la forma que se indica en la siguiente figura:



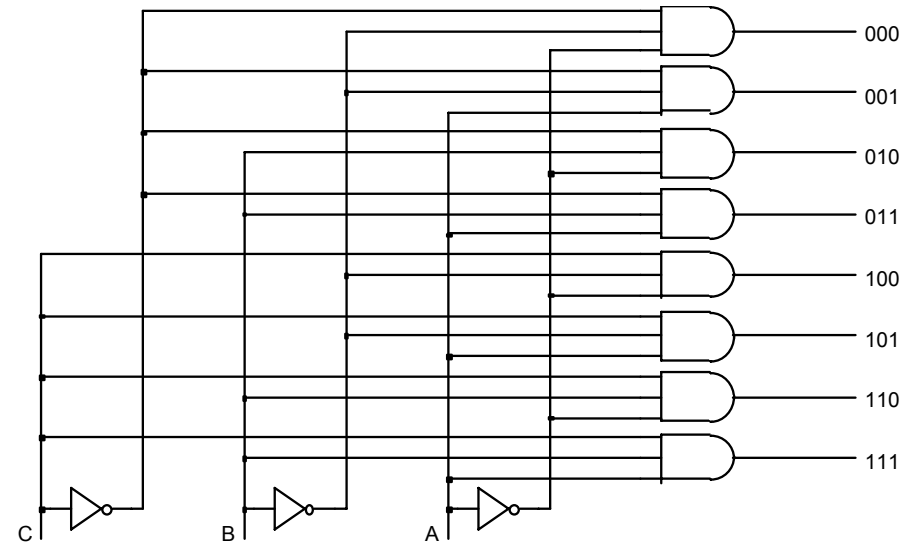
en la que el conmutador C se encuentra en su posición afirmada y negarlo significa efectuar un giro de 90° sobre su centro. Obsérvese que los interruptores de los extremos A y B son del tipo de interruptores conmutados considerados en el tema anterior (I.3.3 Álgebra de Conmutadores), mientras que el interruptor intermedio C realiza una conmutación del tipo «conexión paralela - conexión cruzada»:

- conecta A - B y $\bar{A}$ - $\bar{B}$ en una posición y A - $\bar{B}$ y $\bar{A}$ - B en la otra.

Este esquema de interruptores conmutados es ampliable a n interruptores, siendo los de los extremos de tipo simple A, B y los n-2 intermedios del tipo complejo C.

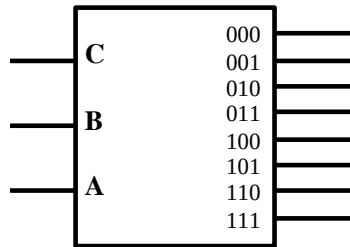
2.4. Decodificadores y multiplexores; otras formas de configurar funciones

Sean tres variables de entrada; podemos construir todos sus términos mínimos: bastarán para ello 8 puertas "y" de 3 entradas y 3 inversores.



Este «bloque digital», cuyas salidas corresponden a los diversos términos mínimos de sus entradas, recibe el nombre de *decodificador*; tal denominación se debe a que «decodifica» un número binario de m dígitos sobre 2^m líneas, de forma que para cada número o vector de entrada activa una salida diferente.

Un decodificador es un bloque digital con m líneas de entrada y 2^m líneas de salida que corresponden a los 2^m posibles vectores de entrada (números binarios de m bits): la línea de salida correspondiente al número binario establecido en las entradas se encontrará a 1 y el resto de líneas de salida estará a 0.



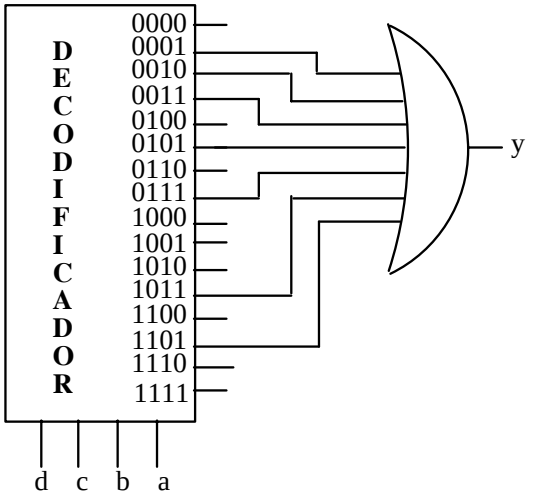
La figura anterior representa un decodificador de 3 líneas de entrada: en cada momento el número binario presente en ellas determina cuál de las 8 líneas de salida se encuentra activada (valor booleano 1).

En un decodificador están presentes todos los términos mínimos de sus entradas; para construir una función booleana de tales entradas, según su forma canónica, bastará reunir sobre una puerta "o" los términos mínimos que corresponden a valor 1 en la tabla de la función, es decir, llevar a una puerta "o" las salidas del decodificador que corresponden a vectores de entrada que «activan» la función (dan resultado 1).

Por ejemplo, para construir de esta forma la función "ser número primo" de 4 dígitos utilizaremos un decodificador de 4 líneas de control seguido de una puerta "o" que recibe aquellas salidas con valor 1 en la «tabla de verdad» de la función: las correspondientes a los números primos 1, 2, 3, 5, 7, 11 y 13.

Tabla funcional

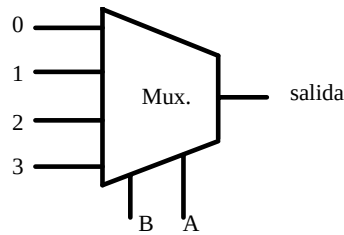
d	c	b	a	y
0	0	0	0	0
0	0	0	1	1 <-
0	0	1	0	1 <-
0	0	1	1	1 <-
0	1	0	0	0
0	1	0	1	1 <-
0	1	1	0	0
0	1	1	1	1 <-
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1 <-
1	1	0	0	0
1	1	0	1	1 <-
1	1	1	0	0
1	1	1	1	0



Así como el decodificador selecciona una de entre sus 2^m líneas de salida, un *multiplexor* es un bloque digital que selecciona una de entre 2^m líneas de entrada; para ello dispone de m líneas de control y en cada momento el número binario establecido en ellas determina la línea de entrada que «queda conectada» a la salida.

Para seleccionar cada línea de entrada por su número binario bastará realizar la operación "y" entre la línea de entrada y el término mínimo que corresponde a dicho número binario; una posterior operación "o" en la salida recogerá el resultado de dicha selección (recibirá las salidas de las citadas puertas "y").

La figura siguiente representa un multiplexor de 4 líneas de entrada y 2 de control; en cada momento el número binario presente en las entradas de control determina cuál de las 4 líneas de entrada se encuentra «conectada» con la salida.

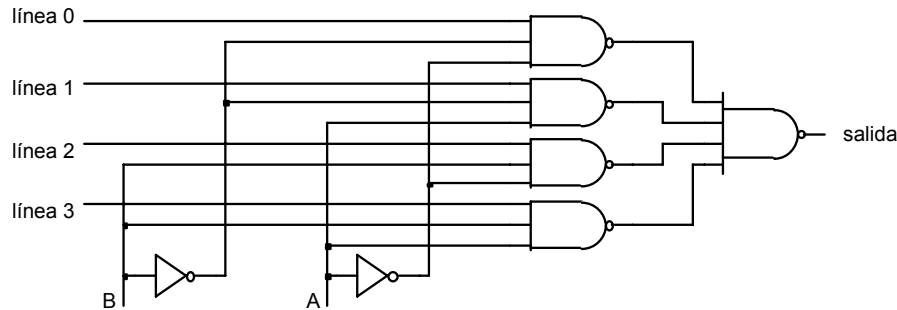


La función booleana que corresponde a este multiplexor es la siguiente:

$$y = L_0 \cdot \bar{B} \cdot \bar{A} + L_1 \cdot \bar{B} \cdot A + L_2 \cdot B \cdot \bar{A} + L_3 \cdot B \cdot A$$

que puede construirse con puertas "y-negada" (Nand) en la siguiente forma:

$$y = \text{Nand}[\text{Nand}(L_0, \bar{B}, \bar{A}), \text{Nand}(L_1, \bar{B}, A), \text{Nand}(L_2, B, \bar{A}), \text{Nand}(L_3, B, A)].$$



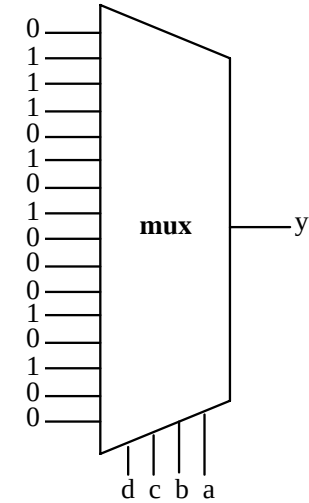
Para construir una función booleana utilizando un multiplexor bastará con fijar en sus líneas de entrada los valores de la «tabla de verdad» de la misma; de esta forma, las variables de control seleccionan sobre la propia tabla de la función el valor booleano que le corresponde al vector de entrada.

Esta forma de configurar funciones booleanas mediante multiplexores, cuyas líneas de entrada reciben los valores de la «tabla de verdad» de la función, se denomina **look-up-table (LUT)**: construcción tabular (*mirar sobre su tabla*).

En el ejemplo de la función "ser número primo" de 4 dígitos, para construirla con un multiplexor de 4 líneas de control, conectaremos a 1 las líneas de entrada del multiplexor que presentan tal valor en la «tabla de verdad» de la función (las correspondientes a los números primos 1, 2, 3, 5, 7, 11 y 13); el resto de las líneas de entrada se conectarán a 0.

Tabla funcional

d	c	b	a	y
0	0	0	0	0
0	0	0	1	1 <-
0	0	1	0	1 <-
0	0	1	1	1 <-
0	1	0	0	0
0	1	0	1	1 <-
0	1	1	0	0
0	1	1	1	1 <-
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1 <-
1	1	0	0	0
1	1	0	1	1 <-
1	1	1	0	0
1	1	1	1	0



Decodificadores y multiplexores son selectores de línea:

- un decodificador activa una de entre **n** líneas de salida
- y un multiplexor selecciona una de entre **n** líneas de entrada y la pone en comunicación con su línea de salida.

Cada uno de ellos proporciona una forma de configurar una función de sus variables de control:

- la forma canónica a partir de un decodificador (suma de términos mínimos $\sum m$)
- la forma tabular sobre un multiplexor (*look-up-table LUT*).

Ambas formas de construir una función booleana no precisan de la expresión algebraica de la misma; se obtienen directamente de su tabla funcional.

3 BLOQUES ARITMÉTICOS Y CODIFICACIÓN NUMÉRICA

- 3.1. Operaciones aritméticas: suma, resta, comparación y producto
- 3.2. Unidad lógica y aritmética: ALU
- 3.3. Codificación de números en binario
- 3.4. Codificación de números en BCD

Las operaciones aritméticas con números binarios pueden ser expresadas en forma de funciones booleanas. El resultado de la suma de dos números de n dígitos puede llegar a tener $n+1$ dígitos, de forma que el correspondiente sumador incluirá $n+1$ funciones booleanas; lo mismo sucederá con la resta, a cuyo resultado (de n bits) habrá que añadir un bit suplementario ($n+1$) para informar si la resta es viable o si el sustraendo es mayor que el minuendo. En cambio, la comparación de números de n bits requiere solamente tres funciones booleanas: igual, mayor, menor.

Suma, resta y comparación pueden ser construidas modularmente, mediante celdas básicas que realicen la operación para un solo dígito. Las celdas básicas de la suma y la resta presentarán dos salidas relativas al resultado y al posible acarreo (me llevo) y dispondrán de las dos entradas correspondientes a los operandos y una entrada adicional para el bit de acarreo o «arrastré», la cual permite la conexión de celdas en cadena para formar bloques de mayor número de bits.

Asimismo, las entradas de «arrastré» permiten conectar sucesivos bloques sumadores, restadores o comparadores para operar con mayor número de dígitos. En el caso de los comparadores son necesarias dos entradas de arrastre (igual, mayor) para recibir la información de la celda o del bloque anterior.

La multiplicación, al no ser operación lineal, no puede construirse mediante celdas básicas en cadena; pero puede configurarse mediante una combinación matricial de operaciones “y” (producto) y celdas sumadoras.

También podemos considerar como bloques operacionales los conjuntos de puertas lógicas que realizan la correspondiente operación booleana entre dos palabras, bit a bit. Las siglas **ALU** (unidad lógica y aritmética) designan a un bloque «multioperación», capaz de realizar diferentes operaciones aritméticas y lógicas sobre dos números de n dígitos; unas entradas de selección determinan la operación a realizar en cada momento.

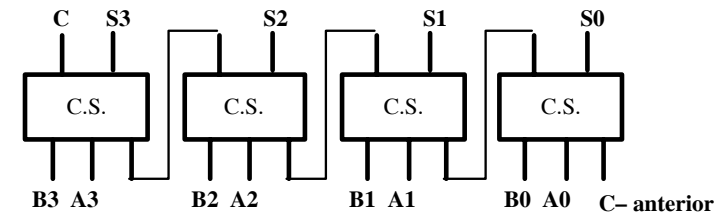
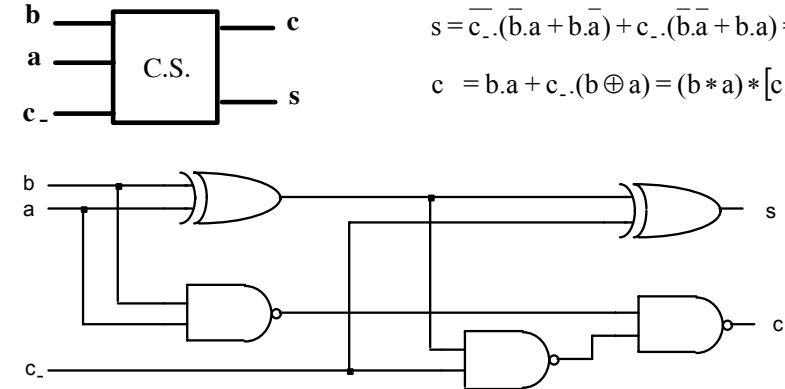
A la par de las propias operaciones aritméticas, resulta oportuno considerar cómo pueden representarse en palabras binarias los números negativos o los que tienen parte decimal (números no enteros), de forma que puedan efectuar sus operaciones con los mismos bloques aritméticos. Además de la forma binaria directa, correspondiente a la numeración en base 2, en muchas ocasiones resulta conveniente conservar la estructura decimal habitual (base 10); para ello, en lugar de convertir a base 2 el número completo, se traslada a binario cada una de sus cifras individuales: esta forma de representar los números en palabras binarias recibe el nombre de **BCD** (decimal codificado en binario).

3.1. Operaciones aritméticas: suma, resta, comparación y producto

A partir de la celda sumadora básica de dos dígitos **a** y **b** más un tercero de acarreo (me llevo: carry) **c**, puede configurarse un sumador de dos números binarios de n dígitos mediante la conexión en cadena de n celdas. [El diseño de una celda sumadora se encuentra detallado en el capítulo anterior (epígrafe II.3.1) como ejercicio de síntesis de funciones booleanas.]

$$s = \bar{c}_- . (\bar{b}.a + b.\bar{a}) + c_- . (\bar{b}.\bar{a} + b.a) = c_- \oplus (b \oplus a)$$

$$c = b.a + c_- . (b \oplus a) = (b * a) * [c_- * (b \oplus a)]$$



Sumador de dos números de 4 dígitos.

A su vez, estos bloques sumadores de n cifras pueden conectarse en cadena, con la salida **C** de cada uno de ellos unida a la entrada **C-** del siguiente más significativo, para formar sumadores de mayor número de cifras binarias.

En la celda sumadora, la función arrastre **C** obtenida mediante un mapa de Karnaugh queda un poco más simplificada que la anterior:

$$c = b.a + c_- . (b + a)$$

esta expresión simplificada nos interesa para compararla con la que resulta en el caso de una celda restadora.

En los sumadores de muchos bits, los tiempos de propagación de las celdas sucesivas se suman y resulta un tiempo de propagación global alto, lo cual puede limitar mucho la velocidad de estos sumadores. El apéndice A2 muestra otra forma de diseño de sumadores aprovechando la recursividad en el cálculo del arrastre: arrastre anticipado (*look ahead carry*) en lugar del arrastre propagado celda a celda (*ripple carry*).

Los *restadores* se configuran con la misma técnica modular: a partir de la tabla funcional correspondiente, se obtienen las funciones booleanas que configuran la célula básica restadora (para **A-B**); las variables de entrada de la tabla funcional son **a** minuendo, **b** sustraendo y **c-** arrastre y para la obtención de la misma ha de tenerse en cuenta que el arrastre de la resta se suma al sustraendo:

a	b	c-	suma: s	acarreo: c+
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$r = \bar{c}_- (\bar{b}.a + b.\bar{a}) + c_- (\bar{b}.\bar{a} + b.a) = c_- \oplus (b \oplus a)$$

$$c = b.\bar{a} + c_- (b + \bar{a})$$

La primera función coincide con la que calcula el resultado de la suma **s** y la segunda difiere del arrastre de la suma solamente en la negación de la variable **b**.

Para restar dos números positivos **A-B** es preciso que **A ≥ B**; en otro caso, el arrastre más significativo valdrá **1**, indicando que **B** es mayor que **A** y, por tanto, el resultado no corresponde a un número positivo. [En el apartado 4 de este mismo capítulo se estudiará la representación y forma de operar con números negativos.]

Comparando las funciones básicas de sumadores y restadores se observa que difieren solamente en las relativas al arrastre y, en éstas, solamente en la negación de una variable:

$$\text{arrastre de la suma} \quad c = b.a + c_- (b + a)$$

$$\text{arrastre de la resta} \quad c = b.\bar{a} + c_- (b + \bar{a})$$

Resulta sencillo construir bloques sumadores/restadores, capaces de efectuar las dos operaciones, con una entrada de selección **d** que diferencie entre ambas; bastará añadir una operación "**o-exclusiva**" $a \oplus d$, para determinar cuándo la variable **a** debe actuar afirmada (**d = 0**: suma) o negada (**d = 1**: resta) en las funciones que calculan los arrastres:

$$\text{resultado} \quad r = c_- \oplus (b \oplus a)$$

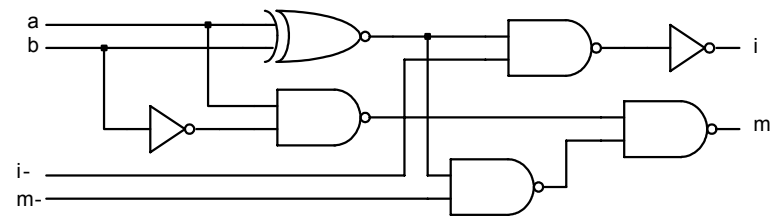
$$\text{arrastre} \quad c = b.a' + c_- (b + a') \quad \text{siendo } a' = a \oplus d$$

Un *comparador* entre dos números binarios de **n** dígitos puede construirse conectando en cadena **n** celdas: la celda básica comparadora tiene como entradas, además de los dígitos correspondientes de los números a comparar **a_i** **b_i**, las que le informan sobre si los conjuntos de dígitos anteriores (de menor valor significativo) son iguales (**i-**) y sobre si es mayor el que corresponde al número **A** (**m-**), y tiene dos salidas que indican igualdad **i** y "**A** mayor que **B**" **m**, respectivamente. La tercera función de comparación "**A** menor que **B**" resulta redundante con las otras dos: $A \text{ menor que } B = \overline{i + m} = \bar{i} . \bar{m}$.

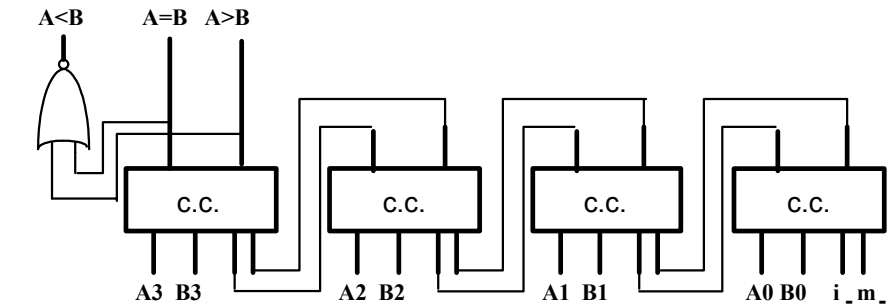
Las funciones booleanas de la célula comparadora responden al siguiente razonamiento [véase el ejemplo de comparación de números de dos cifras detallado en II.3.2]:

- para comparar dos números hay que comenzar por la cifra más significativa y, caso de que ambos dígitos sean iguales, irse desplazando hacia la derecha efectuando la comparación cifra a cifra;
- la función igualdad de una celda comparadora se activará cuando sus dos dígitos **a** **b** son iguales (es decir, cuando ambos valen **0** o ambos valen **1**: $\bar{a}.\bar{b} + a.b$) y, además, la información que reciba por su entrada **i-** respecto a los siguientes dígitos sea de que son iguales (**i- = 1**);
- la función "**A** mayor que **B**" se activará cuando **a = 1** y **b = 0** o cuando ambos dígitos son iguales ($\bar{a}.\bar{b} + a.b$) y la información que reciba por su entrada **m-** respecto a los siguientes dígitos sea de que "**A** mayor que **B**" (**m- = 1**).

$$\begin{aligned} i &= (\bar{a}.\bar{b} + a.b) . i_- = (b \otimes a) . i_- \\ m &= \bar{b}.a + (\bar{b}.\bar{a} + b.a) . m_- \\ &= (\bar{b} * a) * [(b \otimes a) * m_-] \end{aligned}$$



Agrupando **n** células comparadoras «en cadena» (las salidas **i** **m** unidas a las entradas **i-** **m-** de la celda siguiente) se obtendrá un comparador de dos números de **n** dígitos [véase figura en la página siguiente].



Comparador de dos números de 4 dígitos.

[La salida global A<B se activa cuando las otras dos A=B y A>B se encuentran a 0.]

Asimismo, los bloques comparadores de números de *n* cifras pueden conectarse en cascada para configurar comparadores de mayor número de cifras binarias: las salidas *i* (=) y *m* (>) de cada uno de ellos unidas a las correspondientes entradas *i*- y *m*- del siguiente bloque más significativo; las entradas *i*- y *m*- del primero de los bloques (el menos significativo) deberán ir conectadas a 1 y 0, respectivamente (situación inicial: ser iguales).

Un bloque aritmético de mayor complejidad es el *multiplicador* que realiza el producto de dos números binarios de *n* dígitos: ni su diseño ni su utilización son modulares (por células en cadena) sino que es preciso configurar el desarrollo completo de las correspondientes funciones booleanas.

Un bloque multiplicador de 2 números de *n* dígitos dispondrá de **2n** entradas (*n* para cada número) y **2n** salidas, pues tal puede ser el número de dígitos del resultado; el producto de dos números binarios puede organizarse como suma de productos de sus dígitos en la misma forma que el producto decimal:

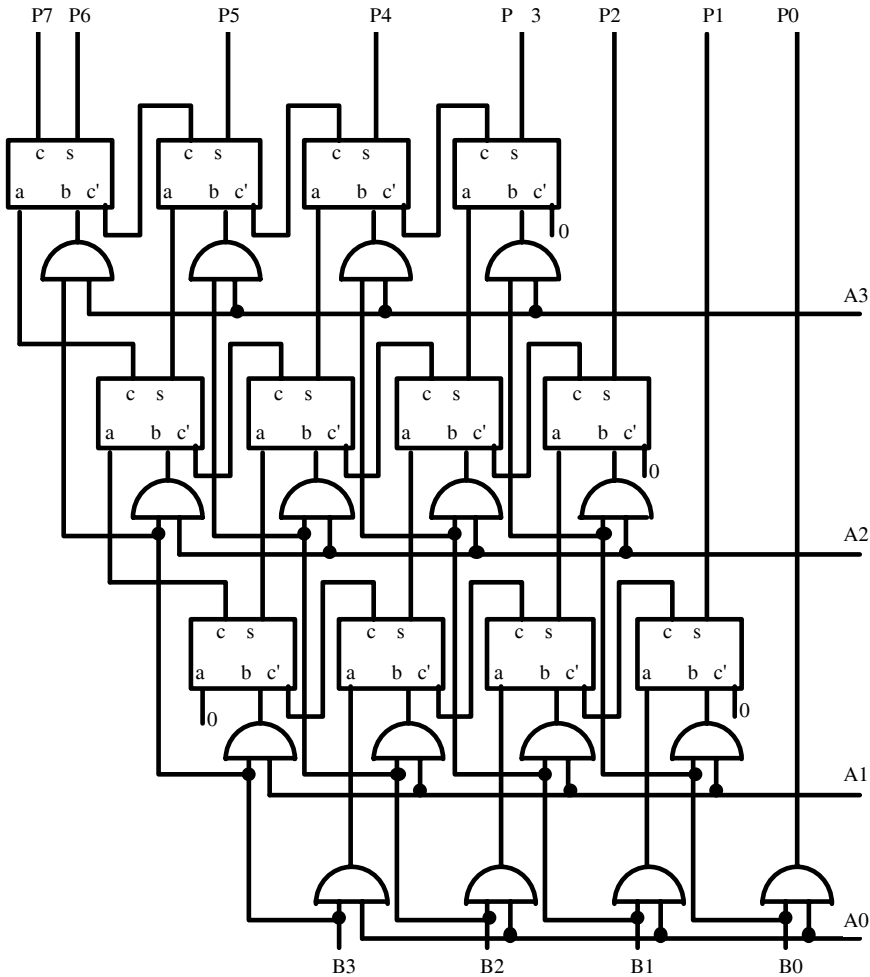
	b _{n-1}	a _{n-2}		b ₄	b ₃	b ₂	b ₁	b ₀
x	a _{n-1}	a _{n-2}		a ₄	a ₃	a ₂	a ₁	a ₀
	b _{n-1} .a ₀	b _{n-2} .a ₀		b ₄ .a ₀	b ₃ .a ₀	b ₂ .a ₀	b ₁ .a ₀	b ₀ .a ₀
	b _{n-1} .a ₁	b _{n-2} .a ₁	b _{n-3} .a ₁		b ₃ .a ₁	b ₂ .a ₁	b ₁ .a ₁	b ₀ .a ₁
	b _{n-1} .a ₂	b _{n-2} .a ₂	b _{n-3} .a ₂	b _{n-4} .a ₂		b ₂ .a ₂	b ₁ .a ₂	b ₀ .a ₂
	b _{n-1} .a ₃	b _{n-2} .a ₃	b _{n-3} .a ₃	b _{n-4} .a ₃	b _{n-5} .a ₃		b ₁ .a ₃	b ₀ .a ₃
								
								
								

Cada una de las columnas contribuye a configurar el correspondiente dígito como suma de productos de los dígitos de los factores:

$$a_i.b_0 + a_{i-1}.b_1 + a_{i-2}.b_2 + ... + a_2.b_{i-2} + a_1.b_{i-1} + a_0.b_i ;$$

pero, además, es preciso tener en cuenta los arrastres (*me llevo*) procedentes de las sumas que configuran los dígitos anteriores.

En conjunto, para multiplicar números de 4 dígitos resulta el esquema de la figura siguiente.



Multiplicador de dos números de 4 dígitos.

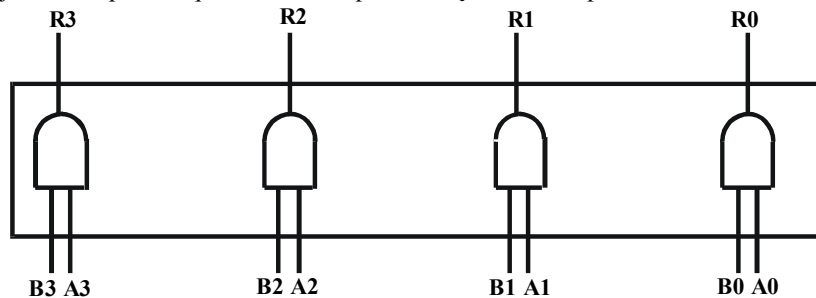
La configuración anterior, con una disposición de tipo matricial de puertas "y" y celdas sumadoras, puede ampliarse directamente a mayor número de dígitos ya que su estructura básica es repetitiva: basta seguir añadiendo filas y columnas de puertas "y" y celdas sumadoras, siguiendo la misma configuración matricial.

Pero no puede separarse en módulos iguales interconectables, de forma que el multiplicador para números de $2n$ dígitos no puede realizarse mediante la simple conexión de varios multiplicadores de n dígitos. [Un multiplicador de números de 8 bits requiere 4 multiplicadores de 4 bits más 5 sumadores de números de 4 bits.]

3.2. Unidad lógica y aritmética ALU

También podemos considerar como bloques operacionales a los que realizan operaciones booleanas, constituidos por conjuntos de n puertas iguales de dos entradas que efectúan la operación lógica correspondiente (sea esta operación "o", "y", "o-exclusiva", "y-negada", "o-negada",...) sobre dos palabras de n dígitos o bits, operación lógica que se realiza individualmente, bit a bit.

En la siguiente figura, se representa uno de estos bloques, configurado por un conjunto de 4 puertas que realizan la operación "y" entre dos palabras de 4 bits.



Bloque operacional lógico ("y") para palabras de 4 bits

Disponemos, pues, de bloques con capacidad de efectuar las operaciones básicas de cálculo binario (suma, resta, comparación,...) y de otros capaces de hacer operaciones booleanas ("+", "-", "⊕", "*", "Δ",...) sobre dos números o palabras binarias de n dígitos o bits.

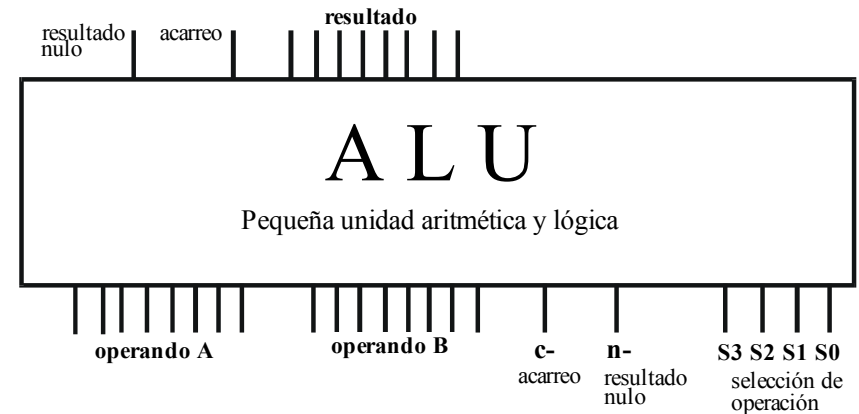
La longitud de palabra de los bloques operacionales integrados n suele ser de 4, 8 o 16 bits; pero este número no es limitativo en modo alguno pues, salvo el caso particular de producto, la longitud de los operandos se puede ampliar indefinidamente sin más que conectar en cadena bloques iguales:

- en el caso de suma o resta, la interconexión entre ellos se refiere al bit de acarreo
- en la comparación, cada bloque ha de comunicar al siguiente sus salidas ("=" y ">")
- en las operaciones lógicas, no es precisa interconexión alguna (son bit a bit).

Un paso más en la complejidad y posibilidades funcionales de los bloques operacionales es la configuración de un «operador genérico», capaz de realizar no una sino toda una amplia serie de operaciones aritméticas y lógicas sobre dos palabras de n bits: un bloque multifunción diseñado para efectuar k operaciones distintas, de forma que en cada momento se selecciona la operación que interesa mediante unas entradas de control.

Un bloque digital de este tipo recibe el nombre genérico de *unidad aritmética y lógica ALU*. Recibirá como entradas dos operandos de n dígitos y los terminales de salida de las diversas operaciones serán únicos, apareciendo sobre ellos el resultado de la operación seleccionada.

La siguiente figura representa una ALU de tipo sencillo que opera sobre palabras de 8 bits y tiene capacidad para 16 operaciones (4 entradas de control).



Unidad lógica y aritmética ALU para palabras de 8 bits

Dos salidas adicionales y sus correspondientes entradas permiten la ampliación de la ALU para palabras de más de 8 dígitos: la entrada y salida de acarreo c para la suma y la resta y la entrada y salida de resultado nulo n , que se activa cuando todos los dígitos del resultado son 0.

La operación de comparación (entre ambos números A y B) se realiza mediante una operación de resta $A-B$, quedando el resultado reflejado sobre las salidas n (resultado nulo) y c (acarreo), en la forma siguiente:

- $n = 1 \Rightarrow A = B$
- $c = 1 \Rightarrow A < B$
- $n = 0$ y $c = 0 \Rightarrow A > B$

La siguiente tabla detalla posibles operaciones a realizar por una **ALU**: la entrada de selección **S₃** distingue entre operaciones aritméticas **S₃ = 0** y operaciones lógicas **S₃ = 1**, mientras que los ocho valores posibles de las otras tres entradas de control **S₂ S₁ S₀** permiten seleccionar ocho operaciones de cada uno de ambos tipos:

S₂	S₁	S₀	S₃ = 0	S₃ = 1
			Operaciones Aritméticas	Operaciones Lógicas
0	0	0	Sumar A y B (B + A)	Invertir A ($\bar{A}$)
0	0	1	Incrementar A (A + 1)	Invertir B ($\bar{B}$)
0	1	0	Incrementar B (B + 1)	"o" (A + B)
0	1	1	Restar A y B (A - B)	"y" (A · B)
1	0	0	Decrementar A (A - 1)	"o-negada" (A Δ B)
1	0	1	Decrementar B (B - 1)	"y-negada" (A * B)
1	1	0	Negativo de A (0 - A)	"o-exclusiva" (A ⊕ B)
1	1	1	Negativo de B (0 - B)	"y-inclusiva" (A ⊗ B)

3.3. Codificación de números en binario

El sistema binario de numeración, con base 2, constituye la forma natural de codificar números en palabras binarias, aptas para ser procesadas por los sistemas digitales; dicho sistema de numeración constituye la base de la aritmética digital.

Este apartado se dedica a la representación de números negativos y de números con parte decimal en forma de palabras binarias (siendo el **0** y el **1** los únicos símbolos permitidos). Se trata de utilizar un convenio de representación que sea compatible con las operaciones de suma y resta de los números enteros, de forma que puedan utilizarse los bloques operacionales tal como han sido construidos en este capítulo.

3.3.1. Codificación binaria de números negativos

Es posible distinguir los números positivos y negativos mediante un bit inicial de signo: **0** para los números positivos y **1** para los negativos. Una primera forma de codificar los números enteros consistirá en añadir un **0** delante del número binario que expresa el valor absoluto para indicar número positivo y añadir un **1** delante del mismo valor absoluto para representarlo en negativo.

Esta codificación no es compatible con las operaciones de suma y resta tal como han sido construidas para los números naturales. Consideremos números binarios inferiores al número decimal 100, cuyo valor absoluto se expresa en 7 bits y utilicemos un octavo bit inicial para el signo:

$$\begin{array}{r}
 +22 \quad \mathbf{00010110} \\
 -22 \quad \mathbf{10010110} \\
 \hline
 \text{suma} \quad \mathbf{10101100} = -44 \text{ erróneo}
 \end{array}$$

Si representamos en una lista los diversos números enteros de 8 bits resultantes de esta codificación (signo - valor absoluto), ordenados según su valor numérico:

01111111	+127
01111110	+126

00000010	+2
00000001	+1
00000000	0
10000001	-1
10000010	-2

11111110	-127
11111111	-128

podemos comprobar que en los números negativos no se conserva el «contaje» binario (a, a+1): el -1 no es el siguiente binario del -2 (**10000010** + 1 ≠ **10000011**); al no conservarse el conteo, tampoco se conservan las operaciones suma y resta, en su forma normal (pues suma y resta no son sino el resultado de conteos/desconteos sucesivos). De hecho los números negativos resultan ordenados en sentido inverso, respecto al conteo.

Interesa una codificación que resulte compatible con el conteo y, por ello, con las operaciones aritméticas de suma y resta tal como son realizadas por los bloques operacionales. Para ello, representemos la lista de los números enteros de 8 bits, construyendo los negativos mediante la operación de restar **1** al anterior (desconteo):

01111111	+127
01111110	+126

00000010	+2
00000001	+1
00000000	0
11111111	-1
11111110	-2

10000001	-127
10000000	-128

Esta representación equivale a obtener el negativo de un número mediante la operación $y = 0 - x$ (olvidando el arrastre que se produce en el bit más significativo):

$$\begin{array}{r}
 0 \quad \mathbf{00000000} \\
 +22 \quad \mathbf{00010110} \\
 \hline
 \text{resta} \quad \mathbf{11101010} \\
 -22 = \quad \mathbf{11101010}
 \end{array}$$

Comprobemos que esta codificación es compatible con las operaciones de suma y resta:

$$\begin{array}{r}
 +22 \quad \mathbf{00010110} \\
 -22 \quad \mathbf{11101010} \\
 \hline
 \text{suma} \quad \mathbf{00000000} = 0
 \end{array}
 \quad
 \begin{array}{r}
 +22 \quad \mathbf{00010110} \\
 -22 \quad \mathbf{11101010} \\
 \hline
 \text{resta} \quad \mathbf{00101100} = +44 \text{ correcto}
 \end{array}$$

(en ambos casos se ha ignorado el arrastre que se produce en el bit más significativo, es decir, los dígitos que van más allá del bit de signo).

Esta codificación se denomina en *complemento a 2ⁿ*, o más abreviadamente, *complemento a 2*; en ella el **0** equivale a **2ⁿ**, siendo **n** el número de bits de la palabra, y los números se cambian de signo restándolos de **2ⁿ**:

$$-A = 2^n - (A) = \mathbf{100000...}(n \text{ «ceros»}) - (A)$$

La anterior sustracción puede ser realizada con mayor facilidad en dos pasos, en la forma que sigue:

$$2^n - (A) = \mathbf{1111...}(n \text{ «unos»}) + 1 - (A) = A' + 1$$

donde $A' = \mathbf{1111...}(n \text{ «unos»}) - (A)$ es precisamente el número que resulta de invertir cada uno de los bits de **A**: $A' = \bar{A}$.

Ejemplo de complemento a 2:

$$\begin{array}{rcl}
 83 & = & \mathbf{01010011} \quad \text{se invierten los bits} \\
 83' & = & \mathbf{10101100} \quad +1 \\
 -83 & = & \mathbf{10101101}
 \end{array}$$

$$2^8 - 83 = 100000000 - 01010011 = 10101101.$$

La conversión inversa, de negativo a positivo, se realiza mediante el mismo proceso:

$$\begin{array}{rcl}
 -83 & = & \mathbf{10101101} \quad \text{se invierten los bits} \\
 -83' & = & \mathbf{01010010} \quad +1 \\
 +83 & = & \mathbf{01010011}
 \end{array}$$

$$2^8 - (-83) = 100000000 - 10101101 = 01010011.$$

Habida cuenta de que la codificación en *complemento a 2* conserva el conteo binario, también resulta compatible con las operaciones de suma y resta aritméticas definidas para los números naturales.

Un número de **n** dígitos *complemento a 2* estará comprendido entre **1000...** = -2^{n-1} (el menor número negativo de **n** bits) y **0111...** = $+2^{n-1} - 1$ (el mayor número de **n** bits).

Existen casos en que el valor absoluto del resultado no cabe en los **n-1** bits disponibles para ello: se produce entonces un *desbordamiento* (*over-flow*) y el resultado no es correcto ya que requiere mayor número de dígitos. En estos casos el valor numérico interfiere con el bit de signo y genera un resultado erróneo.

Por ejemplo, $100_{(10)} + 100_{(10)} = \mathbf{01100100} + \mathbf{01100100} = \mathbf{11001000} = -56_{(10)}$ resultado obviamente erróneo.

El *desbordamiento* (*over-flow*) se produce:

a) cuando se suman dos números positivos y aparece un resultado negativo (lo cual es debido a que el resultado es mayor que **2ⁿ⁻¹-1** y, al no caber en los **n-1** bits del campo numérico, interfiere con el bit de signo y lo modifica erróneamente)

b) cuando se suman dos números negativos y se genera un número positivo (debido a que el resultado es menor que **-2ⁿ⁻¹**, que no cabe en el campo numérico)

c) cuando se restan dos números de distinto signo y en tal resta se produce una de las dos situaciones anteriores a) o b).

En cambio, no puede producirse desbordamiento en caso de sumar números de distinto signo o de restar números del mismo signo, ya que en ambas situaciones el resultado es inferior al mayor de los operandos y tiene perfecta cabida en el campo numérico sin interferir con el signo.

Situaciones de desbordamiento:

$$\begin{array}{rclcl}
 \mathbf{0} \text{-----} & + & \mathbf{0} \text{-----} & = & \mathbf{1} \text{-----} \\
 \mathbf{1} \text{-----} & + & \mathbf{1} \text{-----} & = & \mathbf{0} \text{-----} \\
 \mathbf{0} \text{-----} & - & \mathbf{1} \text{-----} & = & \mathbf{1} \text{-----} \\
 \mathbf{1} \text{-----} & - & \mathbf{0} \text{-----} & = & \mathbf{0} \text{-----}
 \end{array}$$

Así, pues, las operaciones de suma y resta de números enteros en complemento a 2 pueden realizarse con los bloques sumadores y restadores descritos en el primer apartado de este capítulo (ya que dichas operaciones son el resultado de contajes/descontajes sucesivos y esta codificación conserva el conteo binario); en todo caso habrá que comprobar que no se produce desbordamiento (*over-flow*).

La comparación entre números en complemento a 2 no presenta dificultades cuando ambos números son del mismo signo; tanto los positivos como los negativos se encuentran en orden correcto conforme a la comparación aritmética que efectúan los bloques comparadores.

Pero la comparación directa entre números de diferente signo llevaría a declarar el número negativo como mayor (pues comienza por **1** y, en cambio, el positivo lo hace por **0**); este error puede evitarse invirtiendo el bit de signo, es decir, añadiendo al comparador sendos inversores previos que actúen sobre el bit más significativo de cada número.

3.3.2 Codificación binaria de números racionales

Los números racionales presentan una parte entera seguida de una parte decimal; dado que la denominación «parte decimal» da lugar a un doble sentido según que el término «decimal» se refiera a dicha parte o al sistema de numeración decimal, se utilizará en lo sucesivo la denominación «parte no entera».

La forma más directa de codificar los números racionales consiste en reservar un determinado número de dígitos para la parte no entera: se desprecian aquellas cifras decimales que sobrepasen (inferiores en valor relativo) los dígitos disponibles y, en cambio, se completan con ceros cuando el número de cifras de la parte no entera sea menor que el de dígitos fijados.

Esta forma de representar números con parte no entera se denomina codificación en coma fija: el número de bits reservados para la parte no entera del número es fijo, de forma que la longitud de la parte no entera ha de ajustarse a dicho número (despreciando cifras decimales de menor valor significativo o añadiendo ceros, según proceda).

Supongamos una codificación en coma fija de 6 bits: se prescindirá de los dígitos de la parte no entera que ocupan lugares posteriores al sexto y se complementará con ceros caso de que el número de dígitos decimales sea inferior a 6.

Ejemplos:

0,1	= 0,100000	0,101	= 0,101000
0,01011001	= 0,010110	0,00000001	= 0,000000
10	= 10,000000	101,011	= 101,011000

Las operaciones de suma y resta construidas para los números naturales pueden aplicarse directamente a los números racionales en coma fija y el resultado será un número racional expresado en coma fija con el mismo número de dígitos. El establecimiento de una longitud fija para la parte no entera permite tratar a estos números como enteros: al interpretar el resultado se añadirá la coma en la posición que corresponde al número de bits de la parte no entera.

La conversión de la parte no entera de un número desde el sistema decimal al binario se realiza multiplicando sucesivamente por 2 dicha parte no entera:

	719	
1	438	
0	876	0,719 = 0,101110
1	752	
1	504	
1	008	
0	016	
...	...	

La conversión recíproca del sistema binario al decimal se realiza multiplicando cada bit por su valor significativo $2^{-i} = 1/2^i$, siendo i el número de orden que ocupa en la parte no entera:

$$\begin{aligned} \mathbf{0,101110...} &= \mathbf{1} \times 0,5 + \mathbf{0} \times 0,25 + \mathbf{1} \times 0,125 + \mathbf{1} \times 0,0625 + \mathbf{1} \times 0,03125 + ... \\ &= 0,5 + 0,125 + 0,0625 + 0,03125 = 0,71875 \approx 0,719 \end{aligned}$$

La codificación en complemento a 2 es válida para manejar los números negativos en coma fija: en este caso, el cambio de signo de un número se realiza invirtiendo todos sus bits y sumando un bit **1** al número resultante; dicha suma ha de efectuarse sobre el bit menos significativo de la coma fija.

Ejemplo:

2,719	=	00000010,101110	invertir los bits
2,719'	=	11111101,010001	+0,000001
- 2,719	=	11111101,010010	

Con palabras binarias de 32 bits, reservando 16 para la parte entera (de ellas, un bit para el signo) y otras 16 para la parte no entera, pueden representarse números con valor absoluto de hasta $2^{15} = 32.768$ y con una precisión de $2^{-16} = 0,000015$.

Con 64 bits, 40 para la parte entera y 24 para la no entera, se alcanzan números cuya magnitud llega a $2^{39} = 2^9 \cdot 2^{30} \approx 500 \cdot 10^9$ (ya que $2^{10} \approx 10^3$), medio billón, y cuya precisión decimal sea de $2^{-24} = 2^{-4} \cdot 2^{-20} \approx 0,0625 \cdot 10^{-6}$, una diez millonésima.

3.3.3. Codificación en coma flotante

La forma exponencial permite codificar los números racionales, ampliando el rango o la precisión de los mismos.

Todo número **a** puede expresarse mediante su cifra más significativa como parte entera, seguida del resto del número como parte no entera **a'**, multiplicado por la base del sistema de numeración elevada al correspondiente exponente **e**.

En sistema binario **a = 1,a' · 2^e** (mejor, $1,a' \cdot 10^e$, ya que $2 = 10_{(2)}$ donde **a'** es el resultado de prescindir en **a** del bit más significativo (y de la coma de separación decimal) y **2^e** es el valor numérico relativo de dicho bit.

a' recibe el nombre de mantisa, **e** es el exponente y la codificación en coma flotante se configura reservando un bit para el signo, un número fijo de bits para el exponente y otro número, fijo también, de bits para la mantisa:

- el número de bits del exponente determina la magnitud (tanto en grande como en pequeño) de los números que pueden representarse;

- el número de bits de la mantisa obliga a ajustarla a dicho tamaño, despreciando, en su caso, los dígitos menos significativos, o añadiendo, si es preciso, ceros para completar el número establecido de bits; ello equivale a fijar el número de cifras significativas que se utilizan.

a) Coma flotante de simple precisión

La codificación binaria en coma flotante de simple precisión utiliza 32 bits, de los cuales el más significativo expresa el signo del número, los 8 siguientes contienen el exponente y los 23 restantes están reservados para la mantisa.

En principio, el exponente se expresa en código exceso 128, de forma que el exponente **10000000** corresponde al 0 y exponentes que comienzan por **0** son negativos:

00000000 = -128; **01111111** = -1; **10000000** = 0; **11111111** = +127

el rango de los números irá de $2^{127} \sim 10^{38}$, sixtillones, en cuanto a números grandes, hasta $2^{-128} \sim 10^{-38}$, una sixtillonésima, para números pequeños. Los 23 bits de la mantisa equivalen a 7 cifras decimales significativas ($2^{23} \sim 10^7$).

Ahora bien, esta codificación no permite representar el valor **0** (el más cercano posible será el $1,0 \cdot 2^{-128}$); para incluir el **0** (y para mejorar la precisión de los números de valor absoluto muy pequeño) la forma de codificar en coma flotante de simple precisión se ha establecido según las siguientes normas (estándar IEEE 754):

- El primer bit indica el signo (**0** = +; **1** = -), los 8 bits siguientes expresan el exponente y los 23 bits restantes contienen la mantisa, normalizada en la forma **1,mantisa**: **(-1)signo . 1,mantisa . 2^{exponente-127}**
- El exponente se codifica en 8 bits en exceso 127, desde **00000001** = -126 hasta **11111110** = +127, pasando por **01111110** = -1; **01111111** = 0; **10000000** = +1; **10000001** = +2 ;...
- Cuando el exponente es **00000000** se utiliza la normalización **0,mantisa**; si la mantisa es nula representa el cero y si no es nula, representa el número: **0,mantisa . 2⁻¹²⁶**
- El exponente **11111111** se reserva para valores infinitos o indeterminados:
 - cuando la mantisa es nula: ∞ infinito, $+\infty$ o $-\infty$ según el signo (primer bit)
 - si la mantisa no es nula: **NaN** no es un número (por ejemplo: 0/0, etc.;...).

b) Coma flotante de doble precisión

La codificación en coma flotante de doble precisión utiliza 64 bits, el más significativo para el signo, 11 bits para el exponente y los últimos 52 bits expresan la mantisa; las normas de codificación son las siguientes (estándar IEEE 754):

- La mantisa normalizada en la forma **1,mantisa**: **(-1)signo . 1,mantisa . 2^{exponente-1023}**
- El exponente se codifica en 11 bits en exceso 1023, desde **00000000001** = -1022 hasta **11111111110** = +1023
- Para exponente **00000000000** se utiliza la normalización **0,mantisa**: mantisa nula para el cero y si no lo es representa el número: **0,mantisa . 2⁻¹⁰²²**.
- El exponente **11111111111** se reserva para valores infinitos o indeterminados: ∞ para mantisa nula y **NaN** cuando la mantisa es distinta de 0.

Los 52 bits de la mantisa equivalen a 15 cifras decimales significativas ($2^{52} \sim 10^{15}$), mientras que el exponente cubre un rango numérico de $10^{\pm 300}$.

3.4. Codificación de números en BCD

En general, los humanos estamos acostumbrados al sistema de numeración decimal (debido al hecho de que disponemos de 10 dedos) y nos resulta más cómodo utilizar los datos numéricos en base 10. Para ello, la entrada de datos en los sistemas digitales requiere la correspondiente conversión *decimal* $\rightarrow$ *binario*, mientras que la salida de resultados requiere la conversión inversa *binario* $\rightarrow$ *decimal*. Tales conversiones no son sencillas y no pueden realizarse modularmente «a trozos» sino que es preciso operar sobre el número completo a convertir.

Por ello, en muchos sistemas digitales, no se codifican los números en binario sino que se respeta su representación en sistema decimal y se codifica por separado cada una de sus cifras: **codificación BCD** (decimal codificado en binario). De esta forma, el número continúa siendo decimal (base 10), con sus cifras codificadas en binario.

Cada cifra decimal necesita 4 bits (habida cuenta de que 9 = **1001**) y se representan siempre los cuatro bits, incluidos los ceros no significativos:

0	0000	1	0001	2	0010	3	0011	4	0100
5	0101	6	0110	7	0111	8	1000	9	1001

Las palabras binarias **1010**, **1011**, **1100**, **1101**, **1110**, **1111** no tienen significado en codificación **BCD**.

Ejemplos de codificación binaria y **BCD**:

173	=	$10101101_{(2)}$	=	0001 0111 0011 _{BCD}
592	=	$1001010000_{(2)}$	=	0101 1001 0010 _{BCD}
846	=	$1101001110_{(2)}$	=	1000 0100 0110 _{BCD}

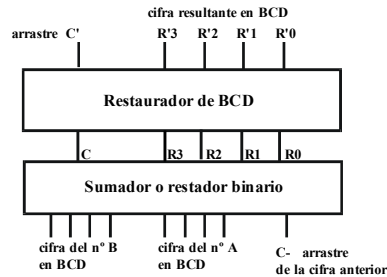
La codificación **BCD** permite introducir directamente los números en un sistema digital a través de 10 teclas, una para cada cifra decimal, y representar directamente los resultados sobre visualizadores de 7 segmentos, a través del correspondiente conversor **BCD** $\rightarrow$ 7 segmentos. En ambos casos no es preciso efectuar cambios de código que afecten al número global; será preciso que cada tecla genere el código **BCD** de su cifra y que cada cifra del resultado sea convertida a 7 segmentos para activar el visualizador.

Las operaciones de suma y resta en **BCD** se efectúan de igual forma que en binario, cifra a cifra, añadiendo las siguientes correcciones:

- en el caso de que el resultado **R** de la suma parcial en una cifra sea superior a 9, es necesario sumar 6 unidades adicionales sobre la misma
- en el caso de que la cifra del minuendo sea inferior a la del sustraendo, se restan 6 unidades.

Esta corrección de 6 unidades se debe al hecho de que en BCD se pasa directamente del 9 al 0 (y *me llevo 1*), mientras que en binario hay 6 unidades intermedias entre el **1001** y el **0000** siguiente: las palabras de 4 bits que corresponden a los números del 10 al 15 (**1010**, **1011**, **1100**, **1101**, **1110**, **1111**).

Tales correcciones pueden realizarse directamente mediante un conversor como el de la figura (**c** = arrastre, **R** = cifra resultante):



cuya configuración booleana corresponde a las siguientes funciones:

SUMA	c	R	c'	R'	RESTA	c	R	c'	R'
0	0000	0	0000	0	0000	0	0000	0	0000
0	0001	0	0001	0	0001	0	0001	0	0001
0	0010	0	0010	0	0010	0	0010	0	0010
0	...	0	...	0	...	0	...	0	...
0	1001	0	1001	0	1001	0	1001	0	1001
0	1010	1	0000	1	1111	1	1111	1	1001
0	1011	1	0001	1	1110	1	1110	1	1000
0	1100	1	0010	1	1101	1	1101	1	0111
0	1101	1	0011	1	1100	1	1100	1	0110
0	1110	1	0100	1	1011	1	1011	1	0101
0	1111	1	0101	1	1010	1	1010	1	0100
1	0000	1	0110	1	1001	1	1001	1	0011
1	0001	1	0111	1	1000	1	1000	1	0010
1	0010	1	1000	1	0111	1	0111	1	0001
1	0011	1	1001	1	0110	1	0110	1	0000

- En la tabla de la suma el «diez» **1010** binario equivale al **1 0000 BCD**, el «once» **1011** al **1 0001 BCD**, el «doce» **1100** al **1 0010 BCD**, etc.;... y el mayor resultado será el que se obtiene al sumar $9 + 9 + \text{arrastre } 1 = 19$: **10011** binario que equivale a **1 1001 BCD**.
- Al restar $0 - 1$ **0000 - 0001** se obtiene **1111** y debería obtenerse **9 1001**, si se resta $0 - 2$ **0000 - 0010** se obtiene **1110** y debería obtenerse **8 1000**, etc.; y el resultado extremo se obtiene al restar $0 - 9 - \text{arrastre } 1 = 0 - 10$ que da **0110** y debería ser **0000**.
- Los vectores de entrada no incluidos en estas tablas no se presentan nunca y pueden ser asignados con valor **X** a efectos de simplificar las funciones.

Las unidades aritméticas y lógicas **ALU** incorporan esta conversión en muchos casos, en particular en los microprocesadores de 8 bits, de forma que admiten la doble posibilidad de suma y resta en binario y en **BCD**.

Para representar los números negativos en sistema de numeración binario se utiliza el *complemento a 2^n* (identificando el 0 con 2^n , siendo n el número de dígitos empleados). De la misma forma, en el sistema decimal los números negativos pueden codificarse en *complemento a 10^n* (complemento a 10), identificando el 0 con 10^n (para n = número de dígitos empleados).

Por ejemplo, para números de valor absoluto inferior a 100.000, añadiendo la cifra de signo en la primera posición resulta un número global de 6 cifras:

$$+1027 = 001027 \quad -8395 = 10^6 - 8395 = 1000000 - 8395 = 99605$$

La cifra inicial 0 es propia de los números positivos, mientras que un 9 inicial indica que el número es negativo.

El cambio de signo puede realizarse sin efectuar la sustracción **$10^n - A$** ; basta calcular el complemento a 9 de cada cifra y sumar al número resultante una unidad:

$$\begin{array}{llll}
 \text{de positivo a negativo} & +1027 & = & 001027 \\
 & 1027' & = & 998972 \\
 & -1027 & = & 998973 \\
 & & & +1 \\
 \text{de negativo a positivo} & -1027 & = & 98973 \\
 & -1027' & = & 01026 \\
 & +1027 & = & 01027 \\
 & & & +1
 \end{array}$$

se complementa a 9 cada cifra

Esta codificación es compatible con la suma y la resta aritmética de números naturales en **BCD** y el desbordamiento (*over-flow*) se produce cuando:

suma	0-----	+	0-----	=	1-----
	9-----	+	9-----	=	8-----
resta	0-----	-	9-----	=	1-----
	9-----	-	0-----	=	8-----

es decir, cuando el bit de signo deja de ser 0 ó 9.

Para operar en **BCD** con números racionales (con parte no entera) se utiliza la representación en coma fija, reservando un número fijo de dígitos BCD para la parte no entera (en forma análoga a la indicada anteriormente para los números racionales en base 2). En **BCD** no se utiliza la representación en coma flotante, ya que, cuando tal codificación resulta necesaria, por razones de precisión, de rango numérico o de velocidad de cálculo, por las mismas razones resulta también conveniente expresar los números y operar con ellos directamente en sistema binario.

4 BLOQUES COMBINACIONALES CONFIGURACIONES RETICULARES

- 4.1. Decodificadores, multiplexores y demultiplexores
- 4.2. Multiplexado mediante puertas de transmisión: tri-estado
- 4.3. Codificadores: conversores de código
- 4.4. Configuración ROM
- 4.5. Sistemas combinacionales; diseño modular
- 4.6. Los bloques combinacionales en VHDL

Un sistema combinacional es aquel cuyas salidas pueden obtenerse por «combinación booleana» de sus entradas, es decir, a través de un conjunto de operaciones lógicas sobre sus variables de entrada; lo cual sucede siempre que a cada vector de entrada le corresponde un solo vector de salida.

En el diseño de sistemas digitales resulta útil aplicar una estrategia de división modular: cuando un sistema resulta complejo es conveniente descomponerlo en subsistemas y realizar su síntesis por separado. Existe una serie de subsistemas conceptuales que aparecen con mucha frecuencia y conviene identificarlos como módulos típicos. Entre ellos ya hemos visto en el capítulo anterior los bloques aritméticos y lógicos (y en el capítulo segundo los decodificadores y multiplexores, a los que volveremos en el presente tema).

Multiplexores y demultiplexores son bloques dedicados a organizar y seleccionar las líneas de transmisión de la información digital, mientras que los decodificadores seleccionan directamente una de entre n posibilidades. Todos ellos aprovechan la propiedad de selección o filtrado que poseen los términos mínimos en relación con su número binario o vector de entrada.

Los codificadores sirven para cambiar el código en que se expresa la información. Una misma información puede representarse en palabras binarias de muy diferentes formas según el código que se utilice; la conversión entre códigos, aunque no modifique la información, es una labor necesaria y útil (por ejemplo, la conversión de una cifra **BCD** al código de 7 segmentos permite representarla «gráficamente» en su forma habitual). Cualquier conjunto de n funciones de m variables puede considerarse como un conversor de un código de m bits a otro de n bits (si bien puede suceder que a diferentes vectores de entrada corresponda un mismo vector de salida, lo cual supone pérdida de información).

Todos estos bloques (los considerados en este capítulo y en el anterior) constituyen los módulos habituales de diseño de un sistema combinacional; sin perjuicio de la necesidad de puertas individuales o conjuntos de funciones booleanas particulares (codificadores específicos). El primer paso en un diseño será efectuar la división del sistema en bloques, es decir, detallar una arquitectura propia en forma de diagrama de bloques.

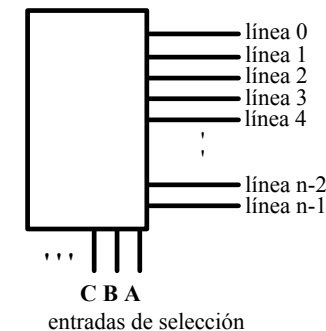
Decodificadores, multiplexores, demultiplexores y codificadores son bloques que suelen presentar un gran número de conexiones internas; la configuración reticular de las mismas simplifica mucho su descripción gráfica.

4.1. Decodificadores, multiplexores y demultiplexores

Una palabra de n bits contiene 2^n posibilidades (numeradas de 0 a 2^n-1), de forma que n entradas permiten seleccionar una de entre 2^n líneas, asignando a cada una de ellas un número de n dígitos. Esta función de selección puede configurarse a través de los términos mínimos de las entradas: un término mínimo adopta valor **1** para su correspondiente vector de entrada y con cualquier otro vector da valor **0**.

4.1.1. Decodificadores

Un decodificador es un «selector de línea» con m entradas y 2^m líneas de salida (numeradas con m dígitos binarios); en cada momento el número binario que recibe como vector de entrada determina que la correspondiente línea de salida se encuentre activa (valor **1**) y el resto de ellas a **0**. Este bloque digital se denomina *decodificador* porque «decodifica» las 2^m posibilidades de un número binario de m dígitos sobre 2^m líneas.



La numeración de las líneas de salida comienza por la línea **0**, pues el primer vector de entrada sobre las líneas de selección será el **0...00**, hasta llegar a la línea **2^m-1** , correspondiente al último vector de selección, **1...11**.

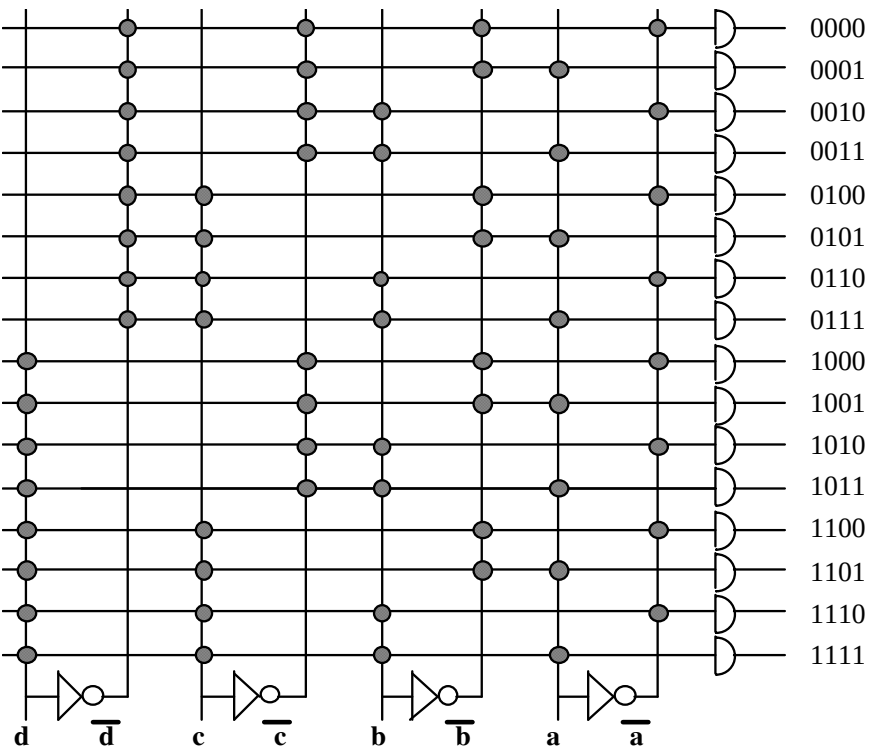
El decodificador es un bloque digital de particular importancia:

- es un selector de línea, que discrimina las líneas de salida según su número binario en las entradas;
- corresponde al conjunto de todos los términos mínimos de sus entradas;
- contiene todas las opciones posibles de sus vectores de entrada y hace corresponder a cada una de ellas con una de las líneas de salida;
- su vector de salida corresponde a un código de «un solo uno» (una sola línea activada, la que corresponde al vector presente en sus entradas).

Asimismo, la disponibilidad de todos los términos mínimos permite utilizar al decodificador para construir funciones booleanas de sus entradas, en su forma canónica; para ello bastará añadir una puerta "o" para cada función [precisamente, los decodificadores fueron introducidos con tal finalidad en 2.4.].

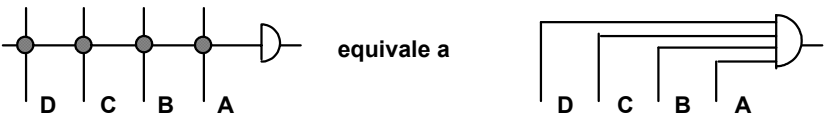
Un decodificador estará configurado por los términos mínimos de sus entradas, cada uno de los cuales dará lugar a una línea de salida: cada salida puede construirse con una simple puerta "y" de *m* entradas, formando el término mínimo que corresponde a su numeración.

Consideremos un decodificador de 4 variables de entrada: sus 16 líneas de salida corresponderán a los 16 términos mínimos de 4 variables y serán las salidas de otras tantas puertas "y" de 4 entradas que podemos dibujar en la forma siguiente:



Decodificador de 4 variables (16 términos mínimos)

La figura anterior presenta una estructura reticular, gracias a que la representación habitual de puertas con varias entradas ha sido sustituida por una sola «línea de entrada» sobre la cual «se conectan» las diversas entradas:



El dibujo de varias entradas «conectadas» sobre una «línea de entrada» única no significa que tales entradas se interconecten formando un nudo común (pues ello daría lugar a un grave conflicto entre sus valores booleanos cuando fueran diferentes) sino que pretende representar, en forma simplificada, las diversas entradas de la puerta lógica (cada una de las cuales actúa por separado).

Esta forma «reticular» de dibujar bloques de muchas salidas (o, también, de muchas entradas) es sumamente útil porque facilita la «lectura» y la comprensión del circuito (evitando el entrecruzamiento de un gran número de líneas) y permite apreciar la distribución de las variables respecto a las diversas puertas lógicas y las regularidades o simetrías existentes en tal distribución.

Además, esta forma de representar las conexiones sobre las puertas lógicas permite trasladar el conjunto de conexiones (o el equivalente conjunto de entradas) sobre una matriz de «ceros» y «unos», en la cual el 1 representa una conexión y el 0 indica la ausencia de ella. Al decodificador de la figura le corresponderá la siguiente matriz de conexiones:

Matriz de conexiones	D	$\bar{D}$	C	$\bar{C}$	B	$\bar{B}$	A	$\bar{A}$
del decodificador	0	1	0	1	0	1	0	1
de 4 variables	0	1	0	1	0	1	1	0
(16 líneas de salida)	0	1	0	1	1	0	0	1
	0	1	0	1	1	0	1	0
	0	1	1	0	0	1	0	1
	0	1	1	0	0	1	1	0
	0	1	1	0	1	0	0	1
	0	1	1	0	1	0	1	0
	1	0	0	1	0	1	0	1
	1	0	0	1	0	1	1	0
	1	0	0	1	1	0	0	1
	1	0	0	1	1	0	1	0
	1	0	1	0	0	1	0	1
	1	0	1	0	0	1	1	0
	1	0	1	0	1	0	0	1
	1	0	1	0	1	0	1	0

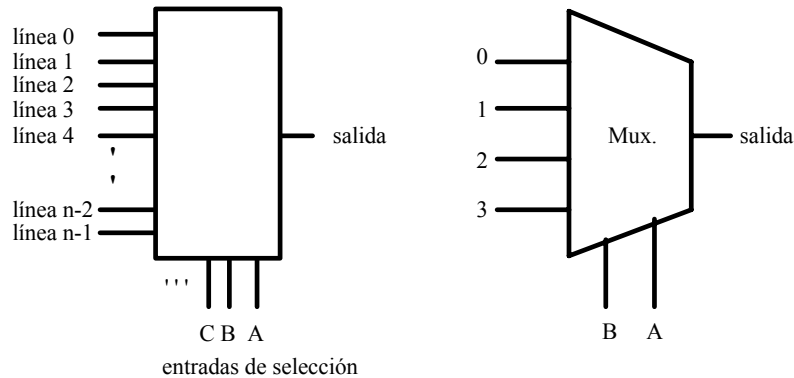
4.1.2. Multiplexores

La función *multiplexado* consiste en seleccionar una de entre varias fuentes de datos para enviarlos por una misma línea; un multiplexor realiza el multiplexado de n líneas de entrada a una sola línea de salida.

Un *multiplexor* es un subsistema digital que selecciona una de entre n fuentes de datos, comunicándola con la línea de salida del mismo; dispone de n líneas de entrada, 1 línea de salida y un conjunto de m líneas de control o selección, tal que $n = 2^m$ (o bien, $2^m \geq n$). En cada momento el vector de entrada en las líneas de selección determina (por su número) la línea de entrada que «queda conectada» a la salida.

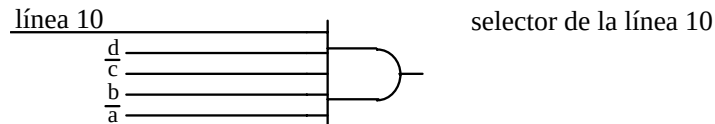
Este tipo de actuación se denomina también *muestreo*, pues selecciona de entre diversas líneas de entrada la que interesa en cada caso: el multiplexor «muestra» la línea cuyo número binario es establecido sobre las entradas de control del mismo.

Las siguientes figuras representan, respectivamente, un multiplexor genérico de n líneas y un multiplexor de 4 líneas:



Los términos mínimos (por su función selectiva respecto a su propio vector de entrada) sirven para filtrar la línea de entrada que corresponde al vector presente en las líneas de selección; para ello basta realizar la operación "y" entre cada línea de entrada y el término mínimo que corresponde a su número en binario.

Por ejemplo, para la línea $10_{10} = 1010_2$:



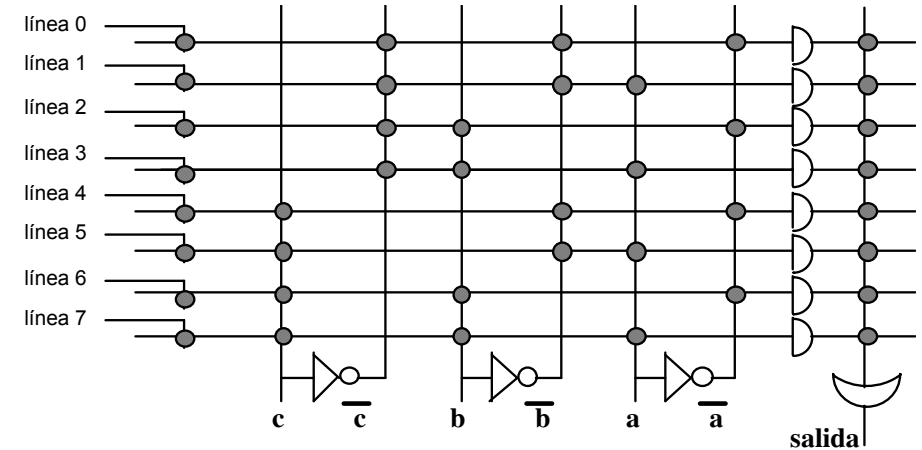
Sólo cuando las variables **dcba** alcanzan el valor $1010_2 = 10_{10}$ dicha puerta presenta en su salida el valor booleano existente en la línea 10; en cualquier otro caso, su salida es **0**.

De esta forma, un multiplexor estará formado por 2^m puertas "y" de $m+1$ entradas confluyendo sobre una puerta "o" de salida (que dará paso a la única línea seleccionada en cada momento):

$$Y = \sum L_i \cdot \text{término mínimo}_i$$

$$= L_0 \cdot \dots \bar{c} \bar{b} \bar{a} + L_1 \cdot \dots \bar{c} \bar{b} a + L_2 \cdot \dots \bar{c} b \bar{a} + L_3 \cdot \dots \bar{c} b a + L_4 \cdot \dots c \bar{b} \bar{a} + \dots$$

Por ejemplo, para 8 líneas de entrada (3 líneas de control) el multiplexor resultante, representado en configuración reticular, es el de la figura siguiente:

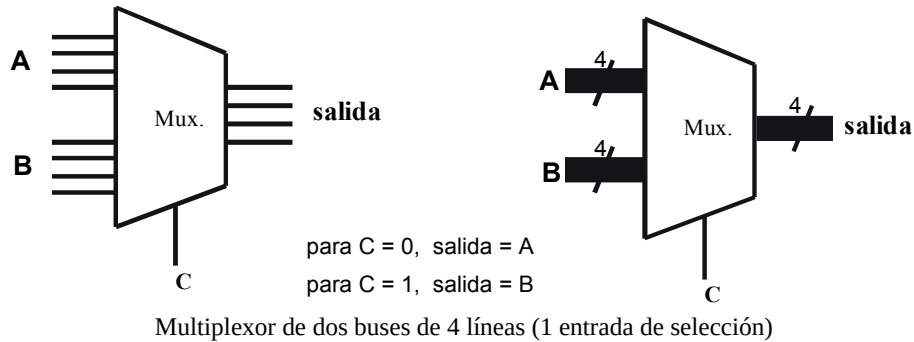


Multiplexor de 8 líneas (3 entradas de control)

Habida cuenta de que $a.b + c.d = (a * b) * (c * d)$ [suma de productos = *Nand de Nands*] las puertas de la figura anterior pueden ser sustituidas, todas ellas a la vez, por puertas "y-negada" (*Nand*).

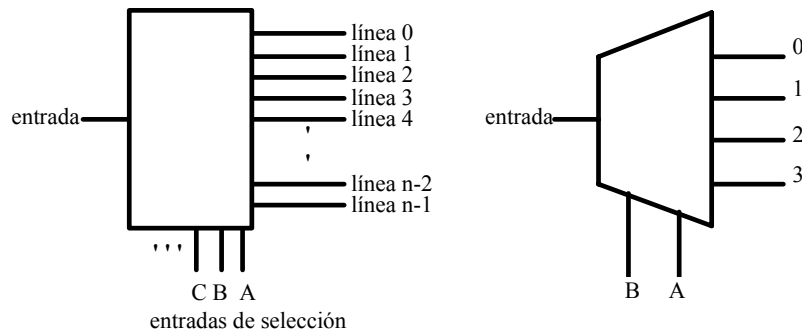
Si asignamos valores booleanos a las líneas de entrada (las conectamos a **0** o a **1**), el multiplexor selecciona uno de dichos valores. De esta forma, podemos construir cualquier función de sus entradas de control, fijando en las líneas de entrada los valores de la «tabla de verdad» de la misma: las variables de control seleccionan sobre la propia tabla de la función el valor booleano que debe adoptar (**look-up-table**) [también los multiplexores fueron introducidos con tal finalidad en II.4]

Cuando los datos a multiplexar constan de varios bits y éstos son enviados en paralelo, en lugar de una única línea se tiene un *bus* o conjunto de líneas que comunican una palabra binaria y la función *multiplexado* consiste igualmente en seleccionar uno de entre varios buses de entrada para comunicarlo con el bus de salida; para un bus de longitud p , es decir compuesto por p líneas, se necesitarán p multiplexores en paralelo.



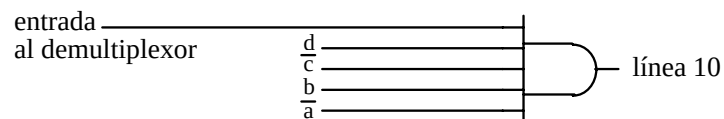
4.1.3. Demultiplexores

Los *demultiplexores* realizan una función contraria a la del multiplexor: reciben los datos por una sola entrada y los dirigen a una de entre n líneas de salida, seleccionables numéricamente por m líneas de control; en cada momento el dato presente en su entrada aparece en la salida cuyo número binario coincide con el establecido en las entradas de control y el resto de líneas de salida permanecen a 0.

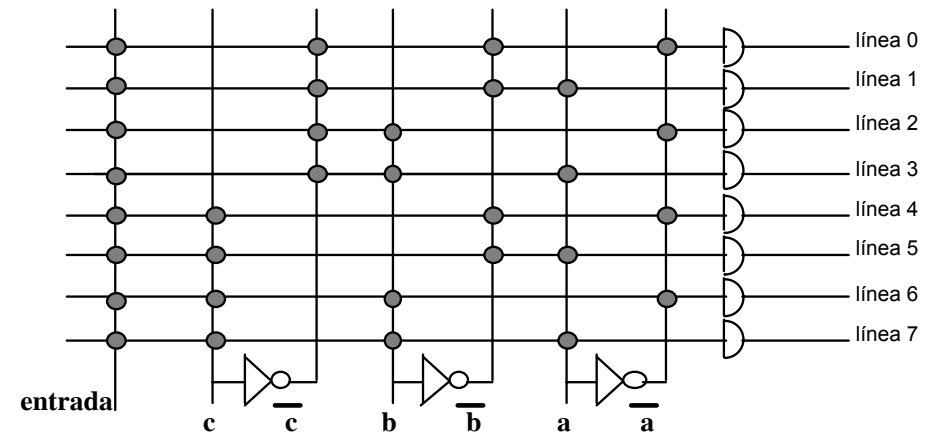


Internamente un demultiplexor está constituido por n puertas "y" a todas las cuales accede la línea de entrada junto con el término mínimo correspondiente de las entradas de control; dicho término mínimo realiza la selección permitiendo la actuación de una sola de las n puertas "y": $\text{salida}_i = \text{entrada} \cdot \text{término mínimo}_i$.

Por ejemplo, para la línea de salida número $10_{(10)} = 1010_{(2)}$:



La figura siguiente representa el esquema de puertas lógicas que configuran un demultiplexor de 8 líneas:



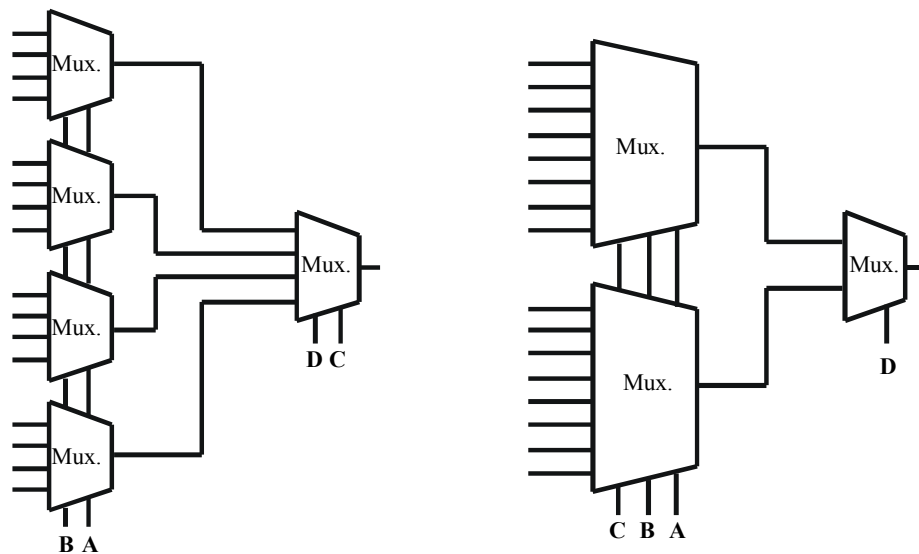
Anulando la línea de entrada del demultiplexor, es decir, conectándola a 1, se obtiene directamente un decodificador: la línea de salida seleccionada por el número binario establecido en las entradas se encontrará a 1 y el resto de líneas de salida estarán a 0.

Tanto los multiplexores como los demultiplexores son modularmente ampliables:

- un multiplexor de n líneas a cuyas entradas se conectan n multiplexores de k líneas, conectándose a su vez las entradas de control de éstos en paralelo, se convierte en un multiplexor de $n \cdot k$ líneas;
- un demultiplexor de n líneas cuyas salidas se conectan a n demultiplexores de k líneas, conectándose también las entradas de control de éstos en paralelo, se convierte en un demultiplexor de $n \cdot k$ líneas;
- en el caso de los decodificadores, su ampliación se realiza conectando sus n salidas a otros tantos demultiplexores de k líneas, con lo que se obtiene un decodificador de $n \cdot k$ líneas.

Por ejemplo, cuatro multiplexores de 4 líneas conectadas sus salidas a las entradas de otro análogo (de 4 líneas) configuran un multiplexor de 16 líneas; el mismo multiplexor de 16 líneas puede configurarse con dos multiplexores de 8 líneas conectados a uno de sólo 2 líneas.

Lo mismo sucede con cuatro demultiplexores de 4 líneas cuyas entradas se conectan a las salidas de otro demultiplexor de 4 líneas o también con dos demultiplexores de 8 líneas conectados a las salidas de uno de 2 líneas.

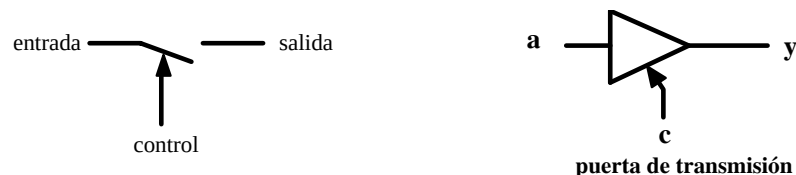


Multiplexor de 16 líneas resultante de la conexión de multiplexores más pequeños

[La figura correspondiente a la conexión de demultiplexores sería la misma, con el flujo de datos de derecha a izquierda: entradas por la derecha, salidas hacia la izquierda.]

4.2. Multiplexado mediante puertas de transmisión: tri-estado

Un interruptor (con una entrada de control) constituye una puerta muy simple: cuando conduce «deja pasar» a su salida el valor booleano presente en su entrada y en corte no permite el paso a su través. Ahora bien, desde el punto de vista de la salida, hay tres situaciones posibles: las dos que corresponden a los valores booleanos **0/1** y una tercera de *desconexión* cuando el interruptor se encuentra en corte.

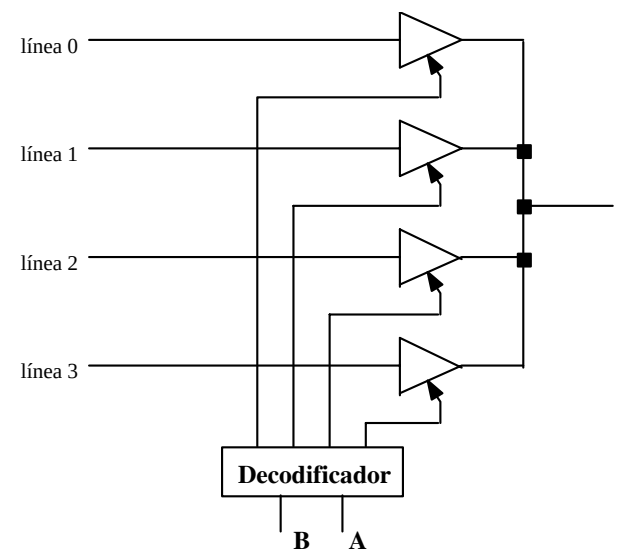


La operación que realiza un interruptor no coincide con las operaciones booleanas básicas (ni con sus derivadas) sino que da lugar a un nuevo tipo de puerta, a la que denominaremos «puerta de transmisión», cuya actuación consiste en permitir/inhibir el paso entre la entrada y la salida.

La salida de una puerta de transmisión presenta tres estados posibles: **0**, **1** y **desconexión**; este tercer estado no corresponde a un valor booleano concreto, sino a una nueva situación física, en la cual la puerta de transmisión no determina (no influye sobre) el valor de la salida.

Se emplea la denominación *tri-estado* para aludir esta triple posibilidad que ofrecen las puertas de transmisión; el estado de desconexión recibe el nombre de «alta impedancia» (pues tal es la situación física a la que corresponde) y se representa en forma abreviada con el símbolo **Z** (o bien, **z↑** en forma más explícita). Los tres valores del tri-estado son **0**, **1** y **Z**.

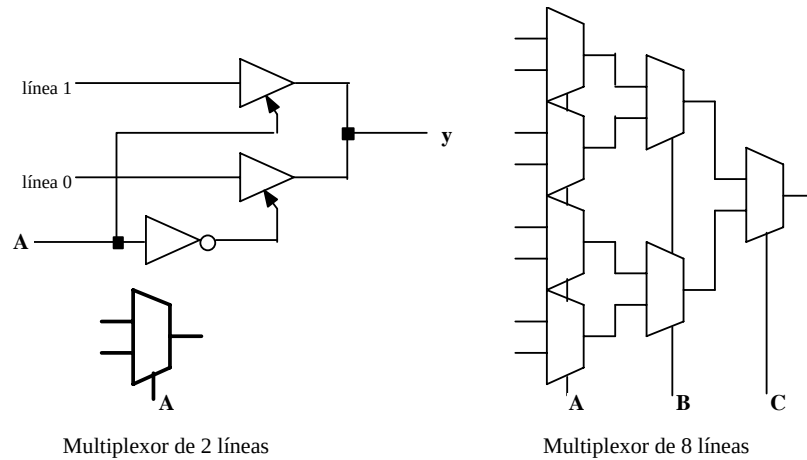
Las puertas de transmisión (o adaptadores tri-estado) no son útiles para configurar funciones lógicas, pero sí para multiplexar señales (para construir multiplexores): podemos conectar las salidas de varias puertas de transmisión y seleccionar una de las señales que reciben, haciendo que la correspondiente puerta esté activada y todas las demás en corte.



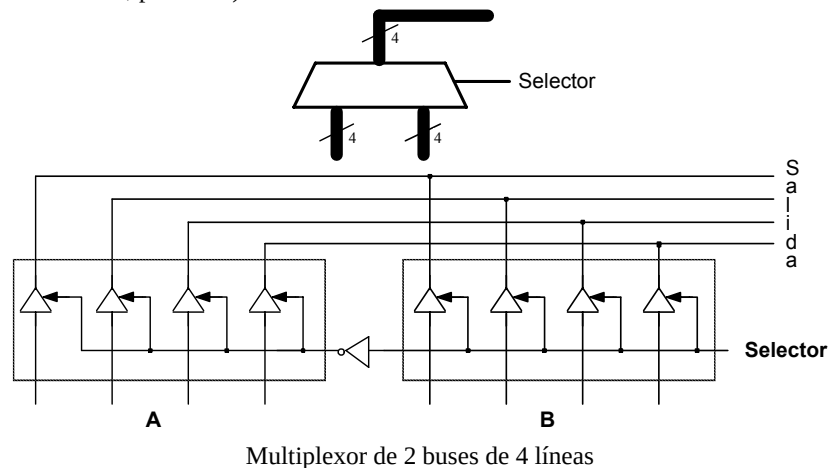
Multiplexor de 4 líneas, construido con 4 puertas de transmisión y un decodificador

Las puertas lógicas no permiten la conexión de varias salidas, unidas en un mismo nudo, porque daría lugar a posibles conflictos entre los dos valores booleanos **0/1**; en cambio, las puertas de transmisión pueden agrupar sus salidas, con tal de que en cada momento conduzca una sola de ellas y las demás se encuentren en corte.

Dos puertas de transmisión con un inversor entre sus entradas de control dan lugar a un pequeño multiplexor de 2 líneas; la conexión de tales multiplexores «en cascada» permite construir multiplexores de mayor número de líneas.



De igual forma la utilización de varias puertas de transmisión en paralelo, con la entrada de control común, permite el multiplexado de buses; la siguiente figura presenta un multiplexor de dos buses de 4 líneas (cuando Selector = 0, pasa el bus A a la salida; con Selector = 1, pasa el B).



Los conjuntos de puertas de transmisión en paralelo reciben también el nombre de adaptadores tri-estado (*buffer tri-state*) y pueden ser, asimismo, de tipo inversor (puertas de transmisión con un inversor previo delante de cada una de ellas).

4.3. Codificadores: conversores de código

En los sistemas digitales toda la información se maneja codificada en palabras binarias. Pero existen múltiples maneras de codificar una misma información; por ejemplo, el valor numérico 94 puede ser representado, entre otros, en los siguientes códigos:

- Binario normal (base 2)	1011110
- Gray	1110001
- BCD (decimal codificado en binario)	1001 0100
- ASCII (representación de texto)	0111001 0110100
- Un solo uno (con 10 niveles)	1000000000 0000010000
- Barras (de 9 niveles)	11111111 00000111
- Siete segmentos (para visualizadores)	1111011 0110011
- ...	

El código binario es la representación normal de números, utilizando el sistema de numeración base 2, mientras que BCD respeta la estructura decimal del número (base 10). El código ASCII se emplea para la representación de caracteres en el tratamiento de textos y es el adecuado para enviar tal información a una impresora.

Los tres códigos siguientes pueden utilizarse para la representación visual de las cifras decimales. El código de «un solo uno» utiliza palabras binarias que contienen solamente un «uno» y corresponde al vector de salida de un teclado decimal (asimismo, era el utilizado por los tubos «Nixie» de 10 filamentos, uno con la forma de cada cifra). El código de barras representa las cifras decimales mediante barras luminosas (de nueve niveles sucesivos) y el código de siete segmentos las representa en su forma habitual (a través de 7 segmentos, con la forma de un 8).

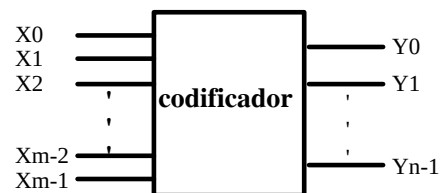
Muchas veces interesa pasar de una codificación a otra; por ejemplo, la entrada de un teclado de 10 pulsadores independientes servirá para introducir cifras decimales en código de «un solo uno» pero, para operar con dichas cifras, generalmente interesará transformarlas a BCD y, una vez obtenidos los resultados, será preciso convertir sus cifras a 7 segmentos para representarlas en un visualizador.

Para efectuar un cambio de código, basta ejecutar la tabla booleana que relaciona ambas codificaciones: si se trata de pasar de una codificación en palabras de m dígitos a otra que utiliza n bits, dicha tabla representa n funciones booleanas (n columnas) de m variables de entrada (2^m filas):

$$X_{m-1} X_{m-2} \dots X_2 X_1 X_0 \rightarrow Y_{n-1} Y_{n-2} \dots Y_2 Y_1 Y_0$$

$$Y_j = f_j(X_i) \quad n \text{ funciones de } m \text{ variables}$$

Un *codificador* o conversor de código es un bloque digital con **m** entradas y **n** salidas, configurado internamente por **n** funciones booleanas de **m** variables.



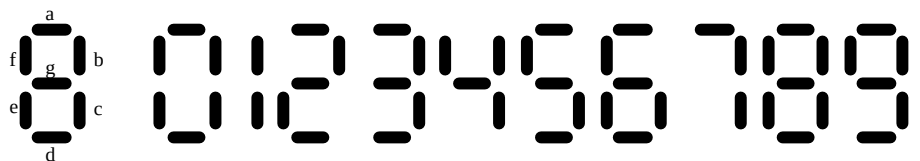
Cada codificador es un bloque específico, cuyas **n** funciones corresponden a la conversión de código que efectúa.

Recíprocamente, cualquier conjunto de **n** funciones de **m** variables puede ser considerado como un conversor de un código de **m** bits a otro de **n** bits; ahora bien, en el caso de que un mismo vector de salida se repita para dos vectores de entrada diferentes, existirá pérdida de información.

Consideremos a continuación, a modo de ejemplos, tres codificadores o conversores de código típicos: un conversor BCD a 7 segmentos, un encodificador de 9 líneas a BCD y un codificador de prioridad de 9 líneas de entrada.

4.3.1. Codificador BCD a 7 segmentos

Las cifras decimales (del 0 al 9) se expresan en BCD con cuatro dígitos binarios (del **0000** al **1001**); pero también dichas cifras se pueden representar mediante un visualizador de 7 segmentos para lo cual hay que codificarlas en 7 bits, cada uno de los cuales controla (*enciende*) uno de los segmentos:



El conversor entre el código BCD y el de 7 segmentos ha de tener 4 entradas y 7 salidas; la tabla de conversión representará las 7 funciones booleanas de 4 entradas que realizan dicho cambio de código; a partir de ella, pueden obtenerse directamente las expresiones algebraicas de dichas funciones que servirán para controlar cada uno de los 7 segmentos.

La tabla correspondiente al paso de BCD a 7 segmentos es la siguiente:

nº decimal	binario				7 segmentos						
	D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1

A partir de ella, utilizando para simplificar los correspondientes diagramas de Karnaugh y expresando las funciones resultantes en términos de puertas "**y-negada**", se obtienen las siguientes expresiones:

a	00	01	11	10
00	1	0	1	1
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

Por términos suma:

$$\begin{aligned}
 a &= (D + C + B + \bar{A}) \cdot (D + \bar{C} + B + A) = \\
 &= D + B + C \cdot A + \bar{C} \cdot \bar{A} = \\
 &= \text{Nand}(\bar{D}, \bar{B}, C * A, \bar{C} * \bar{A})
 \end{aligned}$$

b	00	01	11	10
00	1	1	1	1
01	1	0	1	0
11	X	X	X	X
10	1	1	X	X

Por términos suma:

$$\begin{aligned}
 b &= (\bar{C} + B + \bar{A}) \cdot (\bar{C} + \bar{B} + A) = \\
 &= \bar{C} + B \cdot A + \bar{B} \cdot \bar{A} = \\
 &= \text{Nand}(C, B * A, \bar{B} * \bar{A})
 \end{aligned}$$

c	00	01	11	10
00	1	1	1	0
01	1	1	1	1
11	X	X	X	X
10	1	1	X	X

Por términos suma:

$$\begin{aligned}
 c &= (C + \bar{B} + A) = \\
 &= C + \bar{B} + A = \\
 &= \text{Nand}(\bar{C}, B, \bar{A})
 \end{aligned}$$

d	00	01	11	10
00	1	0	1	1
01	0	1	0	1
11	X	X	X	X
10	1	1	X	X

Por términos producto:

$$\begin{aligned}
 d &= \bar{C}.\bar{A} + \bar{C}.B + C.\bar{B}.A + B.\bar{A} + D = \\
 &= D + C.\bar{B}.A + \bar{C}.B + \bar{C}.\bar{A} + B.\bar{A} = \\
 &= \text{Nand}(\bar{D}, \text{Nand}(C, \bar{B}, A), \bar{C} * B, \bar{C} * \bar{A}, B * \bar{A})
 \end{aligned}$$

e	00	01	11	10
00	1	0	0	1
01	0	0	0	1
11	X	X	X	X
10	1	0	X	X

Por términos producto:

$$\begin{aligned}
 e &= \bar{C}.\bar{A} + B.\bar{A} = \\
 &= \bar{C}.\bar{A} + B.\bar{A} = \\
 &= (\bar{C} * \bar{A}) * (B * \bar{A})
 \end{aligned}$$

f	00	01	11	10
00	1	0	0	0
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

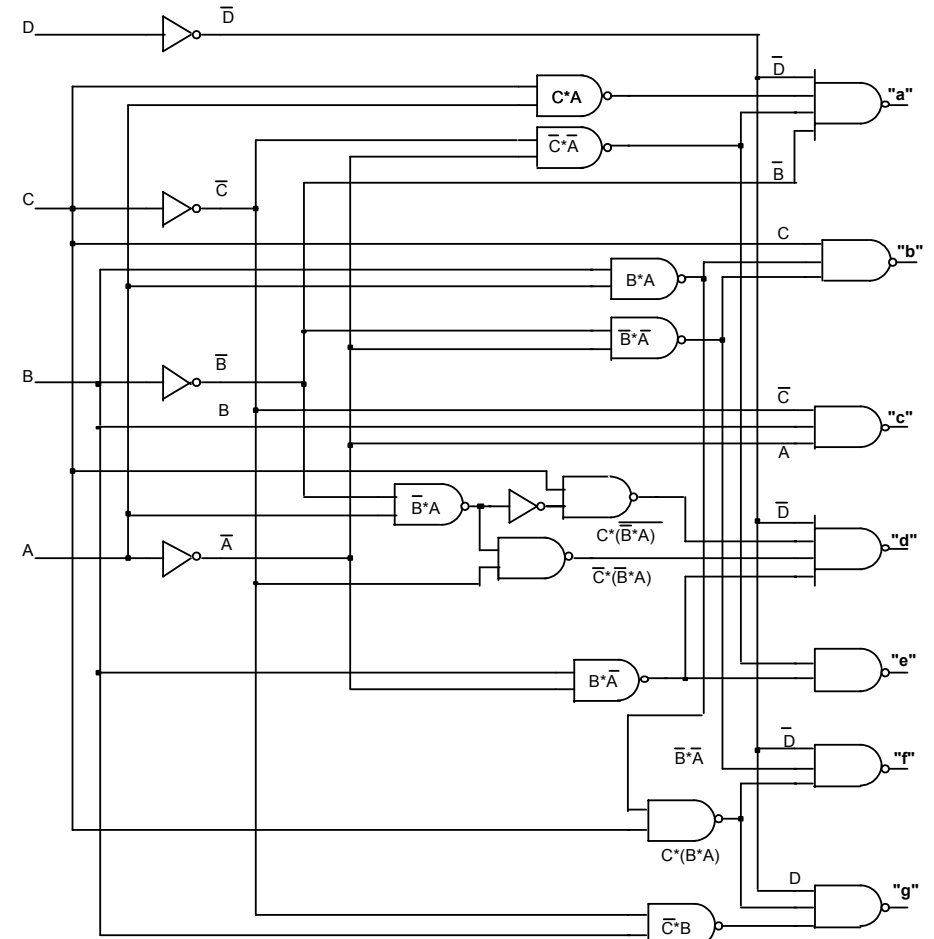
Por términos producto:

$$\begin{aligned}
 f &= \bar{B}.\bar{A} + C.\bar{B} + C.\bar{A} + D = \\
 &= D + C.\bar{B} + C.\bar{A} + \bar{B}.\bar{A} = \\
 &= \text{Nand}(\bar{D}, C * \bar{B}, C * \bar{A}, \bar{B} * \bar{A})
 \end{aligned}$$

g	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	X	X	X	X
10	1	1	X	X

Por términos producto:

$$\begin{aligned}
 g &= \bar{C}.B + C.\bar{B} + C.\bar{A} + D = \\
 &= D + C.\bar{B} + \bar{C}.B + C.\bar{A} = \\
 &= \text{Nand}(\bar{D}, C * \bar{B}, \bar{C} * B, C * \bar{A})
 \end{aligned}$$

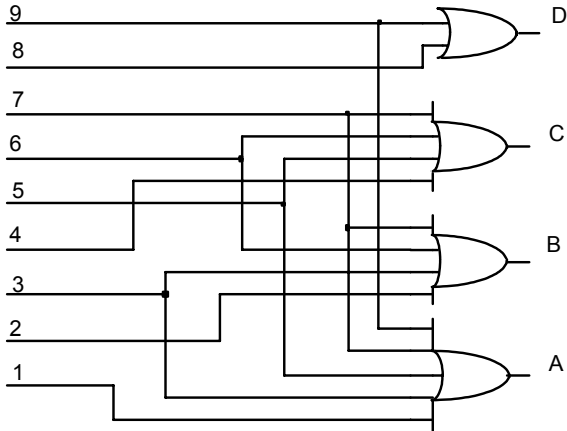
Convertor BCD → 7 segmentos (con puertas *Nand*)

La configuración de la figura utiliza:

- 2 puertas "**y-negada**" de 4 entradas
 - 4 puertas "**y-negada**" de 3 entradas
 - 11 puertas "**y-negada**" de 2 entradas
 - y 5 inversores;
- en total: 22 puertas booleanas.

4.3.2. Codificador de 9 líneas a BCD (encodificador)

Sean nueve líneas numeradas del 1 al 9, de forma que nunca se activan dos de ellas a la vez (código de «un solo uno»), para convertir dichas 9 entradas en su código BCD correspondiente basta utilizar una puerta "o" para cada salida:



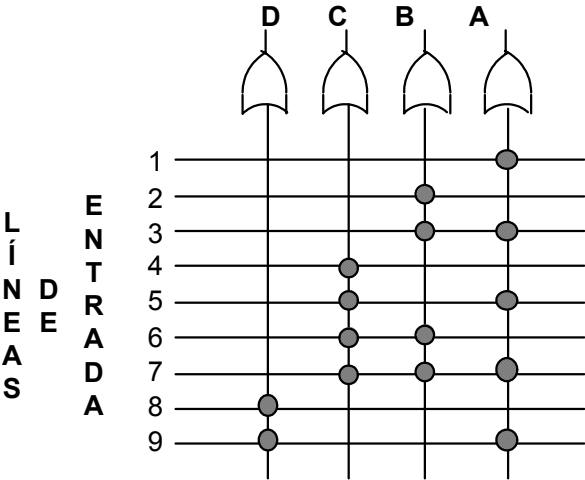
Siempre que el código de entrada es de «un solo uno» el conversor se construye mediante puertas "o" (una para cada salida): cada línea de entrada coincide con uno de los códigos de entrada y debe conectar con aquellas puertas "o" que deben adoptar valor 1 en el correspondiente código de salida. Por ejemplo, el código de la línea 9 es **1001** y, por tanto, debe incidir en las dos puertas "o" de los extremos.

Este tipo de codificador (código de entrada de «un solo uno» - puertas "o" de salida) recibe el nombre de *encodificador* (porque llevan las entradas directamente a las puertas de salida).

La tabla funcional de este encodificador es la siguiente:

l ₉	l ₈	l ₇	l ₆	l ₅	l ₄	l ₃	l ₂	l ₁	D	C	B	A
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0	0	1	0	0	1

La representación reticular resulta también muy apropiada para los encodificadores:



La matriz de conexiones de un encodificador, en estructura reticular, se corresponde directamente con la tabla funcional del encodificador (con la parte derecha de dicha tabla):

Matriz de conexiones	D	C	B	A	
del encodificador	0	0	0	1	línea 1
de 9 líneas	0	0	1	0	línea 2
(4 salidas)	0	0	1	1	línea 3
	0	1	0	0	línea 4
	0	1	0	1	línea 5
	0	1	1	0	línea 6
	0	1	1	1	línea 7
	1	0	0	0	línea 8
	1	0	0	1	línea 9

De forma que, para dibujar un codificador en configuración reticular, basta establecer la correspondiente retícula líneas de entrada – puertas "o" de salida y sobre la misma reflejar la tabla funcional, insertando una conexión cuando el valor de dicha tabla es 1 y dejando sin conexión aquellos cruces que corresponden a valor 0.

4.3.3. Codificador de prioridad de 9 líneas

Sean nueve líneas numeradas del 1 al 9 como en el caso anterior, pero admitiendo la posibilidad de que se activen varias a la vez: el código de salida será el más alto de los números binarios que corresponden a las líneas que se encuentran activadas. La tabla de conversión simplificada es la siguiente:

l ₉	l ₈	l ₇	l ₆	l ₅	l ₄	l ₃	l ₂	l ₁	D	C	B	A
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	0	1	X	0	0	1	0
0	0	0	0	0	0	1	X	X	0	0	1	1
0	0	0	0	0	1	X	X	X	0	1	0	0
0	0	0	0	1	X	X	X	X	0	1	0	1
0	0	0	1	X	X	X	X	X	0	1	1	0
0	0	1	X	X	X	X	X	X	0	1	1	1
0	1	X	X	X	X	X	X	X	1	0	0	0
1	X	X	X	X	X	X	X	X	1	0	0	1

de la cual resultan las siguientes funciones booleanas (simplificadas por «absorción»):

$$D = l_9 + l_8$$

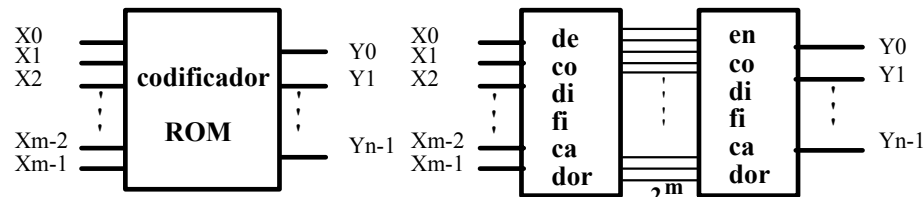
$$C = \overline{l_9} \cdot \overline{l_8} \cdot (l_7 + l_6 + l_5 + l_4)$$

$$B = \overline{l_9} \cdot \overline{l_8} \cdot (l_7 + l_6) + \overline{l_9} \cdot \overline{l_8} \cdot \overline{l_5} \cdot \overline{l_4} \cdot (l_3 + l_2)$$

$$A = l_9 + \overline{l_8} \cdot l_7 + \overline{l_8} \cdot \overline{l_6} \cdot l_5 + \overline{l_8} \cdot \overline{l_6} \cdot \overline{l_4} \cdot l_3 + \overline{l_8} \cdot \overline{l_6} \cdot \overline{l_4} \cdot \overline{l_2} \cdot l_1$$

4.4. Configuración ROM

En general, la tabla funcional correspondiente a un cambio de codificación puede ser muy amplia y la síntesis de las funciones booleanas que corresponden a dicha tabla puede ser sumamente compleja. Sin embargo, la conversión de un código a otro puede resolverse de una manera conceptualmente más sencilla, sin necesidad de construir dichas funciones booleanas, dividiéndolo en dos partes: *decodificación* y *encodificación*.



El primer bloque realiza una *decodificación* completa de los vectores de entrada sobre 2^m líneas intermedias, que adoptarán el código de «un solo uno», propio de las salidas de un decodificador; dichas 2^m líneas intermedias se *encodifican* sobre las n líneas de salida.

De hecho, la figura anterior equivale a configurar las n funciones booleanas de las salidas en su forma canónica: suma de términos mínimos. Primero, se construyen todos los términos mínimos de las entradas y, luego, cada salida recoge en una puerta "o" (suma) los que corresponden a su función, es decir, aquellos cuya columna en la tabla funcional presenta valor 1.

Los codificadores configurados según este esquema reciben el nombre genérico de **ROM** (*read only memory*, memorias de sólo lectura), nombre confuso que procede de la utilización de estos bloques en los computadores y microprocesadores; sin embargo, estos bloques son claramente combinacionales y no tienen, en absoluto, capacidad de memoria, la cual es propia de los sistemas secuenciales. [Por ello, desde el punto de vista de que son bloques combinacionales y a tenor de su estructura funcional, propongo interpretar las siglas **ROM** como **Row Ordered Minterms**: filas de términos mínimos.]

La configuración **ROM** contiene dos partes diferenciadas:

- un *decodificador*, cuya realización solamente depende del número m de entradas, pero es independiente del codificador concreto de que se trate
- y un *encodificador*, constituido por n puertas "o", cada una de las cuales recibe parte de las líneas de salida del decodificador.

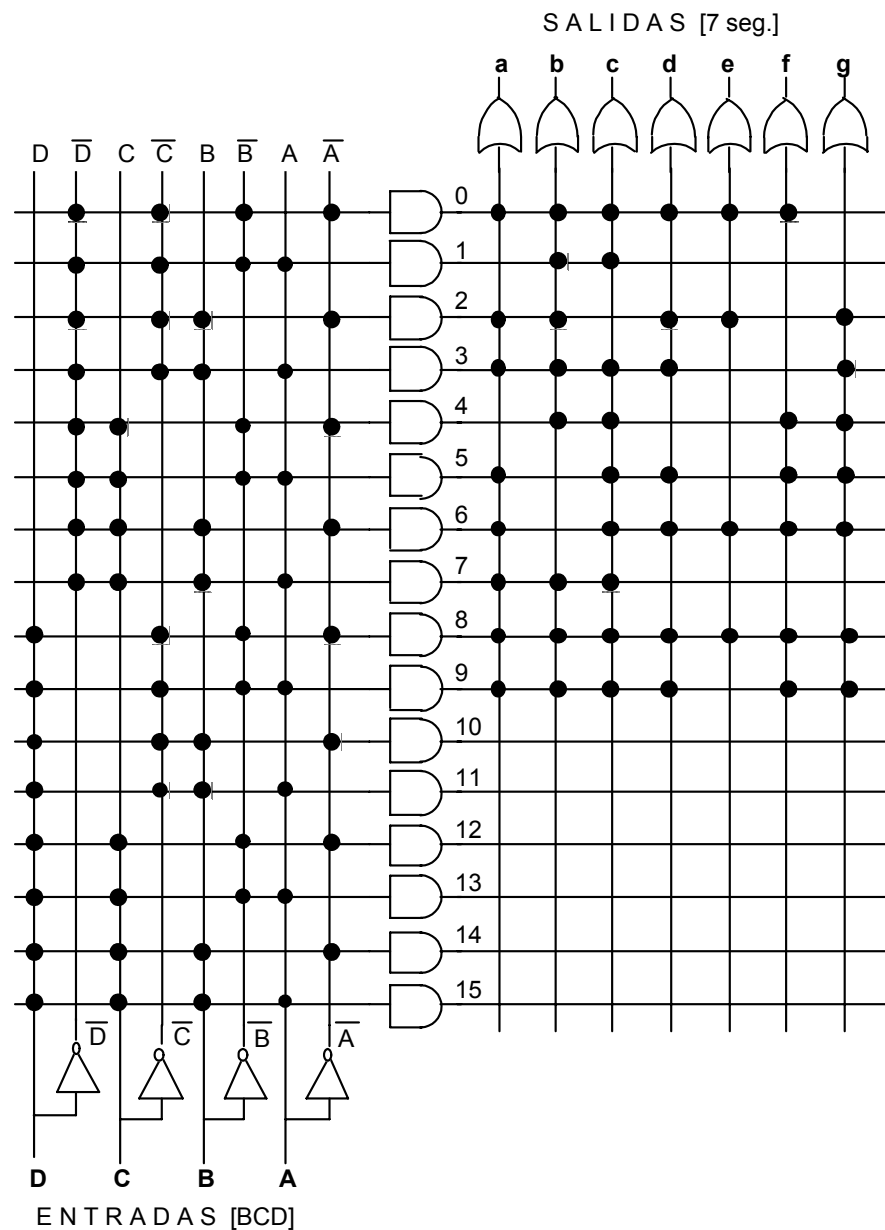
Los subconjuntos de líneas de salida del decodificador conectadas a cada una de las puertas "o" del encodificador personalizan el codificador, es decir, son lo único que varía entre dos codificadores diferentes de igual número de entradas y salidas.

En términos de puertas booleanas tal estructura presenta gran simplicidad y regularidad; está constituida modularmente por dos conjuntos de puertas:

- el primero de puertas "y" (2^m puertas "y" de m entradas que forman el decodif.)
- seguido de un conjunto de puertas "o" (n puertas "o" que forman el encodif.).

Para ambos bloques, la configuración reticular resulta muy apropiada; la matriz de conexiones del encodificador coincide con la tabla funcional (con la parte de la derecha, de asignación de valores a las salidas) de la conversión entre los dos códigos.

Por ejemplo, en el caso del conversor de BCD a 7 segmentos, su configuración ROM (en forma reticular) es la representada en la figura de la página siguiente. [Compruébese en ella que la matriz de conexiones del encodificador es idéntica a la tabla funcional, página 96.]



Codificador ROM BCD → 7 segmentos

Ahora bien, cuando el número de entradas del codificador es pequeño (como sucede en el caso anterior) la configuración **ROM** de un codificador no supone una ventaja apreciable respecto a la obtención de sus funciones booleanas simplificadas.

La estructura **ROM** es realmente apropiada cuando el número de entradas es alto (por encima de las 6 entradas); en tal caso, la configuración **ROM** puede ser construida directamente desde la tabla funcional o tabla de conversión, evitando la necesidad de extraer de ella y de simplificar las funciones booleanas. Dicha construcción consiste en:

- configurar un decodificador genérico de **m** entradas, o sea, construir reticularmente los **2^m** términos mínimos,
- seguido de un encodificador, cuya matriz de conexiones ha de ser la propia tabla de conversión.

Dos ejemplos concretos de cambios de código con un número amplio de entradas los constituyen las *tablas funcionales* y los *generadores de caracteres*.

Podemos considerar un codificador **ROM** como una tabla de valores que para cada valor numérico de entrada nos proporciona el resultado de una determinada función aplicada a dicho número; de esta forma los codificadores **ROM** permiten configurar tablas reducidas de funciones, *tablas funcionales*, como, por ejemplo, tablas trigonométricas (senos, cosenos, tangentes,...), tablas de logaritmos, etc.

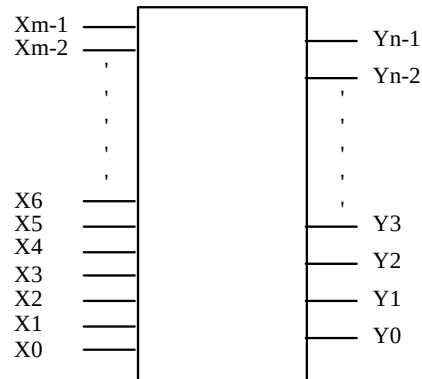
Los *generadores de caracteres* son codificadores que convierten los caracteres alfanuméricos codificados en binario (por ejemplo, en código **ASCII** de 7 bits) en matrices rectangulares de puntos que representan visualmente dichos caracteres. La visualización final puede realizarse bien mediante diodos luminiscentes **LED** o lámparas, bien en monitores o pantallas de rayos catódicos **CRT**, bien sobre papel mediante impresoras de puntos, etc. Los puntos de una matriz **n x m** se activarán (iluminación o impresión) cuando se encuentren con valor **1** y no lo harán si es **0**.

Las pequeñas mini-impresoras por puntos suelen imprimir en columnas de 7 puntos, utilizando 5 columnas para cada letra o número y 2 de separación. El codificador **ROM** para la correspondiente generación de caracteres ha de tener como entradas los 7 dígitos del carácter **ASCII** más 3 dígitos para indicar el número de columna a imprimir y presentará como salidas los 7 bits que van a controlar los solenoides que activan los bastones de impresión.

En cambio, en el barrido de la pantalla de un monitor (**CRT**) las letras y números se dibujan por filas, de forma que el generador de caracteres ha de tener como entradas los 7 dígitos de cada carácter más 3 ó 4 dígitos para indicar el número de la fila y las salidas son los 5 ó 7 bits correspondientes a los puntos que conforman una fila del carácter, caso de representarlos en matrices de 7 x 5 puntos o de 9 x 7 puntos, respectivamente.

4.5. Sistemas combinacionales; diseño modular

Entendemos por sistema lógico o sistema booleano aquel que puede describirse en términos del álgebra de Boole de 2 elementos. En principio, dicho sistema tendrá m variables o entradas, cuyos valores booleanos configuran un *vector de entrada* de dimensión m , y n resultados o salidas, cuyos valores booleanos configuran un *vector de salida* de dimensión n .



Un sistema lógico es *combinacional* si a cada *vector de entrada* le corresponde un solo *vector de salida*, es decir, si la salida es única (aplicación unívoca del conjunto de vectores de entrada en el conjunto de vectores de salida): para un vector de entrada dado no existe más de un vector de salida.

La correspondencia unívoca, es decir, el que a un vector de entrada dado le corresponda un solo vector de salida puede parecer obvia y, sin embargo, no lo es: en el capítulo 11, al introducir los sistemas secuenciales (sistemas digitales que poseen «memoria»), se apreciará que para ellos la aplicación vectores de entrada $\rightarrow$ vectores de salida no es unívoca.

Cada variable de salida y es una función booleana de las m variables de entrada x_i ; tal función puede expresarse mediante su tabla funcional o *tabla de verdad*, haciendo explícito el valor booleano que adopta la variable de salida para cada uno de los 2^m vectores de entrada. En tal sentido, el nombre de *combinacional* proviene de que las salidas o respuestas del sistema son función booleana de sus entradas, es decir, son «combinación» mediante operaciones booleanas de las variables de entrada.

Es posible que el vector de salida correspondiente a uno de entrada sea indeterminado o indiferente (*no importa cual sea*). Esto sucede, por ejemplo, en muchos diseños reales cuando un determinado vector de entrada no se presenta nunca en la práctica o cuando no importa qué respuesta produzca el sistema para un determinado vector de entrada (porque no hay exigencias respecto a tal respuesta y resulte indiferente que sea una u otra). La indeterminación puede referirse a todas las variables de salida o solamente a algunas de ellas: indeterminación parcial.

En los casos de indeterminación se asigna inicialmente a las correspondientes variables de salida el valor X , que puede ser 1 ó 0 según interese; esta doble posibilidad se utiliza para simplificar las funciones booleanas resultantes.

Así pues, en un sistema lógico combinacional a un vector de entrada le corresponde uno y sólo un vector de salida; o bien no le corresponde inicialmente ninguno (la salida es indeterminada) y en el proceso de síntesis se le asigna un vector de salida cuyas componentes se eligen según interese.

El análisis de un sistema combinacional pasa por obtener su *tabla de operación* o conjunto de *tablas de verdad*, expresando para cada vector de entrada el vector de salida que le corresponde.

La síntesis de sistemas combinacionales consiste en construir, a partir de las especificaciones o requisitos que se persiguen, la *tabla funcional* que relaciona los 2^m vectores de entrada posibles con los vectores de salida que deben producir; a partir de dicha tabla se obtienen las correspondientes funciones booleanas que permiten construir el sistema por combinación adecuada de puertas lógicas.

En general, en todo proceso de síntesis resulta sumamente útil la aplicación de estrategias de división modular: cuando un sistema combinacional es de una cierta complejidad es necesario descomponerlo en subsistemas y realizar su síntesis por separado. En tal sentido, existe una serie de subsistemas conceptuales que aparecen con mucha frecuencia y es sumamente útil conocerlos; además, la alta densidad de integración que ofrecen los circuitos integrados ha permitido que dichos subsistemas se encuentren disponibles como circuitos integrados.

Tales subsistemas reciben el nombre de *Bloques Combinacionales*; en los dos últimos capítulos hemos estudiado diversos bloques de utilidad general, que pueden clasificarse en tres grupos:

- *Bloques operacionales aritméticos y lógicos*, capaces de efectuar una operación aritmética entre dos números binarios de m dígitos o de efectuar una operación lógica (bit a bit) entre dos palabras binarias de longitud m .
- *Bloques distribuidores: multiplexores, demultiplexores y decodificadores*, dedicados a organizar y seleccionar las líneas de transmisión de la información.
- *Bloques codificadores o conversores de código*, que realizan la conversión entre códigos, o sea, entre palabras binarias que expresan la misma información.

El interés de los bloques combinacionales no se limita a su disponibilidad en forma de circuitos integrados; incluso la utilización de tales circuitos integrados estándar es, cada vez, menos habitual pues interesa configurar todo el diseño digital en un solo integrado, bien sea éste programable o construido a medida (ASIC).

Los bloques combinacionales interesan por sí mismos, como módulos conceptuales o «piezas de diseño» que facilitan estrategias de división en partes (diagramas de bloques); tal «partición» es imprescindible para abordar el diseño de sistemas complejos.

El tiempo de propagación en los circuitos combinacionales

"El tiempo es un compañero inevitable" en el funcionamiento de los circuitos digitales, al igual que en cualquier otro sistema físico real. "Nada es instantáneo" en la respuesta de los sistemas físicos en relación con su excitación, con las entradas que, al incidir sobre el sistema, causan la respuesta del mismo: la relación causa - efecto requiere un cierto tiempo de ejecución: $\text{estímulo} \rightarrow \Delta t \rightarrow \text{respuesta}$.

En un circuito combinacional se produce un tiempo de retardo entre el vector de entrada y el correspondiente vector de salida, intervalo temporal al que denominamos tiempo de propagación t_p . En la práctica el tiempo de propagación será diferente para cada una de las salidas del circuito combinacional; habida cuenta de que resulta necesario esperar a la más lenta de ellas para obtener la respuesta global, el tiempo de propagación del circuito será el mayor de los tiempos de retraso de sus salidas.

Los tiempos de propagación dependen linealmente del *fan-out* que soportan las puertas, es decir, de la carga capacitiva conectada a su salida; además, aumentan fuertemente con la temperatura. Un circuito electrónico se calienta por la disipación de la energía que consume en su propio funcionamiento que se elimina en forma de calor, con la consiguiente elevación de la temperatura del circuito y disminución de la velocidad de trabajo del mismo; en muchos casos será necesario incluir una refrigeración adecuada para garantizar la frecuencia de trabajo del circuito.

La conexión sucesiva de puertas booleanas implica la correspondiente adición de sus tiempos de propagación: el tiempo de retardo del conjunto será del orden de la suma de los tiempos individuales Σt_p . Si bien dicha suma expresa solamente una cota superior del tiempo de propagación global, ya que los tiempos de propagación de las sucesivas puertas se solapan parcialmente: una puerta no espera para iniciar su conmutación a que la anterior haya completado la suya.

Para valorar, en una primera aproximación, el tiempo de propagación de un bloque combinacional, denominaremos índice de propagación i_p al número de puertas sucesivas que se presentan entre una salida y una entrada, o bien, al mayor de los números de puertas sucesivas que contiene un circuito combinacional. El índice de propagación proporciona una referencia, al menos cualitativa, del tiempo de propagación global.

Las salidas de un bloque combinacional corresponden a funciones booleanas de las entradas del mismo y, en principio, cualquier función booleana puede expresarse como suma de productos; tales sumas de productos se construyen en la práctica mediante dos niveles de puertas "**y-negada**", *Nand* (*Nand de Nands*), junto con un tercer nivel de inversores para las variables de entrada negadas, de forma que el índice de propagación de una función booleana construida como suma de productos es $i_p = 3$.

Como veremos en el próximo capítulo (cap. V), cualquier conjunto de funciones booleanas, en particular cuando el número de sus entradas es relativamente alto, puede construirse mediante un esquema: [matriz Y de entradas] [matriz O de salidas]; en la práctica (precisamente para admitir múltiples entradas), este esquema se construye con puertas "**o-negada**", *Nor pseudoNMOS*, y requiere cuatro niveles de puertas:

[inversores de las entradas] [matriz NOR de entradas]
[matriz NOR de salidas] [inversores de las salidas]

lo cual supone un índice de propagación $i_p = 4$.

Así pues, el índice de propagación necesario para construir cualquier sistema combinacional es 4: según este esquema booleano ningún sistema combinacional requiere un índice superior a 4 y, siempre que ello sea necesario por razones de velocidad, cualquier sistema combinacional puede construirse con los 4 niveles antes detallados.

Ahora bien, hay casos de bloques combinacionales en que la construcción directa de cada una de sus salidas presenta alta complejidad y considerable área de integración, porque tales circuitos se adaptan mejor a un esquema recursivo de celdas en cascada; sistemas combinacionales que, en lugar de ajustarse fácilmente a una configuración en paralelo de sus salidas, se acomodan conceptualmente a un cálculo sucesivo (en serie) de las mismas. Por ejemplo, los sumadores y restadores de n bits, cuya construcción es muy simple a partir de celdas de 1 bit, conectadas en cascada.

En estos casos de diseño celular recursivo (conexión de pequeños módulos en serie), el índice de propagación puede hacerse relativamente alto y, consiguientemente, el tiempo de propagación global del circuito.

El caso más típico y habitual lo constituyen los sumadores de n bits, conformados por n celdas sumadoras de 1 bit; en ellos el arrastre (*carry*) ha de propagarse sucesivamente a través de las n celdas: propagación por onda (*ripple carry*):

$$C_{i+1} = a_i \cdot b_i + (a_i + b_i) \cdot C_i = (a_i * b_i) \cdot [(a_i \Delta b_i) + \overline{C_i}]$$

El índice de propagación del arrastre en una celda básica de 1 bit es $i_p = 2$; un sumador de 64 bits presentará un índice de propagación muy alto $i_p = 128$.

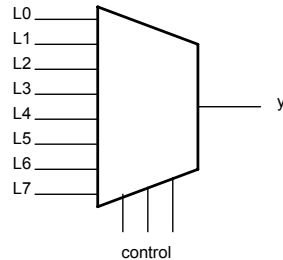
Los esquemas celulares en cascada presentan altos índices de propagación y, por ello, introducen limitaciones fuertes respecto a la frecuencia de trabajo. Cuando interesan velocidades mayores, la forma de conseguirlo se basa en disminuir el número de módulos en cascada, aumentando la «granularidad» de cada módulo; es decir, en lugar de utilizar celdas básicas de 1 bit (granularidad = 1), emplear módulos de p bits (granularidad = p), construyendo directamente (en paralelo) las funciones booleanas de los mismos.

Por ejemplo, en el caso de sumadores, si en lugar de celdas básicas de 1 bit se utilizan módulos sumadores de 4 bits el índice de propagación se reduce en un factor 4. Más aún, el apéndice A2 (referido a la "Propagación rápida de acarreo en los sumadores") muestra cómo la recursividad puede ser aprovechada para conseguir reducciones del índice de propagación aún mayores.

4.5. Los bloques combinacionales en VHDL

Al igual que en el capítulo 1, se incluye aquí la descripción de algunos bloques combinacionales en VHDL a fin de desarrollar una aproximación gradual al diseño con lenguajes de descripción circuital.

Multiplexor de 8 líneas



control : INTEGER range 0 to 7

versión 1

```
y <=  L0 when control = 0 else
      L1 when control = 1 else
      L2 when control = 2 else
      L3 when control = 3 else
      L4 when control = 4 else
      L5 when control = 5 else
      L6 when control = 6 else L7;
```

versión 2

```
with control select
  y <=  L0 when 0,
      L1 when 1,
      L2 when 2,
      L3 when 3,
      L4 when 4,
      L5 when 5,
      L6 when 6,
      L7 when others;
```

descripción utilizando proceso

```
process (control,L7,L6,L5,L4,L3,L2,L1,L0)
begin
```

versión 1

```
if control = 0 then y <= L0; end if;
if control = 1 then y <= L1; end if;
if control = 2 then y <= L2; end if;
if control = 3 then y <= L3; end if;
if control = 4 then y <= L4; end if;
if control = 5 then y <= L5; end if;
if control = 6 then y <= L6; end if;
if control = 7 then y <= L7; end if;
end process;
```

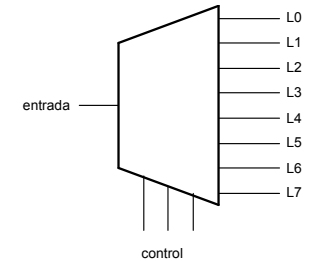
versión 2

```
case control is
  when 0 => y <= L0;
  when 1 => y <= L1;
  when 2 => y <= L2;
  when 3 => y <= L3;
  when 4 => y <= L4;
  when 5 => y <= L5;
  when 6 => y <= L6;
  when 7 => y <= L7;
end case;
end process;
```

Demultiplexor de 8 líneas

control : INTEGER range 0 to 7

```
L0 <= entrada when control = 0   else '0';
L1 <= entrada when control = 1   else '0';
L2 <= entrada when control = 2   else '0';
L3 <= entrada when control = 3   else '0';
L4 <= entrada when control = 4   else '0';
L5 <= entrada when control = 5   else '0';
L6 <= entrada when control = 6   else '0';
L7 <= entrada when control = 7   else '0';
```



utilizando proceso

```
process (control,entrada)
```

```
begin
```

```
L0 <= '0'; L1 <= '0'; L2 <= '0'; L3 <= '0';
L4 <= '0'; L5 <= '0'; L6 <= '0'; L7 <= '0';
```

```
case control is
```

```
  when 0 => L0 <= entrada;
  when 1 => L1 <= entrada;
  when 2 => L2 <= entrada;
  when 3 => L3 <= entrada;
  when 4 => L4 <= entrada;
  when 5 => L5 <= entrada;
  when 6 => L6 <= entrada;
  when 7 => L7 <= entrada;
```

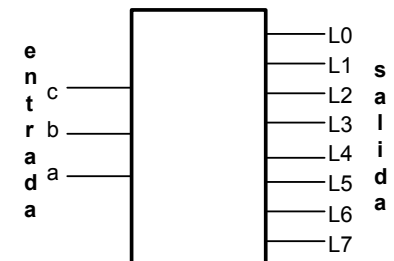
```
end case;
```

```
end process;
```

Decodificador de 8 líneas

with entrada select

```
salida <=  "10000000" when "000",
           "01000000" when "001",
           "00100000" when "010",
           "00010000" when "011",
           "00001000" when "100",
           "00000100" when "101",
           "00000010" when "110",
           "00000001" when "111";
```

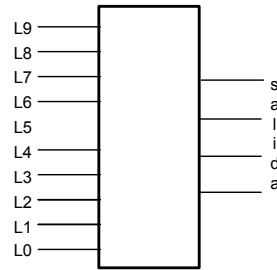


Encodificador de 10 líneas

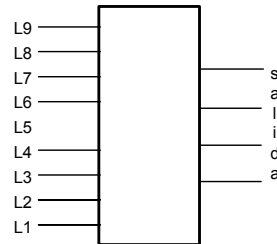
entrada <= L9 & L8 & L7 & L6 & L5 & L4 & L3 & L2 & L1 & L0;

with entrada select

```
salida <= "1001" when "1000000000",
          "1000" when "0100000000",
          "0111" when "0010000000",
          "0110" when "0001000000",
          "0101" when "0000100000",
          "0100" when "0000010000",
          "0011" when "0000001000",
          "0010" when "0000000100",
          "0001" when "0000000010",
          "0000" when "0000000001",
          "1111" when others;
```

**Codificador de prioridad de 9 líneas.**

```
salida <= "1001" when L9 = '1' else
          "1000" when L8 = '1' else
          "0111" when L7 = '1' else
          "0110" when L6 = '1' else
          "0101" when L5 = '1' else
          "0100" when L4 = '1' else
          "0011" when L3 = '1' else
          "0010" when L2 = '1' else
          "0001" when L1 = '1' else
          "0000";
```



utilizando proceso

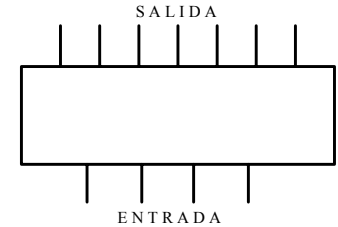
```
process (L9,L8,L7,L6,L5,L4,L3,L2,L1)
begin
if L9 = '1' then      salida <= "1001";
  elsif L8 = '1' then salida <= "1000";
  elsif L7 = '1' then salida <= "0111";
  elsif L6 = '1' then salida <= "0110";
  elsif L5 = '1' then salida <= "0101";
  elsif L4 = '1' then salida <= "0100";
  elsif L3 = '1' then salida <= "0011";
  elsif L2 = '1' then salida <= "0010";
  elsif L1 = '1' then salida <= "0001";
  else                salida <= "0000";
end if;
end process;
```

Conversor BCD a 7 segmentos, ánodo común.

[En un visualizador de ánodo común se «encienden» aquellos segmentos que reciben 0, ya que la línea de alimentación común a todos ellos corresponde a la tensión positiva Vcc es decir, al 1 booleano.]

SALIDA : Out BIT_VECTOR(1 to 7);

ENTRADA : In INTEGER range 0 to 15;



with entrada select

```
SALIDA <= "0000001" when 0,
          "1001111" when 1,
          "0010010" when 2,
          "0000110" when 3,
          "1001100" when 4,
          "0100100" when 5,
          "0100000" when 6,
          "0001111" when 7,
          "0000000" when 8,
          "0000100" when 9,
          "1111111" when others;
```

descripción utilizando proceso

process (ENTRADA)

begin

case ENTRADA is

```
when 0 => SALIDA <= "0000001";
when 1 => SALIDA <= "1001111";
when 2 => SALIDA <= "0010010";
when 3 => SALIDA <= "0000110";
when 4 => SALIDA <= "1001100";
when 5 => SALIDA <= "0100100";
when 6 => SALIDA <= "0100000";
when 7 => SALIDA <= "0001111";
when 8 => SALIDA <= "0000000";
when 9 => SALIDA <= "0000100";
when others => SALIDA <= "1111111";
```

end case; end process;

5 CONJUNTOS DE FUNCIONES BOOLEANAS ESTRUCTURAS MATRICIALES Y BLOQUES PROGRAMABLES

- 5.1. Formas diversas de construir una función booleana
- 5.2. Conjuntos de funciones booleanas: estructuras PAL y ROM
- 5.3. Simplificación multifunción: estructura PLA
- 5.4. Bloques integrados programables

El presente tema trata de dar respuesta a la pregunta: ¿Cómo construir n funciones booleanas de m variables cuando tal número de variables de entrada es un poco alto? O lo que es lo mismo: ¿Cómo manejar conjuntos de varias funciones de muchas variables? En tales casos, el alto número de entradas supone una dificultad importante a la hora de obtener las expresiones algebraicas de las funciones y, también, a la hora de transformar dichas expresiones en circuitos con puertas lógicas.

Lo grande, lo complejo requiere, para ser manejable, de un plus de organización, de una «estructura» que oriente y facilite su manejo. «Estructura = distribución, orden y enlace de las partes para formar el todo.» A lo largo de este tema se muestra cómo la estructura matricial (que se representa mediante una configuración reticular) permite abordar con relativa facilidad conjuntos de funciones de muchas variables.

Además, tal tipo de organización ha permitido construir bloques integrados «programables», en cuyo interior es posible configurar las funciones booleanas por medio de la eliminación de conexiones (fusibles). De manera que disponemos de circuitos integrados programables que constituyen, actualmente, la forma eficaz y habitual de construir los prototipos de sistemas digitales o de fabricar series de pocas unidades.

*Se utilizan tres tipos de estructura matricial (**ROM**, **PAL** y **PLA**) que son útiles en cuanto a modelos o esquemas conceptuales para la realización de conjuntos de funciones booleanas y, también, como configuraciones físicas concretas de los circuitos integrados programables.*

*Los codificadores **ROM** (ya estudiados en el capítulo anterior) expresan las funciones booleanas en su forma canónica, como suma de términos mínimos, diferenciando el decodificador (los términos mínimos, comunes a las diversas funciones) del encodificador (la suma de los mismos, que identifica a cada una de ellas).*

*La estructura **PAL** se corresponde con la forma habitual de expresar las funciones booleanas como suma de productos, una vez simplificada su expresión algebraica; cada función se construye por separado, con sus propios términos producto.*

*Por último, la configuración **PLA** es también del tipo suma de productos, pero los términos producto son compartidos por las diversas funciones, lo cual permite minimizar el número de términos producto necesarios (a través de una simplificación multifunción).*

5.1. Formas diversas de construir una función booleana

Las funciones booleanas son multiformes; pueden representarse y expresarse en formas diversas:

enunciado $\rightarrow$ tabla funcional $\rightarrow$ forma canónica $\rightarrow$ expresión algebraica simplificada

De igual manera, una función booleana puede construirse en formas diversas:

- con un multiplexor cuyas entradas reciben los valores de la tabla funcional: estructura LUT
- con un decodificador, recogiendo sobre una puerta "o" los términos mínimos que corresponden a vectores de entrada que dan valor **1** en la tabla funcional: forma canónica ($\sum m$)
- con puertas básicas, una vez simplificada la expresión algebraica de la función: suma de productos ($\sum p$)
- con puertas unitarias (en concreto, con puertas *Nand* o con puertas *Nor*), utilizando un solo tipo de puertas
- ...

Veamos estas cuatro formas diferentes de construir una función booleana, aplicándolas a una función concreta de 4 variables:

Enunciado: De entre los números binarios de 4 dígitos **dcba** (del 0 al 15 decimales) se excluyen los números 3, 5, 10, 11, 12 y 14, y con el resto se forma el subconjunto **R**; la función "y" indicará la pertenencia ($y = 1$) de un número binario de 4 dígitos al subconjunto **R**.

A continuación, se representa esta función en las cuatro configuraciones citadas:

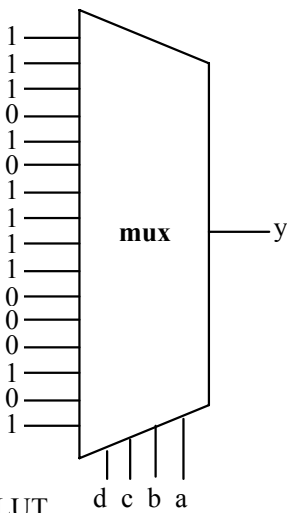
- a) Un multiplexor de 4 líneas de control cuyas entradas reciben los valores booleanos de la «tabla de verdad» (o tabla funcional) de la función.

Esta manera de configurar funciones booleanas, con multiplexores que «muestran» la tabla de la función, se denomina **look-up-table** (LUT): construcción tabular (*mirar sobre su tabla*). [Ver figura en la página siguiente.]

Si bien esta forma de construir funciones no corresponde a las estructuras matriciales consideradas en el presente tema, es utilizada en algunos tipos de circuitos integrados programables (FPGAs), ya que permite la programación de funciones booleanas sin más que almacenar en un registro su tabla funcional.

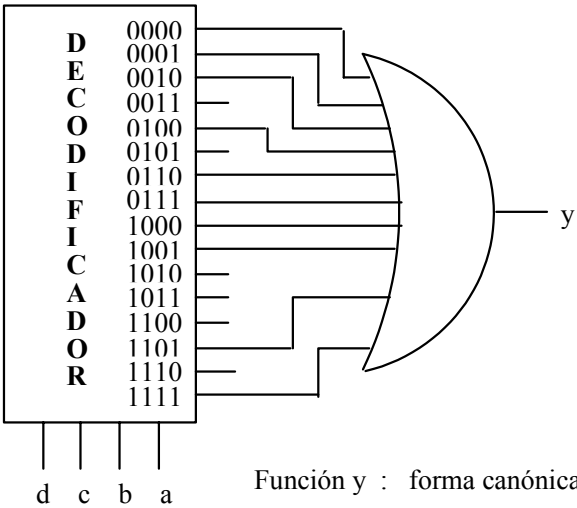
Tabla funcional

d	c	b	a	y
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	0
1	1	0	1	1
1	1	1	0	0
1	1	1	1	1



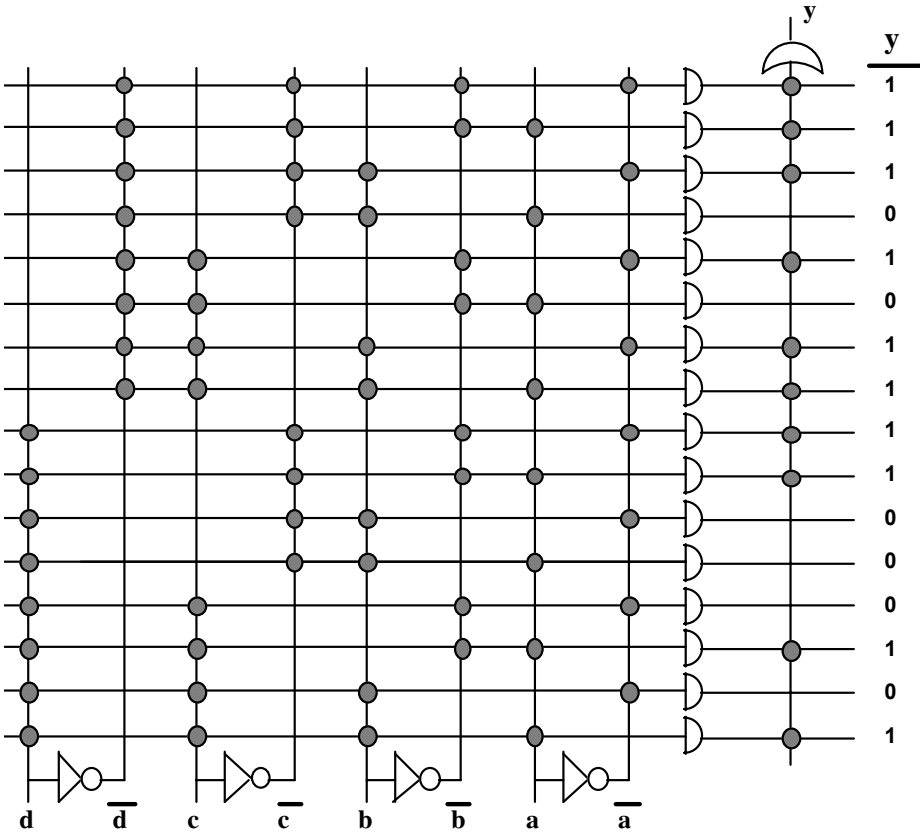
Función y : configuración LUT

b) Un decodificador de 4 líneas de control seguido de una puerta "o" que recibe aquellas salidas que corresponden a valor 1 en la «tabla de verdad» de la función; no es sino la construcción directa de la forma canónica de la función y equivale a la configuración ROM introducida en el capítulo anterior.



Función y : forma canónica

La siguiente figura muestra la representación reticular de esta configuración: un decodificador genérico de 4 variables, más una puerta "o" que recoge los términos mínimos con valor 1 en la tabla funcional. De forma que la línea de conexiones de dicha puerta "o" coincide con la columna de valores de salida de la tabla funcional (entendiendo el valor 1 como conexión y el 0 como ausencia de ella).



Las dos formas anteriores de construir la función (LUT y forma canónica) no precisan de la expresión algebraica de la función booleana y menos aún de su simplificación; se obtienen directamente de su tabla funcional.

En cambio, las siguientes formas de construir la función "y" corresponden a expresiones algebraicas de la misma; pasemos a obtenerlas, en forma simplificada, empleando su mapa de Karnaugh.

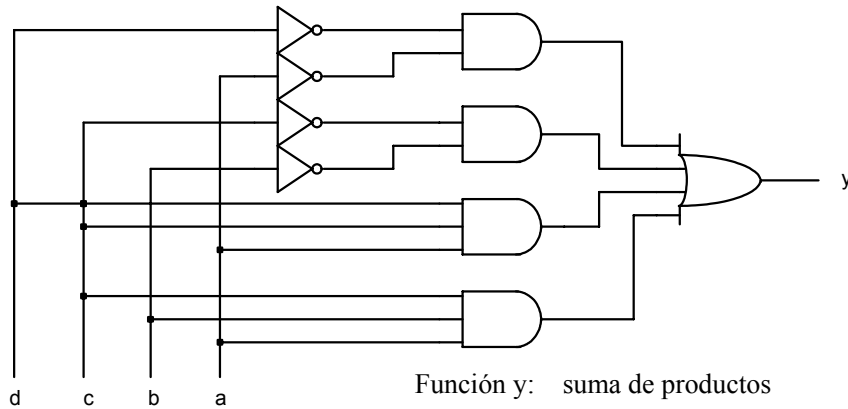
	<u>B A</u>				
<u>D C</u>		0 0	0 1	1 1	1 0
	0 0	1	1	0	1
	0 1	1	0	1	1
	1 1	0	1	1	0
	1 0	1	1	0	0

$$y = \bar{d}.\bar{a} + \bar{c}.\bar{b} + d.c.a + c.b.a \rightarrow \Sigma p$$

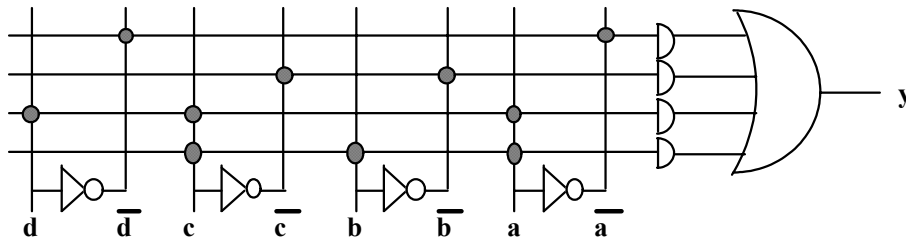
$$= \bar{d}.\bar{a} + \bar{c}.\bar{b} + c.a.(d+b)$$

$$= \text{Nand}[\bar{d} * \bar{a}, \bar{b} * \bar{c}, \text{Nand}(c, a, \bar{d} * \bar{b})] \rightarrow \text{Nand}$$

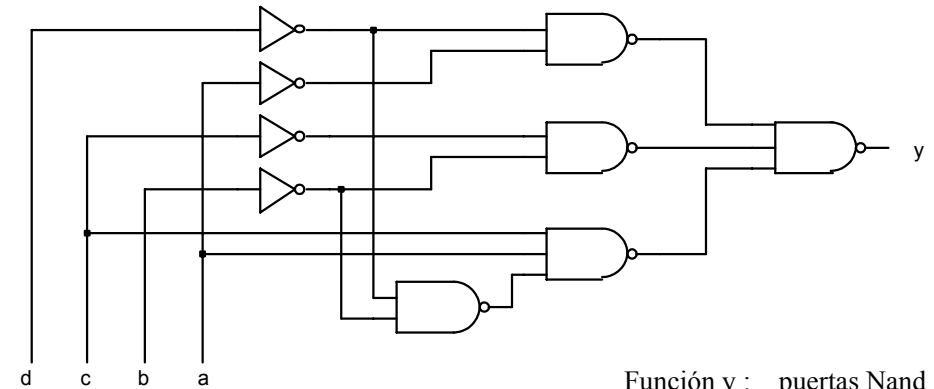
c) La conexión de puertas básicas (inversores, puertas "o" y puertas "y") que corresponde a la expresión algebraica de la función como suma de productos, una vez simplificada (Σp). Es la forma habitual de expresar una función y dará lugar a la estructura matricial que denominaremos **PAL**.



Podemos representar la misma expresión algebraica en configuración reticular en la forma siguiente:



d) La expresión de la función trasladada a puertas "y-negada" (Nand) para utilizar un solo tipo de puertas. Es la forma típica de construir una función con circuitos integrados estándar: el uso de un solo tipo de puertas para minimizar el número de circuitos integrados necesarios.



La utilización de circuitos integrados estándar, aun cuando puede resultar útil en laboratorio (sobre todo para funciones no muy complejas) está siendo sustituida en la práctica por el empleo de circuitos integrados programables, los cuales permiten insertar en el interior de un solo circuito el conjunto completo de funciones booleanas necesarias, evitando el cableado entre las diversas puertas lógicas.

Respecto al presente tema nos interesan, en particular, las formas **b)** y **c)** de construir una función, es decir su forma canónica a partir de un decodificador (suma de términos mínimos Σm) y su forma algebraica como suma de productos Σp , que dan lugar, respectivamente, a las configuraciones **ROM** y **PAL**. Consideraremos también un tercer tipo de configuración, **PLA** (suma de productos compartidos) que solamente tiene sentido cuando tratamos con conjuntos de funciones y no para funciones individuales.

Las siglas utilizadas para estas configuraciones proceden de denominaciones que aportan poco (e incluso son confusas) en relación con la estructura a la que dan nombre: **ROM** (read only memory –memorias de sólo lectura–), **PAL** (programmable and logic –lógica de puertas "y" programables–) y **PLA** (programmable logic array –matriz lógica programable–).

Por ello, a fin de hacer una referencia más directa a la propia estructura de los bloques, propongo una lectura diferente de dichas siglas: **ROM** (row ordered minterms –filas de términos mínimos–), **PAL** (product adding layers –estratos o módulos sumadores de productos–) y **PLA** (product linking adders –sumadores que enlazan o comparten productos–).

5.2. Conjuntos de funciones booleanas: estructuras PAL y ROM

Una advertencia previa: para facilitar la comprensión de los conceptos y, también, por razones obvias de facilidad de realización y de comprensión de los dibujos, en el desarrollo de este capítulo se utilizan como ejemplos funciones de reducido número de variables; ahora bien, debe tenerse en cuenta que las estructuras que estamos estudiando son realmente interesantes y útiles para funciones de amplio número de entradas.

La expresión algebraica más habitual de una función booleana, una vez simplificada, es la forma de suma de términos producto $\sum p$. El circuito correspondiente a tal expresión es un módulo conformado por una puerta "o", que proporciona la salida de la función, conectada a varias puertas "y", que realizan los diversos términos producto. Las conexiones de las entradas (y de sus negadas) sobre las puertas "y" pueden ser representadas gráficamente en forma reticular o matricial.

Veámoslo con el siguiente ejemplo referido a un conjunto de cuatro funciones de cuatro entradas:

Tabla funcional

d c b a	Y1	Y2	Y3	Y4
0 0 0 0	0	0	1	0
0 0 0 1	1	1	1	1
0 0 1 0	0	0	0	0
0 0 1 1	1	1	0	1
0 1 0 0	1	0	1	1
0 1 0 1	1	0	1	1
0 1 1 0	1	1	0	0
0 1 1 1	1	1	0	0
1 0 0 0	0	1	1	1
1 0 0 1	0	1	1	1
1 0 1 0	0	0	0	0
1 0 1 1	1	0	1	1
1 1 0 0	1	0	0	1
1 1 0 1	1	0	0	1
1 1 1 0	1	1	0	0
1 1 1 1	1	1	1	1

Mapas de Karnaugh

Y1

ba \ dc	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	1	1	1	1
10	0	0	1	0

Y2

ba \ dc	00	01	11	10
00	0	1	1	0
01	0	0	1	1
11	0	0	1	1
10	1	1	0	0

Y3

ba \ dc	00	01	11	10
00	1	1	0	0
01	1	1	0	0
11	0	0	1	0
10	1	1	1	0

Y4

ba \ dc	00	01	11	10
00	0	1	1	0
01	1	1	0	0
11	1	1	1	0
10	1	1	1	0

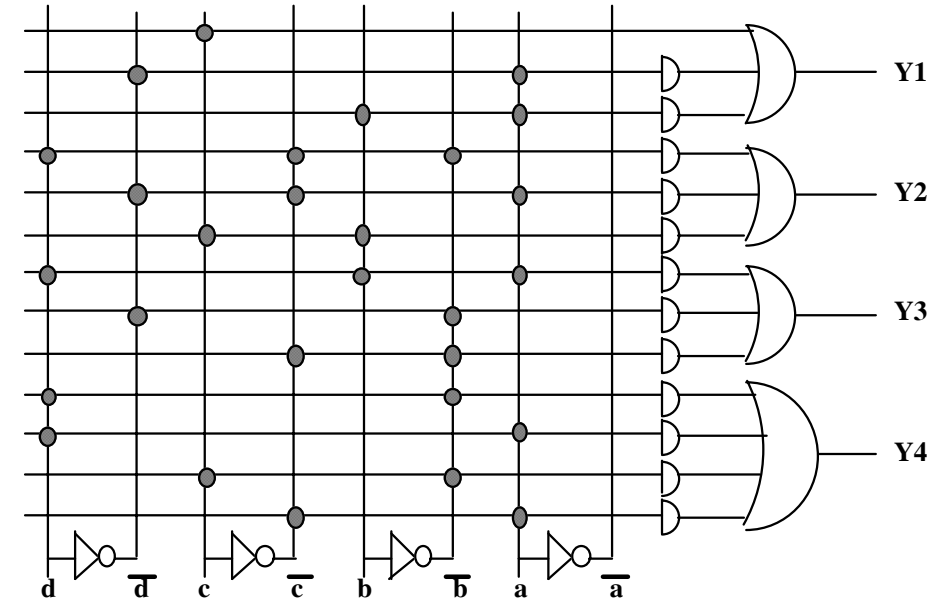
$$Y1 = c + \bar{d}.a + b.a$$

$$Y2 = d.\bar{c}.\bar{b} + \bar{d}.\bar{c}.a + c.b$$

$$Y3 = d.b.a + \bar{d}.\bar{b} + \bar{c}.\bar{b}$$

$$Y4 = d.\bar{b} + d.a + c.\bar{b} + \bar{c}.a$$

La siguiente figura representa, en forma reticular, las cuatro funciones booleanas en configuración suma de productos $\sum p$: cada función ocupa un módulo separado, constituido por una puerta "o" que recibe las salidas de varias puertas "y"; las conexiones de las variables se ajustan a la retícula formada por las entradas y sus negadas y las «líneas de entrada» de las puertas "y".



Esta forma reticular de dibujar la configuración $\sum p$ de un conjunto de funciones recibe el nombre de estructura **PAL** (*product adding layers*: estratos o módulos sumadores de productos); en ella, cada función dispone de sus propios términos producto y las diversas funciones no comparten ninguna puerta booleana, salvo los inversores de las variables de entrada.

Es claro que la configuración **PAL** no es sino una forma de organizar el dibujo de las funciones y no añade nada a su expresión en suma de productos. Pero, si las funciones booleanas son complejas, esta forma facilita el dibujo y la comprensión del mismo; además, esta estructura da lugar a un tipo de bloques integrados programables que permiten construir directamente las funciones booleanas en su interior, mediante la programación de sus conexiones.

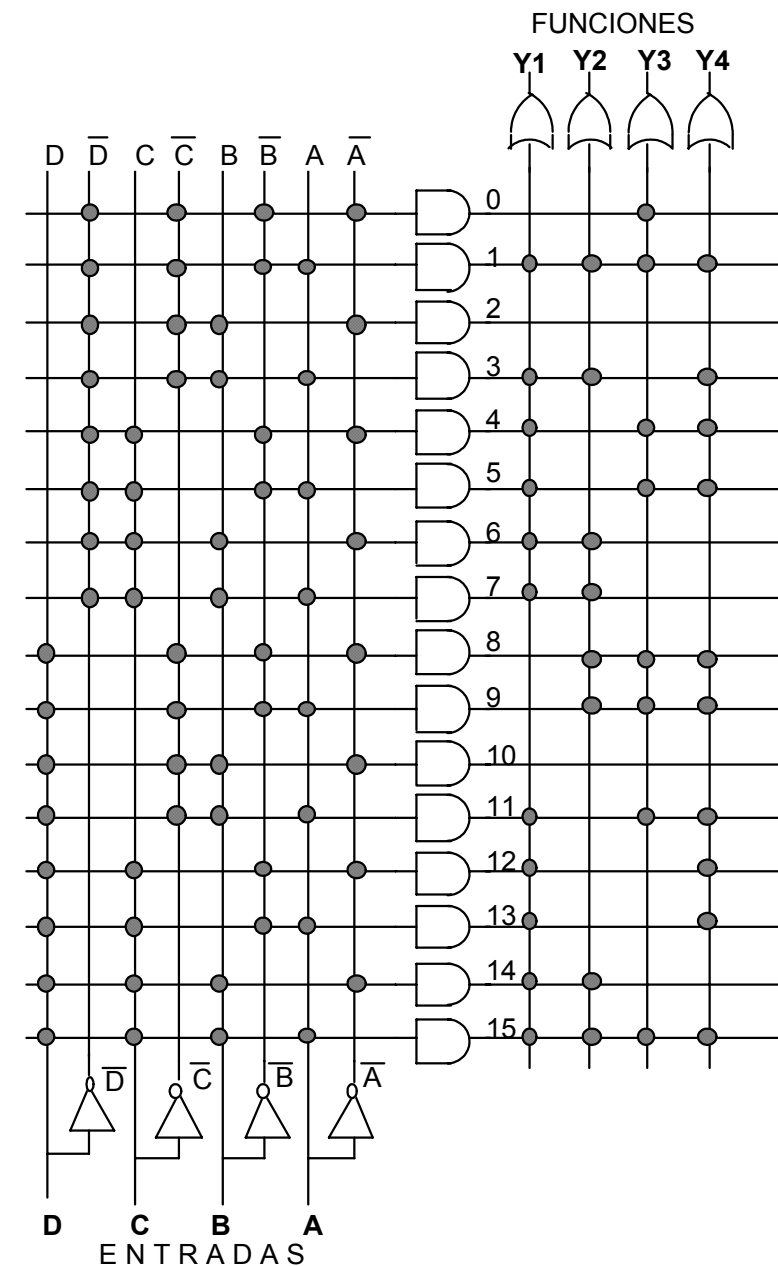
La retícula de conexiones sobre las puertas "y" permite describir tales conexiones en forma de matriz de «ceros» y «unos»; es decir, la configuración reticular equivale a una descripción matricial de las funciones, a la cual podemos denominar *Matriz Y de términos producto*:

Matriz de conexiones:	d	$\bar{d}$	c	$\bar{c}$	b	$\bar{b}$	a	$\bar{a}$
- función Y1:	0	0	1	0	0	0	0	0
	0	1	0	0	0	0	1	0
	0	0	0	0	1	0	1	0
- función Y2:	1	0	0	1	0	1	0	0
	0	1	0	1	0	0	1	0
	0	0	1	0	1	0	0	0
- función Y3:	1	0	0	0	1	0	1	0
	0	1	0	0	0	1	0	0
	0	0	0	1	0	1	0	0
- función Y4:	1	0	0	0	0	1	0	0
	1	0	0	0	0	0	1	0
	0	0	1	0	0	1	0	0
	0	0	0	1	0	0	1	0

La configuración **ROM** (*row ordered minterms*: filas de términos mínimos) constituye otra forma de construir funciones, utilizando un decodificador seguido del encodificador que corresponde a la tabla de las funciones:

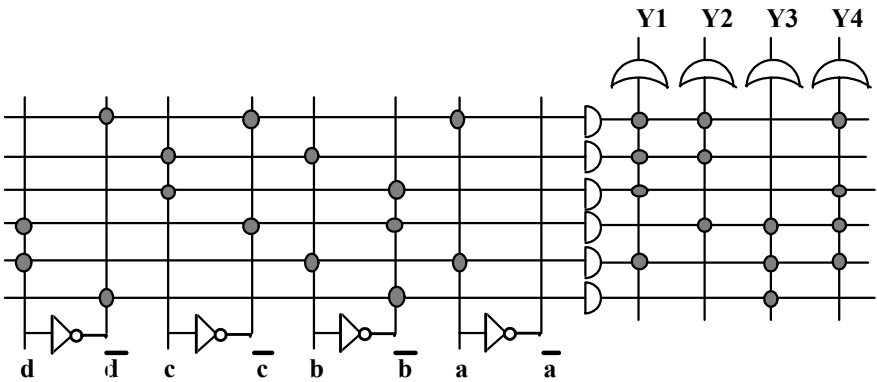
- las **m** entradas del decodificador actúan como variables de entrada para todas las funciones,
- las **2^m** líneas intermedias corresponden a los términos mínimos de las variables de entrada,
- y cada salida realizará una de las funciones booleanas, mediante una puerta "o", haciendo confluír sobre ella los términos mínimos que intervienen en dicha función (forma canónica de la misma).

La figura de la página siguiente representa la configuración **ROM** de las cuatro funciones booleanas que estamos considerando como ejemplo: el decodificador construye físicamente los 16 términos mínimos de las 4 variables de entrada y el encodificador efectúa, para cada una de las funciones, la suma de los términos mínimos que la activan (que producen valor **1** para dicha función).



$$\begin{aligned} Y1 &= \bar{d}.\bar{c}.a + c.\bar{b} + c.b + d.b.a \\ Y2 &= \bar{d}.\bar{c}.a + c.b + d.c.\bar{b} \\ Y3 &= d.b.a + d.c.\bar{b} + \bar{d}.\bar{b} \\ Y4 &= \bar{d}.\bar{c}.a + c.\bar{b} + d.b.a + d.c.\bar{b} \end{aligned}$$

La representación gráfica de estas expresiones da lugar a la siguiente configuración reticular:



La figura anterior representa a las cuatro funciones booleanas compartiendo términos producto como resultado de una simplificación multifunción Σpc (suma de productos compartidos); tal simplificación permite reducir en gran medida el número de términos producto necesarios para el conjunto de funciones, aunque la expresión resultante para cada función no es la más simplificada posible.

Esta forma de construir un conjunto de funciones recibe el nombre de estructura **PLA** (*product linking adders*: sumadores que enlazan o comparten productos); las conexiones conforman dos matrices: la *Matriz Y de términos producto compartidos* y la *Matriz O de salidas*. [Una vez más conviene hacer notar que estos tipos de organización reticular o matricial son particularmente útiles para funciones de un amplio número de entradas.]

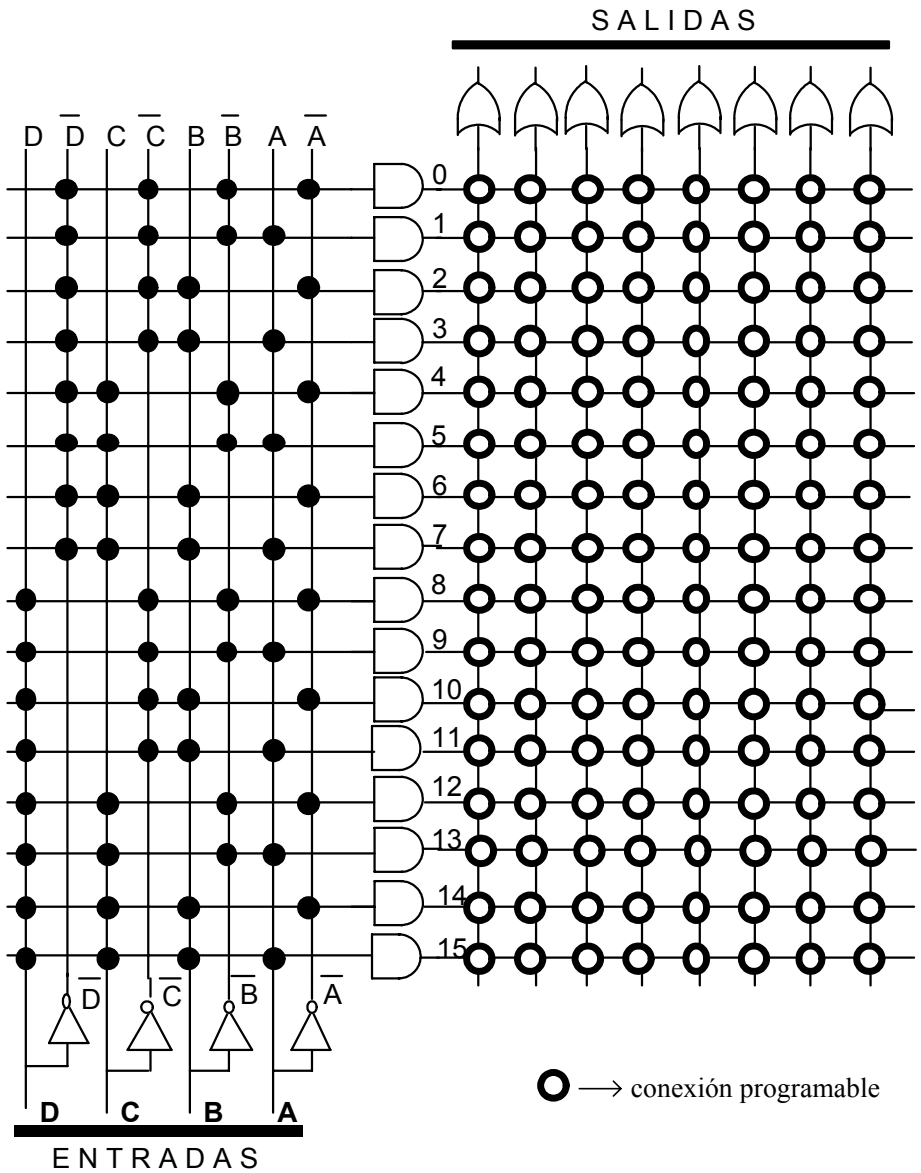
La configuración **PLA** supone un claro ahorro circuital por cuanto que minimiza el conjunto de términos producto necesarios, pero, en cambio, requiere un complejo proceso previo de simplificación multifunción.

5.4. Bloques integrados programables

Consideremos un bloque **ROM** de m entradas y n salidas que, por tanto, contiene n funciones booleanas de m entradas; particularizar dicho bloque a n funciones booleanas dadas consiste en determinar en cuáles de los puntos de la *Matriz O funcional* debe haber conexión y en cuáles no.

Supongamos que todos los puntos de la Matriz O de conexiones sobre las puertas "o" de salida se encuentran conectados (por ejemplo, mediante fusibles) y que el usuario pueda eliminar aquellas conexiones que no interesen (fundiendo los fusibles); de esta forma dispondremos de un bloque **ROM programable** por el usuario: **PROM** [véase la figura de la página siguiente].

La siguiente figura representa un bloque **PROM** de 4 entradas y 8 salidas, apto para la programación de 8 funciones de 4 variables.



Codificador PROM de 4 líneas de entrada y 8 líneas de salida

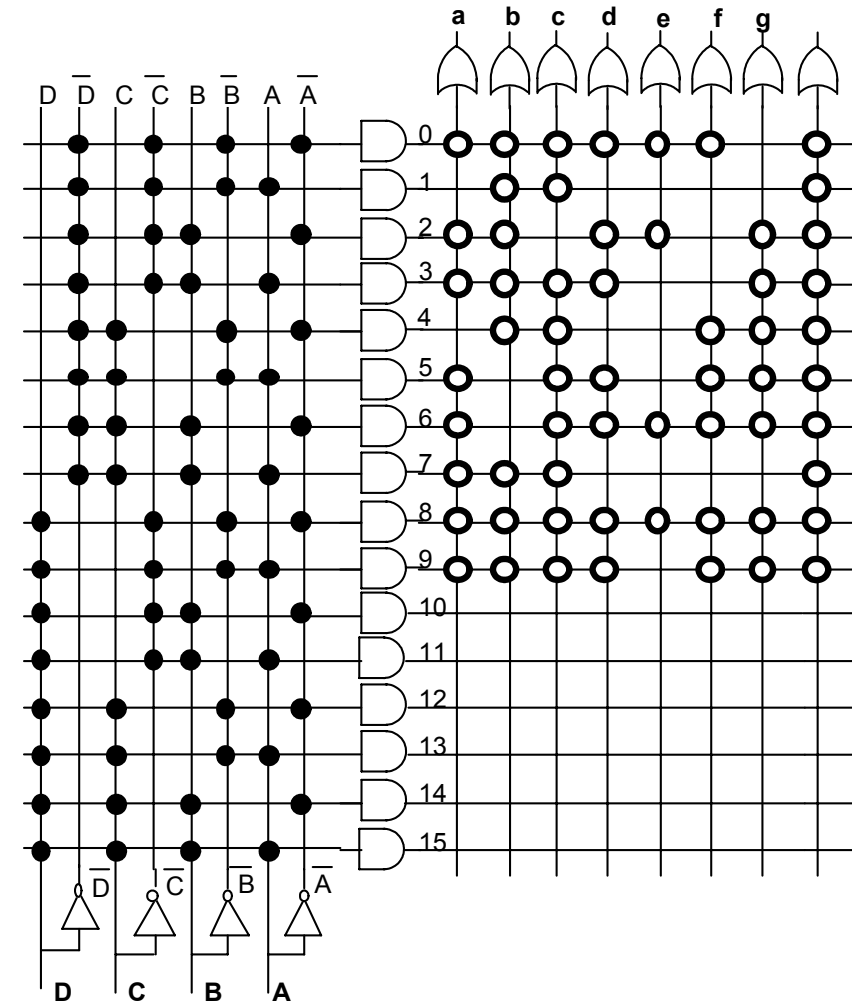
La tecnología electrónica de integración de circuitos ha permitido construir este tipo de bloques **PROM**, programables eléctricamente mediante procesos análogos a la ruptura de fusibles. Su programación consiste en eliminar aquellas conexiones de la Matriz O que no deben estar presentes conforme a la tabla funcional del conjunto de funciones booleanas deseado, es decir, eliminar (*fundir los fusibles de*) aquellas conexiones que corresponden a valor **0** en la tabla funcional.

La programación de un bloque **PROM**, a partir de la tabla de las **n** funciones booleanas equivale a reflejar sobre la Matriz O la tabla de dichas funciones:

- Cada una de las líneas intermedias del bloque **PROM** (salidas del decodificador que conforman las filas de la Matriz O) se corresponde con un término mínimo, al igual que cada fila de la tabla funcional, y cada una de las columnas de la Matriz O se corresponde con una de las funciones, igual que cada columna de dicha tabla.
- Existe una correspondencia directa entre la matriz de valores **0** y **1** que configura la parte de la derecha de la tabla funcional (vectores de salida) y la matriz de conexiones sobre las puertas "o" de salida, Matriz O, asociando el valor **0** a la eliminación o ausencia de conexión y el valor **1** a la conservación o presencia de la misma.
- Para cada columna de la Matriz O los valores **1** presentes en la correspondiente columna de la tabla funcional determinan los términos mínimos que deben estar presentes en la función booleana que tal columna configura y, por tanto, las conexiones que deben incidir sobre la puerta "o" que realiza dicha función.

La figura de la página siguiente corresponde a la programación de las 7 funciones de un conversor BCD $\rightarrow$ 7 segmentos [coincide con la figura de la página 103 del tema anterior, apartado 4.4.]. La tabla de dichas funciones es la siguiente:

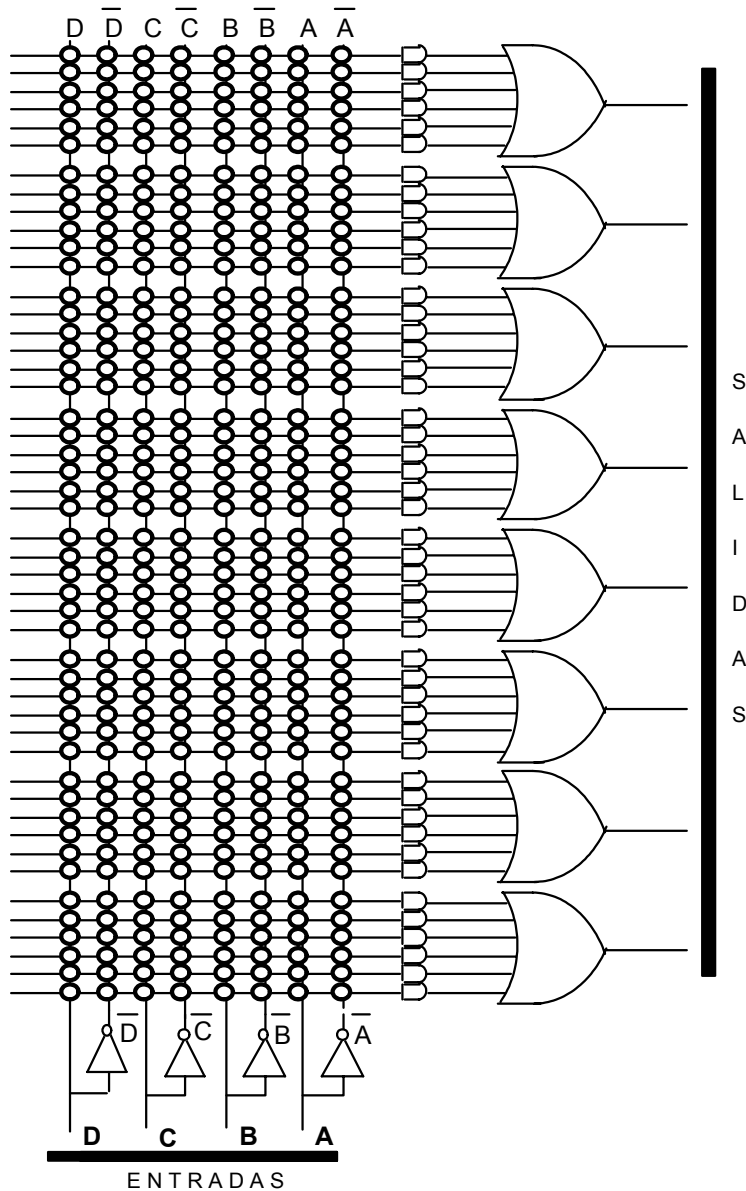
D	C	B	A	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1



Programación del conversor BCD $\rightarrow$ 7 segmentos en la PROM anterior

En los bloques programables tipo **PAL** (véase la figura de la página siguiente) cada puerta "o" de salida está conectada a un subconjunto independiente de puertas "y", siendo programables las conexiones de las entradas sobre dichas puertas "y"; de forma que cada función de salida posee su propia puerta "o" y su propio subconjunto de puertas "y" que inciden sobre ella.

La siguiente figura representa un bloque **PAL** de 4 entradas y 8 salidas, con 6 términos producto en cada módulo.



PAL programable de 4 líneas de entrada y 8 de salida con 6 términos producto

De forma que un bloque **PAL** es un conjunto de n módulos independientes, cada uno de los cuales está constituido por la suma de q términos producto de las m variables de entrada: cada una de las n funciones se programa con independencia de las demás, pues no poseen términos producto compartidos.

La estructura de los bloques **PAL** coincide con las funciones expresadas como suma de términos producto $\sum p$, sin necesidad de preocuparse por las coincidencias de términos entre las diversas funciones. Por ello su programación es muy sencilla y se deduce directamente de la expresión algebraica simplificada de cada una de las funciones a programar.

Ahora bien, un bloque programable **PAL** se encuentra limitado por el número de términos producto disponibles en cada módulo. No es posible programar cualquier función booleana de m variables: no podrán programarse aquellas funciones cuya expresión algebraica, una vez simplificada al máximo, incluya un número de términos producto mayor.

La programación del conversor BCD $\rightarrow$ 7 segmentos sobre un bloque **PAL** consiste en configurar directamente cada una de sus siete funciones, una vez simplificadas:

$$a = D + B + C.A + \bar{C}.\bar{A}$$

$$b = \bar{C} + B.A + \bar{B}.\bar{A}$$

$$c = C + \bar{B} + A$$

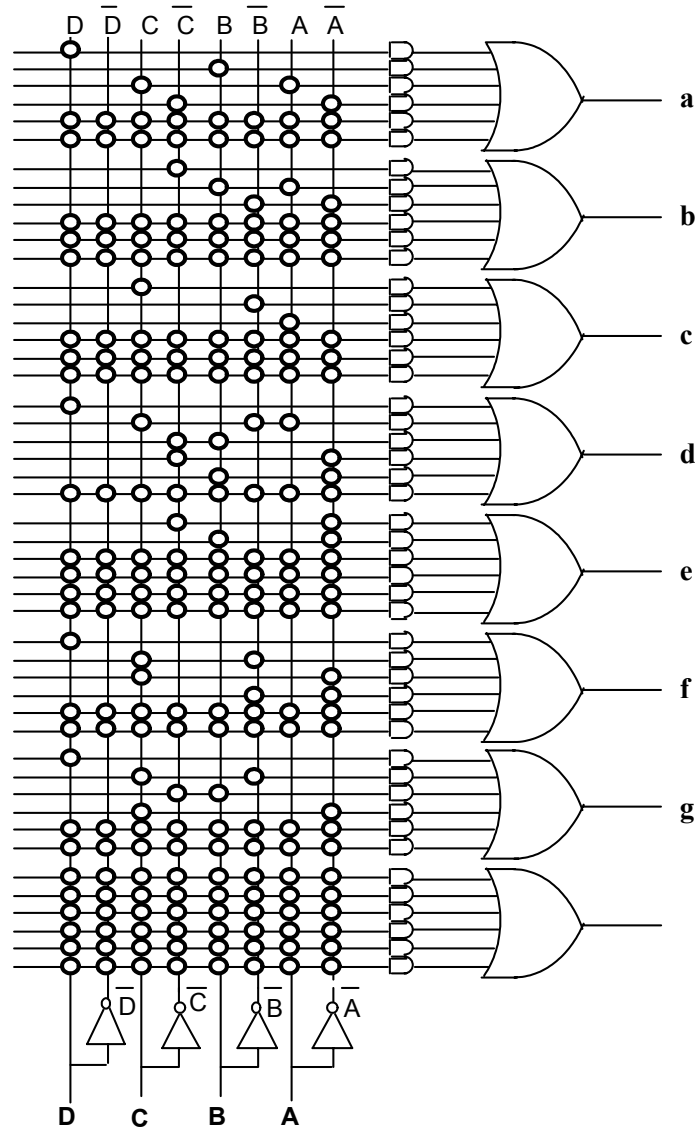
$$d = D + C.\bar{B}.A + \bar{C}.B + \bar{C}.\bar{A} + B.\bar{A}$$

$$e = \bar{C}.\bar{A} + B.\bar{A}$$

$$f = D + C.\bar{B} + C.\bar{A} + \bar{B}.\bar{A}$$

$$g = D + C.\bar{B} + \bar{C}.B + C.\bar{A}$$

Las funciones **a**, **f** y **g** requieren 4 términos producto, la **b** y la **c** 3 términos, mientras que la función **d** requiere 5 términos y la función **e** solamente 2 términos producto.

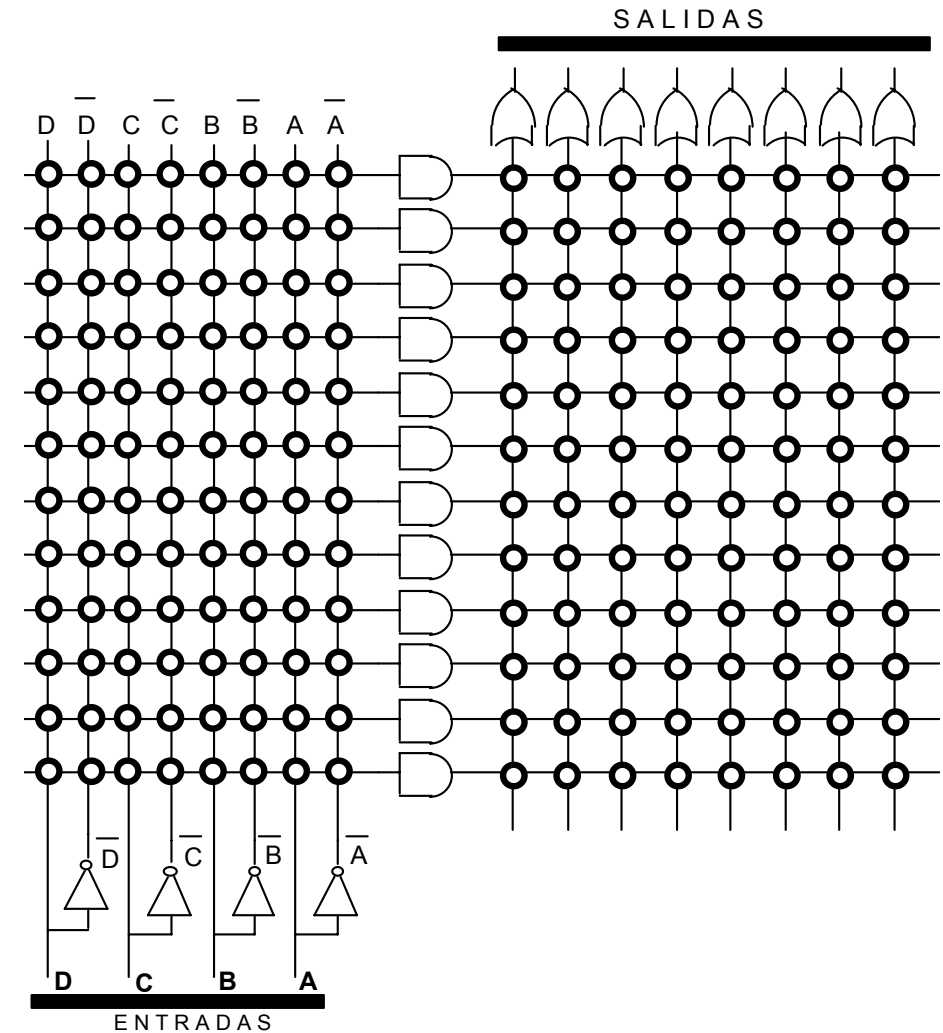


Conversor BCD $\rightarrow$ 7 segmentos programado en la PAL anterior

Obsérvese que en esta figura no se han programado (*no se han eliminado las conexiones de*) aquellos términos producto que no han sido necesarios; un término producto sin programar se anula por sí mismo, ya que contiene el producto de variables por sus negadas ($a \cdot \bar{a} = 0$).

La configuración **PLA** requiere que ambas matrices sean programables, tanto la Matriz Y de entradas como la Matriz O de salidas: sobre la Matriz Y se programarán los términos producto que son necesarios para realizar las n funciones y sobre la Matriz O se programará la inclusión o no de cada término producto en cada función.

La siguiente figura presenta una **PLA** de 4 entradas, 8 salidas y 12 términos producto.



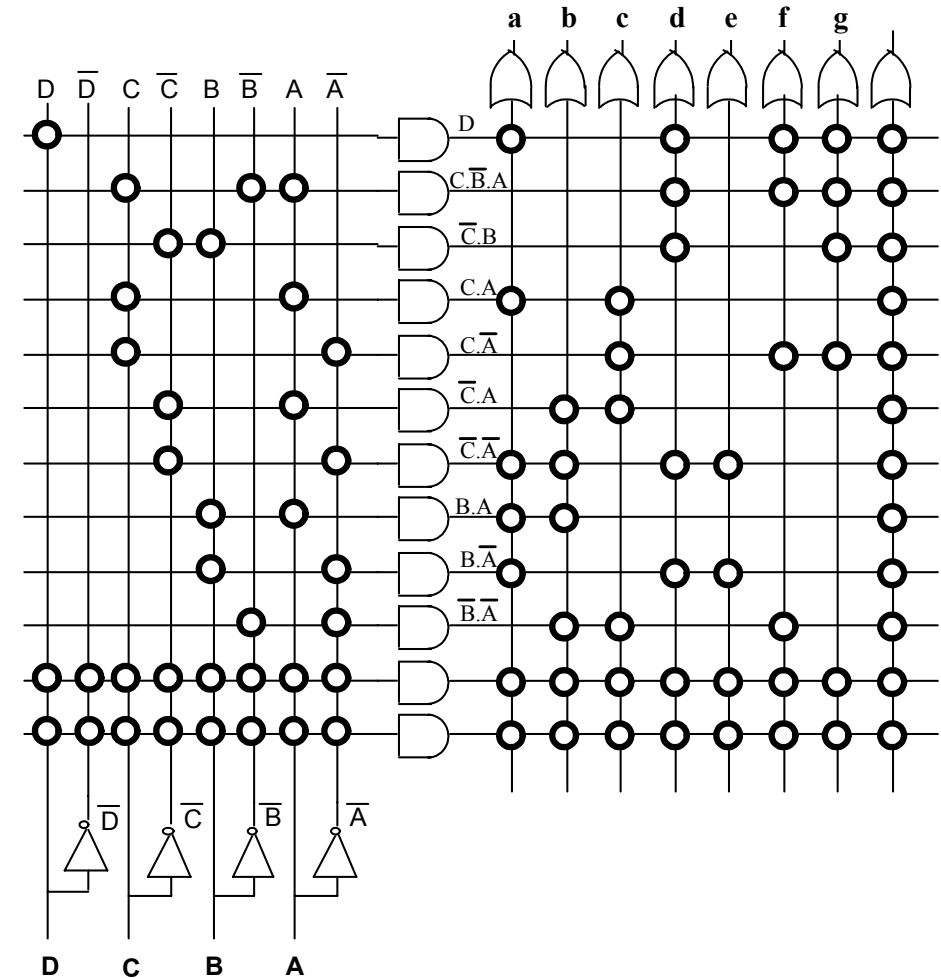
PLA de 4 líneas de entrada, 4 líneas de salida y 12 términos producto.

La programación de un bloque **PLA** resulta relativamente compleja por cuanto que es preciso simplificar conjuntamente las **n** funciones a programar, a fin de minimizar el número de términos producto necesarios.

No basta la simplificación individual de las **n** funciones; al contrario, la minimización de cada una de ellas por separado impedirá, en la mayoría de los casos, encontrar términos producto coincidentes en varias que permitan minimizar el conjunto: el objetivo no es buscar el menor número de términos producto para cada una de las funciones, sino para el conjunto global de ellas.

La programación del conversor BCD $\rightarrow$ 7 segmentos corresponde a las siete funciones siguientes, obtenidas mediante simplificación multifunción:

$$\begin{aligned}
 a &= D + C.A + \bar{C}.\bar{A} + B.A + B.\bar{A} \\
 b &= \bar{C}.A + \bar{C}.\bar{A} + B.A + \bar{B}.\bar{A} \\
 c &= C.A + C.\bar{A} + \bar{C}.A + \bar{B}.\bar{A} \\
 d &= D + C.\bar{B}.A + \bar{C}.B + \bar{C}.\bar{A} + B.\bar{A} \\
 e &= \bar{C}.\bar{A} + B.\bar{A} \\
 f &= D + C.\bar{B}.A + C.\bar{A} + \bar{B}.\bar{A} \\
 g &= D + C.\bar{B}.A + \bar{C}.B + C.\bar{A}
 \end{aligned}$$



Conversor BCD $\rightarrow$ 7 segmentos programado en la PLA anterior

Al igual que en el caso de la **PAL**, no se han programado (*no se han eliminado las conexiones de*) aquellos términos producto que no han sido necesarios, ya que se anulan por sí mismos (producto de variables por sus negadas, $a . \bar{a} = 0$).

En resumen, se dispone de tres bloques de lógica programable que permiten realizar **n** funciones de **m** entradas, expresadas éstas como suma de términos producto (en el caso del bloque **PROM** como suma de términos mínimos). Los bloques **PROM** son programables en cuanto a su Matriz O de salidas, los bloques **PAL** lo son en cuanto a su Matriz Y de entradas y los bloques **PLA** son programables respecto a sus dos matrices Y, O.

Los tres bloques se encuentran configurados por una *Matriz Y* de conexiones de las entradas y de sus negadas sobre puertas "y" y una *Matriz O* de conexiones de las salidas de las puertas "y" sobre las puertas "o" que conforman las salidas del bloque; difieren en cuanto a la capacidad de programación de dichas matrices, conforme a la siguiente tabla:

	<u>Matriz Y</u>	<u>Matriz O</u>	<u>Nº de términos producto</u>
PROM	fija	programable	$2^m = \text{nº términos mínimos}$
PLA	programable	programable	$p \ll 2^m$
PAL	programable	fija	$n \times q \quad (q < p)$

Un bloque **PROM** admite la programación de **n** funciones cualesquiera de **m** entradas; los bloques **PLA** se encuentran limitados por el número total de términos producto ($p \ll 2^m$) disponibles en la Matriz Y, mientras que los bloques **PAL** están limitados en el número de términos producto ($q < p$) de cada módulo.

La programación **PROM** consiste en reflejar sobre la Matriz O la tabla de las funciones (de forma que se eliminen aquellas conexiones que corresponden a valor **0** en dicha tabla). La programación de cada módulo **PAL** corresponde a la expresión algebraica simplificada de la función en forma de suma de términos producto (con tal de que el número de términos sea igual o inferior a los disponibles en el módulo **PAL**). La programación **PLA** supone configurar en su Matriz Y los términos producto resultantes de la simplificación multifunción de las funciones a programar y, posteriormente, configurar sobre la Matriz O las sumas de aquellos términos que dan lugar a cada una de las funciones.

Tanto en el caso PAL como PLA no es necesario programar (*eliminar*) los términos producto no utilizados ya que se anulan ellos solos, por contener productos de una variable por su negada ($a \cdot a = 0$)

El número de conexiones de las matrices puede servir para comparar las tres estructuras **PROM**, **PLA**, **PAL**, en cuanto a sus dimensiones físicas (en su realización electrónica el número de conexiones equivale aproximadamente al número de transistores, ya que cada conexión se realiza a través de un transistor):

	<u>Matriz Y</u>	<u>Matriz O</u>	
PROM	$m \times 2^m$	$n \times 2^m$ programables	
PLA	$2^m \times p$ programables	$n \times p$ programables	$p \ll 2^m$
PAL	$2^m \times q \times n$ programables	$n \times q$	$q < p$

Por ejemplo, comparando una **PROM** de 12 entradas y 8 salidas, una **PLA** del mismo número de entradas y salidas con 28 términos producto y una **PAL** análoga con 6 términos para cada salida:

	<u>Matriz Y</u>	<u>Matriz O</u>
PROM	12×4.096	8×4.096 programables
PLA	24×28 programables	8×28 programables
PAL	$24 \times 6 \times 8$ programables	8×6

resulta que la configuración **PROM** requiere 81.920 conexiones, de las cuales 32.768 son programables, mientras que la **PLA** necesita 896 conexiones (la centésima parte de las que precisa la **PROM**), todas ellas programables, y la **PAL** utiliza 1.200 conexiones, de las cuales 1.152 son programables.

En la anterior **PROM** pueden programarse cualquier conjunto de 8 funciones de 12 entradas, mientras que en la **PAL** no caben funciones cuya expresión algebraica contenga más de 6 términos producto y la **PLA** puede admitir hasta 8 funciones de 12 entradas con tal de que su simplificación multifunción permita reducir el número total de términos producto necesarios a 28 o menos.

Forma de efectuar la programación

La programación de los circuitos integrados programables se realiza habitualmente mediante un programador conectado a un computador; se requieren dos aplicaciones informáticas: un compilador que traduce la descripción del diseño al «mapa de fusibles» del circuito integrado y un programa de control del programador que «ejecuta» el mapa de fusibles sobre el propio circuito integrado.

Se parte de la descripción del diseño específico a programar; tal descripción puede hacerse en forma gráfica (esquemático de puertas lógicas o de bloques), pero lo habitual es hacerla en forma de texto utilizando un lenguaje de descripción circuital (VHDL, Verilog, ABEL, ..., siendo el VHDL el más empleado).

Un compilador apropiado traslada la descripción a funciones booleanas y las «encaja» sobre la configuración del dispositivo programable, obteniendo el «mapa de fusibles» que indica cuales de las conexiones programables se han de eliminar y cuales deben permanecer.

El programador ejecuta el mapa de fusibles sobre el circuito integrado programables, actuando «una a una» sobre las conexiones que deben eliminarse y verificando, posteriormente, que tales conexiones han quedado suprimidas y el resto de ellas mantienen su unión eléctrica.

Ejemplo: conversor BCD $\rightarrow$ 7 segmentos, de cátodo común

a) descripción VHDL del circuito

```
entity CONVERSION is
port( SALIDA      : Out BIT_VECTOR(1 to 7);
      D,C,B,A     : In  BIT);
end CONVERSION;

architecture CATODOCOMUN of CONVERSION is
begin
with D & C & B & A select
  SALIDA <=  "1111110" when "0000",
             "0110000" when "0001",
             "1101101" when "0010",
             "1111001" when "0011",
             "0110011" when "0100",
             "1011011" when "0101",
             "1011111" when "0110",
             "1110000" when "0111",
             "1111111" when "1000",
             "1111011" when "1001",
             "0000000" when others;
end CATODOCOMUN;
```

b) funciones booleanas y mapa de fusibles resultantes de la compilación

A continuación se adjuntan las funciones relativas a las 7 salidas del conversor obtenidas por el compilador WARP sobre la descripción VHDL anterior; inmediatamente debajo de cada una de las funciones se expresa el mapa de fusibles correspondiente para un módulo PAL de 10 entradas y 6 términos producto.

Comentario: las funciones que siguen son diferentes de las indicadas para el mismo conversor en el capítulo anterior (apartado 4.3.1.) y reproducidas en este capítulo en la página 132; ello es debido a que en este ejemplo se asigna vector de salida 0000000 para números de entrada superiores a 9, mientras que en el capítulo 4 se les asignaba vector de salida XXXXXXXX para simplificar en lo posible las funciones resultantes.

salida_1 = $/c * /b * /a + /d * c * a + d * /c * /b + /d * b$

```
00010101000000000000
01100010000000000000
10010100000000000000
01001000000000000000
11111111111111111111
11111111111111111111
```

salida_2 = $d * b * a + /d * /b * /a + /c * /b + /d * /c$

```
10001010000000000000
01000101000000000000
00010100000000000000
01010000000000000000
11111111111111111111
11111111111111111111
```

salida_3 = $/c * /b + /d * a + /d * c$

```
00010100000000000000
01000010000000000000
01100000000000000000
11111111111111111111
11111111111111111111
11111111111111111111
```

salida_4 = $/d * c * /b * a + /c * /b * /a + /d * b * /a + d * /c * /b + /d * /c * b$

```
01100110000000000000
00010101000000000000
01001001000000000000
10010100000000000000
01011000000000000000
11111111111111111111
```

salida_5 = $/c * /b * /a + /d * b * /a$

```
00010101000000000000
01001001000000000000
11111111111111111111
11111111111111111111
11111111111111111111
11111111111111111111
```

salida_6 = $/c * /b * /a + d * /c * /b + /d * c * /a + /d * c * /b$

```
00010101000000000000
10010100000000000000
01100001000000000000
01100100000000000000
11111111111111111111
11111111111111111111
```

salida_7 = $/d * b * /a + d * /c * /b + /d * /c * b + /d * c * /b$

```
01001001000000000000
10010100000000000000
01011000000000000000
01100100000000000000
11111111111111111111
11111111111111111111
```

6 CODIFICACION EN PALABRAS BINARIAS DETECCIÓN DE ERROR

- 6.1. La información codificada en palabras binarias
- 6.2. La paridad para detectar error
- 6.3. Códigos detectores y correctores de error

Los sistemas digitales procesan la información codificada en palabras binarias, constituidas por conjuntos ordenados de ceros y unos. Esta información puede ser cuantitativa (es decir, una cantidad expresada por un número) o cualitativa (referida a la distinción entre diversas posibilidades o cualidades).

La longitud de palabra determina su capacidad para representar informaciones diferentes: una palabra de p dígitos es capaz de expresar 2^p posibilidades, las cuales estarán numeradas desde la 00...000 hasta la 11...111.

Ahora bien, el manejo de palabras binarias largas presenta una cierta complejidad (su lectura y escritura es dificultosa); tal complejidad puede reducirse «compactando» los dígitos de 4 en 4: sistema hexadecimal (base $16 = 2^4$).

La forma más inmediata de codificar la información en binario consiste en numerar en base 2 (código binario directo), pero existen otras posibilidades de interés. Una de ellas respeta la numeración decimal (base 10), que es la habitual para el hombre, y traslada a binario cada una de las cifras por separado: código BCD. También es útil el código Gray, en el cual dos palabras consecutivas difieren en un solo dígito; en ello se basa la simplificación mediante mapas de Karnaugh y la alta seguridad funcional que la codificación Gray aporta en las transiciones entre estados o en las secuencias de vectores.

Una cuestión importante respecto a la transferencia de información es la fiabilidad o ausencia de error en la misma. En la transmisión o almacenamiento de la información pueden producirse errores que afecten a uno o varios bits de algunas palabras binarias; interesa disponer de mecanismos que faciliten la detección de la existencia de errores y, si fuera posible, permitan corregirlos: tales mecanismos se basan en añadir dígitos adicionales (redundancia) para efectuar comprobaciones sobre la información recibida.

La paridad es el más sencillo de los detectores de error, basado en indicar si el número de «unos» de cada palabra es par o impar. La paridad es un código de distancia 2: la distancia (número de dígitos diferentes) entre dos palabras de este código es siempre par y, por ello, detecta errores en número impar.

El concepto de distancia mínima (número de bits en que difieren dos palabras de un código) es básico en el desarrollo de códigos detectores de error. Hamming introdujo un método simple para construir códigos de distancia mínima 4 (basados en la utilización de paridades parciales) que permiten detectar hasta tres errores y efectuar corrección cuando hay uno solo de ellos.

6.1. La información codificada en palabras binarias

Cada dígito de una palabra binaria recibe el nombre de bit (*bit = binary digit*) y puede tener dos valores: **0** y **1**. El número de dígitos o bits que forman una palabra binaria determina su longitud **p**.

La información contenida en una palabra digital puede ser de tipo cuantitativo (numérico) o de tipo cualitativo (distinción entre varias situaciones, posibilidades o cualidades). Por ejemplo, el resultado de la medida de una magnitud física con valor 185 conducirá a la palabra binaria **10111001**, mientras que el estado civil de una persona puede ser codificado con palabras de 2 bits (**00** soltera, **01** casada, **10** divorciada, **11** viuda) y los siete colores del arco iris necesitan para su codificación palabras binarias de 3 dígitos.

Los vectores de entrada y de salida de un sistema digital son palabras binarias que expresan, respectivamente, a través de la correspondiente codificación, la información que recibe el sistema y la información resultante del procesado que el sistema efectúa sobre ella. Muchas veces tales vectores se subdividen en varias palabras digitales con significado propio cada una de ellas.

Internamente los sistemas digitales generan condiciones o informaciones intermedias expresadas también en palabras binarias de uno o varios bits. En particular, los sistemas secuenciales incorporan memoria de su evolución anterior en forma de «estado del sistema», expresado en palabras binarias cuyos **n** bits corresponden a las variables de estado.

A la hora de codificar diversas informaciones, la posibilidad más inmediata consiste en numerarlas en sistema binario: *codificación binaria directa*. Pero también existen otras posibilidades de interés, como pueden ser los códigos BCD y Gray.

La codificación BCD tiene sentido, principalmente, para representar números (información cuantitativa) y con tal finalidad su explicación se encuentra detallada en el capítulo 3 (apartado 3.4.). El código Gray (que ya ha sido utilizado en la numeración de los mapas de Karnaugh, apartado 2.2.) será considerado, con mayor detalle, un poco más adelante en este mismo apartado.

6.1.1. Longitud de palabra y capacidad de información

En principio, las palabras digitales son de longitud variable según la información que vayan a representar. Una palabra binaria de longitud **p** está formada por **p** dígitos **b_{p-1} b_{p-2} b_{p-3} ... b₃ b₂ b₁ b₀** y es capaz de representar o expresar 2^p posibilidades.

Al representar los dígitos de una palabra con subíndices **b_i**, se comienza numerando el dígito menos significativo con 0 ya que dicho dígito (caso de que la palabra sea un número binario) corresponde a las unidades y su valor relativo es $2^0 = 1$; de esta forma el valor relativo del dígito *i*-ésimo **b_i** es 2^i .

Muchos bloques y sistemas digitales adoptan una longitud fija para las palabras que procesan; en tal caso, si la palabra que se desea procesar es de menor número de bits se añade el correspondiente número de ceros para completar su longitud, mientras que cuando el número o información a procesar desborda la longitud de palabra fijada se utilizan varias palabras sucesivas.

Las longitudes de palabra más comunes son las siguientes:

- 4 bits: 16 posibilidades
- 1 *byte* = 8 bits: 256 posibilidades
- 16 bits: 65.536 posibilidades (64K)
- 32 bits: 4.294.967.296 posibilidades (aprox. 4.300 millones)
- 64 bits: aprox. 16×10^{18} posibilidades (dieciséis trillones)

destacando entre ellas la longitud de 8 bits, que se conoce con el nombre de *byte* y es utilizada como longitud de referencia cuando no se indica otra cosa.

Así, por ejemplo, la capacidad global de una memoria suele expresarse en número de registros de 8 bits (en *bytes*, aun en los casos en que el procesador del sistema utiliza palabras de 16 o de 32 bits): al indicar una memoria de 1 Mega nos referimos a una capacidad de almacenamiento de $2^{20} \approx 10^6$ *bytes*.

2^{20} bytes = **1.048.576** bytes = 1.048.576 x 8 bits = 8.388.608 bits $\approx 8 \times 10^6$ bits.

6.1.2. Compactación hexadecimal

El manejo de palabras binarias por parte del hombre, es decir, su lectura o escritura presenta cierta complejidad, habida cuenta del amplio número de dígitos que las palabras binarias tienen por lo general y de la dificultad que supone citar o visualizar sin errores el correspondiente número de ceros y unos. Por ejemplo, al referirnos a una cantidad no muy grande como puede ser la de 234 unidades tenemos que indicar **11101010** (*uno, uno, uno, cero, uno, cero, uno, cero*; o bien, si leemos este número binario como si fuera decimal, *once millones ciento un mil diez*).

El sistema de numeración hexadecimal, cuya base 16 es la cuarta potencia de 2, permite reducir en gran medida dicha complejidad al compactar de 4 en 4 los dígitos de las palabras binarias. El sistema hexadecimal utiliza 16 signos, del **0** al **F** (15), cuya equivalencia binaria y decimal es la siguiente:

0	0000	0	4	0100	4	8	1000	8	C	1100	12
1	0001	1	5	0101	5	9	1001	9	D	1101	13
2	0010	2	6	0110	6	A	1010	10	E	1110	14
3	0011	3	7	0111	7	B	1011	11	F	1111	15

Ejemplos de palabras en binario y en hexadecimal:

$10010101 = \mathbf{95}$ $10001100 = \mathbf{8C}$ $11110000 = \mathbf{F0}$ $101011 = \mathbf{2B}$

$72 = 1110010$ $\mathbf{D1} = 11010001$ $\mathbf{AB} = 10101011$ $\mathbf{8E} = 10001110$

Una memoria de 16 líneas de direccionamiento tiene una capacidad de 64K registros (65.536) y utilizará para numerarlos los códigos hexadecimales que van del **0000** al **FFFF**:

- el registro **2A3B** será el que hace el número de orden decimal *10.811* (comenzando por el número 0) y se seleccionará con los valores binarios **0010101000111011** en el bus de direcciones;
- el registro que hace el número *50.000* de ellos tendrá por referencia hexadecimal **C350** y se seleccionará con los valores **1100001101010000** en el bus de direcciones;
- al activar dicho bus con los valores **0100111110011100** el registro seleccionado será el **4F9C**, que hace el número *20.380* de ellos.

Un bus de direcciones de 24 líneas permite manejar $2^{24} = 16.777.216 \approx 16 \times 10^6$ registros (16 Megas), cuya numeración requiere 6 dígitos hexadecimales.

Una palabra de 1 *byte* da lugar a 2 dígitos hexadecimales, si es de 16 bits se expresa en 4 dígitos hexadecimales y si es de 32 bits necesita 8 dígitos hexadecimales.

Existen otras formas de compactar las palabras binarias, por ejemplo el sistema de numeración octal, con base 8, que permite agrupar los bits de 3 en 3, pero son mucho menos utilizadas que el sistema hexadecimal.

6.1.3. Codificación Gray

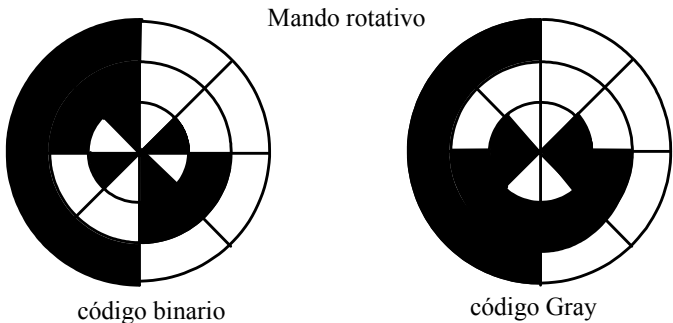
El código Gray se caracteriza por la propiedad de que dos palabras consecutivas del mismo difieren en un solo bit.

Esta propiedad implica que los términos mínimos de vectores de entrada sucesivos son simplificables entre sí (ya que difieren en una sola variable); por ello, la numeración Gray es la base de la simplificación por mapas de Karnaugh. [Con tal finalidad el código Gray fue presentado al tratar dichos mapas en 2.2.]

Pero, además, la codificación Gray proporciona alta seguridad funcional en las transiciones entre palabras binarias: al diferenciarse en un solo bit, el paso de una palabra a la siguiente no puede producir errores o espurios debidos a las diferencias de tiempo en la transición o propagación de cada bit. En tal sentido, el código Gray resulta muy útil en la codificación de estados (es ideal para los grafos de estado), o en la codificación de condiciones o vectores cuando éstos evolucionan siguiendo una secuencia fija.

Por ejemplo, en un mando rotativo con 8 posiciones o niveles el código Gray evita los errores que pueden producirse en las fronteras entre dos posiciones:

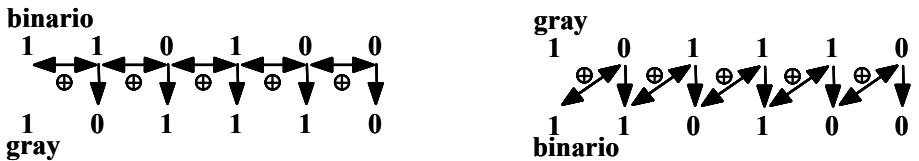
- si la codificación es binaria directa, al pasar del nivel 3 **011** al nivel 4 **100** podría suceder que en la frontera entre ambos apareciese el nivel 0 **000**, por anularse los dos bits menos significativos antes de activarse el otro bit, o bien que se formase el nivel 7 **111** si dicho bit se activa antes que se anulen los dos primeros;
- en código Gray el paso del nivel 3 **010** al nivel 4 **110** no puede producir ningún otro nivel, distinto de ambos, ya que únicamente se modifica el bit más significativo.



La formación sucesiva de las palabras del código Gray, por el método especular, ha sido descrita en el capítulo 2 (apartado 2.2.).

El cambio de código de binario normal a código Gray viene dado por una simple operación "**o-exclusiva**" (sea **b_i** la cifra *i*-ésima en código binario y **g_i** la cifra *i*-ésima en código Gray) : $g_i \text{ (gray)} = b_{i+1} \oplus b_i \text{ (binario)}$; es decir, basta hacer la operación "**o-exclusiva**" entre el bit correspondiente (*i*) del código binario y el anterior (*i+1*) del mismo código.

El cambio de código inverso, de código Gray a binario, es análogo: $b_i \text{ (binario)} = b_{i+1} \oplus g_i$; pero en este caso, es preciso hacer la operación "**o-exclusiva**" entre el bit anterior (*i+1*) del mismo código binario y el bit correspondiente (*i*) del código Gray.



Ejemplos:

$10101101_{(2)} = 11111011_{Gray}$

$10011011_{Gray} = 11101101_{(2)}$

$11100100_{(2)} = 10010110_{Gray}$

$11101011_{Gray} = 10110010_{(2)}$

6.1.4. Codificación de texto

El texto que configura un libro, folleto, carta, documento o cualquier otro tipo de escrito está compuesto por una sucesión ordenada de caracteres alfabéticos y numéricos (alfanuméricos), junto con algunos signos de puntuación, espacios en blanco, separación entre párrafos, tabulaciones, etc.

Nuestro alfabeto utiliza 26 caracteres alfabéticos (25 letras simples más la w) en dos formas, mayúsculas y minúsculas, a los cuales hay que añadir 10 cifras decimales, varios signos de puntuación, algunos signos matemáticos (+, -, =, >, <, ...) y otros caracteres especiales. Un teclado mecanográfico de tipo normal presenta cerca de medio centenar de teclas, con dos posibilidades cada una; en total, unos 100 caracteres. Para su codificación bastarán palabras de 7 bits y sobrará una veintena larga de palabras que se utilizarán para caracteres de control (fin de párrafo, fin de página, salto de línea, tabulaciones, ...).

El código más utilizado para la codificación de textos es el **ASCII** (*American Standard Code for Information Interchange*) que utiliza palabras de 7 bits conforme a la tabla siguiente (existe, también, un código ASCII ampliado que utiliza palabras de 8 bits):

Hexadecimal $b_6b_5b_4$								
$b_3b_2b_1b_0$	0	1	2	3	4	5	6	7
<u>0</u>	NUL	DLE	SP	0	@	P	`	p
<u>1</u>	SOH	DC1	!	1	A	Q	a	q
<u>2</u>	STX	DC2	"	2	B	R	b	r
<u>3</u>	ETX	DC3	#	3	C	S	c	s
<u>4</u>	EOT	DC4	\$	4	D	T	d	t
<u>5</u>	ENQ	NAK	%	5	E	U	e	u
<u>6</u>	ACK	SYN	&	6	F	V	f	v
<u>7</u>	BEL	ETB	'	7	G	W	g	w
<u>8</u>	BS	CAN	(	8	H	X	h	x
<u>9</u>	HT	EM	)	9	I	Y	i	y
<u>A</u>	LF	SUB	*	:	J	Z	j	z
<u>B</u>	VT	ESC	+	;	K	[	k	{
<u>C</u>	FF	FS	'	<	L	\	l	
<u>D</u>	CR	GS	-	=	M	]	m	}
<u>E</u>	SO	RS	.	>	N	^	n	~
<u>F</u>	SI	US	/	?	O	_	o	DEL

Las 10 cifras decimales ocupan los códigos del **30** al **39** mientras que las letras van del **41** (A) al **5A** (Z), las mayúsculas, y del **61** (a) al **7A** (z), las minúsculas. El resto de los códigos se refiere a signos de puntuación y otros caracteres diversos.

Los 32 códigos iniciales, del **00** al **1F**, se destinan a caracteres de control:

00	NUL	Nulo	10	DLE	Anulación de lo transmitido
01	SOH	Inicio de encabezamiento	11	DC1	Control dispositivo 1
02	STX	Inicio de texto	12	DC2	Control dispositivo 2
03	ETX	Final de texto	13	DC3	Control dispositivo 3
04	EOT	Final de transmisión	14	DC4	Control dispositivo 4
05	ENQ	Pregunta	15	NAK	Acuse de recibo negativo
06	ACK	Acuse de recibo	16	SYN	Sincronizador
07	BEL	Aviso acústico	17	ETB	Final del bloque transmitido
08	BS	Retroceder un espacio	18	CAN	Anulación
09	HT	Tabulador horizontal	19	EM	Fin del medio o soporte
0A	LF	Nueva línea	1A	SUB	Substituir
0B	VT	Tabulador vertical	1B	ESC	Escape (anulación de orden)
0C	FF	Nueva página	1C	FS	Separador de archivos
0D	CR	Retorno del carro	1D	GS	Separador de grupo
0E	SO	Fuera de código	1E	RS	Separador de registros
0F	SI	Retorno al código	1F	US	Separador de unidad
10	SP	Espacio (código 10)	7F	DEL	Borrado (código 7F)

Dado que el código **ASCII** es de 7 bits y, en cambio, suelen utilizarse palabras normalizadas a 8 bits (1 *byte*), el octavo dígito se aprovecha para la detección de errores como bit de paridad (véase el apartado siguiente).

El código **ASCII** con paridad utiliza palabras binarias de 8 dígitos, de los cuales el más significativo (el que inicia la palabra) es el bit de paridad; la paridad de la palabra completa es siempre 0. Este código goza de amplia aceptación siendo el utilizado habitualmente en el procesamiento de textos, en la transmisión de la información, en la comunicación con periféricos (impresoras, monitores,...), etc.

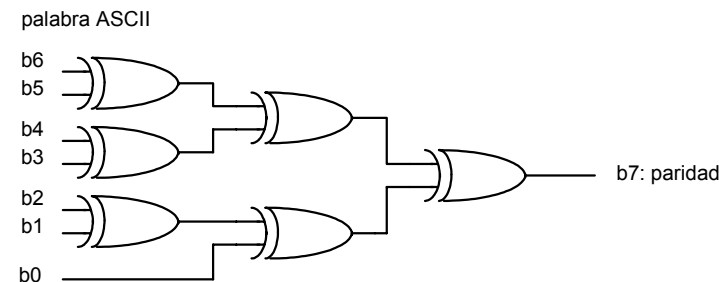
6.2. La paridad para detectar error

La paridad constituye una forma muy simple de detectar errores basada en contabilizar el número de «unos» que cada palabra binaria contiene; se dice que una palabra es par (paridad = 0) cuando el número de «unos» que contiene es par y será impar (paridad = 1) cuando lo sea el número de «unos».

Al añadir a una palabra binaria su paridad se forma una palabra «ampliada» que es siempre par: si la palabra era impar, al añadirle un 1 resulta par; si era par, se le añade un 0 y su paridad no cambia. Caso de recibir una palabra «ampliada» que sea impar, ello supone que, al menos, uno de los dígitos de dicha palabra es erróneo.

De esta manera, la paridad detecta error en una palabra binaria cuando el número de bits erróneos de la misma es impar; en cambio, si en una palabra se modifican (se invierten) un número par de bits la paridad de la palabra no se altera y el error no será detectado.

La paridad de una palabra se calcula circuitalmente mediante puertas "o-exclusiva" en estructura arborescente:



Cada puerta "o-exclusiva" calcula la paridad de sus dos entradas, es decir, genera un 1 al recibir un número impar de «unos» en las mismas. De esta forma, las primeras puertas calculan la paridad de un par de dígitos de la palabra y las puertas siguientes van agrupando las paridades de dos en dos, hasta completar el cálculo de la paridad de la palabra completa.

La paridad es un código de *distancia mínima* 2: al ampliar las palabras binarias añadiendo a las mismas su paridad, las palabras del código resultante son todas ellas de paridad par y se diferencian, al menos, en 2 bits, pues la modificación de un solo bit produce una palabra impar que no pertenece a dicho código.

Un código de distancia mínima 2 permite detectar la modificación o error de un dígito: al invertirse un bit la palabra resultante tiene una distancia 1 respecto a la palabra correcta y no pertenecerá al código (será una palabra errónea), lo cual indica la existencia de error.

En el caso de la paridad, la distancia entre cualesquiera dos palabras «ampliadas» (con su bit de paridad) es siempre múltiplo de 2, pues la paridad de una palabra «ampliada» es siempre par; lo cual permite detectar la existencia de error cuando afecta a un número impar de bits (1, 3, 5, ...) pero, en cambio, la paridad no detecta error cuando el número de bits erróneos es par (2, 4, ...).

Otra forma de aplicar la paridad a conjuntos de **n** palabras binarias consiste en añadir una palabra más, cuyos dígitos correspondan a la paridad global de los bits que ocupan la misma posición en las **n** palabras anteriores; es decir, el bit *i*-ésimo de la nueva palabra es la paridad de los **n** bits *i*-ésimos de las palabras anteriores.

Esta forma de detección de errores es complementaria de la paridad de las palabras individuales, ya que realiza una detección «en vertical» sobre las columnas de bits (supuestas las **n** palabras colocadas en columna, una debajo de otra), mientras que la paridad de cada palabra realiza una detección «en horizontal» sobre la fila de bits que forma la palabra.

En relación con estos dos tipos de paridad (*vertical y horizontal*) las **n** palabras de longitud **p** se organizan matricialmente, formando una matriz de dimensión **n x p**, y se calculan las paridades de las filas (*paridad horizontal*) y de las columnas (*paridad vertical*) de la matriz.

Las dos palabras resultantes (*vertical y horizontal*) tienen idéntica paridad, ya que corresponde a la paridad global de las **n** palabras en bloque; si al conjunto de las **n** palabras se añaden las citadas paridades (la horizontal como columna y la vertical como fila) y se agrega dicho bit de paridad global, se configura una nueva matriz ampliada de dimensiones **n+1 x p+1**, cuyas paridades en ambos sentidos (horizontal y vertical) son todas nulas (matriz de paridad **0**).

Ejemplo: Conjunto de 12 palabras, con su paridad vertical y horizontal.

paridad horizontal ↓		Presencia de un error único:		
1 1 0 1 1 0 1 1	0	1 1 0 1 1 0 1 1	0	0
1 0 0 0 1 0 1 1	0	1 0 0 0 1 0 1 1	0	0
0 1 1 0 0 0 0 1	1	0 1 1 0 0 0 0 1	1	0
1 1 1 0 0 1 1 1	0	1 1 1 0 1 1 1 1	0	← 1
1 0 1 1 0 1 0 1	1	1 0 1 1 0 1 0 1	1	0
1 0 0 0 0 1 0 1	1	1 0 0 0 0 1 0 1	1	0
0 0 1 1 0 1 1 0	0	0 0 1 1 0 1 1 0	0	0
0 1 1 1 1 0 1 0	1	0 1 1 1 1 0 1 0	1	0
1 0 1 1 0 1 1 0	1	1 0 1 1 0 1 1 0	1	0
0 1 0 0 0 1 0 1	1	0 1 0 0 0 1 0 1	1	0
1 1 1 0 0 0 1 1	1	1 1 1 0 0 0 1 1	1	0
paridad 0 1 1 1 1 0 0 0	0	0 1 1 1 1 0 0 0	0	0
vertical → 1 1 0 0 0 0 1 0	1	1 1 0 0 0 0 1 0	1	0
↑				
nuevas paridades del conjunto «ampliado»: → 0 0 0 0 1 0 0 0 0				

Esta combinación de paridades presenta una probabilidad muy alta de detectar la existencia de error, pues solamente falla en los casos en que la situación de todos los bits erróneos corresponda a filas y a columnas que contengan ambas un número par de errores.

Además, en caso de que se detecte error en una sola fila y una sola columna, dicho bit puede corregirse (bajo el supuesto, muy razonable, de que no existen en la misma fila y columna otros errores que, a su vez, estén «compensados» en el resto de filas y columnas). [Véase en el ejemplo anterior la presencia de un error y cómo queda indicado al calcular las nuevas paridades vertical y horizontal del conjunto.] También pueden corregirse varios errores en caso de que todos ellos se encuentren en la misma fila o en la misma columna y que el número de errores sea impar.

La *verificación de suma (checksum)* es un tipo de detección análogo a la paridad vertical: utiliza como palabra adicional el resultado de la suma de las **n** palabras anteriores ejecutada sobre **p** bits (sin tener en cuenta los arrastres superiores), siendo **p** la longitud de palabra de todas ellas; es decir, la suma de las **n** palabras en módulo **2^p** (lo cual asegura que la longitud de palabra del resultado es también **p**). De esta forma se detecta la presencia de errores siempre que no se encuentren «compensados» en la misma columna, en el sentido de que el número de bits que han cambiado de **0** a **1** es el mismo que los bits que han efectuado el cambio opuesto de **1** a **0**.

6.3. Códigos detectores y correctores de error

En la transmisión y en el almacenamiento (conservación en memoria, sea ésta de tipo RAM o sea en disco, cinta o cualquier otro soporte físico de la misma) de las palabras binarias pueden producirse errores que modifiquen el valor booleano de uno o de varios bits. La palabra o palabras correspondientes expresarán una información errónea.

Un código es capaz de detectar errores, es decir, de discriminar palabras con información errónea, cuando la modificación de uno o varios bits de una palabra del código da lugar a una palabra binaria que no pertenece al mismo. El concepto de distancia permite analizar y generalizar la forma de operar de los códigos detectores de error.

La *distancia de Hamming* entre dos palabras binarias de la misma longitud es el número de dígitos en que dichas palabras se diferencian. Se dice que un código es de *distinta mínima D* cuando dos palabras del mismo difieren, al menos, en el valor de **D** de sus bits. Tal código es capaz de detectar cualquier error que afecte a **D-1** dígitos o menos, ya que la modificación en una palabra del código de un número de bits inferior a **D** da lugar a una palabra que no pertenece al código. Para detectar la modificación o error que afecte a **n** bits se requiere un código cuya distancia mínima sea **n+1** o superior.

La paridad es un código de distancia par (múltiplo de 2): las palabras permisibles distan entre sí un número par de dígitos. Cualquier modificación que afecte a un número impar de bits da lugar a una palabra no permitida y, por tanto, reconocida como errónea; en cambio, la paridad no permite detectar error cuando el número de bits afectados es par.

Hamming desarrolló métodos sistemáticos, basados en la misma idea que la paridad, para construir códigos de distancia mínima 3 y 4, con la particularidad de que dichos códigos permiten corregir el error cuando éste afecta a un solo bit.

El código Hamming de distancia mínima 3 utiliza paridades «parciales» referidas a subconjuntos de dígitos de la palabra inicial. El código Hamming de distancia mínima 4 es análogo, añadiendo simplemente un bit de paridad global; ese bit suplementario aporta mayor fiabilidad respecto a la corrección de error.

El código Hamming de distancia mínima 3 permite detectar y corregir errores relativos a un solo bit, pero no es capaz de diferenciar los que afectan a un bit de los que afectan a un número par de ellos; de manera que, en el caso de que exista error en dos bits y se efectúe la corrección como si fuese en uno solo de ellos, la palabra corregida tiene más errores que la original.

El código Hamming de distancia mínima 4 (que supone simplemente añadir un bit de paridad global al de distancia mínima 3) informa si el error es en número par o impar de bits, evitando correcciones erróneas; es cierto que no permite diferenciar si el error afecta a 1 o a 3 (o más) bits, pero la probabilidad de que afecte a 1 ó 2 dígitos es muy superior a la de tener 3, 5, 7, ... errores.

El método Hamming para formar un código de distancia mínima 4 se describe en las dos páginas siguientes: en la página de la izquierda (pág. 150) se explica la manera de actuar y en la de la derecha (pág. 151) se detalla su aplicación a un caso concreto: una palabra inicial de 12 dígitos.

De igual manera en las siguientes dos páginas se describe la forma de detectar y corregir errores en código Hamming de distancia mínima 4: a la izquierda (pág. 152) se explica la manera de proceder y a la derecha (pág. 153) se aplica al caso de un bit erróneo en la palabra obtenida anteriormente (en las páginas 150 y 151).

La generalización de este método para aplicarlo a palabras de diferente longitud es directa.

6.3.1. Construcción del código Hamming

El procedimiento de construcción del código se explica en relación con las sucesivas columnas de la tabla representada en la página siguiente.

- Sea una palabra inicial de 12 bits: **b₁₁ b₁₀ b₉ b₈ b₇ b₆ b₅ b₄ b₃ b₂ b₁ b₀**.
- Se trata de construir una palabra ampliada por un conjunto de paridades parciales, que se entremezclan con los bits de la palabra inicial; para ello, es necesario numerar en binario los bits de la palabra ampliada [columna (1)]. Para facilitar la descripción se utiliza la denominación |número| para indicar el que numera los bits de la palabra ampliada.

*Habida cuenta de que las paridades parciales se refieren a las posiciones de los «unos» en el |número| de cada bit, en la columna (2) se han sustituido los «ceros» por – y los «unos» por símbolos diversos según su posición (@ , * , Δ , # , &) a los que denominaremos grafismos. [Esta representación gráfica es absolutamente superflua una vez comprendido el método.]*

- El primer bit de la palabra ampliada (numerado con 00...) se reserva para la paridad global y es el último que se calcula. Los bits de la palabra ampliada que solamente tienen un «uno» en su |número| (es decir, los que tienen un solo grafismo) se reservan para las paridades parciales [columna (3)].

- En el resto de los bits se colocan ordenadamente los dígitos de la palabra inicial (de menor a mayor valor significativo, tal como están ordenados sus |números|) [columnas (4) y (5)]. Si es preciso se continúa la numeración de bits, hasta que quepan todos los de la palabra inicial, reservando siempre los bits cuyo |número| contiene un solo «uno» para paridades parciales.

- Cada paridad parcial corresponde a un |número| con un solo «uno» y se calcula sobre los bits cuyo |número| contiene un «uno» en la misma posición; es decir, estarán reservados para paridad parcial los bits con un solo grafismo y para calcular una de ellas tomaremos los bits señalados con el mismo grafismo y hallaremos la paridad del conjunto de ellos.

La paridad parcial **P1** es la de los bits cuyo |número| acaba por «uno», es decir, aquellos cuyo grafismo es **&**: **b₁₁ b₁₀ b₈ b₆ b₄ b₃ b₁ b₀**. La paridad parcial **P2** es la de los bits cuyo |número| tiene un «uno» en penúltima posición, o sea cuyo grafismo es **#**: **b₁₀ b₉ b₆ b₅ b₃ b₂ b₀**. La paridad **P3** corresponde a un «uno» en antepenúltima posición y su grafismo es **Δ**: **b₁₀ b₉ b₈ b₇ b₃ b₂ b₁**; **P4** se refiere al grafismo *****: **b₁₀ b₉ b₈ b₇ b₆ b₅ b₄** y **P5** a **@**: **b₁₁**.

- Una vez calculadas y puestas en su lugar las paridades parciales, se calcula la paridad global de la palabra y se coloca en el bit menos significativo (|número| = 0). De esta forma, se tiene la palabra ampliada completa en código Hamming de distancia mínima 4; bien entendido que, conforme a la numeración de los bits, las columnas están ordenadas del bit menos significativo (el primero de arriba) al más significativo (el último de abajo).

Sea la palabra inicial **1 0 0 1 0 1 1 1 0 1 0 0** que deseamos pasar a código Hamming de distancia 4:

(1) número de orden	(2) grafismo	(3) paridades parciales	(4) Posiciones de los bits	(5) palabra inicial	(6) cálculo de paridades	(7) paridad global
00000	————	P0	P0			0
00001	———— &	P1 [&]	P1 [&]		0	0
00010	——— # —	P2 [#]	P2 [#]		1	1
00011	——— # &		bit 0	0	0	0
00100	——— Δ —	P3 [Δ]	P3 [Δ]		0	0
00101	——— Δ — &		bit 1	0	0	0
00110	——— Δ # —		bit 2	1	1	1
00111	——— Δ # &		bit 3	0	0	0
01000	— * ———	P4 [*]	P4 [*]		0	0
01001	— * ——— &		bit 4	1	1	1
01010	— * — # —		bit 5	1	1	1
01011	— * — # &		bit 6	1	1	1
01100	— * Δ ———		bit 7	0	0	0
01101	— * Δ — &		bit 8	1	1	1
01110	— * Δ # —		bit 9	0	0	0
01111	— * Δ # &		bit 10	0	0	0
10000	@ ———	P5 [@]	P5 [@]		1	1
10001	@ ——— &		bit 11	1	1	1

Palabra inicial: **1 0 0 1 0 1 1 1 0 1 0 0**

Código Hamming: **1 1 0 0 1 0 1 1 1 0 0 1 0 0 1 0 0**

(Se han destacado y subrayado los bits que corresponden a las paridades).

Es fácil comprobar la distancia mínima entre dos palabras de este código; consideremos dos palabras iniciales diferentes (cuya distancia sea menor de 3):

- si se diferencian en un solo bit, diferirán también en, al menos, dos bits de paridad parcial (ya que el número de orden del bit modificado tendrá, por lo menos, dos unos),
- si tienen dos bits diferentes, lo será también, al menos, uno de los bits de paridad (ya que los números de orden de los bits modificados diferirán, cuando menos, en un uno), o sea que la distancia entre dos palabras ampliadas no puede ser inferior a 3; además, como el código incluye la paridad global, su distancia será siempre múltiplo de 2, es decir, entre dos palabras ampliadas habrá una distancia mínima de 4.

6.3.2. Detección y corrección de errores

• La verificación respecto a si una palabra ampliada pertenece o no al código se realiza comprobando las paridades parciales y la paridad global de la palabra completa; si la palabra es correcta (si no hay errores detectables), las nuevas paridades deben ser, todas ellas, nulas pues corresponden a conjuntos de bits ampliados con su propia paridad.

• La comprobación de la paridad global se calcula sobre todos los bits de la palabra ampliada y su resultado puede ser:

- comprobación de la paridad global = **0** y, en tal caso, o no existe error, o éste afecta a un número par de dígitos y no se puede hacer corrección de error sobre la palabra recibida
- comprobación de la paridad global = **1** y, en tal caso, existe error y, además, afecta a un número impar de dígitos; en principio, puede suponerse que afecta a un solo bit (pues, salvo sistemas de transmisión o almacenamiento muy defectuosos, es mucho más probable que haya error en un bit que en tres o más de ellos) y, consiguientemente, es viable realizar la corrección de dicho error.

• Cada comprobación de paridad parcial se calcula sobre los bits cuyo |número| contiene un «uno» en la misma posición; es decir, se toman todos los bits señalados con el mismo grafismo y se halla la paridad del conjunto de ellos.

La nueva paridad parcial **CP1** es la de todos los bits cuyo |número| acaba por «uno», es decir, aquellos cuyo grafismo es &: **b11 b10 b8 b6 b4 b3 b1 b0 P1**. La comprobación de paridad parcial **CP2** es la de los bits cuyo |número| tiene un «uno» en penúltima posición, es decir su grafismo es #: **b10 b9 b6 b5 b3 b2 b0 P2**. La comprobación **CP3** corresponde a un «uno» en antepenúltima posición, o sea al grafismo Δ: **b10 b9 b8 b7 b3 b2 b1 P3**; **CP4** se refiere al grafismo *: **b10 b9 b8 b7 b6 b5 b4 P4** y **CP5** a @: **b11 P5**.

• En caso de que la palabra pertenezca al código Hamming todas las comprobaciones de paridad darán resultado **0**, tanto las parciales como la global. Tal cosa sucederá cuando no haya habido error en la transferencia de la palabra; nunca podremos estar absolutamente seguros de la ausencia de error, pero si todas las comprobaciones de paridad son nulas, sabemos que, de haber error hay cuatro o más errores (y siempre en número par), lo cual es altamente improbable.

• La comprobación de paridades parciales da lugar a un número binario ...**CP5 CP4 CP3 CP2 CP1**, con las siguientes posibilidades:

- si dicho número es nulo y la paridad global también lo es, estamos en el caso anterior y, en principio, aceptaremos (con muy alta probabilidad) la ausencia de error;
- si este número no es nulo y la paridad global es **0**, hay error y afecta a un número par de dígitos, por lo cual no podemos corregirlo;
- si tal número no es nulo y la paridad global es **1**, es razonable suponer que el error afecta a un solo bit y es posible corregirlo: el número ...**CP5 CP4 CP3 CP2 CP1** señala al dígito erróneo (ver la justificación en la página 154).

En la palabra ampliada **1 1 0 0 1 0 1 1 1 0 0 1 0 0 0 1 0 0**, obtenida en el subapartado anterior (6.3.1.) que corresponde a la inicial **1 0 0 1 0 1 1 1 0 1 0 0**, introducimos un error (invertimos el bit nº 9) **1 1 0 0 1 0 1 1 0 0 0 1 0 0 0 1 0 0**. La manera de aplicar el procedimiento de detección de error es la siguiente:

(1)	(2)	(3)	(4)	(5)	(6)	(7)
número		Posiciones	comprobación	palabra	cálculo de	error
de orden	grafismo	de los bits	de paridades	ampliada	CP	
00000	—————	P0	CP0	0	1	← Si
00001	————— &	P1	CP1 [&]	0	1	posición
00010	———— # —	P2	CP2 [#]	1	0	del bit
00011	—— — # &	bit 0		0		erróneo:
00100	—— Δ — —	P3 [Δ]	CP3 [Δ]	0	0	01001
00101	—— Δ — &	bit 1		0		
00110	—— Δ # —	bit 2		1		
00111	—— Δ # &	bit 3		0		
01000	— * ———	P4	CP4 [*]	0	1	
01001	— * ——— &	bit 4		0		←
01010	— * — # —	bit 5		1		
01011	— * — # &	bit 6		1		
01100	— * Δ — —	bit 7		0		
01101	— * Δ — &	bit 8		1		
01110	— * Δ # —	bit 9		0		
01111	— * Δ # &	bit 10		0		
10000	@————	P3	CP5 [@]	1	0	
10001	@——— &	bit 11		1		

Se debe corregir el bit cuyo |número| es **0 1 0 0 1**, según indica el número que forman las paridades parciales.

Código Hamming correcto: **1 1 0 0 1 0 1 1 1 0 0 1 0 0 0 1 0 0**

Palabra inicial: **1 0 0 1 0 1 1 1 0 1 0 0**

La palabra inicial se obtiene a partir de la palabra ampliada, eliminando en ella las paridades, tanto las parciales como la global. [Compruébese que es correcta, es decir, que coincide con la palabra inicial del apartado 6.3.1.]

Cuando el número ...**CP5 CP4 CP3 CP2 CP1** no es nulo y la paridad global es **1**, sabemos que hay error y que afecta a un número impar de dígitos; podemos suponer que el error afecta a un solo bit y, en tal caso, el número ...**CP5 CP4 CP3 CP2 CP1** señala al dígito erróneo:

- **CP1 = 1** significa que el error se encuentra entre los bits que corresponden a la paridad **P1** (es decir, entre aquellos cuyo |número| acaba por «uno», grafismo &),
- **CP2 = 1** significa que el error se encuentra entre los bits que corresponden a la paridad **P2** (es decir, su |número| tiene un «uno» en penúltima posición, grafismo #)
- y así sucesivamente ...;

de manera que el número binario que forman ...**CP5 CP4 CP3 CP2 CP1** corresponde precisamente al |número| del bit erróneo.

La siguiente tabla indica el número de bits de paridad necesarios en el código de Hamming de distancia mínima 4 según el número de bits de la palabra inicial:

número de bits			
<u>número de bits</u> <u>de paridad parcial</u>	<u>nº total de bits</u> <u>de paridad</u>	<u>número máximo de bits</u> <u>de la palabra ampliada</u>	<u>nº máximo de bits</u> <u>de la palabra inicial</u>
3	3 + 1 = 4	2 ³ = 8	8 – 4 = 4
4	4 + 1 = 5	2 ⁴ = 16	16 – 5 = 11
5	5 + 1 = 6	2 ⁵ = 32	32 – 6 = 26
6	6 + 1 = 7	2 ⁶ = 64	64 – 7 = 57
7	7 + 1 = 8	2 ⁷ = 128	128 – 8 = 120
8	8 + 1 = 9	2 ⁸ = 256	256 – 9 = 247

Una palabra inicial de 4 dígitos duplica su longitud al pasar a código Hamming de distancia mínima 4, si es de 1 byte requiere 5 dígitos adicionales de paridad (pasa a 13 bits, un aumento del 65 %) y para palabras de 16 bits es preciso añadir otros 6 (40%). En cambio, una palabra inicial de 32 bits aumenta solamente en 7 más (22%) y una de 120 bits se amplía a 128 (un 7%).

Con este mismo tipo de idea conceptual (la de introducir adecuadamente paridades parciales) pueden construirse códigos más complejos de distancia mínima superior. La detección y corrección de errores, es decir, la fiabilidad de la información es un tema de interés cada vez mayor y constituye una rama especializada dentro del amplio campo de la codificación de la información.

Consideremos otro ejemplo, relativo a una palabra inicial de 10 dígitos:

Sea una palabra binaria inicial de 10 bits: **1 1 0 0 1 1 1 0 0 0**
 que en código de Hamming de distancia mínima 4 será: **1 1 0 0 1 1 0 1 0 0 1 0 1 1 0**
 (los dígitos subrayados corresponden a los bits de paridad).

a) si en la palabra inicial se modifica el bit b₅: **1 1 0 0 0 1 1 1 0 0 0**
 su correspondiente palabra codificada será: **1 1 0 0 0 1 1 1 0 0 1 0 0 1 1**
 que dista 4 bits de la anterior.

b) si en la palabra inicial se modifican los bits b₅ y b₆: **1 1 0 1 0 1 1 1 0 0 0**
 su correspondiente será: **1 1 0 1 0 1 0 1 0 0 1 0 1 0 1**
 que dista 4 bits de la primera y otros 4 bits de la anterior.

c) si en la palabra inicial se modifican los bits b₅, b₆ y b₇: **1 1 1 1 0 1 1 1 0 0 0**
 su correspondiente palabra codificada será: **1 1 1 1 0 1 1 1 0 0 0 0 1 0 0**
 que dista 4 bits de la palabras anterior y 6 bits de las dos primeras.

d) si en la transmisión de la primera palabra ampliada se invierte un solo bit, el que hace el número de orden sexto: **1 1 0 0 0 1 1 1 1 0 1 0 0 1 1**
 la palabra recibida corresponde a una palabra errónea, es decir, que no coincide con la primera palabra inicial: **1 1 0 0 0 1 1 1 0 0**

pero la comprobación de la paridad global da **1** (error en número impar de bits) y el número correspondiente a las comprobaciones de paridades parciales valdrá **0110**, lo cual indica error en el bit cuyo número de orden es 6.

e) invirtiendo dicho bit 6 se recupera la palabra correcta: **1 1 0 0 0 1 1 1 0 0 1 0 0 1 1**
 y su correspondiente palabra inicial: **1 1 0 0 0 1 1 0 0 0**

f) si en la transmisión de la primera palabra se invierten los bits que hacen los números de orden sexto y séptimo: **1 1 0 0 0 1 1 0 1 0 1 0 0 1 1**
 el número correspondiente a las comprobaciones de paridades parciales valdrá **0001**, lo cual indica error en el bit cuyo número de orden es 1

corrigiéndolo se generaría la palabra: **1 1 0 0 0 1 1 0 1 0 1 0 0 0 1**
 que es errónea pues corresponde a la palabra inicial: **1 1 0 0 0 1 0 1 0 0**

distinta de la primera. Ello es debido a que el error afectaba a dos bits: la comprobación de la paridad global da **0** (error en número par de bits).

Conviene tener presente la relación entre la detección y corrección de errores y la redundancia en la representación de la información. Como hemos visto en este apartado y en el anterior (6.2. y 6.3.) la detección y corrección de errores se basa en utilizar formas de expresar la información (códigos) que no sean mínimas, sino formas expandidas con suficiente redundancia.

De hecho en el lenguaje hablado continuamente reajustamos la información gracias a la redundancia: palabras que se pronuncian o se escuchan a medias, sílabas trastocadas o suprimidas, discordancias gramaticales, pequeñas faltas de coherencia, etc.,... no suponen ningún problema porque el receptor las completa o corrige en relación con el contexto, gracias a que existe redundancia. Sin ella, cualquier conversación sería muy difícil y cualquier lectura precisaría de una atención agotadora.

En muchas ocasiones, la pregunta: ¿puedes explicarme eso más despacio?, no es sino una solicitud de mayor redundancia.

De igual forma, los códigos binarios pueden detectar errores cuando no son mínimos, cuando se amplían con bits añadidos que permiten efectuar comprobaciones y reajustes; en definitiva, códigos con redundancia.

Al incorporar bits adicionales, existirán palabras que pertenecen al código y otras que no pertenecen al mismo, de forma que, si al producirse error la palabra resultante no pertenece al código, ello nos permite detectar la existencia de error. La adecuada combinación de los bits añadidos permite también la reparación de la palabra errónea cuando el número de errores es limitado.

7 PUERTAS LÓGICAS CON DIODOS E INTERRUPTORES

- 7.1. Tensiones booleanas
- 7.2. Puertas lógicas con diodos
- 7.3. El problema del acoplo en tensión
- 7.4. El interruptor como inversor; puertas inversoras
- 7.5. Características ideales en una puerta lógica
- 7.6. El transistor MOS como interruptor

El álgebra de conmutadores constituye una «materialización» del álgebra booleana binaria, una forma de construirla físicamente mediante la conexión de interruptores con dos estados: paso y corte.

Hay muchos dispositivos físicos, no necesariamente eléctricos, que pueden utilizarse como interruptores (por ejemplo, neumáticos, con aire a presión o con fluidos). Pero entre los diversos tipos de interruptores predominan los componentes electrónicos y, más en concreto, los transistores por su altísima velocidad de conmutación entre los dos estados y por su reducido tamaño (que se traduce en alta densidad de integración, bajo consumo, reducido coste, ...).

Los diodos, como simples «discriminadores de polaridad», pueden actuar a manera de «interruptores de paso» (sin entrada de control) y configurar puertas booleanas no inversoras, "o" e "y"; pero tales puertas sólo pueden utilizarse individualmente, debido a que presentan problemas de acoplo entre ellas.

Para la conexión sucesiva de puertas lógicas resulta indispensable asegurar que su tensión de salida no se modifica (es decir, no se produce una «caída de tensión» apreciable) al conectar sobre ella la entrada o entradas de otras puertas; esto es lo que se entiende por buen acoplo en tensión: conservación de la misma tensión en la salida antes y después de la conexión.

Las puertas lógicas habituales se construyen con transistores y son de tipo inversor, "y-negada", "o-negada",... El estudio de puertas inversoras con interruptores ideales facilita la comprensión de su configuración y funcionamiento y permite considerar las características ideales para ellas.

Precisamente el transistor MOS es un interruptor cuasi-ideal con el que puede construirse una amplísima diversidad de puertas inversoras de muy buenas características.

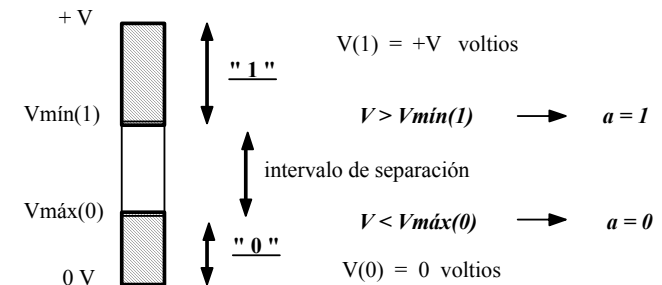
Acoplo en tensión, inmunidad frente a las perturbaciones (frente al «ruido» electromagnético), velocidad de trabajo (o, lo que es lo mismo, tiempo de respuesta) y consumo son aspectos a tener en cuenta en el funcionamiento de las puertas lógicas. Tales cuestiones están relacionadas con las resistencias de entrada y de salida y con la capacidad de entrada de las puertas.

7.1. Tensiones booleanas

Los estados eléctricos representativos de los valores booleanos estarán definidos en términos de tensión o de intensidad; generalmente se expresan en forma de tensión ya que resulta más sencillo razonar con tensiones que con intensidades. [A fin de cuentas, la tensión es una diferencia de potencial «presente» entre dos puntos, mientras que la intensidad es una corriente que fluye, «que pasa».]

A cada estado o valor booleano le corresponderá un intervalo de tensión determinado y entre ambos intervalos, correspondientes al **0** y al **1**, existirá una zona de separación.

En general al **0** booleano se le asocia un intervalo de tensión a partir de **0** voltios y al **1** se le asigna un intervalo de tensión hasta **+V** voltios, siendo **V** la tensión de alimentación.



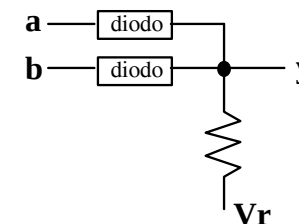
Este tipo de asignación en la que $V(1) > V(0)$ se denomina lógica positiva, para distinguirla de los casos en que $V(1) < V(0)$ que corresponden a lógica negativa.

7.2. Puertas lógicas con diodos

Nota: para quienes no hayan estudiado anteriormente el comportamiento de los diodos, el capítulo T1 describe la forma de actuar de los diodos y presenta un modelo operativo del funcionamiento de los diodos semiconductores; ver T1.2. Unión PN (y, en su caso, ver también T1.1. Semiconductores).

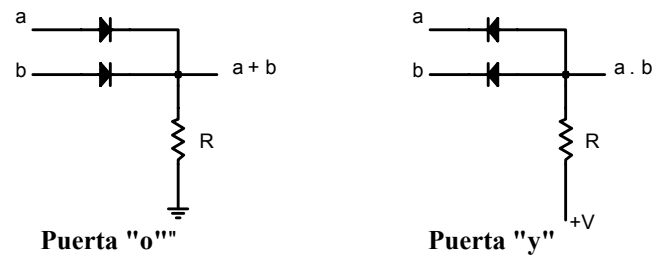
Entendemos por puertas booleanas o puertas lógicas las realizaciones concretas de las operaciones booleanas mediante dispositivos físicos.

Las puertas lógicas con diodos tendrán una estructura del tipo siguiente:



Los diodos actúan dejando pasar la tensión de sus entradas cuando ésta corresponde a uno de los dos valores lógicos y no dejándola pasar para el otro valor; la tensión de referencia V_r debe ser la correspondiente al valor que «no pasa».

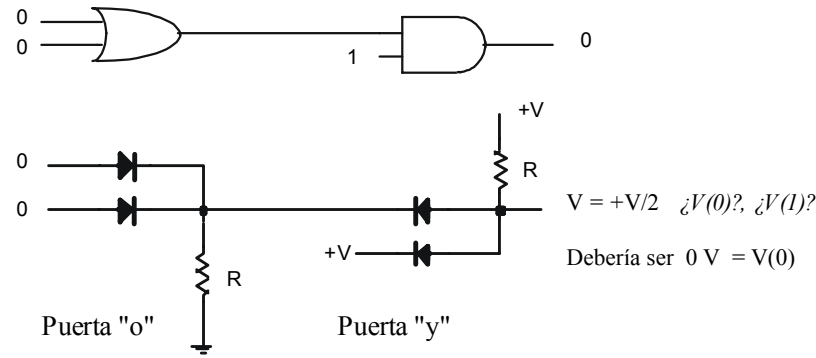
La puerta "o" debe dejar pasar los «unos», de forma que siempre que haya un 1 a la entrada el resultado sea 1 y la puerta "y" debe dejar pasar los «ceros», ya que siempre que una entrada sea 0, el resultado debe ser 0 también.



Estas puertas son directamente ampliables a tres o más entradas, sin más que añadir un diodo por cada nueva entrada.

Desde el punto de vista analógico la puerta "o" constituye un «selector de tensión máxima» (selecciona la mayor de sus tensiones de entrada, supuesto que dicha tensión sea mayor que 0 V) y la puerta "y" es un «selector de tensión mínima» (selecciona la menor de sus tensiones de entrada, supuesto que sea menor que +V).

Ambas puertas son simples y económicas, apropiadas para su utilización como puertas individuales, pero no para formar series de puertas lógicas, conectadas entre sí. Consideremos un ejemplo de los errores que pueden derivarse de la conexión sucesiva de puertas con diodos:



El problema reside en que la impedancia de entrada de la segunda puerta es igual a la de salida de la primera de ellas R ; con ello los efectos de carga de una puerta sobre otra pueden producir errores por desplazamiento de la tensión de salida, como en el caso anterior. No existe un buen acoplo en tensión.

Además, dado que el funcionamiento específico de los diodos consiste en dejar pasar o no una tensión a su través, sin generar nuevos valores de tensión, no es posible realizar con ellos la operación booleana de complementación o negación: no hay inversores con sólo diodos.

Por todo ello, las puertas "o" e "y" con diodos son apropiadas por su sencillez para ser utilizadas individualmente, pero no para conjuntos de puertas lógicas; son útiles para incluir en un diseño operaciones "o" e "y" aisladas (en particular para utilizar dichas puertas en circuitos analógicos o en etapas de potencia), pero no para construir funciones booleanas, que requieren varias puertas conectadas entre sí y, además, necesitan inversores.

7.3. El problema del acoplo en tensión

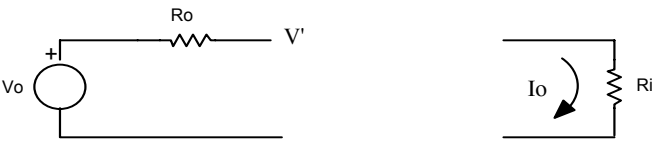
Un circuito digital estará formado por la conexión de múltiples puertas lógicas para configurar funciones booleanas; cada puerta presenta en su salida la tensión correspondiente a uno de los dos valores booleanos (0/1) y debe comunicar dicha tensión a la siguiente puerta o a varias puertas lógicas conectadas a dicha salida. Se requiere, pues, un buen acoplo en tensión, es decir, el valor de tensión de salida de una puerta no debe deteriorarse (no debe sufrir modificaciones importantes) por el hecho de conectarle una o varias entradas de otras puertas booleanas.

La condición de buen acoplo en tensión de un circuito con otro reside en que la resistencia de entrada del segundo circuito sea de valor muy superior a la resistencia de salida del primer circuito:

$$R_i \text{ (segundo circuito)} \gg R_o \text{ (primer circuito)}.$$

En tal caso, la conexión de ambos circuitos no modifica apreciablemente la tensión que presenta el primero de ellos con su salida sin conectar (*al aire*).

Visto desde su salida un circuito presenta un equivalente thevenin consistente en un generador de tensión y una resistencia (o, en el caso más general, una impedancia) de salida. La entrada del segundo circuito puede reducirse, en situación estacionaria, a una resistencia de carga: resistencia de entrada. La conexión entre los dos circuitos viene representada por la siguiente figura.



La tensión de salida del primer circuito V' coincide con la tensión de su generador thevenin V_o cuando no existe ninguna conexión sobre dicha salida; al conectar el segundo circuito, la tensión de salida V' se desplaza un valor ΔV_o :

$$\Delta V_o = I_o \cdot R_o = V_o \cdot R_o / (R_o + R_i)$$

que será despreciable frente a V_o cuando $R_i \gg R_o$ (o lo que es lo mismo, cuando la intensidad I_o que requiere el segundo circuito sea relativamente pequeña):

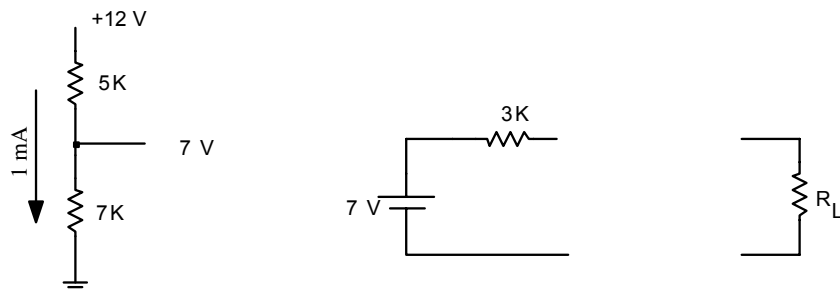
$$V' = V_o \cdot R_i / (R_i + R_o); \text{ si } R_i \gg R_o, \text{ resulta } R_i + R_o \approx R_i \text{ y, por tanto, } V' \approx V_o.$$

En tal caso se dice que existe **buen acoplo en tensión**, es decir, el primer circuito comunica al segundo la tensión presente en su salida sin que dicha tensión se modifique apreciablemente al conectar el segundo circuito sobre el primero.

¿Cuánto debe ser mayor R_i respecto a R_o ? Sea $R_i = k \cdot R_o$, $V' = V_o \cdot k / (k+1)$, $\Delta V_o = V_i / (k+1)$. Para $k = 10$ (R_i diez veces R_o), la tensión de salida disminuye en un 9 % por el hecho de la conexión; si $k = 20$, la disminución es sólo de un 5 %.

Algunos ejemplos pueden servir para aclarar el concepto de acoplo en tensión:

- Divisor de tensión

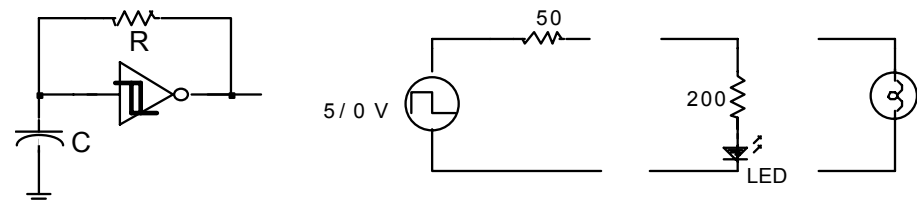


Podemos obtener una referencia de tensión de 7 V a partir de una tensión de alimentación de 12 V con un divisor de tensión formado por dos resistencias de 5K y 7K ($5 + 7 = 12$).

El equivalente thevenin del divisor de tensión contiene una tensión de 7 V en serie con una resistencia de salida de $5K/7K \approx 3K$; para que la referencia de 7 V no se modifique apreciablemente la resistencia de carga debe ser mucho mayor de 3K o, lo que es lo mismo, la intensidad que requiere la carga debe ser muy inferior a la que circula por el propio divisor de tensión ($12 \text{ V} / (5K + 7K) = 1 \text{ mA}$).

Si no se cumple esta condición ($R_L \gg 3K$) la tensión de salida del divisor de tensión disminuirá en forma apreciable al conectar la carga; con una resistencia de carga de 100K, la tensión de salida será de 6,8 V, pero si la resistencia de carga es de 1K la salida será de 1,7 V (la cuarta parte de la que corresponde al divisor de tensión aislado).

b) Oscilador en onda cuadrada (1 Hz)



El circuito de la figura anterior es un oscilador en onda cuadrada (que será de baja frecuencia si R y C son de valores altos), con niveles de tensión de salida de 0 y 5 V y una resistencia de salida del orden de 50 Ω .

Supuesta una frecuencia de oscilación de 1 Hz, si conectamos a la salida del oscilador un diodo luminoso (con una resistencia limitadora de intensidad), el LED se encenderá y apagará una vez cada segundo; en cambio, si conectamos una pequeña bombillita de linterna (4,5 V, 200 mA: resistencia 22 Ω) el oscilador no será capaz de encenderla: el nivel de tensión de 5 V se reducirá fuertemente.

Lo mismo sucede al utilizar como instrumentos de laboratorio generadores de señal cuya resistencia de salida no sea muy baja.

Antiguamente se utilizaban sencillos receptores de radio «galenas» consistentes en un simple diodo (para demodular la señal de radiofrecuencia) y un auricular de alta impedancia; actualmente es difícil encontrar tales auriculares, ya que los habituales son de baja impedancia y con ellos no es posible recibir una señal audible [al usar como carga una resistencia demasiado pequeña se reduce fuertemente la señal].

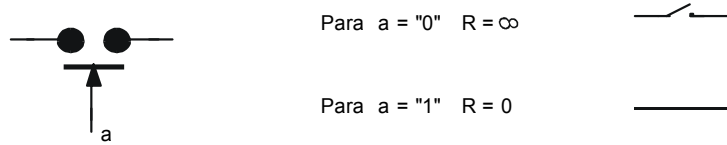
Las puertas lógicas son utilizadas para construir funciones booleanas, mediante la conexión sucesiva de varias puertas: para tener un buen acoplo en tensión interesa que presenten muy alta resistencia de entrada y muy baja de salida.

Ello no ocurre en las puertas con diodos, en las que la resistencia de entrada, cuando el diodo conduce, es la resistencia de polarización R y la resistencia de salida, cuando los diodos no conducen, es la misma resistencia R .

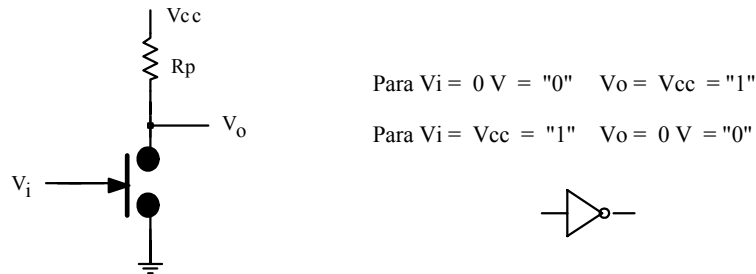
7.4. El interruptor como inversor; puertas inversoras

Un interruptor o conmutador eléctrico es un dispositivo con dos estados: en uno de ellos permite el paso de corriente eléctrica a su través, presentando una resistencia prácticamente nula, y en el otro se comporta como un circuito abierto, ofreciendo una resistencia infinita al paso de corriente.

La entrada controla el estado del interruptor, de forma que cuando la tensión de entrada corresponde al 0 no conduce y cuando es 1 deja pasar libremente la corriente eléctrica; la resistencia de entrada de un interruptor ideal es infinita, es decir, la intensidad que requiere la entrada para mantener el estado es despreciable.



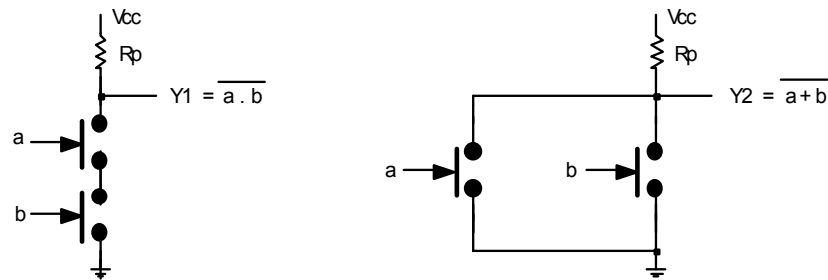
El circuito de la figura siguiente representa un inversor booleano, ya que para $V_i = 0$ el interruptor no conduce y la tensión de alimentación V_{CC} es transmitida a la salida a través de la resistencia de polarización R_p , mientras que para $V_i = 1$ el interruptor conduce y lleva la salida a 0 V.



El carácter inversor de este circuito se debe a que el interruptor cuando conduce no transmite un 1 sino un 0; la resistencia R_p transmite por defecto (cuando el interruptor no conduce) la tensión correspondiente al 1.

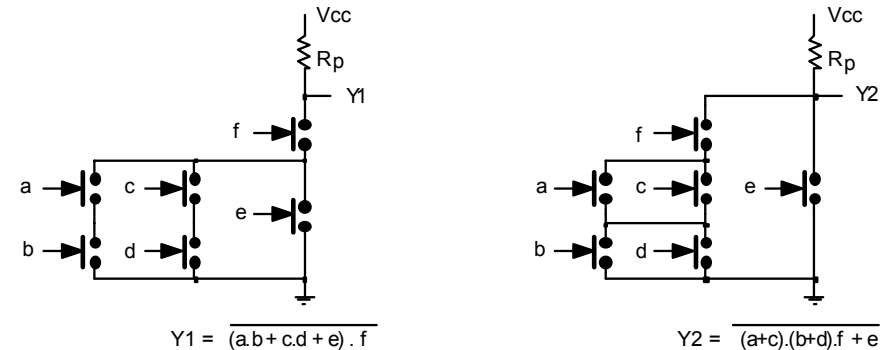
La resistencia de entrada de este inversor es la propia del interruptor, sumamente alta, mientras que la resistencia de salida es nula para salida 0 y R_p para salida 1; en ambos casos, al conectar varios inversores seguidos se cumple la condición de buen acople en tensión $R_i \gg R_o$.

La conexión de interruptores en serie da lugar, de acuerdo con el álgebra de conmutadores, a una operación "y", a la que sigue una inversión (debida a que el valor booleano transmitido por ellos es el 0); así se obtiene una puerta "y-negada" (Nand). De igual forma su conexión en paralelo configura una puerta "o", seguida de una inversión: puerta "o-negada" (Nor).



Puede comprobarse directamente la operación que realiza cada una de estas puertas construyendo su correspondiente «tabla de verdad».

Pero también pueden obtenerse puertas inversoras más complejas, aprovechando operaciones "y", conformadas por interruptores en serie y operaciones "o", con interruptores en paralelo.

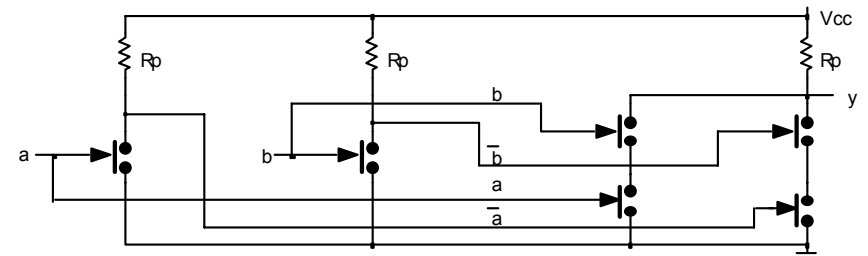


Las puertas resultantes son siempre inversoras: producen una inversión global sobre el conjunto de operaciones "y" y operaciones "o" configuradas por la forma de conexión de los interruptores; proceden de la estructura de un inversor y conservan tal carácter.

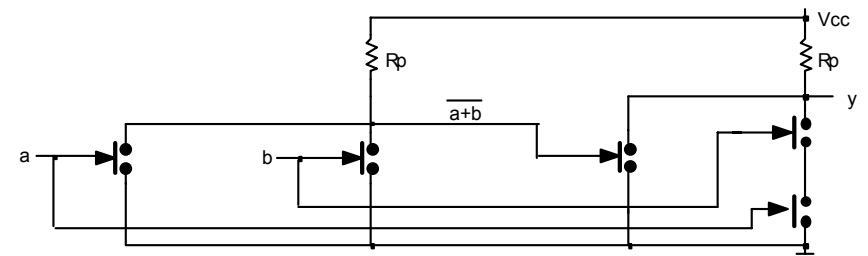
Es posible obtener una puerta "o-exclusiva" en la forma siguiente:

$$y = a \oplus b = \overline{a} \cdot b + a \cdot \overline{b} = \overline{\overline{a} \cdot b + a \cdot \overline{b}} = \overline{(a + b) \cdot (\overline{a} + \overline{b})} = \overline{a \cdot b + \overline{a} \cdot \overline{b}}$$

para cuya configuración necesitamos 6 interruptores (4 para la puerta general y dos para los inversores previos de a y de b):

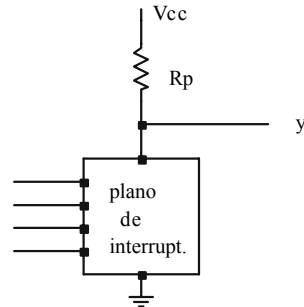


La siguiente transformación permite construir dicha operación con solamente 5 interruptores: $y = a \oplus b = a \cdot b + \overline{a} \cdot \overline{b} = \overline{a \cdot b + (a + b)}$.



En resumen, una puerta inversora está conformada por un «plano de interruptores», conectados en paralelo o en serie para configurar operaciones "o" y operaciones "y", respectivamente, y por una resistencia de polarización R_p :

- cuando el plano de interruptores conduce transmite el valor **0**:
por ello, la puerta es inversora;
- por defecto, R_p transmite el valor **1**, cuando los interruptores no conducen.



$$R_o ("1") = R_p$$

$$R_o ("0") = 0$$

$$R_i = \infty$$

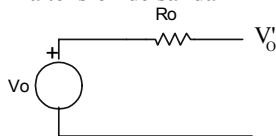
El buen acoplo en tensión queda garantizado por ser $R_i \approx \infty$; el consumo para salida **1** es nulo y para salida **0** es $I_{CC} = V_{CC} / R_p$.

7.5. Características ideales en una puerta lógica

Una puerta lógica ideal, para ofrecer un excelente acoplo en tensión, debe presentar una resistencia de entrada muy alta y una resistencia de salida baja.

$$INTERESA \quad R_i \sim \infty, \quad R_o \sim 0.$$

Además, valores bajos de la resistencia de salida favorecen en gran medida la «inmunidad frente al ruido», es decir, evitan que perturbaciones electromagnéticas de cualquier tipo afecten a la tensión de salida. El equivalente thevenin presenta la resistencia de salida en serie con la tensión de salida:



cuanto más pequeña es R_o menor será el efecto de las perturbaciones que actúen sobre el nudo de salida (se requiere mayor intensidad para producir una modificación de la tensión de este nudo). [Supongamos una perturbación con una potencia P , la variación de la tensión que producirá será $\Delta V_o = \sqrt{P \cdot R_o}$, tanto menor cuanto menor sea R_o .]

También interesa que el tiempo de propagación de la señal, o sea, el retraso físico que se da entre la entrada y la salida sea lo más pequeño posible. Obviamente el cambio de valor en la señal de entrada y el correspondiente cambio en la salida no son simultáneos sino que existe siempre un pequeño intervalo de tiempo entre ambas señales: este retraso recibe el nombre de tiempo de propagación t_p ; cuanto menor sea este tiempo de respuesta, mayor puede ser la velocidad de trabajo de la puerta lógica, mayor será el número de bits que puede procesar en un segundo.

$$INTERESA \quad t_p \sim 0.$$

Los tiempos de propagación son debidos, fundamentalmente, a efectos capacitivos en la conmutación de los interruptores y, en particular, a la capacidad equivalente que presentan las entradas de los mismos y que es preciso cargar o descargar al modificar su valor booleano. Una resistencia de salida baja colabora en la obtención de reducidos tiempos de propagación, ya que permite intensidades altas para efectuar la carga o descarga de la capacidad de entrada de la puerta siguiente.

Por otra parte, interesa que el consumo de las puertas sea lo menor posible y, si ello fuera factible, consumo nulo. El consumo se refleja en la magnitud de la fuente de alimentación que ha de proporcionar la tensión V_{CC} y en el gasto energético que produce el circuito; pero, también, en la disipación de calor en el propio circuito y, en su caso, en la necesidad de incluir sistemas que favorezcan tal disipación.

$$INTERESA \quad I_{CC} \sim 0.$$

En las puertas con interruptores la resistencia de entrada es prácticamente infinita y la de salida es nula cuando conducen los interruptores y R_p cuando no conducen:

$$R_i \approx \infty; \quad R_o(0) \approx 0; \quad R_o(1) = R_p.$$

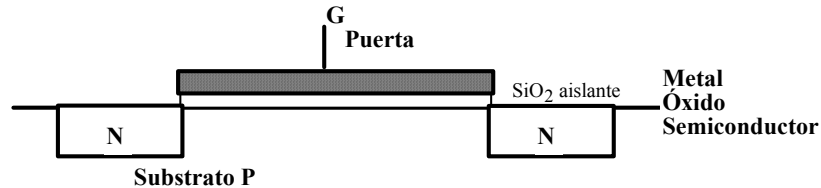
Ya de entrada existe buen acoplo en tensión y, además, en relación con la inmunidad frente al ruido y con la velocidad de trabajo (con los tiempos de propagación) interesa que la resistencia R_p sea de valores bajos. Pero el consumo cuando el plano de interruptores conduce (salida **0**) es V_{CC} / R_p y, a tal efecto, interesan valores altos de la resistencia R_p .

Por ello, la resistencia de polarización R_p ha de elegirse en un compromiso entre velocidad (+ inmunidad al ruido) y consumo. En la práctica, valores aceptables suelen estar entre los $500 \, \Omega$ y los $50 \, K\Omega$, optando por valores bajos ($\sim 500 \, \Omega$) cuando predomina el interés por la velocidad o por la inmunidad frente a perturbaciones y valores altos ($\sim 50 \, K\Omega$) cuando interesa sobre todo reducir el consumo.

7.6. El transistor MOS como interruptor

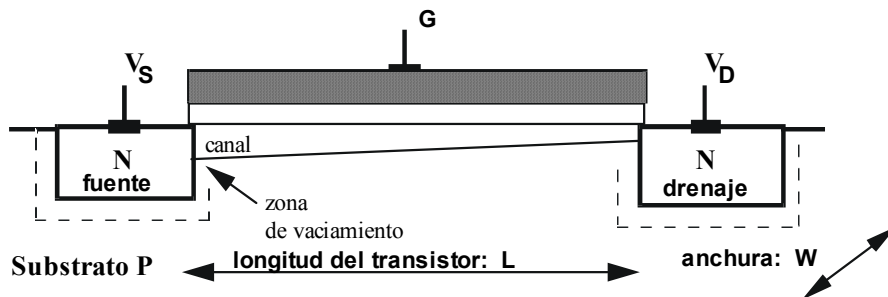
Nota: el capítulo T1 dedicado a “Semiconductores, unión PN y transistores MOS” puede resultar útil para quienes no hayan estudiado anteriormente la forma de actuar de tales transistores, así como para repasar el comportamiento razonado de semiconductores, diodos y transistores; el apartado T1.3. presenta en detalle un modelo operativo de los transistores MOS.

Un transistor NMOS está formado sobre un sustrato P por sendas difusiones N (Fuente **S** y Drenaje **D**) que actúan como contactos y por un «condensador» intermedio conformado por el sustrato, óxido de silicio aislante y un plano conductor que actúa como entrada de control (Puerta **G**).



Un transistor MOS constituye un excelente interruptor electrónico: cuando la tensión de entrada $V_i = V_G$ es suficientemente positiva la puerta atrae a los electrones libres en el sustrato (procedentes de los pares electrón-hueco propios del propio silicio como semiconductor) y forma un canal conductor entre fuente y drenaje:

$V_G > V_{TO}$ se forma canal $\Rightarrow$ el transistor conduce.

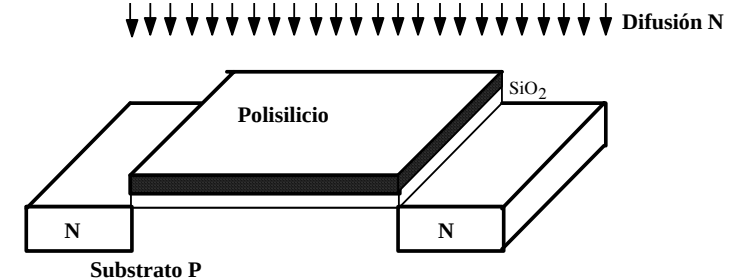


Si la tensión de puerta es claramente inferior al valor de la tensión umbral V_{TO} el canal no se forma y no hay paso de corriente entre las zonas de difusión (fuente y drenaje):

$V_G < V_{TO}$ no existe canal $\Rightarrow$ el transistor no conduce.

La resistencia del canal puede hacerse adecuadamente baja ($\sim 10 \Omega$) aumentando la anchura del transistor y la resistencia de entrada es muy alta (prácticamente infinita), con lo cual queda asegurado un buen acoplo en tensión. Ahora bien, la puerta **G** del transistor es de tipo capacitivo: presenta una capacidad de entrada que habrá que cargar o descargar en las conmutaciones, lo cual limita su velocidad de trabajo ($C_i = C_G \sim 10 \text{ pF}$).

El proceso tecnológico de fabricación MOS es conceptualmente muy simple y consta de un número reducido de etapas básicas (ver capítulo T2):



1. Substrato de silicio cristalino tipo P
2. Capa de óxido fino aislante SiO_2
3. Polisilicio N^{++} formando la puerta

Aunque la denominación MOS procede de la configuración en la zona de puerta «metal-óxido-semiconductor», actualmente el plano de puerta no se construye con metal sino con polisilicio, silicio policristalino fuertemente dopado cuya conductividad es próxima a la del metal.

4. Difusión N^{++} (fuente y drenaje)
5. Contactos metálicos

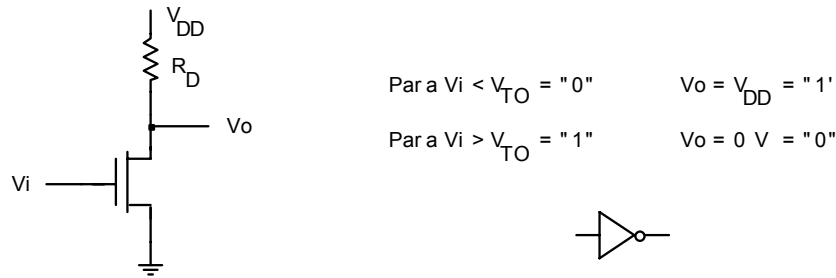
De esta forma, la puerta y las dos zonas de difusión N quedan autoalineadas: la difusión se realiza sobre todo el transistor pero no penetra debajo del polisilicio.

El sustrato P debe conectarse a la tensión más negativa del circuito (a 0 V), de forma que las uniones NP de las difusiones (y del canal) con el sustrato queden polarizadas en inverso, dando lugar a una «capa vacía» (sin portadores de carga) que aísla al conjunto del transistor respecto al sustrato. De esta forma los transistores MOS quedan «autoaislados» (separados del sustrato por una capa de vaciamiento).

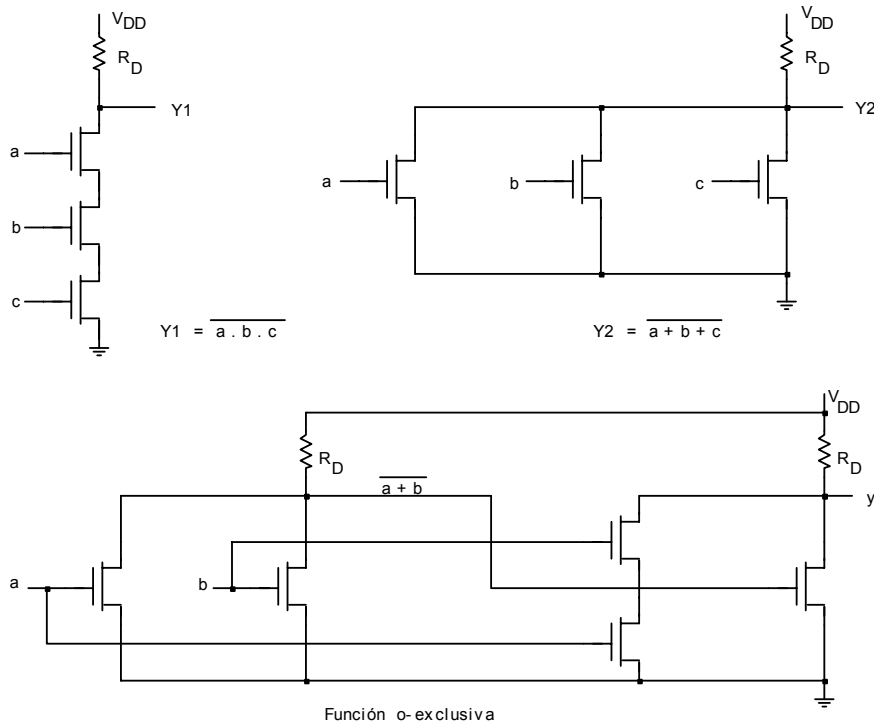
El tamaño del transistor MOS puede ser micrométrico (dimensiones del orden de una micra) y la integración de múltiples transistores en un circuito viene facilitada por las características de autoalineado y autoaislado propias del transistor MOS (ver T2.1).

Nota: el apartado T1.3. explica la formación de la capa de vaciamiento que aísla a los transistores MOS y, al final del apartado T2.1., se vuelve a insistir en dicho aislamiento y se explica el autoalineamiento que se produce entre polisilicio y difusiones del transistor MOS; por otra parte, el capítulo T2 describe en detalle los procesos de fabricación de los circuitos integrados con transistores MOS.

El esquema básico transistor NMOS - resistencia de polarización (del drenaje) R_D conforma un inversor booleano:



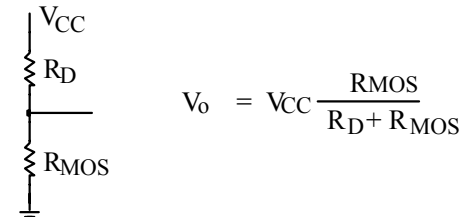
Utilizando los transistores NMOS como interruptores pueden construirse los diversos tipos de puertas inversoras; cada puerta estará conformada por tantos transistores NMOS como entradas tenga, más una resistencia de polarización R_D :



Las puertas NMOS ofrecen un buen acople en tensión, ya que la resistencia de entrada de estos transistores es prácticamente infinita. Las principales diferencias que presenta el transistor MOS respecto de un interruptor ideal son comentadas en la página siguiente.

a) La resistencia de paso de un transistor MOS, cuando conduce, es relativamente baja $R_{MOS} \sim 10\Omega$, pero no nula.

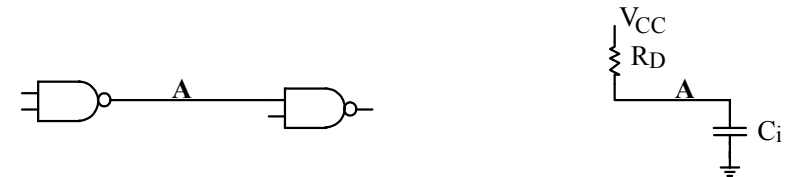
Por ello, cuando el plano de transistores conduce (salida **0**), la tensión de salida no es exactamente 0 V sino que se forma un divisor de tensión entre la resistencia de polarización R_D y la propia resistencia de los transistores en conducción R_{MOS} .



Si $R_D \gg R_{MOS}$ la tensión de salida del divisor de tensión será $V_o \ll V_{CC}$, es decir, $V_o \approx 0 \text{ V}$; para asegurar una tensión de salida **$V_o(0)$** próxima a 0 V es preciso que **$R_D \gg R_{MOS}$** (o sea, $R_D \gg 10 \Omega$).

b) La capacidad de entrada de los transistores MOS $C_i = C_G \sim 10\text{pF}$ limita su velocidad de trabajo.

En un «nudo booleano», es decir, en la conexión de la salida de una puerta a la entrada de otra (o de otras puertas) la capacidad de entrada de esta segunda se carga (en el paso de **0** a **1**) a través de R_D y se descarga (en la conmutación de **1** a **0**) a través de los transistores MOS R_{MOS} ; dado que $R_D \gg R_{MOS}$, el tiempo de carga es muy superior al de descarga y es el que limita la velocidad de trabajo.



El tiempo de propagación de la primera puerta, tiempo de paso de **0** a **1** en el nudo **A**, corresponderá al tiempo de carga del condensador C_i y será del orden de dos veces la constante de tiempo $t_p \approx 2 \cdot \tau = 2 \cdot R_D \cdot C_i$ (en 2τ se efectúa el 86% del proceso de carga).

En consecuencia, los valores que puede adoptar la resistencia de polarización R_D se encuentran acotados en la forma siguiente:

- de un lado, para asegurar la salida **0**, $R_D \gg R_{MOS} \sim 10\Omega$; además, el consumo en tal situación V_{CC}/R_D aconseja emplear valores altos de R_D ;
- en cambio, tanto el acople en tensión, como la inmunidad frente al ruido y la velocidad de trabajo precisan de valores bajos de R_D .

Como se indicaba anteriormente, valores de la resistencia de polarización R_D entre 0,5 K Ω y 50 K Ω suelen ser aceptables, optando por valores muy altos cuando lo prioritario es el consumo y tomando valores más bajos en el resto de los casos.

8 PUERTAS CON INTERRUPTORES COMPLEMENTARIOS LÓGICA CMOS

- 8.1. Puertas con interruptores inversos
- 8.2. El transistor PMOS
- 8.3. El inversor con transistores complementarios
- 8.4. Puertas lógicas CMOS
- 8.5. Familia HCMOS: características según catálogo

Los interruptores inversos se activan con valor booleano 0, de forma que conducen si su entrada es 0 y no lo hacen cuando reciben un 1. Con ellos es posible construir puertas inversoras (al igual que con los interruptores directos), pero, en este caso, el plano de interruptores se sitúa «arriba» para transmitir el valor 1.

Como las entradas se encuentran invertidas, la conexión de interruptores en serie da lugar a operaciones "o" y su conexión en paralelo produce operaciones "y", seguidas siempre de una inversión global. De esta forma los interruptores inversos conforman planos duales respecto a los interruptores directos, para el mismo tipo de puerta lógica.

El interés de los interruptores inversos reside en combinarlos con los directos para formar puertas con dos planos de interruptores (sin resistencia de polarización), cuyo consumo estático es nulo, ya que, en cada momento, solamente conduce uno de los planos. Tales puertas reciben el nombre de lógica «complementaria» y pueden construirse con transistores PMOS (para los interruptores inversos) y NMOS, dando lugar a la familia lógica CMOS (lógica de transistores MOS complementarios).

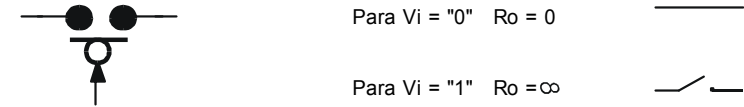
Las puertas CMOS están conformadas por un plano PMOS que transmite el valor 1 y otro NMOS que transmite el 0; cada entrada tiene un transistor en cada uno de los planos y las conexiones en ellos son duales (serie en un plano $\Leftrightarrow$ paralelo en el otro). Estas puertas son de tipo inversor y la combinación de conexiones serie-paralelo permite una gran diversidad de puertas.

Actualmente se utiliza la serie HCMOS (CMOS de alta velocidad); ésta presenta excelentes características en cuanto a resistencias de entrada y de salida (y, por tanto, en cuanto a acople en tensión e inmunidad frente al ruido), consumo estático nulo y tiempos de propagación del orden de los 10 nanosegundos.

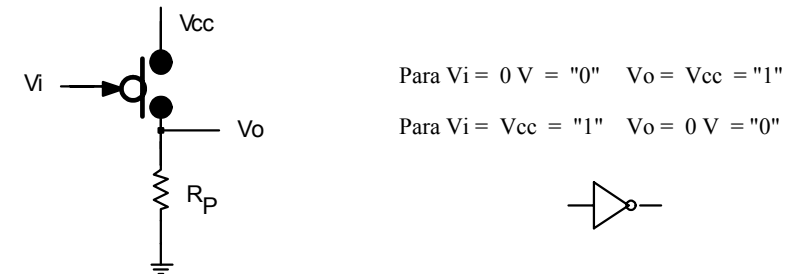
En este capítulo se consideran en detalle tales características, así como el consumo dinámico propio de la conmutación de las puertas. Para ello es necesario habituarse a la forma en que los fabricantes especifican las características en sus catálogos; éstos contienen la información concreta (con datos numéricos medidos) sobre el comportamiento real de los diversos circuitos digitales.

8.1. Puertas con interruptores inversos

Un interruptor inverso permite el paso de corriente eléctrica a su través cuando la tensión de la entrada de control corresponde al 0 y, en cambio, no conduce cuando dicha entrada recibe un 1. Como en el caso del interruptor directo, su resistencia de entrada es infinita, es decir, la intensidad que requiere la entrada para controlar el estado del interruptor es despreciable.



El circuito de la figura siguiente representa un inversor booleano configurado con un interruptor inverso: para $V_i = 0$ el interruptor conduce y transmite a la salida la tensión de alimentación V_{CC} , mientras que para $V_i = 1$ el interruptor no conduce y la resistencia de polarización R_P lleva la salida a 0.



El montaje anterior es inversor ya que el interruptor conduce cuando recibe un 0 y, entonces, lleva la salida a 1; la resistencia R_P transmite por defecto (cuando el interruptor no conduce) la tensión correspondiente a 0.

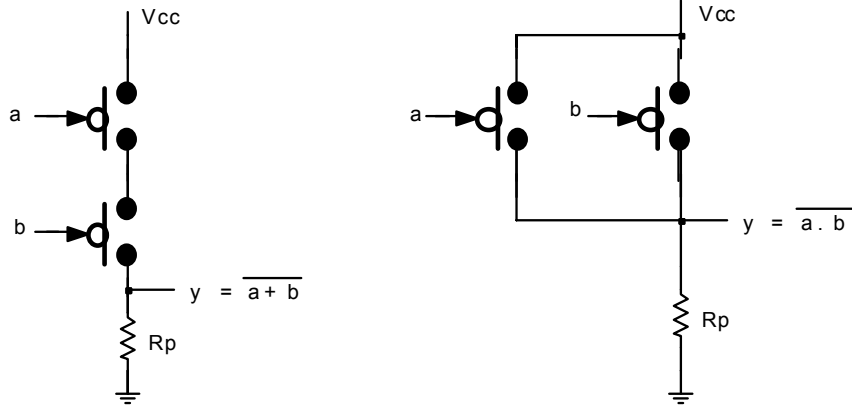
La resistencia de entrada de este inversor es la propia del interruptor, sumamente alta, mientras que la resistencia de salida es nula para salida 1 y R_P para salida 0. En ambos casos, al conectar varios inversores seguidos se cumple la condición de buen acople en tensión $R_i \gg R_o$.

Habida cuenta de que sus entradas se encuentran invertidas (es decir, son activas con valor **0**) la conexión de interruptores inversos en serie da lugar a una operación "**o-negada**" (*Nor*): una operación "**y**" cuyas entradas están negadas:

$$y = \bar{a} \cdot \bar{b} = \overline{a + b}$$

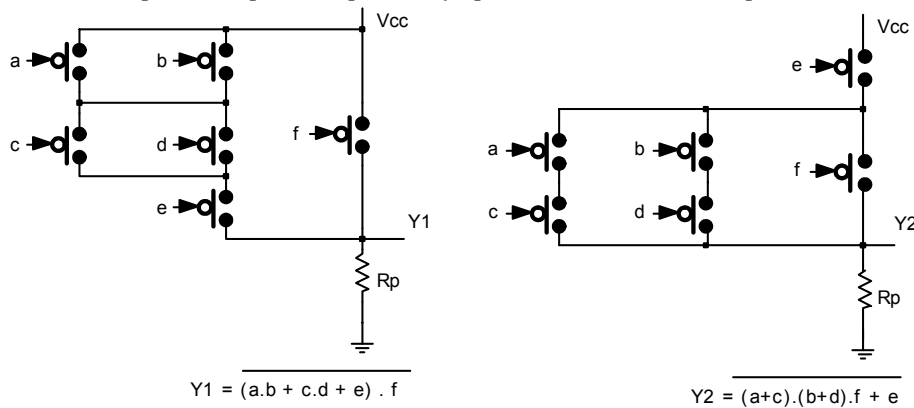
De igual forma, su conexión en paralelo configura una puerta "**y-negada**" (*Nand*):

$$y = \overline{a + b} = \bar{a} \cdot \bar{b}$$



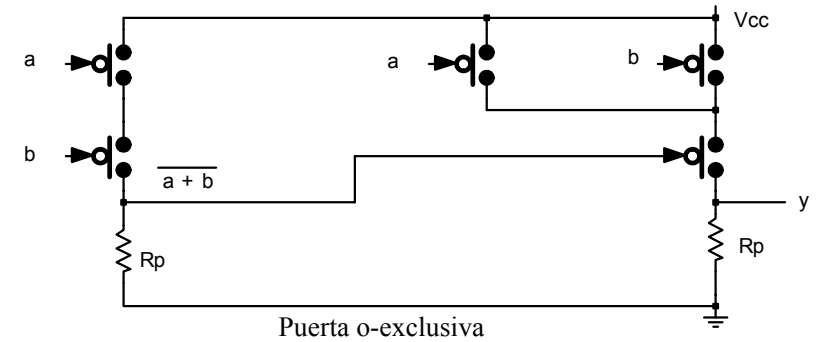
Puede comprobarse directamente la operación que realiza cada una de estas puertas construyendo su correspondiente «tabla de verdad».

Los interruptores inversos se comportan como duales respecto a los interruptores directos. Pueden obtenerse puertas inversoras complejas, aprovechando operaciones "**y**", conformadas por interruptores en paralelo, y operaciones "**o**", con interruptores en serie.



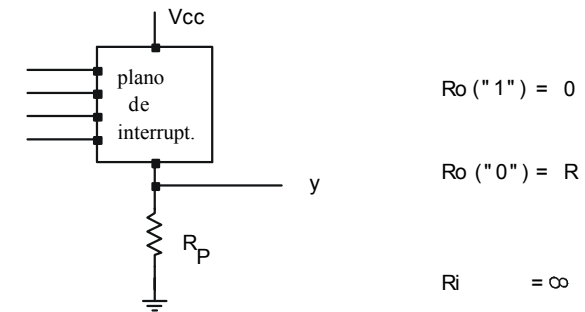
Las puertas resultantes son siempre inversoras: producen una inversión global sobre el conjunto de operaciones "**y**" (paralelo) y operaciones "**o**" (serie) configuradas por la forma de conexión de los interruptores; todas ellas proceden de la estructura de un inversor y conservan tal carácter.

La operación o-exclusiva: $y = a \oplus b = \bar{a} \cdot b + a \cdot \bar{b} = \overline{a \cdot b + \overline{a + b}}$ requiere 5 interruptores:



En resumen, una puerta con interruptores inversos estará conformada por un «plano de interruptores», conectados en serie o en paralelo para configurar operaciones "**o**" y operaciones "**y**", respectivamente, y por una resistencia de polarización R_p :

- cada interruptor conduce cuando su entrada recibe un **0** (por ello, la puerta es inversora)
- cuando el plano de interruptores conduce transmite el valor **1**
- por defecto R_p transmite el valor **0** (cuando el plano de interruptores no conduce).



$$R_o ("1") = 0$$

$$R_o ("0") = R$$

$$R_i = \infty$$

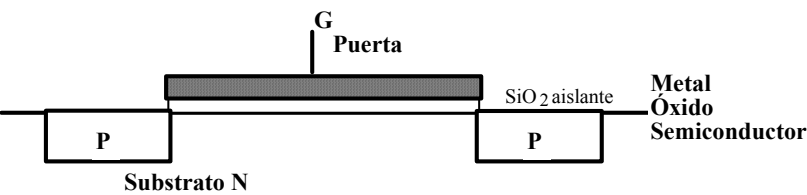
El buen acoplo en tensión queda garantizado por ser $R_i \approx \infty$.

El consumo para salida **0** es nulo y para salida **1** es $I_{CC} = V_{CC} / R_p$.

8.2. El transistor PMOS

Nota: recuérdese que el capítulo T1 puede resultar útil para repasar el comportamiento razonado de semiconductores, diodos y transistores MOS.

Así como el transistor NMOS es un buen interruptor directo, el transistor PMOS lo es como interruptor inverso. Un transistor PMOS está formado sobre un sustrato N por sendas difusiones P (Fuente S y Drenaje D) que actúan como contactos y por un «condensador» intermedio conformado por el sustrato, óxido de silicio aislante y un plano conductor que actúa como entrada de control (Puerta G).

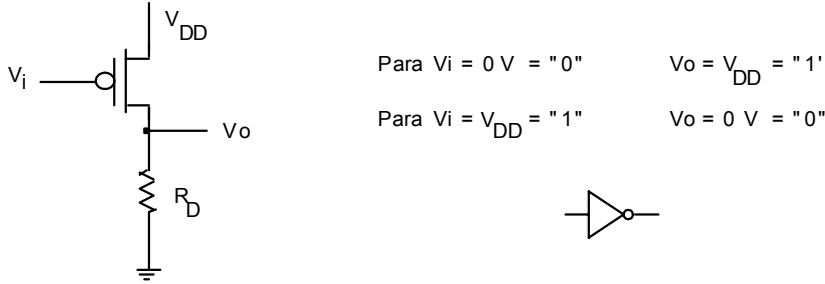


Cuando la tensión de entrada V_i es suficientemente negativa, la puerta atrae a los huecos libres en el sustrato (procedentes de los pares electrón-hueco propios del silicio como semiconductor) y forma un canal conductor entre fuente y drenaje; cuando la tensión de puerta es claramente inferior a la tensión umbral, no existe canal y el transistor no conduce.

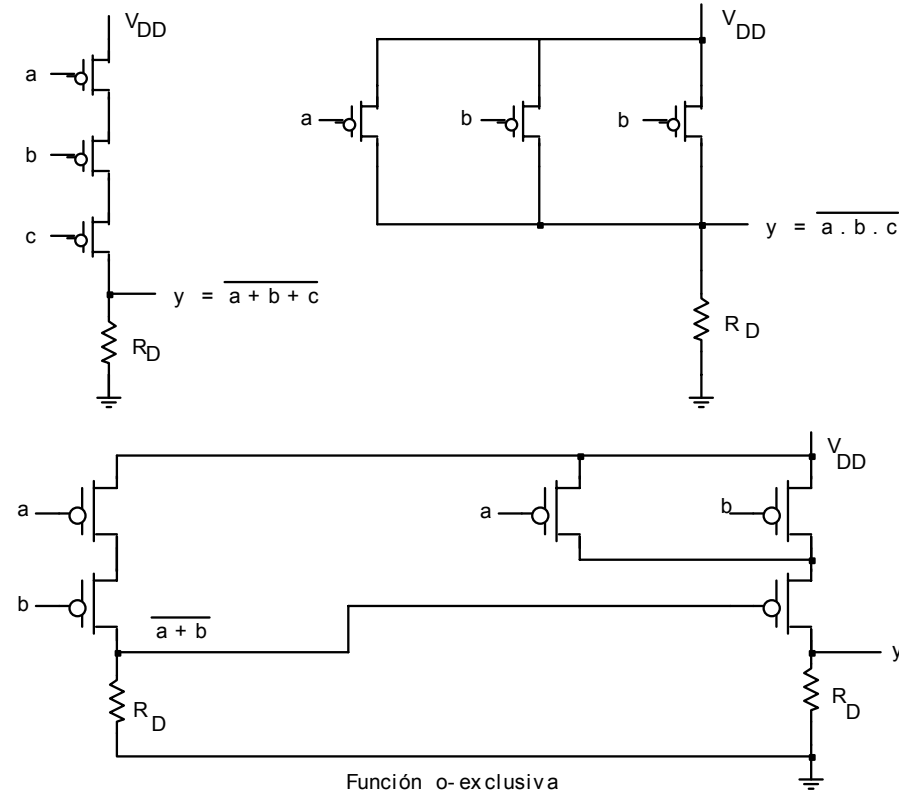
De esta forma, una tensión negativa suficientemente alta sobre el terminal de puerta da lugar, por atracción sobre los portadores minoritarios del sustrato, a un canal continuo entre las dos zonas de difusión P que permite el flujo de corriente entre ellas.

La resistencia del canal se hace convenientemente baja ($\sim 10 \Omega$) aumentando la anchura del transistor; la resistencia de entrada es prácticamente infinita, con lo cual queda asegurado un buen acoplo en tensión.

El esquema transistor PMOS - resistencia de polarización R_D conforma un inversor booleano; en este caso el transistor PMOS se encuentra «arriba», conectado a la tensión de polarización V_{DD} :



Utilizando los transistores PMOS como interruptores inversos pueden construirse los diversos tipos de puertas inversoras; tales puertas estarán formadas por tantos transistores PMOS como entradas tenga la puerta, más una resistencia de polarización R_D :



El tamaño de un transistor PMOS es análogo al del NMOS (dimensiones del orden de la micra) y asimismo conserva las características de autoalineado y autoaislado propias del transistor MOS, que facilitan la alta densidad de integración de múltiples transistores.

El transistor PMOS presenta tiempos de respuesta del orden del nanosegundo, una velocidad similar pero apreciablemente inferior a la del NMOS, ya que la movilidad de los huecos es netamente inferior a la de los electrones: el movimiento de un hueco requiere que un electrón ligado se desplace y lo ocupe, dejando un nuevo hueco en su enlace; obviamente, los electrones ligados presentan menor movilidad que los electrones libres.

8.3. El inversor con transistores complementarios

Las puertas con transistores MOS de un solo tipo, puertas NMOS o puertas PMOS, presentan características análogas:

- Su tamaño es muy reducido, gracias a las dimensiones micrométricas de los transistores y a su autoalineamiento y autoaislamiento: permiten muy alta densidad de integración.
- Son puertas inversoras: las NMOS porque los transistores transmiten el **0** y las PMOS porque las entradas actúan en forma negada.
- Ofrecen buen acoplo en tensión; su resistencia de entrada es elevada $R_i \sim \infty$.
- La resistencia de salida R_o es muy baja cuando el plano de transistores conduce (RMOS) y tiene el valor de la resistencia de polarización R_D cuando no lo hace.
- Cuando el plano de transistores conduce se forma un divisor de tensión entre su resistencia R_{MOS} y la de polarización R_D , siendo necesario que $R_D \gg R_{MOS}$ para que el valor de tensión de salida sea correcto.
- Valores bajos de la resistencia de salida (R_{MOS} , R_D) favorecen en gran medida la «inmunidad frente al ruido»: evitan que cualquier perturbación electromagnética afecte a la tensión de salida.
- Las entradas son de tipo capacitivo $C_i = C_G$, lo cual repercute en el tiempo de propagación de cada puerta ya que ha de cargar o descargar (a través de su resistencia de salida) la capacidad de entrada de la siguiente puerta (o puertas) conectada a ella.
- El consumo I_{CC} es nulo cuando el plano de transistores no conduce y V_{DD} / R_D en el otro caso. Por ello el consumo promedio es $V_{DD} / 2 \cdot R_D$, no nulo.

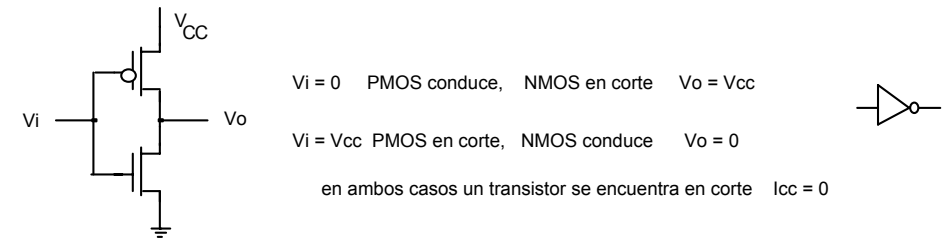
De forma que, en relación con la inmunidad frente al ruido y a la velocidad de trabajo, interesan valores muy bajos de R_D , pero en cuanto al consumo (y en relación con el divisor de tensión R_D / R_{MOS}), interesan valores altos de dicha resistencia.

Ahora bien, podemos resolver esta contradicción y mejorar grandemente las puertas en aquellas características que dependen de R_D utilizando conjuntamente los dos planos de interruptores: el plano de transistores NMOS para transmitir el **0** y el plano de transistores inversos PMOS para transmitir el **1**.

Ambos planos P y N no conducirán nunca a la vez y, consiguientemente, el consumo de estas puertas será nulo. Además, la resistencia de salida será muy baja para ambos valores booleanos, lo cual repercutirá muy favorablemente en la inmunidad frente al ruido y en la velocidad de trabajo de las puertas.

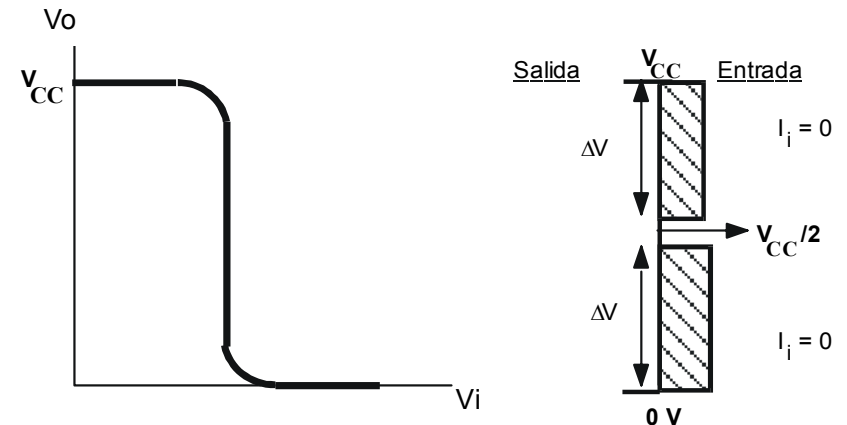
Tales puertas con transistores MOS de ambos tipos reciben el nombre de puertas CMOS (puertas de transistores MOS Complementarios).

El inversor CMOS estará configurado por un transistor de canal N y de otro de canal P; la unión de estos dos transistores complementarios confiere al inversor una gran simetría estructural y reduce su consumo estático a valores nulos.



El consumo estático del inversor CMOS es nulo para los dos valores booleanos, pues en ambos casos uno de los transistores se encuentra en corte; la intensidad en las entradas es también nula, lo cual supone un excelente acoplo en tensión.

Diseñando los transistores PMOS de forma que presenten la misma resistencia efectiva que los NMOS, la simetría propia del circuito hace que la función de transferencia sea aproximadamente simétrica, con la conmutación entre estados situada aproximadamente en la mitad de la tensión de la alimentación ($V_{CC}/2$), obteniéndose márgenes de ruido cercanos al 50 % de dicha tensión.



La resistencia de salida para el valor **1** es la que presenta el transistor PMOS y para el **0** la correspondiente al transistor NMOS, ambas muy bajas ($\sim 10 \Omega$) por el propio diseño de los transistores, adecuadamente anchos.

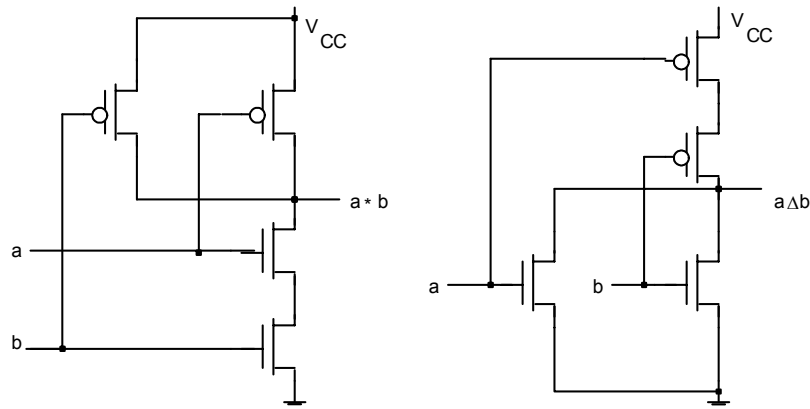
En la conmutación de un valor booleano al otro, una puerta deberá cargar o descargar las capacidades de entrada (condensadores puerta-canal) de las puertas conectadas a ella; ello incide en el tiempo de respuesta de la puerta que será del orden de la constante de tiempo asociada, es decir, del producto de la resistencia de salida de la puerta por la capacidad de carga sobre ella: $t_p \sim 2\tau = 2 \cdot R_o \cdot C_L$.

Asimismo, durante la conmutación de una puerta su intensidad de entrada no es nula sino que corresponde a la carga o descarga de las dos capacidades puerta-canal que conforman cada entrada; además, los transistores MOS presentan otros efectos capacitivos que se suman a la capacidad de puerta. Por ello, si bien el consumo estático (cuando la salida presenta uno de los dos valores booleanos **0/1**) es nulo, cuando la puerta conmuta de un valor al otro se produce un consumo no despreciable ya que han de cargarse o descargarse las diversas capacidades de sus transistores: consumo dinámico.

La evaluación del consumo dinámico de una puerta se realiza a través de su capacidad equivalente a efectos de disipación de potencia C_{pd} : $P = C_{pd} \cdot V_{CC}^2 \cdot f$ (expresión que corresponde a la energía consumida por un condensador en un proceso continuado de carga-descarga con una frecuencia f).

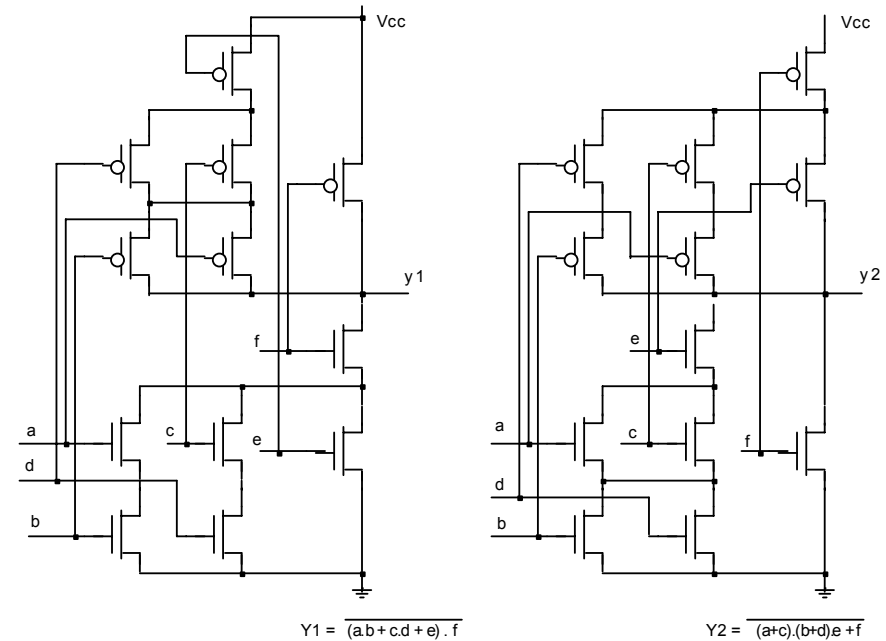
8.4. Puertas lógicas CMOS

A partir de la configuración del inversor CMOS, pueden construirse puertas lógicas "y-negada" (*Nand*) y "o-negada" (*Nor*), incluyendo un par de transistores complementarios (NMOS y PMOS) para cada entrada:

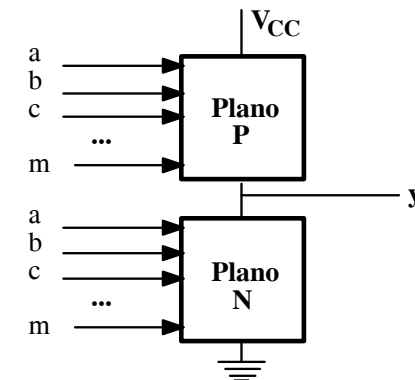


Estas puertas inversoras, conformadas con transistores «MOS complementarios» dan lugar a la familia lógica **CMOS**, con un «plano PMOS» superior y otro «plano NMOS» inferior, ambos con el mismo número de transistores y en conexión «dual» en un plano respecto al otro: si dos transistores de un plano se encuentran en serie, los correspondientes complementarios en el otro plano se encontrarán en paralelo.

Mediante tal conexión de transistores en serie y en paralelo pueden configurarse puertas complejas:



Las puertas lógicas CMOS se construyen mediante dos planos «duales» de transistores: **plano N** y **plano P**; a cada una de las entradas le corresponden sendos transistores, uno en cada plano, conectados de acuerdo con la dualidad serie-paralelo.



Todo transistor de canal N debe estar acompañado por un complementario de canal P: las combinaciones de los PMOS han de ser duales (paralelo ↔ serie) respecto a las que forman sus correspondientes NMOS, a fin de que la salida se encuentre siempre conectada a 0 o a V_{CC} a través de transistores activos.

Al conectar una entrada a un valor booleano **0/1**, uno de sus transistores se encontrará en corte y el otro conducirá; de esta forma todo camino de conducción entre los dos terminales de alimentación incluye siempre un transistor en corte, por lo cual el consumo en reposo es nulo.

La combinación en serie de transistores en el plano N corresponde a una operación "y" (conforme al álgebra de conmutadores), mientras que la conexión en paralelo da lugar a una operación "o"; ahora bien, como el plano N transmite «ceros» (está conectado a tensión 0 V), añade una inversión global en la salida.

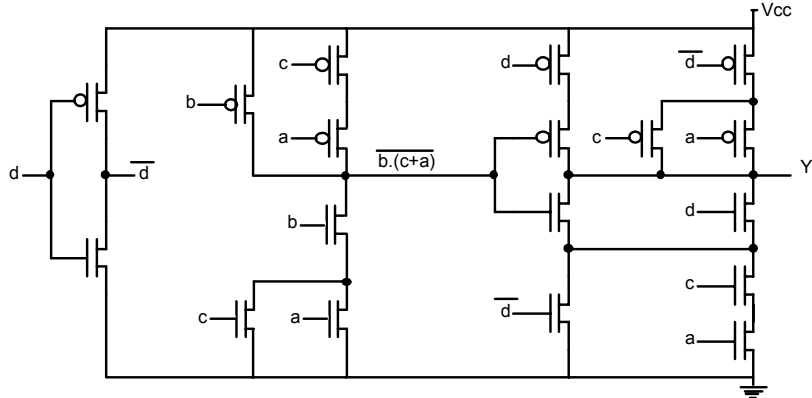
Los transistores del plano P transmiten «unos» (están conectados a tensión V_{CC}) pero conducen con entrada **0**, es decir, sus entradas actúan en forma negada; por ello, la conexión en serie corresponde a una operación "y" de variables negadas o, lo que es lo mismo, a una operación "o-negada" (según el teorema de Morgan), mientras que la combinación en paralelo da lugar a una operación "o" entre variables negadas, que pasa a ser una operación "y-negada". De manera que, en el plano P, la conexión en serie corresponde a una operación "o" y en paralelo a una operación "y" más una negación global, en ambos casos.

De esta forma puede configurarse cualquier función booleana que corresponda a una combinación de sumas y productos con una inversión (negación) global (que afecta al conjunto); en una misma puerta no pueden existir otras inversiones dentro de la expresión algebraica: cada una de ellas dará lugar una nueva puerta.

Ejemplo:

$$Y = \overline{d.c} + \overline{d.a} + \overline{d.c.b} + \overline{d.b.a} = \overline{d.(c + a)} + \overline{d.b.(c + a)} = \overline{(d + c.a) . (d + b.(c + a))}$$

función que requiere 3 puertas: $y1 = \overline{d}$, $y2 = \overline{b.(c + a)}$, $Y = \overline{(y1 + c.a) . (d + y2)}$



Se incluyen a continuación los datos del catálogo de Texas Instruments para el circuito integrado **74HC00** que contiene 4 puertas Nand y es el primero de la familia **HC** (CMOS de alta velocidad).

SN54HC00, SN74HC00

QUADRUPLÉ 2-INPUT POSITIVE-NAND GATES

SCLS181B – DECEMBER 1982 – REVISED MAY 1997

• Package Options Include Plastic Small-Outline (D), Thin Shrink Small-Outline (PW), and Ceramic Flat (W) Packages, Ceramic Chip Carriers (FK), and Standard Plastic (N) and Ceramic (J) 300-mil DIPs

description

These devices contain four independent 2-input NAND gates. They perform the Boolean function $Y = \overline{A \cdot B}$ or $Y = \overline{A + B}$ in positive logic.

The SN54HC00 is characterized for operation over the full military temperature range of -55°C to 125°C. The SN74HC00 is characterized for operation from -40°C to 85°C.

SN54HC00 ... J OR W PACKAGE

SN74HC00 ... D, N, OR PW PACKAGE

(TOP VIEW)

1A 1 14 V_{CC}

1B 2 13 4B

1Y 3 12 4A

2A 4 11 4Y

2B 5 10 3B

2Y 6 9 3A

GND 7 8 3Y

SN54HC00 ... FK PACKAGE

(TOP VIEW)

absolute maximum ratings over operating free-air temperature range†	
Supply voltage range, V _{CC}	-0.5 V to 7 V
Input clamp current, I _{IK} (V _I < 0 or V _I > V _{CC}) (see Note 1)	±20 mA
Output clamp current, I _{OK} (V _O < 0 or V _O > V _{CC}) (see Note 1)	±20 mA
Continuous output current, I _O (V _O = 0 to V _{CC})	±25 mA
Continuous current through V _{CC} or GND	±50 mA
Package thermal impedance, θ _{JA} (see Note 2): D package	127°C/W
N package	78°C/W
PW package	170°C/W
Storage temperature range, T _{stg}	-65°C to 150°C

† Stresses beyond those listed under "absolute maximum ratings" may cause permanent damage to the device. These are stress ratings only, and functional operation of the device at these or any other conditions beyond those indicated under "recommended operating conditions" is not implied. Exposure to absolute-maximum-rated conditions for extended periods may affect device reliability.

NOTES: 1. The input and output voltage ratings may be exceeded if the input and output current ratings are observed.
2. The package thermal impedance is calculated in accordance with JEDEC 51, except for through-hole packages, which use a trace length of zero.

		SN54HC00			SN74HC00			UNIT
		MIN	NOM	MAX	MIN	NOM	MAX	
V _{CC}	Supply voltage	2	5	6	2	5	6	V
V _{IH}	High-level input voltage	V _{CC} = 2 V		1.5	1.5		V	
		V _{CC} = 4.5 V		3.15	3.15			
		V _{CC} = 6 V		4.2	4.2			
V _{IL}	Low-level input voltage	V _{CC} = 2 V		0	0		V	
		V _{CC} = 4.5 V		0	1.35			
		V _{CC} = 6 V		0	1.8			
V _I	Input voltage	0	V _{CC}	0	V _{CC}			V
V _O	Output voltage	0	V _{CC}	0	V _{CC}			V
t _t	Input transition (rise and fall) time	V _{CC} = 2 V		0	1000		ns	
		V _{CC} = 4.5 V		0	500			
		V _{CC} = 6 V		0	400			
T _A	Operating free-air temperature	-55	125	-40	85			°C

SN54HC00, SN74HC00
QUADRUPLE 2-INPUT POSITIVE-NAND GATES

SCLS181B - DECEMBER 1982 - REVISED MAY 1997

electrical characteristics over recommended operating free-air temperature range (unless otherwise noted)

PARAMETER	TEST CONDITIONS	VCC	TA = 25°C			SN54HC00		SN74HC00		UNIT
			MIN	TYP	MAX	MIN	MAX	MIN	MAX	
VOH	VI = VIH or VIL	IOH = -20 µA	2 V	1.9	1.998	1.9		1.9		V
			4.5 V	4.4	4.499	4.4		4.4		
			6 V	5.9	5.999	5.9		5.9		
		IOH = -4 mA	4.5 V	3.98	4.3	3.7		3.84		
			6 V	5.48	5.8	5.2		5.34		
VOL	VI = VIH or VIL	IOL = 20 µA	2 V		0.002	0.1		0.1		V
			4.5 V		0.001	0.1		0.1		
			6 V		0.001	0.1		0.1		
		IOL = 4 mA	4.5 V		0.17	0.26		0.4	0.33	
			6 V		0.15	0.26		0.4	0.33	
II	VI = VCC or 0		6 V		±0.1	±100		±1000	±1000	nA
ICC	VI = VCC or 0, IO = 0		6 V			2		40	20	µA
CI			2 V to 6 V			3	10	10	10	pF

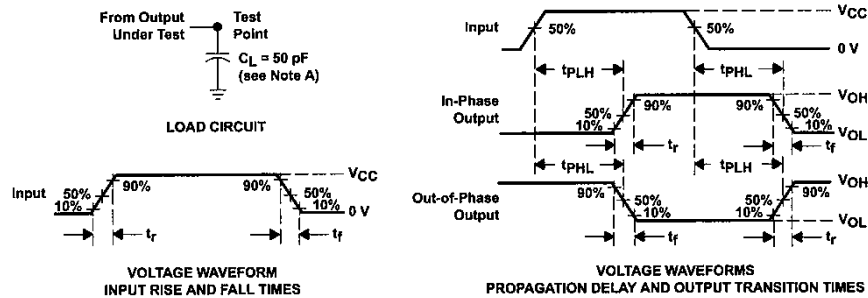
switching characteristics over recommended operating free-air temperature range, CL = 50 pF (unless otherwise noted) (see Figure 1)

PARAMETER	FROM (INPUT)	TO (OUTPUT)	VCC	TA = 25°C			SN54HC00		SN74HC00		UNIT
				MIN	TYP	MAX	MIN	MAX	MIN	MAX	
tpd	A or B	Y	2 V		45	90		135		115	ns
			4.5 V		9	18		27		23	
			6 V		8	15		23		20	
t _t		Y	2 V		38	75		110		95	ns
			4.5 V		8	15		22		19	
			6 V		6	13		19		16	

operating characteristics, TA = 25°C

PARAMETER		TEST CONDITIONS	TYP	UNIT
C _{pd}	Power dissipation capacitance per gate	No load	20	pF

PARAMETER MEASUREMENT INFORMATION



NOTES: A. CL includes probe and test-fixture capacitance.
B. Phase relationships between waveforms were chosen arbitrarily. All input pulses are supplied by generators having the following characteristics: PRR ≤ 1 MHz, ZO = 50 Ω, tr = 6 ns, tf = 6 ns.
C. The outputs are measured one at a time with one input transition per measurement.
D. tpLH and tpHL are the same as tpd.

Las tablas anteriores incluyen los datos de la serie 74HC y, también, los de la serie 54HC; la primera de ellas es la serie «normal», que se utiliza habitualmente.

La serie 54 está preparada para un intervalo de temperaturas de funcionamiento más amplio (de -55 a 125°C, frente al intervalo de -40 a 85°C de la serie normal) y, para ello, su encapsulado es cerámico (en lugar de plástico); suele nombrarse con el calificativo de «militar» y, obviamente, su coste es superior al de la serie «normal».

Conforme a estos datos, la serie **74HC** CMOS de «alta velocidad» presenta las siguientes características funcionales:

• Temperaturas

La temperatura de funcionamiento abarca el intervalo entre -40°C y 85°C.

La temperatura de almacenamiento va de -65° a 150°C.

• Tensión de alimentación

Pueden utilizarse tensiones de alimentación entre 2 y 6 voltios.

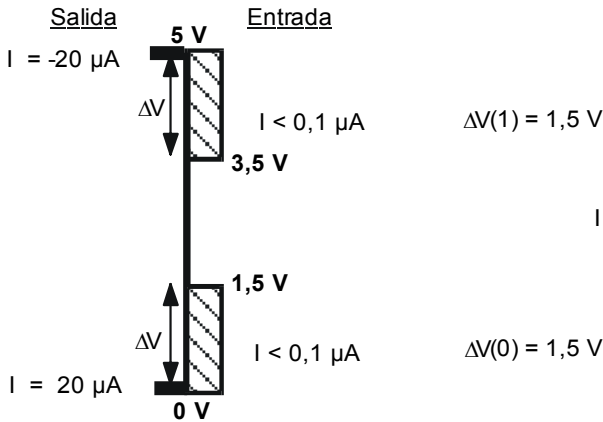
• Tensiones e intensidades: VO-Vi, IO, Ii.

Para referirlos a la tensión de alimentación típica de 5 V se han tomado los datos de tensiones de catálogo para 4,5 V (a 25°C) aumentados en un 10%:

VO_L = 0 V IO_L = 20 µA Vi_{Lmax} = 1,5 V

Ii < 0,1 µA

VO_H = 5 V IO_H = -20 µA Vi_{Hmin} = 3,5 V



- *Inmunidad frente al ruido*

El ruido es una perturbación que actúa sobre los nudos booleanos pudiendo modificar su valor **1/0**: los valores booleanos están presentes en la salida de cada puerta y sobre dicha salida se encuentra conectada la entrada o entradas de las puertas siguientes.

En la figura anterior (diagrama salida – entrada), se aprecia el margen que hay respecto al ruido (indicado como ΔV): si la salida de 5 V (**1** booleano) es afectada por un ruido que causa una variación de tensión inferior a 1,5 V, la entrada siguiente recibirá una tensión superior a 3,5 V y seguirá entendiéndose como valor **1**; de igual forma si la salida de 0 V (**0** booleano) soporta un ruido menor que 1,5 V, la entrada siguiente recibirá una tensión inferior a 1,5 V y seguirá entendiéndose como valor **0**.

El margen de inmunidad frente al ruido en tensión es de 1,5 V para cada uno de los valores booleanos:

$$\Delta V(0) = 1,5 - 0 = 1,5 \text{ V} \qquad \Delta V(1) = 5 - 3,5 = 1,5 \text{ V.}$$

Ahora bien, el ruido es una potencia parásita (energía actuante por unidad de tiempo) que perturba el sistema electrónico: el margen de ruido en potencia resulta mucho más representativo de la inmunidad frente al ruido que el margen en tensión. Para calcularlo es preciso conocer el valor de la resistencia de salida, ya que el efecto del ruido depende de la resistencia sobre la que actúa: la resistencia presente en un nudo booleano corresponde al paralelo de la resistencia R_o de salida de la primera puerta con las de entrada R_i de las siguientes y, en dicho paralelo, predomina la resistencia de salida R_o por ser de valor mucho menor que las de entrada.

$$P = V \cdot I = V \cdot V/R = V^2/R \qquad \Delta P = (\Delta V)^2 / R_o.$$

El valor de la resistencia de salida puede deducirse de las variaciones que se producen en la tensión de salida cuando se requiere intensidad de ella: para alimentación de 4,5 V e intensidad de salida de 4 mA, las tensiones de salida son $V_{oL} = 0,17 \text{ V}$ y $V_{oH} = 4,3 \text{ V}$, lo cual supone una caída de tensión en la salida de aproximadamente 0,2 V en ambos casos.

$$R_o \approx 0,2 \text{ V} / 4 \text{ mA} \sim 50 \Omega$$

El margen de ruido en potencia ($\Delta P = (\Delta V)^2 / R_o$) es análogo para ambos valores booleanos:

$$\Delta P \approx 1,5^2 / 50 \approx 50 \text{ mW.}$$

Como valores de referencia, un margen de 20 mW resulta suficiente en ambiente «normal» (viviendas o lugares públicos) mientras que en un ambiente «industrial» (en donde motores, máquinas y el propio suministro de energía eléctrica generan mayor ruido ambiental) se precisan márgenes superiores a 100 mW para circuitos digitales sin apantallamiento.

- *Velocidad de trabajo*

Los tiempos de propagación típicos son de 9 ns ($V_{CC} = 4,5 \text{ V}$) para una carga capacitiva de 50 pF a 25°C de temperatura. Ello requiere un intervalo para cada bit de, al menos, 18 ns (para permitir tiempo suficiente para la formación del bit y una mínima permanencia estable posterior); en circuitos con reloj, su período mínimo (2 bits: un **1** seguido de un **0**) deberá ser de 36 ns, lo cual permite una frecuencia máxima de reloj de 27 MHz.

- *Consumo*

El consumo estático es despreciable: inferior a 2 μA (a 25°C).

El consumo dinámico depende de la tensión de alimentación y de la frecuencia de conmutación; se evalúa a través de la capacidad equivalente a efectos de disipación de potencia: $C_{pd} \sim 20 \text{ pF}$ a 25°C.

$$P = C_{pd} \cdot V_{CC}^2 \cdot f = 0,5 \text{ mW/MHz para } V_{CC} = 5 \text{ V} \qquad I = 0,1 \text{ mA/MHz}$$

de forma que, para una alimentación de 5 voltios y una onda cuadrada de 10 MHz, el consumo es de 5 mW, que corresponde a una intensidad de 1 mA.

- *Conectividad*

La capacidad de carga, es decir, el número de entradas que puede soportar una salida (*fan-out*) de una puerta CMOS no puede deducirse del cociente entre intensidades de salida y entrada (dicho cociente presenta valores muy altos), ya que al conectar varias puertas a la salida de otra el problema no es de intensidad sino de velocidad: es preciso cargar y descargar las capacidades presentes en dichas entradas y ello limita la velocidad.

Por ello, el *fan-out* debe referirse a la limitación en cuanto a capacidad máxima de carga admisible para una determinada velocidad de conmutación; es decir, la capacidad de carga para la cual han sido medidos los tiempos de propagación (habitualmente una capacidad de 50 pF).

El valor de la capacidad en una entrada C_i es de unos 3 pF; el número máximo de puertas que podrán conectarse a la salida de otra (para preservar su velocidad de trabajo) será de $50 \text{ pF} / 3 \text{ pF} \approx 16$.

9 TECNOLOGÍA CMOS. DIVERSIDAD DE CONFIGURACIONES

- 9.1. Puertas complementarias
- 9.2. Tipos de salidas y de entradas
- 9.3. Puertas de transmisión
- 9.4. Realización CMOS de las estructuras matriciales
- 9.5. Dispositivos programables

La tecnología CMOS, introducida en el capítulo anterior, ha permitido la construcción de puertas booleanas con características cuasi-ideales, pues su resistencia de entrada es prácticamente infinita, la de salida muy baja y el consumo estático es nulo. Además, el juego de conexiones serie-paralelo permite una gran diversidad de puertas, combinando operaciones suma y producto, pero siempre de tipo inversor (con una inversión global sobre el conjunto de operaciones).

Las salidas de estas puertas pueden ser modificadas para incorporar un estado de desconexión (alta impedancia) o para manejar una tensión o una intensidad de salida relativamente altas (drenaje común); por su parte, las entradas pueden ser dotadas de histéresis (dos tensiones de conmutación) para evitar «rebotes».

Pero, además de las puertas complementarias, la tecnología CMOS ofrece otros tipos de puertas que resultan muy útiles para algunas aplicaciones específicas:

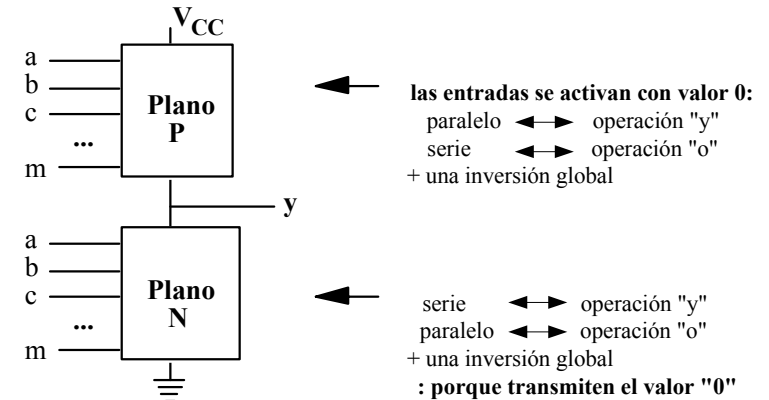
- Puertas de transmisión que actúan como simples interruptores (permitiendo o interrumpiendo el paso del valor booleano presente en su entrada); con ellas pueden construirse multiplexores y biestables, de dimensiones y tiempos de propagación más reducidos que los conformados con puertas inversoras;
- Puertas *Nor* con un solo plano de transistores, el NMOS (junto con un transistor PMOS único, como resistencia de polarización), que pueden tener muchas entradas y, por ello, son adecuadas para construir estructuras matriciales (decodificadores, multiplexores, ROM, PAL, PLA, ...).

Las estructuras matriciales se transforman en programables cuando la conexión de los transistores del plano N se hace a través de «fusibles». La introducción de transistores de doble puerta, la interior de ellas aislada, es otra forma (menos «destruictiva») de anular su conexión; la acumulación de carga eléctrica negativa en la puerta aislada eleva su tensión umbral e impide que el transistor pueda conducir (es como si no estuviera presente), con la ventaja de que esta programación puede ser «borrada», descargando dicha puerta aislada. Las denominaciones EPROM, EEPROM y FLASH corresponden a tres etapas sucesivas en el desarrollo de este tipo de transistores de doble puerta, con tres tipos diferentes de borrado.

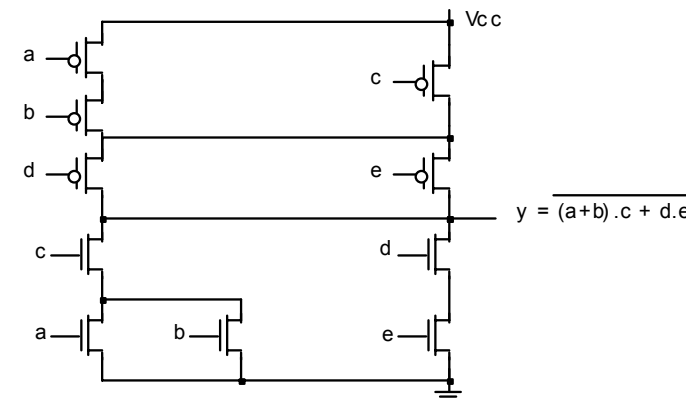
En suma, la aportación de la tecnología CMOS a los circuitos digitales no se limita a su propio tipo de puertas inversoras sino que ofrece una amplia gama de opciones con diferentes prestaciones y campos de aplicación.

9.1. Puertas complementarias

Las puertas lógicas CMOS se construyen mediante dos planos «duales» de transistores: **plano N** y **plano P**; a cada una de las entradas le corresponden sendos transistores, uno en el plano N y otro en el plano P, conectados de acuerdo con la dualidad serie-paralelo.



Consideremos la puerta representada en la siguiente figura:



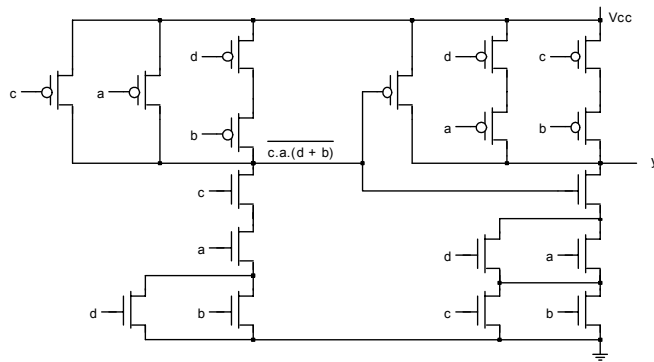
- en su plano N, la configuración de transistores corresponde (de acuerdo con el álgebra de conmutadores) a la función $(a + b).c + d.e$, a la cual debe añadirse una negación global ya que dicho plano transmite el valor 0; de forma que la función conformada por dicho plano es: $y = (a + b).c + d.e$
- en cuanto al plano P, transmite el valor 1 pero sus transistores conducen cuando su entrada es 0, es decir, corresponden a variables negadas ($\bar{a}$, $\bar{b}$, $\bar{c}$, $\bar{d}$, $\bar{e}$) y su configuración (de acuerdo con el álgebra de conmutadores) produce la función $(\bar{a} . \bar{b} + \bar{c}) . (\bar{d} + \bar{e})$; de manera que la función conformada por dicho plano es la misma que la del plano N: $y = (\bar{a} . \bar{b} + \bar{c}) . (\bar{d} + \bar{e}) = (a + b).c + d.e$

La combinación serie-paralelo de transistores permite una amplia diversidad de puertas: puede configurarse cualquier función booleana que corresponda a una serie de sumas y productos con una inversión global; si dentro de la expresión algebraica aparecen otras inversiones, cada una de ellas da lugar a una puerta adicional.

Consideremos la función del apartado 5.1., en el que se estudió su construcción de cuatro formas conceptualmente diferentes; la configuración de esta función con puertas inversoras CMOS, utilizando el menor número de transistores posible, conduce a la siguiente expresión:

$$y = \overline{d}.\overline{a} + \overline{c}.\overline{b} + c.a.(d+b) = \overline{\overline{d}.\overline{a} + \overline{c}.\overline{b} + c.a.(d+b)}$$

$$= \overline{(d+a).(c+b).(c.a.(d+b))}$$



Esta forma de construcción CMOS es la que se utiliza al diseñar un circuito integrado de aplicación específica (ASIC); la elección de las puertas inversoras más apropiadas para cada función concreta permite minimizar el área de integración de las mismas y disminuir sus tiempos de propagación.

9.1.1. Características de las puertas complementarias

Puertas compuestas por sólo transistores MOS

El reducido tamaño de los transistores MOS, su autoaislamiento (que facilita su integración contigua) y la ausencia de resistencias u otros componentes hacen que las puertas CMOS necesiten una área de integración muy reducida.

Conformadas por dos planos de transistores

Cada entrada aporta dos transistores (un PMOS y un NMOS) y las conexiones en cada plano son duales (serie ↔ paralelo) respecto al otro, lo cual supone una cierta complejidad en las líneas de conexión y una ocupación de área que (siendo de por sí muy reducida) es mayor que la necesaria en las tecnologías que utilizan un solo transistor por entrada (NMOS).

Pero con ello se consigue que en cada situación booleana conduzca solamente uno de los dos planos, lo cual supone un consumo estático nulo.

Permiten una amplia diversidad de puertas

El juego de conexiones serie-paralelo permite configurar funciones complejas y muy diversas en una misma puerta, con la limitación de que la expresión de la función tenga una negación global sobre el conjunto de operaciones booleanas (puerta inversora) y de que tal negación sea la única que aparezca: cada negación da lugar a una puerta adicional.

Las entradas son de tipo capacitivo

La resistencia de entrada es muy alta (> 10 MΩ), lo cual asegura un buen acoplo en tensión. Además, los transistores MOS presentan una capacidad de puerta que es preciso cargar o descargar en la conmutación cuando cambia el valor booleano presente en la correspondiente entrada; tal transitorio de carga o descarga:

- da lugar a tiempos de conmutación que limitan la velocidad de trabajo de la puerta
- limita, asimismo, el *fan-out* de la puerta anterior, es decir, el número de entradas de otras puertas que pueden conectarse sobre una salida (pues tal número condiciona la velocidad de trabajo)
- requiere un aporte puntual de intensidad durante la conmutación, que da lugar a un consumo dinámico proporcional a la frecuencia de las conmutaciones
- genera «ruido» sobre las líneas de alimentación como consecuencia del «pulso» de intensidad necesario para la conmutación.

El hecho de que la entrada sea capacitiva (un diminuto condensador) y que el valor de dicha capacidad sea muy reducido hace que una mínima aportación de carga eléctrica sobre el terminal de puerta genere altas tensiones ($\Delta V = Q/C$), con el riesgo de perforación del dieléctrico (de la delgadísima capa de óxido de silicio que forma la puerta). Lo cual constituye un problema de tipo práctico, relativo a la manipulación de los circuitos integrados MOS: la perforación puede producirse por la propia carga estática acumulada en el cuerpo de quien los utiliza.

Por ello, es necesario adoptar una serie de precauciones en el manejo y almacenamiento de los circuitos integrados MOS, evitando en lo posible el roce con los terminales del integrado; los circuitos deben conservarse en las habituales «tiras de plástico» o en almohadillas antiestáticas; los puestos de trabajo dedicados al montaje de placas con circuitos integrados MOS deben tener un diseño apropiado para eliminar (mediante adecuadas «tomas de tierra») toda posible carga en los instrumentos y en las personas.

La salida de cada puerta es resistiva

Cada plano de transistores, cuando conduce, presenta una resistencia que corresponde a la zona lineal u óhmica de sus transistores; tal resistencia depende de las dimensiones de los transistores (disminuyendo en proporción inversa al aumento de su anchura).

La resistencia de salida de una puerta afecta a:

- la intensidad suministrable por la puerta
- los procesos de conmutación (carga y descarga de las capacidades de entrada de las puertas siguientes) y, en consecuencia, a los tiempos de propagación y a la velocidad de trabajo de la puerta
- la inmunidad frente al «ruido» en términos de potencia.

Se presentan varios transistores en serie

La presencia de transistores en serie supone el correspondiente aumento de la resistencia de salida de la puerta, lo cual repercute, de acuerdo con el apartado anterior, en la intensidad suministrable por la puerta, en los tiempos de propagación y la velocidad de trabajo y en la inmunidad frente al «ruido» en términos de potencia.

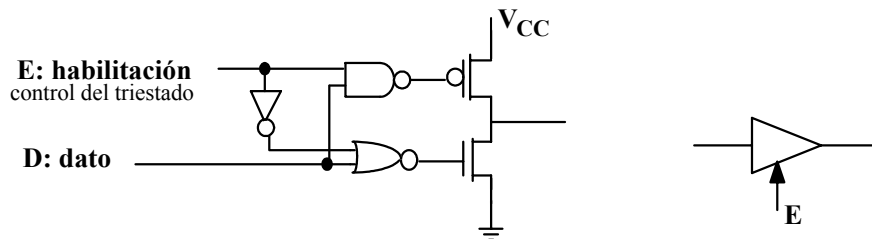
Al aumentar el número de transistores en serie las características de una puerta CMOS empeoran en gran medida: disminuye la intensidad disponible en la salida y el margen de ruido en potencia y aumentan los tiempos de propagación. Además, este efecto no es meramente lineal sino que se ve incrementado por el desplazamiento de la tensión de fuente (en el caso de transistores NMOS el segundo de la serie no tiene su terminal de fuente a 0 V sino conectado al drenaje del primero de ellos, de manera que $V_{GS} < V_G$ y así sucesivamente,...).

Por ello, conviene limitar el número de entradas de las puertas CMOS de forma que el número de transistores conectados en serie no sea superior a 5 o 6 en ninguno de los dos planos (el efecto es aún peor en el plano P que en el N).

9.2. Tipos de salidas y de entradas

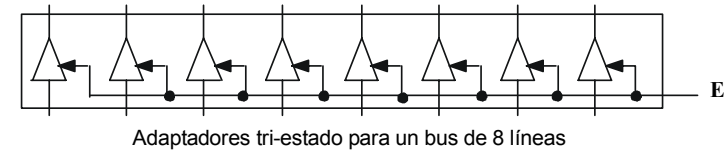
En el apartado 4.2. se introdujo la situación de *desconexión* como un tercer estado de las salidas de las puertas booleanas que permite conectar múltiples salidas en paralelo, con tal de que, en cada momento, actúe una de ellas y las demás se encuentren en *alta impedancia*: el resultado es el multiplexado de las salidas y la selección de una de ellas.

Este tipo de salidas se denomina *tri-estado* y puede conseguirse mediante adaptadores como el representado en la siguiente figura:



Cuando la entrada de habilitación **E** tiene valor nulo, los dos transistores de salida se encuentran en corte, mientras que para **E** = 1 la salida adopta el mismo valor booleano que la entrada **D**.

Este tipo de adaptador tri-estado puede ser incorporado en la salida de una puerta booleana o en las salidas de un bloque digital para incluir la opción de salida en alta impedancia (desconexión); asimismo, un conjunto de **n** adaptadores con una habilitación común configuran un bloque digital típico: el adaptador para **n** líneas o adaptador tri-estado de bus (*buffer tri-estate*).



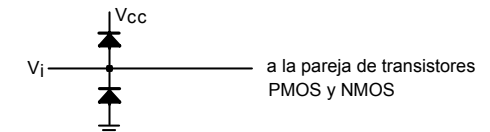
Otro tipo particular de salida elimina el plano P y deja la puerta incompleta, solamente con el plano NMOS: tales salidas se denominan de *drenaje abierto* (aunque es más frecuente el uso del término *colector abierto* que procede del mismo tipo de salidas en la familia TTL). Estas puertas deben completarse exteriormente mediante una resistencia de polarización **RC** conectada a la alimentación **VCC**.

Ahora bien, la resistencia de polarización **RC** puede conectarse a una tensión diferente de la de alimentación del circuito integrado, modificándose así el valor de tensión correspondiente al 1 booleano, lo cual es útil para efectuar el acoplo con otras familias lógicas o con otros componentes que empleen niveles de tensión diferentes.

Además, en los inversores de *drenaje abierto*, la existencia de un único transistor permite fabricarlos con mayor capacidad en tensión y en intensidad, configurando adaptadores que permiten conectar directamente el inversor a dispositivos que requieren mayores valores de tensión o intensidad, por ejemplo, visualizadores, pequeños relés, etc. Tales adaptadores suelen admitir hasta 15 ó 30 V de tensión sobre el transistor de salida (cuando se encuentra en corte) e intensidades del orden de 50 mA (cuando conduce).

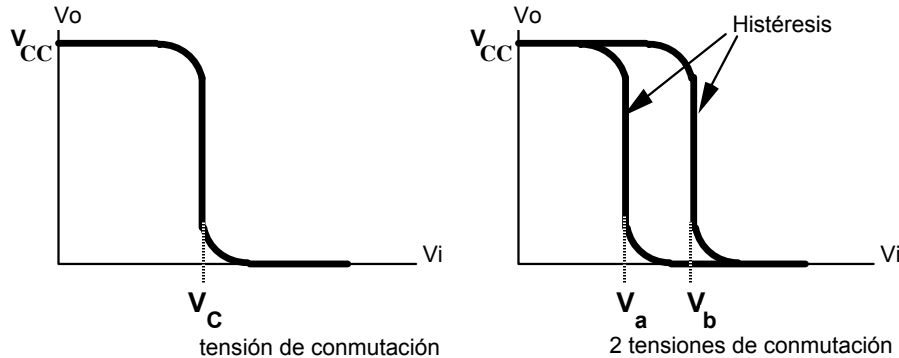
Las salidas de varias puertas de drenaje abierto pueden unirse sobre la misma resistencia de polarización **RC** configurando una operación "y" entre las mismas (siempre que una de las salidas se encuentra a 0, su transistor conduce y la salida global es 0): operación "y" cableada.

Las entradas de los circuitos integrados CMOS están protegidas frente a «sobretensiones» mediante dos diodos en polarización inversa que se añaden en el proceso de fabricación; éstos recortan las tensiones negativas (por debajo de -0,6 V) y aquellas tensiones positivas que superan (en más de 0,6 V) la de alimentación (los datos catálogo incluyen la intensidad máxima que soportan estos diodos con la notación I_{clamp}):



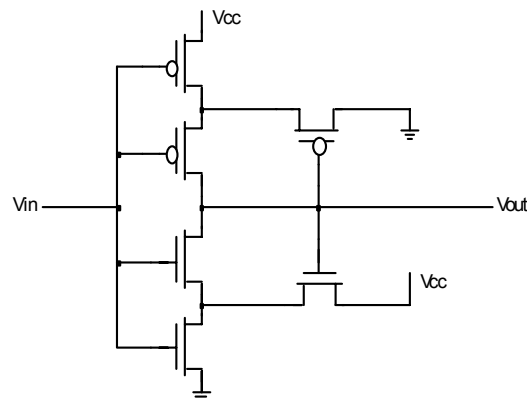
Las salidas de las puertas CMOS presentan un efecto análogo de limitación de tensión, debido a que el drenaje de un transistor MOS forma siempre una unión PN con el sustrato del mismo.

En principio, las entradas presentan una tensión de conmutación por debajo de la cual la entrada es entendida como **0** y por encima como **1**. A veces, resulta útil disponer de entradas con histéresis, que tengan dos tensiones de conmutación: cuando la entrada se encuentra a **0** y se aumenta paulatinamente su tensión, la salida conmuta al alcanzarse el valor V_b ; pero, en cambio, cuando la entrada se encuentra a **1** y se disminuye gradualmente su tensión, la salida conmuta para un valor V_a distinto, siendo $V_a < V_b$ (en la serie HCMOS, alimentada a 5 V, las tensiones de conmutación suelen ser 2 y 3 V).



Este tipo de entradas con histéresis recibe el nombre de entradas *Schmitt*; su comportamiento asimétrico respecto a las variaciones de la tensión de entrada sirve para evitar los rebotes que se producen cuando dicha variación es lenta y la tensión de entrada permanece un cierto tiempo en el entorno de la tensión de conmutación.

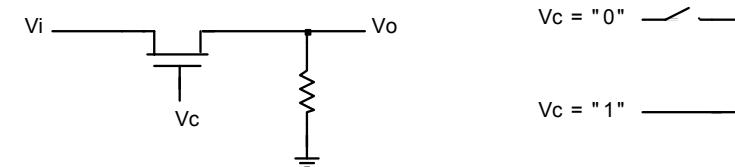
La figura siguiente representa el circuito correspondiente a un inversor con entrada *Schmitt*; los transistores «horizontales», conectados a la tensión de salida, introducen una realimentación que da lugar a la histéresis: con salida **1** los dos NMOS inferiores forman un divisor de tensión cuyo efecto es desplazar hacia arriba la tensión de conmutación, mientras que para salida **0** el divisor de tensión de los dos PMOS superiores desplaza hacia abajo la otra tensión de conmutación.



9.3. Puertas de transmisión

Los transistores MOS pueden ser utilizados como conmutadores para «dejar pasar» o «impedir el paso» de una tensión. La existencia o no de canal entre fuente y drenaje determina que la resistencia de paso sea reducida (inferior a $100\ \Omega$) o muy alta (superior a los $10\ M\Omega$). Esta actuación del transistor MOS como «transistor de paso» o conmutador añade a la diversidad de puertas inversoras una nueva puerta no inversora: la puerta de transmisión.

Un transistor NMOS actúa en la forma siguiente, siendo V_{TO} la tensión umbral:

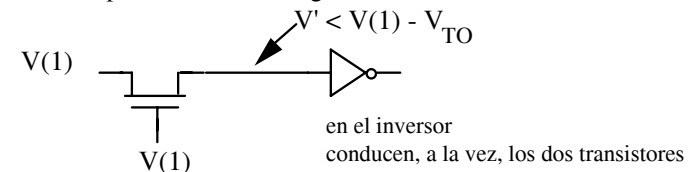


- para $V_C = 0 < V_{TO}$ el transistor se encuentra en corte, presenta una resistencia de paso superior a $10\ M\Omega$ y no comunica a la salida la tensión de su entrada
- cuando $V_C = 1 \gg V_{TO}$ el transistor conduce, su resistencia es inferior a $100\ \Omega$ y transmite hacia la salida la tensión presente en su entrada (los transistores MOS son simétricos, de modo que cada uno de sus terminales extremos actuará como fuente o como drenaje, según convenga al sentido de las intensidades):
 - + si $V_i = 0 = 0\ V$, la tensión en la salida será también nula, $V_o = 0\ V$
 - + si $V_i = 1 = V_{CC}$, la tensión en la salida será $V_o = V_{CC} - V_{TO}$, pues se requiere una tensión V_{TO} entre fuente y puerta para permitir la formación del canal (en este caso, la fuente corresponde al terminal de salida V_o).

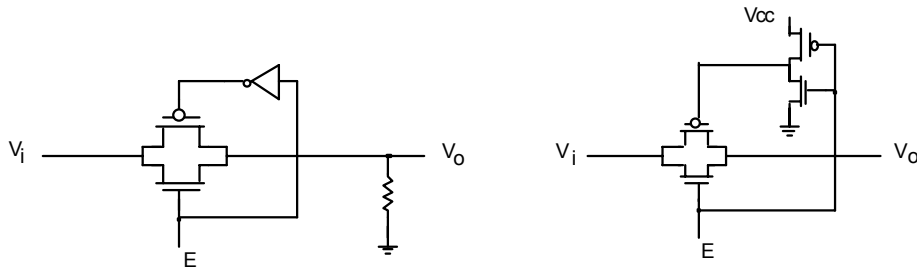
Es decir, al transmitir un **1** la puerta de transmisión produce un desplazamiento de tensión igual a su tensión umbral (para $V_i = V_C = 5\ V$ y $V_{TO} = 1,5\ V$, la tensión transmitida por la puerta será sólo de unos 3,5 voltios).

De esta forma, un transistor NMOS corresponde a un circuito abierto cuando su terminal de control se encuentra a **0** y, en cambio, a **1** permite el paso a su través del valor booleano presente a su entrada; si el valor a transmitir es **1** se produce un desplazamiento en tensión ($\Delta V \sim -V_{TO}$).

Este desplazamiento (del valor booleano **1** al atravesar un transistor NMOS) se ve aumentado por un efecto de segundo orden (denominado efecto sustrato); de forma que la tensión de salida $V_o(1) < V_{CC} - V_{TO}$ no es suficiente para asegurar la situación de consumo nulo en las puertas booleanas siguientes.



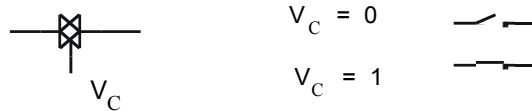
Es posible evitar esta disminución de la tensión del **1** booleano, construyendo las puertas de transmisión con dos transistores complementarios en paralelo; ésto obliga, además, a añadir un inversor entre la tensión de puerta del transistor NMOS y la puerta del PMOS (ya que el transistor PMOS conduce con $V_E = 0$).



• para $V_E = 0$ ambos transistores se encuentran en corte y la puerta de transmisión no conduce;

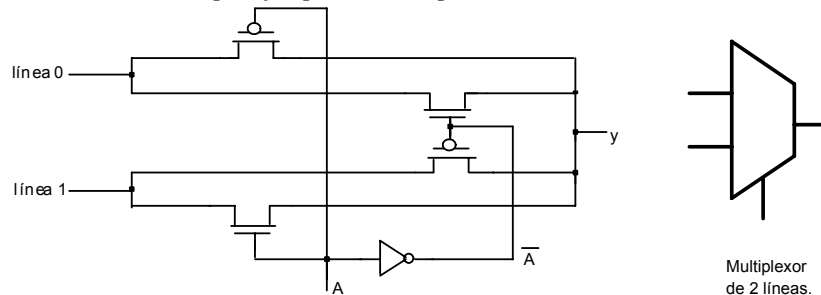
• cuando $V_E = V_{CC}$ (en el terminal de puerta del transistor PMOS habrá 0 V) los dos transistores conducen, transmitiendo a la salida el valor de tensión presente en su entrada, sin producir ningún desplazamiento de nivel: el transistor NMOS conduce siempre que $V_o < V_{CC} - V_{TO}$ y el transistor PMOS lo hace siempre que $V_o > V_{TO}$, de forma que entre los dos cubren el intervalo $[0 - V_{CC}]$.

Para representar las puertas de transmisión puede utilizarse el siguiente símbolo:

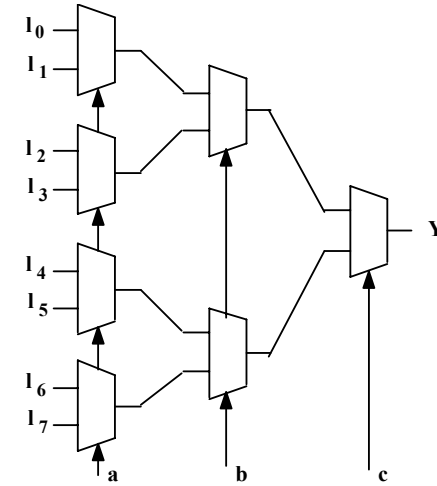


La puerta de transmisión tiene el inconveniente de que la variable de control ha de actuar a través de sus dos transistores en forma invertida (con un inversor intermedio), de manera que a los dos transistores que configuran la puerta han de agregarse otros dos transistores para el correspondiente inversor.

Pero este inconveniente no es tal en aquellas estructuras que siempre utilizan las entradas en forma afirmada y negada (y, por tanto, requieren ya de por sí el invertir la señal de control), como, por ejemplo, los multiplexores:



Un multiplexor de 2 entradas es directamente ampliable conectando módulos iguales en cascada (para n líneas serán necesarios $n-1$ módulos conectados en cascada):



El esquema anterior contiene 14 puertas de transmisión; conformadas por 2 transistores cada una, y 3 inversores para las 3 entradas (el inversor de cada columna de multiplexores es común a todos ellos); en total 34 transistores, mientras que un multiplexor análogo realizado con puertas "y-negada" (*Nand*) necesita 8 puertas de 4 entradas, una de 8 y 3 inversores, es decir, 86 transistores.

De esta forma, la conexión en cascada de puertas de transmisión proporciona una configuración muy simple y modular para los multiplexores y el número de transistores necesarios es claramente inferior a los que se requieren con puertas inversoras; por la misma razón el tiempo de propagación es también inferior.

Habida cuenta de que las puertas de transmisión son simétricas, un multiplexor construido con ellos también lo es y puede ser utilizado como demultiplexor; en tal caso se presenta el problema de que las líneas de salida no seleccionadas quedan en alta impedancia (no en salida **0**, como correspondería a un demultiplexor digital), lo cual puede solventarse añadiendo resistencias que referencien a 0 V dichas líneas.

Incluso, en muchas ocasiones, es útil el estado de alta impedancia de las salidas del demultiplexor pues permite conservar sobre cada línea el último de los valores transmitidos a la misma (para lo cual será necesario que exista una capacidad adecuada conectada en cada línea de salida).

Las puertas de transmisión son, también, buenos interruptores analógicos (*analog switches*) que controlan el paso de señales en el intervalo $[0, V_{CC}]$ establecido por las tensiones de control (de puerta) de sus transistores; caso de que la señal de control utilice como niveles $-V_1$ y $+V_2$, la puerta de transmisión admitirá señales analógicas entre tales valores $[-V_1, +V_2]$.

De esta forma, las puertas de transmisión sirven para muestrear señales analógicas y para realizar su multiplexado o demultiplexado:

- un multiplexor construido con puertas de transmisión puede transmitir cualquier tensión dentro de los límites fijados por sus tensiones de control $[-V_1, +V_2]$: constituye un multiplexor analógico que permite seleccionar por su número una de entre n tensiones (por ejemplo, para medir diversas tensiones con un mismo conversor analógico-digital);
- el mismo multiplexor, utilizado en sentido inverso (una entrada, n salidas) es un demultiplexor analógico, cuyas líneas de salida adoptan el estado de alta impedancia cuando no son seleccionadas y pueden conservar, sobre capacidades, el último valor de tensión transmitido a cada línea (puede servir, por ejemplo, para comunicar varias referencias de tensión).

Asimismo (como se verá en los capítulos 11 y 13), las puertas de transmisión resultan útiles para la construcción de biestables.

9.4. Realización CMOS de las estructuras matriciales

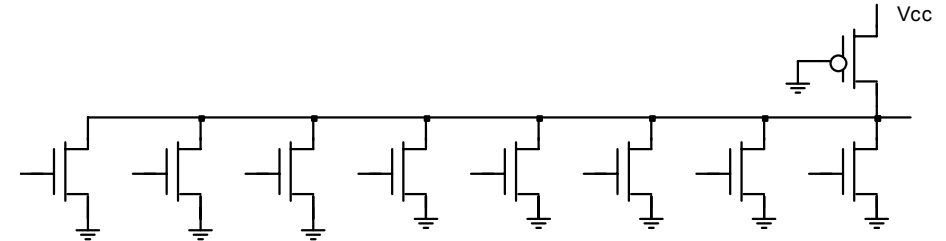
Las puertas CMOS complementarias son inversoras de forma que no pueden construirse directamente las puertas "y" y las puertas "o" propias de las Matrices que conforman las configuraciones reticulares; ello no supone ningún inconveniente por cuanto que toda suma de productos puede ser sustituida por puertas "y-negada": $\sum p = \text{Nand}(\text{Nands})$.

Ahora bien, a la hora de construir funciones booleanas de muchas entradas resulta que las puertas CMOS necesarias para ello presentarán (en uno de sus dos planos P o N) un alto número de transistores MOS en serie; cuando conducen, dichos transistores suman sus resistencias de paso y la resistencia de salida resultante será relativamente alta, empeorándose mucho las características de intensidad de salida disponible, velocidad de trabajo e inmunidad frente al «ruido» de la puerta lógica.

Precisamente las estructuras matriciales son útiles cuando se trata de configurar funciones de un alto número de entradas; en ellas se presentará este problema propio de las puertas de muchas entradas.

La conexión en serie de un número alto de transistores MOS se evita utilizando puertas "o-negada" (*Nor*), cuyo plano N presenta todos sus transistores NMOS en paralelo, y reduciendo el plano P de las mismas a un solo transistor PMOS como resistencia de polarización (con su entrada conectada directamente a 0 V).

Tales puertas se denominan pseudoNMOS, pues su configuración es análoga a la de las puertas propias de las tecnologías NMOS (solo que, en ese caso, el transistor que hace de resistencia de polarización es de tipo NMOS con un canal implantado —NMOS de empobrecimiento—):



Las puertas "o-negada" (*Nor*) pseudoNMOS presentan dos ventajas que las hacen muy apropiadas para estructuras matriciales y programables:

- permiten un alto número de entradas, pues los correspondientes transistores se conectarán en paralelo;
- presentan un solo plano activo de forma que, en cuanto a dispositivos programables solamente será necesario programar sobre el plano NMOS y tal programación consistirá en «desconectar» transistores que se encuentran en paralelo.

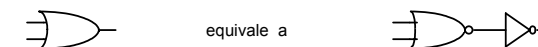
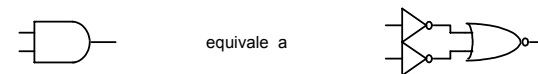
Por contra, estas puertas tienen el inconveniente de que su consumo estático no es nulo.

El transistor PMOS actúa como resistencia de polarización y conduce siempre; cuando la salida es 0, también conduce el plano NMOS, originando un consumo estático a través de la malla de salida. Este consumo puede reducirse conectando la entrada del transistor PMOS a un terminal de habilitación, que permita limitar la actividad de estas puertas al intervalo de tiempo en que interese su actuación, manteniéndolas inactivas y con consumo nulo en el resto:

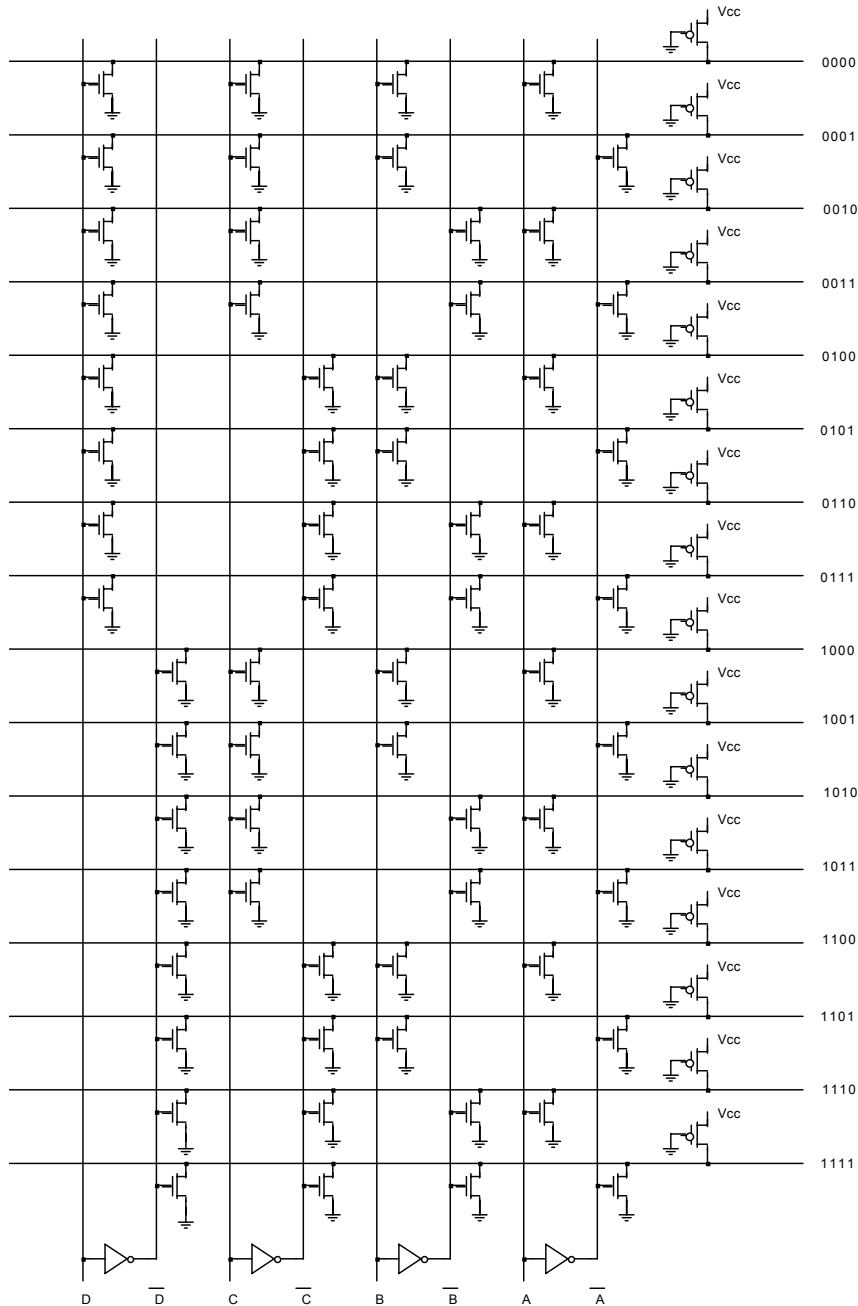
- para $\bar{E} = 1$ el transistor PMOS no conducirá y la puerta estará deshabilitada
- para $\bar{E} = 0$ la puerta se encontrará en normal funcionamiento como puerta *Nor*.

Es fácil construir puertas "y" y puertas "o" a partir de las puertas "o-negada":

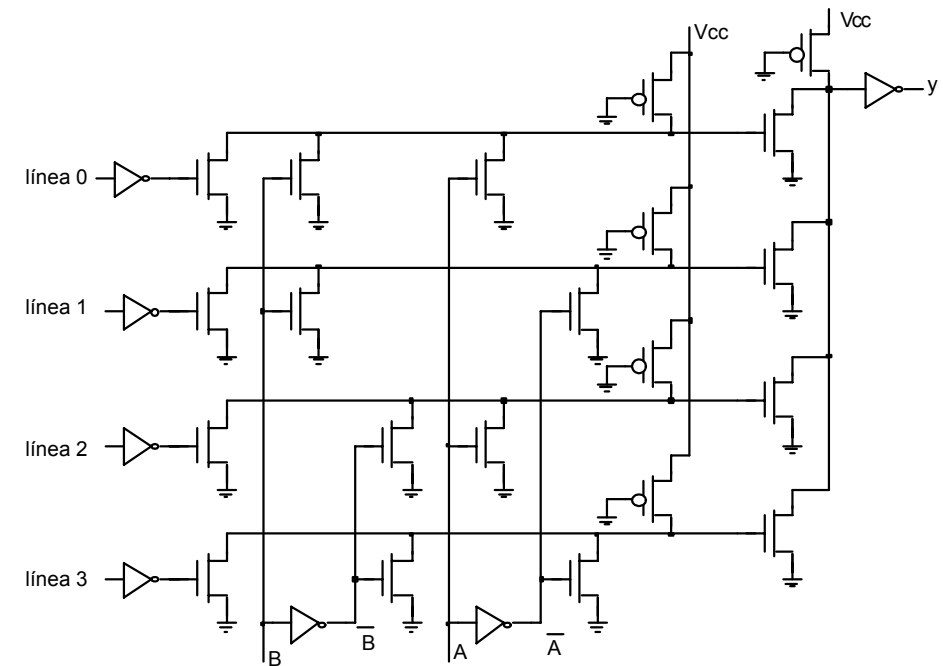
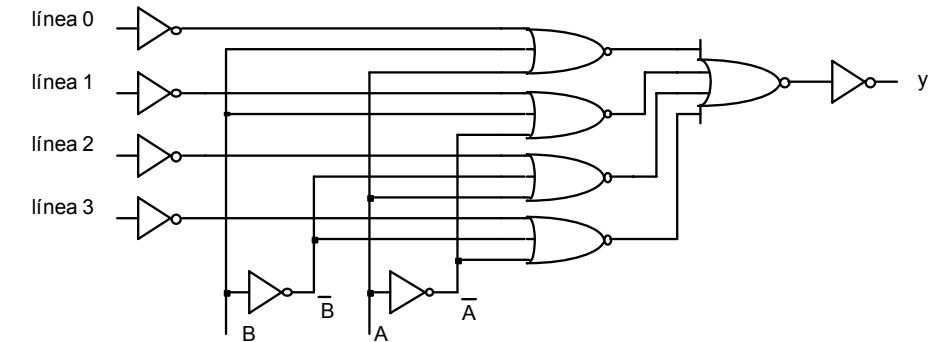
$$a + b = \overline{\overline{a + b}} = \overline{\overline{a} \cdot \overline{b}}; \quad a \cdot b = \overline{\overline{a \cdot b}} = \overline{\overline{a} + \overline{b}} = \overline{a} \Delta \overline{b}$$



La figura de la página siguiente representa un decodificador de 4 variables con puertas "o-negada" pseudoNMOS; obsérvese en ella que (a fin de configurar la operación "y") las puertas *Nor* están conectadas a las variables negadas cuando su valor en el vector de entrada es 1 y a las variables afirmadas cuando es 0.



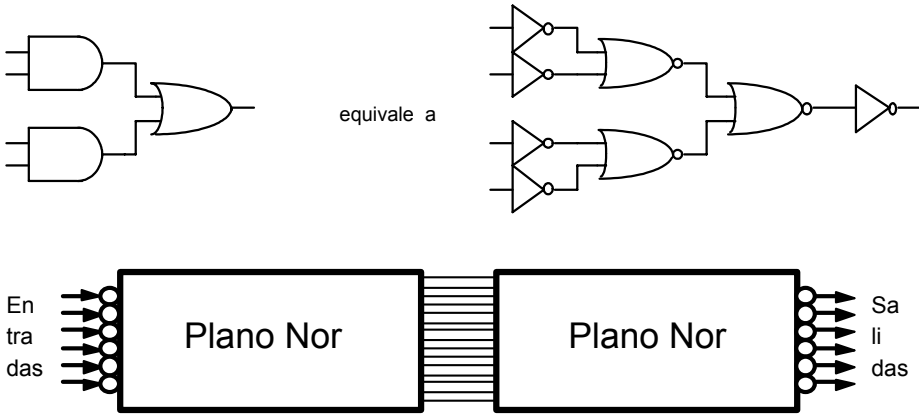
Un segundo ejemplo: la configuración de un multiplexor de 4 líneas de entrada será semejante a la del correspondiente decodificador, añadiendo una entrada a cada una de las puertas y recogiendo las salidas de ellas sobre una puerta *Nor* con una inversión posterior.



Multiplexor de 4 líneas de entrada (2 variables de control)

Conviene insistir en que, por razones obvias de facilidad de realización y de comprensión de los dibujos, las figuras se refieren a bloques con un reducido número de variables y, sin embargo, estas estructuras son realmente interesantes y útiles cuando se trata de un amplio número de entradas.

El conjunto Matriz Y - Matriz O propio de las estructuras ROM, PLA y PAL puede ser sustituido por dos matrices *Nor*, la primera de ellas con una inversión previa de sus entradas y la segunda con una inversión posterior de sus salidas:

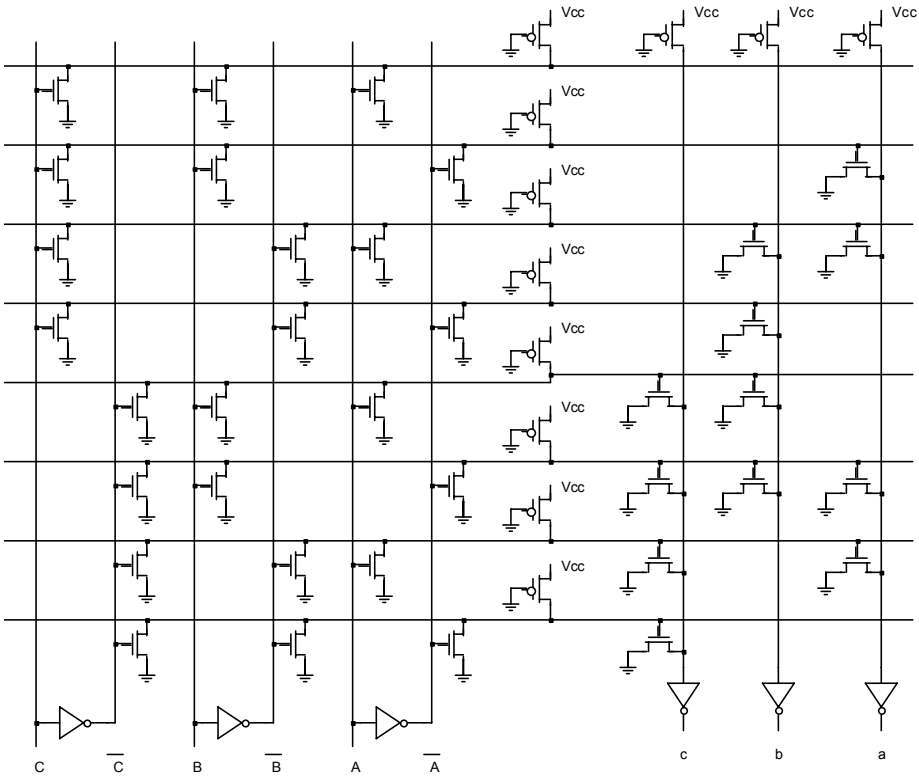


Téngase en cuenta que no es preciso añadir los inversores de las entradas puesto que éstas ya disponen de ellos en la retícula de conexiones de la Matriz Y: bastará tomar las variables adecuadamente negadas (cuando su valor en el vector de entrada sea **1**) o afirmadas (cuando sea **0**).

La figura de la página siguiente representa un sencillo codificador que realiza el cambio de código binario a código Gray para números de 3 dígitos, conforme a la siguiente tabla de conversión:

C	B	A	c	b	a
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	1
0	1	1	0	1	0
1	0	0	1	1	0
1	0	1	1	1	1
1	1	0	1	0	1
1	1	1	1	0	0

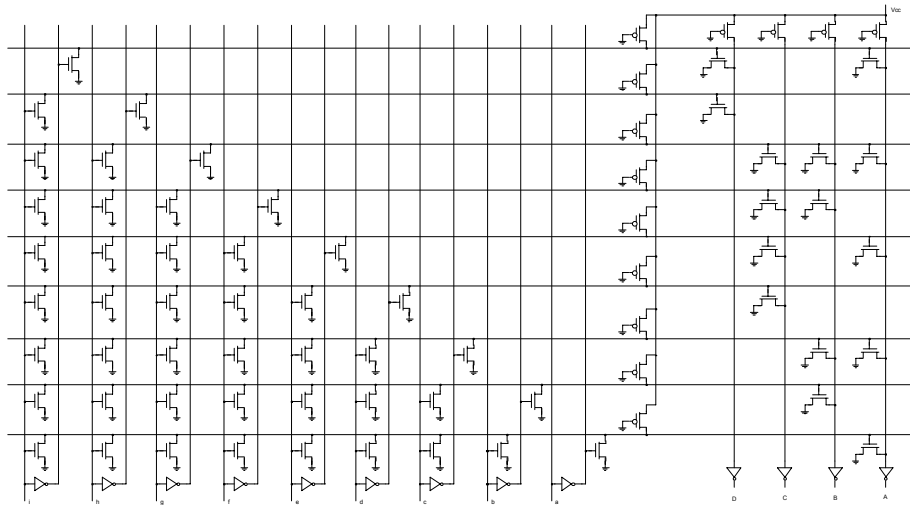
En esta figura pueden apreciarse las dos matrices que forman la configuración ROM: la Matriz Y de entradas (matriz completa que construye todos los términos mínimos) y la Matriz O de salidas (matriz funcional que coincide con la tabla de las funciones). Obsérvese que las puertas *Nor* de la Matriz Y se encuentran dibujadas en horizontal (en cuanto a sus transistores NMOS) mientras que las puertas *Nor* de la Matriz O están dibujadas en vertical.



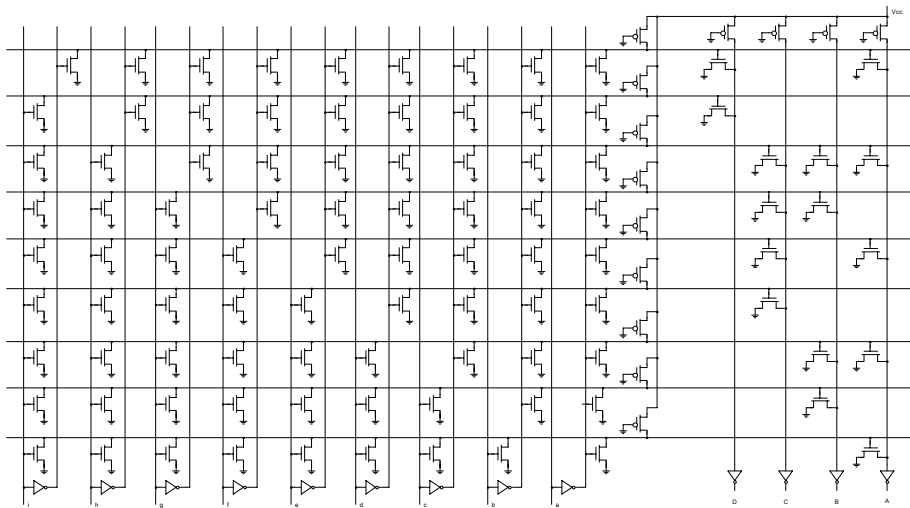
Codificador binario → Gray para números de 3 dígitos, en configuración ROM

Tanto las estructuras matriciales como las puertas pseudoNMOS (*Nor* con un solo transistor PMOS) solamente tienen sentido para muchas entradas. En esa perspectiva, la siguiente página presenta dos ejemplos de bloques de 9 entradas en configuración PLA:

- Un codificador de prioridad de 9 líneas, cuya salida es el número binario que corresponde a la línea activa de mayor número.
- Un conversor de código de barra de 10 niveles a código BCD, cuyos vectores de entrada corresponden a los 10 niveles, de 0 a 9 (00000000, 00000001, 00000011, 00000111, 00001111, 00011111, 00111111, 01111111 y 11111111) y su salida es el número BCD correspondiente al mismo nivel (0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000 y 1001).



Codificador de prioridad de 9 líneas, en configuración PLA

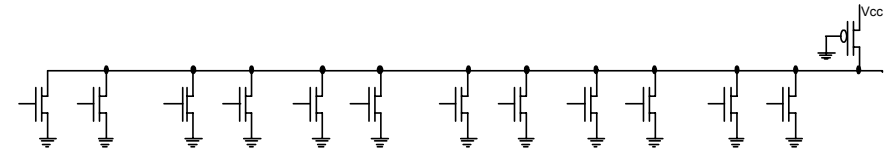


Conversor de código de barra a código BCD, en configuración PLA

9.5. Dispositivos programables

La forma de construir circuitos integrados programables consiste en insertar un transistor en cada uno de los nudos de la retícula de aquella matriz (o matrices, en el caso PLA) que deba ser programable y añadir a cada uno de estos transistores un dispositivo o componente que permita eliminar su presencia.

La configuración pseudoNMOS hace que solamente sea necesario programar el plano N, pues el plano P es fijo, constituido por un transistor PMOS único que actúa como resistencia de polarización: para las entradas booleanas se utilizan solamente transistores de tipo NMOS, todos ellos en paralelo (puerta *Nor*) y su programación consiste en eliminar (*desconectar*) los correspondientes transistores.



Ciertamente, en el caso de puertas CMOS con los dos planos complementarios, la programación de ambos planos sería sumamente compleja, pues en uno de ellos los transistores se encontrarían en serie y su programación consistiría en «puentearlos» (lo cual es mucho más difícil que *desconectarlos*).

Fusibles

Un fusible en serie con cada transistor NMOS (en la matriz programable) permite «desconectar» dicho transistor.

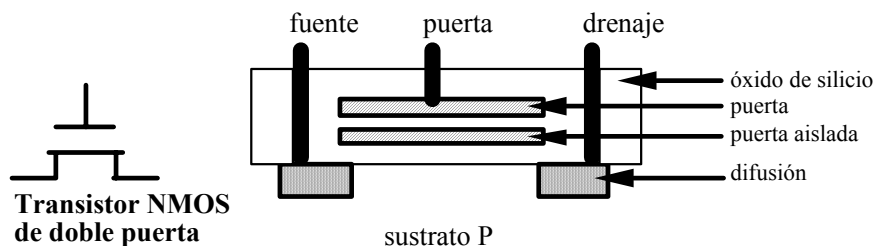
Los fusibles se configuran mediante un estrechamiento en un conductor de un material apropiado, de forma que pueda «fundirse» por efecto térmico mediante el paso de una intensidad relativamente elevada; para ello se utilizan aleaciones de platino-silicio, titanio-tungsteno o níquel-cromo (nicrom, que es quizás la aleación más utilizada por ser la menos costosa).

Los fusibles se programan mediante «fusión térmica» utilizando pulsos de intensidad relativamente alta, con fuerte pendiente dI/dt para generar tensiones puntuales apreciables (por efecto inductivo); de esta forma el efecto en potencia $V.I$ de los pulsos de intensidad es considerable. Se emplean tensiones típicas de 10,5 V, aplicadas en pulsos repetitivos de anchura entre 10 y 50 μs , dando lugar a intensidades de 0,1 - 0,5 A con tiempos de subida del orden de 100 ns.

Los fusibles se utilizan principalmente en tecnologías bipolares, en dispositivos matriciales (PAL, PLA, PROM) de alta velocidad.

Transistores MOS de doble puerta

Los transistores de doble puerta son de tipo NMOS, con una puerta interior flotante, aislada eléctricamente y una segunda puerta, por encima de la anterior, que actúa como entrada. La puerta interior es capaz de almacenar carga eléctrica y mantenerla indefinidamente; si la carga de dicha puerta es negativa (electrones), aumenta la tensión umbral del transistor hasta llegar a situarse por encima de la tensión de alimentación: la tensión umbral V_{TO} pasa del intervalo 1–2 V a ser superior a 8 V, por lo que el transistor nunca llega a conducir.



La programación se produce por inyección de carga sobre la puerta aislada:

- la puerta interior queda cargada negativamente y se opone al efecto que la tensión de entrada en la otra puerta tiene para la creación del canal, lo cual se traduce en una fuerte elevación de la tensión umbral
- el transistor en esta situación no conduce nunca ya que su tensión umbral ha pasado a ser superior a la tensión de alimentación y por tanto al valor de tensión que recibe en su entrada exterior para el valor booleano 1
- el resultado práctico es la «desconexión» efectiva del transistor
- el buen aislamiento que realiza el óxido de silicio permite mantener durante centenares de años la carga electrostática de la puerta interior.

Ahora bien, existen métodos para eliminar la carga almacenada en la puerta aislada y, de ese modo, «borrar» la programación, es decir, volver a «conectar» el transistor. Por eso, los transistores MOS de doble puerta son reprogramables: su programación puede ser anulada (borrada) y, luego, pueden volverse a programar.

EPROM: borrado por luz ultravioleta

Este primer tipo de transistores MOS de doble puerta se programa a partir de una fuerte corriente de canal y de una tensión relativamente alta en la puerta externa; algunos de los electrones que circulan por el canal, acelerados, son capaces de «saltar» a la puerta aislada, a través de la fina capa de óxido que la separa.

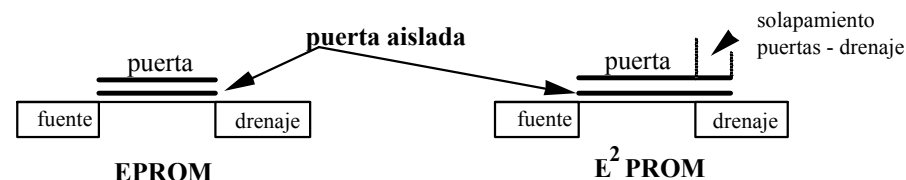
La programación (la acumulación de carga en la puerta aislada) requiere tensiones relativamente altas (~12 V) en drenaje (para provocar una corriente alta a través del canal) y en puerta (para atraer a los electrones).

El «borrado» se efectúa mediante la exposición a luz ultravioleta; ésta proporciona a los electrones atrapados en la puerta aislada energía suficiente para volver al sustrato; se utilizan lámparas de luz ultravioleta de longitudes de onda del orden de 2.537 Å durante unos 15 ó 20 minutos.

Este tipo de transistores es designado con las siglas **EPROM** (*erasable programmable ROM*: ROM programable y borrrable) debido a que su primera utilización fue en los bloques de tipo ROM programables; quizás una denominación más adecuada sería la de UVMOS (transistores MOS borrrables por luz ultravioleta).

EEPROM: borrado por tensión eléctrica

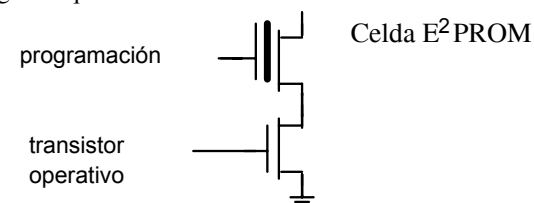
La reducción del espesor del óxido de silicio que separa la puerta interior del sustrato junto con un pequeño solapamiento de las dos puertas con el drenaje ha permitido hacer reversible el proceso de almacenamiento de carga: la puerta aislada se carga y se descarga desde el drenaje por atracción (tensión de puerta positiva) o repulsión de electrones (tensión negativa).



Un pulso de tensión positiva (relativamente alta) entre la puerta exterior y el drenaje produce almacenamiento de carga (atrae electrones del drenaje a la puerta aislada) y un pulso análogo pero negativo elimina la carga almacenada (repele los electrones y los devuelve al drenaje).

Este segundo tipo de transistores MOS de doble puerta da lugar a dispositivos programables y «borrables» eléctricamente; en ambos casos, el tiempo necesario es del orden de varios milisegundos y se requieren adecuadas tensiones positiva y negativa, generalmente ± 12 V.

El borrado es individual (se efectúa sobre un solo transistor), pero presenta un problema de «sobreborrado»: cuando la tensión de la puerta exterior es negativa, además de eliminar los electrones de la puerta aislada, dicha puerta suele cargarse positivamente y dicha carga genera un canal permanente que pone en conducción al transistor. Por ello, es necesario utilizar dos transistores en serie: el de doble puerta que sirve para conectar y desconectar al segundo que es un transistor NMOS normal.



Este tipo de celda con dos transistores es nombrado con las siglas **EEPROM** o **E²PROM** (*electrical erasable programmable ROM*: ROM programable y borrable eléctricamente) y su programación (en ambos sentidos: desconexión o conexión del transistor) puede realizarse en el propio sistema digital.

FLASH: borrado global

Un tercer tipo de transistores MOS de doble puerta proviene de agrupar los dos transistores de la celda E²PROM en uno solo: las dos puertas se solapan con la fuente y la puerta aislada sólo ocupa la mitad de la longitud del canal. De esta forma es como si hubiera dos semitransistores unidos: un transistor de doble puerta, que puede cargarse y descargarse desde la fuente, seguido de un transistor normal de puerta única (conectada con la puerta exterior del anterior).

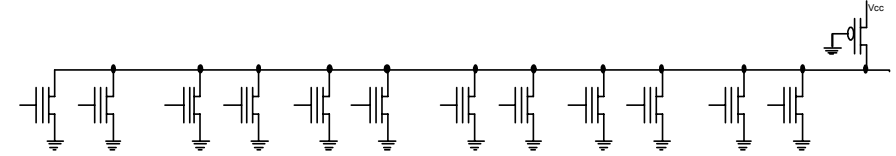


El proceso de carga de la puerta aislada es del tipo 1 EPROM, con una intensidad alta de canal que deriva electrones hacia dicha puerta (tensiones de puerta exterior y de drenaje relativamente altas, ~12 V). La descarga (el borrado) se produce según el tipo 2 EEPROM, por pulsos de tensión positiva sobre la fuente, la cual atrae a los electrones de la puerta aislada. En este caso no hay riesgo de «sobrborrado»; aunque la puerta aislada se cargue positivamente, no puede poner en conducción al transistor, ya que sólo afecta a la mitad del mismo (queda en todo caso otra mitad de transistor, cuyo canal solamente puede ser creado por la tensión sobre la puerta exterior).

Habida cuenta de que la fuente es un nudo común a todos los transistores NMOS (terminal de *tierra* de las puertas *Nor*), el borrado no puede realizarse individualmente (como en el caso EEPROM) sino que afecta a todos los transistores del bloque. Este tipo de borrado global y relativamente rápido en comparación con el borrado por luz ultravioleta ha motivado la utilización del calificativo **FLASH** para designar este tercer tipo de transistores MOS de doble puerta.

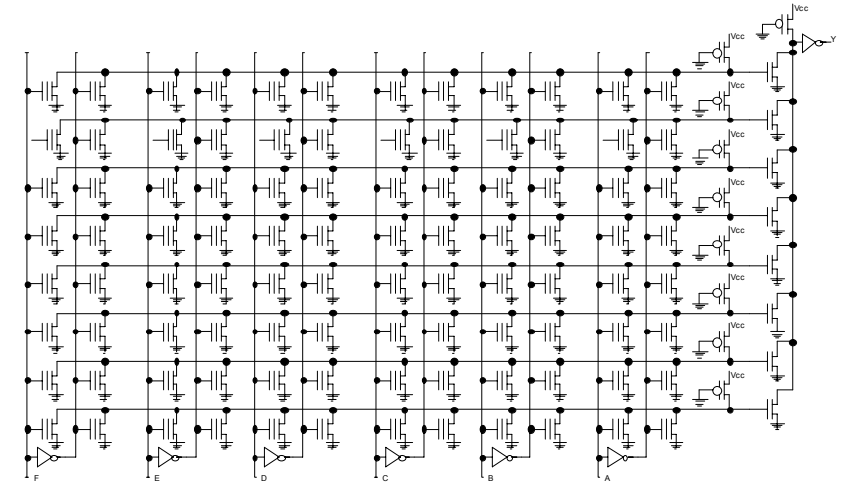
En buena medida, la tecnología **FLASH** incorpora las ventajas de las dos anteriores: requiere un solo transistor (y no dos como la EEPROM) y el borrado es por tensión eléctrica. Además, por ser una tecnología muy reciente, sus transistores son de dimensiones muy reducidas (ésto permite una densidad de integración muy alta), sus tiempos de programación son bajos (~10 µs) y el borrado es muy sencillo (un simple pulso positivo sobre el terminal de fuente, común a los transistores NMOS) y completo (se borra todo el bloque a la vez); el tiempo necesario para el borrado es del orden de 1 segundo.

Una puerta "**o-negada**" (*Nor*) pseudoNMOS con transistores de doble puerta presenta la siguiente configuración:



Al programar uno de dichos transistores, por acumulación de carga negativa en su puerta interior, dicho transistor pasa a tener una tensión umbral muy alta que equivale a eliminarlo (*desconectarlo*) de la puerta.

La figura siguiente representa un módulo **PAL** de 6 entradas y 8 términos producto:



En esta figura, las puertas *Nor* de los 8 términos producto están representadas en «horizontal» y contienen 12 transistores EPROM (correspondientes a las 6 entradas y sus negadas), mientras que la puerta *Nor* de salida está representada en «vertical» con 8 transistores NMOS (correspondientes a la suma de los 8 términos producto).

Un bloque **PAL** que contenga 10 de estos módulos de 6 entradas con 8 términos producto por módulo tendrá el siguiente número de transistores:

- cada módulo: $8 \times 6 \times 2 = 96$ transistores EPROM + 8 NMOS + 9 PMOS
 - los inversores de las entradas: 6 transistores NMOS + 6 PMOS
 - los inversores de las salidas: 10 transistores NMOS + 10 PMOS
- en total: 960 transistores programables + 96 NMOS + 106 PMOS.

10 FAMILIAS LÓGICAS INTEGRADAS

- 10.1. Panorama general de las familias lógicas integradas
- 10.2. Características a tener en cuenta en una familia lógica
- 10.3. El ruido en los sistemas digitales

El proceso de miniaturización de la electrónica, iniciado en la década de los 50 con la utilización del transistor, continuó con un segundo salto cualitativo en la década siguiente (años 60) mediante la integración de subcircuitos completos en un mismo sustrato de silicio (*chip*): subcircuitos correspondientes a módulos digitales tales como puertas booleanas, biestables o bloques combinacionales o secuenciales.

Los circuitos digitales son sumamente apropiados para su inserción en circuitos integrados: de un lado, la ausencia de autoinducciones y el poder prescindir, asimismo, de condensadores reduce los elementos a integrar a transistores y resistencias y a las conexiones de estos entre sí; de otro, la propia modularidad de los sistemas digitales precisa de un número reducido de tipos de puertas lógicas, e incluso, basta con un solo tipo de ellas (*puertas Nand o Nor*).

Por ello, los circuitos integrados invadieron muy pronto el campo digital; en unos pocos años resultó anacrónico y antieconómico construir las puertas booleanas con componentes discretos, una vez que se disponía de una gran variedad de puertas lógicas y de una amplia serie de funciones de gran complejidad construidas dentro de un circuito integrado.

El presente capítulo repasa la evolución de las diversas familias lógicas integradas. En primer lugar, las puertas bipolares que condujeron a la gran familia **TTL** (cuya amplia difusión consolidó la lógica integrada); luego las tecnologías **MOS**, hasta llegar a la predominante **HC MOS**; la mezcla **BiCMOS** (bipolar-CMOS) que resulta muy apropiada para circuitos «interbiús» (en medio de los buses); y la derivación actual hacia series de bajo voltaje (pasando de la alimentación habitual de 5 V a sólo 3 V).

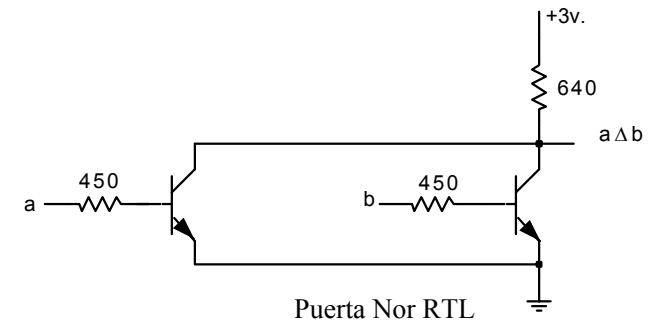
En todo caso, para elegir y utilizar correcta y eficazmente una familia lógica (y, dentro de ella, una serie específica) es preciso tener en cuenta sus características funcionales. Aún más, es requisito previo para ello comprender el significado conceptual y las implicaciones prácticas de tales características y ser capaz de localizar y «hacer una lectura efectiva» de las mismas en los catálogos que suministran los fabricantes de circuitos integrados.

Entre las diversas cuestiones a las que prestar atención aparece el «ruido electromagnético» como un «compañero no deseado» que puede perturbar el correcto funcionamiento de un circuito digital y que requiere una actitud vigilante y un importante esfuerzo de «autoprotección» en el proceso de diseño y puesta a punto del circuito. Pero, a la vez, la producción de «ruido electromagnético» por el propio circuito obliga a un esfuerzo complementario de reducción de la emisión de perturbaciones para cumplir con las normativas de compatibilidad electromagnética.

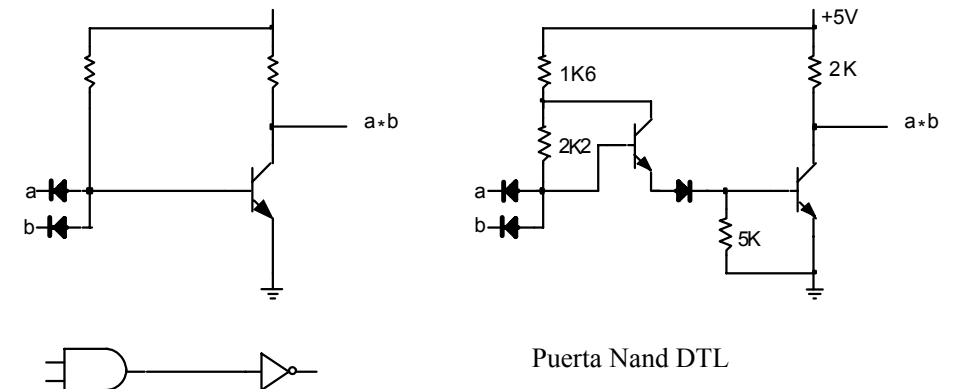
10.1. Panorama general de las familias lógicas integradas

10.1.1. Primeras familias lógicas: C. I. con transistores bipolares

Las primeras puertas lógicas integradas eran mera copia directa de las puertas "o-negada" (*Nor*) con componentes discretos, mediante la conexión en paralelo de varios transistores bipolares NPN en emisor común; tales puertas dieron lugar a la primera familia lógica: **RTL** (lógica de transistores y resistencias).

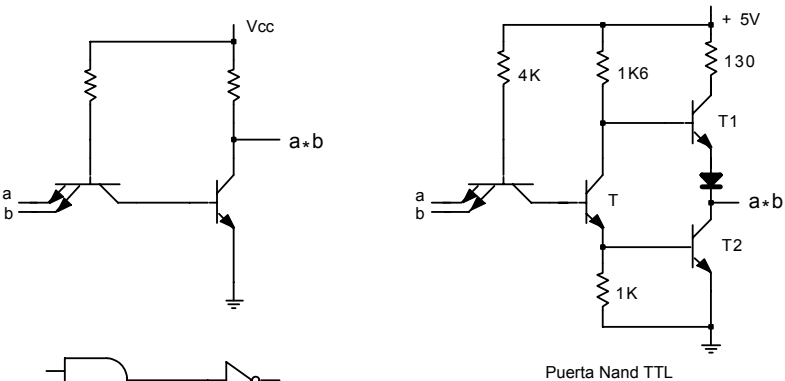


Pronto se mejoraron las características de estas puertas integradas, en cuanto a velocidad y a consumo, combinando una puerta "y" de diodos con un transistor inversor en emisor común; así se configuró la puerta "y-negada" (*Nand*) base de la familia **DTL** (lógica de transistores y diodos) que fue la primera que llegó a alcanzar una difusión apreciable.



A partir de este esquema (puerta "y" + inversor), aprovechando en mayor profundidad las posibilidades que ofrece la integración sobre un sustrato único, se planteó una segunda mejora en velocidad y en consumo, añadiendo una etapa de salida amplificadora de intensidad (dos transistores en *push-pull*) y substituyendo los diodos por un transistor multiemisor.

El resultado fue la gran familia lógica **TTL** (lógica de transistores con transistores).



La etapa de salida de dos transistores NPN (*totem pole*: «palo de tótem») aumenta la intensidad suministrable y disminuye la resistencia de salida; el transistor multiemisor mejora considerablemente la conmutación de la puerta (en una primera aproximación, su comportamiento puede ser analizado en términos de diodos:



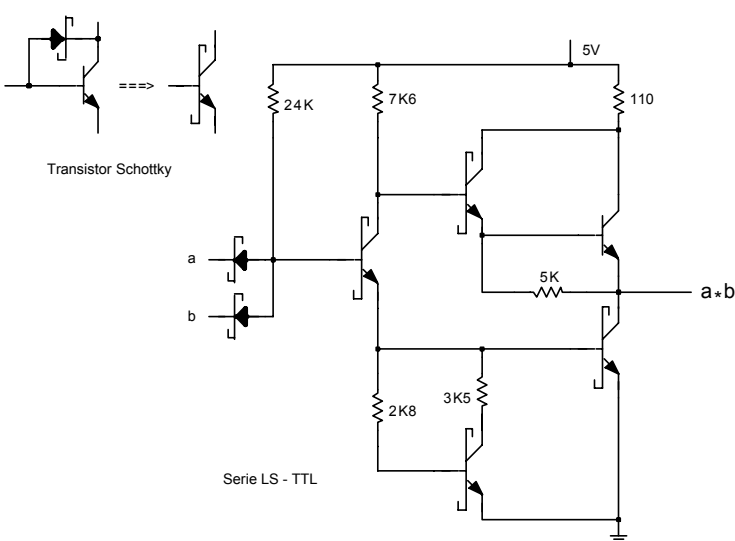
La clave del funcionamiento de la puerta **TTL** es el sentido en que circula la intensidad que la base del transistor multiemisor recibe desde la resistencia de 4K:

- si dicha corriente va «hacia fuera», es decir, si alguna de las entradas está conectada a **0**, el transistor T se encontrará en corte y el transistor T1, en colector común, transmite un **1** a la salida;
- cuando todas las entradas se encuentran a **1** dicha intensidad circula «hacia dentro», hacia la base del transistor T, que se satura y lleva también a saturación al transistor T2, que pone la salida a **0**. [Un **0** en una entrada supone una intensidad «hacia fuera», de forma que una entrada TTL «al aire» equivale a un **1**, salvo efectos de ruido.]

Las puertas **TTL** se alimentan a 5 V; su tensión de conmutación se sitúa en el entorno de 1,2 V, de manera que un **0** en la entrada ha de ser menor de 1 V ($V_{iLmáx} = 1\text{ V}$) y, en cambio, una tensión superior a 1,5 V es entendida como un **1** ($V_{iHmín} = 1,5\text{ V}$); la tensión de salida para el **0** es 0 V, pero la correspondiente al **1** es de solamente 4 V. Los tiempos de propagación de la serie **TTL** estándar son del orden de 10 ns y el consumo promedio es de unos 2 mA (10 mW).

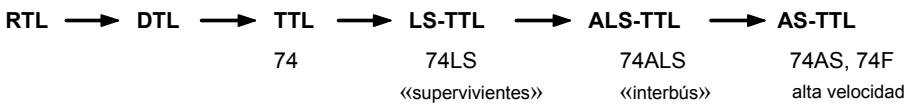
La familia **TTL** proporcionó la base del gran desarrollo que tuvieron los sistemas digitales durante la década de los 70; su amplia difusión y utilización favoreció la aparición de diversas series derivadas de la mejora de características concretas, una de las cuales, la serie **LS** ha sustituido por completo a la serie estándar inicial y es la que se ha seguido utilizando a lo largo de la década de los 80.

La serie **74LS** (*low power Schottky*) mejora en gran medida a la serie estándar en cuanto a consumo (0,4 mA), manteniendo la velocidad de trabajo en valores análogos e incluso, algo superiores. La disminución del consumo se deriva del empleo de resistencias de mayor valor, lo cual acarrea un aumento de las constantes de tiempo asociadas; este efecto queda compensado por la inclusión de un diodo Schottky entre base y colector de los transistores que impide su saturación (desvía la corriente de base hacia el colector antes de entrar en una saturación profunda) y, con ello, aumenta su velocidad de conmutación.



Posterior series «avanzadas» con el mismo esquema circuital han aprovechado la reducción de dimensiones de los transistores y la correspondiente disminución de sus capacidades parásitas para conseguir tiempos de propagación inferiores: la serie **74ALS** (*advanced LS*) presenta tiempos por debajo de 4 ns, mientras que las series **74F** (*fast-TTL*) y **74AS** (*advanced Schottky*) ofrecen tiempos de propagación del orden de 2,5 ns y 1,5 ns, respectivamente, a costa de un mayor consumo (por utilizar resistencias de menor valor).

«Protohistoria»



Esta línea de evolución de las puertas con transistores bipolares constituye la «edad antigua» de los circuitos integrados digitales; actualmente, apenas se utilizan las familias bipolares, salvo en determinadas aplicaciones específicas, en particular, para sistemas de muy alta velocidad.

La serie **74LS** sigue siendo útil para «recambio y mantenimiento» de los numerosos sistemas digitales que han sido contruidos con ella (o con la serie estándar **74**), la serie **74ALS** se emplea en circuitos «interbús» (aplicación que consideraremos un poco más adelante) y la serie **74F** resulta adecuada para diseños de muy alta velocidad de trabajo (frecuencias superiores a los 100 MHz).

[El apéndice A3 describe las puertas con transistores bipolares y el funcionamiento en detalle de la puerta básica TTL.]

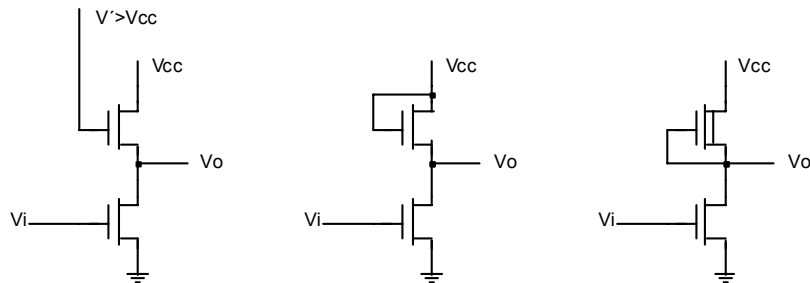
10.1.2. Desarrollo de las tecnologías MOS: familia CMOS

La integración de transistores MOS presentó inicialmente grandes dificultades, derivadas de ser un efecto superficial que es afectado por cualquier impureza o dislocación en la superficie del cristal de silicio; fue preciso desarrollar técnicas de muy alta limpieza ambiental que no estuvieron disponibles hasta mediados de los años 70. Sin embargo, una vez que se dispuso de tales técnicas, las extraordinarias ventajas de los transistores MOS (referidas a autoaislamiento, autoalineamiento, tamaño y consumo) determinaron un rápido desarrollo y difusión de los circuitos integrados digitales MOS.

En una primera fase resultó más sencillo integrar transistores MOS de canal P, pero pronto fueron desplazados por los transistores NMOS cuya velocidad de conmutación es apreciablemente mayor (debido a la mayor movilidad de los electrones respecto de los huecos).

La utilización de transistores MOS como «resistencias de polarización» permite configurar puertas lógicas utilizando únicamente transistores y reduce considerablemente el área de integración, al prescindir de resistencias integradas de valores relativamente altos. De esta forma, las puertas MOS suponen un nuevo avance cualitativo en la miniaturización de la electrónica digital, reducción que afecta no solamente al tamaño y a la densidad de integración, sino también, y en forma aún más significativa, al consumo.

La figura siguiente muestra la evolución de las puertas NMOS en relación con el transistor que actúa como «resistencia de polarización»:



En los tres inversores de la figura el transistor superior se encuentra siempre en conducción y equivale a una «resistencia de polarización».

Para que dicho transistor superior conduzca se requiere una tensión entre puerta y fuente igual o superior a su tensión umbral: por ello, inicialmente (primer inversor) fueron necesarias dos tensiones de alimentación ($V_i > V_{CC} + V_{TO}$); luego (segundo inversor) se utilizó una sola tensión de alimentación pero la tensión de salida para el 1 quedaba reducida a $V_{CC} - V_{TO}$.

Actualmente las tecnologías NMOS emplean como polarización un transistor MOS de empobrecimiento, en cuyo proceso de integración se crea un canal mediante implantación iónica, de forma que conduce incluso en ausencia de tensión entre puerta y fuente; su tensión umbral es negativa $V_{TO} < 0$, de modo que para cortar la conducción se requiere una tensión de puerta aún más negativa que destruya el canal.

El análisis circuital de los tres inversores es análogo:

- para $V_i < V_{TO}$ el transistor inferior se encuentra en corte y el superior comunica a la salida la tensión V_{CC} : $V_o = V_{CC} = 1$ (si bien en el segundo caso se produce un desplazamiento de dicha tensión: $V_o = V_{CC} - V_{TO}$);
- para $V_i \gg V_{TO}$ el transistor inferior conduce, pero también lo hace el transistor superior: es preciso establecer una relación geométrica entre ambos para que el transistor inferior presente una resistencia mucho menor que el superior y la tensión de salida sea muy pequeña: $V_o \ll 1$ V (con lo cual $V_o \approx 0$).

Habida cuenta de que la intensidad que conduce un transistor MOS es directamente proporcional a su anchura W e inversamente proporcional a su longitud L , para asegurar que, cuando conducen ambos transistores, el inferior presente una resistencia mucho menor que la del transistor superior se requiere que:

$$[W/L]_{\text{inferior}} \gg [W/L]_{\text{superior}}.$$

Esta desigualdad expresa una relación entre las geometrías de los dos transistores que ha de mantenerse en el diseño y posterior integración de este tipo de puertas **NMOS**.

La tecnología **NMOS** actual utiliza puertas formadas por un plano de transistores activos NMOS y un transistor MOS de empobrecimiento como resistencia de polarización; aprovecha plenamente la tensión de alimentación, pues $V_{oH} = V_{CC}$ y $V_{oL} \approx 0$ V, y su consumo es muy reducido, ya que $R_i \sim \infty$ y la resistencia del transistor de polarización se hace adecuadamente alta. Esta tecnología resulta muy apropiada para la integración de muy alta densidad (**VLSI**) y sigue utilizándose en grandes bloques digitales (microprocesadores, memorias, etc.) y en los circuitos integrados programables de tipo matricial (PROM, PAL, PLA, PLS).

Ahora bien (como ya hemos visto en el capítulo 8, dedicado a puertas CMOS), la utilización conjunta de transistores de canal N y de canal P (NMOS y PMOS) permite que el consumo estático de las puertas sea nulo; ello dio lugar a la lógica **CMOS** (lógica con transistores MOS complementarios).

La primera serie **CMOS** adoptó el indicativo **40** y presentaba fuertes limitaciones en cuanto a velocidad e inmunidad frente al ruido. Esta serie admite un amplio intervalo de tensiones, desde 3 a 18 voltios, y rizados del 10 % (debido a su reducido consumo), lo cual elimina la necesidad de un buen filtrado y estabilidad en la fuente de alimentación; su velocidad depende fuertemente de la tensión de alimentación, con tiempos de propagación de 200 ns para $V_{CC} = 3\text{ V}$ que pasan a ser de 100 ns para $V_{CC} = 5\text{ V}$ y se reducen a 20 ns cuando $V_{CC} = 15\text{ V}$.

La gran difusión que había tenido la familia **TTL**, con anterioridad a la disponibilidad de integrados **CMOS**, había habituado a quienes trabajaban en el ámbito de la electrónica digital a la utilización de los circuitos integrados de dicha familia y a conocer los números y los terminales de tales circuitos; por ello, atendiendo a la demanda de los usuarios, se desarrolló la serie **74C**, compatible en cuanto a funciones y terminales de los circuitos integrados del mismo número con la familia **TTL** (por ello adopta el mismo indicativo numérico **74**).

Las características de la primera serie **74C** son algo mejores que las de la serie inicial **40**; pero, muy pronto, el desarrollo continuado de las tecnologías de integración **MOS** hizo posible la utilización de transistores de dimensiones cada vez más pequeñas y, consiguientemente, más rápidos.

La serie **74HC** de «alta velocidad» ofrece la misma velocidad de trabajo que la serie **LS-TTL** (tiempos de propagación inferiores a 10 ns) y análoga inmunidad frente al ruido, con un consumo estático nulo; por ello, ha desplazado por completo a la familia **TTL** y es actualmente la más utilizada.

Para facilitar la utilización conjunta de circuitos integrados **TTL** y **CMOS** se introdujo la serie **74HCT**, compatible con los niveles de tensión y de intensidad de la familia **TTL**, que permite la conexión directa entre ambas familias.

Recientemente, se ha presentado una serie avanzada **74AHC**, con tiempos de propagación inferiores a 5 ns y una significativa reducción del «ruido» que las puertas producen en su conmutación. Existe también una serie de alta velocidad **74AC**, con tiempos de propagación del orden de 3 ns, pero con problemas de «ruido» en la conmutación debido a la gran verticalidad de sus flancos.

«Tecnologías MOS»



10.1.3. Lógica «interbús»

Una aplicación particular de los circuitos digitales que requiere prestaciones específicas se refiere a aquellos circuitos que han de situarse en medio de un bus (adaptadores de bus, controladores o decodificadores, etc.); tales circuitos, a los que nos referiremos con el calificativo de «interbús», precisan de tiempos de propagación muy bajos para no retrasar las señales que circulan por el bus y de altas intensidades de salida, ya que el bus que transmiten suele ir conectado a un amplio número de circuitos.

La serie **74ALS**, además de sus reducidos tiempos de propagación, inferiores a 4 ns, permite intensidades de salida de 24 mA para el **1** y 32 mA para el **0**, resultando adecuada para su utilización «interbús».

Por otra parte, el proceso de desarrollo de las tecnologías **MOS** ha proseguido a través de la integración conjunta de transistores **MOS** y transistores bipolares **NPN**, mediante una ampliación directa del proceso de integración **CMOS**. Esta nueva tecnología, mezcla de bipolar y **CMOS**, recibe el nombre de **BiCMOS** y resulta muy apropiada para los circuitos «interbús» y para la integración de circuitos mixtos, con parte analógica y parte digital.

Una puerta (o un bloque integrado) **CMOS** necesita que los transistores que proporcionan la salida sean de gran anchura para que la intensidad suministrable sea del orden de 10 mA. La tecnología **BiCMOS** añade a las puertas **CMOS** una etapa de salida *totem pole* de transistores bipolares, análoga a la que llevan las puertas **TTL**, la cual permite altas intensidades de salida y evita el fuerte efecto capacitivo de los transistores **MOS** de gran anchura.

La tecnología **BiCMOS** ha evolucionado a través de diversas series, de las cuales se ha consolidado y se utiliza actualmente la serie avanzada **74ABT** (*advanced BiCMOS technology*), que permite intensidades de salida de 32 mA para el **1** y 64 mA para el **0** y cuyos tiempos de propagación son menores de 3 ns.

Además, la tecnología **BiCMOS** presenta consumo estático también nulo para su salida en alta impedancia, siendo así que en tecnología **TTL** dicho consumo es aún más elevado que para salida booleana **0/1**.

10.1.4. Lógica de baja tensión

En la última década ha adquirido una gran importancia el desarrollo de circuitos de muy bajo consumo, en particular para sistemas portátiles, aplicaciones médicas, sistemas de alimentación ininterrumpida o por energía solar,... y, en general, para reducir la fuente de alimentación y el consumo energético que requieren los sistemas digitales amplios.

Ciertamente el consumo **CMOS** estático es nulo pero no así el dinámico, el cual es apreciable en el caso de altas frecuencias. Ahora bien, el consumo dinámico depende cuadráticamente de la tensión de alimentación $(V_{CC})^2$ y, por ello, una disminución de la misma tiene una incidencia muy favorable sobre el consumo global de los circuitos **CMOS**: la reducción de la tensión típica de 5 V a otra de 3 V se traduce en una disminución del consumo a la tercera parte (ahorro del 65 %).

Se han desarrollado series de «bajo voltaje» cuya tensión nominal es de 3,3 V que admiten también tensiones de alimentación inferiores, abarcando el intervalo que va desde 1,2 hasta 3,6 V; este intervalo cubre adecuadamente desde las pequeñas baterías de níquel-cadmio 1,2 V hasta las pilas de litio de alta capacidad 3 V, pasando por las diminutas pilas de mercurio 1,3 V y las habituales pilas alcalinas 1,5 V.

La serie **74LV** es equivalente a la **74HC** para bajas tensiones de alimentación, con análogos tiempos de propagación (~10 ns) cuando la tensión de alimentación no es inferior a 3 V; sus tiempos de propagación aumentan fuertemente al disminuir la tensión, situándose en los 50 ns para 1,2 V.

Existe asimismo una serie BiCMOS de baja tensión **74LVT** para trabajar en el intervalo 2,7-3,6 V con tiempos de propagación inferiores a 4 ns.

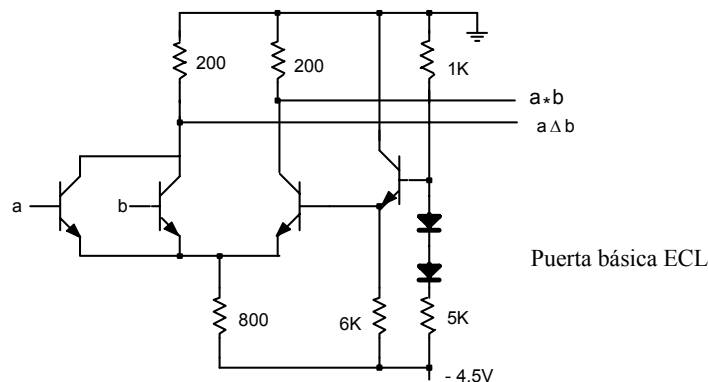
10.1.5. Lógica de muy altas velocidades

Para la construcción de sistemas digitales que han de funcionar a muy altas velocidades (por encima de los 50 MHz) es preciso utilizar series especiales, con muy bajos tiempos de propagación.

La serie CMOS **74AC** (tiempo de propagación de 3 ns) alcanza los 80 MHz de frecuencia de reloj en los sistemas síncronos, mientras que las series TTL **74F** y **74AS** (2,5 y 1,5 ns, respectivamente) permiten trabajar con frecuencias de reloj de 100 y 150 MHz, respectivamente.

Para velocidades aun mayores se cuenta con una familia ultrarrápida **ECL** (lógica de acoplo por emisor) que utiliza como etapa básica un amplificador diferencial, aprovechando la intensidad constante que tal etapa conduce para conmutarla entre las dos ramas que configuran la etapa diferencial.

La familia **ECL** no emplea la conmutación en tensión entre los estados de corte y saturación (con los retrasos inherentes a la transición entre ambos) sino que funciona en «modo de corriente», haciendo corresponder cada valor booleano con la conducción de intensidad por una de las ramas del amplificador diferencial.



La intensidad que circula por la puerta **ECL** es constante y los estados booleanos vienen definidos por el paso de dicha intensidad por una u otra de las dos ramas diferenciales. Los transistores no alcanzan la situación de saturación, por ello su conmutación es muy rápida; sus tiempos de propagación, inferiores a 0,8 ns (serie 100K), permiten alcanzar 300 MHz de velocidad de reloj en los sistemas síncronos.

El consumo de la familia **ECL** es relativamente alto, cercano a los 40 mW por puerta. En cambio, al no existir variaciones de intensidad entre ambos estados booleanos, se evita el «ruido electromagnético» que las conmutaciones producen sobre las líneas de alimentación; esta anulación del ruido «autoinducido» es muy importante cuando se trabaja a muy altas frecuencias y, por tanto, los intervalos de tiempo disponibles para la estabilización de los valores booleanos son muy pequeños.

Conviene expresar aquí la siguiente aclaración: Acostumbrados como estamos en los últimos años a las extraordinarias velocidades de los procesadores (que alcanzan varios GHz) debe tenerse en cuenta que se refieren al interior del circuito integrado y que la velocidad de trabajo «hacia fuera», en sus terminales, es considerablemente inferior. En el interior de un circuito integrado pueden conseguirse velocidades muy altas, debido a que los componentes y conexiones internos son diminutos y los efectos capacitivos son muy reducidos; en cambio, la velocidad de trabajo del circuito integrado en relación con el exterior ha de referirse a capacidades del orden de 10 pF y a etapas de salida amplias, capaces de suministrar varios miliamperios. Al hablar de tiempos de propagación y de velocidad de trabajo en este capítulo y en los anteriores nos referimos al «exterior» del circuito integrado, a las señales en los terminales del mismo.

10.2 Características a tener en cuenta en una familia lógica

Las características funcionales de una familia que es preciso tener en cuenta para su utilización en el diseño, montaje y comprobación de sistemas digitales son las siguientes:

- esquema y comportamiento circuital de su puerta básica
- tensión de alimentación
- tensiones e intensidades V_o , I_o , V_i , I_i , para ambos valores booleanos
- velocidad de trabajo
- consumo
- intervalo de temperaturas
- conectividad (*fan-out*, *fan-in* y flexibilidad de entradas y salidas)
- coste

Otra característica de particular importancia, el comportamiento frente al «ruido electromagnético», será tratada en el siguiente apartado de este capítulo.

Los datos necesarios para analizar las características de cada familia lógica y, más específicamente, las de cada uno de los circuitos integrados que la componen se encuentran en los correspondientes catálogos de las firmas fabricantes. Generalmente dichos catálogos no representan los dos estados booleanos con los valores **0** y **1** sino con los términos **L** (*low*) y **H** (*high*).

- Esquema y comportamiento circuital de la puerta básica

En primer lugar, es preciso conocer el esquema del circuito que configura la puerta básica de la familia lógica, a un nivel genérico y conceptual (que no tiene necesariamente que incluir todos los detalles), y comprender su comportamiento eléctrico en los dos estados booleanos y en las transiciones entre ambos. Parte fundamental de este comportamiento es identificar con claridad la variable física que diferencia la situación que corresponde al **0** booleano de la que corresponde al **1**; es decir, la causa física que determina el que la puerta lógica se encuentre en un estado booleano o en el otro.

Las puertas **CMOS** contienen dos planos de transistores (PMOS y NMOS) y para cada vector de entrada conduce solamente uno de ellos. Cada transistor se encuentra en conducción o en corte según que la tensión entre puerta y fuente supere o no a la tensión umbral del transistor: los transistores NMOS conducen cuando $V_i > V_{TO}$ y los PMOS lo hacen cuando $V_i < V_{CC} - V_{TO}$. La combinación dual serie-paralelo entre ambos planos determina que conduzca uno y solamente uno de ellos; el plano **P** transmite a la salida un **1**, mientras que el plano **N** lleva la salida a **0**.

La puerta básica **TTL** se encuentra conformada por tres etapas sucesivas: una puerta "y" de entrada, un inversor central y un amplificador de intensidad que configura la etapa de salida. La diferencia entre los dos estados booleanos corresponde al hecho de que la intensidad que circula por la etapa inicial (la intensidad en la resistencia de la puerta "y") sea asumida «hacia afuera» a través de alguna de sus entradas (**0** booleano en la misma) o circule «hacia adentro» saturando el transistor intermedio T (situación que corresponde a todas las entradas a **1**).

Del comportamiento circuital de la puerta básica puede deducirse una serie de consecuencias prácticas que deben tenerse en cuenta en la utilización de los circuitos integrados de la familia.

- Tensión de alimentación

La tensión típica de alimentación de los circuitos digitales es de 5 voltios. Dicha tensión es la propia de las diversas series **TTL** (entre 4,75 y 5,25 V) que, además, requieren que su rizado sea muy reducido; en suma, necesitan una fuente de alimentación bien filtrada y estabilizada.

Las puertas **CMOS** admiten tensiones de alimentación diversas dentro de un intervalo relativamente amplio y no requieren estabilidad ni ausencia de rizado en las mismas. La serie **HC** admite una alimentación entre 2 y 6 V.

Actualmente hay una fuerte tendencia a utilizar tensiones de alimentación más reducidas para disminuir el consumo dinámico y para permitir, con ello, mayores velocidades de trabajo y mayores densidades de integración. En tal sentido, la tensión de alimentación de 3 V está sustituyendo paulatinamente a la habitual de 5 V; cada vez es más frecuente que circuitos integrados complejos y de muy alta velocidad se suministren solamente para alimentación de 3 V e incluso para tensiones inferiores (2,5 V; 1,8 V).

- Tensiones e intensidades V_o , I_o , V_i , I_i , para ambos valores booleanos

Interesa conocer los intervalos de tensión propios del **0** y del **1** booleano tanto en la entrada como en la salida, así como las intensidades asociadas a dichos intervalos.

Los valores de las tensiones pueden expresarse en forma conjunta y directa mediante una gráfica «tensión de salida / tensión de entrada» (función de transferencia V_o / V_i), representando para cada valor de tensión de entrada entre 0 y V_{CC} el valor de tensión que adopta la salida.

Generalmente, los catálogos no incluyen estas curvas de transferencia sino que expresan tales datos en forma tabular mediante los siguientes parámetros:

+ referentes a la entrada: $V_{iLm\acute{a}x}$: tensión máxima que la entrada entiende como **0**

$V_{iHm\acute{i}n}$: tensión mínima que la entrada entiende como **1**

I_{iL} : intensidad en la entrada cuando su valor es **0**

I_{iH} : intensidad en la entrada cuando su valor es **1**

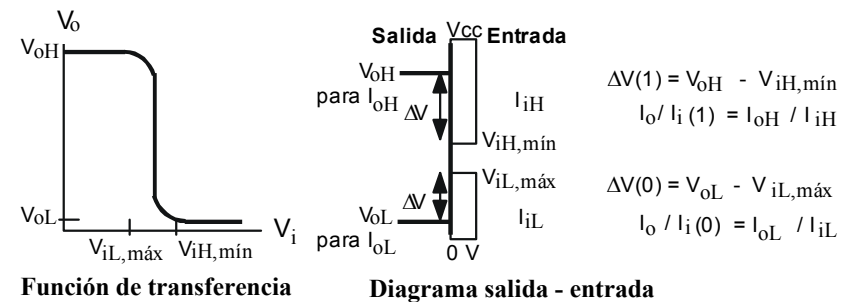
+ referentes a la salida: V_{oL} : tensión de salida para valor booleano **0**

V_{oH} : tensión de salida para valor booleano **1**

ambas tensiones de salida dependen de la intensidad que se requiere de ella, de forma que se expresan siempre para una intensidad determinada: I_{oL} e I_{oH} , respectivamente.

Por convenio se asigna a las intensidades, tanto en las salidas como en las entradas, el signo + cuando circulan hacia dentro de la puerta lógica o circuito integrado y el signo - si lo hacen hacia fuera.

Los datos citados de tensiones e intensidades de entrada y de salida, para los valores booleanos **0** y **1**, pueden expresarse agrupados según el diagrama de la siguiente figura (se representa la salida a la izquierda y la entrada a la derecha para referirse a un nudo booleano, es decir, a la conexión de la salida de una puerta con la entrada de la siguiente, supuestas del mismo tipo):



El anterior diagrama salida/entrada expresa, en forma conjunta y resumida, toda la información de interés sobre las tensiones e intensidades de los dos valores booleanos:

- tensión de salida correspondiente para una determinada intensidad de salida
- intervalo de tensión que la entrada acepta como tal valor booleano
- intensidad que requiere la entrada.

En dicho diagrama queda reflejado, asimismo, el margen de tensión para cada uno de los valores booleanos: $\Delta V(0) = V_{iLm\acute{a}x} - V_{oL}$ y $\Delta V(1) = V_{oH} - V_{iHm\acute{i}n}$ y la relación entre las intensidades de salida y de entrada I_o / I_i en cada uno de ellos.

El margen de tensión constituye un intervalo de seguridad, de forma que modificaciones de la tensión de entrada que se mantengan dentro del mismo no afectan a la tensión de salida, o sea, al valor booleano que proporciona la puerta. El cociente entre intensidades representa el número de entradas que una salida puede soportar en situación estática (en términos de intensidad suministrable a las mismas).

• Velocidad de trabajo

Es obvio que la velocidad es un dato fundamental en sistemas que han de realizar miles de operaciones en tiempos mínimos; la velocidad de trabajo determina la capacidad operativa del sistema. Más aún, habida cuenta de que las operaciones digitales son en gran medida repetitivas, en muchas ocasiones resultan preferibles arquitecturas con unos pocos módulos que realizan grandes series de operaciones sucesivas, en lugar de utilizar un mayor número de módulos en paralelo. De esta forma, si la velocidad de trabajo lo permite, cabe reducir el circuito operativo al mínimo, con la consiguiente reducción de tamaño y de coste económico.

Al llegar una señal a la entrada de una puerta lógica, la respuesta a dicha señal no aparece instantáneamente en la salida, sino que existe un cierto tiempo de retardo; este tiempo es diferente según la transición de estado de la puerta sea de **0** a **1** o de **1** a **0**:

- **t_{PLH}** o **t_{p(1)}**: «tiempo de propagación del **1**», retardo de la salida respecto de la entrada cuando la salida cambia de **0** a **1**;
- **t_{PHL}** o **t_{p(0)}**: «tiempo de propagación del **0**», retardo de la salida respecto de la entrada cuando la salida cambia de **1** a **0**.

Ambos tiempos de propagación suelen tener valores próximos entre sí, lo cual permite utilizar su promedio como tiempo de propagación genérico: $t_p = (t_{PLH} + t_{PHL}) / 2$.

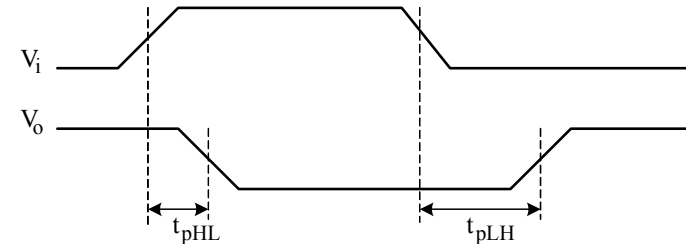
Los tiempos de propagación, es decir, los retrasos de la salida respecto a las variaciones de las entradas dependen de la impedancia de carga conectada sobre la salida, es decir, de la capacidad equivalente que presenta el conjunto de entradas conectadas a ella.

Por ello, los tiempos de propagación se miden en condiciones similares al funcionamiento normal de la puerta, supuesto un número máximo razonable de entradas conectadas a su salida; en el caso CMOS, sus entradas son de tipo capacitivo, del orden de unos pocos picofaradios, de forma que los tiempos de propagación CMOS suelen medirse y expresarse en relación a una carga de 50 pF.

Los valores típicos de los tiempos de propagación se expresan para 25°C, ya que tales tiempos dependen de la temperatura, aumentando con ella; esta dependencia se debe a que las resistencias de paso de los transistores MOS aumentan con la temperatura, por disminuir con ella la movilidad de sus portadores. Interesa, por ello, evitar el funcionamiento de los circuitos digitales a temperaturas altas y, si es necesario, se les dota de adecuados mecanismos de refrigeración.

Al conectar dos puertas, una a la salida de la otra, el tiempo de propagación del conjunto es mayor que los tiempos individuales pero es inferior a la suma de ambos. Es decir, los tiempos de propagación no son linealmente acumulativos ya que la segunda puerta inicia su conmutación antes de que la primera complete la suya.

Para facilitar la suma de tiempos en puertas sucesivas, cada tiempo de propagación suele medirse por el retraso entre el punto medio de conmutación (tensión $V_{cc}/2$) de la onda de entrada y el punto medio de la conmutación de la señal de salida.



Una estimación aproximada de la velocidad de trabajo de un circuito digital puede hacerse en la forma siguiente:

- parece razonable que en una secuencia de operaciones de una puerta lógica o, lo que es lo mismo, en una secuencia de bits, el tiempo dedicado a cada uno de ellos ha de ser mayor que el tiempo de propagación de la puerta;
- si denotamos con **t_{BIT}** el tiempo mínimo para la propagación y formación estable de un bit, ha de asegurarse que, al menos, **t_{BIT} > 2 t_p** para permitir que cada bit se estabilice antes de pasar al siguiente;
- de manera que **f < 1/2 t_p** es una estimación adecuada de la velocidad de trabajo en términos de bits por segundo.

Otra medida de la velocidad de trabajo de una familia lógica la constituye la máxima frecuencia de reloj que admiten sus biestables síncronos **f_{CK}**, que determina la máxima velocidad de trabajo de los sistemas secuenciales y, en particular, de los contadores y de los registros.

Cada una de las semiondas del reloj corresponde a la actuación de uno de los dos biestables básicos que configuran un biestable síncrono (estructura *master/slave*), de forma que cada semionda debe abarcar un intervalo de, al menos, **2 t_{BIT}**; el periodo del reloj debe ser mayor que **4 t_p** y su frecuencia máxima será: **f_{CKmáx} ~ 1/(4 t_p)**.

De todas formas, el problema de la velocidad máxima de un circuito digital será considerado con mayor detalle en el capítulo 15 al estudiar el análisis de tiempos en los sistemas síncronos.

• Consumo

En general, un sistema digital está constituido por un gran número de puertas booleanas, de forma que el consumo energético, en términos de intensidad o de potencia, de una puerta individual queda multiplicado por un número relativamente alto. Ello tiene importancia desde dos puntos de vista, consumo de energía (y consiguiente alimentación del sistema) y disipación de calor (y mecanismos para facilitarla):

- la fuente de alimentación (que proporciona la tensión de alimentación V_{CC}) ha de suministrar suficiente intensidad, de acuerdo con el consumo global del sistema digital;
- la energía consumida por el sistema se disipa en el mismo en forma de calor que ha de ser desalojado para evitar un aumento excesivo de la temperatura, lo cual requiere, en ocasiones, el correspondiente sistema de refrigeración.

Intensidad de alimentación y disipación de calor constituyen dos aspectos a tener en cuenta en el diseño de sistemas digitales; la complejidad, tamaño y coste de la fuente de alimentación y del mecanismo de refrigeración dependen fuertemente del consumo, aumentando drásticamente con él.

Conviene tener en cuenta que el calentamiento de un circuito digital, por efecto de la disipación de potencia en el mismo, puede llevar a su destrucción pero, mucho antes que eso, afecta a los tiempos de propagación de sus puertas lógicas que aumentan con la temperatura. En muchas ocasiones se precisa de un adecuado sistema de refrigeración simplemente para asegurar la velocidad de trabajo del circuito. Por ejemplo, los procesadores Pentium de los PCs, para alcanzar las altas velocidades de trabajo que consiguen (varios GHz), requieren un fuerte disipador con un ventilador situados directamente sobre el propio circuito integrado,

En cada uno de los dos estados booleanos tendremos un consumo estático:

ICCL = intensidad consumida cuando la salida es **0**

ICCH = intensidad consumida cuando la salida es **1**

Los datos de consumo estático que figuran en los catálogos vienen expresados en términos de intensidad global consumida por el circuito integrado; es decir, en el caso de puertas lógicas el consumo no se expresa en términos de intensidad por puerta (ya que no puede medirse el consumo individual de una de ellas), siendo necesario dividir el dato de catálogo por el número de puertas que configuran el integrado.

El consumo estático no incluye el debido a las transiciones entre los dos estados booleanos. Por ello, además del consumo estático, ha de tenerse en cuenta el consumo dinámico, es decir, el que se produce durante las transiciones. En la familia **TTL** prevalece el consumo estático, siendo el dinámico despreciable respecto al estático; en cambio, en la familia **CMOS** el consumo estático es nulo mientras que el dinámico es apreciable para frecuencias por encima del MHz.

El consumo dinámico depende de la frecuencia de conmutación de las puertas booleanas: en cada transición se «gasta» una cantidad determinada de energía utilizada en la carga y descarga de las capacidades presentes en el circuito (la energía que se disipa al cargar o al descargar un condensador C a una tensión V es C.V²/2). Para efectuar el cálculo del consumo dinámico se utiliza la capacidad equivalente de la puerta a efectos de disipación de potencia **C_{pd}** (*power dissipation capacitance*)

$$P = C_{pd} \cdot (V_{CC})^2 \cdot f$$

siendo V_{CC} la tensión de alimentación y f la frecuencia de conmutación (considerada en forma de onda cuadrada: paso de **0** a **1** y posterior paso de **1** a **0**).

Por otra parte, en la evaluación global del consumo dinámico de un sistema digital ha de tenerse en cuenta que no todas sus puertas o biestables conmutan a la vez; aun más, en determinados subsistemas lo hace solamente un pequeño número de ellas. Por ejemplo, en una memoria RAM, al leer o escribir sobre ella solamente se activa uno de los múltiples registros que la forman (obviamente, también conmutan el decodificador que selecciona los registros y el circuito de control de entradas/salidas).

• Intervalo de temperaturas

Existe un intervalo de temperaturas para el que está garantizado el funcionamiento de los circuitos integrados digitales: el intervalo «normal» de funcionamiento va de -40°C a 85°C para CMOS y de 0°C a 70°C en TTL (en ambos casos con indicativo **74**).

Existen, además, series denominadas «militares» para aplicaciones que requieren mayor rango de temperaturas, de -55°C a 125°C; se distinguen porque su numeración empieza por **54** y su encapsulado es cerámico.

Ha de tenerse en cuenta que las características de una puerta lógica varían fuertemente con la temperatura; en general empeoran al aumentar la temperatura, lo cual se refleja en reducción de los márgenes de ruido y de la velocidad de trabajo. El mismo circuito desprende calor, como consecuencia de la disipación de la energía que utiliza en su funcionamiento, y causa una elevación de su propia temperatura que, en ocasiones, puede ser importante. Por ello, el diseño de un sistema digital ha de tener en cuenta el rango de temperaturas en el que va a trabajar y, si es preciso, debe incluir un mecanismo de refrigeración adecuado.

Otra indicación de temperatura que proporcionan los catálogos es el rango que soportan los circuitos integrados para su almacenamiento, que suele ser de -65°C a 150°C.

• *Conectividad.*

Se emplea un parámetro denominado *fan-out* (abanico de salida) o *capacidad de carga* para expresar el número de entradas que pueden conectarse sobre la salida de una puerta lógica. Este parámetro viene determinado por dos factores:

- el cociente entre las intensidades de salida y de entrada I_o/I_i para cada valor booleano, que representa el número máximo de entradas a las que la salida es capaz de suministrar adecuada intensidad sin deteriorar el valor booleano que les transmite;
- el cociente entre la capacidad de carga que la salida puede soportar (con referencia a unos tiempos de propagación determinados) y la capacidad equivalente de las entradas (de las puertas a conectar) C_L/C_i , ya que una capacidad de carga mayor se traducirá en una disminución de la velocidad de trabajo de la puerta (un aumento de sus tiempos de propagación).

En la familia TTL la limitación relativa al *fan-out* viene dada por el cociente entre intensidades I_o/I_i , pero no así en las series CMOS cuya intensidad de entrada es nula y lo que limita es la carga capacitiva que pueden soportar C_L/C_i .

C_L no es un parámetro característico del propio circuito digital, sino la capacidad de carga con la que ha sido medido el tiempo de propagación; es decir, para asegurar dicho tiempo de propagación es preciso que la capacidad que se conecta a la salida sea menor que C_L .

A efectos del *fan-out* o *capacidad de carga* de una puerta debe tenerse en cuenta que en el caso de bloques combinacionales, biestables, registros, etc.,... cada uno de los terminales de entrada se encuentra conectado a las entradas de varias puertas lógicas, de forma que su conexión sobre la salida de otra puerta o bloque supone una carga equivalente a varias entradas; es decir, cada una de las entradas de un bloque digital ha de contabilizarse en términos de su *carga equivalente* o sea del número de entradas individuales o básicas a las que se encuentra conectada.

Asimismo se utiliza el término *fan-in* (abanico de entrada) o *disponibilidad de entradas* para indicar el número de entradas que posee una puerta lógica. La limitación en cuanto al número máximo de entradas con que puede construirse una puerta depende de la estructura electrónica de la misma: en el caso CMOS el número de entradas no debe ser superior a 6 u 8 debido al hecho de que la conexión de múltiples transistores MOS en serie empeora en gran medida las características de velocidad e inmunidad frente al ruido de las puertas.

Conviene recordar, en relación con las entradas de los circuitos integrados MOS, que requieren una manipulación cuidadosa, debido a la posibilidad de perforación de la delgadísima capa de óxido que conforma la puerta de los transistores MOS; tal perforación puede producirse por la propia carga estática acumulada en el cuerpo de quien los maneja. En los catálogos, los fabricantes indican una serie de normas para una manipulación que evite las cargas estáticas (tanto el personal como los soportes e instrumentos deben estar adecuadamente conectados a «tierra»); asimismo, los circuitos integrados MOS deben conservarse en fundas antiestáticas y en ningún caso deben agarrarse por los terminales metálicos (*pines*) sino por la carcasa de plástico.

Interesa conocer la disponibilidad de diversas opciones de entradas y de salidas:

- Entradas con histéresis (entradas tipo *Schmitt*), que presentan dos tensiones de conmutación o comparación V_a y V_b y son útiles para evitar rebotes (y para la construcción de osciladores astables). En la familia CMOS (alimentada a 5 V) las tensiones V_a y V_b de las entradas de tipo *Schmitt* suelen ser simétricas, 2 y 3 V aproximadamente, mientras que en la familia TTL suelen ser mas bajas, 0,8 y 1,6 V.
- Salidas con posibilidad de desconexión (salidas *triestado*) que permiten conectar múltiples salidas en paralelo, actuando en cada momento una de ellas y manteniéndose en alta impedancia las demás.
- En ocasiones, se incluye en la salida una resistencia de valor alto conectada a 0 V (resistencia de *pulldown*) o a V_{CC} (*pullup*), para asegurar que en la inicialización o en estado de alta impedancia la salida adopte valor **0** o valor **1**, respectivamente.
- Salidas de *colector o drenador abierto* que prescinden de los transistores superiores (los que transmiten el **1**). Son puertas incompletas que requieren una resistencia de polarización R_p conectada a la alimentación V_{CC} y precisamente por eso resultan útiles para efectuar acoplos con otras familias lógicas o con otros componentes electrónicos que empleen niveles de tensión diferentes y para la conexión de varias salidas (operación "y" cableada). Existen adaptadores de este tipo (*buffer*) que admiten tensiones o intensidades relativamente altas (hasta 15 ó 30 V de tensión y unos 50 mA de intensidad).

• *Coste*

El coste es un parámetro esencial en cualquier diseño de ingeniería y suele entrar en competencia con otras especificaciones del mismo, como pueden ser la velocidad de trabajo, el conjunto de prestaciones, etc.,... En «productos de consumo», de fabricación en serie, el coste suele ser uno de los parámetros más importantes del diseño; en cambio, en la realización de equipos de producción o de prototipos o pequeñas series especializadas el coste suele quedar en un segundo orden de exigencias, precedido por las prestaciones que se requieren.

••• *Comentario en relación con la forma actual de construir los sistemas digitales*

Desde los años 70 las diversas familias lógicas integradas han permitido la disponibilidad de amplios catálogos de circuitos integrados, relativos a puertas lógicas, biestables, bloques combinacionales y bloques secuenciales. Con estos «elementos constructivos» (con estas piezas o *ladrillos digitales*) el diseño de un sistema digital consiste en la adecuada selección y conexión de circuitos integrados estándar; el resultado es una «tarjeta» formada por un conjunto de circuitos integrados sobre una placa de circuito impreso, cuyas pistas efectúan las conexiones entre ellos.

Los circuitos integrados estándar siguen siendo útiles para probar pequeños diseños, para simular el comportamiento de subcircuitos reducidos o, también, para prácticas de laboratorio en el proceso de aprendizaje (para entrar en contacto con las puertas y los bloques digitales y con las características de la tecnología).

Pero la forma de proceder basada en seleccionar y conectar circuitos integrados estándar «ha pasado a la historia». Hoy día, cualquier diseño digital se construye dentro de un único circuito integrado:

- para la realización de prototipos o de pequeñas series se dispone de circuitos integrados programables para «encajar» sobre ellos, por programación, el diseño específico que interesa (la variedad de los dispositivos programables, en cuanto a tamaño y capacidad de acoger diseños complejos, es enorme);
- cuando el número de ejemplares a utilizar es alto, es preferible fabricar el propio diseño, dando lugar a un ASIC (circuito integrado para una aplicación específica).

Por ello, cada vez se utilizan menos los circuitos integrados estándar y, con ello, se desdibuja la idea de «Familia lógica integrada»; lo que importa, en cuanto al diseño y al producto final, es:

- elegir un circuito integrado programable con capacidad y velocidad suficientes para nuestro diseño y conocer en profundidad las características de tal circuito integrado;
- o bien, seleccionar un fabricante y, dentro de su oferta, un tipo de ASIC adecuado para nuestro diseño y conocer, igualmente, las características de tales ASICs.

En todo caso, los aspectos funcionales a tener en cuenta son los descritos en este apartado y, por lo general, tanto los dispositivos programables como los ASICs pertenecen a la gran familia lógica CMOS, cuyas características hemos analizado en detalle en los tres últimos capítulos.

La tecnología CMOS, con sus diversas variantes y continuas mejoras (en especial, en lo que se refiere a tamaño de los transistores y, con ello, a la densidad de integración y a la velocidad de trabajo) ha sido durante la última década, y lo seguirá siendo en la presente y en la próxima, la más adecuada para la integración de circuitos digitales (incluyendo los programables).

10.3. El ruido en los sistemas digitales

En el entorno físico de los sistemas digitales se encuentran siempre presentes señales de tipo electromagnético que inciden sobre ellos, bien procedentes del medio ambiental en el que se encuentran (motores, relés, transformadores, radiofrecuencias, emisiones de los cables, perturbaciones de la red, etc.,...), bien producidas por los propios sistemas digitales (variaciones de consumo que generan parásitos sobre la alimentación, oscilaciones propias de los circuitos, acoplo entre señales, radiación de las pistas y cables de interconexión, etc.,...).

Todas estas señales reciben el nombre de «ruido» y son siempre indeseables para un sistema electrónico por cuanto pueden afectar a su correcto funcionamiento; en el caso de un sistema digital pueden modificar puntualmente los valores booleanos presentes en el mismo por desplazamiento de las tensiones en los nudos del circuito.

El problema del «ruido electromagnético» es mayor en las plantas industriales, en las que existe un gran número de máquinas, motores y sistemas eléctricos de tipo diverso, cuyo funcionamiento genera ondas electromagnéticas en una amplia gama de frecuencias que se transmiten, no solamente por el aire, sino también por todo tipo de cables o conductores y, en particular, a través de la red de tensión eléctrica. Asimismo, la propia red puede encontrarse distorsionada por las fuertes variaciones de consumo que suelen producirse, principalmente, por la conmutación sobre ella de elementos de potencia.

En un circuito electrónico las señales parásitas debidas al ruido se hacen presentes en términos de tensión transmitida a través de las interconexiones del sistema; pero, también, en términos de potencia que incide como onda electromagnética desde el exterior. En general, el ruido que genera el propio sistema digital se transmite por el mismo en forma de señal de tensión, principalmente por las líneas de alimentación (VCC y 0), y el ruido electromagnético ambiental se recibe como señal de potencia radioeléctrica, captada por las diversas líneas, componentes y bucles del sistema que actúan como pequeñas antenas receptoras, en particular las líneas de interconexión, incluidas las de alimentación.

[Como complemento a este apartado, el capítulo T3 trata con mayor detalle el problema del ruido en los sistemas electrónicos.]

10.3.1. Mecanismos físicos de generación y captación de ruido

Las leyes físicas ignoran las fronteras conceptuales que el diseñador impone a su circuito y parte de la energía del mismo puede alcanzar a otros circuitos de su entorno y a otras partes del propio circuito; además, por causa de esas mismas leyes físicas, los componentes circuitales no se comportan de forma ideal (no se limitan a ser los elementos de circuito en los que el diseñador piensa) sino que presentan multitud de efectos parásitos.

Por otra parte, los circuitos electrónicos, como circuitos eléctricos que son, serán afectados por los campos eléctricos, magnéticos y electromagnéticos que llegan a ellos, procedentes de otros equipos o fenómenos de naturaleza eléctrica.

Los principales mecanismos físicos de generación o captación de «ruido electromagnético» son los siguientes:

- tensiones producidas por variaciones de intensidad sobre elementos autoinductivos
- actuación de los bucles de intensidad (espiras) como receptores y como emisores
- oscilaciones debidas a la presencia de autoinducciones y capacidades parásitas
- acoplo capacitivo entre conductores próximos
- impedancia común en las líneas de retorno de varias señales.

A continuación, analizaremos con un poco de detalle estos fenómenos.

- a) Todo conductor presenta una cierta componente inductiva y responde a las variaciones de la intensidad que conduce con un transitorio de tensión:
- $$\Delta V = L \cdot dI/dt.$$

De esta forma las variaciones de intensidad sobre las líneas de alimentación producen perturbaciones que afectan a la tensión que transmiten. Lo mismo sucede con las variaciones de consumo sobre la red de tensión eléctrica, que provocan perturbaciones que son propagadas a través de la red.

También en las líneas de señal se produce este efecto autoinductivo, pero tiene poca importancia, debido a que, por lo general, tales líneas son muy cortas; una excepción a tener en cuenta puede ser la línea que transmiten el reloj en los sistemas síncronos.

Los picos (*glitches*) de tensión originados dependen de la amplitud de la variación de intensidad y, también, de la velocidad de dicha variación, de la pendiente dI/dt , de forma que ΔI reducidas pero muy rápidas pueden producir ΔV apreciables; una línea conductora de 10 cm, cuya autoinducción será cercana a 0,1 μH , responde a un aumento de intensidad de 10 mA en 1 ns con una variación de tensión de 1 V.

En la conmutación de una puerta booleana se producen importantes dI/dt , debidas a la carga y descarga de sus capacidades parásitas y de las capacidades de entrada de las puertas conectadas a su salida.

Asimismo, en la conmutación suele presentarse un fuerte pico de intensidad, debido a que durante un muy pequeño intervalo de tiempo pueden conducir a la vez los dos transistores de la etapa de salida de la puerta: los dos planos P y N en el caso CMOS o los dos transistores de la configuración *totem pole* en TTL. Este pico de intensidad a través de la malla de salida es debido a que antes de pasar a corte los transistores inferiores comienzan a conducir los superiores o viceversa; no tiene importancia respecto al consumo, ya que su duración es mínima, pero sí respecto a la generación de ruido porque su amplitud es apreciable y su pendiente muy alta.

De esta forma, la conmutación de las puertas booleanas genera perturbaciones sobre la tensión de alimentación que afectan al conjunto del circuito digital. Este ruido, producido sobre las líneas de alimentación por las variaciones de intensidad en las conmutaciones de las puertas, puede reducirse en gran medida utilizando condensadores de desacoplo: pequeños condensadores de unos 10 nF conectados a los terminales de alimentación y situados justo al lado de cada circuito integrado.

La misión de los condensadores de desacoplo es «filtrar las altas frecuencias», suministrando directamente las variaciones bruscas de intensidad. No deben ser condensadores electrolíticos, ya que éstos presentan a altas frecuencias una componente inductiva en serie no despreciable por lo que no resultan efectivos para el filtrado de variaciones muy rápidas.

- b) En los circuitos electrónicos una señal eléctrica es transmitida de un punto a otro del circuito mediante una intensidad que, luego, requiere una línea de retorno; es decir, toda señal eléctrica, incluida la propia tensión de alimentación, forma un «bucle de intensidad» (circula a través de una espira cerrada).

Todo bucle o espira es un receptor de campos magnéticos y electromagnéticos y la efectividad de tal recepción es tanto mayor cuanto lo es el área del bucle. De forma, que los cables y pistas de conducción de un circuito actúan como antenas y como espiras receptoras de interferencias: recogen el efecto de los campos magnéticos y de las ondas electromagnéticas que llegan hasta ellos.

De ahí la conveniencia de minimizar la longitud de las conexiones y el área de los bucles: debe prestarse gran atención al diseño del circuito impreso, tanto en la colocación de los componentes como en las pistas de conexión. Los circuitos impresos no son un mero soporte mecánico y un simple conexionado eléctrico, sino que determinan la topología del circuito y, con ella, los acoplamientos de las perturbaciones: un buen diseño geométrico es fundamental para prevenirlas.

Pero, además, en sentido inverso, cuando la intensidad es variable, los bucles generan campos magnéticos y ondas electromagnéticas, es decir, cada bucle de intensidad puede producir interferencias, que causarán perturbaciones sobre el propio circuito y sobre otros circuitos. También en lo que se refiere a la emisión de interferencias interesa en gran medida minimizar la longitud de las líneas de conexión y el área de los bucles.

- c) Cables o pistas largas presentan una autoinducción parásita apreciable que puede combinarse con las capacidades parásitas del circuito y producir oscilaciones.

Normalmente, este tipo de oscilaciones se evita con condensadores de desacoplo que filtren las alimentaciones, lo más cerca posible de las etapas funcionales.

- d) Dos conductores próximos (por ejemplo, dos conductores de un mismo cable plano o dos pistas que circulan paralelas por una placa) presentan un efecto capacitivo entre ellos, de forma que una variación de tensión en uno de ellos es transmitida parcialmente como transitorio al otro conductor.

Ésta es otra razón para reducir la longitud de las pistas o cables de conexión y procurar que no circulen paralelas entre sí; cuando lo anterior no es posible (buses o cables planos) puede reducirse mucho el acoplo capacitivo intercalando una pista intermedia conectada a tensión 0 (*masa*).

- e) Toda línea conductora presenta una impedancia (R, L) y, cuando por dicha línea viajan dos señales, tal impedancia es compartida por ambas (impedancia común) y las variaciones de tensión producidas por una de ellas afectan también a la otra.

La línea de retorno suele ser utilizada, a la vez, por varias alimentaciones y/o señales, lo cual supone la existencia de una impedancia común, de modo que las caídas o variaciones de tensión generadas sobre dicha impedancia afectan a las diversas etapas conectadas a la línea de retorno.

10.3.2. Medida de la inmunidad frente al ruido

Interesa conocer el margen disponible frente al ruido, es decir, el intervalo dentro del cual el ruido no produce errores sobre los valores booleanos, porque los desplazamientos de tensión que provoca quedan dentro de los intervalos asignados a tales valores.

La inmunidad frente al ruido debe considerarse tanto en términos de tensión, como en términos de potencia y, especialmente, en estos últimos: el ruido es una potencia parásita (energía actuante por unidad de tiempo) que perturba el sistema electrónico. El efecto de tal perturbación depende de la resistencia sobre la que actúa; sobre resistencias bajas generará débiles desplazamientos de tensión, mientras que sobre altas impedancias dará lugar a fuertes modificaciones de tensión.

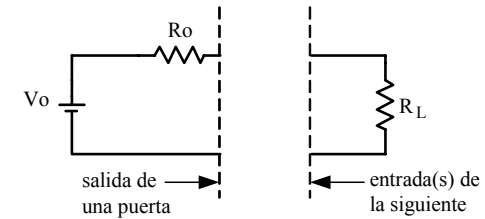
Se considera el efecto del ruido sobre los nudos booleanos: los valores booleanos están presentes en la salida de cada puerta lógica y sobre dicha salida se encuentra conectada la entrada o las entradas de las puertas siguientes. Para cada uno de los valores booleanos la diferencia entre la tensión de salida que corresponde a dicho valor y la tensión límite que la posterior entrada admite para ese mismo valor booleano expresa el margen de tensión, $\Delta V(0)$ y $\Delta V(1)$:

$$\Delta V(1) = V_{oH} - V_{iHmín} \quad \Delta V(0) = V_{iLmáx} - V_{oL}$$

Siempre que el efecto del ruido en tensión sea inferior a dicho margen la segunda puerta lógica actuará correctamente pues el desplazamiento en tensión permanece dentro del intervalo correspondiente al valor booleano.

El margen de tensión no es completamente representativo de la inmunidad frente al ruido; interesa conocer también el margen de potencia (es decir, en términos de energía por unidad de tiempo): $P = V^2/R$ $\Delta P = (\Delta V)^2/R$

Para calcular el margen de ruido en potencia es preciso conocer la resistencia equivalente presente en el nudo booleano:



En la conexión de una salida a una o varias entradas, la resistencia equivalente corresponde al paralelo de la resistencia R_o de salida de la primera puerta con las de entrada R_i de las siguientes y, en dicho paralelo, predomina la resistencia de salida R_o por ser de valor mucho menor que las de entrada (lo cual viene exigido para que exista un buen acoplo en tensión).

La resistencia de salida puede medirse (o calcularse a partir de los datos de catálogo) considerando el desplazamiento en tensión que se produce en la salida cuando se fuerza un consumo de intensidad a través de ella: $R_o = |V_o - V'_o| / I$, siendo V_o la tensión de salida sin consumo de intensidad y V'_o la tensión que corresponde a una intensidad I en la salida.

Conocida la resistencia de salida de la puerta, el margen de ruido en potencia será:

$$\Delta P(0) = (\Delta V(0))^2 / R_o(0) \quad \Delta P(1) = (\Delta V(1))^2 / R_o(1)$$

10.3.3. Diseño para evitar la producción y la recepción de ruido

En el diseño de un sistema digital es importante reducir al mínimo tanto la generación de ruido por parte del propio sistema como la incidencia del ruido electromagnético presente en su entorno. [Los elementos «antirruído» y las consideraciones que siguen serán tratados con mayor detalle en el capítulo T3.]

• Condensadores de desacoplo

La generación de ruido por parte de los circuitos integrados digitales puede deberse, como hemos visto, a dos factores principales: variaciones de consumo u oscilaciones.

Las variaciones de consumo, particularmente en la conmutación de las puertas, causan pequeñas oscilaciones amortiguadas sobre las líneas de alimentación (que necesariamente son largas para alcanzar a todo el circuito); conviene reducir tales oscilaciones mediante condensadores de desacoplo conectados sobre la alimentación: un condensador de unos 10 nF (no electrolítico) al lado de cada circuito integrado.

El condensador de desacoplo de la alimentación proporciona los «picos» de variación rápida de intensidad que se producen en las conmutaciones, evitando que tales dI/dt actúen sobre las autoinducciones parásitas de las pistas de alimentación; además, divide en dos partes el «bucle de alimentación», reduciendo su área efectiva.

Estos mismos condensadores sobre la alimentación eliminan las oscilaciones de alta frecuencia que aparecen en los circuitos electrónicos por acoplo entre capacidades y autoinducciones parásitas (asimismo, conviene utilizar pistas de conexión cortas).

Los condensadores de desacoplo configuran un filtro LC pasa-bajo hacia el circuito integrado que reduce las perturbaciones transmitidas a través de las líneas de alimentación; el filtrado se produce en ambas direcciones, también respecto a las perturbaciones del circuito integrado hacia las líneas de alimentación.

Conviene asimismo filtrar la alimentación en la entrada de la misma a cada placa circuital, mediante un par de condensadores de unos 100 μF y 100 nF, respectivamente (el segundo de ellos, no electrolítico); este filtrado pasa-baja actúa; también; en ambas direcciones: impide el paso de las perturbaciones que llegan por las líneas de alimentación y evita que las producidas en la placa se transmitan al resto del sistema.

• Apantallamiento y separación galvánica

Las interferencias exteriores llegan a un circuito electrónico mediante ondas electromagnéticas o a través de líneas de entrada o salida; se trata de poner pantallas frente a la propagación de tales perturbaciones.

Cuando un sistema digital va a trabajar en situación de alto ruido electromagnético ambiental (plantas industriales con máquinas y motores potentes, etc...), es preciso apantallarlos mediante una carcasa de tipo ferromagnético que constituya una buena jaula de Faraday y aislar galvánicamente, si es posible, sus entradas y salidas, por ejemplo mediante acopladores optoelectrónicos.

La mejor efectividad de las carcasas se consigue mediante la utilización de dos materiales: el exterior conductor (cobre) que refleja los campos electromagnéticos y el interior de tipo ferromagnético que atenúa fuertemente las ondas que lo atraviesan. Es preciso, además, minimizar las dimensiones lineales de las aberturas de la carcasa, asegurar la continuidad eléctrica de la tapa de la misma y apantallar adecuadamente los cables de conexión que salen de la carcasa (cables coaxiales, trenzados,...).

Es importante conectar la carcasa a la «masa» (tensión 0 V) del circuito que contiene para evitar acoplos capacitivos dobles entre conductores del circuito y la carcasa, que pueden causar peligrosas realimentaciones positivas; esta conexión debe hacerse en un solo punto, a través de un condensador.

Los acopladores optoelectrónicos integrados están constituidos por un diodo emisor de radiación y un transistor fotodetector; de esta forma separan físicamente su entrada de su salida y transmiten el correspondiente valor booleano a través de una radiación óptica.

Los optoacopladores permiten establecer una separación galvánica en las entradas y salidas que conectan el circuito con el exterior; esta separación interrumpe la continuidad eléctrica y, con ella, la comunicación de ruido a través de ella, filtra fuertemente el ruido (ya que el escalón energético para atravesar un optoacoplador es alto), divide los bucles en dos partes (reduciendo su área efectiva) y rompe los posibles bucles que se forman al conectar la «masa» de las líneas largas de entrada o salida a «tierra» en sus dos extremos.

• Filtro de red

En relación con la conexión a la red de tensión eléctrica de los circuitos alimentados desde ella, es necesario filtrar las perturbaciones de alta frecuencia que se transmiten a través de la red y, a la vez, impedir que el circuito transmita interferencias hacia la red.

Ambos propósitos se consiguen mediante un filtro de red adecuado; suelen ser filtros de tipo LC en configuración π que realizan el filtrado de las señales de alta frecuencia en ambos sentidos: un par de condensadores conectados a tierra a ambos lados de una bobina, duplicando dicho filtro para cada una de las dos líneas de entrada de la red y con ambas bobinas enrolladas sobre un toroide de ferrita de forma que los campos magnéticos debidos al consumo de intensidad se compensen y no se sature la ferrita.

• Plano de masa

Una última consideración, que resulta ser de suma importancia, se refiere a disponer de una buena distribución de la «masa» (nudo de tensión de referencia, 0 V); se trata de los caminos de retorno (tanto para la alimentación como para las señales), con el objetivo de reducir en lo posible la longitud de las líneas de retorno, el área de los bucles que conforman y la impedancia compartida que presentan.

Lo ideal, a ser posible, es configurar un «plano de masa», dedicando a ella toda una capa de circuito impreso multicapa. Sobre un plano de masa los caminos de retorno de la alimentación y de las señales se ajustan a la condición de recorrido mínimo en el caso de señales de baja frecuencia o de área mínima de bucle para señales de alta frecuencia; ambas situaciones son las más favorables desde el punto de vista de captación o generación de ruido.

Además, este tipo de caminos reduce también al mínimo la impedancia común compartida por varias señales: en el caso de bajas frecuencias prevalece el efecto resistivo de tal impedancia, mientras que para frecuencias altas importa más el aspecto inductivo.

Cuando no es posible disponer de un plano de masa, conviene distribuir dicha tensión 0V en la forma que más se aproxime al mismo: aprovechar toda la superficie libre del circuito impreso para generar «áreas de masa»; utilizar pistas directas, cortas y gruesas; conectar, en la medida de lo posible, las diversas pistas de «masa» en retícula;...

Cuando coexistan en una misma placa circuital partes digitales, analógicas y/o de potencia conviene trazar por separado las líneas de masa de las mismas, de forma que los retornos de intensidad de la parte digital no se vean afectados por los analógicos o de potencia y viceversa.

A1 Apéndice al capítulo 2

Simplificación de funciones por el método de Quine-McCluskey

Sin duda, los mapas de Karnaugh constituyen un método sencillo y eficaz para la simplificación «a mano» de funciones de pocas variables; es un método gráfico que facilita la búsqueda «visual» de términos simplificables entre sí. Pero su aplicación a funciones de más de 6 variables resulta muy laboriosa y desborda la capacidad de «agrupación visual» de términos de un operador humano. En tales casos resulta conveniente disponer de métodos de simplificación automática con ayuda de computador.

Ahora bien, los métodos gráficos presentan una cierta complejidad a la hora de traducirlos en algoritmos para ser ejecutados en computador y dicha dificultad aumenta fuertemente con el número de «dimensiones» a tener en cuenta. En el caso de los mapas de Karnaugh la hoja básica de 4 variables es bidimensional y es necesario añadir una dimensión más (una nueva hoja del mapa) por cada nueva variable: para n variables, el número de dimensiones efectivas del mapa de Karnaugh será de $n-2$.

En cambio, existen algoritmos de simplificación de actuación «lineal» (comparación sucesiva de vectores de entrada) cuya programación sobre computador es directa. Casi todos ellos se basan en el método de simplificación desarrollado por W. V. Quine en 1955 y ampliado posteriormente por E. J. McCluskey.

Esquemáticamente este método consiste en agrupar los vectores de entrada que activan la función (dan resultado **1** para ella) en clases diferenciadas por el número de variables cuyo valor sea **1** y calcular todas las posibilidades de simplificación entre vectores de dos clases sucesivas, eligiendo aquellas que son más eficientes.

Quizás la manera más sencilla de comprender este método de simplificación sea su aplicación a un caso concreto:

Sea la función "ser número primo" para números binarios de 5 dígitos **edcba**. Los vectores de entrada que la activan son los siguientes:

$1 = 00001$ $2 = 00010$ $3 = 00011$ $5 = 00101$ $7 = 00111$
 $11 = 01011$ $13 = 01101$ $17 = 10001$ $19 = 10011$
 $23 = 10111$ $29 = 11101$
 $31 = 11111$

Separados dichos vectores de entrada en clases según el número de unos que contienen y numerados sucesivamente resulta:

```

1      0 0 0 0 1
2      0 0 0 1 0
-----
3      0 0 0 1 1
4      0 0 1 0 1
5      1 0 0 0 1
-----
6      0 0 1 1 1
7      0 1 0 1 1
8      0 1 1 0 1
9      1 0 0 1 1
-----
10     1 0 1 1 1
11     1 1 1 0 1
-----
12     1 1 1 1 1

```

Primera simplificación: cada vector se compara con todos los de la clase siguiente, seleccionando aquellos que tienen valores **1** en la misma posición que dicho vector y por tanto se diferencian en una sola variable; esta variable es eliminable ($a + \bar{a} = 1$) y se indica con "-" su ausencia en el término resultante:

```

1;3     0 0 0 - 1
1;4     0 0 - 0 1
1;5     - 0 0 0 1
2;3     0 0 0 1 -
-----
3;6     0 0 - 1 1
3;7     0 - 0 1 1
3;9     - 0 0 1 1
4;6     0 0 1 - 1
4;8     0 - 1 0 1
5;9     1 0 0 - 1
-----
6;10    - 0 1 1 1
7;      -----
8;11    - 1 1 0 1
9;10    1 0 - 1 1
-----
10;12   1 - 1 1 1
11;12   1 1 1 - 1

```

(1) este término no es simplificable con la clase siguiente, pero no es preciso mantenerlo ya que ha sido recogido al simplificar la clase anterior.

Segunda simplificación: se produce en forma análoga a la primera pero solamente son simplificables aquellos vectores cuyos valores "-" coinciden, difiriendo en una de las restantes variables.

1,3; 4,6	0 0 – – 1	
1,3; 5,9	– 0 0 – 1	
1,4; 3,6	0 0 – – 1	
1,5; 3,9	– 0 0 – 1	
2,3;	0 0 0 1 –	(2) término no simplificable con la clase siguiente; es preciso mantenerlo, ya que no ha sido recogido en los términos anteriores.
3,6; 9,10	– 0 – 1 1	
3,7;	0 – 0 1 1	(2)
3,9; 6,10	– 0 – 1 1	
4,6;	– – – –	(1) término simplificado con la clase anterior
4,8;	0 – 1 0 1	(2)
5,9;	– – – –	(1)
6,10;	– – – –	(1)
8,11;	– 1 1 0 1	(2)
9,10;	– – – –	(1)
10,12;	1 – 1 1 1	(2)
11,12;	1 1 1 – 1	(2)

Agrupando términos iguales en cada una de las clases:

1,3,4,6	0 0 – – 1	A
1,3,5,9	– 0 0 – 1	B
2,3	0 0 0 1 –	C
3,6,9,10	– 0 – 1 1	D
3,7	0 – 0 1 1	E
4,8	0 – 1 0 1	F
8,11;	– 1 1 0 1	G
10,12;	1 – 1 1 1	H
11,12;	1 1 1 – 1	I

No es posible una nueva simplificación de estos términos, pues en ningún caso coinciden las "-" de vectores de dos clases sucesivas.

Tabla de cobertura: finalizado el proceso de simplificación se construye la tabla de cobertura de los vectores iniciales por los términos resultantes de la simplificación.

vectores iniciales	Términos								
	A	B	C	D	E	F	G	H	I
1	x	x							
2			x						
3	x	x	x	x	x				
4	x					x			
5		x							
6	x			x					
7					x				
8						x	x		
9		x		x					
10				x				x	
11							x		x
12								x	x

Se trata de elegir en la tabla de cobertura los términos de forma que con el mínimo número de ellos se cubran todos los vectores iniciales; para hacer esta selección se comienza por los casos en que el vector inicial solamente es cubierto por uno de los términos resultantes y, por tanto, dicho término ha de ser necesariamente recogido (como por ejemplo, para los vectores 2, 5 y 7, que requieren los términos **C**, **B** y **E**, respectivamente).

En la tabla anterior el subconjunto: **A**, **B**, **C**, **E**, **G** y **H** cubre todos los vectores iniciales: $y = A + B + C + E + G + H$

$$y = \bar{e}.\bar{d}.a + \bar{d}.\bar{c}.a + \bar{e}.\bar{d}.\bar{c}.b + \bar{e}.\bar{c}.b.a + d.c.\bar{b}.a + e.c.b.a$$

La aplicación del método de Quine-McCluskey, hecha «a mano», resulta bastante pesada y la selección de términos en el mapa de cobertura puede ser compleja, pero, en cambio, este método resulta muy apropiado para su programación en computador; dicha programación viene facilitada por el hecho de que la diferencia aritmética de los valores decimales correspondientes a dos vectores simplificables es una potencia entera de 2.

En el caso de funciones con vectores de entrada para los que «no importa» (*don't care*) el valor booleano de salida que se les asigne, dichos vectores son tomados también inicialmente, pero se prescinde de ellos en la tabla de cobertura.

Por ejemplo, en el caso de aplicar la función "ser número primo" a las cifras decimales, el proceso de simplificación sería el siguiente:

cifras que son números primos:

1 = **0001** 2 = **0010** 3 = **0011** 5 = **0101** 7 = **0111**

vectores de entrada que no se van a presentar (pues no corresponden a cifras decimales):

10 = **1010** 11 = **1011** 12 = **1100** 13 = **1101** 14 = **1110** 15 = **1111**

Estableciendo las clases según el número de unos que tales vectores contienen y numerados sucesivamente resulta:

1 **0 0 0 1**
2 **0 0 1 0**

3 **0 0 1 1**
4 **0 1 0 1**
(5) **1 0 1 0**
(6) **1 1 0 0**

7 **0 1 1 1**
(8) **1 0 1 1**
(9) **1 1 0 1**
(10) **1 1 1 0**

(11) **1 1 1 1**

Se han indicado entre paréntesis aquellos vectores de entrada para los cuales es indiferente (no importa) la salida que se asigne; dichos vectores se toman sólo a efectos de simplificación.

Primera simplificación:

1;3	0 0 - 1	3;7	0 - 1 1	7;(11)	- 1 1 1
1;4	0 - 0 1	3;(8)	- 0 1 1	(8);(11)	1 - 1 1
2;3	0 0 1 -	4;7	0 1 - 1	(9);(11)	1 1 - 1
2;(5)	- 0 1 0	4;(9)	- 1 0 1	(10);(11)	1 1 1 -
-----		(5);(8)	1 0 1 -		
		(5);(10)	1 - 1 0		
		(6);(9)	1 1 0 -		
		(6);(10)	1 1 - 0		

Segunda simplificación:

1,3; 4,7	0 - - 1	3,7; (8),(11)	- - 1 1
1,4; 3,7	0 - - 1	3,(8); 7,(11)	- - 1 1
2,3; (5),(8)	- 0 1 -	4,7;(9),(11)	- 1 - 1
2,(5); 3,(8)	- 0 1 -	4,(9); 7,(11)	- 1 - 1
-----		(5),(8); (10),(11)	1 - 1 -
		(5),(10); (8),(11)	1 - 1 -
		(6),(9); (10),(11)	1 1 - -
		(6),(10); (9),(11)	1 1 - -

Agrupando términos iguales en cada una de las clases:

1,3,4,7	0 - - 1	A
2,3,(5),(8)	- 0 1 -	B

3,7,(8),(11)	- - 1 1	C
4,7,(9),(11)	- 1 - 1	D
(5),(8),(10),(11)	1 - 1 -	
(6),(10),(9),(11)	1 1 - -	

No es posible una nueva simplificación de estos términos, pues en ningún caso coinciden las "-" de vectores de dos clases sucesivas. Tampoco es necesario recoger los últimos dos términos porque solamente cubren a vectores de entrada para los que es indiferente la salida que se produzca.

La tabla de cobertura para los vectores que interesan (1, 2, 3, 4, 7) es la siguiente:

Términos				
Vectores iniciales	A	B	C	D
1	x			
2		x		
3	x	x	x	
4	x			x
7	x		x	x

Necesariamente ha de tomarse el término **B** para cubrir el vector 2 y junto con el término **A** cubren todos los vectores: $y = \bar{d}.a + c.b$

El método de Quine-McCluskey ha sido extendido de forma que no sea necesario conocer los términos mínimos o vectores de entrada que activan la función, sino que pueda aplicarse a partir de cualquier expresión de la función como suma de términos producto, es decir, a partir de los subconjuntos de valores de las entradas que hacen la función **1**.

Denominemos subvector de entrada a cualquier subconjunto de valores de las entradas que no incluya a todas ellas (por ejemplo, para 4 variables **dcba**: **00-1**, **-101**, **1-0-**, **-1-1**, **-0--**,...). A cada subvector de entrada le corresponde un término producto, que, obviamente, no contiene todas las variables de entrada (en el ejemplo anterior: $\bar{d}.c.a$, $c.\bar{b}.a$, $d.\bar{b}$, $c.a$, $\bar{c}$,...).

En general, en funciones cuyo número de variables no sea pequeño resulta mucho más sencillo conocer y anotar un conjunto completo de los subvectores de entrada que la activan que detallar todos los vectores de entrada que dan resultado **1**, ya que el número de los segundos puede ser muy superior al de los primeros.

Por ejemplo, un codificador de prioridad de 7 líneas de entrada (numeradas de 1 a 7), tal que en su salida señale el número de la línea superior que se encuentre activada, estará configurado por 3 funciones **C B A** de 7 variables **g f e d c b a**, con 128 términos mínimos posibles. Pero los únicos subvectores de entrada que interesan son los siguientes (se expresa al lado de cada uno el vector de salida que debe producir):

	C B A
1 - - - - -	1 1 1
0 1 - - - -	1 1 0
0 0 1 - - -	1 0 1
0 0 0 1 - -	1 0 0
0 0 0 0 1 -	0 1 1
0 0 0 0 0 1	0 1 0
0 0 0 0 0 0 1	0 0 1
0 0 0 0 0 0 0	0 0 0

El procedimiento de aplicación del método de Quine-McCluskey a partir de un conjunto completo de subvectores de entrada que activan la función es diferente del indicado anteriormente. En este caso es preciso comparar cada subvector con todos los demás, para detectar aquellos casos en que un subvector es simplificable con otro o está contenido en él (resulta cubierto por el otro subvector); además, es preciso aplicar un teorema de consenso entre los subvectores, cuando ello sea posible, para generar nuevos subvectores que también activan la función y que pudieran facilitar la simplificación.

El teorema de consenso entre subvectores puede ser enunciado en la siguiente forma: si $a.\alpha$ y $\bar{a}.\beta$ son términos producto que corresponden a subvectores de entrada que activan la función **y**, también activa dicha función el subvector correspondiente al término $\alpha.\beta$ (si lo hubiere, pues $\alpha.\beta$ puede ser nulo como resultado de incluir una variable y su negada).

En el caso del codificador de prioridad de 7 líneas de entrada la primera de sus tres funciones booleanas **C** da valor **1** para los cuatro primeros subvectores, que son los únicos que activan dicha función:

1 - - - - - 0 1 - - - - 0 0 1 - - - 0 0 0 1 - - -

Por simplificaciones sucesivas se obtienen los siguientes subvectores que cubren a los anteriores:

1 - - - - - - 1 - - - - - - 1 - - - - - - 1 - -

$$C = g + f + d + c$$

La segunda de las funciones **B** da valor **1** para el siguiente conjunto de subvectores, el cual resulta completo, es decir, cubre todas las posibilidades de activar dicha función:

1 - - - - - 0 1 - - - - 0 0 0 1 - - 0 0 0 0 1 -

Por simplificaciones sucesivas se llega a los siguientes subvectores:

1 - - - - - - 1 - - - - - - 0 0 1 - - - - 0 0 - 1 -

$$B = g + f + \bar{e}.\bar{d}.c + \bar{e}.\bar{d}.b = g + f + \bar{e}.\bar{d}.(c + b)$$

La tercera función **A** da valor **1** para el siguiente conjunto de subvectores:

1 - - - - - 0 0 1 - - - 0 0 0 1 - - 0 0 0 0 0 1

Simplificando se obtienen los siguientes subvectores:

1 - - - - - - 0 1 - - - - - 0 - 0 1 - - - 0 - 0 - 0 1

$$A = g + \bar{f}.e + \bar{f}.\bar{d}.c + \bar{f}.\bar{d}.\bar{b}.a = g + \bar{f}.[e + \bar{d}.(c + \bar{b}.a)]$$

Una excelente presentación del método de Quine-McCluskey generalizado, de su formulación algorítmica y de su programación informática se encuentra en el libro de J.F. Wakerley, *Digital Design. Principles and Practices* (Prentice-Hall International Editions. 2000. 3ª edición. Páginas 236 - 243).

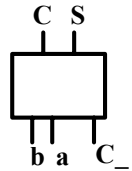
Por otra parte, el método de Quine-McCluskey también permite abordar la simplificación conjunta de múltiples funciones de las mismas variables, de forma que se minimice el número total de términos producto necesarios (y no el número de términos producto para cada una de ellas): *simplificación multifunción*.

A2 Apéndice a los capítulos 3 y 4

Propagación rápida de acarreo en los sumadores

El sumador es un bloque muy común en los sistemas digitales; además, una forma simplificada del sumador, el incrementador ($A + 1$), se utiliza para construir contadores de gran tamaño.

Un sumador se configura en forma modular mediante celdas sumadoras:



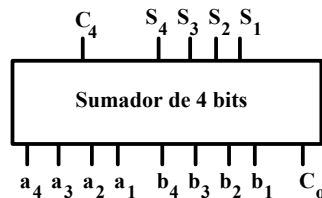
$$S = (a \oplus b) \oplus C_{-}$$

$$C = a \cdot b + (a + b) \cdot C_{-}$$

Al conectar en cascada estas celdas los retrasos con los que se calculan los sucesivos acarreos son acumulativos; este tipo de propagación del arrastre recibe el nombre de «acarreo sucesivo»: *ripple carry* (acarreo propagado por onda).

En el capítulo 2 se introdujo un indicador, el índice de propagación i_p , referido al número de puertas sucesivas que afectan a una función: el índice de propagación de la función $C_i = a_i \cdot b_i + (a_i + b_i) \cdot C_{i-1}$ (construido con puertas inversoras) es 2; un sumador de 64 bits tendrá un índice de propagación 128, referido al bit más significativo $S_{65} = C_{64}$. Los tiempos de propagación asociados a un índice tan elevado limitan mucho la velocidad de trabajo de los sumadores.

Una forma de disminuir los tiempos de propagación de un sumador grande consiste en utilizar celdas sumadoras más amplias (de «granularidad» más gruesa), por ejemplo, celdas sumadoras de 4 dígitos.



Las funciones booleanas que conforman este sumador de 4 bits son las siguientes:

$$S_i = (a_i \oplus b_i) \oplus C_{i-1} \quad \text{para } i = 1 \dots 4$$

$$C_1 = a_1 \cdot b_1 + (a_1 + b_1) \cdot C_0$$

$$C_2 = a_2 \cdot b_2 + (a_2 + b_2) \cdot (a_1 \cdot b_1 + (a_1 + b_1) \cdot C_0)$$

$$C_3 = a_3 \cdot b_3 + (a_3 + b_3) \cdot (a_2 \cdot b_2 + (a_2 + b_2) \cdot (a_1 \cdot b_1 + (a_1 + b_1) \cdot C_0))$$

$$S_5 = C_4 = a_4 \cdot b_4 + (a_4 + b_4) \cdot (a_3 \cdot b_3 + (a_3 + b_3) \cdot (a_2 \cdot b_2 + (a_2 + b_2) \cdot (a_1 \cdot b_1 + (a_1 + b_1) \cdot C_0)))$$

En las funciones anteriores (celda sumadora de 4 dígitos) se ha aplicado la recursividad del acarreo para calcular los sucesivos arrastres directamente, a partir de las entradas de la celda; todos los arrastres (construidos con puertas inversoras) presentan índice de propagación 2, incluyendo el arrastre global de la celda C_4 , que irá conectado a la celda siguiente para construir sumadores más grandes.

Para un sumador de 64 dígitos se necesitan 16 celdas sucesivas de 4 bits, de forma que el dígito más significativo S_{65} tendrá un índice de propagación 32 (4 veces menor que en el sumador de celdas de 1 bit).

Aún es posible conseguir una propagación más «rápida» del arrastre (*lookahead carry*: acarreo anticipado), aprovechando intensivamente la recursividad de la función booleana C_i en la forma que sigue.

Podemos expresar el arrastre C_i en términos de generación y propagación del mismo

$$C_i = a_i \cdot b_i + (a_i + b_i) \cdot C_{i-1} = g_i + p_i \cdot C_{i-1}$$

$$\text{siendo } g_i = a_i \cdot b_i \quad \text{generación de acarreo}$$

$$p_i = a_i + b_i \quad \text{propagación de acarreo}$$

y aplicando la recursividad propia de esta fórmula ($C_i \rightarrow C_{i-1}$)

$$C_i = g_i + p_i \cdot g_{i-1} + p_i \cdot p_{i-1} \cdot g_{i-2} + \dots + p_i \cdot p_{i-1} \cdot p_{i-2} \cdot \dots \cdot p_{m+1} \cdot C_m$$

se obtiene una función que permite calcular, «en un solo paso», un acarreo avanzado (el i -ésimo) a partir de otro inferior (el m -ésimo).

Dado que las puertas lógicas a utilizar son inversoras, conviene efectuar las siguientes transformaciones para configurar la función anterior con puertas inversoras:

$$g'_i = \overline{g_i} = \overline{a_i \cdot b_i} = a_i \cdot b_i$$

$$p'_i = \overline{p_i} = \overline{a_i + b_i} = a_i \Delta b_i;$$

en la expresión de C_i se puede sustituir $g_i = p_i \cdot g_i$, ya que siempre que $g_i = 1$, $p_i = 1$

$$\begin{aligned} C_i &= g_i + p_i \cdot C_{i-1} = g_i \cdot p_i + p_i \cdot C_{i-1} = p_i \cdot (g_i + C_{i-1}) = \overline{\overline{p_i + g_i \cdot C_{i-1}}} \\ &= \overline{p'_i + g'_i \cdot C_{i-1}} \end{aligned}$$

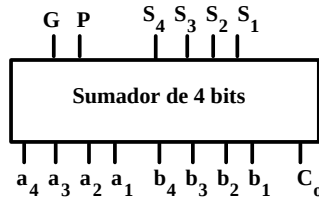
y aprovechando la recursividad

$$C_i = \overline{p'_i + g'_i \cdot p'_{i-1} + g'_i \cdot g'_{i-1} \cdot p'_{i-2} + \dots + g'_i \cdot g'_{i-1} \cdot g'_{i-2} \cdot \dots \cdot g'_{m+1} \cdot C_m}$$

Las operaciones iniciales p'_i , g'_i y $\overline{C_m}$ tienen índice de propagación 1 (una sola puerta lógica) y, también, C_i requiere una sola puerta a partir de ellas; de forma que el cálculo de C_i supone el paso a través de dos series de puertas booleanas sucesivas: índice de propagación $i_p = 2$.

Conforme a lo anterior y como ejemplo demostrativo de la propagación rápida de acarreo (*lookahead carry*), se desarrolla el diseño de un sumador de 64 dígitos:

Primer paso: configuración de un módulo de 4 bits



Las funciones booleanas que conforman este sumador de 4 bits son las siguientes:

$$S_1 = (a_1 \oplus b_1) \oplus C_0$$

$$S_2 = (a_2 \oplus b_2) \oplus C_1; C_1 = \overline{p'_1 + g'_1 \cdot C_0}$$

$$S_3 = (a_3 \oplus b_3) \oplus C_2; C_2 = \overline{p'_2 + g'_2 \cdot p'_1 + g'_2 \cdot g'_1 \cdot C_0}$$

$$S_4 = (a_4 \oplus b_4) \oplus C_3; C_3 = \overline{p'_3 + g'_3 \cdot p'_2 + g'_3 \cdot g'_2 \cdot p'_1 + g'_3 \cdot g'_2 \cdot g'_1 \cdot C_0}$$

El módulo debe contener, además, dos funciones «globales» de generación y propagación del acarreo

$$G = g_4 + p_4 \cdot g_3 + p_4 \cdot p_3 \cdot g_2 + p_4 \cdot p_3 \cdot p_2 \cdot g_1$$

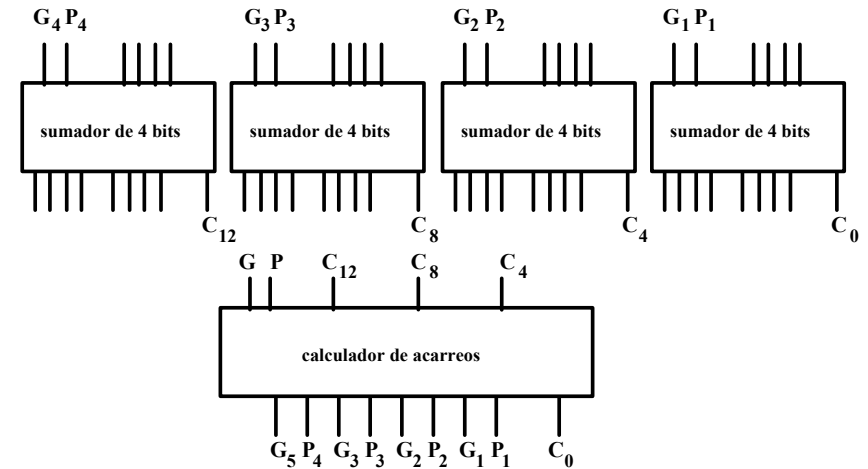
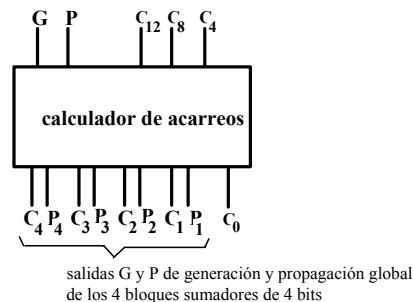
$$= \overline{g'_4 \cdot (p'_4 + g'_3) \cdot (p'_4 + p'_3 + g'_2) \cdot (p'_4 + p'_3 + p'_2 + g'_1)}$$

$$P = p_4 \cdot p_3 \cdot p_2 \cdot p_1 = \overline{p'_4 + p'_3 + p'_2 + p'_1}$$

p'_i , g'_i y $\overline{C_m}$ tienen índice de propagación 1 y, con ello, las funciones que calculan los arrastres C_i y, también, las funciones G y P tienen índice de propagación 2.

Segundo paso: agrupación de 4 módulos para construir un bloque de 16 bits

Además de los cuatro sumadores de 4 bits, se requiere un nuevo módulo que permita calcular los acarreos de entrada a los tres módulos superiores C_4 , C_8 y C_{12} , así como las funciones G P de generación y de propagación globales del sumador de 16 bits:



$$C_4 = G_1 + P_1 \cdot C_0$$

$$G = G_4 + P_4 \cdot G_3 + P_4 \cdot P_3 \cdot G_2 + P_4 \cdot P_3 \cdot P_2 \cdot G_1$$

$$C_8 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot C_0$$

$$P = P_4 \cdot P_3 \cdot P_2 \cdot P_1$$

$$C_{12} = G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot C_0$$

El índice de propagación de estas funciones es, asimismo, 2 (al aplicar una doble negación para formar puertas inversoras, resultan dos puertas sucesivas).

Tercer paso: agrupación de 4 bloques de 16 bits para configurar el sumador de 64 bits

De nuevo se requiere un «calculador de acarreos» idéntico al del apartado anterior para calcular los arrastres de entrada correspondientes a los tres bloques de 16 bits superiores: C_{16} , C_{32} , C_{48} .

Y, a partir de las salidas G P globales de este «calculador de acarreos», se calcula el dígito más significativo de la suma:

$$S_{65} = C_{64} = G + P \cdot C_0$$

El índice de propagación para dicho dígito S_{65} será:

celdas de 4 bits	2
+ «calculador de acarreos» del bloque de 16 bits	2
+ «calculador de acarreos» del sumador de 64 bits	2
+ cálculo de S_{65} , a partir de este último	2

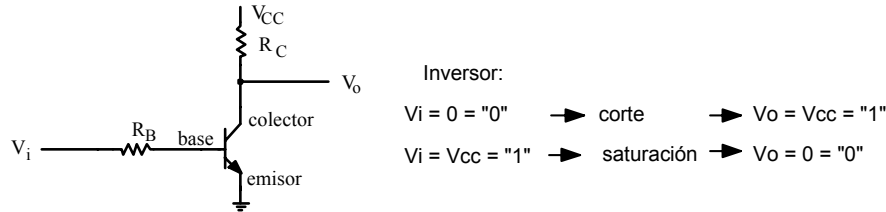
En conjunto, el índice de propagación global del dígito más significativo del sumador rápido (*lookahead carry*) de 64 bits es 8, muy inferior al correspondiente a una propagación sucesiva de acarreo (*ripple carry*) que es 128.

A3 Apéndice al capítulo 10

Puertas lógicas con transistores bipolares

El transistor bipolar como inversor

El circuito básico de un transistor en emisor común, según el esquema de la figura, realiza la operación booleana de negación o complementación: inversor.



Se utilizan los dos estados extremos de este circuito:

- estado de corte: $V_i = 0$ $I_C = 0$ $V_o = V_{CC}$
- estado de saturación: $V_i = V_{CC}$ $V_o = 0$.

La situación de corte queda asegurada siempre que $V_i < V_{\mu}$ (tensión umbral del diodo base-emisor, para el silicio 0,6 V), en cuyo caso $I_B = I_C = 0$ y la tensión de salida es igual a la de alimentación del circuito (no hay caída de tensión sobre R_C), $V_o = V_{CC}$.

La situación de saturación supone que $V_{CE} = 0$ y, por tanto, $I_C = V_{CC}/R_C$; para mantener tal intensidad de colector se necesita una intensidad de base suficiente:

$I_B > I_C / \beta = V_{CC} / (\beta \cdot R_C)$, siendo β la ganancia en corriente del transistor; como la tensión de entrada en este caso será $V_i = V(1) = V_{CC}$

$I_B = (V_{CC} - V_{BE}) / R_B \approx V_{CC} / R_B$ ha de ser mayor que $V_{CC} / (\beta \cdot R_C)$:

$V_{CC} / R_B > V_{CC} / (\beta \cdot R_C)$ o sea, $R_B < \beta \cdot R_C$

condición de saturación para una tensión de entrada igual a la tensión de alimentación.

Por otra parte, la intensidad de base del transistor no debe superar un valor límite $I_{B\text{máx}}$ propio del transistor, lo que impone una segunda condición: $V_{CC} / R_B < I_{B\text{máx}}$, o sea, $R_B > V_{CC} / I_{B\text{máx}}$.

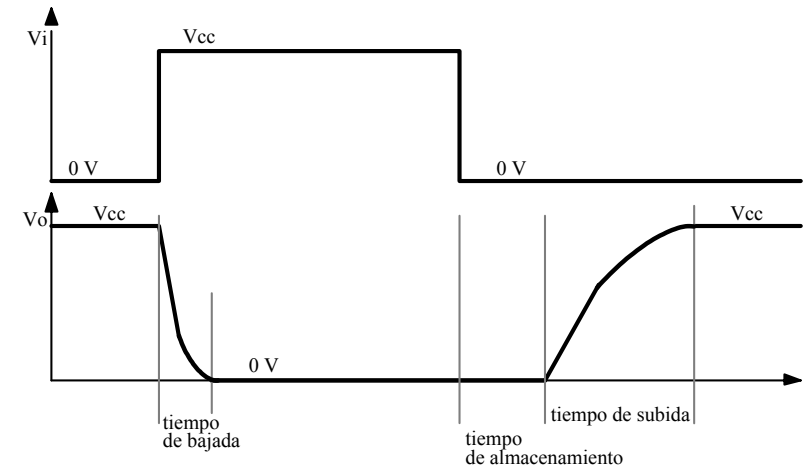
$$V_{CC} / I_{B\text{máx}} < R_B < \beta \cdot R_C$$

Dentro de estos dos límites puede elegirse la resistencia R_B en una amplia gama de valores; cuanto menor sea R_B más profunda será la situación de saturación cuando ésta se alcance, mientras que valores altos de R_B favorecen el estado de corte, siendo precisamente la transición de saturación a corte la que presenta mayores retardos.

La resistencia de colector R_C ha de ser elegida dentro de un compromiso entre velocidad y consumo; cuanto menor es R_C menores son también las constantes de tiempo asociadas al transistor y éste conmuta más rápidamente, pero consume más.

La conmutación de este circuito inversor requiere fuertes variaciones de la concentración de portadores minoritarios en la región de base, habida cuenta de que dicha concentración es muy diferente en corte (prácticamente nula) y en saturación; tales variaciones de carga en la base del transistor necesitan los correspondientes tiempos para la acumulación o eliminación de carga, que se manifiestan como tiempos de retraso.

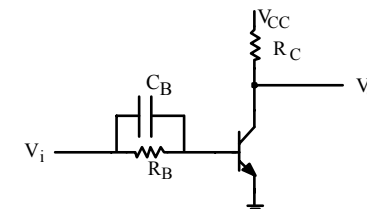
La siguiente figura representa la respuesta de un inversor frente a una onda cuadrada en su entrada:



La transición a corte es más lenta que la de saturación, es decir, es más costoso (en variación de carga en base y, por tanto, en tiempo) el paso de saturación a corte que el recíproco de corte a saturación. En tal sentido, no interesa forzar mucho la saturación, o sea, la desigualdad $R_B < \beta \cdot R_C$.

El orden de magnitud de los tiempos de retraso en la conmutación expresados en la figura anterior depende del tiempo medio de recombinación de portadores minoritarios en la base: τ_B es un parámetro característico del transistor, cuyo orden de magnitud suele encontrarse entre 0,1 y 1 μs para transistores normales de silicio; cuanto menor sea dicho valor, más rápida será la conmutación del transistor.

Es posible mejorar ambas transiciones mediante la inclusión de un condensador C_B en paralelo con la resistencia de base:

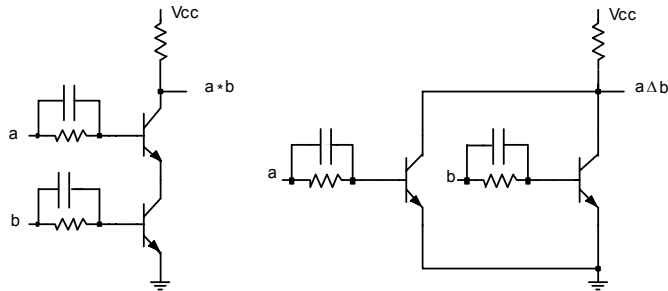


El condensador C_B comunica a la base un impulso de carga en cada transición, que se traduce en una rápida variación de la concentración de portadores minoritarios:

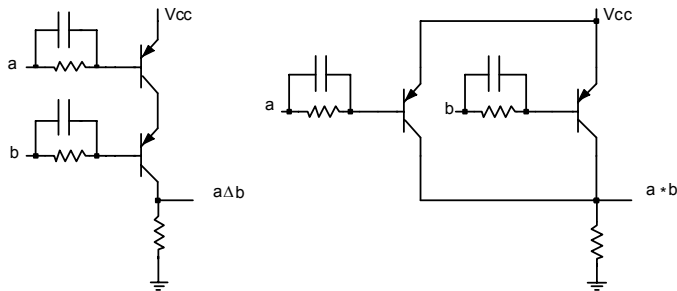
- para $C_B \cdot R_B = \tau_B$ los impulsos de carga comunicados por el condensador C_B coinciden exactamente con la variación global de portadores minoritarios en la base y se consigue una respuesta muy rápida a una onda cuadrada en la entrada del inversor.

Puertas con transistores discretos

A partir del esquema circuital del inversor, la conexión de transistores en serie (operación "y") o en paralelo (operación "o") da lugar a puertas inversoras *Nand* y *Nor*:



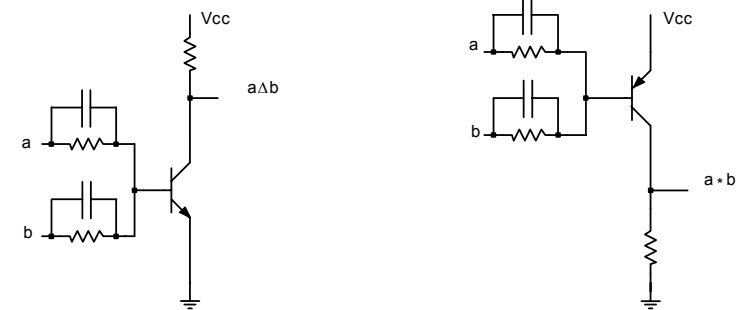
Con transistores PNP resultarán las puertas siguientes:



Habida cuenta la propiedad de «operación única» que posee tanto la puerta "y-negada" (*Nand*) como la "o-negada" (*Nor*), puede construirse cualquier función booleana utilizando solamente puertas *Nand* o puertas *Nor*.

La asociación de transistores en serie no suele utilizarse por presentar ligeros problemas de desplazamiento de nivel (cuando los transistores se encuentran en saturación $V_{CE} = 0,2$ voltios y en el caso de transistores en serie sus tensiones V_{CE} de saturación se suman); la asociación en paralelo resulta un esquema de puerta lógica muy útil y, precisamente, fue el primero en ser utilizado en los circuitos integrados digitales (familia lógica RTL).

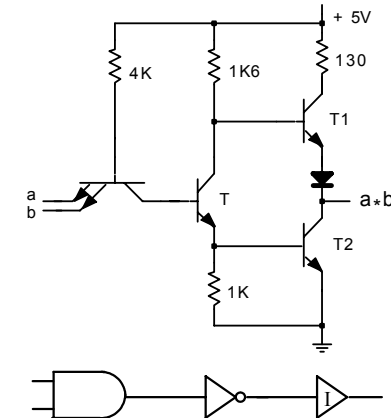
El inconveniente principal de las anteriores puertas lógicas consiste en necesitar tantos transistores como entradas; puede ser evitado utilizando un único transistor y dotándole de varias entradas a través de circuitos $R_B // C_B$.



Estas puertas resultan muy útiles y económicas para realizar funciones booleanas con componentes discretos; con cualquiera de ambas puede construirse toda la lógica booleana.

Puerta básica TTL

La puerta básica **TTL** es de tipo "y-negada" (*Nand*), constituida por tres etapas sucesivas: un transistor multiemisor que actúa como puerta "y", un inversor formado por un simple transistor que conmuta entre corte y saturación y un amplificador de intensidad con dos transistores en *push-pull* (etapa de salida denominada *totem pole*).

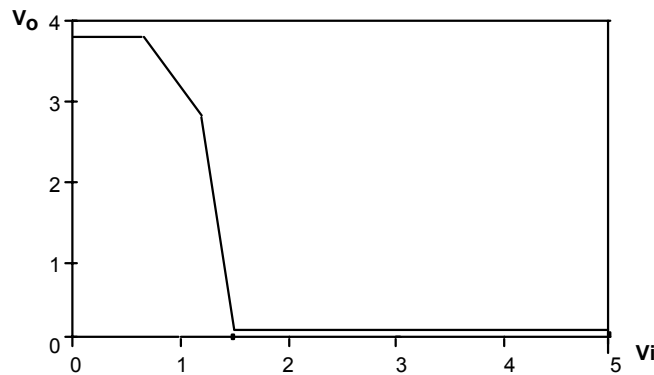


El funcionamiento de la puerta **TTL** es el siguiente:

- Caso de que todas las entradas se encuentren a **1**, la intensidad de corriente que circula por la primera resistencia (4K) llega a la base del transistor central **T** a través del diodo base-colector del transistor multiemisor de entrada; **T** se satura y también lo hace **T₂**, de forma que la tensión de salida se sitúa en **0** ($\approx 0,2$ V); el diodo en serie con **T₁** impide que este transistor conduzca.
- Cuando alguna de las entradas se encuentra a **0**, la intensidad de la mencionada resistencia ($\sim 5 \text{ V} / 4\text{K} = 1,2 \text{ mA}$) es conducida «hacia fuera» por el correspondiente diodo base-emisor del transistor multiemisor; **T** y **T₂** se encontrarán en corte y, por tanto, **T₁** pasa a conducción: la tensión de salida será $5 - V_{BE1} - V_D \approx 3,8 \text{ V}$, que corresponde al **1** (pero que es sensiblemente inferior a la tensión de alimentación).

Dado que para imponer un **0** a una entrada TTL es preciso asumir «hacia fuera» una intensidad de 1,2 mA, una entrada TTL «al aire» equivale a un **1**. Ahora bien, nunca debe dejarse una entrada al aire pues constituirá una pequeña antena capaz de recibir todo tipo de «ruido» electromagnético; las entradas no utilizadas deben anularse conectándolas a **1** (caso de puertas *Nand*).

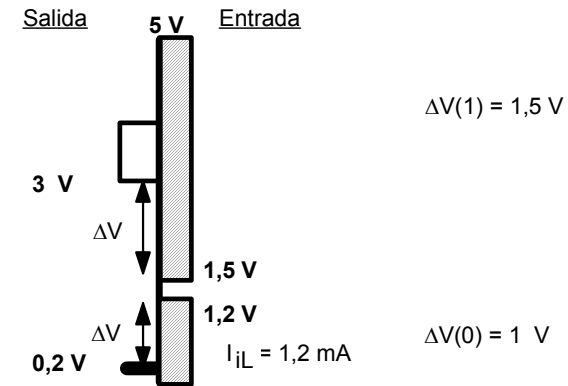
La función de transferencia **V_o-V_i** es la representada en la siguiente figura:



- para $V_i < 0,6 \text{ V}$ la tensión de salida V_o será de 3,8 V
- entre los valores de 0,6 y 1,5 V de la tensión de entrada, la de salida conmuta de 3,8 a 0,2 V; esta transición se realiza en dos tramos: al variar la entrada de 0,6 a 1,2 V la salida pasa de 3,8 a 3 V y de 1,2 a 1,5 V en la entrada la salida desciende abruptamente de 3 a 0,2 V
- para $V_i > 1,5 \text{ V}$, la tensión de salida será de 0,2 V.

Se han utilizado en este análisis tensiones umbrales de 0,6 V para los diodos; 0,7 V para la unión base-emisor en saturación y 0,2 V para la tensión colector-emisor en saturación.

De acuerdo con dicha función de transferencia, el diagrama salida/entrada será:



El margen de tensión para el ruido es de 1,5 V para salida **1** y 1 V para salida **0**.

El consumo de esta puerta TTL es mayor para salida **0**, en cuyo caso el transistor **T** se encuentra en saturación, conduciendo a través del mismo aproximadamente 3,2 mA:

$$\begin{aligned} \text{transistor T} \quad & V_C = V_E + V_{CE,sat} = V_{BE2} + V_{CE,sat} \approx 0,9 \text{ V}; I_C \approx (5 - 0,9)/1,6\text{K} = 2,5 \text{ mA}; \\ & V_B = 2V_{BE,sat} \approx 1,4 \text{ V}; \quad I_B \approx (5 - 1,4 - 0,6)/4\text{K} = 0,7 \text{ mA}. \end{aligned}$$

En cambio, para salida **1** la intensidad más importante es la que circula por la primera resistencia (4K):

$$I \approx (5 - 0,6)/4\text{K} = 1,1 \text{ mA}.$$

El consumo promedio se sitúa en unos 2 mA (10 mW) que representan un consumo relativamente alto: tanto el suministro de intensidad desde la tensión de alimentación como la disipación de potencia en los integrados llegan a alcanzar valores importantes cuando se utiliza un gran número de puertas.

La *velocidad de conmutación* de la puerta TTL está relacionada con el parámetro τ_B de sus transistores, cuyo valor se sitúa en el orden de 10 ns; del mismo orden de magnitud son los tiempos de propagación típicos: **t_{PHL}** y **t_{PLH}**.

Por otra parte, es preciso tener en cuenta que en el momento de la conmutación se produce un ligero pico de consumo debido a que, durante un instante, se superpone la conducción de los dos transistores de la etapa de salida **T₁** y **T₂**; este sobreconsumo dinámico es, en promedio, despreciable pero genera perturbaciones sobre la línea de alimentación que pueden afectar a los valores lógicos.

Diversidad de series TTL

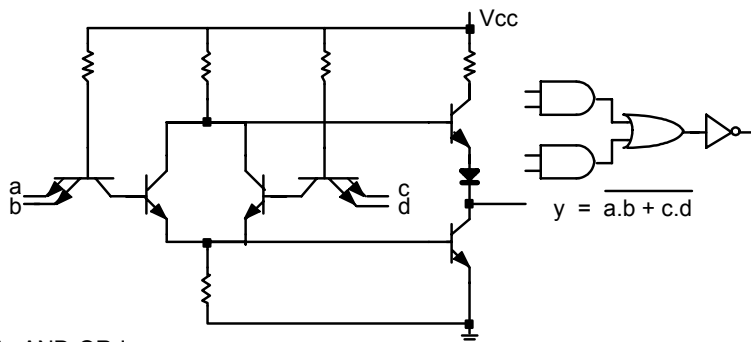
Dentro de la familia TTL se han desarrollado diversas series con prestaciones específicas. Las características numéricas (valores de las resistencias, velocidad, consumo, etc.) detalladas anteriormente corresponden a la primera serie 74, que se conoce como serie normal o estándar (si bien se encuentra hoy en día en desuso).

Las principales mejoras respecto a la serie 74 inicial se deben a tres tipos de modificaciones:

- Aumentando el valor de las resistencias se consigue reducir el consumo (pero, a la vez, aumentan los tiempos de conmutación y disminuye la velocidad de trabajo): serie de «bajo consumo» **74L**, cuyo consumo promedio se sitúa en el orden de 0,2 mA y sus tiempos de propagación en unos 40 ns.
- La utilización de transistores Schottky (bipolares en cuya unión base-colector, en paralelo con la misma, se añade un diodo Schottky que impide al transistor entrar en saturación profunda) permite una conmutación mucho más rápida: serie «TTL Schottky» **74S** con tiempos de conmutación de 3 ns y un consumo promedio de 4 mA.
- Combinando ambas modificaciones se consigue una serie de consumo reducido y velocidad de conmutación análoga a la serie estándar: serie «TTL Schottky de bajo consumo» **74LS**, con tiempos inferiores a los 10 ns y consumo promedio de 0,5 mA.
- La reducción de tamaño de los transistores integrados y la correspondiente reducción de sus efectos capacitivos ha dado lugar a series «avanzadas» con menores tiempos de conmutación: **74ALS** y **74AS** (tiempos del orden de 2 ns y 1 ns, respectivamente).

Suma de productos: puerta and-or-i

Las entradas de la puerta básica "y-negada" (Nand) de la familia TTL pueden ser ampliadas, en la forma representada en la figura siguiente, configurando una puerta con las operaciones "y" – "o" – "negación": puerta "y-o-negada" (AND-OR-I); esta puerta es sumamente útil, ya que realiza directamente la «suma de productos» que corresponde a la forma canónica de una función (tomando, para ello, la suma de productos de la función negada, ya que existe una inversión en la salida de esta puerta).



Puerta AND-OR-I

Características de la serie LS según catálogo

Los datos utilizados en este apartado corresponden al circuito integrado 74LS00 en el catálogo de Texas Instruments.

• Temperaturas

La temperatura de funcionamiento de la serie LS-TTL abarca el intervalo entre 0°C y 70°C y puede almacenarse a temperaturas entre -65°C y 150°C.

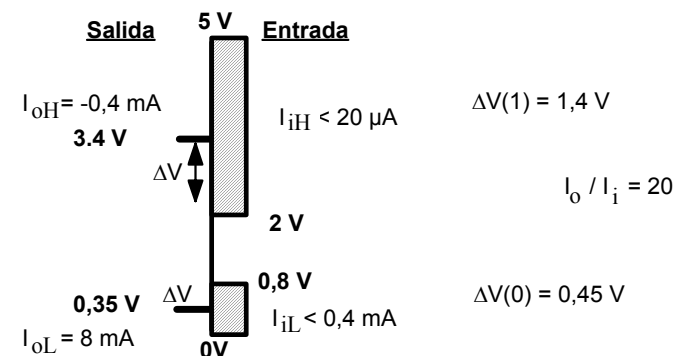
• Tensión de alimentación

La tensión de alimentación es la habitual de 5 voltios; el intervalo de tensiones admitido va de 4.75 a 5.25 voltios, lo cual requiere un buen filtrado y posterior estabilización en la fuente de alimentación.

• Tensiones e intensidades: V_o - V_i , I_o , I_i

$$V_{oL} = 0,35 \text{ V para } I_{oL} = 8 \text{ mA} \quad V_{iL\text{máx}} = 0,8 \text{ V} \quad I_{iL} < -0,4 \text{ mA}$$

$$V_{oH} = 3,4 \text{ V para } I_{oH} = -0,4 \text{ mA} \quad V_{iH\text{mín}} = 2 \text{ V} \quad I_{iH} < 20 \mu\text{A}$$



Es importante tener en cuenta que el valor 0 en una entrada requiere un intensidad «hacia fuera» del orden de 0,2 mA.

• Inmunidad frente al ruido

Los márgenes de ruido en tensión, calculados en la gráfica anterior, son:

$$\Delta V(0) = 0,45 \text{ V}$$

$$\Delta V(1) = 1,4 \text{ V}$$

Para calcular los márgenes de ruido en potencia es necesario deducir las resistencias de salida en el propio esquema circuital de la puerta LS-TTL:

$R_o(0)$ = resistencia del transistor inferior en saturación $\approx 10 \Omega$;

$R_o(1)$ $\approx$ resistencia en serie con el transistor superior $= 110 \Omega$.

Con tales valores los márgenes de ruido en potencia son de:

$$\Delta P(0) = 0.45^2 / 10 = 20 \text{ mW} \quad \Delta P(1) = 1.4^2 / 110 = 18 \text{ mW}$$

El margen de ruido en potencia es de unos 20 mW. Cabe observar que, aunque el margen en tensión es mucho menor para el **0** que para el **1**, en potencia ambos márgenes son parecidos, e incluso es peor para el **1**, ya que su resistencia de salida es mucho mayor.

- *Velocidad de trabajo*

Los tiempos de propagación (**t_{PLH}** y **t_{PHL}**) son del orden de 10 ns; por tanto, se requieren tiempos de bit superiores a 20 ns, lo que permite una frecuencia máxima de reloj de 25 MHz.

- *Consumo*

$$I_{CCL} \sim 0.2 \text{ mA}$$

$$I_{CCH} \sim 0.6 \text{ mA}$$

$$\text{en promedio: } I_{CC} \sim 0.4 \text{ mA}$$

$$P \sim 0.4 \times 5 = 2 \text{ mW}$$

- *Conectividad*

La capacidad de carga *fan-out* viene dada por el cociente entre intensidades de salida y de entrada, ya que al conectar varias puertas a la salida de otra se requiere que ésta suministre intensidad suficiente para asegurar el valor booleano de entrada en las siguientes (en particular, para asegurar el **0**).

$$fan-out = I_o / I_i = 20.$$

En cuanto al *fan-in* existen puertas integradas de hasta 8 entradas.

Se dispone de inversores con entradas con histéresis, tipo *Schmitt*, cuyas tensiones de disparo son, aproximadamente, 0,8 y 1,6 V, y de puertas con salidas en *colector abierto*; asimismo, existe una amplia disponibilidad de puertas y bloques con salidas *triestado*.

T1 SEMICONDUCTORES, UNIÓN PN Y TRANSISTORES MOS

T1.1. Semiconductores

T1.2. Unión PN

T1.3. Transistor MOS

La electrónica se dedica a manejar información y para ello utiliza dispositivos que aprovechan las propiedades de los electrones. Los primeros dispositivos electrónicos fueron las válvulas, derivadas del triodo de Lee de Forest; pero la electrónica actual no es una electrónica de válvulas sino una electrónica de transistores.

El transistor es el punto de partida y, a la vez, el impulsor de un continuado proceso de miniaturización de la electrónica que, a través de los circuitos integrados y de su complejidad cada vez mayor, llega hasta nuestros días.

Los transistores provienen de la utilización eficiente de los semiconductores y, en particular, de estructuras conformadas por zonas semiconductoras de distinto signo: P y N.

Este capítulo presenta en forma conceptual y muy simplificada los fundamentos del comportamiento de los semiconductores, sus dos tipos P y N y la unión entre ambos, hasta llegar a los transistores MOS como interruptores digitales.

Un semiconductor conduce (más bien poco) gracias a los pares electrón-hueco que se forman cuando, por efecto de la temperatura, un electrón deja libre un enlace de valencia («hueco»); pero el número de pares electrón-hueco a temperatura ambiente es insignificante y la resistencia de un semiconductor puro es muy alta.

Ahora bien, la adición de impurezas formadas por átomos de fósforo (con electrones de más, tipo N) o de boro (deficitario de electrones y, por ello, generador de huecos, tipo P) permite aumentar drásticamente la disponibilidad de portadores y lleva al semiconductor a conducir en forma «intermedia» (semi-conduce).

Por su parte, la unión de un semiconductor P y de otro N presenta propiedades no lineales, configurando un discriminador de polaridad: si le llega una señal con parte positiva y parte negativa «deja pasar» solamente una de las polaridades, según el sentido en que se encuentre conectada. Además, la unión PN en polaridad inversa constituye un excelente aislante (para separar zonas de difusión del sustrato que las contiene).

El transistor NMOS aprovecha el efecto capacitivo de una placa (Puerta) para generar un canal conductor entre dos zonas de difusión de tipo N (Fuente y Drenaje, que ofician simplemente como contactos). De esta forma configura un interruptor que conduce cuando la tensión de puerta es positiva y no lo hace si dicha tensión es nula; las características de este interruptor (cuasi-ideal) lo hacen un magnífico candidato para el álgebra de conmutadores que da origen a la electrónica digital.

T.1. Semiconductores

Los metales son materiales conductores de la electricidad debido a que en su estructura cristalina disponen de electrones libres que se mueven por efecto del campo eléctrico cuando se aplica una diferencia de potencial (una tensión eléctrica). La diferencia de potencial eléctrico acelera a los electrones en la dirección del campo eléctrico (aunque en sentido contrario al mismo, ya que su carga eléctrica es negativa); pero en este movimiento de avance los electrones chocan con los átomos de la red cristalina que conforma el metal y la pérdida de energía en tales choques determina un movimiento no acelerado con una velocidad promedio proporcional a la intensidad del campo.

Como materiales conductores que son, la resistencia eléctrica de los metales es muy baja; un hilo de cobre de 1 metro de longitud y 1 mm² de sección, a temperatura ambiente, presenta una resistencia de unos 20 mΩ.

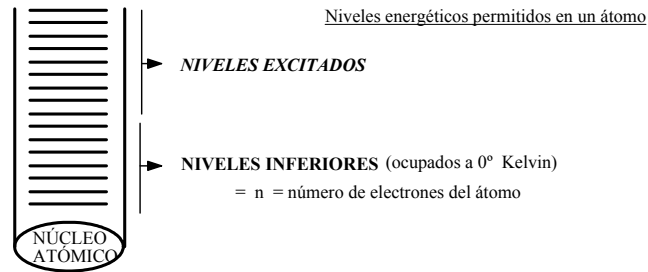
Al aumentar la temperatura, aumenta también el movimiento desordenado de los electrones debido a su agitación térmica y, con ello, aumentan los choques con los núcleos de la red cristalina y la mayor pérdida de energía determina una disminución de la velocidad promedio. Por ello, la resistencia de los metales aumenta al aumentar la temperatura; lo cual puede comprobarse fácilmente midiendo directamente con el polímetro la resistencia de una lámpara apagada y comparándola con el valor que se obtiene por aplicación de la ley de Ohm cuando la lámpara se encuentra encendida.

Existen en la naturaleza unos materiales cuya conductividad es intermedia entre la de los conductores metálicos y la de los aislantes y, además, su resistencia no aumenta con la temperatura sino que disminuye. Su comportamiento no puede ser explicado con el modelo de electrones libres con el que se describe el comportamiento de los conductores, sino que es necesario acudir a los principios cuánticos que rigen la configuración interna de los átomos.

Veamos algunas ideas elementales de la mecánica cuántica, es decir, de la organización de los electrones en los átomos:

Bohr postuló que los electrones de un átomo no pueden situarse en cualquier órbita, o sea, no pueden encontrarse en cualquier nivel energético respecto al núcleo, sino que existe un conjunto discreto de niveles permitidos. Pauli añadió que en un mismo nivel energético no puede haber más que un electrón.

A 0° de temperatura absoluta los electrones se sitúan en los niveles energéticos más bajos, pero a temperaturas superiores, $T > 0^\circ \text{ K}$, la energía térmica correspondiente a la propia temperatura les permite ocupar niveles superiores: niveles excitados.

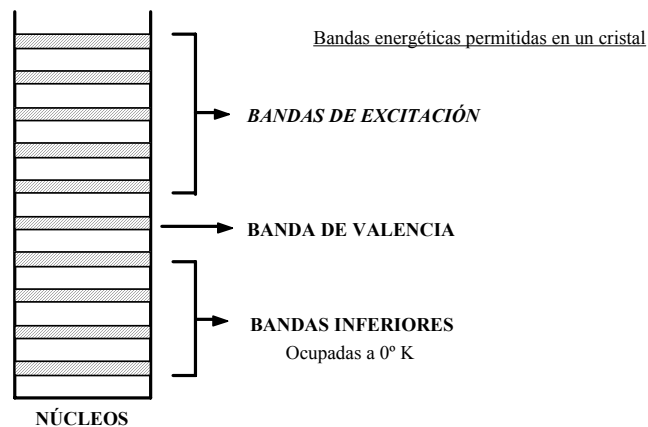


La probabilidad de que un electrón adquiriera una energía de excitación E_i viene dada por $p(E_i) = \alpha \cdot e^{-E_i/KT}$, donde K es la constante de Boltzmann y T la temperatura absoluta: para $T = 0^\circ \text{K}$ $p(E_i) = 0$; para $T > 0^\circ \text{K}$ $p(E_i) > 0$.

La probabilidad de ocupar un nivel excitado aumenta con la temperatura y es tanto mayor cuanto menor es la energía de excitación E_i .

El principio de exclusión de Pauli es válido también para un sistema molecular y para un sistema cristalino: al unirse dos átomos idénticos, desplazan sus niveles energéticos permitidos para que no coincidan dos electrones con la misma energía; este desplazamiento es, claro está, muy pequeño.

En un cristal de un elemento se encuentran multitud de átomos idénticos ligados por la red cristalina, los cuales dan lugar a un conjunto de bandas energéticas formadas por los niveles energéticos de los múltiples átomos; tales niveles eran coincidentes en los átomos aislados pero, al unirse, se han desplazado mínimamente y dan lugar a una banda de energía. Cada banda contiene multitud de niveles energéticos muy próximos, cada uno de los cuales puede ser ocupado sólo por un electrón.



En un sistema cristalino existen bandas energéticas permitidas que pueden ser ocupadas por un número discreto de electrones y que se encuentran separadas por intervalos de energía prohibidos.

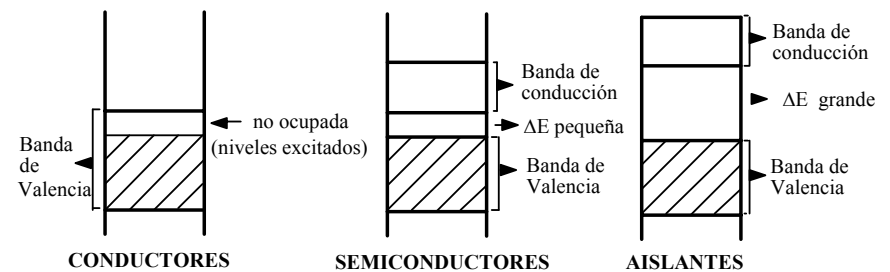
A 0°Kelvin se completan las bandas de energía inferior. La última banda energética que resulta ocupada se denomina banda de valencia; los electrones de esta banda, electrones de valencia, son los que pasarán a niveles excitados ya que los tienen más próximos que el resto de los electrones.

Para contribuir a la conducción eléctrica los electrones han de pasar a niveles excitados, pues han de adquirir la energía cinética correspondiente a su movimiento; de modo que, al aplicar un campo eléctrico, solamente contribuirán a la conducción aquellos electrones que puedan dar el salto a niveles energéticos excitados.

Aquellos materiales que tienen la banda de valencia incompleta, o sea no toda ella ocupada por electrones, son conductores; presentan niveles energéticos excitados lo suficientemente próximos, dentro de la misma banda de valencia, para que la energía correspondiente al campo eléctrico haga pasar electrones a ellos, incluso a temperatura de 0°Kelvin : en los conductores la banda de valencia coincide con la de conducción.

Los materiales cuya banda de valencia está completa no pueden conducir a 0°K , pues los campos eléctricos normales no son capaces de comunicar la energía necesaria para que los electrones de valencia salten a la banda superior, que será en este caso la banda de conducción. Dentro de estos materiales existe una diferencia importante:

- aquellos en los que el intervalo energético entre las bandas de valencia y de conducción es grande y que, por tanto, a temperatura ambiente apenas tendrán electrones en la banda de conducción y, en consecuencia, no conducirán
- y aquellos otros en que este intervalo es pequeño, de forma que a temperatura ambiente tienen un número apreciable de electrones excitados en la banda de conducción y, por tanto, conducen: éstos son los semiconductores.



Ésta es la diferencia entre conductores, aislantes y semiconductores. Los conductores tienen niveles excitados libres en la propia banda de valencia y los electrones pueden pasar a ellos por acción de un campo eléctrico, incluso a 0° absolutos; los semiconductores tienen dichos niveles de excitación en otra banda que se encuentra energéticamente próxima y pueden ocuparlos cuando la temperatura es mayor de 0°K ; mientras que los aislantes tienen la banda de niveles excitados muy distante, de forma que el número de electrones que pasan a ella es despreciable.

En los semiconductores, al aumentar la temperatura, aumenta el número de electrones que pasan de la banda de valencia a la de conducción y contribuyen a la corriente eléctrica; por ello, su resistencia disminuye al aumentar la temperatura.

En el contexto del modelo cuántico de los semiconductores resulta que al pasar un electrón a la banda de conducción deja un hueco en la de valencia, es decir, un nivel energético que puede ser ocupado por otro electrón inferior, adquiriendo una energía de excitación. Este segundo electrón también puede contribuir a la conducción; de forma que contribuyen a la conducción tanto los electrones que han pasado a la banda de conducción como los niveles energéticos «huecos» que han quedado libres en la banda de valencia.

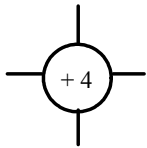
Los pares electrón-hueco se forman por efecto de la energía térmica: a temperatura superior a 0°K un electrón adquiere energía suficiente para pasar a un nivel excitado de la banda de conducción y deja un hueco en la banda de valencia. Un «hueco» contribuye a la corriente eléctrica al ser ocupados por un electrón de un nivel energético inferior y, como el hueco supone «ausencia de electrón», actúa como si fuera una carga positiva.

A la vez sucede que, en ocasiones, un hueco es cancelado por un electrón libre que pasa a formar parte del enlace vacío propio del hueco, cediendo la correspondiente energía; este fenómeno se denomina recombinación y supone pérdida de portadores.

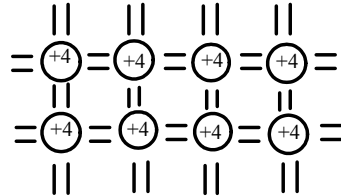
El número de portadores eléctricos disponibles se debe al balance entre la formación de pares electrón-hueco y la recombinación de los mismos; es, pues, una situación de equilibrio dinámico.

Consideremos una imagen más gráfica de los semiconductores: por razones tecnológicas, principalmente por razones de consistencia mecánica y de disponibilidad de ellos, los semiconductores utilizados son cristales constituidos por átomos de valencia 4, cuatro electrones en la última capa, principalmente el germano y el silicio.

Representación esquemática de un átomo de valencia 4

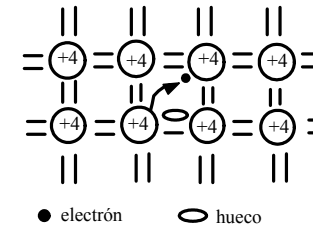


Representación esquemática y plana de un cristal de dichos átomos de valencia 4



La carga positiva indicada dentro del núcleo atómico (+4) queda compensada por los 4 electrones de valencia representados por las líneas periféricas; de forma que, como es obvio, el átomo con sus 4 electrones de valencia es eléctricamente neutro.

Cuando por efecto térmico, a $T > 0^\circ\text{K}$, un electrón adquiere energía suficiente para «saltar» de su enlace queda «libre», es decir, pasa a la banda de conducción, pero, asimismo, al ausentarse del enlace correspondiente deja un hueco (que representa una carga positiva).



Como resultado del paso de un electrón de la banda de valencia a la de conducción queda un hueco, un enlace vacante por ausencia de electrón, en dicha banda de valencia

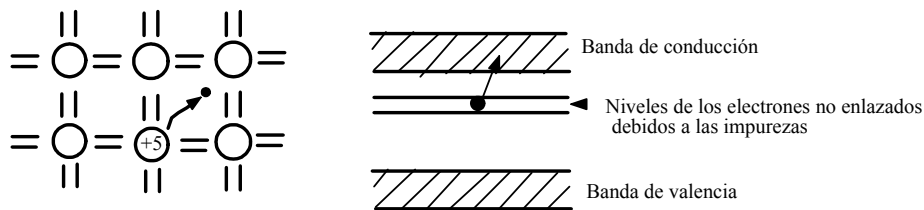
Otro electrón de un enlace próximo puede ocupar este hueco, dejando otro en su enlace anterior, y así sucesivamente, de forma que un hueco se mueve en sentido contrario a los electrones que pasan a ocuparlo.

Si este movimiento es debido a un campo eléctrico, el hueco se mueve como si tuviera carga positiva y el semiconductor contribuye a la conducción con dos tipos de portadores: los electrones en la banda de conducción que se mueven en sentido opuesto al del campo (cargas negativas) y los huecos en la banda de valencia que se mueven en el mismo sentido del campo (cargas positivas). Está claro que los huecos no tienen existencia propia, pero constituyen un modelo válido y simple para representar el movimiento de los electrones de valencia que pasan a ocuparlos.

En el caso del semiconductor más utilizado, el silicio, la concentración de átomos en la red cristalina es del orden de 10^{22} átomos/ cm^3 y, a temperatura ambiente (25°C), se produce una generación de pares electrón-hueco del orden de 10^{10} pares/ cm^3 , es decir, solamente 1 átomo de cada billón (10^{12}) de átomos de silicio suelta un electrón y se queda con un hueco. Esta baja disponibilidad de portadores eléctricos hace que la conductividad de los semiconductores a temperatura ambiente sea muy baja; la resistencia de una línea de silicio de 1 metro de longitud y 1 mm^2 de sección, a temperatura ambiente, es del orden de $1000\text{ M}\Omega$.

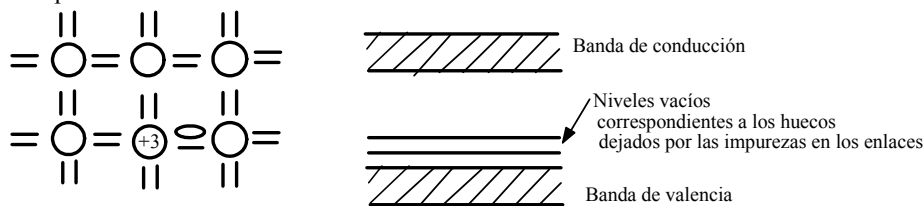
Pero la conductividad de los semiconductores puede ser considerablemente mayor si se aumenta la concentración de portadores eléctricos, añadiendo dentro de la red cristalina pequeñas cantidades de ciertas impurezas. La existencia de dos tipos de portadores (electrones y huecos) da lugar a dos tipos de semiconductores enriquecidos o «dopados»: semiconductores N, con predominio de electrones, y semiconductores P, con predominio de huecos; según que la impureza tengan exceso (fósforo) o defecto (boro) de electrones de valencia resulta un semiconductor tipo N o tipo P.

Si en una red de átomos de valencia 4 se incluyen, en forma muy diseminada, átomos de valencia 5, un electrón de cada uno de estos átomos quedará sin formar parte de los enlaces, por lo cual tendrá mucha mayor facilidad para «saltar» a la banda de conducción. Estos electrones no dejan huecos en la banda de valencia (no dejan enlaces libres), de forma que predominarán los electrones en la banda de conducción y éstos serán los portadores de corriente mayoritarios: semiconductor tipo N.



Si, por el contrario, los átomos de las impurezas son de valencia 3, quedará un hueco en los enlaces de cada átomo de impureza; estos huecos serán ocupados por electrones de la banda de valencia, dejando los correspondientes huecos en ésta pero sin generar electrones libres en la banda de conducción.

En este caso predominan los huecos como portadores de corriente: semiconductor tipo P.



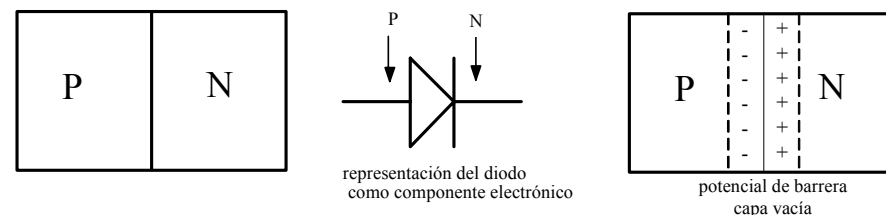
Un dopado normal suele ser del orden de 10^{20} átomos/cm², o sea, 1 átomo de impureza (fósforo o boro) por cada 100 átomos de silicio; ello supone una disponibilidad de portadores eléctricos de 10^{20} /cm² pues todos los átomos de impureza contribuyen con su portador (electrón / hueco) dado que el salto energético para ello es pequeño.

Esta concentración de portadores resulta ser 10.000 millones de veces superior a los portadores disponibles en el silicio puro (10^{10}) y la conductividad del silicio P o N resulta 100 millones de veces mayor que la del silicio sin impurezas. Téngase en cuenta que la conductividad no depende solamente de la concentración de portadores disponibles sino también de la movilidad de tales portadores, que es más baja en los semiconductores fuertemente dopados porque inserta un amplio número de núcleos con carga en la red cristalina; éstos atraen a los portadores mayoritarios y los frenan y, además, el número de choques contra la red cristalina aumenta con el dopado.

La resistencia de una línea de silicio dopado de 1 metro de longitud y 1 mm² de sección, a temperatura ambiente, es del orden de 10 Ω (sólo unas 500 veces superior a la del cobre).

T.2. Unión PN

Un diodo está constituido por una unión P-N, es decir, por la unión de dos semiconductores, uno de tipo P y otro de tipo N, con una superficie común.



Del lado P del diodo predomina la concentración de huecos y, en cambio, en el lado N predomina la concentración de electrones. La diferencia de concentración de portadores entre ambos lados origina sendas corrientes de difusión, de huecos hacia el lado N y de electrones hacia el lado P, que tienden a homogeneizar la concentración de portadores en todo el material.

Los electrones del lado N cercanos a la unión pasan al lado P, lo cual da origen a una carga eléctrica positiva en la zona N próxima a la unión (carga positiva que se produce por la pérdida de electrones, más el aporte de huecos desde P). Asimismo, la zona P próxima a la unión se carga en negativo (por la pérdida de huecos enviados a la otra zona, más el aporte de electrones que le llegan desde N).

Entre ambos lados de la unión se crea una especie de condensador cargado y, consecuentemente, un campo eléctrico que va de N a P y se opone a la difusión de los portadores. Enseguida se llega a un equilibrio en el que la tensión en la unión, asociada al campo eléctrico producido y denominada potencial de barrera (ϕ), adquiere un cierto valor que anula la corriente de difusión.

El potencial de barrera es negativo en el lado P y positivo en el N; de forma que si se conecta una tensión exterior con el terminal negativo hacia P y el positivo hacia N, polarización inversa, dicha tensión se suma al potencial de barrera y ambas juntas impiden la conducción a través del diodo.

En cambio, si la tensión es positiva hacia P y negativa hacia N, polarización directa, dicha tensión exterior se opone al potencial de barrera, disminuyéndolo; en este caso, existirán portadores (huecos del lado P y electrones del lado N) capaces de saltar energéticamente la barrera y contribuir a la conducción en el sentido del campo exterior aplicado, de P a N.

En consecuencia, un diodo sólo puede conducir en el sentido de P a N, que es en el indicado por la flecha en su representación esquemática, y no puede conducir en sentido contrario.



La intensidad que atraviesa el diodo será tanto mayor cuanto menor sea la barrera de potencial que se opone a ella en la unión, es decir, cuanto mayor sea la tensión aplicada al diodo. Esta tensión exterior ha de ser siempre inferior al valor del potencial de barrera propio del diodo, pues de otro modo la barrera se anularía totalmente, la intensidad aumentaría indefinidamente y produciría la destrucción de la unión.

La probabilidad de que un electrón tenga energía suficiente para atravesar la barrera viene dada por:

$$p = \alpha e^{-E/KT}, \text{ siendo } E \text{ la energía de la barrera de potencial.}$$

En una unión PN aislada dicha energía es $E = e \cdot \phi$, (e = carga del electrón) y la probabilidad de atravesar la barrera resulta despreciable, no existiendo corriente a través del diodo. Menos aún si aplicamos una polarización inversa, $E = e \cdot (\phi + V)$, en cuyo caso la probabilidad es aún menor, pues la energía de la barrera de potencial es mayor.

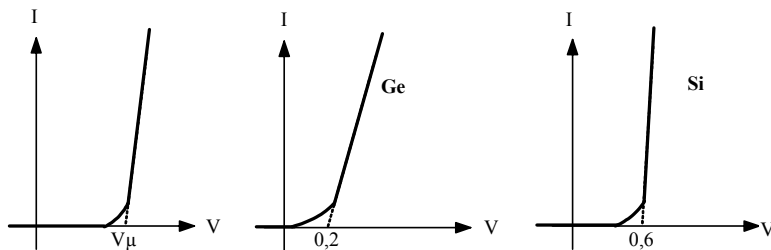
En cambio, en polarización directa la energía de la barrera es disminuida por la tensión exterior aplicada, $E = e \cdot (\phi - V)$, de forma que la probabilidad de atravesarla aumenta exponencialmente con dicha tensión V :

$$p = \alpha e^{-e \cdot (\phi - V)/KT} = \alpha e^{-e\phi/KT} \cdot e^{+eV/KT} = \alpha' e^{+eV/KT}$$

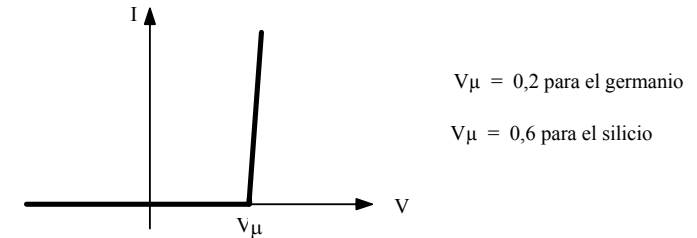
Éste será el tipo de dependencia de la corriente que atraviesa la unión PN con la tensión aplicada

$$I = \alpha'' e^{+eV/KT} = I_s \cdot e^{+eV/KT}$$

cuya representación da lugar a las gráficas de la figura siguiente, que coinciden con las medidas experimentales para diodos de germanio y de silicio.

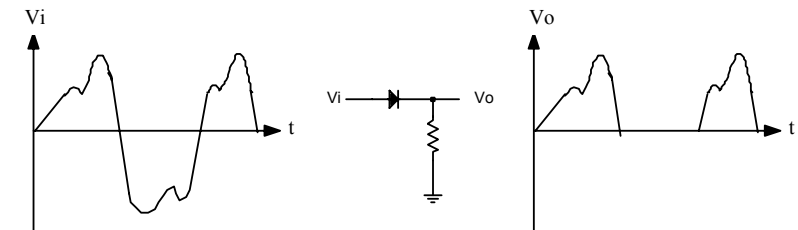


Estas gráficas se pueden linealizar con gran aproximación mediante dos rectas, una horizontal hasta V_μ y la otra prácticamente vertical.



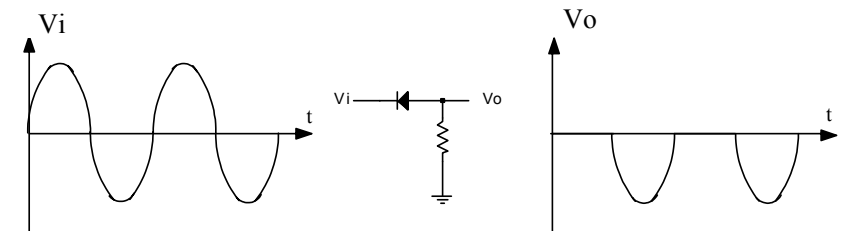
Es decir, un diodo no conduce mientras la tensión aplicada entre sus extremos sea inferior a V_μ y a partir de esta tensión conduce como si fuera prácticamente un cortocircuito; en realidad presenta una pequeña resistencia, una ligera inclinación de la recta, del orden de 10Ω para el germanio y de 2Ω para el silicio (dicha resistencia resulta francamente despreciable en la mayoría de los casos).

Configurando un circuito con un diodo de silicio y una resistencia de carga y aplicando a su entrada una señal con ambas polaridades, positiva y negativa:



pasa a la carga la parte de la onda que se encuentra por encima de los 0,6 voltios necesarios para la conducción del diodo.

Supongamos el diodo al revés y una onda senoidal en la entrada:



Por encontrarse el diodo al revés pasa a la carga la parte negativa de la onda, que es la que polariza al diodo en positivo; en concreto, el diodo deja pasar la parte de onda que se encuentra por debajo de -0,6 voltios.

Tal es la aplicación de los diodos en los circuitos electrónicos: dejar pasar las tensiones de una polaridad (bien positiva, bien negativa, según el sentido del diodo) y no las de la polaridad opuesta. Esta función se denomina «rectificar» una señal eléctrica; el diodo es un componente «rectificador».

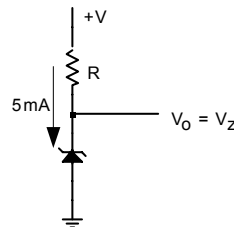
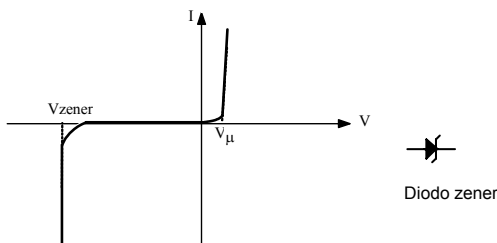
Realmente la operación de rectificar aparece cuando el diodo va seguido de un condensador que filtra la señal de salida y proporciona la envolvente de la onda, dando lugar a una «recta» (tensión continua) con un pequeño rizado; por ello, un nombre más apropiado para el diodo es el de «discriminador de polaridad».

Los diodos se fabrican difundiendo consecutivamente sobre un semiconductor impurezas tipo P por un extremo e impurezas tipo N por el opuesto; así se forman las zonas P y N sobre el mismo sustrato semiconductor. Según la intensidad máxima de corriente que vayan a conducir, se fabrican diodos de 0,1 A, 0,5 A, 2 A, 10 A, etc.

Una unión PN en polarización inversa (incluyendo el caso de $V_D = 0$) presenta una zona vacía de portadores, justo a ambos lados de la unión (en la propia barrera de potencial que la unión PN genera): capa de vaciamiento o capa vacía. La ausencia de portadores se debe a la difusión de los portadores propios hacia el otro lado de la unión y a la recombinación de los portadores que le llegan desde dicho otro lado (que se encuentran con una concentración amplia de portadores de signo opuesto que propicia la cancelación electrón-hueco).

La capa vacía de la unión constituye un excelente aislante que separa eléctricamente la zona semiconductora P de la zona semiconductora N. Este aislamiento propio de la unión PN es aprovechado en la configuración de circuitos integrados para «aislar» las regiones N que se fabrican dentro de un sustrato P y, viceversa, las regiones P fabricadas sobre un sustrato N.

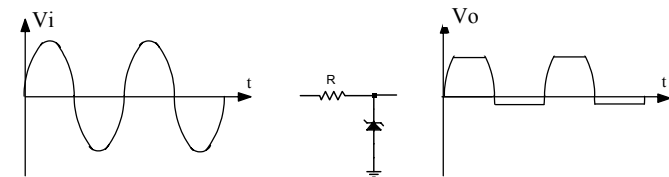
Ahora bien, cuando la tensión aplicada en polarización inversa aumenta, llega un momento en que el potencial creado en la unión, $\phi + V$, origina un campo eléctrico lo suficientemente fuerte como para romper enlaces atómicos, dejando libres huecos y electrones que contribuyen a la conducción y que no han de atravesar la barrera de potencial pues se generan en la propia barrera; este fenómeno se denomina efecto zener y los portadores así generados contribuyen a la conducción de forma que a dicha tensión, tensión zener, el diodo conduce en polarización inversa.



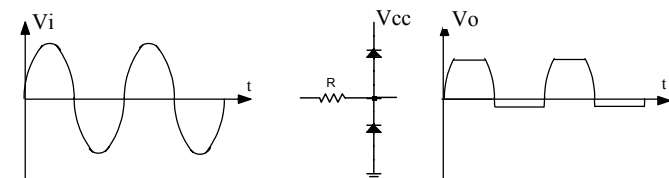
La tensión zener, en los diodos normales, es de varias decenas o de varios cientos de voltios y es otra característica a tener en cuenta junto con la intensidad máxima: la tensión zener de un diodo rectificador ha de ser superior al valor de la máxima tensión inversa que vaya a soportar, a fin de que no conduzca en polarización inversa. En tal sentido, un diodo rectificador viene caracterizado por dos parámetros: la intensidad máxima a conducir y la tensión máxima inversa a soportar (y así, hablaremos de un diodo de 1 A y 300 V y de otro de 10 A y 100 V, etc.)

Pero el efecto zener también es aprovechado en forma contraria en diodos que precisamente se polarizan en zona zener para utilizar dicha tensión como referencia (ver en la figura anterior el circuito de la derecha). Para ello se fabrican diodos con tensión zener de unos pocos voltios, denominados diodos zener: existe toda una gama de zener con valores de tensión que van desde los 2,7 V a los 24 V, muy útiles para configurar tensiones fijas de referencia, para lo cual han de polarizarse inversamente de manera que los atraviese una intensidad inversa (contraria a la flecha del diodo) entre 5 y 20 mA.

Un circuito resistencia-zener, tal como el representado en la figura siguiente, limita la tensión que recibe al intervalo $[-0,6 \text{ V} ; V_Z]$ ya que recorta las tensiones superiores a su tensión zener V_Z (para tensión de entrada positiva el diodo se encuentra en polarización inversa) y también recorta las tensiones negativas superiores a su tensión umbral como diodo $-0,6 \text{ V}$ (para tensión de entrada negativa conduce como diodo a partir de 0,6 V).

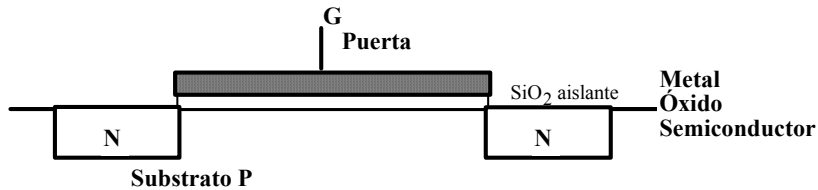


Un circuito limitador análogo puede configurarse con dos diodos en polarización inversa según la figura siguiente; en este caso los diodos recortarán las tensiones positivas superiores a $V_{CC}+0,6$ y las negativas inferiores a $-0,6 \text{ V}$, manteniendo la tensión dentro del intervalo $[-0,6 \text{ V} ; V_{CC}+0,6]$. Ambos circuitos son útiles para reducir al rango de tensiones digitales $[0 ; V_{CC}]$ pulsos de amplitud superior al mismo.



T.3. Transistor MOS

Un transistor NMOS es un interruptor controlado por un terminal de puerta **G**: la conducción se establece sobre un sustrato **P** entre dos difusiones **N** (fuente **S** y drenaje **D**) que actúan como contactos y la puerta configura un «condensador» intermedio cuya carga permite la conducción.



Si la tensión de entrada $V_i = V_G$ es suficientemente positiva, la puerta atrae a los electrones libres en el sustrato y forma un canal conductor entre fuente y drenaje: el transistor conduce. Esto sucede cuando la tensión $V_G > V_{TO}$, la tensión de puerta mayor que la tensión umbral de conducción.

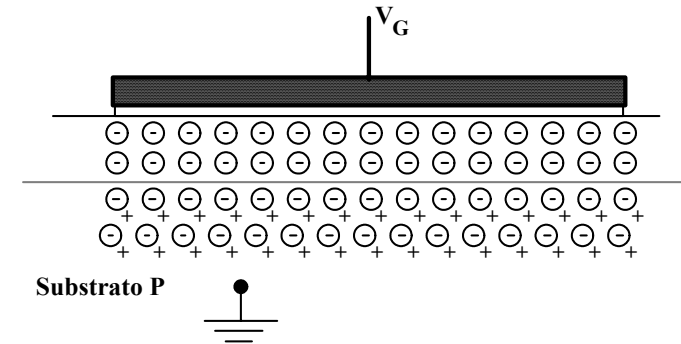
Si la tensión de puerta es claramente inferior al valor de la tensión umbral V_{TO} el canal no se forma y no hay paso de corriente entre las zonas de difusión (fuente y drenaje): el transistor no conduce.

En la zona de puerta (zona activa) la estructura que conforma el transistor está formada por tres capas: conductor – óxido – sustrato; en un principio tales capas fueron Metal – Óxido – Semiconductor (dando nombre al dispositivo: M - O - S), pero muy pronto el metal fue substituido por silicio policristalino fuertemente dopado: polisilicio – óxido – semiconductor. Se utiliza polisilicio en lugar de metal para reducir el potencial electroquímico que se genera en la interfase metal-óxido de silicio y que afecta fuertemente a la tensión umbral.

Analicemos en detalle la formación del canal y puesta en conducción de un transistor NMOS:

a) El sustrato de los transistores NMOS es de tipo P, dopado con átomos de Boro con sólo 3 electrones de valencia, los cuales al formar parte de la estructura cristalina del silicio presentan un hueco (un enlace no cubierto por electrón).

Cuando se aplica una tensión positiva a la puerta $V_G > 0$ respecto al sustrato (conectado a tensión de referencia 0 V) se forma una capa de vaciamiento, desprovista de portadores, debida a la repulsión electrostática que la tensión de puerta (+) ejerce sobre los huecos del sustrato. Al emigrar los huecos más allá de la capa de vaciamiento, los átomos de boro quedan cargados negativamente, pues la ausencia de hueco se debe a que un electrón lo ha cancelado (ha cubierto el enlace que se encontraba libre).



En la figura anterior, los círculos negativos representan a los átomos de impureza P (boro); todos ellos han perdido el hueco resultante de su déficit de electrones (han captado un electrón). En cambio, los signos + representan a los huecos: en la capa de vaciamiento no hay huecos porque han sido expulsados por repulsión electrostática provocada por la tensión positiva de la puerta; los átomos de boro inferiores, situados fuera de la capa vacía, mantienen su hueco y por ello su carga eléctrica es nula.

La densidad superficial de carga, es decir, carga por unidad de superficie Q será:

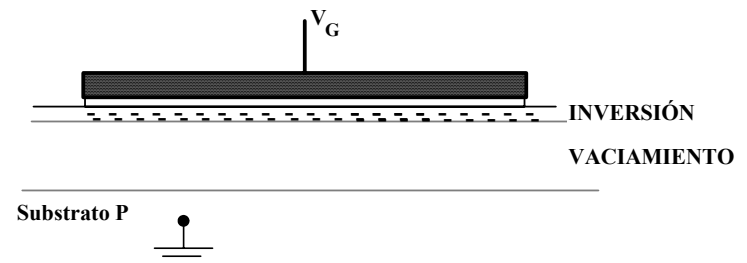
$$Q_v = q \cdot N_A \cdot e \cdot v$$

q = carga del electrón,

N_A = dopado del sustrato = nº de huecos por cm^3 ,

$e \cdot v$ = profundidad de la capa de vaciamiento.

b) Cuando la tensión de puerta es suficientemente positiva (tensión umbral del transistor) se forma una capa superficial de electrones libres, arrastrados por el gradiente de tensión existente en la capa de vaciamiento; ésta es la capa de inversión: el semiconductor P cambia de signo por acumulación de electrones que dan lugar a una zona de tipo N. Dichos electrones libres proceden de los pares electrón hueco del propio silicio semiconductor: son, pues, portadores intrínsecos y su número es reducido.



Para formar una capa de inversión o canal conductor se necesita una tensión en la superficie del semiconductor Φ_B capaz de mantener una concentración de electrones en la banda de conducción equivalente al dopado del sustrato, es decir, el sustrato P se vuelve N en la zona superficial.

La carga total almacenada en el «condensador» puerta-substrato tiene dos componentes: carga correspondiente a la capa de vaciamiento + carga debida al canal o capa de inversión: $Q = Q_V + Q_I$, siendo Q carga por unidad de superficie. Por otra parte, dicha carga será proporcional a la diferencia de potencial existente entre puerta y substrato con un factor de proporcionalidad que expresa la capacidad de dicho condensador c_{ox} .

$$\left. \begin{aligned} Q &= Q_I + Q_V \\ Q &= c_{ox}(V_G - \Phi_B) \end{aligned} \right\} \begin{aligned} Q &= \text{carga por unidad de superficie} \\ c_{ox} &= \text{capacidad por unidad de superficie} \end{aligned}$$

$$Q_I = Q - Q_V = c_{ox}(V_G - \Phi_B) - Q_V = c_{ox}[V_G - (\Phi_B + Q_V / c_{ox})]$$

$$Q_I = C_{ox}(V_G - V_{TO}) \quad \text{donde} \quad V_{TO} = \Phi_B + Q_V / c_{ox}$$

V_{TO} es la tensión umbral de conducción del MOS: tensión de puerta necesaria para que se forme la capa de inversión.

Comentario: aunque este análisis es conceptualmente correcto, el valor concreto de V_{TO} no puede deducirse directamente del mismo ya que sobre V_{TO} inciden otros efectos de segundo orden relativamente complejos y, en la práctica, dicho valor lo fija el fabricante (mediante un proceso de implantación iónica en la superficie del substrato).

c) Si sobre el substrato en su zona superficial se induce, por algún medio, una tensión V' , dicha tensión actúa como referencia o nivel 0 de tensiones, de forma que la tensión efectiva sobre el condensador MOS será $V_G - V'$:

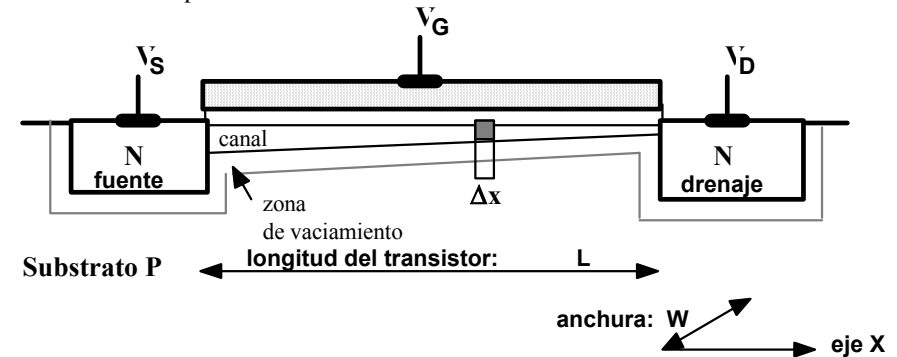
$$Q_I = c_{ox}(V_G - V' - V_{TO}).$$

Precisamente esto es lo que ocurre al polarizar la fuente y el drenaje del transistor MOS para que circule una corriente eléctrica a su través: las tensiones aplicadas a drenaje y fuente $V_D - V_S$ establecen una distribución de tensiones en la zona superficial del substrato, variando desde V_S (en el extremo de la fuente) hasta V_D (en el del drenaje).

Orientando el eje X en la dirección del canal (a lo largo del mismo, de fuente a drenaje), la concentración de portadores de carga en el canal será:

$$Q_I = c_{ox}(V_G - V_{TO} - V(x)) \quad \text{donde } V(x) \text{ varía entre } V_S \text{ y } V_D.$$

d) La diferencia de tensión entre fuente y drenaje $V_D - V_S$ da lugar a una intensidad de corriente I que corresponde a la carga que atraviesa una superficie perpendicular al eje X por unidad de tiempo.



La distribución de portadores libres en la capa de inversión variará a lo largo del canal (eje X): será mayor al lado de la fuente que en el del drenaje ya que $V_{DS} > 0$ y, por ello, $V_G - V_S > V_D - V_S$. En un elemento diferencial Δx la cantidad de carga que corresponde a los portadores libres, a todo lo ancho W del transistor será:

$$\Delta Q_I = [\text{densidad de carga}] \times \text{volumen} = Q_I \cdot W \cdot \Delta x = c_{ox} \cdot (V_G - V_{TO} - V(x)) \cdot W \cdot \Delta x$$

$$I = \frac{dQ_I}{dt} = \frac{\Delta Q_I}{\Delta x} \cdot v \quad \frac{\Delta Q_I}{\Delta x} \text{ es la carga de portadores libres por unidad de longitud}$$

v es la velocidad de los portadores que es proporcional al campo eléctrico:

la constante de proporcionalidad es la movilidad de los portadores de carga μ

$$\left. \begin{aligned} \frac{\Delta Q_I}{\Delta x} &= c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) \\ v &= \mu \cdot E = \mu \cdot \frac{dV}{dx} \end{aligned} \right\} I = \mu \cdot c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) \cdot \frac{dV(x)}{dx}$$

Integrando la anterior ecuación diferencial a lo largo del canal resulta

$$\int_0^L I dx = \int_{V_S}^{V_D} \mu \cdot c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) dv$$

$$I = \mu \cdot c_{ox} \cdot \frac{W}{L} \cdot \int_{V_S}^{V_D} (V_G - V_{TO} - V(x)) dv = \alpha \cdot A$$

donde $A = \int_{V_S}^{V_D} (V_G - V_{TO} - V(x)) dv$ expresa el efecto de las tensiones aplicadas

sobre el transistor y $\alpha = \mu \cdot c_{ox} \cdot W/L$ incluye parámetros tecnológicos y geométricos:

K_P = coeficiente de transconductancia = $\mu \cdot c_{ox}$ (parámetros tecnológicos)

ff = factor de forma = W/L , cociente entre anchura W y longitud L del transistor.

Ambos influyen en proporcionalidad directa sobre la intensidad que conduce el transistor.

e) Para V_{DS} muy pequeño: $V_{DS} \ll V_{GS}$ la expresión $(V_G - V_{TO} - V(x))$ es prácticamente constante a lo largo de todo el canal: $(V_G - V_{TO} - V_S) = V_{GS} - V_{TO}$; la tensión aplicada es la misma a lo largo de todo el canal, de forma que el canal resulta plano, con una distribución de carga uniforme.

En tal caso, la integral es inmediata y la intensidad resultante será:

$$I_D = K_p \cdot \frac{W}{L} \cdot (V_{GS} - V_{TO}) \cdot V_{DS}$$

expresión que muestra una proporcionalidad directa entre la intensidad que pasa por el transistor y la tensión aplicada sobre el mismo, lo cual indica que el transistor se comporta como una simple resistencia cuyo valor será:

$$R_{eq} = \frac{I_D}{V_{DS}} = \frac{1}{K_p \frac{W}{L} (V_{GS} - V_{TO})}$$

valor que disminuye al aumentar la tensión de puerta V_{GS} . Esta región de funcionamiento del transistor se denomina zona lineal o zona óhmica.

Esta es la situación booleana que corresponde a un transistor MOS en conducción: equivale a una resistencia cuyo valor puede hacerse adecuadamente pequeño a través del factor de forma, W/L ; la resistencia es inversamente proporcional a su anchura W .

f) Al aumentar V_{DS} el canal se hace más estrecho (presenta menor número de portadores) en el lado del drenaje (pues $V_{GD} < V_{GS}$) y, al aumentar la tensión del drenaje, llega un momento en que el canal se satura, es decir, la diferencia de tensiones $V_G - V(x)$ no supera la tensión umbral, $V_G - V(x) < V_{TO}$ y, por ello, en tal zona no hay capa de inversión que contribuya a la conducción: para $V(x) > V_G - V_{TO}$ no existe canal.

Esta situación de canal saturado se da si $V_D > V_G - V_{TO}$, en cuyo caso la integral **A** se extiende solamente a la zona en que $V_G - V_{TO} - V(x)$ es positiva y la intensidad resultante es:

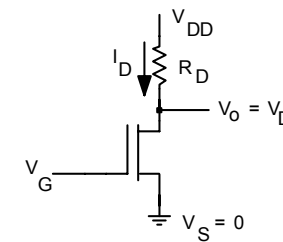
$$I_{D,sat} = \frac{K_p}{2} \frac{W}{L} (V_{GS} - V_{TO})^2$$

expresión que no depende de la tensión de drenaje y que representa la intensidad máxima que el transistor puede conducir para una tensión de puerta V_{GS} determinada; esta intensidad máxima (la de canal saturado) depende de la tensión de puerta, con la cual aumenta cuadráticamente.

g) Entre ambas zonas: zona lineal y canal saturado, la intensidad (resultante de integrar **A**) es:

$$I_D = K_p \cdot \frac{W}{L} \cdot \left(V_{GS} - V_{TO} - \frac{V_{DS}}{2} \right) \cdot V_{DS}$$

Etapa en fuente común:



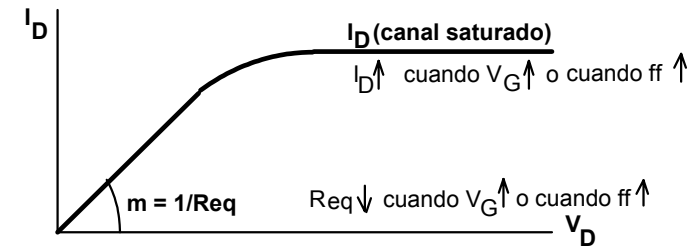
$$V_S = 0$$

zona lineal $R_{eq} = K_p \frac{W}{L} \left(V_G - V_{TO} - \frac{V_D}{2} \right) V_D$

canal saturado $I_{D,sat} = \frac{1}{K_p \frac{W}{L} (V_G - V_{TO})}$

Para canal no saturado: $I_D = K_p \cdot \frac{W}{L} \cdot \left(V_G - V_{TO} - \frac{V_D}{2} \right) \cdot V_D$

Representando la intensidad que circula por el transistor (drenaje-fuente) I_D en función de la tensión entre sus terminales V_D se obtiene la curva característica de la conducción del transistor:



Actuando sobre las dimensiones geométricas del transistor (sobre su factor de forma $ff = W/L$), se modifica su resistencia en zona lineal y, en sentido contrario, la intensidad que conduce con canal saturado.

$$W \uparrow \text{ ó } L \downarrow \text{ (en ambos casos } ff \uparrow) \Rightarrow R_{eq} \downarrow ; I_D \text{ (canal saturado)} \uparrow ;$$

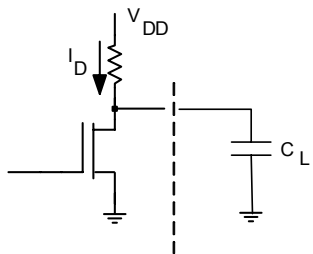
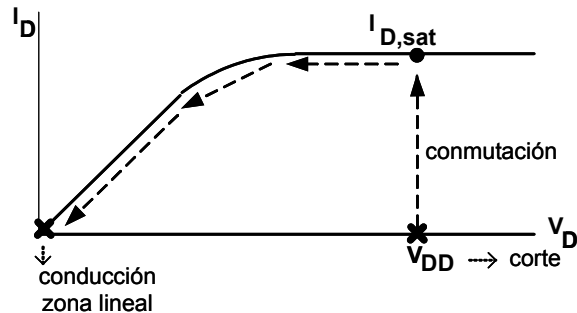
la magnitud de ambas variaciones es proporcional, con el mismo factor de proporcionalidad, a la modificación del factor de forma.

Comportamiento booleano:

Las situaciones booleanas de un transistor MOS corresponden a:

- transistor en corte $I = 0$ $V_{GS} < V_{TO}$
 = 
- transistor en zona lineal $V_{DS} = 0$ $V_{GS} > V_{TO}$
 =  $R_{eq} = \frac{1}{K_p \frac{W}{L} (V_G - V_{TO})}$

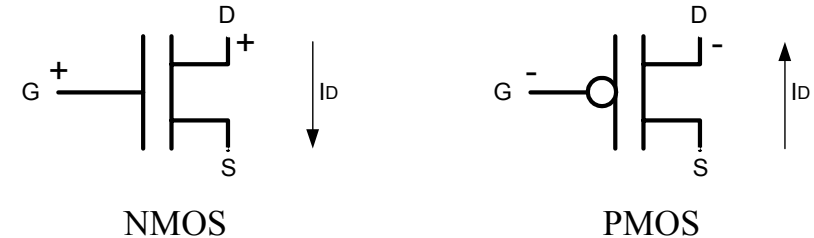
En la conmutación, en la puesta en conducción del transistor, éste recorre la curva característica correspondiente a $V_{GS} = V(1)$ desde la situación de canal saturado hasta alcanzar la zona lineal con $V_{DS} = 0$ V.



La $I_{D,sat}$ señala la máxima intensidad disponible por el transistor, con ella se inicia el proceso de conmutación, para descargar las capacidades equivalentes conectadas a su salida (debidas a otras puertas o dispositivos a los que el transistor comunica el valor booleano 0).

Transistores PMOS

En el transistor MOS de canal P son los huecos (en lugar de los electrones) los portadores que forman el canal y, por ello, sus tensiones e intensidades tienen signo opuesto al que presentan en el transistor NMOS: el transistor PMOS requiere tensiones de polarización V_D y de puerta V_G negativas respecto al sustrato y a la fuente, siendo también negativa su tensión umbral V_{TO} .



Además, la movilidad de los huecos es unas tres veces inferior a la de los electrones:

- Los electrones de la banda de conducción se encuentran efectivamente libres, han dejado su correspondiente enlace de valencia y se mueven por efecto del campo eléctrico con una cierta movilidad μ_e .
- En cambio, los huecos se encuentran en la banda de valencia y no se mueven por sí mismos sino como resultado de que un electrón ligado, que se encontraba en un enlace, pasa a cubrir el hueco y deja un nuevo hueco en su enlace anterior; de manera que el hueco se mueve en sentido contrario a como lo hacen los electrones ligados que lo rellenan.
- Siempre es más costoso movilizar un electrón ligado (que forma parte de un enlace en la banda de valencia) que un electrón libre (que se encuentra ya suelto en la banda de conducción); la movilidad de los huecos es muy inferior a la de los electrones.

En la práctica, la movilidad de los huecos μ_h resulta del orden de un tercio de la movilidad de los electrones μ_e ; por ello, a igualdad de dimensiones (a igualdad de factor de forma W/L), la intensidad conducida por un transistor PMOS será muy inferior a la de un transistor NMOS y la resistencia que presenta en zona lineal será considerablemente mayor para el transistor PMOS respecto al NMOS.

Modelos SPICE

El simulador eléctrico **SPICE** *Simulation Program With Integrated Circuits Emphasis* permite diversos niveles para caracterizar los transistores MOS.

El modelo de nivel 1, modelo de Shichman - Hodges, corresponde a las funciones desarrolladas anteriormente:

$$\begin{cases} I_D = K_p \frac{W}{L} \left(V_{GS} - V_{TO} - \frac{V_{DS}}{2} \right) V_{DS} & \text{canal no saturado} \\ I_D = \frac{K_p}{2} (V_{GS} - V_{TO})^2 & \text{canal saturado} \end{cases}$$

En este modelo la descripción SPICE de un transistor MOS requiere solamente tres parámetros:

KP	K_p parámetro de transconductancia
VTO	V_{TO} tensión umbral del transistor
TOX	t_{ox} espesor del óxido de puerta

TOX es necesario para tener en cuenta la capacidad de puerta; caso de no utilizarlo, se supone capacidad de puerta nula.

Ejemplos del modelo de transistores MOS (nivel 1):

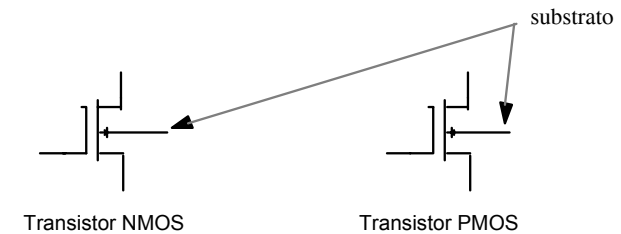
```
MODEL "nombre" NMOS LEVEL=1 KP=40U VTO=1 TOX=50E-9
```

```
MODEL "nombre" PMOS LEVEL=1 KP=15U VTO=-1 TOX=50E-9
```

Es necesario indicar también la dimensión de los transistores **L**, **W** bien dentro del modelo o en la declaración de cada transistor; por defecto configura las dimensiones de los transistores de 1 metro (dichos transistores de un metro cuadrado son tan grandes que dan lugar a resultados de simulación erróneos).

Nota aclaratoria sobre los símbolos utilizados para los transistores MOS

Los transistores MOS son dispositivos de 4 terminales: fuente, drenaje, puerta y sustrato y sus símbolos normalizados son:



En el caso de transistores MOS individuales, ciertamente hay ejemplares comerciales con los cuatro terminales disponibles, pero es más frecuente que tengan solamente tres terminales externos, con el sustrato conectado a la fuente:



Aún más, en el caso de circuitos integrados, el sustrato de todos los transistores NMOS es común y lo mismo sucede con el sustrato de los transistores PMOS, por lo que no resulta necesario dibujar el terminal de sustrato para cada transistor individual.

Por ello, los siguientes símbolos MOS con tres terminales (fuente, drenaje y puerta) resultan sumamente adecuados para representar el funcionamiento de los transistores MOS en los circuitos digitales y son los que se utilizan en este texto:



Estos símbolos no incluyen el terminal correspondiente al sustrato y, por tanto, no reflejan la polarización del sustrato de los transistores (necesaria para que las uniones de la difusión y del canal con el sustrato estén en polarización inversa y el transistor quede aislado del sustrato).

Habrà de tenerse en cuenta en forma implícita que el sustrato de los transistores NMOS ha de conectarse a la tensión más negativa de las de alimentación del circuito (que, generalmente, será el terminal correspondiente a 0 V), mientras que el sustrato de los transistores PMOS ha de estar conectado a la tensión más positiva de ellas (que suele ser el terminal correspondiente a V_{CC}).

T2 LOS PROCESOS DE FABRICACION CMOS

- T2.1. Etapas básicas en la integración CMOS
- T2.2. La litografía para delimitar zonas diferenciadas
- T2.3. Tipos de procesos tecnológicos implicados
- T2.4. Un ejemplo detallado de integración
- T2.5. Parámetros físicos de las regiones CMOS

El objetivo del presente capítulo es presentar el proceso de fabricación de circuitos integrados CMOS de forma que se comprenda la finalidad de cada una de las etapas del proceso y, a la vez, se profundice en la comprensión de la estructura de los transistores MOS y en la manera de conectarlos para formar circuitos digitales.

La integración CMOS consiste en formar zonas semiconductoras *N* y *P* y la zona de óxido de puerta con polisilicio encima de ella e interconectar los diversos transistores entre sí y con la fuente de alimentación, todas estas conexiones mediante líneas de metal (aluminio).

Las regiones citadas no se encuentran en el mismo plano sino en «pisos» sucesivos: las difusiones penetran en la oblea semiconductor, el óxido de puerta y el polisilicio se elevan sobre ella y el metal circula por encima de todo el conjunto. Sendas capas de óxido separan los transistores entre sí y al metal de todo lo que tiene debajo, salvo en los puntos en que debe establecer conexión.

Pero, además, las difusiones *N* requieren un sustrato *P* que es el dopado propio de la oblea mientras que las difusiones *P* precisan de sustrato *N* que habrá que formarlo previamente sobre la oblea *P*: los pozos. El sustrato *P* debe estar polarizado a la tensión más negativa y los pozos *N* a la más positiva, en ambos casos con la finalidad de que las uniones difusión-sustrato queden aisladas, en polarización inversa.

Por ello, en la superficie de la oblea, separados por óxido denominado de campo, tendremos los transistores y los contactos de polarización de los sustratos; al conjunto de todos ellos (transistores, polarizaciones) les denominamos zonas activas.

La presentación de las etapas del proceso de fabricación CMOS se hace en dos vueltas: primero, se explica conceptualmente el esquema básico de integración; luego, se comenta la litografía, necesaria para diferenciar las zonas de actuación sobre la oblea, y se indican los diversos procesos tecnológicos que se utilizan, para pasar a exponer, muy detalladamente, cada uno de los pasos de la integración.

Además, se incluye un apartado final con datos numéricos de los parámetros físicos (concentración de impurezas, de portadores, conductividad, capacidad, ...) de las diversas regiones CMOS a fin de poder disponer de una perspectiva cuantitativa y poder comparar unas regiones con otras (y comparar, también, con las características propias de un conductor).

T2.1.. Etapas básicas en la integración CMOS

La presentación de las etapas del proceso de integración en este apartado es conceptual y esquemática; en los apartados siguientes se explicarán en detalle los diversos procesos tecnológicos y estas mismas etapas desglosadas en acciones concretas. Las «máscaras» son el lazo de unión entre el diseño y la fabricación; son los «dibujos» que muestran la separación entre unas zonas y otras en la superficie de la oblea, diferenciando la región sobre la que se actúa en cada etapa del resto que permanece inalterado en la misma etapa.

Las figuras se refieren a la integración de un inversor, como puerta más simple, conformada por un solo transistor PMOS y otro NMOS.

Los valores referidos a espesores y profundidades dependen del proceso tecnológico concreto; tales datos numéricos se incluyen para poder apreciar su orden de magnitud y para comparar la magnitud relativa entre ellos.

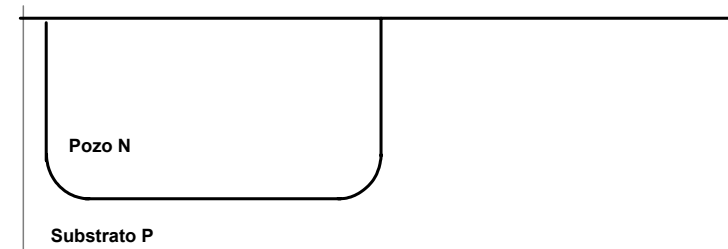
0. Oblea inicial de Silicio dopado tipo P

diámetro: 100 - 125 mm

grosor: 0,5 mm = 500 μm .

1. Formación de los Pozos N

Sobre el sustrato *P*, que es la propia oblea, se formarán los transistores NMOS; es preciso crear zonas de sustrato *N* para los transistores PMOS: los pozos.



MÁQSCARA 1: pozos

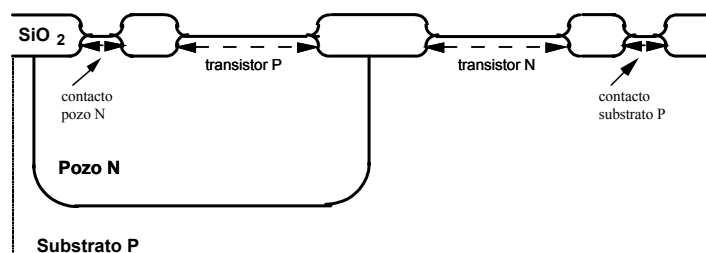
profundidad de los pozos ~ 5 μm

2. Demarcación de las zonas activa:

transistores y contactos de polarización del sustrato

Además de transistores NMOS y PMOS se integran zonas de difusión para polarizar los sustratos. El sustrato P (oblea) se polarizará a la tensión más negativa de las de alimentación (en digital a 0 V), a fin de que todas las uniones NP de las difusiones de los transistores NMOS (incluyendo el canal cuando se forme) queden en polarización inversa y con ello dichos transistores resulten aislados del propio sustrato. Por la misma razón los pozos N se polarizarán a la tensión más positiva (en digital Vcc) y los transistores PMOS quedarán aislados de su sustrato (del pozo en que se encuentran).

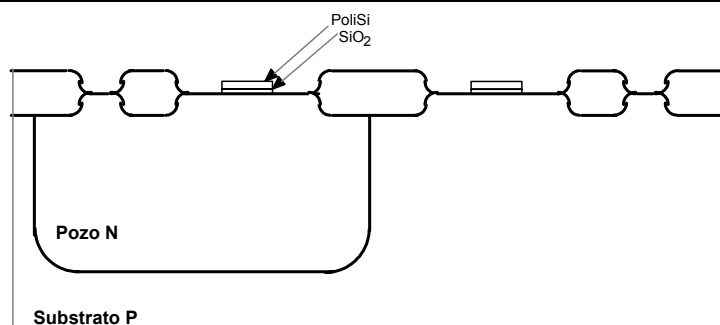
Unas barreras intermedias de óxido de silicio, denominado óxido de campo, separarán los transistores entre sí y éstos de los contactos de polarización; transistores y contactos de polarización son considerados «zonas activas».



MÁSCARA 2: zonas activas

espesor del óxido de campo ~ 1 μm

3. Óxido de puerta y Polisilicio

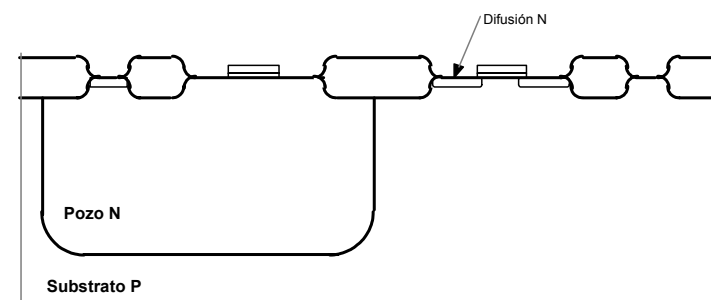


MÁSCARA 3: polisilicio

espesor del óxido de puerta ~ 0,05 μm

espesor del polisilicio ~ 0,2 μm

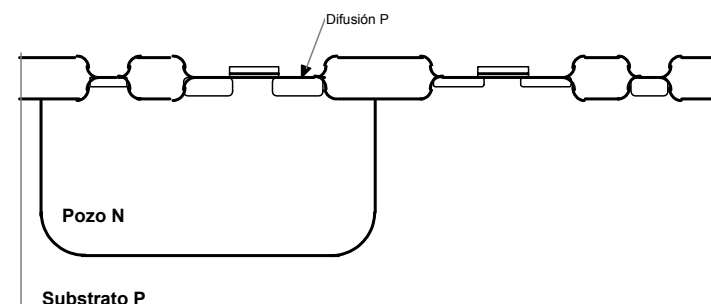
4. Difusión N



MÁSCARA 4: difusión N

profundidad de la difusión N ~ 0,2 μm

5. Difusión P

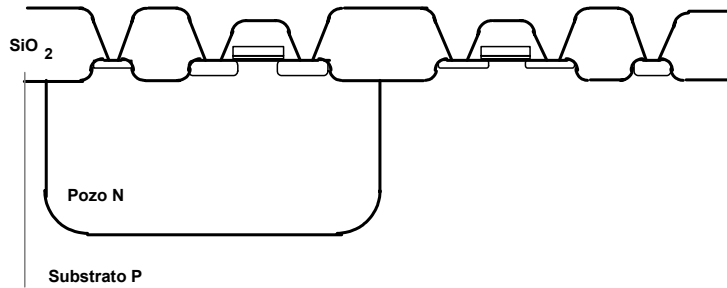


MÁSCARA 5: difusión P

profundidad de la difusión P ~ 0,5 μm (doble que la Difusión N)

6. Contactos

Una capa de óxido de silicio, óxido de aislamiento o de recubrimiento, sirve para separar el metal (que conecta los diversos transistores entre sí y con las tensiones de alimentación) respecto al polisilicio y las difusiones que se encuentran debajo del mismo. Ahora bien, dicha capa de óxido debe llevar «agujeros» para los contactos de las líneas de metal con las diversas zonas que deben ir conectadas (los contactos son como los puntos de soldadura en las placas de circuitos impresos).

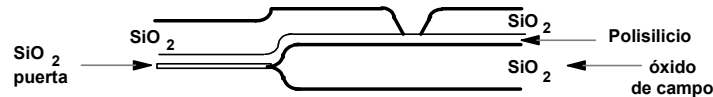
**MÁSCARA 6: contactos**

espesor del óxido de aislamiento ~ **0,7 μm**

dimensiones de los contactos ~ **2 x 2 μm**

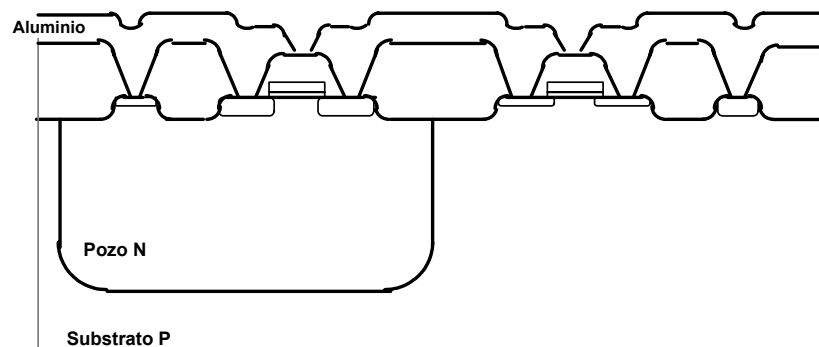
(las dimensiones de los contactos dependen fuertemente de las dimensiones mínimas de la tecnología).

El polisilicio también recibe contactos pero no sobre las zonas de puerta (por eso no se ven en la figura, la cual corresponde a un corte vertical por la zona de puertas):



El polisilicio continúa su recorrido, más allá de las zonas activas, por encima del óxido de campo, efectuando conexiones en polisilicio (por ejemplo, las puertas de los dos transistores MOS complementarios) y conectando también con el metal de conexionado general.

7. Conexiones en metal

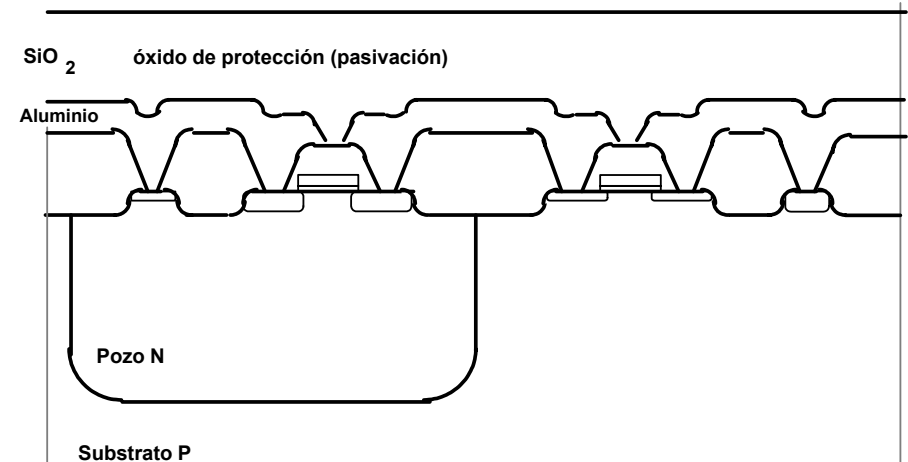
**MÁSCARA 7: metal**

espesor del aluminio ~ **0,5 μm**

Por simplicidad del proceso de integración suponemos que solamente incluye un metal, pero hoy día se integran varios metales superpuestos y cada uno de ellos requiere una capa de óxido de silicio para separarlo del anterior metal, con contactos que se establecen siempre de cada metal con el inferior (*vías*); cada metal precisa de dos máscaras: una para las vías y otra para delimitar las líneas del propio metal.

8. Pasivación

Una gruesa capa superior de óxido de silicio sirve para proteger todo el circuito integrado; en la misma deberán dejarse abiertos «agujeros» (PADs) de las zonas en las que se soldarán los hilos que unen el circuito con los terminales (*pines*) de la carcasa de plástico (o cerámica) que lo contiene.

**MÁSCARA 8: pasivación**

espesor de la capa exterior de óxido ~ **2 μm**

La máscara de pasivación delimita simplemente los espacios de metal (PADs) donde se soldarán los citados hilos que conectan con los terminales de la carcasa.

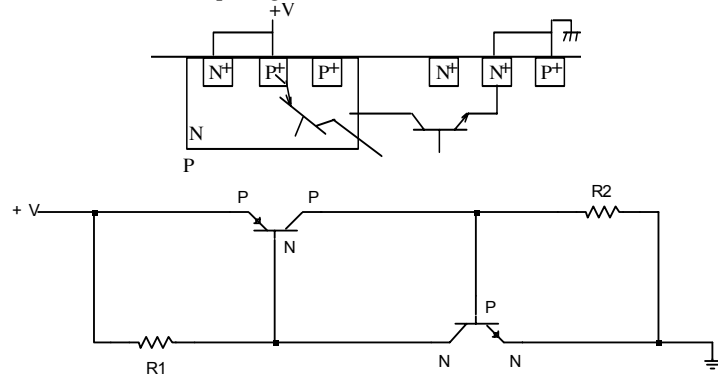
Transistores MOS autoaislados: el problema del "latch-up"

Gracias a la polarización de los sustratos los transistores MOS se encuentran eléctricamente aislados de su sustrato y, por ello, aislados entre sí.

Los transistores NMOS están constituidos por regiones N (fuente - canal - drenaje) en el seno de un sustrato de tipo P; para aislar los transistores del sustrato y para aislar los diversos transistores entre sí se requiere polarizar el sustrato P con la tensión de alimentación más negativa, de forma que todas las uniones NP con el sustrato se encuentren polarizadas inversamente.

De igual forma los transistores PMOS están constituidos por regiones P en el seno de un sustrato de tipo N (pozo); para aislarlos se requiere polarizar el sustrato N (todos los pozos presentes en el integrado) con la tensión de alimentación más positiva, para que todas las uniones PN se encuentren polarizadas en inverso.

Este requisito de aislamiento de los transistores se encuentra reforzado por el hecho de que las regiones N y P propias de las estructuras CMOS se encuentran intercaladas entre sí y permiten la configuración de estructuras parásitas tipo tiristor (PNPN), con características destructivas al ser posible el disparo de una conducción de intensidad incontrolada. Este fenómeno autodestructivo debido al autodisparo de las estructuras PNPN se conoce como *latch-up* (enganche).



Si la caída de tensión en alguna de las resistencias transversales R_1 y R_2 consigue acercarse a 0,5 V, ambos transistores conducen sin limitación de intensidad (de hecho se encuentran realimentados positivamente: el aumento de la I_C de uno de ellos provoca el de la I_B del otro, con el consiguiente aumento de su I_C y, por tanto, de la I_B del primero), configurando un tiristor en situación de disparo y sin limitación de intensidad.

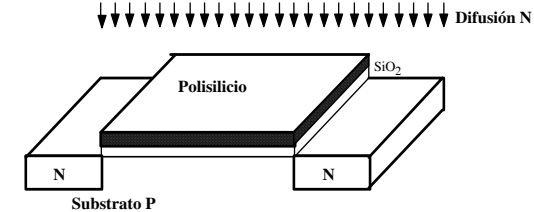
Ambas resistencias se forman a través de los sustratos, de manera que interesa que la resistividad de ambos sustratos (el P de la oblea y el N de los pozos) no sea muy alta y que la tensión de polarización alcance eficazmente a toda la masa del sustrato (para que las diferencias de tensión entre dos puntos del sustrato sean mínimas).

En tal sentido es fundamental asegurar una polarización intensiva de toda la superficie de los sustratos: para evitar la posibilidad de *latch-up* es preciso polarizar cada sustrato con múltiples contactos, de forma que cualquier punto de su superficie se encuentre cercano a un contacto de polarización; generalmente se requiere que la distancia de cualquier punto a un contacto de polarización sea inferior a 50 μm .

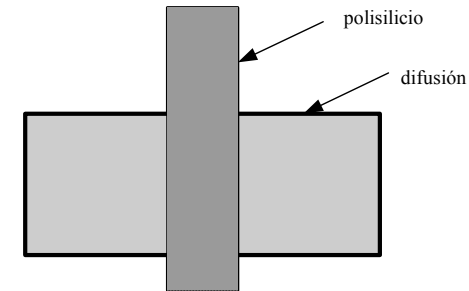
Tengamos presente que la polarización de los sustratos es un requisito previo a las consideraciones sobre *latch-up*: es necesaria para aislar los transistores del sustrato, para establecer uniones PN inversas entre ambos. El fenómeno del *latch-up* añade la exigencia de que tal polarización se haga en forma múltiple e intensiva, de forma que cada punto de la superficie del sustrato se encuentre a menos de 50 μm de un contacto de polarización.

Transistores MOS autoalineados

La puerta y el polisilicio se integran antes que las difusiones y la máscara de difusión es continua y cruza a la del polisilicio; por ello, la puerta y las dos zonas de difusión (fuente y drenaje) quedan autoalineadas: la difusión se realiza sobre todo el transistor pero no penetra debajo del polisilicio.



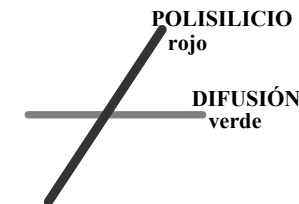
El transistor se forma físicamente por el cruce de 2 regiones: polisilicio y difusión; debajo del polisilicio la difusión se encuentra interrumpida (ha penetrado en el propio polisilicio pero no pasa debajo del mismo).



El cruce de máscaras polisilicio-difusión siempre da lugar a un transistor MOS.

Un pequeño desplazamiento de la máscara de difusión respecto a la de polisilicio no impide la formación del transistor en el cruce de ambas. En tal sentido se dice que el transistor está autoalineado: puede formarse un poco desplazado respecto a ambas máscaras pero se forma siempre que ambas se cruzan.

Abstrayendo las dimensiones superficiales, el transistor puede representarse en un diagrama lineal como un cruce entre dos líneas: difusión - polisilicio.



Las características de autoalineados y autoaislados que poseen los transistores MOS reducen el tamaño necesario para su integración y permiten situarlos muy próximos entre sí, de modo que la densidad de integración en tecnologías MOS es muy alta.

T2.2. La litografía para delimitar zonas diferenciadas

El proceso de fabricación de un circuito integrado CMOS se basa en la configuración de diversas zonas superficiales: pozos, zonas activas, líneas de polisilicio, zonas de difusión N, zonas de difusión P, contactos, líneas de metal (vías entre metal 1 y metal 2, líneas de metal 2,...), puntos de conexión hacia el exterior (PADs), etc. La construcción de estas zonas da lugar a etapas sucesivas en el proceso de integración.

En cada una de las etapas resulta necesario distinguir las áreas sobre las que se actúa del resto de la superficie de la oblea; la delimitación de áreas viene dada por las máscaras resultantes del proceso de diseño del circuito integrado.

La forma de discriminar o dividir la superficie de la oblea en dos regiones, para actuar sobre una de ellas y no sobre la otra, es análoga a la forma de realizar circuitos impresos a partir del fotolito que contiene el dibujo de sus pistas:

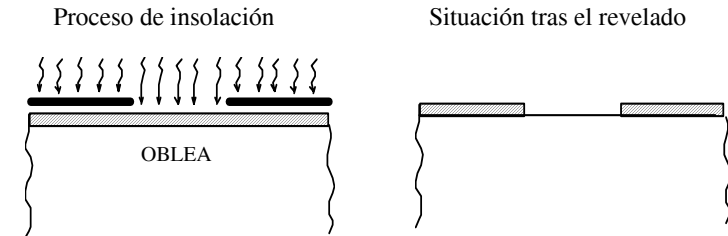
- a) cubrir la superficie metálica de la placa con una resina fotosensible,
- b) insolar con luz ultravioleta a través del fotolito,
- c) revelar la resina dando lugar a la distinción entre áreas protegidas por la misma y áreas sin protección,
- d) ataque con un ácido (grabado) para disolver el metal de las zonas no protegidas.

Existen dos clases de resinas sensibles a la luz (fotorresinas): aquellas en las que el revelado elimina la resina en las zonas expuestas a la radiación (resinas fotopositivas) y aquellas otras en que la resina es eliminada en las zonas no iluminadas (resinas fotonegativas).

Este proceso consistente en trasladar a la superficie de la placa metálica o de la oblea de silicio el dibujo de un fotolito o máscara recibe el nombre de litografía (ya que su objetivo es análogo a la impresión de textos y dibujos sobre papel).

La litografía utilizada en los circuitos impresos y la empleada para fabricar circuitos integrados se diferencian en el factor de escala: en el caso de los circuitos impresos, las anchuras de sus líneas son del orden de la décima del milímetro y en los circuitos integrados actuales corresponden a décimas de micra.

La litografía permite transferir a la superficie de la oblea el dibujo representado en la máscara; la transcripción se realiza mediante una fotorresina que, una vez revelada, establece una diferenciación superficial entre las zonas protegidas por la fotorresina residual y las zonas abiertas, disponibles para ser «grabadas», es decir, para ser atacadas químicamente o sometidas a implantación iónica.



El proceso se realiza en la forma que sigue:

- sobre la superficie de la oblea se depositan unas gotas de resina y mediante giro rápido se forma, por efecto centrífugo, una delgada película homogénea
- insolando con luz ultravioleta, a través de la correspondiente máscara, se impresiona la resina en las zonas en que la máscara permite el paso de luz
- en el revelado, mediante un adecuado disolvente químico, la resina es eliminada de las zonas expuestas a la luz (resina fotopositiva) o de las zonas no iluminadas (resina fotonegativa).

La utilización de radiación ultravioleta (cuya longitud de onda es inferior a la de las radiaciones visibles) permite una resolución de $0,4\ \mu\text{m}$. El interés por seguir reduciendo las dimensiones de transistores y conexiones para aumentar la densidad de integración ha impulsado el desarrollo de litografía por haces de electrones (resolución $\sim 0,1\ \mu\text{m}$) y litografía por rayos X (resolución $0,01\ \mu\text{m}$).

Para fabricar un número de circuitos reducido (en particular, para prototipos) se utilizan procedimientos de litografía sin máscaras, insolando la resina mediante un chorro de electrones (*electron bean*) controlado por ordenador, el cual dibuja secuencialmente (en un barrido por líneas, análogo al de TV) la correspondiente figura geométrica. Esta litografía por *electron bean*, al no utilizar máscaras, abarata considerablemente la fabricación de series reducidas (hace viable la fabricación de circuitos integrados para un reducido número de ejemplares).

La etapa siguiente a una litografía es siempre una grabación, es decir, el ataque de las zonas abiertas (no protegidas) con un disolvente ácido, o una difusión, o sea, la inserción de impurezas (fósforo o boro) a través de dichas zonas para formar regiones P o N.

T2.3. Tipos de procesos tecnológicos implicados

a) Litografía y grabado

LITO Litografía: comentado en el apartado anterior. La litografía comprende varias acciones sucesivas: depósito y homogeneización de la resina, posicionamiento respecto a la máscara, insolación, revelado,...

GRAB Grabado: Consiste en eliminar el óxido de silicio, el polisilicio o el metal de las zonas no protegidas por la fotorresina, una vez que ésta ha sido revelada; la litografía ha reflejado sobre la superficie del material a eliminar el dibujo de la máscara empleada, distinguiendo la zona en que debe permanecer dicho material del resto en que debe ser eliminado. La grabación se efectúa con disolventes químicos adecuados, principalmente ácido fluorhídrico FH, controlando el tiempo de ataque de los mismos. En muchas ocasiones este proceso de grabado se realiza con el atacante vaporizado al vacío y en estado de plasma por aplicación de campos eléctricos de elevada frecuencia.

b) Oxidación

OXter Oxidación térmica: Se realiza directamente sobre la oblea en atmósfera oxidante a temperatura superior a los 1000°C. La oxidación húmeda, con vapor de agua, es un proceso relativamente rápido (~ 0,5 µm/hora) pero no muy preciso; se utiliza para hacer crecer el óxido grueso de separación de las zonas activas (óxido de campo). La oxidación seca, en atmósfera de O₂, es mucho más lenta (~ 0,02 µm/hora), permitiendo controlar con precisión el espesor del óxido; por ello se utiliza para el crecimiento del óxido fino de puerta. En ambos casos, la oxidación térmica consume sustrato, penetrando ligeramente el óxido en el mismo.

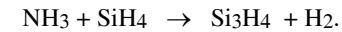
OXdep Depósito de óxido: Los óxidos de recubrimiento y de pasivación han de ser obtenidos mediante depósito; para ello suele emplearse la reacción del hidruro de silicio SiH₄ con el oxígeno, dando lugar, a través de un proceso de combustión, a óxido de silicio SiO₂ que se deposita y agua vaporizada. Esta deposición se realiza a temperatura de unos 500°C.

c) Depósito

POLI Depósito de polisilicio: Se realiza por vía química, forzando la ruptura molecular (*cracking*) del hidruro de silicio SiH₄, lo cual da lugar a deposición de silicio en forma policristalina (múltiples cristales).

ALdep Depósito de metal (Al): Se efectúa por vía física, mediante evaporación del aluminio en vacío o por pulverización catódica, a unos 400°C de temperatura, formando en ambos casos un plasma activo (generado por aplicación de campos de alta frecuencia); se añade al metal una pequeña proporción de silicio que facilita los contactos, al configurar una aleación Al-Si.

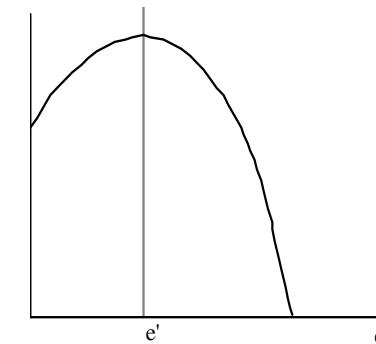
Nldep Depósito de nitruros de silicio: Tales nitruros se utilizan como aislantes en las primeras etapas de la integración, para formar el óxido de campo con perfiles suaves (perfiles de *pico de pájaro* >) en la separación de zonas activas, y luego se eliminan; se depositan por reacción química entre amoníaco e hidruro de silicio:



d) Difusión e implantación iónica

DIF Difusión: Representa la forma clásica de obtener el dopado del semiconductor, a temperaturas muy altas (> 1200°C), en un horno de cuarzo, con una atmósfera rica en la correspondiente impureza (generalmente fósforo para dopado tipo N y boro para tipo P).

Ión Implantación iónica: Se refieren a implantar en la superficie del sustrato, con profundidades del orden de 0,1 µm, átomos de impurezas que han sido previamente ionizados para poder acelerarlos mediante un campo eléctrico. La densidad de concentración de impurezas, tras un proceso de implantación iónica, presenta un perfil gaussiano con una fuerte pendiente de caída:



para campos eléctricos ~ 100 Kev

$e' \approx 0,25 \mu\text{m}$ en el caso del Boro

$e' \approx 0,15 \mu\text{m}$ en el caso del Fósforo

En un principio, la implantación iónica se utilizó para modificar superficialmente el dopado en la zona de canal y con ello ajustar la tensión umbral de los transistores. Actualmente se utiliza también para crear las zonas de difusión (fuente y drenaje), sustituyendo a los procesos de difusión; en este caso, la implantación iónica va seguida de un recocido que aumenta la profundidad de las zonas dopadas.

RECC Recocido: La implantación iónica da lugar a dopados muy superficiales; efectuando un recocido posterior a alta temperatura (> 1000°C) se produce una redistribución de las impurezas, que se difunden por efecto térmico, penetrando hacia el interior de la oblea y ocupando un mayor volumen.

e) Lavados

LAVO Lavado: A lo largo del proceso de integración se realizan numerosos lavados para eliminar la capa superficial (óxido, nitruro, resina,...) mediante el disolvente adecuado.

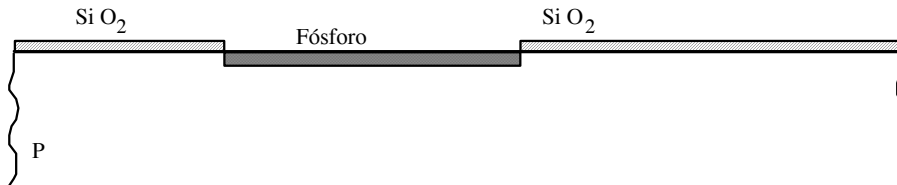
T2.3. Un ejemplo detallado de integración

A. Oblea de silicio, dopada P

dopado $\approx 10^{15}$ at/cm³ Ø: 100 - 125 mm e: 0,5 mm = 500 μ m

1. **LAVO** Lavado inicial de la oblea para limpiar a fondo su superficie.

B. Implantación iónica de fósforo para construir pozos N



2. **OXter** Oxidación de toda la superficie. e $\approx 0,5 \mu$ m

3. **LITO** Definición por litografía de todos los pozos. **M1 = pozos**

(en cada litografía se indica la máscara a utilizar, con la letra **M**)

4. **GRAB** Eliminación del óxido en las zonas de pozos.

5. **LAVO** Eliminación de la fotorresina residual.

6. **IIón(P)** Implantación iónica de fósforo.

concentración $\approx 10^{13}$ at/cm² $\equiv 5 \times 10^{17}$ at/cm³ e $\approx 0,2 \mu$ m

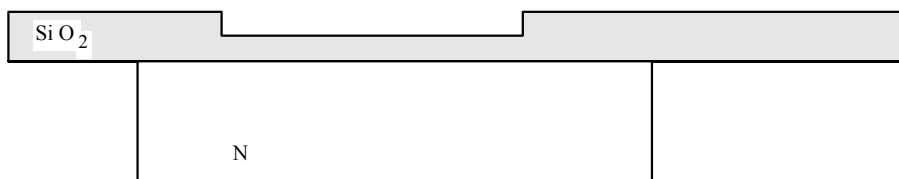
además de la formación del pozo, esta implantación iónica tiene dos utilidades complementarias:

a) da lugar a una zona N⁺ en la superficie que servirá para mejorar la separación entre zonas activas del propio pozo (debajo del óxido de campo) y la separación entre el pozo y el sustrato en los bordes de los pozos;

b) se controla adecuadamente para realizar (junto con la implantación de boro de la etapa H) el ajuste de la tensión umbral de los transistores PMOS.

C. Redistribución del fósforo para conformar el pozo

Se realiza por recocido en ambiente rico en oxígeno de forma que el óxido en la superficie del pozo impida la emigración hacia afuera del fósforo implantado.



7. **RECC** Recocido. 20 horas, rampa de temperatura de 900 a 1200°C

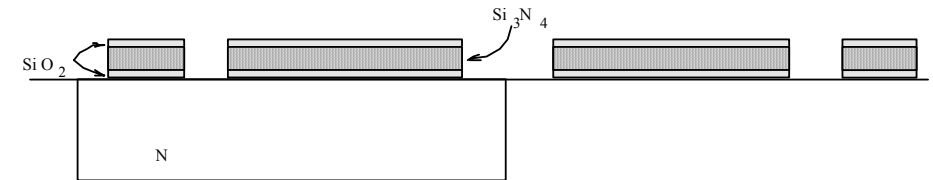
concentración fósforo $\approx 10^{16}$ at/cm² espesor pozo $\approx 5 \mu$ m

concentración N equivalente $10^{16} - 10^{15} \approx 10^{16}$

8. **LAVO** Eliminación de la capa superficial de óxido.

D. Demarcación de zona de activas

Las zonas activas delimitan los transistores y los contactos de los substratos; sobre ellas se deposita una capa protectora óxido-nitruro-óxido (SiO₂ - Si₃N₄ - SiO₂), para inhibir la formación del óxido de campo en ellas y para facilitar que la formación de dicho óxido se efectúe con perfiles suaves (tipo pico de pájaro).



9. **OXter** Oxidación fina de la superficie de la oblea. e $\approx 0,05 \mu$ m

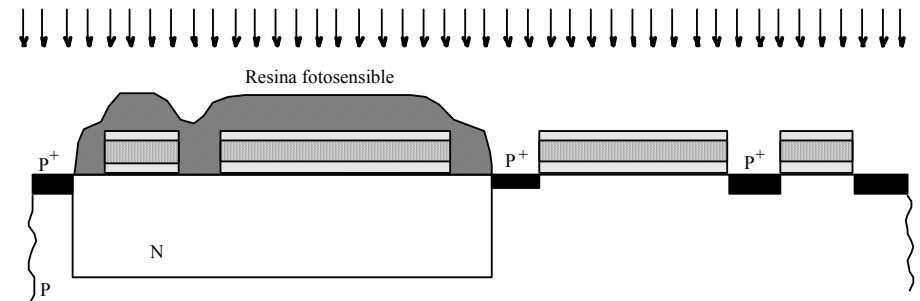
10. **NIdp** Depósito de nitruro de silicio. e $\approx 0,2 \mu$ m

11. **OXdep** Depósito de óxido de silicio. e $\approx 0,05 \mu$ m

12. **LITO** Definición por litografía de las zonas activas. **M2 = zonas activas** se utiliza resina fotosensible negativa para dejar protegidas dichas zonas

13. **GRAB** Eliminación del óxido y nitruro de las zonas no protegidas.

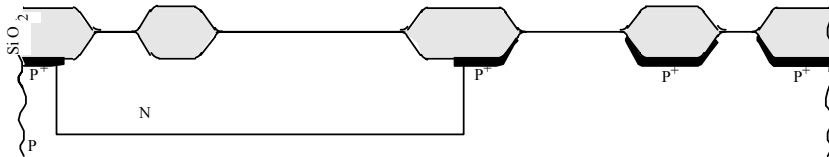
14. **LAVO** Eliminación de la fotorresina residual.

E. Difusión del boro (formación de zonas P⁺) para mejorar las separaciones

15. **LITO** Protección por litografía de los pozos. **M1 = pozos**
se utiliza resina fotosensible negativa para proteger los pozos.
16. **DIF(B)** Difusión de boro en el sustrato P (zonas de óxido de campo).
esta difusión formará zonas semiconductoras P⁺ en los extremos de los pozos
y en las zonas de separación entre las zonas activas del sustrato P
concentración $\approx 10^{17}$ at/cm³ $e \approx 0,2 \mu\text{m}$
17. **LAVO** Eliminación de la fotorresina residual.

Las zonas P⁺ de separación formadas en esta etapa dentro del sustrato P mejoran el aislamiento de los transistores de canal NMOS y el aislamiento de los pozos en sus bordes; el mismo efecto se produce en los pozos mediante la implantación de fósforo en la etapa B.

F. Oxidación gruesa para separar las zonas activas



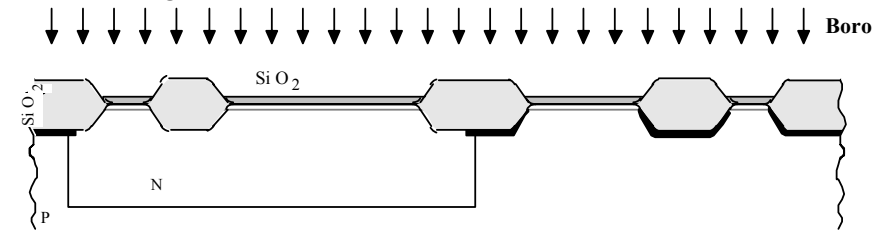
18. **OXter** Oxidación gruesa de la superficie. $e \approx 1 \mu\text{m}$
oxidación de las zonas no protegidas por el óxido-nitruro-óxido
19. **LAVO** Eliminación de la capa superficial óxido-nitruro-óxido.

G. Oxidación fina de puerta

20. **OXter** Oxidación fina para formar el dieléctrico de puerta.
 $e \approx 0,05 \mu\text{m} = 500 \text{ \AA}$

Esta etapa de oxidación fina y muy controlada para formar el óxido de puerta de todos los transistores MOS es la más delicada de todo el proceso de integración, ya que de las características de este óxido de puerta y de su espesor (cuya magnitud es del orden de las centésimas de micra) depende el correcto funcionamiento de los transistores MOS y el valor concreto de su transconductancia.

H. Ajuste de V_{TO} por implantación de boro

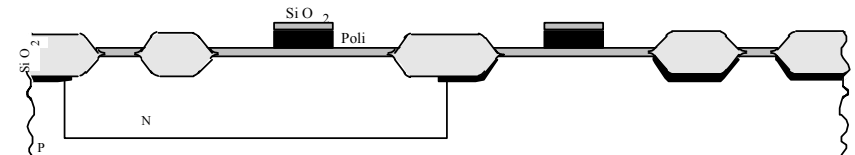


21. **IIón(B)** Implantación iónica de boro en toda la superficie
para ajustar V_{TO} de los transistores N.

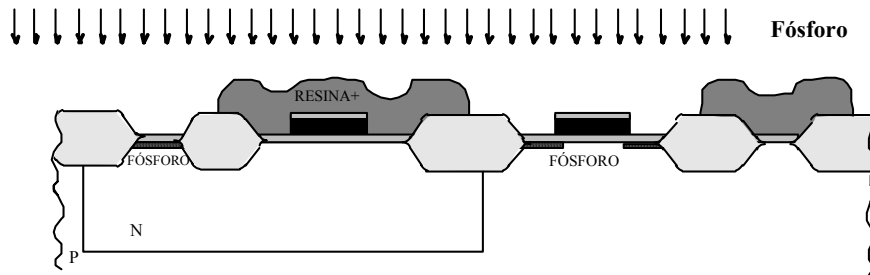
concentración de boro $\approx 10^{12}$ at/cm² $\approx 10^{16}$ at/cm³ $e \approx 0,2 \mu\text{m}$

Con esta implantación iónica se ajusta la tensión umbral V_{TO} de los transistores NMOS; además, esta implantación combinada con la de fósforo en la etapa A sirve para ajustar también la de los transistores PMOS. Inicialmente las tensiones umbrales resultan del orden de 0,4 V para NMOS y - 2 V para PMOS; la implantación de boro aumenta ambas tensiones (en sentido positivo) y sitúa su valor absoluto en el entorno de 1 V.

I. Depósito de polisilicio y recorte del mismo



22. **POLI** Depósito de polisilicio sobre toda la superficie. $e \approx 0,2 \mu\text{m}$.
23. **DIF(P)** Dopado del polisilicio con fósforo
para hacerlo buen conductor (semiconductor N).
concentración $N_D \approx 10^{20}$ at/cm³
24. **OXter** Formación de una capa de óxido protector sobre el polisilicio. $e \approx 1 \mu\text{m}$
25. **LITO** Definición por litografía de las zonas de puerta y conexiones en polisilicio. **M3 = poli**
se utiliza resina fotosensible negativa
26. **GRAB** Eliminación del óxido en el resto de la superficie, no protegida por la resina.
27. **GRAB** Eliminación del polisilicio en la parte abierta, no protegida por el óxido anterior.
28. **LAVO** Eliminación de la resina residual.

J. Implantación de fósforo (transistores NMOS y contactos sustrato N)

29. **LITO** Definición por litografía de las zonas N⁺ (transistores canal N y contactos sustrato N). **M4 = dif.N**

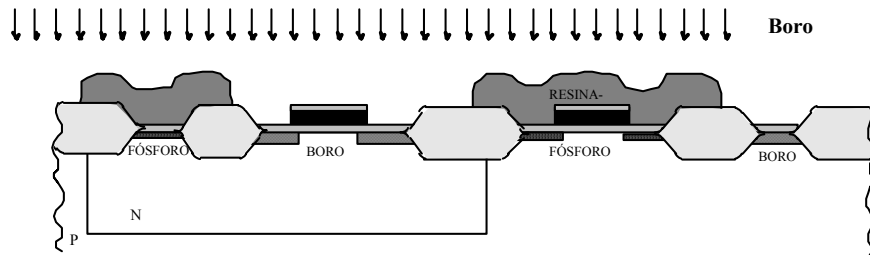
se utiliza resina fotopositiva que deja descubiertas dichas zonas

30. **IIón(P)** Implantación en ellas de fósforo.

$\varepsilon \approx 100 \text{ Kev}$

concentración $\approx 10^{15} \text{ at/cm}^2 \approx 10^{20} \text{ at/cm}^3$ $e \approx 0,1 \mu\text{m}$

31. **LAVO** Eliminación de la resina residual.

K. Implantación de boro (transistores P y contactos sustrato P)

32. **LITO** Definición por litografía de las zonas P⁺ **M4 = dif.N** (transistores canal P y contactos sustrato P).

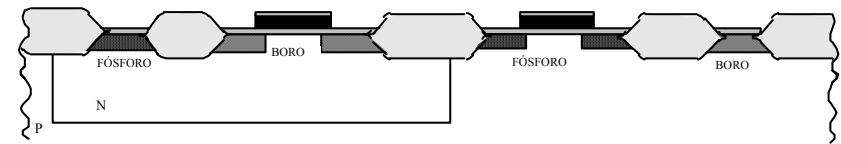
se emplea la misma máscara que en la etapa anterior y resina fotosensible negativa que deja protegidas las zonas N⁺.

33. **IIón(B)** Implantación de boro en el resto.

$\varepsilon \approx 100 \text{ Kev}$

concentración $\approx 10^{15} \text{ at/cm}^2 \approx 5 \times 10^{19} \text{ at/cm}^3$ $e \approx 0,2 \mu\text{m}$

34. **LAVO** Eliminación de la resina residual.

L. Conformación por redistribución de las zonas de difusión N⁺ y P⁺

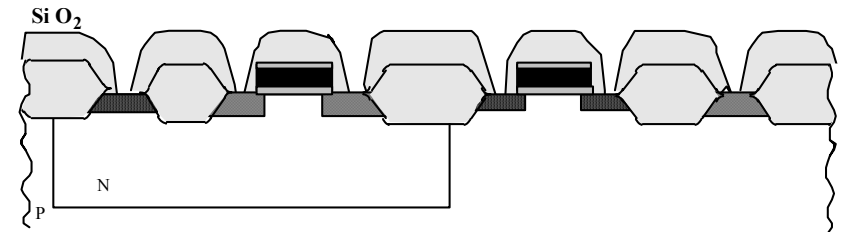
35. **RECC** Recocido para redistribuir las impurezas implantadas ensanchando las zonas N⁺ y P⁺.

duración ≈ 30 minutos

concentraciones resultantes:

difusión N $\approx 10^{20} \text{ at/cm}^3$ $e_{N+} \approx 0,2 \mu\text{m}$

difusión P $\approx 10^{19} \text{ at/cm}^3$ $e_{P+} \approx 0,5 \mu\text{m}$

M. Recubrimiento por óxido y apertura de contactos

36. **OXdep** Deposito de una capa de óxido grueso de recubrimiento.

$e \approx 0,7 \mu\text{m}$

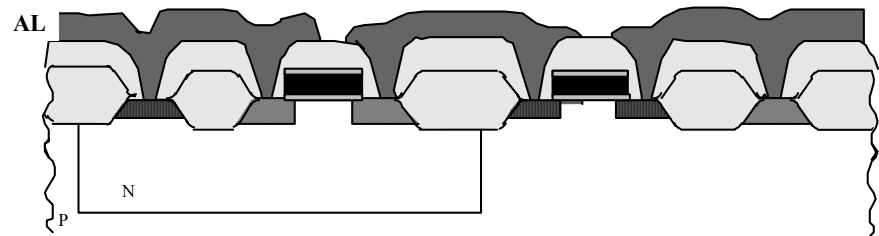
37. **LITO** Definición por litografía de los contactos. **M5 = contactos**

en ellos se elimina el óxido superficial para permitir que el metal conecte con la capa inferior: difusiones, sustratos o polisilicio; se utiliza resina fotosensible positiva.

38. **GRAB** Eliminación del óxido en los contactos.

39. **LAVO** Eliminación de la resina residual.

N. Metalización y recorte de las conexiones sobre el metal



40. **Aldep** Depósito de metal (Aluminio) sobre toda la superficie.
 $e \approx 0,5 \mu\text{m}$
el metal penetra asimismo por los agujeros de contactos
definidos en la etapa anterior.

41. **LITO** Delimitación por litografía de las conexiones metálicas.

M6 = metal

43. **GRAB** Eliminación del metal exterior a las conexiones.
44. **LAVO** Eliminación de la resina residual.

O. Pasivación y delimitación de PAD's

45. **OXdep** Depósito de una gruesa capa de óxido de pasivación.
esta capa de óxido protegerá a todo el circuito integrado. $e \approx 2 \mu\text{m}$

46. **LITO** Demarcación de los puntos de conexión hacia el exterior
M7 = pasivación
se delimitan los puntos de soldadura (PAD's) de los hilos metálicos
que conectarán con las patillas (*pines*) del encapsulado.
47. **GRAB** Eliminación del óxido de pasivación en dichos espacios.
48. **LAVO** Eliminación de la resina residual.

- En resumen:** 7 máscaras: **M1** pozos (sustrato N para transistores P)
M2 zonas activas (transistores y contactos sustratos)
M3 líneas de polisilicio
M4 zonas N⁺ **M4** zonas P⁺
M5 contactos
M6 líneas de metal
M7 puntos de conexión (PAD's)
- 48 operaciones: 8 oxidaciones (1 de altísima precisión: t_{ox})
3 depósitos: nitruro, polisilicio, metal
9 litografías
7 grabados
4 implantaciones iónicas
1 difusión
2 recocidos
14 lavados o limpiezas.

En el caso habitual de utilizar un segundo metal son necesarias dos máscaras adicionales:

M6 bis vías: contactos del segundo metal con el primero

M6 tris líneas de metal referidas al segundo de ellos

y las correspondientes operaciones de depósito del óxido de separación intermedio, litografía de las vías, depósito del segundo metal, litografía de sus líneas de conexión y grabado de las mismas.

Para circuitos integrados digitales complejos se utilizan hasta 6 capas de metal superpuestas. Asimismo, en integrados analógicos se emplean varias capas de metal y suelen utilizarse también dos polisilicios; para el segundo polisilicio es necesaria una máscara adicional:

M3 bis líneas del segundo polisilicio

y las subsiguientes operaciones de depósito del óxido de separación intermedio, depósito del segundo polisilicio, litografía de las líneas del mismo y grabado de ellas.

T2.5. Parámetros físicos de las regiones CMOS

Valores típicos de los diversos parámetros

Los valores de los diversos parámetros dependen del proceso tecnológico concreto; estos datos numéricos se detallan a efectos de poder apreciar su orden de magnitud y comparar la magnitud relativa entre ellos.

SEMICONDUCTORES

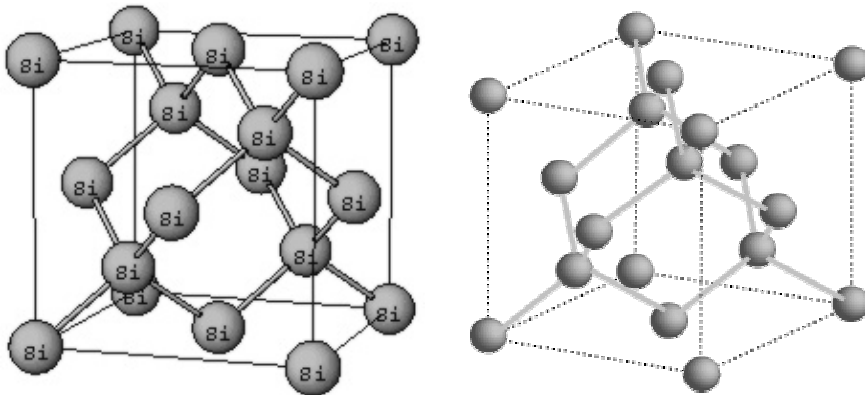
Oblea inicial de silicio dopado tipo P

diámetro: 90 - 125 mm

grosor: 0,5 mm = 500 μm

Silicio Si

El silicio cristaliza en red cúbica centrada en las caras



lado del cubo: $5,43 \text{ \AA} = 543 \text{ pm}$

distancia entre átomos: $2,35 \text{ \AA} = 235 \text{ pm}$

concentración: $5 \cdot 10^{22} \text{ áts/cm}^3 \sim 10^{22} \text{ cm}^{-3}$

orientación cristalográfica **1 0 0** (sección por las caras del cubo).

separación energética Banda de Conducción - Banda de Valencia

$E_G = 1,124 \text{ eV}$ $1 \text{ eV} = 16 \cdot 10^{-20} \text{ julios}$

Una energía normal a escala humana (por ejemplo, la potencial gravitatoria de un kilogramo a una altura de 1 metro o la energía calorífica de un litro de agua al aumentar su temperatura en 2 grados) es cien mil trillones de veces superior al electrón-voltio.

Al recombinarse un electrón con un hueco se produce un fotón, cuya energía será la propia de separación entre la banda de conducción y la de valencia; la mecánica cuántica permite calcular la longitud de onda y la frecuencia del fotón en relación con su energía:

$$E = h \cdot \nu = h \cdot \lambda / c \quad \begin{array}{l} h = 66,25 \cdot 10^{-35} \text{ julios.s} \\ \nu = \lambda / c \quad c = 300 \cdot 10^6 \text{ m/s} \end{array}$$

$$\lambda \sim 1,1 \mu\text{m} \quad \nu \sim 270 \cdot 10^{12} \text{ Hz} \Rightarrow \text{infrarrojo}$$

Esta frecuencia se encuentra fuera (por debajo) del espectro visible, el cual va de los 400 a los 800 THz (1 THz = 10^{12} Hz); los 270 THz corresponden a un infrarrojo muy bajo. [Los diodos LED utilizan esta recombinación electrón-hueco para producir radiación luminosa aprovechando la mayor separación energética entre las bandas de conducción y valencia en el arseniuro de galio y fósforo, As Ga P; de esta forma se consiguen diodos LED rojos, 1,8 eV 450 THz, naranjas, 2 eV 500 THz, y amarillos, 2,1 eV 520 THz.]

número de portadores

pares electrón/hueco (en BC/BV): $n_i^2 \sim 0,2 \cdot 10^{20} \cdot e^{\frac{-q \cdot E_G}{2 \cdot K \cdot T}}$

$$K = 8,62 \cdot 10^{-5} \text{ eV}$$

a "temperatura ambiente" $t = 27^\circ\text{C} = 300^\circ\text{K}$

$$K \cdot T / q = 0,0026 \text{ voltios} \quad n_i \sim 10^{10} \text{ cm}^{-3}$$

Dopado

**** tipo N** átomos de fósforo P $E_G = 0,045 \text{ eV}$ (25 veces < Si)

concentración N_D (nº de átomos donadores / cm^3)

portadores mayoritarios: electrones $n = N_D$

**** tipo P** átomos de boro B $E_G = 0,045 \text{ eV}$ (25 veces < Si)

concentración N_A (nº de átomos aceptores / cm^3)

portadores mayoritarios: huecos $p = N_A$

substrato P (oblea)

dopado $N_A \sim 10^{15} \text{ áts.B/cm}^3$

un átomo de impureza por cada 50 billones de átomos de silicio

en cambio, 100.000 huecos por cada par electrón-hueco del silicio

►► **substrato N (pozo)**

dopado $N_D \sim 10^{16} \text{ áts.P/cm}^3$

el pozo se forma sobre el substrato P (oblea) añadiendo átomos de fósforo para invertir el tipo de semiconductor: el dopado N del pozo tiene que ser, al menos, un orden de magnitud superior (10 veces mayor) que el dopado del substrato P para cambiarlo a N

polisilicio y difusión N

dopado $N_D \sim 10^{20} \text{ áts.P/cm}^3$

un átomo de impureza por cada 500 átomos de silicio

en cambio, 10.000 millones de electrones por cada par electrón-hueco del silicio

100.000 electrones por cada hueco del substrato P

difusión P

dopado $N_D \sim 10^{19} \text{ áts.P/cm}^3$

la difusión P penetra también en el polisilicio en la zona de puerta de los transistores PMOS: es necesario asegurar que el dopado N del polisilicio no se modifica apreciablemente; para ello la difusión P ha de ser un orden de magnitud inferior al dopado del polisilicio

CONDUCTIVIDAD**conducción intrínseca del silicio**

pares electrón/hueco a "temperatura ambiente" $t = 27^\circ\text{C} = 300^\circ\text{K}$ $n_i \sim 10^{10} \text{ cm}^{-3}$

movilidad eléctrica $\mu_e \sim 1360 \text{ cm}^2/\text{V.s}$

$\mu_h \sim 460 \text{ cm}^2/\text{V.s}$ $\mu_e / \mu_h \sim 3$

la movilidad de los huecos es considerablemente inferior a la de los electrones libres, ya que son electrones ligados (a enlaces de valencia) los que han de desplazarse para que un hueco se mueva

conductividad eléctrica $\sigma = q \cdot \mu_e \cdot n + q \cdot \mu_h \cdot p$

n = concentración de electrones en Banda de Conducción

p = concentración de huecos en Banda de Valencia

q = carga del electrón = $16 \cdot 10^{-20}$ culombios

$\sigma = q \cdot (\mu_e + \mu_h) \cdot n_i \sim 3 \cdot 10^{-6} (\Omega \cdot \text{cm})^{-1}$

resistividad eléctrica $\rho = 1 / \sigma \sim 300 \text{ K}\Omega \cdot \text{cm}$

conducción del substrato P

$N_A \sim 10^{15} \text{ áts.B/cm}^3$

movilidad eléctrica $\mu_h \sim 450 \text{ cm}^2/\text{V.s}$

es próxima a la del silicio puro ya que el dopado es bajo

conductividad eléctrica $\sigma = q \cdot \mu_h \cdot N_A \sim 70 \cdot 10^{-3} (\Omega \cdot \text{cm})^{-1}$

resistividad eléctrica $\rho = 1 / \sigma \sim 15 \Omega \cdot \text{cm}$

el substrato P conduce unas 20.000 veces mejor que el silicio puro

conducción del pozo N

$N_D \sim 10^{16} \text{ áts.P/cm}^3$

movilidad eléctrica $\mu_e \sim 1200 \text{ cm}^2/\text{V.s}$

conductividad eléctrica $\sigma = q \cdot \mu_h \cdot N_D \sim 2 (\Omega \cdot \text{cm})^{-1}$

resistividad eléctrica $\rho = 1 / \sigma \sim 0,5 \Omega \cdot \text{cm}$

el pozo N conduce unas 30 veces mejor que el substrato P, ya que se encuentra 10 veces más dopado y conduce mediante electrones (cuya movilidad es 3 veces superior)

conducción del polisilicio y de la difusión N

$N_D \sim 10^{20} \text{ áts.P/cm}^3$

movilidad eléctrica $\mu_e \sim 100 \text{ cm}^2/\text{V.s}$

conductividad eléctrica $\sigma = q \cdot \mu_e \cdot N_D \sim 1600 (\Omega \cdot \text{cm})^{-1}$

resistividad eléctrica $\rho = 1 / \sigma \sim 0,6 \text{ m}\Omega \cdot \text{cm}$

polisilicio y difusión N conducen unas 1.000 veces mejor que el pozo N; en realidad tienen 10.000 veces más portadores pero la movilidad es 12 veces más pequeña, debido al gran número de impurezas en la red cristalina, las cuales atraen a los portadores mayoritarios, frenan su avance y aumentan el número de choques con la red cristalina

conducción de la difusión P

$N_D \sim 10^{19} \text{ áts.P/cm}^3$

movilidad eléctrica $\mu_h \sim 80 \text{ cm}^2/\text{V.s}$

conductividad eléctrica $\sigma = q \cdot \mu_h \cdot N_A \sim 130 (\Omega \cdot \text{cm})^{-1}$

resistividad eléctrica $\rho = 1 / \sigma \sim 8 \text{ m}\Omega \cdot \text{cm}$

la difusión P conduce unas 10 veces peor que la difusión N ya que la concentración de impurezas es menor

conducción en el canal

Canal N: movilidad superficial $\mu_e \sim 600 \text{ cm}^2/\text{V.s}$

Canal P: movilidad superficial $\mu_h \sim 200 \text{ cm}^2/\text{V.s}$

$$\mu_e / \mu_h \sim 3$$

conducción en el metal

Aluminio Al

conductividad eléctrica $\sigma \sim 375 \cdot 10^3 (\Omega.\text{cm})^{-1}$

resistividad eléctrica $\rho \sim 2,65 \mu\Omega.\text{cm}$

el aluminio conduce unas 250 veces mejor que el polisilicio

►► resistencia de los contactos

resistencia metal-difusión N o P $\sim 10 \Omega$

resistencia metal-polisilicio $\sim 5 \Omega$

las resistencias de contacto con las difusiones son mayores que con el polisilicio ya que aquéllas se encuentran a mayor distancia (más abajo) de la capa de metal

ESPESORES

Obviamente dependen mucho de la tecnología; se incluyen estos datos como referencia genérica.

oblea inicial de silicio dopado tipo P

diámetro: 90 - 125 mm

grosor: 0,5 mm = 500 μm

POZO N

espesor

$e_{\text{well}} \sim 5 \mu\text{m}$

POLISILICIO

espesor

$e_{\text{poli}} \sim 0,2 \mu\text{m}$

DIFUSIÓN N

profundidad

$e_N \sim 0,2 \mu\text{m}$

DIFUSIÓN P

profundidad

$e_P \sim 0,5 \mu\text{m}$

para compensar su menor conductividad

ÓXIDO DE PUERTA

espesor

$t_{\text{ox}} \sim 0,05 \mu\text{m} = 500 \text{ \AA}$

la tensión máxima que puede ser aplicada a la puerta sin riesgo de perforación del óxido está en relación con el espesor del mismo:

campo eléctrico máximo que soporta

$E_{\text{máx}} \sim 600 \text{ V}/\mu\text{m}$

voltaje de ruptura para $t_{\text{ox}} \sim 50 \text{ nm}$:

$V_{\text{máx}} = E_{\text{máx}} \cdot t_{\text{ox}} \sim 30 \text{ V}$

la reducción de dimensiones de los transistores tiende también a reducir el espesor del óxido de puerta para aumentar su transconductancia K_p y, con ella, la intensidad disponible, pero ha de tenerse en cuenta la limitación que óxidos de puerta muy finos representan en relación con su tensión de ruptura:

voltaje de ruptura para $t_{\text{ox}} \sim 10 \text{ nm}$:

$V_{\text{máx}} = E_{\text{máx}} \cdot t_{\text{ox}} \sim 6 \text{ V}$

solamente con tensiones de alimentación inferiores a 5 V puede reducirse el espesor del óxido de puerta al orden de 0,01 μm

ÓXIDO DE CAMPO

espesor

$e_{\text{ox,campo}} \sim 1 \mu\text{m}$

ÓXIDO DE AISLAMIENTO

espesor

$e_{\text{ox,aisl}} \sim 0,7 \mu\text{m}$

ÓXIDO DE PASIVACIÓN

capa exterior de óxido de silicio

espesor $\sim 2 \mu\text{m}$

METAL

espesor

$e_{\text{Al}} \sim 0,5 \mu\text{m}$

CAPACIDAD

Las capacidades entre las diversas capas dependen fuertemente (proporción inversa) del espesor del óxido de silicio intermedio, los valores siguientes están en relación con los espesores de óxido indicados en la página anterior

►► coeficiente dieléctrico del silicio

coeficiente dieléctrico del vacío: $\epsilon_0 = 8,85 \text{ pF/m}$

$\epsilon_{\text{Si}} = 117 \epsilon_0 \approx 100 \text{ pF/m} = 0,1 \text{ fF}/\mu\text{m}$

$1 \text{ fF} = 10^{-15} \text{ Faradios}$

►► coeficiente dieléctrico del óxido de silicio

$\epsilon_{\text{ox}} = 3,97 \epsilon_0 \approx 35 \text{ pF/m} = 0,035 \text{ fF}/\mu\text{m}$

►► **capacidad del óxido de puerta SiO₂**

espesor $t_{ox} \sim 0,05 \mu m = 500 \text{ \AA}$

coeficiente dieléctrico óxido de silicio $\epsilon_{ox} = 0,035 \text{ fF}/\mu m$

$$C_{ox} = \epsilon_{ox} \cdot S / t_{ox} \sim 0,7 \text{ fF}/\mu m^2$$

□ □ **capacidad del óxido de campo**

espesor $e_{ox,campo} \sim 1 \mu m$

efecto capacitivo (poli-substrato) $C = \epsilon_{SiO2} \cdot S / e_{ox,campo} \sim 0,04 \text{ fF}/\mu m^2$

□ □ **capacidad del óxido de aislamiento o recubrimiento**

espesor $e_{ox,aisl} \sim 0,7 \mu m$

efecto capacitivo (metal-polisilicio) $C = \epsilon_{SiO2} \cdot S / e_{ox,aisl} \sim 0,06 \text{ fF}/\mu m^2$

efecto capacitivo metal-substrato $C = \epsilon_{SiO2} \cdot S / (e_{ox,aisl} + e_{ox,campo}) \sim 0,02 \text{ fF}/\mu m^2$

►► **efecto capacitivo difusión N - substrato (unión NP en polarización inversa)**

- por área $0,1 \text{ fF}/\mu m^2$

- por perímetro $0,15 \text{ fF}/\mu m$

►► **efecto capacitivo difusión P - substrato (unión PN en polarización inversa)**

- por área $0,6 \text{ fF}/\mu m^2$

- por perímetro $0,4 \text{ fF}/\mu m$

MULTICAPA

2 Metales

el primero de ellos se hace más delgado: espesor $e_{Al(1)} \sim 0,2 \mu m$

resistencia contacto metal-metal $\sim 0,7 \mu \Omega$

separación entre ambos metales $\sim 0,7 \mu m$

2 Polisilicios

separación entre ambos $\sim 0,4 \mu m$

capacidad entre ambos $C = \epsilon_{SiO2} \cdot S / (e_{ox,aisl}) \sim 0,1 \text{ fF}/\mu m^2$

Concentración relativa de átomos y portadores

1 μm (una micra) es una unidad de medida muy adecuada en Microelectrónica. El siguiente cuadro expresa, en número por micra cúbica, las concentraciones de átomos y de portadores y permite comparar unas respecto a otras.

Transistor NMOS	número / μm^3	
átomos de Si red cristalina de Si puro	10^{10} = 10.000 millones	
pares electrón-hueco a temperatura ambiente	1 cada 100 μm^3	1 par e-h por cada 10^{12} átomos de Si
Substrato P	10^3 = 1.000 átomos	1 átomo de Boro por cada 10 millones de áts. de Si
portadores de carga en Substrato P	10^3	100.000 veces mayor que en el Silicio puro
Difusión N	10^8 = 100 millones	1 átomo de Fósforo cada 100 átomos de Si
portadores de carga en Difusión N	10^8	100.000 veces mayor que en el Substrato
Polisilicio	10^8 = 100 millones	dopado análogo al de Difusión N
Transistor PMOS		
Pozo N	10^4 = 10.000 átomos	10 veces más que Substrato P para cambiar su polaridad a N
Difusión P	10^7 = 10 millones	10 veces menor que Difusión N para no cambiar el dopado del Polisilicio

Conductividad relativa

Consideremos un cable de 1 m de largo y de 1 mm² de sección construido con cada uno de los materiales siguientes; el cuadro que viene a continuación expresa su resistencia para los diversos materiales semiconductores que entran a formar parte de la integración CMOS:

$$R = \rho \cdot \frac{\text{Longitud}}{\text{Sección}}$$

Silicio puro	= 3.000 MΩ	
Substrato P	= 150 KΩ.	conduce 20.000 veces mejor que el Silicio puro.
Substrato N (Pozo)	= 5 KΩ	conduce 30 veces mejor que el Substrato.
Difusión N (Transistor NMOS)	= 6 Ω	conduce 800 veces mejor que el pozo N.
Difusión P (Transistor PMOS)	= 80 Ω	conduce 12 veces peor que la Difusión N.
Polisilicio	= 6 Ω.	conduce igual que la Difusión N.
Aluminio	= 26 mΩ	conduce 230 veces mejor que el Polisilicio.

Resistencia por cuadro

La resistencia de un cuadrado es independiente del lado del mismo, ya que se compensa longitud con anchura: $R = \rho \cdot \frac{\text{longitud}}{\text{sección}} = \rho \cdot \frac{L}{L \cdot \text{espesor}} = \frac{\rho}{\text{espesor}}$.

En los circuitos integrados, la resistencia por cuadro es un parámetro muy útil que permite calcular la resistencia que presenta cualquier línea de conexión.

Línea de:	Resistencia por cuadro
Pozo	$R_{\square} = 1 \text{ K}\Omega$
Difusión N	$R_{\square} = 30 \Omega$
Difusión P	$R_{\square} = 160 \Omega$
Polisilicio	$R_{\square} = 30 \Omega$
Aluminio	$R_{\square} = 50 \text{ m}\Omega$

Capacidad por mm²

Para construir un condensador en un circuito integrado analógico, puede aprovecharse la capacidad que presentan las diversas parejas de elementos conductores; este cuadro muestra el orden de magnitud de la capacidad que presenta un condensador

de 1 mm² de superficie: $C = \epsilon \cdot \frac{\text{Superficie}}{\text{espesor}}$

(los valores se refieren a los espesores de las capas de óxido indicados anteriormente)

Placas del condensador	capacidad por mm ²
Polisilicio - Substrato (zona de puerta MOS)	= 700 pF
Polisilicio - Substrato (sobre óxido de campo)	= 35 pF
Metal - Polisilicio	= 50 pF
Metal - Substrato	= 20 pF
Metal2 – Metal1	= 50 pF
Polisilicio - Polisilicio	= 90 pF

T3 EL RUIDO EN LOS SISTEMAS DIGITALES

T3.1. El problema del ruido

T3.2. Consideraciones previas al estudio de interferencias

T3.3. Elementos para reducir el efecto del ruido

T3.4. Mecanismos de producción/transmisión de interferencias

T3.5. Reglas básicas para el diseño

El ruido es un compañero inevitable y molesto de los sistemas digitales, con el que tienen que convivir. La realidad no es nada «limpia»: los circuitos electrónicos han de sobrevivir entre la propia «basura electromagnética» que ellos generan y la basura de los demás, la que se genera en su entorno.

Los circuitos electrónicos trabajan con señales eléctricas, que son, por sí mismas, señales electromagnéticas. Por ello, los circuitos son sensibles a cualquier señal electromagnética y, al recibir señales externas, son perturbados por ellas: se ven afectados por interferencias que pueden alterar su buen funcionamiento, causando errores. Además, los propios circuitos producen señales electromagnéticas parásitas: son generadores de interferencias que les afectan a ellos mismos y que afectan también al resto de los circuitos de su entorno.

Este capítulo pretende tratar en detalle las causas y efectos del «ruido electromagnético» y las formas de actuar frente al mismo. Como punto de partida se presentan los fenómenos físicos que pueden dar lugar a perturbaciones para llegar a identificar los mecanismos concretos de producción y transmisión de interferencias sobre los circuitos digitales.

Desde el conocimiento de sus causas y modos de operar, la defensa frente a las perturbaciones electromagnéticas pasa por un adecuado diseño de las placas circuitales (placas de circuito impreso o similares); esta adecuación del diseño se refiere, por un lado, a la disposición mecánica de componentes y de sus conexiones y, de otro, a la utilización de determinados componentes electrónicos auxiliares que se oponen al ruido.

Como norma general, es preciso minimizar la longitud de las conexiones, el área de los bucles y las impedancias compartidas: la disponibilidad de «un plano de masa», de una capa completa del circuito impreso dedicada a la tensión de referencia, facilita plenamente estos objetivos en lo que se refiere a las líneas de retorno, las cuales tienen una importancia primordial respecto al ruido.

A la vez, filtros separadores como los condensadores de desacople, los optoacopladores o los filtros de red reducen en gran medida la propagación de interferencias y el apantallamiento, en forma de jaula de Faraday, sirve de protección frente a la incidencia de campos electromagnéticos externos.

*Una útil referencia bibliográfica: como complemento a este capítulo, un libro que recoge una muy amplia diversidad de aspectos relativos al ruido electromagnético es el texto de J. Balcells, F. Daura, R. Esparza y R. Pallás, *Interferencias electromagnéticas en sistemas electrónicos*, publicado por Mundo Electrónico. Marcombo Boixareu Editores. Barcelona. 1992.*

T3.1. El problema del ruido

El problema del «ruido» aparece como consecuencia de que el comportamiento de los circuitos electrónicos no se reduce a los términos de «teoría de circuitos», no queda confinado en el simple análisis circuital resultante del diseño; sino que la realidad física actúa en la forma de «campos electromagnéticos», con una doble incidencia: las ondas y los campos externos presentes en el entorno actúan como interferencias sobre el circuito y, además, su propio funcionamiento produce perturbaciones que también le afectan.

Los circuitos electrónicos procesan energía electromagnética para poder llevar a cabo la función para la que han sido diseñados. El diseñador, habitualmente, maneja el problema teniendo en cuenta tensiones y corrientes en el circuito: *análisis de circuitos*.

Pero las leyes físicas ignoran las fronteras conceptuales que el diseñador impone a su circuito y cualquier energía de tipo electromagnético presente en el entorno interfiere sobre el mismo; además, por causa de esas mismas leyes físicas, los componentes no se comportan de forma ideal (no se limitan a ser los elementos de circuito en los que el diseñador piensa) sino que presentan efectos parásitos. Es preciso un punto de vista más amplio que el propio análisis circuital para entender y controlar los fenómenos que provocan la aparición de interferencias: *presencia de campos electromagnéticos*.

De esta forma los sistemas digitales se encuentran sometidos a interferencias que les llegan desde el medio ambiental en que se encuentran (motores, relés, transformadores, radiofrecuencias, emisiones de los cables de red, deformaciones y perturbaciones de la propia red eléctrica, etc,...) y a perturbaciones debidas a los campos electromagnéticos que ellos mismos producen (variaciones de consumo que generan parásitos sobre la alimentación, oscilaciones propias de los circuitos, acoplamiento entre pistas del circuito impreso, radiación de las pistas y cables de interconexión, etc,...).

Todas estas señales son indeseables para un sistema electrónico por cuanto pueden afectar a su correcto funcionamiento; en el caso de un sistema digital las interferencias pueden modificar puntualmente los valores booleanos por desplazamiento de las tensiones en los nudos del circuito.

Ruido es toda perturbación electromagnética que afecta a un circuito digital, toda señal parásita no propia del comportamiento booleano del circuito y que, por tanto, puede producir errores al modificar los valores booleanos correctos.

El ruido puede proceder del exterior o puede ser producido en el propio circuito y puede ser **conducido**, si se propaga a través de conductores y componentes del mismo circuito o de sus líneas de alimentación, de entrada o de salida, o **radiado**, si se acopla a través de campos eléctricos, magnéticos o electromagnéticos. El ruido conducido puede ser razonado en términos de tensión mientras que el radiado requiere términos de potencia (energía perturbativa por unidad de tiempo).

En circuitos combinacionales un ruido momentáneo puede causar un error transitorio en el vector de salida, pero no repercutirá posteriormente; en los secuenciales, el efecto es más duradero y por tanto más peligroso: puede dar lugar a una modificación del estado que se propagará como error hacia el futuro, es decir, provocará un error permanente.

Fuentes de ruido electromagnético

En general, debe tenerse en cuenta que cualquier equipo o circuito eléctrico o electrónico es una fuente potencial de interferencias electromagnéticas **EMI**. Parte de las señales parásitas son producidas por los propios circuitos y afectan a ellos mismos (EMI intraequipo) y también al resto de los circuitos de su entorno (EMI interequipos).

A continuación, se enumeran algunas de las principales fuentes de interferencias; se citan tanto elementos ruidosos como situaciones que llevan aparejadas la aparición de problemas de EMI.

Interferencias generadas por el propio circuito:

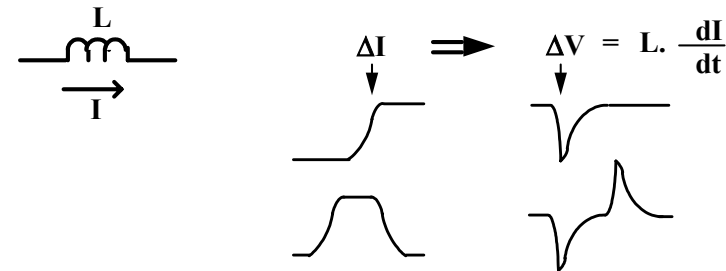
- ♦ conducidas:
 - variaciones de intensidad (dI/dt)
 - por carga y descarga de capacidades
 - por conmutaciones
 - acoplo por impedancia compartida por varias etapas
 - resonancias y oscilaciones asociadas a ellas (LC)
 - acoplo capacitivo entre pistas
- ♦ radiadas:
 - osciladores, señal de sincronismo,...
 - emisión de altas frecuencias (acoplamiento electromagnét.)
 - acoplo inductivo entre bucles (espiras)
 - componentes magnéticos (transformadores, bobinas,...)
 - contactos mecánicos (contactores, relés,...)

Interferencias procedentes del entorno:

- ♦ radiadas:
 - motores y máquinas eléctricas
 - equipos electrónicos que operan a alta frecuencia
 - contactores y relés
 - bobinas y transformadores
 - cables, fluorescentes,...
 - emisoras de radiofrecuencia
 - transistores debidos a fenómenos atmosféricos
- ♦ conducidas:
 - perturbaciones de la red
 - por conexión o desconexión de cargas
 - por variaciones bruscas de consumo
 - por conmutación (troceadores) sobre la red
 - por transferencia de altas frecuencias
 - perturbaciones recogidas por las líneas de entrada y salida

T3.2. Consideraciones previas al estudio de interferencias

- 1 Las autoinducciones responden con transitorios de tensión a las variaciones de intensidad: todo elemento de naturaleza inductiva responde, ante los cambios de intensidad, con alteraciones de tensión.

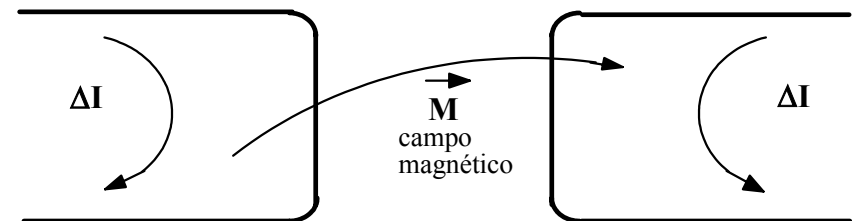


en la práctica, serán oscilaciones amortiguadas, habida cuenta de los efectos capacitivos presentes en el circuito

- los picos (*glitches*) de tensión dependen, no sólo de la amplitud de la variación de intensidad, sino también de la velocidad de dicha variación: de la pendiente dI/dt (de forma que ΔI reducidas pero muy rápidas pueden producir ΔV apreciables);

- todo cable o línea conductora presenta un efecto autoinductivo (una línea de 10 cm, cuya autoinducción será del orden de 0,1 μH , responde a un aumento de intensidad de 10 mA en 1 ns con un transitorio de tensión del orden de 1 V).

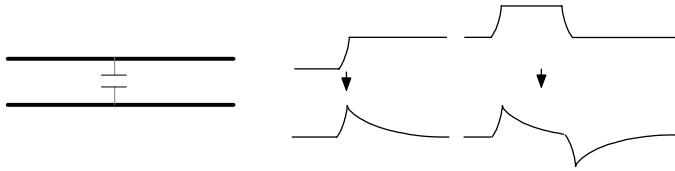
- 2 Los bucles de intensidad (*espiras*) generan campos magnéticos y cuando tales campos son variables (debidos a variaciones de intensidad) producen corrientes inducidas sobre otros bucles (*espiras*) próximos. El acoplo inductivo es proporcional al área de los bucles.



De igual forma, todo bucle conductor recibe el efecto inductivo de los campos magnéticos que le alcanzan y tal efecto es tanto mayor cuanto lo es el área del bucle.

- por ello, es de suma importancia conocer «por dónde circula la corriente»; en ello reside uno de los «secretos» para controlar y reducir las interferencias.

- 3 Dos conductores próximos (por ejemplo, dos conductores de un mismo cable plano o dos pistas que circulan paralelas por una placa) presentan un efecto capacitivo entre ellos: una variación de tensión en uno de ellos es transmitida parcialmente como transitorio al otro conductor.



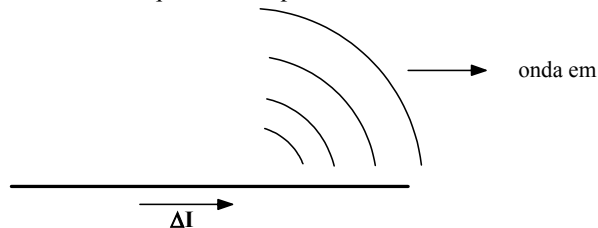
- 4 La presencia de autoinducciones y capacidades parásitas determina la posibilidad de oscilaciones, generalmente amortiguadas (debido a las componentes resistivas presentes también en el circuito): toda variación brusca de tensión o de intensidad puede provocar la activación del circuito oscilante LC y la generación de un transitorio en forma de senoide.



Frecuencia de oscilación:

$$f = 1 / 2\pi \sqrt{LC}$$

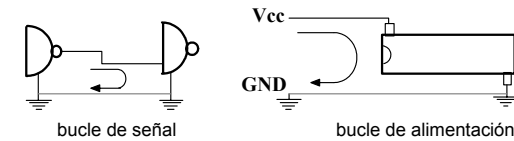
- 5 Cuando una señal variable se propaga por un conductor, parte de la energía que transporta se pierde en forma de radiación electromagnética hacia el entorno; la efectividad de esta transmisión aumenta fuertemente con la frecuencia y depende mucho de la geometría de emisor y receptor. Dicho en otras palabras, cualquier circuito va a comportarse como una antena más o menos efectiva en función de su geometría y de las señales que circulan por el mismo.



Recíprocamente, cualquier línea conductora y, en particular, cualquier bucle de corriente actúa como antena receptora de las ondas electromagnéticas presentes en su entorno y el efecto de tales interferencias es proporcional a la longitud de la línea o, en su caso, al área del bucle.

La radiación electromagnética es la forma en que se transmiten a distancia los efectos de los campos eléctricos y magnéticos variables.

- 6 En un circuito, toda señal eléctrica necesita una línea de retorno; es decir, forma un bucle conformado por dos conductores: el que transmite la señal y el que sirve de referencia, que actúa como línea de retorno.



La representación clásica de la referencia (tensión 0 V: *masa*) como un simple símbolo aislado (incluso a veces la ausencia de tal referencia explícita) tiende a hacernos olvidar que forma parte del circuito: que no es un simple «sumidero omnipresente» sino una línea de retorno que cierra los bucles:

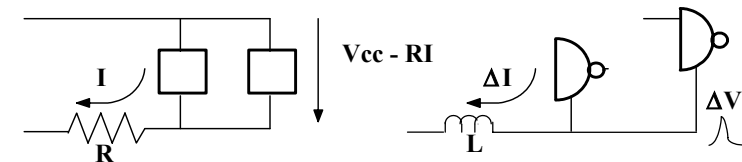
- la tensión de alimentación de un circuito forma bucles de alimentación para cada una de las etapas del mismo
- cada señal que se transmite de una etapa a otra forma un bucle de señal
- la existencia de varias «tomas a tierra» (conexiones de la línea de *masa* del circuito a tierra) dan lugar a «bucles de masa», que también pueden ser receptores de interferencias.

→ Los efectos de emisión/captación magnética/electromagnética de los bucles serán tanto menores cuanto más reducida sea su área.

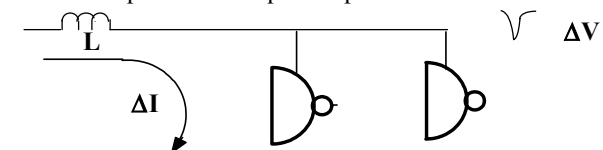
- 7 Toda línea conductora presenta una impedancia (R, L) y, cuando por dicha línea viajan dos señales, es compartida por ambas (IMPEDANCIA COMÚN) y las variaciones de tensión producidas por una de ellas afectan también a la otra.

Téngase en cuenta que, en un circuito real, dos puntos unidos por un conductor no tienen el mismo potencial, como consecuencia de la impedancia existente entre ambos.

La línea de retorno suele ser utilizada, a la vez, por varias alimentaciones y/o señales, lo cual supone la existencia de una impedancia compartida por ellas, de modo que las caídas o variaciones de tensión generadas sobre dicha impedancia afectan a las diversas etapas conectadas a la línea de retorno



De igual forma una línea de alimentación VCC suele estar compartida por múltiples etapas, presentando una impedancia compartida por ellas.



T3.3. Elementos para reducir el efecto del ruido

Conexiones cortas y bucles de área mínima; plano de masa

La primera norma frente al ruido es minimizar la longitud de los cables o líneas de conexión y el área de los bucles que conforman; la reducción de la longitud de las conexiones (en cuanto a antenas) y del área de los bucles (en cuanto a espiras) afecta tanto a la generación como a la captación de ruido electromagnético. La geometría final del circuito electrónico tiene una incidencia trascendental respecto al ruido: longitudes cortas y áreas reducidas emiten y captan menos interferencias.

De ahí la extraordinaria importancia que presenta el diseño de la placa de circuito impreso en relación con el ruido. Los circuitos impresos no son un mero soporte mecánico y un simple conexionado eléctrico, sino que determinan la topología del circuito y, con ella, los acoplamientos de las perturbaciones: un buen diseño geométrico del circuito impreso es fundamental para prevenirlas.

Particular atención merece la distribución de la tensión de alimentación y la de la señal de reloj, por cuanto que tales líneas suelen ir a múltiples integrados y, en general, serán las que mayor trayecto recorren.

La «masa» o referencia de tensión (0 V) actúa como camino de retorno tanto para la alimentación como para las señales y precisa de un tratamiento especial ya que las líneas de retorno cierran los bucles, siendo determinantes en la conformación de su área; además, suelen ser líneas compartidas por varias señales con los consiguientes efectos de «impedancia común» a ellas.

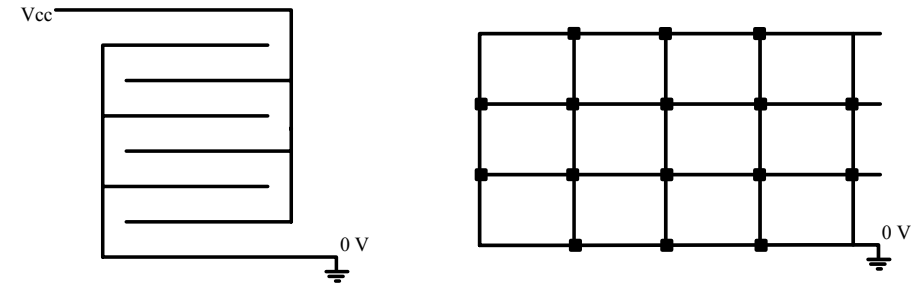
Siempre que sea posible conviene dedicar toda una capa de circuito impreso a la tensión de referencia, es decir, un «plano de masa» que permitirá que los «caminos de retorno» sean lo más directos posibles; de esta forma, el retorno se producirá por el camino de menor impedancia:

- lo cual minimiza la impedancia común a varias señales;
- en el caso de señales de baja frecuencia el camino de mínima impedancia será el de menor resistencia y, por tanto, menor longitud (una línea recta directa);
- mientras que para señales de alta frecuencia dicho camino será el de menor autoinducción que corresponde a mínima área de bucle (camino de retorno siguiendo en paralelo y lo más próximo posible al camino de ida de la señal).

Si no se puede disponer de una capa entera para la tensión de referencia, conviene expandir la masa todo lo posible, aprovechando los huecos libres de pistas del circuito impreso para generar amplios islotes de masa. En general, a falta de un plano de masa, un circuito impreso que presenta amplias zonas «sin cobre» es una mala solución por desaprovechar la oportunidad de utilizar como «islotes de masa» dichos espacios

En todo caso, debe cuidarse que las líneas de masa sean lo más directas y cortas posibles y, también, que los bucles que se forman a través de dichas líneas (como caminos de retorno) sean de la menor área posible:

- distribuciones en forma de «peine» facilitan la reducción de longitud
- y la conexión de las diversas pistas de masa en forma de retícula posibilitan caminos de retorno de área reducida.

Distribución de las líneas de alimentación *en peine*Distribución de las líneas de masa *en retícula*

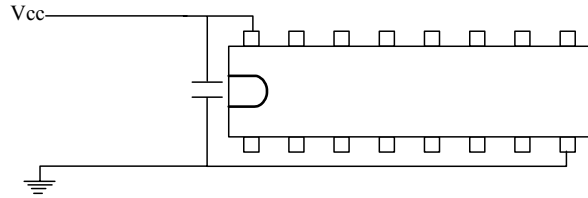
La distribución de la tensión de alimentación «en peine» minimiza la longitud de las líneas que van a múltiples circuitos integrados. Además, dado que la tensión de referencia conforma los caminos de retorno de las señales, es conveniente unir las líneas paralelas de «masa» para formar una retícula (*mallá de masa*) que permita a los caminos de retorno acercarse lo más posible a los caminos de ida de las señales; se esta forma se consiguen áreas de bucle reducidas, en relación con las señales de alta frecuencia.

Asimismo, es conveniente separar las líneas de alimentación de las partes digitales, analógicas y de potencia: para evitar los efectos de impedancia compartida es bueno que, caso de existir en la misma placa de circuito impreso etapas analógicas o de potencia, sus líneas de alimentación sean diferentes de la alimentación de la parte digital; ello evita transferir caídas resistivas o variaciones inductivas en las líneas de alimentación desde la parte analógica o de potencia a la parte digital.

Condensadores de desacoplo sobre la tensión de alimentación

Los condensadores de desacoplo se conectan en paralelo sobre las líneas de alimentación ($V_{cc} - 0\text{ V}$) para formar filtros pasa-baja que reducen fuertemente la posibilidad de que la tensión de alimentación sea afectada por perturbaciones de alta frecuencia. Se utiliza un condensador de desacoplo (del orden de 10 nF) para cada circuito integrado, situado lo más cerca posible de los terminales de alimentación del mismo.

Los condensadores de desacoplo no deben ser electrolíticos ya que éstos son deficientes a altas frecuencias (los de aluminio, por encima de los 25 KHz y los de tantalio para los 100 KHz) por presentar un efecto inductivo en serie; deben utilizarse condensadores cerámicos o de poliéster (no de mylar).



Los efectos «antirruído» de un condensador de desacoplo son los siguientes:

- suministra los rápidos «picos de intensidad» que el circuito integrado precisa en las conmutaciones, evitando que tales variaciones de intensidad actúen sobre la autoinducción que presentan las líneas de alimentación y se reflejen en perturbaciones de la tensión de alimentación; tales perturbaciones afectarían al propio circuito integrado que las provoca y, también, al resto de los circuitos integrados conectados a las mismas líneas de alimentación;
- configura un filtro pasa-baja (constituido por la impedancia resistivo-inductiva de las líneas de alimentación y el condensador de desacoplo) que reduce las perturbaciones de alta frecuencia que pudieran llegar al circuito integrado a través de la tensión de alimentación (el filtrado pasa-baja actúa en ambas direcciones: también filtra el paso de perturbaciones del circuito integrado hacia el exterior);
- desacopla los efectos capacitivos propios del circuito integrado respecto de las autoinducciones que presentan las líneas de alimentación, evitando las oscilaciones de alta frecuencia que se podrían producir por acoplo LC;
- divide en dos el bucle que conforman las líneas de alimentación (ya que el condensador actúa como cortocircuito para frecuencias altas) y consigue que el bucle de alimentación en la parte del circuito integrado sea de área mínima (por hallarse el condensador muy próximo a los terminales del integrado).

En relación con el ruido que el propio circuito integrado produce al conmutar, ha de tenerse en cuenta que el «pico de intensidad» que se genera en la conmutación se debe a dos causas que se suman:

- la carga y descarga de las capacidades efectivas al cambiar su tensión (para pasar de un valor booleano a otro)
- y la conducción simultánea de los dos planos de transistores al iniciar la conducción de uno de ellos dentro del transistor hacia corte del otro.

Los condensadores de desacoplo suministran ambos «picos de intensidad», evitando que repercutan sobre las líneas de alimentación.

Por otra parte, el ruido debido a la conmutación es tanto mayor cuanto lo es la rapidez de la misma: flancos muy verticales producen más ruido que flancos suaves, ya que el efecto inductivo de las variaciones de intensidad aumenta con dI/dt . En las salidas de dispositivos programables complejos o de circuitos integrados de aplicación específica suele existir la posibilidad de optar entre flancos más o menos rápidos: es muy adecuado optar por flancos suaves para aquellas señales que no precisan muy alta velocidad.

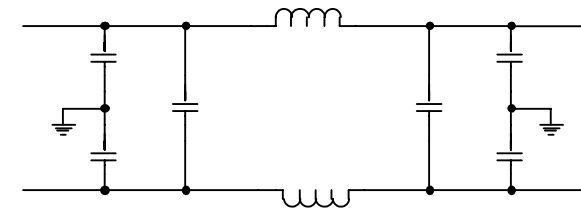
Además del condensador de desacoplo de cada circuito integrado suele incluirse un par de condensadores de desacoplo (uno electrolítico del orden de $100\ \mu\text{F}$ y otro no electrolítico de unos $100\ \text{nF}$) en la entrada de la tensión de alimentación de la placa de circuito impreso (o en la salida de la propia fuente de alimentación si se encuentra en la misma placa). Estos condensadores presentan, en relación con la placa circuital global, los mismos efectos antes comentados (desacoplo de variaciones de intensidad, filtrado en ambos sentidos, eliminación de oscilaciones parásitas y división del bucle de alimentación en dos). El condensador electrolítico, por su mayor capacidad puede suministrar mayores intensidades y asegura un mayor filtrado pero solamente para frecuencias no muy altas, mientras que el otro condensador en paralelo actúa en relación con las frecuencias altas.

Filtros de red para desacoplar la red de suministro eléctrico

Los sistemas digitales conectados a la red de tensión eléctrica (como fuente de energía para su tensión de alimentación) pueden recibir, a través de la misma, perturbaciones electromagnéticas producidas por otros sistemas eléctricos y pueden también transmitir a la red perturbaciones generadas por ellos. Este segundo aspecto (la inserción de perturbaciones sobre la red) resulta, en ocasiones, el más importante, puesto que las normas de compatibilidad electromagnética limitan fuertemente las interferencias que un sistema puede comunicar a la red eléctrica.

Los filtros de red son de tipo pasa-baja y ejercen su efecto en ambas direcciones, limitando el paso de perturbaciones de alta frecuencia tanto de la red hacia el sistema digital como desde dicho sistema hacia la red.

Suelen ser filtros de tercer orden en π (condensador – bobina – condensador), duplicados para filtrar las dos líneas de la red eléctrica, con las dos bobinas enrolladas sobre un mismo núcleo circular de ferrita para evitar su saturación por la propia intensidad que consume el sistema.



Apantallamientos: carcasa y cables

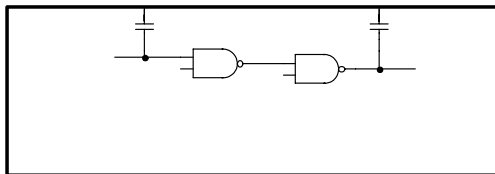
Las perturbaciones presentes en el entorno inciden sobre un sistema digital en forma de ondas electromagnéticas que se propagan a través del espacio (no precisan ni siquiera de un medio material para transmitirse).

Las cajas o carcasas metálicas suponen una barrera que impide en gran medida el paso de tales ondas. Los materiales conductores reflejan parcialmente las ondas electromagnéticas, reduciendo su energía; los materiales ferromagnéticos las atenúan, disipando la energía electromagnética. La combinación de ambos materiales, conductor por fuera de la caja y ferromagnético por dentro, consigue un excelente aislante en relación con los campos electromagnéticos.

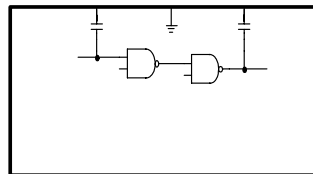
Importa en este caso que la carcasa conforme un recinto cerrado sin fisuras, con conexión completa de la tapa de la misma respecto al resto, para lo cual existen juntas conductoras que facilitan un cierre total. Asimismo importa minimizar las aberturas de la carcasa (necesarias para cables de alimentación y entradas y salidas); como dichas aberturas se encuentran con un frente de onda bidimensional, resulta óptimo que una de las dos dimensiones sea lo más reducida posible.

La carcasa que apantalla a un circuito digital debe ir conectada a la tensión de referencia del mismo (*masa*) para evitar que los posibles acoplos capacitivos que se produzcan entre circuito y carcasa den lugar a peligrosas realimentaciones positivas:

- las líneas del circuito presentan un pequeño acoplo capacitivo con la carcasa metálica
- un amplificador no inversor adquiere una realimentación positiva a través de dichas capacidades: salida del amplificador – carcasa y carcasa – entrada del amplificador
- las propias puertas booleanas (en caso de ser inversoras, las parejas de puertas sucesivas) son etapas amplificadoras
- una realimentación positiva es pernicioso para el ruido, ya que lo amplifica en gran medida.



Amplificador no inversor (formado por dos puertas inversoras sucesivas) con realimentación positiva (a través de los efectos capacitivos con la carcasa)



La realimentación positiva se anula al conectar la carcasa a "masa" (0 V)

La forma de eliminar el efecto de realimentación de las capacidades parásitas que se forman con la carcasa es conectar la carcasa a una tensión fija, por ejemplo a la tensión de referencia del circuito; de esta forma tales capacidades quedan conectadas a 0 V y no forman lazo de realimentación. La conexión carcasa – masa no suele ser directa (mediante un simple cable) sino a través de un condensador (no electrolítico, de valor alto cercano a 1μF); tal conexión debe hacerse en un solo punto para evitar la formación de bucles.

Una vez apantallado un circuito digital, los cables que conectan sus entradas y salidas exteriores pueden actuar como antenas efectivas: pueden recoger perturbaciones electromagnéticas del entorno y conducirlas directamente dentro de la carcasa. Por ello, es necesario plantearse el tipo de cables a utilizar y el apantallamiento de los mismos; existen muchas soluciones en cuanto a blindaje de cables: malla trenzada, laminado recto (en forma de tubo), laminado en espiral, combinaciones laminado y trenzado,...

En caso de cables conectados a equipos que manejan señales de altas frecuencias (como, por ejemplo, los conectados a monitores, pantallas de TV,...) suelen utilizarse abrazaderas de ferrita sobre los propios cables, en el extremo de conexión con tales equipos, para formar una barrera energética que rechaza la transmisión de las componentes de alta frecuencia.

Optoacopladores para las entradas y salidas

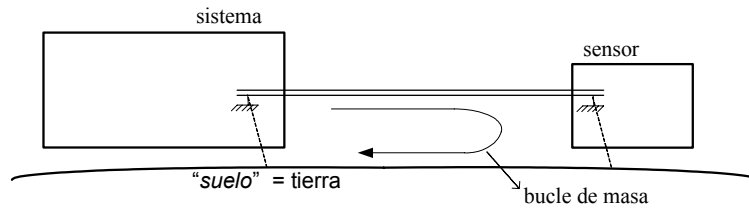
Un optoacoplador está formado por un diodo emisor de radiación luminosa y un fototransistor que detecta dicha radiación. El diodo emisor traduce los valores booleanos 0/1 a situaciones de ausencia/emisión de radiación y de esta forma comunica dichos valores al fotodetector, con separación galvánica entre ambos (emisor y receptor no tienen conexión eléctrica entre ellos).



Los optoacopladores se sitúan en las entradas y salidas de los circuitos digitales hacia el exterior para conformar una separación galvánica respecto de las líneas que conectan el circuito integrado con el entorno exterior al mismo.

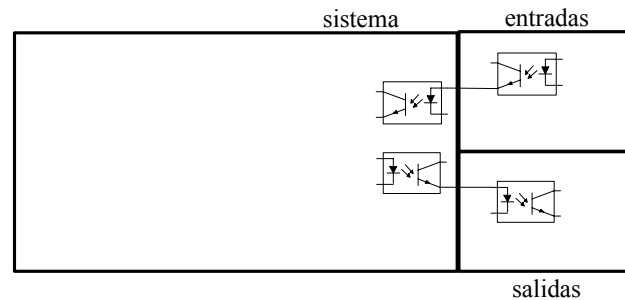
Los efectos de un optoacoplador son los siguientes:

- introduce una ruptura en la continuidad eléctrica de los dos conductores que corresponden a una línea de entrada o de salida exterior; tanto la conexión que transmite la señal como la de retorno quedan interrumpidas y se evita su efecto como antenas de captación de interferencias;
- configura un fuerte filtro pasa-baja frente a las perturbaciones, pues no es posible el paso de interferencias desde el fotodetector al emisor de radiación (pues no hay conexión alguna en tal sentido) y el paso desde el emisor hacia el fototransistor presenta un alto escalón energético;
- divide en dos el bucle que conforma la línea de entrada o salida y consigue que dicho bucle sea de área mínima en la parte que afecta al sistema digital;
- rompe el posible «bucle de tierra» que puede formarse por la línea de masa si está conectada a tierra en más de un punto (dicho bucle se forma entre la línea de masa y la tierra como línea de retorno y puede contener un área muy amplia, ver figura siguiente).



En situaciones de muy alto ruido ambiental puede ser necesario someter a las entradas y salidas digitales a una doble separación galvánica en la forma siguiente:

- transferir todas las líneas de salida y todos los dispositivos de control de potencia (adaptadores de todo tipo: *drivers*, tiristores, triacs, etc,...) a una segunda carcasa adjunta a la del sistema digital, en la cual se aplique un segundo acople optoelectrónico
- de igual forma, recibir todas las líneas de entrada a través de una tercera carcasa adjunta, en la que se efectúe también un acople optoelectrónico adicional
- las fuentes de alimentación de ambas carcasas anexas deben ser diferentes de la alimentación del sistema digital (ubicado en la carcasa principal), a fin de asegurar una efectiva separación galvánica.



Es bueno, también, que las salidas que van a manejar potencia de la red eléctrica lo hagan a través de triacs con disparo en cruce por cero de la red, a fin de que la conmutación de potencia sea lo más suave posible.

El acople optoelectrónico de señales analógicas presenta mayor problema pues se requieren acopladores de extraordinaria precisión para asegurar que los valores de tensión en su salida coincidan con los de su entrada; dicha precisión entraña una dificultad y un coste considerables, lo cual hace que en muchas ocasiones se utilice un simple filtrado en frecuencia que, sin afectar a la señal analógica a transmitir, elimine las perturbaciones de alta frecuencia.

T3.4. Mecanismos de producción/transmisión de interferencias

A CONDUCCIÓN DIRECTA de perturbaciones ya existentes en el circuito o en sus líneas de conexión (alimentación / red, entradas, salidas,...); tales perturbaciones se han producido inicialmente por alguno de los mecanismos que consideraremos a continuación y, una vez presentes en el circuito en términos de tensión, son transmitidas a través de los conductores:

Soluciones: ← filtrado

← separación galvánica

filtros de red
condensadores de desacoplo
optoacopladores
abrazaderas de ferrita sobre los cables
...

B EFECTO AUTOINDUCTIVO que genera transitorios de tensión (*glitches*) en respuesta a los cambios de intensidad:

Soluciones: ← reducir L de las líneas de transmisión

← disminuir dI/dt

← absorber ΔI

líneas cortas, directas
líneas anchas para la alimentación
flancos suaves: *slew-rate*
condensadores de desacoplo
...

C ACOPLA CAPACITIVO entre líneas próximas y paralelas, que transmite de una a otra las variaciones de tensión:

Soluciones: ← reducir la longitud de las líneas

← aumentar su separación

← reducir dV/dt

← interponer un conductor entre ellas

adecuada distribución de los circuitos integrados
líneas de señal cortas
flancos suaves: *slew-rate*
conductor intermedio conectado a 0 V
...

D IMPEDANCIA COMÚN que transite de unas etapas a otras los efectos del consumo de intensidad (resistivos: caídas de tensión RI) y los propios de las variaciones de intensidad (inductivos: transitorios de tensión $L.dI/dt$):

- Soluciones: ← reducir R (por ejemplo, disminuyendo la longitud o aumentando la sección del conductor)
- ← buena distribución de las líneas de alimentación
- ← configuración adecuada de las líneas de retorno (*masa*)
- ← las mismas que en el caso B (efecto autoinductivo)
- líneas cortas y anchas
 - plano de masa
 - esquemas de alimentación en peine o retícula
 - líneas de alimentación directas, anchas y cuidadas
 - condensadores de desacoplo
 - limitar dI/dt (*slew-rate*)
 - separación de alimentaciones: digital, potencia, analógica
 - • •

E ACOPLAMIENTO INDUCTIVO que hace incidir sobre los bucles de corriente el efecto de los campos magnéticos variables (producidos a su vez, por lo general, por otros bucles de corriente variable):

- Soluciones: ← reducir el área de los bucles de corriente
- Normalmente, los bucles se cierran por la línea de retorno o masa; por ello, el diseño geométrico de la «masa» es transcendental
- plano o malla (retícula) de masa
 - expandir al máximo la superficie de masa
 - buena distribución de la línea de alimentación V_{CC}
 - dividir los bucles de alimentación:
 - condensadores de desacoplo
 - evitar los bucles de masa: una sola conexión a tierra
 - romper los bucles de masa: optoacopladores
 - • •

- ← aislar el circuito mediante blindajes
- ← evitar la producción de campos magnéticos
- carcasas ferromagnéticas
 - cables apantallados
 - optoacopladores
 - apantallar transformadores y bobinas
 - no utilizar relés ni contactores
 - • •

F ACOPLAMIENTO ELECTROMAGNÉTICO por el que los diversos conductores y los bucles de corriente actúan como antenas receptoras que recogen el efecto de las ondas electromagnéticas presentes en el entorno:

- Soluciones: ← reducir el área de los bucles
- ← aislar el circuito mediante blindajes
- ← introducir separaciones galvánicas
- carcasas dobles: ferromagnéticas con exterior conductor
 - juntas con continuidad eléctrica
 - ranuras de anchura mínima
 - cables apantallados
 - acoplo optoelectrónico de entradas y de salidas
 - • •

G OSCILACIONES debidas a acoplos entre autoinductancias y capacidades parásitas:

- Soluciones: ← reducir L de las conexiones
- ← filtrado
- líneas cortas y anchas
 - condensadores de desacoplo
 - • •

T3.5. Reglas básicas para el diseño*Reducción del ruido producido por el propio circuito*

- Cuidar mucho el diseño del circuito impreso y los cables de conexionado
- Cables y pistas muy cortos
 - Evitan oscilaciones, acoplos, antenas, caídas de tensión,...
 - Reducen el área de los bucles
- Condensadores de desacoplo en la alimentación
 - Resuelven variaciones de I, dividen los bucles,...
 - * Filtrado en la entrada de VCC de cada placa: 100 μ F // 100 nF
 - * Filtrado de las alimentaciones de cada circuito integrado: 10 nF
- Buena distribución de las alimentaciones
 - Minimiza L, R, bucles e impedancias compartidas
 - * Líneas directas, cortas y anchas
 - * Plano o malla de masa
 - * Separación de alimentaciones: potencia, analógica, digital
- Eliminar / apantallar componentes magnéticos
- Sustituir contactores / relés por triacs que conmuten en cruce por cero
- En los buses de conexión, intercalar líneas de masa intermedias
 - Reducen los acoplos capacitivos

Defensa frente al ruido que le llega del entorno

- Minimizar bucles: reducir su área
 - Reduce su efectividad como receptores
- Aislar el circuito mediante carcasa metálica
 - * aislamiento electrostático: conductores
 - * aislamiento magnético: materiales ferromagnéticos
 - * aislamiento ante ondas electromagnéticas: ambos tipos
 - Los materiales conductores reflejan las ondas, por tanto, deben encontrarse en el lado exterior, y los ferromagnéticos las absorben (interior)
 - * Minimizar las dimensiones lineales de las aberturas
 - * Asegurar la continuidad eléctrica en las juntas
 - * Conectar la masa del circuito a la carcasa mediante un condensador en un solo punto
 - Evita acoplos capacitivos carcasa - circuito
 - El contacto en un solo punto evita bucles

- Aislar las líneas de conexión al exterior.
 - * Filtro de red
 - * Separación galvánica de entradas y salidas mediante optoacopladores
 - Interrumpen la conexión directa y la transmisión de ruido
 - Rompen los bucles de masa
- Apantallar los cables de conexión
 - * Cable coaxial o cable entrelazado
- Evitar los bucles de tierra
 - * Conexión a tierra, a ser posible, en un solo punto.

Limitación del ruido enviado a otros circuitos

- A través de la red
 - * No trocear la intensidad
 - * Ajustar el consumo de intensidad a la senoide de la red
 - * Disparar componentes de potencia en cruce por cero de red
 - * Filtrar adecuadamente la conexión a red
 - * Los flancos de conmutación deben ser lo más suaves posible
- Por ondas electromagnéticas
 - * Apantallar
 - * Evitar antenas
 - * Minimizar el área de los bucles
 - * Evitar contactos mecánicos (pulsadores, contactores, relés, ...)
 - * Evitar componentes magnéticos.

Consideraciones complementarias

- El ruido electromagnético es un problema a tener en cuenta desde las etapas iniciales del diseño: una vez completado el diseño de un equipo se hace más difícil y costosa su protección frente al ruido.

- Los problemas debidos al ruido externo (interequipos o ambiental) no se suelen manifestar en la fase de desarrollo en el laboratorio: son en gran medida problemas de contaminación ambiental que aparecen al llevar el circuito a su ambiente real.

- En los circuitos digitales conviene:

- no ir nunca a muy alta velocidad si no es necesario
- no utilizar series ultrarrápidas si no es preciso
- no emplear flancos muy verticales innecesariamente

ya que, todo ello, además de aumentar el consumo dinámico, colabora en gran medida en aumentar el nivel de ruido.



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

11 INTRODUCCIÓN A LOS SISTEMAS SECUENCIALES I: CONCEPTOS

- 11.1. Necesidad de memoria: concepto de estado
- 11.2. Variables de estado y grafos de estados
- 11.3. La memoria como almacén de información: biestables y registros
- 11.4. Estado, biestables y variables de salida; autómatas de Moore y de Mealy

Los sistemas combinacionales se construyen mediante funciones booleanas de sus variables de entrada. Pero no todo sistema digital es combinacional: existen sistemas en que la correspondencia entre el vector de entrada y el vector de salida no es unívoca; es decir, no se pueden obtener las salidas como funciones de «sólo» las entradas.

Los sistemas secuenciales necesitan recordar su pasado, la «secuencia de vectores de entrada» a través de la cual se ha llegado a la situación presente; para ello han de tener «memoria», que se configura mediante un vector de «estado» que contiene la información que el sistema necesita sobre su pasado. La memoria se consigue mediante «realimentación lógica»: las variables de estado son, a la vez, variables de salida y de entrada en las funciones de evolución del estado.

Tres vectores: entrada, estado y salida, y dos conjuntos de funciones: las de evolución del estado (estado anterior y entradas determinan el nuevo estado) y las de activación de las salidas (que dependen, también, del estado y de las entradas).

La evolución de un sistema secuencial puede ser representada mediante un grafo de estados, a partir del cual pueden construirse las funciones de modificación del estado. En ocasiones, hay dos formas diferenciadas de configurar el grafo de estados: distinguiendo salidas distintas con estados diferentes (autómata de Moore) o reduciendo el número de estados al mínimo y admitiendo que un mismo estado pueda tener varios vectores de salida (autómata de Mealy).

El presente capítulo presenta gradualmente los conceptos básicos necesarios para comprender y describir los sistemas secuenciales, a través de una serie de ejemplos; se reserva para el capítulo siguiente la metodología de diseño secuencial.

La necesidad de memoria, como recuerdo del pasado de un sistema secuencial, se concreta en el concepto de estado. Pero existe otra perspectiva complementaria que nos lleva a la memoria como necesidad de almacenar datos y resultados, es decir, conservar información que puede ser utilizada posteriormente. El biestable como celda capaz de almacenar un bit y el registro (conjunto de biestables) como bloque capaz de almacenar una palabra binaria son los elementos básicos de la memoria.

Precisamente, el siguiente capítulo desarrollará el diseño de circuitos secuenciales, utilizando los biestables como celdas que contienen las variables de estado.

11.1. Necesidad de memoria: concepto de estado

Los sistemas combinacionales presentan una correspondencia unívoca entre el conjunto de vectores de entrada y el de vectores de salida.

Un sistema combinacional es del tipo estímulo-respuesta: al recibir un vector de entrada dado produce un vector de salida determinado, siempre el mismo; la salida es respuesta directa respecto a la entrada y, por ello, puede obtenerse por «combinación booleana» de los valores de las entradas.

En un sistema combinacional las salidas son función booleana directa de los valores presentes en las variables de entrada en dicho momento:

vector de entrada $\mathbf{X} = (x_i) = (x_1, x_2, x_3, x_4, \dots, x_m)$

vector de salida $\mathbf{Y} = (y_j) = (y_1, y_2, y_3, \dots, y_n)$

$\mathbf{Y} = \mathbf{F}(\mathbf{X})$ $y_j = f_j(x_i) = f_j(x_1, x_2, x_3, x_4, \dots, x_m)$ para $j = 1, 2, \dots, n$.

Conocido el funcionamiento requerido para un sistema combinacional, esto es, su tabla funcional o «tabla de verdad», a partir de ella pueden obtenerse directamente las funciones booleanas de las entradas que determinan las salidas del sistema. A cada vector de entrada le corresponde uno y solo un vector de salida: la correspondencia vectores de entrada $\rightarrow$ vectores de salida es unívoca.

No todo sistema digital es combinacional. No toda correspondencia entre el conjunto de vectores de entrada y el de vectores de salida es unívoca.

Existen sistemas digitales que no encajan dentro de este tipo combinacional, sistemas que respecto a un mismo vector de entrada, en momentos diferentes, producen vectores de salida distintos. Un ejemplo sencillo y práctico lo constituye el sistema de apertura y cierre de una puerta de garaje (del tipo de persiana metálica) que actúe de la forma siguiente:

- el sistema dispone de tres entradas: un pulsador de activación o llave de contacto $\mathbf{P}$ y dos detectores o toques de fin de carrera $\mathbf{T_s}$ y $\mathbf{T_i}$ (superior e inferior); dispone asimismo de dos salidas: movimiento hacia arriba $\mathbf{S}$ y movimiento hacia abajo $\mathbf{B}$ de la puerta

- al presionar el pulsador $\mathbf{P}$, si la puerta está abajo (tope inferior $\mathbf{T_i}$ activado), se inicia el movimiento ascendente $\mathbf{S}$, hasta que se activa el tope superior $\mathbf{T_s}$; en cambio, si es el tope superior $\mathbf{T_s}$ el que se encuentra activado en el momento de pulsar $\mathbf{P}$ (puerta arriba), la puerta efectúa el movimiento de bajada $\mathbf{B}$ hasta llegar al tope inferior $\mathbf{T_i}$.

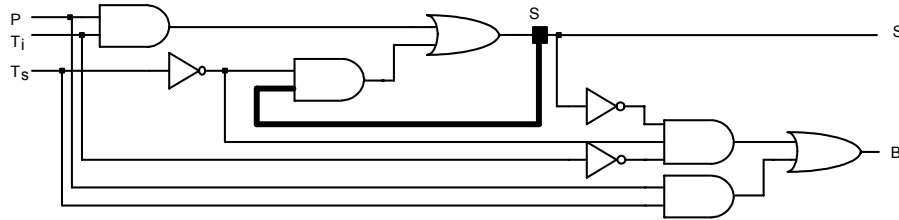
Es decir:	$\mathbf{P=0, T_i=1, T_s=0}$	$\mathbf{S=0}$ y $\mathbf{B=0}$
	$\mathbf{P=1, T_i=1, T_s=0}$	$\mathbf{S=1}$ y $\mathbf{B=0}$
	$\mathbf{P=0, T_i=0, T_s=1}$	$\mathbf{S=0}$ y $\mathbf{B=0}$
	$\mathbf{P=1, T_i=0, T_s=1}$	$\mathbf{S=0}$ y $\mathbf{B=1}$

$S(t + \Delta t) = P \cdot T_i + \overline{P} \cdot T_s$: la variable de estado **S** se activa cuando se pulsa **P** estando la puerta abajo **T_i** y permanece activa hasta alcanzar el tope superior **T_s**.

Una de las funciones de salida consiste en la identificación de la variable de estado **S** con una salida **S = S**; la otra se obtiene de su correspondiente tabla booleana:

$T_s S$					
$P T_i$		00	01	11	10
00		1	0	0	0
01		0	0	X	X
11		0	0	X	X
10		1	0	1	1

$B = P \cdot T_s + \overline{T_i} \cdot \overline{T_s} \cdot \overline{S}$: la variable de salida **B** se encontrará activada al pulsar **P** cuando la puerta se encuentre arriba **T_s** y, también, cuando no lo estén los topes inferior y superior **T_i** y **T_s**, ni la subida **S**.



En la figura anterior se ha resaltado la realimentación booleana que realiza la memoria del estado del sistema (variable **S**).

El estado (el conjunto de variables de estado) representa la información sobre la secuencia previa de vectores de entrada que el sistema secuencial necesita. De forma que un sistema secuencial presenta tres vectores propios: el de entrada, el de salida y el de estado, y dos correspondencias funcionales: la de evolución del estado y la de activación de las salidas

Un sistema secuencial se caracteriza por responder de forma distinta a un mismo vector de entrada en momentos diferentes, dependiendo de la secuencia previa de vectores de entrada. Ahora bien, para que pueda discriminar entre diversas secuencias de vectores de entrada es necesario que posea unas «variables de estado» que, de alguna forma, guarden información sobre la historia del circuito (historia = pasado = secuencia anterior).

En función del valor de las entradas en un momento dado y del valor de las variables de estado en dicho momento el circuito responde con un determinado conjunto de valores en sus salidas y, a la vez, modifica sus variables de estado para tomar en cuenta el nuevo vector de entrada.

Las variables de estado actúan, a la vez, como variables de salida y como variables de entrada en las funciones booleanas que configuran el sistema digital. Como variables de salida expresan el estado en que se encuentra el sistema como consecuencia de la secuencia de vectores de entrada anteriores. Como variables de entrada actúan en la modificación de dicho estado a la llegada de un nuevo vector de entrada y, también, determinan, junto con el vector de entrada, las salidas del sistema en cada momento.

La realimentación de las variables de estado, es decir, el hecho de que, siendo salidas intermedias del sistema, vuelven a la entrada del mismo, es lo que proporciona memoria al sistema secuencial (información sobre su historia pasada).

Algunas de las variables de estado, o en ocasiones todas ellas, pueden coincidir con variables de salida (en el ejemplo anterior, la variable de estado es una de las de salida, **S**), pero no siempre es posible tal identificación; en general, las variables de estado no coincidirán ni tendrán relación directa con las variables de salida del sistema; existen tres conjuntos de variables diferentes: entrada, salida y estado.

Incluso en aquellos casos en que alguna variable de estado coincida con una variable de salida, hay una distinción conceptual neta entre el vector de salida de un sistema y el estado del mismo: el estado representa la memoria que posee de su pasado anterior.

Un sistema secuencial dispone de tres conjuntos de variables:

vector de entrada **X = (x_i)**, vector de salida **Y = (y_j)** y estado **Q = (q_k)**

relacionados a través de dos conjuntos de funciones booleanas:

- función de salida **Y = F(X, Q)**

- evolución del estado **Q(t+Δt) = G(X(t), Q(t))**

La primera función representa la formación de las salidas del sistema como combinación de las entradas y del estado en cada momento; la segunda, muestra la evolución del estado del sistema en función de las entradas y del propio estado en el momento anterior.

El conjunto de variables de estado, el vector de estado, contiene la información que el sistema precisa para distinguir entre las diferentes secuencias de vectores de entrada que, teniendo un mismo vector al final de la secuencia, conducen a vectores de salida distintos.

11.2. Variables de estado y grafos de estados

Las variables de estado de un sistema secuencial representan conjuntamente el estado del sistema; individualmente cada variable de estado puede no tener significado físico alguno.

Conceptualmente, las variables de estado tienen un significado conjunto, el de numerar (y, por tanto, diferenciar) los diversos estados por los que evoluciona el sistema. El vector de estado representa la memoria que el sistema tiene de su evolución anterior.

¿Qué representa cada una de las variables de estado? Individualmente no tiene por qué tener un significado físico concreto; en principio, cada variable de estado es una variable «interna», que no tiene que corresponder necesariamente a ninguna variable de salida ni a ninguna situación determinada.

El significado corresponde al vector de entrada en su conjunto, sin tener cada una de sus variables una significación individualizada; si bien, en muchos casos, pueden tenerla por identificarse directa o indirectamente con alguna variable de salida o con alguna variable situacional del sistema.

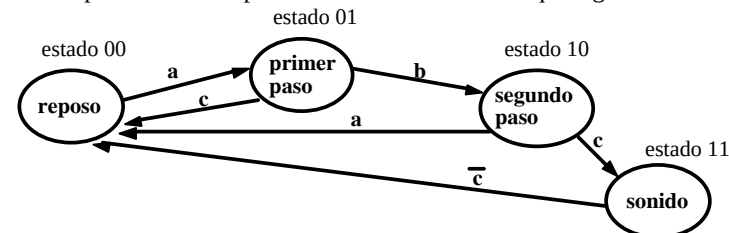
El ejemplo del apartado anterior (puerta de garaje que se activa para subir y para bajar con un pulsador) es muy interesante por su sencillez y por ser un caso práctico muy habitual pero presenta aspectos que pueden inducir a error conceptual:

- la variable de estado coincide con una de las variables de salida, lo cual no tiene porque ser así: el estado y las salidas son, en principio, dos vectores diferentes, cuya función y significado son bien distintos (si bien es cierto que, en algunos casos, pueden coincidir algunas de sus variables);
- precisamente debido a esta identificación salida-estado, en el apartado anterior se dice que el sistema tiene que recordar «lo que estaba haciendo» (subir o bajar), afirmación que no es correcta conceptualmente: el sistema tiene que recordar su pasado en términos de vectores de entrada recibidos anteriormente, ha de recordar la secuencia de vectores de entrada a través de la cual ha llegado hasta la situación actual.

El control de la puerta de garaje, cuando la entrada es **000** (P T_s T_i), debe recordar si anteriormente provenía de la situación de entrada **101** (empezar a subir) o de la situación **110** (comenzar a bajar).

Consideremos otro ejemplo: un simple timbre con tres pulsadores **a b c**, enlazados por un mecanismo que impide pulsar dos a la vez; el timbre debe sonar cuando se pulsan sucesivamente los tres pulsadores, primero el **a**, luego el **b** y, por último, el **c** y el sonido cesa al soltar el pulsador **c**.

Podemos representar este tipo de actuación en la forma que sigue:



El sistema debe acordarse de 4 situaciones o estados diferentes, que pueden ser representados con dos variables de estado **q₂** y **q₁**:

q₂ = 0	q₁ = 0	estado de reposo
q₂ = 0	q₁ = 1	primer paso en la secuencia a b c
q₂ = 1	q₁ = 0	segundo paso en la secuencia a b c
q₂ = 1	q₁ = 1	se ha completado la secuencia: el timbre suena.

La secuencia de vectores de entrada (**a b c**) correcta es la siguiente:

000, 100, 000, 010, 000, 001.

La variable de salida (sonido del timbre) se activa en el estado **11**: **y = q₂·q₁**

¿Qué representan las variables **q₂** y **q₁**? Individualmente cada una de ellas no tienen significado; las dos juntas expresan el estado del sistema. El estado tiene sentido físico; cada variable de estado no tiene por qué tenerlo.

Construyendo la tabla de evolución de los estados **q₂ q₁ a b c → q₂ q₁** (32 filas), pueden obtenerse las funciones **q₂ q₁**:

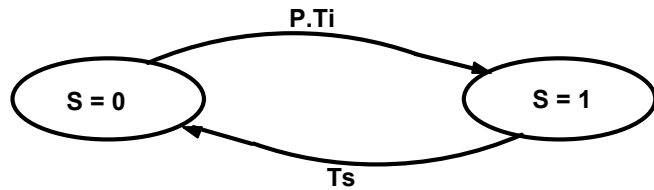
$$q_1^+ = q_2 \cdot c + \overline{q_2} \cdot a + \overline{q_2} \cdot q_1 \cdot \overline{c} \cdot \overline{b} \quad q_2^+ = q_2 \cdot c + q_2 \cdot \overline{q_1} \cdot \overline{a} + \overline{q_2} \cdot q_1 \cdot b$$

La evolución de los estados puede expresarse eficazmente en un grafo.

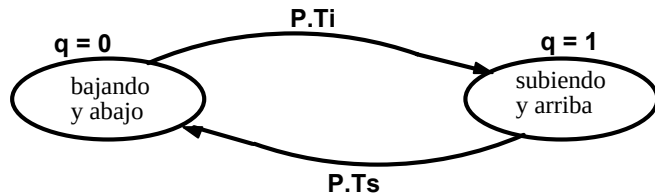
La evolución del estado de un sistema secuencial puede ser representada mediante un grafo de estados. Un grafo de estados detalla los diversos estados del sistema y las transiciones entre estados; las transiciones se representan mediante arcos orientados desde el estado de partida hasta el estado resultante de la transición, con indicación de la condición booleana que determina dicha transición.

La figura anterior (al inicio de esta misma página) corresponde al grafo de estados del circuito de control del timbre con tres pulsadores **a b c**.

El grafo de estados del control de la puerta de garaje del apartado anterior será:



También sería correcto un grafo más simétrico, en la forma que sigue:



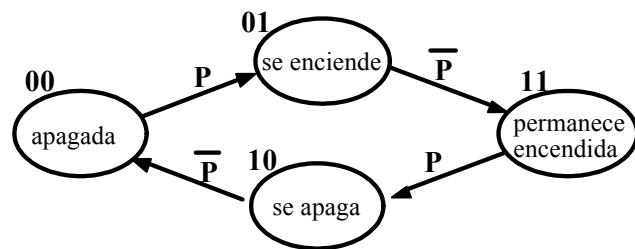
En este caso las funciones de activación de las salidas serán:

$$S \text{ (subir)} = q \cdot \overline{T_s} \quad B \text{ (bajar)} = \overline{q} \cdot \overline{T_i}$$

y la función de evolución del estado: $q^+ = P \cdot T_i + q \cdot (\overline{p} \cdot \overline{T_s})$.

En este segundo grafo de control de la puerta de garaje, la variable de estado **q** no se corresponde con ninguna de las salidas del sistema **S B**, sino que distingue entre dos situaciones: la de subiendo, incluida la puerta parada arriba **q = 1** y la de bajando con inclusión de la puerta parada abajo **q=0**.

Otro ejemplo también muy habitual y sencillo: la lámpara de luz de mesa con un pulsador, que se enciende al pulsar una vez y se apaga al pulsar una segunda vez.



Obsérvese que para diferenciar un pulso del siguiente es necesario detectar **P** y **P-bar**; la omisión de las transiciones con **P-bar** conduciría a un grafo erróneo: con **P = 1** se producirían varias transiciones, que deberían corresponder a pulsos diferentes.

Son necesarios cuatro estados y dos variables de estado **q2 q1**; numerando los estados según el código Gray, la secuencia de funcionamiento es **00 → 01 → 11 → 10** y la variable de salida (lámpara encendida) coincide con **q1: y = q1**.

La tabla de evolución de estados permite obtener las funciones **q2 q1**:

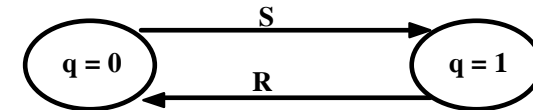
q2	q1	P	q2 ⁺	q1 ⁺
0	0	0	0	0
0	0	1	0	1
0	1	0	1	1
0	1	1	0	1
1	0	0	0	0
1	0	1	1	0
1	1	0	1	1
1	1	1	1	0

$$q_2^+ = q_2 \cdot P + q_1 \cdot \overline{P}$$

$$q_1^+ = \overline{q_2} \cdot P + q_1 \cdot \overline{P}$$

En este caso, la variable **q1** coincide con la salida (encendido de la lámpara) y, en cambio, la variable **q2** no se refleja exteriormente: es una variable de estado necesaria para diferenciar un pulsado del siguiente.

El ejemplo de sistema secuencial más simple posible es aquel que solamente tiene dos estados **q = 0** y **q = 1**; supongamos que posee dos entradas, una **S** (set) para marcar (poner a **1**) a la variable de estado **q** y la otra **R** (reset) para borrarla (llevar a **0**). Por ejemplo, una lámpara con dos pulsadores diferentes **S** para encenderla y **R** para apagarla (y suponemos que nunca se pulsan los dos a la vez, pues resulta contradictorio desear encender y apagar la lámpara al mismo tiempo).



La variable de salida coincide con la de estado y su tabla de evolución será la siguiente:

q	S	R	q ⁺
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	X
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	X

$$q^+ = S + q \cdot \overline{R}$$

Precisamente este tipo de sistema secuencial mínimo será considerado en el próximo apartado y recibe el nombre de biestable por tener solamente dos estados.

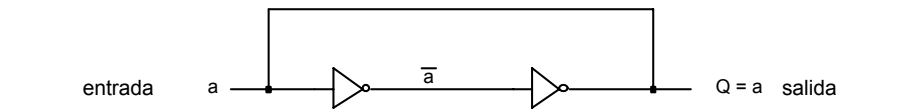
11.3. La memoria como almacén de información: biestables y registros

En los dos apartados anteriores consideramos la memoria referida al estado de un sistema secuencial: la necesidad de recordar la secuencia de vectores de entrada anteriores y el estado como memoria efectiva de dicha secuencia. En este apartado consideraremos la memoria desde otra perspectiva: como almacén de información, es decir, conservación de los valores booleanos de algunas variables.

Los sistemas combinacionales permiten «procesar» la información, tanto numéricamente (operaciones de cálculo) como lógicamente (combinación de proposiciones) y, también, distribuirla adecuadamente (multiplexado) y efectuar cambios entre diferentes codificaciones de la misma información (codificadores).

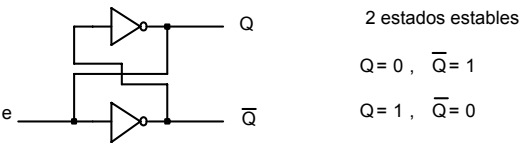
Ahora bien, tales sistemas no bastan para manejar eficientemente números y palabras binarios; se requiere, además, disponer de registros o memorias donde almacenar la información, son necesarios unos «módulos» que almacenen y conserven las palabras digitales (conjuntos de ceros y unos) que contienen la información.

La unidad elemental de memoria será un dispositivo capaz de almacenar un 0 o un 1 y de conservarlo en ausencia de entrada; tal sistema recibe el nombre de *biestable* (dos estados estables). La forma más simple de configurar un biestable (y, a la vez, su forma «conceptual» básica) consiste en conectar dos inversores en lazo cerrado.



Al establecer durante un momento un valor booleano sobre la entrada de este par de inversores, el primero de ellos invierte dicho valor y el segundo vuelve a invertirlo, coincidiendo con el valor de la entrada: no es necesario mantener exteriormente el valor booleano en la entrada pues, aun dejando libre dicha entrada, el acoplo cerrado de los dos inversores conserva el estado adquirido.

El sistema se encontrará en uno de sus dos estados estables ($Q=0$; $Q=1$) dependiendo de que el último valor booleano aportado a su entrada sea 0 ó 1. La existencia de dos estados estables da lugar a la denominación de *biestable*; si bien, con mucha frecuencia se utiliza el nombre de *Flip-Flop FF*, cuya derivación es meramente onomatopéyica.



En un sistema digital no tiene sentido «dejar libre la entrada», pues cada entrada estará conectada a la salida de otra puerta o a una entrada exterior o a un sistema anterior que le proporcionará un 0 o un 1 y no tiene significado afirmar «la ausencia de entrada». En tal sentido, se definen los biestables **RS** y **D** en la forma que a continuación se detalla.

11.3.1. Biestable RS

El biestable **RS** presenta dos entradas **R** y **S**, la primera de ellas **R** (*reset*) de puesta a 0 o borrado y la otra **S** (*set*) de puesta a 1 o marcado, no siendo admisible activar ambas entradas a la vez. De esta forma el marcado (memorización de un 1) y el borrado (almacenamiento de un 0) se realizan por dos entradas distintas.

R	S	Q
0	0	conserva su valor anterior
0	1	1
1	0	0
1	1	valores de entrada no admisibles

Se puede configurar un biestable **RS** directamente con puertas "o-negada" (*Nor*) o con puertas "y-negada" (*Nand*):

R	S	q	q ⁺
0	0	0	0
0	0	1	1
0	1	X	1
1	0	X	0
1	1	X	¿?

¿? = 0 borrado prioritario

$$q^+ = \overline{R} \cdot (S + q) = R \Delta (S \Delta q)$$

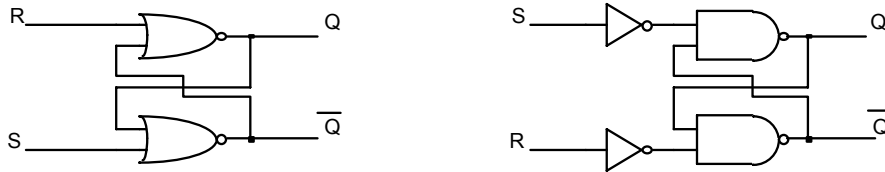
¿? = 1 marcado prioritario

$$q^+ = S + \overline{R} \cdot q = \overline{S} * (\overline{R} * q)$$

Biestable RS configurado con puertas *Nor*

Biestable RS configurado con puertas *Nand*

El vector de entrada **11** no es admisible, pues si va seguido del vector **00**, la situación posterior del biestable no es predecible, ya que dependerá de los transitorios de conmutación de las dos entradas **RS** (en principio, dependerá de cual de ellas se retrase más en pasar a 0).



En ambos casos se utilizan dos puertas en lazo cerrado; los dos esquemas son estructuralmente análogos (difieren en su respuesta para el vector de entrada $R=S=1$, situación que no debe producirse) pero presentan dos diferencias a tener en cuenta:

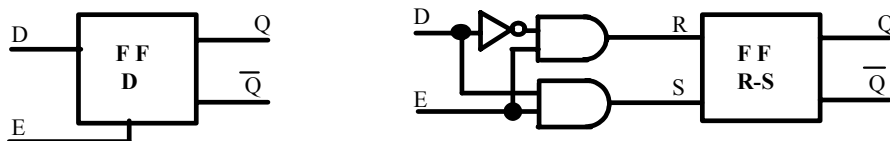
- en el circuito con puertas "o-negada" R actúa sobre el terminal superior y S sobre el inferior mientras que en el que utiliza puertas "y-negada" es al revés
- los valores activos sobre dichas entradas son el **1** en las puertas "o-negada" y el **0** ($\bar{R}$, $\bar{S}$) en la configuración con puertas "y-negada" (o bien, la inclusión de inversores en dicha configuración).

11.3.2. Biestable D

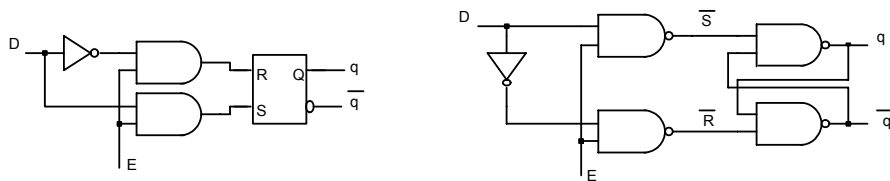
El biestable **D** posee una entrada de almacenamiento **D** y otra de habilitación **E**, de forma que cuando el biestable es habilitado ($E=1$) almacena el valor booleano presente en la entrada **D** y lo conserva hasta una nueva habilitación. Si durante el tiempo de habilitación dicho valor de entrada se modifica, el biestable va aceptando los diversos valores presentes en **D**, reteniendo el último de ellos cuando la habilitación desaparece.

Para configurar un biestable **D** a partir de un biestable **RS**, basta activar sus entradas **R** y **S** con las siguientes condiciones booleanas:

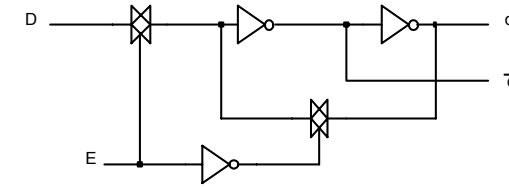
$$S = D \cdot E \quad R = \bar{D} \cdot E$$



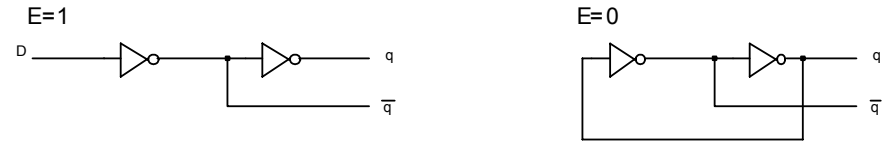
Tal esquema, realizado con puertas "y-negada", queda en la forma:



Este biestable puede construirse también con puertas de transmisión:

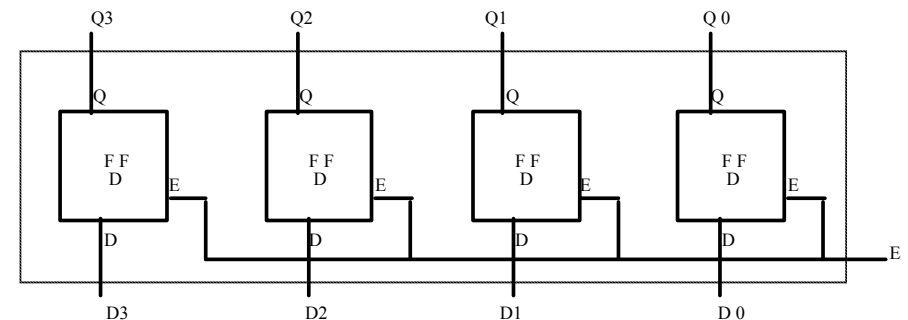


Cuando $E = 1$ la primera puerta conduce y el valor presente en **D** se comunica a la salida **q**; al hacerse $E = 0$, la entrada **D** queda aislada y el valor de la salida se conserva gracias a la realimentación que realiza la segunda puerta de transmisión.



[En tecnologías MOS la opción de puertas de transmisión presenta ventajas respecto a la de puertas *Nand*: menor número de transistores, del orden de la mitad, que repercuten en menor área de integración y menores tiempos de propagación].

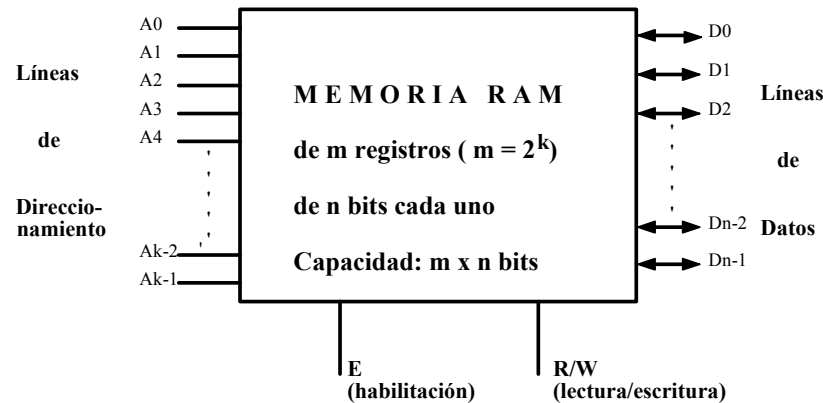
El biestable **D** constituye la célula básica de memoria capaz de almacenar y conserva un valor booleano, bajo control de la entrada de habilitación. Agrupando **n** de estos biestables en paralelo, con su entrada de habilitación común, se configura un registro de longitud **n**, capaz de almacenar una palabra de **n** dígitos o bits. Tal registro recibe el nombre de *memoria de retención* o «memoria cerrojo» (*latch-memory*), porque retiene la información almacenada cuando $E=1$ durante todo el tiempo en que $E=0$, es decir, la conserva entre dos habilitaciones sucesivas.



Estos registros pueden ampliarse a cualquier número de bits, sin más que utilizar varios de ellos, uniendo en una sola las diversas entradas de habilitación **E**.

La agrupación de m registros de n bits, controlados a través de k entradas de direccionamiento ($m=2^k$), de forma que cada registro queda seleccionado por su número en binario, da lugar a un bloque de memoria **RAM**, *memoria de acceso directo* o aleatorio (*random access memory*).

Los terminales de entrada y de salida de los registros son comunes para todos ellos y, en cada momento, el vector presente en las entradas de control o direccionamiento indicará sobre cuál de los registros se actúa.



Además, por simplicidad de acceso, no hay distinción entre terminales de entrada y terminales de salida de los registros (es decir, la misma línea actúa como entrada y como salida), de forma que el tipo de acceso al registro seleccionado (lectura del registro o escritura del mismo) ha de ser indicado por una línea adicional **R/W** (*lectura/escritura*).

Un bloque **RAM** tendrá k líneas de direccionamiento A_i , que actúan como entradas, n líneas de datos D_i , que actúan bidireccionalmente, una entrada de selección de la operación a realizar **R/W**, que distingue entre lectura y escritura y una entrada de habilitación **E**. Su esquema conceptual está conformado por un sub-bloque central que contiene los m registros de n bits, un decodificador de k líneas de entrada que selecciona los registros ($m = 2^k$) y un circuito adaptador de entradas/salidas que controla la actuación de las n líneas de datos a tenor de las entradas de habilitación **E** y lectura/escritura **R/W**.

De esta forma, se dispone de unidades de memoria (biestables **D**) capaces de almacenar y conservar un bit, de registros (conjuntos de biestables **D**) capaces de almacenar una palabra binaria y de bloques de memoria (conjuntos de registros numerados) capaces de almacenar múltiples datos ordenados.

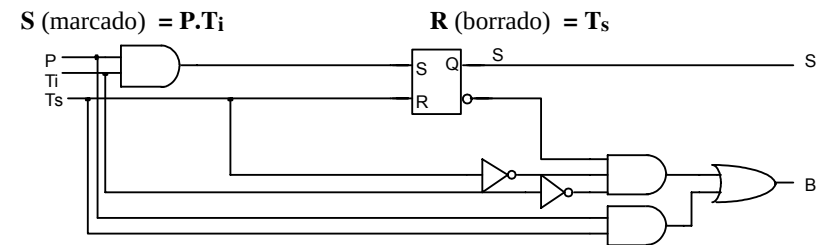
11.4. Estado, biestables y variables de salida; autómatas de Moore y de Mealy

Las variables de estado de un sistema secuencial pueden configurarse sobre biestables RS.

Los biestables **RS** pueden ser utilizados para la configuración de sistemas secuenciales como unidades de memoria capaces de almacenar las variables de estado; será preciso emplear un biestable para cada una de las variables de estado y, en este caso, el resto del circuito será exclusivamente combinacional.

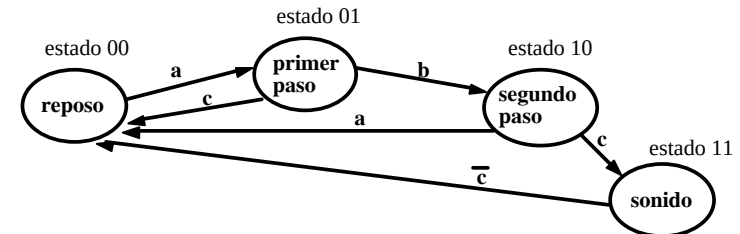
En el sistema de control de la puerta de garaje utilizando la variable de salida **S** como variable de estado, un biestable tipo **RS** servirá para «memorizar» dicha variable, cuyas condiciones de marcado y de borrado serán, respectivamente:

- la variable **S** (subir) se marcará cuando la puerta se encuentre abajo ($T_i = 1$) y se active el pulsador **P** y se borrará al llegar arriba ($T_s = 1$).



En este esquema no queda reflejada la realimentación booleana, ya que la memoria del estado está configurada por el biestable.

En el caso del timbre con tres pulsadores **a b c** (con un mecanismo que impide pulsar dos a la vez y que debe sonar cuando se pulsan sucesivamente los tres pulsadores: **a, b y c**), del grafo de estado pueden extraerse las condiciones de marcado y borrado de cada una de las variables de estado:



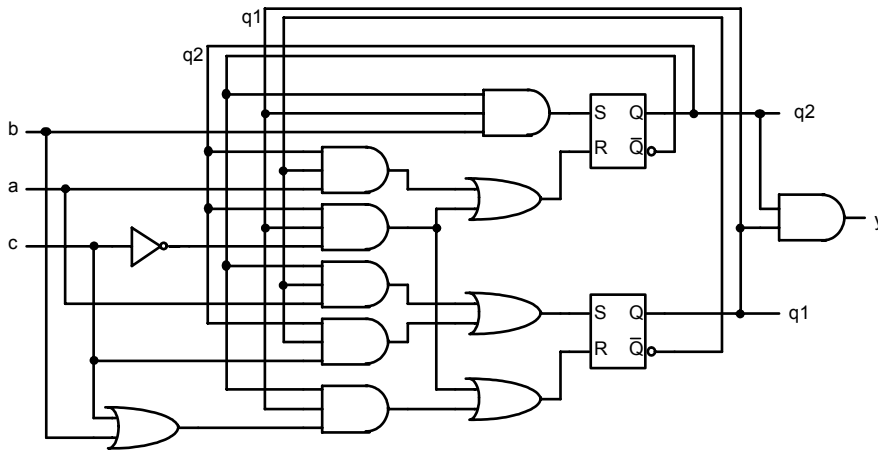
- dos variables de estado $q_2 q_1$ sirven para «dar nombre» a los cuatro estados
- la variable de estado q_1 se marca (pasa de 0 a 1) desde el estado 00 con **a** y desde el estado 10 con **c** y se borra (pasa de 1 a 0) desde el estado 01 con **b** y con **c** y desde el estado 11 con **c** negado
- la variable de estado q_2 se marca (pasa de 0 a 1) desde el estado 01 con **b** y se borra (pasa de 1 a 0) desde el estado 10 con **a** y desde el estado 11 con **c** negado.

$$q_1: S = \overline{q_2} \cdot \overline{q_1} \cdot a + q_2 \cdot \overline{q_1} \cdot c \quad R = \overline{q_2} \cdot q_1 \cdot (b + c) + q_2 \cdot q_1 \cdot \overline{c}$$

$$q_2: S = \overline{q_2} \cdot q_1 \cdot b \quad R = q_2 \cdot \overline{q_1} \cdot a + q_2 \cdot q_1 \cdot \overline{c}$$

- variable de salida: **timbre = y = q₂ · q₁**

La siguiente figura representa el circuito correspondiente a estas funciones:



De esta forma, todo sistema secuencial puede dividirse en tres subsistemas: dos de ellos combinacionales y el tercero constituido por los biestables que almacenan las variables de estado.

Todo sistema secuencial puede ser construido mediante un conjunto de biestables que contengan las variables de estado y el resto del sistema será meramente combinacional:

- las funciones que activan (marcado) y desactivan (borrado) a los biestables
- y las funciones que producen las salidas del sistema.

Dichas funciones de marcado y borrado de los biestables que contienen a las variables de estado pueden obtenerse directamente de los arcos que representan las transiciones en el grafo de estados.

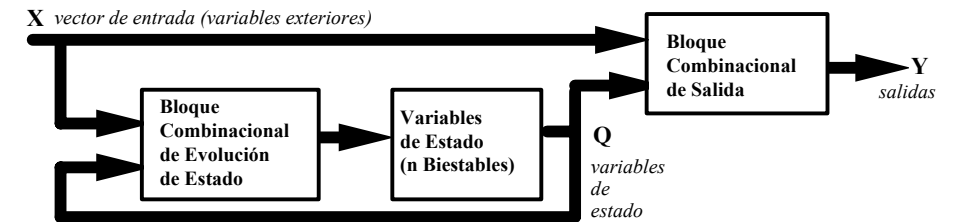
Conviene insistir en que el diseño de un circuito secuencial no se reduce a sus variables de estado, sino que requiere configurar, también, las variables de salida. De hecho, las variables de salida son «lo que se ve» desde fuera, el resultado efectivo de la actuación del circuito; las variables de estado son componentes internas instrumentales que el circuito necesita como memoria de su evolución anterior.

Autómata es el nombre que los matemáticos dan a los sistemas secuenciales, es la denominación de la estructura matemática que describe a dichos sistemas: una quintupla $[X, Q, Y, f, g]$ conformada por tres vectores: el vector de entrada, el vector de estado y el vector de salida, y dos funciones: la correspondencia que determina la evolución del estado y la correspondencia que determina la activación de las salidas.

Un sistema secuencial o autómata engloba tres conjuntos de variables o vectores booleanos, entrada X , salida Y y estado Q , y dos conjuntos de relaciones funcionales:

$$\begin{aligned} &\text{- evolución del estado} && X.Q \longrightarrow Q \\ &\text{- activación de las salidas} && X.Q \longrightarrow Y \end{aligned}$$

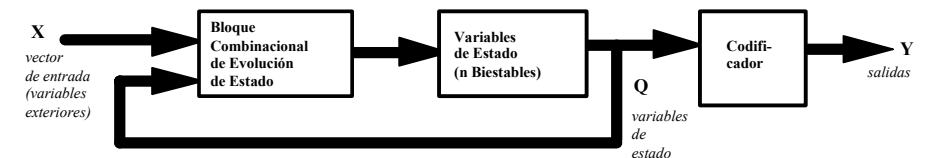
Este tipo de estructura, con separación funcional entre estado y salidas, de forma que a un mismo estado le pueden corresponder diversos vectores de salida (para vectores de entrada diferentes), recibe el nombre de *autómata de Mealy*.



Ahora bien, si nos fijamos en los grafos de estado desarrollados en este capítulo, resulta que en todos (menos en los de la puerta de garaje) el vector de salida es función únicamente del estado (no depende del vector de entrada).

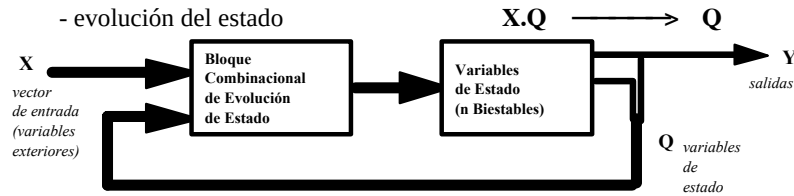
Ello da lugar a un tipo de estructura alternativa denominada *autómata de Moore*, en la cual la correspondencia entre estado y vector de salida es unívoca, de forma que a cada estado le corresponda un solo vector de salida. En este caso el estado ha de contener, además de la información sobre la evolución anterior del sistema, la relativa al vector de salida que debe producir en el momento actual; de forma que las relaciones funcionales en el autómata de Moore son las siguientes:

$$\begin{aligned} &\text{- evolución del estado} && X.Q \longrightarrow Q \\ &\text{- activación de las salidas} && Q \longrightarrow Y \end{aligned}$$



En el *autómata de Moore* el vector de salida se encuentra contenido en el estado, es una simple recodificación del vector de estado. De manera que un estado del autómata de Mealy que produzca diversos vectores de salida tiene que desdoblarse en varios estados del autómata de Moore, uno para cada vector de salida (distinguiéndose estos estados entre sí por los vectores de entrada que, en el caso de Mealy, diferenciaban entre los vectores de salida posibles).

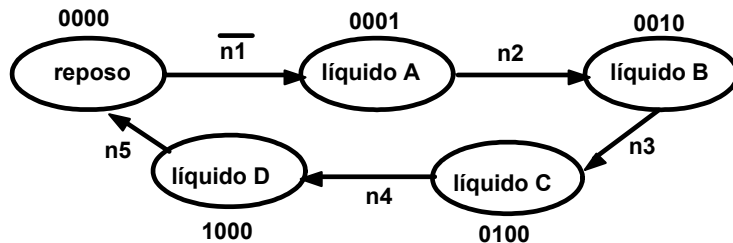
Entre estado y salida hay una simple codificación booleana; más aún, en muchos casos las variables de salida son una parte de las variables de estado: el vector de estado queda configurado por el vector de salida junto con algunas otras variables adicionales. Ello evita el codificador estado-salidas, reduciendo las relaciones funcionales a un solo conjunto:



Ambas estructuras de autómata (Moore y Mealy) son funcionalmente equivalentes, si bien el autómata de Moore puede contener un número de estados considerablemente mayor. Por ello, el autómata de Mealy suele resultar mucho más simple en cuanto a descripción de la evolución del estado y a síntesis de las funciones correspondientes a las variables de estado; en cambio, en el autómata de Moore resultan más sencillas las funciones de activación de las salidas.

Veamos un interesante ejemplo de sistema secuencial, expresando su grafo en las dos formas de Mealy y Moore: *Un depósito se llena con una mezcla de cuatro líquidos diferentes, para lo cual dispone de cuatro electroválvulas A, B, C, D que controlan la salida de dichos líquidos y de cinco detectores de nivel n1, n2, n3, n4, n5 siendo n1 el inferior y n5 el de llenado máximo. Solamente cuando el nivel del depósito desciende por debajo del mínimo n1 se produce un ciclo de llenado: primero con el líquido A hasta el nivel n2, luego el líquido B hasta el nivel n3, el líquido C hasta n4 y, finalmente, D hasta completar el depósito n5.*

El grafo de estado correspondiente al autómata de Moore será el siguiente:



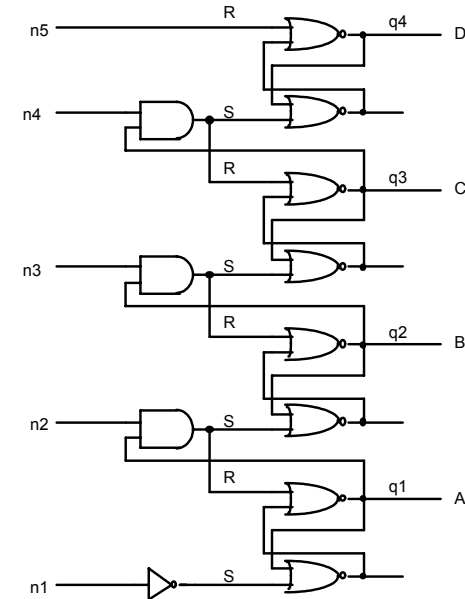
En el grafo anterior se utiliza para los estados el código de «un solo uno» con cuatro variables de estado $q_4 q_3 q_2 q_1$; de esta forma cada variable de estado coincide con una de las variables de salida (q_1 = llenado con líquido A; q_2 = líquido B; q_3 = líquido C; q_4 = líquido D). El marcado y borrado de cada variable de estado coincide, en este grafo circular y codificado con «un solo uno», con las transiciones de entrada y de salida al estado en que la correspondiente variable vale 1:

$$\begin{array}{ll} q_1: & S = \bar{n}_1 \quad R = q_1 \cdot n_2 \\ q_2: & S = q_1 \cdot n_2 \quad R = q_2 \cdot n_3 \\ q_3: & S = q_2 \cdot n_3 \quad R = q_3 \cdot n_4 \\ q_4: & S = q_3 \cdot n_4 \quad R = n_5 \end{array}$$

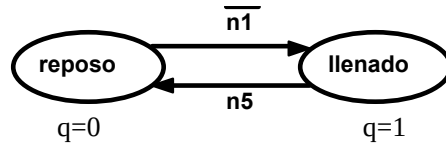
Obsérvese que para identificar cada estado solamente es necesaria una variable (ya que el código es de *un solo uno*).

S_1 no contiene el estado desde el que se marca $\bar{q}_4 \cdot \bar{q}_3 \cdot \bar{q}_2 \cdot \bar{q}_1$ ya que la transición con $n_1 = 0$ solamente puede darse desde dicho estado; lo mismo sucede con R_4 , pues $n_5 = 1$ sólo puede suceder desde el estado correspondiente a q_4 .

La siguiente figura representa el circuito resultante:



El mismo sistema secuencial puede configurarse con un número más reducido de estados, según el siguiente grafo que corresponde a un autómata de Mealy:



En este caso se necesita solamente una variable de estado q , pero las funciones de activación de las salidas resultan más complejas, pues dependen de las entradas, de la información que aportan los detectores de nivel:

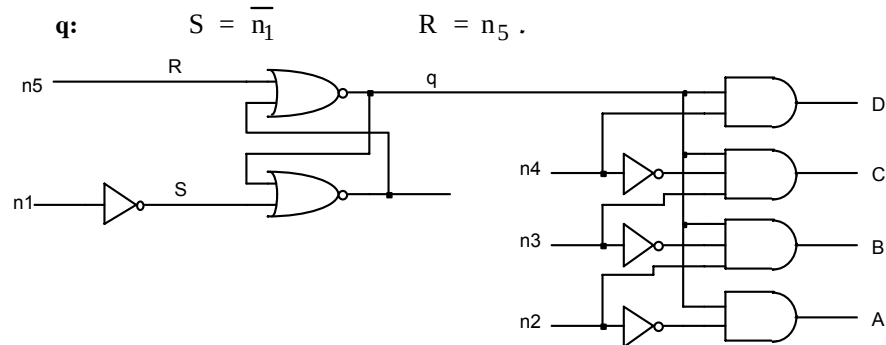
electroválvula A = $q \cdot \overline{n_2}$ líquido A hasta que se alcanza el nivel n_2

electroválvula B = $q \cdot n_2 \cdot \overline{n_3}$ líquido B desde el nivel n_2 hasta el nivel n_3

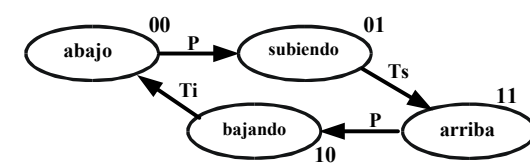
electroválvula C = $q \cdot n_3 \cdot \overline{n_4}$ líquido C desde el nivel n_3 hasta el nivel n_4

electroválvula D = $q \cdot n_4$ líquido D por encima del nivel n_4

y, según el sencillo grafo de estados anterior, la variable q pasa a valor **1** cuando $n_1=0$ y a valor **0** cuando $n_5=1$:



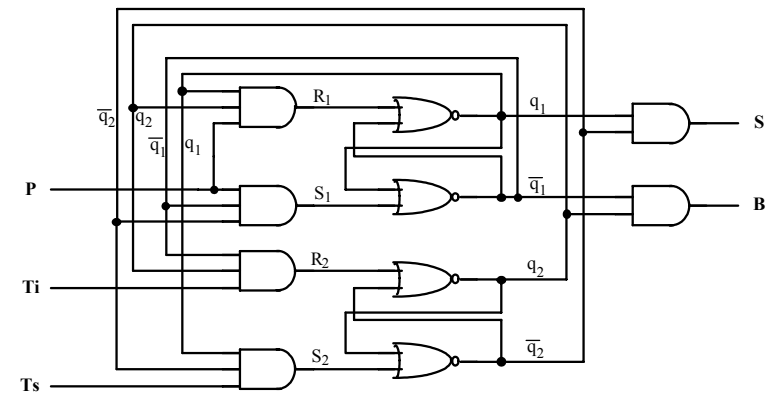
De igual forma, el primer ejemplo considerado en este capítulo, el control de la puerta de garaje, puede expresarse como autómata de Moore:



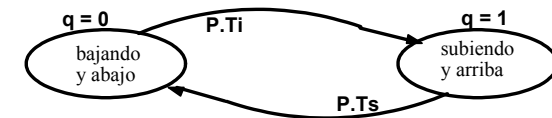
$$q_1: \quad S = P \cdot \overline{q_2} \cdot \overline{q_1} \quad R = P \cdot q_2 \cdot q_1$$

$$q_2: \quad S = Ts \cdot \overline{q_2} \cdot q_1 \quad R = T_i \cdot q_2 \cdot \overline{q_1}$$

$$S \text{ (subir)} = \overline{q_2} \cdot q_1 \quad B \text{ (bajar)} = q_2 \cdot \overline{q_1}$$

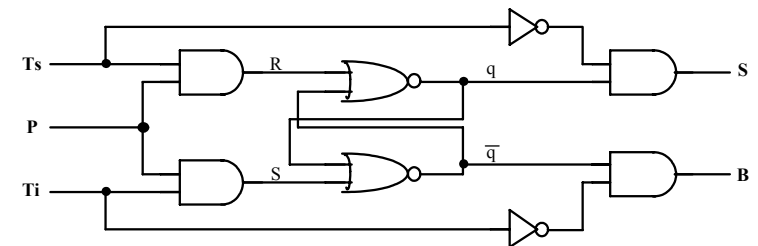


y como autómata de Mealy:



$$q_1: \quad S = P \cdot T_i \quad R = P \cdot T_s$$

$$S \text{ (subir)} = q \cdot \overline{T_s} \quad B \text{ (bajar)} = \overline{q} \cdot \overline{T_i}$$



12 INTRODUCCIÓN A LOS SISTEMAS SECUENCIALES II: DISEÑO

- 12.1. Simplificación de estados con los mismos efectos
- 12.2. Agrupación de estados que se distinguen por variables de entrada
- 12.3. Codificación de los estados
- 12.4. Diseño de circuitos secuenciales con biestables RS
- 12.5. Ejercicios de diseño secuencial

El proceso de diseño de un sistema secuencial parte de la delimitación de los estados necesarios para describirlo, para lo cual resultan sumamente útiles los grafos de estados. La reducción, cuando es posible, del número de estados, contribuye a simplificar el grafo y, en consecuencia, las funciones de evolución del estado. Dos son las situaciones que permiten la agrupación de varios estados en uno solo: cuando los estados tienen los mismos efectos (incluyendo en ellos, tanto las salidas como las transiciones que se producen desde ellos) y cuando no es preciso distinguir los estados entre sí, ya que se diferencian por los valores de algunas variables de entrada.

Una vez establecidos los estados de un sistema, el siguiente paso es «dar nombre» a cada estado: asignarle una palabra binaria. Luego, cada uno de sus dígitos (cada variable de estado) puede almacenarse sobre un biestable, de forma que el diseño secuencial consiste en obtener las funciones de marcado y de borrado de los mismos (biestables RS), junto con las funciones que producen el vector de salida. El sistema secuencial queda distribuido en tres partes: los biestables que contienen las variables de estado, las funciones de activación de dichos biestables y las funciones de activación de las salidas.

El diseño secuencial equivale a un recorrido a través de los diversos niveles de descripción del sistema: a partir de las especificaciones o requisitos se plantea un grafo de estados, desde el cual se obtienen las funciones de evolución de las variables de estado (marcado y borrado de sus biestables) y las de activación de las salidas del sistema para llegar al correspondiente circuito digital de biestables y puertas lógicas.

La síntesis de las funciones de marcado y borrado de los «biestables de estado» puede hacerse a través de la correspondiente tabla de verdad (entradas, estado $\rightarrow$ nuevo estado), según el método habitual de construcción de funciones booleanas; o bien, directamente desde el grafo de estados, identificando las transiciones que modifican el valor de cada variable de estado. La elaboración de la tabla funcional obliga a considerar todas y cada una de las situaciones posibles (para cada estado, cada uno de los vectores de entrada), muchas de las cuales no quedan reflejadas, en forma expresa, en el grafo de estado; además facilita la simplificación de las funciones.

Este capítulo, además de tratar en detalle la simplificación de estados y exponer el proceso de diseño de sistemas secuenciales, incluye un variado y significativo conjunto de ejemplos de diseño de los mismos.

12.1. Simplificación de estados con los mismos efectos

La representación de la evolución del estado del sistema sobre un grafo de estados sirve para determinar los estados necesarios; una vez establecido un grafo de estados, conviene considerar la posibilidad de simplificarlo, disminuyendo el número de estados que contiene.

La reducción del número de estados de un sistema secuencial, cuando tal cosa es posible, supone una simplificación de las funciones booleanas de evolución del estado, la cual resulta particularmente importante cuando permite disminuir el número de variables de estado (ello sucede siempre en codificación con «un solo uno» pero tiene aún mayor interés en código Gray y demás códigos de longitud mínima).

Existen dos mecanismos que permiten agrupar dos o más estados en uno solo y que se refieren a dos situaciones conceptuales, estados que no necesitan diferenciarse entre sí y estados que ya se distinguen por variables exteriores:

- a) son agrupables aquellos estados que no precisan diferenciarse entre sí, por tener las mismas salidas, es decir, los mismos vectores de salida y las mismas transiciones desde ellos;
- b) también son agrupables aquellos estados que, tanto ellos como las transiciones que se producen desde ellos, pueden diferenciarse mediante variables exteriores, es decir, mediante los valores de las variables de entrada.

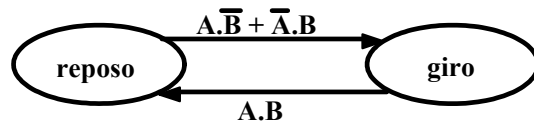
Los estados se refieren a situaciones (secuencias de vectores de entrada) que el sistema necesita conservar en su memoria en forma diferenciada (distinguiendo una situación de otra). Obviamente, no es preciso diferenciar aquellas situaciones que producen los mismos efectos: los mismos vectores de salida y las mismas transiciones. Tampoco es necesario distinguir internamente aquellas situaciones que se diferencian entre sí por el valor de alguna de las variables de entrada (o de varias de ellas). En el primer caso, se evita diferenciar situaciones que son idénticas por serlo sus efectos; en el segundo, puede prescindirse de distinguir en la memoria del sistema aquello que puede diferenciarse externamente.

Son agrupables (reducibles a uno solo) aquellos estados que producen los mismos efectos, es decir, tienen los mismos vectores de salida y las mismas transiciones desde ellos.

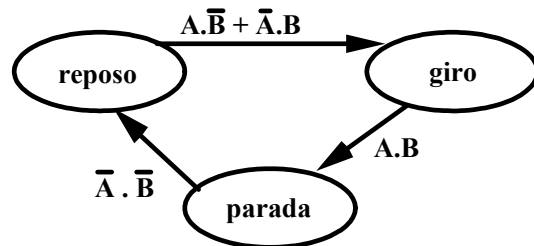
Consideremos un ejemplo muy simple: un motor con giro en sentido único controlado a través de dos pulsadores; activando uno cualquiera de ellos el motor se pone a girar y se para al presionar ambos pulsadores a la vez. Un posible grafo de estados sería el siguiente:



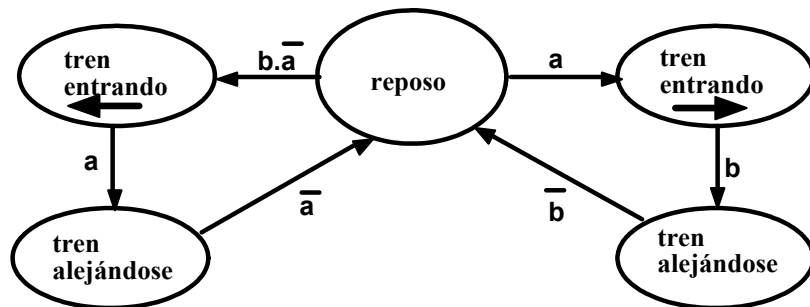
Ahora bien, los dos estados de *giro* tienen el mismo vector de salida (el motor girando en el mismo sentido) y la misma transición de salida (con $A.B$ pasan al reposo); por ello, equivalen a un solo estado:



Se ha propuesto este primer ejemplo por su sencillez, pero presenta un problema sobre el cual conviene razonar: al activar ambos pulsadores el motor se para, pero al soltarlos lo más probable es que un pulsador pase a 0 antes que el otro (los circuitos electrónicos son sensibles a diferencias de nanosegundos) y, entonces, se produce la condición de activación y el motor se pone a girar. El grafo adecuado para evitar esto requiere tres estados:

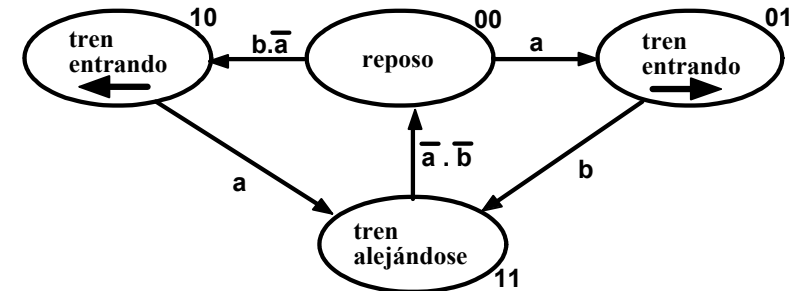


Un segundo ejemplo: *semáforo de aviso de paso de tren en un cruce de vía única bidireccional con un camino; la vía posee, a ambos lados del cruce y a una distancia adecuadamente grande, sendos detectores de paso de tren a y b; los trenes circulan por ella en ambas direcciones y se desea que el semáforo señale presencia de tren desde que éste alcanza el primer sensor en su dirección de marcha hasta que pasa por el segundo sensor tras abandonar el cruce.*



El estado relativo a *tren alejándose* es necesario pues, si se prescinde de dicho estado, un mismo tren al alejarse lleva al sistema al estado de reposo pero, inmediatamente después, será considerado como tren entrando en la dirección opuesta y causará una evolución incorrecta.

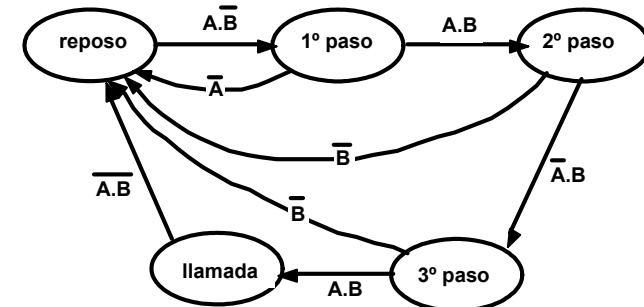
El grafo anterior, tal como está representado no es simplificable. Pero podemos modificar la condición booleana de las transiciones que salen de ambos estados inferiores (*tren alejándose*), de forma que ambas se produzcan con $\bar{a} \cdot \bar{b}$ (en el momento en que el tren abandona el cruce ambos detectores estarán a 0) y, entonces, ambos estados son reducibles a uno solo.



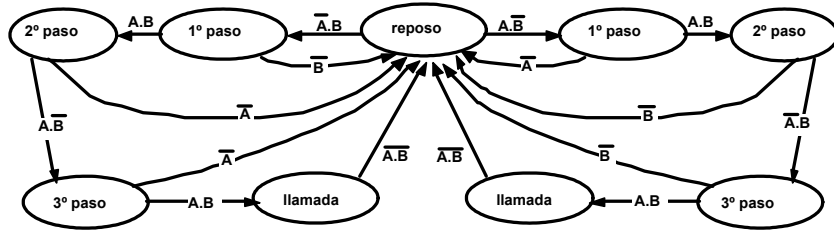
Aunque ambos grafos difieren en un solo estado, el primero de ellos necesita, al menos, tres variables de estado, mientras que el segundo puede codificarse con dos variables de estado.

En la práctica, en el diseño de un sistema secuencial a partir de unas especificaciones funcionales, apenas suele presentarse este tipo de estados con las mismas salidas; es raro que, al formular el correspondiente grafo de estados, el diseñador incluya dos estados que presenten los mismos efectos, pues generalmente los considerará ya de entrada como el mismo estado.

Consideremos un ejemplo que utilizaremos posteriormente para describir las formas de codificar estados: sea un timbre que dispone de dos pulsadores A y B, pero que solamente suena si se ejecuta la siguiente secuencia sobre ellos: A, A y B, B, A y B (se pulsa A; sin soltar, se pulsan ambos, A y B; se suelta A; y, por, último, sin soltar B, se vuelven a pulsar ambos, A y B). Su grafo de estados puede ser el siguiente:



Supongamos que, para evitar el tener que acordarse por cuál de los pulsadores hay que iniciar la secuencia, se admiten las dos secuencias simétricas posibles: (A, A y B, B, A y B) o, también, (B, A y B, A, A y B).



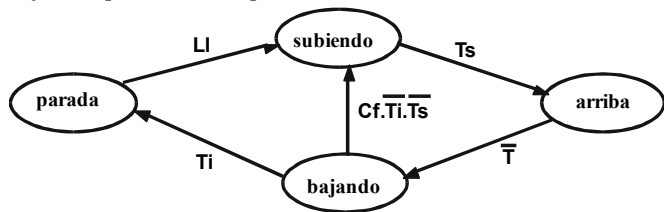
Los dos estados que corresponden a «llamada» en una y otra secuencia se pueden reducir a uno solo; el diagrama resultante puede ser codificado con tres variables de estado, mientras que el grafo sin simplificar necesitaba, al menos, cuatro variables.

12.2. Agrupación de estados que se distinguen por variables de entrada

También son agrupables (reducibles a uno solo) aquellos estados que, tanto ellos como las transiciones que desde ellos se producen, son distinguibles por los valores de las variables de entrada.

Consideremos una puerta de garaje de funcionamiento un poco más complejo que el detallado en el apartado 11.1: la puerta ha de abrirse al accionar una llave **LI** y debe permanecer arriba durante un tiempo prefijado, dado por una temporización **T**, transcurrido el cual la puerta se cierra; si durante la bajada una célula fotoeléctrica situada en la zona inferior **Cf** detecta la presencia de un objeto o persona, la puerta debe volver a subir reiniciando el ciclo de apertura.

Las entradas al sistema de control de esta puerta serán: la llave **LI**, los topes superior **T_s** e inferior **T_i** y la célula fotoeléctrica **Cf**, así como la salida de un circuito auxiliar de temporización **T**. Las salidas de dicho control serán los movimientos hacia arriba **S** y hacia abajo **B** y el disparo de la temporización **DT**.

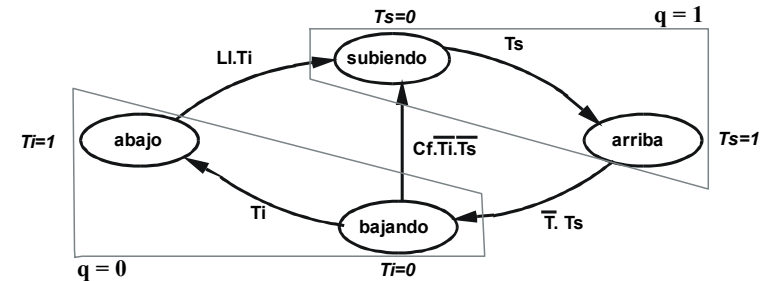


Los cuatro estados de este grafo requieren para su codificación, al menos, dos variables de estado **q₂ q₁**.

Pero en realidad no son necesarios cuatro estados: la necesidad de memoria se reduce a distinguir entre subida y bajada, pudiendo diferenciarse los otros dos estados mediante variables exteriores. El estado de reposo y el de bajada pueden distinguirse por el valor de la variable **T_i** (**T_i=1**, reposo; **T_i=0**, bajada); de igual forma la variable **T_s** permite diferenciar el estado de subida (**T_s=0**) y el de temporización (**T_s=1**).

Una vez agrupados tales estados es preciso comprobar que las transiciones que salen de ellos pueden diferenciarse, de forma que se produzcan correctamente:

- es necesario condicionar con **T_i** la transición que produce **LI** para que solamente se realice desde la situación de puerta parada abajo
- la transición que produce **Cf** se encuentra adecuadamente condicionada con **T_i**, de forma que no se efectúa cuando la puerta se encuentra ya parada abajo
- y, también, hay que añadir **T_s** a la transición determinada por **T** para que no se realice mientras la puerta se encuentra subiendo. De esta forma, las transiciones entre los dos nuevos estados, resultantes de la agrupación de los cuatro iniciales, se producen en las mismas situaciones que en el anterior grafo de cuatro estados.



Basta una variable de estado **q** para diferenciar los dos estados necesarios, que corresponden a la puerta subiendo o parada arriba (**q=1**) y puerta bajando o abajo (**q=0**):

$$q: \quad S = LI.T_i + Cf.T_i.T_s \quad R = T.T_s$$

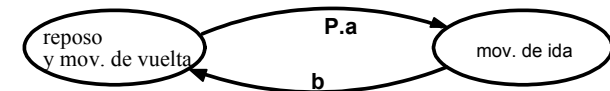
La activación de las salidas vendrá dada por las siguientes funciones:

$$S \text{ (subir)} = q.T_s \quad DT \text{ (disparo de la temporización)} = T_s \uparrow \text{ (al pasar de 0 a 1)}$$

$$B \text{ (bajar)} = \bar{q}.T_i$$

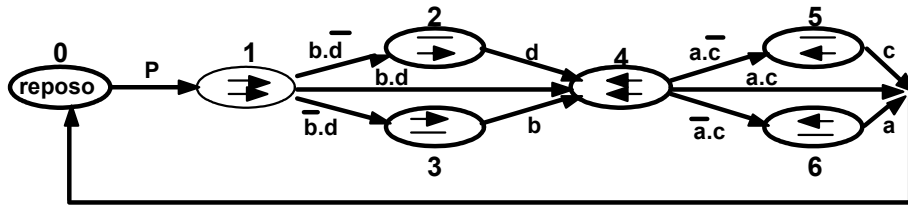
Otro ejemplo semejante: sea un pequeño carrito motorizado que recorre linealmente el camino entre dos puntos **A** y **B**, en los cuales existen sendos contactores «fin de camino» **a** y **b**; al activar un pulsador **P**, el carrito circula desde **A** hasta **B** y vuelve nuevamente a **A**.

Un primer grafo del comportamiento de dicho carrito puede incluir tres estados: reposo, movimiento hacia **B** y movimiento hacia **A**, pero el primero y el último son simplificables, ya que los diferencia la variable exterior **a** (reposo: **a=1**, movimiento hacia **A**: **a=0**). Basta, pues, una variable de estado **q**, cuyo marcado y borrado se producirán con **P.a** y con **b**, respectivamente; el movimiento hacia **B** coincidirá con la variable de estado **q** y el movimiento hacia **A** se producirá con $\bar{q}.a$.



Supóngase que, además del carrito anterior que se mueve entre A y B, un segundo carrito circula entre C y D (contactos «fin de carrera»: **c** y **d**), de forma que ambos carritos inician el movimiento desde A y C con el pulsador **P** y el primero en alcanzar el otro extremo B o D, espera a que el otro alcance el suyo, para iniciar juntos el movimiento de vuelta.

Un grafo detallado de este sistema de dos carritos puede incluir siete estados:



El análisis de la posibilidad de reducir el número de estados puede desarrollarse sistemáticamente considerando:

- 1 estados agrupables
2 variables que los diferencian
3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado.

En el anterior grafo de estado son posibles las siguientes agrupaciones:

- 1.1 estados agrupables: los estados 1, 2 y 3
 1.2 variables que los diferencian: **b y d** : **00, 10, 01**, respectivamente
 1.3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado:

d debe producirse desde el estado 2 (**b=1**) : **d.b**

b debe producirse desde el estado 3 (**d=1**) : **b.d**

el nuevo estado agrupado tendrá una transición de salida **d.b** (cuando se alcancen los dos topes de la derecha), lo cual es correcto.

- 2.1 estados agrupables: los estados 4, 5, 6 y 0
- 2.2 variables que los diferencian: **a y c : 00, 10, 01, 11**, respectivamente
- 2.3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado:

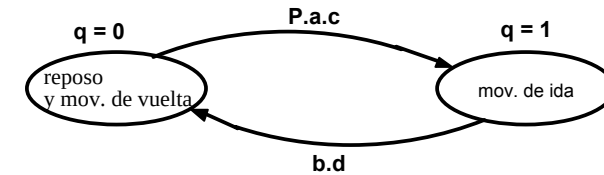
P debe producirse desde el estado 0 (**a=1 c=1**) : **P.a.c**

el estado agrupado tendrá una transición de salida **P.a.c**, lo cual es correcto.

Obsérvese que solamente es necesario considerar las transiciones que salen desde el nuevo estado agrupado y no las que se producen en el interior del mismo entre los estados que se agrupan.

En este caso, conforme al análisis desarrollado, son suficientes dos estados ya que el resto de las situaciones son distinguibles mediante las variables exteriores.

Resulta un grafo del tipo siguiente:



Este grafo corresponde a la necesidad de memoria que se limita a distinguir entre dos situaciones: el movimiento hacia B y D del movimiento hacia A y C; además, el grafo es el mismo (con solamente dos estados) aunque el número de carritos aumente.

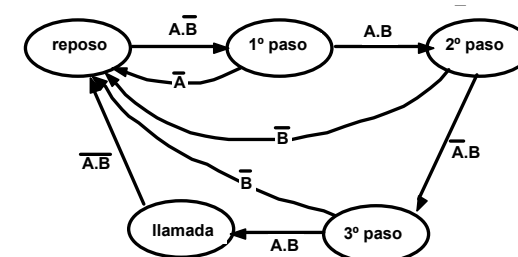
Las funciones de salida correspondientes a los movimientos de cada carrito serán:

- primer carrito: $\rightarrow = \text{mov. AB} = q \cdot \bar{b} \quad \leftarrow = \text{mov. BA} = \bar{q} \cdot \bar{a}$
 - segundo carrito: $\rightarrow = \text{mov. CD} = q \cdot \bar{d} \quad \leftarrow = \text{mov. DC} = \bar{q} \cdot \bar{c}$.

Este grafo de dos estados, en forma de autómata de Mealy, sirve también para cualquier número n de carritos; solamente se requiere diferenciar el movimiento de ida del de vuelta: las salidas referidas a cada uno de los carritos dependerán, además del estado (variable q) de la situación de los correspondientes contactos «fin de carrera».

Otro ejemplo de simplificación de estados que se distinguen por los valores de las variables de entradas es el considerado en el último apartado del capítulo anterior (apartado 11.4, páginas 30 – 32, depósito que se llena con una mezcla de cuatro líquidos diferentes): su grafo como autómatas de Moore presenta cinco estados, que se reducen a dos al tratarlo como autómatas de Mealy, debido a que cuatro de dichos estados pueden diferenciarse por las entradas detectoras de nivel (n_2, n_3, n_4, n_5).

Es conveniente insistir en la importancia que tiene el diferenciar adecuadamente las transiciones desde el estado agrupado. Volviendo al grafo de los dos pulsadores que activan un timbre a través de una secuencia **A, A y B, B, A y B**, (apartado anterior, página 36)



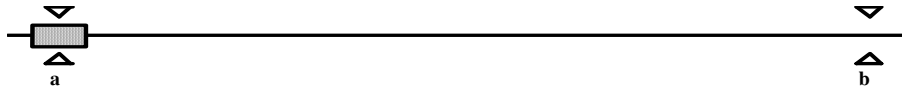
los estados correspondientes a 1° *paso* y a 2° *paso* pueden distinguirse a través de la variable **B** (**B=0** y **B=1**, respectivamente) pero al agruparlos no es posible diferenciar la transición producida por $\bar{B}$ de forma que solamente se efectúe desde el 2° *paso* (**B=1**: $\bar{B} \cdot B$ transición imposible).

También los estados de *reposo* y *1º paso* pueden distinguirse a través de la variable **A** (**A=0** *reposo* y **A=1** *1º paso*) pero al agruparlos no puede diferenciarse la transición producida por **A . B** de manera que sólo se efectúe desde el *1º paso* (por ejemplo, podríamos pulsar **B** y luego **A . B**, con lo cual la transición al *2º paso* se realizaría directamente desde el *reposo*, lo cual es incorrecto).

Ello se debe a que, precisamente, la misma variable exterior que distingue los estados es la que produce la transición. En este caso la necesidad de memoria se extiende a todos los pasos de la secuencia (**A**, **A y B**, **B**, **A y B**); al agrupar dos de ellos se reduce la longitud de la secuencia (si acaso podría prescindirse del último de los estados, el de llamada, y activar el timbre directamente desde el estado que corresponde a *3º paso* cuando la entrada sea **A.B**, tal como veremos en el próximo apartado 12.3.2).

Ejemplo con los dos tipos de simplificación de estados

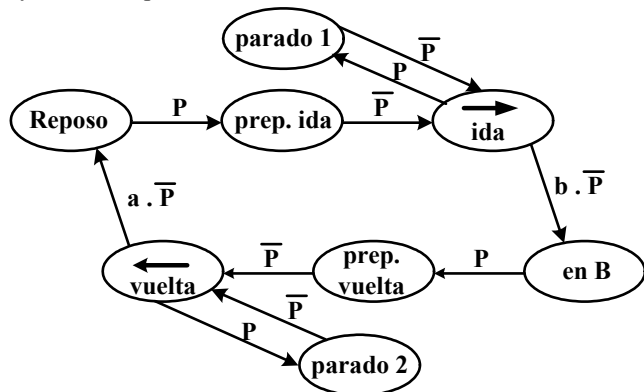
Un determinado mecanismo se mueve a lo largo de un riel entre dos posiciones **A** y **B** que se detectan mediante sendos sensores **a** y **b**:



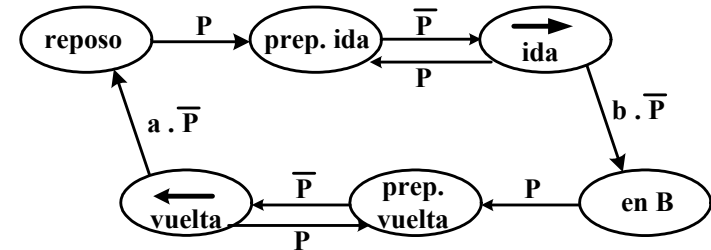
el movimiento se controla mediante un pulsador **P** de la siguiente manera:

- cuando el mecanismo se encuentra en **A** y se activa **P**, se inicia el movimiento hacia **B** al soltar el pulsador **P**
- de la misma forma, cuando se encuentra en **B** se inicia el movimiento hacia **A** al dejar libre el pulsador **P** (una vez activado)
- cuando el mecanismo se encuentra entre **A** y **B**, si se pulsa **P** el móvil se detiene y al soltar **P** continúa su movimiento anterior.

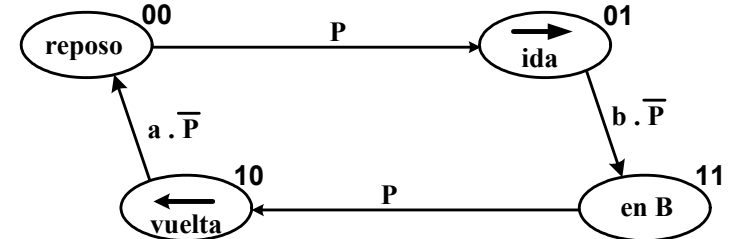
Un posible grafo de estados, en forma de autómatas de Moore con detalle de todas las situaciones posibles, es el siguiente, en el que las salidas corresponden directamente a los estados de *ida* y *vuelta*, respectivamente.



Pero los estados *preparado ida* y *parado 1* son idénticos ya que tienen los mismos efectos (salida nula y transición hacia el estado *ida* con **P** negado) y lo mismo sucede con los estados *preparado vuelta* y *parado 2* (salida nula y transición hacia el estado *vuelta* con **P** negado); de forma que ambas parejas de estados pueden agruparse según el grafo siguiente, que continúa siendo de Moore (salidas en correspondencia con los estados de *ida* y *vuelta*):



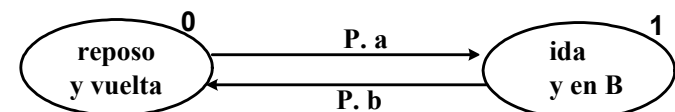
Ahora bien, los estados *preparado ida* e *ida* pueden diferenciarse por el valor de la entrada **P** (**P = 1** *preparado ida*, **P = 0** *ida*) y lo mismo sucede con los estados *preparado vuelta* y *vuelta* (**P = 1** *preparado vuelta*, **P = 0** *vuelta*); lo cual permite agrupar ambas parejas de estados:



En este caso, el autómata es de Mealy y las funciones de activación de las salidas son:

$$\text{ida} = \overline{q_2} \cdot q_1 \cdot \overline{P} \quad \text{vuelta} = q_2 \cdot \overline{q_1} \cdot \overline{P}.$$

Pero también, en este último grafo, los estados *ida* y *en B* pueden diferenciarse por el valor de una entrada (**b = 0** *ida*, **b = 1** *en B*); para agrupar ambos estados es necesario multiplicar la transición **P** que sale del nuevo estado por **b**, a fin de que se produzca desde la situación *en B*. Igualmente sucede con los estados *vuelta* y *reposo* (**P = 0** *vuelta*, **P = 1** *reposo*); ambos pueden agruparse multiplicando la transición **P** que sale del nuevo estado por **a**, para que se produzca desde *Reposo*. Resulta un grafo de Mealy, con solo dos estados:



Las funciones de activación de las salidas son:

$$\text{ida} = q \cdot \bar{P} \cdot \bar{b} \quad \text{vuelta} = \bar{q} \cdot \bar{P} \cdot \bar{a}$$

y la evolución de la variable de estado corresponde a las siguientes funciones de marcado y borrado:

$$S = P \cdot a \quad R = P \cdot b$$

12.3. Codificación de los estados

El grafo de estados establece los estados necesarios; conocidos los estados del sistema secuencial, el siguiente paso será asignar un código binario (un conjunto de valores de las variables de estado) a cada uno de los estados.

La codificación de los estados, es decir, la asignación de vectores de estado a los diversos estados, es un proceso arbitrario, con la única limitación de que a estados diferentes deben corresponder códigos binarios (vectores de estado) distintos. En principio el diseñador puede asignar libremente a los diversos estados las palabras binarias que desee o que considere más adecuadas.

Ahora bien, entre las múltiples codificaciones posibles de los estados, existen dos de particular interés:

- asignar a cada estado (excepto al estado inicial) una variable de estado, de forma que cada estado se corresponda biunívocamente con una de las variables de estado, resultando un código de «un solo uno» (cada vector de estado contiene un solo 1);
- utilizar para numerar los estados el código Gray, de forma que dos estados consecutivos difieran en una sola variable de estado (en cada transición se produce «un solo cambio» en las variables de estado).

La codificación con «un solo uno» (cada vector de estado contiene un solo 1) supone utilizar un biestable para cada estado (excepto para el estado inicial que será el 000...); en cambio, para código Gray en cada transición se produce «un solo cambio» en las variables de estado y, consiguientemente, solamente conmuta uno de los biestables.

El código de «un solo uno» resulta muy sencillo de manejar si el número de estados es reducido, mientras que el código Gray confiere una gran seguridad al funcionamiento del sistema secuencial, evitando evoluciones erróneas. Además, ambas formas de codificación de estados reducen la lógica necesaria para la activación de los biestables (reducen las funciones booleanas de evolución de las variables de estado).

Aplicaremos ambas formas de codificar estados al ejemplo del timbre con dos pulsadores **A** y **B** que suena al aplicar la secuencia: **A, A y B, B, A y B**.

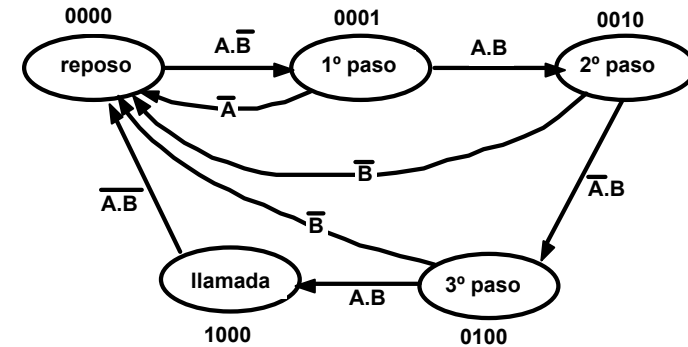
12.3.1. Codificación con un solo uno

El estado inicial o de reposo se codifica con el valor 000... y para el resto de los estados se hace coincidir una variable de estado con cada uno de ellos, de forma que el valor de dicha variable sea 1 para tal estado y 0 para el resto de ellos.

De esta forma se requiere un número de variables igual al número de estados menos uno; en cambio, se consigue que las funciones booleanas, tanto las de evolución de estado como las de salida, sean relativamente simples y fáciles de deducir directamente del grafo de estados.

Con esta codificación el ejemplo relativo al timbre con dos pulsadores requiere cuatro variables de estado **q4 q3 q2 q1** para representar sus cinco estados:

0000 reposo, 0001 primer paso, 0010 segundo paso, 0100 tercer paso, 0001 llamada.



La activación de la salida (sonido del timbre) coincide con el último estado y, por tanto, con una de las variables de estado **q4**. Para configurar el sistema secuencial con biestables RS, se asigna un biestable a cada variable de estado y las condiciones de marcado y borrado pueden obtenerse directamente del grafo de estados.

Resulta útil introducir la variable $q_0 = \text{Nor}(q_4, q_3, q_2, q_1)$, que corresponde al estado inicial 0000, es decir, adopta el valor 1 cuando $q_4=q_3=q_2=q_1=0$; de esta forma, todos los estados, incluido el inicial, se encuentran en correspondencia biunívoca con una variable de estado.

- evolución del estado:

$$q_1: \quad S = q_0 \cdot A \cdot \bar{B} \quad R = q_1 \cdot (\bar{A} + A \cdot B) = q_1 \cdot (\bar{A} + B)$$

$$q_2: \quad S = q_1 \cdot A \cdot B \quad R = q_2 \cdot (\bar{B} + \bar{A} \cdot B) = q_2 \cdot (\bar{A} + \bar{B})$$

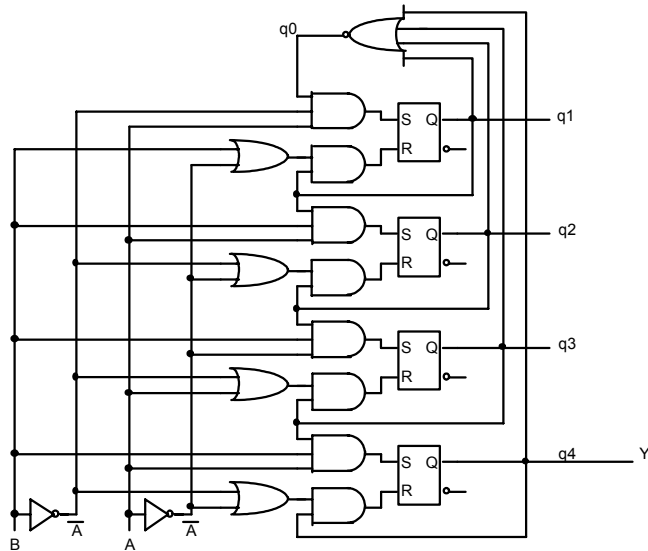
$$q_3: \quad S = q_2 \cdot \bar{A} \cdot B \quad R = q_3 \cdot (\bar{B} + A \cdot B) = q_3 \cdot (A + \bar{B})$$

$$q_4: \quad S = q_3 \cdot A \cdot B \quad R = q_4 \cdot (\bar{A} \cdot \bar{B})$$

- variable de salida:

$$\text{timbre} = q_4$$

La siguiente figura representa el circuito correspondiente a dichas funciones:



Obsérvese que las condiciones de marcado deben incluir la información sobre el estado anterior desde el que se produce el marcado; precisamente dicha información viene dada por la variable que corresponde al estado anterior y en el caso del estado inicial por el producto de las variables de estado negadas: $Nor(q_4, q_3, q_2, q_1) = q_0$.

Lo mismo sucede con las condiciones de borrado que, en general, deben incluir la información sobre el propio estado desde el que se produce el borrado; dicha información se reduce a la variable propia del estado, que es la que pasa a 0.

Se debe poner particular atención a las transiciones entre estados, considerando la posibilidad de que, por error, se marquen dos o más variables de estado o que la variable de estado anterior se borre tan rápidamente que no llegue a marcarse el estado siguiente.

Así puede suceder que al pasar de un estado al siguiente (por ejemplo de q_1 a q_2) el primero de los estados se borre tan rápidamente (q_1 con $A.B$) que no llegue a marcarse el siguiente (q_2); esto puede corregirse condicionando el borrado de una variable al hecho de que la variable de estado siguiente se haya marcado (para q_1 , $R = \bar{A} + B.q_2$).

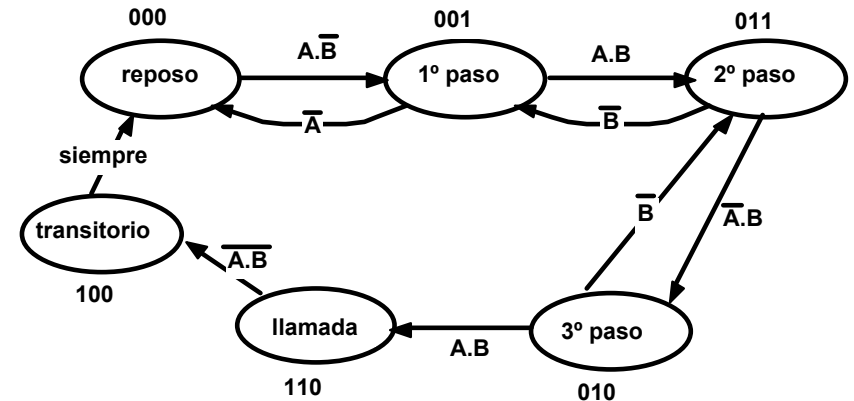
Ahora bien, esta forma de asegurar que el borrado de una variable no se produce hasta que no se marca la siguiente aumenta el riesgo de que se marquen dos variables a la vez. Todos estos problemas son propios de los sistemas asíncronos y no es posible resolverlos globalmente, sino que, en cada caso deben analizarse los riesgos de que tales problemas se produzcan; la forma de evitar todo este tipo de posibles errores pasa por codificar n código Gray o, aún mejor, por efectuar un diseño síncrono (que será estudiado en el capítulo 14).

12.3.2. Codificación con un solo cambio

La utilización del código Gray para numerar los estados permite que dos estados consecutivos se diferencien en el valor de una sola variable de estado, es decir, que en cada transición cambie solamente una variable de estado.

Con esta codificación el ejemplo anterior requiere tres variables de estado q_3 q_2 q_1 para representar sus cinco estados:

000 reposo, **001** primer paso, **011** segundo paso, **010** tercer paso, **110** llamada.



Ha sido preciso reorganizar las transiciones del grafo e introducir un estado adicional para asegurar que en todas las transiciones solamente cambia de valor una de las variables de estado. Puede comprobarse que el anterior grafo de estados es funcionalmente correcto y que cualquiera de las transiciones expresadas en el mismo modifica un sola variable de estados.

Asignando un biestable a cada una de las variables de estado, para obtener las condiciones de marcado y borrado ha de tenerse en cuenta el subconjunto de estados en que la correspondiente variable se encuentra a 1 y considerar las transiciones que conducen y que abandonan, respectivamente, dicho subconjunto.

Tanto en las condiciones de marcado como en las de borrado debe incluirse la información sobre el estado anterior desde el que se produce dicho marcado o borrado.

- evolución del estado:

$$q_1: S = \bar{q}_3 \cdot \bar{q}_2 \cdot \bar{q}_1 \cdot A \cdot \bar{B} + \bar{q}_3 \cdot q_2 \cdot \bar{q}_1 \cdot \bar{B} \quad R = \bar{q}_3 \cdot \bar{q}_2 \cdot q_1 \cdot \bar{A} + \bar{q}_3 \cdot q_2 \cdot q_1 \cdot \bar{A} \cdot B$$

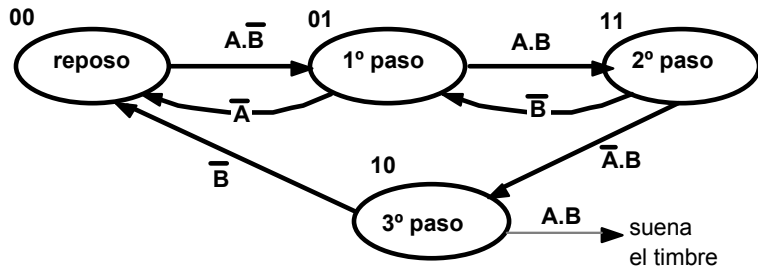
$$q_2: S = \bar{q}_3 \cdot \bar{q}_2 \cdot q_1 \cdot A \cdot B \quad R = \bar{q}_3 \cdot q_2 \cdot q_1 \cdot \bar{B} + q_3 \cdot q_2 \cdot \bar{q}_1 \cdot \bar{A} \cdot B$$

$$q_3: S = \bar{q}_3 \cdot q_2 \cdot \bar{q}_1 \cdot A \cdot B \quad R = q_3 \cdot \bar{q}_2 \cdot \bar{q}_1$$

- variable de salida:

tímbre = q_3

Hubiera sido correcto utilizar para este mismo sistema secuencial un grafo más reducido, conforme a la siguiente figura, ya que no es estrictamente necesario memorizar el último estado de la secuencia; basta con que se active el timbre desde el estado correspondiente a 3º paso cuando la entrada sea $A.B$,



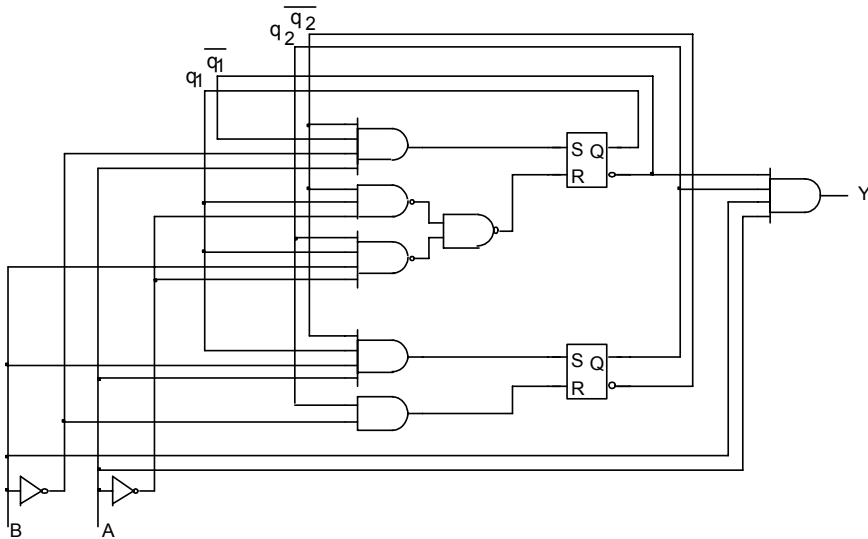
Obsérvese que en el estado final, si después de activar el timbre se deja de pulsar A y se mantiene pulsado B , permanece el mismo estado y puede activarse de nuevo el timbre al pulsar A ; en esto se diferencia este grafo del anterior, en el cual, al dejar de activar uno de los pulsadores A o B , el estado de llamada pasa directamente al estado de reposo.

- evolución del estado:

$$q_1: \quad S = \overline{q_2} \cdot \overline{q_1} \cdot A \cdot \overline{B} \quad R = \overline{q_2} \cdot q_1 \cdot \overline{A} + q_2 \cdot q_1 \cdot \overline{A} \cdot B$$

$$q_2: \quad S = \overline{q_2} \cdot q_1 \cdot A \cdot B \quad R = q_2 \cdot \overline{q_1} \cdot \overline{B} + q_2 \cdot q_1 \cdot \overline{B} = q_2 \cdot \overline{B}$$

- variable de salida: $\text{timbre} = q_2 \cdot q_1 \cdot A \cdot B$.



Cuando la codificación se ajusta por completo a un código Gray (un solo cambio en cada transición) se evita en gran medida la posibilidad de evoluciones erróneas debidas a diferencia en los tiempos de propagación de las funciones que marcan o borran las diversas variables.

Si el código no es Gray, en el paso de un estado a otro en que cambie el valor de más de una variable de estado puede suceder que la modificación de una de ellas sea más rápida que la del resto y se produzca un estado intermedio a partir del cual el sistema desarrolle una evolución errónea. Por ejemplo, en un sistema con dos variables de estado $q_2 q_1$, en el cual para $b=1$ se pase del estado 01 al 10 , siendo la variable q_1 más rápida en modificarse, se pasará primero al estado 00 ; puede suceder que dicho estado intermedio 00 no deba cambiar con $b=1$ y la transición a 10 no se complete, dando lugar a una transición errónea del 01 al 00 .

La codificación en código Gray requiere, en ocasiones, modificar el grafo de estados, añadiendo estados auxiliares para asegurar que en todas las transiciones se modifica una sola variable de estado; ello siempre es posible pero, en algunos casos, resulta muy complejo y supone un aumento significativo del número de estados. La alternativa, en tal caso, pasa por el sincronismo como mecanismo conceptual que proporciona seguridad funcional (al evitar la formación de estados intermedios).

12.4. Diseño de circuitos secuenciales con biestables RS

El proceso de síntesis o «construcción digital» de una función parte del enunciado o especificaciones de la misma, para configurar la «tabla de verdad» de la función y obtener, a través de ella, su expresión algebraica; una vez simplificada, dicha expresión puede ser directamente trasladada a un esquema de puertas como representación gráfica del circuito digital que «hace efectiva» dicha función.

enunciado → tabla funcional → expresión algebraica → esquema de puertas

De forma que el proceso de diseño de un sistema combinacional consiste en recorrer sucesivamente los diferentes niveles de descripción del mismo.

Un sistema secuencial admite, también, varios niveles y tipos de descripción:

- Nivel 1: Enumeración de las especificaciones o requisitos que se le exigen; es una descripción en términos de enunciado del correspondiente problema o proyecto, con indicación de todas aquellas prestaciones que interesan pero sin detallar los estados necesarios ni la evolución entre ellos.

- Nivel 2: Grafo de estados; descripción gráfica de los estados y de la evolución entre ellos, correspondiendo un círculo anotado a cada estado y un arco orientado a cada posible transición, con indicación sobre cada arco de la condición booleana (término de entrada) que produce la transición.

Por su propia definición conceptual dos estados no pueden estar en activo (marcados) a la vez; en cada momento el sistema se encuentra en uno de los estados del grafo, es decir, debe encontrarse marcado uno y solo uno de los estados del grafo y la *marca* pasará de un estado a otro a través de las transiciones (sin posibilidad de duplicarse ni de desaparecer).

Además del grafo de estados será necesario indicar la correspondencia de los estados con las salidas (que, si es autómata de Mealy, también dependerán de las entradas).

- Nivel 3: Funciones booleanas; expresión algebraica de las funciones de evolución de los estados y de activación de las salidas.

- Nivel 4: Configuración circuital en términos de biestables y de puertas lógicas; constituye el objetivo del proceso de diseño del sistema secuencial y consiste en la construcción circuital de las expresiones algebraicas de activación de las salidas y de las funciones de evolución del estado (con los correspondiente biestables).

El proceso de diseño de un sistema secuencial consiste en recorrer sucesivamente los cuatro niveles de descripción:

**especificaciones/
/requisitos** → **grafo
de estados** → **funciones
booleanas** → **realización
circuital**

El paso de grafo de estados a funciones booleanas contiene dos etapas previas: la simplificación del número de estados por agrupación de estados equivalentes y la codificación binaria de los estados:

→ **grafo
de estados** → **simplificación
de estados** → **codificación
de los estados** → **funciones
booleanas** →

En muchos casos la acertada realización de estas etapas, simplificación y codificación, puede reducir en gran medida la complejidad de las funciones booleanas.

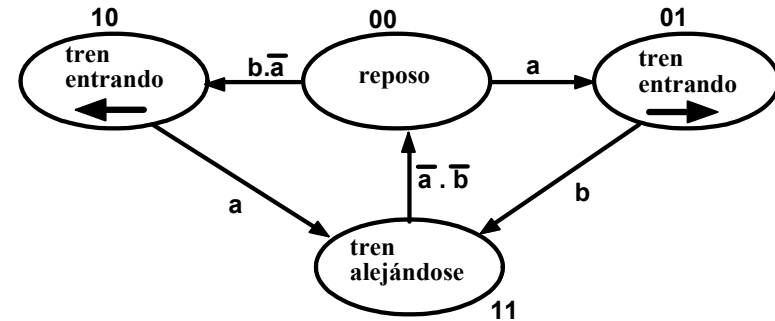
Si se utiliza un biestable RS para almacenar cada variable de estado, las funciones de evolución del estado se dividen en dos partes:

- las funciones de activación de los biestables (marcado)
- y las funciones de desactivación de los mismos (borrado).

A tales funciones es preciso añadir las funciones de activación de las salidas. Los tres conjuntos de funciones son de tipo combinacional (sin realimentación: la memoria es conformada por los biestables).

Las funciones de marcado y borrado de los biestables que contienen las variables de estado pueden obtenerse directamente de los arcos que representan las transiciones en el grafo de estados: para cada variable de estado ha de tenerse en cuenta el subconjunto de estados en que la correspondiente variable se encuentra a **1** y considerar las transiciones que conducen (marcado) y que abandonan (borrado) dicho subconjunto.

Consideremos el ejemplo de un semáforo de aviso de paso de tren en un cruce de vía única bidireccional con un camino (apartado 12.1):



La variable q_1 está activada en los estados **01** y **11**, de forma que se marca cuando pasa a ellos desde los otros estados (**00** y **10**) y se borra al pasar del estado **11** al **00**: se marca con a desde **00** y, también, con a desde **10** y se borra con $\bar{a} \cdot \bar{b}$ desde **11**.

$$q_1: \quad S = \bar{q}_2 \cdot \bar{q}_1 \cdot a + q_2 \cdot \bar{q}_1 \cdot a = \bar{q}_1 \cdot a \quad R = q_2 \cdot q_1 \cdot \bar{b} \cdot \bar{a}.$$

La variable q_2 se encuentra activa en los estados **10** y **11**: se marca con $b \cdot \bar{a}$ desde **00** y con b desde **01** y se borra con $\bar{a} \cdot \bar{b}$ desde **11**.

$$q_2: \quad S = \bar{q}_2 \cdot \bar{q}_1 \cdot b \cdot \bar{a} + \bar{q}_2 \cdot q_1 \cdot b = \bar{q}_2 \cdot b \cdot (\bar{a} + q_1) \quad R = q_2 \cdot q_1 \cdot \bar{b} \cdot \bar{a}.$$

La salida (el semáforo) debe encontrarse activa en los estados **01**, **10** y **11**:

$$\text{semáforo en rojo} = \bar{q}_2 \cdot q_1 + q_2 \cdot \bar{q}_1 + q_2 \cdot q_1 = q_2 + q_1.$$

Ahora bien, si el número de variables (entrada + estado) no es elevado conviene obtener las condiciones de marcado y borrado a partir de la tabla funcional correspondiente a la evolución de las variables de estado: $Q = F(X, Q)$.

En la misma tabla funcional de evolución de los estados pueden expresarse las funciones de marcado y borrado de cada una de las variables de estado; tales funciones se deducen a partir de los cambios que sufren las correspondientes variables de estado según las siguientes consideraciones:

- cuando el valor previo de q_i es **0** y dicho valor permanece, se requiere que $S_i=0$ y, en cambio, no importa que valor adopte R_i (con **0** continúa el valor anterior y con **1** se borra, lo cual es lo mismo);
- cuando el valor previo de q_i es **1** y dicho valor permanece, es necesario que $R_i=0$ y, en cambio, no importa que valor adopte S_i (con **0** continúa el valor anterior y con **1** se marca);
- cuando el valor previo de q_i es **0** y pasa a valor **1**, se requiere que $S_i=1$ y $R_i=0$;
- cuando el valor previo de q_i es **1** y pasa a valor **0**, se requiere que $S_i=0$ y $R_i=1$.

$q_i:$	$0 \rightarrow 0$	$R_i =$	X	$S_i =$	0
	$0 \rightarrow 1$		0		1
	$1 \rightarrow 0$		1		0
	$1 \rightarrow 1$		0		X

Para el ejemplo anterior, semáforo de aviso de paso de tren, la tabla de evolución del estado, ampliada con las columnas de marcado **S** y borrado **R**, es la siguiente:

q_2	q_1	b	a	q_2^+	q_1^+	R_2	S_2	R_1	S_1
0	0	0	0	0	0	X	0	X	0
		0	1	0	1	X	0	0	1
		1	0	1	0	0	1	X	0
		1	1	0	1	X	0	0	1
0	1	0	0	0	1	X	0	0	X
		0	1	0	1	X	0	0	X
		1	0	1	1	0	1	0	X
		1	1	1	1	0	1	0	X
1	0	0	0	1	0	0	X	X	0
		0	1	1	1	0	X	0	1
		1	0	1	0	0	X	X	0
		1	1	1	1	0	X	0	1
1	1	0	0	0	0	1	0	1	0
		0	1	1	1	0	X	0	X
		1	0	1	1	0	X	0	X
		1	1	1	1	0	X	0	X

La elaboración de la tabla de las funciones de evolución del estado obliga a decidir para cada estado y para cada vector de entrada, es decir, para cada una de las situaciones posibles, cuál es la transición más adecuada, evitando errores de diseño por no haber previsto todas las situaciones que pueden presentarse; además, la citada tabla permite realizar las posibles simplificaciones a que hubiere lugar.

A partir de la tabla funcional, se obtienen las expresiones algebraicas de las funciones de marcado y borrado de cada uno de los biestables que conforman las variables de estado del sistema, pudiendo simplificarlas mediante mapas de Karnaugh.

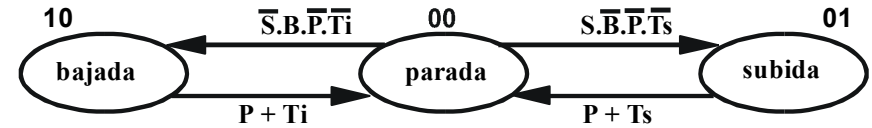
En la tabla anterior, dichas funciones, una vez simplificadas serán:

$$\begin{aligned} q_1: \quad S_1 &= a & R_1 &= q_2 \cdot \bar{b} \cdot \bar{a} \\ q_2: \quad S_2 &= b \cdot (\bar{a} + q_1) & R_2 &= q_1 \cdot \bar{b} \cdot \bar{a} \end{aligned}$$

Estas funciones son algo más reducidas que las expresadas anteriormente, ya que la construcción de la correspondiente tabla funcional facilita su simplificación.

12.5. Ejercicios de diseño secuencial

12.5.1. Montacargas con pulsadores de subida **S**, bajada **B** y paro **P**: para modificar el sentido de la marcha es preciso activar previamente el pulsador de paro y no responde a la activación simultánea de varios pulsadores; sendos toques fin de carrera T_i y T_s le impiden continuar subiendo o bajando cuando alcanza los extremos del recorrido.



La codificación de los estados utiliza «un solo uno», correspondiendo cada variable a una de las direcciones del movimiento del montacargas (subida = q_2 , bajada = q_1). A partir de este grafo de estados pueden expresarse directamente las funciones de marcado y de borrado de los biestables correspondientes a ambas variables de estado q_1 y q_2 .

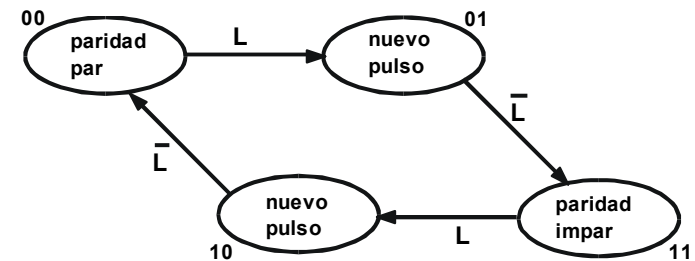
Sea $q_0 = q_2 \Delta q_1$, cuyo valor en el estado de **parada** es 1

$$q_1: \quad S_1 = q_0 \cdot \bar{S} \cdot \bar{B} \cdot \bar{P} \cdot \bar{T}_s \quad R_1 = P + T_s$$

$$q_2: \quad S_2 = q_0 \cdot \bar{S} \cdot \bar{B} \cdot \bar{P} \cdot \bar{T}_i \quad R_2 = P + T_i$$

En este caso, las funciones de borrado no necesitan incluir la correspondiente variable de estado.

12.5.2. Indicador de paridad relativo al número de pulsos que llegan por una determinada línea **L**, de forma que indica paridad par $p=0$ cuando ha llegado un número par de pulsos y paridad impar $p=1$ cuando dicho número es impar; los pulsos se cuentan una vez que han finalizado, es decir, en su bajada ($\downarrow$).



Se han codificado los estados en código Gray; aunque algunos estados son distinguibles por la variable exterior **L** (01 y 11 se distinguen por $L=1$ y $L=0$ e igualmente 10 y 00) no se puede reducir el grafo de estados ya que es precisamente dicha variable la que produce las transiciones.

• función de salida: $p = q_2$.

- evolución del estado:

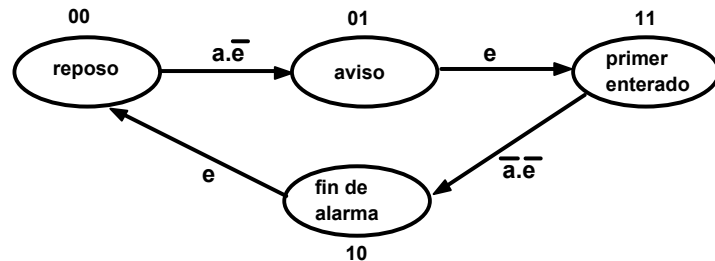
$$q_1: S_1 = \overline{q_2} \cdot \overline{q_1} \cdot L \quad R_1 = q_2 \cdot q_1 \cdot L$$

$$q_2: S_2 = \overline{q_2} \cdot q_1 \cdot \overline{L} \quad R_2 = q_2 \cdot \overline{q_1} \cdot \overline{L}$$

Este grafo de estados coincide con el de control de una lámpara de mesa (*flexo*) considerado en el apartado 11.2 (página 19); asimismo corresponde al grafo de estados del biestable tipo **T**.

12.5.3. Un sistema secuencial actúa de la forma siguiente: cuando aparece una señal de alarma (entrada $a=1$) suena un claxon c y se enciende una lámpara l hasta que el operario pulsa una entrada de enterado e ; con ello se apaga el claxon c pero, si la alarma sigue activa ($a=1$), la lámpara l sigue encendida hasta que desaparezca la alarma. En todo caso, cuando desaparece la alarma ($a=0$) la lámpara l se pone intermitente hasta que el operario vuelve a pulsar por segunda vez el contacto de enterado e . Este comportamiento permite al operario, una vez enterado de la situación de alarma y si ésta persiste, dedicarse a resolver tal situación; finalizada la misma, la lámpara se mantiene intermitente para recordar al operario que debe anotar la hora de la misma en un libro de registros, antes de pulsar un segundo enterado.

El grafo correspondiente a la evolución de los estados puede reducirse a cuatro estados (agrupando adecuadamente aquellos que se diferencian por la variable e):



Los estados han sido codificados en código Gray con dos variables; construyendo la tabla de evolución de estados y, sobre ella, las columnas R_i y S_i , se obtienen las siguientes funciones de evolución del estado simplificadas:

$$q_1: S_1 = \overline{q_2} \cdot a \cdot \overline{e} \quad R_1 = q_2 \cdot a \cdot \overline{e}$$

$$q_2: S_2 = q_1 \cdot e \quad R_2 = \overline{q_1} \cdot e$$

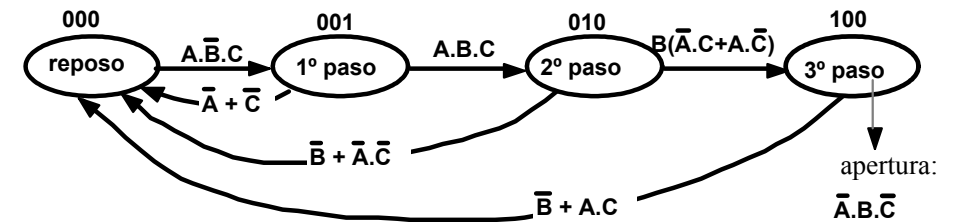
Las funciones de activación de las salidas serán las siguientes:

$$\text{claxon } c = \overline{q_2} \cdot q_1 \quad \text{luz } l = a + i \cdot (q_2 + q_1);$$

la variable i corresponde a una entrada auxiliar que genera la intermitencia (una onda cuadrada de baja frecuencia, por ejemplo, medio segundo en **1** y otro medio en **0**).

12.5.4. Secuencia sobre tres pulsadores para abrir una puerta: la apertura de una puerta está controlada por tres pulsadores A , B y C , de forma que se abre cuando se ejecuta sobre ellos la siguiente secuencia: 1º A y C (pulsadores de los extremos), 2º A y B y C (los tres pulsadores), 3º B y C o A y B indistintamente (dos pulsadores contiguos), 4º B (pulsador central); se progresa en la secuencia sin «soltar» los pulsadores y la puerta permanece abierta mientras se mantiene B pulsado.

El grafo de estados necesario puede reducirse a cuatro estados, agrupando en el último de ellos el tercer y cuarto paso de la secuencia:



Las transiciones de vuelta al estado inicial son las siguientes: el primer paso se anula cuando no se produce $A \cdot \overline{B} \cdot C$ ni $A \cdot B \cdot C$; el segundo paso lo hace cuando no están presentes $A \cdot B \cdot C$ ni tampoco $B \cdot (\overline{A} \cdot C + A \cdot \overline{C})$; el tercer estado pasa a reposo cuando $B = 0$ o cuando se pulsán conjuntamente A y C (ya que, en otro caso, se encontrará bien en una de las condiciones de transición a dicho estado, o bien en la situación de apertura de la puerta). Se ha utilizado la codificación con «un solo uno».

- $q_0 = \text{Nor}(q_3, q_2, q_1)$ corresponde al estado **000**

- evolución del estado:

$$q_1: S_1 = q_0 \cdot A \cdot \overline{B} \cdot C \quad R_1 = q_1 \cdot (\overline{A} + \overline{C} + A \cdot B \cdot C) = q_1 \cdot (\overline{A} + B + \overline{C})$$

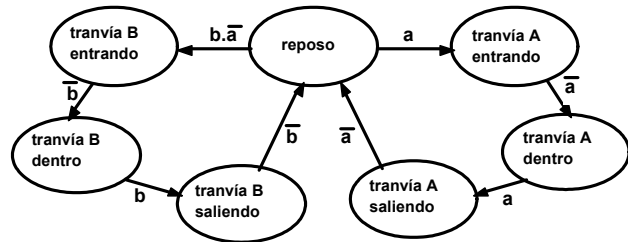
$$q_2: S_2 = q_1 \cdot A \cdot B \cdot C \quad R_2 = q_2 \cdot (\overline{A} + \overline{B} + \overline{C})$$

$$q_3: S_3 = q_2 \cdot B \cdot (\overline{A} \cdot C + A \cdot \overline{C}) \quad R_3 = q_3 \cdot (\overline{B} + A \cdot C)$$

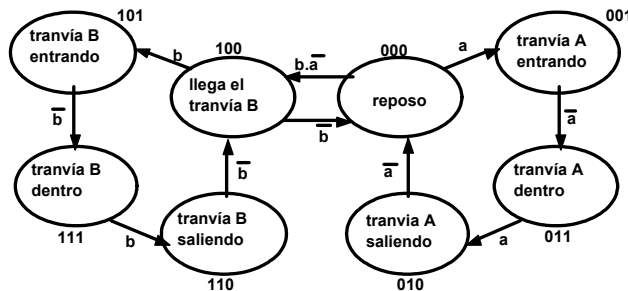
- función de salida: $\text{apertura de la puerta} = q_3 \cdot \overline{A} \cdot B \cdot \overline{C}$.

12.5.5. *Árbitro para la utilización de un recurso común: dos vías de tranvía (de ida y vuelta cada una de ellas), procedentes respectivamente de A y de B se unen en un punto C, de forma que tienen un tramo común desde C hasta D (dicho tramo CD es terminal: una vez que es ocupado por un tranvía, no puede entrar ningún otro, ya que el primero ha de salir por C); previamente al punto C existen sendos detectores de presencia de tranvía (**a** y **b**) y sendos semáforos (**S_a** y **S_b**) para detener a un tranvía cuando el trayecto común se encuentra ocupado por el otro.*

Un grafo de estados de la evolución de este sistema de arbitraje puede ser:



Este grafo incluye un número de estados relativamente alto; para su codificación, caso de utilizar una variable para cada estado (*un solo uno*), son necesarias seis variables de estado. En cambio, utilizando código Gray bastarán tres variables de estado; para ello, (para que en cada transición solamente cambie el valor de una de las variables) es necesario añadir un estado adicional, según el siguiente grafo de estados:



Se ha añadido el estado **100** que separa la evolución correspondiente al tranvía A de la que corresponde al tranvía B.

• funciones de salida:

semáforo **S_a** en rojo (vía común ocupada por B: detención del tranvía A) $S_a = q_3$

semáforo **S_b** en rojo (vía común ocupada por A: detención del tranvía B) $S_b = \overline{q_3}$.

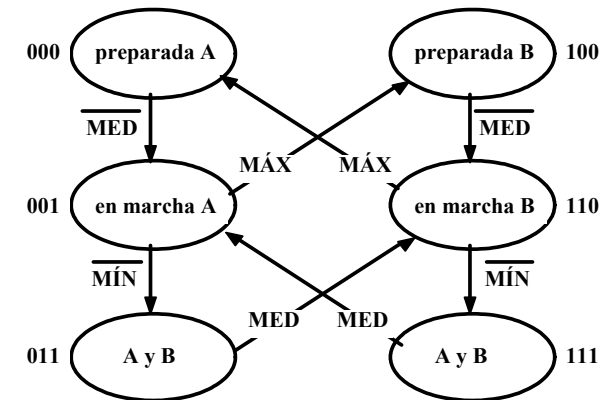
El grafo da prioridad al tranvía A: el semáforo **S_b** se encontrará en rojo en reposo y pasará a verde cuando llegue el tranvía B a su detector, si el otro tranvía no ha llegado al suyo ni tampoco se encuentra dentro del tramo común. La asignación de prioridad evita colisiones en caso de que ambos tranvías alcancen a la vez sus respectivos sensores.

• evolución del estado:

$$\begin{aligned} q_1: S_1 &= \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot a + q_3 \cdot \overline{q_2} \cdot \overline{q_1} \cdot b & R_1 &= \overline{q_3} \cdot q_2 \cdot q_1 \cdot a + q_3 \cdot q_2 \cdot q_1 \cdot b \\ q_2: S_2 &= \overline{q_3} \cdot \overline{q_2} \cdot q_1 \cdot \overline{a} + q_3 \cdot \overline{q_2} \cdot q_1 \cdot \overline{b} & R_2 &= \overline{q_3} \cdot q_2 \cdot q_1 \cdot \overline{a} + q_3 \cdot q_2 \cdot q_1 \cdot \overline{b} \\ q_3: S_3 &= \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot b \cdot \overline{a} & R_3 &= q_3 \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{b}. \end{aligned}$$

12.5.6. *Un depósito de agua dispone de dos bombas A y B para su llenado y de tres detectores de nivel MÁXimo, MEDio y MÍNimo. Cuando el nivel cae por debajo del nivel medio entra en funcionamiento una de las bombas hasta que alcanza el máximo; por debajo del mínimo actúan ambas bombas hasta el nivel medio y luego una sola de ellas hasta el máximo. Para equilibrar el desgaste de las bombas, cuando funciona una sola, lo hacen alternativamente (es decir, si la vez pasada lo ha hecho la bomba A, entrará en funcionamiento la B y viceversa).*

El grafo de estados debe ser simétrico respecto a la actuación de las bombas A y B; debe recordar cual de ambas ha actuado anteriormente y, por tanto, a cual de ellas le corresponde la siguiente puesta en marcha:



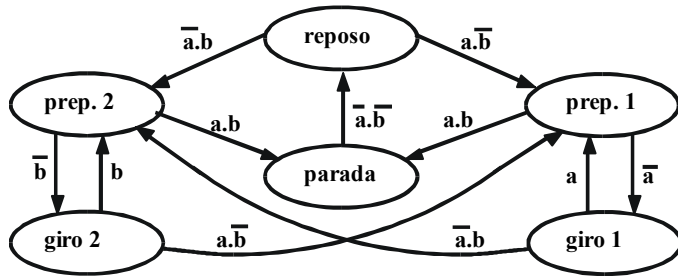
La codificación de estados indicada en este grafo (que no es ni Gray, ni de «un solo uno») resulta adecuada porque permite identificar las variables **q₁** y **q₂** con las bombas de agua A y B: activación de **A** = **q₁**; activación de **B** = **q₂**.

• evolución del estado:

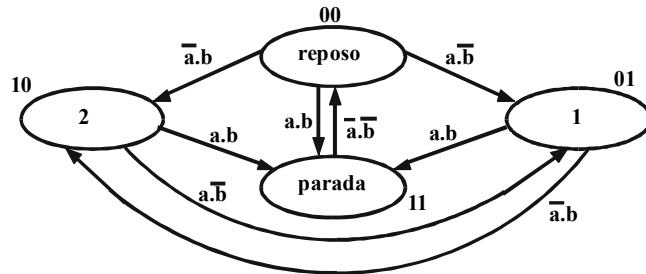
$$\begin{aligned} q_1: S_1 &= \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{\text{MED}} + \overline{\text{MÍN}} & R_1 &= \overline{q_3} \cdot \text{MED} + \text{MÁX} \\ q_2: S_2 &= q_3 \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{\text{MED}} + \overline{\text{MÍN}} & R_2 &= q_3 \cdot \text{MED} + \text{MÁX} \\ q_3: S_3 &= \overline{q_3} \cdot \text{MÁX} + q_3 \cdot \overline{q_2} \cdot q_1 \cdot \text{MED} & R_3 &= q_3 \cdot \text{MÁX} + q_3 \cdot q_2 \cdot q_1 \cdot \text{MED}. \end{aligned}$$

12.5.7. Un motor puede girar en ambas direcciones, controlado por dos pulsadores **a** y **b** en la forma siguiente: al pulsar **a** se detiene el motor (si es que estaba en movimiento) y, al soltar dicho pulsador, el motor se pone a girar en el sentido de las agujas del reloj; lo mismo sucede al pulsar **b**, pero al soltarlo, el motor girará en sentido contrario a las agujas del reloj; la forma de detener el movimiento del motor es pulsar **a** y **b** a la vez, en cuyo caso el motor se para y, al soltar los pulsadores, continuará parado.

El grafo de estados, como autómata de Moore, requiere 6 estados:



Pero los estados *prep. 1* y *giro 1* pueden distinguirse por el valor de la variable **a** e, igualmente, los estados *prep. 2* y *giro 2* pueden distinguirse por el valor de la variable **b**, de manera que el grafo puede reducirse, en forma de autómata de Mealy, a 4 estados:



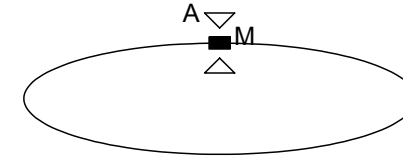
Las funciones de salida correspondientes a los dos giros se producen en los estados **01** y **10** pero, solamente, cuando no están activados los pulsadores:

$$\text{giro 1} = \overline{q_2} \cdot q_1 \cdot \overline{a} \quad \text{giro 2} = q_2 \cdot \overline{q_1} \cdot \overline{b}$$

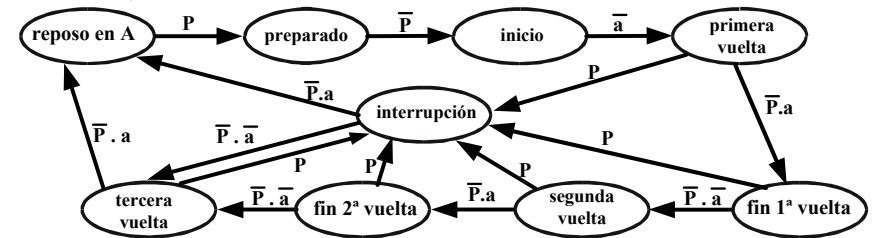
Las funciones de evolución del estado se han obtenido a través de su correspondiente tabla funcional (en la cual, para facilitar la simplificación, se ha hecho que el estado **00** si se activan ambos pulsadores a la vez pase al estado **11**); una vez construidas las columnas **R_i** y **S_i** y simplificadas las condiciones de marcado y borrado son las siguientes:

$$\begin{aligned} q_1: \quad S_1 &= a & R_1 &= q_2 \cdot \overline{a} \cdot \overline{b} + \overline{q_2} \cdot \overline{a} \cdot b = \overline{a} \cdot (q_2 \oplus b) \\ q_2: \quad S_2 &= b & R_2 &= q_1 \cdot \overline{a} \cdot \overline{b} + \overline{q_1} \cdot a \cdot \overline{b} = \overline{b} \cdot (q_1 \oplus a) \end{aligned}$$

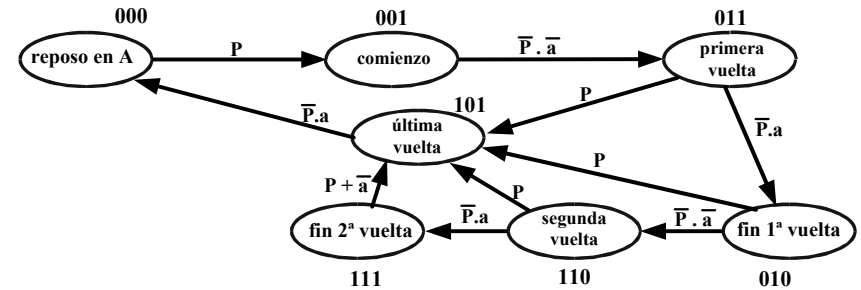
12.5.8. Cierta mecanismo **M** se desplaza en el sentido de las agujas del reloj por un riel elipsoidal a partir de la posición **A**, en la cual existe un sensor que detecta la presencia del mismo: si se activa un pulsador **P**, **M** inicia su movimiento al soltar **P** y da 3 vueltas completas, tras lo cual se detiene en **A**; si durante el transcurso del movimiento se vuelve a pulsar **P**, **M** se para y, al soltar el pulsador, completa la vuelta que esta dando y se detiene al llegar a **A** (en todo caso, al menos da una vuelta).



Un posible grafo de estados sería el siguiente (una vez iniciado el movimiento, si se pulsa **P** se va al estado de interrupción y desde dicho estado, al soltar **P**, se completa la vuelta hasta **A**):



Los estados *preparado* e *inicio* pueden agruparse ya que se diferencian por el valor de **P** y, por lo mismo, pueden agruparse los estados *interrupción* y *tercera vuelta* (de esta forma quedan 7 estados, codificables con 3 variables de estado):



La tabla de evolución de estados presenta solamente 5 variables: 3 de estado **q₃ q₂ q₁** y 2 de entrada **P a** (se deja como ejercicio para el lector). La salida (movimiento del mecanismo) debe activarse en todos los estados menos en el de *reposo*; en los estados de *comienzo* y de *última vuelta* se activa solamente cuando **P** = **0**:

$$\text{movimiento de M} = q_2 + q_1 \cdot \overline{P}$$

13 SINCRONISMO Y BIESTABLES SÍNCRONOS

SECUENCIADORES LÓGICOS PROGRAMABLES

- 13.1. Sincronismo y configuración amo/esclavo: biestables síncronos
- 13.2. Registros síncronos
- 13.3. Tiempos funcionales e inicialización de los biestables
- 13.4. Dispositivos lógicos programables: PAL con biestables
- 13.5. Los biestables en VHDL

«Dividir en partes» es la forma típica de abordar la complejidad. El sincronismo es una partición del tiempo que lo divide en unidades: supone que la variable tiempo deja de ser continua y pasa a variable discreta, fraccionada en unidades numerables.

Un sistema secuencial síncrono es aquel cuyos cambios de estado se producen solamente en los momentos de paso de una unidad de tiempo a la siguiente; tales instantes vendrán definidos por los flancos de una señal digital que cuantifica el tiempo en unidades y a la que, genéricamente, denominaremos reloj.

Para construir sistemas secuenciales síncronos necesitamos biestables síncronos, cuyos cambios de estado se produzcan solamente en el flanco activo de la señal de reloj. Aparece un nuevo «elemento de entrada» que no es ni el 0 ni el 1, sino el paso de 0 a 1 (flanco de subida); para conseguir un biestable «habilitado por flancos» es necesario utilizar dos biestables asíncronos, uno de ellos habilitado por valor 0 y el otro por 1, de forma que el conjunto responda al flanco de la señal: configuración amo/esclavo.

A partir de la combinación amo/esclavo (biestable síncrono tipo **D**) puede construirse una amplia diversidad de tipos de biestables y, en particular, el biestable **JK** que sustituye directamente al biestable **RS** en el diseño síncrono de sistemas secuenciales. Por otra parte, la conexión en serie de biestables **D** da lugar a registros de desplazamiento que permiten la conversión serie-paralelo (y, también, paralelo-serie) de una palabra binaria.

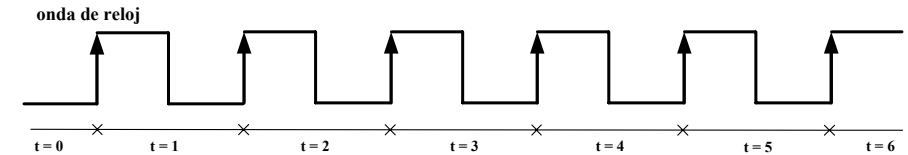
El correcto funcionamiento de un diseño síncrono exige que se respeten los tiempos funcionales de los biestables que lo conforman (tiempos de propagación, de anticipación y de mantenimiento) y que, en el momento inicial, se establezca en ellos el valor adecuado.

Recordemos que los bloques programables tipo **PAL** permiten configurar en su interior conjuntos de funciones booleanas en forma de suma de términos producto. Análogamente, se construyen bloques **PAL + biestables** (añadiendo un biestable tipo **D** en la salida de cada módulo **PAL**) que permiten recoger la evolución de las variables de estado; tales bloques reciben el nombre de **PLS** (secuenciadores lógicos programables).

Diversas mejoras sobre el módulo **PAL + biestable** han dado lugar al concepto de «macrocelda» programable; actualmente se dispone de circuitos integrados con múltiples macroceldas y canales de interconexión entre ellas (**CPLDs**: dispositivos lógicos programables complejos), capaces de configurar, por programación, sistemas digitales completos (pues el número de macroceldas en un integrado puede ser muy alto).

13.1. Sincronismo y configuración amo/esclavo: biestables síncronos

El sincronismo implica una *cuantificación del tiempo* en unidades discretas, definiendo con precisión el instante en que se pasa de una unidad a la siguiente: las transiciones (los cambios de estado) tendrán lugar en dichos instantes y sólo en ellos.

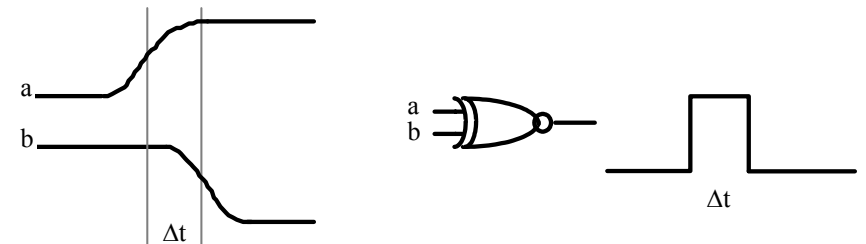


El paso de una unidad temporal a la siguiente vendrá definido por uno de los flancos de la onda de reloj; en la figura se ha considerado como activo el flanco positivo $\uparrow$ pero de igual forma podría tomarse el flanco negativo $\downarrow$.

La sincronización, además de coordinar temporalmente el funcionamiento de los biestables, proporciona una gran seguridad de funcionamiento.

Consideremos, por ejemplo, un biestable **RS** asíncrono que ha de ser activado cuando el valor booleano de dos variables intermedias de un circuito sea idéntico: $S = a.b + \bar{a}.\bar{b}$ supongamos que en un determinado momento la variable **a** que estaba en 0 pasa a 1 y que, a la vez, **b** pasa de 1 a 0: el biestable no debe activarse.

Puede suceder que, por diferencia de tiempos de propagación, las ondas correspondientes a las variables **a** y **b** que lleguen a las entradas de la lógica de activación del biestable sean del tipo de la figura, superponiéndose durante un pequeño intervalo de tiempo Δt , lo cual da lugar a un pulso espurio de breve duración (*glitch*), que pudiera ser suficiente para disparar el biestable.

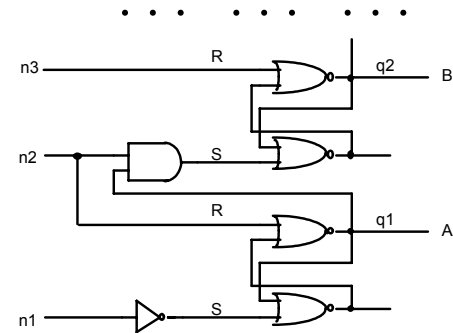


Este marcado no deseado puede ser evitado mediante la utilización de biestables síncronos: con ellos se consigue que todas las variables del circuito conmuten a la vez (en los flancos activos del reloj). Y, lo que es más importante, que los valores que determinan las transiciones sean los presentes en el momento anterior al flanco activo del reloj; no importa, pues, que existan retrasos desiguales en la llegada de las señales a los diversos biestables, ya que son los valores consolidados previos a la transición (y no los que se producen durante ella) los que la controlan.

Otro ejemplo relativo a la seguridad de funcionamiento: en el caso del depósito con mezcla de 4 líquidos, en la codificación con «un solo uno» (apartado 11.4, página 30), simplificando al máximo las condiciones de marcado y de borrado resulta que:

$q_1:$ $S = \overline{n_1}$ $R = n_2$
 $q_2:$ $S = q_1.n_2$ $R = n_3$

el primer biestable se borra con **n2** y el segundo se marca con **n2** cuando el primero está en **1**: la misma señal **n2** que borra al primer biestable debe marcar al segundo de ellos.



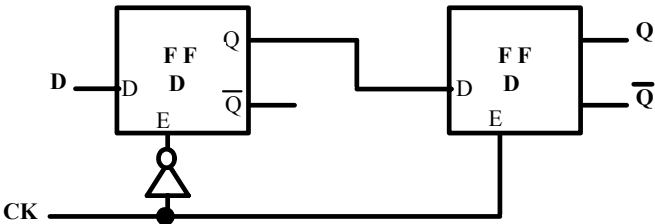
Ahora bien, el borrado de **q1** es más rápido que el marcado de **q2** ya que éste debe atravesar una puerta "y" adicional (que introduce el correspondiente retraso); es fácil (y así ocurre normalmente si se monta este circuito en laboratorio) que se produzca el borrado del primer biestable y, al hacerse **q1 = 0**, desaparezca la condición de marcado del segundo biestable y no se llegue a marcar.

En la solución dada en la página 30 se evitaba esta situación haciendo que el borrado de **q1** también pasara por dicha puerta "y" (lo cual es correcto). Pero la posibilidad de error por diferencia en el tiempo de marcado/ borrado de las diversas variables de estado está siempre presente y ha de tenerse en cuenta, salvo que se utilice código Gray (en cuyo caso nunca se modifican dos variables de estado a la vez).

De nuevo cabe decir que este tipo de errores, debidos a diferencia de tiempo en la propagación de señales que deben actuar simultáneamente, se evita con la utilización de biestables síncronos: en un diseño síncrono son los valores binarios en flanco activo del reloj los que determinan las transiciones (y no les afecta el transitorio de conmutación de las variables de estado en dicho flanco).

Biestables síncronos son aquellos en que los cambios de estado, es decir los cambios del valor booleano memorizado, se producen solamente en el flanco activo de la señal de reloj.

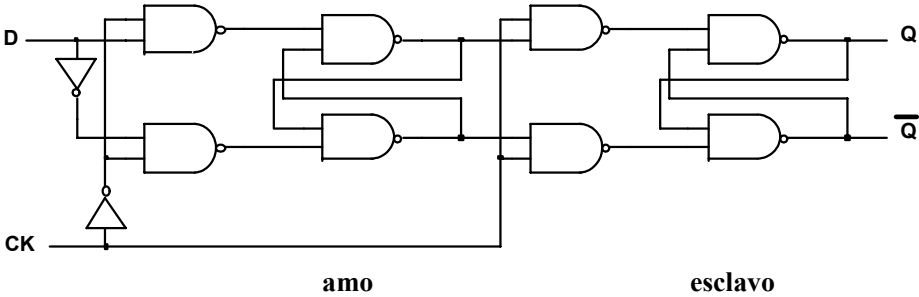
La actuación «por flancos» no corresponde directamente al álgebra booleana: los sistemas lógicos actúan por niveles, correspondientes a los valores booleanos **1 / 0**. Para conseguir que un biestable «síncrono» modifique su salida solamente en los flancos de la señal de reloj es preciso utilizar dos biestables conectados en serie (uno detrás del otro) y de manera que el primero de ellos se habilite con el valor **0** de la onda de reloj y el segundo con el valor **1** de dicha onda.



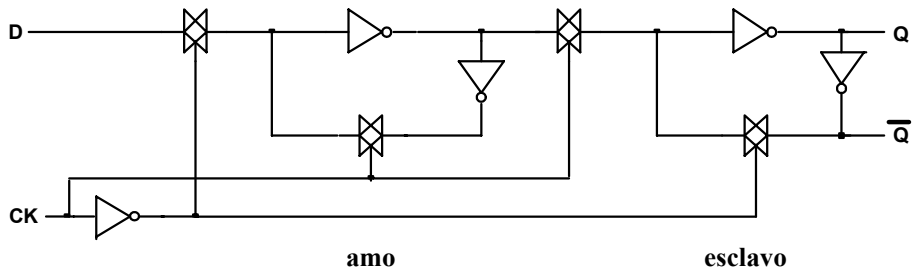
De esta forma para **CK=0** el primer biestable recoge el valor lógico presente en su entrada **D** y lo almacena y cuando **CK=1** el primer biestable transmite el valor almacenado al segundo: el conjunto actúa globalmente cuando **CK** pasa de **0** a **1**, es decir, en las subidas de la señal de reloj **CK**.

Este tipo de estructura que agrupa dos biestables (tipo **D**, con habilitación por niveles **E**) para obtener un biestable síncrono (tipo **D**, con habilitación por flancos **CK**) se denomina configuración *amo/esclavo* (*master/slave*: el segundo biestable actúa como «esclavo» del primero, repite su mismo valor booleano: es «la voz de su amo»).

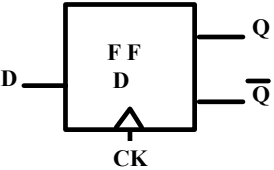
La configuración *master/slave* puede construirse con puertas "y-negada" (*Nand*) o con puertas "o-negada" (*Nor*).



Asimismo, la construcción de biestables síncronos puede hacerse utilizando puertas de transmisión [en tecnologías MOS la configuración de biestables con puertas de transmisión requiere menor número de transistores, por lo cual ocupa menor área de integración y presenta menores tiempos de propagación]:

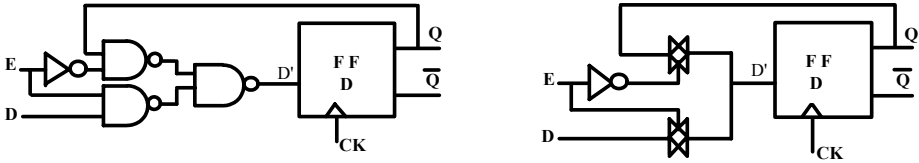
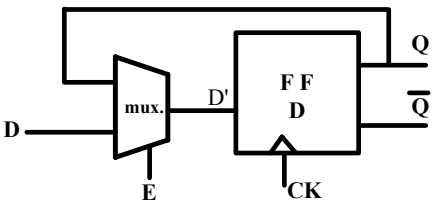
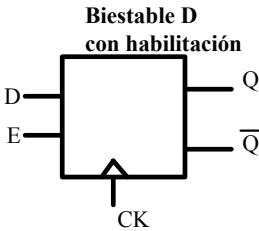


La combinación amo/esclavo da como resultado un biestable tipo **D** síncrono; su funcionamiento es análogo al biestable **D** con habilitación por niveles **E**, pero la adquisición del dato presente en la entrada **D** se produce solamente en los flancos activos de la onda de reloj (entrada **CK**, *Clock*).



La entrada de reloj, como señal de sincronismo, se distingue mediante un pequeño «triángulo» que indica que dicha señal actúa «por flancos» (y sirve, a la vez, para diferenciar a los biestables síncronos). Hemos supuesto que el flanco activo corresponde a la «subida» (paso de valor **0** a valor **1**), pero de igual modo puede serlo la «bajada» de la onda (paso de valor **1** a valor **0**); en tal caso, indicaremos que el reloj actúa con flancos descendentes mediante un círculo (inversor) en la entrada de reloj.

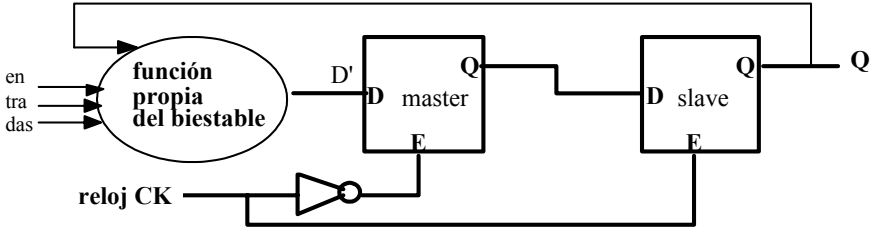
La siguiente figura representa un biestable síncrono tipo **D** dotado de una entrada de habilitación **E**: cuando **E=0** el biestable conserva el valor anterior (para ello, la entrada ha de recibirlo de la salida **Q**), mientras que para **E=1** el biestable recibe un nuevo dato en el flanco activo del reloj: $D' = Q(t+1) = \bar{E} \cdot Q + E \cdot D$



Biestable síncrono D con habilitación E

Dado que para **E = 0** este biestable no debe cambiar de estado, podría pensarse en un diseño «más simple», interrumpiendo el paso de la señal de reloj mediante una puerta "y" controlada por la entrada de habilitación **E**. Tal diseño sería incorrecto, ya que nunca se debe «hacer lógica» sobre la señal de reloj: conformar funciones booleanas con dicha señal rompe el sincronismo, pues provoca la aparición de flancos espurios en la entrada de reloj. [Véase el apartado 15.5. *Precauciones relativas a la señal de sincronismo*].

Como en el caso anterior, a partir de la configuración amo/esclavo (biestable síncrono básico, tipo **D**) pueden construirse otros tipos de biestables síncronos: para ello, delante de la entrada **D'** de la configuración *master/slave* se añadirá la función que corresponda al comportamiento que se desea para el biestable; función que puede obtenerse a través de su tabla funcional: entradas, $Q(t) \rightarrow Q(t+1)$



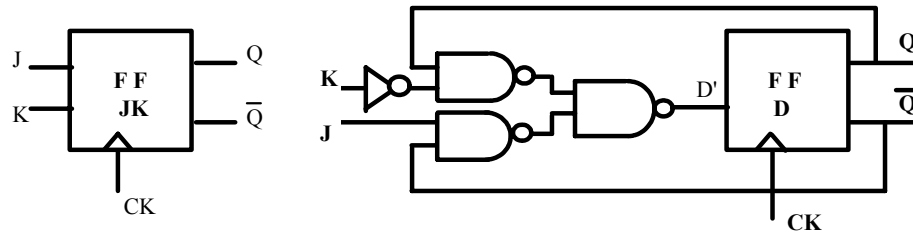
El biestable **JK** corresponde, en forma síncrona, al biestable **RS**, pero su tabla de operación incluye también el caso de activación de ambas entradas a la vez (**J=K=1**): la entrada **K** actúa para el borrado ($\equiv \mathbf{R}$) y la **J** para la puesta a **1** ($\equiv \mathbf{S}$) y cuando se activan ambas a la vez el biestable cambia de estado. Bien entendido que las transiciones se efectúan sólo en los flancos activos de la onda de reloj y lo hacen conforme a los valores booleanos de **J** y **K** inmediatamente anteriores a dichos flancos.

$J(t)$	$K(t)$	$Q(t+1)$
0	0	no cambia
0	1	0
1	0	1
1	1	cambia de estado (0→1, 1→0)

La función booleana previa que ha de añadirse a la configuración amo/esclavo corresponde a la siguiente «tabla de verdad»:

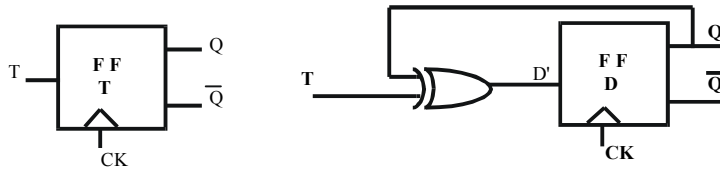
$Q(t)$	J	K	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$D' = Q(t+1) = \bar{K} \cdot Q + J \cdot \bar{Q} = (\bar{K} * Q) * (J * \bar{Q})$$



El biestable síncrono tipo **T** (*Toggle*) resulta muy útil para la construcción de contadores: cuando su entrada $T=0$ el biestable no modifica su estado, mientras que para $T=1$ el biestable conmuta (cambia de estado) en cada flanco activo del reloj:

$$D' = Q(t+1) = \bar{T} \cdot Q + T \cdot \bar{Q} = T \oplus Q$$



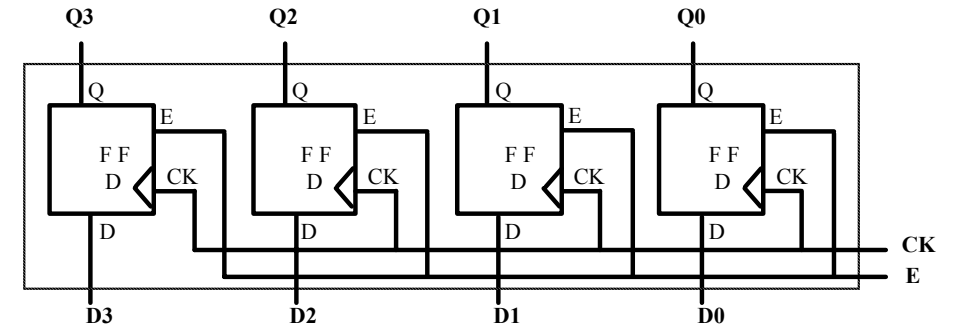
También podemos considerar el biestable tipo **T** sin entrada que cambia de estado cada vez que recibe un flanco activo del reloj: $D' = \bar{Q}$



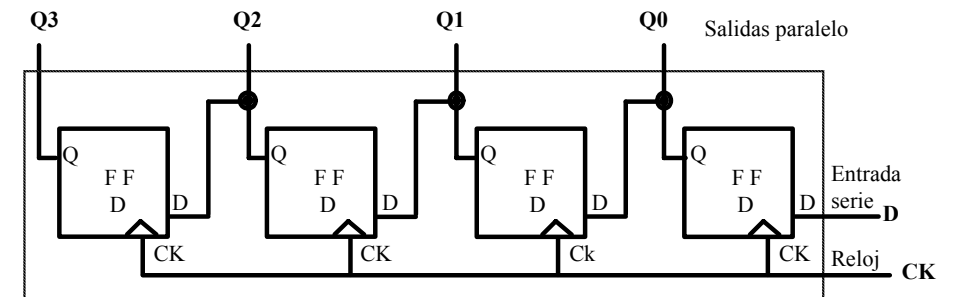
Como complemento a este tema dedicado al «buen manejo del tiempo» (mediante el sincronismo), el apéndice A4 (*Temporizadores: osciladores y monostables*) describe los monostables y los astables, circuitos que, junto con los biestables, forman el grupo de los *multivibradores*. Los biestables son las celdas básicas de memoria y, también, de sincronización de los circuitos secuenciales; astables y monostables son muy útiles en el diseño secuencial, para generar ondas de reloj y como temporizadores, respectivamente.

13.2. Registros síncronos

Agrupando n biestables **D** en paralelo, con sus entradas de reloj y de habilitación comunes, se configura un registro síncrono, capaz de almacenar una palabra de n dígitos.

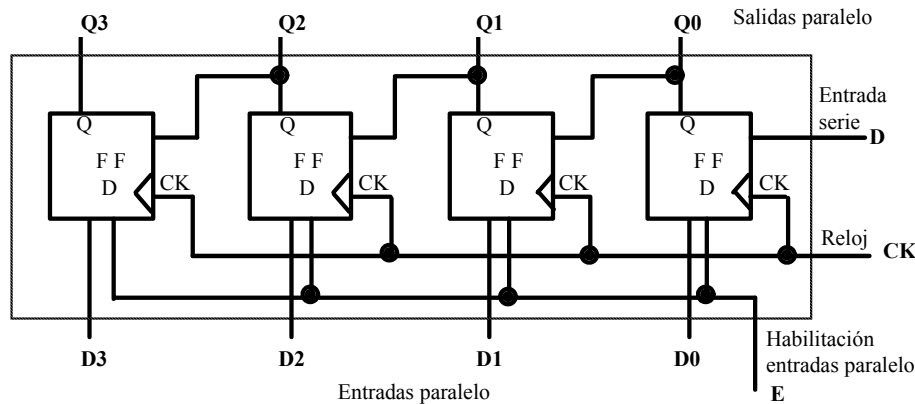


La conexión de n biestables síncronos tipo **D** en serie (la entrada de cada uno unida a la salida del anterior), con reloj común para todos ellos, configura un registro con una única entrada (la del primero) y n salidas en paralelo, en el que la información se desplaza de biestable a biestable «a golpes de reloj»: *registro de desplazamiento (shift-register)*.



Este registro realiza una *conversión serie-paralelo* de la información que recibe en su entrada; con cada pulso de reloj el conjunto de los bits almacenados avanza un biestable y entra un nuevo bit en la cadena (operación de entrada en serie, presentación de valores booleanos en las salidas en paralelo).

Generalmente se añaden a tales registros de desplazamiento las correspondientes entradas paralelo junto con la entrada de habilitación de las mismas, es decir, una entrada **D** a cada uno de los biestables, controlada por una entrada de habilitación común **E**; activando la habilitación (**E=1**) los valores presentes en las entradas paralelo son almacenados en los biestables (*carga en paralelo*).



De esta forma puede recibirse información tanto en serie (entrada serie) como en paralelo (entradas paralelo) y transmitirse también en las dos formas: paralelo (en el conjunto de las salidas) y serie (en la salida del último biestable por desplazamiento).

Ello permite realizar la *conversión serie-paralelo* y la *conversión paralelo-serie* de una palabra binaria:

- *conversión serie-paralelo*: la palabra se recibe bit a bit por la entrada serie del registro de desplazamiento y, una vez recibida, sus dígitos están presentes en paralelo en las salidas de los biestables que configuran el registro;

- *conversión paralelo-serie*: la palabra se recibe a través de las entradas paralelo de los biestables del registro de desplazamiento y es enviada bit a bit por la salida del último de dichos biestables.

Otra aplicación de los registros de desplazamiento se basa en el hecho de que el desplazamiento de un número binario una posición hacia la izquierda equivale a multiplicar dicho número por 2, siendo sumamente útil para el diseño de algoritmos de producto aritmético; recíprocamente, el desplazamiento de una posición hacia la derecha equivale a dividir el número por 2 (registros de desplazamiento hacia la derecha). Para ello, se construyen registros de desplazamiento bidireccionales, con posibilidad de desplazarse en ambos sentidos, y, a su vez, con posibilidad de carga paralelo.

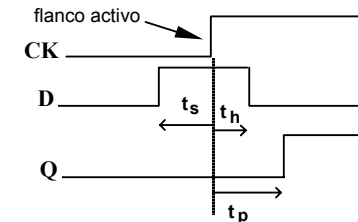
13.3. Tiempos funcionales e inicialización de los biestables

13.3.1. Tiempos de propagación, de anticipación y de mantenimiento de dato

El cambio de estado de un biestable síncrono se produce coincidiendo con el flanco activo de la onda de reloj; en el caso de un biestable tipo **D**, en ese momento (flanco activo del reloj), la salida efectúa una copia del valor presente en la entrada.

Obviamente existe un pequeño retraso entre el flanco activo de la señal de reloj y la consolidación del correspondiente estado en la salida: tiempo de propagación del dato t_p . Además, para asegurar el correcto funcionamiento del biestable **D** cuando llega el flanco activo del reloj es necesario que el valor correcto del dato se encuentre presente en la entrada **D** con una cierta anticipación a dicho flanco (*setup*: t_s) y que tal valor se mantenga durante un cierto intervalo posterior (*hold*: t_h).

Supongamos, por ejemplo, que un biestable **D** que se encuentra con salida **0** debe recibir un **1**, dicho valor booleano **1** debe presentarse en la entrada del biestable, al menos, durante un intervalo t_s (*setup*: tiempo de anticipación) previo al flanco activo del reloj y debe permanecer dicho valor **1** en dicha entrada **D**, al menos, durante un intervalo t_h (*hold*: tiempo de mantenimiento) posterior al flanco activo.



Interesan, en general, tiempos de mantenimiento t_h muy pequeños o, al menos, que t_h sea menor que t_p (tiempo de propagación de los biestables «anteriores», es decir, de aquellos cuyas salidas influyen sobre la entrada del biestable), de manera que el propio retardo de propagación en los biestables sirva para cubrir el tiempo de mantenimiento del dato y evite errores en su captura por el biestable. En general, esto es lo que sucede al utilizar circuitos integrados estándar y, en tal caso, no es preciso prestar atención a los tiempos de mantenimiento (*hold*).

La suma de los tiempos de propagación y de preparación del dato $t_p + t_s$ limita la velocidad máxima de trabajo de los biestables; el período de la onda de reloj deberá ser superior a dicha suma, para permitir que se forme el valor correcto en las salidas de los biestables (retraso t_p) con suficiente anticipación (intervalo previo t_s) para establecer el nuevo valor correcto en sus entradas. De forma que las «violaciones de *setup*» se resuelven disminuyendo la velocidad del reloj o, si es viable, utilizando biestables más rápidos (con menores tiempos de propagación o de anticipación).

El capítulo 15 desarrolla con mayor detalle el «análisis de tiempos» en los circuitos secuenciales síncronos, incorporando al mismo los tiempos de propagación debidos a la parte combinacional existente entre los biestables.

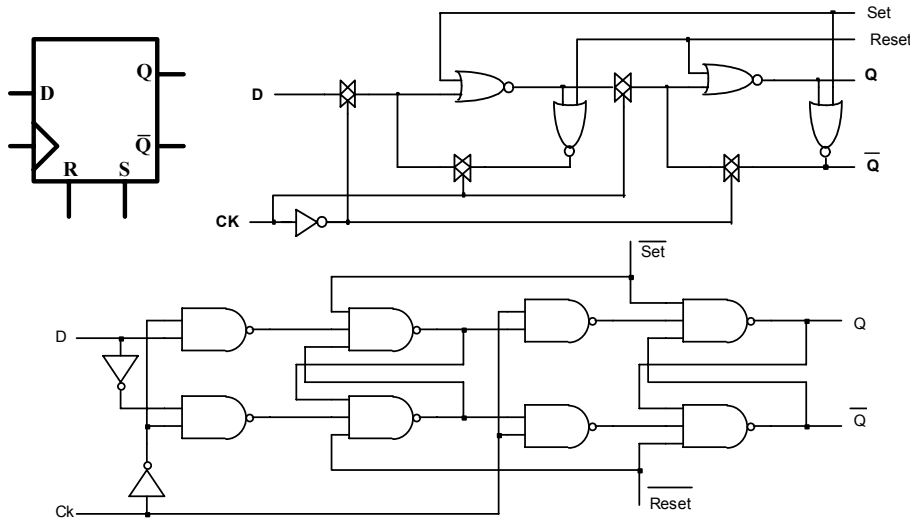
13.3.2. Inicialización de los biestables

Por lo general, un sistema secuencial, es decir, cualquier sistema configurado con biestables, requiere un estado inicial determinado, a partir del cual comienza la evolución del sistema: los biestables deben adoptar, en el primer momento, un estado booleano determinado, en muchos casos el estado 0 (00...).

Al activar la tensión de alimentación de un sistema (encendido: *power-on*), cada uno de los biestables presentes adoptará un estado booleano que, en principio, no es predecible, ya que depende de la configuración electrónica del biestable y del transitorio de encendido de la alimentación.

Para la inicialización de los biestables síncronos (fijación de valores iniciales en los mismos) y, también, para poder efectuar en cualquier momento un borrado o marcado de los mismos (con independencia del funcionamiento normal controlado por el reloj), interesa disponer de entradas asíncronas de marcado y borrado (*set* y *clear* o *reset*) que, al ser activadas, lleven directamente el biestable a estado 1 o estado 0, respectivamente.

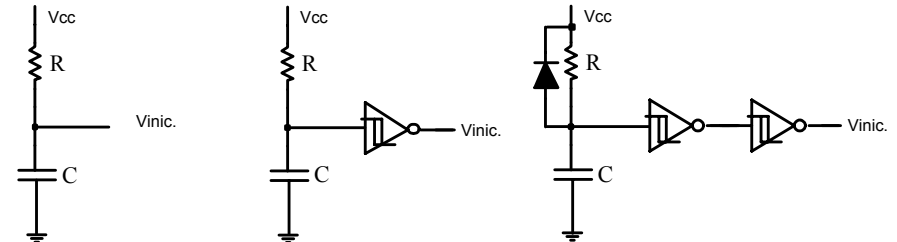
Es sencillo incluir entradas asíncronas de marcado y borrado en las configuraciones circuitales amo/esclavo representadas anteriormente:



Obsérvese (en las dos figuras) que es necesario actuar sobre ambos «semibiestables» (amo y esclavo) para asegurar que el borrado o el marcado permanecen hasta el siguiente flanco activo del reloj (si el borrado o marcado actuaran solamente sobre el esclavo y finalizaran cuando el reloj se encuentra con valor 1, el esclavo recibiría inmediatamente el valor almacenado en el amo).

La inicialización de los biestables consiste en forzarles a adoptar el estado booleano que interese, mediante un primer pulso singular que se produzca tras el encendido (*power-on*) de la alimentación. La activación de las entradas de borrado o de marcado asíncrono (*reset* y *set*) por dicho primer pulso llevará al biestable a estado 0 ó 1 según proceda.

El pulso de inicialización puede generarse «automáticamente» mediante un circuito RC conectado a la tensión de alimentación; el retraso originado por la carga del condensador, a partir del propio «arranque» de la alimentación, determina la permanencia del valor 0 booleano durante un cierto intervalo posterior al encendido.



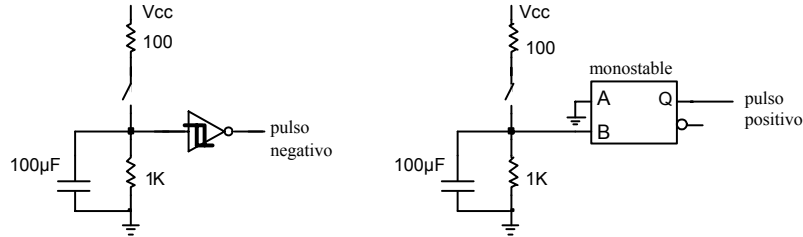
En el caso de que las entradas de borrado o marcado asíncrono se activen con valor 0, bastará conectar la salida del primer circuito RC a la entrada que corresponda. Cuando su valor activo sea 1 será preciso invertir el pulso, mediante una puerta inversora, preferiblemente de entrada con histéresis (tipo *Schmitt*) para mejorar la definición del propio pulso y evitar los rebotes. Incluso, en activación con valor 0, si las entradas de borrado o marcado de los biestables no son de tipo *Schmitt* es conveniente conformar el pulso a través de dos inversores cuyas entradas sí lo sean.

La anchura del pulso de inicialización en los circuitos anteriores será del orden de la constante de tiempo del circuito RC. Los valores del condensador y de la resistencia deben ser adecuadamente altos para asegurar una amplia anchura de pulso que lo prolongue más allá del transitorio de encendido ($RC > 100 \text{ ms}$). En su caso, la inclusión de un monostable disparado por el circuito RC permite fijar con precisión la anchura del pulso de inicialización.

Suele incluirse un diodo en paralelo con la resistencia *R*, en polarización inversa, para descargar rápidamente el condensador *C* cuando hay caídas de tensión y asegurar que se produzca el pulso de inicialización en caso de «microcortes» (caídas de tensión de pequeña duración) de la alimentación.

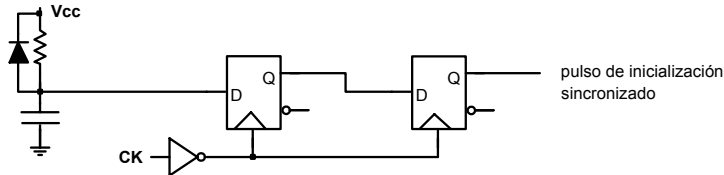
Existen circuitos integrados específicos que supervisan la tensión de alimentación y proporcionan un pulso de anchura fija en el encendido y, asimismo, activan la inicialización cada vez que la tensión de alimentación cae por debajo de un valor prefijado, prolongando después el pulso de inicialización al restablecerse la alimentación.

Caso de que interese una inicialización manual, será necesario conformar el pulso producido sobre el correspondiente pulsador mediante un circuito RC de constante de tiempo relativamente alta (a fin de evitar los rebotes del pulsador), seguido de una puerta con entrada *Schmitt*. La utilización de un monostable permite ajustar a un valor fijo la anchura de este pulso manual de inicialización.



Puede resultar conveniente sincronizar los pulsos de inicialización de los biestables con el reloj del sistema secuencial para evitar posibles coincidencias entre el final de la inicialización y el flanco activo del reloj; tales coincidencias podrían generar errores por cuanto que algunos biestables podrían responder al flanco de reloj y otros no.

Para sincronizar la inicialización basta pasarla a través de un biestable síncrono y, en tal caso, interesa conectar su entrada de reloj al flanco «no activo» del reloj para que la inicialización cubra por completo al anterior flanco activo del mismo; a fin de reducir la incidencia de situaciones «metastables» suelen utilizarse dos biestables sucesivos.

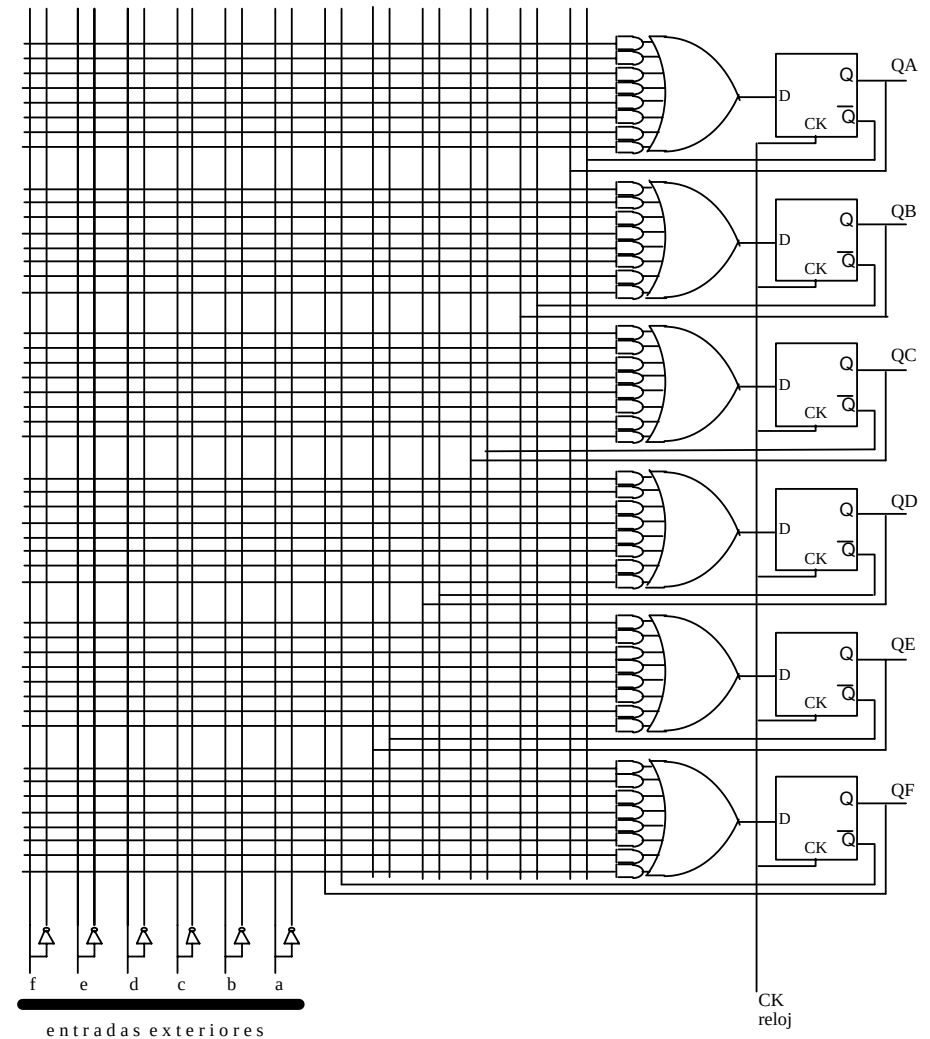


13.4. Dispositivos lógicos programables: PAL con biestables

Existen dispositivos programables basados en la configuración **PAL**, que añaden un biestable en cada una de las salidas; de esta forma, se proporciona a la estructura **PAL** capacidad de memoria y, por tanto, de realización de circuitos secuenciales. En este caso, los términos producto de la **PAL** han de admitir como variables, además de las propias variables de entrada, las salidas de los biestables; es decir, las variables de estado (almacenadas en los biestables) han de realimentarse como entradas sobre los módulos **PAL** que configuran las funciones de evolución del estado.

De esta forma tendremos un bloque con **n** módulos, formados, cada uno de ellos, por un biestable cuya entrada **D** es activada por una puerta "o" precedida por **q** puertas "y", cuyas entradas configuran una Matriz **Y** programable respecto a las **m** variables de entrada exteriores y sus negadas y a las **n** salidas de los biestables del bloque y sus negadas.

Este tipo de bloque programable recibe el nombre de *secuenciador lógico programable* **PLS** (*programmable logic sequencer*) o el de **PAL** con biestables **RPAL** (*registered PAL*).



PLS de 6 módulos con 8 términos producto en cada módulo y 6 variables de entrada

Los bloques **PLS** permiten la programación de las funciones de evolución de estado de un circuito secuencial; para ello habrá que obtener tales funciones de evolución del estado correspondientes a biestables tipo **D**, expresadas en forma de suma de productos.

Asimismo se pueden programar en el mismo bloque **PLS** las funciones de activación de las salidas. En este caso, las salidas pasarán a través de biestables que realizan un sincronismo de las mismas, lo cual suele ser beneficioso en la mayoría de los casos: se añade al sincronismo de las variables de estado el sincronismo de las salidas (que aplica a las mismas un retraso de una unidad de tiempo de reloj).

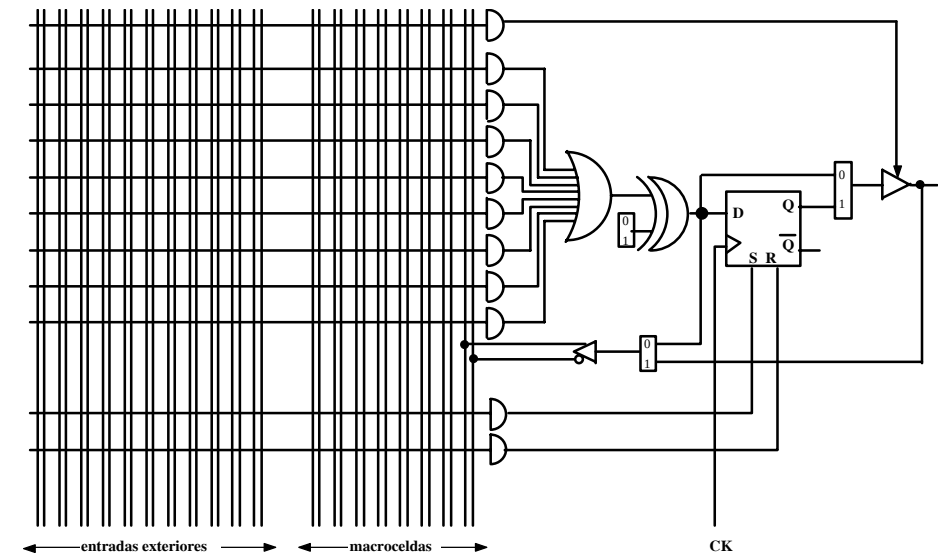
Hay también bloques **PLS** en que el biestable de salida de cada módulo puede utilizarse o no (puede *puentearse*) mediante programación, de forma que permiten la inclusión tanto de variables sincronizadas, como de funciones combinacionales; de esta manera pueden construirse las funciones de activación de las salidas sin añadirles el retraso correspondiente al sincronismo.

Así pues, en el interior de un bloque **PLS** puede configurarse un circuito secuencial completo, mediante programación; ésta es, hoy día, la forma habitual de diseño digital: un solo circuito integrado particularizado que contiene todo el sistema digital (sin necesidad de la conexión de múltiples circuitos integrados estándar).

Los circuitos programables han experimentado un extraordinario desarrollo en la última década (años 90), con la integración de un alto número de módulos en el mismo circuito y el aumento de prestaciones de tales módulos. El módulo básico **PLS** «suma de productos + biestable» ha incorporado diversos «selectores» programables, así como la posibilidad de salida «tri-estado» y recibe el nombre genérico de *macrocelda* (cuya configuración es la representada en la figura de la página siguiente).

Una macrocelda está compuesta por

- una suma de productos programables de múltiples entradas entre las que se encuentran las salidas de todas las macroceldas (realimentación), estando disponible cada entrada en su forma afirmada y negada;
- una puerta "**o-exclusiva**" programable que permite configurar la función de la macrocelda (suma de términos producto) en forma afirmada (y) o negada ($\bar{y}$), pudiendo optar por la que menor número de términos producto requiera (lo cual equivale a poder elegir entre resolver la función por «unos» o por «ceros»);
- un biestable que recibe dicha función;
- un selector que permite «puentear» el biestable y que configura, por tanto, la macrocelda como combinacional o como secuencial;
- y un adaptador triestado con capacidad para «desconectar» la macrocelda respecto del terminal de salida, lo cual da lugar a las siguientes posibilidades:
 - la simple desconexión (alta impedancia) del terminal
 - su utilización bidireccional (I/O)
 - su disponibilidad como entrada, caso de que la macrocelda no sea utilizada.



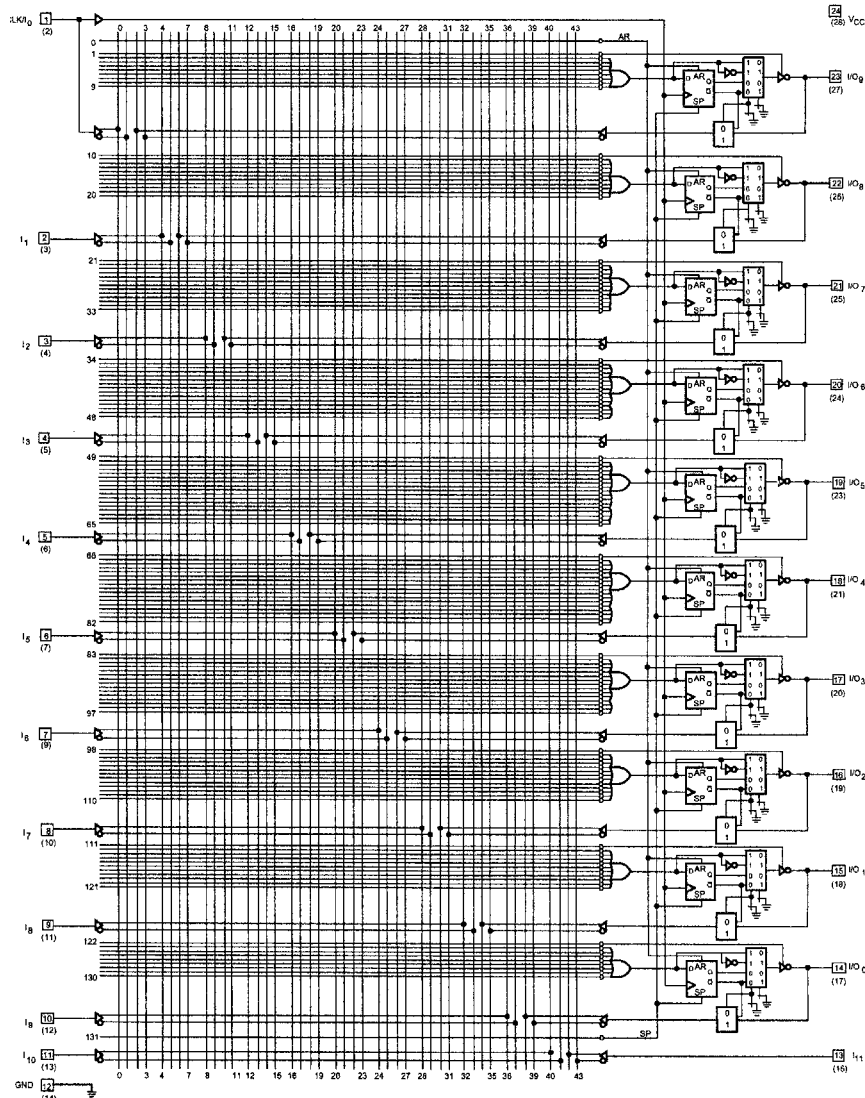
Configuración booleana de una macrocelda

Los circuitos integrados programables que utilizan *macroceldas* suelen ser aludidos con las siglas **PLD** (dispositivos lógicos programables).

Mientras el número de macroceldas contenido en un **PLD** no es alto, cada una de ellas recibe todas las entradas del dispositivo y la realimentación de todas las salidas del mismo, que coinciden directamente con las salidas de todas las macroceldas del dispositivo: en caso de que alguna de las macroceldas no sea utilizada, la salida correspondiente puede ser usada como entrada.

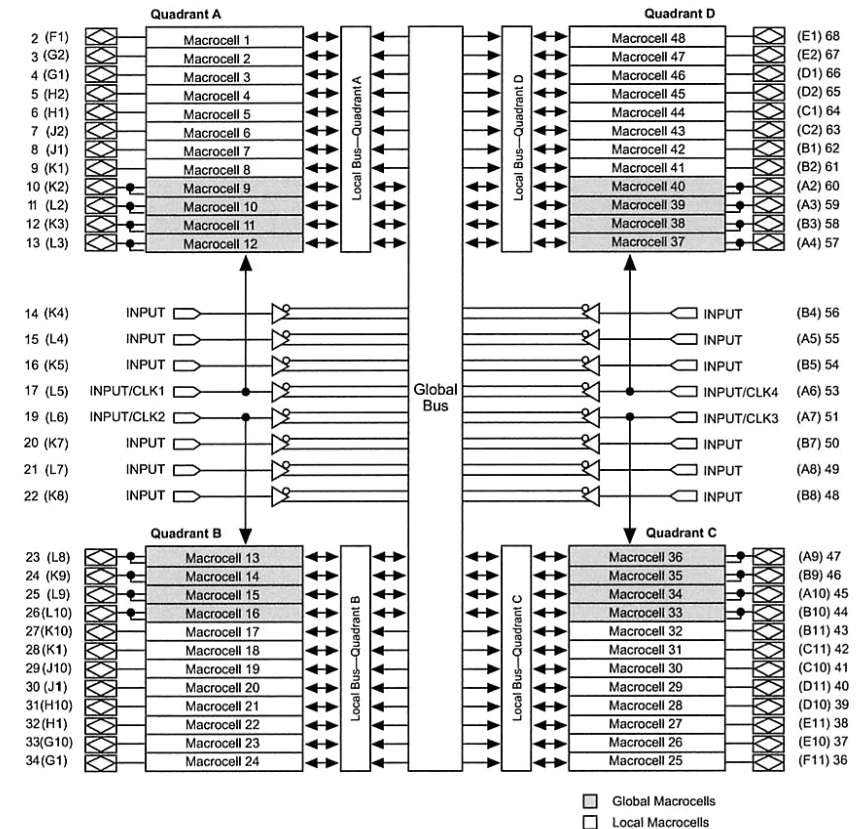
El más conocido de estos dispositivos es el 22V10 que contiene 10 macroceldas con diferente número de términos producto (hay 2 macroceldas con 8 términos producto, otras 2 con 10, 2 con 12, 2 con 14 y 2 con 16 términos producto). [Véase la figura de la página siguiente.]

El circuito integrado programable 22V10 ofrece 22 terminales: 12 de entrada (uno de ellos para el reloj) y 10 de tipo I/O (correspondientes a las 10 salidas de las macroceldas); cuando una macrocelda no es utilizada su correspondiente terminal de salida puede ser aprovechado como una entrada exterior más para el resto de las macroceldas.



Configuración del circuito integrado programable PLD 22V10

La figura siguiente representa un **PLD** aún mayor: el circuito integrado programable EP1800, que contiene 48 macroceldas con 16 entradas exteriores (encapsulado de 68 pines) y posibilidad de aprovechar también como entradas las salidas de las macroceldas que no se utilicen como tales.



Configuración del circuito integrado programable EP1800

Como el número de macroceldas de este circuito integrado ya es considerable, se dividen en cuatro cuadrantes; dentro de cada cuadrante hay cuatro macroceldas «globales» que se realimentan con todas las del integrado y ocho macroceldas «locales» que solamente se realimentan con las del propio cuadrante y con las globales.

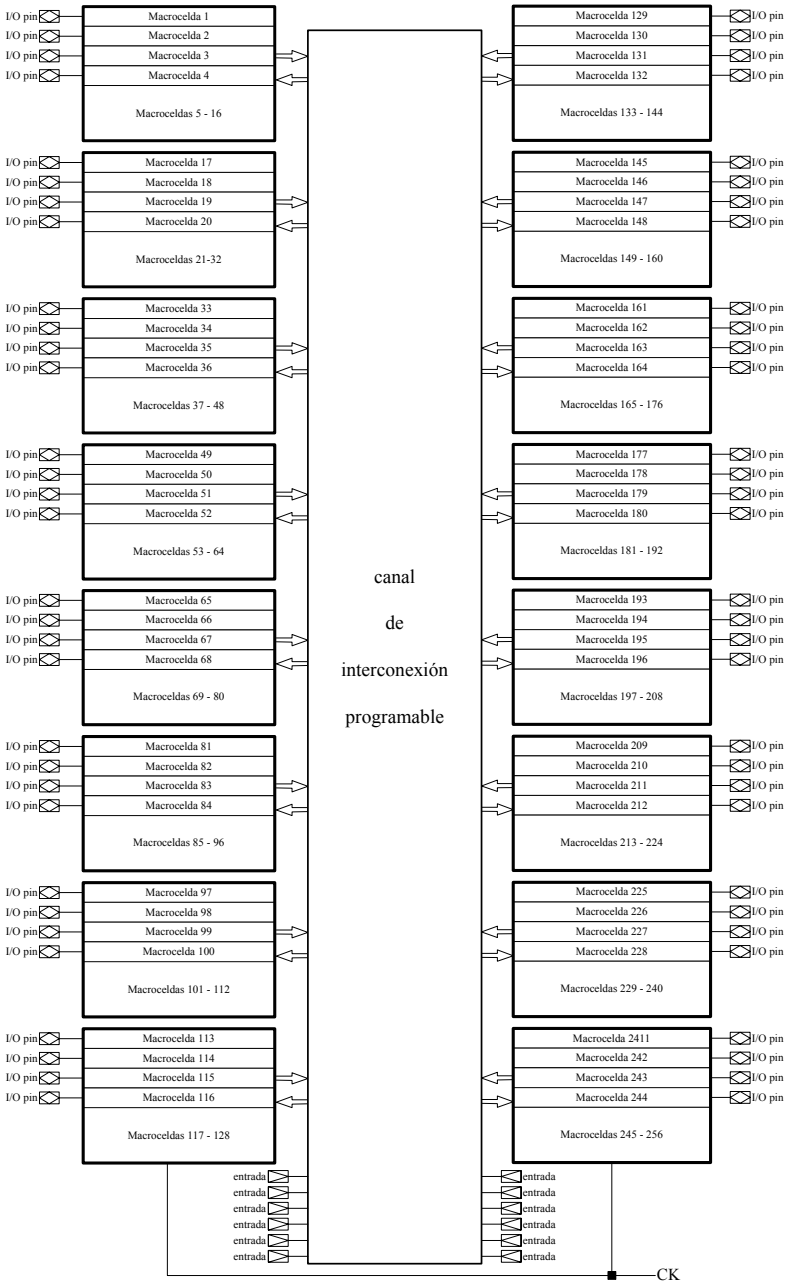
Cuando se trata de un gran número de macroceldas dentro de un mismo integrado no resulta adecuado que cada una de ellas reciba todas las entradas y se conecte con todas las macroceldas ya que la cantidad de conexiones programables sería sumamente elevada; tampoco es viable que las salidas de todas las macroceldas se conecten a terminales de salida del circuito integrado.

Para limitar el tamaño y el número de terminales del circuito, las macroceldas se agrupan en bloques disjuntos y reciben solamente las entradas propias del bloque y la realimentación de las macroceldas del mismo; no todas las salidas de las macroceldas están conectadas a terminales de salida sino solamente un número reducido de cada bloque. Los bloques se conectan entre sí a través de un bus central de conexiones programables conformando una estructura ramificada: un canal o eje de conexiones (*a manera tronco*) sobre el cual se insertan los bloques de macroceldas (*como ramas del mismo*). [Véase la figura de la página siguiente.]

Este tipo de circuitos integrados programables de amplio número de macroceldas con arquitectura ramificada es nombrado con las siglas **CPLD** (dispositivos lógicos programables complejos).

Para reducir la superficie de integración y evitar un alto desaprovechamiento de las estructuras PAL, el número de términos producto de cada macrocelda suele ser pequeño (4 ó 6) y para ampliarlo, cuando se necesite, se incluyen en cada bloque «expansores», en forma de términos producto adicionales que, sin pertenecer directamente a ninguna de las macroceldas del bloque, pueden ser utilizados por cualquiera de ellas.

La figura de la página siguiente representa un CPLD de 256 macroceldas, de las cuales solamente 64 pueden utilizarse como salidas exteriores (las otras 192 macroceldas son internas); el circuito dispone de una entrada específica de reloj y de 12 entradas exteriores, pudiéndose emplear también como entradas los terminales correspondientes a macroceldas que no se usen como tales.



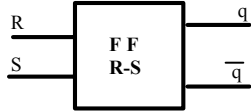
Configuración CPLD con 256 macroceldas

13.5. Los biestables en VHDL

Al igual que en los capítulos 1 y 4, se incluye aquí la descripción VHDL de diversos tipos de biestables a fin de desarrollar una aproximación gradual al diseño con lenguajes de descripción circuital:

Biestables asíncronos

biestable RS



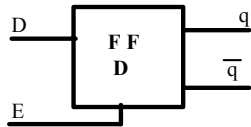
- a) $q \leq '0'$ when $R = '1'$ else $'1'$ when $S = '1'$ else q ;
 b) $y \leq \text{not } R \text{ and } (S \text{ or } q)$;
 ambos casos con borrado prioritario

descripción utilizando proceso

```
process (R,S)
begin
  if R = '1' then q <= '0';
  elsif S = '1' then q <= '1';
  end if;
end process;
```

(téngase en cuenta que *proceso* conserva los valores:
 por ello no es necesario añadir *else q <= q;*)

biestable D



- a) $q \leq D$ when $E = '1'$ else q ;
 b) $y \leq (D \text{ and } E) \text{ or } (q \text{ and not } E)$

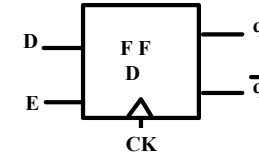
descripción utilizando proceso

```
process (D,E)
begin
  if E = '1' then q <= D;
  end if;
end process;
```

(no es necesario añadir *else q <= q;*)

Biestables síncronos

biestable D

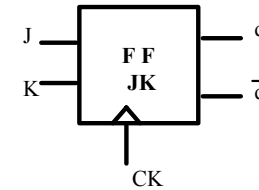


```
process
begin
  wait on CK until CK = '1';
  if E = '1' then q <= D; end if;
end process;
```

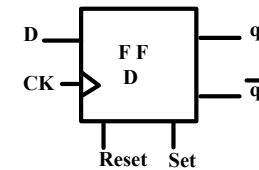
(el reloj se describe siempre dentro de un proceso;
 si afecta a todo el proceso se describe con un *wait*
 y el *until CK = '1'* indica flanco ascendente)

(no es necesario añadir *else q <= q;*)

biestable JK



```
process
begin
  wait on CK until CK = '1';
  if J = '1' and K = '1' then q <= not q;
  elsif J = '1' then q <= '1';
  elsif K = '1' then q <= '0';
  end if;
end process;
```

biestable D
con marcado/borrado
asíncronos

```
process(Reset,Set,CK,D)
begin
  if Reset = '1' then q <= '0';
  elsif Set = '1' then q <= '1';
  elsif CK'event and CK = '1' then
    q <= D;
```

```
  end if;
end process;
```

(la parte asincrónica debe describirse antes del reloj;
 luego el reloj se describe dentro de un *elsif* con *event*
 y *CK = '1'* indica flanco ascendente)

14 DISEÑO SECUENCIAL SÍNCRONO

- 14.1. Diseño de sistemas secuenciales con biestables síncronos
- 14.2. Ejercicios de diseño síncrono
- 14.3. Configuración microprogramada de grafos de estado
- 14.4. Los grafos de estado en VHDL

Los biestables síncronos son las celdas básicas del sincronismo, gracias a las cuales se consigue configurar sistemas síncronos. Las variables de estado se almacenan en biestables síncronos y las funciones de evolución del estado corresponderán a las de activación de dichos biestables.

Hoy día se utilizan habitualmente biestables básicos de tipo $\mathcal{D}$; el diseño con biestables $\mathcal{D}$ requiere la obtención funcional del nuevo valor de la variable de estado, a través de la tabla de evolución del estado ($\mathcal{D}_i = q_i^+$).

Ahora bien, en casos de tabla funcional muy compleja (de gran número de variables) resulta útil separar las condiciones de marcado y de borrado: desde el propio grafo de estados es posible obtener directamente las funciones de marcado $\mathcal{S}$ y borrado $\mathcal{R}$ y el diseño con biestables tipo $\mathcal{D}$ se reduce a agrupar ambas funciones. Ello equivale a razonar en términos de biestables tipo $\mathcal{JK}$, que son el equivalente síncrono (ampliado) del biestable $\mathcal{RS}$, y la traslación del diseño a biestables $\mathcal{D}$ se hace en forma análoga a como se conforman biestables $\mathcal{JK}$ a partir de biestables $\mathcal{D}$.

Un diversificado conjunto de ejemplos de diseño ilustran la forma de aplicar estos procedimientos; entre ellos se incluyen muchos casos típicos, como son: sumadores serie, detección de secuencias, contadores, dados electrónicos, generación de ondas sincronizadas entre sí, detectores de entrada y salida de personas, activación/desactivación mediante detección de secuencias, semáforo con demanda de paso, ...

Al igual que la configuración $\mathcal{ROM}$ permite construir las funciones booleanas sin obtener su expresión algebraica, directamente desde su tabla funcional, la evolución de las variables de estado puede construirse mediante un registro (que contenga el estado actual) y un codificador $\mathcal{ROM}$ (que proporcione el estado siguiente): esta forma de configurar grafos de estado se denomina «microprogramada» ya que el codificador contendrá el «programa» de estados sucesivos, en función del estado actual y del vector de entrada.

14.1. Diseño de sistemas secuenciales con biestables síncronos

Cualquier sistema secuencial de una cierta complejidad (es decir, con excepción únicamente de los sistemas digitales muy simples o muy directos) debe diseñarse en forma síncrona, con una señal de reloj **CK**, común a todo el sistema, que organiza el tiempo en unidades y señala con precisión los momentos de cambio de estado: flancos activos del reloj.

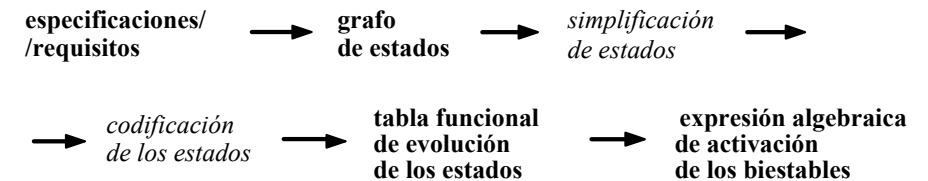
El sincronismo facilita en gran medida el diseño de un sistema complejo, al dividir el tiempo en unidades sucesivas y, con ello, hacer posible el razonamiento con unidades de tiempo discretas y numeradas.

Pero, sobre todo, el sincronismo proporciona seguridad de funcionamiento: evita fallos en las transiciones de estado al dedicar el intervalo correspondiente a cada unidad de tiempo al cálculo del nuevo estado y ejecutar simultáneamente el nuevo estado, ya completamente preparado, al finalizar la unidad de tiempo (en el siguiente flanco activo del reloj). De forma que los «espurios» o valores transitorios de algunas variables que puedan formarse durante la transición de un estado al siguiente no llegan a afectar al circuito porque el nuevo estado no es «admitido» hasta el siguiente flanco activo del reloj, cuando ya ha sido calculado por completo a lo largo de la unidad de tiempo.

El sincronismo de una variable, su modificación coincidente con los flancos activos de la señal de reloj, se consigue mediante una configuración «amo/esclavo» (*master/slave*), es decir, empleando un biestable síncrono para dicha variable. Por ello, los sistemas secuenciales síncronos se diseñan utilizando biestables síncronos: habitualmente biestables síncronos tipo $\mathcal{D}$.

El diseño de un sistema secuencial síncrono, en lo que a sus variables de estado se refiere, consiste en expresar las funciones de evolución del estado en términos de activación de las entradas de sus biestables.

Para ello han de recorrerse los sucesivos niveles de descripción del sistema secuencial:



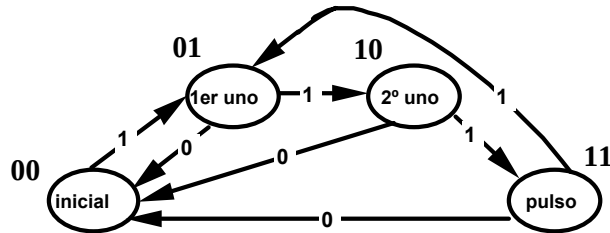
14.1.1. Diseño con biestables D

El biestable tipo D es el biestable síncrono básico y el más utilizado; para diseñar un sistema secuencial con biestables D conviene construir la tabla funcional de evolución del estado y, en ella, la propia columna q_i constituye la función de activación de su entrada:

$$D_i(t) = q_i(t+1) = q_i^+ = f_i(X, Q)$$

La síntesis de las funciones de activación de las salidas constituye un proceso meramente combinacional: $Y = f(X, Q)$.

Consideremos un ejemplo clásico, muy simple, el detector de una secuencia de bits consistente en tres «unos» seguidos: *un sistema con una sola entrada A, por la cual recibe dígitos binarios en unidades de tiempo sucesivas, debe responder con un pulso (de duración igual a una unidad de tiempo) cuando detecta la llegada de tres unos seguidos 111; suponemos que el sistema no admite solapes (una vez recibidos tres unos, para volver a activar la salida es preciso que reciba otros tres unos diferentes).*



Este grafo puede ser codificado con dos variables de estado; las funciones de activación de las entradas de los biestables D, obtenidas a partir de la tabla de evolución del estado serán:

estado anterior	q2	q1	a	q2 ⁺	q1 ⁺	nuevo estado	
	0	0	0	0	0		
			1	0	1		
	0	1	0	0	0		
			1	1	0		
	1	0	0	0	0		$D_1 = q_1^+ = a.(q_2 + \bar{q}_1)$
			1	1	1		
	1	1	0	0	0		$D_2 = q_2^+ = a.(q_2 \oplus q_1)$
			1	0	1		

y la función de activación de la salida corresponde al estado 11: $y = q_2 \cdot q_1$.

14.1.2. Diseño desde el grafo de estados

Razones de tiempo y de complejidad obligan en ocasiones a realizar el diseño directamente desde el grafo de estados (por la dificultad de construir la tabla funcional de evolución de los estados). En tal caso, una vez realizada la simplificación y la codificación de los estados, se pasa a considerar cada variable de estado q_i , dividiendo para cada una de ellas el grafo de estados en dos partes:

- conjunto de estados en que la variable q_i tiene valor 1;
- conjunto de estados en que la variable q_i tiene valor 0.

Para cada una de las transiciones que pasan de la parte en que q_i vale 0 a la parte en que q_i vale 1 se obtiene un término de marcado de dicha variable q_i , dado por el producto booleano del término mínimo correspondiente al estado anterior por el término de entrada que provoca dicha transición.

De igual forma para cada transición que pasa de la parte en que q_i vale 1 a la parte en que q_i vale 0 se obtiene un término de borrado de la variable q_i , dado asimismo por el producto booleano del término mínimo correspondiente al estado anterior por el término de entrada que provoca la transición.

Las funciones de marcado y borrado conducen, de por sí, a un diseño con biestables JK: J_i equivale, en tal sentido, a S_i y debe recoger todos los términos de marcado de q_i y K_i , que equivale R_i , debe recibir todos los términos de borrado:

J_i = suma booleana de los términos de marcado de q_i ;

K_i = suma booleana de los términos de borrado de q_i .

En el ejemplo anterior, la obtención de las funciones de marcado y borrado directamente desde el propio grafo de estados, conduce a:

$$\begin{aligned} q_1: J_1 &= \bar{q}_2 \cdot \bar{q}_1 \cdot a + q_2 \cdot \bar{q}_1 \cdot a = \bar{q}_1 \cdot a & K_1 &= \bar{q}_2 \cdot q_1 + q_2 \cdot q_1 \cdot \bar{a} = q_1 \cdot (\bar{a} + \bar{q}_2) \\ q_2: J_2 &= \bar{q}_2 \cdot q_1 \cdot a & K_2 &= q_2 \cdot \bar{q}_1 \cdot \bar{a} + q_2 \cdot q_1 = q_2 \cdot (\bar{a} + q_1). \end{aligned}$$

Resulta sencillo pasar de las funciones de marcado y borrado $J_i K_i$ a biestables tipo D, mediante la siguiente transformación funcional:

$$D_i = \bar{q}_i \cdot J_i + q_i \cdot \bar{K}_i;$$

las condiciones de marcado actúan cuando la variable de estado se encuentra a 0 y las de borrado cuando se encuentra a 1. La expresión anterior coincide, obviamente, con la forma de construir un biestable JK a partir de uno D (apartado 13.1, página 65).

En el caso que nos ocupa:

$$D_1 = \bar{q}_1 \cdot J_1 + q_1 \cdot \bar{K}_1 = \bar{q}_1 \cdot a + q_1 \cdot a \cdot q_2 = a \cdot (q_2 + \bar{q}_1)$$

$$D_2 = \bar{q}_2 \cdot J_2 + q_2 \cdot \bar{K}_2 = \bar{q}_2 \cdot q_1 \cdot a + q_2 \cdot a \cdot \bar{q}_1 = a \cdot (q_2 \oplus q_1)$$

que coinciden con las obtenidas directamente de la tabla funcional.

Este diseño directo a partir del grafo o de la máquina de estados conduce a funciones booleanas que pueden quedar menos simplificadas que en el caso de construir las tablas funcionales; además, tiene el riesgo de no considerar alguna situación particular que no se encuentre expresamente reflejada en el grafo de estados y que pudiera conducir a errores de funcionamiento. A costa de asumir este riesgo y una menor simplificación, se evita la pesada tarea de construir fila a fila la tabla funcional.

Se indica, a continuación, la forma de obtener las funciones de activación de las entradas J_i K_i desde la tabla de evolución del estado: será preciso ampliar dicha tabla con las condiciones de marcado J_i y borrado K_i de cada una de las variables de estado q_i , a partir de la siguiente correspondencia entre la evolución de q_i y los valores de J_i y de K_i .

$q_i(t)$	$\rightarrow$	$q_i(t+1)$	$\Rightarrow$	J_i	K_i
0		0		0	X
0		1		1	X
1		0		X	1
1		1		X	0

lo cual permite expresar la tabla de verdad para las entradas J_i y K_i de los biestables y, a partir de ella, construir las funciones booleanas que deben actuar sobre dichas entradas:

$$J = f'(X, Q) \quad K = f''(X, Q)$$

En la práctica no tiene mucho interés el diseño con biestables **JK** utilizando la tabla de evolución del estado, ya que es un proceso más complejo que para biestables tipo **D** y, además, es más habitual la disponibilidad de los biestables **D** (entre otras razones porque son más básicos y de circuitería más reducida y porque son los biestables típicos de los dispositivos programables y de las librerías de celdas estándar para el diseño de ASICs).

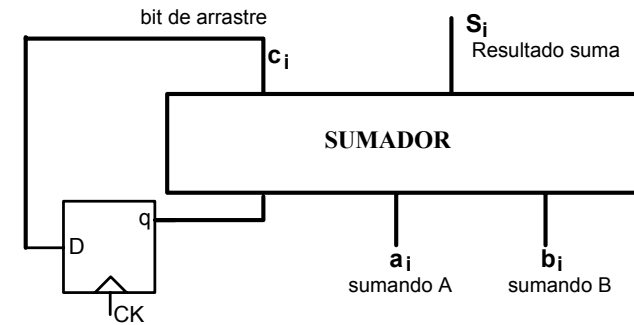
En el caso del detector de una secuencia de bits consistente en tres «unos» seguidos, la tabla funcional de evolución del estado, incluyendo las columnas correspondientes a la activación de las entradas **J** y **K**, es la representada en la página siguiente.

q_2	q_1	a	q_2^+	q_1^+	$J_2 K_2$	$J_1 K_1$	
0	0	0	0	0	0 X	0 X	$q_1:$
		1	0	1	0 X	1 X	
0	1	0	0	0	0 X	X 1	$K_1 = \bar{a} + \bar{q}_2$
		1	1	0	1 X	X 1	
1	0	0	0	0	X 1	0 X	$q_2:$
		1	1	1	X 0	1 X	
1	1	0	0	0	X 1	X 1	$J_2 = q_1.a$
		1	0	1	X 1	X 0	
							$K_2 = \bar{a} + q_1$

14.2. Ejercicios de diseño de sistemas secuenciales síncronos

14.2.1. Sumador secuencial: realiza la suma de dos números binarios, los cuales recibe «en serie», bit a bit, por dos líneas en unidades de tiempo sucesivas (comenzando por el dígito de menor valor significativo).

Este sumador requiere memoria, por cuanto que el bit de arrastre o acarreo que se produce al sumar dos dígitos ha de actuar como operando (como entrada) en la suma de los siguientes dígitos; el correspondiente sistema secuencial ha de ser síncrono pues ha de adaptarse a las unidades de tiempo en las cuales recibe los sucesivos dígitos de los números a sumar.



Este sumador secuencial se reduce a una celda sumadora básica (apartado 3.1, primer volumen) a la que se añade un biestable **D** para almacenar el arrastre.

Las tablas funcionales de evolución del estado c_i (bit de acarreo) y de activación de la salida s_i (resultado de la suma) son las mismas que en la celda sumadora, sólo que aquí la entrada y la salida de acarreo corresponden a una misma variable de estado:

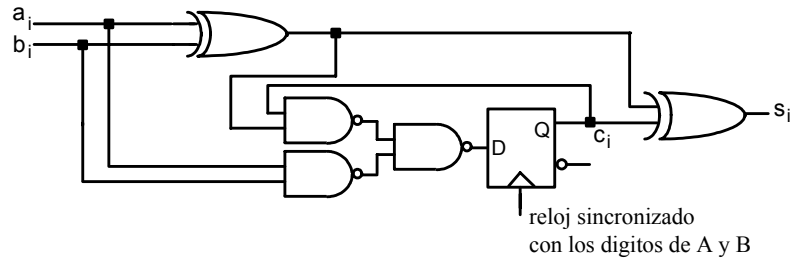
c_i	b_i	a_i	c_i^+	s_i
0	0	0	0	0
	0	1	0	1
	1	0	0	1
	1	1	1	0
1	0	0	0	1
	0	1	1	0
	1	0	1	0
	1	1	1	1

Resolviendo este diseño con un biestable **D**, para almacenar c_i , resulta:

$$\text{variable de salida} \quad s_i = \bar{c}.(\bar{b}.a + b.\bar{a}) + c.(\bar{b}.\bar{a} + b.a) = c \oplus (a \oplus b)$$

$$\text{variable de estado} \quad c_i: \quad c^+ = b.a + \bar{c}.\bar{b}.a + c.b.\bar{a} = b.a + (b \oplus a).c$$

que dan como resultado el circuito de la figura siguiente.



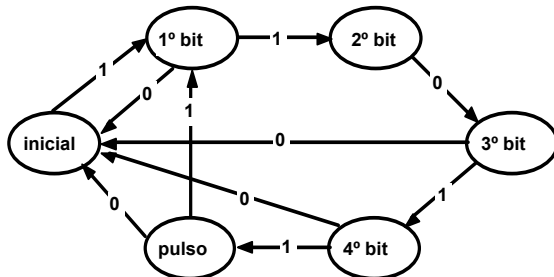
Sumador secuencial

14.2.2. Detector de una secuencia de bits: un sistema con una sola entrada **A**, por la cual recibe dígitos binarios en unidades de tiempo sucesivas, responde con un pulso (de duración igual a una unidad de tiempo) cuando detecta la llegada de una determinada secuencia, por ejemplo, la secuencia **11011**.

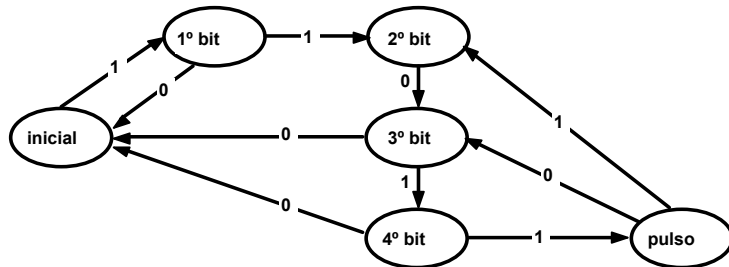
El problema de detección de una secuencia genérica (en este caso, la detección de **11011**) es diferente según se admita o no la posibilidad de solape (es decir, que la sucesión **11011011** produzca doble detección de secuencia, la primera correspondiente a los cinco primeros bits y la segunda relativa a los cinco últimos, dos de ellos solapados en ambas, o solamente produzca una detección).

Los grafos de estado correspondientes a ambas posibilidades serán los siguientes:

a) sin solape



b) con solape



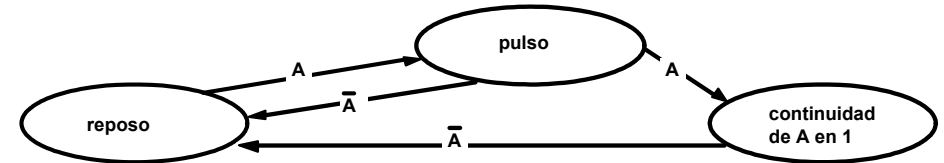
Ambos casos pueden ser codificados con tres variables de estado, su tabla de evolución del estado tendrá a cuatro variables de entrada (las de estado más la entrada **A**) y la salida se activará en el estado anotado como *pulso*.

Se deja para el lector (como ejercicio de aplicación) la codificación de los estados, la construcción de las tablas de evolución del estado y la obtención, a partir de dichas tablas, de las funciones de activación de los biestables D.

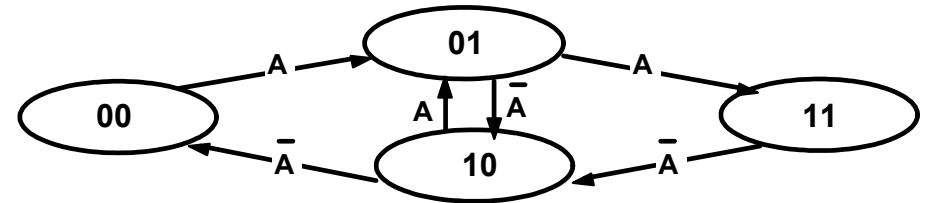
La detección con solape podría resolverse, también, con un registro de desplazamiento (y el término mínimo correspondiente a la secuencia a detectar), pero en este caso serían necesarios cinco biestables, frente a los tres que requiere el anterior grafo de estados.

14.2.3. Detección síncrona de flancos de subida: dada una señal proveniente de un pulsador **A** (o, simplemente, dada cualquier señal digital **A**) se desea generar un pulso cuya duración sea igual a una unidad del tiempo de reloj, cada vez que se active el pulsador (cada vez que se produzca un flanco de subida en la señal **A**).

Un posible grafo de estados de este sistema es el siguiente:



Son necesarias, al menos, dos variables de estado para codificar el grafo anterior; la introducción de un cuarto estado no complica en modo alguno dicho grafo y, en cambio, introduce una gran simetría en el mismo:



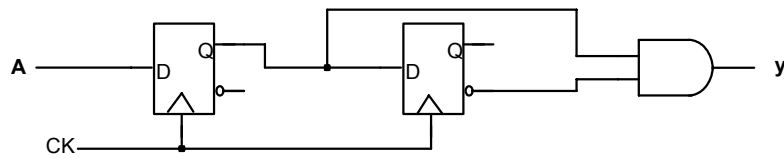
De la codificación del ciclo principal del grafo en código Gray resulta la siguiente tabla funcional:

q2	q1	A	q2 ⁺	q1 ⁺
0	0	0	0	0
		1	0	1
0	1	0	1	0
		1	1	1
1	0	0	0	0
		1	0	1
1	1	0	1	0
		1	1	1

que, realizada con biestables D, da lugar a un registro de desplazamiento de dos bits:

$$D_1 = A \quad D_2 = q_1;$$

siendo la función de salida (estado **01**) la siguiente: $y = \overline{q_2} \cdot q_1$.



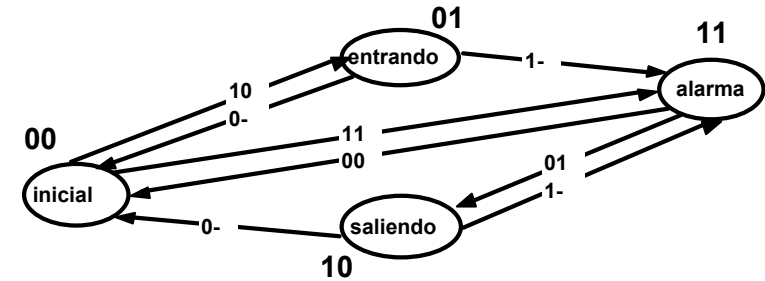
El resultado es obvio: para detectar un flanco de subida en la señal **A** basta retener el valor de dicha señal durante dos unidades de tiempo seguidas y seleccionar la secuencia **01** que corresponde a un flanco ascendente (paso de **0**, valor previo, a **1**, valor siguiente).

Este circuito es un detector de flanco de subida, un módulo muy útil para el diseño secuencial, que aparece frecuentemente en sus diagramas de bloques (ver apartado 16.4, *Contaje de pulsos diferentes del reloj*).

El flanco de bajada se detecta con la secuencia **10**: $y = q_2 \cdot \overline{q_1}$, es decir con un circuito análogo, cambiando las salidas de los biestables que van a la puerta "y"; para detectar ambos flancos (el de subida y el de bajada) será necesaria una puerta "o-exclusiva" que reciba las salidas de ambos biestables: $y = \overline{q_2} \cdot q_1 + q_2 \cdot \overline{q_1} = q_2 \oplus q_1$.

14.2.4. Activación gradual de alarma: un sistema de detección de temperatura proporciona cuatro niveles codificados en binario (**00**, **01**, **10**, **11**); la situación de alarma debe activarse cuando se detecta el nivel 3 **11** (temperatura muy alta), o si se detecta el nivel 2 **10** (alta) en dos ciclos seguidos de reloj y debe desaparecer cuando se detecta el nivel 0 **00** (muy baja), o si se detecta el nivel 1 **01** (baja) en dos ciclos de reloj consecutivos.

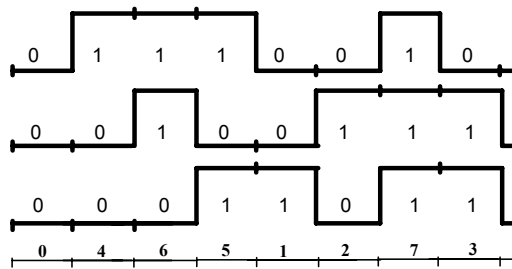
El grafo de estados del sistema de activación de la situación de alarma puede ser el representado en la siguiente figura; su codificación, con dos variables de estado, se ha indicado al lado de cada estado.



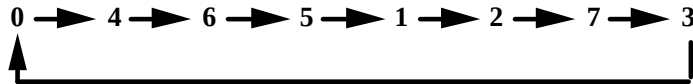
La situación de alarma corresponde a los estados **11** y **10**: $\text{alarma} = q_2$ y la tabla de evolución de estados será la siguiente:

q2	q1	b	a	q2 ⁺	q1 ⁺	
0	0	0	0	0	0	
		0	1	0	0	
		1	0	0	1	
		1	1	1	1	
0	1	0	0	0	0	
		0	1	0	0	
		1	0	1	1	
		1	1	1	1	
1	0	0	0	0	0	
		0	1	0	0	
		1	0	1	1	
		1	1	1	1	
1	1	0	0	0	0	$D_1 = b$
		0	1	1	0	
		1	0	1	1	$D_2 = b \cdot (a + q_2 + q_1) + q_2 \cdot q_1 \cdot a$
		1	1	1	1	

14.2.7. Generador de múltiples ondas repetitivas, sincronizadas entre sí: a partir de una señal de reloj de frecuencia fija se desea generar las formas de onda representadas en la figura, las cuales se repetirán indefinidamente.



La secuencia de valores booleanos correspondientes a estas ondas es : 0, 4, 6, 5, 1, 2, 7, 3,..., de manera que la generación de estas formas de onda corresponde a un contador cuya secuencia de contaje (grafo de estados) será:

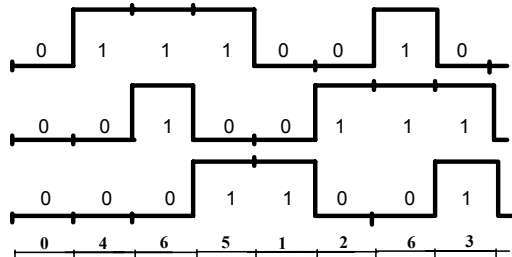


Su diseño requiere tres variables de estado (3 biestables), sin variables de entrada.

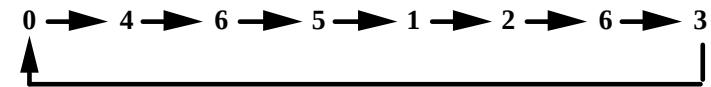
q3	q2	q1	q3 ⁺	q2 ⁺	q1 ⁺
0	0	0	1	0	0
0	0	1	0	1	0
0	1	0	1	1	1
0	1	1	0	0	0
1	0	0	1	1	0
1	0	1	0	0	1
1	1	0	1	0	1
1	1	1	0	1	1

$$D_1 = q_3 \cdot q_1 + q_2 \cdot \overline{q_1} \quad D_2 = q_3 \oplus (q_2 \oplus q_1) \quad D_3 = \overline{q_1}$$

Si las formas de onda hubieran sido las de la figura siguiente:



la secuencia de contaje sería:



en la cual se repite el estado 6.

El código 6 se asigna a dos estados distintos (que presentan transiciones diferentes); será preciso diferenciar ambos estados, lo cual puede conseguirse con una variable adicional **q4**, cuyo valor sea indiferente X en todos los estados salvo en los dos que es preciso distinguir:

q4	q3	q2	q1	q4 ⁺	q3 ⁺	q2 ⁺	q1 ⁺
X	0	0	0	X	1	0	0
X	0	0	1	X	0	1	0
X	0	1	0	1	1	1	0
X	0	1	1	X	0	0	0
X	1	0	0	0	1	1	0
X	1	0	1	X	0	0	1
0	1	1	0	X	1	0	1
1	1	1	0	X	0	1	1
X	1	1	1	X	X	X	X

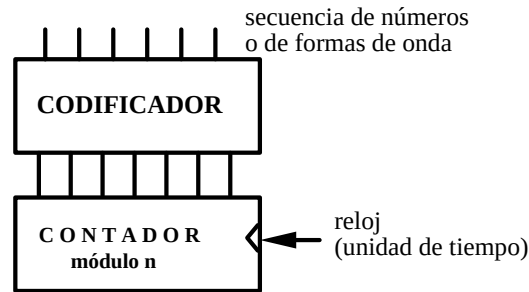
$$D_1 = q_3 \cdot (q_2 + q_1) \quad D_2 = q_3 \oplus (q_2 \oplus q_1) + q_4 \cdot q_3 \cdot q_2$$

$$D_3 = (\overline{q_4} + \overline{q_3} + \overline{q_2}) \cdot \overline{q_1} \quad D_4 = q_2$$

En los dos casos anteriores las correspondientes formas de onda (representadas en las figuras) aparecen en las salidas de los biestables; en el segundo caso, aparecen en las salidas de los tres últimos biestables, mientras que el biestable más significativo sirve simplemente para distinguir dos estados en los que el valor de los otros tres biestables es idéntico.

Esta forma de diseño supone que las formas de onda o, lo que es lo mismo, la secuencia de números binarios a obtener se produce directamente sobre las salidas de los biestables, es decir, las salidas coinciden con las variables de estado.

Otra manera, igualmente válida, de resolver el diseño de una secuencia numérica cualquiera, de longitud **n**, que se repite indefinidamente (**n** pasos de reloj), consiste en utilizar un contador módulo **n**, seguido de un codificador (convertor de código) que transforme los valores numéricos sucesivos del contador (0, 1, 2, 3, ..., **n-1**) en los códigos correlativos de la secuencia deseada.

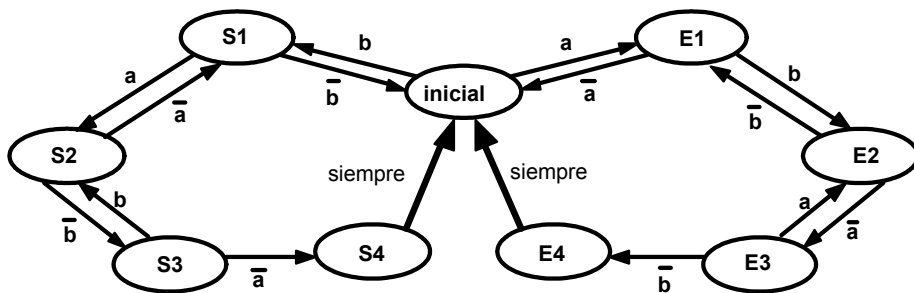


Supongamos que programamos sobre un bloque **PLS** el generador de ondas repetitivas diseñado en la página anterior: se necesitan 4 módulos para las 4 variables de estado **q4 q3 q2 q1** (de las cuales, las tres últimas coinciden con las variables de salida); pero, si la programación corresponde a la configuración anterior contador-codificador, el número de módulos necesarios es de 6 (3 para el contador de 8 estados y 3 para el codificador que proporciona las salidas).

14.2.8. Detector de número de personas presentes: supuesto que exista un solo acceso que es, a la vez, entrada y salida y que las personas entran y salen de una en una y con una mínima distancia entre ellas, un par de células fotoeléctricas próximas **a** y **b** permite detectar la salida o entrada de personas y el sentido en que cruzan dicho acceso.

Siendo **a** la célula fotoeléctrica exterior, las secuencias normales de las situaciones de entrada y de salida son, respectivamente:

- entrada: $\bar{a}.\bar{b}$ $a.\bar{b}$ $a.b$ $\bar{a}.b$ $\bar{a}.\bar{b}$
 - salida: $\bar{a}.\bar{b}$ $\bar{a}.b$ $a.b$ $a.\bar{b}$ $\bar{a}.\bar{b}$



La figura anterior representa un primer grafo de la evolución de los estados de este sistema secuencial; los estados **E4** y **S4** se presentan durante una sola unidad de tiempo y corresponden a las salidas: incrementar y decrementar en una persona, respectivamente.

Supuesto que el conteo de las personas presentes se efectúe con un contador bidireccional, al estado **E4** le corresponde contar una unidad y al estado **S4** le corresponde descontar una unidad:

- habilitación del contador (estados **E4** y **S4**): $E = q_2.\bar{q}_1$
- conteo ascendente (estado **E4**): "control ascendente/descendente" = $\bar{q}_3$.

Habida cuenta de que el número de estados es alto, conviene considerar la posibilidad de que algunos de ellos sean simplificables entre sí:

1.1 estados agrupables: los estados inicial, E1, E2 y E3

1.2 variables que los diferencian: **a** y **b** : **00, 10, 11, 01** respectivamente

1.3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado:

La transición con $\bar{b}$ debe producirse desde el estado **E3** hacia **E4** : $\bar{b}.\bar{a}.b$ ($= 0$); no es posible distinguir correctamente esta transición (una de las variables que diferencian al estado **E3** es la que produce la transición desde dicho estado) y, por tanto, no es posible efectuar esta agrupación de estados.

Lo mismo sucede en la transición con $\bar{a}$ desde el estado **E1** hacia el inicial.

Consideremos otras posibilidades de agrupación de estados:

2.1 estados agrupables: los estados **E1** y **E2**

2.2 variable que los diferencia: **b** : **0, 1**, respectivamente

2.3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado:

Una transición con $\bar{a}$ debe producirse desde el estado **E1** hacia el inicial : $\bar{a}.\bar{b}$

Asimismo, debe producirse otra transición con $\bar{a}$ desde el estado **E2** hacia **E3** : $\bar{a}.b$

El nuevo estado agrupado tendrá una transición de salida $\bar{a}.\bar{b}$ hacia el estado inicial y otra transición $a.b$ hacia el estado **E3**.

3.1 estados agrupables: los estados **S1** y **S2**

3.2 variable que los diferencia: **a** : **0, 1**, respectivamente

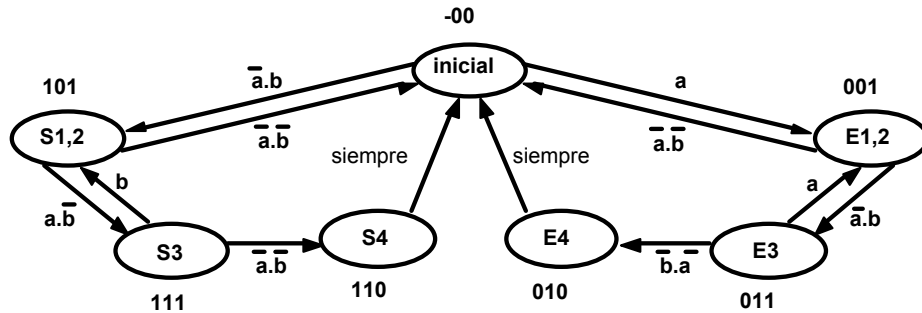
3.3 posibilidad de diferenciar correctamente las transiciones desde el nuevo estado:

Una transición con $\bar{b}$ debe producirse desde el estado **S1** hacia el inicial : $\bar{b}.\bar{a}$

También debe producirse otra transición con $\bar{b}$ desde el estado **S2** hacia **S3** : $\bar{b}.a$

El nuevo estado agrupado tendrá una transición de salida $\bar{b}.\bar{a}$ hacia el estado inicial y otra transición $b.a$ hacia el estado **S3**.

La agrupación de los estados **E1** y **E2** en uno solo **E1,2** y la simétrica de **S1** y **S2** en **S1,2** reduce el grafo a siete estados, codificables con tres variables **q3 q2 q1**.



Respetando la simetría del grafo, el estado inicial se ha codificado con **-00** (es indiferente para dicho estado el valor **q3**) y, para evitar marcados duplicados, se ha condicionado con $\bar{a}$ la transición que produce **b** desde dicho estado; lo mismo se ha hecho (evitar la posibilidad de marcados duplicados) con las transiciones que salen de los estados **E3** y **S3** (**b.a**).

Extrayendo las condiciones de marcado y de borrado del grafo de estados:

$$\begin{aligned} \mathbf{q1:} \quad J_1 &= \bar{q}_2 \cdot \bar{q}_1 \cdot (b + a) & K_1 &= q_1 \cdot \bar{b} \cdot \bar{a} \\ \mathbf{q2:} \quad J_2 &= \bar{q}_2 \cdot q_1 \cdot (\bar{q}_3 \cdot \bar{b} \cdot \bar{a} + q_3 \cdot \bar{b} \cdot a) & K_2 &= q_2 \cdot \bar{q}_1 + q_2 \cdot (q_3 \cdot b + \bar{q}_3 \cdot a) \\ \mathbf{q3:} \quad J_3 &= \bar{q}_2 \cdot \bar{q}_1 \cdot b \cdot \bar{a} & K_3 &= \bar{q}_2 \cdot \bar{q}_1 \cdot a \end{aligned}$$

Para construir este sistema secuencial biestables **D**, aplicaremos la transformación:

$$D = \bar{q} \cdot J + q \cdot \bar{K}$$

$$D_1 = (\bar{q}_2 + q_1) \cdot (b + a)$$

$$D_2 = \bar{q}_2 \cdot q_1 \cdot (\bar{q}_3 \cdot \bar{b} \cdot \bar{a} + \bar{q}_3 \cdot b \cdot \bar{a}) + q_2 \cdot q_1 \cdot (\bar{q}_3 \cdot \bar{b} + \bar{q}_3 \cdot a + \bar{b} \cdot \bar{a})$$

$$D_3 = \bar{q}_3 \cdot \bar{q}_2 \cdot \bar{q}_1 \cdot b \cdot \bar{a} + q_3 \cdot (q_2 + q_1 + \bar{a})$$

En este caso la tabla de evolución de estados presenta 5 variables (es decir, es amplia pero manejable); construyendo dicha tabla se obtienen las siguientes funciones (bastante más simplificadas que las anteriores):

$$D_1 = (\bar{q}_2 + q_1) \cdot (b + a)$$

$$D_2 = q_1 \cdot (\bar{q}_3 \cdot \bar{b} \cdot \bar{a} + \bar{q}_3 \cdot b \cdot \bar{a}) + q_2 \cdot q_1 \cdot \bar{b} \cdot \bar{a}$$

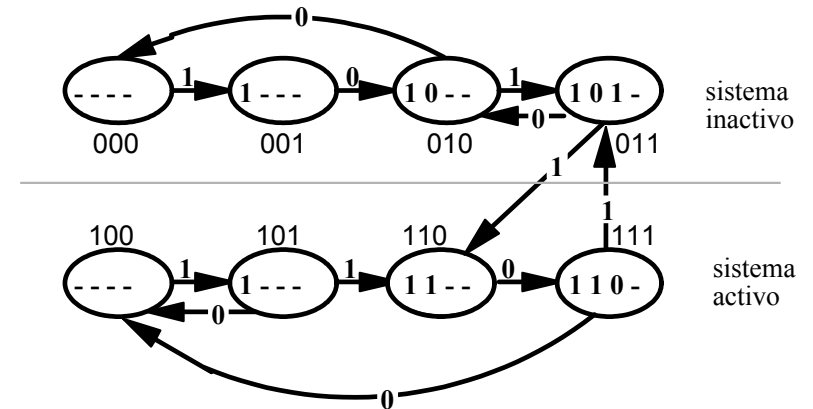
$$D_3 = \bar{q}_1 \cdot \bar{a} + q_3 \cdot q_1$$

14.2.9. Activación y desconexión de un sistema por detección de secuencia: un sistema eléctrico es controlado a través de una línea **a** por la que recibe valores booleanos sucesivos, sincronizados con un reloj de 1 MHz; el sistema se activa al recibir la secuencia **1011** y se desactiva cuando detecta **1101** (se admite el solapamiento entre secuencias. Diseñar el circuito de activación/desactivación, de forma que pueda ser programado sobre un circuito **PLS** y la salida (activo = 1, desactivo = 0) se corresponda con una de las variables de estado.

La cuestión no consiste en detectar dos secuencias **1011** y **1101**, sino en diferenciar dos situaciones activo/inactivo: en la situación de sistema inactivo interesa detectar la secuencia **1011** que lo activa y cuando el sistema se encuentra activo importa detectar la otra secuencia **1101** que lo lleva a inactivo.

Habida cuenta que se admite el solape entre las secuencias a detectar, cuando el sistema se activa se encuentran ya detectados los dos primeros dígitos de la secuencia que lo desactiva **1011** $\equiv$ **11--**; del mismo modo, al pasar a inactivo se encuentran detectados los tres primeros dígitos de la secuencia que lo puede activar **1101** $\equiv$ **101-**.

Un posible grafo de estados es el siguiente, en cuya codificación se ha reservado la variable **q3** para representar la situación de sistema activo **q3 = 1** o inactivo **q3 = 0**:



La función de salida, **y** = sistema activo, corresponde a la variable de estado **q3** y la tabla de evolución de los estados es la representada en la página siguiente.

q3	q2	q1	a	q3 ⁺	q2 ⁺	q1 ⁺
0	0	0	0	0	0	0
			1	0	0	1
0	0	1	0	0	1	0
			1	0	0	1
0	1	0	0	0	0	0
			1	0	1	1
0	1	1	0	0	1	0
			1	1	1	0
1	0	0	0	1	0	0
			1	1	0	1
1	0	1	0	1	0	0
			1	1	1	0
1	1	0	0	1	1	1
			1	1	1	0
1	1	1	0	1	0	0
			1	0	1	1

A partir de esta tabla las funciones de activación de los biestables $D_i = q_i^+$ serán las siguientes:

$$D_1 = q_3 \cdot q_2 \cdot q_1 \cdot a + q_3 \cdot q_2 \cdot \overline{q_1} \cdot \overline{a} + \overline{q_3} \cdot \overline{q_2} \cdot a + \overline{q_3} \cdot q_1 \cdot a + \overline{q_2} \cdot q_1 \cdot a$$

$$D_2 = q_3 \cdot q_2 \cdot q_1 + q_3 \cdot q_1 \cdot a + \overline{q_3} \cdot q_1 \cdot \overline{a} + q_2 \cdot a$$

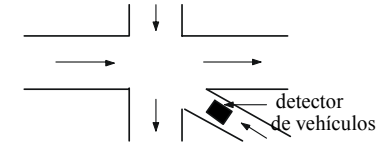
$$D_3 = q_3 \cdot q_2 + q_3 \cdot q_1 + q_3 \cdot a + \overline{q_3} \cdot q_2 \cdot q_1 \cdot a$$

Un diseño alternativo consiste en configurar un registro de desplazamiento de 4 biestables **q4 q3 q1 q1** para recibir la secuencia, detectándola sobre las salidas de dicho registro y utilizar un biestable adicional **q5** que se marque con la secuencia **1011** y se borre con la otra secuencia **1101**:

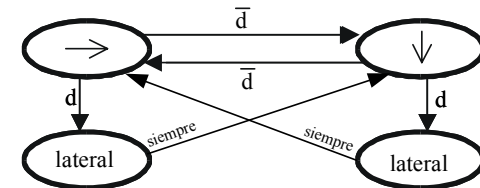
$$\begin{aligned} D_1 &= a & D_2 &= q_1 & D_3 &= q_2 & D_4 &= q_3 \\ D_5 &= \text{marcado} + q_5 \cdot \text{borrado} = q_4 \cdot q_3 \cdot q_2 \cdot q_1 + q_5 \cdot (q_4 \cdot q_3 \cdot q_2 \cdot q_1) = \\ &= q_4 \cdot q_3 \cdot q_2 \cdot q_1 + q_5 \cdot \overline{q_4} + q_5 \cdot \overline{q_3} + q_5 \cdot q_2 + q_5 \cdot \overline{q_1} \end{aligned}$$

Este diseño requiere 5 módulos **PLS** (uno de ellos con, al menos, 5 términos producto), mientras que el diseño anterior (como sistema secuencial directamente a partir del grafo de estados) solamente necesita 3 módulos **PLS**.

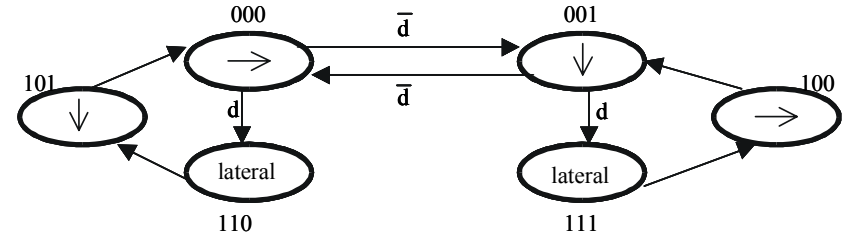
14.2.10 Un sencillo cruce de dos calles perpendiculares y unidireccionales dispone de semáforos (con sólo rojo y verde) que, cada 20" dan paso, alternativamente, a cada una de las dos direcciones; sobre dicho cruce incide una pequeña calle lateral (ver figura) que dispone de un sensor de vehículos **d**, de forma que el paso de dicha calle está normalmente interrumpido, pero, cuando se detecta un vehículo en ella, se espera a finalizar el intervalo de 20" y se da paso a dicha dirección lateral (interrumpiendo las otras dos), también durante 20". Diseñar el circuito de control de los semáforos de las tres calles.



Como los tiempos de los semáforos son todos ellos de 20 segundos, dicha temporización se resuelve directamente mediante un reloj de 20" de período (frecuencia 0,05 Hz). Un posible grafo de estados es el siguiente, siendo **d** la entrada conectada al sensor de vehículos:



Este grafo asigna prioridad a la calle lateral, de forma que cuando hay mucha demanda de paso por la misma, se produce un verde en la calle lateral después de cada paso en una de las otras direcciones. Parece más adecuado asegurar el paso de las otras dos direcciones entre cada dos verdes de la calle lateral, para lo cual sirve el grafo de estados siguiente (todas las transiciones se producen con los flancos activos del reloj, es decir, cada 20" y las transiciones no anotadas se producen siempre).



semáforo calle horizontal:

$$\text{verde} = \overline{q_2} \cdot q_1$$

semáforo calle vertical:

$$\text{verde} = \overline{q_2} \cdot q_1$$

semáforo calle lateral:

$$\text{verde} = q_3 \cdot q_2$$

La tabla de evolución de los estados es la siguiente.

q3	q2	q1	d	q3 ⁺	q2 ⁺	q1 ⁺
0	0	0	0	0	0	1
			1	1	1	0
0	0	1	0	0	0	0
			1	1	1	1
0	1	0	0	X	X	X
			1	X	X	X
0	1	1	0	X	X	X
			1	X	X	X
1	0	0	0	0	0	1
			1	0	0	1
1	0	1	0	0	0	0
			1	0	0	0
1	1	0	0	1	0	1
			1	1	0	1
1	1	1	0	1	0	0
			1	1	0	0

A partir de esta tabla las funciones de activación de los biestables serán las siguientes:

$$D_1 = q_3 \cdot \overline{q_1} + \overline{q_3} \cdot q_1 \cdot d + \overline{q_1} \cdot \overline{d}$$

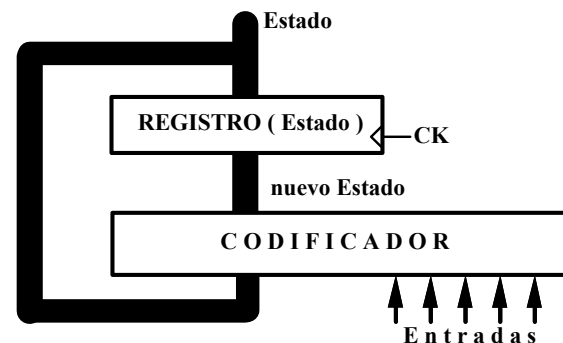
$$D_2 = \overline{q_3} \cdot \overline{q_2} \cdot d$$

$$D_3 = q_2 + \overline{q_3} \cdot d$$

14.3. Configuración microprogramada de grafos de estado

El diseño de un grafo o máquina de estados puede ser abordado en dos formas conceptualmente muy diferentes:

- según el método aplicado en los apartados anteriores, que trata cada variable de estado por separado, reflejando las transiciones en condiciones de activación de sus biestables individuales –forma microcableada–;
- recogiendo el estado sobre un registro global y codificando los cambios de estado a partir del estado anterior y del vector de entradas, mediante un codificador que calcula el nuevo estado –forma microprogramada–.



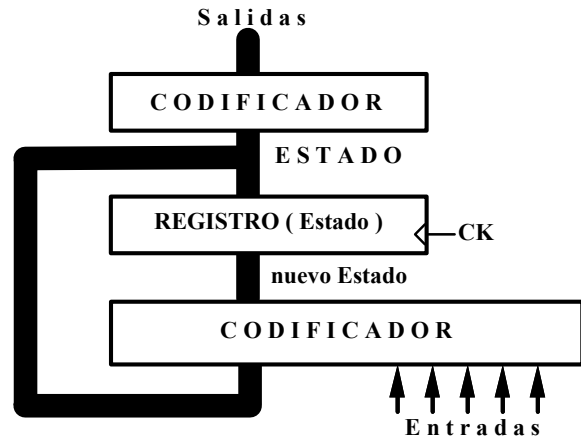
La configuración ROM de los codificadores permite construirlos directamente desde su tabla funcional, sin necesidad de obtener la expresión algebraica de sus funciones: bastará trasladar la tabla de evolución de los estados a la «Matriz O» del codificador; podemos considerar que este codificador contiene «microinstrucciones» relativas al cálculo del estado siguiente, «numeradas» por el estado anterior y el vector de entrada.

La forma microprogramada conserva en forma explícita la estructura del grafo de estados, pues el codificador expresa directamente la correspondencia entre la situación actual (estado y vector de entrada) y el nuevo estado; por ello, resulta fácil efectuar modificaciones de esta máquina de estados, cambiando las correspondientes «microinstrucciones» (la programación) del codificador.

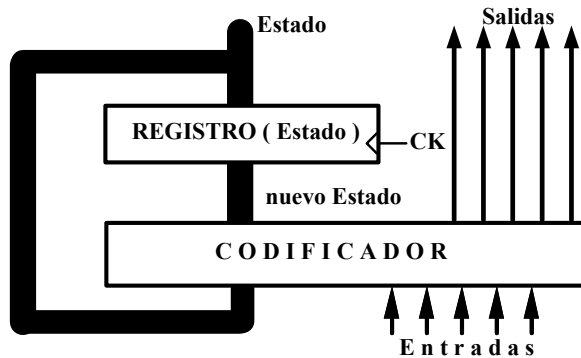
La «programación» del codificador (es decir, su Matriz O) corresponde a la tabla de evolución de estados; en tal sentido, consideremos la configuración «microprogramada» de los ejercicios detallados en el apartado anterior de este mismo capítulo:

- 14.2.4: su tabla de evolución de estados se encuentra en la página 90, requiere 2 variables de estado y 2 de entrada: el codificador tendrá 4 entradas y 2 salidas;
- 14.2.5: (pág. 91) 3 variables de estado: codificador de 3 entradas y 3 salidas;
- 14.2.6: (pág. 92) 3 variables estado y 1 entrada: codificador de 4 entradas y 3 salidas;
- 14.2.7: (pág. 93) 3 variables de estado: codificador de 3 entradas y 3 salidas;
- 14.2.8: (pág. 97) 3 variables estado y 2 entradas: codificador de 5 entradas y 2 salidas;
- 14.2.9: (pág. 98) 3 variables estado y 1 entrada: codificador de 4 entradas y 3 salidas.

Respecto al vector de salida, en el caso de autómata de Moore un segundo codificador puede calcular las salidas a partir del estado (en muchas ocasiones las salidas serán variables de estado y no es necesario el codificador); podemos considerar que este codificador contiene las «microinstrucciones» referidas a las salidas que corresponden a cada estado:



En los autómatas de Mealy, como las salidas dependen del estado y de las entradas, el mismo codificador puede calcular el nuevo estado y el vector de salidas:



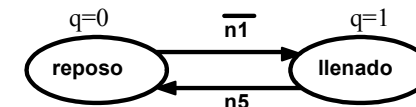
En estas configuraciones «microprogramadas» el tamaño de los codificadores aumenta fuertemente con el número de sus variables de entrada: se duplica cada nueva entrada (dependencia según 2^m); de ahí el interés en reducir el número de variables de estado (codificaciones de «un solo uno» resultan altamente desaconsejables) y, cuando ello sea posible, utilizar técnicas de multiplexado de las variables del vector de entrada (suele suceder que no todas las entradas actúan en todos los estados, de forma que el número efectivo de entradas puede reducirse por multiplexado).

14.4. Los grafos de estado en VHDL

Como ejemplo de descripción VHDL de grafos de estado se incluye la correspondiente al control de llenado de un depósito con una mezcla de cuatro líquidos diferentes (apartado 11.4, páginas 29-31); se detalla su diseño síncrono en las dos formas de autómata de Mealy y autómata de Moore.

Un depósito se llena con una mezcla de cuatro líquidos, para lo cual dispone de cuatro electroválvulas **A**, **B**, **C**, **D**, y de cinco detectores de nivel **n1**, **n2**, **n3**, **n4** y **n5**; cuando el nivel del depósito desciende por debajo del mínimo **n1** se produce un ciclo de llenado: primero **A** hasta el nivel **n2**, luego **B** hasta **n3**, **C** hasta **n4** y, finalmente, **D** **n5**.

Autómata de Mealy



```
constant reposo    : bit := '0';
constant llenado   : bit := '1';
signal estado      : bit ;
```

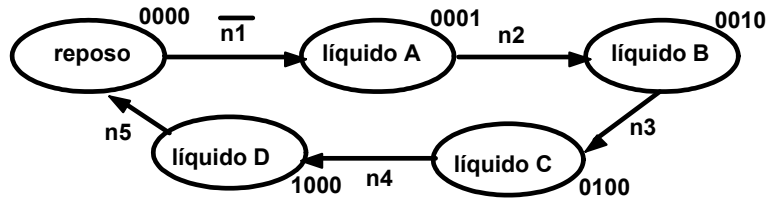
```
process
begin
wait until CK = '1';
if ( Reset = '1' ) then
    estado <= reposo;
else
    case estado is
        when reposo =>
            if (n1 = '0') then
                estado <= llenado;
            end if;
        when llenado =>
            if (n5 = '1') then
                estado <= reposo;
            end if;
    end case;
end if;
end process;
```

(téngase en cuenta que *proceso* conserva el estado cuando no se indica lo contrario: por ello no es necesario añadir *else estado <= estado;*)

--funciones de activación de las salidas:

```
A <= '1' when (estado = llenado) and (n2='0') else '0';
B <= '1' when (estado = llenado) and (n2='1') and (n3='0') else '0';
C <= '1' when (estado = llenado) and (n3='1') and (n4='0') else '0';
D <= '1' when (estado = llenado) and (n4='1') else '0';
```

Autómata de Moore



```

constant reposo      : bit_vector (4 downto 1):="0000";
constant liquido_A   : bit_vector (4 downto 1):="0001";
constant liquido_B   : bit_vector (4 downto 1):="0010";
constant liquido_C   : bit_vector (4 downto 1):="0100";
constant liquido_D   : bit_vector (4 downto 1):="1000";
signal estado        : bit_vector (4 downto 1);

```

```

process
begin
wait until CK = '1';
if ( Reset = '1' ) then
    estado <= reposo;
else case estado is
    when reposo => if (n1 = '0') then estado <= liquido_A; end if;
    when liquido_A => if (n2 = '1') then estado <= liquido_B; end if;
    when liquido_B => if (n3 = '1') then estado <= liquido_C; end if;
    when liquido_C => if (n4 = '1') then estado <= liquido_D; end if;
    when liquido_D => if (n5 = '1') then estado <= reposo; end if;
    when others =>
        end case;
end if;
end process;

```

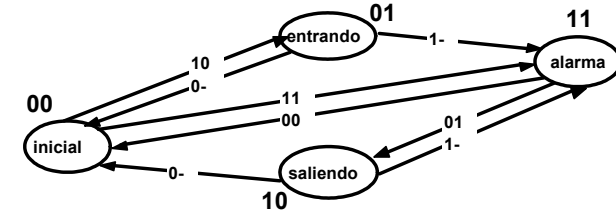
```

--funciones de activación de las salidas:
A <= '1' when estado = liquido_A else '0';
B <= '1' when estado = liquido_B else '0';
C <= '1' when estado = liquido_C else '0';
D <= '1' when estado = liquido_D else '0';

```

El ejemplo anterior solamente presenta una transición desde cada estado y en cada una de ellas actúa solamente una variable de entrada; el siguiente grafo, referido a la activación gradual de alarma (apartado 14.2.4, página 88), incluye mayor número de transiciones entre estados y dos variables de entrada involucradas en ellas.

Un sistema de detección de temperatura con cuatro niveles (00, 01, 10, 11); la alarma debe activarse cuando se detecta 11 (temperatura muy alta), o si se detecta el nivel 10 (alta) en dos ciclos seguidos de reloj y debe desaparecer cuando se detecta 00 (muy baja), o si se detecta el nivel 01 (baja) en dos ciclos de reloj.



```

constant inicial      : bit_vector (2 downto 1):="00";
constant entrando     : bit_vector (2 downto 1):="01";
constant saliendo     : bit_vector (2 downto 1):="10";
constant alarma       : bit_vector (2 downto 1):="11";
signal estado         : bit_vector (2 downto 1);

process
begin
wait until CK = '1';
if ( Reset = '1' ) then
    estado <= reposo;
else case estado is
    when inicial => if (entrada = "10") then estado <= entrando;
                    elsif (entrada = "11") then estado <= alarma; end if;
    when entrando => if (entrada(2) = '1') then estado <= alarma;
                    elsif (entrada(2) = '0') then estado <= inicial; end if;
    when saliendo => if (entrada(2) = '1') then estado <= alarma;
                    elsif (entrada(2) = '0') then estado <= inicial; end if;
    when alarma => if (entrada = "00") then estado <= inicial;
                   elsif (entrada = "01") then estado <= saliendo; end if;
    end case;
end process;

```

```

--función de activación de la salida:
y <= '1' when (estado = alarma) or (estado = saliendo) else '0';

```

15 EL SINCRONISMO: SIGNIFICADO, UTILIDAD Y REQUISITOS

- 15.1. Necesidad y conveniencia del sincronismo
- 15.2. Simultaneidad y estabilidad
- 15.3. Delimitación de caminos
- 15.4. Separación entre las dos fases del reloj
- 15.5. Precauciones relativas a la señal de sincronismo

Para abordar «la complejidad» resulta útil la idea de estructura, en cuanto a «disposición, orden y enlace de las partes para conformar un todo». El tiempo, aunque inmaterial, es también una de las «partes» conformadoras de un sistema complejo y el sincronismo confiere una organización estructural que simplifica el manejo del tiempo al cuantificarlo y numerarlo en unidades sucesivas.

El sincronismo no es sino una partición aplicada al tiempo, una forma de utilizar la estrategia de «divide y vencerás» en relación con una variable particularmente compleja por su carácter «continuo» e impreciso cual es el tiempo.

El sincronismo es un «estilo de diseño» que simplifica la planificación temporal (permite referir su actividad funcional a unidades de tiempo discretas y numerables) y aporta alta seguridad funcional; en grandes líneas, el sincronismo sirve para fijar el tiempo de cada bit en el procesamiento serie, coordinar temporalmente las diversas partes de un sistema, simplificar los cálculos de tiempos funcionales de los componentes y, sobre todo, garantizar la seguridad de funcionamiento del sistema.

Dada su importancia y utilidad, se requiere una comprensión clara del significado conceptual del sincronismo, del mecanismo implícito en el mismo y de sus consecuencias prácticas, tanto las prestaciones o ventajas que ofrece como los requisitos o condiciones que exige. En tal sentido, el presente capítulo trata de profundizar en:

- *los procesos que se desarrollan dentro de cada unidad de tiempo,*
- *su relación con los «caminos» entre biestables (la forma en que la división del tiempo en unidades se refleja en la división del espacio en recorridos entre biestables) y el análisis de tiempos aplicado a dichos caminos,*
- *y los requisitos que hacen efectivo el sincronismo.*

El sincronismo exige una «meta separación entre las dos fases de la señal de reloj» como idea clave para la correcta actuación de los biestables síncronos, lo cual requiere el adecuado diseño de dichos biestables (de forma que no puedan cambiar de estado en el flanco no activo del reloj), el cumplimiento del tiempo de permanencia del dato en sus entradas y el que la señal de reloj sea vertical, simultánea y fuerte (en cuanto a capacidad de intensidad) y no se efectúe ninguna clase de lógica sobre dicha señal.

15.1. Necesidad y conveniencia del sincronismo

El sincronismo puede servir para:

- determinar el tiempo de actuación que corresponde a cada dígito o bit,
- coordinar temporalmente los diversos componentes de un sistema,
- planificar su comportamiento dividiéndolo en acciones sucesivas,
- simplificar los cálculos relativos a los tiempos funcionales de los biestables
- y, sobre todo, garantizar la seguridad de funcionamiento del sistema.

Hay problemas que por su propia naturaleza requieren un diseño síncrono, por ejemplo, la transmisión y el procesamiento serie en los cuales se procesa un bit tras otro (o una palabra tras otra), de forma que es preciso definir el intervalo temporal de duración de cada bit.

En sistemas con partes que trabajan a diferentes velocidades, el sincronismo permite coordinar los diferentes tiempos de retraso entre unas y otras. En general, en sistemas cuyos componentes presentan tiempos de propagación diferentes, interesa un diseño síncrono, con la frecuencia de reloj adaptada a la velocidad del elemento más lento, para evitar problemas con los tiempos de retraso de las señales y asegurar que dichos retrasos no den lugar a transiciones de estado o salidas no deseadas.

De esta forma, el sincronismo facilita el diseño de un sistema complejo, al permitir la planificación temporal de la actividad del sistema referida a unidades de tiempo discretas y ordenadas sucesivamente (numeradas). Sin duda es mucho más fácil razonar respecto a un parámetro cuantificado en intervalos discretos y numerados que frente a una variable de rango continuo. La cuantificación permite establecer correspondencias bien delimitadas entre unidades de tiempo y acciones sucesivas y facilita mucho la adaptación entre partes con diferentes velocidades.

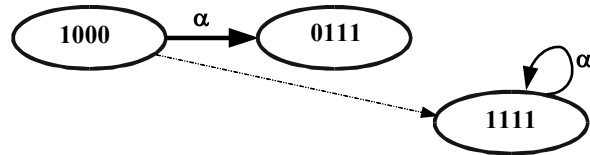
Asimismo, el sincronismo facilita los cálculos relativos a los tiempos de propagación (t_p) y demás tiempos funcionales de los componentes (t_s , t_h , ...) al establecer el concepto de «camino», como recorrido que han de efectuar las variables dentro de la unidad de tiempo.

Pero, sobre todo, el sincronismo proporciona alta seguridad de funcionamiento: todas las variables del sistema conmutarán a la vez (en los flancos activos del reloj) y, lo que es más importante, los valores que determinarán las transiciones serán los valores booleanos presentes en el momento anterior al flanco activo del reloj.

No importa que existan retrasos desiguales en las diversas señales, ya que son los valores consolidados previos a la transición (y no los que se producen durante ella) los que la controlan: cada unidad de tiempo es un intervalo dedicado a calcular el nuevo estado. Los valores transitorios de algunas variables que puedan formarse durante la transición de un estado al siguiente no afectan al sistema, porque el nuevo estado no es admitido hasta el siguiente flanco de reloj, cuando ya ha sido calculado por completo a lo largo de la unidad de tiempo.

En los sistemas combinacionales el tiempo interviene solamente como un retraso entre entradas y salidas (tiempo de propagación) que, en definitiva, limita la velocidad de trabajo del sistema. En los sistemas secuenciales, además de limitar la velocidad de trabajo, tales retrasos pueden dar lugar a errores funcionales que afectan a la evolución de los estados (presentando un comportamiento erróneo continuado, aún a velocidades de procesamiento admisibles).

Veamos un ejemplo sencillo: supongamos que un vector de entrada α activa una transición por la cual el estado **1000** debe cambiar al estado **0111**;



supongamos también que la variable de estado inicial q_4 es mucho más lenta (pasa a través de mayor número de puertas) que las otras tres q_3 q_2 q_1 : se formará transitoriamente el estado **1111** antes de completarse la transición a **0111**.

Pero, si el vector de entrada α no afecta al estado **1111** (si para dicho vector el estado **1111** no debe cambiar), puede suceder que el estado transitorio **1111** se consolide como nuevo estado (pues las condiciones booleanas que determinaban la transición a **0111** - estado **1000** y entrada α - han desaparecido) y, a partir de ese momento, la evolución del sistema será errónea.

Esto no sucederá si el sistema es síncrono (y está bien diseñado), pues el nuevo estado **0111** se encontrará ya calculado previamente al momento de la transición y será capturado en bloque, con el flanco activo de la señal de reloj.

Aparte de sus ventajas, el diseño síncrono resulta obligado en la práctica actual, dado que los módulos constitutivos de los dispositivos programables **CPLD** y **FPGA** son, de por sí, síncronos y también lo son los biestables habituales de las librerías de celdas para el diseño de circuitos integrados de aplicación específica **ASIC**.

Actualmente, habida cuenta de la alta densidad de integración de los bloques digitales disponibles, de la diversidad y del bajo coste de los mismos, no suele haber razones suficientes para configurar un sistema en forma asíncrono: cada vez más el diseño digital es un diseño síncrono.

15.2. Simultaneidad y estabilidad

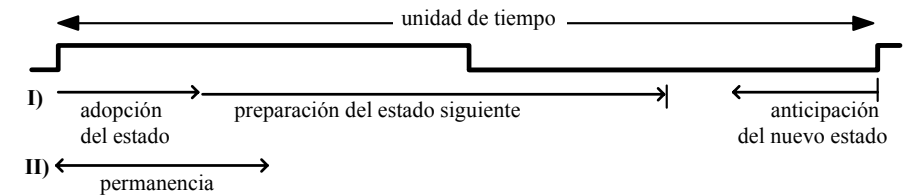
Simultaneidad en los cambios

y estabilidad a lo largo de la unidad de tiempo

El sincronismo se define por el hecho de que «el conjunto de estados» del sistema cambie «simultáneamente», en coordinación directa con la señal que separa las unidades de tiempo, a la que denominamos *reloj*. Ahora bien, el cambio «simultáneo» de variables físicas es mera «entelequia» si tal simultaneidad se entiende con total precisión; por ello, ha de reinterpretarse como cambio «a la vez» con dos condiciones determinantes:

- I) el nuevo estado a adoptar se encuentra preparado previamente al momento de cambio
- II) los valores de las variables correspondientes al nuevo estado no «actúan» hasta el siguiente momento de cambio (es decir, no provocan variaciones inmediatas sobre el propio estado).

El intervalo de una unidad de tiempo se dedica a los tres procesos temporales indicados en la parte I de la figura siguiente:



Habida cuenta de que ningún proceso físico es instantáneo, el cambio de estado que supone el inicio de una nueva unidad de tiempo requiere un cierto intervalo temporal (adopción del estado), después del cual el circuito prepara el estado siguiente (con la correspondiente suma de tiempos de propagación a través de puertas booleanas sucesivas) y dicho nuevo estado ha de estar preparado con suficiente antelación para asegurar que sea capturado correctamente en el cambio a la siguiente unidad de tiempo.

Por otro lado (parte II de la figura), también es necesario un intervalo de seguridad o permanencia en que el «estado preparado anteriormente» no cambie, para que no afecte a la propia adopción de dicho estado. Esta permanencia garantiza la condición II, es decir, la estabilidad del nuevo estado a lo largo de su unidad de tiempo.

En un circuito digital, el estado (en una perspectiva de conjunto) se identifica con las variables que intervienen en la conformación de las salidas del circuito. Además de las propias «variables internas de estado» que constituyen la memoria y que están contenidas en biestables, también será necesario ajustar las variables de entrada a las unidades de tiempo.

La conmutación de una entrada a mitad de la unidad de tiempo puede dar lugar a que sobre unas partes (biestables) del circuito actúe con su nuevo valor y sobre otras con el valor anterior, debido a los diferentes tiempos de propagación de los conjuntos de puertas que atraviesa.

El sincronismo de las entradas es necesario para asegurar la estabilidad de sus valores a lo largo de cada unidad de tiempo y, con ello, garantizar la preparación correcta del nuevo estado; además, en el caso de autómatas de Mealy, evita transitorios intermedios en las salidas.

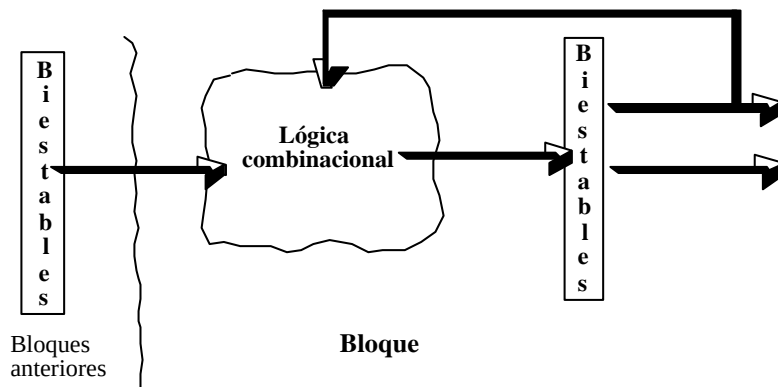
También resulta conveniente sincronizar las variables de salida del circuito, pues ello evita pequeños pulsos transitorios o «fisuras» (*glitches*) que pueden producirse al inicio de las unidades de tiempo, al efectuarse el cálculo de las salidas con el nuevo estado. Ahora bien, la sincronización de las salidas supone retrasarlas una unidad de tiempo de reloj, lo cual, en ocasiones, puede no interesar.

15.3. Delimitación de caminos

Un sistema síncrono se compone de biestables que almacenan las variables (variables de estado, de entrada y, si el sincronismo es total, también las de salida) y de partes combinacionales que conectan los biestables entre sí.

Un circuito secuencial síncrono puede «segmentarse» en bloques o módulos conformados por una parte combinacional que recibe las entradas al bloque y calcula las funciones booleanas que corresponden a la actividad del mismo y un registro o conjunto de biestables que recogen las salidas de la parte combinacional y conectan con el bloque o bloques siguientes; además, las salidas del propio bloque pueden realimentarse como entradas sobre el mismo.

De manera que cada «parte combinacional» se encuentra «emparedada» (a modo de *sándwich*) entre dos registros o conjuntos de biestables, uno de ellos correspondiente al bloque o bloques anteriores (entradas) y el otro, el propio del bloque (salidas).



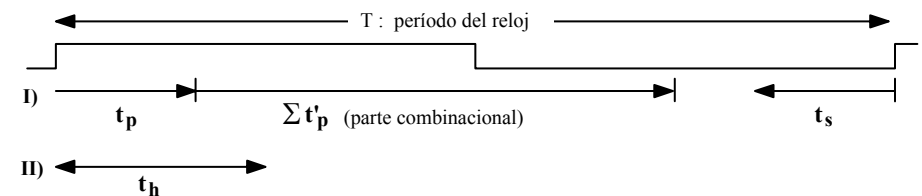
Incluso aquellos bloques que reciben entradas del exterior del sistema digital responden a esta configuración ya que, para sincronizarlas, las entradas se recibirán a través de biestables. Y lo mismo sucede con los bloques que emiten salidas del sistema hacia el exterior pues dichas salidas o bien corresponden directamente a registros o biestables del sistema o, caso de ser conformadas combinacionalmente, suelen incluir biestables que las sincronizan con el reloj central.

La figura anterior puede representar también al sistema secuencial completo, considerado como un único bloque que recibe entradas sincronizadas, a través de los biestables de la izquierda de la figura y emite salidas sincronizadas por biestables de la derecha (quedando, también a la derecha, otros biestables internos de estado que se realimentan sobre la lógica combinacional).

Tomando como referencia esta estructura de bloques secuenciales (figura anterior), entendemos por *camino* el recorrido de una señal desde una de las entradas del bloque hasta la entrada de uno de los biestables del mismo; es decir, un camino es el recorrido que llega a la entrada de un biestable desde la salida de uno de los biestables que inciden sobre el primero.

Un camino va de biestable **FF1** a biestable **FF2** atravesando solamente la parte combinacional del bloque al que corresponde el segundo de ellos **FF2**; obviamente, pueden existir caminos que conectan la salida de un biestable **FF2** con su entrada, caso de darse realimentación de dicho biestable sobre sí mismo.

El sincronismo permite «particionar» (dividir en partes) el cálculo de tiempos, aplicándolo a *caminos* definidos entre biestables, y calcular sobre ellos la velocidad máxima de reloj y las posibles violaciones de permanencia. La unidad de tiempo se refleja sobre el recorrido de cada camino entre biestable y biestable.



El tiempo de propagación de la señal por un «camino» corresponde a la suma de tiempos de propagación del primer biestable y de la parte combinacional que atraviesa el camino, más el tiempo de anticipación que requiere el segundo biestable:

$$t_p(\text{FF1}) + \sum t'_p(\text{parte combinacional}) + t_s(\text{FF2})$$

El tiempo de anticipación (*set-up*) del biestable receptor interviene en esta suma ya que es un tiempo adicional a los de propagación, necesario para el correcto funcionamiento del biestable.

La duración de una unidad de tiempo $T = 1/f_{CK}$ ha de ser suficiente para que todos los caminos completen la propagación de las señales a través de ellos, es decir:

$$T_{CK} > t_p(FF1) + \sum t'_p(\text{parte combinacional}) + t_s(FF2)$$

desigualdad que ha de cumplirse para todos los caminos existentes en el sistema digital. Caso de no respetarse esta desigualdad se produce una violación de la anticipación necesaria para el funcionamiento correcto del biestable **FF2**: «violación de *set-up*»; es decir, el nuevo dato a la entrada de dicho biestable no ha completado su preparación con el tiempo de anticipación suficiente (previo al flanco activo del reloj) para asegurar que será capturado correctamente.

Los tiempos de propagación de diversos caminos serán diferentes, ya que lo es la «parte combinacional» que atraviesa cada camino. La frecuencia máxima de trabajo del sistema digital vendrá limitada por aquellos caminos cuyos tiempos de propagación sean mayores: *caminos críticos*. Resulta, pues, importante detectar los caminos críticos de un sistema digital para tratar de reducir sus tiempos de propagación, caso de que interese aumentar la velocidad de trabajo.

El correcto funcionamiento de los biestables requiere, además de respetar su tiempo de anticipación, el mantenimiento del dato durante un tiempo de permanencia (*hold*) t_h posterior al flanco activo de reloj. El tiempo que tarda un dato en cambiar, en la entrada de un biestable, será el debido al retraso que sufre para recorrer el correspondiente camino, o sea, la suma de tiempos de propagación del biestable anterior y de la parte combinacional que atraviesa el camino:

$$t_p(FF1) + \sum t'_p(\text{parte combinacional}).$$

Para garantizar que el dato se mantiene estable durante un intervalo no inferior al tiempo de permanencia t_h ha de cumplirse que:

$$t_p(FF1) + \sum t'_p(\text{parte combinacional}) > t_h(FF2).$$

Caso de que esta desigualdad no se respete, se produce una violación de la permanencia del dato necesaria para asegurar el correcto funcionamiento del biestable **FF2**: «violación de *hold*»; es decir, el dato no se mantiene en la entrada de dicho biestable suficiente tiempo (después del flanco del reloj) para asegurar que será capturado correctamente.

En la primera desigualdad, referida a t_s , deben aplicarse los valores máximos de los tiempos de propagación, mientras que en la segunda, relativa a t_s , deben aplicarse tiempos de propagación mínimos:

$$T_{CK} > t_{p,máx}(FF1) + \sum t'_{p,máx}(\text{parte combinacional}) + t_{s,máx}(FF2)$$

$$t_h(FF2) < t_{p,min}(FF1) + \sum t'_{p,min}(\text{parte combinacional})$$

Estas dos desigualdades expresan las posibles violaciones de anticipación (*set-up*) o de permanencia (*hold*); con ellas, los analizadores de tiempos calculan los tiempos en cada camino, seleccionan los caminos críticos y detectan las violaciones de anticipación o de permanencia.

Obviamente las «violaciones de la anticipación» se resuelven directamente aceptando una frecuencia de trabajo suficientemente baja:

$$f_{CK} < 1/\text{máx}(t_{pc}) \text{ siendo } t_{pc} = t_{p,máx}(FF1) + \sum t'_{p,máx}(\text{combinacional}) + t_{s,máx}(FF2).$$

Cuando las «violaciones de la anticipación» afectan solamente a unos pocos caminos críticos cabe considerar la reducción de sus tiempos de propagación, mediante un rediseño de la parte combinacional que les afecta: $\sum t'_p(\text{parte combinacional})$.

En suma, la consecución de altas velocidades de trabajo pasa por la reducción de los tiempos de los caminos críticos y para ello se dispone de tres sumandos: el tiempo de propagación del primer biestable, el de la parte combinacional y el tiempo de anticipación del segundo biestable; de estos tres, salvo en casos excepcionales de biestables lentos, el mayor y más maleable suele ser el tiempo de propagación de la parte combinacional.

Para resolver las «violaciones de la permanencia» bastará aumentar el tiempo de propagación de la parte combinacional, incluyendo en ella un par de inversores sucesivos o, si la violación es relativamente pequeña, aumentando el tiempo de propagación de alguna puerta lógica intermedia, sustituyéndola por otra análoga con mayor *fan-out* (una puerta con mayor capacidad de carga contiene transistores de dimensiones mayores y presenta una mayor carga capacitiva sobre la puerta anterior).

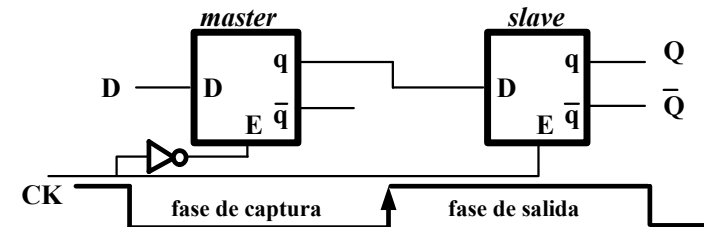
Ha de tenerse en cuenta que la resolución de «violaciones de la permanencia» aumenta los correspondientes tiempos de propagación en los caminos afectados y, por ello, puede disminuir la velocidad de trabajo alcanzable.

15.4. Separación entre las dos fases del reloj

El reloj de un sistema síncrono es una señal particular que organiza el manejo del tiempo y que, por su propia especificidad, requiere un detenido análisis conceptual.

La actividad del reloj no se refiere a niveles booleanos **0** y **1**, sino a las transiciones entre ambos. Esta «actuación por flanco» se refleja en la sucesión de dos valores (**0/1**), dando lugar a dos fases temporales (*captura/salida*) cuya función está en relación directa con la actuación de los dos biestables que conforman uno síncrono (*master/slave*).

El primer biestable básico efectúa la *captura* de los datos, mientras que el segundo biestable realiza la presentación o *salida* de los mismos. Existen pues dos tiempos diferentes en el funcionamiento de un sistema secuencial síncrono, intervalo de captura e intervalo de salida, que coinciden con cada uno de los semiperíodos del reloj.



El reloj habilita sucesiva y selectivamente al *amo* ($CK = 0$) y al *esclavo* ($CK = 1$), lo cual hace que el conjunto actúe como si fuera habilitado con flancos de subida (cuando CK pasa de 0 a 1).

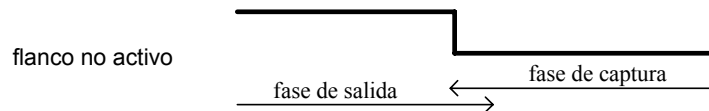
De forma que la existencia de flancos activos en la señal de reloj se debe a la sucesión de dos fases concordantes con los dos valores booleanos: la fase de *captura* o de habilitación del *amo* ($CK = 0$), en la cual «el amo captura el dato de entrada» y la fase de *salida* o de habilitación del *esclavo* ($CK = 1$), en que «el esclavo presenta en la salida el dato anteriormente capturado». Como el conjunto *amo/esclavo* sólo puede capturar el dato en la primera fase y presentarlo en la segunda, sus cambios de estado (del valor booleano presente en su salida) solamente pueden producirse en las transiciones de la fase de *captura* a la de *salida* (al inicio de la fase de salida el dato capturado pasa a ser el valor de la salida).

El buen funcionamiento de un sistema secuencial síncrono se basa en una separación neta entre las fases de captura y de salida: *Principio básico de separación de las fases de reloj*, de manera que la actuación de ambos biestables internos (*master*, *slave*) se encuentre netamente separada en el tiempo.

La correcta actuación del reloj requiere la «separación entre sus dos fases», es decir, la no coincidencia temporal de actuación de las mismas y ello tanto para cada biestable en particular como para los diversos biestables entre sí; lo cual tiene implicaciones directas en el diseño y utilización de los biestables y, también, sobre la señal de reloj.

Separación de las fases de reloj en un biestable

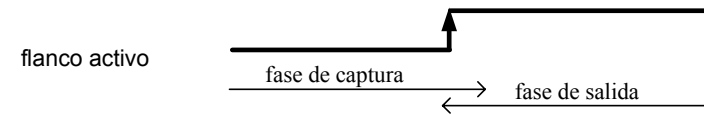
Hemos de considerar los dos momentos en que las fases pueden coincidir, es decir, tanto la transición de la fase de captura a la de salida (flanco activo del reloj), como la otra transición que va de la fase de salida a la de captura (flanco no activo).



Si la fase de captura se solapa con la de salida en el flanco no activo del reloj, puede producirse un cambio de estado en dicho flanco, lo cual constituye un grave error: durante el intervalo de habilitación de ambos (solape de fases), el *amo* podría capturar un nuevo valor y el *esclavo* podría recibirlo, dando lugar a un cambio de estado en el flanco no activo del reloj.

Para evitarlo, el diseño de cada biestable ha de asegurar que la habilitación del esclavo (fase de salida) finaliza antes de iniciarse la habilitación del amo (fase de captura); será necesario, en su caso, retrasar la habilitación del *amo*, de forma que el *esclavo* se bloquee previamente: ésta es una condición básica en el diseño de la configuración amo/esclavo.

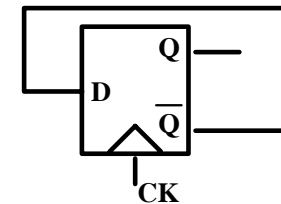
Por ello, en el diseño del conjunto *amo/esclavo*, el inversor necesario para diferenciar las dos fases *captura/salida* (es decir, las dos habilitaciones) se incluye siempre en la habilitación del *amo*, a fin de retrasarla respecto a la del *esclavo*.



Si la fase de salida se solapa con la de captura, en el flanco activo del reloj, puede producirse un «doble cambio de estado» en dicho flanco; es decir, en caso de que el biestable tenga realimentación sobre sí mismo, puede suceder que el nuevo estado traspasado a la salida del biestable dé lugar a un segundo nuevo estado que sea capturado y trasladado a la salida.

En tal caso, el primer nuevo estado asumido en el flanco activo del reloj resulta operativo y afecta al biestable en ese mismo flanco, sin esperar al siguiente flanco del reloj, lo cual es también un error funcional.

Un ejemplo sencillo puede clarificar este tipo de «doble cambio de estado» en un mismo flanco activo de reloj: sea un biestable síncrono **D**, con su entrada conectada a su salida negada,



al llegar el flanco activo del reloj este biestable cambia de estado (por ejemplo, pasa de valor 0 a 1); si la fase de captura se solapa con la de salida, es posible que la entrada capture el nuevo estado de $\bar{Q}$ ($\bar{Q}$ ha pasado de 1 a 0) y, una vez capturado, dicho valor pasará directamente a la salida, ya que se encuentra en fase de salida (Q pasará posteriormente de 1 a 0: no habrá cambiado de valor, salvo un transitorio espurio).

Es precisamente el tiempo de permanencia del dato t_h (*hold*) el que asegura que el nuevo estado no afecta al biestable en su mismo flanco activo de reloj (sino en el siguiente): el tiempo de permanencia entra en juego para impedir el solapamiento entre la fase de captura de un nuevo dato y la de salida del dato anterior (la permanencia excluye la presencia de un nuevo dato y, consiguientemente, la posibilidad de su captura).

En relación con la onda de reloj, la separación de fases requiere que los flancos de dicha señal sean adecuadamente «verticales»; es decir, que el tiempo de transición entre los valores booleanos, en ambos flancos, sea reducido. Si la conmutación entre los valores booleanos es lenta pueden existir intervalos de indefinición en que ambos biestables, el amo y el esclavo, se encuentren habilitados a la vez.

Separación de las fases de reloj entre los diversos biestables

La condición de «no solapamiento» debe cumplirse, no sólo en cada biestable respecto de sí mismo, sino también respecto a todos los demás. De esta forma se garantiza que el nuevo valor que presentan los biestables, tras el flanco activo del reloj, no afecta a ningún biestable hasta que se produce el siguiente flanco activo.

Damos por supuesto que el diseño de los biestables asegura la separación de fases en cada uno de ellos en los flancos no activos del reloj y, por tanto, en dichos flancos no puede producirse cambio del estado de los biestables.

Ahora bien, en los flancos activos del reloj puede suceder que un biestable presente su nuevo valor de salida antes de finalizar la fase de captura del segundo biestable, conectado (a través de la correspondiente parte combinacional) al primero. Es decir, podría solaparse la fase de captura del segundo biestable con la fase de salida del primero de ellos; en tal caso, también puede producirse el «doble cambio de estado» comentado anteriormente, o sea, la actuación de la nueva salida de un biestable sobre otro de ellos en el mismo flanco de reloj.

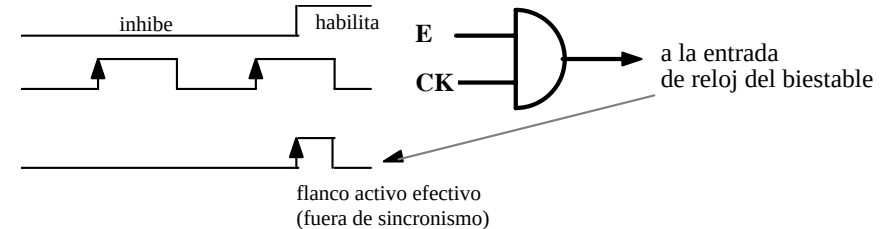
Al igual que en el caso de un simple biestable, el tiempo de permanencia del dato t_H evita la efectividad de este solapamiento de fases entre diversos biestables, con tal, claro está, de que la señal de reloj llegue simultáneamente a todos los biestables. Si los retrasos en la propagación de la onda de reloj hacen que los flancos activos que actúan sobre los diversos biestables no sean simultáneos, pueden existir intervalos de solapamiento entre las fases de salida de uno de ellos y las de captura de otros que den lugar a errores funcionales («dobles cambios de estado», o sea, que una transición de estado produzca efectos sobre la entrada de algún biestable en el mismo flanco en que ésta se produce).

15.5. Precauciones relativas a la señal de sincronismo

El *reloj* es una señal especial que estará conectada a la entrada de sincronismo de todos los biestables para que el cambio de estado de los mismos se produzca en el flanco activo de dicha señal.

Conviene destacar la necesidad de *no contaminación* de la propia señal de reloj: el reloj debe ser enviado directamente a los biestables y, en ningún caso, debe condicionarse la onda de reloj con habilitaciones, ni efectuar ninguna operación booleana sobre la misma: el reloj no debe atravesar ninguna puerta lógica (fuera de los biestables), sino que debe ser comunicada por conexión directa a las entradas de reloj de todos los biestables.

Cualquier operación o función lógica realizada sobre la señal de reloj pone en peligro el concepto de sincronismo, al introducir la posibilidad de que se produzcan «flancos activos espurios» sobre la entrada de reloj del biestable afectado, no simultáneos con el flanco activo propio del reloj.



En el caso de la figura, se pretende inhibir la actuación de un biestable cuando $E = 0$ mediante una puerta "Y" que impide el paso de la señal de reloj; ahora bien, si una vez anulado el paso de un flanco activo E pasa a valor 1 durante el intervalo en que la señal de reloj sigue en 1, se producirá un flanco en la entrada de reloj del biestable coincidente con el flanco de E y fuera de sincronismo respecto a la auténtica señal de reloj CK .

La señal de reloj debe poseer como características propias las siguientes: *verticalidad*, *simultaneidad* y *fuerza*. Estos tres requisitos indicados se derivan de la necesidad de separación neta entre las fases de *captura* y *salida*, de evitar solapes entre la habilitación de todos los primeros biestables básicos del conjunto amo/esclavo ($CK = 0$) y la habilitación de los segundos biestables de dicho conjunto ($CK = 1$).

La onda de reloj debe ser sumamente *vertical* para que no puedan producirse solapes en la habilitación de los dos «semibiestables» (amo/esclavo) que conforman cada biestable síncrono. La verticalidad de la señal de reloj asegura que los cambios booleanos que se producen a partir del flanco activo del reloj no afecten al valor contenido en el *esclavo* ya que el *amo* se encontrará inhibido; también asegura que la habilitación *amo* cuando recibe el otro flanco de reloj coincide con la inhibición del *esclavo*.

Flancos suaves en la onda de reloj pueden dar lugar a intervalos temporales con niveles booleanos imprecisos, en que ambos biestables (*master/slave*) se encuentren habilitados a la vez, rompiendo la condición de no solapamiento entre las fases.

El reloj debe llegar *a la vez* a todos los biestables para que su conmutación sea simultánea; ello exige que no exista ninguna puerta intermedia en las líneas que comunican el reloj y que dichas líneas sean lo más cortas posibles, a fin de evitar todo tipo de retardo en la propagación del reloj.

La falta de simultaneidad hará que la fase de *captura* de algún biestable se solape con la de *salida* de otro biestable previo y pueda producirse «doble cambio de estado», o sea, la actuación de la nueva salida del primer biestable sobre el segundo en el mismo flanco activo de reloj en que tal salida se produce.

Ambas características de la señal de reloj, *simultaneidad* y *verticalidad*, requieren una tercera complementaria: *fuerza*, es decir, capacidad para suministrar una intensidad adecuada para que la transmisión del reloj sea muy rápida (habida cuenta de los efectos capacitivos de las líneas de conexión) y para conmutar rápidamente las múltiples entradas de reloj de los biestables (que, asimismo, suponen una carga capacitiva).

Capacidad de suministro de intensidad equivale a capacidad de carga sobre la señal y se traduce en transmisión rápida a través de líneas directas y conmutación rápida, aún en el caso de un gran número de entradas conectadas sobre tal señal.

En los flancos de la señal de reloj se producen fenómenos de carga y descarga de las capacidades equivalentes de las múltiples entradas a las que dicha señal está conectada. Si su capacidad de carga, es decir, la intensidad que es capaz de suministrar no es suficientemente alta se prolongará en gran medida el tiempo de conmutación (flancos suaves) y se generarán retardos diferentes en la comunicación del reloj a las diversas entradas de los biestables (pérdida de la simultaneidad).

Para facilitar la transmisión de la señal de reloj existen celdas específicas que, además de proporcionar altos valores de intensidad de salida (*buffers* de intensidad), ofrecen múltiples salidas que permiten conectar un *fan-out* elevado, asegurando tiempos iguales en la propagación del reloj a todas ellas; tales adaptadores «múltiples» de reloj reciben el nombre de *clock drivers* (distribuidores de reloj).

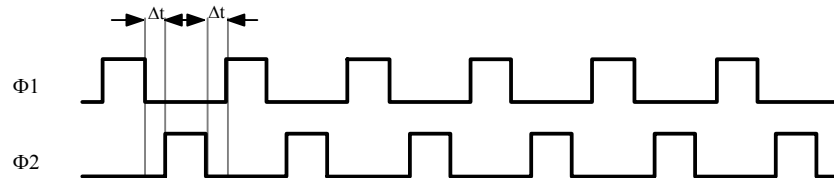
Reloj de dos fases separadas

En general, en sistemas integrados no muy complejos realizados con circuitos integrados estándar no es difícil evitar el solape entre las fases de captura y de salida si se cumplen las recomendaciones relativas a los tiempos de los biestables y a la señal de reloj; recomendaciones que afectan tanto al propio diseño del circuito como al de la placa de circuito impreso que los conecta.

Ahora bien, en sistemas muy amplios (o en aquellos que requieren alta seguridad de funcionamiento) y, sobre todo, en estructuras integradas complejas es frecuente la utilización de un reloj de dos fases no solapadas $\phi 1$ y $\phi 2$, con un intervalo temporal de separación entre ambas; cada una de las fases corresponde a la habilitación de uno de los dos biestables básicos de la configuración amo/esclavo:

$\phi 1$ determina el intervalo de *captura* (habilitación del primer biestable *master*)

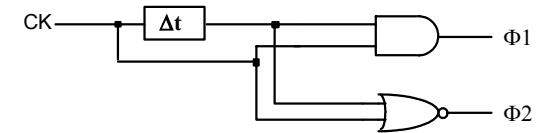
y $\phi 2$ coincide con el intervalo de *salida* (habilitación del segundo biestable *slave*).



De este modo, pequeños retrasos en la propagación de las señales o en los tiempos de subida o bajada de ellas quedan cubiertos por el tiempo de separación entre las dos fases y no provocan errores funcionales, ya que la habilitación de cada biestable (*master/slave*) se produce en intervalos claramente distintos y separados en el tiempo.

El intervalo de separación Δt ha de ser mayor que el tiempo de permanencia del dato t_h de los diversos biestables síncronos sumado al mayor de los tiempos de propagación de las señales de reloj a ellos; si esto se cumple, la utilización de dos fases separadas de reloj, $\phi 1$ $\phi 2$, asegura un correcto funcionamiento del mismo. Tanto el tiempo de mantenimiento, como pequeños retrasos en la propagación de las señales de reloj o los propios tiempos de subida o de bajada de ellas quedan cubiertos por el tiempo de separación entre las dos fases y no provocan errores funcionales.

El siguiente esquema circuital sirve para la generación de dos fases de reloj separadas a partir de una onda normal de reloj **CK**:



El componente señalado como Δt produce simplemente un retraso en la señal y puede hacerse mediante un simple circuito RC o bien mediante una serie de inversores sucesivos (cada uno de los cuales retrasará la señal según su tiempo de propagación).

La separación de fases en este circuito se produce porque, para dar salida **1**, la puerta "y" requiere dos «unos» y la puerta "o-negada" requiere dos «ceros» y, en ambos casos, uno de ellos llega con un retraso Δt .

16 CONTAR PULSOS Y DIVIDIR FRECUENCIAS: DISEÑO Y CONEXIÓN DE CONTADORES

- 16.1. Biestables *T* y su utilidad para conformar contadores
- 16.2. Diversidad de contadores
- 16.3. Diseño de contadores síncronos con biestables *D*
- 16.4. Los contadores y el sincronismo global

Un contador es un sistema secuencial conceptualmente muy simple: con cada pulso que recibe pasa de un estado al siguiente (cuenta el número de pulsos). Un contador módulo *n* presenta *n* estados (de 0 a *n*-1) y su evolución es circular: pasa de cada estado *i* al siguiente *i*+1 y del último *n*-1 al primero de ellos 0; su grafo de estados es un anillo, con una sola transición para cada estado (que le lleva al estado siguiente *i*+1).

A pesar de la sencillez de este comportamiento funcional, los contadores son extraordinariamente útiles en el diseño de sistemas digitales de medida y de control. Por ello, aunque su construcción puede realizarse aplicando la metodología general de diseño de los sistemas secuenciales síncronos (desarrollada en el capítulo anterior), se dedica el presente capítulo al diseño y conexión de contadores.

El biestable tipo *T*, que cambia de estado (conmuta) con cada pulso que recibe si su entrada *T* se encuentra a 1, resulta muy apropiado para la construcción de contadores. El diseño de contadores síncronos con biestables tipo *T* consiste en establecer, en forma de funciones booleanas, las situaciones en que cada biestable debe conmutar.

Es sencillo transferir a biestables *D* el diseño efectuado con los biestables *T*: basta una transformación booleana equivalente a una operación "o-exclusiva" entre la función que activa la entrada *T* y la salida *Q* del biestable.

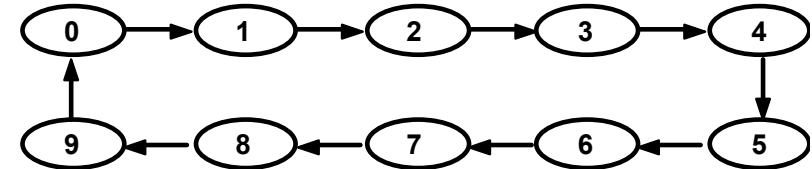
Otro aspecto de interés es la conexión de contadores para conformar otros más grandes, así como la posibilidad de respetar la codificación *BCD* para mantener la estructura de nuestros números decimales (base 10).

El próximo capítulo se dedica a las aplicaciones de los contadores: contaje y control de número de objetos o de sucesos; división de frecuencias; medida y multiplexado de tiempos; medida y control de frecuencias; y el siguiente (capítulo 18) trata de la modulación de anchura de pulsos, técnica que también aprovecha muy eficazmente a los contadores.

La gran utilidad de los contadores se traduce en la disponibilidad de una amplia variedad de los mismos, entre los que se cuentan los contadores «descendentes» (que descuentan) y los contadores «bidireccionales».

16.1. Biestables *T* y su utilidad para conformar contadores

Un contador módulo *n* es un sistema secuencial de *n* estados, numerados sucesivamente de 0 a *n*-1, cuyo grafo de estados es circular, pasando de un estado al siguiente (*i* → *i*+1) y del último al primero de ellos (*n*-1 → 0); con cada pulso el contador avanza un estado (pasa del estado *i* al estado *i*+1) y en el caso del último estado (*n*-1) pasa al primero (0).



Grafo de estados de un contador módulo 10 (década)

Un contador módulo *n* requiere *m* biestables, siendo *m* el menor exponente de base 2 necesario para alcanzar el número *n*:

- si *n* es potencia entera de 2: $n = 2^m$ contador completo
- si no lo es: $2^{m-1} < n < 2^m$ contador parcial.

En el primer caso (*n* potencia de 2) el contador recorre todos los estados posibles de los *m* biestables, por lo cual lo denominaremos contador «completo», mientras que en el segundo caso existen estados no alcanzados por el contador: contador «parcial».

En este capítulo trataremos sobre el diseño de contadores síncronos, aquellos cuyos biestables conmutan a la vez, en sincronía con el flanco activo en su entrada de pulsos. Resulta más sencillo diseñar contadores asíncronos pero no es recomendable en modo alguno su utilización en sistemas digitales complejos. El diseño digital actual se hace en forma síncrona por las amplias ventajas que el sincronismo proporciona (principalmente en cuanto a seguridad funcional).

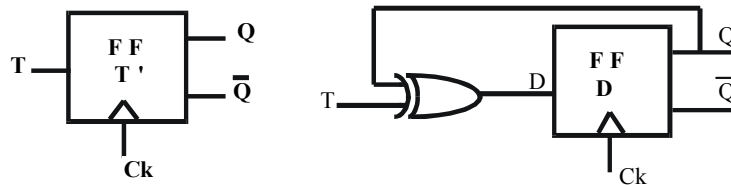
Ahora bien, habida cuenta de que en diseños reducidos pueden resultar adecuados los contadores asíncronos y, sobre todo, que en los catálogos de las familias lógicas integradas hay una amplia oferta de tales contadores no síncronos, se incluye un apéndice (A5 Contadores asíncronos) referido a dichos contadores asíncronos.

El diseño de contadores puede realizarse a través de los métodos generales de diseño secuencial síncrono: a partir de la tabla de evolución de los estados obtener las funciones de activación de los biestables. Ahora bien, los biestables tipo *T* facilitan en gran manera dicho diseño, sin necesidad de construir la tabla de evolución del estado, lo cual es particularmente útil en el caso de contadores de gran tamaño; y, además, la traslación de dicho diseño con biestables *T* a biestables tipo *D* se efectúa a través de una transformación booleana sencilla.

El biestable tipo **T** presenta dos entradas: la de reloj con la cual sincroniza su funcionamiento, y la entrada **T**: el biestable cambia de estado con el flanco activo de la onda de reloj siempre que **T = 1** y permanece en su estado anterior cuando **T = 0**.

El biestable **T** puede construirse a partir del biestable **D** síncrono, anteponiéndole la siguiente función de entrada que corresponde a su comportamiento: el biestable debe cambiar de estado cuando **T = 1** y debe permanecer en el mismo si **T = 0**.

$$D = T \cdot \bar{Q} + \bar{T} \cdot Q = T \oplus Q$$



Al utilizar los biestables **T** para configurar contadores síncronos es necesario activar la entrada **T** de cada biestable con las condiciones booleanas que determinan cuando debe conmutar:

- en el caso de contadores «completos» (módulo 2^m) la condición de conmutación de cada biestable es que todos los anteriores se encuentren a **1** (basta observar la secuencia de conteo y, en ella, el valor de los biestables anteriores al considerado en el estado previo a su conmutación); el primero de los biestables conmuta siempre;
- en caso de contadores «parciales» (módulo $n < 2^m$) será preciso añadir la condición de que todos los biestables pasen a **0** con el pulso de reloj siguiente al estado **n-1**.

De forma que, para construir un contador completo, las entradas **T** de los sucesivos biestables deben recibir las siguientes funciones booleanas:

$$T_0 = 1$$

$$T_1 = q_0$$

$$T_2 = q_1 \cdot q_0$$

$$T_3 = q_2 \cdot q_1 \cdot q_0 \quad \text{Contador módulo 16}$$

... ..

$$T_i = q_{i-1} \cdot q_{i-2} \cdot \dots \cdot q_2 \cdot q_1 \cdot q_0$$

Estos términos booleanos, que contienen el producto de todos los dígitos anteriores en forma afirmada, se corresponden al conteo directo en binario: cuando se completan todos los dígitos anteriores (*se llenan a 1*) es cuando se produce un arrastre de una unidad al dígito siguiente.

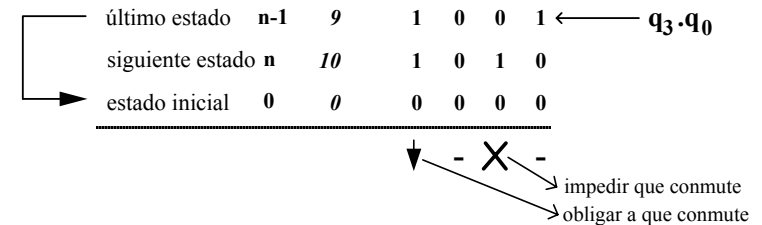
Para convertir un contador «completo» en contador «parcial» módulo **n**, basta incluir en las anteriores expresiones de **T_i** las condiciones booleanas necesarias para que el último estado **n-1** pase al estado inicial **0**. Para ello puede utilizarse el término mínimo reducido correspondiente a **n-1**, definido como el producto de aquellas variables de estado cuyo valor en dicho estado es **1**, y basta considerar el valor booleano de cada biestable en el estado **n-1** y el que tendría en el estado **n**:

- aquellos biestables cuyo valor en el estado **n** sería **0** no requieren modificación alguna ya que a partir del estado **n-1** asumirán directamente, como estado siguiente, el valor booleano **0**;
- cuando un biestable vale **1** en el estado **n-1** y su valor en el estado **n** continúa siendo **1**, es preciso forzar su conmutación añadiendo en la activación de su entrada **T**, mediante suma booleana, el término mínimo reducido correspondiente a **n-1**;
- en el caso de un biestable cuyo valor es **0** en **n-1** y su valor en el estado **n** sería **1** hay que evitar su conmutación, condicionando la activación de la entrada **T**, mediante producto booleano, con el negado del término mínimo reducido de **n-1**.

Por ejemplo, para configurar un contador módulo 10 (década):

- el último estado es el 9 **1001** y tras dicho estado debe alcanzarse el inicial **0000**, siendo así que en su evolución normal el contador pasaría al estado 10 **1010**;
- no es preciso ocuparse de los biestables primero **q₀** y tercero **q₂**: su valor en 10 es **0**;
- es necesario evitar la conmutación del segundo **q₁** y obligar a conmutar al cuarto **q₃**;
- el término mínimo reducido correspondiente al último estado (9 **1001**) es **q₃·q₀**.

Obsérvese que no es necesario utilizar el término mínimo completo, ya que el término mínimo reducido vale **1** para dicho estado; también vale **1** para algunos estados posteriores, pero tales estados no son alcanzables en el conteo. Así, por ejemplo, **q₃·q₀** vale **1** para el 9 y, también, para el 11 (**1011**), el 13 (**1101**) y el 15 (**1111**); pero el conteo módulo 10 pasa de 9 a 0, de forma que nunca se llega a los otros estados con valor **1**.



Con lo cual, las condiciones de activación de las entradas **T** son las siguientes:

$$T_0 = 1$$

$$T_1 = q_0 \cdot \overline{(q_3 \cdot q_0)} = \overline{q_3} \cdot q_0$$

$$T_2 = q_1 \cdot q_0$$

$$T_3 = q_2 \cdot q_1 \cdot q_0 + q_3 \cdot q_0$$

Conexión de contadores síncronos

Para conectar entre sí contadores síncronos, a fin de configurar un contador más amplio, es necesario añadir a cada contador una entrada de habilitación de conteo **E** y una salida que se active cuando el contador alcanza el estado máximo **máx**; de esa forma conectando la salida **máx** de un contador con la entrada **E** del siguiente, el segundo de los contadores se incrementará en una unidad cuando el primero de ellos haya alcanzado el valor máximo de su conteo.

La entrada de habilitación **E** condicionará, a través de producto booleano directo, todas las funciones de activación de las entradas **T_i** de los biestables; la salida **máx** coincide con el término mínimo reducido correspondiente al último estado **n-1** multiplicado booleanamente por la habilitación **E**:

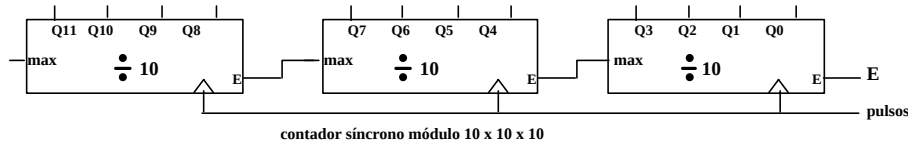
$$T_0 = E$$

$$T_1 = E.\overline{q_3}.q_0$$

$$T_2 = E.q_1.q_0$$

$$T_3 = E.q_2.q_1.q_0 + E.q_3.q_0$$

$$\text{máx} = E.q_3.q_0$$



El contador de la figura anterior es un contador módulo 1000 (10 x 10 x 10) que cuenta en BCD, ya que cada uno de los contadores que lo componen es módulo 10 (contador década); sus salidas pueden representarse sobre visualizadores de 7 segmentos, a través de sendos conversores de BCD en 7 segmentos.

Borrado de contadores

Generalmente los contadores disponen de una entrada de borrado cuya activación lleva a sus biestables al estado **0**. El borrado puede ser asíncrono, a través de la correspondiente entrada **Clr** o **Reset** de borrado de cada uno de los biestables, o síncrono, con una entrada **B** que actúa sobre la función booleana de las entradas **T** de habilitación de los biestables; el borrado asíncrono se produce inmediatamente después de activar la entrada de borrado **Clr**, mientras que el borrado síncrono **B** se ejecuta en el flanco activo de reloj.

Para añadir al contador una entrada de borrado síncrono **B** es necesario inhibir la conmutación (para que ningún biestable que se encuentre a **0** pase a **1**) y, también, es preciso forzar la conmutación de aquellos biestables que se encuentren en **1**: para ello, además de anular, multiplicando por $\overline{B}$, las funciones de activación de las entradas **T_i**, es necesario añadirles un término $+B.q_i$ para obligar a conmutar si **q_i** = **1**.

En el caso del contador módulo 10 (década) las expresiones resultantes serán las siguientes (borrado prioritario: se borra aunque la habilitación sea nula **E** = **0**):

$$T_0 = E.\overline{B} + B.q_0$$

$$T_1 = E.\overline{B}.\overline{q_3}.q_0 + B.q_1$$

$$T_2 = E.\overline{B}.q_1.q_0 + B.q_2$$

$$T_3 = E.\overline{B}.q_2.q_1.q_0 + E.\overline{B}.q_3.q_0 + B.q_3 = E.\overline{B}.q_2.q_1.q_0 + E.q_3.q_0 + B.q_3$$

$$\text{máx} = E.q_3.q_0.$$

Contar pulsos y dividir frecuencias

Contar pulsos puede parecer algo muy simple pero, en la práctica, tiene muchas aplicaciones. Los contadores son sumamente útiles en la realización de sistemas digitales, siendo el núcleo básico de muchos de ellos, como es el caso de relojes, temporizadores, frecuencímetros, dispositivos de sincronización,... y de un amplio número de sistemas de control y de medida. Pocos son los sistemas digitales en los que no se encuentren presentes diversos contadores, realizando operaciones variadas.

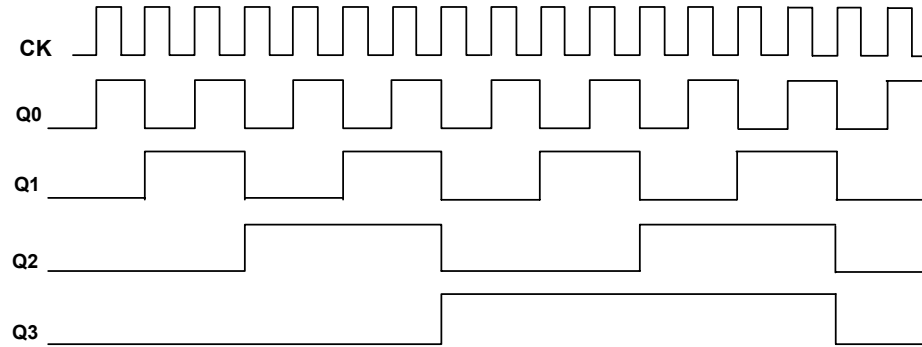
Contar pulsos permite contar objetos o contar sucesos y como resultado del conteo conocer el número de piezas producidas, el número de personas en un recinto, el número de automóviles que circulan por una carretera, el número de unidades de tiempo transcurridas (relojes y cronómetros),... Contar permite también controlar el número de objetos a incluir en un recipiente, el número de unidades de tiempo de un proceso (temporizadores), el número de marcas a recorrer (posicionamiento lineal o angular),...

Asimismo, contar pulsos en una unidad de tiempo equivale a medir la frecuencia de la señal y, con ella, la velocidad de motores (revoluciones por minuto), la velocidad de bicicletas y automóviles, el valor de una variable codificada en frecuencia,...

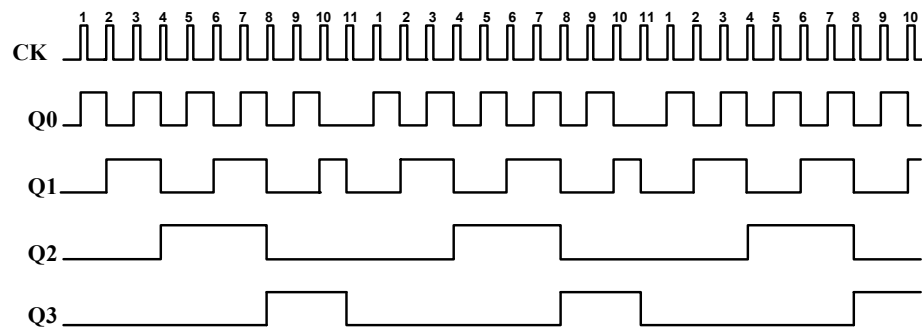
Además, los contadores permiten configurar controles de tipo todo/nada en que cada período de tiempo resulta dividido en dos intervalos activo/inactivo. El resultado de este control *on/off* es una modulación de la anchura de pulsos (capítulo 18) con aplicaciones en control de potencia, conversión número-tensión (conversores digital-analógicos), conversores tensión-tiempo y tensión-número (analógico-digitales), control de amplitud de señales (potenciómetros digitales),...

Un componente básico de los procesadores en los sistemas digitales que actúan bajo programa (computadores, microprocesadores, etc.) es el contador central o contador de programa, que señala la dirección de la instrucción a ejecutar y, una vez ejecutada la misma, pasa a la siguiente instrucción. Este contador ha de disponer de la posibilidad de carga paralelo a fin de poder efectuar saltos en el programa para atender a instrucciones condicionales, a subrutinas y a interrupciones.

El conteo de pulsos se encuentra asociado directamente a la división de la frecuencia de los mismos: los biestables de un contador «completo» (módulo n potencia entera de 2) proporcionan en sus salidas ondas digitales cuyas frecuencias son, respectivamente, la mitad ($1/2$), la cuarta parte ($1/4$), la octava parte ($1/8$),... ($1/2^i$)..., de la frecuencia de los pulsos de entrada.



En el caso de un contador «parcial» (módulo n , siendo n un número cualquiera), tomando como salida la de su biestable de valor más significativo, se obtiene la división por n de la frecuencia de los pulsos que recibe, es decir, produce un pulso en su salida por cada n pulsos en su entrada.



Ondas en las salidas de un contador módulo 11

Al dividir una frecuencia por n (la frecuencia se hace más lenta) su período queda multiplicado por n (el período se hace más largo); de esta forma, pueden obtenerse señales de reloj más lentas, con unidades de tiempo mayores. Así es posible pasar de unidades de tiempo muy precisas del orden del microsegundo, obtenidas mediante osciladores con cristal de cuarzo, a tiempos de milisegundos o de segundos, por simple agrupación de n unidades a través de la división de su frecuencia.

16.2. Diversidad de contadores

16.2.1. Contadores descendentes

Contadores descendentes son aquellos que «descuentan», es decir, pasan del estado i al $i-1$ y, obviamente, del estado inferior 0 al $n-1$ (módulo n).

La condición de conmutación de un biestable en un contador descendente «completo» (módulo n potencia entera de 2) consiste en que todos los anteriores se encuentren a 0 (como puede comprobarse observando en la secuencia de «descontaje» el valor de los biestables anteriores al considerado en el estado previo a su conmutación):

$$T_i = \overline{q_{i-1}} \cdot \overline{q_{i-2}} \cdot \dots \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

Un contador descendente «parcial» (módulo n que no sea potencia entera de 2) ha de forzar la transición desde el estado 0 al estado $n-1$; si el contador es «completo» desde el estado $0000\dots$ se pasa directamente al $1111\dots$, de forma que para alcanzar el estado $n-1$ bastará evitar la conmutación de aquellos biestables cuyo valor en dicho estado sea 0 .

El estado 0 (cero) viene identificado por la operación "y" extendida a las negadas de todas las salidas del contador, mientras que la situación contraria (no cero) corresponde a la operación "o" aplicada a todas las salidas:

$$z = \overline{q_{m-1}} \cdot \overline{q_{m-2}} \cdot \dots \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0} = \text{Nor}(q_{m-1}, q_{m-2}, \dots, q_2, q_1, q_0)$$

$$\overline{z} = q_{m-1} + q_{m-2} + \dots + q_2 + q_1 + q_0$$

Si el biestable q_i no tiene valor 1 en el estado $n-1$ será preciso evitar que conmute a partir del estado 0 :

$$T_i = \overline{q_{i-1}} \cdot \overline{q_{i-2}} \cdot \dots \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0} \cdot z = \overline{q_{i-1}} \cdot \overline{q_{i-2}} \cdot \dots \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0} \cdot (q_{n-1} + q_{n-2} + \dots + q_{i+1} + q_i)$$

Por ejemplo, en el caso de un contador módulo 10 (década) el conteo descendente ha de conducir del estado 0000 al 1001 (cuando, de por sí, pasaría al 1111); para ello, es necesario evitar que, desde el estado 0 , los biestables segundo q_1 y tercero q_2 conmuten. Las funciones correspondientes al conteo descendente en módulo 10 serán las siguientes:

$$z = \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0} \quad \overline{z} = q_3 + q_2 + q_1 + q_0$$

$$T_0 = E$$

$$T_1 = E \cdot \overline{q_0} \cdot \overline{z} = E \cdot \overline{q_0} \cdot (q_3 + q_2 + q_1)$$

$$T_2 = E \cdot \overline{q_1} \cdot \overline{q_0} \cdot \overline{z} = E \cdot \overline{q_1} \cdot \overline{q_0} \cdot (q_3 + q_2)$$

$$T_3 = E \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

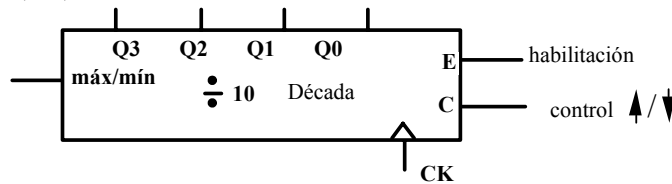
$$\text{mín} = E \cdot z = E \cdot \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

Para la conexión sucesiva de contadores descendentes han de presentar la salida de mínimo **mín** que se activará cuando el contador se encuentre a 0 ; dicha salida se conectará a la entrada de habilitación **E** del siguiente contador.

16.2.2. Contadores bidireccionales

Los contadores bidireccionales (contadores hacia arriba y hacia abajo: *up/down counter*) pueden contar de **0** a **n-1** y pueden también «descontar» de **n-1** a **0**, siendo controlada dicha posibilidad mediante una entrada adicional **C** (*up/down*), cuyo valor **1** determina el conteo ascendente y su valor **0** impone el descendente; obviamente al descontar el estado siguiente al **0** es el **n-1**. En tales contadores la salida para conexión de contadores **máx/mín** ha de activarse cuando el contador se encuentra habilitado y alcanza su máximo valor (**n-1**), caso de estar contando (*up*), o su valor mínimo (**0**), caso de encontrarse descontando (*down*).

Las funciones booleanas de activación de los biestables en un contador bidireccional pueden obtenerse agrupando las funciones correspondientes al conteo ascendente y las propias del conteo descendente, diferenciando ambas posibilidades mediante una entrada de control **C** ($\uparrow/\downarrow$).



En el caso de un contador módulo 10 (década) tomando ambas funciones de conteo ascendente (apartado 1 de este capítulo) y de conteo descendente (16.2.1):

Contaje ascendente

$$T_0 = E$$

$$T_1 = E \cdot \overline{q_3} \cdot q_0$$

$$T_2 = E \cdot q_1 \cdot q_0$$

$$T_3 = E \cdot q_2 \cdot q_1 \cdot q_0 + E \cdot q_3 \cdot q_0$$

$$\text{máx} = E \cdot q_3 \cdot q_0$$

Contaje descendente

$$T_0 = E$$

$$T_1 = E \cdot \overline{q_0} \cdot (q_3 + q_2 + q_1)$$

$$T_2 = E \cdot \overline{q_1} \cdot \overline{q_0} \cdot (q_3 + q_2)$$

$$T_3 = E \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

$$\text{mín} = E \cdot z = E \cdot \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

y, agrupando las funciones correspondientes a ambos conteos, multiplicando booleanamente por **C** las relativas al conteo ascendente y por su negado las que producen el conteo descendente, resulta:

$$T_0 = E$$

$$T_1 = C \cdot E \cdot \overline{q_3} \cdot q_0 + \overline{C} \cdot E \cdot \overline{q_0} \cdot (q_3 + q_2 + q_1)$$

$$T_2 = C \cdot E \cdot q_1 \cdot q_0 + \overline{C} \cdot E \cdot \overline{q_1} \cdot \overline{q_0} \cdot (q_3 + q_2)$$

$$T_3 = C \cdot E \cdot (q_2 \cdot q_1 \cdot q_0 + q_3 \cdot q_0) + \overline{C} \cdot E \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

$$\text{máx/mín} = C \cdot E \cdot q_3 \cdot q_0 + \overline{C} \cdot E \cdot \overline{q_3} \cdot \overline{q_2} \cdot \overline{q_1} \cdot \overline{q_0}$$

16.2.3. Contadores complejos. Contadores universales

Dada la utilidad que los contadores ofrecen para la realización de sistemas digitales existe una amplia disponibilidad de tipos diferenciados dentro de las familias lógicas integradas; algunos de ellos con prestaciones complejas como veremos a continuación.

Existe un tipo de **contadores para división de frecuencias** (*rate multipliers*) que, a partir de un número total de pulsos **m** que llegan a su entrada, permiten el paso de un número **n** de ellos (**n < m**), programable en sus entradas de control, de forma que realizan un cambio de frecuencia según el factor **n / m**.

Asimismo, orientados a la división de frecuencias y a la temporización, existen contadores descendentes (*down*) programables a través de sus entradas paralelo (carga síncrona) y dotados de una salida que se activa cuando el contador se encuentra a cero. Conectando la salida indicadora de estado cero a la habilitación de entradas paralelo se obtiene un divisor de frecuencia por **n+1**: tras alcanzarse su valor mínimo (**0**) el contador pasa al valor **n** programado en tales entradas. En cambio, cargando inicialmente el valor **n** y activando con la salida indicadora de cero una entrada de inhibición de conteo se consigue un temporizador: el contador descuenta desde **n** hasta alcanzar el estado **0**.

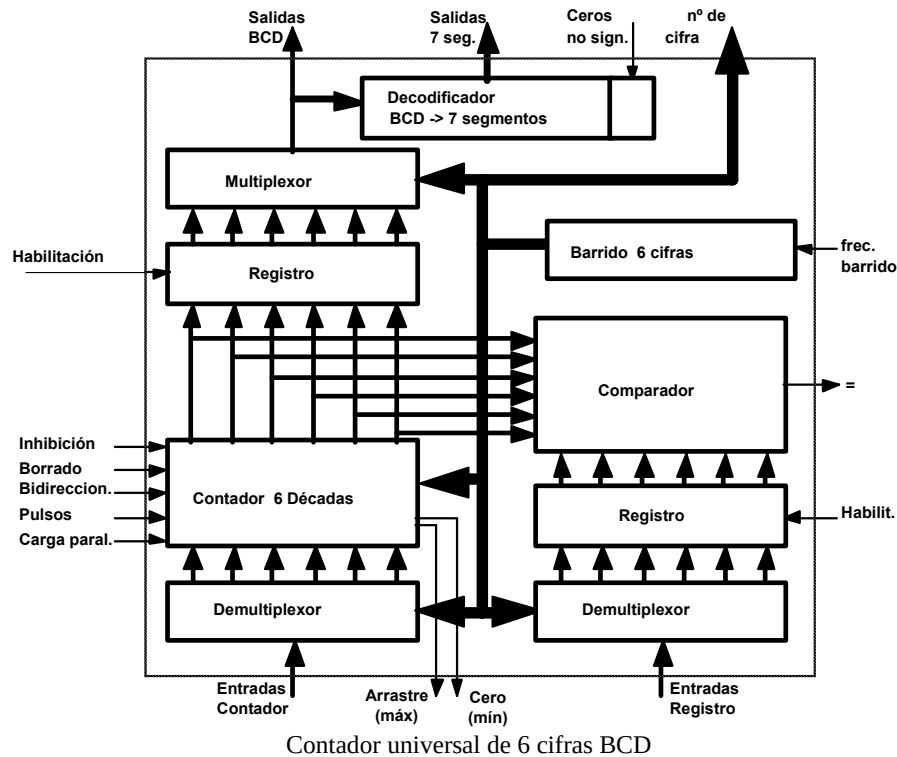
Existen integrados conteniendo varias décadas, cuyas salidas son multiplexadas sobre las mismas cuatro líneas BCD, de forma que presentan sucesivamente una a una las cifras decimales. El mismo circuito integrado incluye el multiplexor y el correspondiente contador de control del mismo, recibiendo la frecuencia deseada para el multiplexado a través de pulsos por una entrada de muestreo (SCAN) o mediante un simple condensador exterior que determina la frecuencia de muestreo.

Un paso más consiste en incluir un registro de retención que reciba las cifras de los contadores BCD, estando en este caso las salidas del registro multiplexadas sobre los cuatro terminales de salida; ello permite utilizar directamente tal integrado en aplicaciones más complejas que el simple conteo, siendo muy apropiado para la realización de frecuencímetros.

El desarrollo de esta línea de creciente complejidad y potencia de cálculo ha culminado en el concepto de **contador universal**:

- un contador suficientemente amplio, generalmente de 6 décadas, bidireccional, con posibilidad de carga paralelo multiplexada (de forma que cada vez se almacena una de las cifras BCD), con salidas indicadoras de que se encuentra en su valor máximo y en su valor mínimo (**0**), junto con las correspondientes entradas de borrado e inhibición;
- un registro de retención conectado a la salida del contador, con salidas multiplexadas cifra a cifra sobre cuatro terminales BCD y, a la vez, decodificadas en 7 segmentos para atacar directamente a un visualizador;
- un segundo registro de retención programable exteriormente en forma multiplexada análoga a la del contador y cuyo contenido se compara aritméticamente con el del contador de forma que una salida exterior indica la igualdad entre ambos;

- y el sistema de multiplexado necesario para la salida de las cifras del primer registro y para la programación del contador y del segundo registro; la velocidad de muestreo se fija mediante un condensador exterior y 6 líneas individuales indican cuál de las cifras BCD se encuentra activa en cada momento.



El anterior es un posible esquema de bloques de un «contador universal»; una aplicación inmediata de tales contadores es el diseño de frecuencímetros o temporizadores para lo cual se requiere añadir muy poca circuitería adicional.

Por otra parte, existen contadores amplios de propósito particular entre los que destacan los dedicados a relojes digitales que contienen toda la circuitería necesaria para su configuración como sistemas autónomos sin más que añadir el visualizador, la alimentación de tensión y el cristal de cuarzo que genera la frecuencia inicial; tales integrados incluyen, el sistema de programación de la hora mediante simples pulsadores y la función de despertador o alarma.

Se utilizan contadores análogos de contaje horario (*real time clock* RTC) como periféricos de microprocesadores (o de otros sistemas digitales complejos), a los cuales pueden comunicar la hora mediante transmisión serie (utilizando muy pocas líneas para tal comunicación).

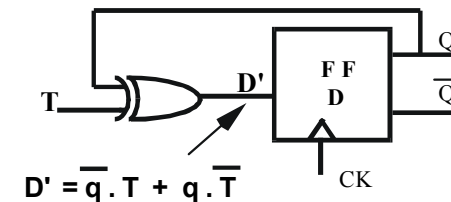
16.3. Diseño de contadores síncronos con biestables D

Para construir contadores con biestables síncronos tipo **D**, las funciones booleanas de activación de sus entradas **D_i** no sólo han de contener las condiciones en que el biestable cambia de estado, sino también aquellas en que conserva el estado **1**.

Tomando como referencia el diseño de contadores síncronos con biestables **T**, las funciones **T_i** expresan las situaciones en las cuales el correspondiente biestable debe conmutar y, en cambio, cuando **T_i = 0** el biestable debe conservar el valor anterior; ello nos permite escribir:

$$D_i = \overline{q_i} \cdot \text{cuando debe conmutar} + q_i \cdot \text{cuando debe permanecer} \\ = \overline{q_i} \cdot T_i + q_i \cdot \overline{T_i}$$

expresión que se corresponde con la forma de construir un biestable **T** a partir de un biestable síncrono tipo **D**.



Por tanto, una forma sencilla de diseñar contadores con biestables tipo **D**, consiste en construir las funciones **T_i** correspondientes al diseño con biestables **T** y, a partir de ellas, aplicar la anterior transformación a biestables **D**:

$$D_i = \overline{q_i} \cdot T_i + q_i \cdot \overline{T_i}$$

Por ejemplo, sea un contador módulo 12:

último estado	n-1	11	1	0	1	1	← q₃·q₁·q₀
siguiente estado	n	12	1	1	0	0	
estado inicial	0	0	0	0	0	0	
							↓ X - -

$$T_0 = E$$

$$T_1 = E \cdot q_0$$

$$T_2 = E \cdot q_1 \cdot q_0 \cdot \overline{(q_3 \cdot q_1 \cdot q_0)} = E \cdot \overline{q_3} \cdot q_1 \cdot q_0$$

$$T_3 = E \cdot q_2 \cdot q_1 \cdot q_0 + E \cdot q_3 \cdot q_1 \cdot q_0 = E \cdot q_1 \cdot q_0 \cdot (q_3 + q_2)$$

quedando la activación de las entradas **D_i** en la forma que sigue.

$$D_0 = \overline{q_0}.E + q_0.\overline{E} = E.\overline{q_0} + \overline{E}.q_0$$

$$D_1 = \overline{q_1}.E.q_0 + q_1.(\overline{E} + \overline{q_0}) = E.\overline{q_1}.q_0 + q_1.\overline{q_0} + \overline{E}.q_1$$

$$D_2 = \overline{q_2}.E.\overline{q_3}.q_1.q_0 + q_2.(\overline{E} + q_3 + \overline{q_1} + \overline{q_0}) \\ = E.\overline{q_3}.\overline{q_2}.q_1.q_0 + q_3.q_2 + q_2.\overline{q_1} + q_2.\overline{q_0} + \overline{E}.q_2$$

$$D_3 = \overline{q_3}.E.q_1.q_0.(q_3 + q_2) + q_3.(\overline{E} + \overline{q_1} + \overline{q_0} + \overline{q_3}.\overline{q_2}) \\ = E.\overline{q_3}.q_2.q_1.q_0 + q_3.\overline{q_1} + q_3.\overline{q_0} + \overline{E}.q_3$$

$$\max = E.q_3.q_1.q_0$$

En la función correspondiente a **D₂** puede prescindirse del término $+ q_3.q_2$ ya que nunca se alcanza dicho valor (corresponde a números mayores de 11).

Para añadir una entrada de borrado síncrono **B**, basta con multiplicar por $\overline{B}$ las expresiones de activación de las entradas **D_i**, de forma que cuando **B** = 1 el dato que reciben los biestables sea 0.

$$D_0 = \overline{B}.E.\overline{q_0} + \overline{B}.\overline{E}.q_0$$

$$D_1 = \overline{B}.E.\overline{q_1}.q_0 + \overline{B}.\overline{E}.q_1.\overline{q_0} + \overline{B}.\overline{E}.q_1$$

$$D_2 = \overline{B}.E.\overline{q_3}.\overline{q_2}.q_1.q_0 + \overline{B}.\overline{E}.q_2.\overline{q_1} + \overline{B}.\overline{E}.q_2.\overline{q_0} + \overline{B}.\overline{E}.q_2$$

$$D_3 = \overline{B}.E.\overline{q_3}.q_2.q_1.q_0 + \overline{B}.\overline{E}.q_3.\overline{q_1} + \overline{B}.\overline{E}.q_3.\overline{q_0} + \overline{B}.\overline{E}.q_3$$

En las expresiones anteriores el borrado **B** es prioritario (respecto a la habilitación **E**); si se quisiera dar prioridad a la habilitación (es decir, que no se borre si no hay habilitación) será necesario dejar sin multiplicar por $\overline{B}$ el término $+ \overline{E}.q_1$, responsable de conservar el valor del biestable en ausencia de habilitación.

A veces los contadores no empiezan en valor 0, sino que su primer estado es un número no nulo; la forma de diseñarlos es análoga, con el mismo tipo de razonamiento. Consideremos, por ejemplo, un contador módulo 6 que simule el funcionamiento de un dado y, por tanto, cuente de 1 a 6 (en lugar de 0 a 5):

último estado	6	1	1	0	← $q_2.q_1$
siguiente estado	7	1	1	1	
estado inicial	1	0	0	1	
					↓ ↓ -

$$T_0 = E$$

$$T_1 = E.q_0 + E.q_2.q_1$$

$$T_2 = E.q_1.q_0 + E.q_2.q_1 = E.q_1.(q_2 + q_0)$$

$$D_0 = E.\overline{q_0} + \overline{E}.q_0$$

$$D_1 = E.\overline{q_1}.q_0 + \overline{q_2}.q_1.\overline{q_0} + \overline{E}.q_1$$

$$D_2 = E.\overline{q_2}.q_1.q_0 + q_2.\overline{q_1} + \overline{E}.q_2$$

Si se utiliza como reloj de pulsos de dicho contador una onda de frecuencia relativamente alta y se activa la habilitación a través de un pulsador manual, el número en que se detenga al finalizar la habilitación será aleatorio y tendremos un dado «electrónico» (análogo al diseñado en 14.2.6, página 92).

Un segundo ejemplo: en algunos juegos de rol se utilizan dados de 25 caras, numeradas de 1 a 25; supongamos que deseamos diseñar un contador para simular dichos dados (este contador módulo 25, que cuente de 1 a 25, ha de contar en BCD para poder representar el resultado en sendos visualizadores de 7 segmentos).

El contador tendrá 6 biestables, los cuatro inferiores **q₃ q₂ q₁ q₀** para representar las unidades y los dos superiores **q₅ q₄** para contar hasta 2; el contador formado por los cuatro biestables inferiores **q₃ q₂ q₁ q₀** ha de ser módulo 10 (contando solamente de 0 a 9 para respetar las cifras decimales)

9	1	0	0	1	← $q_3.q_0$
10	1	0	1	0	
0	0	0	0	0	
					↓ - X -

y el contador global **q₅ q₄ q₃ q₂ q₁ q₀** ha de ser módulo 25, de 1 a 25:

25	1	0	0	1	0	1	← $q_5.q_2.q_0$
26	1	0	0	1	1	0	
1	0	0	0	0	0	1	
							↓ - - ↓ X X

- en **q₀** hay que imponer la condición de que no conmute en el estado 25;
- **q₁** no debe conmutar ni en el estado 9 (BCD) ni en el estado 25;
- a **q₂** es preciso obligarle a conmutar en el estado 25;
- a **q₃** hay que obligarle a conmutar en el estado 9 (BCD);
- **q₄** y **q₅** solamente deben contar cuando el contador **q₃ q₂ q₁ q₀** está en 9 (BCD)
- y a **q₅** hay que obligarle a conmutar en el estado 25;

$$T_0 = E(\overline{q_5 \cdot q_2 \cdot q_0}) = E.\overline{q_5} + E.\overline{q_2} + E.\overline{q_0}$$

$$T_1 = E.q_0(\overline{q_3 \cdot q_0})(\overline{q_5 \cdot q_2 \cdot q_0}) = E.\overline{q_3} \cdot q_0(\overline{q_5} + \overline{q_2})$$

$$T_2 = E.q_1.q_0 + E.q_5.q_2.q_0$$

$$T_3 = E.q_2.q_1.q_0 + E.q_3.q_0 \quad \text{contador BCD (módulo 10)}$$

$$T_4 = E.q_3.q_0 \quad \text{cuenta cuando el anterior está en 9}$$

$$T_5 = E.q_3.q_0.q_4 + E.q_5.q_2.q_0$$

$$D_0 = E.\overline{q_0} + q_5.q_2.q_0 + \overline{E}.q_0$$

$$D_1 = E.\overline{q_5}.\overline{q_3}.\overline{q_1}.q_0 + E.\overline{q_3}.\overline{q_2}.\overline{q_1}.q_0 + q_3.q_1 + q_1.\overline{q_0} + \overline{E}.q_1$$

$$D_2 = E.\overline{q_2}.\overline{q_1}.q_0 + q_2.\overline{q_0} + \overline{q_5}.q_2.\overline{q_1} + \overline{E}.q_2$$

$$D_3 = E.\overline{q_3}.q_2.q_1.q_0 + q_3.\overline{q_0} + \overline{E}.q_3$$

$$D_4 = E.\overline{q_4}.q_3.q_0 + q_4.\overline{q_3} + q_4.\overline{q_0} + \overline{E}.q_4$$

$$D_5 = E.\overline{q_5}.q_4.q_3.q_0 + q_5.\overline{q_0} + q_5.\overline{q_3}.\overline{q_2} + q_5.\overline{q_4}.\overline{q_2} + \overline{E}.q_5$$

En la función correspondiente a **D1** se ha prescindido del término $+ q_5.q_2.q_1$ ya que nunca se alcanza dicho valor (corresponde al número 26).

Este contador requiere 6 biestables y puede ser programado en un bloque **PLS**, cuyas macroceldas dispongan de, al menos, 5 términos producto; utiliza solamente una entrada exterior **E**.

Contador descendente

Al pasar las funciones booleanas correspondientes al conteo descendente en módulo 10 (apartado 16.2.1), a biestables síncronos tipo **D**, resulta:

$$T_0 = E \quad D_0 = E.\overline{q_0} + \overline{E}.q_0$$

$$T_1 = E.\overline{q_0} \cdot (q_3 + q_2 + q_1) \quad D_1 = E.q_3.\overline{q_1}.\overline{q_0} + E.q_2.\overline{q_1}.\overline{q_0} + q_1.q_0 + \overline{E}.q_1$$

$$T_2 = E.\overline{q_1}.\overline{q_0} \cdot (q_3 + q_2) \quad D_2 = E.q_3.\overline{q_2}.\overline{q_1}.\overline{q_0} + q_2.q_1 + q_2.q_0 + \overline{E}.q_2$$

$$T_3 = E.\overline{q_2}.\overline{q_1}.\overline{q_0} \quad D_3 = E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0} + q_3.q_0 + \overline{E}.q_3$$

$$\text{mín} = E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0} \quad \text{mín} = E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0}$$

En la función correspondiente a **D3** se ha prescindido de los términos $+ q_3.q_2$ y $+ q_3.q_1$ ya que nunca se alcanzan dichos valores (12 y 10).

Contador bidireccional

En el caso del contador módulo 10 (década) bidireccional (apartado 16.2.2):

$$T_0 = E$$

$$T_1 = C.E.\overline{q_3}.q_0 + \overline{C}.E.\overline{q_0} \cdot (q_3 + q_2 + q_1)$$

$$T_2 = C.E.q_1.q_0 + \overline{C}.E.\overline{q_1}.\overline{q_0} \cdot (q_3 + q_2)$$

$$T_3 = C.E.(q_2.q_1.q_0 + q_3.q_0) + \overline{C}.E.\overline{q_2}.\overline{q_1}.\overline{q_0}$$

$$\text{máx / mín} = C.E.q_3.q_0 + \overline{C}.E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0}$$

construyendo este mismo contador con biestables síncronos tipo **D**, queda:

$$D_0 = E.\overline{q_0} + \overline{E}.q_0$$

$$D_1 = C.E.\overline{q_3}.q_1.q_0 + C.q_1.\overline{q_0} + \overline{C}.E.q_3.\overline{q_1}.\overline{q_0} + \overline{C}.E.q_2.\overline{q_1}.\overline{q_0} + \overline{C}.q_1.q_0 + \overline{E}.q_1$$

$$D_2 = C.E.\overline{q_2}.\overline{q_1}.q_0 + C.q_2.\overline{q_1} + C.q_2.\overline{q_0} + \overline{C}.E.q_3.\overline{q_2}.\overline{q_1}.\overline{q_0} + \overline{C}.q_2.q_1 + \overline{C}.q_2.q_0 + \overline{E}.q_2$$

$$D_3 = C.E.\overline{q_3}.q_2.q_1.q_0 + C.q_3.\overline{q_0} + \overline{C}.E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0} + \overline{C}.q_3.q_0 + \overline{E}.q_3$$

$$\text{máx / mín} = C.E.q_3.q_0 + \overline{C}.E.\overline{q_3}.\overline{q_2}.\overline{q_1}.\overline{q_0}$$

16.4. Los contadores y el sincronismo global

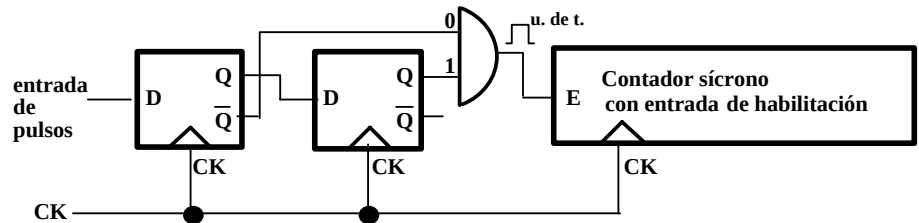
Contaje de pulsos diferentes del reloj

El diseño de sistemas digitales complejos se realiza en forma síncrona; para ello, además de utilizar contadores síncronos, conviene prestar atención a la forma de incorporarlos dentro del diseño respetando el sincronismo global del sistema.

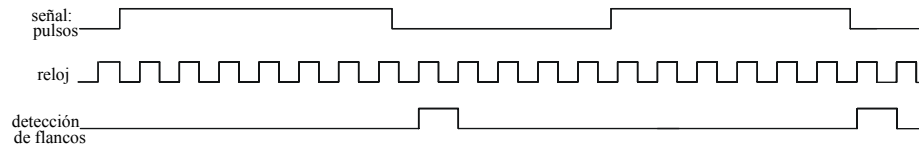
No basta con que los contadores sean síncronos, también hay que asegurar que su conexión se hace en forma síncrona. Si, por ejemplo, un contador recibe en su entrada de pulsos (entrada que actúa por flancos) cualquier señal diferente del propio reloj del sistema se pierde el sincronismo, ya que el contador cambiará su estado según el flanco activo de esa señal y no con referencia al flanco activo del reloj. Lo mismo sucede si se producen borrados asíncronos del contador (por entradas del tipo *Clear* o *Reset*).

El reloj central de un sistema secuencial síncrono es la única señal que actúa «por flancos»; cualquier otra señal debe actuar por niveles booleanos **0** y **1**. En tal sentido, cuando se desea contar pulsos diferentes a los propios de la señal de reloj, ha de transformarse el correspondiente flanco de los mismos (generalmente el de bajada, con el cual finaliza el pulso) en un pequeño pulso coincidente con una unidad de tiempo del reloj central y habilitar con dicho pulso el conteo del contador síncrono, cuyo reloj seguirá siendo el propio del sistema secuencial global.

El esquema necesario para detectar un flanco de bajada de un pulso cualesquiera (de mayor duración que la unidad de tiempo del reloj central) y transformar dicho flanco en un nivel activo **1**, cuya duración coincida con una unidad de tiempo, es simple: dos biestables sucesivos, formando un reducido registro de desplazamiento, que detecten la secuencia **10** (en una *bajada* el valor anterior del pulso será **1** y el siguiente valor del pulso será **0**). [Véase este mismo diseño realizado como ejercicio en 14.2.3, página 86.]



Este tipo de esquema de «detección de flanco» (detectar un **1** seguido de un **0** en el caso del flanco descendente o viceversa para flancos de subida) debe ser utilizado en cualquier actuación «por flancos» de una señal distinta de la del reloj central del sistema.



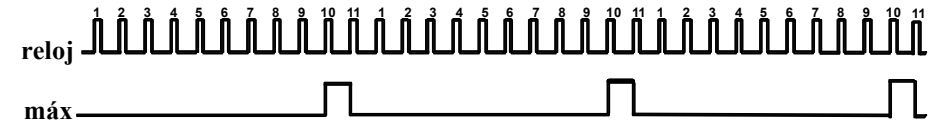
El detector de flancos es un bloque secuencial que aparece con mucha frecuencia en los diagramas de bloques correspondientes al diseño de sistemas digitales.

La señal resultante de la detección de flancos está sincronizada con el reloj del sistema, tiene la misma frecuencia que la señal de pulsos (un pulso por cada uno de entrada) pero su «tiempo en **1**» coincide con una unidad de tiempo del reloj: de esta forma, solamente habilita una vez (coincidiendo con el reloj) por cada pulso que se desea contar. Esta señal es del tipo de las ondas de temporización que trataremos a continuación.

Ondas de temporización

La señal que se obtiene en la salida **máx** de un contador módulo **n**, alimentado con la frecuencia de reloj **CK** del sistema digital correspondiente, es una onda que se repite cada **n** pulsos de reloj (frecuencia = f_{CK} / n) y cuyo intervalo en valor **1** coincide con una unidad de tiempo del reloj **CK**.

Por ejemplo, en el caso de un contador módulo 11, dicha señal, con amplitud en **1** de una unidad de tiempo de reloj, dividirá la frecuencia del reloj por 11:



Este tipo de formas de onda pueden ser utilizadas como señales de temporización en los sistemas síncronos: su actuación solamente se hará presente durante un pulso de reloj y podrá servir para condicionar transiciones de forma que se realicen después de haber transcurrido el intervalo de tiempo correspondiente a su período.

Asimismo, si deseamos temporizar en forma síncrona con alguna señal (bien sea una entrada al circuito digital o una señal intermedia generada por el mismo) deberá configurarse una onda de temporización en forma análoga: intervalo activo igual a una unidad de tiempo de reloj. Lo cual puede hacerse mediante un detector de flancos como el detallado en la página anterior.

Así, por ejemplo, en sistemas de control de la potencia suele interesar temporizar con la señal de cruce por cero de la red, la cual se repite cada 10 ms (dos veces por período, frecuencia de 50 Hz). Dicha señal puede obtenerse comparando la señal de la red (reducida mediante un transformador), rectificada en doble onda, con una referencia baja; posteriormente habrá que utilizar un detector de flancos para que la señal resultante actúe una sola vez (esté activa durante un solo pulso de reloj).

Dividiendo la anterior señal por 100 se tiene un período de 1 segundo; la salida **máx** del contador módulo 100 que produce tal división de frecuencia presenta un «tiempo en **1**» de una sola unidad de tiempo de reloj, de forma que puede ser utilizada como onda de temporización.

17 APLICACIONES DE LOS CONTADORES

- 17.1. Contaje de objetos y sucesos
- 17.2. División de frecuencias y ampliación de períodos
- 17.3. Medida de tiempos
- 17.4. Multiplexado temporal: reparto de tiempos
- 17.5. Medida de frecuencias

Sin duda, los contadores son los bloques digitales más utilizados, estando presentes en la mayor parte de los sistemas digitales, habida cuenta del amplio número y diversidad de sus aplicaciones.

En primer lugar, el propio contaje directo de unidades que, además de la información sobre número de objetos, personas o sucesos, permite el control de dicho número; por ejemplo controlar el número de objetos a insertar en un envase, el número máxima de personas presentes en un recinto, ... Y la división de frecuencias, consecuencia directa del contaje de sus pulsos, que ofrece la posibilidad de disminuir la frecuencia de las señales y aumentar la unidad temporal que señalan sus períodos.

El tiempo es una variable «omnipresente» que afecta a todo tipo de procesos y actividades; además, puede ser aprovechada indirectamente para medir otros tipos de magnitudes. Los contadores son una buena herramienta para el manejo de la variable tiempo, por cuanto que permiten medirla con precisión y permiten, también, definir intervalos temporales precisos.

La forma de medir tiempos admite diversidad de opciones: desde la medida horaria propia de los relojes o la medida con mayor resolución y precisión que realizan los cronómetros, hasta la determinación de intervalos temporales precisos (temporizadores). Asimismo, la medida de tiempos puede ser utilizada para medir velocidades (al recorrer espacios conocidos) y distancias (conocida la velocidad con que se recorren).

La distribución del tiempo en partes (multiplexado temporal) permite la generación repetitiva de secuencias, la selección de señales y su medida con un instrumento común, la realización de temporizaciones sucesivas, el control temporal de procesos, etc.

Así como medir tiempos consiste en contar pulsos de frecuencia fija conocida durante el intervalo a medir; el contaje recíproco de pulsos durante un intervalo de tiempo fijo y conocido da como resultado la medida de la frecuencia de los pulsos. Los frecuencímetros configuran otro campo de las aplicaciones de los contadores, con utilidad para la caracterización de señales, medida de velocidades de motores, medida de velocidad lineal de vehículos, medida de magnitudes codificadas en frecuencia, ...

Se deja para el capítulo siguiente la gran variedad de aplicaciones resultantes de la modulación de anchura de pulsos PWM.

17.1. Contaje de objetos y de sucesos

La cuenta directa de unidades (pulsos, objetos, sucesos,...) encuentra aplicación en muchos procesos. Para ello se precisa que el fenómeno (evento) a contabilizar sea primeramente transformado en señal eléctrica, mediante el correspondiente sensor al que seguirá un circuito de conformación de pulsos adecuado.

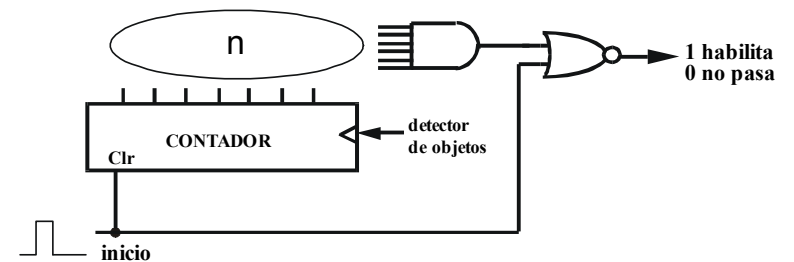
Por ejemplo, se pueden contar objetos haciéndolos pasar en fila de a uno por una cinta transportadora entre una célula fotoeléctrica y un foco luminoso. La utilización de fotodetectores y otros tipos de sensores de interposición o de proximidad para detectar presencia de objetos, personas o marcas es tan amplia que existe una gran diversidad en la oferta de tales componentes.

Por otra parte, se da el caso de operaciones que pueden realizarse indirectamente por contaje; por ejemplo, el control de posición o de ángulo de algunos mecanismos (cabezales de impresoras, posicionamiento de taladros, etc.) puede efectuarse mediante regletas o discos graduados, con marcas que se cuentan a partir de un origen.

En tareas de control es muy útil el contaje hasta un número predeterminado; lo cual puede realizarse de dos formas:

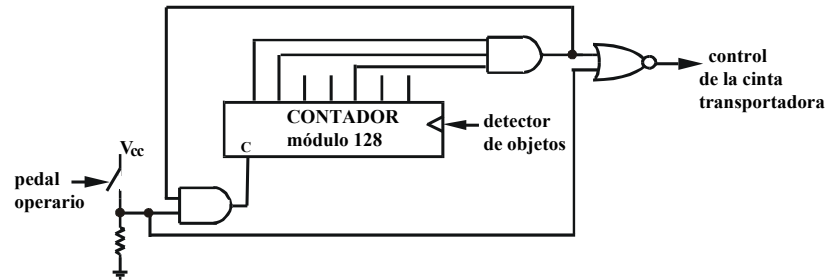
- comparando el resultado del contaje de un contador normal (ascendente) con el número deseado, a través del correspondiente comparador;
- prefijando en un contador descendente (imponiendo mediante carga paralelo) el número a contar y detectando cuándo el contaje inverso llega a cero.

Contadores de este tipo pueden emplearse, por ejemplo, para contar el número de objetos que entran en un recipiente o envase; al alcanzarse el número fijado, el pulso de salida determina el fin de la serie de **n** objetos (inhibe el paso de más objetos) y, para dejar pasar una nueva serie de **n** objetos, un pulso de inicio debe borrar (poner a 0) el contador.



En la figura anterior, la puerta "y" debe conformar el término mínimo reducido del número **n**, es decir, debe recibir las salidas del contador que corresponden a dígitos con valor 1 en dicho número **n**.

Consideremos un sencillo ejemplo de control de número de unidades: *una cinta transportadora mueve pequeños objetos de uno en uno; al final de ella, un operario coloca una caja de embalaje y, al presionar un pedal, deben caer 100 objetos en la caja.* La figura siguiente muestra un posible circuito para controlar el movimiento de la cinta, de forma que no se produce error aunque se presione el pedal mientras la cinta se mueve.

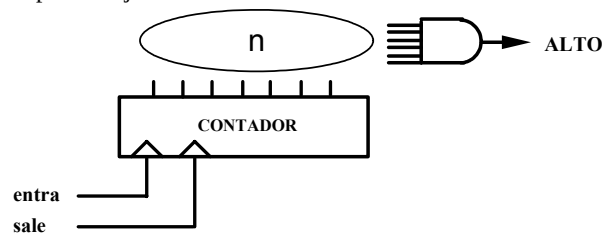


La activación del pedal borra el contador, pero durante dicho intervalo la cinta transportadora permanece inmóvil (entrada inferior de la puerta "o-negada"); posteriormente, al soltar el pedal, la cinta transportadora avanza y el detector de objetos envía los correspondientes pulsos al contador, hasta alcanzar el número 100 (**1100100**) que detiene el movimiento de la cinta (entrada superior de la puerta "o-negada") hasta una nueva activación del pedal.

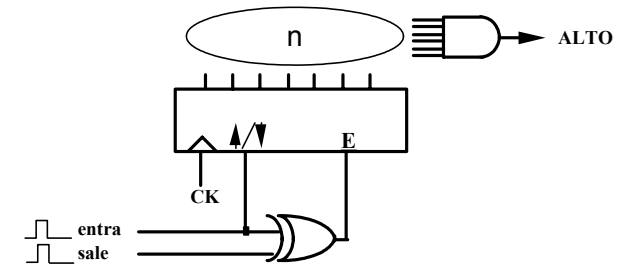
Obsérvese que el borrado del contador se encuentra condicionado a que se encuentre en el número 100, para evitar que activaciones erróneas del pedal durante el proceso de llenado de una caja incrementen el número de objetos en la misma.

Configuraciones análogas pueden utilizarse para cualquier control de número de unidades, por ejemplo, para dejar pasar n pulsos cada vez que se activa su entrada de pulsos, para avanzar n posiciones (determinadas por marcas), etc.

De igual forma, para controlar un número máximo (de personas o de objetos presentes en un recinto), supuesto que se disponga de sendos detectores de entrada y de salida (que comunican un pulso por cada entrada o salida individual), puede emplearse un contador bidireccional; el contador cuenta los pulsos provenientes del sensor de entradas y descuenta los que recibe del sensor de salidas y, al alcanzar el número máximo, produce una señal que avisa o cierra el paso (señal de alto: *stop*) hasta que se producen salidas que sitúan el contador por debajo del número máximo.



No es frecuente disponer de contadores con entradas de pulsos separadas para el conteo y desconteo y, además, un diseño síncrono no admite varias entradas de reloj; lo habitual será configurar el contador anterior en la forma que se representa en la figura siguiente (en ella los pulsos de entradas y de salidas se han ajustado a una unidad de tiempo del reloj mediante los correspondientes detectores de flanco).



El contador debe actuar cuando recibe un solo pulso de entrada o de salida y debe hacerlo en sentido ascendente si el pulso es de entrada. La puerta "y", que produce la señal de salida, corresponde al término mínimo reducido del número n (recibe los dígitos con valor 1 en dicho número n).

17.2. División de frecuencias y ampliación de períodos

La división de frecuencia se utiliza para obtener frecuencias inferiores a partir de una frecuencia patrón o, lo que es lo mismo, para obtener unidades de tiempo múltiplos del período que corresponde a dicha frecuencia patrón.

Por ejemplo, en los relojes digitales de pulsera la unidad de tiempo básica (un segundo) suele obtenerse a partir de un cristal de cuarzo de 32,768 KHz, dividiendo dicha frecuencia mediante un contador completo de 15 biestables ($2^{15}=32.678$).

Se consigue mayor precisión con cristales de cuarzo de 1 MHz (generalmente 1000000 ± 1 Hz, precisión del uno por millón, que corresponde a una desviación inferior a 3 segundos al mes), cuya división a través de 4 contadores década (módulo 10) proporciona el período de 0,01" básico para cronómetros o relojes de alta precisión.

La unidad de tiempo puede obtenerse, también, a partir de la frecuencia de la red de tensión alterna (220 voltios, 50 Hz): una vez reducida adecuadamente la tensión a través de un transformador, su rectificación da lugar a pulsos iniciales de 50 Hz (rectificación en media onda) o de 100 Hz (en onda completa) que, por división de frecuencia, permiten obtener períodos de un segundo o de una décima, según interese.

De igual modo se pueden definir unidades de tiempo diferentes para intervalos de medida, control de procesos, etc.; por ejemplo, los frecuencímetros suelen utilizar un intervalo de 6" o 10" como tiempo de medida e intervalo de visualización de la misma. Asimismo se utilizan divisores de frecuencia en aplicaciones de sincronismo, adaptación a la velocidad de trabajo de dispositivos más lentos, transmisión de datos, etc.

Los módulos de comunicación digital serie (comunicación con periféricos o entre sistemas informáticos) permiten seleccionar la velocidad de transmisión; para ello disponen de un amplio divisor de frecuencia a partir de un oscilador inicial que suele emplear un cristal de 1,8432 MHz. Dicha frecuencia dividida por 48 (3×16) da lugar a 38,4 KHz y sucesivas divisiones por 2 generan las frecuencias típicas de transmisión de datos: 19.200, 9.600, 4.800, 2.400, 1.220, 600, 300, 150 baudios (bits por segundo).

Los osciladores de cristal de cuarzo proporcionan señales digitales con frecuencias muy precisas, pero las frecuencias de vibración de los cristales de cuarzo son relativamente elevadas; no se utilizan cristales por debajo de 10 KHz y los más habituales y de mayor precisión se sitúan en el rango de los megaciclos.

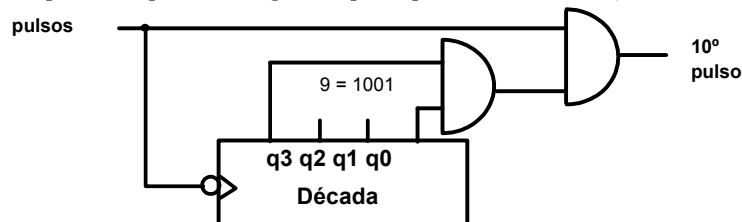
La forma de obtener osciladores de precisión para frecuencias inferiores al MHz consiste en dividir la frecuencia superior de un oscilador de cristal de cuarzo; la división de frecuencia se realiza mediante un contador módulo n , tomando la salida del biestable que corresponde al bit más significativo del contador o la salida **máx** del mismo.

Si la señal se toma de la salida **máx** del contador (o bien se utiliza un detector de flancos) se obtiene una onda de temporización con el período señalado y con «tiempo en 1» igual a una unidad de tiempo de reloj; esta onda de temporización permite habilitar cualquier cambio o transición en forma síncrona y solamente durante un pulso de reloj.

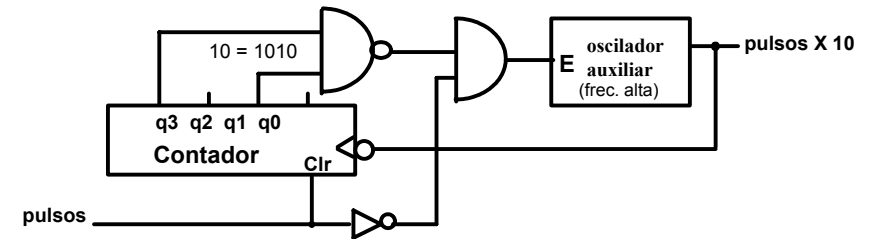
Cambio de escala en el número de pulsos

Un contador módulo n configura un divisor de escala por n que genera un pulso por cada n pulsos recibidos en su entrada; este pulso se obtiene en la salida del último biestable (el más significativo) del contador (o en su salida **máx**), la cual proporciona un pulso por cada «vuelta» del contador (por cada n pulsos).

Un caso particular de división de frecuencia, representado a continuación, es un divisor de escala que deja pasar precisamente el n -ésimo de los pulsos recibidos en su entrada; el circuito de la figura de cada 10 pulsos que recibe permite el paso al décimo de ellos (para lo cual la puerta "y" de salida del circuito es habilitada cuando el contador ha detectado el paso de 9 pulsos: el siguiente pulso pasa a través de ella).



La posibilidad inversa en el cambio de escalas se refiere a multiplicar el número de pulsos que se reciben por un factor n , lo cual puede hacerse mediante un contador que, al recibir cada pulso, permita el paso de n pulsos de frecuencia superior. La siguiente figura representa un sistema que multiplica por 10 el número de pulsos que le llegan.



El pulso de entrada borra el contador y, al finalizar dicho pulso, permite que el oscilador auxiliar se active y proporcione pulsos hasta que se alcanza el número 10, cuyo término mínimo reducido se utiliza para bloquear de nuevo al oscilador; durante el borrado del contador el oscilador se bloquea (mediante el inversor) para impedir que genere pulsos y no sean contados. La frecuencia de los pulsos de salida (oscilador auxiliar) ha de ser adecuadamente alta para que puedan producirse 10 pulsos en el intervalo entre cada pulso de entrada y el siguiente.

17.3. Medida de tiempos

A partir de un generador de pulsos de frecuencia fija y muy precisa, cuyo período sea mucho menor que los intervalos temporales a medir, la medida de tiempos se reducirá a contar el número de pulsos en cada intervalo; dicha medida quedará expresada en unidades equivalentes al período de los pulsos.

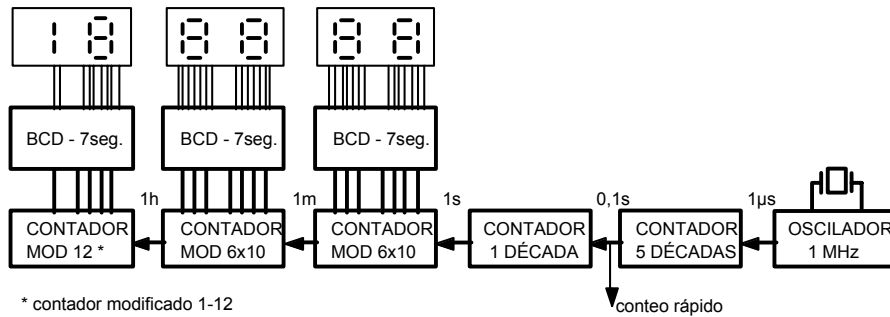
Como generador de pulsos de frecuencia precisa suele utilizarse un oscilador con cristal de cuarzo o la propia señal de la red de tensión alterna (50 Hz).

Consideraremos, a continuación, diversas utilidades relativas a la medida de tiempos:

- la medida del tiempo horario (horas, minutos, segundos): relojes digitales,
- la medida de intervalos con precisión y resolución: cronómetros,
- la delimitación de intervalos temporales precisos: temporizadores

17.3.1. Relojes digitales

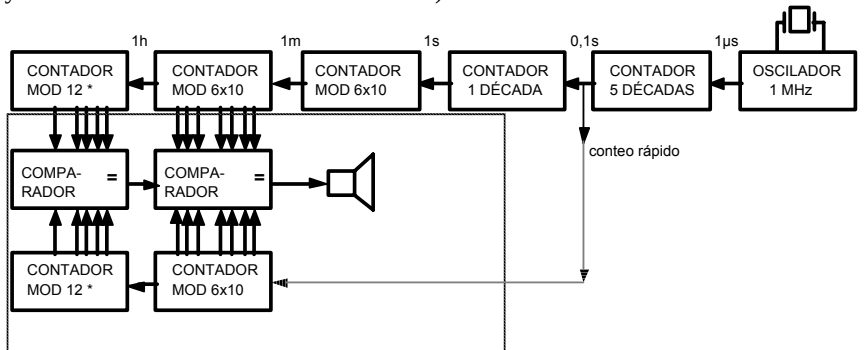
El caso más general de medida de tiempos corresponde a los relojes digitales (reloj horario en horas, minutos y segundos) cuyo esquema de bloques puede ser el siguiente:



Un contador adicional módulo 7 permite indicar los días de la semana y un nuevo contador hasta 31 señalará el día del mes; al cual seguirá un contador módulo 12 para obtener el número del mes y la correspondiente lógica de ajuste para los meses de 30 y de 31 días (y los 28/29 días de febrero).

La puesta en hora de este reloj suele realizarse llevando directamente la señal rápida de 0,1 segundos, mediante pulsadores apropiados, al contador de minutos y al contador de horas, hasta que en cada uno de ellos se contabilice el número deseado.

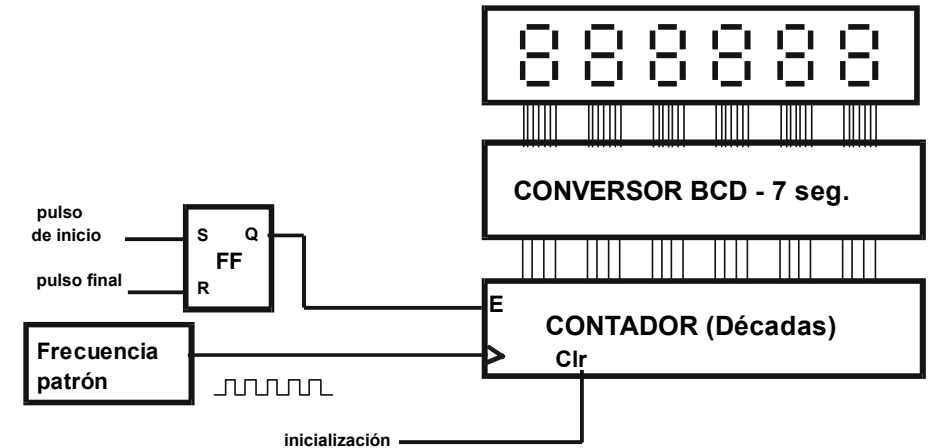
Es sencillo dotar a este reloj de alarma o despertador mediante un contador duplicado de horas y minutos en el que se fija, por conteo directo con la señal de 0,1 s, la hora y el minuto en el que debe sonar la alarma; un comparador entre ambos contadores (horas y minutos) activa, con su salida de igualdad, un pequeño zumbador (en cuyo caso la alarma sonará durante 1 minuto).



En el capítulo anterior se comentó la disponibilidad de circuitos integrados específicos para relojes con toda la circuitería necesaria para su configuración, sin más que añadir el visualizador y el cristal de cuarzo; así como contadores horarios periféricos de microprocesadores a los cuales comunican la hora mediante transmisión serie.

17.3.2. Cronómetros

Otra forma de medida de tiempo es la realizada por los cronómetros que permiten medir con muy alta precisión el tiempo transcurrido entre dos sucesos; su actuación viene definida por un pulso de comienzo y otro de final de medida, los cuales abren y cierran, respectivamente, la habilitación de un contador, previamente borrado, que recibe en su entrada de reloj pulsos de frecuencia fija y muy precisa.



Con este esquema funcional es posible realizar medidas de tiempo sumamente precisas sin más que disponer de la frecuencia patrón adecuada (cristal de cuarzo).

Un cronómetro permite conseguir una extraordinaria precisión en la medida de intervalos relativos a un determinado fenómeno físico, realizando una transformación de las condiciones físicas que definen sus instantes inicial y final en pulsos eléctricos; por ejemplo, el paso de una bala entre dos detectores fotoeléctricos permite medir el tiempo en que recorre tal distancia.

Tal medida de intervalos temporales es aplicable indirectamente a la medida de otras magnitudes físicas tales como velocidad de vehículos y proyectiles (midiendo el tiempo empleado en recorrer una distancia prefijada), distancia (midiendo el tiempo que tarda en recorrerla una señal de velocidad conocida con precisión), volumen de llenado de líquidos a través de una tubería (conocido su caudal), etc. El radar y el sonar calculan la distancia a que se encuentran los objetos detectados a partir de la medida del tiempo que tarda en volver a ellos la señal emitida, tras ser reflejada por dichos objetos.

17.3.3. Temporizadores

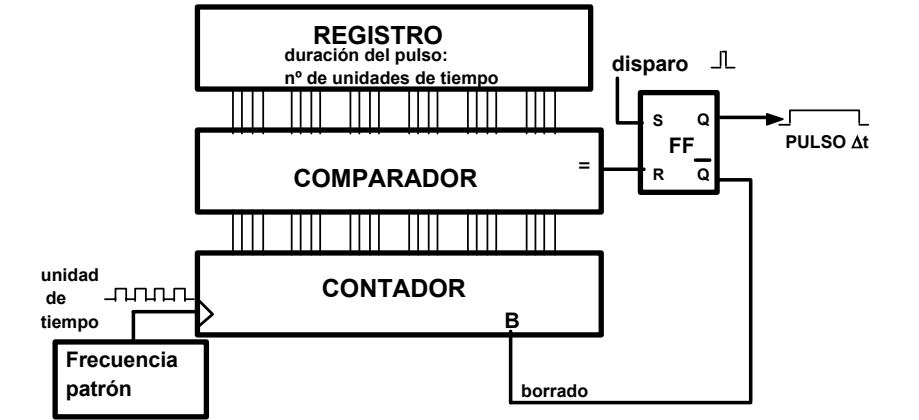
Otra medida particular de tiempos es la temporización, en la cual se establece el tiempo que debe durar un proceso, produciendo un pulso cuya duración coincida con el tiempo prefijado; dicho pulso determina el intervalo de activación del proceso, al finalizar el cual se produce su desconexión o desactivación o bien se genera una señal de alarma acústica o visual.

Una temporización no es sino un monostable de alta precisión, la cual se consigue a partir de una frecuencia patrón muy precisa, multiplicando su período por *n* mediante el correspondiente contador.

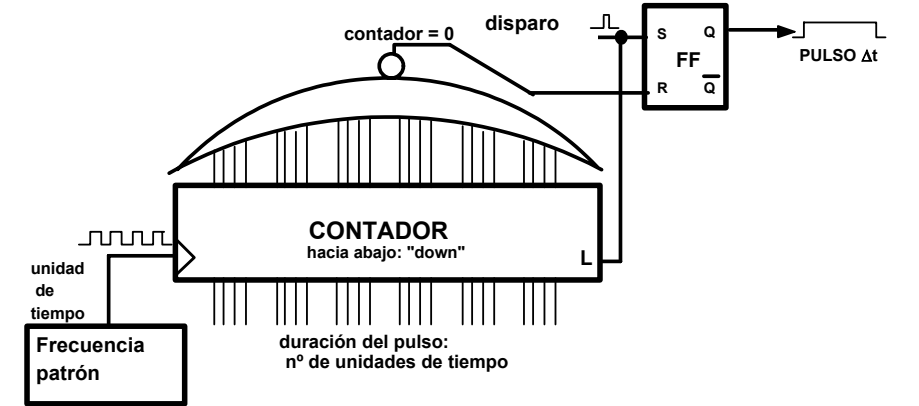
Temporizaciones, con diferentes grados de precisión, se utilizan en todo tipo de actividades, desde el control del tiempo de encendido de la iluminación de las escaleras comunitarias, hasta la apertura del objetivo de las máquinas fotográficas, pasando por la programación de tiempos en microondas o en hornos de cocina, y por los ciclos de funcionamiento de lavadoras y lavavajillas, por el control de luces de los semáforos, etc., que no son sino un conjunto de temporizaciones sucesivas.

En general, sistemas de temporización están presentes en el control de los procesos industriales de todo tipo, pues será preciso establecer en ellos los diversos tiempos de duración de cada fase o etapa.

La temporización puede ser por conteo ascendente, desde cero hasta el número prefijado, en forma análoga al sistema de alarma descrito para un reloj digital; sobre un registro o contador auxiliar se programa la duración del intervalo a temporizar y el final del mismo será detectado por comparación entre el contador activo y el registro programado.



Otra posibilidad consiste en utilizar un contador bidireccional (*up/down*), en el cual se programa inicialmente (mediante carga en paralelo o por conteo rápido ascendente), el tiempo de temporización deseado. Durante la temporización el contador va descontando las unidades de tiempo que transcurren, de forma que el contenido del contador muestra en cada momento el tiempo que falta para completarla; la llegada del contador a cero señala el final de la temporización.

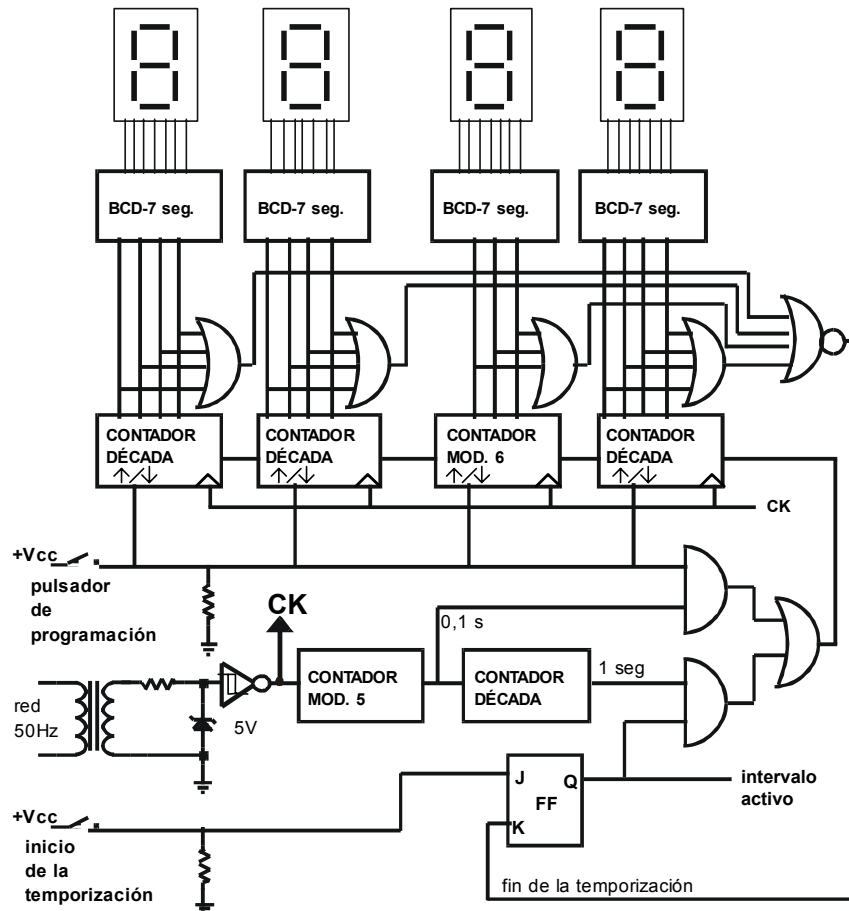


En el contador anterior, la entrada *L* (*load*) determina la carga en paralelo del número presente en las entradas del mismo; a partir de ahí, el contador descuenta hasta llegar a 0.

La figura de la página siguiente detalla un ejemplo de temporizador con una capacidad máxima de 99 minutos, una resolución de 1 segundo y programación por conteo rápido ascendente con pulsos de 0,1 s.

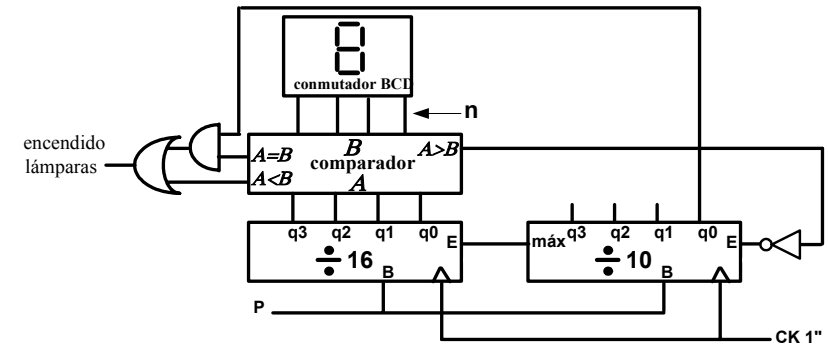
Mientras se mantiene activado el pulsador de programación el contador cuenta «hacia arriba» (*up*) a una velocidad de 10 pulsos por segundo que permite interrumpir el conteo en el número que se desea prefijar. Un segundo pulsador determina el inicio de la temporización al situar el biestable *JK* a 1; durante la temporización el contador descuenta segundo a segundo hasta llegar a cero, en tal momento la puerta "o-negada" (*Nor*) que recibe todas las salidas del contador borra el biestable *JK*, dando por finalizada la temporización.

La frecuencia patrón ha sido tomada de la semionda de 50 Hz de la red de tensión alterna, rectificada y limitada en amplitud mediante un zener de 5 V, para obtener pulsos positivos cuyos flancos se conforman mediante un inversor con entrada Schmitt; esta frecuencia actúa como reloj general del sistema y, además, dividida por 5 genera una onda de temporización de 0,1" para la programación del temporizador mientras que una segunda división por 10 produce el período base de la temporización (1").



Otro ejemplo de temporizador: Las lámparas de una escalera deben encenderse cada vez que se pulsa un interruptor **P** y mantenerse encendidas durante un intervalo de tiempo a programar mediante un conmutador BCD entre 10" y 90" (en múltiplos de 10"); durante los 10" siguientes las lámparas deben apagarse y encenderse 5 veces, con intervalos de 1".

Un posible diagrama de bloques del correspondiente circuito de control puede ser el representado en la figura de la página siguiente: se utilizan dos contadores sucesivos, el primero de ellos tiene un ciclo de 10" que es la unidad de tiempo de conteo del segundo; la salida de éste se compara con el número **n** fijado por el conmutador BCD, de forma que la señal de salida tendrá valor **1** durante un número de estados igual a **n** (es decir, durante **n x 10"**).

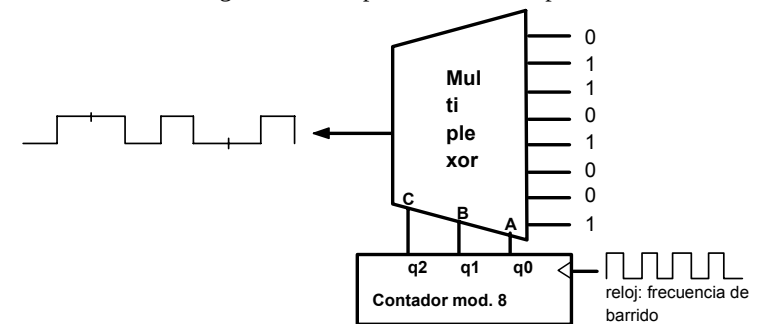


Durante el conteo del segundo contador de 0 a **n-1**, las lámparas permanecen encendidas: cuando dicho contador llega a **n**, «deja pasar» la intermitencia producida por la primera salida del primer contador (1" a **0** y 1" a **1**) y al pasar a **n+1** se deshabilitan los contadores, hasta un nuevo borrado de los mismos con el pulsador **P**.

17.4. Multiplexado temporal: reparto de tiempos

Generador de palabras digitales y de secuencias de señales

Un multiplexor digital puede ser utilizado como generador de formas de onda digitales o generador de palabras; conectando sus entradas de control a las salidas de un contador, éste determina un muestreo sucesivo de los valores booleanos impuestos en las **n** líneas de entrada, dando lugar a la correspondiente onda repetitiva.



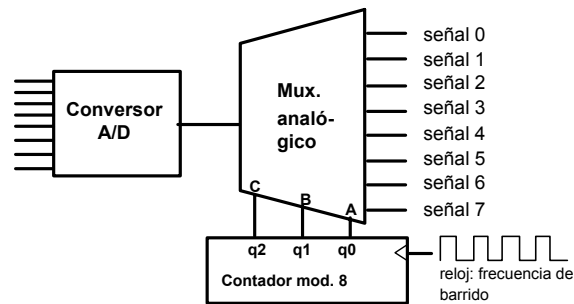
Análogamente, si se sustituye el multiplexor por un pequeño codificador ROM, se obtiene una secuencia repetitiva de los vectores de salida programados en dicho bloque, es decir, una secuencia de señales sincronizadas con el reloj del contador; lo cual puede ser útil para el control de maniobras repetitivas como, por ejemplo, el control de motores paso a paso o el control de máquinas herramientas (el codificador o «memoria ROM» sustituye en este caso a las antiguas cintas perforadas de programación).

Multiplexado de señales analógicas

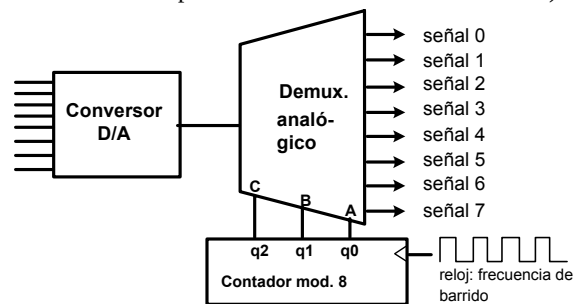
Un conjunto de n puertas de transmisión CMOS, conectadas a una misma línea de salida y controladas por un decodificador que active una sola de dichas puertas, constituye un multiplexor analógico que permite seleccionar una de entre n señales de entrada. El mismo multiplexor utilizado en dirección opuesta (1 entrada hacia n salidas) se convierte en demultiplexor analógico habida cuenta el carácter bidireccional que presentan las puertas de transmisión.

Conectando las entradas de control de un multiplexor analógico a un contador se realizará el muestreo sucesivo y cíclico de las n señales de entrada; la frecuencia del muestreo será la de los pulsos que reciba el contador.

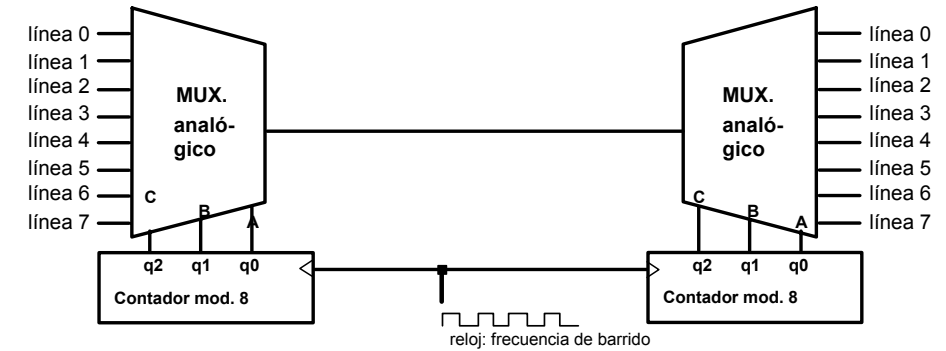
El muestreo de señales analógicas puede ser aprovechado para medir dichas señales con un mismo instrumento o circuito de medida (convertor analógico/digital). Resulta así una configuración típica de los sistemas de adquisición de datos para medir o controlar periódicamente diversas magnitudes físicas: el transductor correspondiente las transforma en tensiones y el multiplexado de ellas permite tomar sus valores con un solo convertor analógico-digital.



Asimismo, un demultiplexor analógico puede ser utilizado para generar varias referencias de tensión o varias señales lentas; un convertor digital/analógico puede proporcionar los diversos valores de tensión y el demultiplexor (controlado por un contador) los trasladará a sus salidas (será necesario que exista una capacidad adecuada conectada en cada línea de salida para mantener los valores de tensión).



También se utiliza el muestreo analógico para la transmisión de múltiples señales por una misma línea de comunicación: el multiplexor enviará muestras sucesivas de las n señales y, en la recepción, un sistema simétrico demultiplexor-contador, funcionando con la misma frecuencia de muestreo, separará las muestras hacia n líneas de salida.



Un teorema de muestreo, introducido por Claude E. Shannon, garantiza que, cuando se muestrea una señal a una frecuencia superior al doble de la frecuencia máxima contenida en ella, no hay pérdida de la información de la señal, es decir, es posible recuperar la señal inicial mediante filtrado de la señal muestreada.

El citado teorema de muestreo condiciona la integridad de la señal a transmitir a que la frecuencia de muestreo sea superior al doble de la frecuencia máxima de dicha señal; en cambio, no depende de la anchura de las muestras.

De esta forma, el muestreo a frecuencias adecuadas (superiores al doble de su frecuencia máxima) permite enviar un gran número de señales (en pequeños trozos de cada una de ellas) por la misma línea, multiplexadas en el tiempo. En la recepción de tales señales, una vez demultiplexadas las muestras recibidas, para obtener las señales primitivas basta con filtrar las líneas de salida mediante filtros pasa-baja con frecuencias de corte del orden de la mitad de la frecuencia de muestreo.

Reparto de tiempos

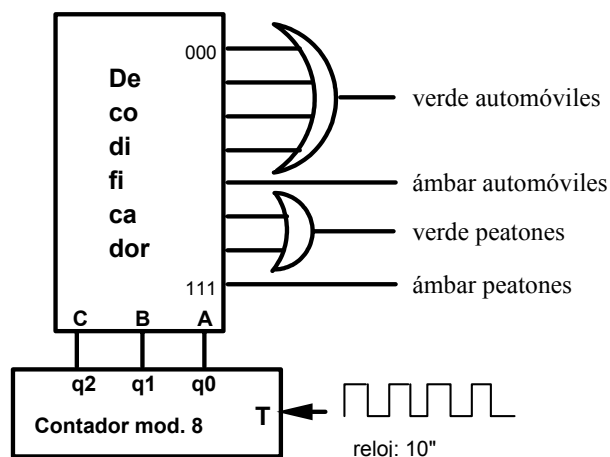
El multiplexado en tiempo (multiplexor controlado por un contador que recibe una señal de frecuencia fija) equivale a una distribución del tiempo en sucesivos intervalos, de igual duración, dedicados a diferentes señales o acciones. Con el mismo esquema circuital, sustituyendo el multiplexor por un decodificador de n líneas, se obtendrá un circuito de n salidas que se activarán sucesiva y cíclicamente; cada salida permite habilitar una actuación particular, que se irán ejecutando una tras otra.

De esta manera el ciclo se divide en partes o acciones sucesivas que se ejecutan en un orden dado (y se repetirán posteriormente en el mismo orden); el resultado es un conjunto de temporizaciones sucesivas, con igual duración de tiempo cada una de ellas.

También es posible dedicar varios intervalos a una misma acción y obtener así temporizaciones sucesivas de duraciones diferentes (siempre múltiplos del intervalo temporal básico).

Un ejemplo ilustrativo de este tipo de reparto de tiempos puede ser el comportamiento de un semáforo: *consideremos un semáforo para un simple paso de peatones que desarrolla el siguiente ciclo: 40" en verde para automóviles, 10" en ámbar para detenerlos; 20" en verde para paso de peatones y 10" en ámbar para completar el cruce de los mismos.*

El circuito de control de dicho semáforo puede ser el representado en la figura siguiente: un contador que recibe pulsos de 10" de período señala los ocho intervalos de 10" contenidos en el ciclo anterior, un decodificador diferencia cada uno de dichos intervalos y, a través de puertas "o", se agrupan tales intervalos para formar las cuatro temporizaciones a las que alude el enunciado.



Obviamente, el decodificador no es imprescindible, pues las diferentes salidas pueden ser definidas a través de funciones booleanas:

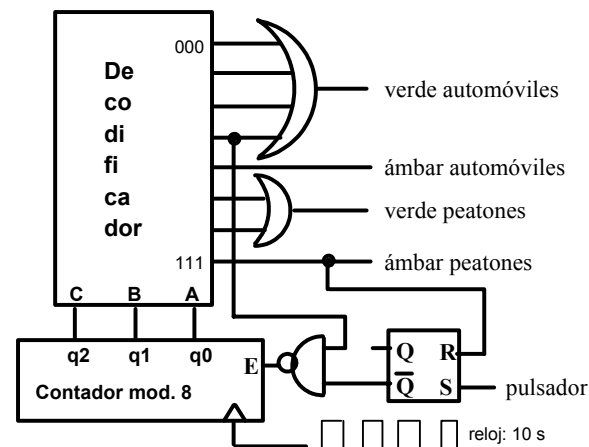
$$\text{verde autom3viles} = \overline{q_2} \qquad \text{3mbar autom3viles} = q_2.\overline{q_1.q_0}$$

$$\text{verde peatones} = q_2.(q_1 \otimes q_0) \qquad \text{ámbar peatones} = q_2.q_1.q_0$$

pero, desde una perspectiva conceptual, el decodificador expresa muy gráficamente el reparto del tiempo (4 intervalos de 10" para paso de automóviles, 1 intervalo para detenerlos, 2 intervalos para paso de peatones y 1 intervalo de aviso a éstos).

Podemos perfeccionar el ejemplo anterior de manera que los peatones deban activar un pulsador como demanda de paso: sea un cruce de peatones que cuenta con un semáforo para detener a los automóviles, con un pulsador **P** que debe ser activado por los peatones cuando desean cruzar; la activación de **P** da lugar al siguiente ciclo: 10" en amarillo para detener a los automóviles, 20" en rojo (verde para peatones), 10" en amarillo para peatones, pasando finalmente al estado de circulación de automóviles (rojo para peatones); cuando en dicho estado de circulación se recibe una nueva demanda de paso, es atendida pero asegurando siempre que el intervalo mínimo de paso de automóviles sea de 40".

Puede utilizarse un biestable **RS** para recoger la demanda de paso por parte de los peatones; dicho biestable se borra en el intervalo de ámbar para peatones (que es cuando se completa el paso de peatones, en respuesta a una demanda anterior). Mientras hay nuevas solicitudes de paso se ejecuta normalmente el ciclo completo; pero, si no hay demanda de paso, el ciclo se detiene en el cuarto intervalo de 10" de paso de automóviles (ya que la puerta *Nand* anula la habilitación del contador) y permanece en dicha situación (paso de automóviles) hasta que se produce una petición de paso por parte de peatones.

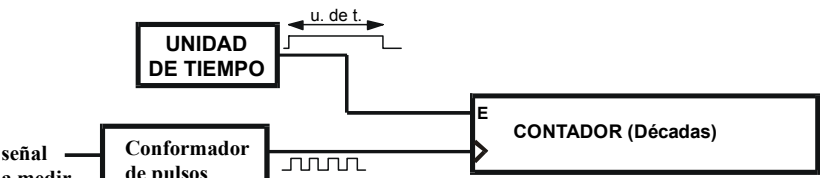


17.5. Medida de frecuencias

La frecuencia de una señal o de un proceso periódico es el número de repeticiones o pulsos por unidad de tiempo; su medida consiste en contar pulsos durante un intervalo de tiempo unidad o durante uno de sus múltiplos.

La medida de frecuencias es en cierta manera el recíproco de la medida de tiempos: mientras que en esta segunda se cuentan pulsos de frecuencia precisa y conocida durante el intervalo temporal a medir, para la medición de una frecuencia se cuentan los pulsos de la señal durante un intervalo de tiempo de duración precisa y conocida.

Para ello se requiere un subsistema que determine con precisión los intervalos temporales, a partir de una frecuencia patrón; durante la unidad de tiempo definida se permite el conteo de pulsos de la señal a medir y, al final de la misma, se inhibe dicho conteo, quedando recogida en el contador la medida de la frecuencia: número de pulsos por unidad de tiempo.

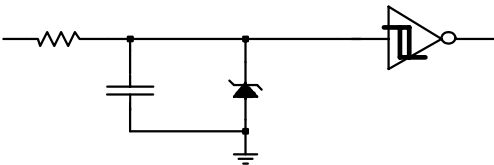


Esquema básico conceptual de un frecuencímetro

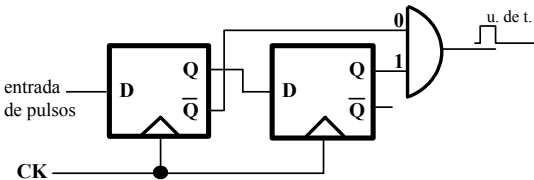
Un adaptador previo deberá «digitalizar» los pulsos de la señal de entrada, filtrando los posibles rebotes, ajustando sus valores de tensión y conformando flancos adecuadamente verticales; para ello, el bloque conformador de pulsos debe efectuar un filtrado pasa-baja (integración) a fin de evitar rizados y rebotes, recortar las tensiones en los niveles booleanos y «verticalizar» los flancos:



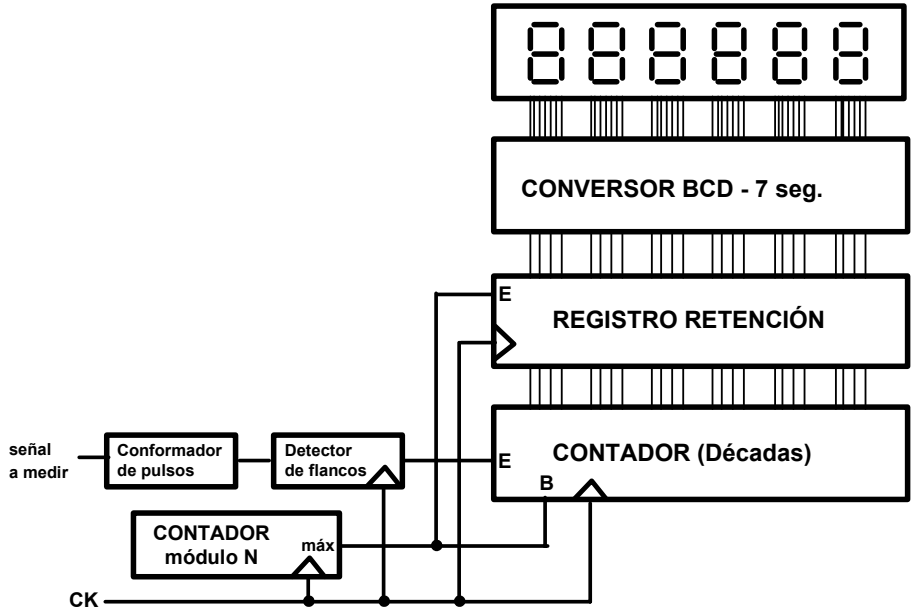
La integración puede ser hecha mediante un simple filtro RC (o con un filtro activo), un diodo zener puede recortar los niveles de tensión y un inversor con entrada Schmitt sirve para evitar rebotes y conformar flancos verticales:



Normalmente el diseño será síncrono con un reloj diferente de los pulsos de la señal de entrada, de forma que después del conformador de pulsos se necesitará un detector de flancos de los mismos para proceder a su conteo síncrono:



La figura siguiente representa un frecuencímetro de funcionamiento continuo; el intervalo de medida ha de ser del orden de 10", a fin de que cada medida se visualice durante un tiempo mínimo adecuado; como el contador va a ser utilizado continuamente, el resultado de cada medida será almacenado en un registro de retención (*latch memory*) para su visualización.



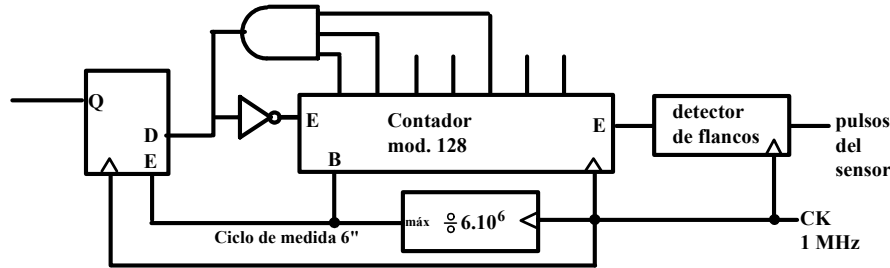
El contador que define el ciclo de medida divide por N la frecuencia del reloj, de forma que el período de medida será $T = N \cdot T_{CK}$ y el pulso de salida de dicho contador es una onda de temporización (cuyo tiempo en 1 es una sola unidad de tiempo de reloj). Cada ciclo de medida comienza con la puesta a cero del contador; a partir de dicho momento éste cuenta los pulsos de la señal de entrada durante el intervalo temporal $T = N \cdot T_{CK}$, al final del cual el mismo pulso que borra el contador recoge el conteo anterior sobre el registro y lo representa en el visualizador.

Esta configuración circuital permite un diseño síncrono a partir del reloj que genera la frecuencia patrón; el detector de flancos servirá para el conteo síncrono de los pulsos de entrada al frecuencímetro.

Medida y control de la velocidad de un motor

Una aplicación característica de la medida de frecuencias es la medida de la velocidad angular de un motor, expresada en número de revoluciones por minuto; para ello se detectan las vueltas del eje del motor mediante un sensor de tipo magnético o fotoeléctrico que genera los pulsos a contar.

La estructura circuital básica de un frecuencímetro de funcionamiento continuo sirve para configurar circuitos de control de la velocidad de un motor; por ejemplo, el de la figura cuya salida se activa si el número de revoluciones por minuto es superior a 1000.



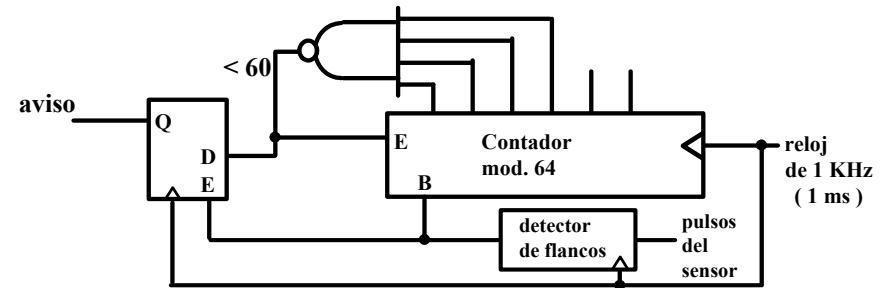
La velocidad de 1000 revoluciones por minuto equivale a 100 revoluciones en 6 segundos. Adoptando 6" como tiempo de medida, el contador deberá «vigilar» si se alcanza o no el número 100 (**1100100**); cuando el contador llega a tal cantidad, se inhibe su habilitación y la puerta "y" señala que se ha superado la velocidad máxima, dato que es recogido por el biestable **D** al finalizar el ciclo de medida e iniciarse uno nuevo (con el borrado del contador).

El mismo pulso (la onda de temporización de 6" generada por el contador inferior) produce la habilitación del registro (en este caso, un simple biestable para almacenar el aviso de superación de velocidad) y el borrado síncrono del contador para iniciar una nueva medida.

El anterior circuito podría utilizar un comparador con el número 100 (**1100100**), en lugar de la puerta "y"; en todo caso, es importante inhibir el contador una vez que se ha detectado la velocidad máxima, para evitar que «se de la vuelta» (llegue a su valor máximo y, luego, siga contando desde 0).

El esquema anterior presenta un tiempo de respuesta de 6 segundos (el tiempo de medida); puede obtenerse una respuesta más rápida midiendo el tiempo de cada vuelta (su período), en lugar de contar el número de revoluciones.

1000 revoluciones por minuto corresponden a un período de $60"/1000 = 60 \text{ ms}$; períodos inferiores a 60 ms (**111100**) suponen velocidad superior a la máxima. El circuito para medir dicho período puede ser el de la figura siguiente: el contador cuenta ahora el tiempo transcurrido entre dos pulsos del sensor y si dicho tiempo es inferior a 60 la puerta "y-negada" señala velocidad superior a la máxima.



En este caso son los pulsos del sensor de vueltas del motor los que determinan el intervalo de medida y, para ello, generan los pulsos de habilitación de dato y borrado del contador; al igual que en el caso anterior, es necesario inhibir el conteo una vez que se han detectado 60 ms, para evitar que el contador pase a valor 0. Igualmente se podría utilizar un comparador con el número 60 (**111100**), en lugar de la puerta "y-negada".

Medida de la velocidad de un vehículo

En cualquier vehículo que circula sobre ruedas, sea bicicleta, motocicleta o automóvil, el producto del número de revoluciones por minuto por la longitud de la circunferencia exterior de sus ruedas expresa la velocidad lineal del vehículo.

Por ejemplo, para una bicicleta con ruedas de 70 cm de diámetro (220 cm de circunferencia) la velocidad en kilómetros por hora coincide aproximadamente con el número de revoluciones que da en 8 segundos:

$$\frac{n \text{ vueltas}}{8 \text{ segundos}} \cdot 220 \text{ cm} = n \cdot \frac{220 \text{ cm}}{8"} = n \cdot \frac{220 \text{ cm}}{8"} \cdot \frac{100000 \text{ cm}}{3600"} \cdot \frac{\text{Km}}{\text{h}} =$$

$$= n \cdot \frac{220 \cdot 3600}{8 \cdot 100000} \cdot \frac{\text{Km}}{\text{h}} = 0,99 n \approx n \frac{\text{Km}}{\text{hora}}$$

De manera que un frecuencímetro con una unidad de tiempo de 8 segundos proporciona directamente la velocidad de la bicicleta (cuyo diámetro de rueda sea de 70 cm) en kilómetros por hora; con diferentes diámetros de rueda o tiempos de medida distintos será necesario efectuar el correspondiente cambio de escala.

Codificación en frecuencia

En ocasiones para la transmisión de valores medidos por sensores alejados se utiliza su codificación en frecuencia (se envía una frecuencia proporcional al valor medido); este tipo de comunicación en frecuencia ofrece alta fiabilidad, minimizando el efecto de los errores propios de la transmisión; un frecuencímetro transformará la frecuencia recibida en un número que expresa la medida del sensor.

18 MODULACIÓN DE ANCHURA DE PULSOS

- 18.1. Control todo/nada
- 18.2. Modulación de anchura de pulsos
- 18.3. Modulación PWM en pulsos distribuidos
- 18.4. Conversión tensión-tiempo de tipo rampa
- 18.5. Conversores sigma-delta

La modulación de anchura de pulso es una codificación analógica, alternativa a la representación habitual en amplitud de tensión, que se acomoda muy fácilmente a las técnicas digitales y permite configurar módulos de control de potencia, conversión DC-DC, potenciómetros digitales, conversión digital-analógica y analógico-digital,... Este tipo de etapas resulta de gran interés en el diseño microelectrónico puesto que permiten reducir, en forma considerable, la parte analógica que acompaña a la digital.

El objeto propio de la electrónica es la información; las señales eléctricas son el soporte físico de la información. Generalmente dicha información se representa sobre la amplitud de la tensión eléctrica que conforma la señal: las variaciones de tensión corresponden, en su caso, a las variaciones del valor de la magnitud física representada por la señal. Pero también es posible representar la misma información sobre la anchura de pulsos de tensión fija: la señal soporte de la información quedará determinada por las variaciones en la duración de los sucesivos pulsos; este procedimiento de codificación de la información se denomina modulación de anchura de pulsos (**PWM**).

Ambas codificaciones analógicas, en amplitud de tensión o en anchura de pulso, son equivalentes: el teorema de muestreo de Shannon garantiza que ambas señales contienen la misma información, siempre que la frecuencia de los pulsos de anchura modulada sea superior al doble de la frecuencia máxima de la señal. Ahora bien, resulta que los pulsos de anchura modulada son relativamente fáciles de manejar mediante técnicas digitales, aprovechando que los contadores son una herramienta adecuada para manejar el tiempo.

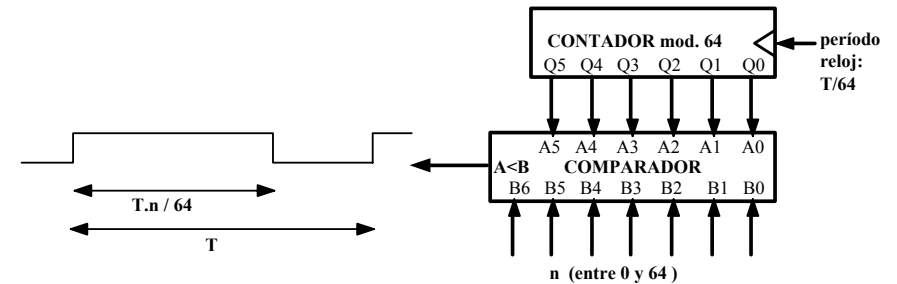
Contadores y sumadores permiten generar pulsos de anchura controlada por un número y, por otra parte, la carga de condensadores, seguida por comparadores analógicos, proporciona pulsos de anchura controlada por una tensión; de esta forma se configuran circuitos que pueden sustituir (si la velocidad de trabajo requerida no es muy elevada) a etapas típicamente analógicas mucho más complejas.

En concreto, la modulación de anchura de pulsos resulta muy útil para el control de potencia (por el mecanismo todo/nada), para la conformación de potenciómetros digitales (para controlar la amplitud de señales o efectuar una conversión DC-DC) y, también, para efectuar sencillas conversiones de señal digital a analógica (de número a tiempo o a tensión) y de analógica a digital (de tensión a número). Todos estos casos serán desarrollados en los apartados que siguen.

18.1. Control todo/nada

Uno de los métodos más simples de regulación de potencia es el control todo/nada (*on/off*): a partir de la potencia máxima a suministrar y de un período **T** que marca la duración del ciclo, se efectúa una conmutación «todo/nada» de la potencia, de forma que durante un primer intervalo **T₁** se aplica la potencia máxima **P_{máx}** y durante el resto del ciclo **T - T₁** no se aplica potencia alguna; promediando en el tiempo, la potencia eficaz aplicada será **P_{máx}·T₁/T**.

La señal de control *todo/nada* puede ser generada mediante un contador módulo **N**, activado con una señal de reloj cuyo período sea **T/N**, seguido de un comparador en cuya segunda entrada se establece el número que actúa como referencia **n**; **n** puede variar de **0** a **N**. Tomando la salida "<" del comparador (**contador < n**) se obtiene una señal rectangular de período **T** y cuyo «tiempo en 1» será **T·n/N**, es decir, el porcentaje de tiempo activo (tiempo de *on*) respecto al total será **n/N**.



Si durante el tiempo de *on* se suministra a la carga la potencia máxima y durante el tiempo de *off* no se le suministra potencia alguna, la potencia promedio será:

$$P = P_{\text{máx}} \cdot \frac{T_1}{T} = P_{\text{máx}} \cdot \frac{n}{N}$$

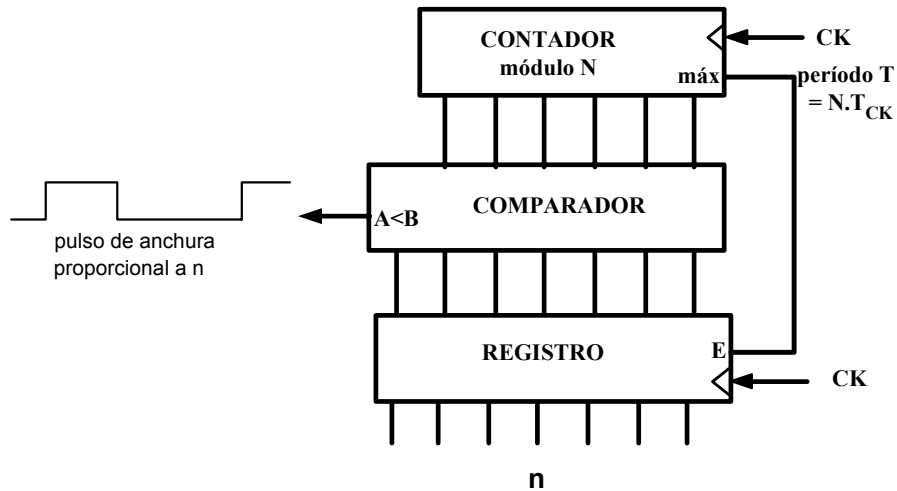
Un contador década permite una regulación todo/nada con 11 niveles (de 0 a 10) y un paso entre niveles del 10 % mientras que un contador de 6 bits (módulo 64) permite disminuir el paso entre dos niveles consecutivos al 1,5%.

Esta forma de suministrar potencia «a trozos» (*todo/nada*), en lugar de variar la tensión que se transmite a la carga, resulta sumamente adecuada en relación con los dispositivos y las etapas electrónicas de potencia:

- determinados componentes de potencia, como los tiristores y triacs no permiten otro tipo de actuación (solamente presentan dos estados que corresponden a las dos situaciones de conducción total o no conducción);
- incluso, para los transistores (que admiten tensiones variables de entrada y de salida) las situaciones *todo/nada* son muy favorables en relación con la disipación de potencia (cuando conducen «todo» su tensión es muy baja $V_{CE} \approx 0$ y cuando no conducen su intensidad es nula $I_C = 0$: en ambos casos la potencia disipada en el transistor, producto tensión por intensidad $V_{CE} \cdot I_C$, es muy reducida).

18.1.1. Conversión número-tiempo

Un control todo/nada realiza una conversión número $\rightarrow$ tiempo: el número que llega al comparador como referencia es transformado en anchura de pulso. El resultado es una modulación de la anchura de pulso en relación con el número que actúa como referencia: $n \rightarrow \Delta t$.



Si el número n que incide sobre el comparador es constante se producen pulsos de anchura fija (proporcional a dicho valor n). Al variar n se produce una variación de la anchura de los pulsos en su salida, siendo dicha anchura proporcional al valor de n ; de esta forma una señal expresada numéricamente es convertida en pulsos de duración proporcional a su valor.

Este esquema de modulación en anchura de pulsos (conversión de números en pulsos de anchura proporcional a ellos) puede ser utilizado en la reproducción de música digitalizada (discos compactos CD). Los valores numéricos de la señal digitalizada que contiene el CD son leídos con un período T , en sincronismo con el ciclo del conversor todo/nada, de forma que cada valor numérico controla la duración de un pulso.

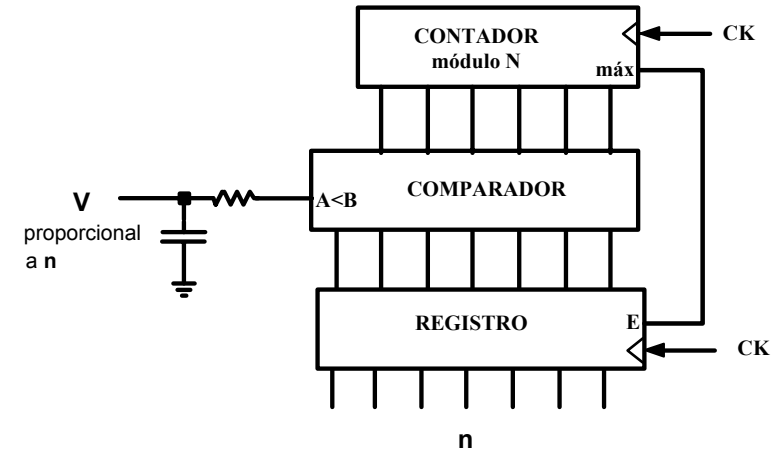
Si tales pulsos de amplitud modulada son enviados a un altavoz, a través de un amplificador de potencia, el propio altavoz filtrará la frecuencia del ciclo *on/off* y responderá a la anchura de los pulsos con la correspondiente señal de audio. En lugar de enviar la potencia al altavoz en forma de amplitud de tensión, se transmite en forma de duración de pulso; el resultado es el mismo, con la ventaja de que las pérdidas en potencia en el amplificador serán muy inferiores, pues actúa en situaciones todo/nada.

18.1.2. Conversión número-tensión

Si la salida del anterior conversor número-tiempo es filtrada mediante un filtro pasabaja (cuya frecuencia de corte sea inferior a la del ciclo todo/nada) el resultado es una tensión proporcional al número que actúa como referencia. De esta manera se dispone de un sencillo conversor digital-analógico cuyos componentes son bloques digitales (con excepción del filtro de la salida):

- si la referencia del modulador de anchura de pulso es fija (n fijo) se tiene un conversor DC-DC en que la tensión de salida V_{OH} (supuesto $V_{OL} = 0$ V) es convertida en una tensión de valor inferior, controlado por el número n que actúa como referencia;
- si n es variable se obtiene una onda que es el resultado de convertir a tensión la señal definida numéricamente.

Ésta es una de las formas más sencillas de efectuar la conversión de un valor digital en una tensión analógica (conversor D/A) y, además, utiliza procedimientos digitales (salvo el filtrado); existen conversores más rápidos y precisos (que son descritos en el apéndice A8), pero requieren la utilización de bloques analógicos (lo cual dificulta su integración en circuitos digitales). Este esquema permite generar digitalmente tensiones de referencia o formas de onda (señales de tensión variable).



Para que el rizado debido al ciclo *on/off* sea despreciable debe cumplirse que la frecuencia de corte del filtro sea muy inferior a la de dicho ciclo; por otra parte, también es necesario que la frecuencia de la señal a obtener sea muy inferior a la de corte del filtro, a fin de que dicha señal de salida no se vea afectada por el filtrado:

$$f_{\text{máx señal}} \ll f_{\text{corte filtro}} \ll f_{\text{ciclo on/off}}$$

$$1/f_{\text{máx señal}} \gg RC \gg T = N \cdot T_{CK}$$

El comportamiento del filtro y el valor de la tensión de salida sobre el condensador V_C , en condiciones de pequeño rizado, puede razonarse en la siguiente forma:

En un ciclo on/off la tensión del condensador se mantiene prácticamente constante ya que la frecuencia de la señal de salida es sumamente inferior a la frecuencia del ciclo (por un lado, la propia frecuencia de corte del filtro solamente permite el paso de frecuencias muy inferiores a la del ciclo on/off y, además, se ha impuesto la condición de que la señal de salida sea de frecuencia muy inferior a la de corte del filtro).

Para que dicha tensión se mantenga constante, el balance de carga sobre el condensador debe ser nulo, es decir, la cantidad de carga que recibe el condensador durante el intervalo de on ha de ser igual a la que el condensador cede durante el intervalo de off

$$\Delta Q_{on} \approx \Delta Q_{off} ; I_{on} \cdot t_{on} \approx I_{off} \cdot t_{off}$$

$$I_{on} = \frac{V_{oH} - V_C}{R} ; I_{off} = \frac{V_C}{R} \text{ (supuesto } V_{oL} = 0 \text{ V)}$$

$$t_{on} = \frac{n \cdot T}{N} ; t_{off} = T - t_{on}$$

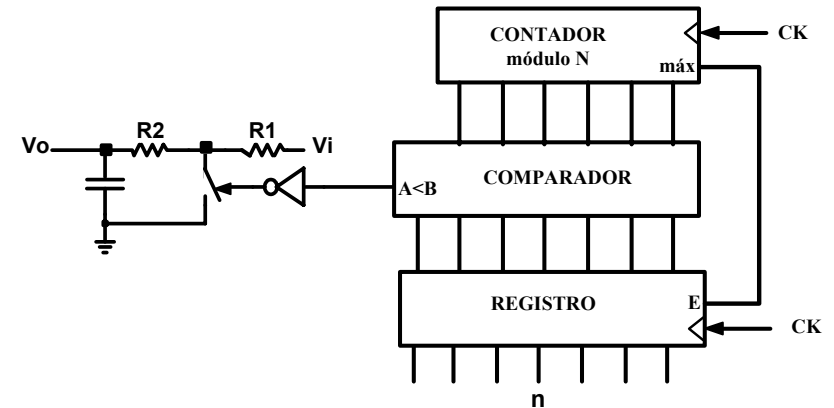
$$\frac{V_{oH} - V_C}{R} \cdot \frac{n \cdot T}{N} = \frac{V_C}{R} \cdot (T - \frac{n \cdot T}{N})$$

$$V_C = V_{oH} \cdot \frac{n}{N} = k \cdot n \text{ donde } k = \frac{V_{oH}}{N}$$

es decir, la tensión de salida es proporcional a la referencia n y la constante de proporcionalidad es $V_{oH} / N \approx V_{CC} / N$, que puede ajustarse adecuadamente (por ejemplo, con $V_{CC} = 5 \text{ V}$ y $N = 50$, a cada unidad de n le corresponden 0,1 V).

18.1.3. Potenciómetro digital

El mismo esquema conceptual empleado en el epígrafe anterior (PWM más filtrado pasa-baja) puede ser utilizado para controlar la amplitud de una señal analógica; su muestreo mediante un control todo/nada sobre un divisor de tensión, seguido de un filtro pasa-baja, en la forma representada en la figura siguiente, permite reducir la amplitud de dicha señal, controlándose el factor de proporcionalidad numéricamente mediante n . Esta reducción proporcional de la amplitud es precisamente lo que hace un potenciómetro cuando se utiliza como divisor de tensión.



Al igual que en el caso anterior, para que el rizado sea despreciable y la señal de salida no se vea afectada por el filtrado, debe verificarse que:

$$f_{\text{máx. señal}} \ll f_{\text{corte filtro}} \ll f_{\text{ciclo on/off}}$$

$$1/f_{\text{máx señal}} \gg RC \gg T = N \cdot T_{CK}$$

y el análisis del comportamiento del circuito es análogo al caso anterior (igualdad de carga en intervalos de on y off):

$$\Delta Q_{on} \approx \Delta Q_{off} ; I_{on} \cdot t_{on} \approx I_{off} \cdot t_{off}$$

$$I_{on} = \frac{V_i - V_C}{R1 + R2} ; I_{off} = \frac{V_C}{R2} ; t_{on} = \frac{n \cdot T}{N} ; t_{off} = T - t_{on}$$

$$\frac{V_i - V_C}{R1 + R2} \cdot \frac{n \cdot T}{N} = \frac{V_C}{R2} \cdot (T - \frac{n \cdot T}{N})$$

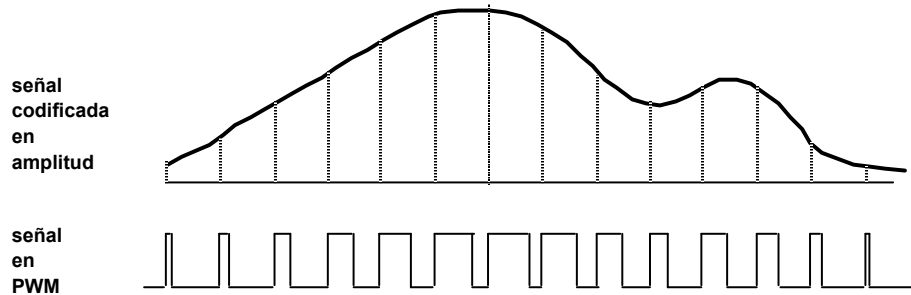
tomando las resistencias $R1 \ll R2$, puede aproximarse $R1 + R2 \approx R2$

$$V_C = V_i \cdot \frac{n}{N} = k \cdot V_i \text{ donde } k = \frac{n}{N}.$$

Si la tensión de entrada es continua resulta una tensión de salida continua y menor, según un factor $k < 1$, y si la entrada es una señal variable resulta una señal de salida de menor amplitud, proporcional a la entrada conforme a dicho factor $k = n/N < 1$.

18.2. Modulación en anchura de pulsos

La modulación de anchura de pulso **PWM** (*pulse width modulation*) es una forma de codificar la información que utiliza, para ello, pulsos de anchura variable; es decir, en lugar de una señal que expresa la información a través de su amplitud (variaciones de tensión), se utilizan pulsos de amplitud fija cuya anchura (la duración de los pulsos) es variable, proporcional al valor de la señal en cada momento.



El paso de señal en amplitud a pulsos modulados en anchura se consigue mediante una transformación $V-\Delta t$ tensión-anchura de pulso. Precisamente en el apartado anterior (*Control todo/nada*) se ha utilizado una transformación $n-\Delta t$, número-anchura de pulso, que efectúa la modulación **PWM** a partir del valor numérico de la señal.

El teorema de muestreo de Shannon garantiza que la información contenida en los pulsos de anchura modulada es la misma que transmite la amplitud de la señal si la frecuencia de los pulsos es superior al doble de la máxima frecuencia de dicha señal.

El control todo/nada es la forma más simple y directa de transformar un número en un pulso de anchura proporcional al mismo. Esta modulación *on/off* actúa cíclicamente con un período T dividido en dos intervalos: activo/inactivo ($1/0$), de forma que, al inicio de cada período T se produce un pulso cuyo «tiempo en 1», proporcional al número n , señala el intervalo activo.

Como se ha visto en el anterior apartado, la conversión número $\rightarrow$ anchura de pulso, propia de la modulación todo/nada, requiere simplemente un contador (que desarrolla el ciclo de N unidades) y un comparador (del contador con la referencia n) y resulta muy útil para control de potencia, conversión digital-analógica y control de amplitud de señales (potenciómetro digital).

Dentro de cada ciclo *on/off*, el «tiempo en 1» (*on*) se encuentra agrupado en un mismo pulso al comienzo del período; un procedimiento alternativo consiste en dividir el tiempo de *on* en pulsos disjuntos repartidos a lo largo de todo el período.

En lugar de diferenciar dos intervalos separados y sucesivos (activo/inactivo) es viable, también, producir pulsos (de duración igual a una unidad de tiempo de reloj) cuya suma de «tiempos en 1» sea igual al tiempo de *on* y que se distribuyan a lo largo del ciclo «homogéneamente»; a esta otra forma de producir pulsos de anchura modulada la denominaremos *modulación PWM en pulsos distribuidos*.

Ambas modulaciones son equivalentes y ofrecen las mismas aplicaciones; en algunos casos la segunda puede tener ventajas en relación con filtrados pasa-baja posteriores (el rizado en un filtro pasa-baja es menor si los pulsos se encuentran distribuidos homogéneamente). Las dos modulaciones corresponden a una conversión número $\rightarrow$ anchura de pulsos: entrada, el número n que es la referencia; salida, pulsos de anchura modulada.

Otra conversión de interés que también produce pulsos modulados en anchura, es la transformación tensión $\rightarrow$ anchura de pulso. Puede configurarse a través de la comparación de la tensión de entrada con una rampa de tensión creciente (apartado 18.4), dando lugar a pulsos cuya anchura (el tiempo que tarda la rampa en alcanzar la tensión de entrada) es proporcional a la misma: convertidores de tensión-tiempo de tipo rampa.

De esta forma, al igual que en el control todo/nada, en cada ciclo se produce un pulso inicial de duración controlada por la tensión de entrada, seguido por un intervalo inactivo. También existe la alternativa de pulsos distribuidos a lo largo del ciclo, que se consigue con convertidores sigma-delta (apartado 18.5).

Estos dos últimos tipos de convertidores (por rampa o sigma-delta) corresponden a una conversión tensión $\rightarrow$ anchura de pulsos: entrada, tensión analógica; salida, pulsos de anchura modulada. Si durante el «tiempo en 1» se habilita un contador que efectúa el conteo de unidades de tiempo (a partir de una señal de reloj apropiada) se consigue una conversión tensión $\rightarrow$ número, es decir, analógico-digital.

Así, pues, consideramos en este capítulo cuatro tipos de moduladores de anchura de pulsos que podemos clasificar en la forma siguiente:

- ➔ conversión número-anchura de pulso:
 - pulsos de frecuencia fija: modulación *on/off*,
 - pulsos «aleatorios»: modulación en pulsos distribuidos;
- ➔ conversión tensión-anchura de pulso:
 - pulsos de frecuencia fija: rampa de tensión,
 - pulsos «aleatorios»: convertidor sigma-delta.

Todos estos moduladores dividen el tiempo en intervalos activos e inactivos; en consecuencia, los cuatro permiten el control de potencia por el método todo/nada y el control de amplitud de señales (potenciometro digital), tal como están detallados en el apartado anterior; en los dos primeros la variable de control es un número n y en los otros dos el control lo realiza la tensión de entrada V_i .

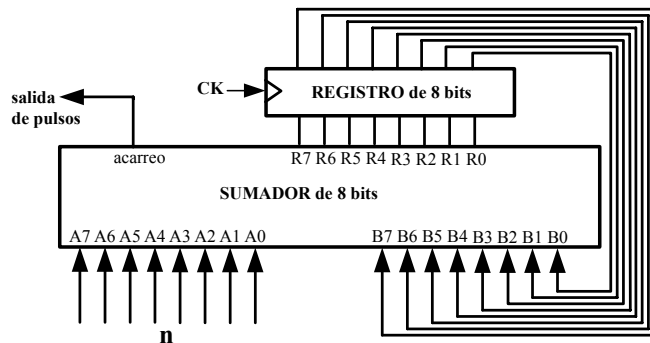
Además, los dos primeros (con un filtro pasa-baja a su salida) pueden ser utilizados como conversores digital-analógicos (D/A) y los otros dos (con un contador de unidades de tiempo habilitado por ellos) como conversores analógico-digitales (A/D).

Tales conversores entre el «mundo» analógico y el digital son relativamente lentos, pues hacen la conversión «a lo largo del tiempo», a través de la producción de pulsos de anchura controlada por su entrada (por un número en los primeros y por una tensión en los segundos), pero pueden alcanzar gran precisión y resultan útiles para muchas aplicaciones.

Como complemento a este capítulo, en el aspecto de conversión D/A y A/D, y para completar el estudio de los sistemas digitales con la interfase entre ellos y el mundo analógico, el apéndice A7 (*Conversión Digital-Analógica y Analógica-Digital*) describe otros tipos de conversores directos (más rápidos) digital-analógicos y analógico-digitales.

18.3. Modulación PWM en pulsos distribuidos

Una conversión número $\rightarrow$ anchura de pulsos con pulsos de salida distribuidos «homogéneamente» a lo largo del intervalo de conversión puede conseguirse mediante sumas repetitivas del número a convertir, según el circuito de la figura.



Este sumador (de números de p dígitos, cuyo resultado es un número de longitud p y un bit de acarreo) produce arrastre cuando el resultado de la suma alcanza o sobrepasa el número $N = 2^p$ (p es el número de dígitos del sumador que coincide con el número de biestables del registro que almacena el resultado de la suma).

Al realizar $N = 2^p$ sumas sucesivas del número de entrada n , el resultado total debería ser $N \cdot n = n \cdot N = n \cdot 2^p$; durante la realización de dichas N sumas el arrastre se debe activar n veces ya que n es la parte numérica resultante más allá de los p dígitos del sumador: n es la parte del resultado situada por encima de los p bits que aparecen en las salidas de resultado de la suma y que se almacenan en el registro. El resultado global $n \cdot 2^p$ indica que, por encima de los p bits, «se han tenido que llevar» (arrastre) n unidades puesto que la parte más significativa del resultado (a partir de los p dígitos inferiores) vale n y tal es el número de veces que ha tenido que activarse el arrastre para producirla.

Insistiendo en la misma explicación, consideremos un segundo sumador (suficientemente largo) situado a partir del anterior y que suma simplemente los arrastres del primero. Después de N sumas sucesivas del número n el sumador total debe contener el resultado $n \cdot N$ y, como $N = 2^p$ es el valor relativo del segundo sumador (pues se encuentra p dígitos por encima de las unidades), el contenido del mismo debe ser el número n . Para ello el arrastre del primer sumador ha tenido que activarse n veces, es decir, ha producido n pulsos de duración un período de reloj.

El intervalo de conversión es N unidades de tiempo de reloj y el resultado son n pulsos de duración igual a una unidad de tiempo, siendo n el número de entrada a este conversor. Tales pulsos de salida se encontrarán distribuidos «homogéneamente»: si n es pequeño estarán más espaciados entre sí pues hace falta un mayor número de sumas para alcanzar N y si n es grande (cercano a N) el espaciado de los pulsos será pequeño e, incluso, podrán encontrarse consecutivos (formar un mismo pulso de duración doble, ...).

Las aplicaciones de esta segunda forma de conversión número $\rightarrow$ anchura de pulsos son las mismas que se detallan en el apartado 18.1:

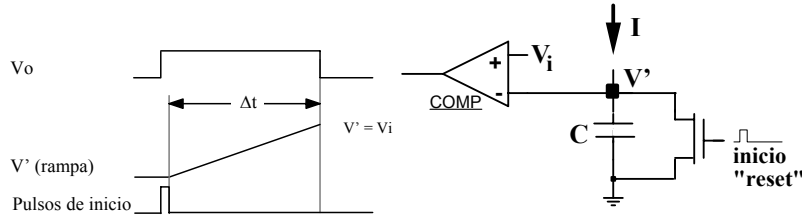
- control de potencia todo/nada
- conversión número-tensión (digital-analógica)
- control de amplitud de señal (potenciometro digital).

Comparando ambas formas de modular la anchura de pulsos, resulta que la modulación PWM en pulsos distribuidos aumenta el número de pulsos y, con ello, el número de conmutaciones, lo cual puede ser un inconveniente en cuanto a control de potencia (ya que aumenta el esfuerzo a realizar por los componentes de potencia debido al mayor número de conmutaciones de los mismos y, también, aumenta el consumo dinámico producido en tales conmutaciones).

En cambio, la distribución más «homogénea» de los pulsos de salida hace que la separación entre pulsos sea menor, lo cual es una ventaja en las aplicaciones que utilizan un filtrado de salida pasa-baja (pues se reduce el rizado del filtro al estar más próximos los pulsos).

18.4. Conversión tensión – tiempo de tipo rampa

La carga de un condensador, con una corriente constante, desde 0 V hasta el valor de la tensión de entrada V_i determina un intervalo de tiempo proporcional a dicha tensión:

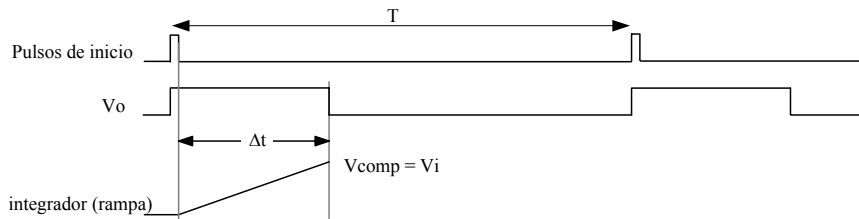


El condensador recibe una intensidad constante, de forma que la tensión del condensador variará en forma de rampa lineal de pendiente I/C ; cuando dicha rampa alcanza el valor V_i finaliza el pulso de salida cuya anchura será proporcional a V_i .

$$V_C(t) = \frac{I}{C} \cdot t ; \text{ para } V_C(t) = V_i \rightarrow t = \frac{C}{I} \cdot V_i = k \cdot V_i$$

El circuito requiere pulsos de inicialización, que descarguen el condensador, para comenzar cada pulso de salida. La carga del condensador a intensidad constante puede hacerse con un generador de intensidad (por ejemplo, la etapa típica de transistor bipolar en base común); si no se requiere una precisión muy alta, puede utilizarse una simple resistencia, aprovechando la parte inicial de la exponencial del proceso de carga RC (por ejemplo, en un circuito RC conectado a 12 V, si se acota el intervalo de carga del condensador de 0 a 5 V, el error de linealidad resultante es inferior al 5 %).

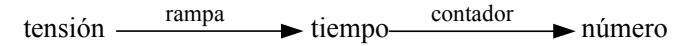
Si los pulsos de inicio (*reset*) tienen una frecuencia fija, con un período T , al comienzo de cada período se produce un pulso de salida cuya duración será proporcional a V_i : pulsos de anchura modulada, obtenidos por una conversión tensión-tiempo.



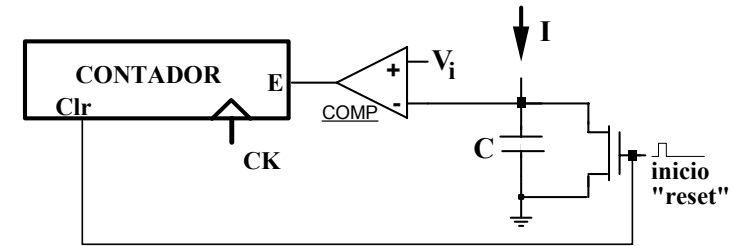
Este modulador de anchura de pulso se diferencia de los dos anteriores en que la variable de entrada (de control de la anchura de los pulsos) es una tensión (en lugar de un número); sirve para las mismas aplicaciones: control de potencia todo/nada y control de amplitud de señal (potenciómetro digital), realizándose ambos controles por tensión.

Además, la duración de cada pulso puede ser medida por un contador cuyo reloj se ajuste a una unidad de tiempo precisa, de forma que se efectúe una segunda conversión tiempo-número, dando como resultado un convertor analógico-digital.

A través de la rampa de tensión del condensador el circuito efectúa la conversión tensión-tiempo y el contador completa el proceso con una conversión tiempo-número:



La segunda conversión conforma una medida del intervalo de tiempo (duración de la rampa hasta V_i) que, a tenor de la primera conversión, es también una medida del valor de la tensión de entrada; se consigue así una conversión tensión-número que corresponde a un convertor analógico-digital.



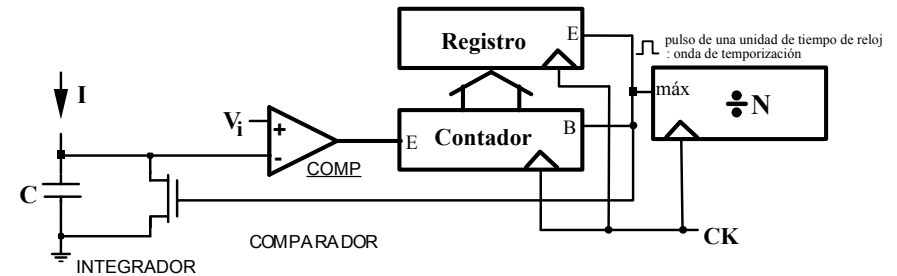
$$V_C = \frac{I}{C} \cdot t$$

sea $t' = n \cdot T_{CK}$ el tiempo que tarda la rampa en alcanzar el valor de V_i , donde n será el número alcanzado por el contador en dicho tiempo y T_{CK} el período del reloj:

$$V_i = \frac{I}{C} \cdot n \cdot T_{CK} ; \quad n = \frac{C}{I \cdot T_{CK}} \cdot V_i = k \cdot V_i ; \quad k = \frac{C}{I \cdot T_{CK}}$$

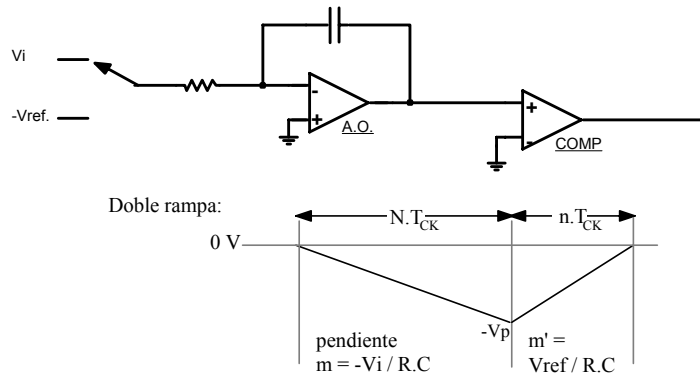
El resultado n , expresado en el contador, es proporcional a la tensión de entrada y la constante de proporcionalidad puede ser ajustada adecuadamente (por ejemplo, $I = 1 \text{ mA}$, $C = 100 \text{ nF}$ y $T_{CK} = 1 \mu\text{s}$ hacen que n exprese la medida de V_i en centésimas de voltio, 0,01 V).

El siguiente circuito representa un convertor tensión-número (analógico-digital) de funcionamiento continuo, que cada determinado tiempo $T = N \cdot T_{CK}$ efectúa una medida de la tensión de entrada:



El contador módulo N determina el tiempo en que comienza cada medida de la tensión de entrada; para ello produce los pulsos de inicialización, los cuales almacenan en el registro el resultado de la medida anterior y, a la vez, borran el contador (borrado sincrónico durante una sola unidad de tiempo). A partir del borrado del contador, el integrador genera una rampa de pendiente I/C que es comparada con la tensión de entrada hasta que ambas tensiones son iguales.

Este conversor requiere que los términos que intervienen en el factor de escala (I , C y T) sean de adecuada precisión, así como el amplificador operacional y el comparador. Es posible mejorar en gran medida la precisión de la conversión tensión-tiempo utilizando doble rampa: un integrador genera dos rampas, una de ellas descendente y la otra ascendente, integrando primero la tensión a medir V_i durante un tiempo fijo y posteriormente integrando una tensión de referencia negativa $-V_{ref}$, el comparador, en este caso, sirve para comparar la rampa ascendente con 0 V .



La tensión $-V_p$ alcanzada por la rampa descendente al cabo del tiempo fijo $N.T_{CK}$ de integración sobre V_i será:

$$\Delta V = V_p = \frac{V_i}{R.C} \cdot N.T_{CK}$$

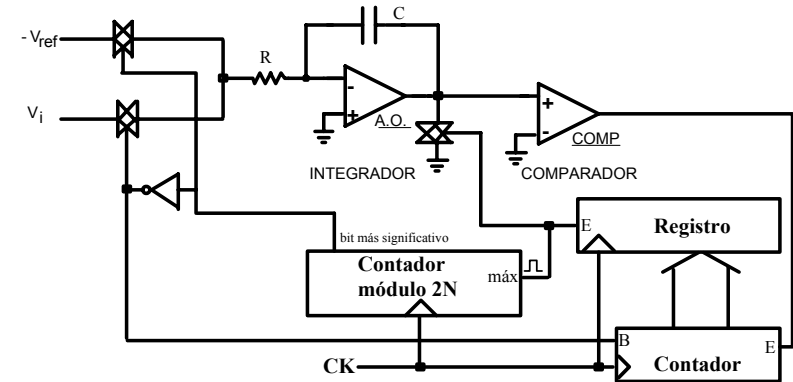
La rampa ascendente posterior, conformada por integración sobre $-V_{ref}$, tardará un tiempo $n.T_{CK}$ en alcanzar los 0 V :

$$\Delta V = V_p = \frac{V_{ref}}{R.C} \cdot n.T_{CK}$$

$$\frac{V_i}{R.C} \cdot N.T_{CK} = \frac{V_{ref}}{R.C} \cdot n.T_{CK} \quad n = \frac{N}{V_{ref}} \cdot V_i = k.V_i \quad \text{siendo } K = \frac{N}{V_{ref}}$$

La duración de la segunda rampa, expresada en número de unidades de tiempo de reloj n , es proporcional a la tensión a medir, con un factor de escala que no depende de R , ni de C ni de T_{CK} , sino solamente de la tensión de referencia V_{ref} ; tomando, por ejemplo, $V_{ref} = 10\text{ V}$ y $N = 10.000$, el número n expresará la medida de V_i en milivoltios.

Un posible esquema del circuito necesario para un conversor de doble rampa de funcionamiento continuo es el representado en la siguiente figura:



El tiempo del ciclo es $2N.T_{CK}$; durante la primera mitad de dicho intervalo el contador módulo $2N$ pone en conducción la puerta de transmisión superior, de forma que el integrador genera la rampa descendente de pendiente $-V_i/R.C$; durante todo este intervalo el contador que efectúa la medida de la tensión permanece borrado. En el otro semiperíodo conduce la puerta de transmisión inferior, que genera la rampa ascendente de pendiente $+V_{ref}/R.C$, hasta alcanzar la tensión de 0 V , tiempo durante el cual el segundo contador realiza el conteo hasta el valor n .

Al finalizar el intervalo de medida, se habilita el registro para almacenar el resultado de la misma (valor n) y se descarga el condensador; a partir de aquí se inicia un nuevo ciclo de conversión.

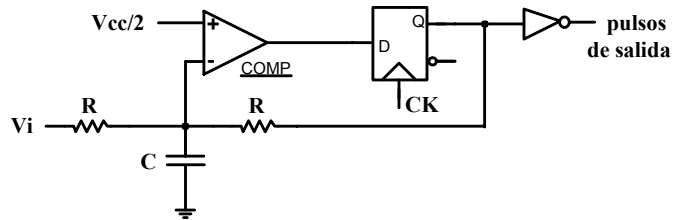
La utilización de dos rampas compensa la falta de precisión de R , C y T_{CK} y, en gran medida, los posibles errores introducidos por el amplificador operacional; basta asegurar una alta precisión en la tensión de referencia y que la frecuencia de reloj sea estable para obtener una medida digital n muy precisa.

Este tipo de conversión tensión-tiempo es relativamente lento en cuanto a realizar la conversión A/D pero proporciona una razonable precisión a bajo coste; por ello es muy utilizada cuando no se necesitan altas velocidades de conversión, por ejemplo, en los voltímetros digitales de bajo coste (el operador humano necesita varios segundos para leer el resultado de la medida, disponiéndose por tanto de intervalos de tiempo relativamente amplios para completar la conversión).

18.5. Conversores sigma – delta

El esquema de conversi3n tensi3n-tiempo-n3mero constituye, tambi3n, la base operativa de los denominados conversores sigma-delta ($\Sigma\Delta$), pero en ellos los pulsos se encuentran distribuidos «homog3neamente» a lo largo del intervalo de tiempo en que se efect3a la conversi3n.

La figura siguiente representa un conversor sigma-delta tensi3n-tiempo muy simple: el circuito RC de la entrada realiza una integraci3n de la se3al V_i y la realimentaci3n a trav3s del biestable conforma un camino de descarga que, bajo el control del comparador anal3gico, hace que la tensi3n del condensador se mantenga en el entorno de $V_{CC}/2$.



La realimentaci3n negativa, que se efect3a a trav3s del biestable, intenta mantener la tensi3n del condensador en el valor de referencia fijado en la otra entrada del comparador:

- el biestable se pone a **0** cuando la tensi3n del condensador es mayor que la de referencia y de esa forma proporciona un camino de descarga al condensador
- el biestable se pone a **1** cuando la tensi3n del condensador es menor que la de referencia y suministra carga al condensador.

A mayor tensi3n de entrada mayor tiempo tiene que encontrarse el biestable a **1** (salida a **0**), en situaci3n de descarga del condensador, para compensar la mayor carga que produce dicha tensi3n de entrada.

Habida cuenta que la salida del circuito est3 invertida respecto a la del biestable, el resultado es un conjunto de pulsos de salida cuyo «tiempo en **1**» es proporcional a la tensi3n de entrada: Tales pulsos se distribuir3n de manera «homog3nea» a lo largo del tiempo, pues se ajustan a la necesidad de carga/descarga del condensador para mantener su tensi3n en el valor establecido por la referencia.

Para que el rizado del condensador sea peque3o es necesario que la constante de tiempo del integrador RC sea mucho mayor que el per3odo del reloj que mueve al biestable T_{CK} : $RC \gg T_{CK}$ (o, lo que es lo mismo, que la frecuencia de corte del filtro pasa-baja RC sea muy superior a la frecuencia de actuaci3n del biestable).

Para una tensi3n de entrada V_i continua, la realimentaci3n negativa mantendr3 la tensi3n del condensador en el entorno de la tensi3n de referencia (comportamiento como etapa lineal); se verificar3 que

$$V_+ \approx V_- = \frac{V_{CC}}{2}$$

y para ello (para que la tensi3n del condensador se mantenga constante), el aporte de carga al mismo ha de ser igual a la cesi3n de carga:

$$\Delta Q_{carga} = \Delta Q_{descarga}$$

La carga del condensador proviene de la tensi3n de entrada V_i y del biestable en aquellos intervalos de tiempo en que se encuentra a **1** (salida del circuito a **0**), mientras que la descarga se debe solamente al biestable cuando se encuentra a **0** (salida a **1**).

Supuesto un tiempo de ciclo T , relativamente amplio respecto al per3odo del reloj $T = N \cdot T_{CK}$, y siendo t_{on} la suma de los intervalos de tiempo en los cuales la salida se encuentra a **1** (el biestable a **0**, $V_{OL} \approx 0$ V, produce descarga del condensador) y t_{off} la de los intervalos con salida a **0** (biestable a **1**, $V_{OH} \approx V_{CC}$, con efecto de carga):

$$\Delta Q_{carga} = \frac{V_i - \frac{V_{CC}}{2}}{R} \cdot T + \frac{V_{CC} - \frac{V_{CC}}{2}}{R} \cdot t_{off};$$

$$\Delta Q_{descarga} = \frac{\frac{V_{CC}}{2} - 0}{R} \cdot t_{on}$$

como $\Delta Q_{carga} = \Delta Q_{descarga}$:

$$(V_i - \frac{V_{CC}}{2}) \cdot T + (V_{CC} - \frac{V_{CC}}{2}) \cdot t_{off} = \frac{V_{CC}}{2} \cdot t_{on}$$

$$V_i \cdot T = \frac{V_{CC}}{2} \cdot T - V_{CC} \cdot t_{off} + \frac{V_{CC}}{2} \cdot t_{off} + \frac{V_{CC}}{2} \cdot t_{on}$$

dado que $t_{on} + t_{off} = T$:

$$V_i \cdot T = \frac{V_{CC}}{2} \cdot T - V_{CC} \cdot t_{off} + \frac{V_{CC}}{2} \cdot T = V_{CC} \cdot (T - t_{off}) = V_{CC} \cdot t_{on}$$

siendo $T = N \cdot T_{CK}$:

$$t_{on} = \frac{V_i \cdot T}{V_{CC}} = \frac{T}{V_{CC}} \cdot V_i = \frac{N \cdot T_{CK}}{V_{CC}} \cdot V_i = k \cdot V_i \quad \text{con } k = \frac{N \cdot T_{CK}}{V_{CC}}.$$

El circuito efect3a una conversi3n de la tensi3n de entrada V_i en tiempo de on t_{on} : cuanto mayor es la tensi3n de entrada, mayor es el tiempo del biestable a **0** necesario para mantener el condensador en el entorno de $V_{CC}/2$ (mayor tensi3n de entrada implica mayor tiempo de descarga). La relaci3n es directamente proporcional, como lo muestra el anterior balance de carga-descarga y la constante de proporcionalidad $N \cdot T_{CK}/V_{CC}$ puede ajustarse adecuadamente: para $N = 5000$, $T_{CK} = 1\mu s$ y $V_{CC} = 5$ V, a 1 voltio le corresponde 1 milisegundo.

Si la tensión de entrada es una señal (tensión variable), conformada por tensiones positivas ($V_i \geq 0V$), la conversión tensión-tiempo *de on* sigue siendo válida con la condición de que la frecuencia máxima de la señal sea muy inferior a la de corte del filtro que introduce el condensador; tal condición es necesaria a fin de que la realimentación sea adecuadamente rápida para «seguir la señal», es decir, para ajustarse a los cambios de la tensión de entrada siendo capaz de mantener la tensión del condensador en el entorno de la referencia $V_{CC}/2$; para ello ha de verificarse que

$$f_{\text{máx señal}} \ll f_{\text{corte filtro}} \\ 1/f_{\text{máx señal}} \gg RC \gg T_{CK}$$

La segunda desigualdad ($RC \gg T_{CK}$) es la indicada anteriormente para condiciones de bajo rizado.

Si durante el «tiempo en 1» de la salida se habilita el conteo de unidades de tiempo de reloj en un contador, al final del intervalo de medida, el número resultante en el contador n será proporcional a la tensión de entrada:

$$t_{on} = \frac{N \cdot T_{CK}}{V_{CC}} \cdot V_i ;$$

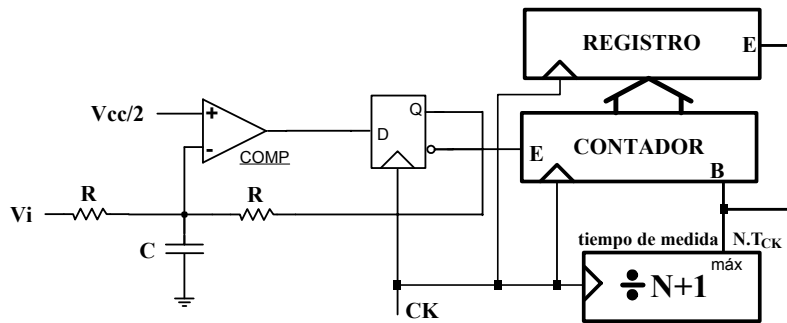
$$\text{si el contador alcanza el número } n \quad t_{on} = n \cdot T_{CK} ;$$

$$n = \frac{N}{V_{CC}} \cdot V_i = k \cdot V_i \quad \text{siendo } k = \frac{N}{V_{CC}} .$$

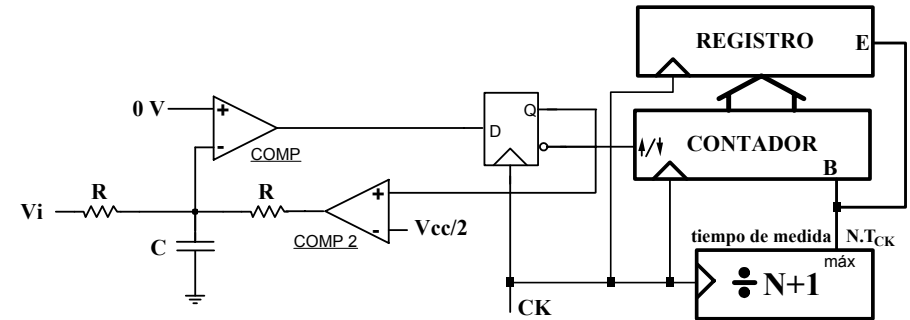
para $N = 5000$ y $V_{CC} = 5V$, n expresa la medida de la tensión V_i en milésimas de voltio.

De esta forma, al igual que en el caso del convertor por rampa, se efectúan dos conversiones seguidas: tensión-tiempo y tiempo-número:

tensión $\xrightarrow{\text{sigma-delta}}$ tiempo $\xrightarrow{\text{contador}}$ número
resultando un convertor analógico-digital:



Este convertor puede admitir, también, tensiones negativas; para ello hay que situar la tensión de comparación a 0 V y hacer que las tensiones que proporciona el bucle de realimentación a través del biestable sean simétricas: $V_{oH} = +V_{CC}$ y $V_{oL} = -V_{CC}$ (circuito de la figura siguiente, en el cual el segundo comparador suministra tensiones de salida $+V_{CC}$ y $-V_{CC}$ como resultado de la comparación).



En este caso y con la condición de bajo rizado $RC \gg T_{CK}$:

$$\Delta Q_{\text{carga}} = \frac{V_i}{R} \cdot T + \frac{V_{CC}}{R} \cdot t_{off}$$

$$\Delta Q_{\text{descarga}} = \frac{V_{CC}}{R} \cdot t_{on}$$

$$V_i \cdot T = V_{CC} \cdot (t_{on} - t_{off})$$

El contador ha de ser bidireccional y debe contar durante el *tiempo de on* (biestable a 0) y descontar en el *tiempo de off* (biestable a 1), de forma que

$$T = N \cdot T_{CK} \quad \text{tiempo del ciclo (N unidades de tiempo de reloj),}$$

$$t_{on} = n_1 \cdot T_{CK} \quad \text{tiempo de on durante el cual el contador cuenta en forma ascendente,}$$

$$t_{off} = n_2 \cdot T_{CK} \quad \text{tiempo de off durante el cual el contador descuenta,}$$

siendo $N = n_1 + n_2$ y $n = n_1 - n_2$ el resultado global del conteo en el contador:

$$V_i \cdot N = V_{CC} \cdot (n_1 - n_2) = V_{CC} \cdot n ;$$

$$n = \frac{N}{V_{CC}} \cdot V_i = k \cdot V_i ; \quad k = \frac{N}{V_{CC}} .$$

El resultado del conteo n se presentará en codificación en complemento a 2: cuando las tensiones de entrada son negativas, el *tiempo de off* será superior al *tiempo de on* y el contador presentará números negativos expresados en dicha codificación. Para evitar su desbordamiento (*over-flow*) el contador debe ser de módulo $\geq 2N$: debe poder llegar a contener el número N y el número $-N$ en complemento a 2. La tensión de entrada admisible en este convertor se encontrará en el intervalo $[-V_{CC} ; +V_{CC}]$.

19 MEMORIAS DE ACCESO DIRECTO

- 19.1. Biestables, registros, pilas y memorias
- 19.2. Configuración de los bloques de memoria de acceso directo
- 19.3. Arquitectura de buses: mapa de memoria
- 19.4. Direccionamiento de un bloque RAM en sectores separados
- 19.5. Memorias de sólo lectura

En el capítulo 11 se considera la memoria de los sistemas secuenciales desde dos puntos de vista complementarios: el estado del sistema y el almacenamiento de datos. Dentro de esta segunda perspectiva, la mayoría de los sistemas digitales (salvo sistemas muy simples o directos) necesitan «memoria» para conservar en ella información (datos y resultados intermedios) que será utilizada posteriormente.

Un biestable *D* es una unidad de memoria capaz de almacenar un bit, un registro de *n* biestables tiene capacidad para conservar una palabra de *n* bits, una pila es una fila de registros a los que se accede secuencialmente (uno tras otro) y se denomina memoria de acceso directo *RAM* a un bloque de *m* registros numerados, con capacidad para *m* palabras de *n* bits.

Los bloques integrados *RAM* permiten acceder (leer o escribir) a cada registro por medio de su número, a través de un conjunto de líneas de direccionamiento o bus de direcciones. La transferencia de información se lleva a cabo por unas líneas bidireccionales (permiten tanto leer como escribir sobre el registro seleccionado) que conforman el bus de datos; tales líneas presentan, además, la capacidad de desconexión, pasando a un estado de alta impedancia (tri-estado).

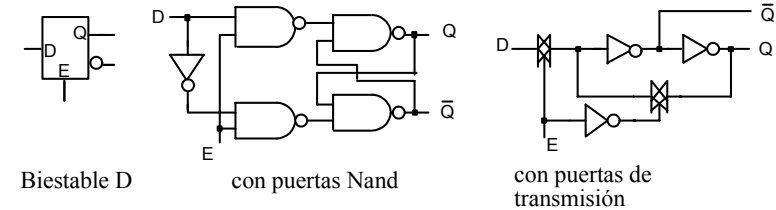
La memoria de un sistema digital puede incluir múltiples bloques *RAM*; la arquitectura de buses (datos y direcciones) permite organizar la memoria en forma sencilla, asignando a cada registro un número binario (con longitud igual al número de líneas del bus de direcciones). La correspondencia entre los registros y su numeración da lugar a un «mapa de memoria»; un bloque *RAM* ocupará un sector de dicho mapa y dos bloques diferentes deberán situarse en sectores diferentes.

El presente capítulo describe en detalle la configuración de los circuitos integrados de memoria de acceso directo y la forma de insertarlos en un mapa de memoria: cómo situar uno o varios bloques *RAM* en posiciones determinadas de un mapa de memoria e, incluso, cómo ubicar segmentos de un mismo bloque en sectores separados del mapa de memoria.

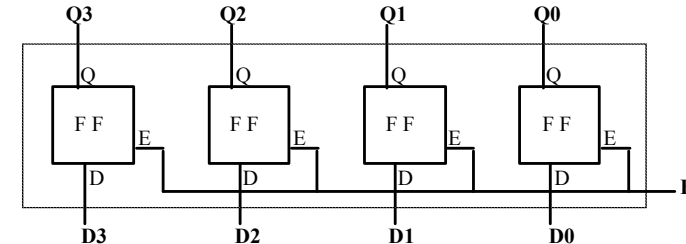
Por otra parte, los codificadores *ROM* (estudiados en el capítulo 4) pueden ser considerados como conjuntos de palabras binarias numeradas, seleccionables por su vector de entrada o «dirección»; en tal sentido, pueden ser utilizados como «memorias de sólo lectura». De forma que, además de los bloques *RAM* que solamente pueden suministrar datos después de que el sistema los haya escrito en ellos, pueden existir bloques *ROM* que corresponden a «conjuntos de registros de información fija» (programada previamente sobre ellos).

19.1. Biestables, registros y memorias

Una célula o unidad de memoria digital es un dispositivo capaz de almacenar y conservar un bit de información, es decir, un **0** o un **1** booleanos. La unidad de memoria básica es el biestable y, en concreto, el biestable tipo **D**: la entrada de habilitación determina el momento de aceptación de un nuevo bit, que será conservado hasta una nueva habilitación

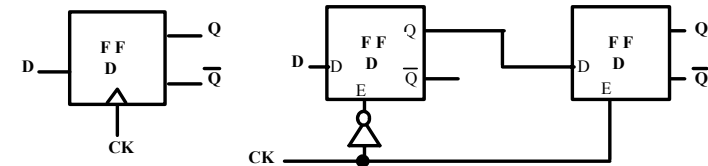


Un conjunto de *n* biestables *D* con habilitador común conforma un registro con capacidad de una palabra binaria de *n* bits: registro de retención (*latch memory*).



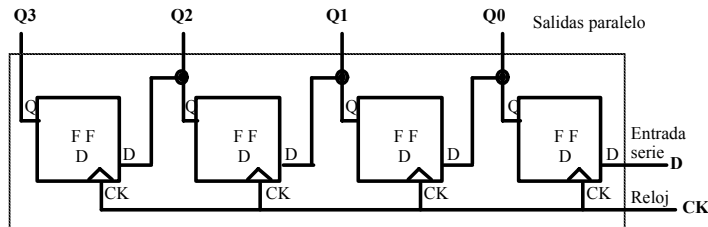
Este tipo de registros, habilitados por nivel (entrada de habilitación $E = 1$), constituyen el elemento básico de almacenamiento masivo de información digital, ya que generalmente son de este tipo los registros internos de las memorias de acceso directo (bloques integrados *RAM*).

En los biestables síncronos la habilitación se realiza por flancos ($CK = \uparrow$) en lugar de por niveles; cada biestable síncrono se construye por asociación de dos biestables habilitados por nivel, en configuración «amo-esclavo» (*master-slave*) de forma que la habilitación combinada del primer biestable con **0** y del segundo con **1** produce una habilitación global con flancos de subida (paso de la señal de reloj de **0** a **1**).

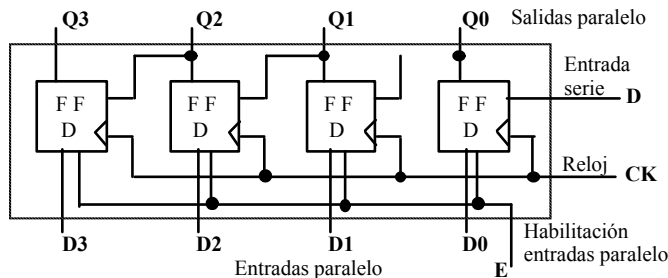


Con biestables síncronos, además de registros de retención síncronos, pueden conformarse otros dos tipos de registros de particular interés: los registros de desplazamiento y los contadores. [Los contadores, su configuración y sus aplicaciones, han sido descritos en detalle en los tres capítulos anteriores (16, 17 y 18).]

Los registros de desplazamiento (*shift register*, apartado 13.2) resultan de conectar un conjunto de biestables **D** síncronos en serie, de forma que la información que reciben avanza un biestable con cada pulso de reloj; de esta manera pueden recibir una palabra binaria, bit a bit, a través de su entrada y presentarla completa en las salidas de los biestables: conversión serie-paralelo.



Añadiendo a un registro de desplazamiento la posibilidad de carga paralelo (propia del registro de retención) se dispone, también, de la conversión paralelo-serie: una palabra recibida a través de las entradas paralelo puede ser transmitida, bit a bit, a través de la salida del último biestable.

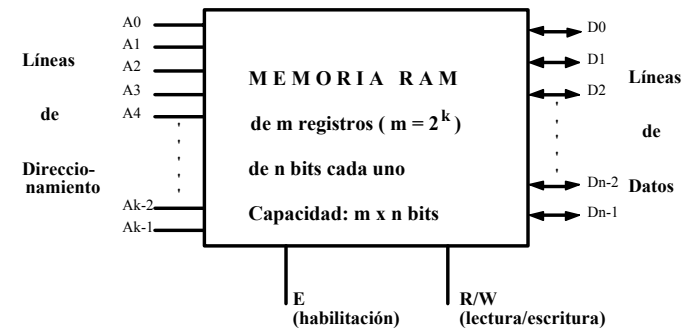


Además, los registros de desplazamiento resultan útiles para los algoritmos de multiplicación y de división, ya que el desplazamiento de un número un lugar hacia la izquierda equivale a multiplicar dicho número por 2. [Asimismo los registros de desplazamiento permiten la configuración de *pilas*, «memorias de acceso secuencial», que serán consideradas al final de este mismo apartado.]

En sistemas digitales reducidos bastan unos pocos registros junto con algunos contadores y algunos biestables de estado para configurar la memoria global del sistema. Ahora bien, en sistemas complejos suele requerirse un alto número de registros para memorizar el conjunto de datos y resultados (e instrucciones en el caso de sistemas que actúan bajo programa).

Resulta muy útil disponer, en un mismo bloque digital, de un amplio número de registros, capaces, cada uno de ellos, de memorizar una palabra binaria de **n** dígitos; los terminales de entrada y salida a estos registros han de ser comunes para todos ellos y unas entradas adicionales de direccionamiento indicarán en cada momento a cuál de los registros interesa acceder.

Esta agrupación de **m** registros de **n** bits, seleccionables por **k** entradas de direccionamiento ($m = 2^k$), recibe el nombre de memoria de acceso directo (*random access memory*): **RAM**.



La denominación de memoria de acceso directo («acceso aleatorio») indica que, en cualquier momento, puede leerse o escribirse directamente sobre cualesquiera de sus registros. El calificativo de aleatorio se utilizó por contraposición a las memorias de acceso secuencial, en las cuales para acceder a un dato es preciso desplazar previamente todos los anteriores; un ejemplo característico de memorias de acceso secuencial son las de cinta magnética.

En un bloque RAM el vector presente en las entradas de direccionamiento selecciona el registro sobre el que se lee o se escribe y los terminales de datos actúan como entradas y como salidas para todos los registros, de forma que el tipo de acceso (lectura del registro o escritura del mismo) ha de ser controlado por una línea adicional **R/W** (*lectura/escritura*).

Una memoria de acceso directo tendrá **k** líneas de direccionamiento **A_i**, que actúan como entradas, **n** líneas de datos **D_i**, bidireccionales, una entrada de selección de la operación a realizar **R/W**, que distingue entre las dos operaciones siguientes:

- **R/W = 1** operación de lectura del registro seleccionado por **A_i**
- **R/W = 0** operación de escritura sobre el registro seleccionado por **A_i**

y una o varias entradas de habilitación **CE**, que permiten (**CE = 1**) o inhiben (**CE = 0**) el funcionamiento global de la memoria.

La transferencia de datos de un bloque de memoria de acceso directo **RAM** presenta dos nuevas posibilidades, no contempladas hasta ahora en los bloques digitales: *la bidireccionalidad* (los terminales de datos son bidireccionales, es decir, actúan en unos momentos como entradas y en otros como salidas) y *la desconexión* (los terminales de datos pueden situarse en estado de alta impedancia).

En las funciones y bloques digitales considerados anteriormente cada terminal era claramente unidireccional, entrada o salida, y los terminales se conectaban a través de líneas unidireccionales en las que los valores booleanos se comunicaban desde una salida a una o a varias entradas. Las líneas de datos de las memorias de acceso directo son bidireccionales, ya que tales bloques utilizan los mismos terminales cuando reciben la información a memorizar (escritura) y cuando la envían (lectura).

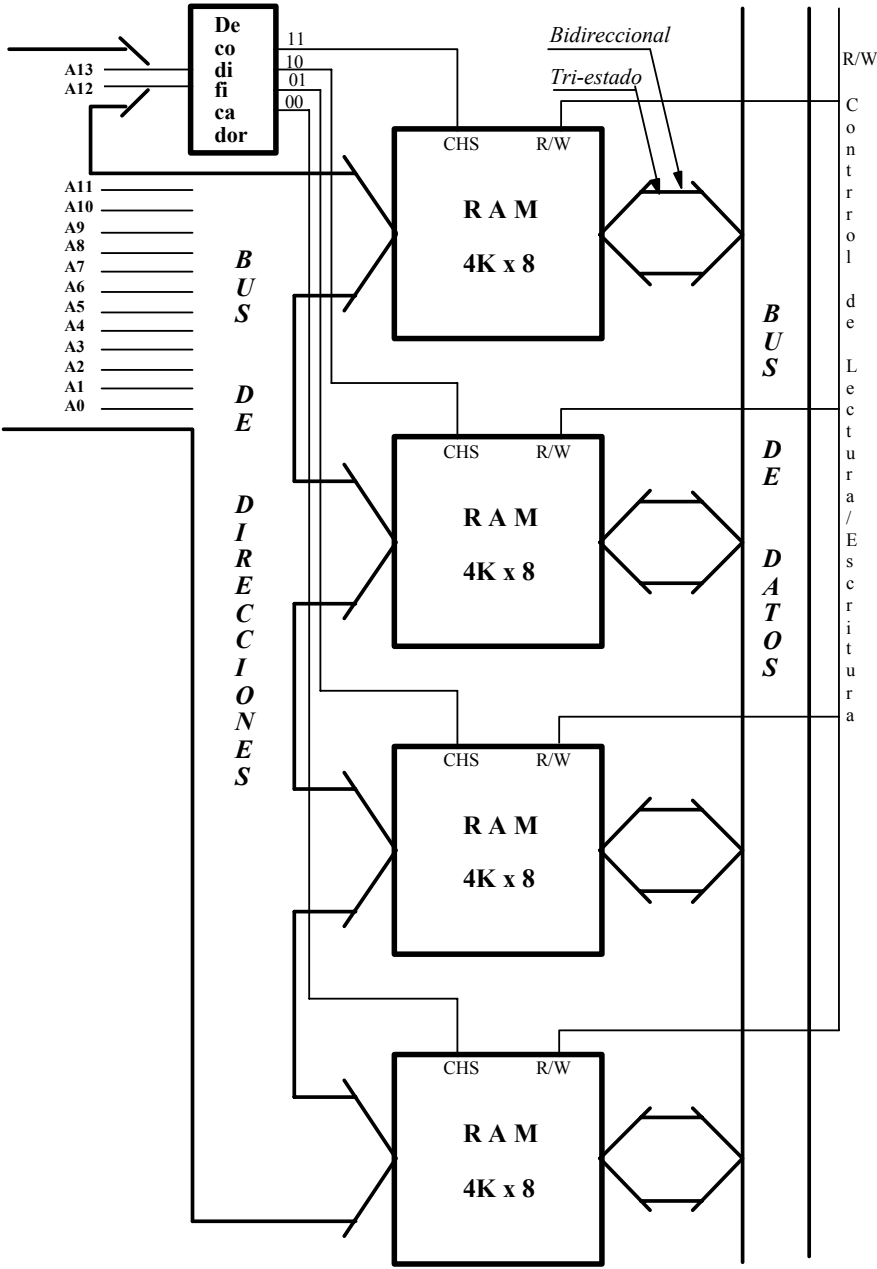
Además, se dota a los terminales de datos de una tercera posibilidad en la forma de un *tercer estado*, la *desconexión*: cuando **CE** = 0 las líneas de datos de dicho bloque no actúan ni como entradas ni como salidas, sino como si el bloque no estuviese conectado a ellas. Esta capacidad de desconexión, estado de *alta impedancia*, permite la conexión de varios bloques **RAM** sobre las mismas líneas de datos, actuando en cada momento uno de ellos y estando inhibidos los demás.

La figura de la página siguiente representa la conexión de 4 bloques **RAM** de 4K registros de 8 bits, formando una memoria de 16K registros; se utilizan 12 líneas de direccionamiento para seleccionar el registro interno de entre los 4K que posee cada uno de los integrados y 2 líneas más de direccionamiento, convenientemente decodificadas en sus cuatro posibilidades, para habilitar uno de los cuatro bloques.

La *bidireccionalidad* y la desconexión o *triestado*, permiten introducir el concepto de *bus de datos* como conjunto de líneas que enlazan a diversos bloques digitales y por las que pueden transmitirse los datos en ambas direcciones. El bus de datos configura una forma sencilla de comunicación entre la parte que procesa la información (procesador) y la parte que la almacena (memoria).

Análogamente, las líneas de direccionamiento (que seleccionan un registro de entre los muchos que forman cada bloque) son compartidas por los diversos elementos de memoria y reciben el nombre de *bus de direcciones*; dicho bus es unidireccional (comunica hacia la memoria el número del registro).

Bus de datos y *bus de direcciones* facilitan en gran medida la organización estructural de aquellos sistemas que requieren amplia capacidad de memoria; se necesitan, además, unas pocas líneas de control que determinen el sentido de la operación a realizar (lectura/escritura) y el momento de su ejecución (sincronización).

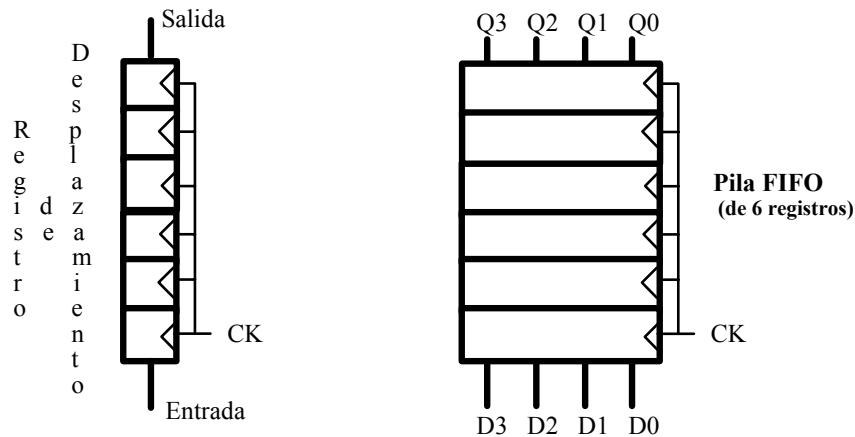


Memorias de acceso secuencial

Consideraremos brevemente otra forma de agrupar conjuntos de registros consistente en «apilarlos» (cada uno «encima» del anterior) formando una columna de ellos; el acceso a los registros será secuencial: habrá que leer y escribir en ellos según el orden de la columna.

La agrupación de varios registros «en vertical», de manera que reciban la información por las entradas del primero de ellos y la devuelvan por las salidas del último registro, da lugar a una *pila*; el conjunto equivale a una «memoria de desplazamiento», capaz de almacenar secuencialmente varias palabras binarias y devolverlas en el mismo orden en que las ha recibido: **pila FIFO** (*first in, first out*), la primera palabra en entrar será también la primera en salir.

El desplazamiento se produce a través de los sucesivos registros; en tal sentido, una pila FIFO de n registros de m bits puede construirse con m registros de desplazamiento de n bits cada uno de ellos, orientados «verticalmente» y colocados unos al lado de otros:

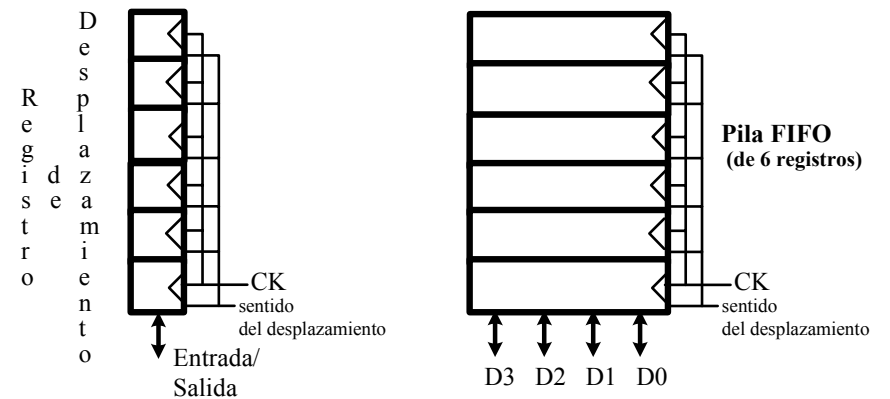


Una memoria RAM puede ser utilizada como pila FIFO si se le añaden dos contadores que realicen su direccionamiento en la siguiente forma:

- los contadores deberán ser de módulo igual a la capacidad de la memoria (para recorrer exactamente las direcciones de la misma);
- uno de ellos contendrá la dirección de escritura y deberá incrementarse (pasar a la dirección siguiente) cuando se escribe en la pila;
- el otro contador tendrá la dirección de lectura y se incrementará también cuando se produzca una operación de lectura en la misma;
- cuando ambos contadores señalen la misma dirección la pila se encontrará vacía y, en cambio, cuando el contador de escritura alcance la dirección anterior a la de lectura la pila estará llena (y no deben escribirse nuevos datos en ella).

Las pilas FIFO se utilizan para almacenar temporalmente palabras binarias (es decir, informaciones sucesivas) que serán utilizadas posteriormente en el mismo orden con que se han recibido o producido; una aplicación típica es la comunicación entre sistemas de diferente velocidad (por ejemplo entre un computador y una impresora), de forma que el emisor deposita a su propia velocidad un fichero sobre la pila y el receptor lo recoge a una velocidad distinta.

Otro tipo diferente es la **pila LIFO** (*last in, first out*) en la que la última palabra en entrar es la primera en salir; tendrá los mismos terminales para almacenar una palabra y para extraerla de la pila, de forma que las palabras se leen en orden inverso al que se han escrito en ella. Puede construirse con registros de desplazamiento bidireccionales, conformando un bus, también bidireccional, con las entradas y salidas de los primeros bistables de dichos registros.



Los procesadores de los sistemas que actúan bajo programa (computadores, microprocesadores, etc. ...) utilizan las pilas LIFO para el servicio de subrutinas, interrupciones y otros tipos de saltos que se realizan con «intención de volver»; la información relativa a la vuelta al punto desde el que se efectúa el salto ha de ser almacenada de forma que, si se producen varios saltos sucesivos, la información correspondiente al último de ellos será la primera en ser recuperada.

También es posible utilizar una memoria RAM como pila FIFO; bastará añadirle un contador de direccionamiento (de módulo igual a la capacidad de la memoria), que indique la posición de memoria en la que se debe escribir (la primera posición que se encuentra vacía). La escritura debe efectuarse sobre dicha posición de memoria y el contador debe incrementarse (señalando la siguiente), mientras que la lectura debe hacerse decrementando previamente el contador y leyendo de la posición de memoria resultante. La pila LIFO se encontrará vacía cuando el contador indique la primera posición de la memoria RAM y estará llena cuando el contador contenga la última posición de la misma.

19.2. Configuración de los bloques de memoria de acceso directo

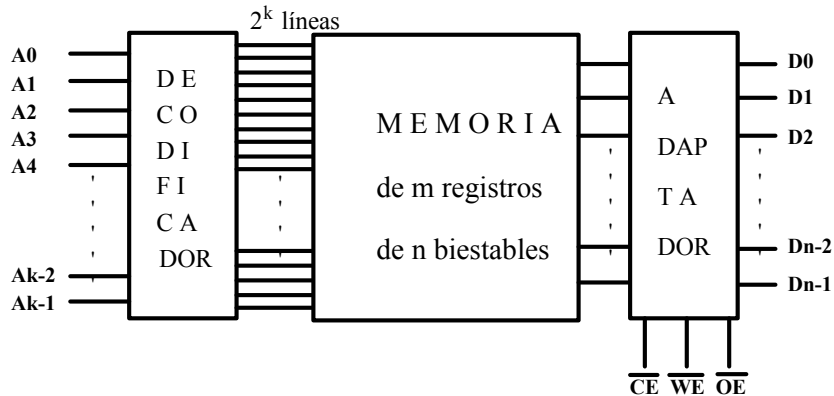
La mayoría de las memorias de acceso directo integradas presentan tres terminales de control:

- $\overline{CE}$ que habilita el circuito integrado ($\overline{CE} = 0$), de forma que si $\overline{CE} = 1$ es como si dicho circuito no se encontrara presente, es decir, todas sus líneas de datos se encuentran en alta impedancia y las diversas partes que conforman el integrado adoptan un estado de consumo mínimo (*standby*)
- $\overline{WE}$ que habilita la operación de escritura ($\overline{WE} = 0$), posicionando las líneas de datos como entradas y activando el correspondiente circuito de escritura
- $\overline{OE}$ que habilita las líneas de datos como salidas ($\overline{OE} = 0$), permitiendo la ejecución de una operación de lectura.

De acuerdo con ello, el circuito integrado **RAM** se puede encontrar en una de las siguientes cuatro situaciones:

$\overline{CE} = 1$	desconexión y consumo mínimo
$\overline{CE} = 0$ y $\overline{WE} = 0$	escritura; líneas de datos como entradas
$\overline{CE} = 0$, $\overline{WE} = 1$ y $\overline{OE} = 0$	lectura; líneas de datos como salidas
$\overline{CE} = 0$, $\overline{WE} = 1$ y $\overline{OE} = 1$	conectado; las líneas de datos en alta impedancia.

La estructura interna de una memoria de acceso directo es, en términos conceptuales, la representada en la siguiente figura:



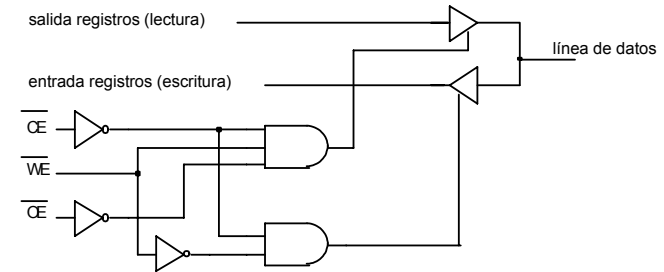
El módulo central de la memoria contiene $m = 2^k$ registros, constituido cada uno de ellos por n biestables. El decodificador es un bloque combinacional típico que selecciona numéricamente una de entre m líneas, cada una de las cuales va a corresponder a uno de los registros.

Un circuito adaptador determina la dirección del flujo de información del registro direccionado:

- lectura del contenido del mismo, operando las líneas de datos como salidas
- escritura de un nuevo dato sobre sus biestables, actuando las líneas de datos como entradas.

Obviamente, la lectura de un registro de un bloque **RAM** es una operación no destructiva: el registro no se borra ni se modifica, sino que sigue conservando la palabra binaria que se ha leído; solamente la escritura de una nueva palabra en tal registro modifica su contenido.

El circuito adaptador realiza el control de las operaciones de la memoria (lectura/escritura/desconexión), a través de adaptadores triestado cuya entrada de habilitación determina su situación de conexión o desconexión (estado de alta impedancia); la bidireccionalidad se consigue mediante la utilización de dos adaptadores triestado, que permiten actuar como salidas de los registros (lectura) o como entradas de los mismos (escritura).



Debido al gran número de registros presentes, resulta necesario minimizar el área de integración y la forma de selección de los mismos, para lo cual se adoptan configuraciones muy simples.

Basta con un solo decodificador para seleccionar el registro sobre el que se ejecuta la correspondiente operación (lectura/escritura); las k líneas de direccionamiento serán decodificadas en sus 2^k posibilidades, cada una de las cuales sirve para habilitar uno de los registros de la memoria. Ahora bien, el tamaño del decodificador y el número de líneas de selección de registros puede reducirse mucho dividiendo la decodificación en dos partes (fila y columna), de forma que cada registro corresponda a dos líneas de selección (su fila y su columna).

De manera que un bloque **RAM** de 1 Mega, en lugar de decodificar de una vez sus 20 líneas de direccionamiento sobre sus 1.048.576 registros, divide dichas líneas en dos grupos de 10, con dos decodificadores mucho más pequeños, cuyas líneas de salida se reducen a 2×1.024 (un número quinientas veces inferior al anterior). El decodificador de 20 líneas de entrada hubiera requerido 10^6 puertas con, al menos, 20×10^6 transistores, mientras que dos decodificadores de 10 líneas de entrada requieren del orden de 2×10^3 puertas con unos 20×10^3 transistores).

Por ello, para reducir el número de líneas necesarias para la selección de registro y el tamaño de los decodificadores, la distribución de los biestables de los registros adopta una estructura de tipo «bidimensional», conformando una matriz en la que cada registro queda identificado por la fila y la columna que ocupa; las líneas de direccionamiento se agrupan en dos subconjuntos, uno de los cuales indica la fila y el otro la columna a la que pertenece el registro seleccionado. [Véase la correspondiente figura en la página siguiente.]

Los biestables de cada registro se localizan en «hojas» sucesivas; cada una de ellas contiene un biestable de cada uno de los registros y se encuentra recorrida horizontalmente por las líneas de selección de fila y verticalmente por líneas que enlazan cada una de las columnas; cada hoja se corresponde con una de las líneas del bus de datos y cuenta con un amplificador de lectura/escritura.

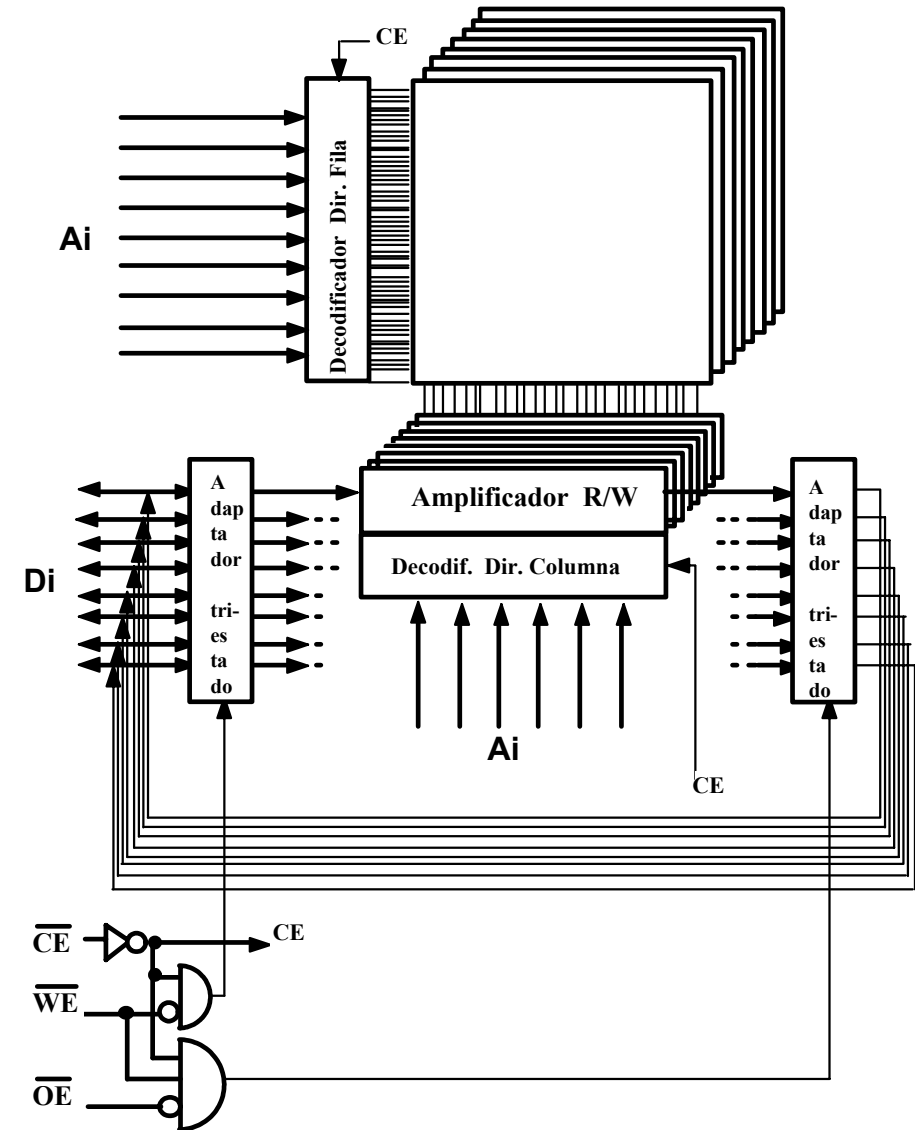
Cada columna cuenta con dos líneas una para Q y otra para $\bar{Q}$, conectadas ambas a los correspondientes terminales de los biestables; al seleccionar una de las filas, mediante la activación de la correspondiente línea de selección de fila, los biestables que se encuentran en ella quedan unidos a las «líneas de columna» que les corresponden. [Véanse en relación con este párrafo y siguientes las figuras de la página 189.]

Las «líneas de columna» establecen la comunicación de los biestables con el amplificador de lectura/escritura; un amplificador para cada una de las hojas, es decir, para cada uno de los bits de los registros. Las líneas de selección de columna determinan que, en cada momento, solamente una de las columnas se encuentre unida a dicho amplificador de lectura/escritura.

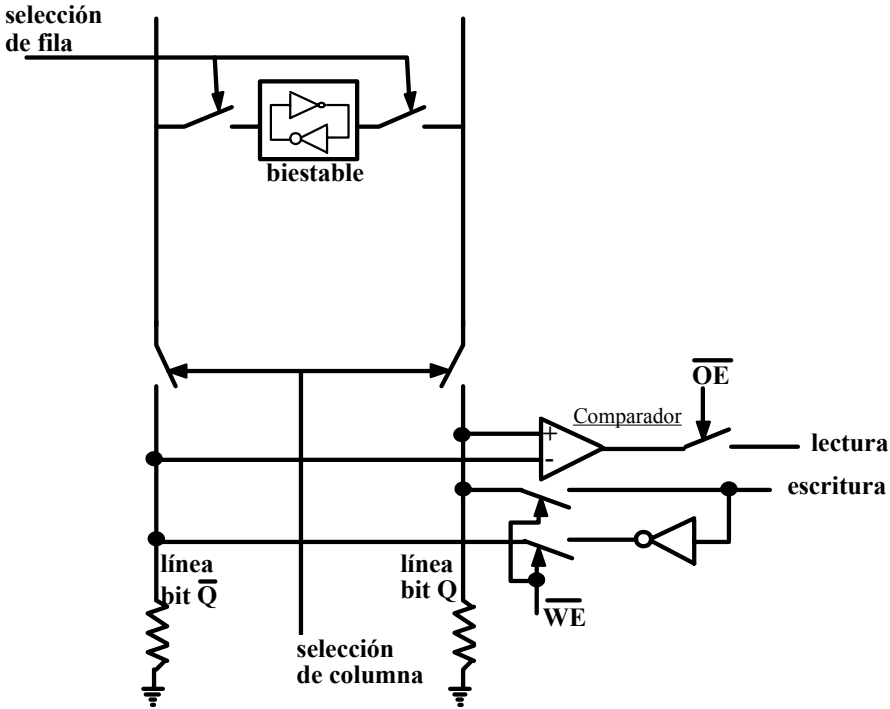
Las operaciones de lectura y de escritura son realizadas por el citado amplificador de lectura/escritura actuando sobre las «líneas de columna», esto es, comparando las tensiones existentes en ambas (lectura) o imponiéndoles el correspondiente valor booleano (escritura):

- una operación de escritura se ejecuta estableciendo en la línea Q de la columna seleccionada el valor booleano a almacenar en el correspondiente biestable y en la línea $\bar{Q}$ el valor booleano inverso
- una operación de lectura se realiza por comparación entre las tensiones de ambas «líneas de columna» para discriminar cuál de ellas se encuentra a **1** (a mayor tensión).

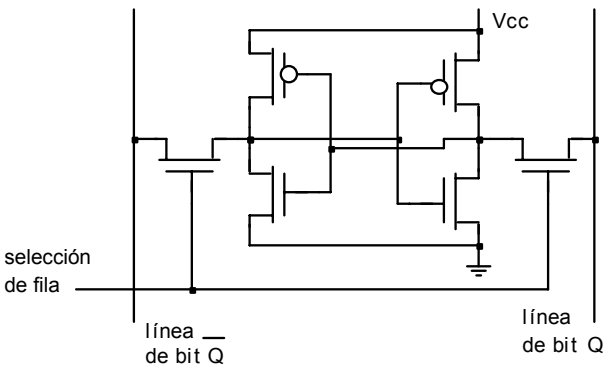
La configuración electrónica de cada biestable ha de ser también muy simple para reducir su área de integración: basta un par de inversores en lazo cerrado, la salida de cada uno de ellos conectada a la entrada del otro.



Estructura de una memoria de acceso directo (diagrama de bloques)



Conexión y forma de selección de los biestables en una memoria de acceso directo



Configuración y conexiones de uno de los biestables en tecnología CMOS

19.3. Arquitectura de buses: mapa de memoria

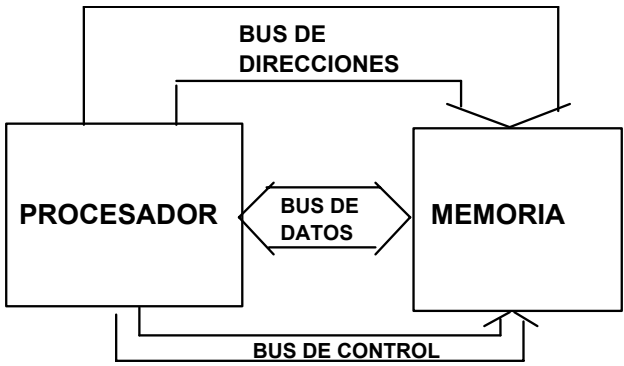
Un bus es un conjunto de líneas de comunicación entre varios subsistemas o bloques digitales, en concreto, entre varios circuitos integrados, entre varias placas de circuitos en los sistemas amplios o, incluso, entre sistemas digitales diferenciados.

La utilización conjunta del bus de datos y del bus de direcciones ha permitido organizar la memoria en forma muy simple: todos los registros se numeran correlativamente accediendo a ellos (lectura o escritura) por las mismas líneas de datos mientras que las líneas de direccionamiento seleccionan el registro sobre el que se opera. Las líneas de datos son bidireccionales (lectura/escritura), mientras que las de direccionamiento son unidireccionales (se dirigen siempre hacia la memoria).

A cada registro se le hace corresponder circuitalmente un número binario diferenciado; a cada número o dirección le corresponde un sólo registro o ninguno (posición de memoria vacía): dicho número de selección (dirección) de registro es recibido por la memoria, a través del bus de direcciones A_i . La correspondencia entre cada registro o conjunto de registros y la dirección o direcciones que ocupan configura el mapa de memoria del sistema.

Los buses determinan una división estructural del sistema digital en dos partes: el procesador que efectúa el procesamiento de la información (unidad operativa y de control) y la memoria que almacena la información. Procesador y memoria se comunican a través de tres buses:

- el bus de datos, por el que viaja la información en forma de palabras digitales
- el bus de direcciones, que selecciona el registro sobre el que se opera
- el bus de control, que determina la dirección de transferencia de la información y sincroniza dicha transferencia.



El bus de datos es bidireccional; puede recibir la información del procesador y comunicarla hacia la memoria o, al revés, actuar la memoria como salida de la información y el procesador como entrada. El número de líneas de datos determina la longitud de palabra y de los registros de la memoria.

Los buses de direcciones y de control están constituidos por líneas que salen del procesador y llegan a los múltiples bloques que constituyen la memoria. El número de líneas de direcciones determina la capacidad máxima de memoria que puede manejar directamente el procesador: con 10 líneas se pueden seleccionar solamente 1.024 registros (1K); 16 líneas alcanzan a discriminar entre 64K registros (65.536); 20 líneas de direccionamiento permiten manejar 1 Mega (1.048.576) y 32 líneas suponen la posibilidad de 4×10^9 registros (4.000 Megas).

El bus de control incluirá al menos tres líneas referidas, respectivamente, a la selección de la operación a efectuar **R/W** (lectura/escritura), a la validación de la dirección presente en el bus de direcciones **DIRV** (dirección válida) y a la validación del dato enviado por el procesador al bus de datos en las operaciones de escritura **DATV** (dato válido). **DIRV** evita que se efectúen operaciones sobre la memoria en momentos en que no hay una dirección válida en el bus de direcciones y **DATV** impide que se produzcan operaciones de escritura cuando no se dispone del dato correcto en el bus de datos.

Entendemos por mapa de memoria la distribución de las posiciones de memoria posibles entre los registros que la integran, efectuando una asignación numérica a cada registro. Una dirección o posición de memoria viene caracterizada por el correspondiente número binario de **m** bits, tantos como líneas tiene el bus de direcciones encargado de seleccionarla; para evitar la complejidad de manejo de largos números binarios, suele utilizarse el sistema hexadecimal para numerar las direcciones.

De esta forma, si el bus de direcciones es de 16 líneas, cada posición de memoria estará numerada por 16 dígitos binarios, pero bastan 4 cifras hexadecimales para expresar tal numeración; la capacidad total de memoria direccionable con dicho bus irá de la posición **0000H** a la **FFFFH** (65.535₁₀)

En los siguientes ejemplos se considera que el bus de direcciones es de 16 líneas **A15 - A0** y se utilizan las siguientes líneas de control:

- selección de lectura/escritura **R/W**
- validación de dirección **DIRV**
- y validación del dato enviado por el procesador para su escritura **DATV** (se supone que cuando se indica que el dato es válido, **DATV = 1**, también lo es la dirección, **DIRV = 1**).

Para diferenciar las líneas del bus de direcciones de las entradas de direccionamiento de los bloques **RAM**, denotaremos con mayúsculas **A_i** las líneas del bus y con minúsculas las entradas **a_i** de cada bloque; un bus de direcciones de 16 líneas corresponderá al conjunto **A15 - A0**, mientras que un circuito integrado de 1K registros tendrá 10 entradas de direccionamiento ($1K = 2^{10}$) que corresponderán a **a9 - a0**. Asimismo, se utiliza la palabra «sector» para referirse a un conjunto de posiciones sucesivas (a un «trozo») del mapa de memoria y la palabra «segmento» para un conjunto de registros seguidos (un «trozo») de un bloque RAM.

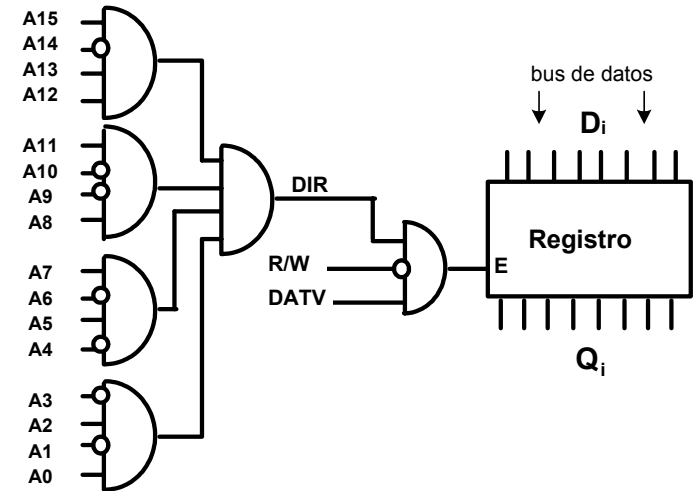
19.3.1. Posicionamiento de un simple registro: dado un registro de retención (entrada de habilitación **E**), se desea situarlo en la posición **B9A5H** del mapa de memoria.

$$\mathbf{B\ 9\ A\ 5\ H} = \mathbf{1011\ 1001\ 1010\ 0101}$$

Para ello será necesario activar la entrada de habilitación del registro con el término mínimo correspondiente a dicho número binario:

$$A_{15} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot A_8 \cdot A_7 \cdot \overline{A_6} \cdot A_5 \cdot \overline{A_4} \cdot \overline{A_3} \cdot A_2 \cdot \overline{A_1} \cdot A_0$$

Además, es preciso multiplicar dicho término por $\overline{R/W}$ para que la habilitación sólo se produzca en las operaciones de escritura y por **DATV** para que dicha escritura se efectúe cuando el dato sea válido (que también lo será la dirección).



19.3.2. Posicionamiento de un bloque RAM: dado un circuito integrado RAM de 2K registros, se desea situarlo a partir de la posición **5800H** del mapa de memoria.

$$\mathbf{5\ 8\ 0\ 0\ H} = \mathbf{0101\ 1000\ 0000\ 0000}$$

Un bloque de 2 K requiere 11 líneas de direccionamiento **a10 - a0** para los 2048 registros que contiene ($2K = 2 \times 2^{10}$), de forma que ocupará las posiciones de memoria:

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	5 8 0 0 H
0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	5 F F F H

es decir todas las posiciones correspondientes a los números binarios

$$\mathbf{0\ 1\ 0\ 1\ 1\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ -\ 5800 - 5FFF}$$

donde el signo - representa la posibilidad de ambos valores booleanos **0, 1**.

Por ello, es preciso ocupar los siguientes sectores de memoria:

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1 0 0 0 H
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1 F F F H
0	0	0	1	-	-	-	-	-	-	-	-	-	-	-	-	4K
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	2 0 0 0 H
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	3 F F F H
0	0	1	-	-	-	-	-	-	-	-	-	-	-	-	-	8K
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	4 0 0 0 H
0	1	0	0	1	1	1	1	1	1	1	1	1	1	1	1	4 F F F H
0	1	0	0	-	-	-	-	-	-	-	-	-	-	-	-	4K.

En el primero de los sectores anteriores caben 4K (dos de los bloques), el segundo sector es de 8K (cuatro bloques de 2K) y, por último, quedan 4K (dos bloques) que irán en el tercero de los sectores.

La situación de los 8 bloques sucesivos será la siguiente:

0	0	0	1	0	-	-	-	-	-	-	-	-	-	-	-	1000 – 17FF
0	0	0	1	1	-	-	-	-	-	-	-	-	-	-	-	1800 – 1FFF
0	0	1	0	0	-	-	-	-	-	-	-	-	-	-	-	2000 – 27FF
0	0	1	0	1	-	-	-	-	-	-	-	-	-	-	-	2800 – 2FFF
0	0	1	1	0	-	-	-	-	-	-	-	-	-	-	-	3000 – 37FF
0	0	1	1	1	-	-	-	-	-	-	-	-	-	-	-	3800 – 3FFF
0	1	0	0	0	-	-	-	-	-	-	-	-	-	-	-	4000 – 47FF
0	1	0	0	1	-	-	-	-	-	-	-	-	-	-	-	4800 – 4FFF.

Como los 8 bloques van seguidos, las líneas **A13 A12 A11** sirven para diferenciarlos (si bien el primer bloque en el mapa de memoria corresponde al valor **010** de dichas líneas, el segundo al valor **011**,... y el último al valor **001**) y puede utilizarse un decodificador análogo al del ejercicio anterior; pero, en este caso, el decodificador debe habilitarse en las siguientes situaciones:

$$A_{15} = 0, A_{14} = 0, A_{13} = 0, A_{12} = 1$$

$$A_{15} = 0, A_{14} = 0, A_{13} = 1$$

$$A_{15} = 0, A_{14} = 1, A_{13} = 0, A_{12} = 0$$

$$E(\text{decodif.}) = \overline{A_{15}} \cdot (\overline{A_{14}} \cdot \overline{A_{13}} \cdot A_{12} + \overline{A_{14}} \cdot A_{13} + A_{14} \cdot \overline{A_{13}} \cdot \overline{A_{12}}) \cdot \text{DIRV}$$

$$a_{10} - a_0 = A_{10} - A_0; \quad \overline{WE} = \overline{R/W} \cdot \text{DATV}; \quad OE = R/W.$$

19.3.4. Posicionamiento de 4 bloques RAM en sectores no contiguos: sean 4 circuitos integrados RAM de 2K registros que deben colocarse en el mapa de memoria

a) a partir de las posiciones **0000H**, **2000H**, **4000H**, y **6000H**, respectivamente.

La situación de los 4 bloques sucesivos será la siguiente:

A15	A14	A13	A12	A11	A10	A9	A8	A7	A6	A5	A4	A3	A2	A1	A0	
0	0	0	0	0	-	-	-	-	-	-	-	-	-	-	-	0000 – 07FF
0	0	1	0	0	-	-	-	-	-	-	-	-	-	-	-	2000 – 27FF
0	1	0	0	0	-	-	-	-	-	-	-	-	-	-	-	4000 – 47FF
0	1	1	0	0	-	-	-	-	-	-	-	-	-	-	-	6000 – 67FF

Se observa que los 4 bloques pueden diferenciarse por los valores de las líneas **A14 A13** de forma que pueden seleccionarse mediante un decodificador de 2 líneas de entrada (**A14 A13**) y 4 de salida (para las habilitaciones **CE** de cada uno de los circuitos integrados); las líneas **A14** y **A13** adoptan los cuatro vectores posibles (**00**, **01**, **10** y **11**) y las otras tres líneas **A15 A12** y **A11** tienen el mismo valor en los cuatro bloques, de forma que el decodificador debe habilitarse cuando: $A_{15} = 0, A_{12} = 0, A_{11} = 0$.

$$E(\text{decodificador}) = \overline{A_{15}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

$$a_{10} - a_0 = A_{10} - A_0; \quad \overline{WE} = \overline{R/W} \cdot \text{DATV}; \quad OE = R/W.$$

b) a partir de las posiciones **1000H**, **5000H**, **9800H**, y **B000H**.

La situación de los 4 bloques sucesivos será la siguiente:

0	0	0	1	0	-	-	-	-	-	-	-	-	-	-	-	1000 – 17FF
0	1	0	1	0	-	-	-	-	-	-	-	-	-	-	-	5000 – 57FF
1	0	0	1	1	-	-	-	-	-	-	-	-	-	-	-	9800 – 9FFF
1	0	1	1	0	-	-	-	-	-	-	-	-	-	-	-	B000 – B7FF

Como no existe ninguna regularidad en las posiciones que ocupan, los 4 bloques deberán habilitarse independientemente, a través de la función correspondiente a su situación en el mapa de memoria:

$$CE\ 1 = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot A_{12} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

$$CE\ 2 = \overline{A_{15}} \cdot A_{14} \cdot \overline{A_{13}} \cdot A_{12} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

$$CE\ 3 = A_{15} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot A_{12} \cdot A_{11} \cdot \text{DIRV}$$

$$CE\ 4 = A_{15} \cdot \overline{A_{14}} \cdot A_{13} \cdot A_{12} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

Estas funciones pueden ser programadas sobre un bloque **PAL** (precisamente el direccionamiento para configurar mapas de memoria fue una de las primeras aplicaciones de dichos bloques programables).

19.4. Direccionamiento de un bloque RAM en sectores separados

Aunque los ejemplos siguientes constituyen casos muy particulares de ubicación de un circuito integrado RAM en un mapa de memoria (que no se presentan habitualmente), son de gran interés de cara a comprender en profundidad el posicionamiento de registros en un mapa de memoria.

19.4.1. Sea un circuito integrado RAM de 8K (13 líneas de direccionamiento), cuyos registros se desean colocar en el mapa de memoria, en sectores no contiguos:

a) 4K registros en el sector inicial del mapa de memoria y 4K en el final.

Un segmento de 4K registros requiere 12 líneas de direccionamiento $a_{11} - a_0$, de forma que el situado en el sector inicial tendrá las restantes líneas $A_{15} - A_{12}$ a 0, mientras que su valor para el sector final será 1; el circuito integrado de 8K tendrá 13 líneas de direccionamiento $a_{12} - a_0$, 12 de las cuales $a_{11} - a_0$ sirven para conformar segmentos de 4K y la línea a_{12} distinguirá entre los dos segmentos (inicial y final del mapa de memoria):

0	0	0	0	-	-	-	-	-	-	-	-	0000 – 0FFF
1	1	1	1	-	-	-	-	-	-	-	-	F000 – FFFF

$$CE = (\overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} + A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12}) \cdot DIRV$$

$$a_{12} = A_{12} \text{ o, también, } a_{12} = A_{15}$$

$$a_{11} - a_0 = A_{11} - A_0; \quad WE = \overline{R/W} \cdot DATV; \quad OE = R/W.$$

b) 2K registros en los sectores del mapa de memoria cuya dirección inicial es, respectivamente, 0000H, 4000H, 8000H y C000H.

Un segmento de 2K registros requiere 11 líneas de direccionamiento $a_{10} - a_0$; las posiciones de memoria ocupadas por los cuatro segmentos de 2K serán:

0	0	0	0	0	-	-	-	-	-	-	-	0000 – 07FF
0	1	0	0	0	-	-	-	-	-	-	-	4000 – 47FF
1	0	0	0	0	-	-	-	-	-	-	-	8000 – 87FF
1	1	0	0	0	-	-	-	-	-	-	-	C000 – C7FF

Para situar el bloque en estas posiciones, debe habilitarse cuando

$$A_{13} = 0, A_{12} = 0, A_{11} = 0: \quad CE = \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot DIRV.$$

$$a_{10} - a_0 = A_{10} - A_0; \quad WE = \overline{R/W} \cdot DATV; \quad OE = R/W$$

Para diferenciar los 4 segmentos del bloque RAM que van a encontrarse en 4 sectores separados del mapa de memoria, disponemos de las líneas $A_{15} A_{14}$ que diferencian dichos 4 sectores ($A_{15} A_{14}$ recorren todos vectores posibles); las entradas $a_{12} a_{11}$ que distinguen los 4 segmentos del bloque han de conectarse a dichas líneas $A_{15} A_{14}$: $a_{12} = A_{15}$; $a_{11} = A_{14}$

Téngase en cuenta que, si se conectasen las líneas a_{12} y a_{11} a las del mismo número del bus de direcciones, solamente se utilizaría el primer segmento de 2K del bloque RAM, pues los valores de las entradas $a_{12}a_{11} = A_{12}A_{11} = 00$ serían iguales ($a_{12}a_{11} = 00$) en los cuatro sectores del mapa de memoria (lo cual significa que los otros segmentos $a_{12}a_{11} = 01, 10, 11$ no serían utilizados, sino que dicho segmento $a_{12}a_{11} = 00$ se encontraría repetido cuatro veces en sectores distintos del mapa de memoria).

c) 6K registros en el sector inicial del mapa de memoria y 2K en el final.

Distribuyendo el circuito integrado en segmentos de 2K (11 líneas $a_{10} - a_0$):

0	0	0	0	0	-	-	-	-	-	-	-	0000 – 07FF
0	0	0	0	1	-	-	-	-	-	-	-	0800 – 0FFF
0	0	0	1	0	-	-	-	-	-	-	-	1000 – 17FF
1	1	1	1	1	-	-	-	-	-	-	-	F800 – FFFF

Los tres primeros segmentos han de situarse en el sector inicial de la memoria ($A_{15} = 0, A_{14} = 0, A_{13} = 0$, y $A_{12}A_{11} = 00, 01, 10$) y el cuarto segmento en el sector final ($A_{15} = 1, A_{14} = 1, A_{13} = 1, A_{12} = 1, A_{11} = 1$):

$$CE = (\overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot (\overline{A_{12}} + \overline{A_{11}}) + A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11}) \cdot DIRV$$

Los cuatro segmentos pueden diferenciarse por los valores de las líneas $A_{12} A_{11}$ que recorren todos vectores posibles: $a_{12} = A_{12}$; $a_{11} = A_{11}$

$$a_{10} - a_0 = A_{10} - A_0; \quad WE = \overline{R/W} \cdot DATV; \quad OE = R/W.$$

d) 2K registros a partir de la posición 4000H y 6K a partir de A000H

En segmentos de 2 K:

0	1	0	0	0	-	-	-	-	-	-	-	4000 – 47FF
1	0	1	0	0	-	-	-	-	-	-	-	A000 – A7FF
1	0	1	0	1	-	-	-	-	-	-	-	A800 – AFFF
1	0	1	1	0	-	-	-	-	-	-	-	B000 – B7FF

el primer segmento ha de situarse en $A_{15} = 0, A_{14} = 1, A_{13} = 0, A_{12} = 0$ y $A_{11} = 0$ y los otros tres segmentos en $A_{15} = 1, A_{14} = 0, A_{13} = 1$ y $A_{12}A_{11} = 00, 01, 10$:

$$CE = (\overline{A_{15}} \cdot A_{14} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} + A_{15} \cdot \overline{A_{14}} \cdot A_{13} \cdot (\overline{A_{12}} + \overline{A_{11}})) \cdot DIRV$$

En este caso, para diferenciar los cuatro segmentos y determinar la conexión de las entradas de direccionamiento que los numeran **a12 a11** podemos construir una pequeña tabla en la forma siguiente:

A15	A14	A13	A12	A11	a12	a11	
0	1	0	0	0	0	0	A partir de esta «tabla de verdad» se obtienen: $a_{12} = A_{12} + A_{11}$ $a_{11} = A_{13} \cdot A_{11}$
1	0	1	0	0	0	1	
1	0	1	0	1	1	0	
1	0	1	1	0	1	1	

19.4.2. Sea un circuito integrado RAM de 4K (12 líneas de direccionamiento), cuyos registros se desea situar en las posiciones iniciales del mapa de memoria, pero sabiendo que el primer sector de 1K del mapa se encuentra ocupado previamente.

El sector de 1K ocupado corresponde a:

0 0 0 0 0 0 - - - - - 0000 – 03FF

A continuación de dicho sector podemos situar los 4K registros en la siguiente forma:

0 0 0 0 0 1 - - - - - 0400 – 07FF
0 0 0 0 1 - - - - - 0800 – 0FFF
0 0 0 1 0 0 - - - - - 1000 – 17FF

El primero de los sectores anteriores es de 1K (10 líneas), el segundo permite colocar 2K (11 líneas) y el tercero corresponde a 1K restante; la habilitación del circuito integrado ha de producirse en las tres situaciones:

$A_{15} = 0, A_{14} = 0, A_{13} = 0, A_{12} = 0, A_{11} = 0, A_{10} = 1$ 1 sector de 1K
 $A_{15} = 0, A_{14} = 0, A_{13} = 0, A_{12} = 0, A_{11} = 1$ 2 sectores de 1K
 $A_{15} = 0, A_{14} = 0, A_{13} = 0, A_{12} = 1, A_{11} = 0, A_{10} = 0$ 1 sector de 1K

$$CE = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot (\overline{A_{12}} \cdot \overline{A_{11}} \cdot A_{10} + \overline{A_{12}} \cdot A_{11} + A_{12} \cdot \overline{A_{11}} \cdot \overline{A_{10}}) \cdot \text{DIRV}$$

Como los 4 sectores de memoria de 1K van seguidos, las líneas **A11 A10** sirven para diferenciarlos (si bien el primer segmento del bloque integrado en el mapa de memoria corresponde al valor **01** de dichas líneas, el segundo al valor **10**, el tercero a **11** y el último al valor **00**):

$$a_{11} - a_0 = A_{11} - A_0; \quad WE = \overline{R/W} \cdot \text{DATV}; \quad OE = R/W.$$

19.4.3. Sea un circuito integrado RAM de 8K (13 líneas de direccionamiento), cuyos registros se desean situar en la parte inicial de un mapa de memoria, en el cual se encuentra ocupado el sector **0800H a 6FFFH**.

0 8 0 0 H = 0000 1000 0000 0000
6 F F F H = 0110 1111 1111 1111

Previamente a la posición **0800H** se pueden poner registros en:

0 0 0 0 0 - - - - - 0000 – 07FF
 sector de 2K; quedarán por situar otros 6K registros detrás de la posición 6FFFH:

0 1 1 1 - - - - - 7000 – 7FFF
1 0 0 0 0 - - - - - 8000 – 87FF

sectores de 4K y 2K, respectivamente.

La habilitación del circuito integrado ha de producirse en:

$A_{15} = 0, A_{14} = 0, A_{13} = 0, A_{12} = 0, A_{11} = 0$ 1 segmento de 2K
 $A_{15} = 0, A_{14} = 1, A_{13} = 1, A_{12} = 1$ 2 segmentos de 2K
 $A_{15} = 1, A_{14} = 0, A_{13} = 0, A_{12} = 0, A_{11} = 0$ 1 segmento de 2K
 (de los tres vectores anteriores, primero y tercero pueden simplificarse entre sí)
 $CE = (\overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} + \overline{A_{15}} \cdot A_{14} \cdot A_{13} \cdot A_{12}) \cdot \text{DIRV}$

El bloque RAM queda organizado en cuatro segmentos de 2K registros, de los cuales los tres últimos se sitúan juntos; para diferenciar los segmentos y determinar la conexión de sus entradas de selección **a12 a11** podemos utilizar la siguiente tabla:

A15	A14	A13	A12	A11	a12	a11	
0	0	0	0	0	0	0	A partir de esta tabla se obtienen: $a_{12} = A_{11} + A_{15}$ $a_{11} = A_{12} \cdot \overline{A_{11}} + A_{15}$
0	1	1	1	0	0	1	
0	1	1	1	1	1	0	
1	0	0	0	0	1	1	

$$a_{10} - a_0 = A_{10} - A_0; \quad WE = \overline{R/W} \cdot \text{DATV}; \quad OE = R/W.$$

19.4.4. Determinación de la posición en un mapa de memoria: ¿cuál será el mapa de memoria ocupado por un bloque RAM de 8K, cuya habilitación es la siguiente:

$$CE = \overline{A_{15}} \cdot (\overline{A_{14}} \cdot A_{13} \cdot (\overline{A_{12}} + \overline{A_{11}})) + A_{14} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

El bloque se habilitará en cada una de las siguientes situaciones:

$A_{15} = 0, A_{14} = 0, A_{13} = 1, A_{12} = 0$
 $A_{15} = 0, A_{14} = 0, A_{13} = 1, A_{11} = 0$
 $A_{15} = 0, A_{14} = 1, A_{13} = 0, A_{12} = 0, A_{11} = 0$

que podemos expresarlas ordenadamente en la siguiente tabla

A15	A14	A13	A12	A11	
0	0	1	0	0	sector 2000-27FF
0	0	1	0	1	sector 2800-2FFF
0	0	1	1	0	sector 3000-37FF
0	1	0	0	0	sector 4000-47FF

Los sectores anteriores son de 2K (11 líneas **A10** - **A0** para direccionar sus registros) y los tres primeros son contiguos (6K), de forma que el mapa de memoria ocupado está dividido en dos trozos: **2000-37FF** y **4000-47FF** (entre ambos queda un sector de 2K 3800-3FFF).

¿Cómo deben conectarse las entradas de direccionamiento **a12** - **a0** de este bloque?

A15	A14	A13	A12	A11	a12	a11	
0	0	1	0	0	0	0	
0	0	1	0	1	0	1	
0	0	1	1	0	1	0	$a_{12} = A_{14} + A_{12}$
0	1	0	0	0	1	1	$a_{11} = A_{14} + A_{11}$

19.5. Memorias de sólo lectura

Los codificadores **ROM**, aunque no contienen registros, pueden ser considerados como «memorias de sólo lectura» (a lo cual alude su propia denominación ROM: *read only memory*, debida a que ésta fue la primera aplicación de los grandes codificadores con estructura ROM).

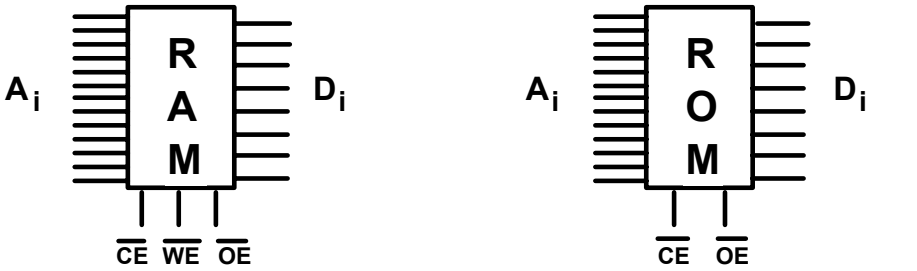
Un codificador proporciona un vector de salida (dato) para cada vector de entrada (dirección) que recibe; es como si a una «dirección» recibida a través de sus entradas respondiese con un «dato» en sus líneas de salida; el resultado es la obtención en las salidas de una palabra binaria, seleccionada por el número binario que hay en sus entradas.

Ciertamente la relación entrada – salida (dirección – dato) es meramente combinatorial y no hay «memoria» en el sentido propio de los sistemas secuenciales; pero, funcionalmente, un codificador presenta un conjunto de palabras binarias numeradas.

En tal sentido, el comportamiento de un codificador equivale al de una memoria de acceso directo cuyos registros estuviesen ya escritos con información fija y solamente se pudieran leer: *memoria de sólo lectura*. Las entradas del codificador corresponden a las líneas de direcciones y las salidas a las líneas de datos en una operación de lectura.

Por otra parte, para poder conectar el codificador ROM a un bus de datos será necesario dotar a sus salidas de capacidad tri-estado, con la posibilidad de desconexión (alta impedancia) controlada por una línea de habilitación (CE), de forma que pueda compartir las líneas de datos con otros bloques RAM o ROM.

Por analogía con los circuitos integrados **RAM**, los codificadores **ROM** suelen tener dos entradas de control: **CE** para habilitar el bloque y **OE** para habilitar las salidas del mismo (lectura), ambas activas con valor booleano **0**. [**WE** no tiene sentido por cuanto que la escritura en este tipo de bloques no es posible.]



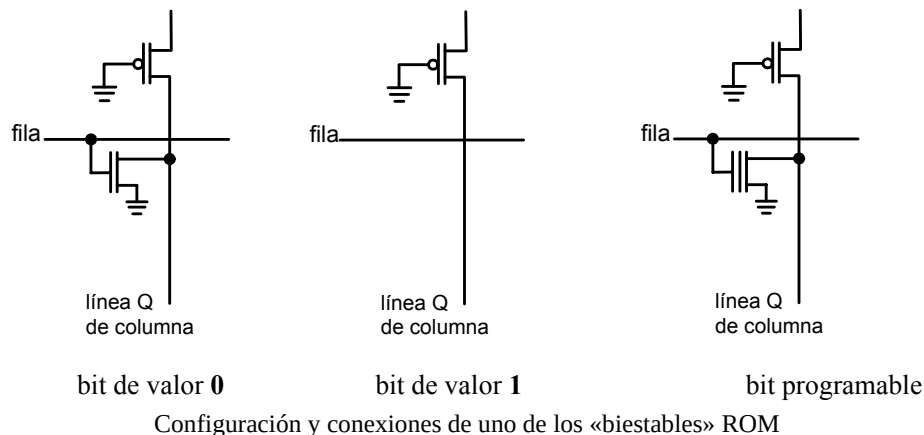
Las memorias de sólo lectura son útiles para datos fijos, tablas de valores, tablas de conversión, tablas funcionales, etc., y, en particular, para los programas específicos de los microprocesadores dedicados a aplicaciones de control.

La disponibilidad de codificadores **ROM programables** permite contar con «memorias de sólo lectura» cuya «escritura» previa puede realizarse mediante programación. De forma que las palabras binarias contenidas en una «memoria» ROM pueden venir fijadas «de fábrica», resultantes de la inclusión o no de transistores en los correspondientes nudos de la matriz "O", o bien pueden ser «escritas» por el diseñador (codificadores programables **PROM**), a través de un programador, previamente a su utilización circuital.

La estructura de una memoria ROM es análoga a la de una RAM, prescindiendo de los circuitos correspondientes a la escritura y utilizando una sola línea Q (en lugar de las dos líneas Q y $\bar{Q}$); la celda básica, para cada bit, consistirá en la presencia/ausencia de un transistor en el nudo correspondiente, según que el valor de dicho bit sea **0/1**.

La figura de la página siguiente muestra tal estructura; en ella se observa que el transistor PMOS «pone» la línea Q a **1** y dicho valor permanece cuando no hay transistor NMOS en la fila seleccionada; en cambio, cuando el transistor NMOS está presente en dicha fila, conducirá si está seleccionada y llevará la línea Q a **0**.

En el caso de codificadores **PROM** se incluye un transistor **EPROM** programable para cada bit, de forma que equivale a valor **1** (el transistor no conduce nunca) cuando se carga negativamente su puerta aislada y, en caso de que dicha puerta esté descargada, implica valor **0** (el transistor conduce al ser seleccionado y lleva la línea Q a 0 V).



Existen tres tipos de transistores MOS programables: EPROM, E²PROM y FLASH (ver apartado 9.5., primer volumen). En cuanto a la forma de programarlos (y, también, en cuanto a la velocidad de lectura) no hay diferencias funcionales entre ellos; se distinguen, en cambio, en que el «borrado» de los primeros puede hacerse con luz ultravioleta y los otros dos permiten hacerlo mediante tensión eléctrica (de signo contrario a la de su programación). En el caso E²PROM el borrado es individual (cada transistor se puede programar –bit a 1– o borrar –bit a 0– individualmente), mientras que en el caso FLASH el borrado es global (se borra todo el bloque, mediante una tensión eléctrica positiva aplicada al terminal de fuente que es común a todos los transistores).

Cualquiera de estos tres tipos puede ser utilizado como ROM, memoria de sólo lectura, programando previamente su contenido a través de un programador.

Memorias de sólo lectura reprogramables

Los transistores MOS tipo E²PROM son reprogramables eléctricamente, es decir, puede «escribirse» en ellos un 1 (transistor programado) o un 0 (transistor borrado) mediante la aplicación a la puerta de una tensión positiva o negativa relativamente alta. Esta capacidad de programación individual de un 0 o un 1 sobre cada bit ha permitido la construcción de memorias ROM que se pueden re-escribir en el propio circuito funcional, mediante una operación de «escritura» a través de los buses.

Dicha operación requiere tiempos más amplios que los normales de acceso a una memoria RAM ya que es necesario efectuar la programación de la correspondiente palabra binaria sobre los transistores E²PROM, cargando con electrones su puerta aislada (caso de un bit a 0) o descargándola (caso de programar un 1).

Además tal operación requiere, por lo general, tensiones más elevadas que las habituales de alimentación digital y necesita los correspondientes circuitos de control de la programación; tanto las tensiones como los circuitos adicionales se incorporan dentro del circuito integrado en su diseño y fabricación, de forma que, desde el exterior, actúa como una memoria de acceso directo con capacidad de lectura y de escritura, solo que la escritura es lenta y requiere varios ciclos de reloj.

Incluso, se fabrican memorias RAM duplicadas con otra ROM dentro del propio circuito integrado para evitar la pérdida de información cuando dejan de estar alimentadas a la correspondiente tensión eléctrica: cuando se detecta una «caída» de tensión, un circuito de control determina el traspaso de todos los datos del bloque RAM al bloque ROM disponible en el interior del mismo circuito integrado y, viceversa, al recuperarse la tensión de alimentación el control reescribe el bloque RAM con los datos guardados en la parte ROM del integrado.

Con la misma finalidad, existen series especiales de circuitos integrados RAM no volátiles, dotados de una batería interna (recargable con la propia alimentación del integrado) que conserva los valores almacenados en sus registros, en ausencia de tensión de alimentación.

Memorias FLASH de acceso secuencial

Un bloque ROM programable tipo FLASH constituye una memoria de lectura rápida, con capacidad de escritura lenta (ya que debe hacerse por programación) si el bloque ha sido previamente «borrado»; en las memorias FLASH el borrado se efectúa en forma global (se borra todo el bloque a la vez) ya que los transistores, una vez programado, no pueden borrarse individualmente.

Las reducidas dimensiones de los transistores FLASH y de su agrupación en configuración ROM han permitido la integración de memorias FLASH de muy alta capacidad. Tales memorias se utilizan para el almacenamiento masivo de datos, con una funcionalidad análoga a la de los disquetes o discos compactos CDs y con importantes ventajas sobre ellos al no necesitar un sistema mecánico para su lectura.

Para estas aplicaciones de memorias de conservación y transporte de datos (o de memoria de acumulación de datos en un sistema de adquisición de los mismos) se utilizan memorias FLASH serie, con acceso secuencial que se gestiona a través de contadores incluidos en la propia memoria. Estas memorias permiten escribir los datos en ellas y recuperarlos posteriormente a través de una simple entrada serie.

20 SISTEMAS CON ARQUITECTURA DE BUS

- 20.1. Memoria en sentido amplio
- 20.2. Configuración circuital de mapas de memoria
- 20.3. Ciclos de lectura y de escritura: tiempos de acceso
- 20.4. Problemas relativos a los buses: adaptadores de bus

La arquitectura basada en buses (datos y direcciones), que determina una división estructural entre procesador y memoria, sirve también para organizar las transferencias de información con el exterior.

De esta forma la memoria engloba, no sólo los registros que conservan la información (datos y resultados intermedios) sino, también, aquellos registros a través de los cuales se envía información al exterior o se recibe información externa (salidas y entradas). La memoria incluye dos unidades de diferente funcionalidad: la unidad de almacenamiento de información y la unidad de comunicación con el exterior.

De manera que los elementos de memoria son diversos: RAM, ROM, registros de salida, adaptadores de entradas, adaptadores de periféricos,... Para el procesador, cualquier elemento de memoria no es sino un conjunto de registros numerados (en ocasiones un registro individual con su número), a los que puede acceder a través del bus de datos, seleccionando cada registro concreto por su número, a través del bus de direcciones.

Este capítulo, después de considerar en detalle tales elementos de memoria, trata de la forma de situarlos adecuadamente en el mapa de memoria, de manera que el procesador pueda escribir o leer sobre los correspondientes registros mediante su número. También se estudian las formas de onda y requisitos temporales (tiempos de acceso y de habilitación) necesarios para ejecutar un ciclo de lectura o de escritura sobre los diversos elementos de memoria.

Por último, se consideran las cuestiones específicas que afectan a los buses: la necesidad de no retrasar las señales y la de proporcionar intensidades relativamente altas. En ocasiones resulta necesario amplificar la intensidad disponible mediante «adaptadores de bus», que, en el caso del bus de datos, han de ser bidireccionales.

Las exigencias de velocidad e intensidad en los buses dan lugar a la utilización de series específicas de circuitos integrados: lógica «interbus», de muy bajos tiempos de propagación y altas intensidades de salida; en particular, resulta muy apropiada la tecnología BiCMOS, desarrollada precisamente para esta clase de aplicaciones aprovechando las ventajas de ambos tipos de transistores (bipolares y MOS).

*Una útil referencia bibliográfica: como complemento a este capítulo, un libro dedicado a sistemas con microprocesadores, que recoge una amplia diversidad de aspectos relativos a memorias, periféricos y mapas de memoria, es el texto de Bonifacio Martín del Brío, **Sistemas electrónicos basados en microprocesadores y microcontroladores**, publicado en esta misma colección de Textos Docentes, nº 61. Pressas Universitarias de Zaragoza. 1999.*

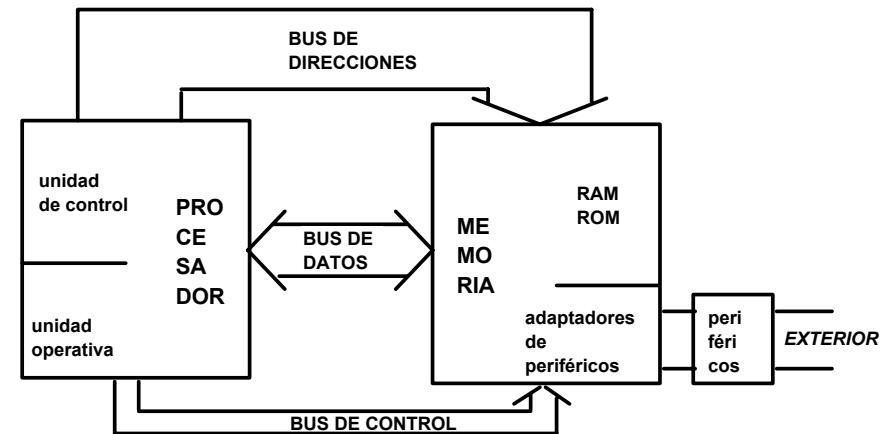
20.1. Memoria en sentido amplio

La combinación estructural de bus de datos y bus de direcciones organiza no solamente el almacenamiento de información sino también la comunicación de la misma con el exterior. Ambas funciones se refieren a transferencia de información: en un caso, transferencia a memoria para su utilización posterior (transferencia en el tiempo) y, en el otro, transferencia hacia el exterior (transferencia en el espacio).

La memoria de un circuito digital complejo, como concepto genérico, engloba a todo tipo de registros de los que se extrae directamente o a los que se envía directamente información digital. En este sentido, el concepto de memoria incluye no sólo a los registros de trabajo donde se memorizan datos y resultados (generalmente englobados en bloques de tipo RAM), sino también a los registros de información fija que contienen tablas de valores o programas (como pueden ser los bloques ROM) y a los registros de adaptación de entradas y de salidas del circuito (adaptadores de periféricos).

De esta forma un sistema digital complejo puede dividirse, conceptualmente, en dos partes diferenciadas:

- el procesador que controla el proceso y efectúa las operaciones pertinentes
- y la memoria o conjunto amplio de registros que memorizan, reciben o envían la información, es decir, que realizan la transferencia de información.



Este concepto amplio de memoria engloba dos unidades claramente diferenciadas por su finalidad operativa:

- la unidad de memoria, como lugar de almacenamiento de la información disponible,
- y la unidad de entradas/salidas, como conjunto de periféricos que comunican con el exterior.

La comunicación con el exterior se realiza a través de registros adaptadores de salida que presentan la información hacia un periférico (visualizador, conversor D/A, impresora, monitor, módem, etc.) o a través de adaptadores de entrada, asimilables conceptualmente a registros, que reciben la información desde el periférico (teclado, pulsadores o conmutadores, conversor A/D, módem, etc.). Desde su punto de vista, el procesador se encuentra con un conjunto de registros (generalmente unidireccionales) que, posteriormente, se comunican con un periférico a través del cual reciben o transmiten información respecto al exterior.

Para el procesador todo lo demás son registros, seleccionables (a través del bus de direcciones) por su número dentro del mapa de memoria, y sobre los cuales escribe o lee, a través del bus de datos, información contenida en palabras binarias.

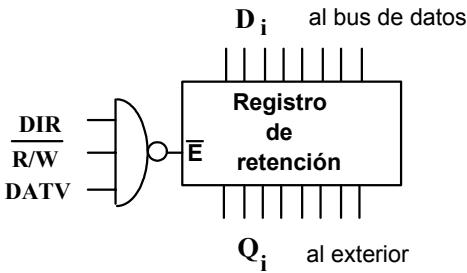
Los buses de datos y direcciones, no solamente dividen al sistema en dos partes estructurales (procesador, memoria), sino que permiten dividir la memoria en elementos constituidos por conjuntos de registros (o registros individuales) que ocupan los correspondientes sectores del mapa de memoria.

La unidad de memoria propiamente dicha, esto es, el conjunto de registros de trabajo del procesador (almacenamiento de información), estará constituida por integrados **RAM**, capaces de memorizar datos y resultados operativos, y por circuitos integrados **ROM**, con registros «de sólo lectura» cuya información es fija.

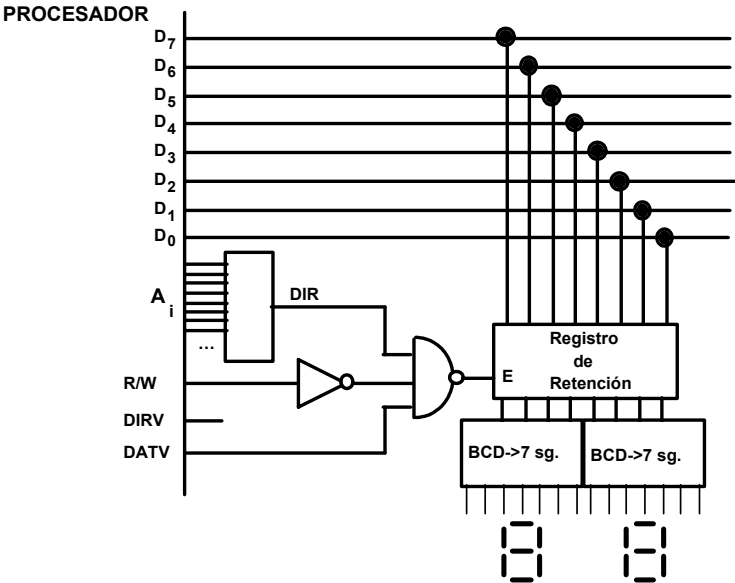
Otra parte diferenciada de la memoria está formada por los registros de adaptación de periféricos, configurando la unidad de entradas/salidas del sistema; tales registros suelen ser unidireccionales, distinguiéndose entre los registros de salida y los adaptadores de entrada.

Los registros de salida serán del tipo de registros de retención (*latch memory*), conformados por *n* biestables con una entrada de habilitación común **E**, la cual será activada al realizar una operación de escritura sobre la dirección que ocupa el registro en el mapa de memoria **DIR**; tales registros han de habilitarse al enviar la correspondiente dirección **DIR**, cuando la operación es de escritura **R/W = 0** y una vez que el dato es válido **DATV = 1**.

$$E = DIR \cdot \overline{R/W} \cdot DATV$$

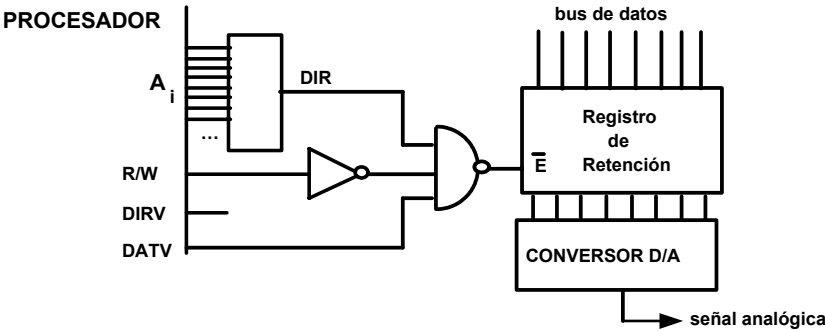


La figura siguiente representa un ejemplo de periférico de salida muy simple: se trata de un visualizador de dos cifras decimales adaptado al bus de datos, a través de un registro de salidas y de los correspondientes decodificadores de BCD a 7 segmentos; cuando el procesador efectúa una operación de escritura sobre la dirección ocupada por dicho registro (**DIR, R/W = 0, DATV = 1**), éste recibe por el bus de datos dos cifras BCD que quedan permanentemente representadas en los visualizadores de 7 segmentos.



Periférico que permite la visualización de dos cifras decimales

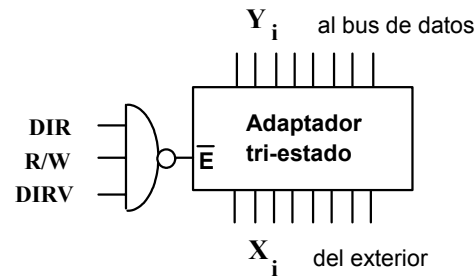
Si las salidas del registro se conectan a un conversor digital/análogo, la salida de éste proporcionará la señal de tensión que corresponde a los sucesivos valores numéricos que el procesador escriba sobre la dirección de memoria correspondiente; de esta forma, el procesador puede generar una determinada onda o una señal de referencia.



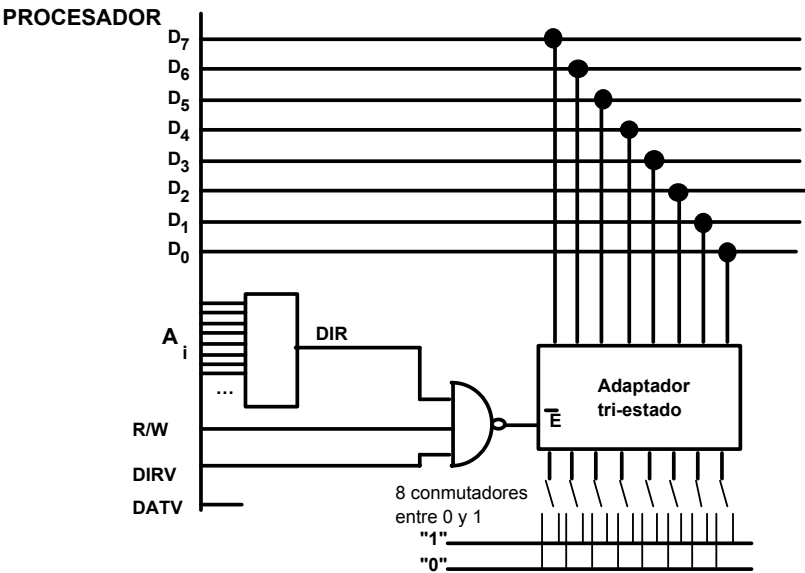
Periférico que permite la generación de una señal analógica

Los adaptadores de entrada serán simples adaptadores tri-estado (*buffers*) que, al seleccionados en una operación de lectura, vuelcan sobre el bus de datos los valores booleanos presentes en sus entradas; su habilitación ha de producirse cuando la operación es de lectura **R/W = 1** sobre la correspondiente dirección **DIR**, una vez que se valide dicha dirección como correcta **DIRV = 1**.

$$E = DIR \cdot R/W \cdot DIRV$$

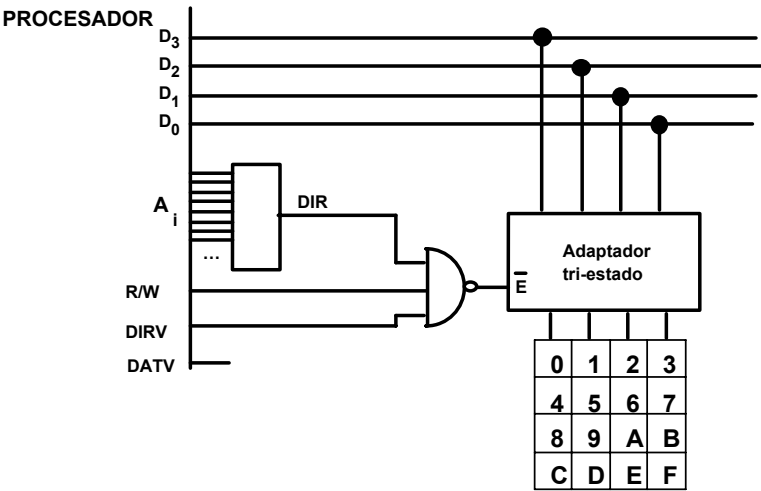


La siguiente figura representa un periférico de entrada muy simple constituido por ocho conmutadores, conectados al bus de datos a través de un adaptador de entradas; cuando el procesador efectúa una operación de lectura sobre la dirección ocupada por dicho adaptador (**DIR**, **R/W = 1**, **DATV = 1**), éste resulta habilitado, comunicando al procesador los valores booleanos fijados sobre los conmutadores.



Periférico que permite leer el estado de 8 conmutadores

Un circuito análogo al anterior sería el correspondiente a un teclado hexadecimal o decimal o a un conmutador BCD, que necesitará solamente 4 líneas del bus de datos.

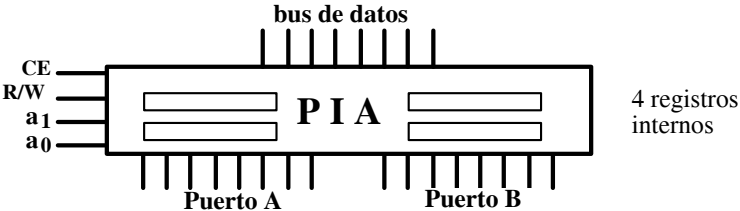


Periférico que permite leer un teclado hexadecimal

Otro tipo de adaptadores de entrada lo constituyen los registros de retención con salida tri-estado, los cuales pueden memorizar una palabra binaria en cualquier momento y transferirla al bus de datos cuando su salida sea habilitada.

Los adaptadores anteriores son de tipo unidireccional, para conectar periféricos de entrada o de salida; existen muchos otros tipos de adaptadores de propósito particular para periféricos específicos y, también, existen adaptadores de tipo genérico, dotados de una cierta programación de su funcionamiento. Como integrantes de la memoria, tales adaptadores equivalen, desde el punto de vista del procesador, a reducidos conjuntos de registros, algunos de ellos unidireccionales.

Un ejemplo de adaptadores de entradas/salidas de tipo genérico es el bloque **PIA** (*Peripheral Interface Adapter*) que ofrece dos puertos de 8 líneas, las cuales pueden ser configuradas como líneas de entrada o de salida y contiene 4 registros: dos de ellos para la transferencia de información (uno para cada puerto) y los otros dos para determinar si las líneas actúan como entradas o como salidas; por tanto, este bloque ocupa 4 posiciones de memoria.



20.2. Configuración circuital de un mapa de memoria

Se pretende en este apartado abordar el diseño del circuito necesario para que los diversos circuitos integrados que constituyen la memoria de un procesador, incluidos los adaptadores de periféricos, queden situados en las direcciones que les corresponden, es decir, la realización circuital de un mapa de memoria.

Por lo general no suele utilizarse la capacidad total de la memoria, es decir, no suelen ocuparse todas las posiciones de memoria que el bus de direcciones permite numerar, sino que puede quedar un buen número de posiciones vacías; ello permite elegir de entre los posibles mapas de memoria el que resulte más sencillo en cuanto al circuito necesario para su realización.

En los ejercicios que siguen se utiliza un bus de direcciones de 16 líneas y 3 líneas de control: lectura/escritura **R/W**, validación de dirección **DIRV** y validación del dato en la escritura **DATV** (se supone que cuando **DATV = 1**, también **DIRV = 1**); los circuitos integrados utilizados son:

- bloques **RAM** de 2K x 8 (entradas de control $\overline{CE}$, $\overline{WE}$ y $\overline{OE}$),
- bloques **ROM** de 2K x 8 (entradas de control $\overline{CE}$ y $\overline{OE}$),
- registros de salida de 8 biestables (entrada de habilitación $\overline{E}$)
- y adaptadores de entrada de 8 líneas (entrada de habilitación $\overline{E}$).

20.2.1. Caso de una memoria reducida: sea una memoria con un circuito integrado RAM de 2K, otro ROM también de 2K, un registro de salida y un adaptador de entrada, es decir, un solo elemento de memoria de cada tipo; la memoria RAM debe situarse al inicio del mapa de memoria y la ROM al final del mismo.

a) Configuración de los 4 elementos con ocupación mínima del mapa de memoria

Los integrados RAM y ROM requieren 11 líneas de direccionamiento **a₁₀ - a₀** para los 2K registros que contienen; quedan 5 líneas disponibles **A₁₅ - A₁₁** para la selección de estos circuitos integrados que deben situarse, respectivamente, al inicio y al final del mapa de memoria:

A ₁₅	A ₁₄	A ₁₃	A ₁₂	A ₁₁	A ₁₀	A ₉	A ₈	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0 0 0 0 H
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	0 7 F F H
0	0	0	0	0	-	-	-	-	-	-	-	-	-	-	-	0000 - 07FF
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	F 8 0 0 H
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	F F F F H
1	1	1	1	1	-	-	-	-	-	-	-	-	-	-	-	F800 - FFFF

El registro de salida **RS** y el adaptador de entrada **AE** pueden situarse en una misma dirección de memoria, accediendo al segundo en las operaciones de lectura y al primero en las de escritura; supongamos que los ubicamos en la posición **8000H**:

1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 8 0 0 0 H.

De acuerdo con los sectores del mapa de memoria indicados, las habilitaciones de los cuatro componentes de la memoria serán:

$$\begin{aligned}\text{RAM: } CE &= \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \text{DIRV} \\ WE &= \overline{R/W} \cdot \text{DATV}; \quad OE = R/W \\ a_{10} - a_0 &= A_{10} - A_0\end{aligned}$$

$$\begin{aligned}\text{ROM: } CE &= A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot \text{DIRV} \quad OE = R/W \\ a_{10} - a_0 &= A_{10} - A_0\end{aligned}$$

AE (adaptador de entradas):

debe habilitarse en la posición **8000H** cuando la operación sea de lectura:

$$E = A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot A_{10} \cdot A_9 \cdot A_8 \cdot A_7 \cdot A_6 \cdot A_5 \cdot A_4 \cdot A_3 \cdot A_2 \cdot A_1 \cdot A_0 \cdot R/W \cdot \text{DIRV}$$

RS (registro de salidas):

debe habilitarse en **8000H** cuando la operación sea de escritura y el dato sea válido:

$$E = A_{15} \cdot A_{14} \cdot A_{13} \cdot A_{12} \cdot A_{11} \cdot A_{10} \cdot A_9 \cdot A_8 \cdot A_7 \cdot A_6 \cdot A_5 \cdot A_4 \cdot A_3 \cdot A_2 \cdot A_1 \cdot A_0 \cdot R/W \cdot \text{DATV}$$

Estas funciones pueden ser programadas sobre un bloque **PAL** (de un amplio número de entradas; se utilizan 19 de ellas: A₁₅ - A₀, R/W, DIRV y DATV).

b) Configuración de los 4 elementos con circuito de posicionamiento reducido

Los cuatro elementos de memoria presentes pueden ser reducidos a tres, al situar el registro de salida **RS** y el adaptador de entrada **AE** en la misma dirección de memoria; para discriminar entre los tres elementos (**RAM**, **ROM**, **RS-AE**) resultantes bastan dos líneas de direccionamiento **A₁₅** y **A₁₄**:

A₁₅ = 1	A₁₄ = 1	Bloque ROM en posiciones finales de memoria
A₁₅ = 1	A₁₄ = 0	Registro de salida RS y adaptador de entrada AE
A₁₅ = 0	A₁₄ = X	Bloque RAM en posiciones iniciales.

Con esta asignación de valores a las líneas **A₁₅** y **A₁₄** resulta que:

a) el integrado **ROM** ocupa todo el sector de memoria numerado por

1 1 X X X - - - - - - - - - - - C000 - FFFF

que supone una ocupación de 16K de memoria (14 líneas); el signo - indica que dichas posiciones se encuentran ocupadas por registros del bloque y el signo X indica que tales líneas del bus de direcciones no son utilizadas.

b) el integrado **RAM** ocupa el sector numerado por

0 X X X X - - - - - 0000 - 7FFF
ocupando 32K de memoria (15 líneas)

c) y los dos adaptadores, aun cuando constituyen una sola posición de memoria, se encuentran en todo el sector numerado por

1 0 X X X X X X X X X X X X X X 8000 - BFFF
que corresponde a una ocupación de 16K de memoria (14 líneas).

El símbolo **X** significa que «no importa» el valor booleano presente en dicha línea, ya que tales líneas no se utilizan en la selección de registros del correspondiente elemento, y el signo - indica que dicha línea direcciona registros internos del bloque integrado.

En suma, en esta configuración (*decodificación parcial*) los registros no se encuentran unívocamente direccionados en una sola posición, sino que aparecen en varias posiciones distintas (por ejemplo, cada adaptador, que en realidad es un solo registro, llega a ocupar 16K posiciones de memoria). Ello es debido a que no se han utilizado todas las líneas del bus de direcciones para situar cada elemento de memoria, lo cual supone un gran desperdicio de posiciones de memoria (al ocupar un mismo registro múltiples posiciones), pero resulta admisible por cuanto que solamente se precisa utilizar una parte reducida de la capacidad total de la memoria.

Dentro de la multiplicidad de posiciones que corresponden a un mismo registro puede elegirse un mapa de memoria concreto, conviniendo una asignación unívoca que constituirá el mapa de memoria con el que trabajará el procesador:

RAM	de	0000 0000 0000 0000	=	0 0 0 0 H
	hasta	0000 0111 1111 1111	=	0 7 F F H
RS-AE	en	1000 0000 0000 0000	=	8 0 0 0 H
ROM	de	1111 1000 0000 0000	=	F 8 0 0 H
	hasta	1111 1111 1111 1111	=	F F F F H

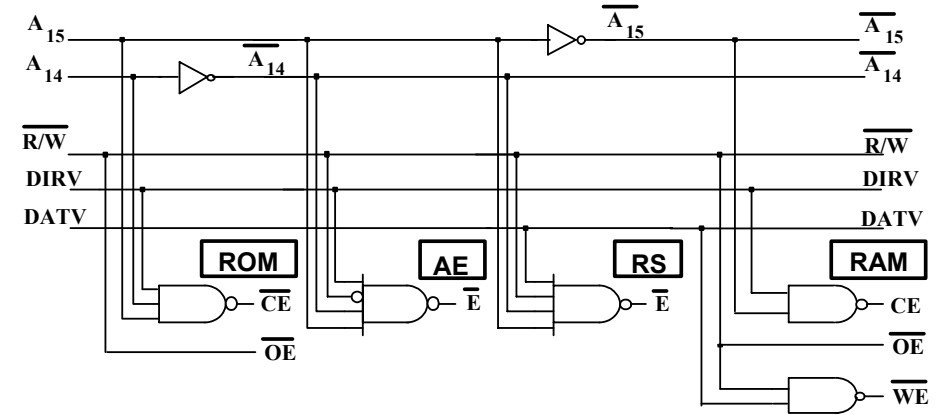
Las condiciones booleanas de habilitación, incluyendo las líneas de control, son las siguientes:

RAM: $CE = \overline{A_{15}} \cdot DIRV$ $WE = \overline{R/W} \cdot DATV$; $OE = R/W$
 $a_{10} - a_0 = A_{10} - A_0$

ROM: $CE = A_{15} \cdot A_{14} \cdot DIRV$ $OE = R/W$
 $a_{10} - a_0 = A_{10} - A_0$

AE (adaptador de entradas): $E = A_{15} \cdot \overline{A_{14}} \cdot R/W \cdot DIRV$

RS (registro de salidas): $E = A_{15} \cdot \overline{A_{14}} \cdot \overline{R/W} \cdot DATV$



Ha de tenerse en cuenta que las entradas de control de los elementos de memoria utilizados se activan con valor booleano **0**, lo cual hace que las anteriores funciones booleanas conduzcan directamente a puertas "y-negada" (Nand).

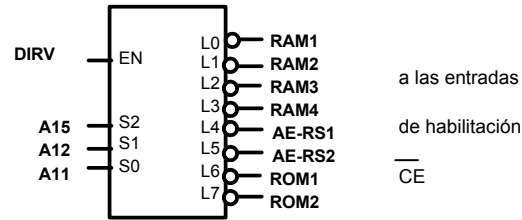
Todas las puertas de direccionamiento deben ser muy rápidas a fin de minimizar los retrasos en las señales de habilitación y, con ello, asegurar que se cumplen los diversos tiempos exigidos por los ciclos de lectura y de escritura sobre los elementos de memoria.

20.2.2. Caso de una memoria de tipo medio: una memoria con 8 K de RAM (en bloques integrados de 2 K), 4 K de ROM (también en bloques de 2 K), 2 registros de salida y 2 adaptadores de entrada; la memoria RAM al inicio del mapa de memoria y la ROM al final del mismo.

Los 10 elementos de memoria citados pueden ser reducidos a 8 elementos diferenciados ya que los registros de salida y los adaptadores de entrada pueden ser colocados sobre las mismas posiciones de memoria.

Los bloques de 2 K requieren 11 líneas para el direccionamiento de sus registros **a10 - a0** y para situar conjuntamente los cuatro circuitos integrados **RAM** se requieren dos líneas más **A11** y **A12**, a fin de que los cuatro bloques ocupen sectores sucesivos de memoria. Para seleccionar los 8 elementos diferenciados de memoria se requieren tres líneas de direccionamiento, dos de las cuales han de ser **A11** y **A12**, según lo anterior, y la tercera línea puede ser cualquiera de las otras tres superiores (por ejemplo **A15**).

Un decodificador de 8 líneas proporciona directamente la discriminación entre los bloques; dado que las líneas de habilitación de los mismos se activan con valor **0** ha de utilizarse un decodificador con salidas negadas y su entrada de habilitación se condiciona con la validación de la dirección **DIRV**.



Con ello puede asignarse el siguiente mapa de memoria:

RAM1	de	0000 0000 0000 0000	=	0 0 0 0 H
	hasta	0000 0111 1111 1111	=	0 7 F F H
RAM2	de	0000 1000 0000 0000	=	0 8 0 0 H
	hasta	0000 1111 1111 1111	=	0 F F F H
RAM3	de	0001 0000 0000 0000	=	1 0 0 0 H
	hasta	0001 0111 1111 1111	=	1 7 F F H
RAM4	de	0001 1000 0000 0000	=	1 8 0 0 H
	hasta	0001 1111 1111 1111	=	1 F F F H
RS-AE1	en	1000 0000 0000 0000	=	8 0 0 0 H
RS-AE2	en	1000 1000 0000 0000	=	8 8 0 0 H
ROM1	de	1111 0000 0000 0000	=	F 0 0 0 H
	hasta	1111 0111 1111 1111	=	F 7 F F H
ROM2	de	1111 1000 0000 0000	=	F 8 0 0 H
	hasta	1111 1111 1111 1111	=	F F F F H

Es claro que este mapa de memoria no es único, pues no se han utilizado todas las líneas para direccionar cada bloque; obsérvese que a las líneas **A14** y **A13**, que no son utilizadas se les asigna arbitrariamente valor **0** en el caso de los bloques **RAM** para situarlos al inicio de memoria y, en cambio, se les asigna valor **1** para los bloques **ROM** a fin de ubicarlos al final.

Cada registro, según este direccionamiento, ocupa múltiples posiciones de memoria: en el caso de los registros de los bloques **RAM** o **ROM**, cada registro ocupa 4 posiciones separadas de memoria, resultantes de dar valores a las líneas **A14** y **A13**; pero en el caso de los adaptadores de entrada/salida, cada uno de ellos ocupa 8K posiciones de memoria:

RS-AE1 en **1XX0 0XXX XXXX XXXX**
ocupa los sectores **8000 - 87FF**, **A000 - A7FF**, **C000 - C7FF** y **E000 - E7FF**

RS-AE2 en **1XX0 1XXX XXXX XXXX**
ocupa los sectores **8800 - 8FFF**, **A800 - AFFF**, **C800 - CFFF** y **E800 - EFFF**.

La conexión de las entradas de control de los diversos circuitos integrados que constituyen la memoria se completa en la forma siguiente:

RAM: La salida correspondiente del decodificador se conecta a la habilitación global $\overline{CE}$, mientras que las otras dos entradas de habilitación $\overline{OE}$ y $\overline{WE}$ han de conectarse a $\overline{R/W}$ y a $\overline{R/W} \cdot \overline{DATV}$, respectivamente; las entradas de direcciones se conectan a las correspondientes líneas del bus $a_{10} - a_0 = A_{10} - A_0$.

ROM: La salida del decodificador se conecta a la habilitación $\overline{CE}$ y la otra entrada de habilitación $\overline{OE}$ a $\overline{R/W}$; las entradas de direcciones se conectan a las correspondientes líneas del bus $a_{10} - a_0 = A_{10} - A_0$.

AE (adaptadores de entradas): Cada uno de ellos debe habilitarse cuando la línea del decodificador que le corresponde **L4** o **L5** se encuentra a **0** y la operación es la lectura $\overline{R/W} = 1$: $\overline{E} = \overline{L_i} \cdot \overline{R/W} = L_i + \overline{R/W}$.

RS (registros de salidas): Su habilitación debe tener lugar para $L_i = 0$, $\overline{R/W} = 0$ (operación de escritura) y **DATV** = **1** (dato válido):
 $\overline{E} = \overline{L_i} \cdot \overline{R/W} \cdot \overline{DATV} = L_i + \overline{R/W} \cdot \overline{DATV}$.

Resulta fácil incluir nuevos registros de salida o adaptadores de entrada en el esquema anterior; consideremos concretamente la forma de añadir un tercer registro de salida **RS3** a partir de la línea de habilitación de **RS1** (**L4**), desdoblándola en dos a través de su combinación con la primera línea de direccionamiento **A0**.

RS1 deberá habilitarse cuando **L4 = 0** y **A0 = 0**

$$\overline{E} = \overline{L_4} \cdot \overline{A_0} \cdot \overline{R/W} \cdot \overline{DATV} = L_4 + A_0 + \overline{R/W} \cdot \overline{DATV}$$

RS3 lo hará cuando **L4 = 0** y **A0 = 1**

$$\overline{E} = \overline{L_4} \cdot A_0 \cdot \overline{R/W} \cdot \overline{DATV} = L_4 + \overline{A_0} + \overline{R/W} \cdot \overline{DATV}$$

Con ello el registro **RS1** conserva su anterior dirección **8 0 0 0 H** y el registro añadido **RS3** queda situado en la siguiente **8 0 0 1 H** (**A0 = 1**).

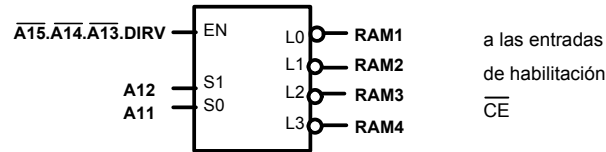
20.2.3. El mismo caso anterior de una memoria de tipo medio, pero con ocupación mínima del mapa de memoria

Consideremos los 8 elementos de memoria del caso anterior con el mismo mapa:

RAM (4)	de	0000 0000 0000 0000	=	0 0 0 0 H
	hasta	0001 1111 1111 1111	=	1 F F F H
RS-AE1	en	1000 0000 0000 0000	=	8 0 0 0 H
RS-AE2	en	1000 1000 0000 0000	=	8 8 0 0 H
ROM (2)	de	1111 0000 0000 0000	=	F 0 0 0 H
	hasta	1111 1111 1111 1111	=	F F F F H

Los 4 bloques RAM de 2 K pueden situarse al comienzo del mapa mediante un decodificador de 4 líneas de salida, con entradas **A11** y **A12**, habilitado con la función:

$$E = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \text{DIRV}$$



Cada adaptador de entrada/salida ocupa una sola posición de memoria y requiere, para ello, de las 16 líneas de direccionamiento en su habilitación:

$$E(\text{AE1}) (\text{posición de memoria } \mathbf{8\ 0\ 0\ 0\ H}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} \cdot R/W \cdot \text{DIRV}$$

$$E(\text{RS1}) (\text{misma posición de memoria } \mathbf{8\ 0\ 0\ 0\ H}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} \cdot R/W \cdot \text{DATV}$$

$$E(\text{AE2}) (\text{posición de memoria } \mathbf{8\ 8\ 0\ 0\ H}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} \cdot R/W \cdot \text{DIRV}$$

$$E(\text{RS2}) (\text{misma posición de memoria } \mathbf{8\ 8\ 0\ 0\ H}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \overline{A_3} \cdot \overline{A_2} \cdot \overline{A_1} \cdot \overline{A_0} \cdot R/W \cdot \text{DATV}$$

Y los dos bloques de memoria ROM para situarse al final del mapa de memoria:

$$E(\text{ROM1}) (\text{posiciones de memoria } \mathbf{1111\ 0--- ---}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

$$E(\text{ROM2}) (\text{posiciones de memoria } \mathbf{1111\ 1--- ---}) = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \text{DIRV}$$

Estas funciones de habilitación y, también, la habilitación del decodificador de las memorias RAM pueden ser programadas sobre un bloque **PAL**.

20.2.4. Posicionamiento de 4 adaptadores de periféricos: se desea situar 4 adaptadores PIA (cada uno de ellos tiene 4 registros y, por tanto, dos líneas de direccionamiento) a partir de la posición **B000H** del mapa.

Cada **PIA** requiere 2 líneas de direccionamiento **a1 a0** y para situar conjuntamente las cuatro se requieren dos líneas más **A3** y **A2**, a fin de que ocupen sectores sucesivos de memoria.

La ocupación de los 4 adaptadores será la siguiente:

1 0 1 1	0 0 0 0	0 0 0 0	0 0 - -	B000 – B003
1 0 1 1	0 0 0 0	0 0 0 0	0 1 - -	B004 – B007
1 0 1 1	0 0 0 0	0 0 0 0	1 0 - -	B008 – B00B
1 0 1 1	0 0 0 0	0 0 0 0	1 1 - -	B00C – B00F

Un decodificador de 4 líneas permite situar los cuatro adaptadores, tomando como entradas las líneas **A3** y **A2**; la habilitación del decodificador deberá ser:

$$E = \overline{A_{15}} \cdot \overline{A_{14}} \cdot \overline{A_{13}} \cdot \overline{A_{12}} \cdot \overline{A_{11}} \cdot \overline{A_{10}} \cdot \overline{A_9} \cdot \overline{A_8} \cdot \overline{A_7} \cdot \overline{A_6} \cdot \overline{A_5} \cdot \overline{A_4} \cdot \text{DIRV}$$

20.2.5. Placa de memoria RAM de 16K y direccionamiento versátil: se trata de diseñar una placa de 16K de memoria RAM con circuitos integrados de 2K, que pueda ser posicionada en cualquiera de los cuatro posibles sectores de 16 K (0000-3FFF, 4000-7FFF, 8000-BFFF, C000-FFFF).

Cada bloque **RAM** de 2 K requiere 11 líneas de direccionamiento **a10 - a0** y para agrupar los ocho bloques que configuran 16K se requieren tres líneas de direcciones más **A13 A12 A11**; quedan dos líneas **A15** y **A14** con las cuales puede situarse globalmente la placa en los cuatro sectores de memoria indicados.

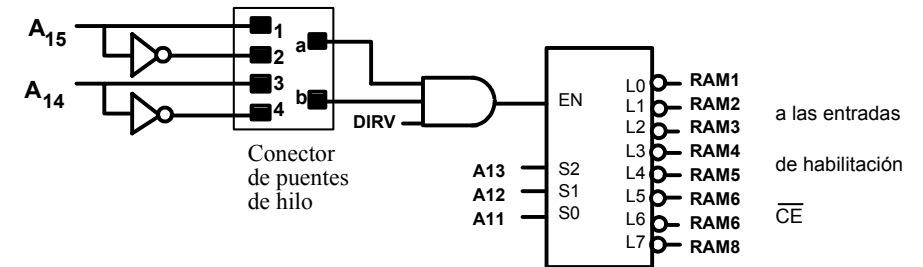
Un decodificador de 8 líneas permite discriminar entre los 8 bloques integrados **RAM**; la selección del sector en que se sitúa globalmente la placa se realiza mediante un conector de puentes de hilo, que permite condicionar la habilitación del decodificador con los valores de las líneas **A15** y **A14**:

$$A_{15} = 0 \text{ y } A_{14} = 0 : \mathbf{0000-3FFF}$$

$$A_{15} = 1 \text{ y } A_{14} = 0 : \mathbf{8000-BFFF}$$

$$A_{15} = 0 \text{ y } A_{14} = 1 : \mathbf{4000-7FFF}$$

$$A_{15} = 1 \text{ y } A_{14} = 1 : \mathbf{C000-FFFF}$$



La línea **DIRV** evita direccionamientos falsos y los puentes de hilo del conector determinan el sector de memoria en el que se sitúa globalmente la placa:

conexión **1-a** y **3-b** : **C000-FFFF** conexión **1-a** y **4-b** : **8000-BFFF**
 conexión **2-a** y **3-b** : **4000-7FFF** conexión **2-a** y **4-b** : **0000-3FFF**.

Las otras dos entradas de habilitación $\overline{OE}$ y $\overline{WE}$ han de conectarse a $\overline{R/W}$ y a $\overline{R/W} \cdot \text{DATV}$, respectivamente, y las entradas de direcciones a las correspondientes líneas del bus $a_{10} - a_0 = A_{10} - A_0$.

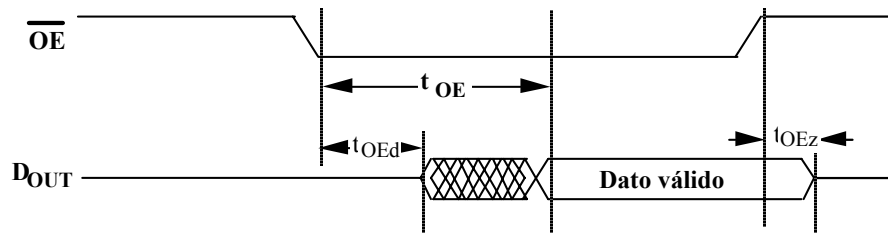
Esta placa de memoria contiene ocho circuitos integrados conectados a los buses y, en general, se utilizará junto con otras placas, de modo que será apreciable el número de integrados cargados sobre los buses; conviene incluir en cada placa adaptadores de bus que amplifiquen la intensidad de los mismos (véase apartado 20.4).

20.3. Ciclos de lectura y de escritura: tiempos de acceso

Lectura a través de un adaptador tri-estado

Los adaptadores tri-estado conectados sobre un bus mantienen normalmente sus líneas de salida en alta impedancia (desconexión) y requieren un pulso de habilitación de lectura $\overline{OE}$ para ejecutar tal operación.

A partir del momento en que se inicia dicho pulso de habilitación de lectura y con un cierto retraso sobre el mismo (tiempo de activación de las líneas de datos t_{OE_d}) las salidas dejan de estar en alta impedancia para transmitir, posteriormente (tiempo de habilitación de lectura t_{OE}) la palabra binaria presente en sus entradas. El pequeño intervalo existente entre los tiempos señalados ($t_{OE} - t_{OE_d}$) corresponde a un transitorio durante el cual las líneas de salida actúan como tales pero el dato que presentan no es válido (o bien los valores de las tensiones eléctricas no lo son).



Una vez finalizado el pulso de habilitación de lectura las líneas de salida tardan un cierto tiempo (tiempo de desactivación t_{OEz}) en pasar al estado de alta impedancia, durante el cual el dato permanece en dichas salidas; corresponde por tanto a un tiempo de permanencia de la salida.

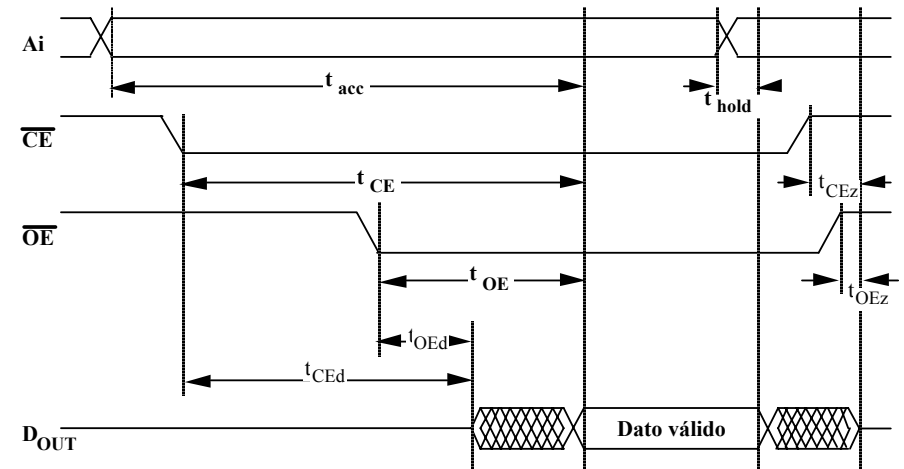
Ciclo de lectura en una memoria de acceso directo (RAM o ROM)

En el caso de una memoria de acceso directo, que contiene múltiples registros seleccionables por su número binario o dirección, la ejecución de una operación de lectura sobre uno de dichos registros requiere:

- seleccionar el registro a través de las líneas de direcciones A_i ,
- habilitar la memoria a través de la línea $\overline{CE}$
- y habilitar, también, la correspondiente operación de lectura $\overline{OE}$.

Por razones de protección, a fin de evitar inserciones (escrituras) falsas sobre los registros, la habilitación de escritura $\overline{WE}$ se mantiene a **1** permanentemente y sólo pasa a **0** en los momentos de escritura; a tal fin los procesadores mantienen siempre la línea $\overline{R/W}$ a **1**, salvo en las operaciones de escritura.

El diagrama de señales en un ciclo de lectura adopta la forma representada en la figura siguiente:



Para que aparezcan válidamente los datos de un registro en las líneas de entrada/salida D_i es preciso:

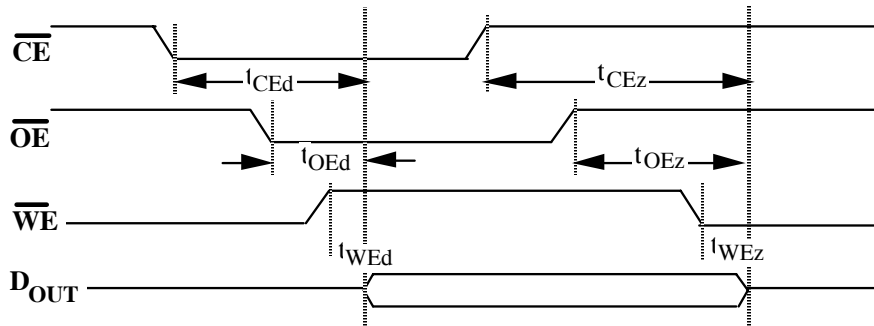
- establecer la dirección correspondiente a dicho registro en las líneas de direcciones a_i durante, al menos, un mínimo tiempo de acceso t_{acc} ,
- habilitar la memoria ($\overline{CE} = 0$) durante un intervalo superior al tiempo de habilitación t_{CE}
- y habilitar, también, las líneas de datos como salidas ($\overline{OE} = 0$) durante un intervalo superior al tiempo de habilitación de lectura t_{OE} .

El tiempo de acceso es el retraso que transcurre desde que se activan convenientemente las líneas de direccionamiento hasta que aparecen en las líneas de salida los datos válidos del registro direccionado, supuesto que la memoria se encuentra habilitada globalmente y que también lo esté la operación de lectura. Este tiempo es el más largo de todos debido al gran número de puertas que involucran los decodificadores de dirección del registro y, por ello, caracteriza la velocidad del circuito integrado.

Los tiempos de habilitación son los retrasos entre la activación de la correspondiente entrada de habilitación y la aparición de los datos en las líneas de datos, que pasan en este caso desde su estado de alta impedancia a actuar como salidas y a presentar en ellas el contenido del registro seleccionado, supuesta una dirección establecida previamente. Los datos permanecen en la salida un pequeño tiempo posterior al establecimiento de una nueva dirección (tiempo de permanencia t_{hold}).

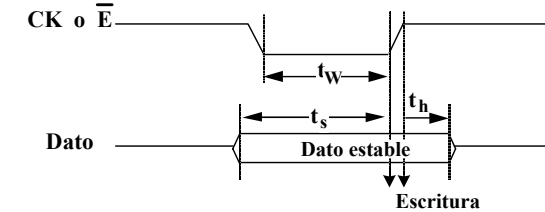
La figura anterior incluye, asimismo los tiempos de paso de las líneas de datos del estado de alta impedancia a su funcionamiento como salidas (t_{CEd} , $t_{OE d}$) y el paso contrario a alta impedancia (t_{CEz} , t_{OEz}); la activación de tales líneas como salidas requiere la habilitación de la pastilla $\overline{CE} = 0$ y de las salidas $\overline{OE} = 0$ y, también, la no habilitación de una operación de escritura $\overline{WE} = 1$.

La siguiente figura representa los tres tiempos involucrados en el paso de alta impedancia a salida de datos y en el paso inverso a alta impedancia:



Escritura en un registro

La operación de escritura sobre un registro requiere que la palabra binaria (dato) que va a ser escrita se encuentre presente en las líneas de entrada del mismo con una cierta anticipación (tiempo de preparación del dato: *setup* t_s) al momento de la escritura y que permanezca durante un pequeño tiempo posterior (tiempo de mantenimiento: *hold* t_h) a dicho momento.



En un registro síncrono el momento de ejecución de una operación de escritura viene determinado por el flanco activo del reloj **CK**, mientras que en un registro «habilitado por nivel» (*latch*) dicho momento coincide con el final del pulso de habilitación. En ambos casos, el «pulso de escritura» (es decir, el nivel previo al flanco activo del reloj o el pulso de habilitación) ha de tener una duración superior a un valor mínimo: tiempo de escritura t_w .

Uno de los dos tiempos referentes al dato (tiempo de preparación t_s o tiempo de mantenimiento t_h) puede ser nulo; de hecho existe un cierto compromiso entre ambos tiempos, de forma que, en el diseño de un registro, es posible disminuir uno de ellos a costa de aumentar el otro.

El respeto a los tiempos mínimos de anticipación, de mantenimiento del dato y de escritura asegura la correcta ejecución de la operación; la violación de tales tiempos puede dar lugar a una escritura incorrecta o a la permanencia del dato anterior en el registro.

Tras una operación de escritura sobre un registro, la palabra binaria escrita en él se presentará en sus líneas de salida con un pequeño retraso respecto al momento de escritura (tiempo de propagación t_p) y permanecerá en las salidas del registro hasta la siguiente operación de escritura.

Escritura en una memoria de acceso directo RAM

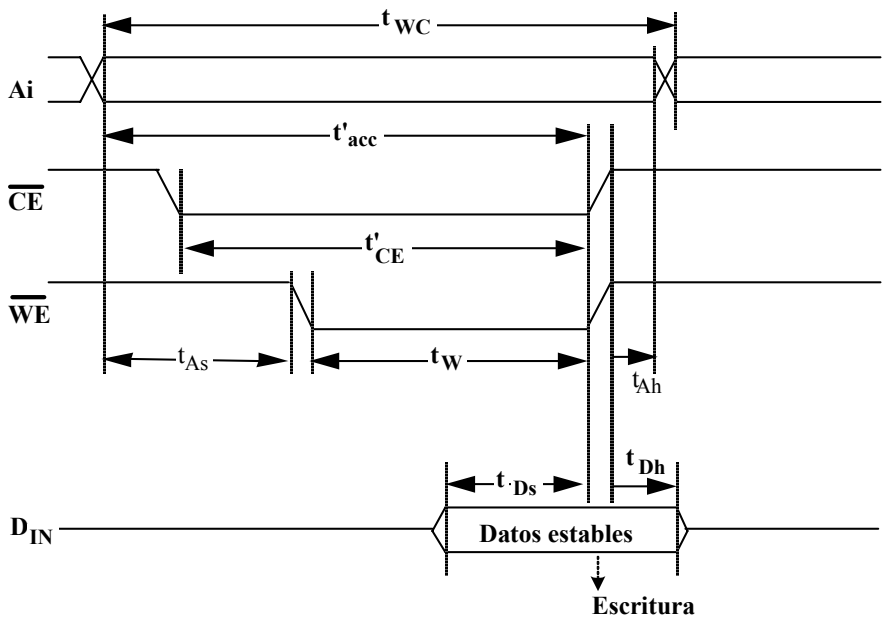
Para realizar una operación de escritura sobre un registro RAM se requiere:

- seleccionar dicho registro, a través de las líneas A_i , durante un intervalo de tiempo previo análogo al tiempo de acceso t'_{acc} ;
- habilitar la memoria ($\overline{CE} = 0$) durante un tiempo mínimo t'_{CE} ;
- ordenar la escritura a través de la línea $\overline{WE}$ ($\overline{WE} = 0$), durante un tiempo mínimo de escritura t_w ;
- y, además, el dato ha de estar presente en las líneas de entrada/salida de la memoria D_i cumpliendo los correspondientes tiempos de anticipación (*setup* t_{Ds}) y de mantenimiento (*hold* t_{Dh}).

El pulso de escritura puede actuar bien sobre la entrada de habilitación de escritura $\overline{WE}$ estando la memoria habilitada $\overline{CE} = 0$ o bien sobre la propia habilitación de la memoria $\overline{CE}$ estando la escritura habilitada $\overline{WE} = 0$.

El pulso de escritura controla la inserción de nuevo dato en la pastilla, de forma que el instante de escritura coincide con el flanco de subida del mismo; previamente ha de encontrarse el dato en las líneas de entrada/salida durante un intervalo temporal superior a t_{Ds} y debe mantenerse presente durante un pequeño intervalo posterior t_{Dh} ; todo ello para asegurar una escritura correcta.

La secuencia de tiempos para el ciclo de escritura es la siguiente:



En la escritura, el tiempo de acceso o selección de registro t'_{acc} es análogo al considerado en el ciclo de lectura (ligeramente inferior); dicho tiempo expresa el retraso entre la implantación de una dirección en las líneas de direccionamiento y el acceso a dicho registro por las líneas de entrada/salida.

El tiempo de escritura t_w es el intervalo mínimo de habilitación a través de la línea de selección de escritura. Los tiempos de preparación t_{Ds} y de mantenimiento del dato t_{Dh} aseguran la estabilidad del mismo en las líneas de datos en el momento de memorización o escritura.

Pero, además, la dirección correcta ha de establecerse previamente al pulso de escritura t_{As} y ha de permanecer posteriormente al mismo t_{Ah} para evitar que se produzca una escritura errónea en algún otro registro.

El tiempo global del ciclo de escritura t_{WC} suele ser análogo al tiempo de acceso t_{acc} del ciclo de lectura (que es también el tiempo mínimo posible para dicho ciclo de lectura t_{RC}); por ello, el tiempo de acceso t_{acc} caracteriza la memoria en cuanto a velocidad.

Los diversos tiempos dependen del circuito integrado específico y deben ser consultados en las correspondientes hojas de características; dentro de un mismo tipo de memoria integrada suelen ofrecerse diversas series con tiempos de acceso y, consiguientemente, velocidades de trabajo diferentes; por ello, al número que identifica al circuito integrado suele añadirse su tiempo de acceso.

Tiempos de acceso y velocidades de trabajo habituales son los siguientes:

a) Memorias «lentas»:

(en la segunda línea de datos se indican las velocidades de trabajo admisibles, suponiendo que los retrasos intermedios, procesador-memoria, en las líneas de buses, incluida la decodificación de dirección del bloque, son inferiores a 20 ns)

120 ns	100 ns	85 ns
6 MHz	8 MHz	10 MHz

b) Memorias «rápidas»:

(supuestos retrasos intermedios en las líneas de buses que no superen los 10 ns)

70 ns	55 ns	30 ns
12 MHz	15 MHz	25 MHz

c) Memorias ultrarrápidas:

(retrasos intermedios inferiores a 5 ns)

25 ns	15 ns	10 ns
33 MHz	50 MHz	66 MHz

20.4. Problemas relativos a los buses: adaptadores de bus

- La conexión de circuitos integrados o de placas (conjuntos de ellos) sobre los buses ha de respetar la velocidad de trabajo, asegurando que no se introducen retrasos importantes en las señales transmitidas a través de los buses; para ello:
- a) cualquier «circuito intermedio» (*interbus*) ha de ser suficientemente rápido
 - b) y ha de limitarse el número de circuitos integrados conectados a un bus, incluyendo, si es necesario, adaptadores de bus (*buffers*: amplificadores de intensidad).

Los circuitos que realizan la decodificación de las direcciones (para situar adecuadamente los registros en el mapa de memoria), así como el resto de circuitos de habilitación y control de los diversos elementos que configuran la memoria (y, también, los adaptadores de bus) se interponen en medio de las líneas de los buses entre el procesador y la memoria, retrasando la transmisión de señales en los buses.

Por ello, tales circuitos (*interbus*) han de tener tiempos de propagación muy pequeños: para ciclos de operación no inferiores a 200 ns (5 MHz) pueden ser suficientes tiempos de propagación inferiores a 25 ns, mientras que ciclos con duración inferior precisan tiempos no superiores a 15 ns que deberán bajar a menos de 10 ns cuando la duración del ciclo sea inferior a 40 ns (25 MHz).

Las diversas familias y series lógicas integradas presentan, para un mismo circuito integrado, tiempos de propagación muy diferentes; por ejemplo, para un decodificador de 8 líneas muy utilizado en la decodificación de direcciones (**74138**), los tiempos de propagación son:

HC	(CMOS)	t _p máximo = 50 ns	típico (25°C): 25 ns
LS	(TTL)	t _p máximo = 40 ns	típico (25°C): 20 ns
ALS	(TTL)	t _p máximo = 20 ns	típico (25°C): 10 ns
FAST	(TTL)	t _p máximo = 10 ns	típico (25°C): 6 ns
AC	(CMOS)	t _p máximo = 8 ns	típico (25°C): 5 ns.

La serie **ALS** es adecuada para velocidades de trabajo altas (no superiores a 20 MHz) ya que asegura retrasos en la propagación de las señales relativamente bajos (del orden de 10 ns) y, en cambio, su repercusión como carga sobre el bus es inferior a 0,1 mA (intensidad para entrada **0**).

Para velocidades mayores es necesario utilizar series de menor tiempo de propagación, como puede ser la serie **F** (FAST) bipolar (cuyo consumo global es relativamente elevado y su carga sobre el bus es superior a 0,5 mA para entrada **0**) o la serie **AC** en CMOS (que requiere un diseño muy cuidadoso de la placa de circuito impreso pues presenta graves problemas de ruido en su conmutación).

Como veremos a continuación, una alternativa mejor es utilizar la familia BiCMOS.

Los buses han de conectarse a múltiples circuitos integrados (*fan-out*), cuyo número en ocasiones puede ser muy alto; sin embargo, la intensidad suministrable por cada línea de bus suele ser relativamente baja, inferior a 10 mA.

Los circuitos integrados **TTL** requieren una intensidad de entrada no despreciable cuando su valor booleano es **0**: 0,2 mA para la serie LS, 0,1 para la serie ALS, 0,5 para la serie rápida FAST,...; por ello el número de circuitos integrados **TTL** conectados directamente sobre una línea de un bus debe ser muy pequeño.

De hecho los bloques de memoria (incluidos los adaptadores de periféricos) suelen ser integrados de tecnologías **MOS**, cuya repercusión sobre las líneas de los buses es capacitiva: cada entrada equivale a un condensador que hay que cargar o descargar en cada transición, cuyo valor suele estar entre 1 y 10 pF, siendo 3 pF el valor típico.

La carga o descarga de las múltiples entradas capacitivas que soporta un línea de un bus supone un aumento del tiempo de conmutación de las señales; supuesto un *fan-out* **n**, una intensidad de 10 mA y una tensión de alimentación de 5 V:

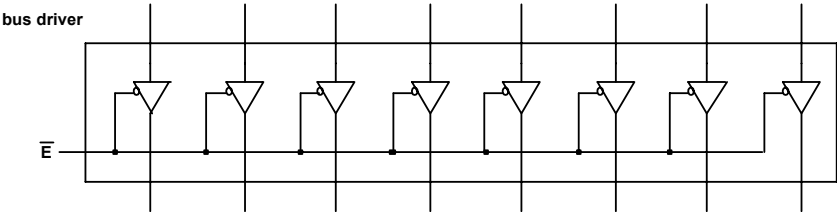
$\Delta Q = n.C_i.\Delta V \quad I.t = n.C_i.\Delta V \quad t = n.C_i.\Delta V/I \approx n.(3 \text{ pF}).(5 \text{ V})/(10 \text{ mA}) = 1,5.n \text{ ns}$

Para **n** = 10 el tiempo de conmutación de la línea de bus es de 15 ns, que suponen un cierto retraso en la propagación de las señales, pero para **n** = 100 el retraso sería del orden de 150 ns, sumamente alto.

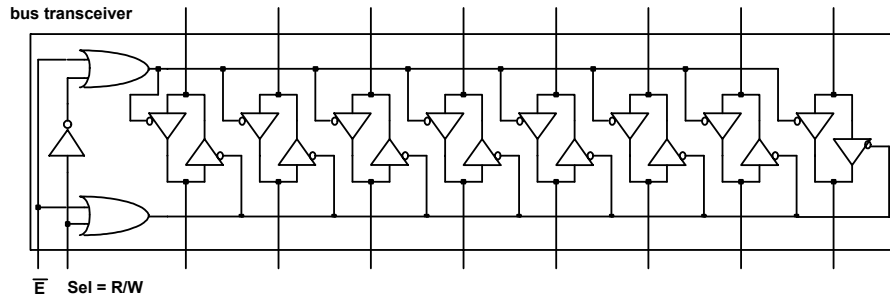
Tanto en el caso bipolar como en el MOS, cuando el número de circuitos a conectar sobre los buses es relativamente alto, interesa aumentar la capacidad de carga del bus, es decir, la intensidad suministrable por sus líneas. Esto se consigue mediante «adaptadores de bus» (*buffers*), que transmiten directamente el valor booleano presente en el bus y proporcionan en su salida intensidades relativamente altas.

Un adaptador de bus puede ser unidireccional *bus driver* (caso del bus de direcciones y del bus de control) o bidireccional *bus transceiver* (para el bus de datos); ambos se configuran mediante adaptadores triestado de alta intensidad de salida:

- un adaptador unidireccional *bus driver* está constituido por un conjunto de adaptadores triestado con una sola entrada de habilitación común para todos ellos (por ejemplo, el circuito integrado **74244** de 8 líneas)



- un adaptador bidireccional *bus transceiver* contiene un conjunto de parejas de adaptadores triestado en «antiparalelo» (para una operación de lectura debe ser habilitado uno de ellos y para la escritura ha de habilitarse el de la dirección opuesta), de forma que, además de la entrada de habilitación ha de tener una entrada común **Sel** que selecciona la dirección en que se produce la habilitación (por ejemplo, el circuito integrado **74245** de 8 líneas).



Como «circuitos intermedios» en los buses, los adaptadores deben presentar tiempos de propagación muy pequeños y, además, por su propia función, han de proporcionar intensidades de salida relativamente altas; constituyen una clase particular de circuitos digitales que han de ofrecer, a la vez, alta velocidad y alta intensidad de salida.

Para los buses de direcciones y de control han de utilizarse adaptadores unidireccionales (*bus driver*), que han de estar continuamente habilitados. Para el bus de datos serán necesarios adaptadores bidireccionales (*bus transceiver*), que han de habilitarse sólo cuando se accede al correspondiente sector del mapa de memoria (a la zona o «placa» de circuitos integrados a la que transmiten los datos); además, se necesita distinguir el sentido de transmisión de los datos, diferenciando entre la habilitación de lectura (hacia el procesador) y la habilitación de escritura (en sentido contrario).

La serie **ALS** proporciona adaptadores de bus con intensidades de salida de 15 mA y 24 mA para el **1** y el **0** lógicos, respectivamente, y tiempos de propagación inferiores a 10 ns en el caso unidireccional (*bus driver 74ALS244*) y a 20 ns en el caso bidireccional (*bus transceiver 74ALS245*).

Lógica interbús y tecnología BiCMOS

En el capítulo 10 (primer volumen) se ha introducido la denominación de «lógica interbús» (apartado 10.1.3) para referirse a los circuitos situados en medio de los buses; dentro de este tipo se encuentran tanto los adaptadores de bus como aquellos otros circuitos intermedios que, situados sobre los buses, han de conectar sus salidas a múltiples bloques de memoria.

La lógica interbús requiere tiempos de propagación muy reducidos para no retrasar las señales que transmiten los buses y altas intensidades de salida para poder conectarse a múltiples bloques. Para mejorar dichas características de alta velocidad e intensidad se ha desarrollado recientemente una nueva tecnología de integración aprovechando las ventajas de los transistores bipolares y las propias de la integración CMOS: tecnología **BiCMOS**.

En la integración **BiCMOS** la etapa de salida es de tipo TTL (*totem pole*), que permite conmutar rápidamente altas intensidades, mientras que las entradas y la lógica booleana son de tipo CMOS, aprovechando su reducida carga (como entradas) y sus mejores características de funcionalidad lógica y de conmutación; además, el consumo de las puertas BiCMOS en la situación de alta impedancia es muy reducido (mientras que el consumo de las puertas bipolares en alta impedancia es alto).

La tecnología **BiCMOS** se utiliza para circuitos integrados específicamente dirigidos a los buses (adaptadores de bus, decodificación de direcciones, control de lectura y escritura, etc.) y, también, para circuitos integrados mixtos (ASICs digitales con parte analógica).

La serie **ABT** (*Advanced BiCMOS Technology*) proporciona adaptadores de bus con intensidades de salida de 32 mA (para el **1**) y 64 mA (para el **0**) y tiempos de propagación inferiores a 5 ns (*bus driver 74ABT244*) y a 7 ns en el caso bidireccional (*bus transceiver 74ABT245*); existe asimismo una serie **BiCMOS** de baja tensión **LVT** para el intervalo 2,7-3,6 V, con tiempos de propagación inferiores a 4 ns.

A4 Apéndice al capítulo 16

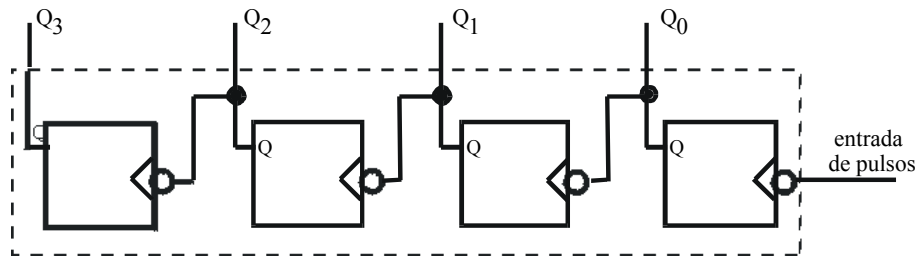
Contadores asíncronos

Si anulamos la entrada de habilitación de un biestable **T** resulta un biestable muy simple con una única entrada que corresponde a la entrada de reloj o entrada de pulsos y que cambia de estado con cada pulso que recibe; estos biestables utilizan como flanco activo el de bajada $\downarrow$ (el flanco con el que finaliza cada pulso).



La evolución de este biestable corresponde a la función $Q(t+1) = \overline{Q}(t)$ y equivale a un biestable **D** en el cual $D = \overline{Q}$ o a un biestable **JK** en el cual $J = K = 1$.

Este biestable permite la construcción de contadores en forma muy sencilla: mediante la conexión directa, en cadena, de n biestables se configura un contador módulo 2^n ; en el caso de 4 biestables será un contador módulo 16, que contará desde 0 hasta 15.

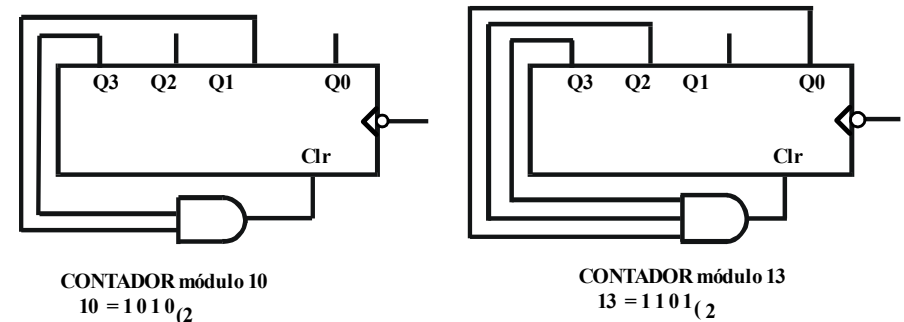


El primer biestable cambia de estado con cada pulso que le llega y cada uno de los biestables siguientes cambia cuando el biestable anterior pasa de 1 a 0 (al contar un biestable de 1 a 0 «se lleva una unidad» al siguiente biestable). Esto equivale a contar en binario desde el valor inicial, $000...00 = 0$, hasta el máximo que pueden contener los biestables, $111...11 = 2^m - 1$ (véase la figura de la página siguiente).

Q3	Q2	Q1	Q0	
0	0	0	0	situación inicial: 0
0	0	0	1	primer pulso
0	0	1	0	segundo pulso
0	0	1	1	tercer pulso
0	1	0	0	cuarto pulso
0	1	0	1	quinto pulso
1	1	0	1	13º pulso
1	1	1	0	14º pulso
1	1	1	1	15º pulso
0	0	0	0	16º pulso
0	0	0	1	17º pulso

Generalmente se incluye en los contadores una entrada de borrado asíncrono **Clr** (clear) común a todos los biestables que, al ser activada (**Clr = 1**), lleva el contador a cero; su adecuada utilización permite transformar un contador completo módulo 2^m en otro contador parcial módulo n , siendo n un número cualquiera inferior a 2^m .

Para configurar tal contador módulo n se parte de un contador de m biestables tal que $2^m > n$ y se hace que dicho contador se borre cuando aparezca en sus salidas el número n ; para ello basta llevar los dígitos con valor 1 del número n en binario a una puerta "y" que actúe sobre la entrada de borrado, de manera que al formarse el número n en las salidas se borre inmediatamente el contador y pase a 0: *contador módulo n*.



CONTADOR módulo 10
 $10 = 1010_2$

CONTADOR módulo 13
 $13 = 1101_2$

Así pues, para construir contadores módulo $n \neq 2^m$, basta añadir una puerta "y" que reciba las variables cuyo valor en n es **1**: término mínimo reducido correspondiente a n . Al no utilizar el término mínimo completo, el borrado no solamente se produce con n sino, también, con otros números superiores (con todos aquellos que tengan valor **1** en las mismas posiciones que n y en alguna otra más); siendo n el menor de ellos, los demás números que producen el borrado no pertenecen a la evolución del contador (el contador vuelve al estado inicial antes de llegar a ellos).

Aunque cada biestable individualmente es síncrono (si su entrada se conecta a la señal de reloj), su conexión en cadena hace que el contador resultante sea asíncrono, pues los biestables van conmutando sucesivamente y no en el mismo instante; aun más, en los contadores módulo $n \neq 2^m$ se presenta durante unos instantes el estado n , que se utiliza para borrar el contador y que no pertenece al conteo módulo n .

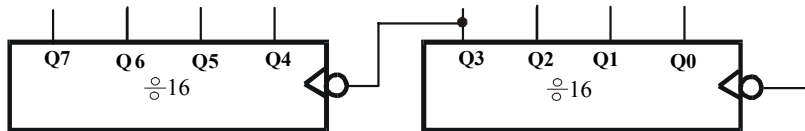
De forma que este tipo de configuración de contadores no es globalmente síncrona y ello por dos razones:

a) no conmutan todos los biestables a la vez, sino que lo hacen sucesivamente; la conmutación de cada biestable es provocada por el paso de **1** a **0** del biestable anterior y, por tanto, la transición entre dos estados que difieren en más de una variable de estado (por ejemplo el paso de 7 **0111** a 8 **1000**) recorre fugazmente toda una serie de estados intermedios (**0111**→**0110**→**0100**→**0000**→**1000**).

b) en el caso de contadores «parciales» (de que n no sea potencia entera de 2) la transición del estado $n-1$ al estado inicial 0 genera un estado adicional (el estado n) que ha de estar presente durante un intervalo de tiempo pequeño pero suficiente para producir el borrado de los biestables; tal estado n no pertenece a la evolución del contador módulo n y no es meramente un estado fugaz ya que tiene una actuación booleana concreta (borrado de los biestables); además, existe el peligro de que, por diferencias en los tiempos de propagación entre los biestables, no se complete el borrado de todos ellos y la transición se detenga (erróneamente) en un estado intermedio entre el 0 y el n , distinto de ambos.

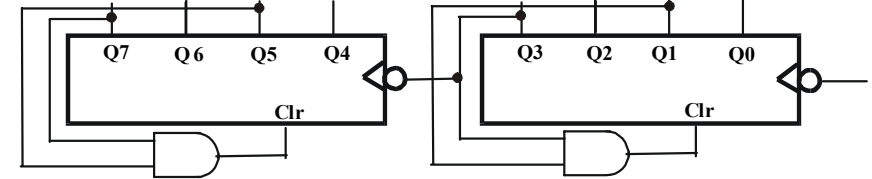
Conexión de contadores asíncronos

Los contadores asíncronos pueden conectarse entre sí uniendo la salida más significativa de cada uno de ellos con la entrada de pulsos del siguiente.



Si conectamos de esta forma dos contadores síncronos (uniendo la salida más significativa del primero con la entrada de reloj del segundo), el contador resultante será asíncrono ya que el segundo de los contadores no cambiará de estado con los pulsos de reloj del sistema, sino después de que haya cambiado de estado el primer contador.

En el caso de conexión de contadores «parciales» asíncronos un posible error de diseño sería llevar a dicha entrada el pulso de borrado del contador anterior; la duración de dicho pulso es tan pequeña que no asegura su reconocimiento como tal, a efectos de avanzar un unidad en el conteo (y además, no es necesario utilizarlo, pues se dispone del flanco de bajada del biestable más significativo del correspondiente contador).

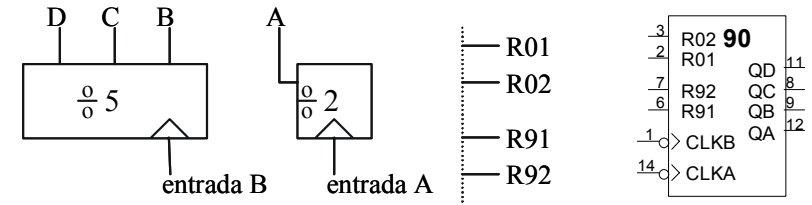


Contador módulo 100 (10 x 10)

Contador 7490

En los catálogos de circuitos integrados digitales existe un amplio número de contadores integrados asíncronos de tipos muy diversos.

Uno de ellos muy utilizado es el 7490, que contiene dos partes separadas: un biestable **QA** (módulo 2) y los otros tres biestables **QB** **QC** **QD** unidos configurando un contador módulo 5, de forma que para conformar un contador módulo 10 es necesario unir la salida del primer biestable a la entrada de pulsos de los otros tres. Dos entradas **R01** y **R02** permiten poner a **0** todos los biestables (cuando ambas se encuentran a **1**) y otras dos entradas **R91** y **R92** llevan al contador a 9 (**1001**).



La separación del primer biestable permite la configuración de contadores módulo 10 BCD (**DCBA**: entrada por A y la salida de A unida a la entrada de B) y bi-quinarios (**ADCB**: entrada por B y la salida de D unida a la entrada de A); estos últimos también dividen la frecuencia por 10 pero, además, proporcionan una onda cuadrada como resultado de dicha división (en la salida más significativa **QA**: 5 unidades de tiempo en **0** y otras 5 unidades de tiempo en **1**).

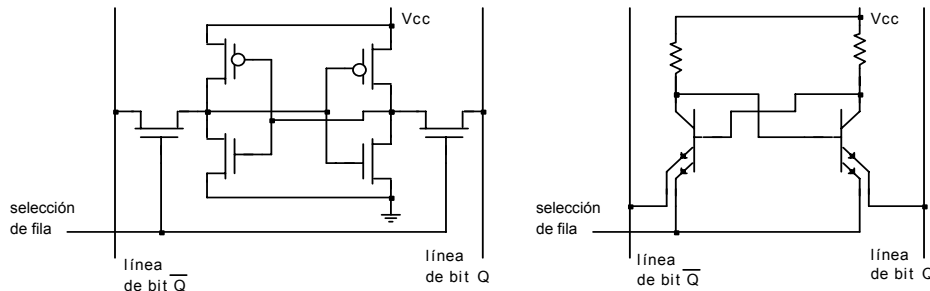
La existencia de dos entradas de borrado (configurando internamente una puerta "y" entre ellas) permite construir contadores módulo 3, módulo 6 y módulo 9 (cuyos términos mínimos reducidos tienen dos variables) sin necesidad de añadir ninguna puerta exterior. Además, este circuito ofrece de por sí contadores módulos 2 y 5 y es directo configurar contadores módulos 4 y 8 (cuyos términos mínimos reducidos tienen una sola variable). [Un contador módulo 7, con conteo un poco «extraño», puede conseguirse uniendo las salidas C y B a las entradas **R91** y **R92**, de manera que contará 0 1 2 3 4 5 9 0 1 2 ...]

A5 Apéndice al capítulo 19

Memorias de acceso directo dinámicas

Una memoria de acceso directo con k líneas de direccionamiento y n líneas de datos contiene $n \times 2^k$ biestables, número que puede ser considerablemente alto; por ello, la configuración electrónica de cada biestable ha de ser muy simple para minimizar el área de integración: basta un par de inversores en lazo cerrado (la salida de cada uno de ellos conectada a la entrada del otro).

La siguiente figura representa la configuración y conexiones de un biestable CMOS y de un biestable bipolar propios de un bloque RAM:



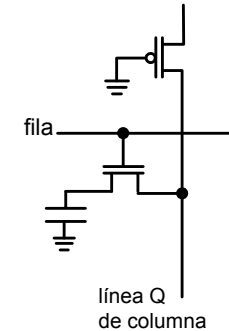
En el caso CMOS las dos salidas Q y $\bar{Q}$ del biestable se encuentran conectadas a través de sendos transistores de paso a las líneas que enlazan la correspondiente columna; cuando se selecciona la fila en que se encuentra dicho biestable los transistores de paso conducen y comunican las dos salidas del biestable con las líneas de columna, haciendo posible una operación de lectura o de escritura sobre él.

En el caso bipolar la conexión con las líneas de columna se establece a través de los emisores de los transistores; un segundo emisor conecta los transistores con la línea de selección de fila, de forma que cuando ésta se encuentra a **0** (no activada) dichos emisores conducen e impiden que lo hagan los que conectan con las líneas de columna.

Este tipo de memorias de acceso directo, con biestables que conservan la información a lo largo del tiempo (con tal de que se encuentren adecuadamente alimentados, V_{CC}), reciben el nombre de RAM estáticas: **SRAM**.

Existe una amplia oferta de circuitos integrados **SRAM** para cubrir las diferentes necesidades relativas a capacidad de memoria (número de registros) y a velocidad de trabajo (tiempos de acceso).

Para reducir el área de integración de los biestables y, consiguientemente, aumentar la capacidad de la memoria, se ha desarrollado otro tipo de memorias de acceso directo que utilizan un simple condensador, en lugar de un biestable, para almacenar y mantener el valor que deben conservar; se denominan memorias RAM «dinámicas»: **DRAM**.



Estas memorias incorporan condensadores análogos a las capacidades de puerta de los transistores MOS para retener sobre ellos la tensión correspondiente a cada bit memorizado (la capacidad de tales condensadores suele ser del orden de 0,1 pF).

La conservación de la tensión por efecto capacitivo no es permanente pues se producirá un lento proceso de descarga. Por ello, en estas memorias la información no permanece indefinidamente, sino que es necesario «refrescar» (leer y volver a escribir) cada cierto tiempo las palabras binarias almacenadas; de ahí el calificativo de «dinámicas» **DRAM**. En cambio, al disminuir el número de componentes y, también, por el hecho de utilizar una sola línea de columna, el área necesaria para su integración es muy inferior al de las memorias estáticas **SRAM**, lo que permite aumentar en gran medida el número de registros que contienen (la capacidad de la memoria).

Una operación de lectura de uno de los «biestables» (en realidad, condensadores) de estas memorias supone una redistribución de la carga del correspondiente condensador sobre la capacidad equivalente de la línea de columna a que se encuentra conectado; es decir, supone una modificación de la tensión «memorizada» de forma que no se alcanzan los valores propios del **0** y del **1** booleanos. Ello exige mayor complejidad en las operaciones de lectura y la posterior «reescritura» del registro leído para evitar la destrucción de su información.

En una memoria **DRAM** cada línea de columna se encuentra conectada a un amplificador de lectura/escritura de forma que una operación de lectura se ejecuta sobre todos los «biestables» (condensadores) de la fila correspondiente; posteriormente la dirección de columna seleccionará los bits que han de ser comunicados a las salidas de la memoria. En cada una de las «hojas» de la memoria, un «registro de fila», cuya longitud es igual al número de «biestables» que componen una fila, almacena el resultado de la lectura de todos ellos.

La lectura se efectúa a través de un comparador, cuyas dos entradas son «precargadas» a una tensión intermedia (entre las que corresponden al **0** y al **1** booleanos); al poner en comunicación al condensador de la línea seleccionada con una de las entradas de este comparador, la tensión de dicha entrada aumentará o disminuirá según que el valor almacenado sea un **1** o un **0**, de forma que el comparador será capaz de discriminar entre dichos valores. Cada comparador comunica su salida hacia uno de los biestables del «registro de fila»; una vez finalizada una operación de lectura, el contenido de dicho registro es «reescrito» sobre los condensadores que conforman la fila, de forma que se produce un «refresco» de toda la fila sobre la que se ha efectuado la lectura.

La operación de escritura se realiza en forma análoga: refiriéndonos a una de las «hojas» de la memoria, en primer lugar se ejecuta una lectura de la correspondiente fila sobre el «registro de fila», luego se actualiza la porción de dicho registro que corresponde a la columna sobre la que se desea escribir y, por último, se «reescribe» el contenido del «registro de fila» sobre la fila seleccionada.

El refresco de toda la memoria se consigue ejecutando una operación de lectura sobre cada una de las filas que componen la memoria. Generalmente las memorias **DRAM** se organizan en «hojas» con matrices de 256 x 256 «biestables» (64K) o de 512 x 512 «biestables» (256K = 1/4 Mega); por lo cual el «refresco» supone, respectivamente, 256 ó 512 operaciones de lectura sobre la memoria.

Por lo general las memorias **DRAM** disponibles actualmente conservan la información por períodos superiores a 8 ms, que son el intervalo típico de «refresco». Supuesto que cada operación de lectura precise 1 µs, para «refrescar» toda la memoria se necesitarán 256 µs ó 512 µs según el número de sus filas; lo cual supone, en relación con el intervalo de 8 ms, un 3 % y un 6 % del tiempo total, respectivamente.

El manejo de las memorias **DRAM** integradas se complica aún más por cuanto que, para reducir el número de sus terminales (*pines*), suelen recibir la dirección de fila y la dirección de columna por los mismos terminales; una línea de validación de la dirección de fila $\overline{RAS}$ y otra de validación de la dirección de columna $\overline{CAS}$ gestionan el almacenamiento de dichas direcciones en sendos registros en el inicio de cada operación de lectura o escritura.

De forma que una operación de lectura desarrolla la siguiente secuencia:

- 0) situación al inicio del ciclo $\overline{RAS} = 1$ y $\overline{CAS} = 1$
- 1) $\overline{RAS} = 1$: **precarga** de los comparadores
- 2) $\overline{RAS} = \downarrow$: almacenamiento de la **dirección de fila**
- 3) $\overline{RAS} = 0$: lectura de toda la fila de «biestables»
sobre el «registro de fila»
- 4) $\overline{CAS} = \downarrow$: almacenamiento de la **dirección de columna**
- 5) $\overline{CAS} = 0$: salida del correspondiente dato (**lectura**)

- 6) $\overline{RAS} = \uparrow$: escritura del «registro de fila»
sobre la fila de «biestables» (**refresco**)
- 7) $\overline{RAS} = 1$: fin del ciclo de operación (nueva precarga).

Como puede apreciarse la línea $\overline{RAS}$ es la que controla realmente las «filas de condensadores», determinando su lectura $\overline{RAS} = 0$ y su escritura $\overline{RAS} = 1$; asimismo con su flanco negativo dicha línea memoriza la dirección de la fila sobre la que se actúa. La línea $\overline{CAS}$ actúa como habilitación de salidas, determinando la presentación de un dato $\overline{CAS} = 0$ o, alternativamente, el estado de alta impedancia $\overline{CAS} = 1$; con su flanco negativo memoriza la dirección de la columna.

Una operación de escritura coincide con la secuencia anterior salvo en:

- 4) $\overline{CAS} = \downarrow$ con $\overline{WE} = 0$: almacenamiento de la dirección de columna
y recepción del dato desde las líneas de entrada
- 5) $\overline{CAS} = 0$ con $\overline{WE} = 0$: almacenamiento del dato en el «registro de fila»

en esta quinta etapa el dato que había sido recibido por las líneas de entrada en la etapa anterior es almacenado en el «registro de fila», en la posición que corresponda a la columna seleccionada y en la etapa siguiente el «registro de fila» es escrito en la fila seleccionada.

Cada operación de lectura o de escritura produce el «refresco» de la correspondiente fila de la memoria. Cuando lo único que se pretende es refrescar la memoria no es necesario referirse a las columnas ni utilizar la correspondiente línea de validación $\overline{CAS}$; basta conmutar la línea de validación de la dirección de fila $\overline{RAS}$, presentando en cada uno de sus flancos negativos la dirección de una de las filas de la memoria.

Los cronogramas correspondientes a los ciclos de lectura y de escritura de las memorias **DRAM** incluyen medio centenar de tiempos diferenciados (acceso, anticipación, mantenimiento,...). La complejidad de tales operaciones y la necesidad de producir cíclicamente un «refresco» global de la memoria han motivado la aparición de **controladores de DRAM** integrados que logran que, desde el lado del procesador, las operaciones de lectura y escritura se ejecuten como si fueran memorias **SRAM** y, a la vez, aseguran que se realice el «refresco» de la memoria con adecuada periodicidad.

En muchas memorias dinámicas, se incluye en el propio bloque integrado el circuito de refresco de la memoria (un simple contador que controla la lectura de sus filas, operando internamente cuando la memoria no se encuentra habilitada); de esta forma, vistas desde fuera, se comportan como si fueran estáticas: **pseudo static RAM**.

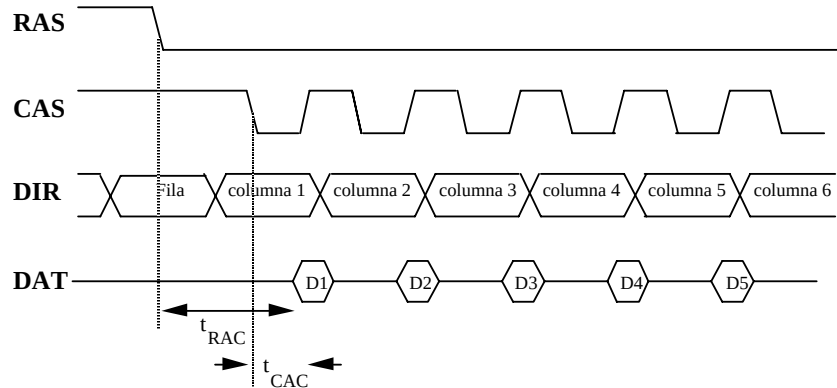
La velocidad de acceso a una memoria **DRAM** aumenta cuando se realizan operaciones en una misma fila, es decir, cuando no se modifica la dirección de fila sino solamente la de columna; en tal sentido se considera que los «registros» situados en una misma fila se encuentran en una página única: operación en modo de página.

Cuando se producen varias operaciones seguidas sobre la misma página, solamente es necesario que la primera de ellas ejecute un ciclo normal de lectura o escritura; para los siguientes accesos basta modificar la dirección de columna y conmutar la correspondiente línea de validación **CAS**, sin necesidad de gastar tiempo en las etapas relativas a la línea **RAS**.

Este tipo de bloques **DRAM** se denomina **FPM** (*fast page mode*), ya que accesos sucesivos a una misma página requieren tiempos de ciclo inferiores al primero de ellos: una vez determinada la página, basta enviar las sucesivas direcciones de las columnas sobre las que se ha de efectuar la operación.

Los bloques **DRAM FPM** de alta velocidad (y alta capacidad de memoria) suelen tener tiempos de acceso de 70 ns que se reducen a 40 ns cuando las siguientes operaciones se realizan sobre la misma página. [Como memorias RAM de los computadores se utilizan en placas con relojes de hasta 66 MHz, pero, en tal caso, requieren 5 ciclos de reloj para el primer acceso a página y 3 ciclos para los accesos siguientes.]

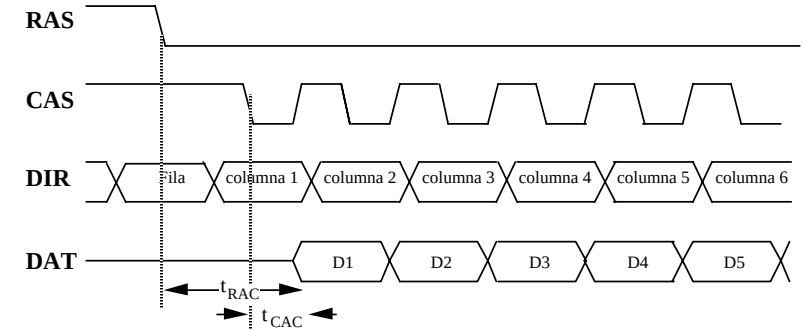
La lectura limita la velocidad de trabajo en mayor medida que la escritura, ya que en esta segunda el procesador mantiene el dato a escribir en el bus durante todo el ciclo de escritura; por ello, las siguientes figuras se refieren a operaciones de lectura sobre los bloques **DRAM**.

Operaciones sucesivas de lectura sobre una página DRAM **FPM**

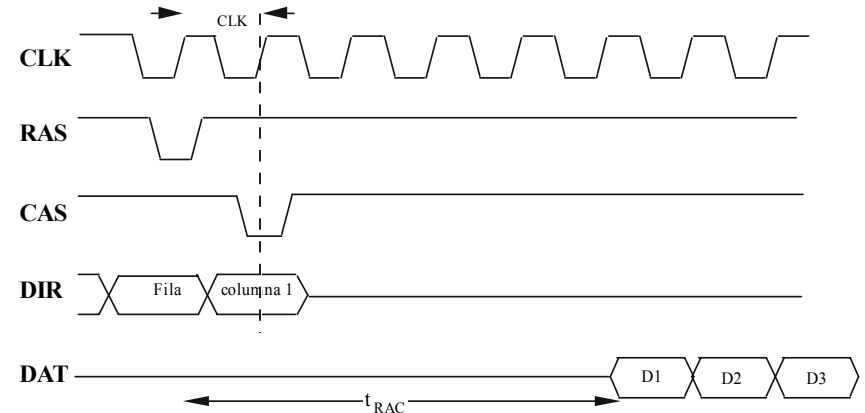
Las operaciones de lectura sobre estas memorias presentan el inconveniente de que la salida de dato es habilitada por la línea **CAS**, de manera que se interrumpe después de que dicha línea pasa a valor 1 (y, en lecturas sucesivas, hay intervalos de tiempo muerto en los cuales no hay salida de ningún dato correcto).

Para aumentar la velocidad de trabajo (en relación con la lectura de registros en la misma página) se ha desarrollado un tipo de memorias en que el dato se mantiene en las salidas hasta que, como resultado de una nueva operación de lectura, se presenta el dato siguiente (es decir, el dato de salida se mantiene durante el intervalo en que $\overline{\text{CAS}} = 1$); este tipo de **DRAM** se denomina **EDO** (*extended data output*).

De esta forma, los tiempos de acceso para lecturas sucesivas en una misma página se reducen: de 60 ns en el primer acceso se pasa a 25 ns en los siguientes. [Se utilizan en placas de computadores con relojes de hasta 75 MHz, reservando 5 ciclos de reloj para el primer acceso a página y 2 ciclos para los accesos siguientes.]

Operaciones sucesivas de lectura sobre una página DRAM **EDO**

Aún se consiguen velocidades mayores cuando se trata de operar sobre registros sucesivos de una misma página; para ello el registro de dirección de columna se configura en forma de contador con una entrada de reloj que permite incrementar sucesiva y rápidamente la columna: memorias dinámicas síncronas **SDRAM** (*synchronous DRAM*). Los tiempos de accesos sucesivos en una misma página son aún menores: de 40 ns en primer acceso se baja a 8 ns en los siguientes. [Se utilizan relojes de hasta 133 MHz, 5 ciclos para el primer acceso y 1 ciclo para los siguientes.]

Operaciones sucesivas de lectura sobre una página **SDRAM**

T4 TEMPORIZADORES: OSCILADORES Y MONOSTABLES

T4.1. Comportamiento circuital de los condensadores

T4.2. Monostables

T4.3. Circuitos astables

T4.4. El circuito temporizador 555

T4.5. Osciladores de precisión: cristal de cuarzo

T4.6. Acomodación de pulsos externos

Este tema se dedica a la configuración y diseño de circuitos auxiliares que tienen que ver con el tiempo, con la delimitación de intervalos de tiempo: circuitos monostables que proporcionan un pulso con la anchura temporal que interese y osciladores astables que generan una señal de frecuencia fija.

El tiempo es una variable necesaria en muchos circuitos digitales: se trata de disponer de intervalos temporales de una duración dada, bien en forma de pulsos individuales producidos a partir de una señal de disparo (monostables) o bien en forma de señal repetitiva cuyos períodos determinan unidades de tiempo sucesivas (astables). El primer caso sirve para controlar la duración de un proceso, mientras que el segundo proporciona ondas de sincronismo o de reloj (imprescindibles en los sistemas síncronos).

En ambos casos hay que delimitar la duración temporal de intervalos, conforme al valor deseado, mediante pulsos cuya anchura o cuyo período de repetición se ajusten a dicho valor.

La carga o descarga de un condensador a través de una resistencia proporciona una manera sencilla para «fijar» tiempos: el condensador recorre una exponencial y, tomando un intervalo de la misma (entre dos tensiones V_1 y V_2), tardará en recorrerlo un tiempo determinado. De esta forma, tanto la duración del pulso de un circuito monostable como la del período de un astable pueden controlarse mediante un circuito RC.

A veces, sobre todo en la señal de reloj de sistemas síncronos, interesa mayor precisión de la que puede obtenerse con circuitos RC: el empleo de cristales de cuarzo, con frecuencias de resonancia sumamente precisas, permite construir osciladores apropiados.

Comienza este capítulo repasando el comportamiento circuital de un condensador en una red RC, para utilizarlo, luego, en circuitos monostables y astables; se considera, también, la configuración de los monostables digitales integrados y la del temporizador típico 555.

Asimismo se describe la utilización de cristales de cuarzo y la configuración de los correspondientes osciladores de precisión. Y se incluye un último apartado sobre la adaptación de pulsos externos, acomodándolos a los niveles de tensión y a la verticalidad de los flancos propios de los sistemas digitales.

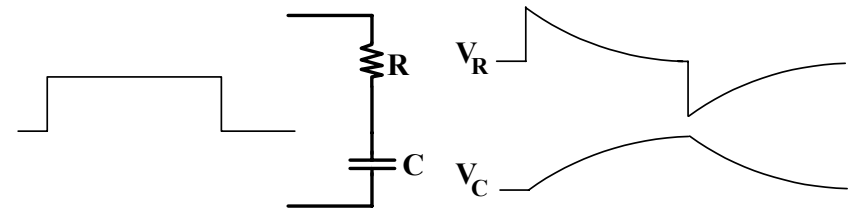
T4.1. Comportamiento circuital de los condensadores

Un condensador es un «depósito» capaz de almacenar carga eléctrica; la cantidad de carga almacenada determina la tensión del condensador: $q = C \cdot V$

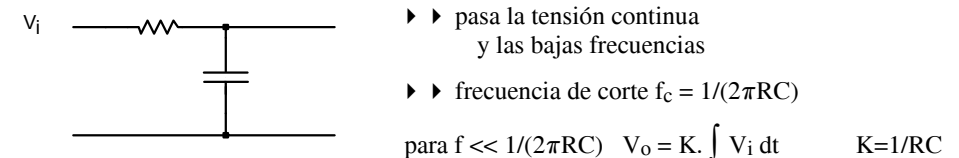
La cantidad de carga en un condensador no puede modificarse «instantáneamente», sino a través de un proceso de carga o de descarga. Por tanto, la tensión de un condensador no puede variar bruscamente sino a través de las correspondientes funciones de carga y de descarga, que serán exponenciales si se produce a través de una resistencia.

Consideremos un circuito RC, una resistencia y un condensador en serie:

- Al aplicar una tensión continua a un circuito RC, tras el correspondiente transitorio, toda la tensión continua quedará aplicada sobre el condensador: un condensador es un circuito abierto para tensión continua, es como si el condensador no estuviera presente para tal tensión (la tensión continua sobre la resistencia será nula).
- Cualquier variación brusca de una tensión aplicada al circuito RC se proyecta de inmediato sobre la resistencia, después de lo cual el condensador desarrollará el correspondiente proceso de carga o descarga.



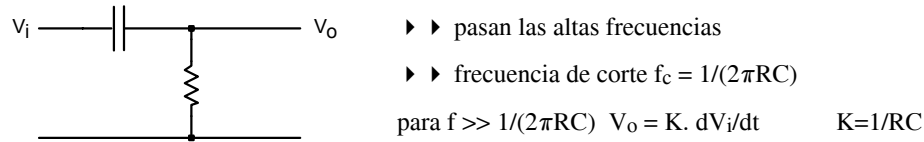
- Un circuito RC cuya salida se toma sobre el condensador «suaviza» las tensiones que recibe, se comporta como un integrador: filtro pasa-baja.



En los circuitos digitales se emplean «condensadores de desacoplo» en paralelo con la tensión de alimentación, situados junto a los circuitos integrados y muy próximos a sus terminales de alimentación, con una doble utilidad:

- por una parte, suministran los «picos» de intensidad que se requieren en las conmutaciones, evitando que produzcan transitorios de tensión sobre las líneas de alimentación (a causa de la autoinducción que presentan)
- y, también, junto con las inductancias de dichas líneas de alimentación, conforman filtros pasa-baja que impiden el paso de los transitorios de alta frecuencia presentes en las mismas.

- En cambio, si la salida del circuito RC se toma sobre la resistencia, se refuerzan las variaciones de la tensión recibidas, el circuito se comporta como un diferenciador o derivador: filtro pasa-alta.



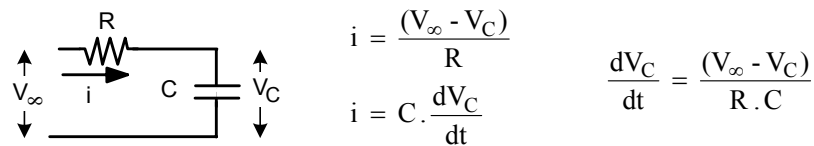
- Al condensador le lleva su tiempo cargarse o descargarse, siendo así que para responder a las variaciones de la tensión de entrada, el condensador (en un circuito RC) ha de ejecutar el correspondiente proceso de carga o de descarga.

Supuesto que la tensión de entrada al circuito RC varíe mediante un escalón, de un valor V_a a otro V_b (ambos de tensión continua), el condensador pasará «suavemente» de tener una tensión V_a a otra V_b y el correspondiente proceso de carga o descarga será exponencial con constante de tiempo $\tau = RC$. La constante de tiempo expresa la velocidad de variación de las exponenciales.

Téngase en cuenta que la constante de tiempo de un circuito RC no es el tiempo que el condensador tarda en cargarse o descargarse (que matemáticamente es infinito, ya que es una curva asintótica), sino el tiempo en que se recorre el 63 % del intervalo de carga o descarga: $1 - 1/e = 1 - 1/2,72 = 0,63$. En el caso de la descarga de un condensador desde una tensión V , una constante de tiempo es el tiempo en que la tensión del condensador disminuye desde V hasta el valor $V/e = 0,37 \cdot V$.

En dos constantes de tiempo se recorre el 86 % del intervalo de carga o descarga, en tres el 95 % y en cuatro constantes de tiempo se recorre el 98 %, porcentaje que, en la práctica, equivale a completar el proceso de carga o de descarga.

Los temporizadores aprovechan la función de carga o descarga de un condensador a través de una resistencia para determinar intervalos de tiempo de duración prefijada Δt : la ecuación de carga o descarga (circuito RC conectado a una tensión V_∞) es:



exponencial: ecuación diferencial cuya solución es de tipo $V_C = A + B \cdot e^{-t/RC}$,

obteniéndose los valores de A y B a través de las condiciones de contorno:

para $t = 0$ $V_C = A + B = V_{\text{inicial}}$;

para $t = \infty$ $V_C = A = V_\infty$ con lo cual $V_C = V_\infty - (V_\infty - V_{\text{inicial}}) \cdot e^{-t/RC}$.

(Se utiliza la notación V_∞ para la tensión aplicada al circuito RC para destacar que tal es el valor hacia el cual tiende la tensión del condensador: el condensador adquiere la tensión V_∞ al cabo de un tiempo suficientemente grande).

Si el proceso de carga o descarga se realiza hasta alcanzar la tensión V_{final} , su duración Δt será:

$$V_{\text{final}} = V_\infty - (V_\infty - V_{\text{inicial}}) \cdot e^{-\Delta t/RC}$$

$$(V_\infty - V_{\text{inicial}}) \cdot e^{-\Delta t/RC} = V_\infty - V_{\text{final}} ; \quad \Delta t = R \cdot C \cdot \ln \frac{V_\infty - V_{\text{inicial}}}{V_\infty - V_{\text{final}}}$$

En la carga o descarga de un condensador C hacia una tensión V_∞ a través de una resistencia R , el intervalo de tiempo Δt en que la tensión del condensador pasa de un valor inicial V_{ini} a un valor final V_{fin} será:

$$\Delta t = R \cdot C \cdot \ln \frac{V_\infty - V_{\text{ini}}}{V_\infty - V_{\text{fin}}}$$

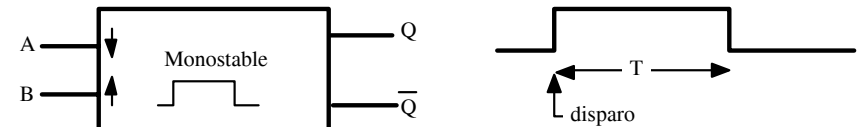
T4.2. Monostables

Un *monostable* (o temporizador) es un bloque digital con un estado estable **0** y otro estado inestable **1**; el monostable pasa a estado **1** cuando se produce su disparo y permanece en dicho estado durante un intervalo de tiempo constante T .

Un *monostable* produce un pulso de duración T ; para ello ha de ser disparado a través de sus correspondientes entradas: normalmente los monostables integrados presentan dos entradas de disparo, una de ellas **A** se activa con bajadas $\downarrow$ (paso de **1** a **0** en dicha entrada) y la otra **B** se activa con subidas $\uparrow$ (paso de **0** a **1** en la misma), existiendo dos posibilidades de disparo definidas por las condiciones siguientes:

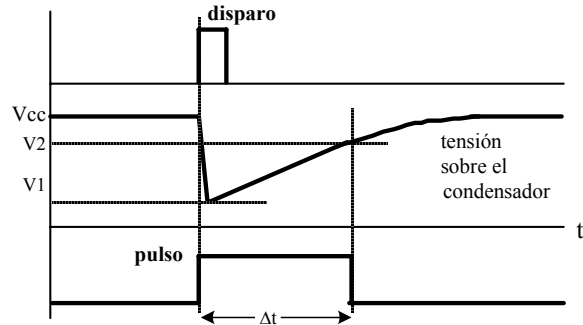
- disparo por A : $A = \downarrow$ y $B = 1$
- disparo por B : $A = 0$ y $B = \uparrow$.

El monostable en reposo se encuentra a **0**; en el disparo pasa a **1** y permanece en dicho estado durante un tiempo T prefijado (generalmente dicho tiempo se fija mediante un circuito RC externo); es, pues, un *temporizador* que se activa durante tiempos de duración prefijada o, lo que es lo mismo, produce pulsos de una anchura temporal dada.



Se dice que el monostable es *redisparable* si al producirse un nuevo disparo durante su pulso activo prolonga la duración del pulso durante un nuevo intervalo de tiempo T ; en cambio, un monostable *no redisparable* finaliza siempre sus pulsos cuando éstos han alcanzado la duración T prefijada.

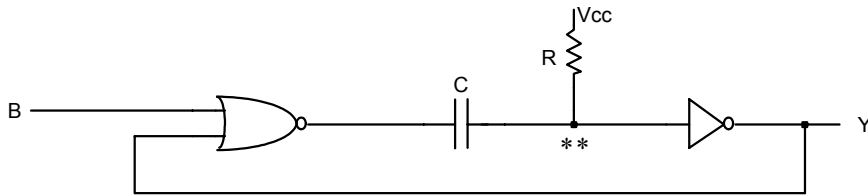
Los monostables suelen utilizar un circuito RC conectado a la tensión de alimentación: el condensador se encuentra inicialmente cargado a la tensión de alimentación V_{CC} y, en el momento del disparo del monostable, se fuerza una descarga rápida hasta una tensión V_1 a partir de la cual el condensador se carga a través de la resistencia hasta alcanzar una tensión V_2 que determina el final del pulso.



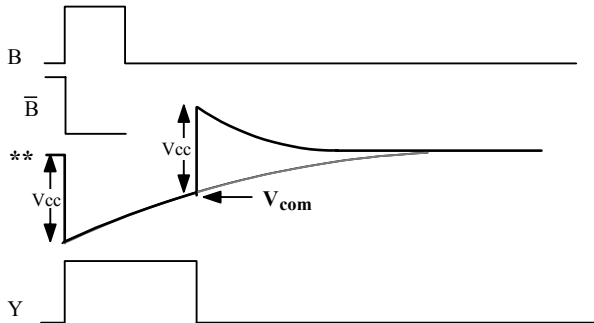
La anchura del pulso Δt viene dada por el tiempo de carga entre las tensiones V_1 y V_2 , habida cuenta de que el tiempo de descarga hasta V_1 es despreciable frente a la carga a través de R :

$$\Delta t = R.C . \ln \frac{V_{\infty} - V_{ini}}{V_{\infty} - V_{fin}} = R.C . \ln \frac{V_{CC} - V_1}{V_{CC} - V_2}.$$

La figura siguiente representa un monostable sencillo que se dispara con flancos de subida en su entrada **B**:



Las formas de onda en los nudos de este circuito son las siguientes, siendo V_{com} la tensión de conmutación del inversor (y supuesto $V_{OH} = V_{CC}$ y $V_{OL} = 0$ V):

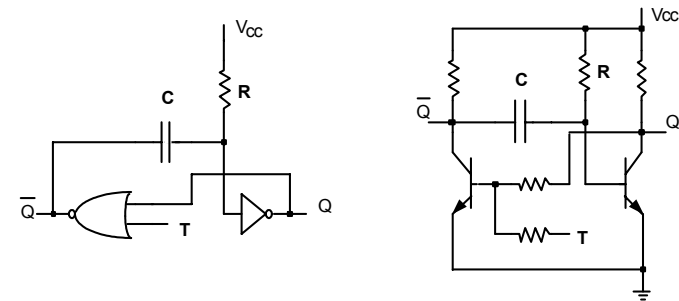


La realimentación sobre la puerta "o-negada" (*Nor*) impide que la bajada del pulso de disparo en la entrada **B** se transmita (invertida) al condensador e interrumpa el pulso de salida.

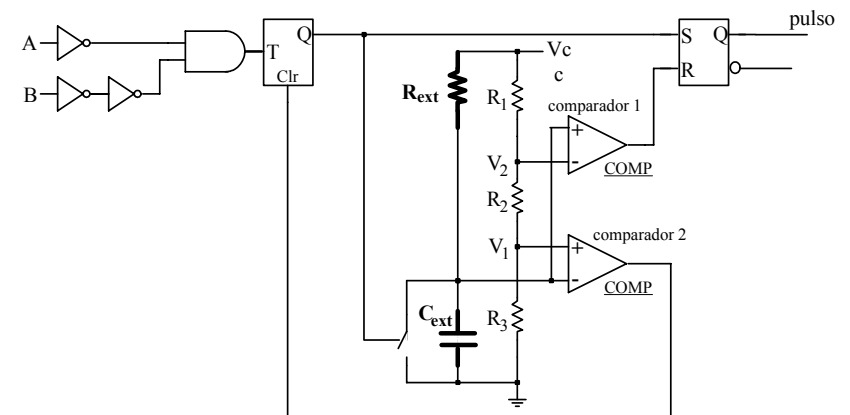
Las puertas integradas CMOS (serie HC) tienen, por lo general, su tensión de conmutación V_{com} aproximadamente a mitad de la de alimentación:

$$\Delta t = R.C . \ln \frac{V_{CC} - 0}{V_{CC} - V_{com}} = R.C . \ln \frac{V_{CC}}{V_{CC} - (V_{CC}/2)} = R.C . \ln 2 \approx 0,7 . R.C$$

Este monostable puede ser construido empleando dos puertas "o-negada" (*Nor*) integradas (74HC02). También puede ser construido con transistores discretos según la figura siguiente (con la ventaja de que, en este caso, pueden utilizarse tensiones de alimentación y, en consecuencia, tensiones de salida más elevadas).



Los monostables integrados suelen utilizar un esquema del siguiente tipo:



La entrada de disparo **A** actúa con bajadas siempre que **B** = 1 mientras que la entrada **B** lo hace con subidas cuando **A** = 0; en ambos casos se dispara el biestable **T** (que causa la descarga rápida del condensador hasta que alcanza la tensión V_1) y se marca el biestable **RS** que proporciona el pulso de salida. El conmutador que descarga al condensador puede ser realizado mediante un simple transistor NMOS (o un NPN).

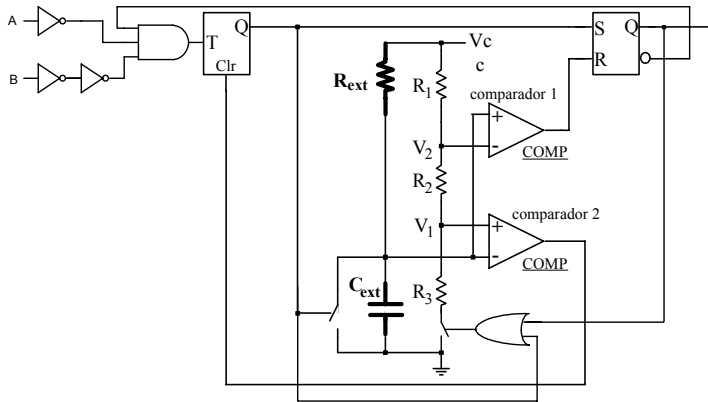
Dos comparadores de tensión detectan el cruce de la tensión del condensador con V_1 y V_2 ; el condensador se descarga hasta que su tensión es V_1 (momento en que el primer comparador borra al biestable **T** y finaliza la descarga) y, luego, se carga hasta V_2 (en que el segundo comparador borra al biestable **RS** y finaliza el pulso de salida).

En los monostables integrados CMOS (serie HC) suele hacerse $R_2=2.R_1=2.R_3$ con lo cual $V_1 = V_{CC}/4$, $V_2 = 3V_{CC}/4$ y la anchura de pulso: $\Delta t = R.C. \ln 3 \approx 1,1 RC$.

El monostable representado en la figura anterior es redispensible, es decir, si durante el intervalo temporal que corresponde a un pulso vuelve a actuar el disparo, el condensador se descarga de nuevo hasta la tensión V_1 y el pulso se prolonga durante un intervalo igual a la anchura de pulso Δt .

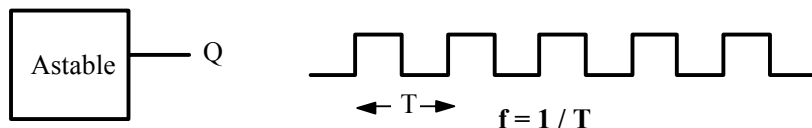
Puede evitarse el redisparo añadiendo a la puerta "y" que dispara el biestable **T** una entrada conectada a la salida negada del monostable; de esta forma, durante el pulso dicha puerta "y" se encuentra inhibida y, con ella, el disparo del monostable.

La figura siguiente muestra un monostable no redispensible; esta figura incluye, asimismo, un conmutador que evita el consumo de intensidad a través de la red de resistencias fuera de los intervalos correspondientes al pulso.

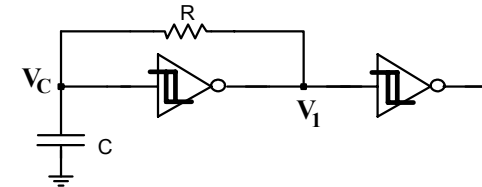


T4.3. Circuitos astables

Un *astable* (u oscilador digital) es un bloque que no tiene ningún estado estable sino que conmuta sucesivamente entre sus dos estados (**0** y **1**) produciendo una onda de período T : frecuencia $f = 1/T$. Un *astable*, como oscilador en onda cuadrada (o bien en onda rectangular si los semiperíodos de la misma son de distinta duración) sirve para generar la onda de reloj de los sistemas síncronos o cualquier otra señal de frecuencia fija que interese.

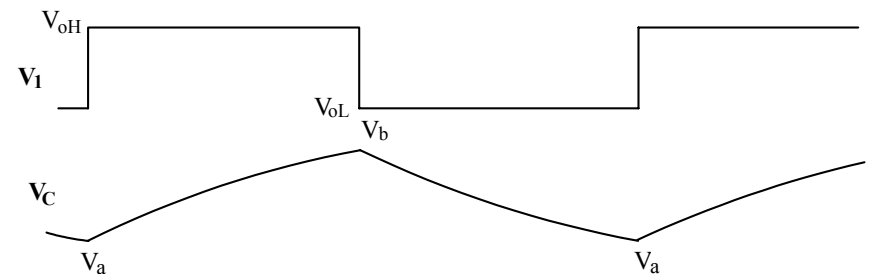


Es posible configurar un sencillo oscilador en onda rectangular mediante un lazo de realimentación RC sobre un inversor con entrada de tipo *Schmitt*; la realimentación a través de R determina la carga y descarga del condensador según que la salida del inversor sea **1** ó **0** y, a su vez, la tensión del condensador fuerza la conmutación del inversor al alcanzar las tensiones umbrales V_a y V_b .



En la figura anterior se añade un segundo inversor para mejorar la verticalidad de los flancos de la onda y, a la vez, proteger funcionalmente al circuito oscilador evitando el efecto de carga de múltiples entradas conectadas sobre el mismo.

La tensión sobre el condensador será aproximadamente triangular (constituida por sendos tramos de las exponenciales de carga y descarga) entre los valores de tensión V_a y V_b (de disparo de la entrada *Schmitt*) de forma que las ondas en la salida y la entrada del primer inversor serán las siguientes:



La onda rectangular de salida tendrá como semiperíodos:

$$\text{- carga } T_1 = R.C. \ln \frac{V_{OH} - V_a}{V_{OH} - V_b} \quad \text{- descarga } T_2 = R.C. \ln \frac{V_{OL} - V_b}{V_{OL} - V_a}$$

$$\text{Supuesto que } V_{OH} \approx V_{CC} \text{ y } V_{OL} \approx 0 \quad T = T_1 + T_2 = R.C. \ln \frac{(V_{CC} - V_a) \cdot V_b}{(V_{CC} - V_b) \cdot V_a}$$

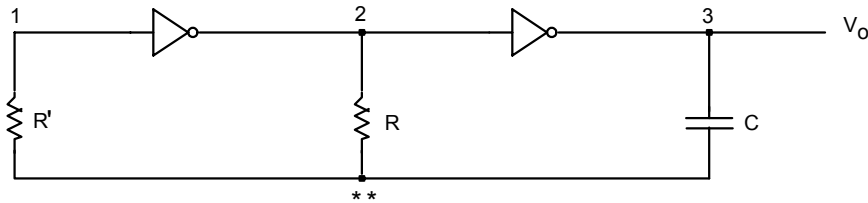
En inversores integrados CMOS (serie HC) para una tensión de alimentación de 5 V ($V_{CC} = 5$ V) las tensiones de disparo de la entrada *Schmitt* suelen ser 2 y 3 V:

$$T = RC \ln(9/4) \approx 0,8 RC$$

$$f = 1/T \approx 1,25/RC$$

La realimentación negativa que efectúa la resistencia ha de ajustarse en forma adecuada para evitar tanto el posible bloqueo del oscilador si la realimentación es muy fuerte, como la presencia de oscilaciones parásitas o ruido; valores de la resistencia que una entrada y salida del inversor entre 5K y 50K suelen ser apropiados.

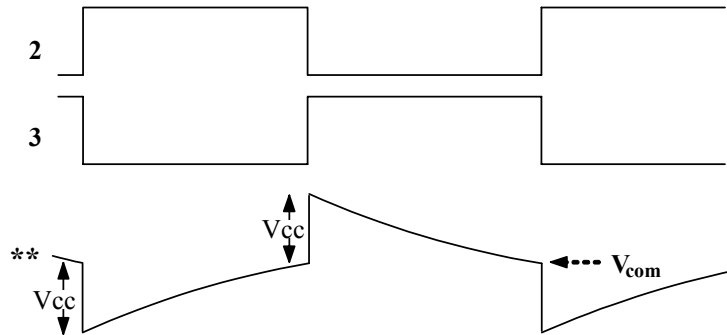
También puede construirse un oscilador astable con inversores CMOS normales (sin entrada *Schmitt*), utilizando el circuito de la figura.



Téngase en cuenta que el nudo ** no está conectado a masa. La resistencia **R'** sirve simplemente para separar el nudo ** de la entrada del primer inversor, evitando que los diodos limitadores que suelen incluirse en las entradas de las puertas integradas recorten las ondas de dicho nudo **: **R' >> R** para que pueda despreciarse su efecto en relación con la carga y descarga del condensador.

La tensión del nudo ** evoluciona exponencialmente hacia la del nudo 2, pero, a la vez, dicha tensión ** actúa sobre el nudo 1 y fuerza a la conmutación de ambos inversores cuando alcanza el valor **V_{com}** (tensión de conmutación del primer inversor).

Las formas de onda en los nudos de interés son las siguientes:



Los escalones de tensión del nudo 2 pasan, a través del condensador, al nudo **, después de lo cual, la tensión de dicho nudo ** tiende exponencialmente, a través de la resistencia R, hacia la tensión del nudo 2.

Por otra parte, la tensión del nudo ** se proyecta directamente sobre el nudo 1 (resistencia de entrada de las puertas CMOS infinita), de forma que cuando alcanza la tensión de conmutación **V_{com}**, cambia el valor booleano en los nudos 2 y 3.

Las ondas de carga y de descarga son simétricas y su semiperíodo (calculado en la semionda de carga) será:

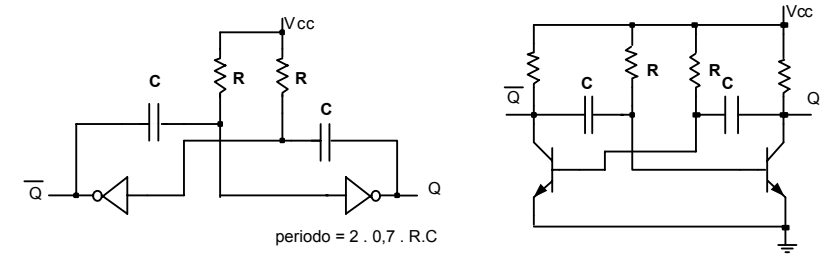
$$T/2 = R.C . \ln \frac{(V_{CC} - (V_{com} - V_{CC}))}{V_{CC} - V_{com}}$$

y para el caso de **V_{com} = V_{CC}/2** :

$$T/2 = R.C . \ln \frac{V_{CC} + V_{CC}/2}{V_{CC} - V_{CC}/2} = R.C . \ln 3 \approx 1,1 . R.C$$

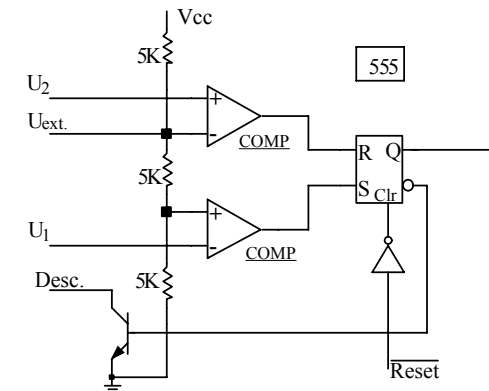
$$T \approx 2,2 . R.C ; \quad f = 1/T \approx 0,45/RC$$

Asimismo puede configurarse un oscilador astable con transistores discretos (que permiten tensiones de alimentación y de salida más elevadas):



T4.4. El circuito temporizador 555

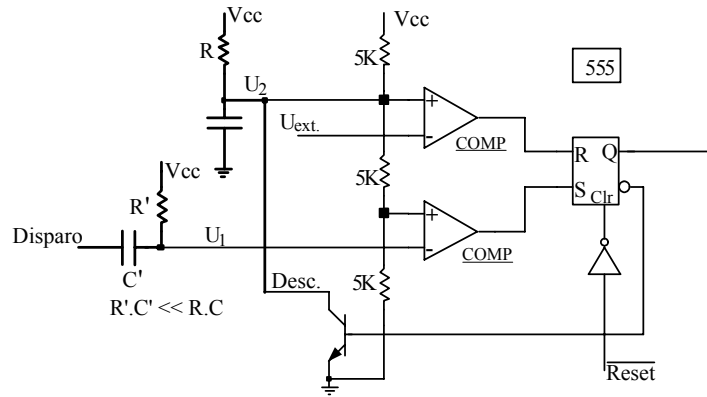
Un monostable es un temporizador específico que se considera como circuito digital; ahora bien, los catálogos de circuitos integrados lineales ofrecen también una amplia gama de temporizadores (*timers*), de los cuales el más conocido y utilizado es el **555** cuya configuración interna es la representada en la siguiente figura:



El esquema circuital del **555** es similar al del monostable integrado y sirve, en forma análoga, para configurar temporizaciones utilizando un circuito RC exterior para determinar los intervalos temporales.

Para ello se dispone de sendos comparadores con sus entradas **U₁** y **U₂** y de un circuito de descarga **Desc.** del condensador; una entrada auxiliar **U_{ext}** permite modificar ambas tensiones de comparación, sirviendo, asimismo, para configurar sistemas de modulación de anchura de pulsos **PWM**.

Para completar este circuito como monostable basta conectar una red RC a los terminales **U₂** y **Desc.**, produciendo el disparo a través de **U₁**, mediante un circuito derivador que, a partir de un flanco de bajada, proporcione un pequeño pulso invertido.



Al producirse el disparo el condensador, que se encuentra descargado por su conexión al terminal **Desc.**, iniciará un proceso de carga hacia **V_{CC}** hasta alcanzar la tensión de referencia del comparador superior; de esta forma, si no existe una tensión exterior conectada al terminal **U_{ext.}**, la anchura del pulso del monostable será:

$$T = R.C \cdot \ln \frac{V_{\infty} - V_{ini}}{V_{\infty} - V_{fin}} = R.C \cdot \ln \frac{V_{CC} - 0}{V_{CC} - 2V_{CC}/3} = R.C \cdot \ln 3 \approx 1,1 \cdot R.C$$

Es necesario que el pulso de disparo que actúe sobre la entrada **U₁** tenga una anchura inferior a la duración del pulso del monostable; para ello **R'C' << RC**.

Los temporizadores integrados (*timers*), configurados como monostables, pueden ser utilizados como moduladores de la anchura de los pulsos (**PWM**, véase 18.2).

Para utilizar un circuito integrado **555** como modulador de anchura de pulso basta dispararlo con una frecuencia fija (período **T**) y conectar al terminal **U_{ext}** la tensión a transformar; la anchura del pulso del monostable será:

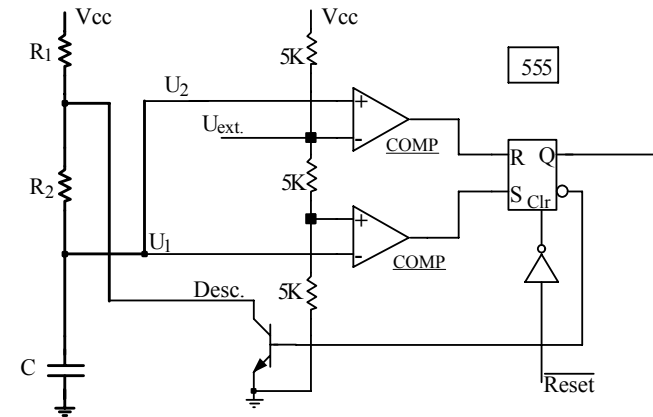
$$\Delta t = R.C \cdot \ln \frac{V_{CC}}{V_{CC} - U_{ext.}} \quad (\text{a mayor } U_{ext.}, \text{ mayor anchura de pulso}).$$

Esta transformación tensión-anchura de pulso es aproximadamente lineal cuando **U_{ext.} << V_{CC}**, en cuyo caso la expresión anterior puede reducirse al primer término de su desarrollo en serie:

$$\Delta t = -R.C \cdot \ln \frac{V_{CC} - U_{ext.}}{V_{CC}} = -R.C \cdot \ln \left(1 - \frac{U_{ext.}}{V_{CC}} \right) \approx R.C \cdot \frac{U_{ext.}}{V_{CC}} = K \cdot U_{ext.}$$

Es posible mejorar la linealidad de la transformación anterior y evitar la restricción sobre **U_{ext.}** utilizando para cargar el condensador un generador de intensidad (intensidad constante), en lugar de una resistencia.

Asimismo, los temporizadores (*timers*) pueden ser utilizados para construir astables; la configuración de un astable con el **555** requiere dos resistencias, **R₁** y **R₂**, en la red RC, a fin de descargar el condensador a través de una de ellas.



La carga del condensador se produce a través de ambas resistencias en serie **R₁+R₂** conectadas a **V_{CC}**, mientras que la descarga se realiza a través de la segunda de ellas **R₂**, cuando el transistor interno conduce. Habida cuenta de que las tensiones de comparación, en ausencia de tensión exterior conectada al terminal **U_{ext.}**, son **V_{CC}/3** y **2V_{CC}/3**, los semiperíodos de carga y descarga del condensador serán:

$$T_1 = (R_1 + R_2).C \cdot \ln \frac{V_{CC} - V_{CC}/3}{V_{CC} - 2V_{CC}/3} = (R_1 + R_2).C \cdot \ln 2$$

$$T_2 = R_2.C \cdot \ln \frac{-2V_{CC}/3}{-V_{CC}/3} = R_2.C \cdot \ln 2$$

$$T = T_1 + T_2 = (R_1 + 2R_2).C \cdot \ln 2 \approx 0,7 \cdot (R_1 + 2R_2).C$$

La onda resultante será rectangular, con el semiperíodo correspondiente al **1** mayor que el correspondiente al **0**, y su frecuencia será:

$$f = \frac{1}{T} \approx \frac{1}{0,7 \cdot (R_1 + 2R_2).C} \approx \frac{1,4}{(R_1 + 2R_2).C}$$

T4.5. Osciladores de precisión: cristal de cuarzo

La utilización de cristales de cuarzo permite disponer de circuitos astables de gran precisión en cuanto a la frecuencia de oscilación. Su configuración circuital se basa en realimentar positivamente una etapa amplificadora a través del cristal de cuarzo, que determina que la realimentación se produzca únicamente para su frecuencia propia. La ganancia de la etapa amplificadora combinada con la realimentación selectiva que produce el cuarzo dan lugar a un oscilador en onda cuadrada a la frecuencia del cristal.

El cristal de cuarzo como componente electrónico es un resonador que presenta dos modos de resonancia: *serie* con desfase nulo para la frecuencia de resonancia y *paralelo* que produce un desfase ligeramente inferior a 180° ; en ambos casos el cristal ofrece muy pequeña resistencia de paso para su frecuencia de resonancia y alta resistencia para cualquier otra frecuencia.

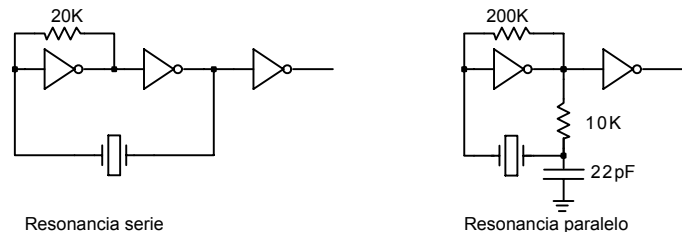
La frecuencia de resonancia de un cristal de cuarzo depende del corte cristalográfico con que ha sido tallado y de su espesor, abarcando el intervalo de frecuencias que va de los 10 KHz a los 10 GHz. Las dos frecuencias de resonancia (*serie* y *paralelo*) se encuentran muy próximas entre sí, pero para osciladores de muy alta precisión se fabrican cristales de cuarzo especialmente adaptados para la oscilación en serie y otros para la oscilación en paralelo con indicación precisa del valor de tales frecuencias.

La etapa amplificadora necesaria para configurar el oscilador se consigue polarizando un inversor en la zona de su función de transferencia que corresponde a la conmutación; en dicha zona el inversor se comporta como un amplificador, ya que pequeñas variaciones de la tensión de entrada V_i producen mayores variaciones en la tensión de salida V_o .

En los inversores CMOS basta unir la entrada a la salida a través de una resistencia de alto valor para polarizar ambas en un valor intermedio entre 0 y V_{CC} ; de esta forma se consigue que $V_o = V_i$, situación que corresponde a la zona de conmutación del inversor. Un solo inversor, así polarizado, configura una etapa amplificadora con desfase de 180° (inversora), mientras que un par de inversores dan lugar a un amplificador no inversor.



Ambos amplificadores, al ser realimentados a través de un cristal de cuarzo, oscilan según las frecuencias de resonancia del cristal.

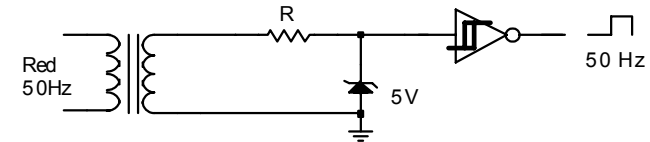


Si la etapa es no inversora la oscilación corresponde al modo serie; en cambio, cuando la etapa es inversora oscilará a la frecuencia de resonancia en modo paralelo. En este segundo caso la realimentación ha de efectuar un desfase de 180° para compensar la inversión que produce la etapa y, para ello, ha de añadirse un pequeño circuito RC en serie con el cristal de cuarzo (ya que el cristal por sí solo, en su modo paralelo, produce un desfase cercano pero inferior a los 180°).

La figuras anteriores incluyen un inversor a la salida para proteger funcionalmente al oscilador separándolo de las múltiples puertas que reciben su onda como señal de reloj.

Circuitos análogos pueden ser utilizados con resonadores piezocerámicos, componentes que sustituyen a los cristales de cuarzo con la ventaja de su menor precio y el inconveniente de la menor precisión y estabilidad de su frecuencia de resonancia.

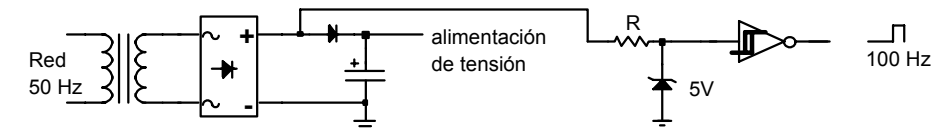
En los sistemas conectados a la red eléctrica es posible extraer, de la propia onda de red, una señal de reloj de la misma frecuencia: 50 Hz.



En el circuito anterior el diodo zener actúa como rectificador y como limitador de amplitud: por un lado rectifica la señal alterna de salida del transformador, de manera que la semionda negativa queda sobre la resistencia y la positiva pasa hacia la salida del circuito, y de otro lado recorta dicha semionda positiva, limitando su valor máximo a 5 V.

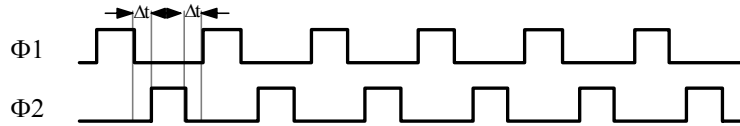
El inversor de entrada con histéresis (*Schmitt*) sirve para mejorar la verticalidad de los flancos de la semionda que recibe, de manera que la señal de salida es rectangular (con el semiperíodo correspondiente al **1** ligeramente mayor que el que corresponde al **0**) con la misma frecuencia que la red: 50 Hz.

De igual forma, si la rectificación es en doble onda, la señal resultante será de frecuencia doble: 100 Hz.



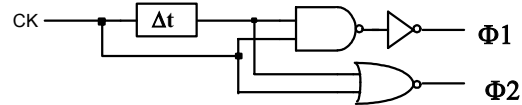
El circuito anterior aprovecha el esquema típico de una fuente de alimentación lineal (transformador-puente rectificador de 4 diodos-condensador de filtrado); ahora bien, para utilizar la señal de red rectificada en doble onda se separa la rectificación del filtrado mediante un diodo, de manera que cada semionda, una vez limitada su amplitud a 5 V por el diodo zener y digitalizada por el inversor de entrada *Schmitt*, da lugar a un pequeño pulso en la salida. La señal de reloj así obtenida es rectangular (el semiperíodo del **1** bastante menor al del **0**) con frecuencia doble respecto a la de la red: 100 Hz (debido a la rectificación en doble onda).

En arquitecturas digitales complejas o que requieran alta seguridad de funcionamiento suele utilizarse un reloj de dos fases no solapadas que incluyen un intervalo temporal de separación entre la habilitación de cada uno de los «semibiastables» de la configuración *amo/esclavo*.



De este modo, pequeños retrasos en la propagación de las señales de reloj o los propios tiempos de subida o de bajada de ellas quedan cubiertos por el tiempo de separación entre las dos fases y no provocan errores funcionales.

El siguiente esquema circuital permite la generación de dos fases no solapadas (*non-overlapping*) a partir de una onda rectangular de reloj **CK**:



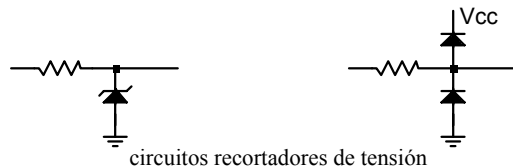
El retardo Δt puede realizarse mediante un simple circuito RC o, en forma integrada, con una cadena de inversores que acumulan sus tiempos de propagación.

La puerta "y" superior requiere dos «unos» en sus entradas para que la salida sea de valor **1**, mientras que la puerta "o-negada" (*Nor*) inferior requiere dos «ceros» para que su salida sea **1**; en ambos casos uno de los valores de entrada provenientes de **CK** llega con un retardo Δt , lo que determina que en dicho intervalo ambas salidas sean **0** (separación entre los «unos» de ambas ondas).

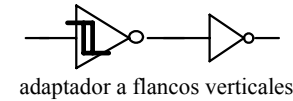
T4.5. Acomodación de pulsos externos

Cuando un circuito digital recibe pulsos externos es preciso efectuar un ajuste de sus niveles de tensión a los correspondientes a los valores booleanos V_{OH} y V_{OL} y un ajuste de flancos de forma que resulten adecuadamente verticales; en caso de que la amplitud de los pulsos no alcance los niveles V_{OH} y V_{OL} será necesaria una amplificación previa.

Para ajustar los niveles a las tensiones booleanas basta un pequeño circuito recortador con diodo zener en inversa (que limita la tensión positiva a V_z y la negativa a $-0,6$ V) o bien con sendos diodos en polarización inversa conectados a ambas líneas de alimentación (que limitarán la tensión positiva a $V_{CC} + 0,6$ V y la negativa a $-0,6$ V).

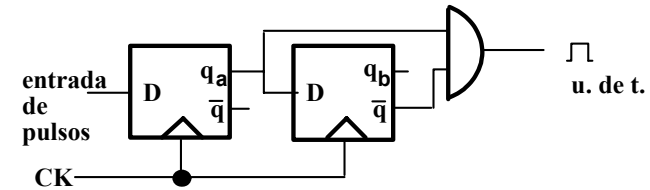


El ajuste de flancos se consigue con un inversor con entrada *Schmitt* (con dos tensiones de comparación para evitar rebotes o transiciones suaves en el entorno de la tensión de conmutación); un segundo inversor restituirá la polaridad del pulso (evitará que el pulso resultante quede invertido respecto al de entrada).

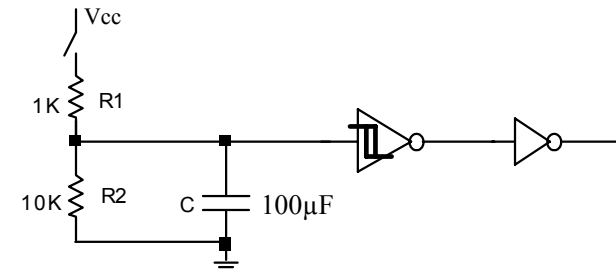


En ocasiones (por ejemplo, para el conteo síncrono de pulsos) será necesario añadir un detector de flancos que, por cada pulso recibido, produzca otro pulso cuya anchura sea una unidad de tiempo del reloj del circuito (onda de temporización, véase 16.4); dicho detector de flancos se configura con dos biestables D síncronos seguidos, q_b y q_a (q_a recibe el pulso exterior y q_b conectado a la salida del anterior):

la condición $\overline{q_b} \cdot q_a$ detecta flanco ascendente (**01**: valor anterior **0**, valor siguiente **1**), $q_b \cdot \overline{q_a}$ descendente (**10**: valor anterior **1**, siguiente **0**) y $q_b \oplus q_a$ detecta ambos flancos.



La actuación con pulsos manuales puede efectuarse mediante un pulsador conectado a V_{CC} , con una resistencia a 0 V que suministre entrada **0** en ausencia de pulsado. Ahora bien, los pulsadores mecánicos pueden producir rebotes (con duración del orden del milisegundo) tanto al pulsar como al soltar, de forma que en lugar de un pulso se produce una serie de ellos; es necesario incluir un filtrado de los rebotes, seguido de un ajuste a flancos verticales. Un simple filtro RC (con constante de tiempo del orden de $0,1$ s) puede servir para filtrar los rebotes, según el circuito de la figura siguiente.



R_2 ha de ser bastante mayor que R_1 para que al activar el pulsador la tensión comunicada sea próxima a V_{CC} ; el condensador conforma un filtro pasa bajo, cuya constante de tiempo es $0,1$ s al pulsar y 1 s al soltar, y el inversor con entrada *Schmitt* acomoda los flancos a verticales.

T5 CONVERSIÓN DIGITAL/ANALÓGICA Y ANALÓGICO/DIGITAL

T5.1. Conversores D/A

T5.2. Conversores A/D

T5.3. Resolución, linealidad y errores en los conversores

T5.4. Conversión tensión-frecuencia

El mundo real es básicamente analógico. La medida directa de una magnitud física (sonido, temperatura, presión, etc.) es convertida por el correspondiente transductor (sensor) a un valor de tensión analógica capaz de ser procesada por un sistema electrónico. Asimismo, el sistema electrónico proporcionará a los correspondientes efectores (altavoces, motores, calefactores, etc.) una tensión analógica que determine su actuación.

Los sistemas digitales emplean los valores numéricos codificados en binario, en palabras digitales compuestas por ceros y unos; ello proporciona a los sistemas digitales alta fiabilidad y precisión, conseguidas por la perfecta distinción física entre el 0 y el 1, y una gran potencia de cálculo, derivada de la utilización de un sistema de numeración y de la capacidad de integración de funciones booleanas de altísima complejidad.

En la frontera (interfase) entre las señales analógicas procedentes del medio físico o destinadas a interferir con él y las señales digitales que procesa el sistema electrónico se requieren conversores que pasen los valores numéricos del campo analógico al digital y viceversa: conversores A/D y D/A.

Mediante una suma ponderada de los dígitos de valor 1 se consigue, en forma muy simple, un conversor digital-analógico rápido; la ponderación puede hacerse con una serie de resistencias en progresión geométrica (cada una mitad de la anterior), lo cual obliga a utilizar un amplio rango de resistencias, o bien mediante una red R-2R que efectúa sucesivas divisiones por 2.

Puede convertirse una tensión en número binario utilizando un conversor opuesto D/A, a través de la comparación entre la tensión de entrada y la proporcionada por dicho conversor D/A aplicado a un generador de números binarios; se trata de aproximar el número-resultado a aquel cuya correspondiente tensión analógica es igual a la de entrada. La aproximación puede hacerse de unidad en unidad, mediante un simple contador, o dígito a dígito mediante un circuito secuencial específico.

En los sistemas digitales la precisión viene dada por la utilización de dos símbolos 1/0 y por la separación entre las tensiones que los representan. En cambio, en el tratamiento de tensiones analógicas y, por tanto, en los conversores D/A y A/D, hemos de preocuparnos de la precisión y de las diversas causas de error que le afectan: desplazamiento del origen, linealidad, resolución,...

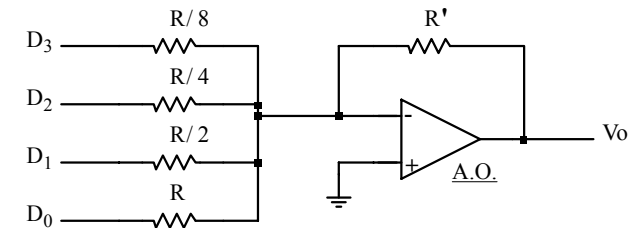
Se incluye en este capítulo, por completitud, la conversión tensión-frecuencia ($V \rightarrow f$), que puede servir también (añadiéndole un frecuencímetro) como conversión A/D. La conversión tensión-tiempo ($V \rightarrow t$) ha sido tratada en detalle en el capítulo 18 (PWM).

T5.1. Conversores D/A

Conceptualmente la conversión analógica-digital consiste en realizar la suma ponderada de los diversos dígitos que configuran el número binario; el valor relativo de cada uno de ellos viene dado por la correspondiente potencia de 2:

$$N = a_0 + 2.a_1 + 4.a_2 + 8.a_3 + 16.a_4 + \dots \\ = 2^0.a_0 + 2^1.a_1 + 2^2.a_2 + 2^3.a_3 + 2^4.a_4 + \dots = \sum 2^i.a_i$$

Esta suma puede realizarse mediante un sencillo circuito sumador con resistencias ponderadas (según la relación R, R/2, R/4, R/8, ...) como el de la figura:



Supuesto que las tensiones que corresponden a los valores booleanos sean 0 y +V:

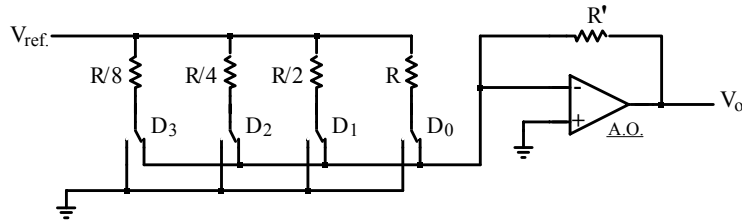
$$V_o = -(R'/R) \cdot (+V) \cdot (D_0 + 2.D_1 + 4.D_2 + 8.D_3 + \dots)$$

El último paréntesis de la expresión anterior expresa el valor del número binario ... D₃ D₂ D₁ D₀ y el factor inicial V.R'/R determina el valor de tensión asignado a cada unidad; las resistencias R' y R permiten ajustar dicho valor a la tensión unitaria que se desee.

Resulta un circuito sumamente sencillo para obtener una tensión analógica a partir de las tensiones de los dígitos binarios del número que se desea convertir. Habida cuenta de que la etapa sumadora es inversora, se obtendrá una tensión negativa, que puede transformarse fácilmente en positiva mediante una segunda etapa amplificadora inversora de ganancia unidad.

Las tensiones booleanas que presentan los diversos dígitos de un número binario (salidas de los correspondientes terminales del circuito digital, generalmente salidas de circuitos integrados) no ofrecen adecuada precisión: ambas tensiones, V_{oL} ≈ 0 V y V_{oH} ≈ +V, no son valores muy precisos.

Por ello, para aumentar la precisión del conversor, no se utilizan directamente las tensiones de los dígitos a convertir sino una tensión única de referencia de alta precisión, la cual se conecta (caso de dígito de valor 1) o no (valor 0) a las correspondientes resistencias sumadoras mediante interruptores; además, para disminuir los efectos capacitivos propios de los conmutadores y aumentar la velocidad de conmutación, ésta se efectúa entre dos posiciones de igual tensión.

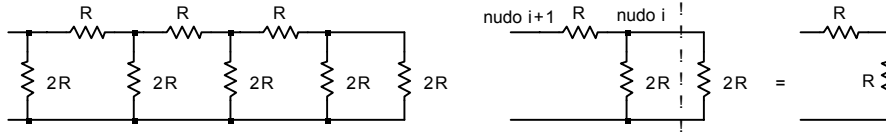


Cada conmutador se conecta hacia la entrada del amplificador cuando el valor del correspondiente dígito es 1; en otro caso, se conecta directamente hacia la línea de 0 V.

$$V_o = -(R'/R) \cdot V_{ref} \cdot (D_0 + 2.D_1 + 4.D_2 + 8.D_3 + \dots)$$

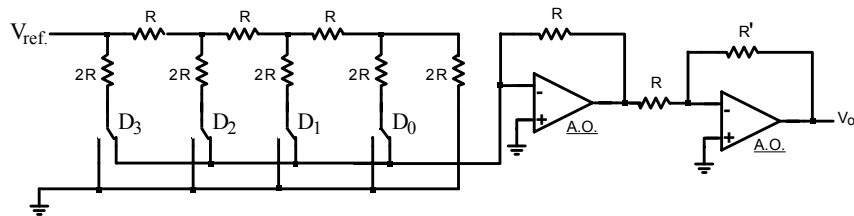
La precisión de este convertor depende de la precisión de las resistencias y de la tensión de referencia así como de las características del amplificador operacional, especialmente en lo relativo a tensión y corrientes de offset.

Ahora bien, esta red sumadora requiere resistencias de valores muy diferentes (por ejemplo para 12 bits ha de llegarse desde R hasta R/4096), siendo extremadamente difícil integrar tal diversidad de resistencias con la precisión necesaria. Por ello, resulta preferible utilizar una red de resistencias R-2R en escalera o red divisora de tensión, que posee la propiedad de que la resistencia de carga vista desde cualquier nudo de la red hacia adelante es de idéntico valor: 2R.



Esta red de resistencias tiene la propiedad de que en cada nudo se encuentran en paralelo sendas resistencias de igual valor 2R, una de las cuales es la equivalente del resto del circuito; de forma que en cada nudo la intensidad se divide en dos partes iguales y, de esta forma, cada nudo realiza una división de la tensión del nudo anterior por 2.

Utilizando este tipo de red como sumadora, mediante conmutadores entre dos posiciones (ambas con tensión de referencia 0 V) según el esquema siguiente, puede obtenerse un convertor D/A que solamente utiliza dos valores de resistencias R y 2R.



La segunda etapa amplificadora sirve para que la tensión de salida sea positiva e introduce la amplificación con el factor R'/R . Habida cuenta la sucesiva división de tensiones e intensidades que se produce en cada nudo:

$$\begin{aligned} V_o &= (R'/R) \cdot V_{ref} \cdot (D_3 + D_2/2 + D_1/4 + D_0/8) \\ &= (R'/16R) \cdot V_{ref} \cdot (16.D_3 + 8.D_2 + 4.D_1 + D_0) \end{aligned}$$

Con este tipo de red sumadora se configura una amplia gama de convertidores D/A integrados, de alta precisión, ya que es posible conseguir gran precisión en la red de resistencias y en la tensión de referencia (utilizando un zener de alta precisión bien estabilizado). Ello permite asegurar una fuerte linealidad en la conversión, con errores inferiores a la mitad del paso en tensión correspondiente a una unidad.

Los convertidores D/A más comunes de este tipo son de 8 y de 12 bits; un convertor de 8 bits permite una resolución de 256, es decir, para un intervalo de conversión 0-10 V a cada unidad le corresponden aproximadamente 40 mV; la resolución de un convertor de 12 bits es de 4096 pasos, 2.5 mV.

En tecnología MOS los conmutadores se realizan mediante transistores NMOS alternativos, entre cuyos terminales de puerta se conecta un inversor; se consiguen tiempos de respuesta globales (desde que se presenta el valor digital, hasta que aparece el correspondiente valor analógico) inferiores al microsegundo. Además, en aplicaciones relativas a la generación de ondas, en las cuales la salida va siguiendo sucesivamente valores próximos de la onda a generar, el tiempo de transición entre un valor y otro resulta mucho menor, pudiéndose alcanzar frecuencias superiores a 10 MHz.

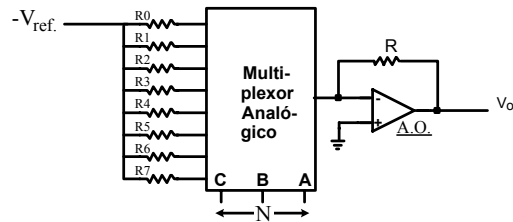
En el caso bipolar se configuran generadores de intensidad ponderados, mediante redes R-2R incluyendo transistores en las mismas; la configuración en amplificador diferencial permite conmutar tales intensidades entre las dos posiciones con altas velocidades de respuesta, consiguiéndose tiempos de conmutación del orden de 10 ns.

La utilización de una referencia de tensión negativa evita la necesidad de utilizar el segundo amplificador inversor.

En todos los convertidores D/A anteriormente considerados la tensión de salida es proporcional al número binario aplicado a sus entradas: $V_o = V_u \cdot N$, siendo V_u el paso en tensión correspondiente a una unidad; a veces (por ejemplo en la generación digital de ondas senoidales o de otras formas de onda) interesa otro tipo de funciones $V_o = f(N)$ distintas de la simple proporcionalidad.

Para ello puede efectuarse una transformación digital previa del número N a un número N' tal que $f(N) = V_u \cdot N'$, de manera que un convertor D/A proporcional aplicado sobre N' servirá para generar la tensión analógica deseada; la conversión intermedia (de N a N') puede ser realizada por un convertor de código o codificador ROM.

Cuando no se requiere gran precisión en la tensión de salida, puede obtenerse directamente la función $V_o = f(N)$ mediante un multiplexor analógico (formado por puertas de transmisión) controlado por el número N , según el esquema siguiente.

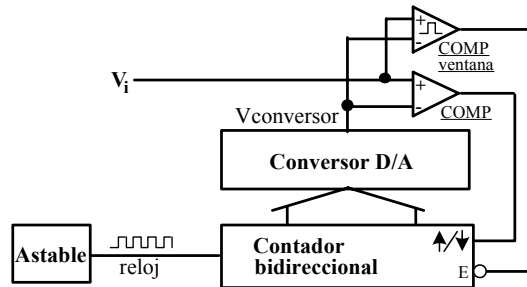


A un valor concreto N le corresponderá una tensión $V_o = R' \cdot V_{ref} / R_N$, que puede ser ajustada al valor deseado mediante la resistencia R_N ; caso de que la función $f(N)$ adopte también valores negativos, bastará conectar las resistencias correspondientes a una tensión de referencia positiva $+V_{ref}$.

T5.2. Conversores A/D

La utilización de los conversores D/A considerados en el apartado anterior permite realizar la conversión inversa, analógica-digital A/D, a través de un sencillo esquema funcional basado en la comparación entre la señal a digitalizar y la proporcionada por el conversor D/A; un circuito secuencial de aproximación deberá generar los números binarios cuya correspondiente tensión analógica es comparada con la tensión a convertir, de forma que la conversión finaliza en el momento en que ambas tensiones se igualan.

El circuito de aproximación más sencillo lo constituye un contador bidireccional (*up/down*), que cuente «hacia arriba» o «hacia abajo» según que el resultado de la comparación entre la tensión de entrada y la tensión generada por el conversor D/A sea favorable a la primera o a la segunda de dichas tensiones.



Cuando los valores de tensión (la exterior y la resultante del conversor D/A) se igualan el contador se sitúa en una secuencia alternativa (contar-descontar), oscilando entre dos números contiguos; para evitarlo se añade un comparador de ventana, cuya tensión central se sitúa en la tensión a medir y la anchura de la ventana se hace algo mayor que el paso en tensión correspondiente a una unidad.

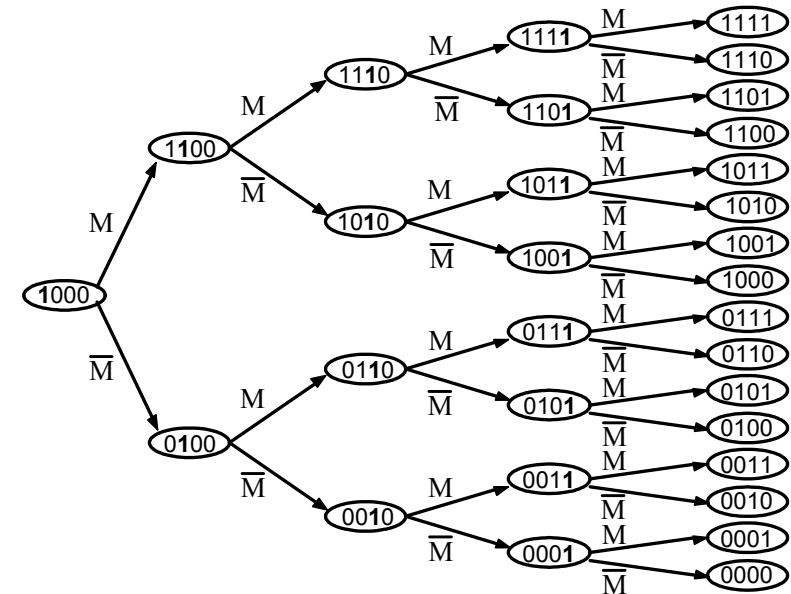
Para realizar una medida de tensión, estos conversores han de efectuar un conteo de pulsos desde la situación en que se encuentre el contador hasta la correspondiente a la medida, empleando para ello los ciclos de reloj necesarios: en el peor de los casos tiene que llegar a realizar 2^n pasos (4096 pulsos de reloj para una conversión de 12 bits). Por ello este conversor resulta lento para efectuar conversiones aisladas, como, por ejemplo, para efectuar las medidas sucesivas de varias señales multiplexadas.

Sin embargo, el conteo hacia arriba y hacia abajo resulta apropiado para seguir la evolución de una señal en un proceso de medida continuada; para tales aplicaciones este esquema de conversión A/D resulta muy atractivo por su sencillez.

Conversores A/D más rápidos se consiguen utilizando, en lugar del contador, un circuito secuencial que actúe por aproximaciones sucesivas, bit a bit:

- inicialmente se pone a **1** el dígito más significativo y el resto de ellos a **0**;
- se compara la tensión analógica correspondiente (dada por el conversor D/A) con la tensión de entrada a medir: si es mayor esta última se consolida el valor **1**, en otro caso se pasa a **0** dicho dígito;
- se procede de igual forma con el dígito siguiente, de modo que cada bit se sitúa a valor **1** y se respeta dicho valor si la tensión correspondiente es menor que la tensión a medir, pasándolo a **0** en caso contrario.

Designando con M el valor booleano que expresa el resultado de la comparación entre la tensión de entrada y la tensión del conversor D/A, $M = "V_i > V_{conversor}"$, el diagrama de estados del circuito secuencial es el siguiente:



De esta forma, para realizar la conversión se requieren solamente n pulsos de reloj, tantos como dígitos ha de tener el número digital resultante. El proceso comienza con una señal de inicio que pone a **1** el bit más significativo y borra todos los demás, a partir de la cual cada pulso de reloj determina una de las transiciones del grafo de estados anterior.

El circuito secuencial que efectúa las aproximaciones sucesivas, bit a bit a partir del más significativo, estará compuesto por n biestables cuyas condiciones booleanas de marcado y de borrado son las siguientes:

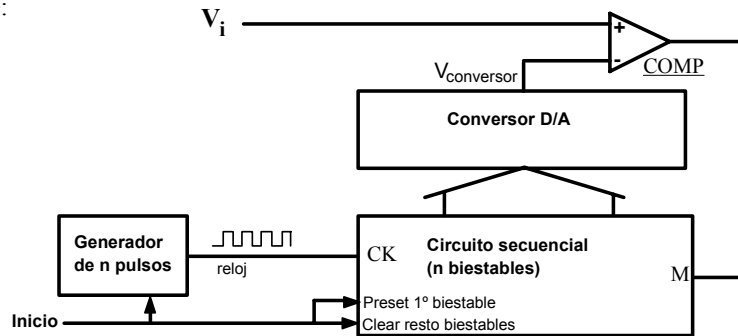
- el pulso de comienzo (*start*) marca el biestable correspondiente al dígito más significativo (primer biestable) y borra todos los demás biestables;
- cualquier biestable, salvo el primero, debe marcarse al llegar un pulso de reloj, cuando el anterior biestable se encuentra a **1** y todos los siguientes, incluido el mismo, a **0**:

$$J_i = q_{i+1} \cdot \overline{q_{i-1}} \cdot \overline{q_{i-2}} \cdot \dots \cdot \overline{q_1} \cdot \overline{q_0}$$

- cualquier biestable, incluido el primero, debe borrarse con un pulso de reloj cuando el mismo se encuentre a **1** y todos los siguientes están a **0** y, además, el resultado de la comparación **M** es **0**:

$$K_i = q_i \cdot \overline{q_{i-1}} \cdot \overline{q_{i-2}} \cdot \dots \cdot \overline{q_1} \cdot \overline{q_0} \cdot \overline{M}$$

El diagrama de bloques del conversor por aproximaciones sucesivas será el siguiente:



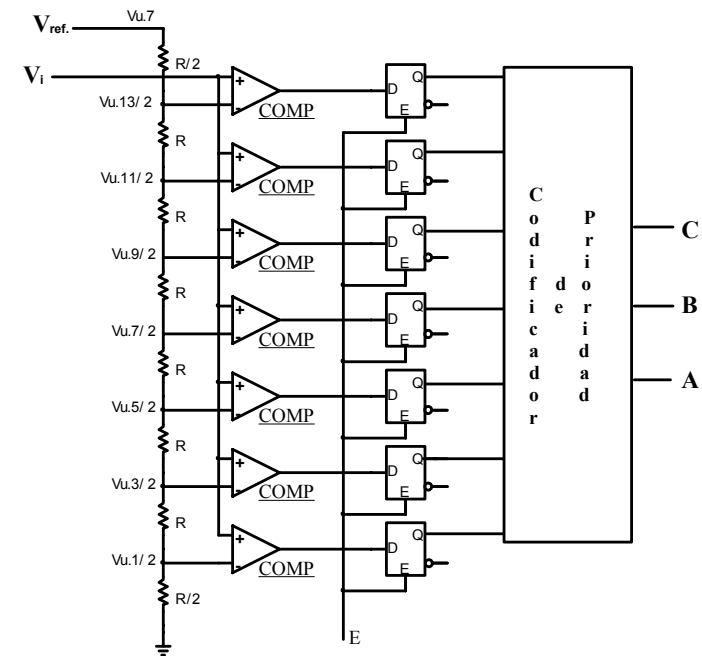
Éste es el esquema conceptual típico de los conversores A/D integrados de uso general, siendo los más frecuentes los de 8 ó 12 bits, con intervalos de conversión de [0,5], [0,10], [-5, +5] ó [-10, +10] voltios, con precisión equivalente al paso en tensión correspondiente al último bit y con tiempos de respuesta que se sitúan en el orden de los microsegundos (1-100 μ s.).

Los conversores A/D requieren que la tensión analógica a convertir permanezca constante durante el tiempo de conversión; para ello, si es necesario, se utilizan circuitos específicos de muestreo y mantenimiento (*sample and hold*) que toman un valor puntual de la señal presente en su entrada (muestreo) y lo mantienen en su salida (por efecto capacitivo) durante un cierto intervalo de tiempo.

Velocidades de conversión muy altas requieren convertidores de tipo paralelo, muy rápidos (*flash*), que comparan internamente la tensión a medir con los 2^m-1 niveles de tensión intermedia posibles (siendo m el número de bits del resultado).

Dichos 2^m-1 niveles se generan por división de tensión sobre 2^m resistencias y, a partir de ellos, un conjunto de 2^m-1 comparadores realiza la comparación de la tensión exterior con cada uno de los niveles. El vector de salida de dichos comparadores será un número digital formado por dos conjuntos sucesivos de ceros y unos; el número de unos presentes determina el nivel al que equivale la tensión exterior; un «codificador de prioridad» efectúa la conversión de dicho vector en el número binario que expresa el número de «unos» contenidos en él.

Para evitar errores debidos a transiciones durante la comparación suelen incluirse 2^m-1 biestables tipo D que reciben las salidas de los comparadores después de haberse estabilizado la comparación.



Este circuito de conversión A/D (*flash*) es sumamente rápido, existiendo series comerciales para 4 y 8 bits con tiempos de conversión inferiores a los 100 ns. El inconveniente es el gran número de bloques circuitales repetidos necesarios para realizar la conversión en paralelo (para 8 bits se necesitan 255 comparadores), lo cual limita el número de dígitos a obtener y eleva el coste de estos integrados.

Otros conversores de 8 y 12 bits utilizan un proceso de división en intervalos en dos pasos sucesivos (*conversores pipeline*) mediante dos conjuntos de redes de resistencias y comparadores, el primero de los cuales realiza una división «gruesa» (bits más significativos) y, restando de la tensión de entrada la tensión analógica correspondiente a tales dígitos más significativos, obtiene la diferencia de tensión sobre la cual realiza la comparación «fina» la segunda red. De esta forma para 8 bits bastan dos redes de 15 comparadores y para 12 bits se requieren dos redes de 63 comparadores.

Asimismo existen conversores A/D integrados que utilizan a la vez la conversión en paralelo y la conversión mediante aproximaciones sucesivas (*semiflash*); la conversión en paralelo se utiliza para una parte de los dígitos (más significativos) y se resta la tensión correspondiente a ellos, para efectuar luego, por aproximaciones sucesivas, la conversión «fina» que produce la otra parte de ellos (los de menor valor relativo).

T5.3. Resolución, linealidad y errores en los conversores

Este apartado se refiere expresamente a conversores D/A y A/D de tipo lineal, es decir, aquellos en que la correspondencia entre tensión analógica y valor numérico binario (en sistema de numeración de base 2) es de proporcionalidad directa. El «recorrido» del conversor irá de 0 a N en cuanto a valor numérico digital ($N = 2^m - 1$, siendo m el número de dígitos del conversor) y de $V_{\min}$ a $V_{\max}$ en lo que se refiere a tensión analógica; de forma que a una unidad digital (bit menos significativo MSB) le corresponderá una tensión analógica V_u (tensión unitaria), tal que

$$V_u = \frac{V_{\max} - V_{\min}}{N}, \text{ que es el «paso» o salto en tensión entre un número digital y el}$$

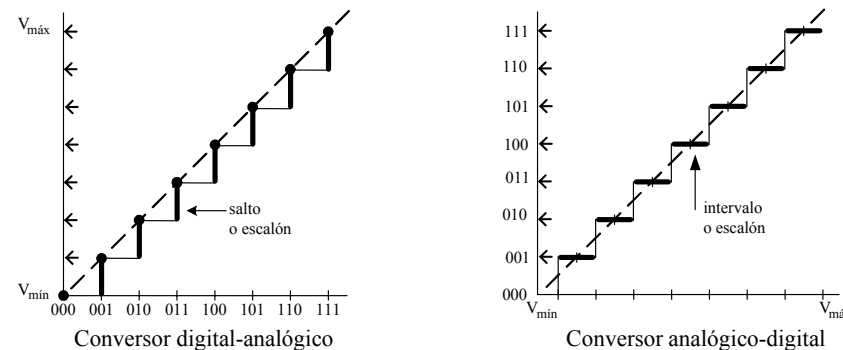
siguiente (entre dos valores digitales consecutivos).

Así, pues, en toda conversión (entre digital y analógica) se aplica un proceso de cuantificación de la tensión analógica, habida cuenta de que los valores digitales son discretos; la tensión analógica no interviene en su forma propia de «rango continuo» de valores (entre dos extremos $V_{\max}$ y $V_{\min}$) sino que actúa a través de «escalones»:

- la conversión D/A presenta un «paso» o «escalón» vertical, dado por la diferencia entre las dos tensiones que corresponden a dos números binarios sucesivos; los valores de tensión situados «dentro» de dicho «escalón» (entre las dos tensiones citadas) nunca se producirán como tensiones de salida;
- en la conversión A/D, a cada número binario de salida le corresponde todo un «intervalo» o «escalón» horizontal de tensiones analógicas; las tensiones situadas dentro de un mismo «escalón» son indistinguibles en cuanto a que proporcionan la misma salida digital.

Se produce, de por sí, un error de cuantificación o discriminación, pues valores de tensión próximos pero diferentes corresponden al mismo valor digital, no pueden ser diferenciados por el conversor: si éste es analógico/digital no distingue entre ellos y si es digital/analógico no los genera como tensiones de salida.

De esta forma las funciones de transferencia (salida – entrada: $V_o - n$ en el conversor D/A y $n - V_i$ en el A/D) son de tipo «escalonado» (ver figuras siguientes); si la conversión es lineal la anchura de los «escalones» es constante: todos ellos son de la misma «altura» V_u (conversor D/A) o de la misma «longitud» V_u (conversor A/D).



La función de transferencia de los conversores lineales viene caracterizada por una línea recta (que denominaremos «recta de conversión»), que pasa por el origen (0, $V_{\min}$):

- en el caso D/A esta recta contiene los «puntos de conversión», es decir, de correspondencia entre los números binarios de entrada –eje X– y los valores de tensión analógica de salida –eje Y–
- y en el caso A/D la recta de conversión pasa por los puntos medios de los «escalones», o sea, de los intervalos de correspondencia entre las tensiones analógicas de entrada –eje X– y los números binarios de salida –eje Y–.

Resolución: rango y sensibilidad

La resolución de un conversor vendrá dada, desde el lado digital, por el número de dígitos (bits) que admite para el número binario y, desde el lado analógico, por la anchura del «escalón» (su «altura» en el conversor D/A y su «longitud» en A/D).

Resolución digital: **m** dígitos.

Resolución analógica o sensibilidad: **V_u** , anchura del escalón.

Por ejemplo, un conversor cuya tensión analógica varíe entre -10 y $+10$ V y su número binario sea de 12 dígitos (resolución digital, 12 bits) tendrá una anchura de escalón $V_u = 20 / 2^{12} = 20 / 4096 \approx 5$ mV; tal será su resolución analógica.

El número de dígitos determina el *rango numérico* dentro del cual se efectúa la conversión: $0 - N$, siendo $N = 2^m - 1$.

La anchura de escalón o «paso» entre tensiones analógicas, V_u , expresa la *sensibilidad* con que actúa el convertor: la mínima diferencia entre tensiones que es percibida por el convertor como correspondiente a dos números binarios diferentes (dos números consecutivos).

El *intervalo de tensión* va de $V_{\min}$ a $V_{\max}$, siendo $V_{\max} = V_{\min} + N \cdot V_u$.

Proporcionalidad lineal y errores

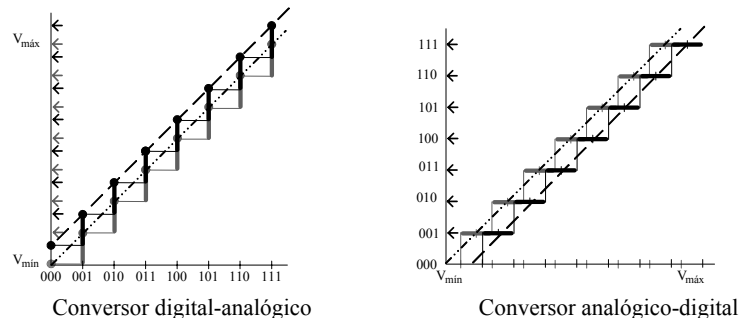
En los sistemas digitales la precisión queda garantizada por la codificación en dos símbolos diferenciados 0/1 y por la separación de los valores de tensión que los representan: salvo problemas de ruido electromagnético o de mal funcionamiento, un circuito digital proporciona con absoluta precisión los vectores de salida que corresponden a su diseño lógico.

No ocurre así en los circuitos analógicos, como son los conversores D/A y A/D, en los cuales hay múltiples causas de imprecisión que determinan desviaciones entre los resultados que teóricamente deberían proporcionar y los que realmente suministran.

Los posibles errores de estos conversores pueden detectarse y clasificarse en relación con la «recta de conversión»:

- cuando esta recta no pasa por el origen: error de cero o de *offset*;
- cuando la pendiente de la recta no es la apropiada: error de ganancia;
- cuando dicha línea no es una recta: error de linealidad.

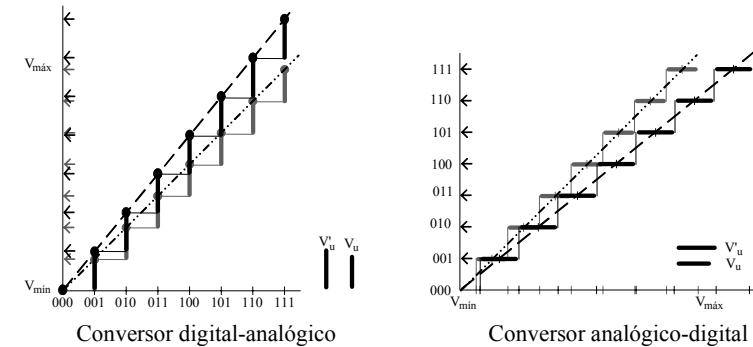
El error de cero (*offset*) existe cuando no se corresponde el valor numérico 0 (00...0) con la tensión analógica inicial $V_{\min}$, es decir, cuando la «recta de conversión» está desplazada y no corta al eje de tensiones analógicas en dicho valor $V_{\min}$ y, por tanto, el valor numérico máximo N (11...1) tampoco se corresponde con la tensión $V_{\max}$.



En la figura anterior (y en las siguientes de este mismo apartado) se representa «en gris» la escala de conversión sin error, como referencia para apreciar la desviación provocada por el error; asimismo se representa en línea de raya y dos puntos la «recta de conversión» teórica.

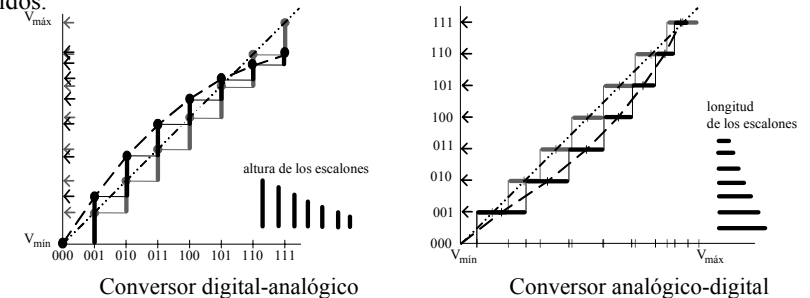
Si solamente hay error de cero (*offset*), el intervalo de tensión [$V_{\min}$, $V_{\max}$] se desplaza: [$V_{\min} + V_{\text{offset}}$, $V_{\max} + V_{\text{offset}}$], pudiendo V_{offset} ser positiva o negativa.

El error de ganancia se presenta cuando la pendiente de la «recta de conversión» es mayor o menor del valor que le corresponde según la relación de conversión. Tal error se produce cuando la anchura de los escalones no coincide con la tensión unitaria V_u , sino que es mayor o menor a la que corresponde al cociente $(V_{\max} - V_{\min})/N$.



Si existe error de ganancia el rango de la tensión analógica no coincidirá con el previsto; $V'_{\max} = V_{\min} + N \cdot V'_u$ será mayor o menor que $V_{\max}$, según que $V'_u > V_u$ o $V'_u < V_u$. El intervalo de conversión [$V_{\min}$, $V'_{\max}$] será más amplio o más pequeño que el previsto [$V_{\min}$, $V_{\max}$].

Cuando la anchura de los escalones no es constante la «recta de conversión» deja de ser una línea recta y decimos que hay error de linealidad. Dicho error puede ser puntual, referido a un escalón específico o general, afectando a un conjunto de escalones seguidos.



El error de linealidad puede expresarse (para cada valor binario) en forma integral por la desviación respecto de la recta de conversión ideal y en forma diferencial por la diferencia entre la anchura real de cada escalón y la anchura (V_u) que deberían tener todos ellos.

Sea un valor numérico digital **a**:

- sea V'_a el valor de tensión analógica que corresponde a dicho número **a** (en el caso de un conversor D/A el valor V'_a es, obviamente, la tensión de salida para entrada **a**, en el caso A/D V'_a será el punto medio del intervalo de tensiones que generan **a** como valor digital de salida);
 - habida cuenta que la pendiente de la «recta de conversión» es el cociente entre el intervalo de tensiones $V(N) - V(0)$ y el intervalo de números $N - 0$; el valor V_a que corresponde a **a** según la linealidad es $V_a = V(0) + a.(V(N) - V(0))/N$;
 - sea $\Delta V'_a$ el valor de la anchura del escalón correspondiente al número binario **a**;
 - la anchura ΔV que deben tener todos los escalones, en un conversor lineal, coincide con la pendiente de la «recta de conversión»: $\Delta V = (V(N) - V(0))/N$
- error de linealidad (integral) en el punto **a** = $V'_a - V_a$:
diferencia entre la tensión analógica real y la teórica (si fuera lineal)
 - error de linealidad (diferencial) en el punto **a** = $\Delta V'_a - \Delta V$:
diferencia entre la anchura de escalón real y la teórica (para ser lineal).

La forma integral expresa la desviación global respecto a la linealidad en el punto considerado, mientras que la forma diferencial expresa el error «puntual», es decir, en que medida la diferencia con el valor anterior es errónea (en que medida contribuye el punto **a** al error de linealidad).

Obviamente, los tres tipos de error (cero, ganancia y linealidad) no son excluyentes sino que pueden darse a la vez: consideraremos como «error absoluto» o desviación máxima la mayor de las diferencias entre la «recta de conversión» teórica y la «línea real de conversión»:

- ➔ En el caso del conversor digital-analógico el «error absoluto» cada diferencia se calcula entre la tensión de salida real y la tensión de salida teórica para un valor numérico de entrada, $V'_a - (V_{\min} + a.V_u)$, y de las $N+1$ diferencias (para los valores digitales de 0 a N) se toma la mayor de ellas.
- ➔ Respecto al conversor analógico-digital para cada valor digital se toman los valores de tensión analógica máximo y mínimo que proporcionan dicho valor de salida y se calcula su diferencia con el valor de tensión teórico según la «recta de conversión» (punto medio del escalón de tensión que corresponde a ese valor digital); del conjunto de estas $2(N+1)$ diferencias (para cada valor numérico se calculan en ambos extremos de tensión) se toma la mayor de ellas. De esta forma, el error absoluto incluye, también, el error de cuantificación ($\frac{1}{2}V_u$): si no hubiera otro tipo de errores, el error absoluto no sería nulo sino igual al error de cuantificación.

Limitación respecto a la frecuencia de muestreo (efecto de aliasing)

La conversión analógico-digital de una señal implica tomar «muestras» puntuales de la misma cada cierto intervalo de tiempo Δt , lo cual da lugar a una frecuencia de muestreo $f_m = 1/\Delta t$.

El teorema de Shannon aplicado a este «muestreo» de la señal (conversión de la misma en pulsos de anchura mínima y frecuencia f_m) reclama que, para asegurar la integridad de la señal, la frecuencia de muestreo ha de ser superior al doble de la frecuencia máxima presente en la señal analógica.

Si no se respeta esta limitación, se corre el peligro de que la señal digitalizada sea muy diferente a la señal analógica de entrada al conversor.



Efecto «exagerado» de *aliasing* (muestreo a una frecuencia excesivamente baja): la señal continua es la entrada analógica y la discontinua el resultado de la digitalización.

Una señal real suele tener una «anchura de banda» amplio, es decir, en las señales reales suele haber múltiples frecuencias (incluso «ruido», generalmente de frecuencias más altas que las propias de la señal) y, normalmente, interesan las componentes de la señal por debajo de una frecuencia dada.

El teorema de Shannon obliga, de un lado, a muestrear y efectuar la conversión A/D a una frecuencia superior al doble de la máxima frecuencia de interés y, por otro, a filtrar previamente la señal eliminando sus componentes por encima de dicha frecuencia. En toda conversión analógico-digital, es sumamente conveniente incluir un filtro *antialiasing* previo, con frecuencia de corte inferior a la mitad de la frecuencia a la cual se realiza la conversión.

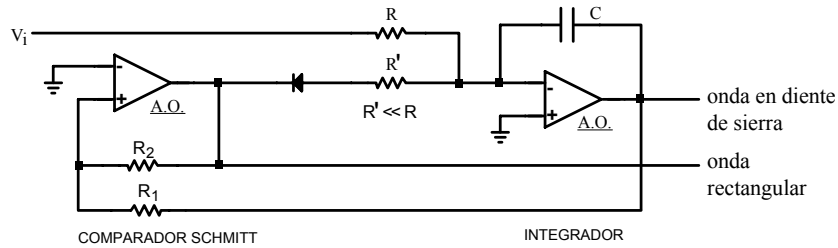
T5.4. Conversores tensión-frecuencia

Otra forma de realizar la conversión A/D consiste en convertir la tensión analógica en un tiempo o en una frecuencia directamente proporcionales al valor de dicha tensión.

En el primer caso un contador inicialmente a cero y cuyos pulsos de entrada tengan como período la unidad elemental de tiempo expresará, al finalizar el tiempo resultante de la conversión, la medida digital de la tensión analógica. En el segundo caso dicha medida puede ser obtenida mediante un frecuencímetro que reciba la señal resultante de la conversión tensión-frecuencia.

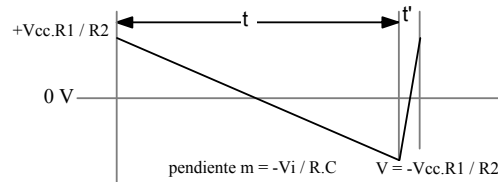
La conversión tensión-tiempo da lugar a pulsos de anchura modulada y, como tales moduladores, se describen en detalle en el capítulo 18; se consideran en dicho capítulo dos tipos de conversores tensión-tiempo: por rampa y sigma-delta.

La conversión tensión-frecuencia puede hacerse mediante un integrador y un comparador con histéresis, configurando un generador de onda triangular y rectangular. El circuito siguiente representa un conversor tensión-frecuencia de baja precisión y linealidad, pero muy simple.



La onda triangular de salida del integrador tiene dos semiperíodos muy diferentes, debido a la desigualdad entre las resistencias R' y R . Suponiendo despreciable el semiperíodo más pequeño respecto al otro mayor, resulta una frecuencia directamente proporcional a la tensión exterior V_i .

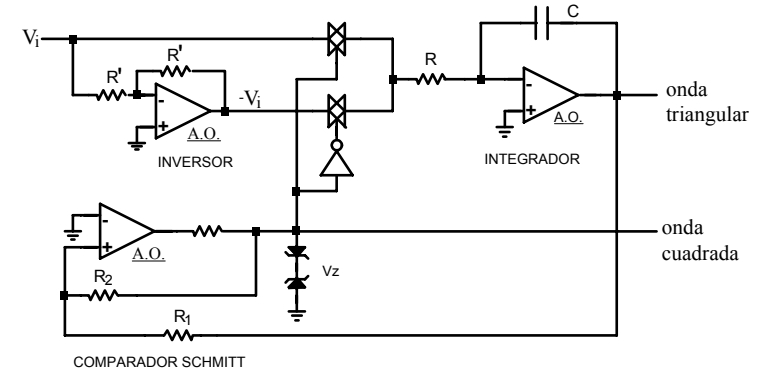
Diente de sierra:



$$T = t + t' \approx t = \frac{\Delta V}{m} = \frac{2 \cdot V_{CC} \cdot R_1 / R_2}{V_i / R \cdot C} = \frac{2 \cdot R \cdot C \cdot R_1 \cdot V_{CC}}{R_2 \cdot V_i}$$

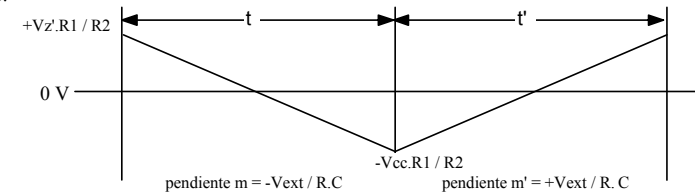
$$f = 1/T \approx K \cdot V_i \quad \text{con } K = \frac{R_2}{2 \cdot R \cdot C \cdot R_1 \cdot V_{CC}}$$

Puede mejorarse considerablemente la linealidad de este circuito conformando una onda triangular simétrica por integración sobre V_i y $-V_i$, respectivamente, de forma que no será preciso despreciar uno de los semiperíodos frente al otro; además, la utilización de tensiones *zener* a la salida del comparador *Schmitt* proporciona mayor precisión a sus tensiones de disparo.



Se consigue así un conversor tensión-frecuencia con buena linealidad y cuya precisión dependerá de la red RC del integrador, de las resistencias R_1 y R_2 del comparador y de las tensiones de referencia de los diodos zener $\pm V_z'$ ($V_z' = V_z + V_\mu$), siendo V_z la tensión del diodo en su zona zener y V_μ su tensión de conducción en directo, así como de las características de los amplificadores operacionales.

Onda triangular:



$$T = t + t' = 2 \cdot t = 2 \cdot \frac{\Delta V}{m} = 2 \cdot \frac{2 \cdot V_z' \cdot R_1 / R_2}{V_i / R \cdot C} = \frac{4 \cdot R \cdot C \cdot R_1 \cdot V_z'}{R_2 \cdot V_i}$$

$$f = 1/T \approx K \cdot V_i \quad \text{con } K = \frac{R_2}{4 \cdot R \cdot C \cdot R_1 \cdot V_z'}$$

Existen conversores integrados tensión-frecuencia de alta linealidad y precisión para diversos intervalos de frecuencia, siendo comunes los de 1-10 KHz, 10-100 KHz y 100KHz-1MHz. Un pequeño frecuencímetro conectado a su salida completa la conversión A/D; por otra parte, en algunos tipos de control automático se utiliza directamente la conversión tensión-frecuencia para transmitir con precisión el valor de una variable a través de un lazo de realimentación.



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

21 MICROELECTRÓNICA:

CIRCUITOS INTEGRADOS ESPECIFICADOS POR EL USUARIO

- 21.1. Circuitos integrados programados por el usuario: tipos
- 21.2. Circuitos integrados diseñados por el usuario: tipos
- 21.3. Herramientas de diseño
- 21.4. Validación de circuitos integrados: simulación, análisis de tiempos y test

La palabra «microelectrónica» alude a la «electrónica» en tamaño «micro», es decir, a la configuración miniaturizada de los circuitos electrónicos. Ahora bien, añadir sin más la referencia al reducido tamaño de los circuitos electrónicos no deja de ser una redundancia: el desarrollo de la electrónica se encuentra marcado, desde los años 50, por un continuado proceso de miniaturización, del cual deriva su propia funcionalidad (su capacidad de abordar eficazmente funciones complejas) y su viabilidad (mediante equipos de tamaño, consumo y coste reducido).

A partir de los años 80, algunos autores han aprovechado, en forma relativamente oportunista y confusa, el título de «microelectrónica» en textos de electrónica realizada con circuitos integrados estándar, en cuyo contexto el prefijo «micro» no aporta ninguna precisión porque no hay «otro tipo de electrónica»: carece de sentido práctico (y estaría fuera de contexto tecnológico) un diseño que no utilice los circuitos integrados disponibles.

El término «microelectrónica», para que resulte significativo y conceptualmente relevante, debe ser entendido como la configuración del circuito electrónico completo, resultante de un diseño específico, en el interior de un solo circuito integrado. Ello supone pasar de la interconexión de circuitos integrados estándar a la configuración de un circuito integrado específico: se trata de insertar nuestro propio diseño completo en un integrado.

Es decir, en lugar del ensamblaje de «microcircuitos» genéricos (de catálogo) para configurar nuestro diseño, la microelectrónica trata de la realización de dicho diseño específico en un «microcircuito» particular; para lo cual disponemos de dos alternativas:

- programar nuestro diseño sobre un circuito integrado programable (CPLD, FPGA)
- o, fabricar dicho diseño como circuito integrado específico (ASIC).

Ambos casos (programación o fabricación del diseño) utilizan las mismas tecnologías de fabricación de los circuitos integrados (proceso de integración CMOS), el mismo esquema de desarrollo del proceso de diseño, el mismo tipo de herramientas y las mismas consideraciones metodológicas a tener en cuenta.

El presente capítulo describe las diversas posibilidades existentes en cuanto a circuitos integrados programables (PLDs, basados en la estructura PAL, y FPGAs, con estructura PGA) y respecto a la fabricación del propio diseño (ASIC); presenta las diversas herramientas necesarias para desarrollar un diseño en sus diferentes fases y opciones y reflexiona sobre tres aspectos diferenciados de la comprobación del diseño.

21.1. Circuitos integrados programados por el usuario: tipos

La finalidad de la microelectrónica es *configurar un circuito digital específico en el interior de un circuito integrado* (es decir, «insertar nuestro diseño en un integrado») y los circuitos integrados programables ofrecen una posibilidad de realizar este objetivo.

Un circuito integrado programable, adecuadamente grande, permite configurar, a través de su programación, un diseño digital específico y ello puede ser realizado directamente por el propio usuario.

En tal sentido, en la construcción de prototipos y en la fabricación de series limitadas la *lógica programada* sobre un único circuito integrado ha desplazado a las placas de múltiples circuitos integrados estándar. La utilización de integrados estándar, aún cuando puede resultar útil en laboratorio (para casos de diseños relativamente simples), ha sido sustituida en la práctica por el empleo de circuitos integrados programables, de forma que un solo circuito recoge el conjunto completo de funciones booleanas necesarias, evitando el cableado entre los diversos bloques digitales.

Los circuitos integrados programables presentan siempre una configuración interna modular: una *celda tipo* repetida múltiples veces; cada celda cuenta con una parte combinacional, sobre la que puede programarse una función booleana, seguida de un biestable. La parte combinacional de la celda o módulo básico permite establecer una división de los dispositivos programables en dos grandes grupos:

- dispositivos de tipo PAL: funciones expresadas en forma de suma de productos;
- dispositivos basados en multiplexores (funciones LUT), organizados en forma de matriz de módulos programables, tipo PGA.

Los dispositivos de tipo PAL están conformados por macroceldas (que incluyen las funciones como suma de productos seguidas de sendos biestables) y fueron estudiados en el apartado 4 del capítulo 13 (volumen 2). Las macroceldas han de contener sumas de términos producto de muchas variables; son módulos de «granularidad gruesa»: funciones de muchas variables de entrada (pudiendo cada módulo realizar una función compleja o bien desaprovechándose muchas de sus conexiones programables).

Los dispositivos de tipo PGA (*Programmable Gate Array*) corresponden a una «granularidad más fina», con funciones internas de pequeño número de entradas (4, 5 ó 6) y, en cambio, ofrecen mayor potencialidad en las conexiones entre módulos; están compuestos por una matriz de «celdas lógicas» (de reducido tamaño), separadas por canales de conexión verticales y horizontales, junto con «celdas de entrada/salida» situadas en la periferia, que conectan con los terminales del circuito.

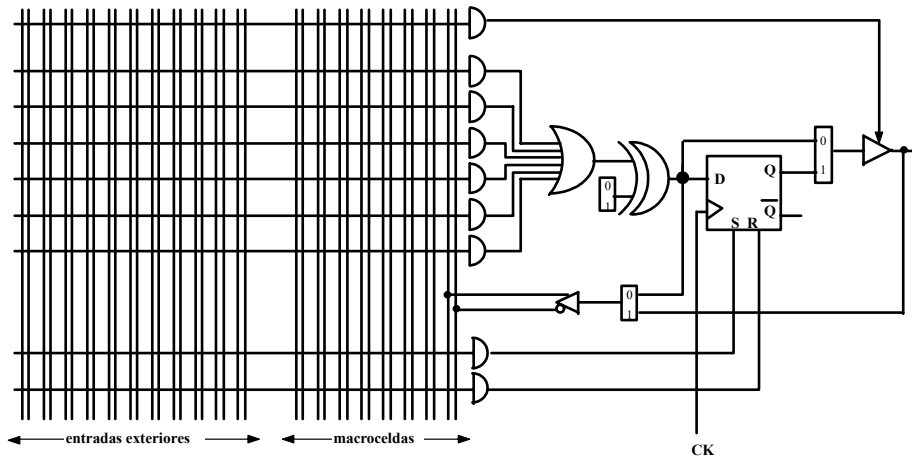
Los módulos de tipo PAL son más potentes (y, por ello, más susceptibles de ser desaprovechados) en cuanto a su parte combinacional interna y se presentan en estructuras de interconexión (arquitecturas) relativamente directa: lineal o ramificada. Los dispositivos PGA contienen módulos más simples (y, por lo tanto, de limitada capacidad booleana) en una organización matricial con canales verticales y horizontales de interconexión, con una gran capacidad en cuanto a conexiones programables.

Dispositivos lógicos programables complejos: CPLDs

La manera más habitual de expresar una función booleana es su forma algebraica como suma de productos $\sum p$; la configuración reticular de varias funciones booleanas expresadas en forma de suma de productos recibe el nombre de **PAL**: cada función ocupa un módulo separado, conformado por una puerta "o" que recibe las salidas de varias puertas "y" sobre las que se programan los términos producto de la función.

Para poder programar sistemas secuenciales es necesario añadir a la configuración **PAL** la capacidad de memoria que proporcionan los biestables: la inclusión de un biestable en cada módulo, cuyos términos producto han de admitir como variables, además de las propias variables de entrada, las salidas de los biestables, para permitir la realimentación de las variables de estado.

El módulo básico resultante, «suma de productos + biestable», ha sido ampliado con diversos «selectores» programables, así como con la posibilidad de salida «tri-estado», y recibe el nombre genérico de *macrocelda*.



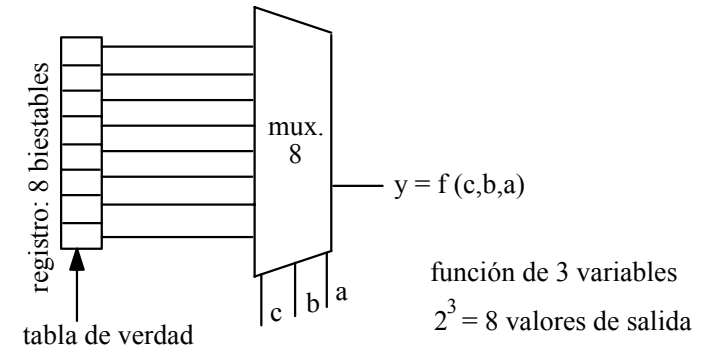
Los circuitos integrados programables que utilizan macroceldas suelen ser aludidos con las siglas **PLD** (dispositivo lógico programable) y **CPLD** (PLD complejo, cuando el número de módulos o macroceldas que contiene es alto).

En los **PLDs** pequeños cada macrocelda está conectada a todas las demás (recibe la realimentación de las otras macroceldas). En los **CPLDs** las macroceldas se organizan en bloques, existiendo un canal de interconexión (líneas con conexiones programables para conectar los bloques entre sí y para comunicarlos con las entradas exteriores).

[Véase una descripción más detallada de los CPLDs en el apartado 13.4 del volumen 2 y en el siguiente capítulo, apartado 22.1, que incluye un repaso de dicha descripción.]

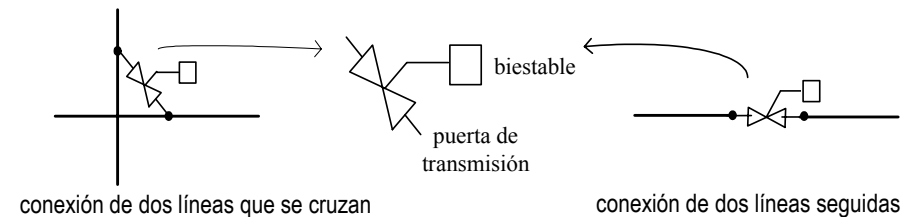
Matrices de celdas lógicas configurables: FPGAs

Una función booleana puede expresarse, también, a través de su tabla funcional y puede construirse mediante un multiplexor que «toma valores» sobre dicha tabla: configuración **LUT** (*look-up-table*).

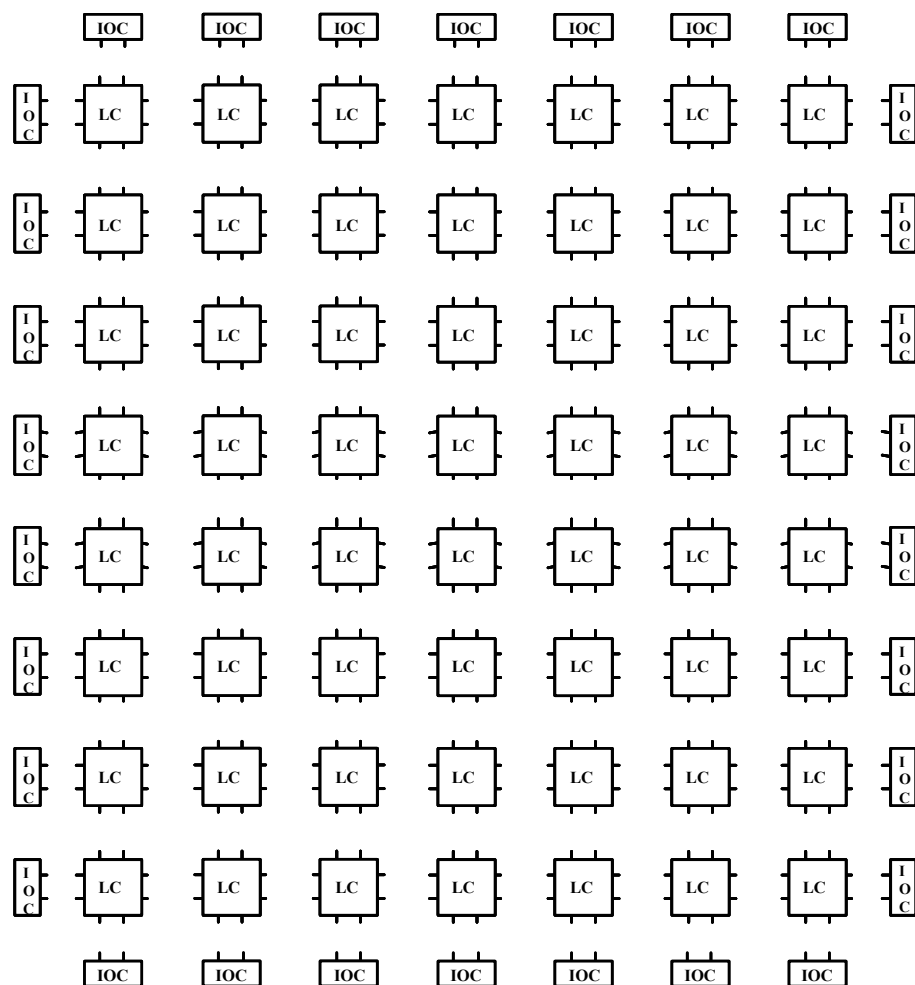


De esta forma, las funciones se configuran mediante su tabla de verdad que se memoriza en un «registro»: un multiplexor cuyas entradas son las salidas del registro selecciona (por «búsqueda en la tabla», **LUT**) la salida de la función.

Asimismo, las conexiones entre funciones (o entre módulos) pueden ser controladas mediante biestables: una puerta de transmisión entre dos líneas permite su conexión cuando la puerta conduce o su separación en caso de que se encuentre en corte; la situación de conducción o corte de la puerta de transmisión puede ser determinada por el valor booleano almacenado en un biestable. Téngase en cuenta que las puertas de transmisión, en tecnología CMOS, son bidireccionales: la conexión que establecen entre dos líneas puede actuar en ambas direcciones.



Los circuitos programables resultantes de esta configuración (funciones LUT y conexiones por puertas de transmisión) disponen de un conjunto de celdas lógicas (LC), organizadas matricialmente en dos dimensiones; a través de las franjas (verticales y horizontales) de separación entre ellas se establecen canales (conjuntos de líneas) para la interconexión de las celdas y, además, en la periferia se disponen celdas específicas de adaptación de las entradas y salidas (IOC).



Estos circuitos programables reciben el nombre de PGA (*Programmable Gate Array*): matrices de puertas programables, entendiendo el término puerta (G *gate*) en su acepción amplia de módulo lógico (como celda que contiene una función booleana de pocas variables). Su denominación habitual es FPGA (*Field Programmable Gate Array*), donde la F inicial (*field*: in situ), viene a indicar que la programación por el usuario puede hacerse en la propia placa circuital de aplicación, sin tener que extraer la FPGA del circuito de que forma parte.

Las matrices de celdas programables FPGAs son dispositivos de «granularidad fina», compuestos por una amplia matriz de celdas lógicas de reducido tamaño (y limitada capacidad booleana), separadas por canales de conexión verticales y horizontales, y por celdas de entrada/salida situadas en la periferia, que conectan con los terminales del dispositivo. El calificativo «programable» adjetiva tanto a las celdas (de ambos tipos: lógicas y de entrada/salida) como a la matriz de conexiones, es decir, tiene una doble repercusión: son programables tanto las celdas internamente (su función booleana y su configuración) como las conexiones de las celdas entre sí.

El próximo capítulo (cap. 22) se dedica a los circuitos integrados programables, repasando brevemente la configuración y características de los CPLDs y describiendo en detalle las FPGAs.

21.2. Circuitos integrados diseñados por el usuario: tipos

El extraordinario desarrollo alcanzado por las tecnologías de integración MOS y la continuada reducción de costes de los circuitos fabricados con ellas, junto con la sistematización de los procesos de diseño y la disponibilidad de eficaces herramientas CAD, han permitido que el diseño de los circuitos integrados sea realizado por los propios usuarios de los mismos.

De esta manera, cualquier diseñador de sistemas digitales puede abordar el diseño de su propio circuito integrado y, por tanto, cualquier empresa puede incorporar a los productos que fabrica circuitos integrados propios, diseñados específicamente para el producto en cuestión y pensados desde la perspectiva de mejorar, ampliar y personalizar las prestaciones de dicho producto.

La utilización de circuitos integrados específicos no sólo resulta viable, sino también económicamente rentable; aun más, contribuye en gran medida a aumentar la competitividad de los productos fabricados en serie, por su repercusión en las prestaciones y en el precio de los mismos.

Un ASIC (*application specific integrated circuit*) es un circuito integrado, no disponible en catálogo, diseñado para una aplicación concreta, de forma que sus prestaciones se ajustan específicamente a dicha aplicación.

En realidad, el término ASIC puede aplicarse tanto a los circuitos integrados programados por el usuario como a los fabricados según el diseño del mismo; en ambos casos, tales circuitos integrados resultan configurados en forma «no estándar», para aplicaciones específicas. Ahora bien, suele ser habitual reservar la denominación de ASICs para los circuitos fabricados con diseño propio y, en tal sentido, lo utilizaremos en este texto.

Un **ASIC** permite sustituir una o varias placas de circuitos integrados estándar, reduciendo drásticamente su coste, su tamaño y su consumo y aumentando su fiabilidad y su velocidad así como las prestaciones del circuito global; además un **ASIC** personaliza el circuito electrónico, asegurando en gran medida su confidencialidad, es decir, la dificultad de reproducción del mismo mediante duplicación de las placas circuitales o mediante el rediseño de las mismas con ligeras modificaciones.

En relación con una placa de circuitos integrados estándar, un ASIC presenta:		pero, también, supone:
↑ menor coste	↑ mayor velocidad	↓ mayor inversión inicial NRE
↑ menor consumo	↑ mayores prestaciones	↓ necesidad de un n° mínimo
↑ menor tamaño	↑ mayor fiabilidad	↓ menor flexibilidad al cambio
↑ menor peso	↑ mayor confidencialidad	↓ test más complejo

La fabricación de circuitos integrados específicos ofrece un coste por ejemplar sumamente inferior al coste global de la placa (o placas) de circuitos integrados estándar a los que sustituye y muy inferior, también, al coste de un circuito integrado programable equivalente.

Se requiere, en cambio, una inversión inicial (**NRE**: *non recursive engineering*) relativamente elevada, referente al diseño, desarrollo y construcción de las máscaras de integración; inversión que es independiente del número de ejemplares a fabricar. La repercusión de la inversión **NRE**, en el coste de cada unidad, es pequeña cuando el número de unidades que posteriormente se integran es de varios miles; de otro modo, el coste de fabricación individual se verá fuertemente incrementado por la amortización de dicha inversión inicial.

La última etapa del diseño a nivel físico de un circuito integrado, la que conecta con el proceso de fabricación, tiene un carácter netamente geométrico, para culminar en el dibujo de las máscaras que determinan, en cada una de las etapas del proceso de integración, la distinción entre unas y otras zonas superficiales de la oblea; respetando, además, las reglas de diseño que impone la tecnología de integración.

Existen tres posibilidades para abordar el diseño y la fabricación de un circuito integrado, según se expresa en el siguiente diagrama:

	<i>fabricación completa</i> <i>partir de una matriz de puertas prefabricadas</i>	
<i>diseñar completamente cada una de sus partes</i>	diseño físico completo FULL CUSTOM	
<i>utilizar una librería de celdas prediseñadas</i>	celdas prediseñadas STANDARD CELL	puertas prefabricadas GATE ARRAY

Un *diseño físico completo (full custom)* incluye el dibujo de las máscaras por parte del propio diseñador, ayudado por las correspondientes herramientas **CAD**. Este tipo de diseño completo puede resultar sumamente complejo; en cambio, permite, si el diseñador es suficientemente hábil, un óptimo aprovechamiento de la superficie de la oblea de silicio. De esta forma se diseñan los circuitos integrados estándar, cuyo alto volumen de producción justifica el esfuerzo de un diseño físico completo, en orden a minimizar la superficie de silicio de cada integrado y, con ello, su precio.

Sin embargo, el diseño de un **ASIC** no suele descender hasta el nivel de las máscaras de fabricación, sino que aprovecha librerías de celdas lógicas prediseñadas (puertas, biestables, pequeños bloques combinacionales, reducidos registros o contadores,...). El circuito específico se describe a través de la correspondiente interconexión de tales celdas y, una vez capturado su esquema, el compilador lo traduce automáticamente a las correspondientes máscaras físicas (utilizando para ello potentes algoritmos de colocación óptima de las celdas, *placement*, y de conexionado de ellas, *routing*).

A partir del diseño con *celdas prediseñadas (standard cell)* pueden obtenerse todas las máscaras necesarias para fabricar el correspondiente circuito integrado, personalizadas todas ellas para cada **ASIC**, y, consiguientemente, puede efectuarse el proceso completo de fabricación del mismo.

Otra posibilidad consiste en disponer de circuitos integrados parcialmente prefabricados, con un gran número de puertas en su interior, y configurar el circuito digital específico mediante el conexionado sobre dichas puertas. Este tipo de fabricación que utiliza una *matriz de puertas prefabricadas (gate array* o, también, *sea of gates*) simplifica el proceso de fabricación de cada **ASIC** individual, pues solamente las máscaras correspondientes al conexionado son personalizadas para cada **ASIC**.

En ambos casos (*standard cell* y *gate array*) el diseño se realiza utilizando una librería de celdas estándar y no es muy diferente del diseño clásico con circuitos integrados estándar, seleccionándolos en el correspondiente catalogo y realizando la correspondiente interconexión entre ellos.

La *fabricación completa de celdas prediseñadas (standard cell)* permite un buen aprovechamiento de la superficie de la oblea de silicio, por cuanto que se incorporan solamente las celdas funcionalmente necesarias, ya optimizadas en su diseño, y tales celdas se posicionan (mediante algoritmos de *placement* y *routing*) de forma que su interconexión sea lo más directa posible; por tanto permite fabricar un mayor número de unidades en cada oblea.

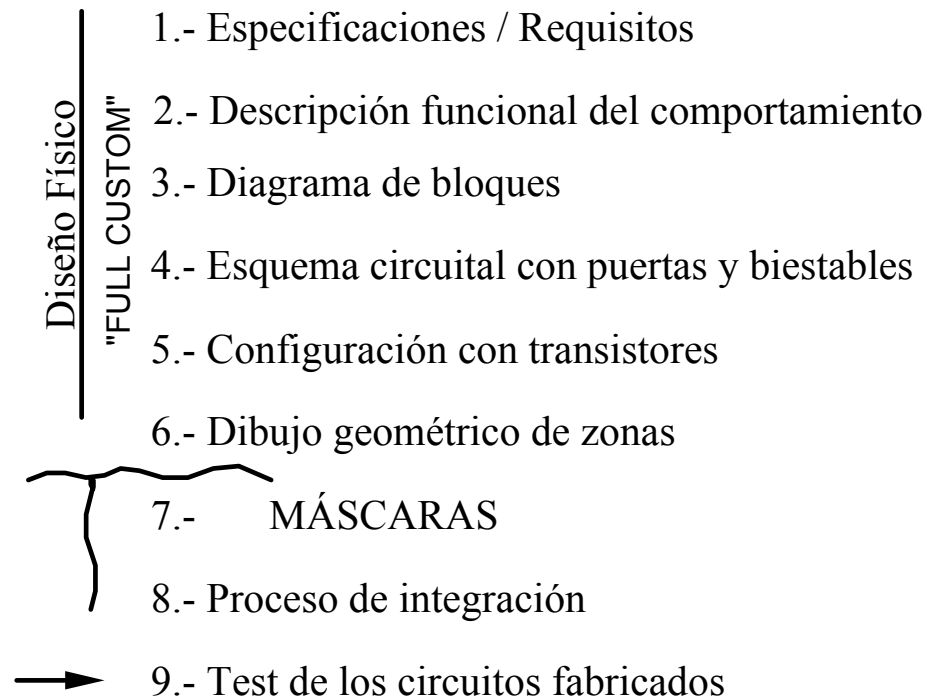
En cambio la utilización de *matrices de puertas prefabricadas (gate array)* requiere mayor área (no es posible utilizar todas las puertas prefabricadas, ni tampoco conseguir un óptimo aprovechamiento de las mismas), pero disminuye fuertemente el coste de la inversión inicial **NRE**, ya que solamente es necesario personalizar las máscaras y las etapas de fabricación correspondientes al conexionado (es decir, a las líneas de metal).

La elección entre uno u otro método de fabricación de un **ASIC** (*standard cell* o *gate array*) depende del número de ejemplares a integrar, es decir, al término de coste que predomina: el del proceso de fabricación concreta de cada ejemplar o el de la inversión inicial **NRE** distribuida entre todos los ejemplares fabricados.

En todo caso, el proceso de diseño incluirá la descripción del circuito, su simulación funcional (comprobar que funciona conforme se desea) y la generación de los vectores de test que permitan la comprobación física de cada circuito una vez fabricado; todo ello sobre computador, utilizando potentes herramientas CAD.

Diseño físico completo

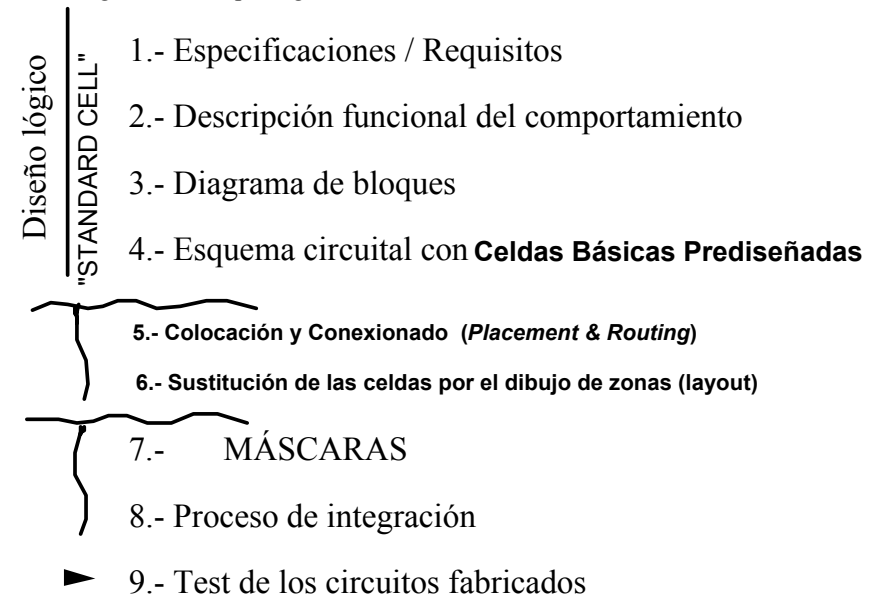
Los procesos de diseño completo (*full custom*) de un circuito integrado, junto con su fabricación, abarcan las siguientes etapas:



En un diseño completo, la tarea del diseñador parte de las especificaciones o requisitos que debe cumplir el circuito hasta llegar a su dibujo geométrico (*layout*), que dará lugar a las máscaras que el fabricante utilizará en el proceso de fabricación. También, es responsabilidad del diseñador la obtención de los vectores de test que el fabricante ha de aplicar para verificar cada uno de los circuitos resultantes del proceso de fabricación.

Diseño con celdas prediseñadas

Otra posibilidad de diseño consiste en utilizar una librería de celdas prediseñadas (*standard cell*) por el fabricante, de forma que el diseñador no ha de descender al nivel físico de los transistores que configurarán el circuito, sino que efectúa el diseño en el nivel lógico (booleano). Los procesos de diseño y fabricación utilizando una librería de celdas seguirán las etapas siguientes:



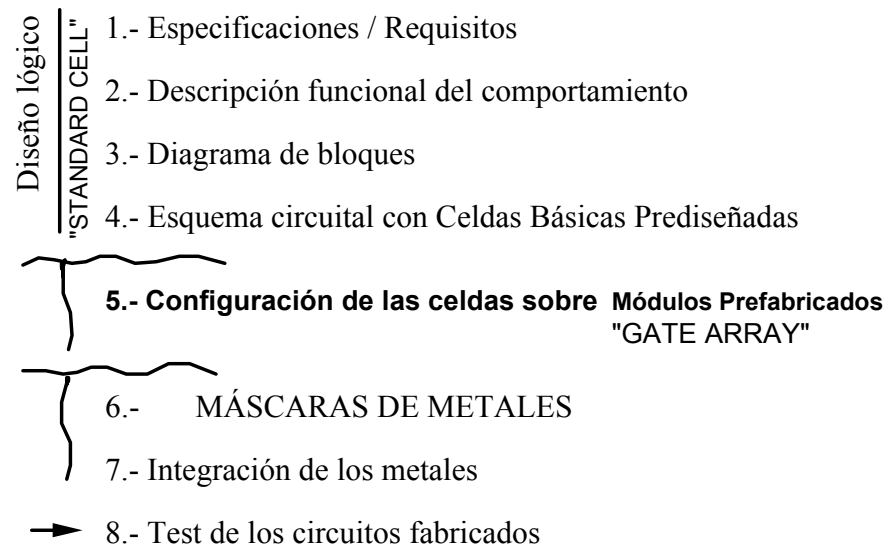
Las celdas básicas incluidas en una librería tienen dimensiones geométricas ajustadas entre sí, para facilitar el encaje de unas con otras. Las tareas de ubicación de las celdas y conexinado de las mismas (*placement and routing*) son ejecutadas con la ayuda de potentes herramientas informáticas y suelen ser realizadas por el diseñador, en contacto y colaboración directa con el fabricante. La sustitución de las celdas por su *layout* es realizada directamente por el fabricante.

La descripción del circuito puede ser gráfica (captura de esquemas) o textual (lenguajes de descripción circuital o descripción de *hardware*) o mezcla de ambas. A medida que aumenta la complejidad de los circuitos integrados resultan cada vez más útiles los lenguajes de descripción circuital (VHDL, Verilog,...), que permiten describir un circuito a través de su funcionamiento, sin descender a su configuración (puertas, biestables o bloques); de construir tal configuración se encarga el correspondiente compilador.

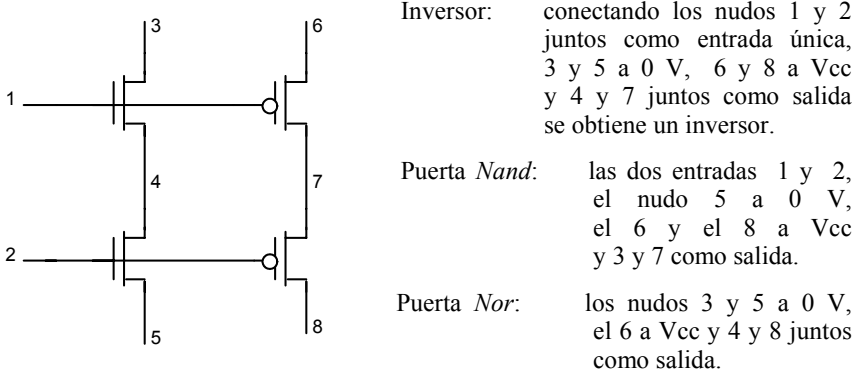
Fabricación con matrices de puertas prefabricadas

En cuanto al proceso de fabricación, una posibilidad de reducir costes consiste en utilizar circuitos integrados ya fabricados a nivel de puertas (*gate array*) o de módulos equivalentes, sobre los cuales se configurarán (por conexión mediante las capas de metales) las celdas de la librería.

De esta forma, el circuito particularizado sólo requiere las máscaras correspondientes a los metales y a sus contactos:



La configuración típica de una puerta o módulo básico *gate array* suele ser del tipo representado en la figura siguiente:



21.3. Herramientas de diseño

El proceso de diseño de un circuito integrado digital (en sus diferentes modalidades) se desarrolla sobre computador, en forma de tareas de tipo CAD, para cuya ejecución se dispone de eficaces y potentes herramientas informáticas. Las herramientas que se utilizan para el diseño digital con circuitos programables y para el diseño de ASICs digitales son básicamente las mismas y se emplean de la misma forma, diferenciándose luego en la forma de compilar el diseño y en los resultados de tal compilación.

El diseño de un circuito integrado digital es básicamente un proceso de descripción del mismo, junto con la simulación de su comportamiento para verificar que el diseño es correcto, conforme a las especificaciones o requisitos del mismo. La descripción digital puede hacerse a diferentes niveles, en particular, a nivel geométrico (*layout*), a nivel lógico (esquemático) o a nivel funcional («programa»).

El nivel geométrico (descripción física del circuito integrado) no tiene sentido en el diseño sobre circuitos integrados programables (pues ya están fabricados) y es utilizado en forma muy limitada en el diseño de ASICs con librería, es decir en el diseño *standard cell* o *gate array* (solamente se usan algoritmos de colocación y conexionado de las celdas de la librería, que ya están diseñadas).

Los niveles lógico y funcional ofrecen dos alternativas de diseño: en forma gráfica de esquema de puertas y biestables, adecuadamente conectadas, o en forma de texto, describiendo el comportamiento del circuito; actualmente, y cada vez más, predomina la descripción en texto, utilizando un lenguaje de descripción circuital (HDL, *Hardware Description Language*).

Existen, pues, tres niveles de descripción de los diseños digitales:

- nivel geométrico (*layout*): descripción gráfica de las diferentes regiones físicas que conforman el circuito integrado (difusiones, polisilicio, metal, contactos,...), o sea, el dibujo que corresponde a las máscaras con las que se fabricará el integrado; este nivel es propio y casi exclusivo del diseño completo (*full custom*),
- nivel lógico (esquemático): descripción, en términos de puertas lógicas y biestables; es el nivel que corresponde directamente al diseño con librería,
- nivel funcional (texto): descripción del comportamiento del circuito en un lenguaje de descripción circuital (como puede ser VHDL o Verilog).

El tipo de tareas que permiten abordar las herramientas informáticas, según los tres niveles anteriores, está resumido en la tabla de la página siguiente.

1 Nivel Funcional**➔ Lenguajes de Descripción Funcional**

- Edición y análisis de la descripción.
- Simulación funcional.
- Compilación lógica.

2 Nivel Lógico**➔ Descripción**

- Captura de esquemas.
- Generación de celdas matriciales (ROM, PLA, RAM, ...).

➔ Simuladores Lógico-temporales**➔ Análisis Temporal**

- Verificación de tiempos de anticipación y de mantenimiento.
- Cálculo de retardos y detección de caminos críticos.

➔ Test

- Simulación de fallos.
- Cobertura de test.
- Generación automática de test: ATPG.

3 Nivel Geométrico**➔ Descripción**

- Colocación y conexionado de celdas: *Placement and Routing*.
- Edición gráfica.

➔ Simulación

- Extractores de parámetros: *back-annotation*.
- Simuladores Eléctricos.

➔ Comprobación Lógica

- Extractores lógicos.
- Verificación de correspondencia.

➔ Verificadores de reglas de diseño: DRC.

En todos los niveles es preciso comprobar que el diseño es correcto, a través de la simulación de su comportamiento:

- en el nivel geométrico, se utilizan simuladores eléctricos, en particular el simulador SPICE, extrayendo previamente, sobre el *layout*, los transistores y los componentes capacitivos y resistivos presentes;
- en el nivel lógico, se emplean simuladores lógico-temporales que actúan en términos de funciones booleanas y de retrasos temporales (tiempos de propagación);
- en el nivel funcional, los lenguajes de descripción circuital ofrecen simuladores del comportamiento del circuito, tal como queda descrito en dichos lenguajes.

Además, en el nivel lógico se aplica el análisis de tiempos para verificar que se respetan los tiempos de propagación, de anticipación y de mantenimiento (y, en el caso de ASICs, se obtiene el conjunto de vectores de test necesarios en el proceso de integración para comprobar que cada ejemplar fabricado responde efectivamente al diseño).

En el nivel funcional (lenguajes de descripción circuital), se dispone de compiladores que trasladan la descripción al nivel lógico. En dicho nivel lógico, los compiladores para circuitos integrados programables, traducen el diseño en el «mapa de fusibles» que corresponde a su programación; en el caso de ASICs (*standard cell* y *gate array*), del nivel lógico se pasa al nivel geométrico mediante algoritmos de ubicación y conexionado que sustituyen la descripción lógica por la conexión de las correspondientes celdas de la librería, adecuadamente situadas.

El compilador para CPLDs trasladará la descripción (esquemática o en texto) a funciones booleanas, expresadas en suma de productos, y biestables tipo D y tratará de encajar dichas funciones y biestables en las macroceldas que conforman el dispositivo programable. El resultado será un «mapa de fusibles» que describe la forma en que deben quedar los diversos términos producto y, en su caso, las conexiones entre macroceldas; un programador ejecutará dicho mapa de fusibles sobre el correspondiente circuito integrado programable.

El compilador para FPGAs pasará a funciones booleanas de un número de variables limitado, descomponiendo cada función en varias si es preciso, para insertarlas en las celdas lógicas que componen el dispositivo; el «mapa de valores» (denominación que es más adecuada que la de «mapa de fusibles», habida cuenta de que predomina en este caso la programación de tipo SRAM) describirá las funciones booleanas, mediante su «tabla de verdad», la configuración de las celdas y las conexiones entre ellas.

En el caso de ASICs, la descripción funcional se compila utilizando la librería de celdas disponible para pasarla a un esquemático de celdas (nivel lógico) y, posteriormente, se traslada a nivel geométrico, mediante los algoritmos de colocación y conexionado de las celdas (cuya dimensión geométrica ya se conoce pues su *layout* se ha diseñado previamente).

21.4. Validación de circuitos integrados: simulación, análisis de tiempos y test

En el proceso de diseño y fabricación de un circuito integrado se utilizan tres tipos de procedimientos de simulación, verificación y supervisión que tienen objetivos y metodologías de ejecución totalmente diferentes:

- la *simulación funcional*
- el *análisis de tiempos*
- y el *test del circuito fabricado*.

Estos tres procedimientos se refieren, respectivamente, a la verificación del comportamiento del circuito resultante del proceso de diseño, a la verificación de los tiempos implicados en tal circuito y a la verificación de cada circuito integrado obtenido tras el proceso de fabricación:

1 verificación del comportamiento

- **Simulación** → **Funcional**
- **Lógica-temporal**
- **Eléctrica**

2 verificación de tiempos

- **Análisis de tiempos** → **Tiempos de anticipación y de mantenimiento**
- **Tiempos de propagación y caminos críticos**

3 verificación del circuito obtenido

- **Test** → **Simulación de fallos**
- **Cobertura de fallos**

La *simulación funcional* trata de comprobar si el diseño realizado funciona adecuadamente, es decir, si se ajusta a las especificaciones o requisitos que se pretendían alcanzar. Consiste en reproducir el funcionamiento real del circuito para verificar que es correcto, incluyendo las más diversas situaciones posibles de sus entradas y comprobando la ausencia de errores en la respuesta del circuito. Es un estudio estímulo/respuesta (entradas/salidas) en relación con el funcionamiento normal del circuito; se realiza con simuladores lógico-temporales que evalúan la respuesta booleana a los vectores de entrada y calculan también el tiempo de respuesta: cada celda básica es sustituida por la correspondiente función booleana y por un tiempo de propagación que tiene en cuenta la carga (*fan out* y capacidad equivalente) que soporta su salida.

El *análisis de tiempos* trata de comprobar que se respetan las restricciones temporales internas (compatibilidad entre los tiempos de propagación y los tiempos de anticipación *set-up* y de mantenimiento *hold*) y de calcular la velocidad máxima de trabajo del circuito; asimismo, aporta información sobre los caminos en que se violan las restricciones temporales y sobre los caminos que resultan críticos respecto a la velocidad máxima de funcionamiento del circuito.

El análisis de tiempos se efectúa por caminos: un camino es el recorrido existente entre dos biestables, se inicia en la salida de un biestable y finaliza en la entrada de otro (sin atravesar ningún biestable intermedio). Se realiza mediante un cálculo directo de tiempos entre la entrada de cada biestable y las salidas de aquellos biestables que influyen sobre ella: cada celda básica combinacional es sustituida por sus tiempos de propagación (teniendo en cuenta en ellos la situación real de carga *-fan out* y capacidad equivalente— que les afecta) y para cada biestable se tienen en cuenta sus tiempos de propagación (biestable origen) y los de anticipación y de mantenimiento (biestable final del camino).

El análisis de tiempos no tiene en cuenta, en cambio, la función concreta que el circuito va a realizar: no utiliza vectores de entrada y de salida, ni efectúa la simulación del circuito sino que meramente calcula tiempos entre biestables; supone de partida que el diseño es síncrono y los tiempos se calculan con referencia al flanco activo del reloj.

Ambos procedimientos de comprobación/verificación (*simulación funcional* y *análisis de tiempos*) se ejecutan en las diferentes etapas del diseño: tanto en el nivel lógico, sobre el esquemático del circuito, como en el nivel geométrico, sobre las máscaras que conectan con el proceso de fabricación. Cuanto más cercana es la información que utilizan al nivel físico definitivo del circuito más precisa es la simulación temporal y el análisis de tiempos: cuanto más próximos nos situemos a la configuración física real del circuito integrado, mayor precisión puede obtenerse en los aspectos de carga efectiva que soportan las salidas de las celdas básicas y en los retardos debidos a los efectos resistivos y capacitivos que introducen las conexiones (es decir, en aquellas cuestiones que afectan a los cálculos temporales).

Obviamente, el análisis de tiempos es la verificación propiamente «temporal», pero también la simulación funcional tiene en cuenta el tiempo ya que involucra dos aspectos: el lógico relativo a las funciones booleanas que ejecutan las celdas y el temporal correspondiente a sus tiempos de propagación; es una simulación *lógico-temporal*.

La *simulación funcional* se efectúa «globalmente» (calculando las salidas que produce el circuito en función de las ondas de entrada que recibe) y *suministra información sobre la validez del diseño* (sobre si es correcto o no); en su caso, obliga a rediseñar parcial o totalmente el circuito para que su funcionamiento coincida con el que se pretende.

El análisis de tiempos se efectúa por caminos (calcula tiempos entre la entrada de un biestable y cada una de las salidas de los biestables que le afectan), *ayuda a aumentar la velocidad de trabajo del circuito* (detecta los caminos críticos, aquellos que presentan mayor tiempo de propagación entre biestables) y *proporciona información sobre las violaciones de los tiempos de mantenimiento (hold)* de los biestables. Los tiempos de anticipación (*set-up*) de los biestables intervienen en la velocidad máxima del circuito sumándose a los tiempos de propagación en los diversos caminos; asimismo, aun no existiendo problemas de velocidad, han de respetarse los tiempos de mantenimiento.

La violación de un tiempo de *set-up* requiere reducir los tiempos de propagación intermedios (disminuyendo el número de puertas intermedias o utilizando puertas más rápidas) o bien admitir una velocidad de trabajo inferior (mayor período de reloj, es decir, mayor intervalo temporal para formar el nuevo dato y presentarlo en la entrada del biestable). La violación de un tiempo de *hold* requiere aumentar el tiempo de permanencia del dato en la entrada del biestable respecto del flanco activo del reloj, es decir, aumentar el tiempo de propagación a partir de la salida del biestable anterior (incluyendo un par de inversores para retrasar la señal o utilizando puertas de mayor tiempo de propagación). Obviamente puede suceder que la corrección de una violación de *hold* produzca una violación de *set-up* y, consiguientemente, exija una velocidad de trabajo menor.

Tanto la simulación funcional como el análisis de tiempos son comprobaciones que se ejecutan en el proceso de diseño y pueden determinar una vuelta atrás en el mismo para corregir lo que «no funciona bien». En cambio, *el test de un circuito integrado* es la comprobación, posterior a la fabricación del mismo, que *garantiza que el circuito se ha fabricado bien*, es decir, que todas las celdas que configuran el circuito han sido integradas correctamente de forma que el resultado físico coincide con el esquema circuital resultante del diseño.

El test no equivale, ni mucho menos, a la simulación funcional: no se trata de verificar que el funcionamiento «normal» del circuito es el deseado sino de comprobar que todos y cada uno de los transistores han sido fabricados y conectados correctamente. Téngase en cuenta que, si el circuito es relativamente complejo, su funcionamiento «total» no puede ser simulado: requeriría una secuencia enormemente alta de vectores de test.

El test es, por tanto, mucho más exigente que la simulación funcional: que el diseño es correcto queda garantizado, en la misma etapa de diseño, por la simulación funcional y, complementariamente, por el análisis de tiempos en lo que se refiere a restricciones temporales; con el test de lo que se trata es de garantizar, para cada uno de los circuitos integrados, que tal diseño ha sido fabricado correctamente, es decir, que cada uno de los circuitos físicos obtenidos (*chips*) corresponde exactamente al diseño.

El test va más allá de una comprobación funcional de los casos de interés: los vectores de test pretenden verificar que todos los nudos booleanos presentes en el circuito integrado son capaces de actuar correctamente, lo cual asegura, en gran medida, que el circuito físico responde efectivamente al diseño efectuado y no contiene errores debidos al proceso de integración. En la práctica, es imposible la comprobación funcional completa de un circuito digital complejo, recorriendo todos los casos posibles; el test permite efectuar una comprobación «completa» de todos los nudos booleanos del mismo, conforme a un modelo conceptual establecido para tal finalidad.

El test se efectúa, también, mediante un estudio estímulo/respuesta (entradas/salidas) utilizando una secuencia apropiada de vectores test (que no coincide con la secuencia propia de la simulación funcional, aunque ésta puede formar parte del conjunto global de vectores de test). Los vectores de test han de permitir «controlar», desde las entradas del circuito integrado, cada uno de los «nudos booleanos» internos del mismo (pudiendo situarlos a valor **0** y a valor **1**) y «observar», desde las salidas del circuito integrado, el estado booleano de cada uno de dichos nudos; de esta forma, podremos comprobar que cada celda básica está en su sitio y que actúa correctamente.

Aun cuando la aplicación del test es posterior a la fabricación, afecta directamente al diseño por cuanto que el circuito ha de ser «testeable» (no todos lo son) y, además, la obtención de los vectores de test forma parte del diseño del circuito. Es decir, todo el proceso de diseño ha de estar orientado al test; no basta un diseño funcional sino que hay que efectuar, a la vez, un diseño para el test, siendo así que, en muchas ocasiones, los aspectos relativos al test resultan más difíciles y complejos que el propio diseño funcional. [El capítulo 25 se dedica íntegramente al test de circuitos integrados digitales.]

22 CIRCUITOS INTEGRADOS PROGRAMABLES: MATRICES DE CELDAS CONFIGURABLES

- 22.1. Resumen de la configuración y características de los CPLDs
- 22.2. Los biestables como componentes programables: SRAM
- 22.3. Matrices de celdas programables: FPGAs

Los circuitos integrados programables constituyen una de las opciones de «construir» la electrónica digital en tamaño «micro», es decir, de insertar nuestro diseño digital en un único circuito integrado; la otra es fabricar específicamente dicho diseño.

Dos son los grandes tipos de dispositivos programables, que se diferencian según su «granularidad» (según el tamaño de su celda básica) y por la forma de configurar las funciones booleanas: los de granularidad gruesa y configuración PAL (funciones en forma de suma de términos producto) y los de granularidad fina y configuración LUT (tabla funcional a través de multiplexores): CPLDs y FPGAs. Este capítulo incluye un comprensivo repaso de los CPLDs que ya fueron objeto de estudio en el capítulo 13 (segundo volumen) y la presentación en detalle de las FPGAs.

Frente a la forma física de programación, habitual en los CPLDs (fusibles y transistores NMOS de doble puerta), las FPGAs utilizan una programación más «suave», por almacenamiento de valores sobre biestables: programación SRAM, cuyo fichero debe ser nominado «mapa de valores» (en lugar del clásico «mapa de fusibles»).

Una FPGA contiene bloques lógicos (formados por varias celdas lógicas elementales), bloques de entrada/salida y conexiones entre ellos; dichos tres componentes (cada uno de ellos) son de una gran potencia, en cuanto a posibilidades de programación.

Cada celda lógica permite su utilización como función booleana, como registro de desplazamiento, como pequeña memoria RAM o ROM, como multiplexor rápido,...; además, incluye recursos específicos para operaciones aritméticas: sumadores, restadores, multiplicadores y contadores de muy alta velocidad.

Los adaptadores de entrada/salida son celdas periféricas que ofrecen su programación como entradas, como salidas o como terminales bidireccionales, con sincronización o no de la entrada y la salida, con capacidad de alta impedancia y con la posibilidad de seleccionar los niveles de las tensiones lógicas, las pendientes de las señales (slew rate), ...

Una tupida malla de conexiones programables, de diversos tipos de longitud (cortas, dobles y largas), a través de los canales horizontales y verticales que hay entre los bloques, ofrece una amplísima conectividad entre las diversas celdas, optimizando los retrasos que introducen las líneas de conexión.

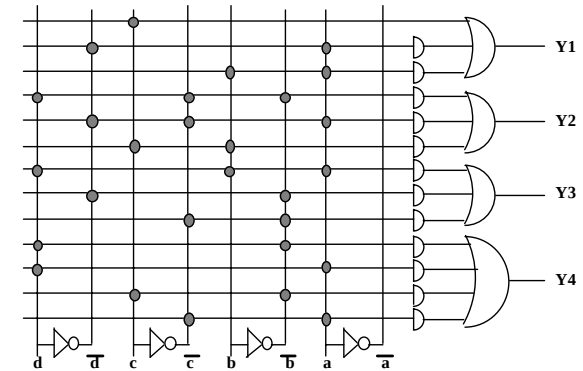
A todo ello se une la disponibilidad de bloques específicos de memoria RAM, de multiplicadores e, incluso, de microprocesadores, todo dentro de una misma FPGA. En suma, cualquier sistema digital, por grande y complejo que sea, puede ser «insertado» en una FPGA, en forma eficiente.

22.1. Resumen de la configuración y características de los CPLDs

Estructura PAL + biestables

Recordemos, una vez más, que la finalidad de la microelectrónica es *configurar un circuito digital específico en el interior de un circuito integrado* (es decir, «insertar nuestro diseño en un integrado») y que una manera de conseguir este objetivo consiste en programar el circuito específico en un circuito integrado programable.

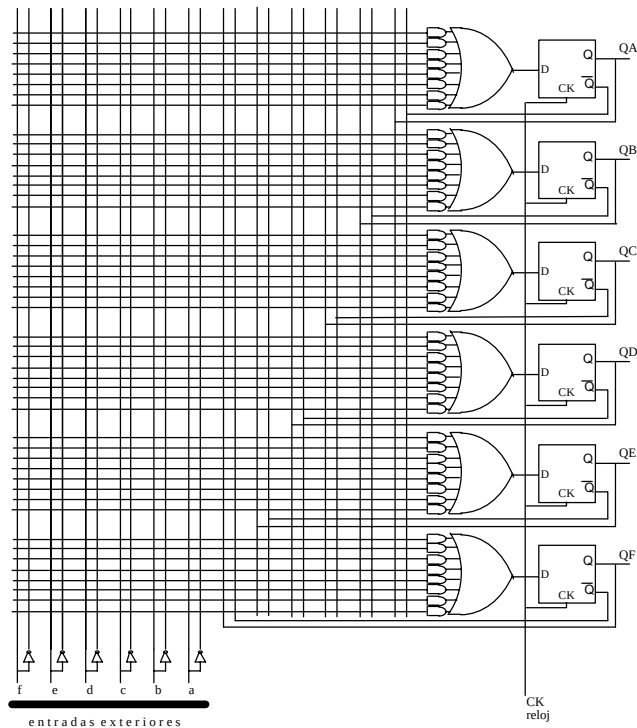
La configuración reticular de varias funciones booleanas, expresadas en forma de suma de productos, recibe el nombre de **PAL** (*product adding layers* –estratos o módulos sumadores de productos–).



$$\begin{aligned} Y1 &= c + \bar{d}.a + b.a & Y2 &= d.\bar{c}.b + \bar{d}.\bar{c}.a + c.b \\ Y3 &= d.b.a + \bar{d}.\bar{b} + \bar{c}.\bar{b} & Y4 &= d.\bar{b} + d.a + c.\bar{b} + \bar{c}.a \end{aligned}$$

Cada función (suma de productos $\sum p$) ocupa un módulo separado y dispone de sus propios términos producto; las conexiones de las variables se ajustan a la retícula formada por las entradas y sus negadas sobre las «líneas de entrada» de las puertas "y".

Para poder programar sistemas secuenciales se requiere, además de funciones booleanas, biestables como «memoria» para las variables de estado. Bata, para ello, añadir a la configuración **PAL** la capacidad de memoria que proporcionan los biestables: incluir un biestable en cada una de las salidas de un bloque **PAL**. De esta forma tendremos bloques con **n** módulos, cada uno de ellos con un biestable cuya entrada **D** es activada por una función en forma de suma de productos, cuyas variables serán las propias variables de entrada y, también, las salidas de los biestables (realimentación). Tales bloques reciben la denominación de **PLS** (secuenciadores lógicos programables) o, también, la de **PLD** (dispositivos lógicos programables).



Componentes programables

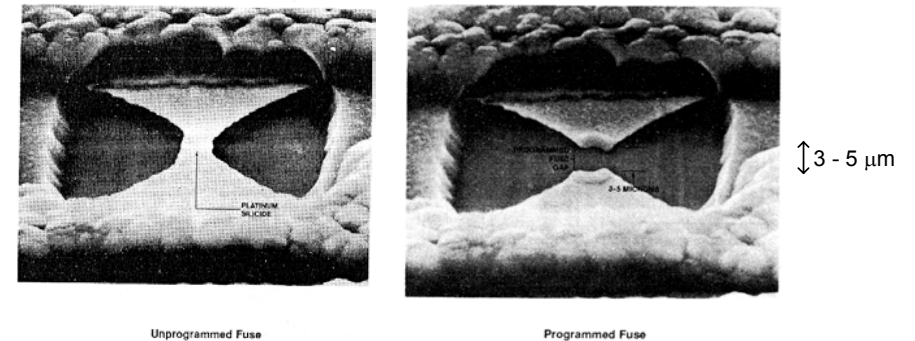
Recordemos las formas de conseguir que una puerta lógica sea programable: ¿cuáles son los mecanismos físicos que permiten tal programación? Básicamente hay dos formas de dotar de «capacidad de programación» a una puerta lógica, dos tipos de componentes circuitales que permiten la conexión-desconexión física entre líneas del circuito:

- > fusibles
- > y transistores MOS de doble puerta

y, dentro de los transistores de doble puerta, se dispone de tres tipos diferentes:

- ◁ EPROM (borrables por luz ultravioleta)
- ◁ EEPROM (borrables eléctricamente)
- ◁ FLASH (con borrado eléctrico global y mayor densidad de integración).

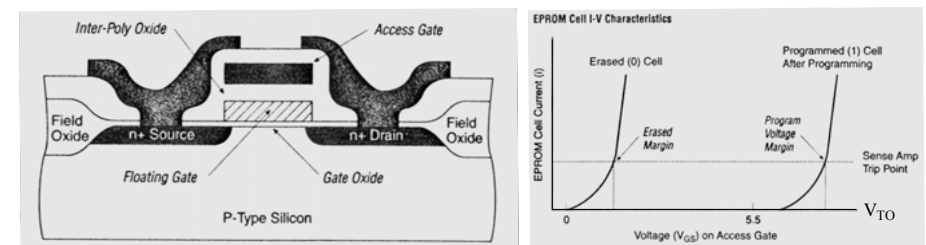
Un fusible se construye mediante un estrechamiento en un conductor de un material apropiado, de forma que pueda «fundirse» por efecto térmico mediante el paso de una intensidad relativamente elevada; suele tener un tamaño aproximado de $5 \times 5 \mu\text{m}$ y se utilizan aleaciones de platino-silicio, titanio-tungsteno o níquel-cromo (nicrom).



Los fusibles se utilizan principalmente en tecnologías bipolares de alta velocidad; se programan mediante «fusión térmica», a través de pulsos de intensidad alta, con una fuerte pendiente dI/dt para generar tensiones puntuales apreciables (por efecto inductivo): tensiones típicas de 10,5 V aplicadas en pulsos repetitivos de anchura entre 10 y 50 μs , dando lugar a intensidades de 0,1 - 0,5 A con tiempos de subida inferiores a 100 ns.

Los fusibles son de tipo destructivo en cuanto a su programación; por ello, no son «reprogramables», es decir, una vez programados no pueden «borrarse» y volver a la situación inicial ya que la conexión ha sido «cortada» y no puede repararse. Este tipo de dispositivos se denomina **OTP** (*one time programmable*), a diferencia de los dispositivos **EPROM** que son reprogramables.

Los transistores MOS de doble puerta (MOS EPROM) se programan mediante almacenamiento de carga en la puerta interior; dicha carga modifica la tensión umbral del transistor MOS, situándola por encima de la tensión de alimentación: la tensión umbral V_{TO} pasa del entorno de 1 V a unos 8 V y el transistor no entra nunca en conducción.



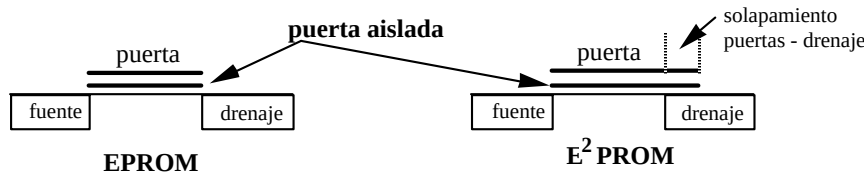
Hay tres tipos de transistores MOS programables:

- i) EPROM, borrables mediante luz ultravioleta (para descargar la puerta aislada).

Su programación se hace a través de una fuerte corriente de canal y de una tensión relativamente alta en la puerta externa (pulsos de tensión de unos 12 V sobre drenaje y sobre puerta). El alto coste que supone el encapsulado con ventana hace que estos dispositivos EPROM se ofrezcan también en versión **OTP** (sin ventana).

- ii) E²PROM, borrables eléctricamente, cuya programación y borrado se hace mediante tensión entre la puerta externa y el drenaje.

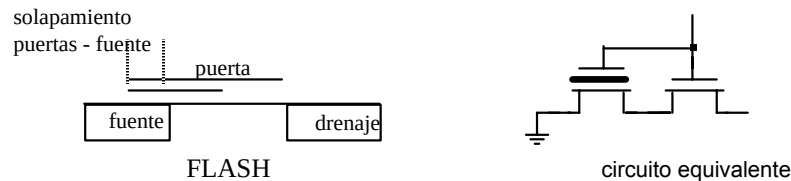
Un pequeño solapamiento de las dos puertas con el drenaje permite que el proceso de carga de la puerta aislada sea reversible: se carga y se descarga desde el drenaje por atracción (tensión de puerta positiva) o repulsión de electrones (tensión negativa).



El borrado es individual (se efectúa sobre un solo transistor), pero presenta un problema de «sobreborrado»: cuando la tensión de la puerta exterior es negativa la puerta aislada suele cargarse positivamente y se genera un canal permanente que pone en conducción al transistor. Por ello, es necesario utilizar dos transistores en serie: el de doble puerta para conectar y desconectar al segundo, que es un transistor NMOS normal.

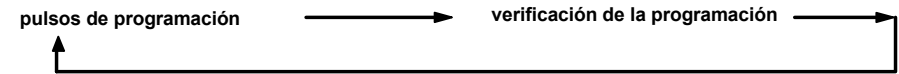
- iii) FLASH, borrables eléctricamente, cuya programación se hace por corriente de canal y el borrado por tensión entre puerta y fuente.

Este tercer tipo de transistores proviene de agrupar los dos transistores de la celda E²PROM en uno solo: las dos puertas se solapan con la fuente y la puerta aislada sólo ocupa la mitad de la longitud del canal.



El proceso de carga de la puerta aislada es del tipo EPROM, con una intensidad de canal alta que deriva electrones hacia dicha puerta y la descarga se produce según el tipo E²PROM, por pulsos de tensión positiva sobre la fuente, que atrae a los electrones de la puerta aislada. El borrado no puede realizarse individualmente sino que afecta a todos los transistores del bloque (la fuente es común a todos los transistores NMOS) y no hay riesgo de «sobreborrado» (aunque la puerta aislada se cargue positivamente, no puede poner en conducción al transistor, ya que sólo actúa sobre la mitad del mismo).

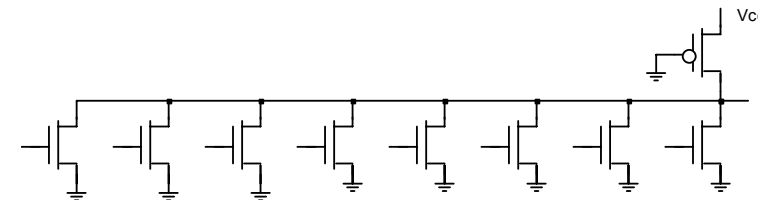
Los dispositivos programables (fusibles y EPROM) se diferencian en el proceso físico que determina su programación: ruptura por efecto térmico y almacenamiento de carga, respectivamente; sin embargo, el proceso de programación de ellos es análogo: se aplica una secuencia de pulsos de tensión relativamente elevada que se repite cíclicamente hasta estar seguros de que la correspondiente conexión ha quedado programada.



Configuración pseudoNMOS

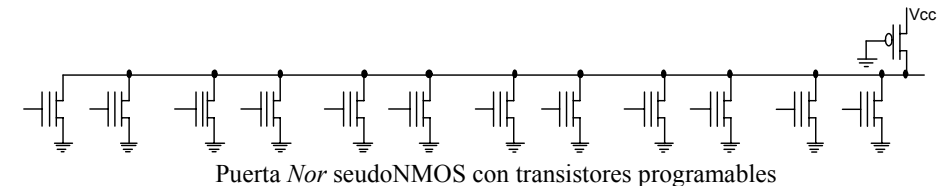
En general, los bloques programables **PAL** requieren puertas con un alto número de entradas: cada término producto recibe un alto número de variables (sumando las de entradas, las salidas de los biestables realimentadas y las negadas de ambas) y en cada módulo el número de términos producto puede ser relativamente alto.

La conexión en serie de un número alto de transistores se evita utilizando puertas "o-negada" (*Nor*) pseudoNMOS: el plano N tiene todos sus transistores NMOS en paralelo y el plano P se reduce a un solo transistor como resistencia de polarización (tales puertas tienen el inconveniente de que su consumo estático, para salida **0**, no es nulo).

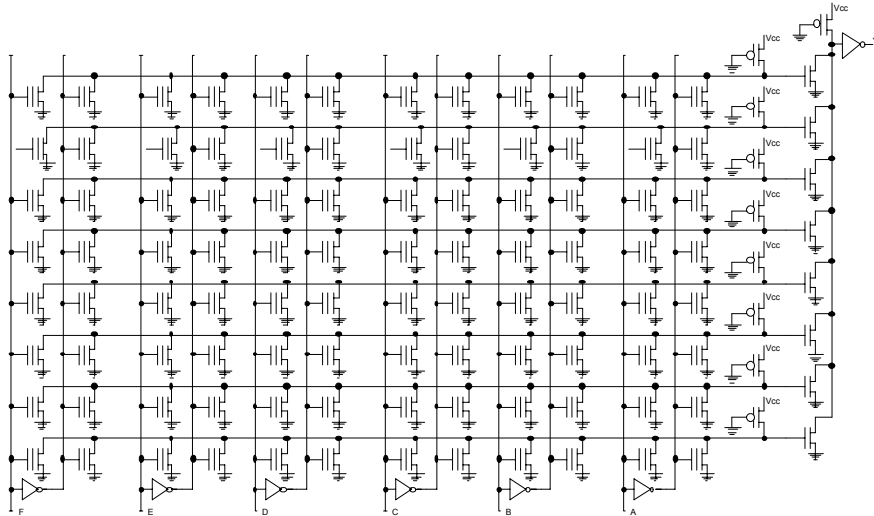


Además, en cuanto a dispositivos programables, esta configuración pseudoNMOS hace que solamente sea necesario programar el plano N, pues el plano P es fijo, constituido por un solo transistor: solamente se utilizan transistores EPROM de tipo NMOS y su programación consiste en eliminar («desconectar») los correspondientes transistores.

Ciertamente la programación de ambos planos sería sumamente compleja, pues, en uno de ellos, los transistores se encontrarían en serie y su programación consistiría en «puentearlos» (lo cual es mucho más difícil que «desconectarlos»).



La figura siguiente representa un módulo **PAL** de 6 entradas y 8 términos producto.

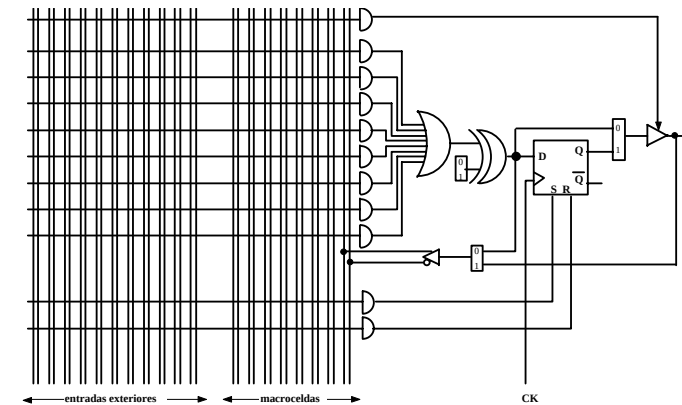


Macroceldas

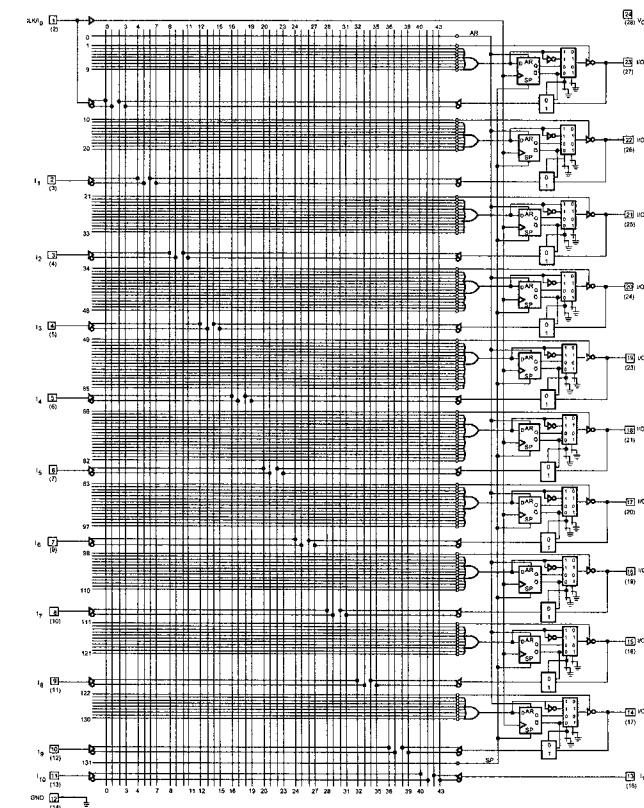
Una macrocelda está compuesta por:

- una suma de productos programables de múltiples entradas entre las que se encuentran su salida y las salidas de otras macrocelas (realimentación), estando disponible cada entrada en su forma afirmada y negada,
- una puerta "**o-exclusiva**" programable que permite configurar la función de la macrocelda (suma de términos producto) en forma afirmada (y) o negada ($\bar{y}$, que equivale a producto de términos suma),
- un biestable que recibe dicha función,
- un selector que permite «puentear» el biestable y que configura, por tanto, la macrocelda como combinacional o como secuencial,
- y un adaptador triestado con capacidad para «desconectar» la macrocelda respecto del terminal de salida, lo cual da lugar a las siguientes posibilidades:
 - la simple desconexión (alta impedancia) del terminal,
 - su utilización bidireccional (I/O),
 - su disponibilidad como entrada, caso de que la macrocelda no sea utilizada.

Mientras el número de macrocelas de un circuito integrado programables no es alto, cada una de ellas recibe todas las entradas del dispositivo y la realimentación de todas las salidas del mismo, que coinciden directamente con las salidas de todas sus macrocelas: en caso de que alguna macrocelda no sea utilizada, la salida correspondiente puede ser usada como entrada.



Macrocelda con 8 términos producto



Configuración del circuito integrado programable PLD 22V10

Si bien el nombre de **PLD** es genérico (y, en principio, engloba a cualquier circuito integrado programable) suele utilizarse para referirse a los dispositivos programables de tipo PAL, es decir, módulos PLS: PAL + biestable.

Cuando se trata de un gran número de macroceldas dentro de un mismo integrado no resulta adecuado que cada una de ellas reciba todas las entradas y se conecte con todas las macroceldas ya que la cantidad de conexiones programables sería sumamente elevada; tampoco es viable que las salidas de todas las macroceldas se conecten a terminales de salida del circuito integrado.

Para limitar el tamaño del circuito y el número de sus terminales, las macroceldas se agrupan en bloques disjuntos y reciben solamente las entradas propias del bloque y la realimentación de las macroceldas del mismo; además, no todas las salidas de las macroceldas están conectadas a terminales de salida sino solamente un número reducido de cada bloque.

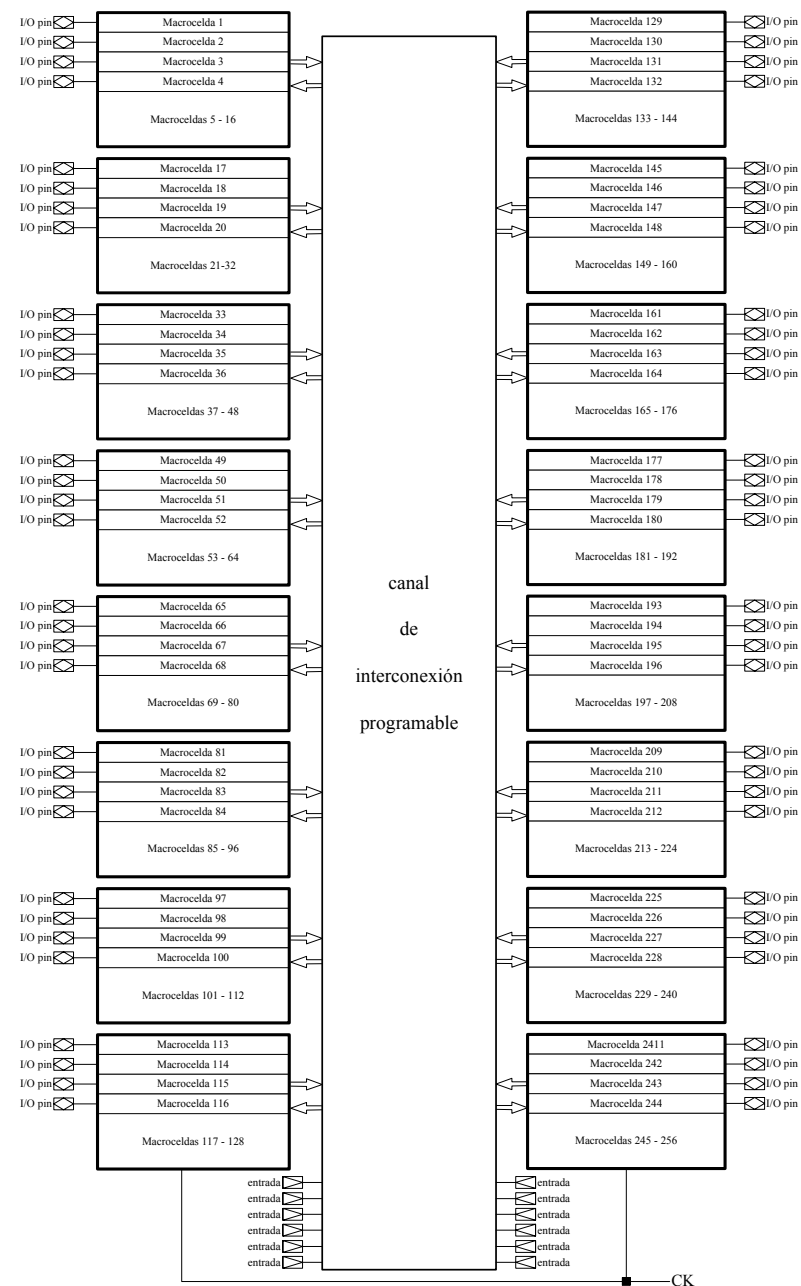
Los bloques se conectan entre sí a través de un bus central de conexiones programables conformando una estructura ramificada: un canal o eje de conexiones (*a manera de tronco*) sobre el cual se insertan los bloques de macroceldas (*como ramas del mismo*). Este tipo de circuitos integrados programables de amplio número de macroceldas con arquitectura ramificada es nombrado con las siglas **CPLD** (dispositivos lógicos programables complejos).

Para reducir la superficie de integración y evitar un alto desaprovechamiento de las estructuras PAL, el número de términos producto de cada macrocelda suele ser pequeño (4 ó 6) y para ampliarlo, cuando se necesite, se incluyen en cada bloque «expansores», en forma de términos producto adicionales que, sin pertenecer directamente a ninguna de las macroceldas del bloque, pueden ser utilizados por cualquiera de ellas.

La figura de la página siguiente representa un CPLD de 256 macroceldas, de las cuales solamente 64 pueden utilizarse como salidas exteriores (las otras 192 macroceldas son internas); el circuito dispone de una entrada específica de reloj y de 12 entradas exteriores, pudiéndose emplear también como entradas los terminales correspondientes a macroceldas que no se usen como tales.

Para proteger los diseños almacenados en estos dispositivos, frente a duplicación o copia de los mismos, suelen disponer de un «fusible de seguridad» que, una vez programado, impide la «lectura» del «mapa de fusibles» y, con ello, la duplicación del circuito.

Además, suelen ofrecer la posibilidad de seleccionar la velocidad de trabajo (y con ello, el consumo) a través de la programación de un «fusible de velocidad»: el dispositivo puede trabajar con dos conjuntos de tiempos funcionales diferenciados; cuando se selecciona máxima velocidad el consumo es mayor, pudiendo actuar con menores intensidades si la velocidad no es tan alta.



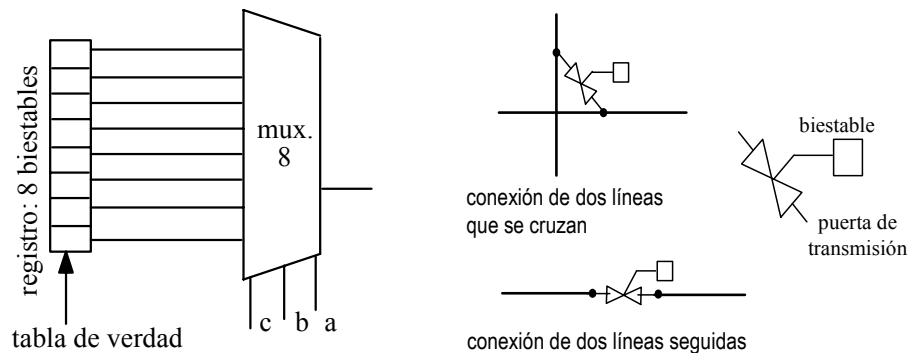
Configuración CPLD con 256 macroceldas

En paralelo con el desarrollo de las FPGAs, las series recientes de CPLDs incluyen también celdas periféricas adaptadoras de entradas/salidas análogas a las que se describen más adelante. Asimismo existen CPLDs de muy alto número de macroceldas con la estructura reticular propia de las FPGAs: los bloques de macroceldas separados por canales de conexión horizontales y verticales. Estas características de los CPLDs «avanzados» serán estudiadas en el apartado 3.

22.2. Los biestables como componentes programables: SRAM

En la configuración LUT (*look-up-table*) las funciones se construyen a través de su tabla funcional, mediante un multiplexor que «toma valores» sobre dicha tabla; dichos valores pueden ser almacenados en un registro.

Las conexiones entre funciones (o entre módulos) son seleccionadas, también, a través de biestables que actúan sobre puertas de transmisión: el valor booleano (1/0) almacenado en un biestable determina que las dos líneas conductoras enlazadas por la puerta de transmisión queden conectadas entre sí o permanezcan independientes.



De esta forma, las funciones se configuran mediante su tabla de verdad que se memoriza en un registro y las conexiones se establecen a través de los valores que se almacenan también en registros (en conjuntos de biestables).

En ambos casos, se establece una programación por memoria (*software*), a través de un amplio número de biestables que almacenan las tablas de las funciones y la selección de conexiones entre líneas. De forma que tanto las funciones booleanas de los módulos (*look-up-table*) como las conexiones (selección de caminos posibles) se programan almacenando valores booleanos sobre registros; las siglas **SRAM** identifican este tipo de «programación por memoria estática».

Este tipo de dispositivos, con programación SRAM, se denomina FPGA y será estudiado en detalle en el próximo apartado; están formados por:

- celdas con funciones lógicas configurables y biestables
- celdas adaptadoras de entradas y salidas configurables
- conexiones configurables
- y la memoria (el conjunto de biestables) donde se almacena la configuración de todo lo anterior junto con el circuito de control de la misma.

La programación se refiere tanto a las «tablas de verdad» de las funciones como a la configuración interna de las celdas en que éstas se encuentran y la de las celdas adaptadoras de entrada/salida y a las conexiones entre celdas. En conjunto será un amplio número de biestables cuyas salidas actúan sobre líneas de entrada de multiplexores (en el caso de «tablas de verdad») o sobre las entradas de control de otros multiplexores (en el caso de configuración de celdas o de conexiones entre ellas).

El número de biestables es muy alto: una FPGA mediana, de unos 2000 módulos lógicos suele tener del orden de 10^6 biestables de programación, es decir, unos 500 biestables por cada módulo, de los cuales unos 50 son de configuración del propio módulo y el resto de programación de las conexiones entre módulos.

En relación con su programación, es decir, con el almacenamiento de los valores booleanos correspondientes a un determinado diseño, todos estos biestables se encuentran conectados formando un largo registro de desplazamiento, en el cual se introduce, bit a bit, el «mapa de valores» que corresponde a dicho diseño. Este registro de desplazamiento es análogo al que se forma con los biestables de un circuito secuencial para efectuar el test del mismo: *scan path* (que será estudiado en 25.4).



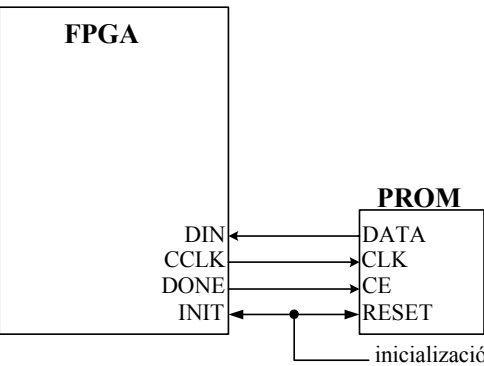
Una entrada de control PROG (selector de modo de programación), determina el paso del circuito a la situación de programación, en la cual todos los biestables de configuración (de programación) SRAM se unen, en cadena, formando un registro de desplazamiento, con una entrada DIN (entrada de datos para la programación) que recibirá los dígitos del «mapa de valores», bit a bit conforme a una señal de reloj CCLK que desplaza los dígitos a lo largo del registro. Un contador auxiliar sirve para contar el número de dígitos recibidos y señalar (salida DONE) el momento en que se completa la programación: se ha llenado todo el registro.

Sin tensión de alimentación el dispositivo carece de programación; en la puesta en marcha (*power-on*), la primera tarea que realiza el dispositivo consiste en leer su programación: ésta puede ser proporcionada por una PROM auxiliar (generalmente una PROM serie que se comunica con la FPGA a través de un reducido número de líneas) o bien suministrada directamente desde un computador o microprocesador.

La figura siguiente muestra la conexión entre una FPGA y una memoria PROM serie que contiene su programación; las líneas de conexión para efectuar la programación son las siguientes:

	PROM	FPGA	
- línea de control	RESET	INIT	señal externa de inicio
- línea de datos	DATA →	DIN	lo envía la PROM
- línea de reloj	CLK ←	CCLK	lo envía la FPGA
- línea de finalización	CE ←	DONE	lo envía la FPGA.

Al activar una señal de inicio (RESET / INIT) la FPGA lee el contenido de la memoria PROM a través de la línea de datos (DATA → DIN), lectura que es controlada mediante dos líneas que la FPGA envía a la PROM: una línea de reloj (CLK ← CCLK) y otra de habilitación (CE ← DONE); la segunda línea determina la finalización de la lectura cuando se ha completado el fichero necesario para la programación.



También pueden utilizarse las memorias PROM habituales de 8 líneas de datos; su lectura será más rápida pero requieren un mayor número de líneas dedicadas a la programación.

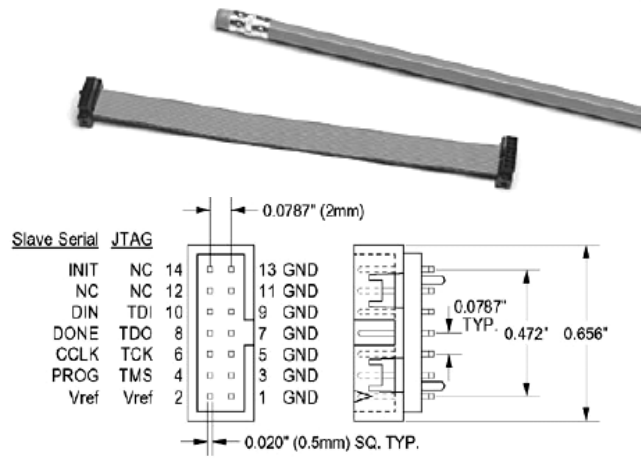
Asimismo, el fichero de programación de una FPGA puede ser enviado desde un computador o desde un microprocesador a través de 4 líneas de interconexión:

	procesador	FPGA	
- línea de control	TMS →	PROG	control de la programación
- línea de datos	TDI →	DIN	lo envía el procesador
- línea de reloj	TCK →	CCLK	lo envía el procesador
- línea de finalización	TDO ←	DONE	lo envía la FPGA.

Basta, pues, con dejar 4 terminales en la placa de la FPGA preparados con este fin; es el computador el que envía el reloj a la FPGA para sincronizar la transmisión de datos, mientras que con una PROM es la FPGA la que envía el reloj a la memoria para leerla.

En ambos casos la FPGA recibe los datos de la programación a través de la línea DIN e informa de que se ha completado tal programación a través de la línea DONE.

La siguiente figura muestra un cable normalizado de interconexión: a las 4 líneas básicas de programación se les añaden otras líneas con la referencia de «tierra» y la alimentación.



La figura que sigue muestra un adaptador para conectar el cable anterior al puerto paralelo de un PC:



De igual forma la programación puede realizarla un microprocesador presente en la misma placa circuital, a través de 4 líneas del circuito impreso.

Como puede apreciarse, la programación de los dispositivos SRAM se efectúa en el mismo sistema digital del que forman parte: es una programación en el propio circuito global aplicado **ISP** (*in system programmable*). Estos dispositivos, programables directamente en el propio sistema digital del que forman parte, permiten, incluso, el cambio de su programación durante el funcionamiento del mismo (de manera que pasen a realizar funciones diferentes de las que estaban realizando).

No todas las FPGAs son de tipo SRAM; la configuración LUT y selección de línea por multiplexores puede también conformarse con programación E²PROM o FLASH, reflejando sobre simples transistores MOS programables los valores 0/1 que en el caso SRAM se almacenan en los biestables.

Las FPGAs de tipo E²PROM o FLASH suelen incluir internamente la generación de las tensiones y la circuitería necesaria para su borrado y programación, de forma que también pueden ser programadas en el propio sistema, de manera análoga a las de tipo SRAM; son pues dispositivos **ISP** (programables en el sistema a través de 4 líneas).

Existen también FPGAs con programación destructiva, mediante antifusibles. Los antifusibles consisten en contactos entre polisilicio y difusión separados inicialmente por un aislante de óxido de silicio SiO₂ muy fino, de forma que la perforación del aislante pone en contacto la difusión con el polisilicio; la resistencia inicial del antifusible es superior a 100 Ω y se reduce por debajo de los 500 Ω al perforar el óxido de silicio.

Los antifusibles se programan por «perforación eléctrica» a través de la acción de un campo eléctrico fuerte, producido por una tensión relativamente elevada; para ello se utilizan pulsos de tensión de unos 21 V muy repetitivos (en dos pasos: una precarga de todos los antifusibles a 10,5 V, para evitar la perforación simultánea de varios de ellos, seguida de la aplicación de 21 V al fusible particular que se quiere perforar).

22.3. Matrices de celdas lógicas programables: FPGAs

Las matrices de celdas programables FPGA son dispositivos de «granularidad fina», compuestos por una matriz de bloques lógicos, formados por 2 ó 4 parejas de celdas lógicas de reducido tamaño (y limitada capacidad booleana), separados por canales de conexión verticales y horizontales, con celdas de entrada/salida situadas en la periferia, que conectan con los terminales del dispositivo.

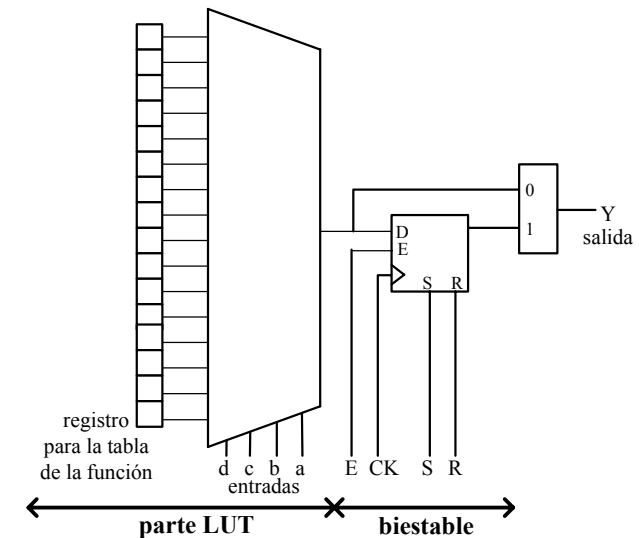
El calificativo programable adjetiva a las celdas y a la matriz de conexiones, es decir, tiene una doble repercusión: son programables tanto las celdas internamente (su función booleana y su configuración) como las conexiones de las celdas entre sí.

La descripción que, a continuación, se detalla pretende ser una aproximación conceptual y genérica a la conformación de las FPGAs, que proporcione una idea clara y justificada de su configuración y de sus posibilidades; la configuración concreta de una FPGA depende del fabricante y de la serie específica y alcanza progresivamente mayores niveles de complejidad.

Celdas, módulos y bloques lógicos

Las celdas lógicas cuentan con una parte combinacional, capaz de realizar una función booleana de pocas entradas (generalmente 4 entradas), un biestable y algunos «multiplexores de configuración»; las funciones se construyen mediante multiplexores por el procedimiento de «búsqueda sobre tabla» LUT (*look-up-table*).

El biestable ofrece una entrada de habilitación **E** para mantener su valor cuando interese (**E = 0**), lo cual simplifica mucho el diseño de determinados registros (como los de retención o los contadores) y sendas entradas asíncronas de marcado **S** y de borrado **R**, para la inicialización del biestable.



De esta forma, una celda lógica ofrece una función de 4 variables, con salida directa (combinacional) o «registrada» (a través de un biestable). Caso de que no se utilice la celda como función booleana, pueden emplearse los biestables de programación de la misma en las tres formas siguientes:

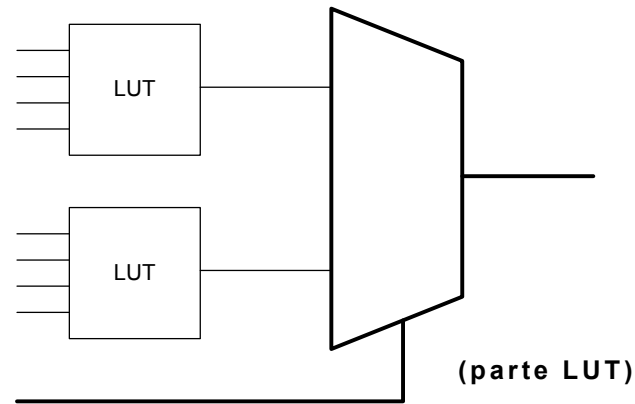
- como registro de desplazamiento (muy rápido) de hasta 16 biestables (el multiplexor LUT permite controlar la longitud de dicho registro) o de 17 biestables utilizando el propio de la celda lógica;
- como memoria ROM de 16 x 1 bits, fijando los 16 valores sobre el registro de programación y tomando la salida del multiplexor LUT;
- como memoria RAM de 16 x 1 bits, utilizando los 16 biestables de programación como biestables numerados; en principio, la línea de entrada y la de salida de dicha memoria son independientes: se incluye en la celda la lógica necesaria para escribir selectivamente en uno de los biestables y la lectura es a través del multiplexor LUT (que selecciona el biestable que aporta su valor).

Asimismo, la función LUT puede ser configurada como multiplexor rápido de 2 líneas.

De manera que una celda lógica equivale a:

- una función de 4 variables + biestable,
- un registro de desplazamiento de hasta 17 bits,
- una pequeña memoria ROM de 16 x 1 bits,
- una pequeña memoria RAM de 16 x 1 bits,
- un multiplexor de 2 líneas de entrada.

Dos celdas lógicas se agrupan en un módulo lógico, con un multiplexor de las dos salidas LUT a fin de poder configurar funciones de 5 variables.



De esta forma, un módulo lógico puede ser utilizado como:

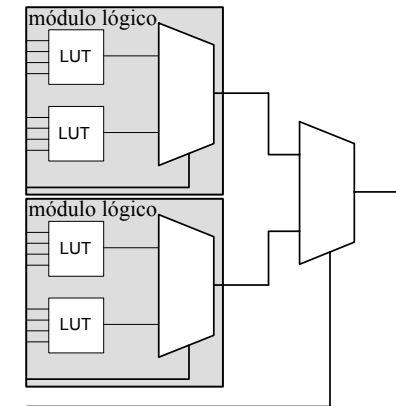
- dos funciones de 4 variables + 2 biestables,
- una función de 5 variables + 2 biestables (el bloque permite la programación de cualquier función de 5 variables y, también, algunas funciones, no todas, de hasta 9 variables),
- dos registros de desplazamiento de hasta 17 bits cada uno o uno de hasta 34 bits,
- una pequeña memoria ROM de 16 x 2 bits o de 32 x 1 bits,
- una pequeña memoria RAM de 16 x 2 bits o de 32 x 1 bits,
- un multiplexor de 4 líneas de entrada.

Además, según veremos en el próximo subapartado, los módulos lógicos incluyen una circuitería adicional específica para operaciones aritméticas rápidas.

Los módulos lógicos se agrupan para formar bloques lógicos que, separados por canales horizontales y verticales de conexión, forman la «matriz» básica de las FPGAs. Los bloques lógicos contienen dos o cuatro módulos lógicos, o sea, 4 u 8 celdas lógicas.

Un bloque lógico de 2 módulos (ver la figura siguiente), además de las prestaciones que ofrecen sus dos módulos por separado, puede ser utilizado, conjuntamente, como:

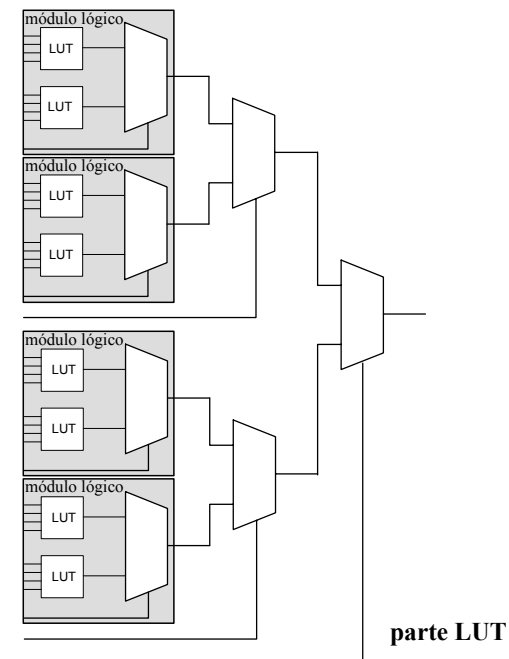
- una función de 6 variables + 4 biestables (el bloque permite la programación de cualquier función de 6 variables y algunas funciones de hasta 19),
- un registro de desplazamiento de hasta 68 bits,
- una memoria ROM de 16 x 4 o de 32 x 2 o de 64 x 1 bits,
- una memoria RAM de 16 x 4 o de 32 x 2 o de 64 x 1 bits,
- un multiplexor de 8 líneas de entrada.



Bloque lógico formado por 2 módulos lógicos

Cualquiera de las prestaciones que se configuran dentro de un bloque lógico son de alta velocidad, ya que las funciones y conexiones, en su interior, están optimizadas en cuanto a tiempos de propagación.

Las FPGAs «avanzadas», de alta capacidad, contienen 4 módulos lógicos en cada bloque lógico (en cada celdilla de su matriz):



Bloque lógico formado por 4 módulos lógicos

Un bloque lógico de 4 módulos, además de las prestaciones de dichos módulos, puede ser utilizado como:

- una función de 7 variables + 8 biestables (permite la programación de cualquier función de 7 variables y algunas funciones de hasta 39 variables),
- un registro de desplazamiento de hasta 136 bits,
- una memoria ROM de 16 x 8 o de 32 x 4 o de 64 x 2 o de 128 x 1 bits,
- una memoria RAM de 16 x 8 o de 32 x 4 o de 64 x 2 o de 128 x 1 bits,
- un multiplexor de 16 líneas de entrada.

Recursos para operaciones aritméticas

A fin de poder configurar operaciones aritméticas y que éstas puedan efectuarse a alta velocidad, se incluye en cada módulo lógico una circuitería adicional específica para el diseño de sumadores, restadores y multiplicadores. Dicha circuitería permite, también, la configuración de contadores rápidos, que pueden ser bidireccionales, con borrado síncrono y con carga paralelo.

Una celda sumadora requiere las siguientes funciones:

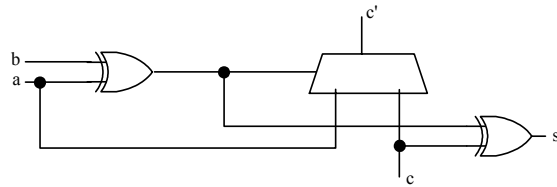
$$\text{suma: } s = c_{-} \oplus (b \oplus a)$$

$$\text{acarreo: } c' = b \cdot a + c_{-} \cdot (b + a);$$

en relación con la velocidad, hay que prestar particular atención a la función de acarreo porque es recursiva: el acarreo de cada dígito c' debe esperar al acarreo del dígito anterior c_{-} .

Ahora bien, dentro de la función acarreo hay dos terminos: la generación de acarreo $a \cdot b$ y la propagación del acarreo anterior $(a+b) \cdot c_{-}$, siendo este segundo el que limita la velocidad; resulta eficaz la utilización de un multiplexor rápido que transmita el acarreo «propio» cuando éste es generado ($a = b = 1$) o el anterior cuando a o b son 1.

Las dos funciones de la celda sumadora pueden construirse de la siguiente forma, que aprovecha una puerta "o-exclusiva" para ambas:

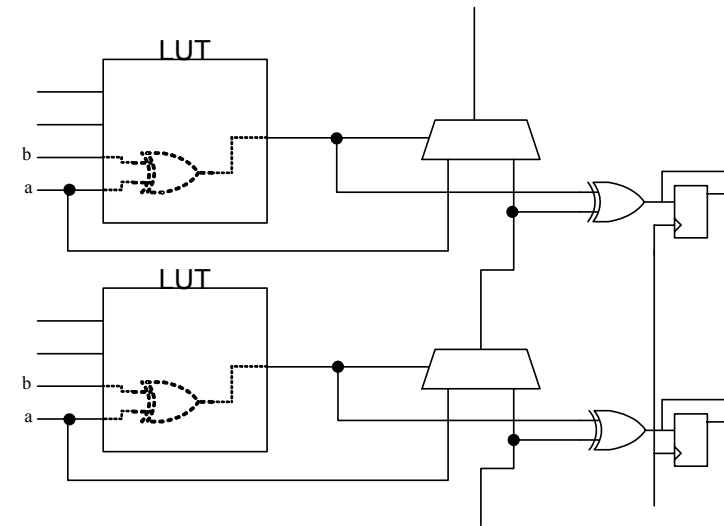


$$\text{suma: } s = c_{-} \oplus (b \oplus a)$$

$$\begin{aligned} \text{acarreo: } c' &= (\overline{b \oplus a}) \cdot a + (b \oplus a) \cdot c_{-} = (\overline{b \cdot a} + b \cdot a) \cdot a + (b \cdot \overline{a} + \overline{b} \cdot a) \cdot c_{-} \\ &= b \cdot a + (b \cdot \overline{a} + \overline{b} \cdot a) \cdot c_{-} = b \cdot a + c_{-} \cdot (b + a) \end{aligned}$$

Con ello, un módulo lógico queda en la forma de la figura siguiente, donde la primera puerta "o-exclusiva" se configura en la parte funcional (LUT) de cada celda y se

añade el multiplexor y la segunda puerta "o-exclusiva" como elementos específicos para operaciones aritméticas:



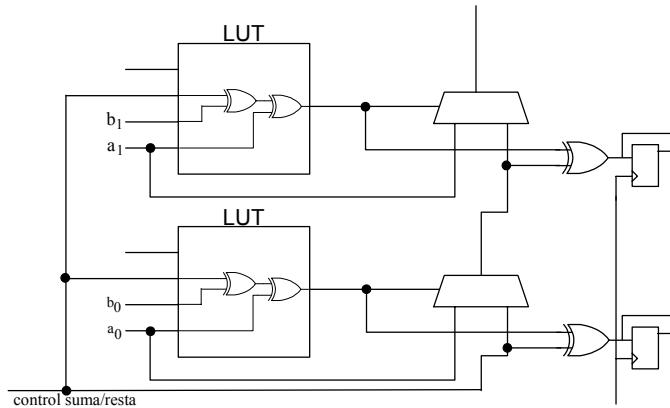
De esta forma, cada celda lógica se convierte en una celda sumadora (el módulo lógico es un sumador de 2 dígitos). Esta configuración presenta muy bajos tiempos de retraso en lo que se refiere a la propagación del arrastre, ya que el paso a través de multiplexores (que se encuentran todos ellos previamente habilitados, conforme al valor de su entrada de control) es muy rápido y las líneas de conexión son directas y cortas. Téngase en cuenta que, en muchas ocasiones, los tiempos de propagación a través de las conexiones son mayores que los que presentan las puertas lógicas.

Además, la salida de «propagación de arrastre» de cada módulo lógico se encuentra unida, a través de una línea directa y corta, con la entrada de «propagación de arrastre» del siguiente módulo lógico de la misma columna (el que se encuentra «encima» del mismo de acuerdo con la disposición matricial de las FPGAs); es decir, existe una línea directa y «mínima» de propagación de arrastre a lo largo de cada columna de módulos lógicos que permite la configuración de sumadores ultrarrápidos en dicha columna.

Aprovechando aún más la parte funcional (LUT) de las celdas lógicas, el anterior sumador puede convertirse en un sumador/restador (en complemento a 2) controlado por una entrada de selección. La resta $A-B$, en complemento a 2, equivale a sumar a A el inverso de B más una unidad, ya que el cambio de signo de B equivale a invertirlo y sumarle una unidad:

$$A - B = A + (-B) = A + (\overline{B} + 1) = A + \overline{B} + 1.$$

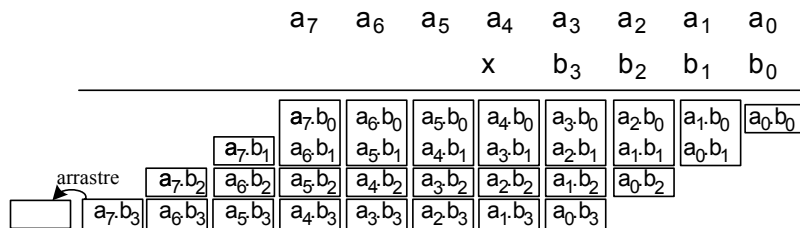
La suma de dicha unidad (+1) se efectúa haciendo el arrastre inicial igual a 1. La siguiente figura representa la configuración del primer módulo lógico para un sumador/restador que efectúa la suma cuando $s = 0$ y la resta cuando $s = 1$; la primera puerta "o-exclusiva" de cada celda invierte el dígito de B cuando $s = 1$ (dejándolo pasar sin invertir cuando $s = 0$) y la entrada de selección s actúa como arrastre inicial para sumar una unidad en el caso de la resta.



Habida cuenta de que contar es ejecutar la suma $n + 1$ (sumar una unidad) sobre un registro y que contar «hacia abajo» equivale a restar una unidad ($n - 1$), esta misma configuración, utilizando sus biestables como registro (número n) permite construir contadores bidireccionales que admiten frecuencias de reloj muy altas: siendo Q el estado de los biestables, basta hacer $A = Q$ y $B = \dots 00001$; la entrada de habilitación de los biestables sirve también para la habilitación de los contadores.

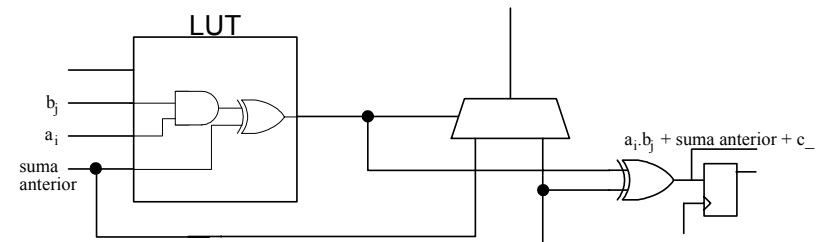
Además del multiplexor de propagación de arrastre y de la puerta "o-exclusiva", se añade en cada celda una puerta "y" (en el control del citado multiplexor) para facilitar la construcción de multiplicadores y de contadores con carga paralelo y borrado síncrono.

La forma habitual de efectuar un producto

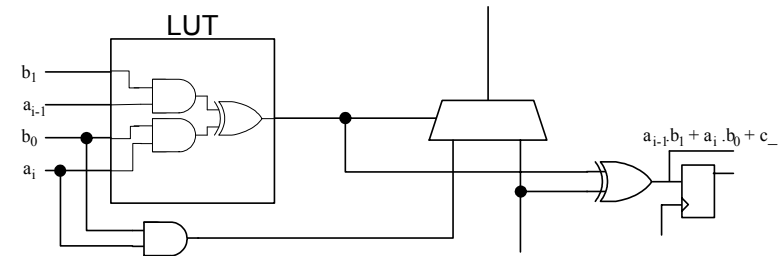


requiere sumadores para los productos lógicos $a_i b_j$; en la operación anterior, cada cuadrado es una suma de tipo $a_i b_0 + a_{i-1} b_1$ y cada rectángulo lo es en la forma $a_i b_j + \text{suma anterior}$; ambos tipos de sumadores pueden ser realizados utilizando una sola celda lógica (en total, 26 celdas).

Los rectángulos son sumadores $a_i b_j + \text{suma anterior}$ que pueden ser configurados en una celda en la siguiente forma:



En cambio, para la suma $a_i b_0 + a_{i-1} b_1$ de las dos primeras filas se necesita disponer de una puerta "y" adicional en la forma representada en la figura siguiente:



De esta forma un multiplicador de $8 \times m$ dígitos requiere un sumador de 9 bits (y una puerta "y": $a_0 b_0$) para las dos primeras filas y otro sumador de 8 bits para cada una de las filas siguientes más una celda lógica que recoja el arrastre más significativo. El producto de $n \times m$ dígitos necesita $m - 1$ sumadores con un total de $n(m - 1) + 2$ celdas lógicas.

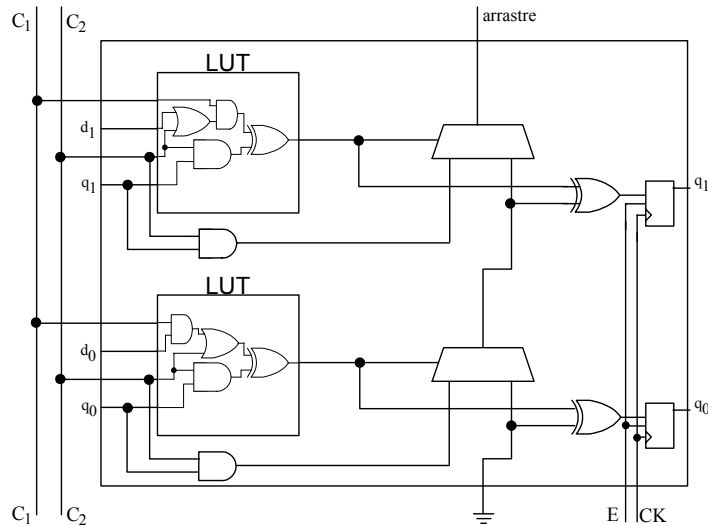
Un multiplicador de números binarios de 16 dígitos requiere un amplio número de celdas lógicas (242 celdas: 121 módulos lógicos) pero realiza el producto muy rápidamente, en un solo ciclo de reloj. En el capítulo 24 se considera otra opción de construcción de multiplicadores, en forma de máquina algorítmica, de menor tamaño (36 celdas: 18 módulos lógicos) pero, también, con menor velocidad pues requiere 32 ciclos de reloj para realizar el producto. Es habitual, en el diseño digital, la existencia de un compromiso entre velocidad y tamaño de los circuitos.

Además, habida cuenta del gran número de celdas lógicas que requiere la configuración de un multiplicador largo, las FPGAs avanzadas incorporan, como módulos específicos, multiplicadores de 16 o de 32 bits

La puerta "y" que controla el multiplexor del arrastre permite configurar la celda en la forma $a.b + F(a,b,c,d)$ (suma aritmética) que da lugar a contadores bidireccionales, con borrado síncrono y carga paralelo, a través de las siguientes expresiones:

$$q_i^+ = \text{Suma aritmética}[\langle q_i, C_2 \rangle \text{ "+" } \langle C_1, (d_i + C_2) \rangle \text{ "+" } \langle \text{arrastre del bit anterior} \rangle]$$

$$\text{para } i \neq 0 \text{ y para el primer dígito } q_0^+ = \text{Suma aritmética}[\langle q_0, C_2 \rangle \text{ "+" } \langle C_1, d_0 + C_2 \rangle].$$



Esta figura representa un módulo lógico con los dos primeros dígitos del contador (unidades y «dosenas»); las celdas correspondientes a los siguientes dígitos tendrán la misma configuración que la celda superior (la de arriba, q_1) de este módulo. La entrada de control C_2 puede ser considerada como habilitación de conteo (cuando $C_2 = 1$) o bien como habilitación de carga negada ($C_2 = 0$):

- cuando $C_2 = 0$ se efectúa la carga si $C_1 = 1$; si $C_1 = 0$ se produce borrado síncrono;
- cuando $C_2 = 1$ el contador realiza su operación propia de conteo: si $C_1 = 0$ el conteo es ascendente y si $C_1 = 1$ es hacia abajo.

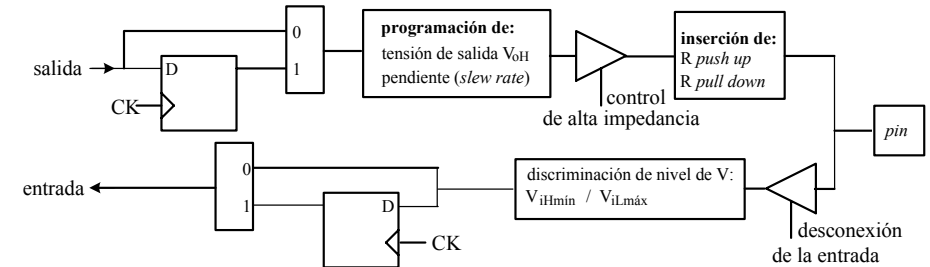
$C_2 = 0$	$C_1 = 0$	$q_i^+ = 0$	$Q = 000000....$	borrado síncrono
$C_2 = 0$	$C_1 = 1$	$q_i^+ = d_i$	$Q = D$	carga paralelo
$C_2 = 1$	$C_1 = 0$	$q_i^+ = \text{Suma aritmética}[q_i, "+", 0, "+", \langle \text{arrastre} \rangle]$ $q_0^+ = \text{Suma aritmética}[q_0, "+", 1]$ $\text{suma } Q + ...00001$		conteo hacia arriba
$C_2 = 1$	$C_1 = 1$	$q_i^+ = \text{Suma aritmética}[q_i, "+", 1, "+", \langle \text{arrastre} \rangle]$ $q_0^+ = \text{Suma aritmética}[q_0, "+", 1]$ $\text{suma } Q + ...11111$		conteo hacia abajo

Este contador bidireccional (cuya entrada de habilitación del conteo es la propia de los biestables), con borrado síncrono y carga paralelo, requiere solamente una celda lógica para cada dígito.

Celdas periféricas adaptadoras de entradas/salidas

Las celdas de entrada/salida contienen sendos biestables para sincronizar la entrada y la salida, adaptadores triestado para «desconectarlas» y una serie de multiplexores de configuración que permiten programar:

- la utilización de la celda como entrada, como salida o como terminal bidireccional
- respecto a la entrada:
 - su desconexión como tal entrada
 - su paso a través de biestable: sincronismo de la entrada
 - los niveles de tensión para discriminar los valores booleanos **0** / **1**
- respecto a la salida:
 - su paso o no a través de biestable: sincronismo de la salida
 - el control de la situación de alta impedancia: triestado
 - el valor de la tensión de salida para el **1** lógico
 - la pendiente de los flancos de salida (*slew rate*): lento o rápido
 - la inclusión de una resistencia de *push-up* o de *pull-down*.



En relación con las tensiones de salida (es decir, con V_{OH} , ya que $V_{OL} = 0$ V) y de entrada (con V_{IHmin} y V_{ILmax}) las FPGAs suelen presentar múltiples posibilidades (en ocasiones hasta una decena de ellas) de niveles de tensión para facilitar su conexión con otros circuitos integrados digitales o analógicos; en general admiten todo tipo de tensiones lógicas normalizadas (tanto TTL como CMOS y las diversas series de baja tensión), de las cuales debe seleccionarse en cada caso la que interesa mediante programación.

Conexión entre celdas

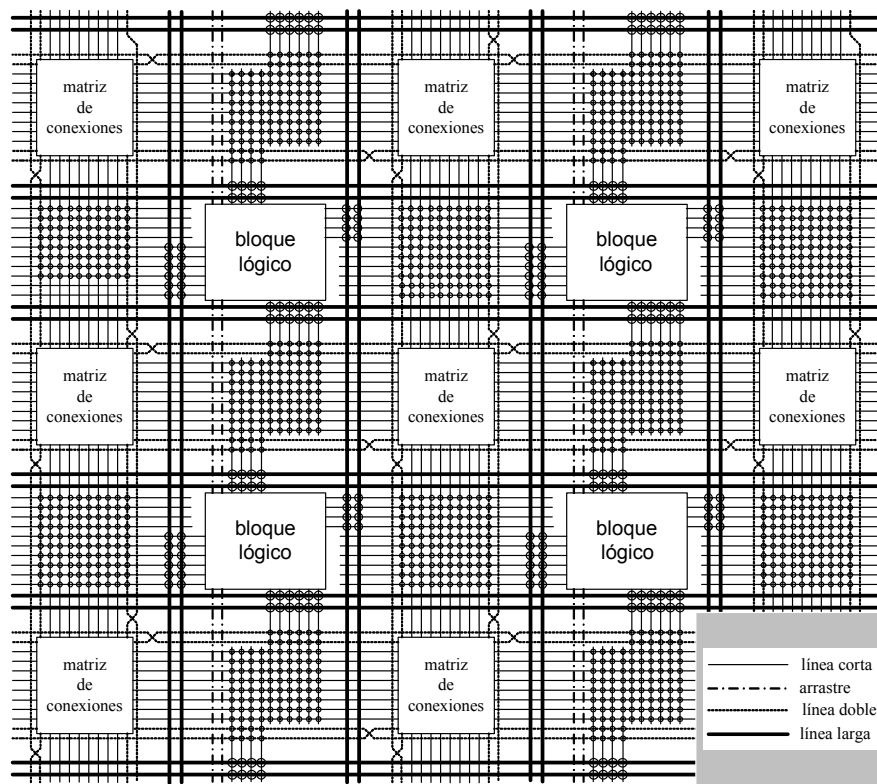
Los canales de conexión, verticales y horizontales, situados entre los bloques lógicos, contienen líneas de diferentes longitudes, con muy alta capacidad de interconexión entre ellas; a través de tales líneas se efectúan las conexiones de las celdas lógicas entre sí y de éstas con las celdas de entrada/salida.

Se trata de poder conectar dos elementos cualesquiera, optimizando, en lo posible, los tiempos de retardo que introducen las líneas de conexión. Ya, de entrada, se minimizan las longitudes de las líneas dentro de cada bloque lógico y en los canales entre bloques se dispone de tres tipos de líneas de conexión: *cortas* para conexión de bloques lógicos vecinos, *dobles* para los bloques siguientes y *largas* que recorren toda la fila (si son horizontales) o toda la columna (si son verticales).

En cada cruce de los canales de conexión horizontales con los verticales (es decir, en las cuatro esquinas de los bloques lógicos), hay una *matriz de conexiones* que permite prolongar las líneas que llegan a ella y conectarlas con las líneas que se cruzan en ella.

De forma que cada canal de conexiones está formado por *líneas cortas*, que van de una matriz de conexiones a la siguiente, junto con unas pocas *líneas dobles* (que enlazan matrices alternas) y algunas *líneas largas* que recorren todo el canal (y no pasan por las matrices de conexión). Las entradas y salidas de cada bloque lógico se prolongan en forma de líneas cortas perpendiculares a los canales de conexión.

La figura siguiente presenta una hipotética distribución de líneas y conexiones:

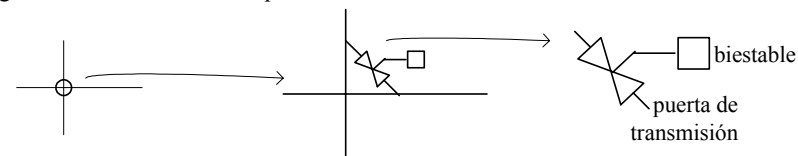


Las líneas cortas (de mínima resistencia y capacidad) son las más numerosas y relacionan a cada bloque y a cada matriz de conexión con los cuatro bloques y con las cuatro matrices inmediatamente contiguas: los bloques vecinos se conectan a través de sus propias líneas cortas (utilizando las líneas de canal que las cruzan); los bloques no contiguos lo hacen a través de las matrices de conexión.

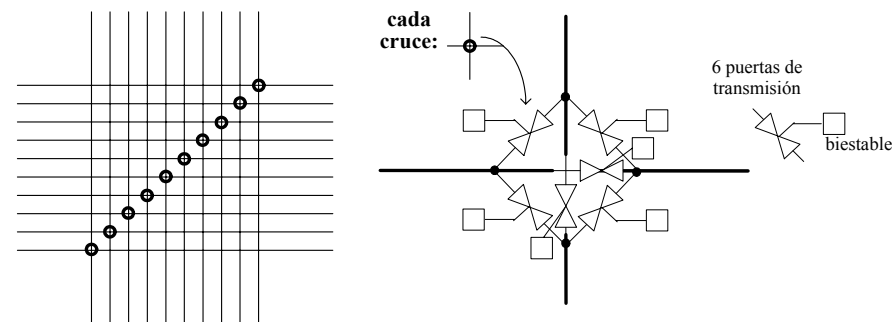
Las líneas dobles permiten la conexión directa entre matrices de conexión alternas y las líneas largas (más lentas) son para señales de tipo global (que van a muchos bloques) y para conexiones rápidas entre bloques muy alejados.

Las conexiones se configuran a través de puertas de transmisión (que equivalen a interruptores estáticos) controladas, cada una de ellas, por un biestable que determina su situación de conexión (biestable a **1**) o desconexión (biestable a **0**); las conexiones son siempre bidireccionales (ya que los transistores MOS y las puertas de transmisión formadas con ellos también lo son).

En los cruces señalados con círculo (en la figura anterior) hay una conexión programable entre las líneas que se cruzan:



En las matrices de conexión cada línea corta (o doble) puede prolongarse hacia la siguiente (situada «en línea», a continuación de ella) y también puede unirse a las otras dos líneas perpendiculares a ella.



En principio, las puertas de transmisión están formadas por sendos transistores NMOS (que transmite bien los «ceros») y PMOS (para transmitir los «unos») en paralelo; pero también es viable utilizar solamente transistores NMOS de paso (en cuyo caso conviene aumentar la tensión umbral de los transistores PMOS de la tecnología para recuperar la situación de consumo nulo de las puertas complementarias –ver T8.2–).

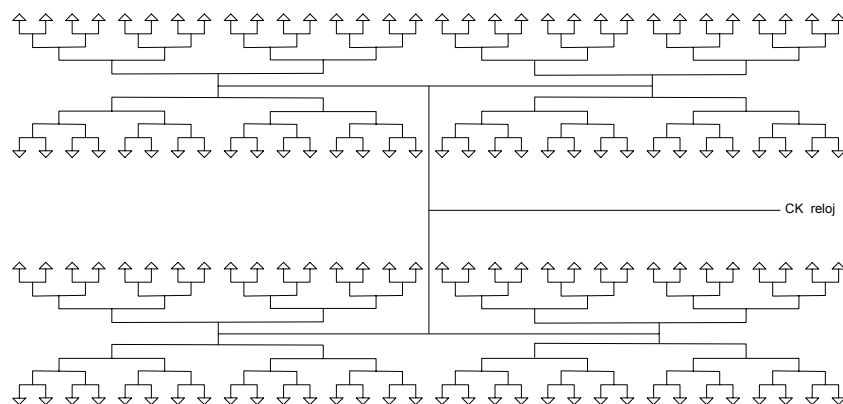
El número de conexiones en una FPGA es muy alto, de manera que los biestables de programación de las conexiones son siempre un número muy superior al de biestables de configuración: un módulo lógico suele tener unos 60 biestables para su programación (32 para las «tablas de verdad» de las 2 funciones de 4 variables que contiene y unos 28 de configuración del propio módulo); una FPGA de unos 2000 módulos tiene del orden de 10^6 conexiones programables (es decir, unas 500 por módulo).

La distribución del diseño entre los diversos bloques lógicos es de vital importancia respecto a la velocidad de trabajo que puede alcanzar dicho diseño (en forma análoga a la importancia que tiene la distribución de circuitos integrados en una placa de circuito impreso): los bloques que se relacionan entre sí deben estar lo más cercanos posible y los sumadores, restadores, multiplicadores y contadores deben situarse en la misma columna de módulos lógicos (para aprovechar la propagación directa del arrastre).

Distribución de la señal de reloj

Dada la importancia de la simultaneidad de la señal de reloj, en relación con el sincronismo, se incorporan esquemas de distribución de dicha señal que aseguran que recorre caminos de longitud prácticamente igual para los diversos bloques lógicos.

Distribuciones de tipo «fractal», como la representada en la figura siguiente, consiguen que la longitud de todas las líneas de distribución de reloj sea la misma: desde la conexión que la recibe (señalada como «CK reloj») hasta su aplicación a cada módulo lógico (indicada con un triángulo) la señal de reloj recorre caminos de igual longitud.



Esta forma de asegurar la simultaneidad de la señal de reloj aumenta, en forma apreciable, la longitud (ya de por sí grande) de las líneas de reloj y el largo recorrido que debe efectuar para llegar a los diversos biestables determina un retraso significativo entre el reloj interno y el reloj externo que recibe la FPGA.

Para evitar los efectos del retardo de propagación de la señal de reloj, las FPGAs avanzadas incorporan módulos de «resincronización de flanco» (bloques denominados **DLL**, *Delay Locked Loop*) que permiten sincronizar el reloj interno con el externo, es decir, retrasan el reloj interno de forma que sus flancos coinciden con los de la señal de reloj externa. De esta forma, en cada flanco activo (en los momentos de transición de una unidad de tiempo a la siguiente) el flanco actual del reloj interno procede de un flanco anterior del reloj de entrada pero coincide con el flanco actual de dicho reloj.

Memoria RAM y bloques incrustados

Al describir la celda lógica, se comentó la posibilidad de configurar los registros SRAM de la programación LUT (biestables previos a los multiplexores que conforman las funciones) como memorias RAM internas y rápidas; este tipo de memoria RAM, utilizando celdas no empleadas para funciones booleanas, se denomina memoria RAM distribuida.

Además, las FPGAs avanzadas suelen incluir bloques específicos de memoria RAM de varios *Kbytes*, a fin de disponer de módulos de memoria más amplios y compactos; esta memoria es, también, muy rápida por ser interna al dispositivo. Tales bloques de memoria suelen ser configurables; por ejemplo, un bloque de 8K x 8 bits puede utilizarse como 16K x 4, 32 K x 2 ó 64K x 1 bit y, también, como 4K x 16 ó 2K x 32 bits.

Asimismo, las FPGAs más grandes incorporan otros módulos operativos de utilidad «incrustados» (*embedded*) dentro del mismo circuito integrado; de esta forma, en el interior de la FPGA se encuentran disponibles algunos bloques grandes de utilidad general, sin necesidad de programarlos sobre módulos lógicos (lo que daría lugar a un mayor tamaño del bloque «distribuido» resultante y menor velocidad de trabajo del mismo). Por ejemplo, multiplicadores de 16 ó de 32 bits (ya citados en páginas anteriores), microprocesadores de 8 ó de 16 bits, etc.

23 LENGUAJE DE DESCRIPCIÓN CIRCUITAL: V H D L

- 23.1. VHDL como lenguaje para describir, simular, validar y diseñar
- 23.2. VHDL básico para diseñar circuitos combinacionales
- 23.3. Descripción de circuitos secuenciales y de sistemas síncronos
- 23.4. Descripción de grafos de estado
- 23.5. Otros recursos de VHDL

A la hora de describir un diseño microelectrónico (es decir, de realizar la descripción de un circuito digital, en formato informático, para ser, posteriormente, «compilado» sobre un dispositivo programable o sobre una librería de celdas de un ASIC), tal descripción puede hacerse en forma gráfica (esquema circuital) o en forma de texto (programa).

*El "Versatile Hardware Description Language" VHDL (cuyas siglas proceden de un nombre aún más largo y restrictivo: *Very high speed integrated circuit HDL*) se desarrolló inicialmente como lenguaje de documentación, de simulación y de «validación» (simulación en el contexto en que debe funcionar) de circuitos integrados digitales. Para documentar y simular se requiere una descripción precisa, carente de ambigüedades y estructurada y tal descripción puede ser directamente utilizada para diseñar el circuito descrito (mediante su «compilación» sobre los recursos booleanos disponibles). Las dos páginas que siguen (apartado 1 de este capítulo) amplían la breve presentación que del lenguaje VHDL se hace en este párrafo y deben ser leídas como parte de esta introducción.*

Habida cuenta de que un circuito integrado no tiene «finalidad propia», sino que forma parte de un sistema más amplio, cuyo funcionamiento controla o supervisa, uno de los propósitos de VHDL era simular el circuito en el contexto del sistema de que forma parte; de manera que no se limita a describir sistemas digitales sino que abarca, también, cualquier otro tipo de sistema activo (eléctrico, mecánico,...).

Es obvio que un tratado sobre VHDL requeriría todas las páginas de este volumen y muchas más. Por ello, el contenido de este capítulo se restringe a la parte de VHDL que se utiliza habitualmente en el diseño digital. Su objetivo es enseñar las bases de la descripción de circuitos digitales en VHDL a quienes desconozcan por completo este lenguaje.

Quizás la mejor forma de aprender un lenguaje sea utilizarlo y practicarlo. Por eso este capítulo, que es simplemente una presentación parcial y utilitaria de VHDL, está construido, fundamentalmente, con ejemplos de diseño; se apoya en múltiples descripciones y diseños de subcircuitos y de pequeños sistemas digitales y prescinde, en gran medida, de lo que pudiera ser una exposición académica o descriptiva del propio lenguaje.

La misma organización del capítulo está dirigida directamente al diseño digital y, así, los diversos epígrafes de introducción del lenguaje se refieren, sucesivamente, a la descripción de sistemas combinacionales (apartado 2), de sistemas secuenciales y síncronos (apartado 3) y de grafos de estado (apartado 4) y a otros recursos avanzados de diseño digital, aplicados también a ejemplos concretos (apartado 5).

23.1. VHDL como lenguaje para describir, simular, validar y diseñar

En un principio, la «captura de esquemas» fue la forma habitual de diseño CAD (apoyado y almacenado en un computador). Pero, hoy día, ha sido sustituida (casi por completo) por su descripción funcional en texto (programa que detalla el funcionamiento de las diversas partes del circuito y la conexión entre ellas), utilizando para ello un lenguaje de descripción de *hardware* (HDL).

La forma textual presenta numerosas ventajas: suele requerir menor tiempo y esfuerzo para comprender lo que el circuito hace (en el caso de sistemas complejos); es independiente de la implementación a bajo nivel (en puertas, bloques, biestables y registros); es directamente trasladable a los diversos dispositivos programables y a las diversas librerías de ASICs;... y, sobre todo, resulta mucho más sencillo revisar e introducir modificaciones en el texto descriptor que en el esquema gráfico del mismo.

Actualmente, son dos los lenguajes de descripción circuital que se han impuesto como estándares para el diseño digital: VHDL y Verilog; y, de entre ellos, en el contexto europeo predomina el VHDL (si bien Verilog resulta, en buena medida, más cercano al *hardware* y a los esquemas gráficos que se utilizaban anteriormente).

El lenguaje de descripción de sistemas digitales VHDL nació como herramienta de documentación y de comunicación en relación con los circuitos integrados digitales. Un circuito integrado complejo, a lo largo del tiempo pasa por muchas manos («por muchas mentes»): quien lo define y quien lo utiliza no suele ser el mismo que quien lo diseña; quien repara o quien modifica los sistemas en que participa el circuito no suele ser el propio diseñador; e, incluso, quien lo ha diseñado, al volver a su circuito cuando ha transcurrido un cierto tiempo, difícilmente recordará los detalles de las diversas funciones y recursos con los que lo ha configurado.

De ahí la necesidad de un lenguaje que ofrezca una descripción funcional precisa, carente de ambigüedades, estructurada y de fácil lectura e independiente de la «implementación» concreta a bajo nivel. Que quien lea esa descripción sea capaz de comprender, con poco esfuerzo y absoluta claridad, las funciones que hace el circuito y cómo las hace; sin tener que descender al nivel booleano de puertas y biestables que, por su mayor amplitud en componentes, resultaría más difícil de analizar.

Además, una misma descripción funcional puede configurarse circuitalmente de formas muy variadas y tal configuración dependerá de la librería de celdas estándar disponibles (para el diseño de un ASIC) o, en su caso, del dispositivo programable en que se inserte; en principio no es necesario conocer su configuración a nivel booleano para utilizar eficientemente un circuito digital.

Lo que sí es necesario es disponer de una adecuada y detallada descripción funcional. La escritura es la herramienta de las ideas; el vehículo hacia la «claridad», la precisión, la estructura (disposición, orden y enlace de las partes que configuran el todo) y la comunicación (transmisión de las ideas «en el espacio y en el tiempo»); la escritura es el mejor medio para poder trasladar las ideas a otras personas (comunicación espacial) y poder contrastarlas y debatirlas y, también, para recordar las ideas (comunicación temporal) y poder recuperarlas y revisarlas pasado el tiempo.

La documentación es un requisito inexcusable para considerar que un diseño se ha completado y hecho efectivo y, cuando el diseño se refiere a un sistema complejo, requiere un lenguaje preciso y estructurado (estructurado tanto en su forma de expresar el diseño como en el sentido de que confiera estructura al propio diseño).

Por otra parte, antes de «construir» un diseño complejo, con el coste económico y de tiempo que ello supone, conviene saber si el diseño es correcto; para ello es de gran ayuda efectuar una simulación «virtual» de su funcionamiento. Un lenguaje que describa con precisión el comportamiento de un sistema servirá, sin duda, para simular su comportamiento: bastará con una aplicación informática que «ejecute» la descripción del sistema en relación con el transcurso del tiempo.

Aún más, un circuito integrado no tiene sentido funcional «por sí mismo», sino que formará parte de un sistema más amplio; generalmente será una pieza de control de un sistema con partes eléctricas y mecánicas. El objetivo «final» no es que el circuito integrado funcione (individualmente) bien, sino que el sistema global actúe correctamente: en definitiva, el objetivo que se persigue no es el funcionamiento del propio circuito integrado, sino el del sistema de que forma parte.

De manera que el propósito de un lenguaje eficiente se refiere a la capacidad de describir y simular, además de los circuitos integrados digitales, los mecanismos y sistemas controlados por ellos. Es decir, «validar» al circuito integrado en el entorno funcional para el que ha sido diseñado.

Estamos, pues, planteando un objetivo cada vez más ambicioso: de la documentación se pasa a la simulación y se pretende, también, la validación de los circuitos integrados complejos. Con esa intencionalidad ha sido desarrollado el *Very high speed integrated circuit Hardware Description Language*, que conocemos como VHDL, siglas que podemos referir mejor a *Versatile Hardware Description Language* (pues la referencia a la alta velocidad de los circuitos es superflua; de igual forma puede ser utilizado para documentar, describir, simular y validar circuitos integrados lentos).

Un lenguaje con capacidad para describir con precisión y simular con eficiencia puede ser fácilmente utilizado para diseñar: una vez descrito un sistema digital, basta «compilar» la descripción sobre una «librería de recursos».

Se entiende por compilación el paso de la descripción funcional a la configuración circuital (del algoritmo al circuito), utilizando como componentes de dicho circuito los contenidos en una «librería de recursos»:

- en el caso de CPLDs dicha librería se refiere a las funciones booleanas en forma de suma de productos (configuración PAL) y biestables tipo D;
- para las FPGAs los recursos son las funciones booleanas, con un limitado número de variables (dividiendo, en su caso, las funciones más amplias), expresadas en forma de «tabla de verdad» (configuración LUT) y los biestables D;
- y en el diseño de ASICs, la librería de celdas básicas prediseñadas, propia del diseño con librería (*standard cell*).

El compilador (la herramienta informática de compilación) «traslada» la descripción funcional al circuito que la «materializa», conformado por componentes disponibles en la librería de recursos sobre la que se compila.

VHDL es una excelente herramienta de documentación, simulación, validación y diseño de sistemas digitales, estandarizada y en permanente proceso de actualización bajo los auspicios del IEEE (*Institute of Electrical and Electronics Engineers*). El único lenguaje alternativo que goza, también, de amplia aceptación es Verilog, pero su difusión en el contexto europeo es mucho menor (aunque, en buena medida, Verilog es un lenguaje más directo y más cercano al propio circuito digital).

23.2. VHDL básico para diseñar circuitos combinatoriales

El lenguaje VHDL no se refiere solamente a sistemas digitales sino que está abierto a la descripción de sistemas de cualquier tipo; por otra parte, VHDL es un lenguaje tremendamente versátil y potente. Su descripción requiere todo un amplio volumen y su estudio precisa de, al menos, un curso específico dedicado solamente a este lenguaje.

Interesa aquí la utilización de VHDL para diseñar sistemas digitales, es decir, aquella parte del lenguaje que es «compilable» para dar como resultado un circuito digital. A continuación, se expone un breve resumen, a la vez parcial y útil, como primera aproximación a este lenguaje y, a fin de ser lo más directa y práctica posible, esta introducción se realiza a través de ejemplos concretos de descripción circuital.

Nota: Todos los ejemplos de código VHDL (incluso las descripciones parciales) que vienen a continuación han sido comprobados en cuanto a su compilación (con MAX+plus II de ALTERA).

23.2.1. Primeras nociones

VHDL no distingue entre MAYÚSCULAS y minúsculas (salvo unas pocas excepciones referidas a valores de las señales). Pueden incluirse comentarios y, para identificarlos, se inician con el símbolo repetido "--" que indican al compilador que ignore todo lo que sigue hasta final de línea. Cada «módulo» descriptivo y cada «asignación» se cierran con el símbolo ";".

Los elementos básicos de la descripción digital son las señales (*signal*); para ellas suele utilizarse el tipo *std_logic* (*standard logic*) que admite los siguientes nueve valores:

'0'	-- cero	'1'	-- uno	valores booleanos típicos
'X'	-- desconocido			no se conoce el valor
'Z'	-- alta impedancia			propio de tri-estado
'U'	-- sin inicializar			biestables en su situación previa
'-'	-- no importa (don't care)			indiferente (para simplificación)
'L'	-- 0 débil	'H'	-- 1 débil	'W', -- desconocido débil

Los valores de una señal se expresan siempre entre comillas simples: '0', '1' y los valores X y L no admiten la minúscula (x, l no son válidas).

Los tres primeros valores (0, 1, X) son de tipo fuerte, si se «encuentran» dos de ellos aplicados sobre un nudo el resultado es X (desconocido). Los valores débiles corresponden a determinadas situaciones circuitales que, si confluyen con algún valor fuerte, dan como resultado dicho valor fuerte; en cambio, si se encuentran dos valores débiles sobre un nudo el resultado es W (desconocido débil).

Un conjunto de señales constituye un vector, *std_logic_vector*, que puede ser declarado en forma ascendente *std_logic_vector(0 to 7)* o descendente *std_logic_vector(7 downto 0)*, siendo más frecuente esta segunda declaración porque corresponde a la forma típica en la que el dígito más significativo es el de mayor subíndice; el conjunto de valores que adopta un vector se expresa entre comillas dobles: por ejemplo, "11010001".

Para conjuntos de señales (vectores), se utiliza también el tipo *integer* (entero) que debe ser declarado para un rango determinado: *integer range 0 to 15* (señal de 4 bits).

Los tipos de señales, *std_logic* y *std_logic_vector*, aunque son los más habituales en diseño digital (ya que describen bien las señales electrónicas en todas sus posibilidades), no se encuentran definidos en el propio VHDL básico (están definidos los tipos *bit* y *bit_vector*, que admiten sólo los dos valores booleanos 0 y 1). Los tipos *standard logic* han sido introducidos en la normalización hecha por IEEE y requieren la declaración de la librería (y de los paquetes) que los definen al principio de la descripción:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

(con el paquete *ieee.std_logic_unsigned* las operaciones se realizan en binario natural; si se desea efectuarlas en complemento a 2 debe utilizarse el paquete *ieee.std_logic_signed*)

En estos paquetes se dispone de dos funciones muy útiles:

CONV_INTEGER(**a**) que convierte el *std_logic_vector* **a** en *integer*

CONV_STD_LOGIC_VECTOR(**b,n**) que convierte el *integer* **b** en vector de longitud **n**.

Las **operaciones básicas** entre señales son:

asignación:	<=
operaciones booleanas	and or not xor
comparaciones	= /= > < >= <=
aritméticas	+ - *
concatenación	&

(la concatenación se refiere a poner señales o vectores juntos, formando un vector «más largo», cuyo número de bits es la suma de los números de ambas señales).

La base de la descripción VHDL es la asignación de valores a una señal, la cual puede hacerse directamente o por medio de operaciones entre señales.

Ejemplos de asignaciones directas:

```
signal a, b, c, Y: std_logic_vector(3 downto 0);
signal m, n: integer range 0 to 15;      -- 4 bits
y <= "1001";
Y <= (3 => '1', 0 => '1', others => '0'); -- equivale a la anterior ("1001")
m <= 9;
Y <= ((not a) and b) or (a and not b); -- equivale a y <= a xor b;
y <= a + b;                             -- suma aritmética
```

El lenguaje VHDL es muy disciplinado: una señal de un tipo no admite asignación de valores o de señales de otro tipo.

Ejemplos de asignaciones incorrectas:

```
y <= 9;          -- y no es de tipo integer
m <= "1001";     -- m no es del tipo std_logic
Y <= m;          -- tipos de señal diferentes
M <= a;          -- tipos de señal diferentes
Y <= "001";      -- faltan componentes
m <= 18;         -- fuera de rango
y <= '1001';     -- faltan comillas dobles
M <= 4           -- falta ;
```

23.2.2. Estructura de una descripción: librerías, entidades y arquitecturas

En VHDL se describe por un lado la «caja» del circuito con sus entradas y salidas, o sea, los terminales de conexión hacia el exterior, y eso se hace en un módulo denominado *entity*, y en otro módulo posterior, denominado *architecture*, se describe «lo que hace» el circuito, es decir, su funcionamiento interno. Además, es preciso declarar previamente las librerías necesarias para compilar el circuito (sobre librerías se trata en el apartado 23.5.6).

En consecuencia, la descripción VHDL tiene la siguiente estructura:

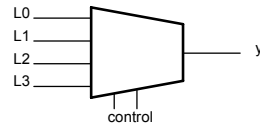
➔ declaración de librerías

➔ módulo de terminales

```
entity nombre_de_la_entidad is
port(
    declaración de entradas y salidas
);
end nombre_de_la_entidad ;
```


Ejemplos de asignaciones concurrentes:

Descripción de un multiplexor de cuatro líneas de entrada



```
signal control : integer range 0 to 3;
```

versión 1

```
y <= L0 when control = 0 else
      L1 when control = 1 else
      L2 when control = 2 else
      L3;
```

versión 2

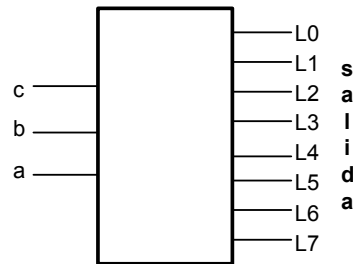
```
with control select
y <= L0 when 0,
      L1 when 1,
      L2 when 2,
      L3 when others;
```

Descripción de un decodificador de ocho líneas

```
entrada <= c & b & a;
```

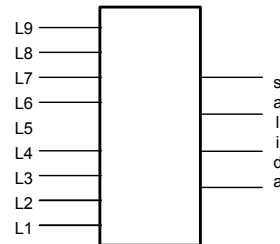
```
with entrada select
```

```
salida <= "10000000" when "000",
          "01000000" when "001",
          "00100000" when "010",
          "00010000" when "011",
          "00001000" when "100",
          "00000100" when "101",
          "00000010" when "110",
          "00000001" when others;
```



Descripción de un codificador de prioridad de ocho líneas

```
salida <= "1001" when L9 = '1' else
          "1000" when L8 = '1' else
          "0111" when L7 = '1' else
          "0110" when L6 = '1' else
          "0101" when L5 = '1' else
          "0100" when L4 = '1' else
          "0011" when L3 = '1' else
          "0010" when L2 = '1' else
          "0001" when L1 = '1' else "0000";
```



Comparación de números:

```
igual <= '1' when A = B else '0';
mayor <= '1' when (A > B) else '0';
menor <= '1' when (A < B) else '0';
```

Suma de dos números:

```
R <= A + B;
```

También los procesos (*process*), que se describen en el próximo subapartado, son concurrentes (cada uno de ellos considerado globalmente es una asignación concurrente) y no puede asignarse valores a una señal en dos procesos diferentes.

23.2.4. Asignaciones secuenciales

Las asignaciones secuenciales se encuentran dentro de un módulo denominado proceso (*process*) y no se ejecutan hasta que «se ha terminado de leer» todo el módulo. Dentro de un proceso puede haber dos o más asignaciones referidas a la misma señal y es válida la última de ellas; en el caso de asignaciones condicionales (que será el caso general), es válida la última de ellas que resulta «efectiva» (es decir, cuyas condiciones se cumplen). Las asignaciones secuenciales no corren peligro de imponer doble valor a una misma señal, pues son consideradas en el orden en que están escrita (igual que un programa de computador) y solamente se aplica la última «efectiva» de ellas.

Los procesos se declaran y se concluyen de la siguiente forma:

```
nombre_del proceso (opcional): process (lista de sensibilidad)
begin
    asignaciones
end process;
```

La lista de sensibilidad se refiere a las señales que «despiertan» el proceso (que lo hacen operativo) y, en el caso de descripción circuital, debe contener todas las señales que actúan como entradas sobre el proceso. En principio, los compiladores no tienen en cuenta la lista de sensibilidad pero suelen avisar si ésta es incompleta.

Cada proceso, considerado globalmente, es una asignación concurrente (o, si asigna valor a varias señales, un conjunto de asignaciones concurrentes sobre ellas): no se puede efectuar asignación a una misma señal en dos procesos diferentes.

Las asignaciones secuenciales pueden ser fijas, condicionales o múltiples. Las fijas utilizan la misma sintaxis que las asignaciones concurrentes fijas, pero las condicionales y las múltiples utilizan sintaxis diferentes:

condicionales:

```
if ..... then señal <= -----; señal <= -----; señal <= -----;
elseif ..... then señal <= -----; señal <= -----; señal <= -----;
elseif ..... then señal <= -----; señal <= -----; señal <= -----;
else      señal <= -----; señal <= -----; señal <= -----;
end if;
```

múltiples:

```
case ..... is
when "....." => señal <= -----; señal <= -----; señal <= -----;
when "....." => señal <= -----; señal <= -----; señal <= -----;
when "....." => señal <= -----; señal <= -----; señal <= -----;
when others => señal <= -----; señal <= -----; señal <= -----;
end case;
```

Dentro de un proceso no pueden utilizarse asignaciones con *when* o con *with* y, de igual forma, las estructuras *if* y *case* no pueden utilizarse fuera de procesos.

Ejemplos de asignaciones secuenciales:

Descripción de un multiplexor de cuatro líneas de entrada

```
process (control,L3,L2,L1,L0)
begin
```

versión 1

```
if control = 0 then y <= L0; end if;
if control = 1 then y <= L1; end if;
if control = 2 then y <= L2; end if;
if control = 3 then y <= L3; end if;
end process;
```

versión 2

```
case control is
  when 0 => y <= L0;
  when 1 => y <= L1;
  when 2 => y <= L2;
  when others => y <= L3;
end case;
end process;
```

Descripción de un codificador de prioridad de nueve líneas

```
process (L9,L8,L7,L6,L5,L4,L3,L2,L1)
begin
```

```
if L9 = '1' then      salida <= "1001";
  elsif L8 = '1' then salida <= "1000";
  elsif L7 = '1' then salida <= "0111";
  elsif L6 = '1' then salida <= "0110";
  elsif L5 = '1' then salida <= "0101";
  elsif L4 = '1' then salida <= "0100";
  elsif L3 = '1' then salida <= "0011";
  elsif L2 = '1' then salida <= "0010";
  elsif L1 = '1' then salida <= "0001";
  else               salida <= "0000";
```

```
end if;
end process;
```

Descripción de un demultiplexor de ocho líneas.

```
control : integer range 0 to 7
```

```
process (control,entrada)
```

```
begin
```

```
-- asignaciones por defecto
```

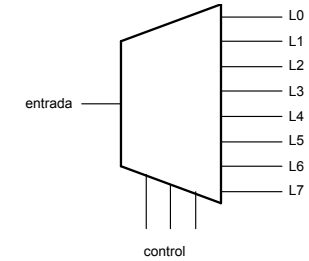
```
L0 <= '0'; L1 <= '0'; L2 <= '0'; L3 <= '0';
```

```
L4 <= '0'; L5 <= '0'; L6 <= '0'; L7 <= '0';
```

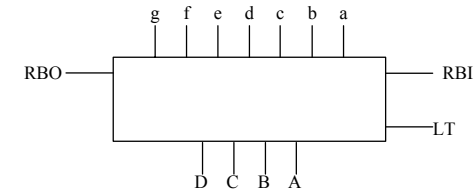
case control is

```
when 0 => L0 <= entrada;
when 1 => L1 <= entrada;
when 2 => L2 <= entrada;
when 3 => L3 <= entrada;
when 4 => L4 <= entrada;
when 5 => L5 <= entrada;
when 6 => L6 <= entrada;
when 7 => L7 <= entrada;
```

```
end case;
end process;
```



23.2.5. Conversor BCD a 7 segmentos de ánodo común



El conversor recibe las 4 entradas BCD y proporciona las 7 salidas (g f e d c b a) correspondientes a la activación de los 7 segmentos (que se activarán con valor 0, ya que son de ánodo común); además, dispone de una entrada LT para test de lámparas y otra entrada RBI, con su correspondiente salida RBO, para apagado de ceros no significativos.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

entity **BCD_7SEG** is

```
port ( D,C,B,A,LT,RBI : in std_logic;
        SALIDA          : out std_logic_vector(1 to 7); -- a b c d e f g
        RBO              : out std_logic );
```

end **BCD_7SEG**;

-- versión 1: con asignaciones concurrentes

architecture **CODIFICADOR** of **BCD_7SEG** is

```
signal bcd: std_logic_vector(3 downto 0);
```

```
signal ENTRADA: integer range 0 to 9;
```

```
signal AUX: std_logic_vector(1 to 7); -- señal auxiliar;
```

```
begin
```

```
bcd <= D & C & B & A; ENTRADA <= conv_integer(bcd);
```

```
RBO <= '1' when (RBI = '1') and (entrada = 0) else '0';
```

```
SALIDA <= "0000000" when LT = '1' else
```

```
"1111111" when (RBI = '1') and (entrada = 0) else AUX;
```

```

with ENTRADA select
    AUX <=
        "0000001" when 0,
        "0010010" when 2,
        "1001100" when 4,
        "0100000" when 6,
        "0000000" when 8,
        "1111111" when others;

end CODIFICADOR;

versión 2: con asignaciones secuenciales

architecture CODIFICADOR of BCD_7SEG is
    signal bcd: std_logic_vector(3 DOWNTO 0);
    signal ENTRADA: integer range 0 to 9;
begin
    bcd <= D & C & B & A;
    entrada <= conv_integer (bcd);
    process(ENTRADA,LT,RBI)
    begin
        RBO <= '0';
        if ( LT = '1' ) then
            SALIDA <= "0000000" ;
        elsif ((RBI = '1') and (ENTRADA = 0)) then
            SALIDA <= "1111111"; RBO <= '1';
        else
            case ENTRADA is
                when 0 => SALIDA <="0000001";
                when 1 => SALIDA <="1001111";
                when 2 => SALIDA <="0010010";
                when 3 => SALIDA <="0000110";
                when 4 => SALIDA <="1001100";
                when 5 => SALIDA <="0100100";
                when 6 => SALIDA <="0100000";
                when 7 => SALIDA <="0001111";
                when 8 => SALIDA <="0000000";
                when 9 => SALIDA <="0000100";
                when others => SALIDA <="1111111";
            end case;
        end if;
    end process;
end CODIFICADOR;

```

23.2.6. Decodificador de mapa de memoria

Se trata de situar, en un mapa de memoria cuyo bus de direcciones tiene 16 líneas, un circuito integrado RAM de 8 K al comienzo del mapa, de 0000(H a 1FFF(H, un adaptador de puertos PIA que tiene 4 registros a partir de la posición A000(H, y dos circuitos integrados ROM al final de memoria, uno de 2K de F000(H a F7FF(H y el otro de 4K de F800(H a FFFF(H; el decodificador de direcciones utiliza, para ello, solamente las 6 líneas superiores del bus de direcciones.

```

library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;

entity MAPA is
    port ( A15, A14, A13, A12, A11, A10 : in std_logic;
           RAM, IO,ROM1,ROM2 : out std_logic); end MAPA;

architecture HABILITACIONES of MAPA is
    signal DIR :std_logic_vector (15 downto 0);
    -- se introduce la señal auxiliar DIR para referir las direcciones a las 16 líneas
    -- del bus de direcciones (y por tanto 16 bits del mapa de memoria)
begin
    DIR <= A15 & A14 & A13 & A12 & A11 & A10 & "00000000000";
    RAM <= '1' when DIR <= 16#1FFF# else '0';
    -- la notación 16#. ....# indica que el número es hexadecimal
    IO <= '1' when DIR = 16#A000#;
    ROM1 <= '1' when (DIR >= 16#F000#) and (DIR <= 16#F7FF#) else '0';
    ROM2 <= '1' when DIR >= 16#F800# else '0';
end HABILITACIONES;

```

23.2.7. Comentarios

Las asignaciones condicionales de tipo concurrente han de ser completas: debe especificarse «qué pasa» en caso de que no se cumplan las condiciones; es decir, deben llevar *else* en el caso de *when* y deben llevar *when others* en el caso de *with*.

La asignación múltiple *case* también debe ser completa; debe llevar *when others* =>. También la asignación con *if* debe ser completa (llevar *else* o, alternativamente, haber dado valores por defecto a las señales) si el circuito es combinacional, pues en caso de no serlo introduce biestables para conservar el valor de las señales (según se verá en el próximo apartado: los procesos producen *memoria implícita*).

En el caso de *if* o de *case*, dentro de una misma condición, se pueden hacer asignaciones a varias señales: *señal* <= -----; *señal* <= -----; *señal* <= -----; por eso es necesario acabar cada asignación con ";" y finalizar el conjunto de condiciones con *end if* o *end case*.

En una asignación múltiple (*with* o *case*), para referirse a varios casos se utiliza el símbolo "|" para separarlos (no es correcto utilizar "or", ya que, en tal caso, se aplicará la operación booleana "o"); por ejemplo: *case entrada is when 1 | 3 | 6 => y <= '1'; ... with entrada select y <= 1 when 1 | 3 | 6, ...*

23.3. Descripción de circuitos secuenciales y sistemas síncronos

23.3.1. Descripción de biestables

En los biestables, la salida actúa también como entrada (realimentación) y, habida cuenta que las salidas VHDL (*port out*) no pueden «ser leídas» desde dentro del circuito (es decir, no pueden actuar como entradas de ninguna asignación), es necesario utilizar para la realimentación una señal interior, del mismo valor que la salida.

```
.....
port( q :out std_logic;
.....

architecture nombre_de_la_architectura of nombre_de_la_entidad is
signal q_interna: std_logic;
begin
q <= q_interna;
```

Por otra parte, los procesos tienen *memoria implícita*: si una señal recibe una asignación condicional dentro de un proceso y el conjunto de asignaciones no es «completo» (es decir, existe alguna condición en que la asignación a dicha señal no está especificada), el proceso asigna por defecto la conservación del valor de dicha señal. Es como si, al comienzo del proceso existiera la asignación *señal <= señal;*, referida, por defecto, a cada una de las señales que reciben alguna asignación dentro del proceso.

Ejemplo:

```
process(a,b)
begin
if a = '1' then p <= b; end if;
end process;
```

En este caso, cuando **a = 1**, **p** adopta el valor de **b** y, cuando **a = 0**, como no se especifica nada dentro del proceso, **p** conserva el valor que tenía anteriormente; es equivalente a cualquiera de las dos descripciones siguientes:

<pre>process(a,b) begin if a = '1' then p <= b; else p <= p; end if; end process;</pre>	<pre>process(a,b) begin p <= p; if a = '1' then p <= b; end if; end if; end process;</pre>
---	--

Diversas formas de describir un biestable RS

```
q_interna <= '0' when R = '1' else '1' when S = '1' else q_interna;
```

```
process (R,S)
begin
if S = '1' then q_interna <= '1'; end if;
if R = '1' then q_interna <= '0'; end if;
end process; -- borrado prioritario
```

```
process (R,S)
begin
if R = '1' then q_interna <= '0'; elsif S = '1' then q_interna <= '1'; end if;
end process; -- también borrado prioritario
```

(téngase en cuenta que un proceso conserva los valores: por ello no es necesario añadir en los dos procesos anteriores *else q_interna <= q_interna ;*)

Diversas formas de describir un biestable D

```
q_interna <= D when E = '1' else q_interna;
```

```
with E select q_interna <= D when '1', q_interna when others;
```

```
process (D,E)
begin
if E = '1' then q_interna <= D; end if;
end process;
```

23.3.2. Circuitos síncronos: descripción del reloj

La descripción de la señal de reloj **CK** ha de hacerse dentro de un proceso, de las siguientes formas:

- si todo el proceso es síncrono


```
process -- sin lista de sensibilidad
begin
wait on CK until CK = '1'; -- flanco ascendente
```
- si hay una parte asíncrona (por ejemplo, un borrado asíncrono con **R**)


```
process(R,CK)
begin
if R = '1' then .....
elsif CK'event and CK = '1' then -- flanco ascendente
o, también, elsif rising-edge(CK) then -- flanco ascendente
```

Biastable D con habilitación y con borrado asíncrono

```

process (R,D,E,CK)
begin
if R = '1' then q_interna <= '0';
elsif CK'event and CK = '1' then
    if E = '1' then q_interna <= D; end if;
end if;
end process;

```

Biastable JK con marcado y borrado asíncronos

```

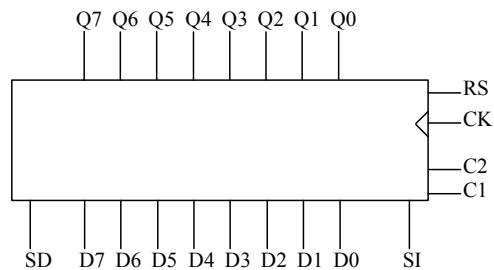
process (R,S,J,K,CK)
begin
if R = '1' then q_interna <= '0';
elsif S = '1' then q_interna <= '1';
elsif CK'event and CK = '1' then
    if J = '1' and K = '1' then q_interna <= not q_interna;
    elsif J = '1' then q_interna <= '1';
    elsif K = '1' then q_interna <= '0';
    end if;
end if;
end process;

```

Registro de desplazamiento bidireccional de 8 bit con carga paralela síncrona

Se trata de diseñar un registro de desplazamiento con las cuatro posibilidades funcionales siguientes, controladas por dos entradas de selección (**C2** y **C1**):

- **00**: retención del valor anterior
- **01**: desplazamiento hacia la izquierda (entrada **SI**)
- **10**: desplazamiento hacia la derecha (entrada **SD**)
- **11**: carga paralelo (entradas **D**)



```

library ieee; use ieee.std_logic_1164.all;

```

entity **REGDESP** is

```

    port (
        CK,RS,C1,C2,SI,SD : in  std_logic;
        D                  : in  std_logic_vector(7 downto 0);
        Q                  : out std_logic_vector(7 downto 0) );

```

end **REGDESP**;

architecture **SINCRONA** of **REGDESP** is

```

    signal Q_interior      :std_logic_vector(7 downto 0);

```

```

    signal control         :std_logic_vector(2 downto 1);

```

begin

```

    Q <= Q_interior;      control <= C2 & C1;

```

REGISTRO: process

-- un proceso puede llevar una etiqueta o «nombre» identificativo delante del mismo

begin

```

wait until CK = '1';

```

```

if ( RS = '1' ) then      Q_interior <= (others => '0');

```

```

    else      case control is

```

```

        when "01" =>      Q_interior <= Q_interior(6 downto 0) & SI;

```

```

        when "10" =>      Q_interior <= SD & Q_interior(7 downto 1);

```

```

        when "11" =>      Q_interior <= D;

```

```

        when others =>

```

```

    end case;

```

end if;

end process;

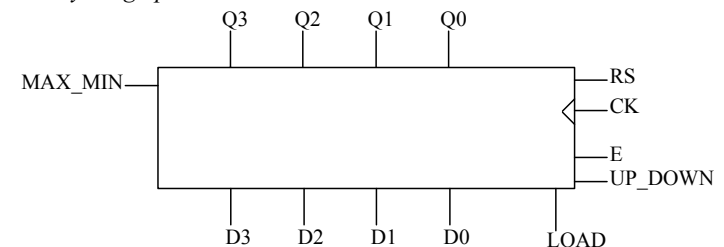
end **SINCRONA**;

Obsérvese que no es necesario añadir $Q_interior \leq Q_interior$; en *when others* ya que un proceso tiene *memoria implícita* (conserva los valores).

23.3.3. Contador década

Contador módulo 10, bidireccional, con habilitación

y con borrado y carga paralela síncronos



```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;

entity DECADA is
    port (
        CK,RS,E,UP_DOWN,LOAD : in std_logic;
        D                     : in std_logic_vector(3 downto 0);
        MAX_MIN               : out std_logic;
        Q                     : out std_logic_vector(3 downto 0));
end DECADA;
```

```
architecture CONTADOR of DECADA is
    signal Q_interior : std_logic_vector(3 downto 0);
begin
    -- COMBINACIONAL:
    Q <= Q_interior;
    MAX_MIN <= '1' when ((UP_DOWN = '1') and (Q_interior = "1011"))
        or ((UP_DOWN = '0') and (Q_interior = "0000")) else '0';

    SINCRONO: process
    begin
        wait until CK = '1';
        if ( RS = '1' ) then
            Q_interior <= "0000";
        elsif ( LOAD = '1' ) then
            Q_interior <= D;
        elsif ( E = '1' ) then
            if ( UP_DOWN = '1' ) then
                if Q_interior = "1001" then Q_interior <= "0000";
                else
                    Q_interior <= Q_interior + 1; end if;
            else if Q_interior <= "0000" then Q_interior <= "1001";
                else
                    Q_interior <= Q_interior - 1; end if;
            end if;
        end if;
    end process;
end CONTADOR;
```

El mismo contador módulo 10, bidireccional, con habilitación pero con borrado y carga paralela asíncronos

Basta cambiar, en la descripción anterior, el proceso SINCRONO por el siguiente, denominado ASINCRONO

```
ASINCRONO: process(CK,RS,Q_interior,LOAD,E,UP_DOWN)
begin
    if ( RS = '1' ) then
        Q_interior <= "0000";
    elsif ( LOAD = '1' ) then
        Q_interior <= D;
    elsif (CK'event and CK='1') then
        if ( E = '1' ) then
```

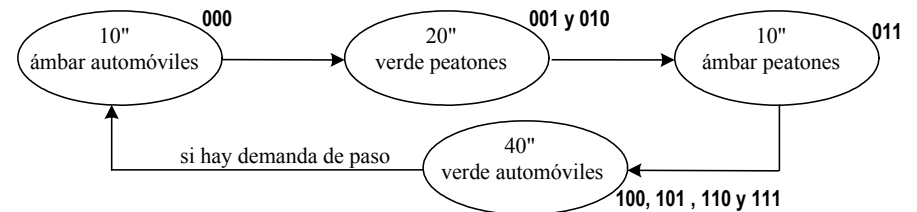
```
        if ( UP_DOWN = '1' ) then
            if Q_interior = "1001" then Q_interior <= "0000";
            else
                Q_interior <= Q_interior + 1; end if;
        else if Q_interior <= "0000" then Q_interior <= "1001";
            else
                Q_interior <= Q_interior - 1; end if;
        end if;
    end if; end if; end process;
```

23.3.4. Temporizaciones sucesivas: semáforo con demanda de paso para los peatones

Sea un cruce de peatones que cuenta con un semáforo para detener a los automóviles, con un pulsador **P** que debe ser activado por los peatones cuando desean cruzar; la activación de **P** da lugar al siguiente ciclo: 10" en amarillo para detener a los automóviles, 20" en rojo (verde para peatones), 10" en amarillo para peatones, pasando finalmente al estado de circulación de automóviles (rojo para peatones); cuando en dicho estado de circulación se recibe una nueva demanda de paso, es atendida pero asegurando siempre que el intervalo mínimo de paso de automóviles sea de 40".

Se utiliza un biestable **RS** para recoger la demanda de paso por parte de los peatones; dicho biestable se borra en el intervalo de ámbar para peatones (que es cuando se completa el paso de peatones, en respuesta a una demanda anterior). El reloj del sistema es de 1 Hz (1 segundo de período).

El ciclo comienza por el servicio a la demanda de paso (ámbar para automóviles) y dura un total de 80"; mientras hay solicitudes de paso se ejecuta normalmente el ciclo completo, pero, si no hay demanda, el ciclo se detiene al final del mismo (cuarto intervalo de 10" de paso de automóviles) y permanece en dicha situación (paso de automóviles) hasta que se produce una petición de paso por parte de peatones.



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity PEATONES is
    port (
        CK, RS, DEMANDA : in std_logic;
        AMBAR, ROJA, VERDE, PAMBAR, PROJA, PVERDE : out std_logic);
end PEATONES;
```


architecture **TEMPORIZADOR** of **PEATONES** is

signal contador_1 : std_logic_vector(3 downto 0);

signal contador_2 : std_logic_vector(2 downto 0);

signal activo : std_logic;

begin

-- biestable para guardar la solicitud de paso

SOLICITUD: process(activo, RS, contador_2, DEMANDA)

begin

if RS = '1' or contador_2 = "011" then activo <= '0';

elseif DEMANDA = '1' then activo <= '1';

end if;

end process;

TEMPORIZACION: process

-- el contador_1 divide por 10: pasa del reloj de 1" a 10"

begin

wait until CK = '1';

if RS = '1' then contador_2 <= "000"; contador_1 <= "0000";

elseif contador_1 = "1001" then -- cada 10"

if contador_2 = "111" then -- fin de ciclo

if activo = '1' then contador_2 <= "000";

contador_1 <= "0000"; end if;

contador_2 <= contador_2 + 1;

contador_1 <= "0000";

else

contador_1 <= contador_1 + 1;

end if;

end if;

end process;

SALIDAS: process(contador_2)

begin

VERDE <= '0'; AMBAR <= '0'; ROJA <= '0';

PVERDE <= '0'; PAMBAR <= '0'; PROJA <= '0';

case contador_2 is when "000" => AMBAR <= '1'; PROJA <= '1';

when "001" => ROJA <= '1'; PVERDE <= '1';

when "010" => ROJA <= '1'; PVERDE <= '1';

when "011" => ROJA <= '1'; PAMBAR <= '1';

when others => VERDE <= '1'; PROJA <= '1';

-- paso de automóviles :others => contador_2 de 100 a 111: 40"

end case;

end process;

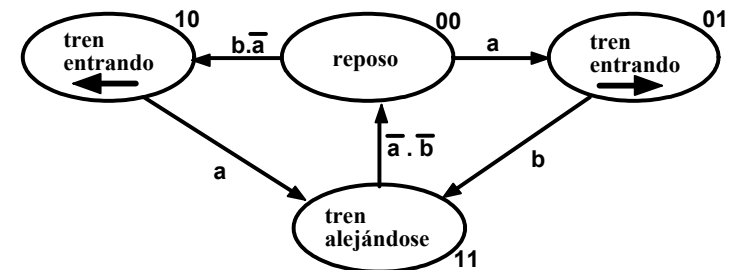
end **TEMPORIZADOR**;

23.4. Descripción de grafos de estados

La evolución del estado de un sistema secuencial se describe muy bien con la asignación múltiple *case* para referirse a cada uno de los estados y, dentro de ella, utilizando adecuadamente la asignación condicional *if* para las transiciones. Existen diversas posibilidades para asignar nombres y códigos binarios a los estados; si se prefiere puede dejarse al compilador la tarea de codificar los estados. A continuación se detallan las descripciones VHDL de varios sistemas secuenciales, a partir de sus grafos de estados.

23.4.1. Autómata de Moore: semáforo para cruce de una vía de tren bidireccional con un camino rural

Se trata de un semáforo de aviso de paso de tren en un cruce de vía única bidireccional con un camino; la vía posee, a una distancia adecuadamente grande, sendos detectores de paso de tren **a** y **b**; los trenes circulan por ella en ambas direcciones y se desea que el semáforo señale presencia de tren desde que éste alcanza el primer sensor en su dirección de marcha hasta que pasa por el segundo sensor tras abandonar el cruce.



library ieee;

use ieee.std_logic_1164.all;

entity **SEMAFORO** is

port (CK,RS,A,B : in std_logic;

SEMF : out std_logic);

end **SEMAFORO**;

architecture **GRAFO** of **SEMAFORO** is

-- definición de los estados

subtype mis_estados is std_logic_vector(1 downto 0);

constant reposo : mis_estados := "00";

constant entra_por_a : mis_estados := "01";

constant entra_por_b : mis_estados := "10";

constant saliendo : mis_estados := "11";

signal estado : mis_estados;

begin

-- evolución del estado:

SECUENCIAL: process

begin

wait until CK = '1';

if (RS = '1') then

estado <= reposo;

else case estado is

```

when reposo => if (A = '1') then estado <= entra_por_a;
                  elsif (B = '1') then estado <= entra_por_b; end if;
when entra_por_a => if (B = '1') then estado <= saliendo; end if;
when entra_por_b => if (A = '1') then estado <= saliendo; end if;
when saliendo => if (A = '0') and (B = '0') then estado <= reposo; end if;
when others =>
end case;
```

end if;

end process;

-- funciones de activación de las salidas:

SEMF <= '0' when estado = reposo else '1';

end **GRAFO**;

23.3.2. Otras formas de describir los estados

En el ejemplo anterior se ha dado nombre y número binario a los estados mediante la definición de un tipo *mis_estados* y la enumeración de los estados y asignación de valores a los mismos, a través de su declaración como constantes.

```

subtype mis_estados is std_logic_vector(1 downto 0);
constant reposo : mis_estados := "00";
constant entra_por_a : mis_estados := "01";
constant entra_por_b : mis_estados := "10";
constant saliendo : mis_estados := "11";
signal estado : mis_estados;
```

Otra forma, más breve, que conduce exactamente a la misma declaración de estados y asignación de valores, es la siguiente:

```

type mis_estados is (reposo, entra_por_a, entra_por_b, saliendo);
attribute enum_encoding: string;
attribute enum_encoding of mis_estados: type is "00 01 10 11";
signal estado: mis_estados;
```

También puede hacerse una declaración de estados sin asignar valores a los mismos, permitiendo que el compilador efectúe esta asignación:

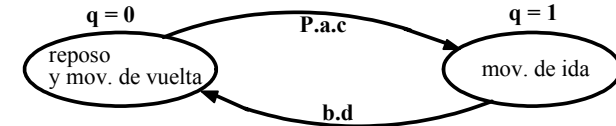
```

type mis_estados is (reposo, entra_por_a, entra_por_b, saliendo);
signal estado: mis_estados;
```

23.4.3. Automáta de Mealy: dos carritos con movimiento de ida y vuelta sincronizados

Sean dos carritos motorizados que se mueven linealmente, entre sendos detectores **a** y **b** el primero y entre **c** y **d** el segundo, de forma que, al activas un pulsador **P**, ambos carritos inician el movimiento desde **a** y **c** y el primero en alcanzar el otro extremo **b** o **d**, espera a que el otro alcance el suyo, para iniciar juntos el movimiento de vuelta.

Un grafo detallado de este sistema de dos carritos puede incluir siete estados (como autómata de Moore) pero puede ser simplificado dando como resultado el grafo siguiente (autómata de Mealy: la necesidad de memoria se limita a distinguir entre dos situaciones: el movimiento de ida hacia **b** y **d** y el de vuelta hacia **a** y **c**):



library ieee; use ieee.std_logic_1164.all;

entity **CARRITOS** is

```

port ( CK,RS,A,B,C,D,P : in std_logic;
       iz_1,der_1,iz_2,der_2 : out std_logic);
```

end **CARRITOS**;

architecture **GRAFO** of **CARRITOS** is

type mis_estados is (vuelta_y_reposo, ida);

signal estado: mis_estados;

begin

-- evolución del estado:

SECUENCIAL: process

begin

wait until CK = '1';

if (RS = '1') then

estado <= vuelta_y_reposo;

else case estado is

```

when vuelta_y_reposo => if (P and A and C) = '1' then estado <= ida; end if;
when ida => if (B and D) = '1' then estado <= vuelta_y_reposo; end if;
```

end case;

end if;

end process;

-- funciones de activación de las salidas:

der_1 <= '1' when (estado = ida) and (B='0') else '0';

iz_1 <= '1' when (estado = vuelta_y_reposo) and (A='0') else '0';

der_2 <= '1' when (estado = ida) and (D='0') else '0';

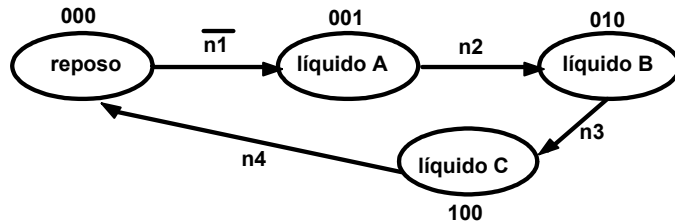
iz_2 <= '1' when (estado = vuelta_y_reposo) and (C='0') else '0';

end **GRAFO**;

23.4.4. Ejemplo realizado con ambas posibilidades, Moore y Mealy: depósito con mezcla de 3 líquidos

Un depósito se llena con una mezcla de tres líquidos diferentes, para lo cual dispone de tres electroválvulas **A**, **B**, **C** que controlan la salida de dichos líquidos y de cuatro detectores de nivel **n1**, **n2**, **n3**, **n4**, siendo **n1** el inferior y **n5** el de llenado máximo. Solamente cuando el nivel del depósito desciende por debajo del mínimo **n1** se produce un ciclo de llenado: primero con el líquido A hasta el nivel **n2**, luego el líquido B hasta el nivel **n3** y, finalmente, el líquido C hasta completar el depósito **n4**.

El grafo de estado correspondiente al autómata de Moore será el siguiente:



```
library ieee;      use ieee.std_logic_1164.all;
```

```
entity DEPOSITO is
```

```
    port (      CK,RS,n1,n2,n3,n4      : in  std_logic;
              A,B,C                      : out std_logic);
```

```
end DEPOSITO;
```

```
architecture MOORE of DEPOSITO is
```

```
    subtype mis_estados is std_logic_vector(3 downto 1);
```

```
-- código de un solo uno
```

```
constant reposo      : mis_estados := "000";
```

```
constant liquido_A    : mis_estados := "001";
```

```
constant liquido_B    : mis_estados := "010";
```

```
constant liquido_C    : mis_estados := "100";
```

```
signal estado: mis_estados;
```

```
begin
```

```
-- funciones de activación de las salidas:
```

```
A <= estado(1);  B <= estado(2);  C <= estado(3);
```

```
-- evolución del estado:
```

```
SECUENCIAL: process
```

```
begin
```

```
wait until CK = '1';
```

```
if ( RS = '1' ) then
```

```
    estado <= reposo;
```

```
else
```

```
    case estado is
```

```
        when reposo =>      if (n1 = '0') then estado <= liquido_A; end if;
```

```
        when liquido_A =>   if (n2 = '1') then estado <= liquido_B; end if;
```

```
        when liquido_B =>   if (n3 = '1') then estado <= liquido_C; end if;
```

```
        when liquido_C =>   if (n4 = '1') then estado <= reposo; end if;
```

```
        when others =>
```

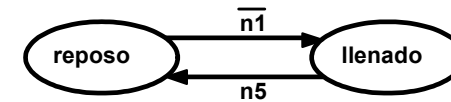
```
    end case;
```

```
end if;
```

```
end process;
```

```
end MOORE;
```

El mismo sistema secuencial puede configurarse con un número más reducido de estados, según el siguiente grafo que corresponde a un autómata de Mealy (en este caso se necesita solamente una variable de estado **q**, pero las funciones de activación de las salidas resultan más complejas, pues dependen de las entradas, de la información que aportan los detectores de nivel):



```
architecture MEALY of DEPOSITO is
```

```
    type mis_estados is (reposo, llenado);
```

```
    signal estado: mis_estados;
```

```
begin
```

```
-- evolución del estado
```

```
SECUENCIAL: process
```

```
begin
```

```
wait until CK = '1';
```

```
if ( RS = '1' ) then
```

```
    estado <= reposo;
```

```
    else case estado is
```

```
        when reposo =>      if (n1 = '0') then estado <= llenado; end if;
```

```
        when llenado =>     if (n4 = '1') then estado <= reposo; end if;
```

```
    end case;
```

```
end if;
```

```
end process;
```

```
-- funciones de activación de las salidas:
```

```
A      <= '1' when (estado = llenado) and (n2='0') else '0';
```

```
B      <= '1' when (estado = llenado) and (n2='1') and (n3='0') else '0';
```

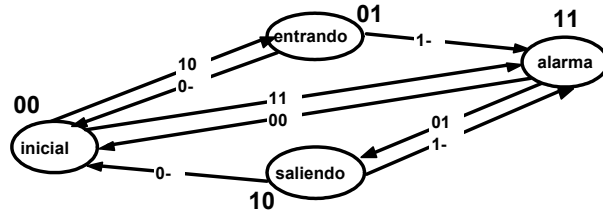
```
C      <= '1' when (estado = llenado) and (n3='1') else '0';
```

```
end MEALY;
```

23.4.5. Ejemplo de grafo con varias transiciones desde cada estado: activación gradual de alarma

Los ejemplos anteriores presentan solamente una transición desde cada estado y en muchos casos actúa solamente una variable de entrada en cada transición; el siguiente grafo, incluye mayor número de transiciones entre estados y dos variables de entrada involucradas en ellas.

Un sistema de detección de temperatura con cuatro niveles (**00**, **01**, **10**, **11**); la alarma debe activarse cuando se detecta **11** (temperatura muy alta), o si se detecta el nivel **10** (alta) en dos ciclos seguidos de reloj y debe desaparecer cuando se detecta **00** (muy baja), o si se detecta el nivel **01** (baja) en dos ciclos de reloj.



```

architecture GRAFO of ALARMA is
  type mis_estados is (inicial,entrando,alarma,saliendo);
  attribute enum_encoding: string;
  attribute enum_encoding of mis_estados: type is "00 01 11 10";
  signal estado: mis_estados;
begin
  SECUENCIAL: process
  begin
    wait until CK = '1';
    if ( Reset = '1' ) then
      estado <= inicial;
    else case estado is
      when inicial => if (entrada = "10") then estado <= entrando;
                       elsif (entrada = "11") then estado <= alarma; end if;
      when entrando => if (entrada(2) = '1') then estado <= alarma;
                       else estado <= inicial; end if;
      when saliendo => if (entrada(2) = '1') then estado <= alarma;
                       else estado <= inicial; end if;
      when alarma => if (entrada = "00") then estado <= inicial;
                     elsif (entrada = "01") then estado <= saliendo; end if;
    end case;
  end if;
end process;

--función de activación de la salida:
y <= '1' when (estado = alarma) or (estado = saliendo) else '0';
end GRAFO;

```

23.5. Otros recursos de VHDL

23.5.1. Tipos de datos

Como ya se hizo en el caso de la declaración de estados en los sistemas secuenciales, se pueden definir y dar valores a constantes, después de la declaración de arquitectura y antes del *begin* de la misma y dichas constantes pueden ser utilizadas para asignación de valores a señales o como parámetros:

```

constant nueve : std_logic_vector(3 downto 0):="1001";
y <= nueve;
constant k : integer:=8;
signal ww : std_logic_vector(k-1 downto 0);

```

También pueden definirse parámetros o valores constantes en la declaración de entidad, antes de los puertos, en la forma siguiente:

```

generic( m : integer := 8 );
port( ....

```

Los vectores pueden utilizarse en forma parcial o por componentes; por ejemplo:

```

signal a : std_logic_vector(15 downto 0);
signal b,c : std_logic_vector(7 downto 0);
signal d,e : std_logic;
begin
  b <= a(15 downto 8); -- mitad más significativa del vector a
  c <= a(11 downto 4); -- parte «central» del vector a
  d <= a(6); -- dígito número 6 del vector a
  e <= a(0); -- bit menos significativo del vector a

```

Al igual que un vector es un conjunto ordenado y numerado de valores (bits), una matriz (*array*) es un conjunto ordenado y numerado de vectores:

```

signal tt : array(1 to 64) of std_logic_vector(7 downto 0);
begin
  tt(21) <= "10101100";
  tt(52,7) <= '1';

```

La declaración anterior de señal introduce una matriz formada por 64 vectores de tipo *standar logic* de 8 bits, numerados de 1 a 64; *tt(21)* se refiere al vector número 21 y *tt(52,7)* señala al bit más significativo del vector número 52.

Variables

En los procesos (y en las funciones, que se explican a continuación), pueden definirse variables internas a los mismos. Mientras que las señales no cambian de valor dentro del propio proceso, sino al final de la aplicación del mismo, las variables cambian de valor dentro del proceso, en cuanto reciben una asignación aplicada sobre ellas.

Las asignaciones a variables se hacen con los símbolos "!=" en lugar de "<=".

Ejemplo de la diferencia funcional entre señales y variables:

<i>p señal</i>	<i>p variable</i>
architecture ejemp1 of entidad is	architecture ejemp1 of entidad is
signal a,b,y,w: std_logic_vector(3 downto 0);	signal a,b,y,w: std_logic_vector(3 downto 0);
signal p : std_logic_vector(3 downto 0);	
begin	begin
....	
p1: process(a,b,p)	p2: process(a,b)
	variable p : std_logic_vector(3 downto 0);
begin	begin
p <= a or b;	p := a or b;
y <= p;	y <= p;
p <= a and b;	p := a and b;
w <= p;	w <= p;
end process;	end process;
...	...

En el ejemplo de la izquierda (proceso p1) la asignación final que reciben ambas señales **y** y **w** será **a and b**, ya que, como señal, **p** recibe asignación al final del proceso y la anterior asignación (**a or b**) resulta ignorada; en cambio, en el ejemplo de la derecha (proceso p2) la señal **y** recibe la asignación **a or b**, y la señal **w**, **a and b**, (como variable **p** ejecuta sus asignaciones inmediatamente).

Por ejemplo, si el vector **a** tiene valor "1010" y el vector **b** vale "0101", los valores resultantes para **y** y **w** serán:

<i>p señal</i>	<i>p variable</i>
y = "0000" "1010" and "0101" = "0000"	y = "1111" "1010" or "0101" = "1111"
w = "0000"	w = "0000"

En relación con el diseño digital, las señales se corresponden más directamente con los nudos del circuito; en tal sentido, es preferible la utilización de señales para la propia descripción circuital, mientras que las variables pueden usarse para dar valores a índices, parámetros, etc.

23.5.2. Alta impedancia y bidireccionalidad

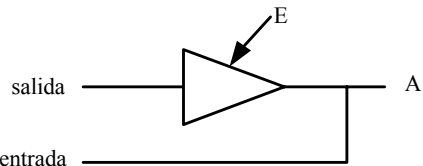
Forma de configurar el estado de alta impedancia en señales tri-estado

```
salidas: out std_logic_vector(7 downto 0)
salidas <= "ZZZZZZZZ" when E = '0' else y;

-- para no especificar el número de valores,
salidas <= (others => 'Z') when E = '0' else y;

-- dentro de un proceso
if E = '0' then salidas <= (others => 'Z')
else salidas <= y; end if;
```

Forma de configurar un terminal bidireccional



```
A
: inout std_logic;
....
signal entrada,E : std_logic;
signal salida :std_logic;
begin
A <= salida when E = '1' else 'Z';
entrada <= A;
```

Cuando el terminal bidireccional **A** actúa como entrada (**E = '0'**), recibe el valor de fuera y su asignación como salida debe hacerse a **Z** (alta impedancia).

Ejemplo: contador cuyas salidas actúan, también, como entradas paralelo

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;

entity contador is
port( salidas :inout std_logic_vector(3 downto 0);
      CK,oe, load :in std_logic);
end contador ;

architecture bidir of contador is
signal qq: std_logic_vector(3 downto 0);
begin
-- actuación como salidas
salidas <= qq when oe = '1' and load = '0' else (others => 'Z');
process
```

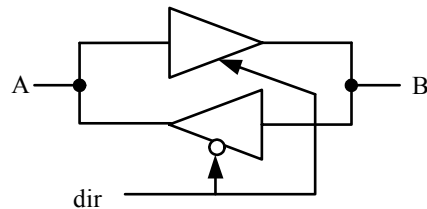
```

begin
wait until CK = '1';
if load = '1' and oe = '1'      then    qq <= "0000";
    elsif load = '1'            then    qq <= salidas;
    elsif oe = '1'              then    qq <= qq + 1;
end if;
end process;
end bidir;

```

El contador es completamente síncrono; se borra cuando *load* y *oe* se ponen a **1**, toma el valor de las *salidas* cuando solamente *load* está a **1** y cuenta y presenta el valor del conteo en las *salidas* cuando *oe* se encuentra **1** (con *load* a **0**)

Forma de configurar un adaptador bidireccional (buffer)



```

library ieee; use ieee.std_logic_1164.all;

entity adaptador is
port( A,B      :inout std_logic_vector(7 downto 0);
      dir      :in  std_logic);
end adaptador ;

architecture bidir of adaptador is
begin
process(A,B,dir)
begin
if dir = '0' then A <= B; else A <= (others => 'Z'); end if;
if dir = '1' then B <= A; else B <= (others => 'Z'); end if;
end process;
end bidir;

```

La descripción anterior corresponde a un adaptador bidireccional de 8 líneas; cuando los terminales actúan como entradas su asignación de salida debe hacerse a Z.

23.5.3. Bucles

Los bucles sirven para aplicar repetitivamente una «instrucción» según un índice que recorre el intervalo de valores que se le fija:

```

for i in ... to ...      loop
                                asignaciones
end loop;

```

Para recorrer todo el rango del parámetro *i* puede utilizarse "*for i in q'range*".

Ejemplo: largo registro de desplazamiento (50 biestables)

```

signal q: std_logic_vector(50 downto 1);
begin
process(reset,clk)
begin
if reset = '1' then
    for i in 50 downto 1      --- también for i in q'range
                                loop      q(i) <= '0';      end loop;
elsif clk'event and clk = '1' then    q(1) <= entrada;
    for i in 50 downto 2 loop      q(i) <= q(i-1);      end loop;
end if;
end process;

```

Otro ejemplo: pila FIFO (64 registros de 8 bits)

```

type pila is array(1 to 64) of std_logic_vector(7 downto 0);
signal qqqq      :pila;
signal entrada    : std_logic_vector(7 downto 0);
begin
salida <= qqqq(64);
process(reset,clk)
begin
if reset = '1' then
    for i in qqqq'range      loop      qqqq(i) <= "00000000";      end loop;
elsif clk'event and clk = '1' then    qqqq(1) <= entrada;
    for i in 2 to 64          loop      qqqq(i) <= qqqq(i-1);      end loop;
end if;
end process;

```

23.5.4. Funciones

Una función consiste en un conjunto de asignaciones, cuya aplicación a las entradas de la función sirve para devolver un valor; se describe al inicio de la arquitectura, antes del *begin* de la misma y puede ser llamada, dentro de la arquitectura, cuantas veces sea necesaria. Dentro de una función no puede ir una instrucción de espera (*wait*), ni una actuación por flancos (*reloj*).

Función:

```
function nombre_de_la_función (entradas: tipo)
    return tipo_de_la_salida is
begin
    asignaciones
return .....;
end;
```

llamada a la función: *señal* <= *nombre de la función (entradas)*;

Ejemplo: descripción de una simple celda sumadora utilizando funciones

```

library ieee; use ieee.std_logic_1164.all;

entity CELDA is
port(  a,b,arrastre_in      : in std_logic;
       suma,arrastre_out   : out std_logic);      end CELDA ;

architecture SUMADOR of CELDA is

function paridad (a,b,c: std_logic) return std_logic is
begin  return ((a xor b) xor c);      end;

function mayoria (a,b,c: std_logic) return std_logic is
begin  return ((a and b) or (b and c) or (c and a));      end;

begin
suma <= paridad(a,b,arrastre_in);
arrastre_out <= mayoria(a,b,arrastre_in);
end SUMADOR ;

```

Otra función: multiplexor que deja pasar el mayor de 2 números

```

function mayor(a,b: std_logic_vector) return std_logic_vector is
variable y: std_logic_vector(7 downto 0);
begin  if a > b then y := a;   else y := b;   end if;
return y;      end;

```

a) dejar pasar el mayor de 3 números A, B, C

```
t <= mayor(A,B);      Y <= mayor(t,C);
```

o también, $Y \leq \text{mayor}(\text{mayor}(A,B),C);$

b) dejar pasar el número intermedio de entre 3 números A, B, C

```

process(A,B,C)
variable t,u,v: std_logic_vector(7 downto 0);
begin
t := mayor(A,B);          u := mayor(B,C);          v := mayor(C,A);
if (t = u) then Y <= v;   elsif (u = v) then Y <= t;   else Y <= u;   end if;
end process;

```

ahora bien, esta solución, aunque «ingeniosa», requiere mucha circuitería (del orden de cinco comparadores y dos multiplexores de 2 números) y puede resolverse en forma mucho más reducida; por ejemplo,

```
Y <= A;  
if (A > B) = (B > C) then Y <= B; end if;  
if (A > C) /= (B > C) then Y <= C; end if;
```

que ocupa poco más de la mitad (unos dos multiplexores y tres comparadores).

Otro ejemplo de función: contador de minutos, segundos y décimas (cronómetro)

Se trata de un cronómetro, con resolución de una décima de segundo y capacidad de conteo hasta una hora; se utilizan dos funciones para configurar los diversos contadores módulo 10 y módulo 6, respectivamente.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

```
entity CRONO is
port (    clk,rs,hab          : in std_logic;
        max                  : out std_logic;
        decimas              : out std_logic_vector(3 downto 0);
        minutos,segundos     : out std_logic_vector(6 downto 0));
end CRONO;
```

architecture **CONTADORES** of **CRONO** is

-- *contaje módulo 10*

```
function mod10 (e :std_logic; q:std_logic_vector) return std_logic_vector is
variable cont :std_logic_vector(3 downto 0);
begin
    if e = '1' then
        if q = "1001" then cont := "0000"; else cont := q + 1; end if;
        else cont := q;    end if;
return cont;  end;
```

-- *contaje módulo 6*

```
function mod6 (e :std_logic; q:std_logic_vector) return std_logic_vector is
variable cont:std_logic_vector(6 downto 4);
begin
    if e = '1' then
        if q = "101" then cont := "000"; else cont := q + 1; end if;
        else cont := q;    end if;
    return cont;    end;

signal seg,min                : std_logic_vector(6 downto 0);
signal dec                    : std_logic_vector(3 downto 0);
signal max1,max2,max3, max4    : std_logic;
-- señales para indicar el máximo ( 9 ó 5 ) de los contadores

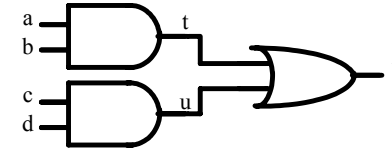
begin
decimas <= dec;
segundos <= seg;
minutos <= min;
max1 <= '1' when dec(3 downto 0) = "1001" and hab = '1' else '0';
max2 <= '1' when seg(3 downto 0) = "1001" and max1 = '1' else '0';
max3 <= '1' when seg(6 downto 4) = "101" and max2 = '1' else '0';
max4 <= '1' when min(3 downto 0) = "1001" and max3 = '1' else '0';
max <= '1' when min(6 downto 4) = "101" and max4 = '1' else '0';

process(rs,clk)
begin
if rs = '1' then dec <= (others => '0');  seg <= (others => '0');  min <= (others => '0');
elsif clk'event and clk = '1' then
    dec(3 downto 0) <= mod10(hab,dec(3 downto 0));
    seg(3 downto 0) <= mod10(max1,seg(3 downto 0));
    seg(6 downto 4) <= mod6(max2,seg(6 downto 4));
    min(3 downto 0) <= mod10(max3,min(3 downto 0));
    min(6 downto 4) <= mod6(max4,min(6 downto 4));
end if;
end process;
end CONTADORES;
```

23.5.5. Descripción estructural: conexión de módulos

Hasta aquí, la descripción VHDL de sistemas digitales se ha desarrollado en forma «funcional», pero este lenguaje admite también una descripción «estructural», detallando las conexiones entre celdas o módulos circuitales.

Ejemplo: descripción estructural de una sencilla combinación de puertas en forma de suma de productos



```
library ieee; use ieee.std_logic_1164.all;

entity puerta_and is port(a,b : in std_logic;  y : out std_logic); end puerta_and;
architecture uno of puerta_and is  begin y<='1' when a='1' and b='1' else '0'; end uno;

library ieee; use ieee.std_logic_1164.all;

entity puerta_or is port(a,b : in std_logic;  y : out std_logic); end puerta_or;
architecture dos of puerta_or is  begin y<='1' when a='1' or b='1' else '0'; end dos;

library ieee; use ieee.std_logic_1164.all;

entity suma_de_productos is
port(a,b,c,d : in std_logic;  y : out std_logic);
end suma_de_productos;

architecture andor of suma_de_productos is
component puerta_and port(a,b : in std_logic; y : out std_logic); end component;
component puerta_or  port(a,b : in std_logic; y : out std_logic); end component;

signal t,u: std_logic;

begin
u1: puerta_and port map(a=>a, b=>b, y=>t);
u2: puerta_and port map(a=>c, b=>d, y=>u);
u3: puerta_or  port map(a=>t, b=>u, y=>y);
end andor;
```

Obsérvese que es necesario declarar la librería y el paquete que se utilizan en cada entidad: la declaración de librerías y paquetes se «satura» (se gasta) cada vez que dicha librería se utiliza.

Los componentes del ejemplo anterior son muy pequeños (simples puertas lógicas básicas), pero de igual forma se procedería para conectar entre sí módulos más amplios (descritos cada uno de ellos con su entidad y su arquitectura), a fin de formar un circuito con ellos.

Otro ejemplo: descripción estructural del cronómetro anterior

Cronómetro con resolución de una décima de segundo y capacidad hasta una hora; se utilizan en esta descripción como módulos los contadores módulo 10 y módulo 6.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
entity mod10 is
port(e,ck,rs : in std_logic; q : out std_logic_vector(3 downto 0); max : out std_logic);
end mod10;
architecture contador1 of mod10 is signal qq : std_logic_vector(3 downto 0); begin
q <= qq; max <= '1' when qq = "1001" and e = '1' else '0';
process(rs,ck) begin
if rs = '1' then qq <= (others => '0');
elsif ck'event and ck = '1' then
if e = '1' then
if qq = "1001" then qq <= (others => '0'); else qq <= qq + 1; end if;
end if; end if; end process; end contador1;
```

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
entity mod6 is
port(e,ck,rs : in std_logic; q : out std_logic_vector(2 downto 0); max : out std_logic);
end mod6;
architecture contador2 of mod6 is signal qq : std_logic_vector(2 downto 0); begin
q <= qq; max <= '1' when qq = "101" and e = '1' else '0';
process(rs,ck) begin
if rs = '1' then qq <= (others => '0');
elsif ck'event and ck = '1' then
if e = '1' then
if qq = "101" then qq <= (others => '0'); else qq <= qq + 1; end if;
end if; end if; end process; end contador2;
```

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

```
entity CRONO is
port ( clk,rs,hab : in std_logic;
max : out std_logic;
decimas : out std_logic_vector(3 downto 0);
minutos,segundos : out std_logic_vector(6 downto 0));
end CRONO;
```

architecture **CONTADORES** of **CRONO** is

```
component mod10 port(e,ck,rs : in std_logic; q : out std_logic_vector(3 downto 0);
max : out std_logic); end component;
component mod6 port(e,ck,rs : in std_logic; q : out std_logic_vector(2 downto 0);
max : out std_logic); end component;
signal max1,max2,max3, max4 : std_logic;
begin
u1: mod10 port map(e=>hab, ck=>clk, rs=>rs, q=>decimas, max=>max1);
u2: mod10 port map(e=>max1, ck=>clk, rs=>rs, q=>segundos(3 downto 0),
max=>max2);
u3: mod6 port map(e=>max2, ck=>clk, rs=>rs, q=>segundos(6 downto 4), max=>max3);
u4: mod10 port map(e=>max3, ck=>clk, rs=>rs, q=>minutos(3 downto 0), max=>max4);
u5: mod6 port map(e=>max4, ck=>clk, rs=>rs, q=>minutos(6 downto 4), max=>max);
end CONTADORES;
```

Bucle para generar módulos: generate

```
etiqueta: for i in ... to ... generate
inserción de módulos
end generate;
```

Ejemplo: descripción estructural de un registro de desplazamiento de 25 biestables

```
library ieee; use ieee.std_logic_1164.all;
entity biestable is
port(ck,rs,d : in std_logic; q : buffer std_logic); end biestable;
architecture ff of biestable is begin
process(ck,rs,q) begin
if rs='1' then q<='0'; elsif ck'event and ck='1' then q<=d; end if;
end process; end ff;
library ieee; use ieee.std_logic_1164.all;
entity registro is
port(ck,rs,entrada : in std_logic; y : buffer std_logic_vector(25 downto 1));
end registro;
architecture desplazamiento of registro is
component biestable port(ck,rs,d : in std_logic; q : out std_logic); end component;
begin
u1: biestable port map(ck=>ck, rs=>rs, d=>entrada, q=>y(1));
```

```
gen1 : for i in 2 to 25 generate
    u_resto: biestable port map(ck=>ck, rs=>rs, d=>y(i-1), q=>y(i));
end generate; end desplazamiento;
```

En este ejemplo, la instrucción *generate* da lugar a la inserción de 24 componentes de tipo biestable (numerados del 2 al 25).

Otro ejemplo: divisor de frecuencia por 10^6

Un modulo divisor de frecuencia por un millón sirve, por ejemplo, para pasar de una frecuencia de 1 MHz (período de 1 microsegundo) a la de 1 Hz (período 1 segundo).

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
entity contador is port(ck,rs,E : in std_logic; max : out std_logic); end contador;
architecture modulodiez of contador is begin
    process(ck,rs,E)
        variable q: std_logic_vector(3 downto 0); begin
        if E = '1' and q = "1001" then max <= '1'; else max <= '0'; end if;
        if rs='1' then q := "0000";
        elsif ck'event and ck='1' then
            if E = '1' then if q = "1001" then q := "0000"; else q := q + 1; end if; end if;
        end if; end process; end modulodiez ;

library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
entity divisor is port(ck,rs,hab : in std_logic; y : out std_logic); end divisor;
architecture unmillon of divisor is
    component contador port(ck,rs,E : in std_logic; max : out std_logic); end component;
    signal aa :std_logic_vector(6 downto 0) ;
    begin
    y <= aa(6); aa(0) <= hab;
    gen1 : for i in 1 to 6 generate
        u_resto: contador port map(ck=>ck, rs=>rs, E=>aa(i-1), max=>aa(i));
    end generate; end unmillon;
```

23.5.6. Librerías y paquetes

Una *librería* es una «carpeta» (un directorio) en la que se almacenan diseños (con su entidad y su arquitectura, cada uno de ellos) y paquetes, que pueden ser utilizados en otros diseños. Un *paquete* es un fichero, dentro de una librería, en el que se declaran componentes, constantes, tipos o funciones.

Librerías y paquetes sirven para organizar y ordenar los diseños y para aprovechar el trabajo generando módulos reutilizables. La librería directa de trabajo, en la que se desarrolla el diseño actual, se denomina *work* y no es necesario declararla.

Ejemplo de librerías y paquetes son los declarados habitualmente al inicio del texto:

```
library ieee;
use ieee.std_logic_1164.all; use ieee.std_logic_arith.all; use ieee.std_logic_unsigned.all;
```

Esta declaración indica el uso de la librería "ieee"; dentro de ella se usan los paquetes "std_logic_1164", "std_logic_arith" y "std_logic_unsigned"; y dentro de dichos paquetes se usan todos los elementos contenidos en los mismos. Si se necesitase sólo un elemento, bastaría poner el nombre del elemento (en lugar de *all*).

Un paquete tiene una parte declarativa y otra descriptiva:

declaración de un paquete:

```
package nombre_del_paquete is
```

-- declaración de tipos, constantes, componentes, funciones y/o procedimientos

```
subtype nombre is ... ..;
```

```
constant nombre :tipo;
```

```
component nombre port( ... .. ) end component;
```

```
function nombre (entradas) return tipo;
```

```
end;
```

descripción de un paquete:

```
package body nombre_del_paquete is
```

-- valores de las constantes y descripción de componentes, funciones y procedimientos

```
entity y architecture de cada componente
```

```
constant nombre: tipo := valor;
```

```
function nombre (entradas) return tipo is
```

```
begin
```

```
    asignaciones
```

```
return expresión;
```

```
end nombre;
```

```
end;
```

para utilizar el paquete

```
library nombre_de_la_librería ;
```

```
use nombre_de_la_librería.nombre_del_paquete.all;
```

24 APROXIMACIÓN ESTRUCTURAL AL DISEÑO DE SISTEMAS COMPLEJOS

- 24.1. Hacer manejable la complejidad
- 24.2. Sistemas con mucha transferencia de información
- 24.3. Sistemas con esquema de cálculo complejo
- 24.4. Máquinas algorítmicas: varios ejemplos
- 24.5. Dividir en partes los tiempos de retraso: segmentación

El estudio de la electrónica digital suele recorrer un itinerario acumulativo de funciones booleanas, conjuntos de funciones, bloques combinacionales, biestables y registros, contadores y sus derivados, memorias, ..., en una aproximación «bottom-up», de lo sencillo hacia arriba; de esta forma llegamos a conocer los elementos disponibles, los recursos constructivos, es decir, las «piezas del diseño». Pero el diseño real supone algo más que conocer las «piezas» disponibles, es un ejercicio de síntesis, un proceso en dirección contraria «top-down», del todo hacia abajo.

Resulta difícil enfrentarse a «la complejidad», pero es algo que los diseñadores de sistemas digitales venimos haciendo en el «día a día» y conviene transferir una pautas o «formas de actuar» que orienten y ayuden a quienes se inician en las tareas de diseño.

Si nos acercásemos al templo de Apolo en Delfos, con confianza en el oráculo, y nuestra pregunta fuese «¿Cómo abordar el diseño de sistemas complejos?», es seguro que la pitonisa, en la forma oscura y ambigua que acostumbra, respondería con dos palabras: «Divide y vencerás». Si, además, la pitonisa tuviera conciencia de la complejidad de nuestros sistemas actuales, seguramente añadiría: «Divide, sí, pero con estructura».

Dividir en partes, pero sin perder de vista la globalidad. Es decir, dividir dotando al conjunto de una estructura adecuada, conociendo «la relación, orden y enlace de las partes para formar el todo» (que es la definición apropiada de «estructura»).

En esta perspectiva, al tratar de los sistemas secuenciales, se introduce el sincronismo como división estructurada de algo tan complejo como es el tiempo: el sincronismo separa el tiempo en unidades sucesivas, como forma de manejarlo con seguridad, eficacia y sencillez.

El presente capítulo, además de plantear, en forma conceptual y genérica, la estrategia de «dividir con estructura», presenta en detalle dos procedimientos sumamente útiles de efectuar tal división: la arquitectura de buses como estructura eficaz para sistemas con mucha transferencia de información y la separación parte de control/parte operativa para esquemas de cálculo complejos. Se incluyen varios diseños de máquinas algorítmicas (parte de control) relativamente complejas (división, raíz cuadrada, conversión BCD-binario) así como dos ejemplos «no aritméticos», referidos a control de cronómetros.

Además, se comenta, en forma breve, el mecanismo de segmentación (pipe-line) como forma de aumentar la velocidad de trabajo, a través de la división en partes de los tiempos de propagación de los bloques combinacionales (cuando tales tiempos son altos).

24.1. Hacer manejable la complejidad

¿Cómo abordar el diseño de sistemas digitales complejos? Divide y vencerás, es una buena respuesta; pero, como complemento a la división, conviene añadir la exigencia de «estructura» (dividir teniendo clara la relación de cada parte con el conjunto).

¿Cómo abordar un diseño que nuestra mente no puede abarcar en conjunto?¹ «Fraccionar sin perder la globalidad», podría ser una respuesta concisa: la idea es dividir el sistema en partes, conservando la visión global del mismo; aún más, dando particular importancia a la estructura, a la relación de cada parte con el conjunto.

Estructura: relación, orden y enlace de las partes para formar un todo (un sistema).

La relación de cada parte con el conjunto presenta una triple referencia:

- funcionalidad de cada parte en relación con el sistema, con las demás partes,
- comunicación física (líneas de conexión) de las partes,
- y ordenación temporal de las señales entre ellas.

Por partes (divide y vencerás) y, a la vez, con estructura (con perspectiva organizada del conjunto): una metodología que permita poner en relación el diseño de cada parte con los requisitos globales que se pretenden, combinando adecuadamente los aspectos de partición, jerarquía y coordinación.

Esto es lo que hacemos cuando formulamos la arquitectura de un sistema en términos de «diagrama de bloques»; cada bloque constituye una parte diferenciada del sistema y la conexión entre ellos sitúa cada parte en la perspectiva de conjunto: confiere estructura a la división.

Tal conexión presenta el triple aspecto citado: conexión funcional (funcionalidad de cada bloque respecto a los demás), conexión física (comunicación entre los bloques) y conexión temporal (ordenación de las señales entre ellos).

Un diagrama de bloques representa una de las posibles soluciones (*arquitecturas*) que realizan el sistema, es decir, que cumplen las especificaciones o requisitos del proyecto; supone una *división estructural* en la que cabe destacar ambos aspectos:

- una *división en partes* (bloques) que permitirá el diseño independiente de cada parte,
- y una *estructura* que expresa la función de cada parte respecto al conjunto (al sistema) y, asimismo, la comunicación entre las partes.

El aspecto estructural, de relación entre partes, exige que se preste especial atención a la conexión entre ellas: tanto la coordinación funcional entre las diversas partes como la comunicación (adecuadamente sincronizada) entre las mismas.

¹ Every engineering construction is an organized system. If it contains only a few components, the designer has no difficulty in bearing them all in mind at once, but if there are many components, the designer almost instinctively creates a hierarchical design with subassemblies and sub-subassemblies as necessary so that he can shape the design without exceeding his span of understanding. By such means in every field of engineering the design task is broken down into manageable sections.

G. G. Scarrott From computing slave to knowledgeable servant: the evolution of computers.

El nombre de *arquitectura* resulta sumamente apropiado, ya que destaca la idea de una estructura que conecta, funcional y físicamente, partes diversas; conocida la situación de cada parte en la estructura, es posible un tratamiento individualizado de la misma.

El diseño es un trayecto desde las especificaciones o requisitos que definen al sistema hasta las funciones o bloques booleanos que determinan su configuración circuital. En esta perspectiva, la división en partes se ejecuta, por lo general, en dos etapas sucesivas (a través de dos «escalones» diferenciados que descienden del todo a las partes): una primera división en módulos, referidos a las especificaciones o prestaciones del sistema y su posterior acomodación a bloques, en relación con los recursos (subsistemas digitales).

En la primera aproximación se divide el sistema en módulos con significado funcional respecto al propio sistema, respecto a las especificaciones o funciones que se pretende que el sistema realice; en un segundo paso se configuran o reajustan dichos módulos en bloques o subsistemas digitales conocidos (y, si es preciso, se introducen bloques «específicos»), definidos todos ellos por su comportamiento booleano.

En todo este proceso de división, conservando la globalidad a través de la estructura, resulta importante dedicar tiempo a:

- dejar claras las especificaciones del sistema,
- llegar a comprenderlas en profundidad,
- conocer sus razones, sus efectos y las relaciones entre ellas,
- idear un esquema de funcionamiento detallado y coherente,

todo ello para llegar a establecer la arquitectura correspondiente, representada en un diagrama de bloques.

De todo lo cual debe dejarse constancia por escrito, porque la escritura es la «herramienta» de las ideas, la única forma de dejarlas claras, de transmitir las y de contrastarlas y de recuperarlas andando el tiempo.

Nunca insistiremos demasiado en la necesidad de «escribir», de documentar el diseño, de describir cada detalle, cada variable, cada registro,... (tanto su función propia como su relación con el conjunto); en el ahorro de tiempo que, en definitiva, supone el texto escrito tanto para clarificar ideas, como para transmitir las, no solamente a otras personas, sino también a uno mismo con el transcurso del tiempo.

La escritura nos impone la disciplina de aclarar y concretar las ideas, de «darles forma» expresándolas en modo explícito y de recogerlas en un soporte material. A cambio, la escritura es un eficaz vehículo de comunicación para transmitir las ideas y permitir su revisión y debate y para almacenarlas y permitir recuperarlas después.

Precisamente, el lenguaje de descripción circuital VHDL nació como herramienta de documentación; como respuesta a la necesidad de describir, en forma clara, precisa y exenta de ambigüedades, el comportamiento detallado de los circuitos digitales complejos. La propia potencialidad de la «escritura estructurada» hizo que la descripción VHDL pasase a ser, también, una excelente herramienta de diseño.

En la perspectiva de «dividir en partes», cabe destacar que el sincronismo no es sino una partición aplicada al tiempo que, al cuantificarlo en unidades, facilita el diseño y le confiere seguridad funcional; es una forma de «modularizar» un aspecto particularmente complejo por su carácter de «continuo» e impreciso cual es el tiempo.

En buena medida el sincronismo en los sistemas digitales equivale a un «buen manejo del tiempo»: al dividir el tiempo en unidades sucesivas facilita la planificación temporal en el diseño de los sistemas complejos (permite referir su actividad funcional a unidades de tiempo discretas y numerables), simplifica los cálculos relativos a los tiempos de propagación y tiempos funcionales de los biestables (el análisis de tiempos) y, a la vez, proporciona una gran seguridad de funcionamiento.

[En tal sentido, se invita, a quienes no lo hayan hecho, a estudiar el capítulo 15 (segundo volumen) dedicado al "significado, requisitos y utilidad del sincronismo". Por no aumentar el número de páginas, dicho capítulo no ha sido repetido en este libro; pero hubiera sido muy apropiado hacerlo.]

En los próximos apartados consideraremos en detalle otros dos casos genéricos de división estructural que presentan particular interés y, además, son muy frecuentes:

- Sistemas con mucha transferencia de información, en los que resulta sumamente útil la división procesador/memoria y, dentro de ésta, la numeración de los diversos registros que permite el tratamiento individualizado de grupos de registros (y, en su caso, de periféricos conectados a tales registros).
- Sistemas con esquema de cálculo complejo, para los cuales es apropiada la división parte operativa/parte de control, que, a su vez, permite tratar por separado cada uno de los recursos de cálculo.

En ambos casos, la división conceptual en dos partes diferenciadas (procesador/memoria, operaciones/control) se refleja sobre una de las partes en una partición de múltiples elementos cuyo diseño se puede abordar por separado: los diversos elementos de la memoria (diversos bloques de memoria o diferentes periféricos) y los varios recursos de cálculo (operadores y registros activos) pueden individualizarse en cuanto a su definición y diseño.

Ni qué decir tiene que muchos sistemas digitales presentan, a la vez, las dos situaciones anteriormente descritas (mucha transferencia de información y esquema de actuación complejo) y, en consecuencia, les son de aplicación las dos particiones antes indicadas.

Por otra parte, las citadas no son las únicas formas de dividir un diseño digital en partes, sino que la estrategia de división es una metodología de tipo conceptual que puede ser aplicada en forma diversa y resulta de particular interés plantearse cuál es el tipo de división estructural que resulta más adecuado a cada caso.

El esquema de «fraccionar sin perder la globalidad» puede (y debe) ser aplicado sucesivamente en forma jerárquica (iterativamente), es decir, a partir de un primer esquema de bloques, los bloques complejos pueden ser divididos en «subbloques», dando lugar a un segundo nivel de «esquemas de bloques» y así sucesivamente hasta «hacer manejable» la complejidad.

En los casos citados en la página anterior, la división se refleja sobre una de las partes en forma múltiple:

- en los sistemas con mucha transferencia de información, la memoria se subdivide naturalmente en bloques y en periféricos (y, respecto al procesador, suele interesar la división en parte de control y parte operativa);
- en los sistemas con esquema de cálculo complejo, la parte operativa se descompone en los diversos operadores y registros necesarios (y la máquina de estado relativa a la «parte de control» puede segmentarse, caso de resultar compleja, en varias máquinas de estado comunicadas entre sí).

Conceptualmente, la idea de «particionar» se opone a la práctica «chapucera» de incorporar «parches» y hacer añadidos «al paso»; por ejemplo, incluyendo al socaire del diseño nuevas condiciones mediante biestables adicionales. Toda especificación o variable de diseño debe estar integrada en el conjunto y quedar reflejada en el diagrama de bloques y, en su caso, en el correspondiente grafo de estado; no es buena técnica desarrollar máquinas de estado dejando biestables de control fuera de las mismas.

Por ello, de vez en cuando, en el desarrollo de prototipos, conviene volver atrás y reescribirlo todo, replanteando situaciones y estados en esquemas de conjunto, que se reflejarán en los correspondientes grafos de estado.

Junto con las anteriores reflexiones conceptuales y formulaciones metodológicas, no puede dejarse de lado la importancia de la experiencia: *Sabe más el diablo por viejo que por diablo*. La experiencia es un requisito insustituible para el «saber hacer» (*know-how*): difícilmente será posible abordar con eficacia el diseño de sistemas complejos sin el bagaje de una experiencia conseguida a través de la práctica, con la dedicación personal y el proceso temporal que ello supone.

24.2. Sistemas con mucha transferencia de información

En el caso de elevada transferencia de información (bien porque el sistema necesite almacenar gran cantidad de datos, bien porque su comunicación con el exterior sea muy intensa o diversa) resulta muy eficaz la división en dos partes, procesador/«memoria», conectadas por buses y la identificación de los registros mediante un número. La sencilla idea de numerar los registros y seleccionarlos a través de su número determina una división estructural múltiple, que permite tratar cada registro o grupo de registros por separado.

Entendemos aquí por procesador la parte operativa y de control, en contraposición con la «memoria» que almacena y transfiere la información; en tal sentido, el término es genérico y no se refiere solamente a los procesadores típicos que actúan bajo programa.

Por otra parte, téngase en cuenta que el nombre de «memoria» se utiliza aquí en su sentido amplio, para designar a la parte «no procesador» que no solamente contiene elementos de memoria que almacenan la información (archivo) sino también dispositivos de entrada y de salida que intercambian información con el exterior (comunicación).

La *memoria* de un sistema digital complejo engloba a todo tipo de registros de los que se extrae directamente o a los que se envía directamente información digital. En tal sentido, incluye los registros de trabajo donde se memorizan datos y resultados (generalmente englobados en bloques de tipo RAM), los registros de información fija que contienen tablas de valores (como pueden ser los bloques ROM) y, también, los registros de adaptación de entradas y de salidas (adaptadores de periféricos).

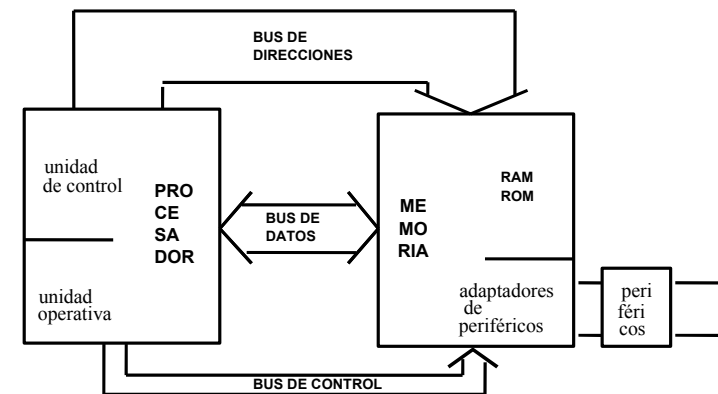
La *memoria* alude a dos unidades diferenciadas por su finalidad funcional:

- la unidad de memoria, como lugar de almacenamiento de la información disponible,
- y la unidad de entradas/salidas, conjunto de periféricos que comunican con el exterior.

La comunicación del sistema con el exterior se realiza a través de registros adaptadores de salida que presentan la información hacia un periférico (optoacoplador, LED, visualizador, conversor D/A, etc.) o a través de adaptadores de entrada, asimilables conceptualmente a registros, que reciben la información desde el periférico (pulsador o conmutado, comparador, conversor A/D, etc.). Desde su punto de vista, el procesador se encuentra con un conjunto de registros que, a su vez, se comunican con un periférico a través del cual reciben o transmiten información respecto al exterior.

De esta forma, el sistema digital puede dividirse en dos partes conceptuales:

- el procesador que controla el proceso y efectúa las operaciones pertinentes,
- y la *memoria*, conjunto de registros que almacenan, reciben o envían la información:



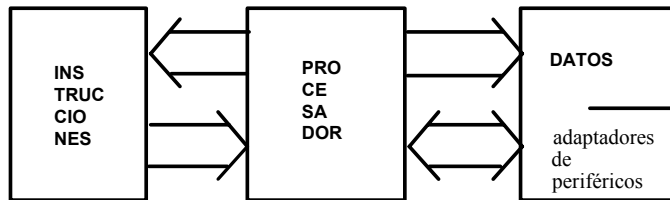
Arquitectura de buses con «memoria única»

La utilización de buses diferenciados permite organizar tanto el almacenamiento como la transferencia de información en forma simple: todos los registros se numeran correlativamente accediendo a ellos por las mismas líneas (bus de datos), un segundo conjunto de líneas seleccionan el registro sobre el que se opera (bus de direcciones) y unas pocas líneas de control determinan la operación a realizar y sincronizan la transferencia de información (bus de control).

[En los capítulos 19 y 20 (segundo volumen) se describe en detalle la arquitectura de buses y la configuración de mapas de memoria, con un amplio número de ejemplos de ubicación de bloques de memoria o adaptadores de periféricos en ellos.]

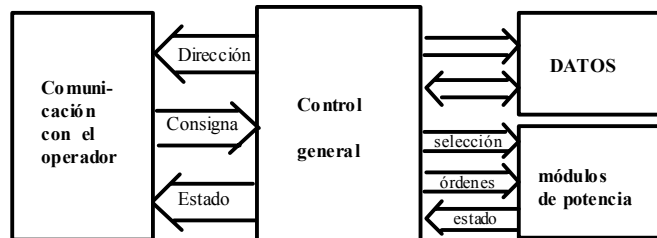
La organización mediante un mapa de memoria, gestionado a través de buses, es una estrategia de tipo genérico que puede ser aplicada de formas muy diversas (memoria única, arquitectura Harvard, módulos especializados, ...): lo básico de esta metodología es el acceso a múltiples registros a través de su número. Además de la división en dos partes, procesador y «memoria», esta arquitectura permite abordar por separado el diseño de cada elemento de la «memoria» (de los diversos bloques o partes de memoria RAM y ROM y de los distintos periféricos), una vez establecida la numeración de sus registros.

A diferencia de la configuración de memoria única, la arquitectura Harvard separa las instrucciones que constituyen el programa de la parte de datos y comunicación con el exterior; de esa forma consigue mayor velocidad de procesamiento, a costa de utilizar un mayor número de líneas debido a la duplicación de los buses.



Arquitectura Harvard de procesadores que actúan bajo programa

En sistemas complejos de control de potencia, puede resultar útil la separación en varias partes, comunicadas a través de buses de reducido número de líneas (incluso algunos de una sola línea); suele interesar diferenciar entre sí los diversos módulos de potencia, la comunicación con el operador y la memoria de datos, controlados a través de un módulo central de control general.



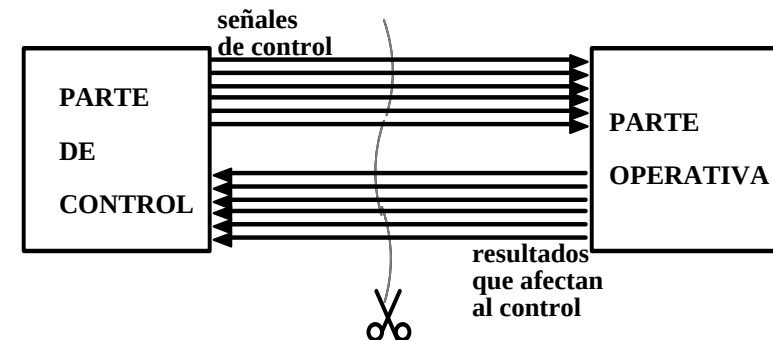
Ejemplo de arquitectura con módulos especializados para sistema de control de potencia

24.3. Sistemas con esquema de cálculo complejo

Un esquema apropiado para este tipo de diseño puede ser el siguiente:

1. Idear el método de operación (el esquema de cálculo), es decir la secuencia genérica de operaciones a realizar y la forma de efectuarlas.
2. Describir dicho esquema mediante un algoritmo o un grafo de estados.
3. Identificar los recursos operativos necesarios (la parte operativa), a saber:
 - los recursos de cálculo (operadores) que permiten efectuar todas las operaciones incorporadas en el algoritmo,
 - los registros necesarios para ejecutar las operaciones en comunicación con los operadores,
 - el conexionado de tales elementos entre sí y su comunicación con el exterior.
4. Expresar la secuencia de control (la parte de control) en forma de máquina de estados que ejecuta el algoritmo de control, activando secuencialmente, a través de sus salidas, los recursos operativos establecidos en el punto anterior. Cada estado corresponde a un conjunto de acciones que se realizan a la vez («en paralelo»), mientras que acciones que requieren un orden correlativo dan lugar a varios estados sucesivos.
5. Abordar el diseño por separado de la parte operativa delimitada en el punto 3 (y, dentro de ella, los diversos operadores y registros) y de la parte de control definida funcionalmente en el punto 4.

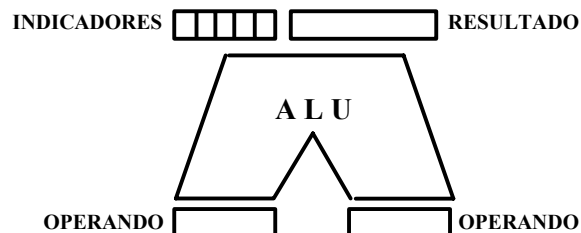
La distinción entre parte operativa y parte de control, establecida en los puntos 3 y 4, simplifica en gran manera el diseño, por cuanto que permite tratar por separado (en momentos diferentes y con una cierta independencia) las operaciones a realizar y la secuencia en que tales operaciones se realizan.



De forma que al diseñar los recursos de cálculo no es preciso considerar el orden con que tales operaciones se ejecutan, ni el número de veces que se repite cada operación, ni los necesarios sincronismos entre operaciones y en la transferencia de datos. Asimismo, al diseñar la parte de control se evita el considerar la configuración en detalle de los bloques que efectúan las operaciones.

La parte operativa será construida, en general, mediante bloques operacionales, registros (algunos de ellos con capacidad de desplazamiento), contadores y multiplexores (para estructurar las comunicaciones); también suelen requerirse algunos biestables individuales (que reciben el nombre de indicadores, *flags*) para resultados parciales (de un solo bit) necesarios en operaciones posteriores (como el «arrastre» de suma o resta) o para informar a la parte de control respecto a posibles saltos condicionados (resultado nulo de una operación, resultado negativo, «desbordamiento», comparaciones, ...).

En muchas ocasiones los bloques operacionales se reducen a uno solo: una ALU que contenga las operaciones aritméticas y lógicas necesarias.



Parte de control: máquina algorítmica

La parte de control puede ser sintetizada por los métodos generales de los sistemas secuenciales, a partir de su máquina de estados.

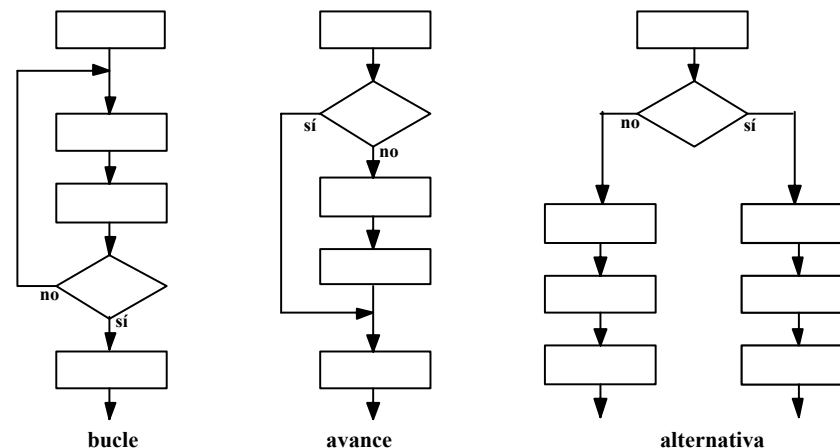
Ahora bien, los algoritmos de control suelen corresponder a máquinas de estados de tipo Moore, relativamente particulares:

- cuyos estados están presentes una sola unidad de tiempo (salvo los estados de «espera» a que se cumpla una condición, que también pueden ser formalizados en términos de «una sola unidad de tiempo»),
- y su estructura es predominantemente lineal (series de estados sucesivos), junto con «bucles», «avances» y «alternativas».

Tales máquinas de estados pueden formularse en términos de series de estados sucesivos y de saltos condicionados, los cuales pueden determinar:

- el recorrido de una serie de estados en forma cíclica, repetidos varias veces hasta que se cumple la correspondiente condición (bucle),
- una modificación en el recorrido de los siguientes estados, de forma que se produce un salto hacia adelante sin pasar por los inmediatamente siguientes (avance),
- la existencia de varias posibilidades o caminos que conducen a diferentes series de estados (alternativa).

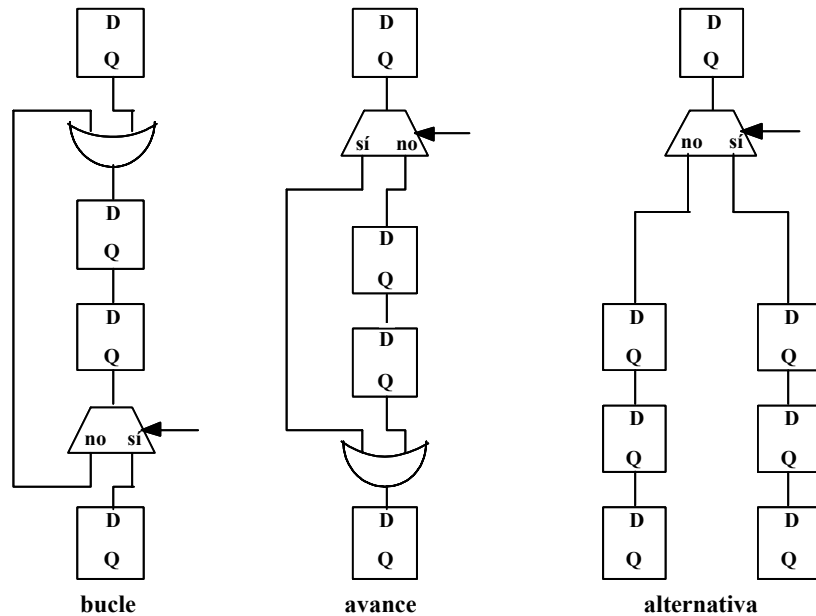
Este tipo de estructura queda muy bien reflejado en los organigramas que se utilizaban para expresar gráficamente la estructura de los programas de computador. En ellos las «cajas» rectangulares representan los estados (conjunto de acciones que se ejecutan en paralelo) y los «rombos» representan los saltos condicionales, que pueden dar lugar a bucles (caso de establecer un lazo cíclico sobre la serie de estados ya recorridos), a avances (caso de saltar en la serie de estados sin recorrer los inmediatamente siguientes) o alternativas (caso de dar paso a dos series diferentes de estados).



La codificación de los estados con «un solo 1» resulta sumamente apropiada, ya que permite construir la máquina de estados reflejando directamente el organigrama sobre un conjunto de biestables, cada uno de ellos correspondiente a un estado («caja» rectangular), y un conjunto de demultiplexores de dos líneas, corresponden a los saltos condicionales («rombos»). [Véase figura de la página siguiente.]

Comentario importante: Conviene tener en cuenta que las acciones propias de un estado, si son de tipo síncrono (y lo serán la mayoría de ellas), no se ejecutan en la unidad de tiempo que corresponde a ese estado, sino cuando llega el flanco de reloj y el estado pasa al siguiente. Por ello, un salto condicional que dependa de acciones síncronas propias del estado anterior no se encontrará con el valor derivado de la aplicación de tales acciones, sino con el valor anterior a las mismas, dando lugar a una transición errónea. Esto puede evitarse añadiendo un *estado intermedio* que permita la ejecución de tales acciones, actualizando los valores relativos a las condiciones antes de abordar el salto.

Por ejemplo, sea un estado en el cual se carga un determinado valor en un registro **P**, seguido de una transición condicionada a $P = 0$; si la carga es síncrona y la condición (el «rombo» que indica el salto) es inmediatamente posterior a dicho estado, la evaluación de la condición se hará sobre el anterior valor del registro **P** y no sobre el nuevo valor. Puede resolverse este problema con un estado adicional entre el citado y el salto condicional, de forma que la actualización de **P** se hará en la transición a este estado, antes de llegar a la evaluación de la condición para ejecutar o no el salto.



Esta forma de construir la parte de control, configurando el correspondiente sistema secuencial de forma que refleje directamente la estructura del algoritmo de control, recibe el nombre de *máquina algorítmica*.

Como ejemplo de descomposición en parte operativa y de control se desarrolla a continuación el diseño (uno de los posibles diseños) de un procesador dedicado específicamente a calcular el producto de dos números binarios **P** y **Q** de 64 dígitos, cuyo resultado **R** será un número binario de hasta 128 dígitos.

[Para facilitar su exposición, este ejemplo se refiere a un cálculo numérico conceptualmente simple y bien conocido; el próximo apartado contiene otros ejemplos de cálculo y un par de ejemplos «no aritméticos» referidos a control de cronómetros.]

1. El *esquema de cálculo* puede ser el siguiente:

- multiplicar el multiplicando **P** por cada dígito del multiplicador **Q** (comenzando por el menos significativo); dicho producto da como resultado **P** cuando el bit de **Q** vale 1 y 0 si dicho bit es nulo,
- efectuar el correspondiente desplazamiento sobre los productos parciales anteriores para ajustarlos a su valor relativo,
- y sumar los productos parciales una vez desplazados.

Este esquema de actuación se corresponde con el que se emplea al hacer «a mano» una multiplicación: cada fila de dicha multiplicación es un producto parcial por uno de los dígitos del multiplicador, cada fila se desplaza un lugar respecto a la anterior para tener en cuenta su valor relativo y, una vez desplazadas, se suman todas las filas.

El esquema de cálculo propuesto necesita un sumador de 128 bits (ya que el resultado puede alcanzar tal longitud); ahora bien, es posible utilizar un sumador de solamente 64 dígitos si la suma se efectúa sobre los bits más significativos del resultado y lo que se desplaza, en cada suma parcial, es el propio resultado; es decir:

- hacer 64 veces lo que sigue
- recorrer sucesivamente los dígitos del multiplicador **Q** desde el bit menos significativo y, cuando el correspondiente bit de **Q** valga 1, sumar el multiplicando **P** sobre el resultado **R**; efectuar esta suma sobre la parte más significativa del resultado: $R' = \{65 \text{ bits superiores de } R\}$ (obsérvese que las sumas parciales se efectúan sobre los 65 bits más significativos de **R**, ya que el resultado de una suma con operandos de 64 bits puede llegar a tener 65 bits)
- desplazar el resultado **R** hacia la derecha (dividir por 2) para ajustar el valor relativo de cada suma parcial respecto a la siguiente a efectuar; no hacer este desplazamiento tras la última de las sumas.

Este esquema da lugar a 64 sumas parciales posibles, correspondientes a los 64 dígitos del multiplicador (dichas sumas se producen sobre los bits 127-63 del resultado); pero sólo produce 63 desplazamientos, de forma que el bit menos significativo del producto, que se forma en la primera de las sumas parciales sobre el bit 63 del resultado, pasa finalmente al bit 0 (unidades) del mismo.

2. El *algoritmo* correspondiente al anterior esquema de cálculo puede ser el siguiente:

Se introduce un registro **C** (contador) para expresar el número de veces que se ha realizado el ciclo básico del algoritmo (sumas y desplazamientos).

Inicio del algoritmo

Leer **P** y **Q**;

Borrar **R** y **C**;

Repetir

Si bit_0 (unidades) de **Q** = 1

Sumar **P** + **R'** sobre **R'**;

Fin_de_Si;

Si **C** = 111111 (63 $\equiv$ 64 ejecuciones de la repetición;
la primera vez **C** = 0)

Fin del algoritmo.

Fin_de_Si;

Desplazar **Q** hacia la derecha;

Desplazar **R** hacia la derecha;

Incrementar **C** en una unidad;

Fin_de_Repetir (volver al inicio de la repetición);

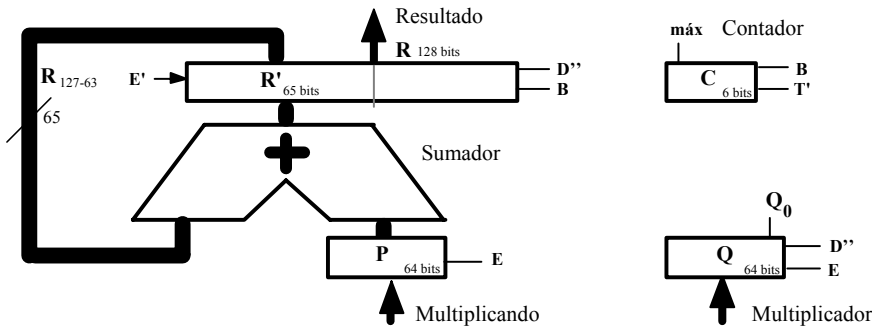
3. Recursos de cálculo necesarios:

Operadores:

- un sumador de 64 bits (salida en 65 bits),
- capacidad de desplazamiento hacia la derecha del resultado **R** y del multiplicador **Q**,
- un contador **C** módulo 64 con entrada de borrado y detección de máximo (11111).

Registros:

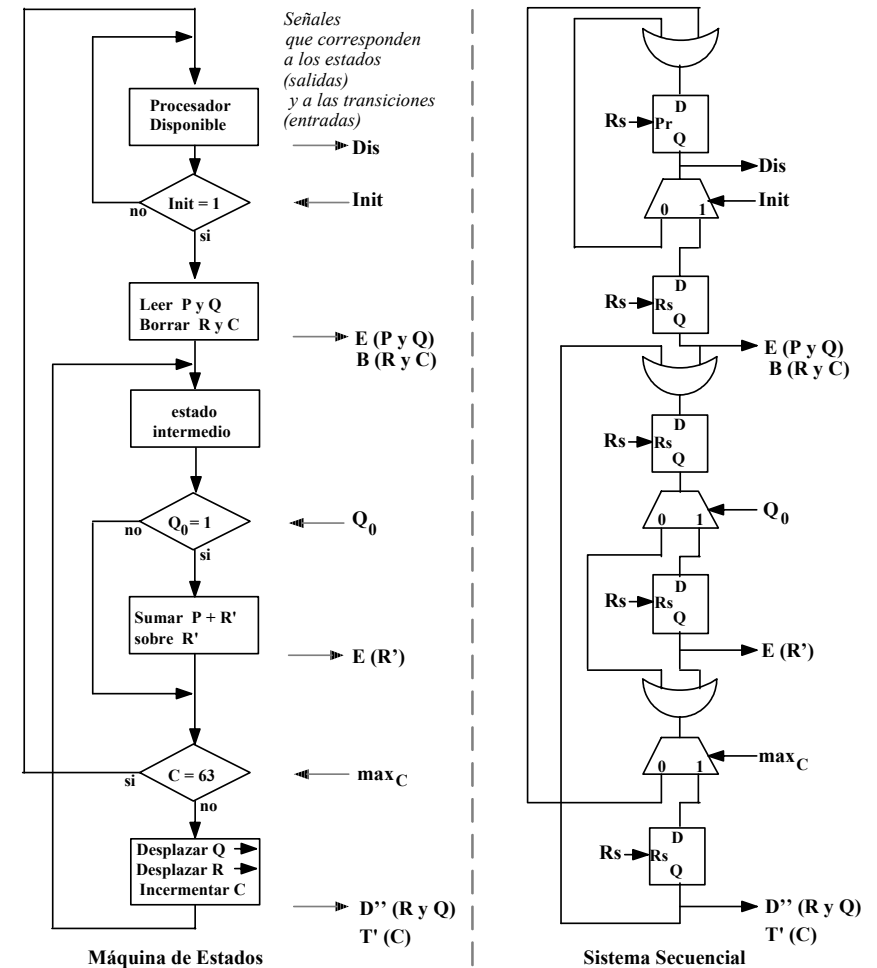
- un registro de retención de 64 bits para el multiplicando **P**,
- un registro de retención de 64 bits para el multiplicador **Q** (con desplazamiento hacia la derecha según se ha dicho en los requisitos de operadores),
- un registro de 128 bits para el resultado **R**, con entrada de borrado (y desplazamiento hacia la derecha según también se ha dicho), cuyos 65 bits más significativos **R'** han de poderse cargar en paralelo (registro de retención respecto a dichos 65 bits),
- un contador de 6 bits **C**, con entrada de borrado (y detección de máximo).



Las señales expresadas en la figura anterior, correspondientes a señales de control de los recursos de cálculo, son las siguientes:

- **E** : entrada de habilitación del correspondiente registro de retención; cuando **E = 1** el registro recoge la palabra binaria situada en sus entradas (en el caso del registro de resultados, su habilitación **E'** solamente actúa sobre los 65 bits más significativos: **R'**),
- **B** : entrada de borrado síncrono del correspondiente registro (incluido el contador); cuando **B = 1** todos los bits del registro pasan a valor 0,
- **D''** : entrada de desplazamiento hacia la derecha (división por 2) del correspondiente registro; cuando **D'' = 1** cada biestable recoge el valor que tenía el biestable de su izquierda y el biestable más significativo pasa a valor 0,
- **T'** : entrada de habilitación de conteo del contador; cuando **T' = 1** el contador incrementa su valor en una unidad (con el reloj del sistema secuencial),
- **máx** : salida indicadora de que el contador se encuentra en su valor máximo 11111
- **Q0** : salida correspondiente al biestable menos significativo (unidades) del registro **Q**.

4. La *maquina de estados* que expresa la secuencia de control puede ser la representada en el organigrama de la figura siguiente (parte de la izquierda).



5. En esta misma figura (parte de la derecha) se representa el *sistema secuencial* que realiza dicha máquina de estados (en una codificación de estados con «un solo 1»).

Se ha supuesto que el proceso de cálculo se activa con una señal **Init** (inicio) y que cuando acaba se vuelve al estado inicial, de forma que la señal **Disp** (disponible) señala que se ha completado el último proceso de cálculo que ha sido activado. La señal **Rs** (reset) de los biestables sirve para inicializar el sistema: borra todos los biestables (**Rs**) menos el primero (**Pr**), que lo pone a 1.

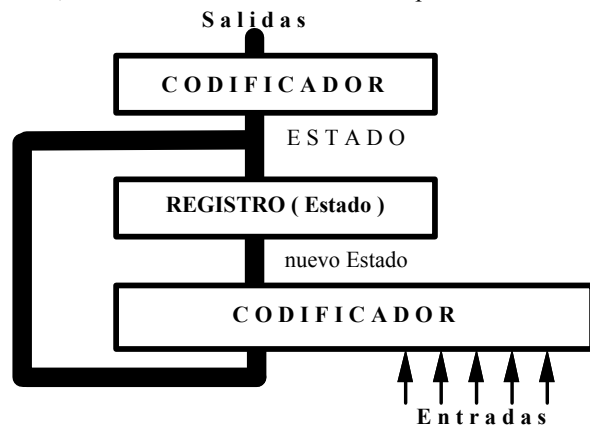
En la anterior máquina algorítmica se ha introducido un *estado intermedio* para que las acciones *cargar Q* e *incrementar C* (propias, respectivamente, del segundo estado y del último) se ejecuten antes de llegar a los saltos condicionales que dependen de ellas ($Q_0 = 1$ y $C = 63$).

Sin dicho estado intermedio, la primera vez que se evalúe $Q_0 = 1$ se encontrará con el registro **Q** a cero (pendiente de cargar el nuevo valor de **Q**); asimismo, cuando, desde el último estado, se pase directamente a la condición $C = 63$ (por ser $Q_0 = 0$), el valor de **C** estará pendiente del último incremento correspondiente a dicho estado. Un solo estado adicional, previo a las dos condiciones de salto citadas, facilita, en este caso, la ejecución de las dos acciones señaladas (*cargar Q* e *incrementar C*).

Configuración microprogramada de la parte de control

El circuito de la página anterior corresponde a la forma «microcableada» que construye la parte de control, con codificación de «un solo uno» que permite identificar cada estado (y el conjunto de acciones asociadas al mismo) con un biestable y trasladar directamente el grafo de estados (especialmente si se dibuja en forma de «ordinograma») a su configuración circuital.

La parte de control puede ser construida, también, en forma «microprogramada» (ver apartado 14.3, volumen segundo), recogiendo el estado sobre un registro y utilizando un codificador para calcular el nuevo estado a partir del estado anterior y del vector de entradas; no resulta oportuna la codificación con «un solo uno» sino que, para reducir el tamaño del codificador, interesa utilizar el menor número posible de variables de estado.



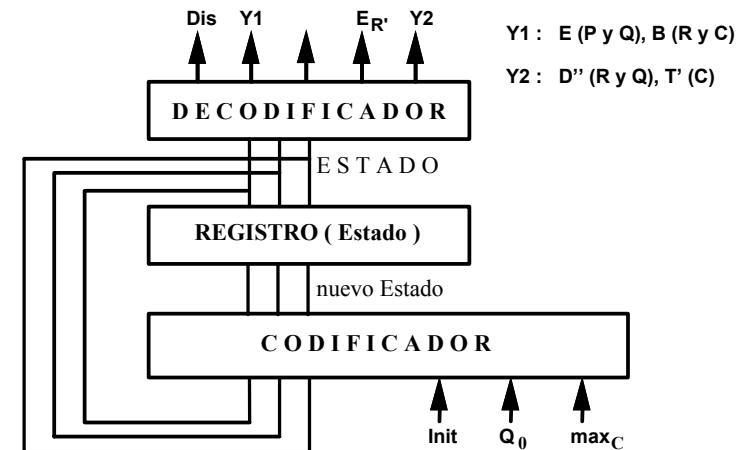
En estructura ROM, los codificadores se configuran directamente desde su tabla de conversión: su «Matriz O» coincide con la propia tabla funcional; podemos considerar que los codificadores contienen «microinstrucciones» en la siguiente forma:

- el primer codificador contiene la tabla relativa al cálculo del estado siguiente, a partir del estado anterior y el vector de entrada;
- el segundo contiene la tabla de los vectores de salida en relación con los estados.

La forma microprogramada conserva, «en forma de programa», la estructura de la parte de control, pues el codificador expresa directamente la correspondencia entre la situación actual (estado y vector de entrada) y el nuevo estado; por ello, resulta fácil efectuar modificaciones de esta máquina de estados, cambiando las correspondientes «microinstrucciones» (la programación) del codificador.

En principio, la configuración «microcableada» es más rápida (menores tiempos de propagación) y ocupa un área de integración más reducida, pero puede ser muy compleja. La forma «microprogramada» es más directa, más fácil de modificar (basta cambiar las correspondientes «microinstrucciones» en los codificadores), pero puede necesitar bloques ROM grandes.

En el caso del multiplicador de dos números binarios, el número de estados necesarios es 5, que pueden codificarse con 3 variables de estado; la parte de control requiere un codificador de 6 entradas (3 para el estado anterior y 3 para el vector de entradas) y 3 salidas para calcular el nuevo estado y un simple decodificador de 3 entradas para el vector de salida, según el diagrama de bloques siguiente:



Sistemas que actúan bajo programa

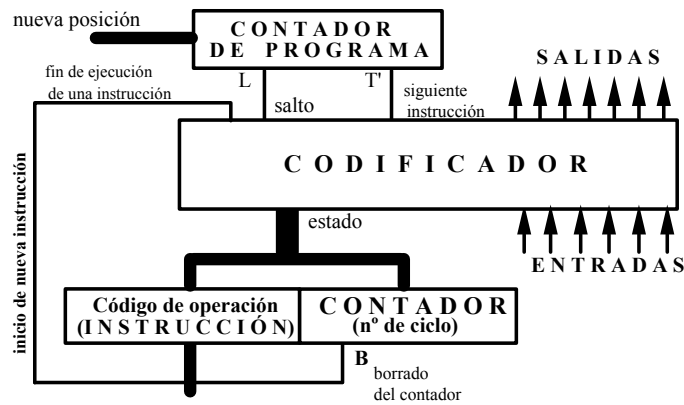
En los sistemas digitales que actúan bajo programa (procesadores programables, computadores, microprocesadores, microcontroladores,...) es clásica la separación entre la parte operativa (ALU y registros) y la parte de control; en esta segunda, el estado se encuentra conformado básicamente por el contador de programa, la instrucción que se está ejecutando y un contador de ciclos que desglosa dicha ejecución.

Dentro de una instrucción, es preciso distinguir dos partes claramente diferenciadas por su funcionalidad: el código de operación (que establece cuál es la actuación específica de la instrucción) y la dirección (o, en su caso, direcciones) del operando (o de los diversos operandos).

El código de operación de una instrucción constituye la parte más relevante del estado del procesador; junto con ella, un contador numera sucesivamente los ciclos de ejecución de la instrucción (ya que, en general, cada instrucción empleará varios ciclos de reloj para su ejecución):

estado = código de operación (de la instrucción) + nº de ciclo (contador)

La realización microprogramada de la máquina de estados que controla la ejecución de las instrucciones es de particular interés y el correspondiente codificador coincide con el denominado «decodificador de instrucciones».



El contador de programa señala la ubicación (en el mapa de memoria) de la siguiente instrucción a ejecutar y, en tal sentido, es parte del estado del procesador (pues es relevante en el control de la secuencia de instrucciones), pero el «estado del contador» no se realimenta hacia dentro de la parte de control.

El primer ciclo de todas las instrucciones (nº de ciclo = 0) consiste en la búsqueda de la propia instrucción, comenzando por su código de operación. La «lectura» de una instrucción completa requiere, en ocasiones, varios ciclos ya que puede ocupar varias palabras sucesivas; tras lo cual, el contador de programa avanza una unidad, pasando a señalar la instrucción siguiente; los «saltos» se producen modificando el contenido de contador mediante «carga en paralelo» de la nueva localización.

24.4. Máquinas algorítmicas: varios ejemplos

Se detallan, a continuación, varios ejemplos de sistemas digitales relativos a cálculos numéricos: división, raíz cuadrada, conversión BCD-binario y binario-BCD, seguidos de un par de ejemplos «no aritméticos» referidos a control de cronómetros.

Nota: Todas las máquinas algorítmicas de este apartado (así como la del multiplicador de 64 bits, descrita en el apartado anterior) han sido comprobadas, a través de su descripción en VHDL y la posterior simulación de la misma (utilizando el compilador MAX+plus II de ALTERA).

24.4.1. Divisor de dos números binarios

Sean P y Q dos números binarios, de 64 y 16 dígitos, respectivamente.

Esquema de cálculo

- recorrer el dividendo **P** dígito a dígito (comenzando por el más significativo), restando el divisor **Q** cuando se pueda;
- en cada desplazamiento, añadir un **1** al resultado **R** si se efectúa la resta y un **0** cuando ésta no es posible.

Este esquema se deduce directamente del que se emplea para hacer «a mano» la división en binario:

- se toman los primeros dígitos del dividendo (los más significativos), de forma que correspondan a un número mayor o igual que el divisor, se les resta el divisor y se pone un **1** en el resultado;
- a partir de aquí, se toma el resto resultante y se le añade (se «baja») un nuevo dígito del dividendo y, si es posible, se le resta el divisor y se añade un **1** al resultado y, caso de que la resta no sea factible, se añade un **0** al resultado;
- y se repite, sucesivamente, el anterior proceso de «bajar» un nuevo dígito del dividendo y restar, si es posible, hasta completar el recorrido del mismo.

Algoritmo

Se utilizan dos registros auxiliares: un contador **C** para expresar el número de veces que se ha realizado el ciclo y un registro **S**, «resto parcial» o «dividendo actual», que recoge el número binario que, en cada momento, se compara con el divisor; dicho registro **S** se conecta con el **P**, formando un registro de desplazamiento (hacia la izquierda) **S-P**.

Inicio del algoritmo

Leer **P** y **Q**; Borrar **R**, **S** y **C**;

Repetir

Desplazar hacia la izquierda **S-P**; Incrementar **C** en una unidad;

Si $S \geq Q$ Restar **S - Q** sobre **S**;
Desplazar **R** hacia la izquierda con entrada **1**;

Si no Desplazar **R** hacia la izquierda con entrada **0**;

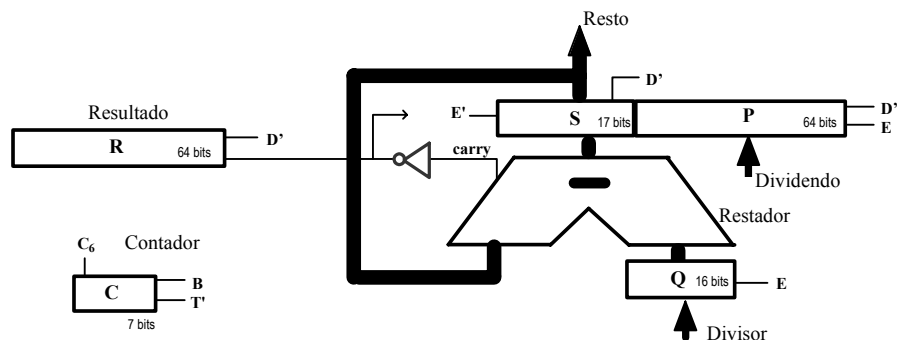
Fin_de_Si;

Si bit₆ (más significativo) de **C** = **1** (64 ejecuciones de la repetición)

Fin del algoritmo.

Fin_de_Si;

Fin_de_Repetir (volver al inicio de la repetición);

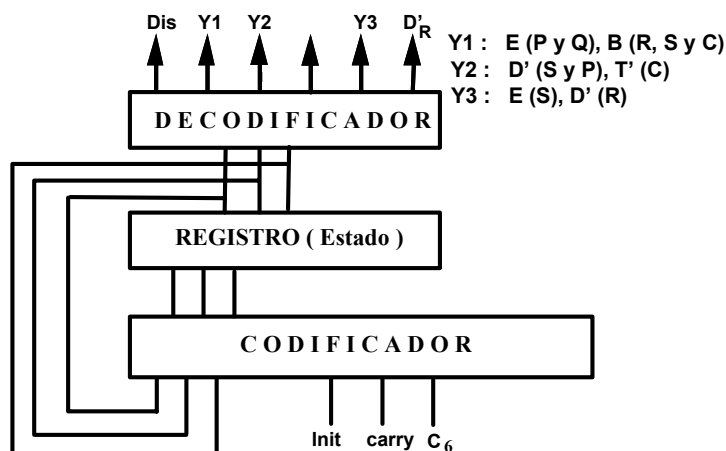


El registro **S** unido al **P**, como registro de desplazamiento **S-P**, permite ir añadiendo al «resto parcial» o «dividendo actual» **S** los dígitos del número **P** uno a uno; la entrada de control **D'** se refiere a desplazamiento hacia la izquierda (en el caso del multiplicador se utilizaba **D''** para el desplazamiento hacia la derecha).

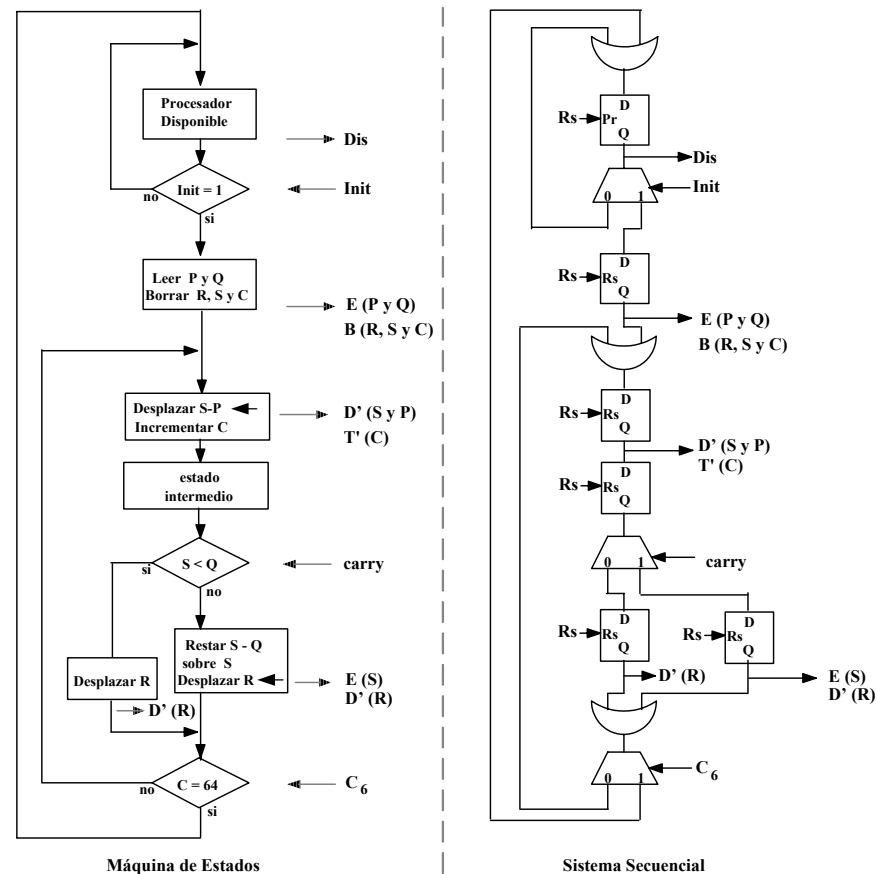
La salida de arrastre del restador (acarreo **carry**) sirve para comparar **S** (el «resto parcial») con el divisor **Q**: cuando dicha salida es **0** ($S \geq Q$) se ejecuta la resta (se habilita el resistro **S**) y se carga un **1** (por desplazamiento) en el resultado; cuando es **1** ($S < Q$) no se efectúa la resta (habilitación del resistro **S** a **0**) y en el resultado se carga un **0**.

Configuración microprogramada de la máquina de estados

La parte de control requiere 6 estados, según la correspondiente máquina de estados representada en la figura de la página siguiente:



Máquina de estados y su configuración microcableada



Ha sido necesario introducir un *estado intermedio* antes de la condición $S > Q$ para que la acción *desplazar S-P* se ejecute antes de llegar al salto condicional que depende de ella ($S < Q$). Sin dicho estado intermedio, al llegar al salto, el registro **S** no habría tomado, por desplazamiento, un nuevo dígito de **Q**.

24.4.2. Raíz cuadrada de un número binario

Sea P un número binario de 64 dígitos.

Esquema de cálculo

- recorrer el número **P** de dos en dos dígitos (comenzando por los más significativos), restando, cuando se pueda, el resultado parcial **R** con el añadido **01** al final;
- en cada desplazamiento, añadir un **1** al resultado **R** si se efectúa la resta y un **0** cuando ésta no es posible.

Este esquema se deriva del que se emplea para hacer «a mano» la raíz cuadrada de un número binario:

- se toman los dos primeros dígitos (los más significativos; si el número de dígitos es impar se toma solamente el primero de ellos) y, si es posible, se les resta **01** y se pone un **1** en el resultado (si no es factible, no se hace la resta y el resultado es **0**);
- a partir de aquí, se toma el resto resultante y se le añaden (se «bajan») dos nuevos dígitos del número de manera que, si resulta un resto mayor o igual que el doble del resultado parcial con un **1** añadido (que equivale a añadir, simplemente, **01** al resultado parcial), se resta y se añade un **1** al resultado y, caso de que la resta no sea factible, se añade un **0** al resultado;
- y se repite, sucesivamente, el anterior proceso de «bajar» dos nuevos dígitos y restar, si es posible, hasta completar el recorrido del número inicial.

Algoritmo

Se utilizan dos registros auxiliares: un contador **C** para el número de veces que se ha realizado el ciclo y un registro **S**, «resto parcial», que recoge el número binario que, en cada momento se compara con el **R01** (el resultado parcial, seguido de **01**).

Al igual que en el caso anterior, el registro **S** se une como registro de desplazamiento al **P** (registro **S-P**), para ir añadiendo al «resto parcial» **S** los dígitos del número **P** de dos en dos.

Inicio del algoritmo

Leer **P**; Borrar **R, S y C**;

Repetir

Desplazar hacia la izquierda **S-P**; Incrementar **C** en una unidad;

Desplazar hacia la izquierda **S-P**;

$$\text{Si } S \geq R$$

Restar **S - R&01** sobre **S**;

Desplazar **R** hacia la izquierda con entrada 1;

Si no Desplazar **R** hacia la izquierda con entrada **0**;

Fin de Si;

Si bit5 (más significativo) de C = 1 (32 ejecuciones de la repetición)

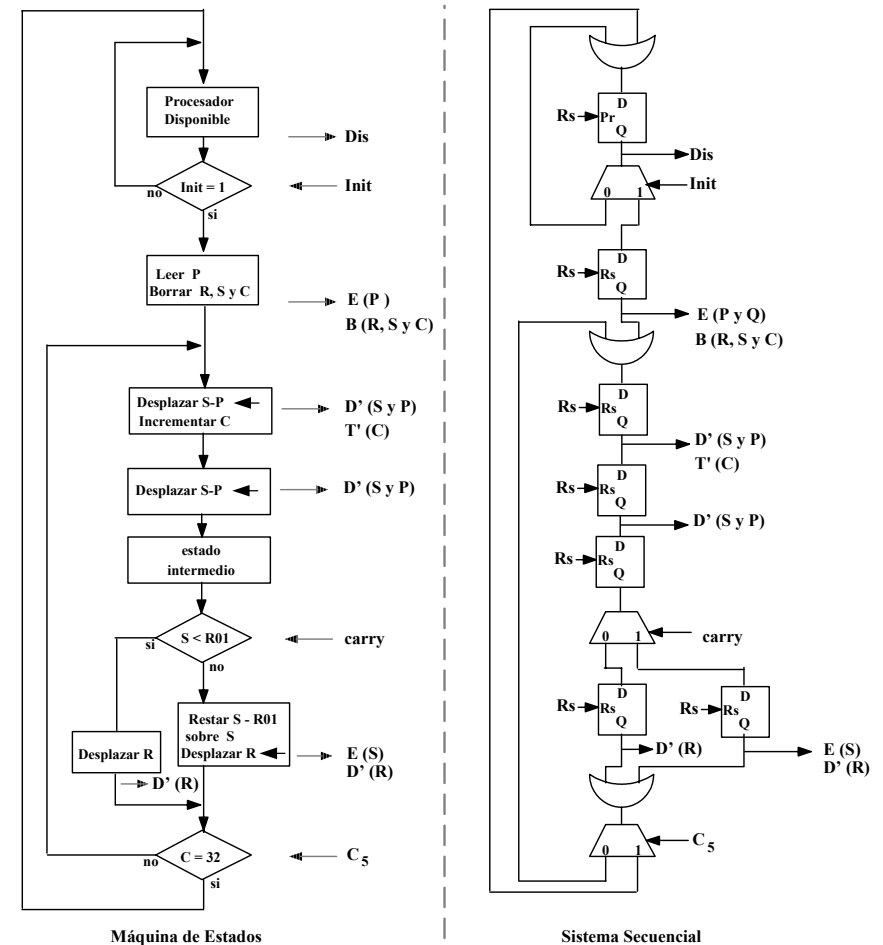
Fin del algoritmo.

Fin de Si;

Fin de Repetir (volver al inicio de la repetición);

Máquina de estados y su configuración microcableada

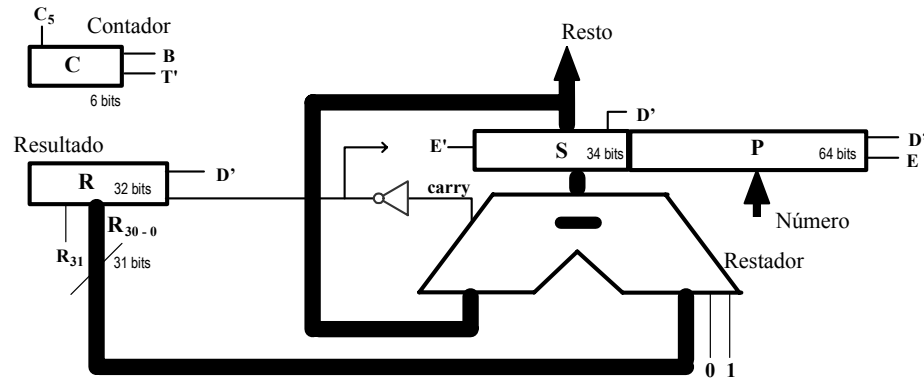
El control expresado en el algoritmo anterior requiere 6 estados (más un *estado intermedio*), según la máquina de estados representada en la figura siguiente.



Ha sido necesario introducir un *estado intermedio* antes de la condición $S < R01$ para que la acción *desplazar S-P* se ejecute antes de llegar al salto condicional que depende de ella ($S < R01$); de otra forma, el registro S no hubiera tomado, por desplazamiento, dos nuevos dígitos de Q sino solamente uno.

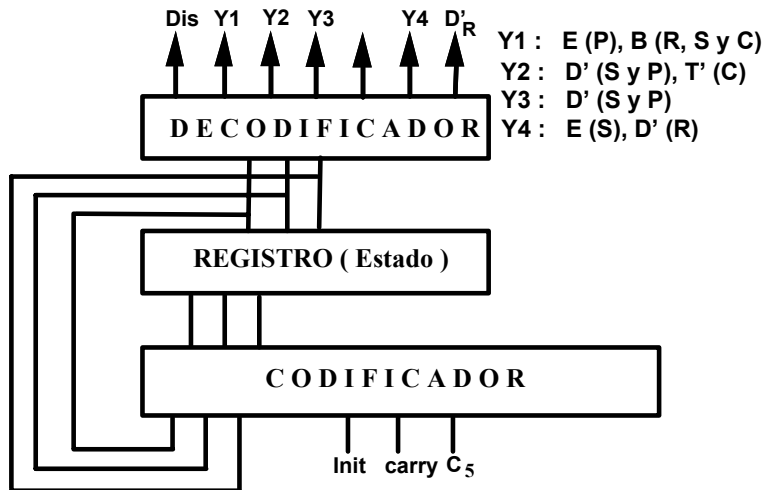
Puede ahorrarse un estado (y el consiguiente biestable) haciendo que el desplazamiento del registro **S-P** sea de dos bits en una sola vez; para ello bastará conectar la salida de cada biestable con la entrada del biestable que está dos lugares más hacia la izquierda del mismo (no del inmediatamente siguiente).

Recursos de cálculo



La entrada de control **D'** controla el desplazamiento hacia la izquierda y la salida de arrastre del restador sirve para comparar **S** (el «resto parcial») con **R01** y para almacenar en el resultado **R** un **1** (cuando $S \geq Q$) o un **0** (cuando $S < Q$).

Configuración microprogramada de la máquina de estados



24.4.3. Conversión binario a BCD

Sea **P** un número binario de 64 dígitos.

Esquema de cálculo

- recorrer el número **P** de bit en bit (comenzando por el más significativo) y sumar (en BCD) dicho bit al resultado;
- en cada desplazamiento, salvo en el último, multiplicar por dos (en BCD) el resultado.

Este esquema se deriva de la expresión del valor relativo de los dígitos de un número binario; consideremos un número más pequeño **n'** de 16 dígitos, ponmlkjihgfedcba(2):

$$\begin{aligned} n' &= p.2^{15} + o.2^{14} + n.2^{13} + m.2^{12} + l.2^{11} + k.2^{10} + j.2^9 + i.2^8 + h.2^7 + g.2^6 + \\ &\quad + f.2^5 + e.2^4 + d.2^3 + c.2^2 + b.2 + a \\ &= ((((((((((((((p.2) + o).2 + n).2 + m).2 + l).2 + k).2 + j).2 + \\ &\quad + i).2 + h).2 + g).2 + f).2 + e).2 + d).2 + c).2 + b).2 + a \end{aligned}$$

El producto por 2 puede hacerse sumando **R** consigo mismo: $R + R = 2.R$; las operaciones han de hacerse en BCD para que el resultado quede en tal codificación.

El esquema de cálculo puede ser reordenado de la siguiente manera:

- recorrer el número **P** de bit en bit (comenzando por el más significativo) haciendo lo siguiente: (multiplicar **R** por 2) sumar en BCD el resultado consigo mismo ($R + R$); y sumar también el correspondiente bit del número **P**; dicho bit puede sumarse (en la misma operación ($R + R$) a través del arrastre inicial.

Algoritmo

Inicio del algoritmo

Leer **P**; Borrar **R** y **C**;

Repetir Sumar (en BCD) $R + R$ + primer bit de **P** en **R**;

Desplazar hacia la izquierda **P**;

Incrementar **C** en una unidad;

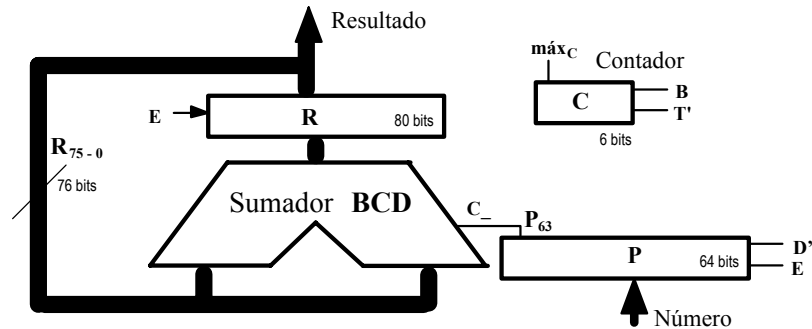
Si **C** = 111111 (64 ejecuciones de la repetición; la primera vez **C** = 0)

Fin del algoritmo.

Fin_de_Si;

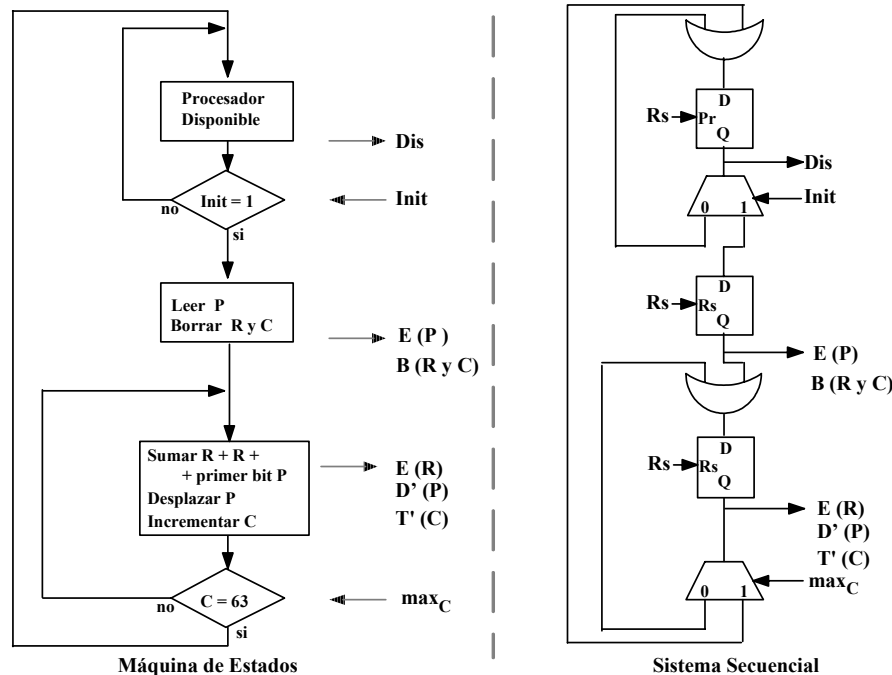
Fin_de_Repetir (volver al inicio de la repetición);

Recursos de cálculo



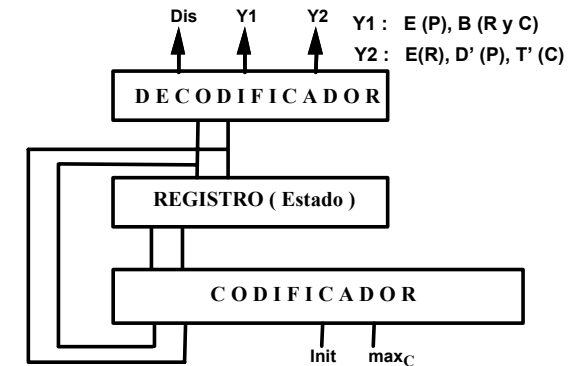
La conversión de un número binario de 64 bits a BCD ocupa un máximo de 20 dígitos BCD ($10^{20} > 2^{64}$), que equivalen a 80 bits; el número BCD (en **R**) anterior a la última suma (corresponde a 63 bits) ocupa un máximo de 19 dígitos ($10^{19} > 2^{63}$), 76 bits.

Máquina de estados y su configuración microcableada



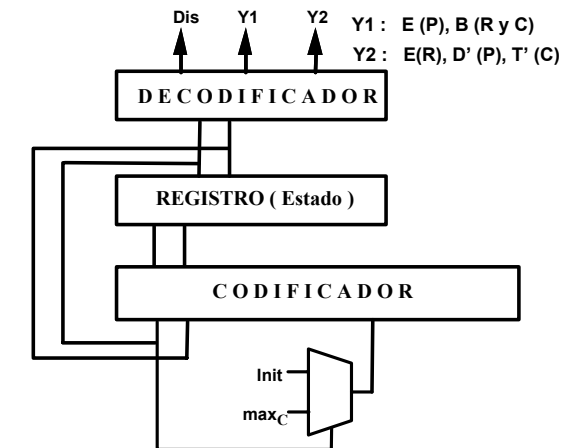
No se ha añadido ningún *estado intermedio* y, por ello, la condición $C = 63$ se verifica cuando el ciclo se ha recorrido 64 veces, ya que la acción *incrementar C* se ejecuta en forma síncrona en el mismo flanco de reloj que el salto condicionado (la primera vez $C = 0$).

Configuración microprogramada de la máquina de estados



Habida cuenta de que, en la configuración «microcableada» el tamaño de los codificadores aumenta fuertemente con el número de sus variables de entrada (se duplica cada nueva entrada), puede resultar conveniente utilizar técnicas de multiplexado de las entradas: suele suceder que no todas las entradas actúan en todos los estados, de forma que el número efectivo de ellas puede reducirse por multiplexado.

En este caso, la entrada *Init* actúa solamente con el primer estado y la entrada *maxC* lo hace con el último estado; es posible multiplexarlas mediante la variable de estado más significativa y el codificador se reduce a la mitad de tamaño.



24.4.4. Conversión BCD a binario

Sea P un número BCD de 16 dígitos (64 bits).

Esquema de cálculo

- recorrer el número P de dígito en dígito (es decir, de 4 en 4 bits) comenzando por el dígito BCD más significativo y sumar (en binario) dichos cuatro bits al resultado;
- en cada desplazamiento, salvo en el último, multiplicar por diez el resultado.

Este esquema corresponde a la expresión del valor relativo de las cifras de un número BCD; sea n de 16 dígitos, PONMLKJIHGFEDCBA₍₁₀₎, que serán 64 bits:

$$n = P \cdot 10^{15} + O \cdot 10^{14} + N \cdot 10^{13} + M \cdot 10^{12} + L \cdot 10^{11} + K \cdot 10^{10} + J \cdot 10^9 + I \cdot 10^8 + H \cdot 10^7 + G \cdot 10^6 + F \cdot 10^5 + E \cdot 10^4 + D \cdot 10^3 + C \cdot 10^2 + B \cdot 10 + A$$

$$= ((((((((((((((P \cdot 10) + O) \cdot 10 + N) \cdot 10 + M) \cdot 10 + L) \cdot 10 + K) \cdot 10 + J) \cdot 10 + I) \cdot 10 + H) \cdot 10 + G) \cdot 10 + F) \cdot 10 + E) \cdot 10 + D) \cdot 10 + C) \cdot 10 + B) \cdot 10 + A$$

El producto por 10 puede hacerse de la siguiente forma: $R \cdot 8 + R \cdot 2$ (se suma R desplazado 3 dígitos $R000$ –que equivale, en binario, a multiplicar por 8– con R desplazado 1 dígito, $R0$ –que equivale a multiplicar por 2–).

El esquema de cálculo anterior puede ser reordenado en la forma siguiente:

- recorrer el número P de cuatro en cuatro bits (comenzando por los más significativos) haciendo lo siguiente: (multiplicar R por 10) sumar el resultado R desplazado tres bits (resultado con 3 ceros añadidos al final: $R000$) con el mismo resultado desplazado un bit (resultado con 1 cero añadido detrás: $R0$); y sumar también dichos cuatro dígitos del número P .

Algoritmo

Se utiliza el contador C para contar el número de veces que se ha realizado el ciclo.

Inicio del algoritmo

Leer P ; Borrar R y C ;

Repetir

Sumar $R \&000 + R \&0 + 4$ primeros bits de P en R ;

Si $C = 1111$ (15 $\equiv$ 16 ejecuciones de la repetición; la primera vez $C = 0$)

Fin del algoritmo.

Fin_de_Si;

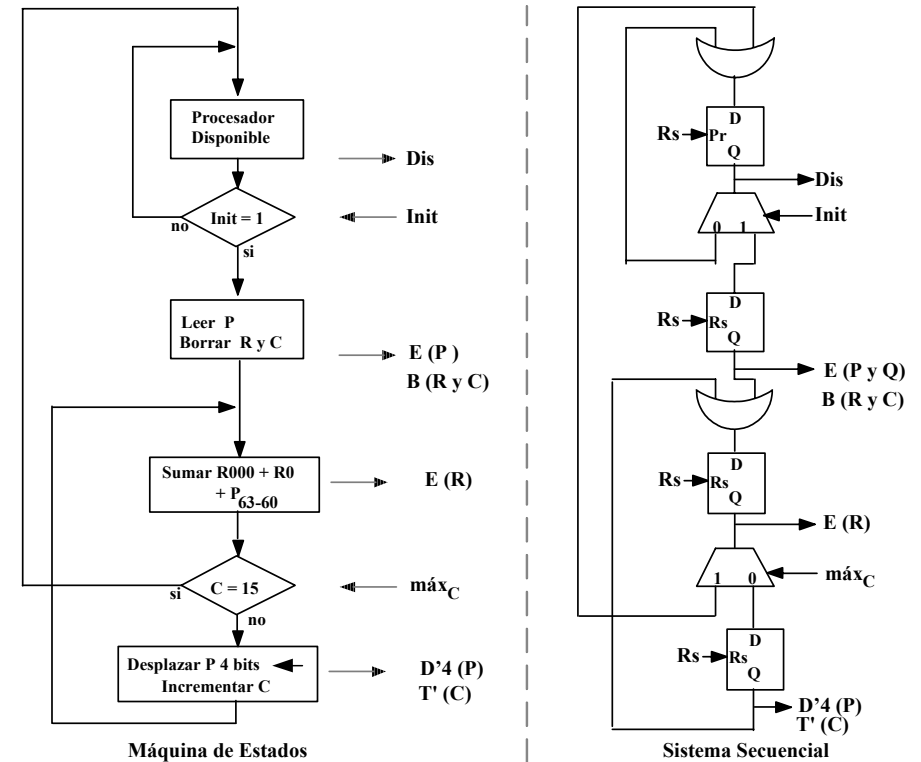
Desplazar hacia la izquierda P 4 dígitos;

Incrementar C en una unidad;

Fin_de_Repetir (volver al inicio de la repetición);

Máquina de estados y su configuración microcableada

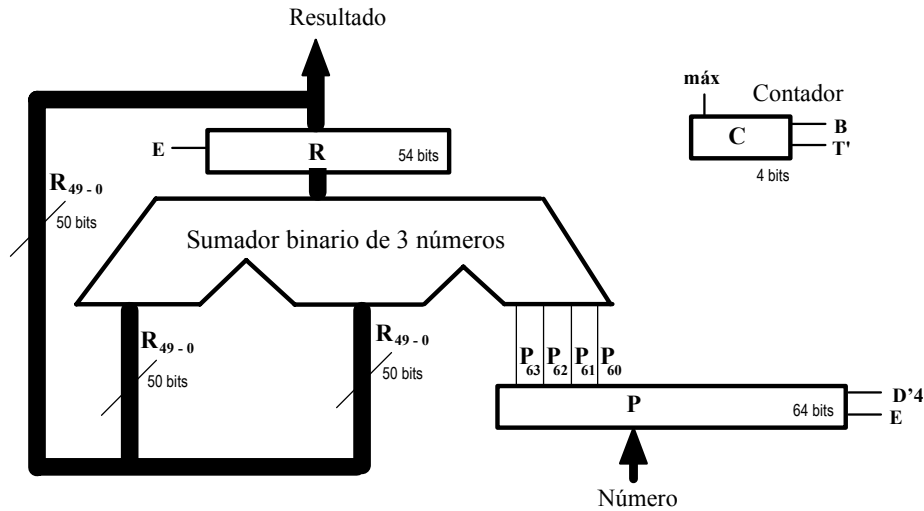
El control necesita solamente cuatro estados, conforme a la siguiente máquina algorítmica:



El desplazamiento de P cuatro dígitos hacia la izquierda puede efectuarse en un solo estado, a través de la conexión serie adecuada de los biestables del registro P ; si fuera un registro de desplazamiento normal, requeriría cuatro estados para hacerlo bit a bit. De esta forma, el cálculo es más rápido, la parte de control requiere menor número de estados y el circuito secuencial menor número de biestables.

No ha sido necesario introducir ningún *estado intermedio* pues la condición $C = 15$ no se encuentra inmediatamente detrás de la acción *incrementar* C , sino que hay un estado «en medio» que permite ejecutar el incremento.

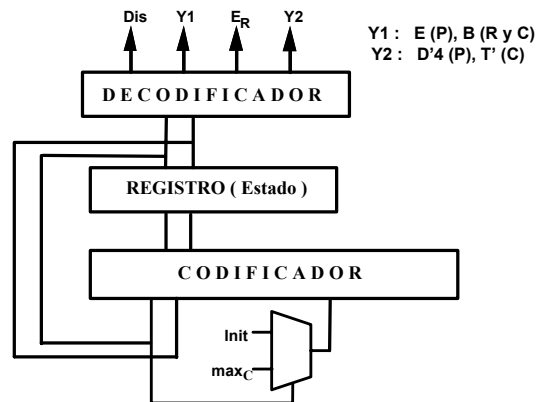
Recursos de cálculo



La conversión de un número de 16 dígitos BCD a binario ocupa un máximo de 54 bits ($2^{54} > 10^{16}$); el número binario (en **R**) anterior a la última suma (el correspondiente a 15 dígitos BCD) ocupa un máximo de 50 bits ($2^{50} > 10^{15}$).

La entrada **D'4** controla el desplazamiento cuatro bits hacia la izquierda: para ello es necesario que las conexiones serie del registro de desplazamiento sean de la salida de cada biestable con la entrada del situado cuatro lugares más a la izquierda del mismo.

Configuración microprogramada de la máquina de estados



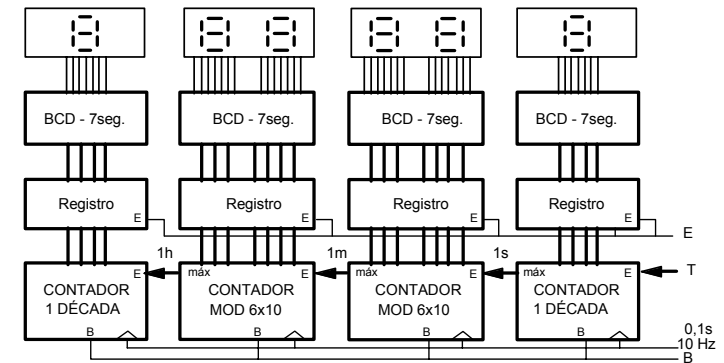
Las cinco máquinas algorítmicas descritas anteriormente se refieren a operaciones aritméticas. A fin de dejar constancia de que este método de división *parte operativa* – *parte de control* es de tipo general (y no solamente para cálculos matemáticos) se incluyen, a continuación, dos ejemplos de máquinas algorítmicas relativas a control de cronómetros.

24.4.5. Control de un cronómetro mediante pulsadores

Se desea controlar un cronómetro mediante dos pulsadores **P** y **Q** tal que el conteo de tiempo se inicie al activar **P** y se detenga al pulsar **Q** y un segundo pulso de **Q** sirva para poner a cero el cronómetro; en cambio, una vez detenido el conteo, si se pulsa **P** se reanuda el mismo. Si mientras el cronómetro está activo se pulsa **P**, el conteo prosigue pero el visualizador queda detenido en el último valor previo a dicho pulso; se vuelve a la visualización normal del conteo pulsando nuevamente **P**.

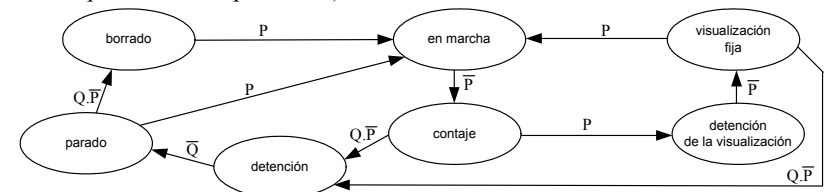
Parte operativa

Dado que es un sistema relativamente simple, resulta adecuado comenzar identificando los recursos operativos necesarios. El cronómetro requiere un contador a partir de una frecuencia de 10 Hz (período de 0,1 s), con entradas de habilitación **T** y de borrado **B**, un registro para «retener» el conteo, con entrada de habilitación **E**, y un visualizador apropiado. La siguiente figura muestra el diagrama de bloques para un cronómetro con capacidad hasta 10 horas (9 h 59 m 59 s 9 décimas).

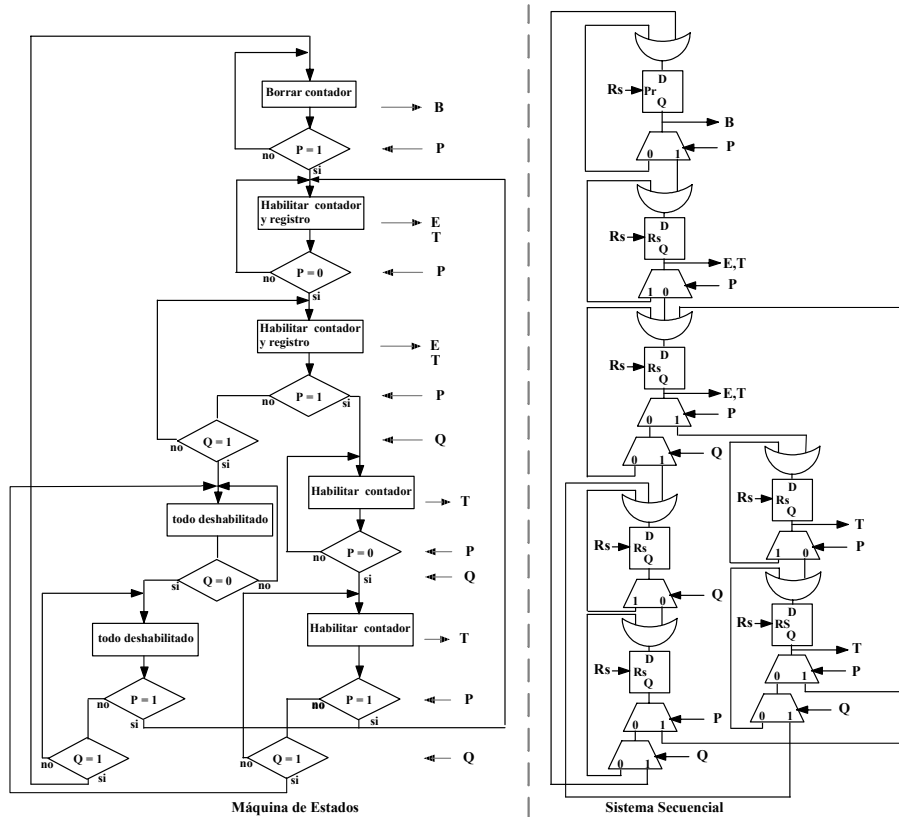


Parte de control

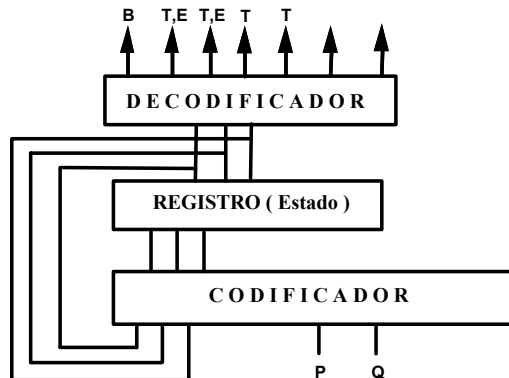
El control de este sistema responde al siguiente grafo de estados (se ha dado prioridad al pulsador **P** respecto a **Q**):



Máquina de estados y su configuración microcableada



Configuración microprogramada de la máquina de estados



24.4.5. Control de un cronómetro para medir tiempos de 8 nadadores

Interesa un cronómetro manual para carreras de natación de ocho nadadores, con capacidad de medida hasta 100 minutos y precisión de una décima de segundo; un pulsador R servirá para su puesta a cero y otro C para todo el control de medidas y visualización: para iniciar el conteo se pulsa C y, luego, se pulsará sucesivamente ocho veces más para registrar el tiempo de cada uno de los nadadores. Una vez finalizada la medida de los ocho tiempos, aparecerá en el visualizador el tiempo del primer nadador y cada vez que se pulse C el visualizador pasará a representar el tiempo del siguiente (obviamente, del octavo vuelve a pasar al primero de ellos).

Parte operativa

Al igual que en el caso anterior, el cronómetro requiere un contador con un reloj de 10 Hz (período de 0,1 s) y entradas de habilitación T y de borrado B y un visualizador; también se necesitará un registro para almacenar el tiempo de cada nadador.

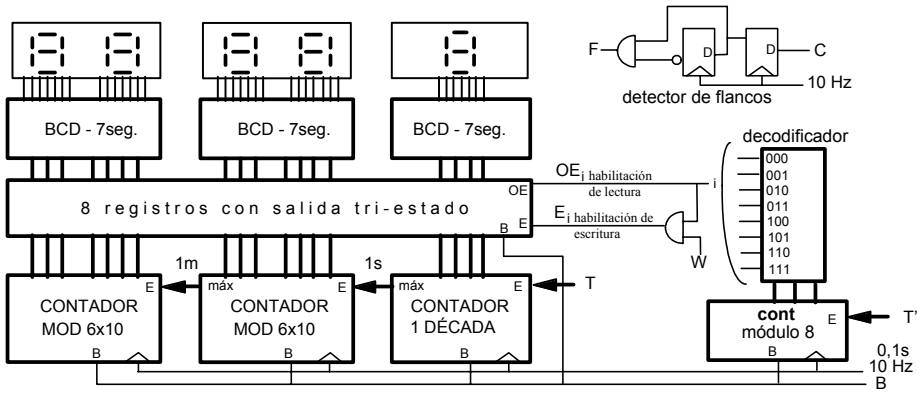
Los ocho registros han de tener sus entradas conectadas a las salidas del contador y sus salidas deben poder conectarse, en forma multiplexada, al visualizador. Para facilitar el diseño (y el dibujo del mismo) el multiplexado se realiza dotando a los registros de salida tri-estado, de forma que todos ellos estarán conectados al visualizador y, en cada momento, se activan las salidas del registro que corresponda. Cada uno de los registros tendrá una entrada de habilitación de escritura E_i (i variando de 0 a 7), otra de habilitación de lectura $O E_i$ y una entrada común de borrado B .

En principio, el grafo de estados de este sistema tendrá, al menos, 17 estados, debido a la necesidad de diferenciar los registros, en las dos situaciones: escritura (ocho estados) y lectura (otros ocho); para evitar tan amplio número de estados se puede utilizar un contador módulo 8 ($cont$), seguido de un decodificador, que seleccione, en cada momento, el registro al que le corresponde almacenar la salida del contador de tiempo (situación de escritura) o representar su valor en el visualizador (lectura).

De igual forma, resulta conveniente utilizar un detector de los flancos de subida del pulsador C , para evitar tener que detectar cada pulsado a través de una secuencia $C - \overline{C}$.

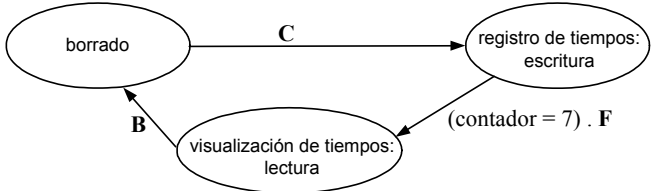
El contador auxiliar módulo 8 tendrá una entrada de habilitación para el conteo de registros T' y la entrada común de borrado B ; su habilitación T' se producirá en ambas situaciones (escritura y lectura) pero solamente cuando se detecta flanco F de la señal C : $T' = (escritura + lectura) \cdot F$.

De manera que la parte operativa ampliada con el contador-selector de registro y con el detector de flancos quedará en la forma representada en la figura siguiente:



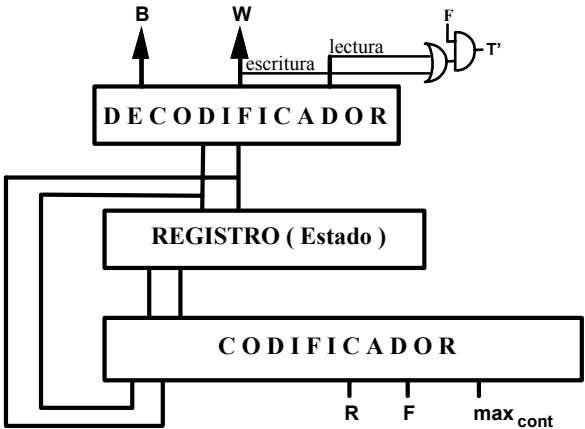
Parte de control

Un posible grafo de estados para representar el control de este sistema es el siguiente:

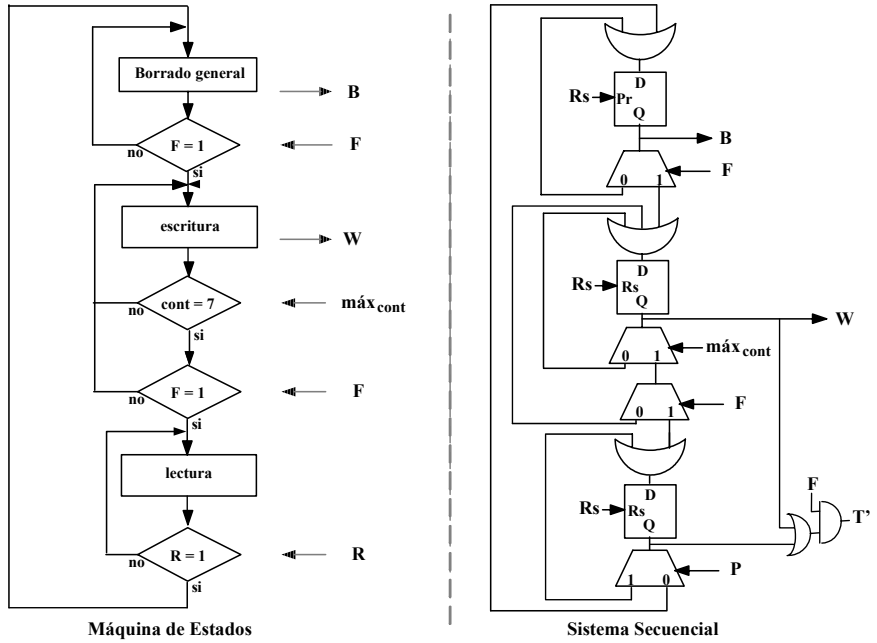


La escritura corresponde al registro de tiempos de los ocho nadadores y la lectura a la posterior visualización cíclica de los mismos. El contador auxiliar, que recorre los ocho registros, debe «contar» tanto en el estado de lectura como en el de escritura cuando se detecta un flanco del pulsador C.

Configuración microprogramada de la máquina de estados

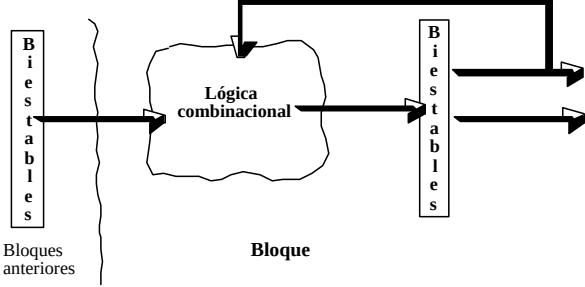


Máquina de estados y su configuración microcableada



24.5. Dividir en partes los tiempos de retraso: segmentación

Los bloques o módulos constitutivos que conforman un sistema síncrono están configurados por una parte combinacional situada entre dos registros (o conjuntos de biestables), según la figura siguiente (véase el apartado 15.3, segundo volumen, páginas 111-114):



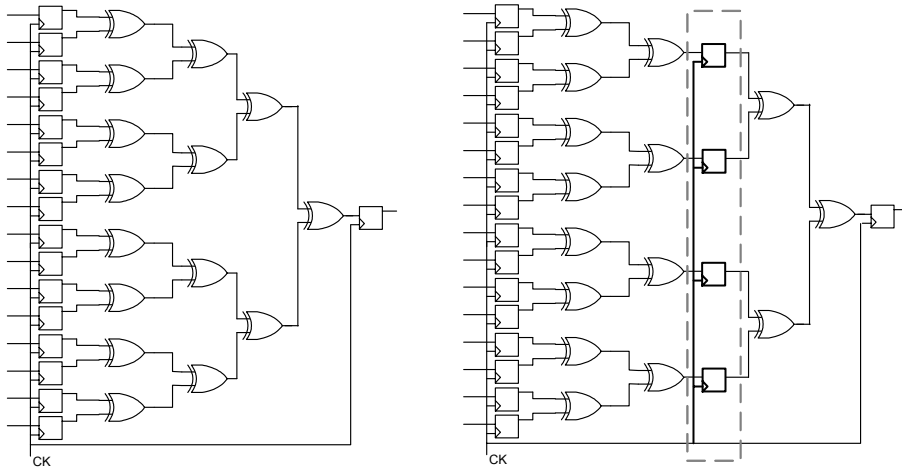
La velocidad de trabajo de uno de estos módulos queda limitada por el tiempo de propagación de los diversos «caminos» entre biestables, según la expresión:

$$T_{CK} > t_p (\text{FF1}) + \sum t'_p (\text{parte combinacional}) + t_s (\text{FF2})$$

donde t_p es el tiempo de propagación del primer biestable y $\sum t'_p$ la suma de tiempos de propagación en la parte combinacional que atraviesa el camino, y t_s el tiempo de anticipación que requiere el segundo biestable. Esta desigualdad impone un límite a la frecuencia máxima de reloj en que el módulo puede funcionar correctamente (y, con ello, limita la frecuencia de reloj del sistema digital).

Si la parte combinacional es amplia (en el sentido de muchas puertas sucesivas), la suma de retrasos en ella $\sum t'_p$ es apreciable y requiere un período de reloj largo, limitando fuertemente la velocidad de trabajo del sistema digital. Un recurso de diseño que permite aumentar dicha velocidad consiste en dividir la parte combinacional en segmentos (en *rodajas*) insertando registros intermedios; esta técnica, conocida con la denominación de *pipeline* o segmentación, es aplicable solamente a bloques sin realimentación.

Consideremos un primer ejemplo de aplicación de la segmentación a un bloque muy simple: se trata de calcular la paridad de números de 16 dígitos binarios, a través de una sucesión de puertas "o-exclusiva", según las figuras siguientes:



Supongamos que los tiempos de anticipación y de propagación de los biestables son iguales entre sí e iguales, asimismo, al tiempo de propagación de cada una de las puertas "o-exclusiva". En la figura de la izquierda, el período de reloj deberá ser mayor que

$$t_p + 4.t_p + t_s = 6.t_p; \quad f_{CK} < 1/(6.t_p);$$

en la figura de la derecha, se ha insertado un registro en medio de la parte combinacional, de forma que el período de reloj deberá ser, en este caso, mayor que

$$t_p + 2.t_p + t_s = 4.t_p; \quad f_{CK} < 1/(4.t_p);$$

se ha conseguido una ganancia en la frecuencia de reloj según un factor 1,5 ($4 = 6/1,5$).

Ha de tenerse en cuenta que, para obtener el resultado (la paridad del número de 16 bits), el segundo circuito requiere un ciclo más de reloj. La frecuencia de reloj puede ser mayor pero el cálculo de la paridad del primer número que llega al registro inicial tarda dos ciclos (en lugar de uno solo); en cambio, los resultados de paridad de los números siguientes (supuesto que cada ciclo el registro inicial reciba un nuevo número) se obtienen en ciclos de reloj sucesivos. Para una serie de 100 números binarios seguidos son precisos 101 ciclos de reloj: dos para el primer número y uno más para cada uno de los siguientes.

Este aumento de un ciclo de reloj, debido al registro que se ha añadido al dividir la parte combinacional en dos segmentos (dos «rodajas»), ha de ser tenido en cuenta en el diseño del resto del sistema digital que utilice el cálculo de la paridad.

Pueden incluirse tres registros intermedios (en lugar de uno solo), de forma que cada segmento sea de profundidad igual a una puerta, es decir, esté formado por un grupo de puertas "o-exclusiva" en paralelo y, entonces, el período de reloj deberá ser mayor que

$$t_p + 1.t_p + t_s = 3.t_p; \quad f_{CK} < 1/(3.t_p);$$

lo cual supone una ganancia en la frecuencia de reloj según un factor 2 ($3 = 6/2$), en relación con el circuito inicial (sin ningún registro intermedio). En este caso, se añaden 2 nuevos ciclos de reloj para obtener el resultado: la paridad del primer número binario requiere 4 ciclos de reloj y, luego, cada ciclo de reloj suministra una nueva paridad; para una serie de 100 números seguidos son precisos 103 ciclos de reloj.

La técnica de segmentación se aplica a bloques síncronos cuya parte combinacional presenta altos tiempos de propagación y tal aplicación es efectiva cuando dichos bloques actúan repetitivamente con vectores de entrada sucesivos; en tal caso, la reducción del tiempo global es muy significativa. En el ejemplo considerado, para el cálculo de paridad de series de 100 números seguidos, resultan los siguientes tiempos mínimos:

- sin segmentación: $T_{CK} > 6.t_p$ tiempo total: $100.T_{CK} \sim 600.t_p$
- con un registro intermedio: $T_{CK} > 4.t_p$ tiempo total: $101.T_{CK} \sim 404.t_p$
- con segmentación total: $T_{CK} > 3.t_p$ tiempo total: $103.T_{CK} \sim 309.t_p$.

Un segundo ejemplo, algo más complejo, servirá para poner de manifiesto la potencia y, también, los requisitos de la segmentación.

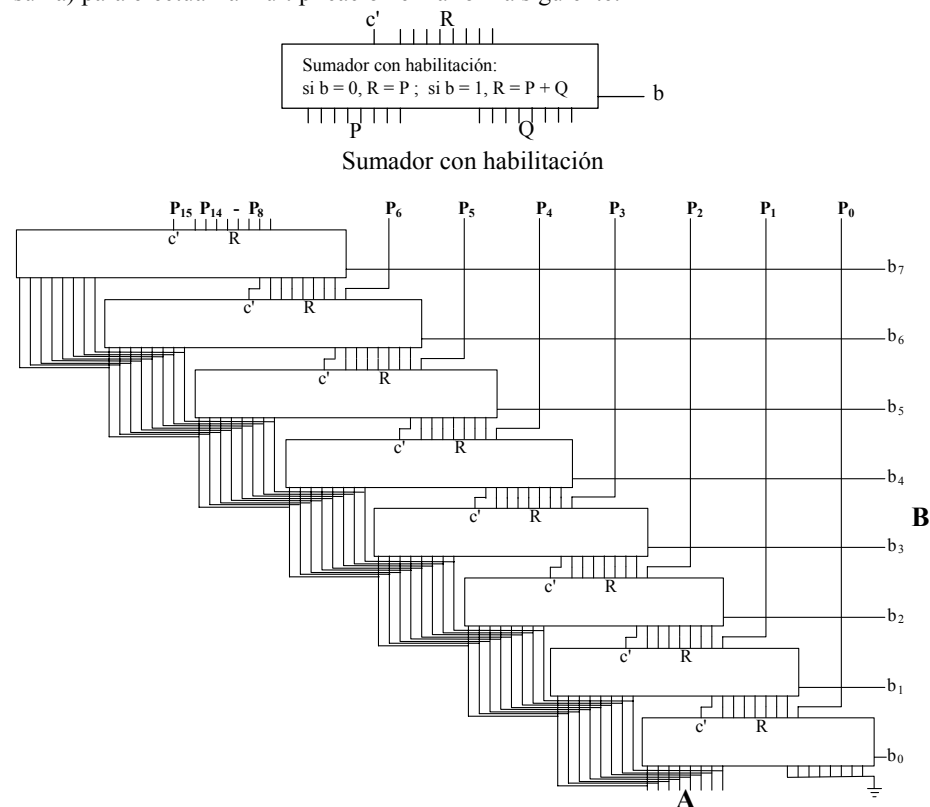
Un multiplicador de números de ocho dígitos, $A \times B$, puede construirse mediante una máquina algorítmica, en la forma descrita en el apartado anterior (páginas 107 a 112), pero esta configuración secuencial resulta relativamente lenta: requiere 26 ciclos de reloj.

Como referencia cuantitativa supongamos que los tiempos de anticipación y de propagación de los biestables son iguales entre sí y que el tiempo de propagación de un sumador de ocho dígitos es unas tres veces mayor (el sumador estará formado por ocho celdas sumadoras sucesivas, cuyo arrastre ha de propagarse a través de ellas); en tal caso el tiempo de ciclo de reloj deberá ser mayor que

$$t_p + 3.t_p + t_s = 5.t_p; \quad f_{CK} < 1/(5.t_p);$$

y el tiempo de cálculo de la multiplicación será (en el peor de los casos): $26.t_{CK} \sim 130.t_p$.

Otra posibilidad es utilizar ocho sumadores (con entrada de habilitación respecto a la suma) para efectuar la multiplicación en la forma siguiente:



Multiplicador de $A \times B$ (de 8 dígitos)

El tiempo de propagación de este bloque será del orden de $8.4.t_p$ y el período de reloj (de este multiplicador, situado entre sendos registros) deberá ser mayor que

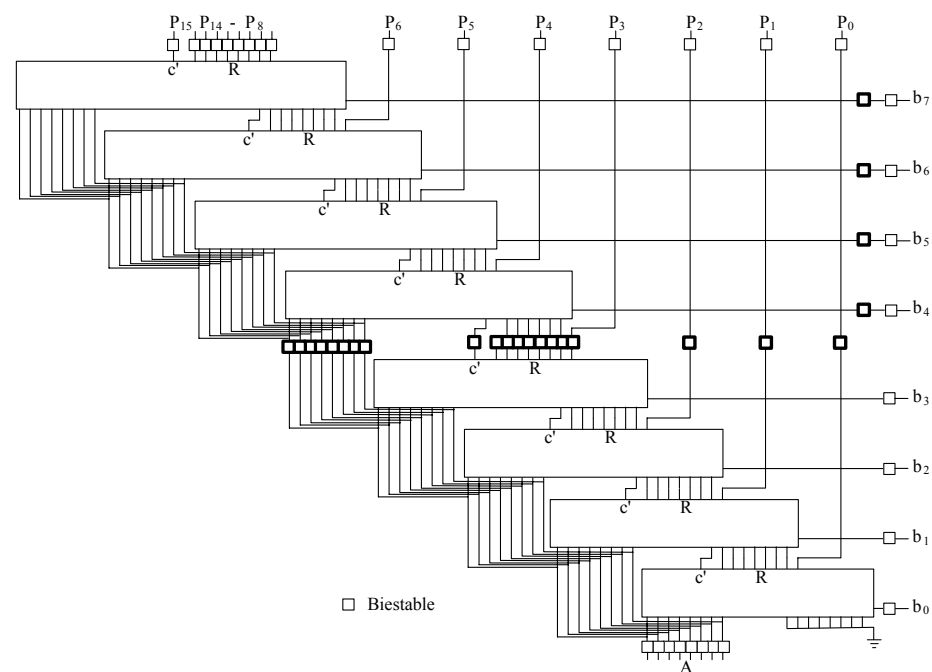
$$t_p + 8.3.t_p + t_s = 26.t_p; \quad f_{CK} < 1/(26.t_p)$$

con un tiempo de cálculo (1 ciclo: $26.t_p$) cinco veces más pequeño que el correspondiente al anterior multiplicador ($130.t_p$) configurado como máquina algorítmica.

En el caso de segmentar en dos «rodajas», es decir, de insertar un registro en mitad del multiplicador (ver figura siguiente), el período de reloj deberá ser mayor que

$$t_p + 4.3.t_p + t_s = 14.t_p; \quad f_{CK} < 1/(14.t_p);$$

una frecuencia de reloj cercana al doble de la anterior.



Multiplicador de 8 dígitos segmentado en dos partes

En la figura anterior, además de los biestables propios de la segmentación, se han incluido los registros de entrada de los dos números A y B y de salida P .

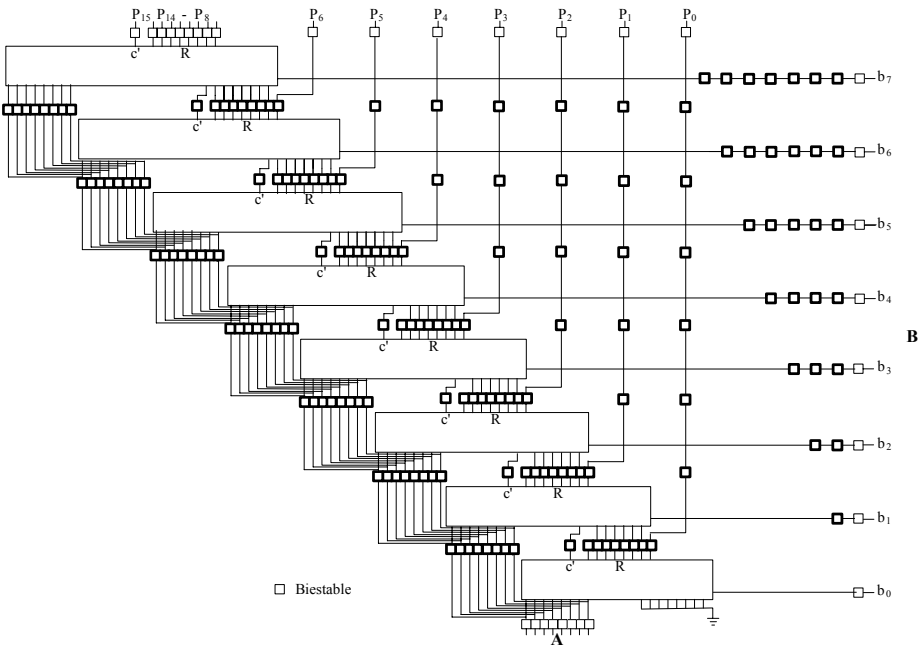
Obsérvese que es necesario añadir biestables, tanto en las salidas del cuarto sumador, como en las salidas $P_3 - P_0$ de los anteriores para retrasarlas un ciclo, acompasándolas a las de los sumadores siguientes; también, hay que añadirlos en el número A y en los dígitos $b_7 - b_4$, en cuanto a entradas a dichos sumadores. En total, han sido precisos 24 biestables para segmentar en dos partes el multiplicador.

De nuevo cabe resaltar que, para obtener el resultado de la multiplicación en este circuito segmentado, se requiere un ciclo más de reloj; la frecuencia puede duplicarse respecto al circuito anterior pero el cálculo del producto de los dos primeros números tarda dos ciclos; en cambio, los productos de las parejas de números siguientes (supuesto que en cada ciclo los registros iniciales reciban nuevos operandos) se obtienen en ciclos de reloj sucesivos. Para una serie de 100 números binarios seguidos se necesitan 101 ciclos de reloj: dos para el primer producto y uno más para cada uno de los siguientes.

Una segmentación total (es decir, cada sumador un segmento o «rodaja») requiere 168 biestables de segmentación (ver figura siguiente) con un periodo de reloj mayor que

$$t_p + 1.3.t'_p + t_s = 5.t_p; \quad f_{CK} < 1/(5.t_p);$$

frecuencia unas tres veces más rápida que la del circuito anterior y cinco veces mayor que la del circuito sin segmentación.



Multiplicador de 8 dígitos segmentado en 8 partes

En este caso de segmentación completa, se necesitan ocho ciclos de reloj para obtener el resultado: el producto de la primera pareja de números binarios requiere tales ocho ciclos y, luego, en cada ciclo de reloj se obtiene un nuevo producto; para una serie de 100 números seguidos son precisos 107 ciclos de reloj.

Comparando las cuatro posibilidades descritas de realizar el producto, para series de parejas de 100 números sucesivos:

- máquina algorítmica: $TCK > 5.t_p$ 26 ciclos de reloj para cada producto
tiempo total: $100.26.TCK > 13000.t_p$
- sin segmentación: $TCK > 26.t_p$ tiempo total: $100.TCK > 2600.t_p$
- con un registro intermedio: $TCK > 14.t_p$ tiempo total: $101.14.TCK > 1414.t_p$
- con segmentación total: $TCK > 5.t_p$ tiempo total: $107.5.TCK > 535.t_p$

Un bloque síncrono (una parte combinacional situada entre dos registros o conjuntos de biestables), *sin realimentación* (sus biestables de salida no se realimentan sobre el propio bloque), puede segmentarse, es decir, dividirse en varios bloques síncronos sucesivos, de menor profundidad; lo cual permite aumentar la velocidad de trabajo del sistema, su frecuencia de reloj, a costa de insertar un amplio número de biestables.

Ciertamente el mayor número de biestables incrementa en forma importante el área de silicio, en el caso de ASICs, o el número de celdas necesarias, caso de FPGAs. Pudiera parecer que también aumenta el consumo del circuito, al existir más biestables conmutando en cada flanco de reloj; sin embargo, a igualdad de frecuencia, suele suceder que la segmentación disminuya el consumo, pues la división en segmentos de la parte combinacional limita la propagación de los espurios producidos en la conmutación, que son responsables de buena parte del consumo cuando hay muchas puertas seguidas.

La segmentación (*pipeline*) es un recurso de división en partes utilizado para conseguir mayores velocidades en procesamientos que actúan sobre series de datos sucesivos; en el diseño del sistema global, ha de tenerse en cuenta que cada división (cada nuevo segmento) introduce un registro y un ciclo adicional de reloj.

Lo que hace la segmentación es aplicar una concurrencia de tareas, poniendo a operar en paralelo a los diversos segmentos, sin tener que esperar, cada uno de ellos, a que los anteriores hayan finalizado su cálculo.

Esta concurrencia se aplica de formas diversas; por ejemplo, en los procesadores que actúan bajo programa la ejecución de instrucciones, mediante una segmentación del tipo siguiente, permite el tratamiento simultáneo de cinco instrucciones de operación sobre dos operandos:

búsqueda de instrucción	lectura de operando A	lectura de operando B	ejecución de operación	escritura de resultado
instrucción n + 4	instrucción n + 3	instrucción n + 2	instrucción n + 1	instrucción n.

25 TEST DE CIRCUITOS INTEGRADOS

- 25.1. El test como última etapa del proceso de fabricación
- 25.2. Modelo de fallos para el test
- 25.3. Análisis de fallos y simulación de fallo
- 25.4. Test de circuitos secuenciales
- 25.5. Test de placas y de sistemas completos
- 25.6. Test interno

El test es el procedimiento que se aplica a cada circuito integrado, al culminar el proceso de fabricación, para comprobar que dicho ejemplar (cada chip individual) se ha fabricado bien, es decir, para verificar que el resultado de la fabricación (en ese integrado particular) es correcto; el resultado del test es binario:

Sí : el ejemplar es correcto, se encapsula y da lugar a un circuito integrado disponible para ser utilizado.

No : tal ejemplar ha resultado «mal fabricado» y se desecha.

En el caso de un circuito integrado muy pequeño, el test puede hacerse probando todas las situaciones del mismo: todos los vectores de entrada posibles, junto con todos los estados si el circuito es secuencial. Pero en circuitos de tipo medio o grandes este tipo de comprobación no es factible pues se necesitarían años para ella (un simple sumador de números de 26 dígitos precisa de más de un siglo, a una velocidad de 10^6 vectores/seg).

Y, sin embargo, es necesario comprobar que cada una de las celdas (cada puerta, cada biestable) se ha fabricado bien. Para ello se han desarrollado «modelos de test» que permiten identificar los fallos posibles, encontrar los vectores que detectan dichos fallos y conocer qué fallos detecta cada vector de test.

En el caso de circuitos secuenciales, la extraordinaria complejidad que implica controlar y observar el estado del circuito se resuelve añadiendo (en cada biestable) circuitería específica para el test; creando un «camino de exploración» (scan path) que permita establecer, directamente sobre los biestables, el estado que se desee y observar directamente el estado anterior.

Esta técnica de «scan path» ha resultado tan simple y útil que ha sido extendida al test de placas circuitales (test de conjuntos de circuitos integrados y de las pistas de interconexión de los mismos) y al test de sistemas digitales completos.

Asimismo, la incorporación de circuitería añadida para el test permite la realización del mismo en el interior del propio circuito integrado: test interno. Sencillos algoritmos de test de memorias (BIST), registros realimentados para generación aleatoria de vectores de test (LFSR), compactación en una sola palabra de los resultados del test (FIRMA) y aprovechamiento conjunto de LFSR y FIRMA para ejecución del test, son mecanismos de test interno que requieren añadidos circuitales relativamente pequeños. Ello simplifica el proceso de comprobación de la fabricación y, además, ofrece la posibilidad de verificar el integrado posteriormente, en el sistema digital del que forma parte.

25.1. El test como última etapa del proceso de fabricación

El test de un circuito integrado es la comprobación, posterior a la fabricación del mismo, que garantiza que el circuito se ha fabricado bien; tiene que garantizar que todas las celdas que configuran el circuito han sido integradas correctamente de forma que el resultado físico coincide con el esquema circuital resultante del diseño.

El test no puede plantearse desde la funcionalidad del circuito sino desde su configuración física; no es una simulación de su comportamiento respecto a las especificaciones que se pretenden, sino una verificación de su realidad física interna.

Téngase en cuenta que el resultado de la fabricación para cada circuito integrado individual (para cada uno de los muchos chips que se fabrican en una sola oblea) es incierto; basta una microscópica partícula de polvo, una irregularidad en la superficie de la red cristalina de la oblea, un fallo en el proceso litográfico de reproducción de las máscaras sobre la oblea, etc.,... para que un transistor no se construya y el circuito resultante sea erróneo.

El test se realiza sobre la propia oblea, previamente a la etapa de corte y separación de los numerosos chips que contiene, mediante costosas máquinas de test que activan los terminales (PADs) de entrada del circuito integrado, con un conjunto de vectores de entrada sucesivos (secuencia de vectores de test), y observan los terminales de salida, comparando los vectores de salida con la secuencia de resultados correctos. De forma que cada vector de test ha de incluir ambas partes: entradas y salidas:

$$\text{Vector de test} = \{ \text{vector de entrada, vector de salida} \}$$

Los vectores de test han de permitir «controlar», desde las entradas del circuito integrado, cada uno de los «nudos booleanos» internos del circuito (para situarlos a valor 0 y a valor 1) y «observar», desde las salidas del integrado, el estado booleano de cada uno de dichos nudos; de esta forma, podremos comprobar que cada celda básica está en su sitio y actúa correctamente.

El test se ejecuta posicionando automáticamente un conjunto de «puntas» o sondas de la máquina de test sobre los «puntos de soldadura» (PADs) terminales del chip que, posteriormente, se conectarán a los terminales (pins) del encapsulado. Se dispone para el test solamente de los terminales (entrada/salida) del circuito integrado, sin poder actuar sobre nudos internos.

El test se aplica «globalmente» sobre el circuito integrado completo; aun más, sobre el circuito «encapsulado», es decir, siendo accesibles solamente los terminales (PADs) del circuito: se dispone únicamente de las entradas físicas del circuito (PADs de entrada), para actuar sobre el mismo (vectores de entrada), y de sus salidas físicas (PADs de salida), para observarlo.

Además, es preciso minimizar el tiempo de utilización de la máquina de test, debido al elevado coste de tales equipos.

Por ello, el test presenta los siguientes inconvenientes:

1. Hay que comprobar todos y cada uno de los circuitos integrados: mientras que el proceso de fabricación opera sobre múltiples obleas a la vez y cada oblea contiene un amplio número de circuitos (es decir, centenares o miles de *chips* en cada «hornada»), el test ha de aplicarse individualmente a cada uno de ellos.
2. Un circuito integrado tiene muchos componentes (transistores) configurando una red con muchos nudos: todos ellos han de ser comprobados, pero el número de nudos accesible para el test es muy reducido, se dispone solamente de las entradas y salidas globales del circuito integrado.
3. La secuencia de vectores de test no puede ser excesivamente larga porque ello supondría un tiempo de máquina de test muy prolongado y repercutiría gravemente en el coste del circuito.

En suma, muchos circuitos integrados y muchos nudos (en cada uno de ellos) a comprobar y, en cambio, pocos nudos accesibles y relativamente pocos vectores de test a utilizar. Como respuesta a estos inconvenientes, hay que desarrollar metodologías de test y de diseño orientado al test que permitan superarlos.

Cuantifiquemos un poco la magnitud y complejidad del test:

¿Cómo estar seguros de que un circuito integrado se ha fabricado bien? Una primera respuesta podría ser: probando todas y cada una de sus posibilidades.

En el caso de un circuito combinacional, el número de posibilidades es sencillo: si cuenta con m entradas, las situaciones posibles son 2^m (que conforman su «tabla de verdad»). Por ejemplo, un sumador de números de 16 bits cuenta con 32 entradas que suponen 2^{32} situaciones posibles, es decir, unos 4.300 millones de vectores de test que necesitan más de 1 hora, a una velocidad de un millón de vectores por segundo. Realmente se consiguen mayores velocidades de test, $\sim 10^7$ vectores/s, que suponen unos 7 minutos de test, que sigue siendo un tiempo alto para un simple sumador de 16 dígitos; un sumador de 32 bits, 2^{64} posibilidades, necesita cerca de 600 siglos de test (a 10^7 v/s).

En el caso de circuitos secuenciales, la magnitud del problema crece aún mucho más, por dos razones que se suman recíprocamente:

- los estados añaden posibilidades en progresión geométrica:

m entradas n biestables = 2^n estados $2^m \times 2^n$ posibilidades = 2^{m+n}

Por ejemplo: con 16 entradas y 32 biestables, 2^{48} posibilidades, unos 100 billones, que, a 10^7 vectores/s, suponen más de 4 meses dedicados al test.

- el orden de los vectores de entrada influye decisivamente:

Para conseguir cada estado en el conjunto de biestables, es necesaria una secuencia de vectores de entrada adecuada. Para verificar cada posibilidad no basta un solo vector de entrada sino que hay que conseguir llevar a los biestables al estado deseado, mediante una secuencia de vectores de entrada; esto no sólo aumenta el número de vectores de test, sino que requiere encontrar tal secuencia, lo cual no es obvio.

La magnitud del problema del test (puesta de manifiesto en los ejemplos anteriores, que, también, ponen en evidencia el mayor grado de complejidad que presentan los sistemas secuenciales), hace necesaria la utilización de *modelos de fallos* que permitan:

- a) determinar con precisión los fallos posibles y contar su número,
- b) encontrar vectores de test para cada fallo,
- c) conocer qué fallos detecta cada vector de test,
- y d) calcular la cobertura de fallos, es decir, el conjunto de fallos que pueden ser detectados por un conjunto dado de vectores de test.

En la práctica, se utiliza el modelo de fallos por bloqueo *stuck-at* cuya justificación y descripción se detallan en el apartado siguiente.

Ahora bien, en los razonamientos anteriores nos hemos referido solamente a uno de los aspectos del test de circuitos integrados digitales: el aspecto *funcional-booleano*: comprobar que cada celda básica está en su sitio (es decir, en sus salidas realiza la función booleana correspondiente) y adecuadamente conectada (ya que el resultado de dicha función se detecta en el nudo adecuado, tras haber actuado a través de los nudos que corresponden a sus entradas).

Existe un segundo aspecto que también debería ser objeto de test: el comportamiento *temporal*, referido a los tiempos de propagación (y de anticipación y mantenimiento), comprobando que tales tiempos coinciden con (o no son peores que) los estimados.

En la práctica, el test temporal no se realiza en forma individualizada sobre cada *chip* sino que se supone que si un transistor ha sido fabricado en su sitio sus características coinciden con las previstas.

Para tener una cierta garantía de ello suele realizarse un test global de fabricación: en la oblea se incluyen, aparte de los múltiples circuitos integrados iguales (*chips*), una serie de componentes electrónicos de tipo simple (resistencias, capacidades, uniones, PN, transistores individuales,...) cuyas características se miden (generalmente en unas pocas obleas de cada «hornada») para comprobar que el proceso de fabricación ha sido correcto, es decir, ha construido componentes de las características eléctricas previstas. Es pues un «test de calidad» del proceso de fabricación, análogo al que se realiza en otros tipos de procesos de fabricación, tomando varias muestras al azar y comprobando en detalle sus características.

25.2. Modelo de fallos para el test

Definir los fallos posibles en un circuito integrado es tarea ardua y compleja; las posibilidades son enormes:

- en principio los circuitos integrados digitales en tecnologías MOS están compuestos por transistores MOS conectados entre sí;
- en cuanto a los transistores puede haber fallos por quedar cortocircuitadas dos de sus cuatro regiones constitutivas: fuente, drenaje, puerta y sustrato, o por quedar en circuito abierto (o no haberse formado, lo cual es lo mismo) la fuente, el drenaje o la puerta;
- respecto a las conexiones puede darse el caso de pistas rotas, pistas interconectadas entre sí (puenteadas), etc;
- ...

Para poder manejar tan gran número de posibilidades es precisa una fuerte simplificación, se requiere un modelo simplificado de fallos.

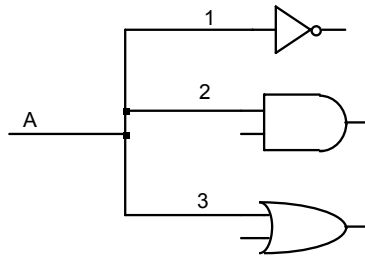
Un modelo de fallos muy simple pero que ha dado excelentes resultados y es el que se utiliza actualmente en la fabricación industrial es el modelo de «fallos por bloqueo», el cual admite solamente dos posibilidades de fallo: *stuck-at-0* (bloqueo a **0**) y *stuck-at-1* (bloqueo a **1**) y supone las siguientes hipótesis:

1. Se consideran como elementos componentes del circuito las celdas básicas («celdas estándar» con las que se efectúa el diseño: puertas, biestables,...) y se utiliza la *net-list* (lista de conexiones del circuito) configurada por dichas celdas básicas.

2. Se consideran como «nudos» todas y cada una de las entradas a las celdas básicas y, además, las salidas del circuito.

Por ello, estos «nudos a efectos de test» no coinciden exactamente con los «nudos topológicos» del circuito, sino que cada nudo eléctrico ha de ser «desplegado» en varios, tantos como entradas a las que está conectado. De forma que el número de nudos a efectos de test es muy superior al número de nudos de la *net-list* SPICE.

Por ejemplo, el nudo A de la figura siguiente ha de ser contabilizado como tres nudos (1, 2 y 3), ya que conecta con 3 entradas.



3. Para cada nudo se consideran como fallos posibles solamente dos:

- que el nudo esté bloqueado a **0** *STUCK-AT-0*
 - que el nudo esté bloqueado a **1** *STUCK-AT-1*
- ("stuck" es el participio del verbo "to stick": clavar, pegar, adherir ...).

4. Cada nudo ha de ser comprobado dos veces, para asegurar que no se encuentra bloqueado a **0** ni a **1**.

5. El no-bloqueo (la puesta a **1** y a **0**) de cada entrada (nudo interior) ha de ser detectado pasando a través de la celda básica a la que corresponde dicha entrada, es decir, para cada celda básica ha de verificarse el paso de un **1** y el de un **0** desde cada una de sus entradas; además, ha de verificarse la posibilidad de **1** y de **0** para cada salida del circuito.

6. Se supone que en el circuito no se presentan múltiples fallos que se compensen entre sí: *hipótesis de fallo único en el circuito*; en realidad lo que se supone es que si un vector de test es capaz de detectar un fallo individual, dicho vector detecta siempre dicho fallo, es decir, no se da el caso de que el fallo quede enmascarado (compensado) por otros fallos que tengan lugar a la vez.

Las simplificaciones que introduce este modelo (en particular, las hipótesis de «solo dos tipos de fallos» y de «fallo único en el circuito») son muy fuertes y es obvio que no asegura, en principio, la detección de todas las posibilidades de fallo. Sin embargo, el modelo se ha mostrado muy eficaz en su utilización práctica y es, hoy por hoy, el único que se emplea a nivel industrial; ha demostrado que detecta un porcentaje muy alto (aunque no todos) de los fallos posibles y se ha comprobado que, efectivamente, sirve para asegurar que un circuito integrado ha sido fabricado bien.

Habida cuenta que cada nudo se comprueba dos veces (bloqueos a **0** y a **1**), que se consideran como nudos independientes cada una de las entradas a las celdas básicas y por cada una de dichas celdas se hace pasar un **0** y un **1** desde cada una de sus entradas (detectando específicamente dicho paso),... es sumamente probable que un test completo conforme al modelo *stuck-at* detecte cualquier fallo resultante del proceso de fabricación.

El modelo *stuck-at* es capaz de detectar los siguientes fallos:

- un cortocircuito entre dos terminales de un transistor MOS generalmente equivale a un *stuck-at*;
- un transistor MOS en circuito abierto en muchas ocasiones será detectado como *stuck-at*; en este caso, el orden de los vectores de test influye en la detección del fallo pues generalmente la salida de la correspondiente puerta lógica depende del anterior valor de salida (que se conserva capacitivamente) [Se ha desarrollado un modelo de fallos *stuck-open* para tratar los correspondientes a circuitos abiertos, basado en detectar las transiciones (de **0** a **1** y de **1** a **0**) pero requiere un número de vectores mucho mayor que el *stuck-at* porque es preciso detectar cada nudo en valor **0** y de detectar su paso a **1** y viceversa.];
- lo mismo podría decirse de pistas cortadas;

- la interconexión entre pistas (*bridging*: puentes) no es detectada directamente, pero habida cuenta que ha de verificarse el no-bloqueo a **0** y a **1** de cada una de ellas (al menos 4 pruebas) y que dicha verificación se hace varias veces en el caso de estar unidas a más de una entrada es muy probable que tal fallo se detecte (existen también modelos de fallos por puentes);
- algo parecido puede decirse de los fallos múltiples: en principio se encuentran excluidos del modelo pero es sumamente probable que alguno de ellos sea detectado, en alguna de las múltiples veces que los vectores de test observan los «nudos» afectados por tales fallos (téngase en cuenta que carece de interés conocer si hay más de un fallo: basta uno de ellos para que el circuito sea erróneo y deba ser desechado).

La detección de un fallo presenta dos aspectos complementarios:

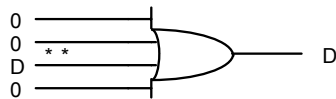
a) el CONTROL del correspondiente nudo desde las entradas del circuito integrado para situarlo en el valor booleano adecuado (el que demuestra que el nudo no está bloqueado, es decir, el contrario al bloqueo que se investiga)

b) la OBSERVACIÓN del valor booleano presente en dicho nudo desde las salidas del circuito, pasando a través de la celda básica a la que pertenece el nudo como entrada de la misma.

En la observación hay que pasar a través de las celdas básicas «hacia adelante» hasta llegar a las salidas; en el control hay que pasar también a través de las celdas básicas pero «hacia atrás» (de las salidas hacia las entradas) hasta llegar a las entradas del circuito.

Para realizar este paso «a través» de las celdas básicas se utiliza un «modelo de fallos» de cada celda que permite razonar cuál debe ser el valor booleano de todas sus entradas para atravesar la celda y, también, qué bloqueos detecta en cada caso:

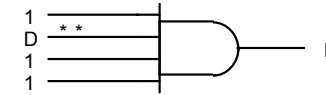
- denotemos con **D** el valor booleano que debe atravesar la celda (**D** será el valor presente en el nudo que estamos considerando: **D = 1** para investigar bloqueos a **0** y **D = 0** para tratar los bloqueos a **1**);
- una puerta "o", para que pase **D** desde una de sus entradas, ha de tener el resto de ellas a **0** y la salida será **D**; cuando **D** sea **0**, la salida **y = 0** detecta «no-bloqueo a **1**» en todas las entradas, cuando **D** sea **1**, la salida **y = 1** detecta solamente «no-bloqueo a **0**» en dicha entrada **D**:



D = 0 **y = 0** $\Rightarrow$ no-bloqueo a 1 de ninguna entrada

D = 1 **y = 1** $\Rightarrow$ no-bloqueo a 0 de la entrada **.

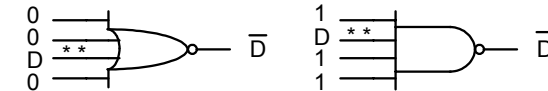
- una puerta "y", para dejar pasar **D** desde una entrada, requiere que el resto se encuentre a **1** y la salida será **D**; cuando **D** sea **0**, **y = 0** detecta solamente «no-bloqueo a **1**» de dicha entrada, pero cuando **D** sea **1**, **y = 1** detecta «no-bloqueo a **0**» de todas las entradas:



D = 0 **y = 0** $\Rightarrow$ no-bloqueo a 1 de la entrada **

D = 1 **y = 1** $\Rightarrow$ no-bloqueo a 0 de ninguna entrada.

- el modelo para puertas "o-negada" *Nor* e "y-negada" *Nand* es el siguiente:



- la puerta "o-exclusiva" presenta dos posibilidades:



25.3. Análisis de fallos y simulación de fallos

25.3.1. Generación de vectores de test: análisis de fallos

Para detectar un fallo es preciso encontrar un vector de test capaz de:

- a) *controlar* el nudo correspondiente, es decir, actuar sobre dicho nudo desde las entradas de forma que adquiera el valor booleano deseado;
- b) *observar* dicho nudo, es decir, establecer a partir del mismo un conjunto de valores sobre los nudos que interaccionan posteriormente con él, de forma que el valor booleano de una salida sea diferente en el circuito con fallo en relación con el correcto.

La parte **a** («control»), se consigue llevando «hacia atrás» los valores booleanos desde el nudo considerado hacia las entradas, lo cual, en principio, siempre es posible (en un circuito combinacional). La parte **b** («observación») trata de llevar «hacia adelante» el valor booleano del nudo hacia las salidas, imponiendo en el resto de los nudos que influyen sobre este camino «hacia adelante» el valor booleano que permita distinguir el valor que está pasando; ello no siempre es posible pues puede haber problemas de compatibilidad entre los valores booleanos de unos nudos y otros, habida cuenta de que todos ellos han de ser impuestos desde las entradas del circuito.

La parte **b** («observación») ha de aplicarse antes que la **a** ya que determina valores booleanos en otros nudos internos del circuito que, posteriormente (parte **a**: «control»), han de ser llevados (todos ellos, además del correspondiente al nudo objeto de investigación) hacia las entradas.

Se demuestra que, en el modelo de fallos por bloqueo STUCK-AT, si el circuito es combinacional y no contiene redundancias todos los fallos son «testeables»: para cada fallo existe un vector de test que, actuando desde la entradas del circuito, diferencia el comportamiento del circuito con y sin fallo, produciendo salidas distintas en ambos casos.

El empleo de la notación **D** (**D** = **1** para investigar bloqueos a **0**, **D** = **0** para los bloqueos a **1**) permite encontrar, a la vez, los vectores de test para ambos bloqueos de un nudo dado. El proceso de obtención de los vectores de test para un determinado nudo se realiza mediante el siguiente esquema procedimental, denominado **D-algoritmo**:

1. se asigna el valor **D** al nudo a «testear»;
2. a partir de dicho nudo y pasando por la correspondiente celda básica se lleva **D** hacia una salida asignando valores booleanos adecuados a los nudos de las puertas por las que se pasa;
3. el valor **D** y el resto de los valores booleanos asignados se llevan hacia las entradas.

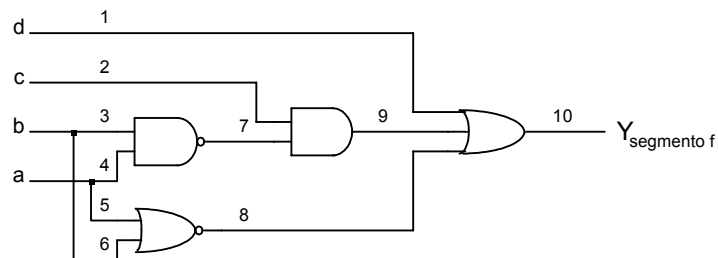
Para ejecutar los pasos 2 y 3 (consistentes en atravesar «hacia adelante» y «hacia atrás», respectivamente, las celdas básicas) se utilizan los «modelos de fallos» de las correspondientes celdas. En todo caso, es preciso tener bien presente la «hipótesis de fallo único en el circuito»: si en el circuito existe un fallo, éste es único (se desecha la posibilidad de que se produzca un segundo fallo que compense el primero).

El *D-algoritmo* resulta fácilmente programable (de hecho, existen varios algoritmos equivalentes desarrollados en detalle para su programación), lo cual permite la generación automática de vectores de test (ATPG).

Ejemplo de aplicación manual del D-algoritmo

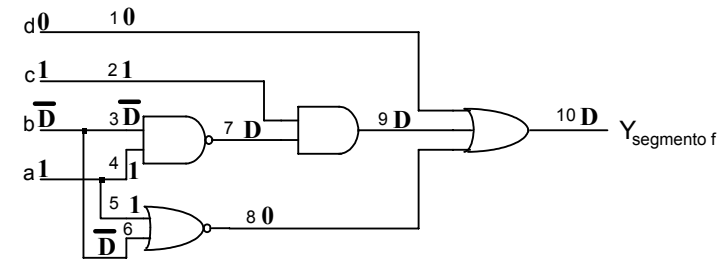
Consideremos el caso de una simple función booleana, la correspondiente a la activación del segmento **f** de un conversor BCD a 7 segmentos, expresada en la siguiente forma algebraica:

$$Y_{\text{segmento f}} = d + c.\bar{b} + c.\bar{a} + \bar{b}.\bar{a} = d + c.(b * a) + (b\Delta a)$$



«A efectos de test» el circuito presenta 10 nudos, tal como han quedado numerados en la figura; por tanto, el número de fallos posibles es de 20: 10 *bloqueos a 0* y 10 *bloqueos a 1*.

Efectuemos el análisis de fallos del circuito anterior, respecto al nudo 7:



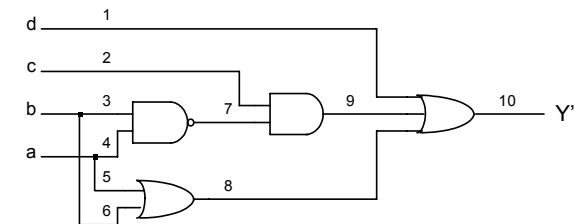
- 1 asignamos **D** a 7;
- 2 observación: nudo 2 a **1**; nudo 9 a **D**, nudos 1 y 8 a **0**, 10 a **D**;
- 3 control: nudo 1 a **0** (por tanto, entrada $D = 0$); nudo 2 a **1** (entrada $C = 1$); nudo 7 a **D** y, para ello, nudo 3 a $\bar{D}$ y nudo 4 a **1** (entradas $B = \bar{D}$ y $A = 1$); nudo 8 a **0**, lo cual es correcto ya que «entrada A» = nudo 6 = **1**.

vectores de test: **01 $\bar{D}$ 1** **0101** $\Rightarrow$ **D = 1** : detecta *bloqueo a 0* en el nudo 7

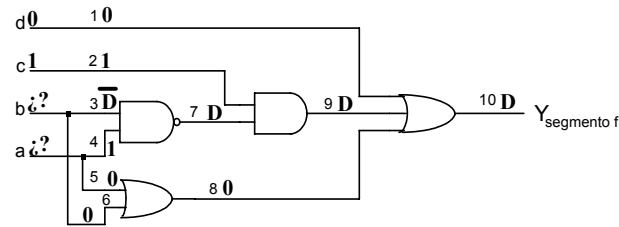
0111 $\Rightarrow$ **D = 0** : detecta *bloqueo a 1* en dicho nudo.

Cuando en el circuito hay redundancia, o sea, cuando es posible simplificar la función y eliminar algunas de las puertas que configuran el circuito los nudos afectados por la redundancia no son «testeables», es decir, no es posible encontrar un vector de test que diferencie el circuito con y sin fallo. Lo cual es consecuencia de que el valor booleano de tales nudos no «determina» por sí mismo el valor de las salidas, al existir caminos lógicos redundantes.

Por ejemplo, si en el circuito anterior cambiamos la puerta «o-negada» por una «o»:



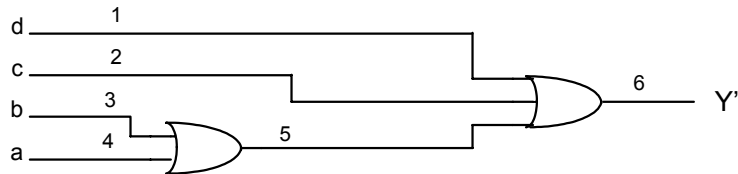
- 1 nudo 7 a **D**;
- 2 observación: nudo 2 a **1**; nudo 9 a **D**, nudos 1 y 8 a **0**, nudo 10 a **D**;
- 3 control: para que el nudo 8 se sitúe a **0** se requiere que los nudos 5 y 6 estén a **0**; en cambio para que el nudo 7 se sitúe a **D** es preciso o bien que el nudo 3 esté a $\bar{D}$ y el 4 a **1** o sino que el nudo 3 esté a **1** y el nudo 4 a $\bar{D}$ o que ambos nudos 3 y 4 estén a $\bar{D}$;



Como los nudos 3 y 6 han de tener el mismo valor booleano y lo mismo sucede con los nudos 4 y 5, para $D = 0$ tales condiciones (nudos 3 y 4 a 1 y nudos 5 y 6 a 0) son incompatibles entre sí; para $D = 0$, si existe vector de test, ya que corresponde a los 4 nudos (3, 4, 5 y 6) a 0:

- el vector **0100** detecta el *bloqueo a 0* del nudo 7;
- pero, en cambio, ningún vector es capaz de detectar el *bloqueo a 1* del nudo 7.

Ello es debido a que la función contiene redundancia, como se demuestra al simplificarla: $Y' = d + c.(b * a) + (b + a) = d + c + b + a$.



Las puertas "y-negada" e "y" son redundantes y pueden eliminarse.

Por tanto, en relación con el test, interesa evitar toda redundancia circuital: en ocasiones se ha recomendado introducir redundancia en el diseño de circuitos combinatoriales para eliminar «espurios» (*glitches*) debidos a la conmutación de las puertas (a sus tiempos de propagación); esto resulta nefasto para el test: los *glitches* deben ser evitados a través de un diseño adecuadamente síncrono.

25.3.2. Cobertura de test: simulación de fallos

No es preciso utilizar un vector de test para cada uno de los fallos posibles, pues un mismo vector de test sirve para detectar varios fallos a la vez.

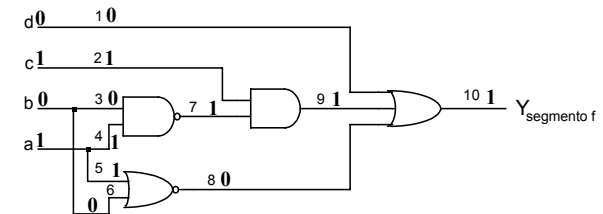
Por eso, una vez obtenido un vector de test conviene averiguar qué fallos es capaz de detectar. Esta operación se denomina *simulación de fallos* y se realiza yendo desde las salidas del circuito hacia las entradas y deduciendo, para cada celda básica a partir de su «modelo de fallos», los fallos que son detectables en las entradas de dicha celda.

Por ejemplo, recordando el modelo de fallos de las puertas "o" e "y":

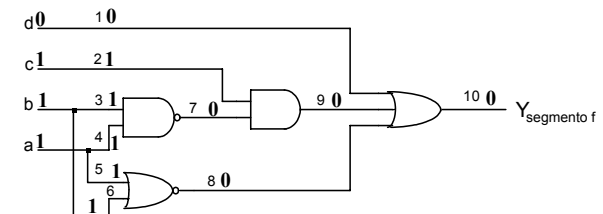
- una puerta "o" con salida **0** detecta cualquier *bloqueo a 1* en cualquiera de sus entradas; dicha puerta "o" con salida **1** solamente es capaz de detectar el *bloqueo a 0* de una de sus entradas y ello con la condición de que el resto de ellas esté a 0;
- una puerta "y" con salida **1** detecta *bloqueo a 0* en cualquier entrada y con salida **0** sólo es capaz de detectar el *bloqueo a 1* de una de sus entradas y con la condición de que el resto de ellas se encuentre a 1;

En el ejemplo anterior (segmento f de un convertidor BCD a 7 segmentos):

- el vector de entrada **0101** es capaz de detectar *bloqueo a 0* en los nudos 10, 9, 2, 7 y *bloqueo a 1* en el nudo 3;



- el vector de entrada **0111** es capaz de detectar *bloqueo a 1* en los nudos 10, 1, 9, 8, 7 y *bloqueo a 0* en los nudos 3 y 4.



Bastan los dos vectores obtenidos para testear el nudo 7 (**01 $\bar{D}$ 1**) para conseguir una cobertura de test del 60 % (12 fallos/20 posibles).

Siempre sucede que los primeros vectores de test detectan muchos fallos pero, a medida que se añaden nuevos vectores para aumentar la cobertura de test, éstos detectan muy pocos fallos nuevos.

En este ejemplo (segmento f), para una cobertura de fallos completa (100 %) son necesarios los siguientes vectores (en la tabla que sigue se indican, para cada vector de entrada, los fallos que son detectados por dicho vector; se señalan con # los «nuevos» fallos y con * los fallos que ya han sido detectados por vectores anteriores):

	bloqueos a 0										bloqueos a 1										
nudos :	1	2	3	4	5	6	7	8	9	10	1	2	3	4	5	6	7	8	9	10	
vector 0101		#						#		# #			#								25 %
vector 0111			# #								#						# #	# #			60 %
vector 0000								#		*					# #						75 %
vector 0001					#						*	#						*	*	*	85 %
vector 0010						#					*	*						*	*	*	90 %
vector 0110		*					*		*	*				#							95 %
vector 1010	#									*											100 %

en total se necesitan 7 vectores de test para un test completo del circuito.

Éste es el procedimiento común para generar los vectores de test:

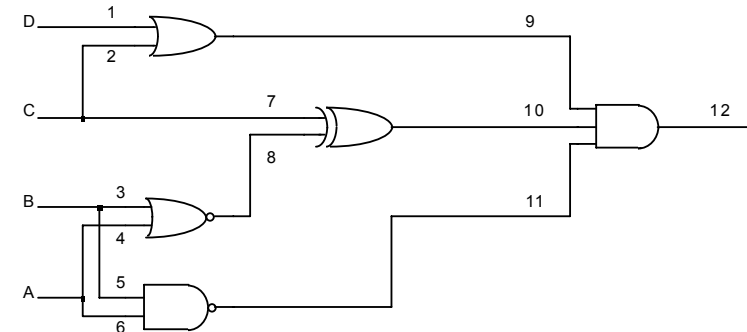
- se escoge un *subconjunto inicial de vectores de test*: tal subconjunto puede ser el de los vectores correspondientes a una simulación funcional del circuito (conjunto de vectores que habremos utilizado en el proceso de diseño para verificar que el circuito diseñado es correcto) o bien, sencillamente, un conjunto de vectores generados al azar (método que suele dar muy buenos resultados para el test);
- mediante *simulación de fallos* se averigua qué fallos son detectados por dichos vectores y cuáles no;
- mediante *análisis de fallos* se obtienen nuevos vectores para fallos aún no cubiertos;
- se reitera la *simulación de fallos* y se repite el proceso hasta que se consigue un grado de cobertura adecuado (a ser posible el 100 %).

En los ASICs digitales, los vectores de test son la referencia «legal» entre fabricante y cliente: el fabricante se compromete a que cada uno de los circuitos integrados que suministra verifica los vectores de test (la secuencia de vectores de test) aceptados por ambos (son objeto de firma específica en el contrato) y el cliente no puede exigirle legalmente otra cosa sino el cumplimiento de los vectores de test.

Los fabricantes de ASICs suelen exigir, como condición previa, una cobertura de test cercana al 100 % (en muchos casos, cobertura total), a fin de poder comprobar con adecuadas garantías los circuitos integrados resultantes de la fabricación.

Además, el fabricante señala un límite en el número de vectores test, a fin de que el tiempo de utilización de la máquina de test para cada *chip* no sea muy alto (y, en consecuencia, costoso). Generalmente el número máximo de vectores de test suele ser del orden de 64K o de 256K; el cliente puede solicitar la utilización de mayor número de vectores de test, pero ello repercutirá en el precio por unidad del circuito integrado.

Otro ejemplo de búsqueda de vectores de test



El circuito tiene 12 nudos «a efectos de test»: son posibles 24 fallos: 12 *bloqueos a 0* y otros 12 *bloqueos a 1*.

1. Comenzamos por el nudo 1 y buscamos su vector de test

vector de test para el nudo 1: D000

- el vector **0000** detecta *bloqueo a 1* en los nudos: **1, 2, 9 y 12**
- el vector **1000** detecta *bloqueo a 0* en los nudos: **1, 8, 9, 10, 11 y 12**
y detecta *bloqueo a 1* en los nudos: **3, 4 y 7**

La **cobertura de test** conseguida con ambos vectores superior al **50 %**: 13 / 24

2. Tomando un segundo nudo, no cubierto por los anteriores vectores, por ejemplo el nudo 5, buscamos su vector de test

vector de test para el nudo 5: 01D1

- el vector **0101** detecta *bloqueo a 0* en los nudos: **2, 4 y 7**
y detecta *bloqueo a 1* en los nudos: **5 y 8**
- el vector **0111** detecta *bloqueo a 0* en los nudos: **5 y 6**
y detecta *bloqueo a 1* en los nudos: **11**

La **cobertura de test** conseguida es cercana al **90 %**: 21 / 24. Solamente quedan por detectar el *bloqueo a 0* del nudo 3 y los *bloqueos a 1* de los nudos 6 y 25.

3. Buscamos el vector de test que detecta el *bloqueo a 0* del nudo 3: **1010**

- dicho vector **1010** detecta *bloqueo a 0* en el nudo: **3**
y detecta *bloqueo a 1* en el nudo: **10**

Con ello, la **cobertura de test** alcanzada es superior al **95 %**: 23 / 24

4. Buscamos el vector de test que detecta el *bloqueo a 1* del nudo 6: **1110**

- dicho vector **1110** completa el **100 % de cobertura de test** del circuito.

Los vectores utilizados en la simulación funcional, a lo largo del proceso de diseño, suelen proporcionar una cobertura de test entre el 60 % y el 80 %, según los casos; además, gran parte de ellos son redundantes respecto al test, no aumentan la cobertura que ya realizan los demás: el problema del test no se reduce al de la simulación funcional.

A lo largo de este apartado (y también en el anterior), al hablar de vectores de test puede parecer que solamente se hace referencia a «vectores de entrada» para el test; debe tenerse siempre presente que un vector de test ha de incluir ambas partes: {vector de entrada, vector de salida}; no sólo es preciso activar adecuadamente las entradas del circuito, sino, también, comparar los valores de las salidas con los que corresponden a un circuito correcto (bien fabricado).

25.3.3. Diseño orientado al test

Aunque la aplicación efectiva de los vectores de test la realiza el fabricante (la ejecuta la máquina de test sobre cada uno de los *chips* obtenidos en el proceso de integración), la obtención de los vectores de test es responsabilidad del diseñador y debe constituir un punto de referencia de todo el proceso de diseño: es necesario diseñar para el test.

El diseño condiciona el test y, sobre todo, puede facilitarlo en gran manera. No basta un diseño funcional sino que hay que efectuar, a la vez, un diseño para el test y, en la práctica, los aspectos relativos al test pueden resultar más difíciles y complejos que el propio diseño funcional. Ya hemos comentado que la redundancia da lugar a nudos no testeables: ha de evitarse toda redundancia, lo cual queda asegurado si los procesos de simplificación se realizan bien.

Por otra parte, comprender bien la dinámica del test permite definir estrategias que facilitan el test: el test se basa en *controlar* desde las entradas y en *observar* desde las salidas los nudos internos del circuito. Por ello interesa aumentar la observabilidad y la controlabilidad del circuito, facilitar el control y la observación de los nudos del circuito:

- el aumento del número de salidas: conectar nudos internos del circuito a salidas adicionales del mismo; lo cual se puede conseguir físicamente, aumentando en igual medida el número de terminales (*pines*), o por medio de multiplexado, utilizando los mismos terminales para varios nudos;
- el control de entradas intermedias: hacer que nudos internos adquieran el valor booleano directamente; lo cual requiere distinguir la situación de test de la correspondiente al funcionamiento normal, a través de un multiplexado apropiado;
- la división del circuito en bloques separables: configurar particiones, de forma que cada bloque sea controlable y observable desde el exterior, bien por multiplexado, bien mediante un bus interno, etc.,...;
- en general, cualquier medida que permita «acercar» nudos internos a las entradas (para controlarlos) o «aproximarlos» a las salidas (para observarlos) facilitará en gran manera el test (la obtención de vectores de test y la reducción del número de ellos necesario para una cobertura total).

25.4. Test de circuitos secuenciales

Los circuitos combinacionales presentan, en relación con el test, la ventaja de que el orden de los vectores no afecta a la comprobación de los fallos; en cambio, en los circuitos secuenciales el orden de los vectores de test es transcendental: los valores booleanos de los nudos del circuito no dependen solamente del vector de entrada, sino también del estado (interno) del circuito, es decir, del conjunto de valores almacenados en los biestables contenidos en él.

En un circuito secuencial, para imponer un determinado valor en un nudo, no basta con un vector de entrada, sino que es necesaria una secuencia adecuada de tales vectores. Ello aumenta, en gran medida, el número de vectores de entrada necesarios para el test y, lo que es mucho peor, aumenta extraordinariamente la complejidad de la obtención de dichos vectores de test; no hay una solución general que permita obtener la *secuencia de vectores de test* necesaria para detectar el bloqueo de un determinado nudo.

Por ejemplo, en un contador compuesto por 20 biestables, para analizar el bloqueo a **0** del más significativo de ellos es preciso realizar el conteo de 2^{19} unidades, es decir, más de medio millón de ciclos de reloj (el doble en cuanto a vectores de entrada) para llegar a establecer un **1** en dicho biestable.

Imaginemos la complejidad que presenta un circuito de control con varios contadores, diversos registros y algunos grafos de estado entrelazados mutuamente: intentar imponer valores booleanos determinados sobre un conjunto de nudos internos que dependan de tales bloques (y, además, hacerlo desde las entradas del circuito que, en principio, no conectarán directamente con ellos) es, sin duda, una labor ardua y, en muchos casos, inabordable.

En los circuitos secuenciales el problema del test no puede ser abordado a posteriori, sino que ha de ser un condicionante básico del diseño, desde el inicio del mismo: es indispensable un diseño orientado al test y desde tal perspectiva (diseñar para el test) no resulta difícil llevar a cabo un diseño bien estructurado y obtener la correspondiente secuencia de vectores de test para el mismo.

En general, resulta sumamente conveniente:

- 1 que el circuito sea completamente síncrono y esté bien diseñado como tal (y, como es obvio, con adecuada distribución del reloj que asegure la simultaneidad del mismo, y con ausencia de violaciones de los tiempos de *set-up* y de *hold*);
- 2 que el estado inicial de los biestables sea conocido, es decir, que exista un procedimiento directo de inicialización (*reset*) que permita llevar a los biestables a un estado determinado (**0** ó **1**).

La condición de «sincronismo» suele ser una exigencia de partida en el diseño de ASICs con celdas estándar pero, además, es una situación deseable desde el punto de vista de la seguridad funcional. El sincronismo resulta sumamente conveniente, tanto desde el punto de vista del test, como desde la propia seguridad funcional del circuito: un buen diseño secuencial es, hoy día, un diseño síncrono.

La posibilidad de inicialización viene exigida no sólo por el test sino también por la propia simulación funcional: no es simulable un circuito cuyo «estado inicial» no sea conocido; todo proceso de simulación comenzará efectivamente a partir de un estado inicial, que se establece (se impone) previamente sobre los biestables. Por ello, todo diseño secuencial debe contar con su entrada de inicialización (*reset*) que actúe, directa o indirectamente, sobre todos los biestables forzándoles a adoptar un estado conocido (por ejemplo, su borrado: puesta a 0).

25.4.1. Reducir el test secuencial al combinacional: SCAN PATH

Supuesto un circuito síncrono, la idea es poder *controlar* y *observar* cada uno de sus biestables, de forma que el test se reduzca al caso combinacional; es decir, manejar «desde fuera» el vector de estado del circuito (el estado de todos los biestables del mismo), convirtiéndolo en un vector de entrada (*control* directo sobre el estado anterior) y en un vector de salida (*observación* del estado siguiente), de manera que el circuito «deja de ser secuencial»: si el estado anterior es controlable como entrada y el estado siguiente es observable como salida, la función de memoria que da carácter a los circuitos secuenciales desaparece.

Supongamos que podemos fijar el valor booleano en la salida de cada biestable y, después de un solo pulso de reloj, podemos observar las salidas de todos ellos, el test a formular se referirá únicamente a la parte combinacional del circuito. De esta forma y en lo que se refiere al test, se prescinde de los biestables como tales (como elementos de memoria interna) y se produce una «duplicación» de ellos: por un lado, se convierten en «entradas» del circuito (sobre ellos se fija, desde fuera, el estado anterior) y por otro actúan como «salidas» (se observa el nuevo estado tras el pulso de reloj).

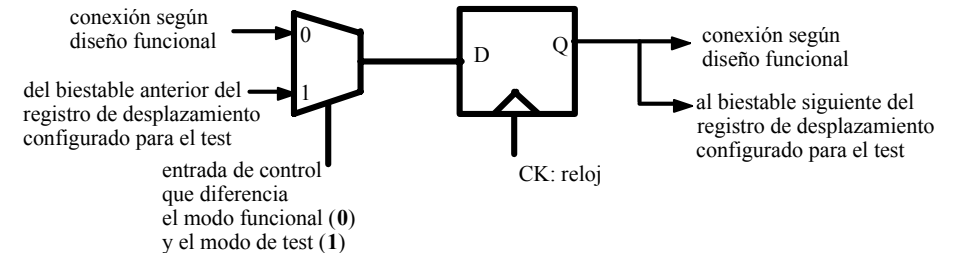
¿Cómo puede controlarse y observarse, en forma directa, desde el exterior del circuito integrado, el estado de todos los biestables del mismo? Una forma relativamente simple de hacerlo es conectando, a efectos de test, todos los biestables en largos registros de desplazamiento con entrada y salida en terminales específicos del circuito integrado.

Haciendo que, a los efectos de control y de observación para el test, los biestables se encuentren, todos ellos, conectados en serie, formando un registro de desplazamiento, con una entrada (serie) exterior y una salida externa (la del último biestable). Para dar valores a los biestables bastará almacenarlos sucesivamente, a través de la entrada, utilizando *n* pulsos de reloj (tantos como biestables contengan el registro de desplazamiento); pero, a la vez, durante tales *n* pulsos de reloj, la salida del registro presentará sucesivamente los valores contenidos en los biestables. De forma que, la *observación del estado* de los biestables (adquirido tras el pulso de reloj) y la *fijación de un nuevo estado anterior* en ellos se realiza simultáneamente.

Esta forma de organizar el test de los circuitos secuenciales, mediante el control y la observación «en serie» de todos sus biestables, recibe el nombre de *scan path* (camino de exploración): conexión de todos los biestables en cadena, permitiendo acceder a ellos por desplazamiento con doble finalidad, como entrada para fijar su valor y como salida para observar su estado.

Cada biestable ha de tener dos modos de actuación:

1. el modo funcional, que corresponde al diseño propio del circuito integrado;
2. el modo test, de conexión al registro de desplazamiento, para efectuar el control (del estado anterior) y la observación (del nuevo estado) durante el test.



El circuito integrado ha de disponer de 3 terminales adicionales:

- la entrada serie al registro de desplazamiento *test-in*
- la salida de dicho registro (del último biestable) *test-out*
- y una entrada de control del modo de operación *test-control*.



Obviamente, si el número de biestables (del *scan path*) es muy alto, pueden organizarse en varios registros de desplazamiento (cada uno de ellos precisará dos nuevos terminales: su entrada y su salida).

En el test con *scan path*, el primer paso será la comprobación de «no bloqueos» en el propio «camino de observación», en los propios biestables del circuito; para ello, se introduce en el registro de desplazamiento una secuencia de «ceros» y «unos» alternados y se comprueba la salida de dicha secuencia, después del retraso correspondiente a los *n* biestables del registro de desplazamiento.

Una vez comprobado el *no bloqueo* de los biestables, el *análisis de fallos* y la *simulación de fallos* se efectúan como si el circuito fuera combinacional, suponiendo que el estado anterior son entradas externas del circuito y el nuevo estado (tras un pulso de reloj) se observa en salidas externas del mismo. De esta forma, por métodos puramente combinacionales, se generan los vectores de test y se estudia la cobertura de test que proporcionan, teniendo en cuenta que cada vector de test incluye cuatro componentes:

vector de test = {vector de entrada, estado anterior, vector de salida, estado siguiente}

La aplicación del test, al final del proceso de fabricación, ha de efectuarse según la siguiente secuencia:

1. modo de test: se carga «en serie» el estado anterior sobre los biestables, lo cual requiere tantos pulsos de reloj y tantos «vectores de entrada» como sea el número de biestables n ,
2. modo de funcionamiento: se fija el vector de entrada, se aplica un pulso de reloj y se observa el vector de salida,
3. modo test: se observa el estado siguiente, extrayéndolo «en serie» de la cadena de biestables (n pulsos de reloj).

Obviamente, a la vez que se observa el estado siguiente (etapa 3) se efectúa simultáneamente la carga del «nuevo estado anterior» (etapa 1), correspondiente al siguiente vector de test; de manera que el número efectivo de pulsos de reloj que requiere cada vector de test es $n + 1$, siendo n el número total de biestables que conforman el registro de desplazamiento (o cada uno de los registros de desplazamiento, caso de configurarse varios de ellos).

25.5. Test de placas y de sistemas completos

La extraordinaria complejidad que alcanzan hoy día las placas de circuitos digitales y los sistemas digitales completos repercute fuertemente sobre su comprobación (test), que resulta sumamente difícil.

La filosofía desarrollada en el método *scan path* para el test de circuitos secuenciales es aplicable al test de placas de circuitos integrados digitales (y al test de sistemas digitales completos) en la siguiente forma:

- ha de incluirse un biestable en cada terminal (entrada o salida) de los circuitos integrados; dicho biestable actuará en «modo de test» formando parte (junto con los biestables internos) del registro de desplazamiento que conforma el camino de exploración (*scan path*) para el test del circuito integrado;
- los caminos de exploración (registros de desplazamiento) de los diversos circuitos integrados serán conectados en serie para formar un largo camino de exploración global para el test de la placa;
- este gran *scan path* permite controlar en serie todas las entradas de los circuitos integrados y observar en serie todas las salidas (y, también, el estado interno) de los mismos, de forma que podemos «testear» cada uno de dichos circuitos utilizando sus propios vectores de test;
- asimismo este *scan path* permite fijar valores en las salidas de los circuitos integrados y observar, luego, las entradas de todos los circuitos, lo cual hace posible comprobar las pistas de conexión (el circuito impreso), verificando si los valores fijados en las salidas de los circuitos son recibidos correctamente en las entradas de los circuitos conectados a tales salidas.

Esta forma de abordar el test de placas digitales, mediante la inclusión de «biestables en el contorno» de todos los circuitos integrados, para formar con ellos (junto con los biestables internos propios) un camino de exploración (*scan path*) de la placa, constituye un método de comprobación que recibe el nombre de *boundary scan* (exploración de contorno). Se encuentra en proceso de estandarización a través de un grupo de IEEE, el JTAG (*Joining Test Action Group*) encargado de redactar la correspondiente normativa.

El *boundary scan* requiere disponer de circuitos integrados especiales, preparados para dicho test de contorno; tales circuitos incorporan «biestables periféricos» en todos sus terminales (*pins*), es decir, en todas sus entradas y salidas, y cuatro terminales adicionales dedicados específicamente al «test»:

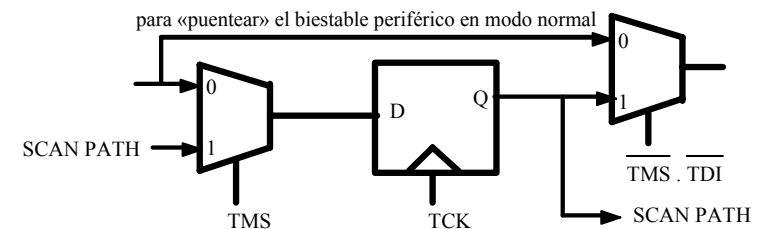
- dos para la entrada **TDI** y salida **TD0** del registro de desplazamiento (*scan path*),
- uno como entrada de control para la ejecución del test **TMS** (*test mode switch*)
- y otro como entrada de reloj para el test **TCK**.

Si el circuito integrado es secuencial, los «biestables propios» del circuito son conectados, asimismo, al registro de desplazamiento (*scan path*) junto con los «biestables periféricos», para poder efectuar el test completo de los circuitos.

El circuito funciona en tres modos, completamente diferentes:

- el «modo normal» en el cual no actúan los biestables periféricos (se encuentran «puenteados») y «no existe» el *scan path*: **TMS = TDI = 0**;
- el «modo de desplazamiento» **TMS = 1**, **TCK = pulsos de reloj**, en que se produce el desplazamiento del registro que configura el *scan path* y, por tanto, la entrada y la salida de los vectores de test;
- el «modo de ejecución de test»: **TMS = 0**, **TDI = 1**, **TCK = un solo pulso de reloj** en que los «biestables periféricos» actúan como entradas de los circuitos integrados (para realizar su test interno) y como salidas de los mismos (para realizar el test de las conexiones entre ellos).

Estos tres modos de funcionamiento se diferencian, en los biestables periféricos, mediante dos multiplexores:



El test de cada circuito integrado se efectúa en la forma siguiente:

- ① vector de entrada para dicho test:
 - entradas al circuito integrado: se cargan sobre los biestables periféricos de las entradas del mismo;
 - estado anterior: se carga sobre los biestables internos propios del circuito.
- ② un pulso de reloj en modo de ejecución de test;
- ③ vector resultante «de salida» del test:
 - salidas del circuito integrado: se encontrarán en los biestables periféricos de las salidas del mismo;
 - nuevo estado: en los mismos biestables internos propios del circuito.

El test de las pistas de la placa de circuito impreso se efectúa así:

- ① valores a comunicar a través de las pistas: se cargan sobre los biestables periféricos de las salidas de los diversos circuitos integrados;
- ② un pulso de reloj en modo de ejecución de test;
- ③ valores transmitidos a través de las pistas: se encontrarán en los biestables periféricos de las entradas de los circuitos integrados.

El test se realiza mediante la ejecución de las siguientes fases:

- fase de desplazamiento: **TMS = 1**, **TCK = n** pulsos de reloj (siendo **n** el número de biestables, la longitud del *scan path*); se produce el desplazamiento del registro que configura el camino de exploración y, por tanto, la entrada del vector, bit a bit, de test a dicho registro;
- fase de ejecución: **TMS = 0**, **TDI = 1**, **TCK =** un solo pulso de reloj; dicho pulso de reloj ejecuta el test y los biestables recogen el resultado del mismo: los biestables propios y los biestables periféricos de las salidas reciben el resultado del test interno de cada circuito integrado; los biestables periféricos de las entradas reciben el test de conexiones entre circuitos (pistas del circuito impreso);
- de nuevo una fase de desplazamiento: **TMS = 1**, **TCK = n** pulsos de reloj; permite observar el resultado del test y, a la vez, introducir el vector para el siguiente test.

Al igual que cada circuito integrado, la placa circuital dispondrá de cuatro terminales específicos para el test:

- **TDI** (*Test Data In*): entrada de vectores de test
- **TDO** (*Test Data Out*): salida de resultados del test
- **TCK** (*Test Clock*): reloj para la ejecución del test
- **TMS** (*Test Modo Switch*): control «modo test»

Dentro de la placa, las entradas **TDI** y las salidas **TDO** de test de los circuitos integrados que contiene han de estar unidas en cascada (**TDO → TDI'**), formando una larga cadena de registros de desplazamiento (un *scan path* global). Siendo **p** el número total de biestables presentes en la placa, el test «global» de la misma requiere **p** pulsos de reloj (con **TMS = 1**) para la «fase de desplazamiento» (en la cual, se observan los resultados del test anterior y se introducen, a la vez, los vectores de entrada para el test siguiente) y **1** pulso de reloj (con **TMS = 0** y **TDI = 1**) para la «la fase de ejecución».

Los «vectores de entrada» necesarios son los correspondientes al test de los diversos circuitos integrados contenidos en la placa, adecuadamente encadenados entre sí (según el *scan path* que tales circuitos configuran), intercalados (conforme a los «biestables periféricos» de las salidas de los integrados) con los vectores para el test de conexiones entre circuitos (test de las pistas del circuito impreso), que serán conjuntos aleatorios de ceros y unos.

Del mismo modo (con la misma filosofía y con los mismos recursos) que se ha planteado el test de una placa de circuitos digitales, puede desarrollarse el test de un conjunto de placas o, lo que es lo mismo, el test de un sistema digital completo.

25.6. Test interno

En circuitos integrados de gran complejidad (en particular, en bloques correspondientes a estructuras matriciales tales como RAM, ROM, PLA, etc.), conviene incluir, en el propio circuito integrado módulos que permitan realizar «internamente» el test; es decir, circuitos complementarios para:

- generar por sí mismos un conjunto apropiado de vectores de test (vectores de entrada)
- y analizar los resultados (vectores de salida) del test.

Téngase en cuenta que un *scan path*, para ejecutar un sólo test, requiere **n + 1** ciclos (siendo **n** el número de biestables presentes en dicho camino de exploración), ya que es preciso almacenar, bit a bit, el vector de entrada (y también extraer, bit a bit, el resultado del test); el número de ciclos puede reducirse por un factor **n + 1** si los vectores de entrada se generan internamente y los resultados se analizan (o se almacenan en forma compacta) en el interior del circuito integrado. Tal es el objetivo del *test interno*.

Consideremos una memoria RAM de gran capacidad: su test implica escribir y leer en cada uno de sus registros varias veces, lo cual requiere un alto número de ciclos. Sin embargo, la generación y el análisis de vectores de test es muy sencilla (como veremos a continuación); resulta conveniente incluir tales funciones (generación y análisis) dentro del circuito integrado, lo cual permite prescindir (en buena medida) de las costosas máquinas de test y realizar el test con un equipo de observación mucho más simple.

El test interno permite comprobar los circuitos posteriormente, una vez instalados en el sistema electrónico concreto en que vayan a ser utilizados, y verificar (en cualquier momento y de forma sencilla) que dichos circuitos integrados siguen funcionando bien. Es decir, el test interno no es sólo una facilidad para la necesaria verificación del circuito en el proceso de fabricación, sino que configura un recurso permanente (una prestación de la cual se dispone siempre) que permite efectuar nuevas verificaciones «in situ», a lo largo del tiempo de utilización del circuito integrado; en ambos casos, la ejecución del test es muy simple ya que, básicamente, se realiza en el interior del circuito.

El *test interno* plantea la realización del test, por medio de un circuito auxiliar (por *hardware*), dentro del propio circuito integrado; es decir, incluir en su diseño (y, posteriormente, en su fabricación) un añadido circuital específico para la ejecución del test. Dicho circuito auxiliar ha de contener, obviamente, dos partes (entradas / salidas):

- un generador de los vectores de test (vectores de entrada)
- y un analizador de los resultados del mismo (vectores de salida).

Una forma sencilla de «generar vectores» de **p** bits la constituye un contador de **p** biestables, que producirá todos los vectores de **p** bits en orden numérico binario; otra posibilidad, tan sencilla o más que la anterior, es generar vectores «aleatoriamente» utilizando un registro de desplazamiento. La segunda opción tiene la ventaja de que los vectores contiguos presentan menor correlación (son más independientes entre sí) que los que proceden de un contador.

En ocasiones (por ejemplo, en el test BIST para memorias RAM que se describe a continuación), los resultados del test presentan una gran uniformidad y basta un pequeño circuito combinacional para detectar si son correctos; una salida del circuito integrado servirá para informar de que los sucesivos resultados del test son válidos (dicha salida se mantendrá constantemente a **0** durante el test: un simple pulso a **1** indica mal funcionamiento del circuito).

Este tipo de solución combinacional para analizar el test no siempre es posible y en muchas ocasiones hemos de limitarnos a almacenar, en forma compactada, los sucesivos vectores de salida de test; la *firma sobre p bits* (una extensión de la paridad a **p** bits) permite compactar, sobre una longitud **p**, cualquier serie de bits por larga que sea, detectando error en la misma siempre que el número de errores no sea superior a **p**.

En los siguientes subapartados se detallan diversas configuraciones relativas al test interno de circuitos integrados:

- el test interno de memorias, **BIST**,
- la generación aleatoria de vectores de test, **LFSR**,
- el almacenamiento compacto de vectores resultantes del test, **FIRMA**
- y el aprovechamiento de un registro como LFSR y como FIRMA, **BILBO**.

25.6.1. BIST (Built In Self Test): test interno de memorias

Genéricamente el término **BIST** (autotest construido internamente) alude a cualquier procedimiento de *test interno*, pero suele emplearse más específicamente para el test de memorias, en el cual las direcciones se generan mediante un contador y los contenidos de los registros corresponden a *todo unos*, a *todo ceros* y a un «ajedrezado» de *ceros* y *unos* alternativos.

El test de las memorias RAM suele realizarse mediante el siguiente algoritmo (supuesta una memoria de N registros, cuya numeración irá de 0 a N-1):

- 1 de 0 a N-1 escribir todos los registros con **11111 ...**
- 2 de 0 a N-1 leer y comprobar cada registro y escribir en el mismo **00000 ...**
- 3 de 0 a N-1 leer y comprobar cada registro y escribir **11111 ...**
- 4 de N-1 a 0 leer y comprobar cada registro y escribir en el mismo **00000 ...**
- 5 de N-1 a 0 leer y comprobar cada registro y escribir **11111 ...**
- 6 de 0 a N-1 escribir sucesivamente registros impares con **101010 ...**
y registros pares con **010101 ...**
- 7 de 0 a N-1 leer y comprobar cada registro
- 8 de 0 a N-1 escribir sucesivamente registros impares con **101010 ...**
y registros pares con **010101 ...**
- 9 de 0 a N-1 leer y comprobar cada registro
- 10 test completo, memoria disponible para ser utilizada.

De esta forma, se observa no sólo la escritura y lectura de un **0** y de un **1** sobre cada biestable de la memoria, sino también la influencia de los biestables contiguos: de ahí que, a la vez que progresa la lectura de «unos», se escriban «ceros» antes de leer el registro contiguo y viceversa y de ahí, también, la utilización de ajedrezados con «ceros» y «unos» intercalados en las dos direcciones y en las dos posibilidades.

Puede comprobarse que, con el algoritmo anterior, cada biestable (i, j, siendo i la fila del biestable y j la columna del mismo) se prueba en sus dos valores (**0** / **1**) respecto a todas las posibilidades de los ocho biestables «vecinos» que le rodean: los dos biestables contiguos de la misma fila (i, j-1 ; i, j+1), los tres biestables análogos de la fila anterior (i-1, j-1; i-1, j ; i-1, j+1) y los tres de la siguiente (i+1, j-1; i+1, j ; i+1, j+1). Asimismo, se comprueban las diversas posibilidades de interacción entre biestables del mismo registro (que se encontrarán en «hojas» diferenciadas, véase la estructura de las memorias RAM en 19.2., segundo volumen).

Este test requiere un contador para generar las direcciones de los registros; un segundo contador de cuatro bits y un decodificador para distinguir las diez fases de ejecución del test; un circuito combinacional muy simple que genera, a la vez, los vectores de entrada y de salida del test y un comparador entre la lectura de los registros y los citados vectores de salida.

Los vectores a generar son muy simples:

- dos componentes: **D_{impar}** biestables impares y **D_{par}** biestables pares de los registros;
- en la fase 1, **D_{impar}, D_{par} = 11**; en la fase 2 y en la fase 4 **11** (lectura) y **00** (escritura) alternativos; en la fase 3 y en la 5 **00** (lectura) y **11** (escritura) uno tras otro;
- en la fase 6 y en la 7, **D_{impar}, D_{par} = 10** (registros pares) y **01** (impares) sucesivamente y en las fases 8 y 9 **01** (registros pares) y **10** (impares).

Basta un pequeño circuito combinacional, a partir del decodificador de fases citado, de un biestable que distinga entre lectura y escritura (en las fases 2, 3, 4 y 5) y de la línea de direccionamiento **A0** (registros pares e impares), para generar los citados vectores de test que cubren tanto la entrada (escritura) como la salida (lectura) del mismo.

El comparador entre la lectura de los registros y el vector de referencia generado por el circuito combinacional ha de habilitarse solamente en los momentos de lectura; de forma que su salida se encontrará normalmente a **0** y solamente pasa a **1** si la lectura del registro no coincide con el valor esperado.

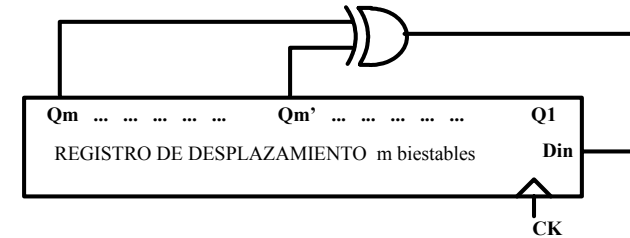
Este tipo de test interno (**BIST**) requiere solamente tres terminales: **TMS** de control del «modo de test», **TCK** para enviar un reloj que haga avanzar a los contadores y **TDO** el terminal de salida del comparador; el test se realiza haciendo **TMS = 1**, enviando una alta frecuencia de reloj por **TCK** y observando la ausencia de **1** en **TDO**. No es necesario contar los pulsos enviados a **TCK**: basta dedicar suficiente tiempo al test para asegurar que se ha llegado a la fase 10.

25.6.2. LFSR (Linear Feedback Shift Register): generación «aleatoria» de vectores

La ejecución del test (sobre un bloque combinacional) requiere «vectores de entrada» con **p** componentes (**p** entradas); tales vectores pueden generarse mediante un contador de **p** biestables (módulo 2^p) y, en tal caso, aparecerán según el orden numérico binario.

Otra posibilidad consiste en generar vectores de forma «aleatoria», que asegura menor correlación (nula si realmente fueran aleatorios) entre vectores sucesivos, por lo cual resultan mucho más adecuados para el test; en particular, cuando no se utiliza un conjunto completo (2^p) de «vectores de entrada» (se ha comentado antes que los vectores de entrada aleatorios consiguen «coberturas de fallos» relativamente altas).

Los registros de desplazamiento (SR: *Shift Register*) con realimentación lineal (LF: *Linear Feedback*) constituyen sencillos generadores de vectores «seudoaleatorios». Para determinados valores de longitud de un registro de desplazamiento (**m** biestables, siendo **m** el más significativo), se verifica que, realimentando hacia la entrada, a través de una puerta «o-exclusiva», la salida del biestable **m** y otra de las salidas **m'**, adecuadamente escogida, el registro genera sucesivamente y en forma «seudoaleatoria» todos los vectores de **m** bits menos el nulo **0000...**: $2^m - 1$ vectores.



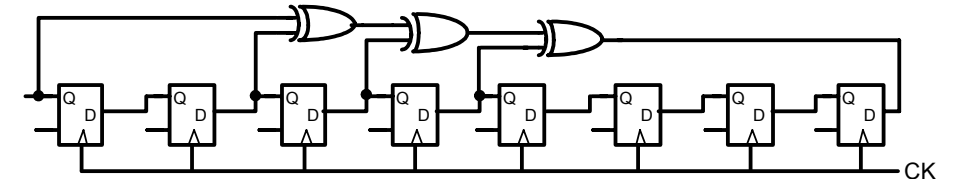
El número **m'** del segundo biestable a realimentar, para conseguir dicha generación «máxima» de vectores pseudoaleatorios ($2^m - 1$), se calcula por técnicas matemáticas de «polinomios generadores»; por ejemplo:

m = 10	m' = 7	1.023 vectores;
m = 15	m' = 14	32.767 vectores;
m = 20	m' = 17	1.048.575 vectores;
m = 25	m' = 22	33.554.431;
m = 31	m' = 28	2 mil millones de vectores;
m = 39	m' = 35	0,5 billones.

Si se añade un inversor detrás de la puerta «o-exclusiva» se obtienen todos los vectores de **m** bits menos el «todo unos» **1111...**, lo cual permite utilizar el vector nulo **0000...** para iniciar la generación de vectores pseudoaleatorios.

Para longitudes habituales (8, 16, 24, 32,...) es preciso realimentar, a través de puertas «o-exclusiva» (paridad), más de 2 biestables hacia la entrada:

para m = 8 hay que realimentar los biestables 8°, 6°, 5° y 4° (255 vectores),



para m = 16: se realimentan 16°, 15°, 13° y 14° y produce 65.535 vectores distintos,

para m = 24: se realimentan 24°, 23°, 22° y 17° y genera 16.777.215 vectores.

Los registros de desplazamiento con realimentación lineal **LFSR** son los módulos utilizados normalmente como generadores internos de vectores de test. Respecto a contadores de igual longitud tiene la ventaja de que su circuitería es más simple (**m** biestables y una o varias puertas «o-exclusiva», sin tener que constituir las complejas funciones que corresponden al conteo) y que generan los vectores en forma «cuasi aleatoria» y no en orden numérico binario.

25.6.3. FIRMA: almacenamiento compacto de vectores

Al realizar un test con n vectores de entrada obtenemos n vectores de salida, es decir, una secuencia de n palabras de p bits: $n \cdot p$ bits; la idea es «comprimir» dichos resultados de forma que la cantidad de memoria necesaria para almacenarlos sea pequeña (en la práctica una sola palabra de igual longitud que cada vector salida: p bits) y de forma, también, que la obtención global del resultado del test sea sencilla: bien en cuanto a número de ciclos para leer el registro de resultados, bien en cuanto al «circuito de comparación» que evalúa si el resultado es o no correcto.

Entendemos por «firma sobre p bits» (del inglés *firm*) la reducción a una sola palabra de p dígitos de n palabras binarias, cada una de ellas de p bits, con las siguientes propiedades:

- la firma es una aplicación unívoca de n palabras de p dígitos sobre una palabra de p bits,
- dos conjuntos distintos, de n palabras de p dígitos cada uno, que no se diferencien en más de p bits, tendrán firmas diferentes entre sí,
- es decir, si en un conjunto de n palabras de p dígitos se modifican (se invierten) p de los $n \cdot p$ bits (o menos), la firma del conjunto modificado será diferente de la firma del conjunto inicial.

De manera que la *firma sobre p bits* permite detectar errores que afectan a p bits o a menos de p bits, pues el «análisis de firma» de dos conjuntos de palabras diferentes conduce a dos firmas distintas, salvo que dichos conjuntos difieran en más de p bits (en cuyo caso sus firmas pueden ser diferentes o iguales).

Sea S el conjunto de $n \cdot p$ dígitos, su «firma» es una aplicación sobre p bits

$$S (n \cdot p \text{ bits}) \longrightarrow F(S) (p \text{ bits})$$

tal que, si dos conjuntos S_1 y S_2

$$S_1 \longrightarrow F(S_1) = F_1$$

$$S_2 \longrightarrow F(S_2) = F_2$$

tienen la misma firma $F_1 = F_2$, puede afirmarse que:

- o bien ambos conjuntos son iguales ($S_1 = S_2$)
- o bien difieren en más de p bits ($S_1 \oplus S_2$ contiene más de p «unos»).

Por ello, la firma detecta errores cuando su número no es superior a p , ya que la firma de un conjunto S' erróneo será distinta de la firma del conjunto correcto S :

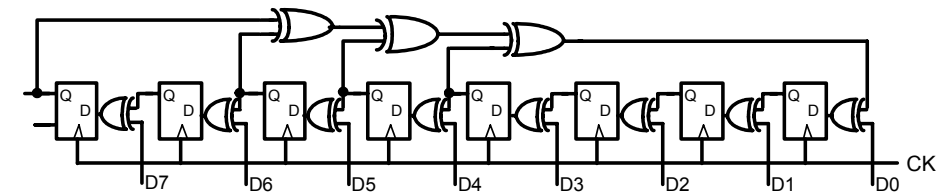
$$F' \neq F \text{ si } S \neq S' \text{ y } S \oplus S' \text{ no contiene más de } p \text{ «unos»}.$$

Obviamente, la bien conocida «paridad» corresponde a la «firma sobre 1 bit».

Para calcular la «firma sobre p bits» basta modificar un registro de desplazamiento realimentado linealmente **LFSR**, añadiendo puertas "o-exclusiva" en las entradas de sus biestables en la forma que sigue:

- cada palabra de p dígitos es recogida por el registro de desplazamiento a través de p «entradas paralelo especiales» y es «mezclada» con el estado del registro;
- cada biestable recibe, en su entrada D, la operación "o-exclusiva" entre la salida del biestable anterior (conexión serie propia del registro de desplazamiento) y el bit correspondiente de la palabra recibida (entradas paralelo especiales);
- el registro se encuentra, además, realimentado linealmente en la forma propia de los registros de desplazamiento con realimentación lineal.

Tal registro de desplazamiento, con entrada paralelo en forma de "o-exclusiva" y realimentación lineal, realiza la «firma sobre p bits» siendo p el número de sus biestables y el número de dígitos de las palabras que recibe:

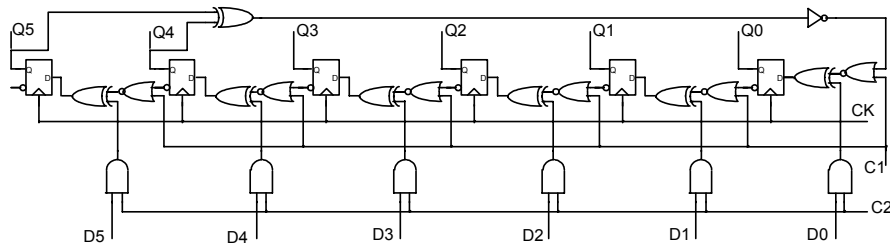


En el «registro de desplazamiento» anterior, al recibir uno tras otro los n «vectores de salida» de test (supuesto que sean de 8 bits), se realiza la firma del conjunto de dichos n vectores ($n \cdot 8$ bits) sobre una palabra de 8 dígitos. Puede demostrarse que dos conjuntos diferentes (de n palabras de 8 dígitos cada uno), cuya firma sea la misma, diferirán en más de 8 bits; es decir, permite detectar errores que afecten a 8 bits o menos.

Los circuitos que realizan la firma sobre p bits reciben el nombre de analizadores de firma.

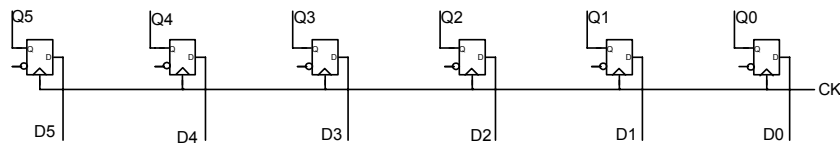
25.6.4. BILBO (Built In Block Observer): test interno mediante LFSR y FIRMA

Un registro síncrono, de entradas paralelo (o, lo que es lo mismo, un conjunto de biestables de tipo D), puede ser «reconfigurado» de forma que, además de actuar como registro retención, pueda hacerlo como generador de vectores (LFSR) y como analizador de firma o, también, pueda borrarse en forma síncrona. Basta, para ello, añadir al registro la lógica correspondiente al analizador de firmas, junto con dos entradas de control **C1** y **C2** que actúan a través de sendas puertas "y" y "o-negada", según el esquema de la figura.



Cuando ambas entradas tienen valor **1**, **C1 = C2 = 1**, el bloque actúa como un simple registro de entradas paralelo:

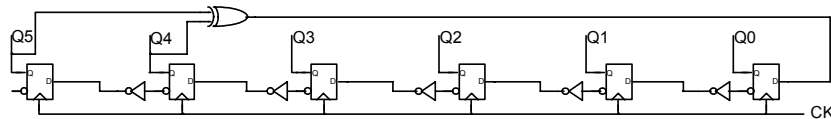
C1 = C2 = 1



Registro de entradas paralelo

Para ambas entradas a valor **0**, **C1 = C2 = 0**, el bloque queda en forma de registro de desplazamiento realimentado linealmente **LFSR**, o sea, un generador de vectores de test:

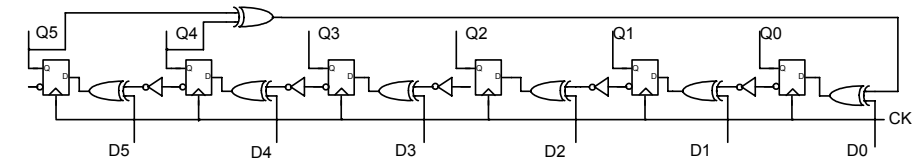
C1 = C2 = 0



Generador de vectores de test

Para **C1 = 0** y **C2 = 1**, el bloque se configura como analizador de firma:

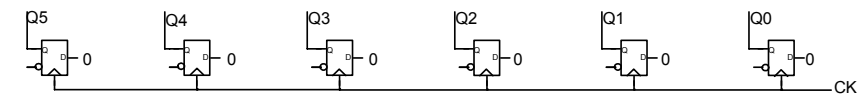
C1 = 0 C2 = 1



Analizador de firma

Y con **C1 = 1** y **C2 = 0**, se produce el borrado síncrono del registro:

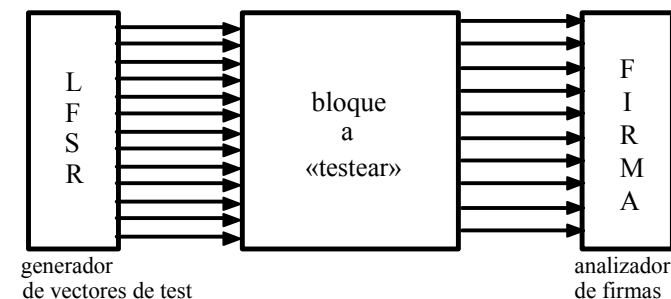
C1 = 1 C2 = 0



Borrado síncrono

Este tipo de registro, que puede ser empleado bien como registro de retención (o biestables individuales) con borrado síncrono, bien como generador de vectores o como analizador de firmas, recibe el nombre de registro **BILBO** (*Built In Block Observer*). Con esta estructura cualquier registro interno, o cualquier conjunto de biestables, de un circuito integrado puede ser utilizado, a efectos de test, como generador de vectores de test y como analizador de firma.

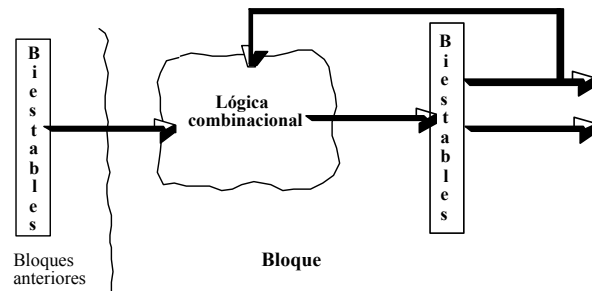
Los registros **BILBO** dan lugar a una configuración de test interno que consiste en un generador de vectores de test (**LFSR**) y un analizador de firmas; el primero genera vectores de test sobre las entradas del bloque a testear y el segundo almacena, en forma comprimida, los vectores de salida resultantes del test.



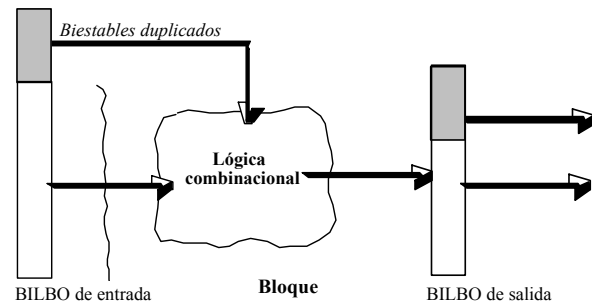
Para efectuar el test, esta configuración requiere tres terminales: **TMS** para controlar el modo de test, **TCK** como reloj y **TDI** para la salida del analizador de firmas; dicha salida puede obtenerse en forma de firma global (**p** bits) que se extrae «por desplazamiento» (mediante *scan path*) bit a bit o en forma unitaria (1 bit), añadiendo un comparador que confronte la firma obtenida con la esperada en el caso correcto.

Los registros de «entrada» (generador de vectores) y «salida» (analizador de firma) pueden ser registros añadidos a los solos efectos del test, pero también pueden ser registros o conjuntos de biestables propios del diseño, configurados en forma de **BILBO**.

Según se comentó en el capítulo 15 (volumen 2), un circuito secuencial síncrono puede «segmentarse» en bloques o módulos conformados por una parte combinacional y un registro o conjunto de biestables que recogen las salidas de ella y conectan con el bloque o bloques siguientes; algunas salidas del propio bloque se realimentarán sobre el mismo. De manera que cada «parte combinacional» se encuentra entre dos registros o conjuntos de biestables, uno de ellos correspondiente al bloque o bloques anteriores (entradas) y el otro, el propio del bloque (salidas).

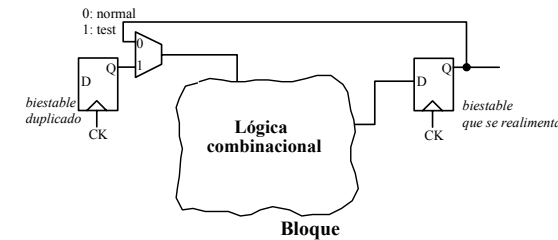


A partir de la estructura representada en esta figura, para obtener una configuración BILBO es necesario «duplicar» (en el modo de test) los biestables que se realimentan sobre el propio bloque, de forma que una mitad siga siendo biestables de salida del bloque para configurar, con el resto de los biestables de salida, un registro BILBO que actúe como analizador de firma y la otra mitad se agregue a los biestables de entrada, formando otro registro BILBO como generador de vectores de test.



De manera que un circuito integrado complejo puede dividirse (a efectos de test) en partes, formadas (cada una de ellas) por un núcleo combinacional entre dos registros **BILBO** y, de esta forma, puede realizarse el test interno de cada parte utilizando el registro anterior, como generador de vectores de test, y el registro siguiente, como analizador de firmas; para cada una de las firmas, ha de comprobarse su coincidencia (o diferencia) con las que corresponden al circuito sin fallos.

La configuración necesaria para «duplicar» los registros que se realimentan sobre el bloque puede ser la siguiente:



25.6.5. Autotestabilidad, seguridad y tolerancia frente a fallos

La inclusión en el propio circuito integrado de módulos que permiten realizar su test internamente introduce un nuevo concepto, la autotestabilidad: la capacidad de verificar «in situ» y en cualquier momento la configuración funcional del circuito integrado. De esta forma, el circuito puede autocomprobarse repetidamente y avisar cuando tal comprobación detecte fallos.

Cada vez son más los sistemas digitales que se «autochequean», antes de comenzar su actuación (o, también, periódicamente a lo largo de la misma, aprovechando pausas funcionales), para asegurar que todas sus partes se encuentran en perfecto estado de funcionamiento.

Un paso más allá en la detección de errores consiste en realizar (al menos parcialmente) el test del circuito en forma simultánea a su funcionamiento como tal, de manera que el circuito integrado supervise su actuación continuamente y en paralelo con la misma y avise de la existencia de fallos lo antes posibles y sin necesidad de efectuar una solicitud exterior de test. En este caso el circuito presentará una gran seguridad frente a fallos (permitiendo «desechar» con inmediatez cualquier funcionamiento erróneo).

Una solución que ya se utiliza es la de duplicar determinados sistemas o partes digitales y verificar la coincidencia de los cálculos realizados por ambos.

Lo óptimo sería que el circuito, además de avisar en cuanto existe un fallo (seguridad frente a fallos), fuera capaz de corregir su propio funcionamiento y su respuesta fuera correcta, aun en presencia de fallos: circuito tolerante a fallos. Al igual que existen códigos autocorrectores de errores (que permiten tal corrección cuando el número de errores es limitado) es posible diseñar circuitos que, para un número limitado de fallos, la presencia de ellos no introdujera errores en la respuesta del circuito.

Al igual que la duplicación de partes digitales permite que se «vigilen» mutuamente, su «triplicación», añadiendo un circuito de «decisión mayoritaria», da lugar a que, si una de las tres partes iguales que funcionan en paralelo «falla» (si proporciona una respuesta diferente a la de las otras dos), sus resultados sean desechados y se adopte como respuesta correcta la común de las otras dos partes.

A6 Apéndice al capítulo 23

Ejemplos de diseño con VHDL

Este apéndice ofrece una serie de ejemplos de diseño de circuitos de control descritos en VHDL. Los seis primeros se refieren a pequeños sistemas, basados en la utilización de contadores, a saber:

- temporización de la luz de una escalera,
- control de anchura de pulsos de entrada,
- supervisión de velocidad de un motor,
- comparación de anchura de pulsos sucesivos,
- generación de n pulsos, siendo n la medida del pulso de entrada,
- y pulsos de anchura progresivamente creciente/decreciente.

En estos ejemplos, como referencia para el texto VHDL y, también, como ejercicio de diseño modular, se incluye y se comenta el correspondiente diagrama de bloques.

Otros tres diseños se refieren a máquinas algorítmicas descritas en el capítulo 24:

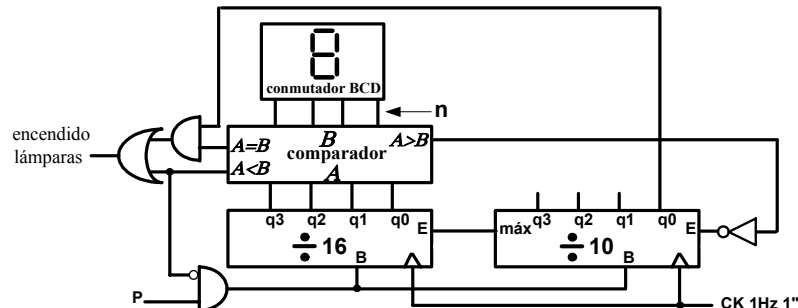
- producto de dos números de 64 dígitos binarios (apartado 24.3.),
- raíz cuadrada de un número de 64 dígitos (ejemplo 24.4.2.),

y, por último, se desarrollan dos diseños más amplios, relativos a un cronómetro controlado mediante dos pulsadores (ejemplo 24.4.5. del capítulo 24) y al control de una encimera de cocina vitrocerámica con cuatro «fuegos».

[Los diez diseños de este apéndice han sido comprobados, mediante compilación y simulación del texto VHDL con la herramienta MAX+plus II de ALTERA.]

1. Temporización de la luz de una escalera

Un conmutador BCD controla las lámparas de iluminación de la escalera de una comunidad de vecinos; de forma que, si el número fijado en el conmutador BCD es n , las lámparas deben mantenerse encendidas durante $n \cdot 10$ segundos cada vez que se pulsa un interruptor P ; durante los 10 segundos siguientes, las lámparas deben apagarse y encenderse 5 veces sucesivas, con intervalos de 1 segundo. Cuando las lámparas están encendidas, si se pulsa P no hace nada, pero al hacerlo durante el intervalo de intermitencia, o posteriormente al mismo, se reinicia el ciclo de encendido.



A partir de un reloj de 1 Hz (1 segundo de período), un contador década determina ciclos de 10" y un segundo contador realiza el conteo de dichos ciclos y la comparación con el número n fijado en el conmutador BCD. Mientras el segundo contador se encuentra por debajo de n (n estados, de 0 a $n-1$) las lámparas se mantienen encendidas; cuando ambos números son iguales, se produce la intermitencia (10") y, al superar el contador el valor n se detiene el conteo para evitar que «se dé la vuelta» (vuelta a 0) e inicie un nuevo ciclo de iluminación.

Para la intermitencia se aprovecha la primera salida (unidades) del primer contador, cuyo semiperíodo (tiempo en 1 y tiempo en 0) es 1 segundo. El pulsador borra ambos contadores para comenzar una nueva temporización, pero no lo hace cuando las lámparas se encuentran en encendido fijo.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

entity **ESCALERA** is

```
port ( CK, RS, P : in std_logic;
      conmutador : in std_logic_vector(3 downto 0);
      luces : out std_logic);
```

end **ESCALERA**;

architecture **TEMPORIZADOR** of **ESCALERA** is

```
signal cont_1, cont_2 : std_logic_vector(3 downto 0);
```

```
-- el contador cont_1 divide por 10: pasa del reloj de 1" a 10"
```

```
-- cont_2 cuenta intervalos de 10" (cuando cont_1 está en 9)
```

```
begin
```

```
-- si cont_2 < conmutador BCD, luz fija; cuando son iguales, luz intermitente
```

```
-- la intermitencia se hace con cont_1(0): semiperíodo 1"
```

```
luces <= '1' when (cont_2 < conmutador) or ((cont_2 = conmutador) and (cont_1(0) = '1'))
      else '0';
```

```
TEMPORIZACION: process
```

```
begin wait until CK = '1';
```

```
-- inicialización RS y activación de la temporización P
```

```
if ((RS = '1') or ((P = '1') and (cont_2 >= conmutador))) then cont_2 <= "0000";
                                                                cont_1 <= "0000";
```

```
-- conteo módulo 10 para cont_1 y hasta cont_2 > conmutador BCD para ambos
```

```
elsif (cont_2 <= conmutador) then
  if (cont_1 = "1001") then cont_2 <= cont_2 + 1; cont_1 <= "0000";
  else cont_1 <= cont_1 + 1; end if;
```

```
end if; end process;
```

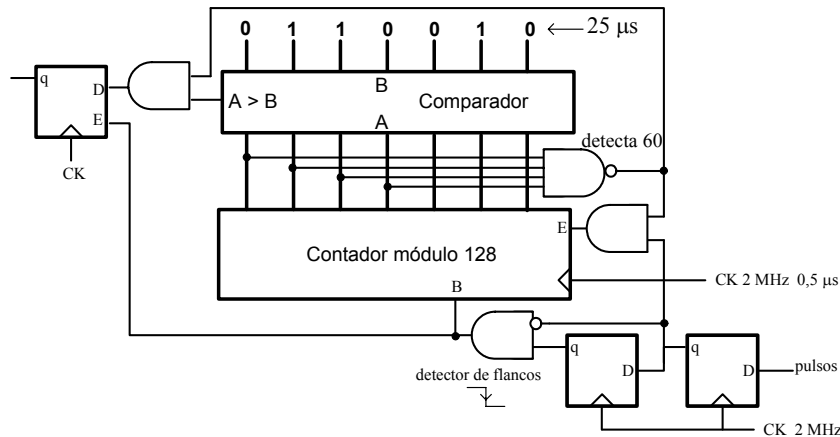
```
end TEMPORIZADOR;
```

2. Control de anchura de pulsos de entrada

Un circuito posee una única línea de entrada que recibe pulsos de diferente duración y presenta una sola línea de salida que debe situarse a **1** si la anchura del último pulso recibido es mayor que $25\ \mu\text{s}$ y menor de $60\ \mu\text{s}$, con una precisión de $0,5\ \mu\text{s}$: la salida se actualiza cuando finaliza cada pulso (flanco de bajada).

Se requiere un reloj de 2 MHz para que la precisión sea de $0,5\ \mu\text{s}$ y con dicho reloj:

$$25\ \mu\text{s} = 11001\ \mu\text{s} = 110010 \times 0,5\ \mu\text{s} \quad 60\ \mu\text{s} = 111100\ \mu\text{s} = 1111000 \times 0,5\ \mu\text{s}$$



Un contador «mide» la anchura del pulso de entrada con el reloj de 2 MHz ($0,5\ \mu\text{s}$) y la compara con $25\ \mu\text{s}$ (comparador) y con $60\ \mu\text{s}$ (puerta *Nand*); cuando llega a $60\ \mu\text{s}$ el contador se detiene para que no «se dé la vuelta» (vuelva a 0) e inicie un nuevo conteo. La comparación se ejecuta (sobre el biestable de salida) al finalizar el pulso de entrada (detección del flanco de bajada) y, a la vez, se borra el contador para la medida del siguiente pulso.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

```
entity ANCHURA is port ( CK, RS, P : in std_logic; Y : out std_logic);
end ANCHURA;
```

```
architecture MEDIR of ANCHURA is
```

```
signal cont :std_logic_vector(6 downto 0);
```

```
-- cont mide la anchura del pulso; se detiene si llega a 60 μs
```

```
-- se borra al detectarse el flanco de bajada
```

```
signal detector :std_logic_vector(2 downto 1);
```

```
signal flanco :std_logic;
```

```
-- «detector» son dos biestables para detectar el flanco de bajada:
```

```
-- cuando detector(2) = 1 y detector(1) = 0, entonces flanco = 1
```

```
signal yy :std_logic;
```

```
-- yy es el biestable de salida que se activa cuando flanco = 1 y 25 < contador < 60
```

```
begin
```

```
Y <= yy; flanco <= detector(2) and not detector(1);
```

```
RELOJ: process
```

```
begin wait until CK = '1';
```

```
detector(1) <= P; detector(2) <= detector(1);
```

```
-- medida de la anchura de pulso
```

```
if (( RS = '1' ) or (flanco = '1')) then
```

```
cont <= "0000000";
```

```
elsif (cont < "1111000") and (detector(1) = '1') then
```

```
cont <= cont + 1;
```

```
end if;
```

```
-- ejecución de las comparaciones para dar lugar a la salida
```

```
if flanco = '1' then
```

```
if (cont >= "0110010") and (cont < "1111000") then
```

```
yy <= '1';
```

```
else
```

```
yy <= '0'; end if;
```

```
end if;
```

```
end process;
```

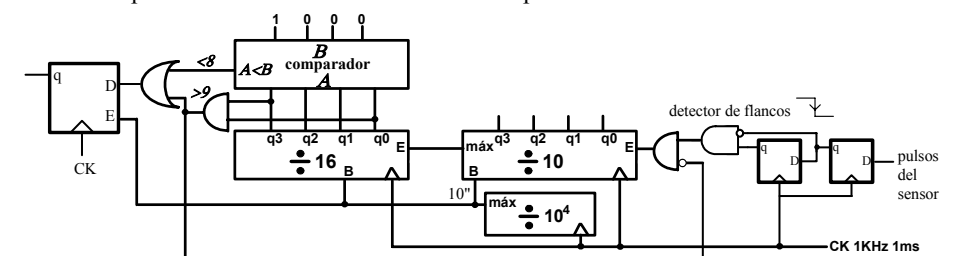
```
end MEDIR;
```

3. Supervisión de velocidad de un motor

Un motor debe girar con un mínimo de 480 r.p.m. hasta un máximo de 540 (que no debe ser alcanzado); el circuito que vigile la velocidad del motor recibirá pulsos de un sensor que detecta las vueltas del motor y debe proporcionar una señal de alarma cuando se encuentre fuera de dicho intervalo. Se utilizará una frecuencia de reloj de 1 KHz.

Corresponde a un frecuencímetro con una sola salida (de alarma), referida a la comparación de la medida de frecuencia con las dos frecuencias extremas; tomando un tiempo de medida de 10 segundos:

$$480\ \text{r.p.m.} = 80\ \text{rev. en } 10'' = 1000 \quad 540\ \text{r.p.m.} = 90\ \text{rev. en } 10'' = 1001$$



Ahora lo que se cuentan son milisegundos (se mide el período) y dicho conteo se hace directamente en binario y se compara con 125 y con 111; cuando se alcanza el 125 se detiene el contador para evitar que «se dé la vuelta». Las comparaciones se ejecutan al finalizar cada vuelta (flanco de bajada de los pulsos del sensor) y, a la vez, se borra el contador para un nuevo conteo.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;

entity MOTOR2 is port ( CK, RS, S : in std_logic; Y : out std_logic);
end MOTOR2;

architecture PERIODO of MOTOR2 is

function mod10 (e :std_logic; qq :std_logic_vector(3 downto 0)) return std_logic_vector is
    variable cont:std_logic_vector(3 downto 0);
    begin
        if e = '1' then
            if qq = "1001" then cont := "0000"; else cont := qq + 1; end if;
            else cont := qq; end if;
    return cont; end;

    signal contador :std_logic_vector(6 downto 0);
    signal q3, q2, q1 :std_logic_vector(3 downto 0);
    signal max3, max2, max1, miliseg :std_logic;

    signal detector :std_logic_vector(2 downto 1);
    signal flanco :std_logic;
    signal yy :std_logic;

    begin
        Y <= yy; flanco <= detector(2) and not detector(1);

        max1 <= '1' when q1 = "1001" else '0';
        max2 <= '1' when (q2 = "1001") and max1 = '1' else '0';
        max3 <= '1' when (q3 = "1001") and max2 = '1' else '0'; miliseg <= max3;

    RELOJ: process
    begin wait until CK = '1';
        detector(1) <= S; detector(2) <= detector(1);
        q1 <= mod10('1', q1); q2 <= mod10(max1, q2); q3 <= mod10(max2, q3);

        -- contaje de milisegundos
        if (( RS = '1' ) or (flanco = '1')) then contador <= "0000000";
        elsif (miliseg = '1') and (contador < "1111101") then contador <= contador +1;
        end if;
```

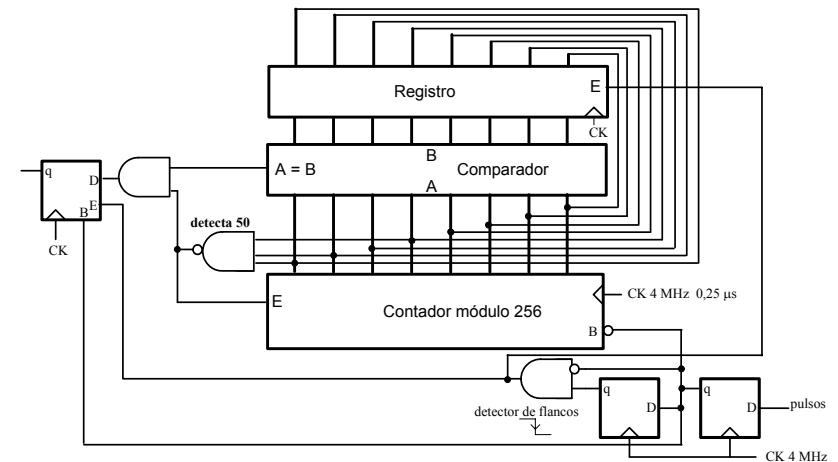
-- ejecución de las comparaciones al finalizar cada vuelta del motor

```
if flanco = '1' then
    if (contador < "1110000") or (contador >= "1111101") then yy <= '1';
    else yy <= '0'; end if;
end if; end process; end PERIODO ;
```

4. Comparación de anchura de pulsos sucesivos

Interesa disponer de un circuito con una entrada por la que recibirá pulsos de duración variable y una salida que debe avisar cuando los dos últimos pulsos recibidos sean iguales. El circuito será síncrono con un reloj de 4 MHz y el aviso (salida a **1**) se producirá cuando finalice cada pulso hasta que llegue el nuevo pulso; pulsos de duración igual o superior a 50 μ s no son tenidos en cuenta.

Con un reloj de 4 MHz (0,25 μ s): 50 μ s = **110010** μ s = **11001000** x 0,25 μ s.



El contador «mide» la anchura de los pulsos y el registro almacena la medida del pulso anterior. La comparación entre contador (pulso de entrada) y registro (pulso anterior) y el almacenamiento en el registro se ejecutan cuando acaba cada pulso (detección de su flanco de bajada). El contador permanece borrado mientras no hay pulso de entrada y, en cambio, el biestable de salida se borra cuando existe pulso de entrada.

```
library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;
```

```
entity COMPARACION is
port ( CK, RS, P : in std_logic; Y : out std_logic);
end COMPARACION ;
```

architecture ANCHURA of COMPARACION is

signal contador, registro :std_logic_vector(7 downto 0);
 -- contador y registro son de 8 bits (para abarcar los 50 μ s, medidos con 4 MHz)

signal detector :std_logic_vector(2 downto 1);
 signal flanco :std_logic;
 -- detector son dos biestables para detectar el flanco de bajada:
 -- cuando detector(2) = 1 y detector(1) = 0, entonces flanco = 1

signal yy :std_logic;
 -- yy es el biestable de salida

begin

Y <= yy; flanco <= detector(2) and not detector(1);

RELOJ: process

begin wait until CK = '1';

detector(1) <= P; detector(2) <= detector(1);

-- medida de la anchura de pulso: máximo 50 μ s

if ((RS = '1') or (detector(1) = '0')) then contador <= "00000000";
 elsif contador < "11001000" then contador <= contador + 1;
 end if;

-- señal de salida: ejecución de la comparación al finalizar un pulso

if detector(1) = '1' then yy <= '0';
 elsif flanco = '1' then
 if (contador < "11001000") and (contador = registro) then yy <= '1';
 else yy <= '0'; end if;
 end if;

-- paso de la medida del pulso al registro para la comparación con el siguiente pulso

if flanco = '1' then registro <= contador;

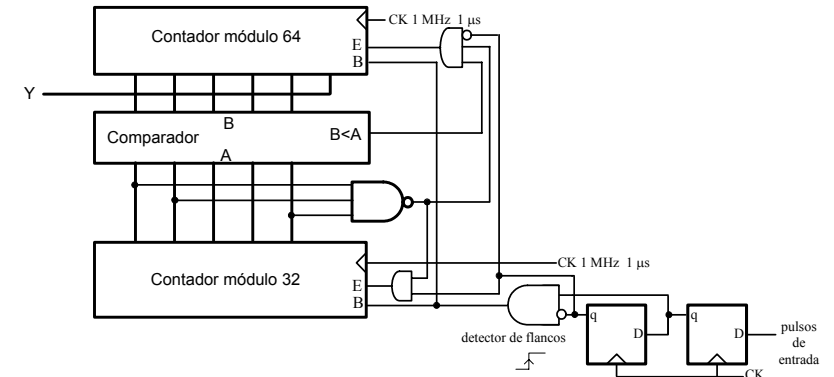
end if;

end process;

end ANCHURA;

5. Generación de n pulsos

Un sistema síncrono con un reloj de 1 MHz recibe pulsos de diferente duración, separados, al menos, 50 μ s; el sistema mide la anchura del pulso recibido (n μ s) y, si n es inferior a 25 μ s, genera n pulsos (tantos como la propia duración del pulso de entrada) de 1 μ s, separados 1 μ s cada uno del siguiente.



La primera salida **q0** de un contador completo genera pulsos cuyos semiperíodos (tiempo en 1 y en 0) son iguales al período del reloj; las salidas siguientes del mismo contador **q5 ... q1** «cuentan» el número de dichos pulsos.

De esta forma, un contador (módulo 32) mide la anchura de los pulsos de entrada y otro contador (módulo 64) genera los pulsos de salida (**q0**) y los cuenta. Ambos contadores se borran al inicio de un nuevo pulso de entrada (detección de flanco de subida). La condición para que se generen los pulsos de salida (habilitación del segundo contador) es que haya finalizado el pulso de entrada, que su anchura haya sido inferior a 25 μ s y que el número de pulsos de salida sea aún menor que la anchura del pulso de entrada (comparador).

25 μ s = **11001** μ s.

library ieee; use ieee.std_logic_1164.all; use ieee.std_logic_unsigned.all;

entity PULSOS is

port (CK, RS, P : in std_logic; Y : out std_logic);
 end PULSOS;

architecture GENERAR of PULSOS is

signal cont1 :std_logic_vector(4 downto 0);

-- cont1 mide la anchura del pulso de entrada; se detiene si llega a 25 μ s

signal cont2 :std_logic_vector(5 downto 0);

-- cont2 genera los pulsos de salida y los cuenta

```

signal detector      :std_logic_vector(2 downto 1);
signal flanco        :std_logic;

-- detector son dos biestables para detectar el flanco de subida:
-- cuando detector(2) = 0 y detector(1) = 1, entonces flanco = 1

begin

Y <= cont2(0);      flanco <= not detector(2) and detector(1);

RELOJ: process
begin
    wait until CK = '1';
    detector(1) <= P;          detector (2) <= detector(1);
    if (RS = '1') or (flanco = '1') then
        cont1 <= "000000";          cont2 <= "000000";

-- contaje del primer contador: medida de la anchura del pulso de entrada (máximo 25)
    elsif detector(2) = '1' then
        if (cont1 < "11001") then
            cont1 <= cont1 + 1;      end if;

-- segundo contador: generación de los pulsos de salida y conteo de los mismos
    else if (cont1 < "11001") and (cont2(5 downto 1) < cont1) then
        cont2 <= cont2 + 1;
    end if;
end if;
end process;

end GENERAR ;

```

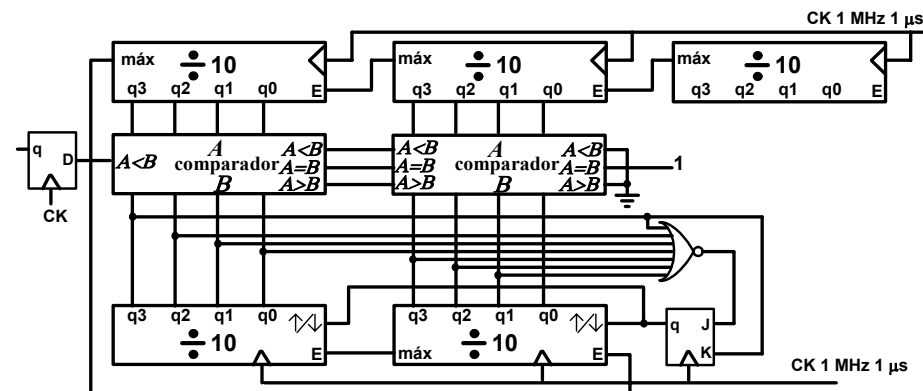
6. Pulsos de anchura progresivamente creciente/decreciente

Interesa un circuito digital que proporcione pulsos de amplitud modulada en la forma siguiente: un pulso cada milisegundo, el primero de ellos de 10 μ s y los siguientes con anchura creciente de 20 μ s, 30 μ s, ... (cada uno 10 μ s más largo que el anterior), hasta alcanzar los 800 μ s; luego los pulsos decrecerán (de 10 en 10 μ s) hasta reducirse a 10 μ s, momento en el cual volverán a crecer; y así sucesivamente.

Se trata de configurar un modulador de anchura de pulso: un contador que produzca el ciclo on/off y un comparador que compare el estado del contador con una referencia. Dicha referencia ha de ir creciendo, ciclo a ciclo, desde 10 μ s hasta 800 μ s (con un paso de 10 μ s), mediante un contador bidireccional cuyo sentido de conteaje sea controlado por un biestable; éste debe ponerse a **0** (borrarse) cuando la referencia llegue a 800 μ s en el conteaje ascendente y debe pasar a **1** (marcarse) cuando el «descontaje» llega a 10 μ s.

Si ambos contadores (el del ciclo y el de la referencia) cuentan en BCD y se refieren a un intervalo base de 10 μ s: 10 μ s = **0000 0001** x 10 μ s, 800 μ s = **1000 0000** x 10 μ s.

Con reloj de 1 MHz hará falta otro contador década para la temporización en 10 μ s. El contador del ciclo PWM será habilitado por esta señal de 10 μ s y de módulo 100 para que el ciclo sea de 1 ms (100 x 10 μ s); el contador que genera la referencia se habilitará con la salida de máximo del anterior (que determina el final de cada ciclo).



```
library ieee;    use ieee.std_logic_1164.all;    use ieee.std_logic_unsigned.all;
```

```
entity PWM is port ( CK, RS : in std_logic; Y : out std_logic);
end PWM;
```

architecture VARIA of PWM is

-- *contaje módulo 10, que puede ser bidireccional: hay 5 contadores década en el diseño*

-- rs actúa como inicialización, dir como control arriba/abajo y e como habilitación

```
function mod10 (rs, dir, e :std logic; qq :std logic vector(3 downto 0))
```

return std::logic_vector{is}

```
variable contador :std_logic_vector(3 downto 0);
```

begin

```
if rs = '1' then contador := "0000";
```

```
elseif e = '1' then
```

```
if dir = '1' then if qq = "1001" then contador := "0000";  
                  else contador := qq + 1;                end if;
```

```

else      if qq = "0000" then contador := "1001";
           else contador := qq - 1;           end if;  end if;

```

```
else contador := qq;   end if;
```

```
return contador; end;
```

```
signal cont1, cont2, cont3, cont4, cont5      :std logic vector(3 downto 0);
```

```
signal max1, max2, ciclo, m m4          :std logic;
```

-- contadores módulo 10: 1, 2 y 3 generan el ciclo PWM y 4 y 5 la referencia

```
signal dir, yy                                :std_logic;
```

-- 2 biestables: dir controla el contejo up/down de la referencia; yy sincroniza la salida

```
signal uno                                     :std_logic;
```

-- señal para establecer un 'l'

```

begin
Y <= yy;
uno <= '1';

-- salidas para la interconexión de los 5 contadores
max1 <= '1' when cont1 = "1001" else '0';
max2 <= '1' when (cont2 = "1001") and max1 = '1' else '0';
ciclo <= '1' when (cont3 = "1001") and max2 = '1' else '0';
-- "ciclo" señala el final del ciclo PWM ( corresponde a max3)

m_m4 <= '1' when ciclo = '1' and ( ((cont4 = "1001") and (dir = '1'))
                                or ((cont4 = "0000") and (dir = '0')) ) else '0';
-- "m_m4" es la señal de máximo/mínimo de cont4, necesaria para habilitar cont5

RELOJ: process
begin
wait until CK = '1';

-- inserción de los 5 contadores
-- cont1, cont2 y cont3 que forman el contador de ciclo son ascendentes
cont1 <= mod10(RS, uno, uno, cont1);
cont2 <= mod10(RS, uno, max1, cont2);
cont3 <= mod10(RS, uno, max2, cont3);
-- cont4 y cont5 que generan la referencia son bidireccionales (dir)
cont4 <= mod10(RS, dir, ciclo, cont4);
cont5 <= mod10(RS, dir, m_m4, cont5);

-- biestable que controla el sentido de variación de la referencia: contaje/descontaje
if (RS = '1') or (cont5 = "1000") then      dir <= '0';
elsif (cont5 = "0000") and (cont4 <= "0001") then  dir <= '1';
end if;

-- modulación de anchura de pulso: comparación entre contador y referencia
if (cont3 < cont5) or ((cont3 = cont5) and (cont2 < cont4)) then  yy <= '1';
else                                                              yy <= '0';
end if;
end process;
end VARIA;

```

7. Multiplicador de dos números de 64 dígitos

Sea un procesador dedicado específicamente a calcular el producto de dos números binarios **P** y **Q** de 64 dígitos, cuyo resultado **R** será un número binario de hasta 128 dígitos; corresponde al ejemplo de descomposición en parte operativa y de control desarrollado en el capítulo 24 (apartado 3, páginas 107 a 112).

El algoritmo aplicado consiste en recorrer sucesivamente los bits del multiplicador **Q** desde el menos significativo y, cuando el correspondiente bit de **Q** vale **1**, se suma el multiplicando **P** sobre el resultado (esta suma se efectúa sobre la parte más significativa del resultado); luego, se desplaza el resultado hacia la derecha (dividir por 2) para ajustar el valor relativo de cada suma parcial respecto a la siguiente a efectuar.

```
library ieee;          use ieee.std_logic_1164.all;          use ieee.std_logic_unsigned.all;
```

entity **MULTIPLICADOR** is

```

port (  CK, RS, Init   : in  std_logic;
        P, Q           : in  std_logic_vector(63 downto 0);
        Dis            : out  std_logic;
        R              : out  std_logic_vector(127 downto 0));

```

end **MULTIPLICADOR**;

architecture **ALGORITMICA** of **MULTIPLICADOR** is

subtype mis_estados is std_logic_vector(4 downto 1);

-- codificación de los estados: código de un solo uno

-- conforme a la máquina de estados de la página 110

```

constant reposo      : mis_estados := "0000";
constant inicio      : mis_estados := "0001";
constant intermedio   : mis_estados := "0010";
constant sumar        : mis_estados := "0100";
constant desplazar    : mis_estados := "1000";

```

signal estado : mis_estados;

signal pp,qq : std_logic_vector(63 downto 0);

-- registros para los operandos P y Q

signal rr : std_logic_vector(127 downto 0);

-- registro para el resultado R

signal cont : std_logic_vector(5 downto 0);

-- contador del número de bit del multiplicador

begin

R <= rr;

-- PARTE DE CONTROL: máquina algorítmica

CONTROL: process

```
begin      wait until CK = '1';
if ( RS = '1' ) then
    estado <= reposo;
else
    case estado is
        when reposo => if (Init = '1') then estado <= inicio; end if;
        when inicio => estado <= intermedio;
        when intermedio => if (qq(0) = '1') then estado <= sumar;
                           elsif (cont = "111111") then estado <= reposo;
                           else estado <= desplazar; end if;
        when sumar => if (cont = "111111") then estado <= reposo;
                       else estado <= desplazar; end if;
        when desplazar => estado <= intermedio;
        when others =>
    end case;
end if;
end process;
```

-- PARTE OPERATIVA: operaciones correspondientes a los diversos estados

OPERACION: process

```
begin      wait until CK = '1';
Dis <= '0'; -- valor por defecto
if ( RS = '1' ) then pp <= (others => '0'); qq <= (others => '0'); rr <= (others => '0');
               cont <= (others => '0');
else
    if estado = reposo then Dis <= '1'; end if;
    if estado = inicio then pp <= P; cont <= (others => '0');
                           qq <= Q; rr <= (others => '0'); end if;
    if estado = sumar then rr(127 downto 63) <= rr(127 downto 63) + pp;
                           end if;
    if estado = desplazar then rr <= '0' & rr(127 downto 1);
                              qq <= '0' & qq(63 downto 1);
                              cont <= cont + 1; end if;
end if;
end process;
end ALGORITMICA ;
```

8. Raíz cuadrada de un número de 64 dígitos

El resultado **R** será un número binario de hasta 32 dígitos; el algoritmo de cálculo y la correspondiente máquina de estados se encuentran detalladas 24.4 (páginas 116 a 119).

El algoritmo aplicado consiste en recorrer sucesivamente los bits del operando, de dos en dos (comenzando por los más significativos), y restando, cuando se pueda, el resultado parcial **R** con el añadido **01** al final; en cada desplazamiento, se añade un **1** al resultado si se efectúa la resta y un **0** cuando ésta no es posible.

```
library ieee;      use ieee.std_logic_1164.all;      use ieee.std_logic_unsigned.all;
```

entity **RAIZ** is

```
port (   CK, RS, Init   : in  std_logic;
        P               : in  std_logic_vector(63 downto 0);
        Dis             : out  std_logic;
        R               : out  std_logic_vector(31 downto 0));
```

end **RAIZ** ;

architecture **CUADRADA** of **RAIZ** is

-- declaración de estados

```
type mis_estados is ( reposo, inicio, desplazar1,desplazar2,
                      intermedio, restar, desplazar);
```

```
signal estado      :mis_estados;
```

```
signal pp          : std_logic_vector(63 downto 0);
```

-- registros para el operando P

```
signal rr          : std_logic_vector(31 downto 0);
```

-- registro para el resultado R

```
signal ss          : std_logic_vector(33 downto 0);
```

-- registro para el resto actual

```
signal cont        : std_logic_vector(5 downto 0);
```

-- contador de número de parejas de bits del operando

```
begin
```

```
R <= rr;
```

-- PARTE DE CONTROL: máquina algorítmica

CONTROL: process

```
begin
```

```
wait until CK = '1';
```

```
if ( RS = '1' ) then                                estado <= reposo;
```

```

else case estado is
    when reposo => if (Init = '1') then estado <= inicio; end if;
    when inicio => estado <= desplazar1;
    when desplazar1 => estado <= desplazar2;
    when desplazar2 => estado <= intermedio;
    when intermedio => if ss < (rr & "01") then estado <= desplazar;
                        else estado <= restar; end if;
    when restar => if cont(5) = '1' then estado <= reposo;
                    else estado <= desplazar1; end if;
    when desplazar => if cont(5) = '1' then estado <= reposo;
                        else estado <= desplazar1; end if;

    when others =>
end case;
end if;
end process;

```

-- PARTE OPERATIVA: operaciones correspondientes a los diversos estados

OPERACION: process

```

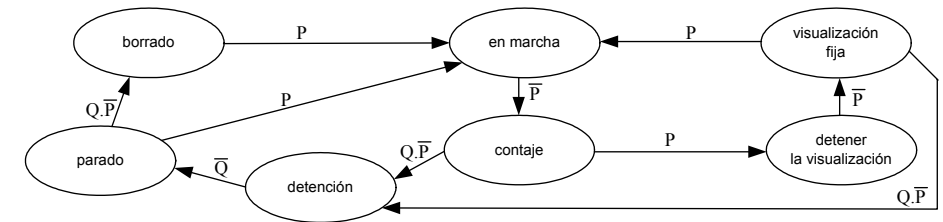
begin
wait until CK = '1';
Dis <= '0';
if ( RS = '1' ) then pp <= (others => '0'); ss <= (others => '0'); rr <= (others => '0');
                    cont <= (others => '0');
else
    if estado = reposo then Dis <= '1'; end if;
    if estado = inicio then pp <= P; cont <= (others => '0');
                          ss <= (others => '0'); rr <= (others => '0');
    end if;
    if estado = desplazar1 then ss <= ss(32 downto 0) & pp(63);
                                pp <= pp(62 downto 0) & '0';
                                cont <= cont + 1;
    end if;
    if estado = desplazar2 then ss <= ss(32 downto 0) & pp(63);
                                pp <= pp(62 downto 0) & '0';
    end if;
    if estado = restar then ss <= ss - (rr & "01");
                           rr <= rr(30 downto 0) & '1'; end if;
                           rr <= rr(30 downto 0) & '0'; end if;
    if estado = desplazar then rr <= rr(30 downto 0) & '0'; end if;
end if;
end process;
end CUADRADA;

```

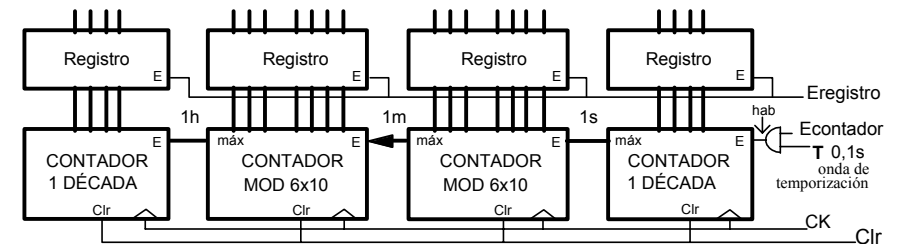
9. Cronómetro controlado por dos pulsadores

Se desea controlar un cronómetro mediante dos pulsadores **P** y **Q** de forma que el conteo de tiempo se inicie al activar **P** y se detenga al pulsar **Q** y un segundo pulso de **Q** sirva para poner a cero el cronómetro; una vez detenido el conteo, si se pulsa **P** se reanuda el mismo. Si mientras el cronómetro está activo se activa **P**, el conteo prosigue pero el visualizador queda detenido en el último valor previo a dicho pulso; se vuelve a la visualización normal del conteo pulsando nuevamente **P**.

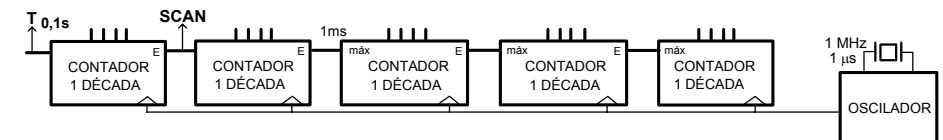
Este ejemplo corresponde al desarrollado en el capítulo 24 (apartado 4, páginas 126 y 127); el control responde al siguiente grafo de estados:



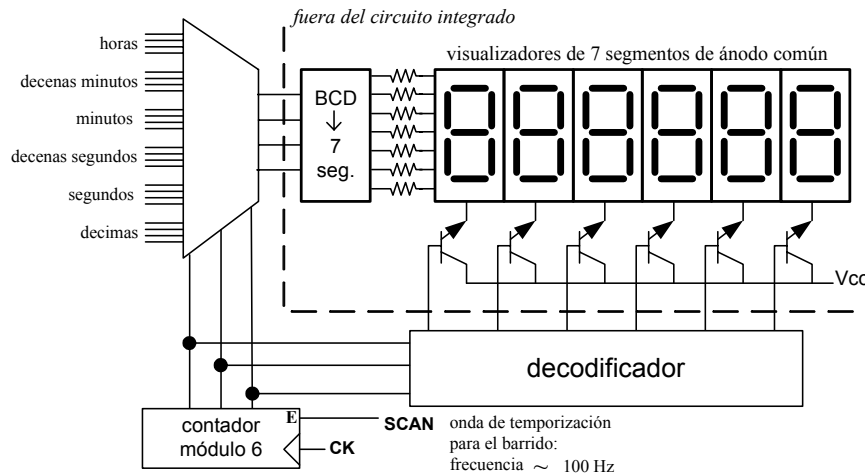
La siguiente figura muestra el diagrama de bloques de la parte operativa:



Como reloj se utilizará un oscilador de precisión con cristal de cuarzo de 1 MHz; por división de esa frecuencia (mediante 5 décadas) se obtendrá la onda de temporización de 0,1 segundo (100 Hz) que es la unidad de tiempo para el contador de la figura anterior. Asimismo, se obtendrá la señal de temporización para el barrido de los visualizadores (véase la figura de la página siguiente) que será de 100 Hz.

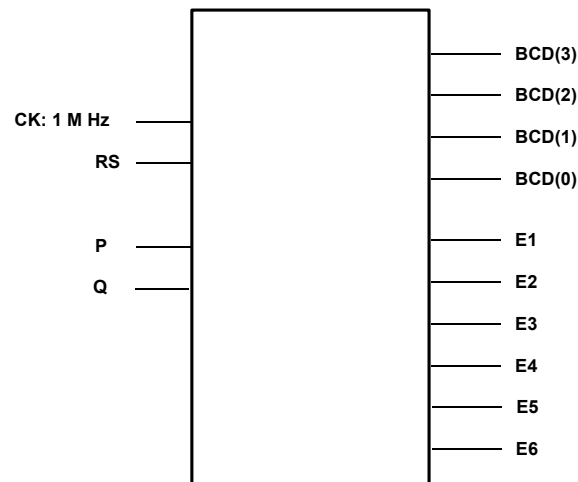


A fin de reducir líneas de conexión (6 cifras BCD requieren 24 líneas, que pasan a ser 42 después de los conversores a 7 segmentos), la representación sobre el visualizador será de tipo dinámico, según el esquema de multiplexado que sigue:



El circuito integrado, supuesto el conversor a 7 segmentos fuera del mismo, tendrá los siguientes terminales:

- entradas: reloj CK, inicialización RS y los dos pulsadores P y Q;
- salidas: 4 comunes para las cifras BCD y 6 para el barrido de los 6 visualizadores.



```
library ieee;          use ieee.std_logic_1164.all;          use ieee.std_logic_unsigned.all;
```

entity **CRONOMETRO** is

```
port (   CK, RS      : in  std_logic;
        P, Q         : in  std_logic;
        BCD          : out  std_logic_vector(3 downto 0);
        E            : out  std_logic_vector(6 downto 1));
```

end **CRONOMETRO**;

architecture **PULSADORES** of **CRONOMETRO** is

-- *contaje módulo 10*

```
function mod10 (e1 :std_logic; q1:std_logic_vector) return std_logic_vector is
    variable contador1 :std_logic_vector(3 downto 0);
    begin
        if e1 = '1' then
            if q1 = "1001" then contador1 := "0000";
            else contador1 := q1 + 1;      end if;
        else contador1 := q1;              end if;
    return contador1; end;
```

-- *contaje módulo 6*

```
function mod6 (e2 :std_logic; q2:std_logic_vector) return std_logic_vector is
    variable contador2 :std_logic_vector(6 downto 4);
    begin
        if e2 = '1' then
            if q2 = "101" then contador2 := "000";
            else contador2 := q2 + 1;      end if;
        else contador2 := q2;              end if;
    return contador2; end;
```

```
signal seg, min          : std_logic_vector(6 downto 0);
signal dec, hor          : std_logic_vector(3 downto 0);
signal max1, max2, max3, max4,max5 : std_logic;
```

-- *contadores del propio cronómetro (medida de tiempo)*

```
signal regseg, regmin    : std_logic_vector(6 downto 0);
signal regdec, reghor    : std_logic_vector(3 downto 0);
```

-- *registros del propio cronómetro (conectados a los contadores anteriores)*

-- *declaración de estados de la parte de control*

```
type mis_estados is ( borrado, en_marcha, contaje, detencion, parado,
                      detener_vis, vis_detenida);
```

```
signal estado            :mis_estados;
```

```
signal Clr, Econtador, Eregistro, hab :std_logic;
```

-- *señales de salida de la parte de control que actúan sobre el propio cronómetro*


```

signal div1, div2, div3, div4, div5      : std_logic_vector(3 downto 0);
signal maxdiv1, maxdiv2, maxdiv3, scan, decima : std_logic;
-- contadores del divisor de la frecuencia del reloj

```

```

signal contscan                          : std_logic_vector(6 downto 4);
-- contador para el barrido de la representación dinámica (módulo 6)

```

```
begin
```

-- CONTADORES Y REGISTROS DEL PROPIO CRONOMETRO

```
hab <= decima and Econtador;    -- habilitación global del contador de tiempo
```

```
-- señal de máximo para conectar los contadores sucesivamente
```

```

max1 <= '1' when dec(3 downto 0) = "1001" and hab = '1' else '0';
max2 <= '1' when seg(3 downto 0) = "1001" and max1 = '1' else '0';
max3 <= '1' when seg(6 downto 4) = "101" and max2 = '1' else '0';
max4 <= '1' when min(3 downto 0) = "1001" and max3 = '1' else '0';
max5 <= '1' when min(6 downto 4) = "101" and max4 = '1' else '0';

```

```
-- inserción de los contadores
```

```
CONTADORES: process(RS, CK)
```

```
begin
```

```

if (RS = '1') or (Clr = '1') then    dec <= (others => '0');    seg <= (others => '0');
                                     min <= (others => '0');    hor <= (others => '0');

```

```
elsif CK'event and CK = '1' then
```

```

    dec(3 downto 0) <= mod10(hab, dec(3 downto 0));
    seg(3 downto 0) <= mod10(max1, seg(3 downto 0));
    seg(6 downto 4) <= mod6(max2, seg(6 downto 4));
    min(3 downto 0) <= mod10(max3, min(3 downto 0));
    min(6 downto 4) <= mod6(max4, min(6 downto 4));
    hor(3 downto 0) <= mod10(max5, hor(3 downto 0));

```

```
end if;
```

```
end process;
```

```
-- inserción de los registros
```

```
REGISTROS: process(RS, CK)
```

```
begin
```

```

if RS = '1' then    regdec <= (others => '0');    regseg <= (others => '0');
                  regmin <= (others => '0');    reghor <= (others => '0');

```

```
elsif CK'event and CK = '1' then
```

```

    if Eregistro = '1' then    regdec <= dec;    regseg <= seg;
                              regmin <= min;    reghor <= hor;
    end if;

```

```
end if;
```

```
end process;
```

-- CONTROL A TRAVÉS DE LOS DOS PULSADORES

```
-- PARTE DE CONTROL: máquina de estados
```

```
CONTROL: process
```

```
begin    wait until CK = '1';
```

```
if ( RS = '1' ) then
```

```
    estado <= borrado;
```

```
else    case estado is
```

```
        when borrado =>    if (P = '1') then    estado <= en_marcha;    end if;
```

```
        when en_marcha =>    if (P = '0') then    estado <= contaje;    end if;
```

```
        when contaje =>    if (P = '1') then    estado <= detener_vis;
                           elsif (Q = '1') then    estado <= detencion;    end if;
```

```
        when detencion =>    if (Q = '0') then    estado <= parado;    end if;
```

```
        when parado =>    if (P = '1') then    estado <= en_marcha;
                           elsif (Q = '1') then    estado <= borrado;    end if;
```

```
        when detener_vis =>    if (P = '0') then    estado <= vis_detenida;    end if;
```

```
        when vis_detenida =>    if (P = '1') then    estado <= en_marcha;
                           elsif (Q = '1') then    estado <= detencion;    end if;
```

```
        when others =>
```

```
            estado <= borrado;
```

```
    end case;
```

```
end if;
```

```
end process;
```

```
-- PARTE OPERATIVA: señales correspondientes a los diversos estados
```

```
SEÑALES: process
```

```
begin
```

```
Clr <= '0';    Econtador <= '0';    Eregistro <= '0';    -- valor por defecto
```

```
case estado is
```

```
    when borrado =>    Clr <= '1';    Eregistro <= '1';
```

```
    when en_marcha =>    Econtador <= '1';    Eregistro <= '1';
```

```
    when contaje =>    Econtador <= '1';    Eregistro <= '1';
```

```
    when detencion =>
```

```
        when parado =>
```

```
            when detener_vis =>    Econtador <= '1';
```

```
            when vis_detenida =>    Econtador <= '1';
```

```
        when others =>
```

```
end case;
```

```
end process;
```

-- DIVISIÓN DE FRECUENCIAS DEL RELOJ

-- señal de máximo para conectar los contadores sucesivamente

```

maxdiv1 <= '1' when div1(3 downto 0) = "1001" else '0';
maxdiv2 <= '1' when div2(3 downto 0) = "1001" and maxdiv1 = '1' else '0';
maxdiv3 <= '1' when div3(3 downto 0) = "1001" and maxdiv2 = '1' else '0';
scan    <= '1' when div4(3 downto 0) = "1001" and maxdiv3 = '1' else '0';
decima  <= '1' when div5(3 downto 0) = "101"  and scan = '1' else '0';

```

-- inserción de los divisores por 10

```

DECADAS: process(RS, CK)      begin
if RS = '1' then
    div1 <= (others => '0');   div2 <= (others => '0');
    div3 <= (others => '0');   div4 <= (others => '0');
    div5 <= (others => '0');

elsif CK'event and CK = '1' then
    div1 <= mod10('1', div1);   div2 <= mod10(maxdiv1, div2);
    div3 <= mod10(maxdiv2, div3); div4 <= mod10(maxdiv3, div4);
    div5 <= mod10(scan, div5);

end if;
end process;

```

-- BARRIDO DINAMICO DE LOS VISUALIZADORES

-- contador que realiza el barrido

```

MODULO6: process(RS, CK)      begin
if RS = '1' then
    contscan <= (others => '0');

elsif CK'event and CK = '1' then
    contscan <= mod6(scan, contscan);

end if;
end process;

```

-- multiplexado

```

MULTIPLEXADO: process(contscan) begin
case contscan is
    when "000" =>   BCD <= regdec;           E <= "000001";
    when "001" =>   BCD <= regseg(3 downto 0); E <= "000010";
    when "010" =>   BCD <= '0' & regseg(6 downto 4); E <= "000100";
    when "011" =>   BCD <= regmin(3 downto 0); E <= "001000";
    when "100" =>   BCD <= '0' & regmin(6 downto 4); E <= "010000";
    when "101" =>   BCD <= reghor;           E <= "100000";
    when others =>   BCD <= "0000";          E <= "000000";

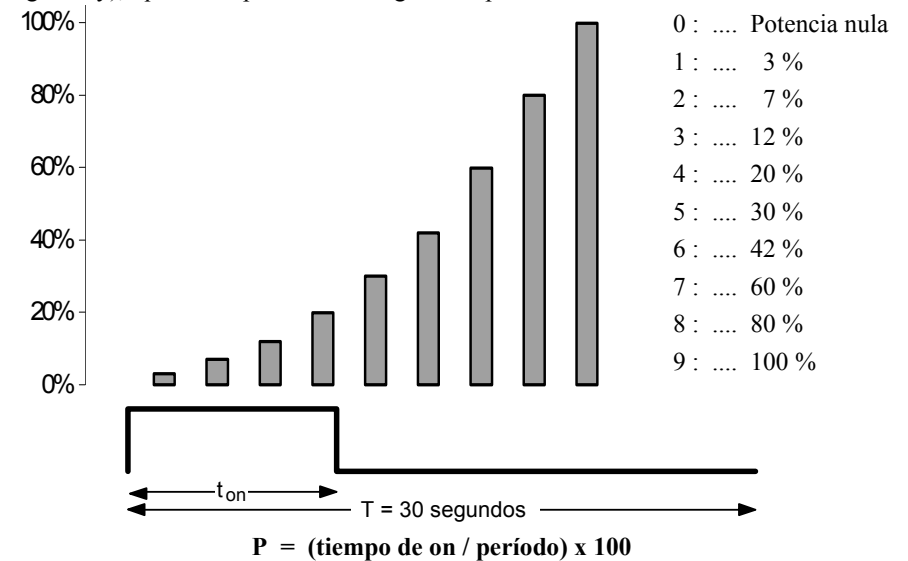
end case;
end process;

end PULSADORES;

```

10. Control on/off para una placa térmica vitrocerámica

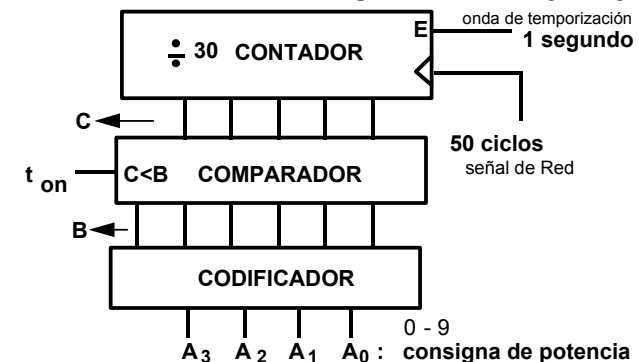
Consideramos una «cocina vitrocerámica» de cuatro «fuegos»; para controlar la potencia suministrada a la resistencia calefactora de cada fuego, se utiliza un ciclo de aproximadamente 30 segundos dividido en 30 pasos de 1" y, para fijar la potencia a suministrar, se emplea un mando circular de control con 10 posiciones (de 0 a 9 en código Gray), que corresponden a las siguientes potencias relativas:



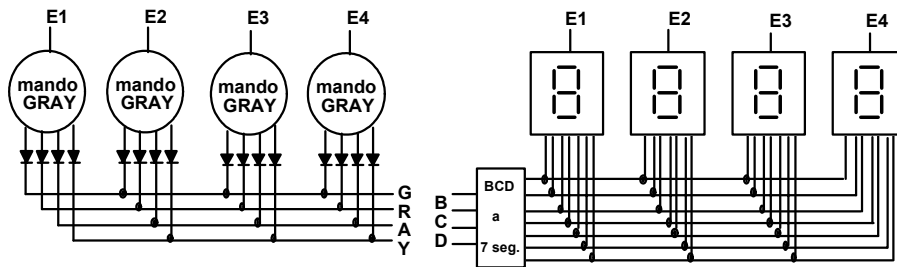
La anterior distribución de potencias (relativas) corresponde, en el caso de un ciclo de 30 unidades, a los siguientes intervalos activos (tiempos de on):

0 : 0	1 : 1	2 : 2	3 : 4	4 : 6
5 : 9	6 : 13	7 : 18	8 : 24	9 : 30

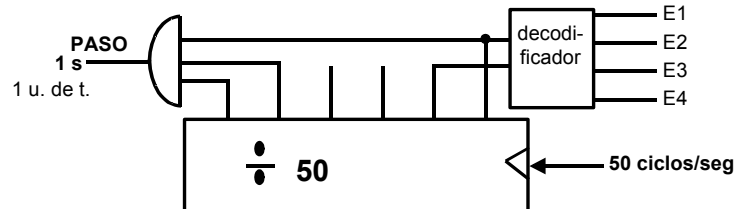
y el esquema circuital del control on/off es el representado en la figura siguiente:



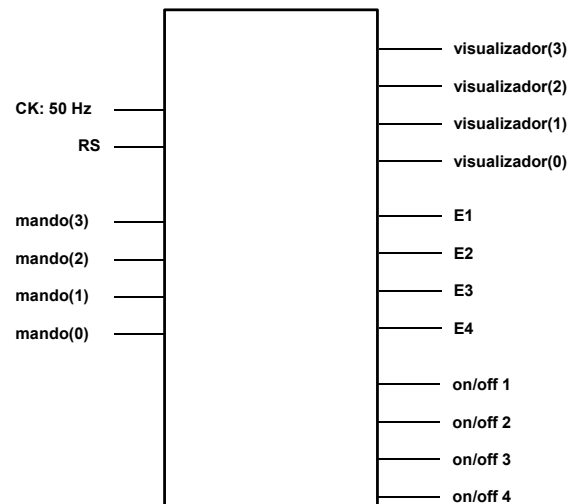
Los niveles de potencia de los cuatro «fuegos», fijados en los mandos rotativos (en código Gray) serán leídos dinámicamente, a través de 4 líneas comunes; tales niveles serán representados (en decimal, de 0 a 9) en cuatro visualizadores de siete segmentos, también en forma dinámica, a través de 4 líneas BCD:



El «barrido» para la lectura y representación dinámica será gestionado por el siguiente selector:



De manera que el circuito integrado completo tendrá los siguientes terminales (se aprovecha como reloj la propia señal de la red eléctrica: 50 Hz, 10 ms):



La descripción VHDL se hace mediante un módulo de «control», repetido para los cuatro «fuegos», que ejecuta la modulación de anchura de pulso, y un módulo «común» que realiza la temporización de 1 segundo, la lectura de los mandos y la visualización:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;
```

entity **CONTROL** is

```
port (
    CK,RS,HAB,PASO: in std_logic;
    CONSIGNA       : in std_logic_vector( 3 downto 0);
    BCD             : out std_logic_vector( 3 downto 0);
    ONOFF           : out std_logic );
```

end **CONTROL** ;

architecture **ON_OFF** of **CONTROL** is

```
signal salida      :std_logic;
signal cont, tiempo :std_logic_vector( 4 downto 0);
signal reg1, reg2   :std_logic_vector( 3 downto 0);
```

begin

```
ONOFF <= salida;      BCD <= reg2;
```

-- validación de la CONSIGNA: actúa cuando HAB = 1

-- se guarda en reg1 el valor recibido

-- y, si el siguiente valor que se recibe es igual, se almacena en reg2

```
process(RS, CK) begin
```

```
if RS = '1' then reg1 <= "0000"; reg2 <= "0000";
```

```
elsif CK'event and CK = '1' then
```

```
    if HAB = '1' then reg1 <= CONSIGNA; end if;
```

```
    if HAB = '1' and (reg1 = CONSIGNA) then reg2 <= reg1; end if;
```

```
end if;
```

```
end process;
```

-- contador base de tiempos: módulo 30

-- PASO es una onda de temporización de 1 segundo de período

```
process(RS, CK) begin
```

```
if RS = '1' then cont <= "00000";
```

```
elsif CK'event and CK = '1' then
```

```
    if PASO = '1' then if cont = "11101" then cont <= "00000";
```

```
                        else cont <= cont + 1; end if;
```

```
    end if;
```

```
end if; end process;
```

-- conversión *CONSIGNA* → *referencia (tiempo de on)*

```
process(reg2)
begin
case reg2 is
  when "0000" => tiempo <= "00000";
  when "0001" => tiempo <= "00001";
  when "0010" => tiempo <= "00011";
  when "0011" => tiempo <= "00101";
  when "0100" => tiempo <= "00111";
  when "0101" => tiempo <= "01010";
  when "0110" => tiempo <= "01111";
  when "0111" => tiempo <= "10100";
  when "1000" => tiempo <= "11000";
  when "1001" => tiempo <= "11110";
  when others => tiempo <= "00000";

end case;
end process;
```

-- *comparador del contador con el tiempo de on*

```
process(RS, CK) begin
if RS = '1' then salida <= '0';
elsif CK'event and CK = '1' then
  if tiempo > cont then salida <= '1'; else salida <= '0'; end if;
end if;
end process;
```

end **ON_OFF**;

El módulo «común» divide por 50 la frecuencia de reloj, generando una onda de temporización (PASO) de 1 segundo, efectúa la lectura multiplexada (E1, E2, E3, E4) de los cuatro mandos y la escritura multiplexada sobre los cuatro visualizadores.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all; use ieee.std_logic_unsigned.all;

entity COMUN is
  port ( CK,RS          : in std_logic;
        mando          : in std_logic_vector(3 downto 0);
        BCD1,BCD2,BCD3,BCD4 : in std_logic_vector(3 downto 0);
        visualizador    : out std_logic_vector(3 downto 0);
        E1,E2,E3,E4     : out std_logic;
        PASO            : out std_logic;
        CONSIGNA        : out std_logic_vector(3 downto 0));
end COMUN;
```

architecture **OPERATIVA** of **COMUN** is

```
signal ciclo      :std_logic;
signal cont       :std_logic_vector(5 downto 0);
signal EEEE       :std_logic_vector(1 to 4);

begin
PASO <= ciclo; E1 <= EEEE(1); E2 <= EEEE(2); E3 <= EEEE(3); E4 <= EEEE(4);

-- contador módulo 50 : generación de la señal de 1 segundo (PASO)
process(RS,CK) begin
if RS = '1' then cont <= "000000"; ciclo <= '0';
elsif CK'event and CK = '1' then
  if cont = "10111" then cont <= "000000"; ciclo <= '1';
  else cont <= cont + 1; ciclo <= '0'; end if;
end if;
end process;
```

-- *ciclo de habilitación de los 4 mandos/visualizadores*

```
with cont(1 downto 0) select EEEE <=
  "1000" when "00",
  "0100" when "01",
  "0010" when "10",
  "0001" when "11",
  "0000" when others;
```

-- *representación dinámica en los 4 visualizadores*

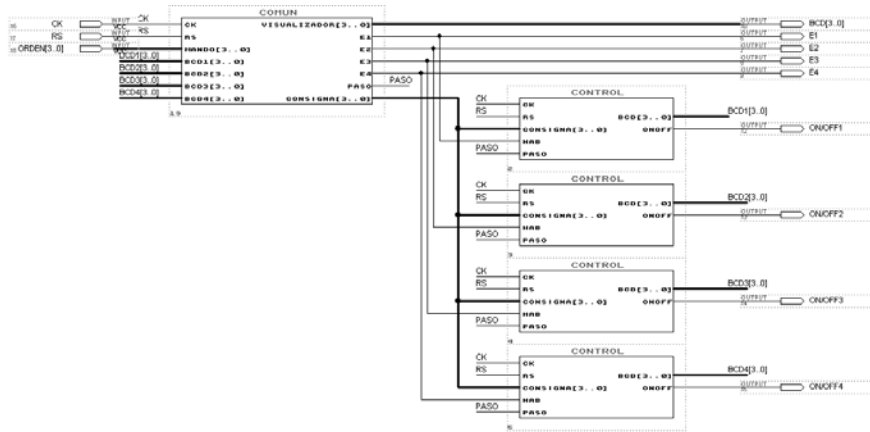
```
with cont(1 downto 0) select visualizador <=
  BCD1 when "00",
  BCD2 when "01",
  BCD3 when "10",
  BCD4 when "11",
  "0000" when others;
```

-- *conversión de la orden (gray) en consigna (BCD)*

```
process(mando) begin
case mando is
  when "0000" => CONSIGNA <= "0000";
  when "0001" => CONSIGNA <= "0001";
  when "0011" => CONSIGNA <= "0010";
  when "0010" => CONSIGNA <= "0011";
  when "0110" => CONSIGNA <= "0100";
  when "0111" => CONSIGNA <= "0101";
  when "0101" => CONSIGNA <= "0110";
  when "0100" => CONSIGNA <= "0111";
  when "1100" => CONSIGNA <= "1000";
  when "1101" => CONSIGNA <= "1001";
  when others => CONSIGNA <= "0000";

end case;
end process; end OPERATIVA;
```

La siguiente figura representa el esquemático correspondiente al circuito completo: conexión de cuatro módulos de «control» junto con un módulo «común».



La descripción VHDL (descripción estructural de conexión de módulos previamente descritos) del esquema anterior es la siguiente:

```
library ieee; use ieee.std_logic_1164.ALL;
entity VITRO is
    port ( CK,RS          : in  std_logic;
          mando          : in  std_logic_vector (3 downto 0);
          visualizador    : out std_logic_vector (3 downto 0);
          E1,E2,E3,E4     : out std_logic;
          ON_OFF1,ON_OFF2,ON_OFF3,ON_OFF4 : out std_logic);
end VITRO;

architecture CONEXIONES of VITRO is
    signal CONSIGNA,BCD1,BCD2,BCD3,BCD4 : std_logic_vector (3 downto 0);
    signal PASO                          : std_logic;
    signal E1interior,E2interior,E3interior,E4interior : std_logic;

    -- declaración de componentes: 2 componentes, COMUN y CONTROL

    component COMUN
        port ( CK,RS          : in  std_logic;
              mando          : in  std_logic_vector (3 downto 0);
              BCD1,BCD2,BCD3,BCD4 : in  std_logic_vector (3 downto 0);
              E1,E2,E3,E4,PASO : out std_logic;
              visualizador    : out std_logic_vector (3 downto 0);
              CONSIGNA        : out std_logic_vector(3 downto 0));
    end component;
```

component CONTROL

```
    port ( CK,RS,HAB,PASO : in  std_logic;
          CONSIGNA        : in  std_logic_vector (3 downto 0);
          ONOFF           : out std_logic;
          BCD             : out std_logic_vector (3 downto 0));
```

end component;

begin

```
E1 <= E1interior; E2 <= E2interior; E3 <= E3interior; E4 <= E4interior;
```

-- conexión de los módulos: 1 módulo COMUN y 4 módulos CONTROL

U1 : COMUN

```
    port map ( CK=>CK, RS => RS, PASO => PASO, CONSIGNA => CONSIGNA,
              BCD1 => BCD1, BCD2 => BCD2, BCD3 => BCD3, BCD4 => BCD4,
              E1 => E1interior, E2 => E2interior, E3 => E3interior, E4 => E4interior,
              mando => mando, visualizador => visualizador );
```

U2 : CONTROL

```
    port map ( CK => CK, RS => RS, CONSIGNA => CONSIGNA, PASO=>PASO,
              HAB => E1interior, BCD => BCD1, ONOFF => ON_OFF1 );
```

U3 : CONTROL

```
    port map ( CK => CK, RS => RS, CONSIGNA => CONSIGNA, PASO => PASO,
              HAB => E2interior, BCD => BCD2, ONOFF => ON_OFF2 );
```

U4 : CONTROL

```
    port map ( CK => CK, RS => RS, CONSIGNA => CONSIGNA, PASO => PASO,
              HAB => E3interior, BCD => BCD3, ONOFF => ON_OFF3 );
```

U5 : CONTROL

```
    port map ( CK => CK, RS => RS, CONSIGNA => CONSIGNA, PASO => PASO,
              HAB => E4interior, BCD => BCD4, ONOFF => ON_OFF4 );
```

end CONEXIONES;

A7 Apéndice al capítulo T9

Circuitos integrados mixtos

Necesidad de etapas analógicas

Los sistemas digitales, a no ser los meramente informáticos, necesitan, en algunas de sus entradas y salidas, circuitos electrónicos analógicos para relacionarse con el mundo físico, que es, básicamente, analógico. Conversores digital-analógicos o analógico-digitales, comparadores, filtros, amplificadores,... son etapas auxiliares que, muy frecuentemente, acompañan a los sistemas digitales.

Cuando se integra un sistema digital en un ASIC, dejar fuera, en la periferia, estas etapas analógicas puede suponer una limitación en cuanto a tamaño, velocidad, fiabilidad y coste. Por ello, cada vez es más frecuente integrar también la parte analógica auxiliar dentro del mismo circuito integrado, dando lugar a un circuito integrado mixto.

Un ASIC mixto es predominantemente digital (generalmente más del 90 % en cuanto a funciones o módulos), con una parte auxiliar que realiza la interfase de la parte digital con algunas entradas o salidas analógicas. A pesar de ser minoritaria, el diseño de la parte analógica suele recurrir un esfuerzo mucho mayor que el de toda la parte digital; no es extraño que, suponiendo las etapas analógicas menos del 10 % del conjunto del circuito, su diseño requiera un esfuerzo relativo superior al 90 % del total.

¿Por qué el diseño analógico es más costoso y se encuentra menos desarrollado que el diseño digital? La siguiente tabla presenta un resumen de motivos que confieren a «lo analógico» un sello de diversidad y dispersión frente a la uniformidad y regularidad de lo «digital».

<i>lo digital</i>	<i>lo analógico</i>
unos y ceros: valores extremos	valores continuos
puertas y biestables con un solo parámetro: <i>fan-out</i>	diversidad de etapas con gran número de parámetros: alimentaciones, ganancia, R_i , R_o , ancho de banda, <i>offset</i> , CMRR, ...
tres tiempos funcionales: t_p , t_s , y t_h	necesidad de estudiar su respuesta en frecuencia: polos y ceros
se aprovecha la «no estabilidad», fuera de la zona lineal: se utilizan los extremos de la alimentación	se trabaja en zona lineal y es preciso asegurar la estabilidad a través de: situación de polos y ceros, margen de fase, redes de compensación
la precisión se deriva de la utilización de dos estados diferenciados: conducción y corte	la precisión se consigue mediante técnicas de compensación: apareamiento, realimentación y componentes adicionales
hay un amplio margen para la precisión, gracias a los intervalos de cero y uno	afectan fuertemente los errores de linealidad y desplazamiento
buena inmunidad al ruido	alta sensibilidad al ruido y deriva térmica

A pesar de las dificultades derivadas de los aspectos señalados en la tabla anterior, en muchas ocasiones merece la pena insertar todo el sistema electrónico, incluidas sus etapas analógicas, en el mismo circuito integrado; en su caso, quedarán fuera las etapas de potencia, aunque va siendo viable, cada vez más, incorporar también en un ASIC etapas de media potencia.

La modulación de anchura de pulsos

Conviene tener en cuenta que las técnicas de modulación en anchura de pulso permiten reducir, en gran medida, la parte analógica de los diseños, cuando no se requieren velocidades muy altas. El manejo de la información codificada en anchura de pulso, en lugar de en amplitud, se acomoda bien a las técnicas digitales, de manera que la modulación PWM es una alternativa eficaz a procesos típicamente analógicos.

En el capítulo 18 (segundo volumen) se describen cuatro moduladores de anchura de pulso y sus aplicaciones:

- control de potencia todo/nada, en lugar de control por tensión o intensidad,
- generación de referencias de tensión y de formas de onda,
- conversión DC – DC,
- control de amplitud de señal, conformando potenciómetros digitales divisores de tensión,
- conversión digital – analógica,
- y conversión analógica – digital.

Los correspondientes circuitos PWM requieren algunos componentes analógicos, pero su control tiene una importante parte digital, en base a intervalos temporales (anchura de pulsos) que pueden ser gestionados por contadores, comparadores y otros bloques digitales. De esta forma, aunque siguen estando presentes, estas técnicas reducen mucho los componentes analógicos que se necesitan: mantener estos componentes fuera del ASIC no suele suponer una grave limitación a sus prestaciones y, en todo caso, pueden integrarse dentro con menor esfuerzo.

Componentes en las etapas analógicas

Los sistemas digitales, en tecnología CMOS, solamente requieren transistores NMOS y PMOS; ningún otro componente más. Las etapas analógicas necesitan, además, resistencias, condensadores y transistores bipolares (al menos, transistores NPN).

La integración de resistencias se hace mediante trozos alargados de regiones semiconductoras:

- en polisilicio las resistencias son de bajo valor ($< 1K$) y buena precisión ($< 10 \%$),
- la difusión P^{++} permite mayores valores (hasta $10 K$) con menor precisión (20%)
- y la difusión N del pozo ofrece resistencias de valores altos, con malos coeficientes de variación respecto a la temperatura o a la tensión aplicada (lo cual es admisible en muchos de los circuitos que necesitan resistencias grandes).

En los dos primeros casos (polisilicio y difusión P⁺) puede conseguirse una precisión del 1 % en la utilización de pares de resistencias como divisor de tensión, pues se compensan las desviaciones de ambas resistencias por estar fabricadas con las mismas características tecnológicas, en el mismo proceso de fabricación y muy cercanas entre sí.

Los condensadores integrados suelen ser de valores pequeños (< 1 nF) ya que, si fueran mayores, ocuparían enormes áreas de silicio. Se aprovecha la capacidad de puerta de los transistores NMOS (capacidad puerta sustrato P) para condensadores conectados a 0 V y de los transistores PMOS (capacidad puerta pozo N) si están conectados a V_{CC}.

Para condensadores no conectados a alimentación o de mayor precisión (del orden del 10 %) se utilizan tecnologías de dos polisilicios, configurando el condensador entre ambos con el aislante de óxido de silicio que los separa. Al igual que en el caso de las resistencias, los condensadores apareados presentan precisiones del 1 % en relación a su cociente C₁/C₂.

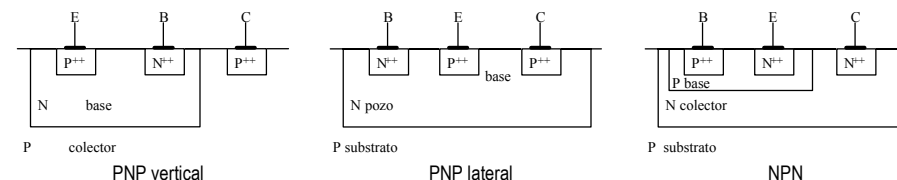
Interesa disponer, también, de transistores bipolares NPN ya que sus prestaciones son mejores que las de los transistores MOS en lo que se refiere a ganancia, intensidad, velocidad y apareamiento:

- la ganancia en tensión V_o/V_i puede ser del orden de 20 a 200 en los transistores MOS, en fuerte dependencia con su anchura W, y mayor de 2000 en los bipolares;
- la intensidad depende, en los transistores MOS, de dicha anchura W y, en cambio, es relativamente independiente de las dimensiones de los bipolares (ya que en ellos, la conducción no se debe a un efecto superficial sino volumétrico);
- la frecuencia de transición f_T, a la cual la ganancia desciende a la unidad, disminuye en los transistores MOS al aumentar su anchura y, en los bipolares, es independiente del tamaño del transistor;
- de forma que requisitos de alta ganancia o alta intensidad imponen, en los transistores MOS, una anchura W grande, lo cual reduce su velocidad de trabajo;
- en relación al apareamiento de transistores fabricados a la vez y en zonas próximas, las etapas con transistores MOS presentan tensiones de *offset* del orden de 10 mV frente a menos de 1 mV con bipolares y, además, los diversos efectos de «desapareamiento» quedan multiplicados por un factor relativo del orden de 500 en el caso MOS y del orden de 20 en el caso bipolar.

Precisamente en etapas digitales que requieren, a la vez, altas intensidades de salida y reducidos tiempos de propagación ha sido conveniente recurrir a los transistores bipolares dentro de la propia evolución de las tecnologías MOS: en su momento (apartados 10.1, panorama general de las familias lógicas integradas, y 20.4, problemas relativos a los buses) destacamos la tecnología BiCMOS (CMOS con transistores bipolares) como muy apropiada para circuitos «interbús» (necesitados de intensidades altas y mínimos tiempos).

De igual forma resultan convenientes los transistores bipolares en las múltiples configuraciones en que se aprovecha el apareamiento de transistores para compensar la falta de precisión de algunas de sus características: espejos de corriente, amplificadores y comparadores con bajo offset, conversores D/A,...

En tecnología CMOS pueden obtenerse transistores PNP parásitos «verticales» conectados a 0 V (difusión P⁺⁺ como emisor, pozo N que actúa de base y sustrato P de colector) o laterales (dos difusiones P⁺⁺, emisor y colector, en el pozo N como base), ambos de baja calidad. Utilizando una máscara más para formar la base pueden obtenerse transistores NPN (sobre un pozo N que actúa de colector, una difusión adicional P como base y una difusión N⁺⁺ sobre ella), pero presentan una resistencia alta en serie con el colector que limita mucho su funcionalidad.



Tecnología BiCMOS

Dadas las malas características de los transistores bipolares que pueden obtenerse en tecnología CMOS, se ha desarrollado una tecnología ampliada, BiCMOS, que requiere dos zonas semiconductoras adicionales y un proceso de rectificación del óxido de campo.

En esquema resumido, las etapas del proceso CMOS son las siguientes:

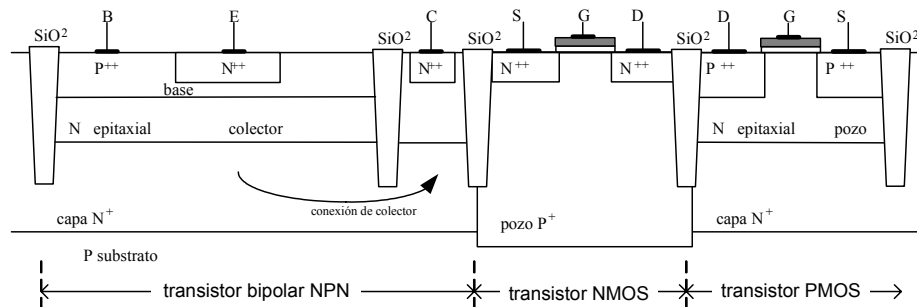
pozos N → óxido de campo → óxido de puerta y polisilicio → difusiones N⁺⁺ → difusiones P⁺⁺ → contactos → metales → pasivación

En cambio, el proceso BiCMOS pasa por las siguientes etapas (ver la figura al final de la descripción de las mismas):

- dopado superficial N⁺ sobre toda la oblea, formando una capa que será utilizada como conexión de colector de los transistores bipolares (con baja resistencia ya que su dopado es alto) y se aprovecha, también, para mejorar la separación de los pozos N respecto al sustrato P que forma la oblea;

- formación de pozos P^+ en las zonas en que irán los transistores NMOS (en tales zonas la anterior capa N^+ es reconvertida en P^+);
- crecimiento de una capa epitaxial N en toda la superficie de la oblea: esta capa actuará como colector de los transistores NPN y como pozo para los transistores PMOS y queda transformada en sustrato P en la zona que se forma encima de la anterior capa P^+ (por deriva de portadores gracias al mayor dopado P^+);
- oxidación para separar los transistores: este óxido de campo debe ser relativamente grueso ($> 1 \mu m$), parte del mismo penetra en el material (su profundidad es algo superior a la de la capa epitaxial pero sin llegar a cruzar del todo la capa inferior N^+) y otra parte sobresale respecto a la superficie de la oblea;
- proceso mecánico de rectificación del óxido de campo que sobresale, a fin de dejar una superficie lisa para los procesos siguientes (ya que se constató que las irregularidades superficiales debidas al óxido de campo producían defectos apreciables en las capas que se depositan encima del mismo);
- formación del óxido de puerta y depósito del polisilicio;
- [en tecnologías de dos polisilicios, depósito de óxido fino de aislamiento entre ellos y del segundo polisilicio encima del mismo]
- implantación P^{++} que forma los transistores PMOS y la base de los NPN;
- implantación N^{++} , menos profunda, para los transistores NMOS y el emisor de los NPN;
- contactos, metales y pasivación.

La figura siguiente representa la configuración de los tres tipos de transistores (bipolar NPN, NMOS y PMOS) en tecnología BiCMOS; en ella se han omitido las polarizaciones de los substratos (del pozo P^+ que actúa como sustrato de los NMOS y de la capa epitaxial N que lo hace para el PMOS).



Tipos de etapas y módulos analógicos

En los ASICs mixtos suelen utilizarse los siguientes tipos de configuraciones analógicas, de las cuales, las inferiores sirven también para conformar las de los niveles superiores:

→ Componentes individuales:

- resistencias
- condensadores
- transistores NPN
- transistores NMOS
- transistores PMOS
- uniones PN: diodos
- ...

→ Etapas de tipo básico:

- referencias de tensión
- espejos de corriente
- generadores de intensidad
- amplificadores diferenciales
- adaptadores de impedancias
- puertas de transmisión
- ...

→ Bloques operativos:

- amplificadores operacionales
- comparadores
- OTAs
- ...

→ Módulos de alto nivel:

- filtros
- convertidores D/A
- convertidores A/D
- osciladores controlados por tensión
- ...

A8 Apéndice a los capítulos T6, T7, T8 y T9

Simuladores eléctricos: S P I C E

Simuladores eléctricos

Un simulador es una herramienta informática que permite reproducir sobre el computador el funcionamiento de los circuitos eléctricos y electrónicos, de forma que:

- pueda compararse tal funcionamiento con el deseado hasta comprobar que el diseño funciona correctamente, que cumple las especificaciones que se desean alcanzar;
- se detecte cualquier defecto o anomalía en el funcionamiento del circuito, para corregirlo o, en su caso, verificar que es admisible;
- se conozca cualquier transitorio o cualquier particularidad funcional del circuito para evaluar sus efectos y valorar la necesidad o no de evitarlo.

Los simuladores eléctricos reproducen el comportamiento físico real de los circuitos, es decir, su comportamiento eléctrico, a través de los valores de las tensiones e intensidades en los diversos nudos y ramas del circuito a la largo del tiempo; lo cual permite conocer la respuesta real del circuito frente a las señales de entrada que interesan y, en concreto, los transitorios reales, los tiempos de propagación efectivos, los posibles espurios (*glitches*) u oscilaciones, la violación, en su caso, de los tiempos de anticipación o de mantenimiento de las entradas en las partes secuenciales, etc.

Los simuladores eléctricos actúan a nivel eléctrico: leyes de Kirckoff para el análisis de nudos y de mallas y ecuaciones $I = f(V)$ para describir el comportamiento de los componentes circuitales; han de resolver sistemas de un gran número de ecuaciones, con relaciones I-V no lineales, teniendo que efectuar análisis transitorios con cálculos punto a punto a lo largo del eje de tiempos, para lo cual requieren prolongados tiempos de cálculo (tiempos de CPU) y amplias capacidades de memoria (memoria RAM).

El análisis en términos eléctricos del comportamiento real de los circuitos:

- resulta imprescindible en la simulación de cada una de las celdas o módulos básicos de un circuito integrado (o de un diseño con componentes discretos) para poder calcular su comportamiento temporal (tiempos de propagación, anticipación, mantenimiento, etc., en el caso digital, respuesta en frecuencia, *slew rate*, etc., en el caso lineal) y para estudiar los efectos de carga, fan-out, transitorios, oscilaciones,...
- es necesario para analizar el comportamiento de temporizadores, osciladores, multivibradores,... y, en general, cualquier etapa con realimentación;
- es muy útil en la simulación de pequeños circuitos multietapa;
- es, en la práctica, la única herramienta efectiva para circuitos analógicos;
- pero presenta dificultades (por razones de tiempo y de capacidad de memoria) cuando se trata de circuitos grandes con alto número de componentes.

SPICE *Simulation Program with Integrated Circuit Emphasis* es el simulador eléctrico más difundido y utilizado; fue desarrollado en los años 70 por la Universidad de California (Berkeley) y ha sido mejorado constantemente, existiendo múltiples versiones comerciales, continuamente actualizadas por los suministradores de aplicaciones informáticas.

Descripción circuital: «net list» SPICE

La descripción del circuito se efectúa mediante su **NET LIST**, lista circuital formada por **nudos** numerados (siempre debe existir un nudo de referencia global del circuito, que se numera con 0), **componentes** conectados a los nudos, **valores** de los componentes y **modelos** de los mismos.

La primera línea de la *net list* debe contener el nombre de la misma y es ignorada (aun cuando sea, por equivocación, una línea de descripción de un componente); el final de la «lista circuital» se indica mediante la instrucción **.END**.

Ejemplos de componentes:

id = identificador, generalmente un número						
resistencia	R_{id}	nudo	nudo	valor		
condensador	C_{id}	nudo	nudo	valor		
autoinducción	L_{id}	nudo	nudo	valor		
diodo	D_{id}	nudo (P)	nudo (N)	modelo		
transistor	Q_{id}	nudo (C)	nudo (B)	nudo (E)	modelo	
MOS	M_{id}	nudo (D)	nudo (G)	nudo (S)	nudo (subst)	modelo
alimentación	V_{id}	nudo (+)	nudo (-)	valor		
señal	V_{id}	nudo	nudo	descripción de la señal de tensión		
intensidad	I_{id}	nudo	nudo	descripción del generador de intens.		
...						

Para los **valores** se puede utilizar notación exponencial o las siguientes letras:

$$\begin{aligned} \mathbf{K} &= \mathbf{E3} \equiv 10^3 & \mathbf{MEG} &= \mathbf{E6} \equiv 10^6 & \mathbf{G} &= \mathbf{E9} \equiv 10^9 & \mathbf{T} &= \mathbf{E12} \equiv 10^{12} \\ \mathbf{M} &= \mathbf{E-3} \equiv 10^{-3} & \mathbf{U} &= \mathbf{E-6} \equiv 10^{-6} & \mathbf{N} &= \mathbf{E-9} \equiv 10^{-9} & \mathbf{P} &= \mathbf{E-12} \equiv 10^{-12} & \mathbf{F} &\equiv 10^{-15} \end{aligned}$$

Los valores pueden incluir la correspondiente unidad (VOLTS, AMPS, HERZ) o abreviaturas de la misma (V, A, H), sin espacio de separación, pero tal indicación es ignorada por el simulador (por lo cual, puede ser errónea o sin sentido).

Los **modelos** pueden estar definidos en una librería: existen librerías típicas de diodos, transistores bipolares, FETs, MOSFETs, amplificadores operacionales, etc.,... que incluyen grandes series de tales dispositivos con su nombre comercial; en este caso, es necesario indicar la librería o librerías en las cuales deben buscarse los modelos mediante la instrucción:

.LIB nombre de librería.

Un modelo puede definirse también expresando los valores de los parámetros físicos del dispositivo mediante la instrucción:

.MODEL nombre_del_modelo tipo_de_dispositivo parámetros

Ejemplos: **.MODEL** 1N4148 D IS=100F BV=500

.MODEL tipoN NMOS LEVEL=1 VTO=1 K=40U TOX=5E-8

+ L=0.2E-6 LAMBDA=0.01 L=6U W=2U

El signo + al comienzo de una línea quiere decir que dicha línea es continuación directa de la anterior, es decir, son datos relativos al componente o modelo a que se refiere la línea anterior.

Pueden añadirse líneas de comentarios, comenzando los comentarios con el signo *; al reconocer dicho símbolo, SPICE ignora lo que sigue hasta final de la línea.

Generadores de señal

Un generador de tensión continua o una tensión de alimentación se expresa en la forma:

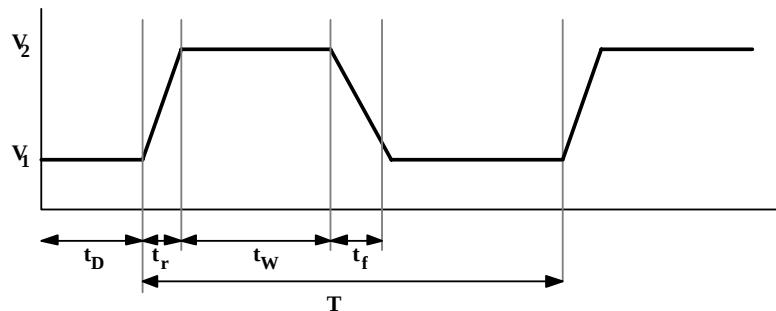
Vid nudo nudo valor_de_dicha_tensión

donde en valor_de_dicha_tensión se indicará la magnitud de la tensión en voltios.

Un generador o señal en onda cuadrada se describe en la forma siguiente:

Vid nudo nudo **PULSE**(V₁ V₂ t_D t_r t_f t_w T)

Este tipo de señal genera una onda rectangular, cuyos niveles son V₁ V₂, que se repite con período T; t_w es la anchura del pulso (en nivel V₂), t_D es el tiempo previo anterior al comienzo del primer pulso durante el cual la tensión es continua (en nivel V₁) y t_r y t_f son los tiempos de subida (paso de V₁ a V₂) y de bajada (paso de V₂ a V₁), respectivamente.



Un pulso o conjunto de pulsos también se puede definir por puntos, expresando las coordenadas (tiempo y tensión) de los puntos de conmutación:

Vid nudo nudo **PWL**(T₁ V₁ T₂ V₂ T₃ V₃ T₄ V₄ ...)

siendo T_i y V_i el tiempo y la tensión, respectivamente de cada punto i de conmutación; es una forma de descripción lineal a trozos, que corresponde a una señal compuesta exclusivamente por segmentos rectilíneos, de forma que en el intervalo que va de T_i a T_{i+1} la tensión varía linealmente de V_i a V_{i+1}.

Una tensión senoidal se describe en la forma:

Vid nudo nudo **SIN**(V_{DC} amplitud frecuencia t_D)

siendo V_{DC} la tensión de continua de la senoide y t_D el tiempo previo anterior al comienzo de la senoide durante el cual la tensión es continua según el valor V_{DC}.

La señal senoidal admite también amortiguamiento exponencial

Vid nudo nudo **SIN**(V_{DC} amplitud frecuencia t_D α)

que corresponde a la siguiente descripción:

$$V_{id} = V_{DC} \quad \text{para } t < t_D$$

$$V_{id} = V_{DC} + \text{amplitud} \cdot e^{-\alpha t'} \cdot \sin(2 \cdot \pi \cdot \text{frecuencia} \cdot t') \\ \text{para } t > t_D \quad (t' = t - t_D).$$

Asimismo, la senoide admite desfase:

Vid nudo nudo **SIN**(V_D amplitud frecuencia t_D α θ)

$$V_{id} = V_{DC} + \text{amplitud} \cdot e^{-\alpha t'} \cdot \sin[2 \cdot \pi \cdot \text{frecuencia} \cdot t' - \theta].$$

Otra posibilidad es una entrada senoidal modulada en frecuencia:

Vid nudo nudo **SFFM**(V_{DC} amplitud freq_portadora mod freq_señal)

donde mod es el índice de modulación con el cual actúa la señal sobre la portadora:

$$V_{id} = V_{DC} + \text{amplitud} \cdot \sin[2 \cdot \pi \cdot \text{freq_portadora} \cdot t' + \text{mod} \cdot \sin(2 \cdot \pi \cdot \text{freq_señal} \cdot t')]$$

Obviamente pueden definirse, también, generadores de intensidad:

Iid nudo nudo parámetros_de_la_intensidad

con iguales tipos y la misma forma de descripción que los expresados anteriormente para las tensiones.

El circuito puede contener generadores de tensión o de intensidad dependientes:

E_{id} nudo nudo nudoA nudoB factor_de_ganancia

generador de tensión controlado por una tensión (la existente entre los nudos A y B)

G_{id} nudo nudo nudoA nudoB factor_de_ganancia

generador de intensidad controlado por tensión (la existente entre los nudos A y B)

H_{id} nudo nudo V_{id} factor_de_ganancia

generador de tensión controlado por una corriente: V_{id} es un generador de tensión auxiliar (con valor de tensión nula) que se coloca en la rama por la que circula dicha corriente para detectarla.

F_{id} nudo nudo V_{id} factor_de_ganancia

generador de intensidad controlado por una corriente (la que circula por V_{id}).

Tipos de análisis

Al efectuar la simulación de un circuito, SPICE puede realizar, entre otros, los siguientes tipos de análisis:

RESPUESTA TEMPORAL: Cálculo de tensiones e intensidades en el dominio del tiempo; para ello se utiliza la instrucción:

.TRAN intervalo tiempo_de_simulación

Con esta instrucción, SPICE realiza la simulación temporal del circuito durante el tiempo indicado en el segundo parámetro (tiempo_de_simulación) y almacena los resultados cada intervalo (primer parámetro). Ello no quiere decir que los cálculos los efectúe con $\Delta t = \text{intervalo}$; la variación del tiempo, a efectos del cálculo, es sumamente fina y el simulador ajusta automáticamente el paso de simulación Δt a los valores necesarios para seguir las variaciones de tensión e intensidad con suma precisión.

RESPUESTA FRECUENCIAL: Cálculo de tensiones e intensidades en el dominio de la frecuencia, mediante las instrucciones:

.AC LIN n°_de_puntos frec_inicial frec_final

.AC OCT n°_de_puntos frec_inicial frec_final

.AC DEC n°_de_puntos frec_inicial frec_final

La primera de ellas utiliza el eje de frecuencias en forma lineal y las otras dos lo usan en forma logarítmica: por octavas y por décadas, respectivamente: n°_de_puntos indica el número de puntos, con igual espaciado entre ellos, para los cuales se efectúa el análisis; en el primer caso (LIN) es el número total de puntos entre la frecuencia inicial y la final, mientras que en los otros dos casos se refiere a número de puntos por octava o por década.

El análisis frecuencial requiere una señal de entrada de tipo **AC**

V_{id} nudo nudo **AC** amplitud

en donde el tipo AC indica que se utiliza una señal senoidal de frecuencia variable, cuya amplitud viene fijada por el parámetro siguiente. Este tipo de análisis permite representar diagramas de Bode ($20 \cdot \log(V_o/V_i)$) y, también, la variación de la fase en relación con la señal de entrada (para lo cual se utiliza el indicativo **VP(nudo)**).

BARRIDO: Respuesta en continua frente a la variación de una variable:

.DC V_a valor_inicial valor_final Δ valor

donde V_a, señal que realiza el barrido, debe ser definida mediante asignación de nudos:

V_a nudo(+) nudo(-)

y el barrido se realiza entre el valor_inicial y el valor_final con un paso de variación igual a Δ valor.

Por ejemplo, para representar la función de transferencia (V_o respecto a V_i):

V_i nudo(+) nudo(-)

.DC V_i valor_inicial valor_final Δ valor

Puede utilizarse, también, para obtener las curvas características de un componente; por ejemplo, las de un transistor MOS, I_D-V_{DS} para diversos valores de V_{GS}:

VDS nudo nudo

VGS nudo nudo

.DC VDS valor_inicial valor_final Δ valor

+ VGS valor_inicial valor_final Δ valor

Se representarán en la misma gráfica varias curvas, una para cada valor de V_{GS}.

Otras posibilidades de análisis se refieren a:

- análisis de Fourier **.FOUR** frecuencia
calcula los coeficientes de Fourier para la frecuencia indicada y ocho armónicos superiores

- cuádrupolos **.TF** V_{salida} V_{entrada}
realiza el cálculo de los parámetros del circuito considerado como un cuádrupolo

- análisis de sensibilidad **.SENS** variable
calcula el efecto porcentual que tiene sobre la variable indicada la variación del valor de cada uno de los componentes del circuito.

- análisis de ruido **.NOISE** V(nudo, nudo) fuente_de_ruido

-

El simulador SPICE realiza sus análisis a la temperatura ambiente típica, 27° C (300°K) pero puede indicarse que los realice a otra o a varias temperaturas:

.TEMP temp1 temp2 temp3 (en grados centígrados).

Asimismo se puede ordenar que repita un análisis para varios valores de una variable:

.STEP variable valor_inicial valor_final Δvalor

en este caso, los resultados para los diversos valores de la variable se representarán conjuntamente, es decir, se representarán varias curvas en la misma gráfica, una para cada valor de la variable.

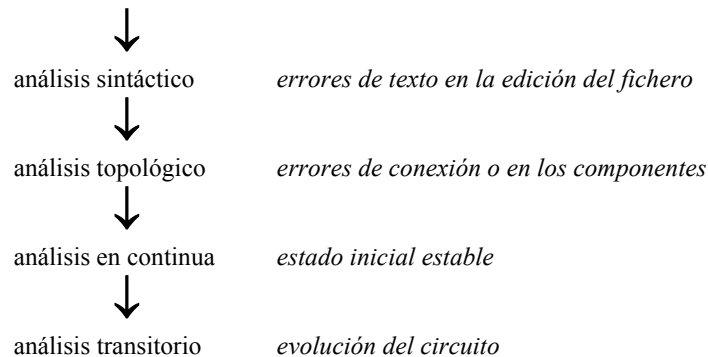
La instrucción **.NODESET** V(nudo)=valor permite fijar las tensiones iniciales (en continua) de los nudos; tales tensiones permanecen fijas en dicho valor en el cálculo del estado inicial, pero evolucionan adecuadamente al calcular la respuesta transitoria a lo largo del tiempo.

Una instrucción análoga **.IC** V(nudo)=valor permite ayudar en los cálculos del punto inicial, indicando un valor para uno o varios nudos, a partir del cual se efectúan los cálculos del estado inicial; a diferencia de **.NODESET**, **.IC** no fija la tensión de los nudos, tan sólo señala el primer valor a utilizar en el correspondiente cálculo iterativo de resolución de las ecuaciones (lo cual ayuda a resolver muchos casos de falta de convergencia).

La instrucción **.PROBE** determina el paso de los resultados de la simulación a un escenario gráfico interactivo que permite representar en pantalla cualquier combinación de las entradas y de las salidas y efectuar todo tipo de ampliaciones y medidas con cursores. Y las instrucciones **.PRINT** y **.PLOT** sirven para imprimir los resultados a través de impresora y de «plotter», respectivamente.

Forma de actuar

El simulador SPICE, en el caso de un análisis temporal (que es el más habitual), actúa de la siguiente forma:



El circuito queda organizado en **nudos** (puntos de conexión de dos o más componentes) y **ramas** (cada una de las conexiones existentes entre dos nudos: en cada rama hay un componente y sólo uno) y las ecuaciones circuitales se construyen según el siguiente proceso:

1. Ecuaciones de conservación de la corriente en los nudos: n ecuaciones
2. Sustitución en ellas de las corrientes en función de las tensiones de rama, a partir de las ecuaciones $I = f(V)$ de los componentes.
3. Sustitución de las tensiones de rama por diferencias de tensión de nudo.
4. Se añaden las ecuaciones de rama en aquellos casos en que las intensidades de rama no han podido ser sustituidas (por ejemplo, para un generador de tensión V_a no puede expresarse la intensidad en función de la tensión y su ecuación de rama es simplemente $V(n) = V_a$) a ecuaciones

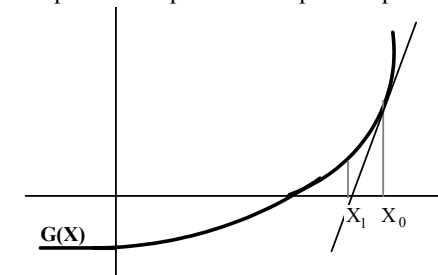
De esta forma resultan $n+a$ ecuaciones (n nudos + a ramas añadidas) con $n+a$ incógnitas, que se resuelven por técnicas de cálculo numérico.

Ahora bien, los dispositivos electrónicos son componentes no lineales; la relación $I = f(V)$ no es lineal y, por tanto, el sistema de ecuaciones tampoco lo es. Para resolverlo, se utilizan métodos numéricos de aproximación lineal mediante iteraciones sucesivas:

Fijado un punto V_0 se aproxima la función $I = f(V)$ por su primer término según el desarrollo en serie de Taylor:

$$I(V) = I(V_0) + \left. \frac{dI(V)}{dV} \right|_{V_0} \cdot (V - V_0) = a + b \cdot \Delta V$$

lo cual permite calcular un nuevo valor V_1 y volver a iterar hasta que V_{i+1} se diferencia mínimamente de V_i ; lo que en realidad se hace es sustituir la curva $I-V$ por su tangente en un punto e ir aproximando punto a punto la solución de las ecuaciones.



Por ejemplo, sea la función $G(X) = 0$; para hallar una solución de la misma, se toma el punto X_0 , se sustituye la curva, propia de la función, por su tangente en dicho punto y se calcula un segundo punto X_1 , repitiéndose con este nuevo valor el proceso anterior y así sucesivamente ... hasta que X_i y X_{i+a} están muy próximos.

Este tipo de métodos de aproximación lineal de ecuaciones no lineales resulta muy eficaz, pero presenta problemas de convergencia por desbordamiento (valores muy altos) o por mínimos locales (en los cuales la iteración puede centrarse recursivamente sobre él y no salir del intervalo en torno al mínimo local).

En caso de no convergencia pueden intentarse las siguientes soluciones:

1. Respecto a valores altos de las impedancias:

- evitar que algún nudo quede eléctricamente aislado, incluyendo, cuando exista dicha posibilidad, una pequeña capacidad (por ejemplo 1 fF) entre dicho nudo y el nudo de referencia (nudo 0); también puede incluirse, en lugar de la capacidad, una resistencia de valor muy alto (pero, en circuitos digitales, da mejor resultado la inclusión de pequeños efectos capacitivos y esto se aproxima más a la situación real).
- limitar el valor máximo de la impedancia que puede presentar una rama, o lo que es lo mismo, limitar el valor mínimo de la admitancia de cualquier rama, para lo cual se emplea la instrucción **.OPTIONS** GMIN=valor (puede darse a la admitancia mínima el valor 1E-7 que corresponde a una impedancia máxima de 10 MΩ).

2. Respecto a la posibilidad de mínimos locales:

- mejorar los modelos de los componentes de forma que no presenten mínimos locales,
- definir valores iniciales para calcular la tensión de los nudos mediante la instrucción **.IC** V(nudo)=valor.

Resolviendo el sistema de ecuaciones, por iteraciones sucesivas, se obtiene un estado inicial estable: la solución estacionaria (de partida); a partir de ese punto ha de calcularse la evolución temporal V(t), I(t), es decir, el análisis transitorio.

El sistema se transforma en diferencial y dependiente del tiempo:

$$f_j\left(t, X, \frac{dX}{dt}\right) = 0 \text{ para } j = 1, 2, 3, \dots, n+a, \text{ y para resolverlo se define un paso o intervalo de tiempo } \Delta t, \text{ que permite sustituir las derivadas por incrementos:}$$

$$\frac{dX}{dt} = \frac{X - X_{\text{anterior}}}{\Delta t}.$$

De esta forma, las ecuaciones vuelven a ser de primer orden (aunque no lineales) y se resuelven por los mismos métodos numéricos de aproximación lineal, a través de iteraciones sucesivas.

De nuevo pueden producirse problemas de convergencia y, además, el tamaño del paso influye en tal posibilidad. Por otra parte, el tamaño del paso de simulación Δt introduce los correspondientes errores de aproximación, al sustituir una derivada puntual por un incremento que corresponde a un intervalo finito.

Para minimizar tales errores de aproximación y evitar, en lo posible, los problemas de convergencia, el simulador SPICE utiliza tamaños de paso variable (disminuyendo el tamaño en los puntos de fuerte variación de tensiones o intensidades y aumentándolo cuando su variación es lenta). Por ejemplo, en la simulación de un circuito digital, el tamaño del paso se hace sumamente pequeño en el entorno de los flancos de las señales de entrada y pasa a ser grande en las situaciones estables: estados booleanos 0 y 1.

Subcircuitos

El simulador SPICE admite la utilización de subcircuitos como módulos para la construcción de circuitos más complejos, sin necesidad de detallar su contenido más que una sola vez. La declaración de subcircuito se realiza con las siguientes instrucciones (de cabecera y de final de subcircuito):

.SUBCKT nombre_del_subcircuito nudo nudo nudo nudo

descripción (net list) del subcircuito

.ENDS

Los nudos declarados en la cabecera serán los necesarios para la conexión externa del subcircuito en su utilización como tal y su numeración ha de corresponderse con de la «lista circuital» que describe el subcircuito. El nudo 0 es siempre un nudo global para todo el circuito, es decir, no puede utilizarse el 0 para numerar un nudo de un subcircuito que no vaya a coincidir con el nudo 0 de referencia del circuito global.

Para insertar un módulo o subcircuito en un circuito mayor se utiliza la instrucción:

Xid nudo nudo nudo nudo nombre_del_subcircuito

El orden de los nudos ha de corresponderse con el indicado en la cabecera de la declaración del subcircuito, pero su numeración (al utilizarlo como tal) será la propia del circuito global en el que se incluye.

Un mismo número de nudo (por ejemplo, el 2) puede ser utilizado dentro de la descripción de uno o de varios subcircuitos y, también, en el circuito global; pero, en cada caso, se refiere a un nudo diferente, es decir, los nudos dentro de los subcircuitos y los de fuera de ellos no quedan conectados por asignación del mismo número. La única excepción a esto es el nudo 0, que es común, con tal número a todos los subcircuito y al circuito global, pues es el nudo de referencia para todos ellos.

Un mismo módulo o subcircuito puede ser utilizado con valores distintos para alguno o algunos de los valores de sus componentes (por ejemplo, un inversor MOS puede ser utilizado con diferentes valores de la anchura de sus transistores). Es decir, la declaración de un subcircuito admite la parametrización (expresar en función de un parámetro) de alguna de sus características, para lo cual se utiliza la declaración:

.SUBCKT nombre_del_subcircuito nudo nudo

+ **PARAMS:** parámetro = valor_por_defecto

descripción (net list) del subcircuito y dentro de ella: **variable = {parámetro}**

.ENDS

«parámetro» representa el nombre del parámetro que se va a utilizar.

Al utilizar el subcircuito se indica el valor a dar al parámetro:

Xid nudo nudo nudo nombre_del_subcircuito
+ **PARAMS:** parámetro = valor_actual

Si al incluir un subcircuito se asigna un valor al parámetro utilizado (como sucede en la instrucción anterior) el subcircuito se construye dando tal valor a la variable parametrizada; en caso contrario (si no se asigna valor al parámetro), se construye con el valor por defecto expresado en la declaración del subcircuito.

Ejemplo: Cuatro inversores seguidos «en escalera» (con la anchura de sus transistores progresivamente creciente).

Circuito de cuatro inversores en escalera

*Subcircuito inversor 1 entrada y 1 salida

.SUBCKT inver 1 2 PARAMS: a = 1.5U

M1 2 1 3 3 mipmos L=1U W={a}

M2 2 1 0 0 minmos L=1U W={a}

VCC 3 0 5

.model mipmos PMOS level=1 KP = 15U VTO = -1 TOX = 50E-9

.model minmos NMOS level=1 KP = 40U VTO = 1 TOX = 50E-9

.ENDS

* Conexión de los 4 inversores

X1 1 2 inver

X2 2 3 inver PARAMS: a = 15U

X3 3 4 inver PARAMS: a = 150U

X4 4 5 inver PARAMS: a = 1500U

* Carga capacitiva de la salida

C4 5 0 50P

* Tensión de entrada: onda cuadrada de 40 ns de periodo

Vi 1 0 PULSE(0 5 10N 0 0 20N 40N)

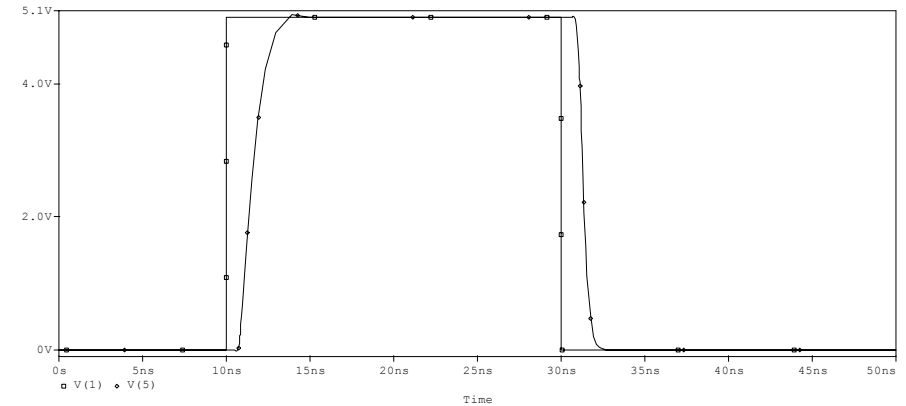
* Análisis temporal

.TRAN 0.01N 50N

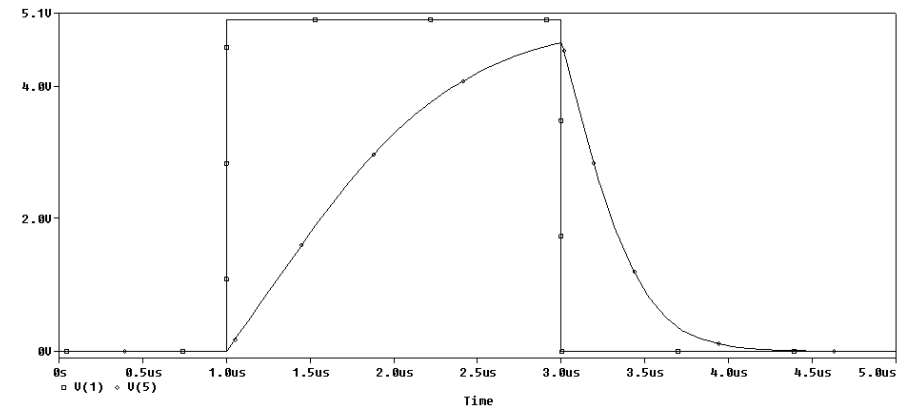
.PROBE

.END

En la página siguiente se muestra el resultado de la simulación de este circuito de 4 inversores en escalera (el primero de $W = 1,5 \mu\text{m}$ y creciendo W diez veces en cada uno de los siguientes) y, a efectos comparativos, se muestra el resultado de la simulación con los cuatro inversores iguales (todos ellos de $W = 1,5 \mu\text{m}$).



Respuesta del circuito de 4 inversores «en escalera» a un pulso de tensión (el eje X, eje de tiempos, se presenta con 5 ns/división)



Respuesta a un pulso de tensión de un circuito de 4 inversores iguales, de tamaño mínimo (téngase en cuenta que, en esta segunda gráfica, el eje de tiempos está en 500 ns/división, es decir, sus divisiones son 100 veces mayores, en tiempo, que en la gráfica anterior).

T6 MODELO FUNCIONAL DEL TRANSISTOR MOS

T6.1. Funcionamiento del transistor NMOS

T6.2. Etapa en fuente común

T6.3. El transistor MOS real: efectos de segundo orden

T6.4. Modelos SPICE

El estudio de la microelectrónica digital, además de abarcar el proceso de diseño del circuito electrónico y las posibilidades de programación o fabricación del mismo, debe atender, también, a la tecnología, al «soporte material» sobre el cual se va a configurar el circuito diseñado.

Se trata de insertar el circuito digital resultante de un diseño específico en el interior de un circuito integrado, bien por configuración de un circuito integrado programable, bien por fabricación del circuito integrado según el diseño propio.

Es necesario conocer el soporte material que permite tal programación o fabricación, estudiar con un poco de detalle la tecnología que permite «materializar» nuestro diseño «particular». Disponer de un modelo básico conceptual y operativo de dicha tecnología permitirá conocer sus posibilidades, sus prestaciones, sus limitaciones y sus exigencias (los requisitos que la tecnología impone sobre el diseño).

Tal es el objeto de estudio de los capítulos T6, T7, T8 y T9. En éste, primero de ellos, se presenta un modelo funcional del comportamiento de los transistores MOS, empleando los «diagramas de Memelink» como herramienta gráfica muy útil para representar la distribución de carga en el canal de los transistores y para efectuar cálculos relativos a tensiones e intensidades (y, también, a tiempos de propagación y consumos).

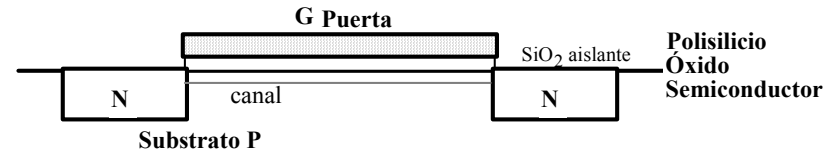
El comportamiento de los transistores MOS ya fue descrito, con cierto detalle, en el capítulo T1 (volumen 1). En los apartados que siguen se repetirá, ampliándola, tal descripción; se ha procurado resumir en lo posible lo ya dicho en T1.3., de manera que la consulta a dicho apartado puede ser útil para razonar y comprender aquellas cuestiones que, por resumidas, no hayan quedado claras al estudiar el presente capítulo. Asimismo, se recomienda la lectura de todo el capítulo T1 como introducción previa.

Junto con el modelo funcional (gráfico) del comportamiento MOS, se detallan aquellos aspectos de segundo orden que separan a los transistores reales de dicho modelo y que permiten comprender mejor y simular más fielmente su funcionamiento.

Respecto a este capítulo y a los siguientes, debe apreciarse que el estudio de la tecnología con la cual pueden fabricarse nuestros diseños (y con la cual se fabrican, también, los dispositivos en los que pueden programarse) no es una mera «opción cultural» no es una simple oportunidad de conocer cómo son «por dentro» los circuitos integrados reales. Sino que, de dicho estudio, se derivan muchas cuestiones prácticas, muchas limitaciones, condiciones o requisitos que han de tenerse en cuenta para hacer efectiva la integración de un sistema digital.

T6.1. Funcionamiento del transistor NMOS

La idea básica del transistor MOS fue anterior a la del bipolar (1947); la estructura MOS presenta una gran sencillez conceptual, por su analogía directa con un interruptor:



Una tensión positiva suficientemente alta sobre el terminal de puerta V_G da lugar, por atracción sobre los portadores minoritarios del sustrato, a un canal continuo entre las dos zonas de difusión N, que permite el flujo de corriente entre ellas:

$V_G < V_{TO}$ no existe canal $\Rightarrow$ el transistor no conduce

$V_G > V_{TO}$ se forma canal $\Rightarrow$ el transistor conduce

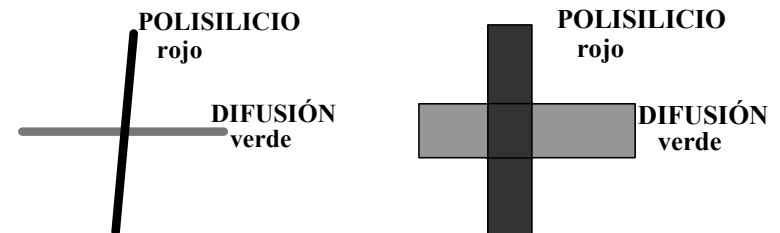
Siendo V_{TO} la tensión umbral de conducción.

La fabricación efectiva de transistores MOS se retrasó hasta los años 60 por problemas técnicos: el transistor es un efecto superficial, no volumétrico (caso de los transistores bipolares) y se requieren técnicas de extraordinaria limpieza en la superficie.

En el proceso de integración se forma primero la «banda» de óxido de puerta y polisilicio (cuya anchura determina la longitud del transistor) y, luego, en sentido perpendicular, la «banda» de difusión que forma la fuente y el sustrato; de esta forma, la puerta y las dos zonas de difusión N quedan autoalineadas (la difusión se realiza sobre todo el transistor pero no penetra debajo del polisilicio).

El transistor se forma físicamente por el cruce de dos regiones: polisilicio y difusión. Debajo del polisilicio la difusión se encuentra interrumpida, pero puede inducirse, por medio de la tensión de puerta, su continuidad dando lugar a la formación de un canal.

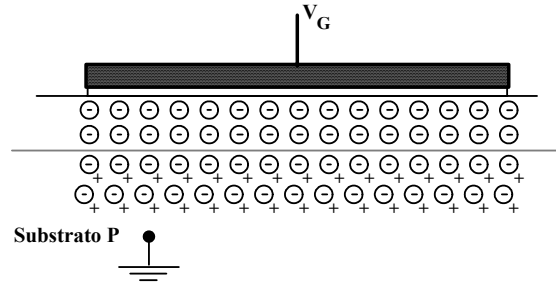
Abstrayendo las dimensiones superficiales, el transistor puede representarse en un diagrama lineal como un cruce entre dos líneas: difusión - polisilicio.



[Al dibujar las correspondientes máscaras, suele utilizarse color rojo para la de polisilicio y color verde para la difusión.]

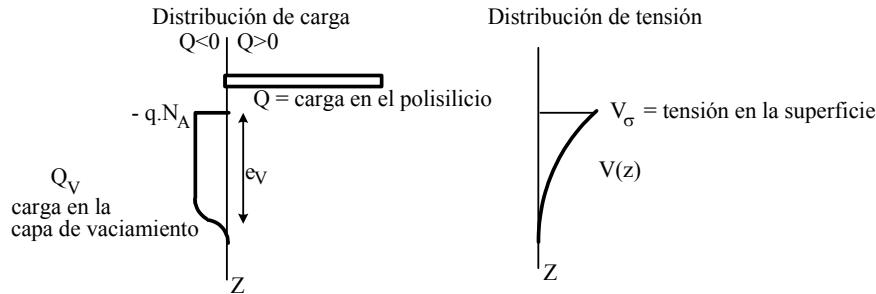
a) Capa vacía

El sustrato de los transistores NMOS es de tipo P, dopado con átomos de Boro con sólo 3 electrones de valencia, que dejan un hueco en la estructura cristalina. Cuando se aplica una tensión positiva a la puerta, $V_G > 0$, se forma una capa de vaciamiento, desprovista de portadores, debida a la repulsión de la tensión de puerta sobre los huecos del sustrato; los átomos de boro quedan cargados negativamente.



La densidad superficial de carga (carga por unidad de superficie) Q será:

$$Q_V = q \cdot N_A \cdot e v \quad \begin{array}{l} q = \text{carga del electrón,} \\ N_A = \text{dopado del sustrato} = n^\circ \text{ de huecos por cm}^3, \\ e v = \text{profundidad de la capa de vaciamiento.} \end{array}$$



Integrando la ecuación de Poisson para la tensión, resulta:

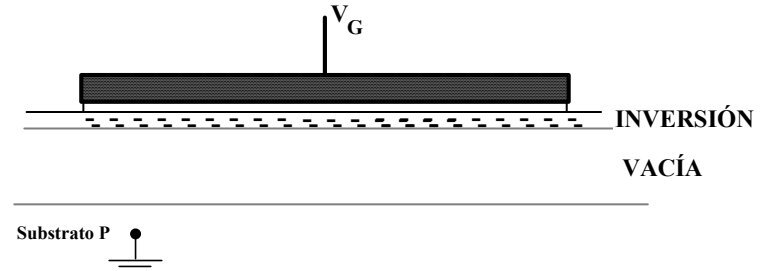
$$\frac{d^2 V}{dz^2} = \frac{\rho(z)}{\epsilon} \Rightarrow V_\sigma = \frac{q \cdot N_A}{2 \cdot \epsilon_{Si}} \cdot e v^2$$

siendo V_σ la tensión en la superficie del semiconductor; sustituyendo $e v$ en Q_V se obtiene la relación entre la carga de la capa de vaciamiento y la tensión en la superficie:

$$Q_V = \sqrt{2 \cdot \epsilon_{Si} \cdot q \cdot N_A} \sqrt{V_\sigma} = K_1 \sqrt{V_\sigma}, \quad \text{donde } K_1 = \sqrt{2 \cdot \epsilon_{Si} \cdot q \cdot N_A}.$$

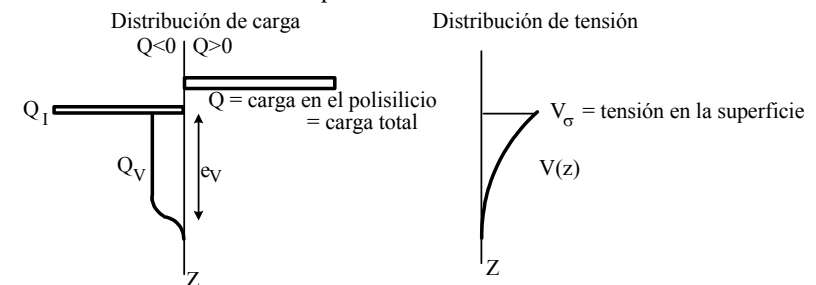
b) Capa de inversión: canal conductor

Cuando la tensión de puerta es suficientemente positiva, se forma una capa superficial de electrones libres, arrastrados por el gradiente de tensión de la capa de vaciamiento: capa de inversión o canal; tales electrones libres proceden de los pares electrón-hueco del propio silicio semiconductor y su número es reducido.



Para formar una capa de inversión o canal conductor se necesita una tensión en la superficie del semiconductor Φ_B capaz de mantener una concentración de electrones equivalente al dopado (concentración de huecos) del sustrato, es decir, el sustrato P se vuelve N en la zona superficial; el valor de la tensión superficial necesaria para un «dopado inverso» equivalente al del sustrato N_A , *condición de fuerte inversión*, es:

$$\phi_B = 2 \frac{KT}{q} \ln \frac{N_A}{N_i} \quad \begin{array}{l} \text{siendo } K \text{ la constante de Boltzman, } T \text{ la temperatura absoluta} \\ \text{y } N_i \text{ la concentración de pares electrón-hueco del silicio a esa temperatura.} \end{array}$$



La carga almacenada en el «condensador» puerta-sustrato tiene dos componentes: carga correspondiente a la capa vacía + carga debida al canal o capa de inversión, $Q = Q_V + Q_I$; la carga total Q será proporcional a la diferencia de potencial existente entre puerta y sustrato y el factor de proporcionalidad corresponderá a la capacidad de dicho condensador c_{ox} . En las siguientes expresiones las diversas cargas están expresadas por unidad de superficie y lo mismo sucede con la capacidad de puerta c_{ox} .

$$\left. \begin{array}{l} Q = Q_I + Q_V \\ Q = c_{ox} (V_G - \Phi_B) \\ Q_V = K_1 \sqrt{V_\sigma} \end{array} \right\} \quad \begin{array}{l} Q = \text{carga por unidad de superficie} \\ c_{ox} = \text{capacidad por unidad de superficie} \end{array}$$

$$Q_I = Q - Q_V = c_{ox}(V_G - \phi_B) - K_1 \sqrt{\phi_B} = c_{ox} [V_G - (\phi_B + \gamma \sqrt{\phi_B})]$$

$$Q_I = C_{ox} (V_G - V_{TO}) \quad \text{donde} \quad V_{TO} = \phi_B + \gamma \sqrt{\phi_B}$$

$$y \quad \gamma = \frac{K_1}{c_{ox}} = \frac{\sqrt{2 \cdot \epsilon_{Si} \cdot q \cdot N_A}}{c_{ox}}$$

V_{TO} es la tensión umbral de conducción del MOS (tensión de puerta necesaria para que se forme la capa de inversión, en condiciones de fuerte inversión) y γ es el parámetro de efecto sustrato (expresa el efecto de la capa de vaciamiento sobre **V_{TO}**).

Comentario: sobre **V_{TO}** inciden otros efectos de segundo orden (ver apartado 3); en la práctica, dicho valor lo fija el fabricante, mediante un proceso de implantación iónica en la superficie del sustrato.

Valores de los diversos parámetros (tecnología Ψ):

Para tener una idea más concreta y una referencia cuantitativa de los resultados, en este capítulo y en los siguientes se incluyen valores de los diversos parámetros y se efectúan los cálculos correspondientes, en relación a una tecnología hipotética pero realista a la que denominaremos tecnología Ψ .

La tecnología Ψ es de 1 micra; sus transistores tendrán las siguientes dimensiones: 1 μm de longitud mínima y 1,5 μm de anchura mínima y el espesor del óxido de puerta t_{ox} será de 0,05 μm .

La dimensión mínima de una tecnología de integración CMOS viene dada por la anchura mínima del polisilicio, que corresponde a la longitud del transistor: $L_{\min}$. En general, la anchura de la difusión (que coincide con la del transistor W) no puede ser tan reducida como la del polisilicio; lo habitual en la mayoría de las tecnologías es que $W_{\min} = 1,5 \cdot L_{\min}$ y, por ello, los transistores de tamaño mínimo no serán cuadrados sino que su factor de forma será $ff = W/L = 1,5$.

A temperatura ambiente (27°C = 300°K):

$$N \text{ (densidad átomos de Si)} \quad \sim 10^{22} \text{ átomos Si/cm}^3 = 10^{10} \text{ ats}/\mu\text{m}^3$$

$$N_i \text{ (pares e-h Si puro)} \quad \sim 10^{10} \text{ pares e-h/cm}^3$$

1 par e-h intrínseco por cada billón de átomos de Si

$$N_A \text{ (dopado sustrato)} \quad \sim 10^{15} \text{ ats B/cm}^3 = 10^3 \text{ ats}/\mu\text{m}^3$$

1 átomo de Boro por cada 10 millones de átomos de Si

1 hueco extrínseco por cada 10 millones de átomos de Si

100.000 veces más portadores extrínsecos que intrínsecos

La capacidad de puerta, por unidad de superficie, será:

$$c_{ox} = \frac{\epsilon_{SiO_2}}{t_{ox}} \quad \left| \begin{array}{l} \epsilon_{SiO_2} = 3,97 \cdot \epsilon_0 \approx 0,035 \text{ fF}/\mu\text{m} \\ t_{ox} \text{ (espesor óxido)} \approx 0,05 \mu\text{m} \end{array} \right. \quad c_{ox} \approx 0,7 \text{ fF}/\mu\text{m}^2$$

Un condensador MOS de 1 mm^2 presentará una capacidad ~ 1 nF; dicho efecto capacitivo se refiere al cociente $Q_I / (V_G - V_{TO})$: es una capacidad de tipo incremental $C = \Delta Q / \Delta V_G$.

Un transistor MOS de tamaño mínimo (1 $\mu\text{m} \times 1,5 \mu\text{m}$) tendrá una capacidad ~ 1 fF en su zona de puerta.

Respecto a la capa de inversión o canal conductor:

$$\Phi_B = 2 \frac{KT}{q} \ln \frac{N_A}{N_i}; \quad \frac{KT}{q} \approx 0,026 \text{ V (a } 25^\circ\text{C)} \quad \Phi_B \approx 0,6 \text{ V}$$

$$\gamma = \frac{\sqrt{2 \cdot \epsilon_{Si} \cdot q \cdot N_A}}{c_{ox}} \quad \left| \begin{array}{l} \epsilon_{Si} = 11,7 \cdot \epsilon_0 \approx 0,1 \text{ fF}/\mu\text{m} \\ q \text{ (carga electrón)} = 16 \times 10^{-20} \text{ C} \end{array} \right. \quad \gamma = 0,25 \text{ V}^{1/2}$$

$$V_{TO} = \Phi_B + \gamma \cdot \sqrt{\Phi_B} \quad V_{TO} \approx 1 \text{ V}$$

También puede calcularse la profundidad de la capa de vaciamiento:

$$\Phi_B = \frac{q \cdot N_A}{2 \cdot \epsilon_{Si}} \cdot e_v^2 \quad e_v = \sqrt{\frac{2 \cdot \epsilon_{Si}}{q \cdot N_A}} \cdot \sqrt{\Phi_B} \approx 1 \mu\text{m}$$

c) Tensión de referencia (en el canal) $\neq 0$

Si sobre el sustrato en su zona superficial se induce, por algún medio, una tensión **V'**, dicha tensión actúa como referencia o nivel 0 de tensiones, de forma que la tensión efectiva sobre el condensador MOS será **V_G - V'**:

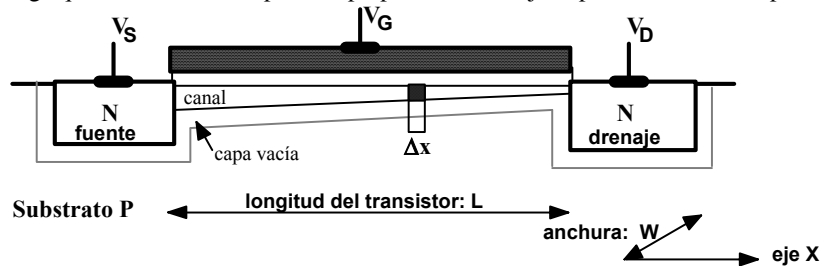
$$Q_I = c_{ox} \cdot (V_G - V' - V_{TO}).$$

Esto ocurre al polarizar la fuente y el drenaje para que circule una corriente: las tensiones **V_D** y **V_S** establecen una distribución de tensiones en la zona superficial del sustrato, de **V_S** (en el extremo de la fuente) hasta **V_D** (en el del drenaje). Orientando el eje X en la dirección del canal, la concentración de portadores de carga en el canal será:

$$Q_I = c_{ox} \cdot (V_G - V_{TO} - V(x)) \quad \text{donde } V(x) \text{ varía entre } V_S \text{ y } V_D.$$

d) Intensidad de corriente a través del canal

La diferencia de tensión V_{DS} da lugar a una intensidad de corriente que corresponde a la carga que atraviesa una superficie perpendicular al eje X por unidad de tiempo.



La distribución de portadores libres en la capa de inversión variará a lo largo del canal: será mayor al lado de la fuente que en el del drenaje ($V_{DS} > 0$, $V_G - V_S > V_G - V_D$). En un elemento diferencial Δx la cantidad de carga que corresponde a los portadores libres, a todo lo ancho W del transistor será:

$$\Delta Q_I = [\text{densidad de carga}] \times \text{volumen} = Q_I \cdot W \cdot \Delta x = c_{ox} \cdot (V_G - V_{TO} - V(x)) \cdot W \cdot \Delta x.$$

$$I = \frac{dQ_I}{dt} = \frac{\Delta Q_I}{\Delta x} \cdot \frac{\Delta x}{\Delta t} = \frac{\Delta Q_I}{\Delta x} \cdot v \quad \frac{\Delta Q_I}{\Delta x} : \text{carga de portadores por unidad de longitud}$$

v es la velocidad de los portadores, que es proporcional al campo eléctrico:

la constante de proporcionalidad es la movilidad de los portadores de carga μ

$$\left. \begin{aligned} \frac{\Delta Q_I}{\Delta x} &= c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) \\ v &= \mu \cdot E = \mu \cdot \frac{dV}{dx} \end{aligned} \right\} I = \mu \cdot c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) \cdot \frac{dV(x)}{dx}$$

Integrando la anterior ecuación diferencial a lo largo del canal resulta

$$\int_0^L I dx = \int_{V_S}^{V_D} \mu \cdot c_{ox} \cdot W \cdot (V_G - V_{TO} - V(x)) dV$$

$$I = \mu \cdot c_{ox} \cdot \frac{W}{L} \cdot \int_{V_S}^{V_D} (V_G - (V_{TO} + V(x))) dV = \alpha \cdot A$$

donde $A = \int_{V_S}^{V_D} (V_G - (V_{TO} + V(x))) dV$ expresa el efecto de las tensiones aplicadas sobre

el transistor y $\alpha = \mu \cdot c_{ox} \cdot W/L$ incluye parámetros tecnológicos y geométricos:

K_p = coeficiente de transconductancia = $\mu \cdot c_{ox}$ (parámetros tecnológicos)

ff = factor de forma = W/L , cociente entre anchura W y longitud L del transistor.

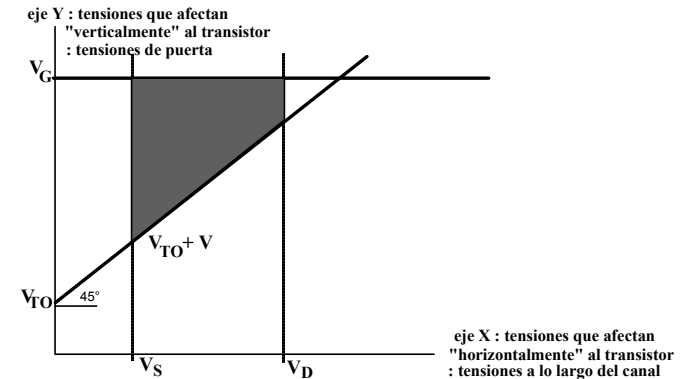
Ambos influyen en proporcionalidad directa sobre la intensidad que conduce el transistor.

Para los transistores NMOS de la tecnología Ψ :

$$\mu_e \sim 600 \text{ cm}^2/\text{V.s}, \quad c_{ox} \sim 0,7 \text{ fF}/\mu^2 = 0,07 \text{ }\mu\text{F}/\text{cm}^2, \quad K_p \sim 600 \cdot 0,7 \approx 40 \text{ }\mu\text{A}/\text{V}^2.$$

Diagramas de Memelink

La integral A puede resolverse en forma gráfica mediante los denominados *diagramas de Memelink* que representan, en el eje horizontal X , las tensiones aplicadas a fuente y drenaje (y por tanto la distribución de tensiones a lo largo del canal) y, en el eje vertical Y , la tensión aplicada a la puerta y la tensión umbral efectiva $V_{TO} + V$ (de modo que su diferencia $V_G - (V_{TO} + V)$ expresa la tensión efectiva generadora del canal):



$$\begin{aligned} A &= \int_{V_S}^{V_D} (V_G - (V_{TO} + V(x))) dV = \text{área del trapecio} \\ &= \left(\frac{(V_G - V_{TO} - V_S) + (V_G - V_{TO} - V_D)}{2} \right) \cdot (V_D - V_S) \\ &= \left(\frac{(V_{GS} - V_{TO}) + (V_{GS} - V_{TO} - V_{DS})}{2} \right) \cdot V_{DS} = \left(V_{GS} - V_{TO} - \frac{V_{DS}}{2} \right) \cdot V_{DS} \end{aligned}$$

En un *diagrama de Memelink* el área limitada por las cuatro rectas $x = V_S$, $x = V_D$, $y = V_G$ y $y = V_{TO} + V$ representa la distribución de carga a lo largo del canal; es, por tanto, una imagen del propio canal. Téngase en cuenta que el eje de abscisas no es espacial, sino un eje de tensiones; por ello, el diagrama representa la distribución de carga en el canal, en función de la distribución de tensiones a lo largo el mismo.

Integrando dicho área y multiplicándola por $\alpha = \mu \cdot c_{ox} \cdot W/L$ (que expresa los efectos tecnológicos y geométricos) se obtiene la ecuación de la corriente en un transistor MOS (canal no saturado):

$$I = K_p \cdot \frac{W}{L} \cdot \left(V_{GS} - V_{TO} - \frac{V_{DS}}{2} \right) \cdot V_{DS}$$

e) Diferencia de tensión drenaje-fuente pequeña

Para V_{DS} muy pequeño: $V_{DS} \ll V_{GS}$, $V_D \approx V_S$, la expresión $(V_G - V_{TO} - V(x))$ es prácticamente constante, $V_{GS} - V_{TO}$; la tensión aplicada es la misma a lo largo de todo el canal, que resulta plano, con una distribución de carga uniforme:

$$I_D = K_p \cdot \frac{W}{L} \cdot (V_{GS} - V_{TO}) \cdot V_{DS}$$

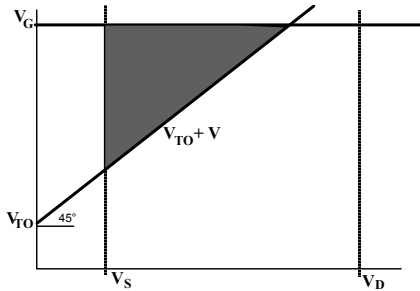
expresión que muestra una proporcionalidad directa entre la intensidad que pasa por el transistor y la tensión aplicada sobre el mismo. El transistor se comporta como una simple resistencia cuyo valor disminuye al aumentar la tensión de puerta V_{GS} :

$$R_{eq} = \frac{I_D}{V_{DS}} = \frac{1}{K_p \frac{W}{L} (V_{GS} - V_{TO})}$$

Esta región de funcionamiento del transistor se denomina *zona lineal* o *zona óhmica* y es la situación booleana que corresponde a un transistor MOS en conducción: equivale a una resistencia cuyo valor puede hacerse adecuadamente pequeño, a través del factor de forma W/L (la resistencia es inversamente proporcional a su anchura W).

f) Canal saturado

Al aumentar V_{DS} el canal se hace más estrecho (presenta menor número de portadores) en el lado del drenaje ($V_{GD} < V_{GS}$) y llega un momento en que se satura, es decir, la diferencia de tensiones $V_G - V(x)$ no supera la tensión umbral $V_G - V(x) < V_{TO}$: en tal zona no hay capa de inversión que contribuya a la conducción: si $V_{Gx} \leq V_{TO}$ se cierra el canal.



A = área del triángulo rectángulo isósceles

$$= \frac{1}{2} (V_G - (V_{TO} + V_S))^2 = \frac{1}{2} (V_{GS} - V_{TO})^2$$

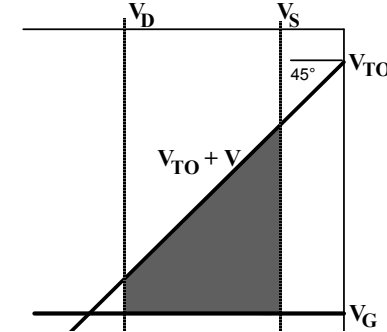
La situación de canal saturado se da si $V_G < V_{TO} + V_D$, $V_D > V_G - V_{TO}$, en cuyo caso la integral **A** se extiende solamente a la zona en que $V_G - V_{TO} - V(x)$ es positiva y viene dada por el área del triángulo formado por las rectas $x = V_S$, $y = V_G$ e $y = V_{TO} + V$:

$$I_{D,sat} = \frac{K_p}{2} \frac{W}{L} (V_{GS} - V_{TO})^2$$

expresión que representa la intensidad máxima que el transistor puede conducir para una tensión de puerta V_{GS} determinada; esta intensidad no depende de la tensión de drenaje y aumenta fuertemente con la tensión de puerta.

Transistores PMOS

En el transistor MOS de canal P son los huecos (en lugar de los electrones) los portadores que forman el canal; por ello, requiere tensiones de alimentación V_{DD} y de puerta V_G negativas respecto al sustrato y a la fuente, siendo también negativa su tensión umbral V_{TO} . Por ello, su *diagrama de Memelink* se sitúa en el tercer cuadrante:



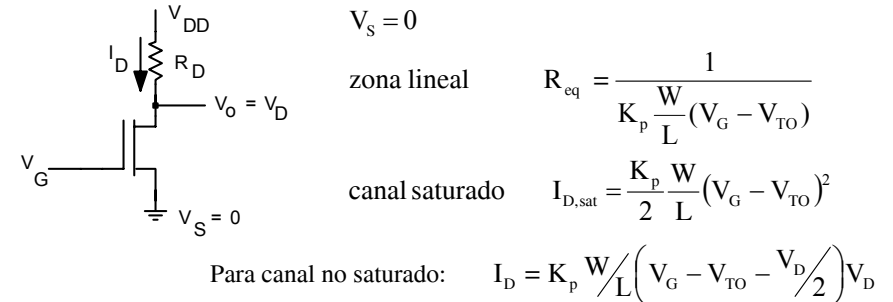
Además, la movilidad de los huecos es unas tres veces inferior a la de los electrones:

- Los huecos no se mueven por sí mismos sino como resultado de que un electrón ligado, que se encontraba en un enlace, pasa a cubrir el hueco y deja un nuevo hueco en su enlace anterior; de manera que el hueco se mueve en sentido contrario a como lo hacen los electrones ligados que lo rellenan.
- Siempre es más costoso movilizar un electrón ligado (que forma parte de un enlace) que un electrón libre (que se encuentra ya suelto en la banda de conducción).
- A igualdad de dimensiones (a igualdad de factor de forma W/L) la intensidad conducida por un transistor PMOS será muy inferior a la de un transistor NMOS.

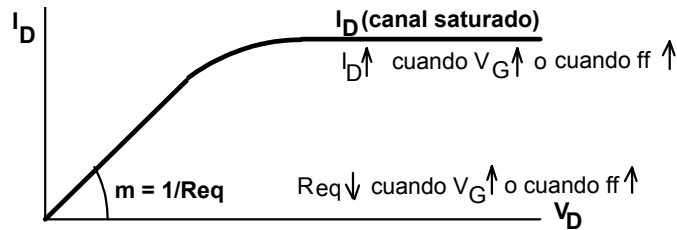
Para los transistores PMOS de la tecnología Ψ :

$$\mu_h \sim 200 \text{ cm}^2/\text{V.s}, \quad c_{ox} \sim 0,7 \text{ fF}/\mu^2 = 0,07 \text{ pF}/\mu^2, \quad K_p \sim 200 \cdot 0,7 \approx 15 \text{ pA}/\text{V}^2.$$

T6.2. Etapa en fuente común



Representando la intensidad que circula por el transistor (drenaje-fuente) I_D en función de la tensión entre sus terminales V_D , se obtiene la curva característica de la conducción del transistor:



Para pequeños valores de V_D , $V_D \ll V_G - V_{TO}$, zona lineal, el transistor se comporta como una resistencia cuyo valor disminuye al aumentar la tensión de puerta y, también, si aumenta el factor de forma ($ff = W/L$):

$$R_{eq} = \frac{1}{K_p \frac{W}{L} (V_G - V_{TO})}.$$

Para canal saturado $V_D > V_G - V_{TO}$,
$$I_D = \frac{K_p}{2} \frac{W}{L} (V_G - V_{TO})^2$$

la intensidad que atraviesa el transistor no depende de la tensión de drenaje y aumenta fuertemente con la tensión de puerta y linealmente con el factor de forma.

Actuando sobre las dimensiones geométricas del transistor ($ff = W/L$), se modifica su resistencia en zona lineal y, en sentido contrario, la intensidad con canal saturado.

$$W \uparrow \text{ ó } L \downarrow \text{ (en ambos casos } ff \uparrow) \Rightarrow R_{eq} \downarrow ; I_D \text{ (canal saturado)} \uparrow ;$$

la magnitud de ambas variaciones es proporcional a la modificación del factor de forma.

Valores de los diversos parámetros (tecnología Ψ):

Para un transistor cuadrado ($W=L$; $ff=1$) con tensión umbral de 1 V y tensión de puerta de 5 V los valores de la resistencia equivalente en zona lineal y de la intensidad de canal saturado serán:

NMOS

$$K_p \approx 40 \mu A/V^2$$

$$R_{eq} = 1 / (40 \cdot 10^{-6} \cdot (5-1)) \approx 6 \text{ k}\Omega$$

$$I_D = (40/2)(5-1)^2 \approx 0,3 \text{ mA}$$

PMOS

$$K_p \approx 15 \mu A/V^2$$

$$R_{eq} = 1 / (15 \cdot 10^{-6} \cdot (5-1)) \approx 17 \text{ k}\Omega$$

$$I_D = (15/2)(5-1)^2 \approx 0,1 \text{ mA}$$

Para un transistor de tamaño mínimo ($ff=1,5$):

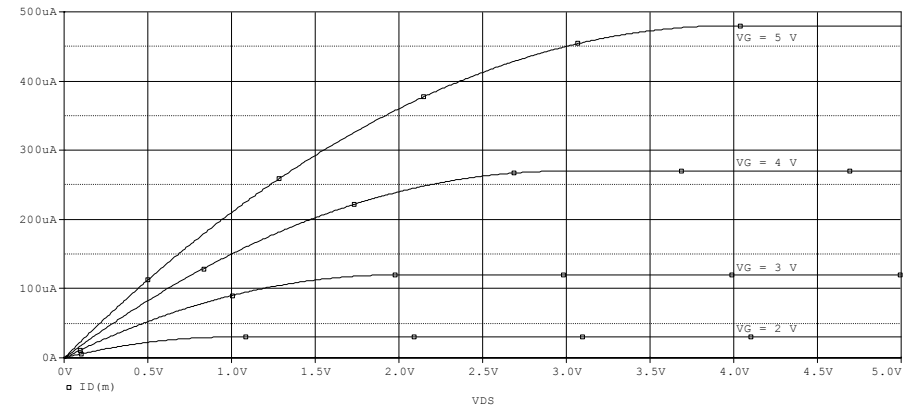
$$R_{eq} = 1 / (40 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 4 \text{ k}\Omega$$

$$I_D = (40/2) \cdot 1,5 \cdot (5-1)^2 \approx 0,5 \text{ mA}$$

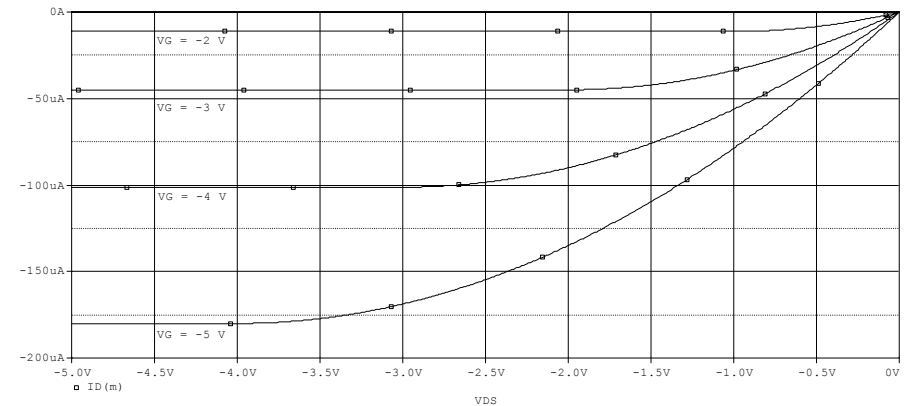
$$R_{eq} = 1 / (15 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 11 \text{ k}\Omega$$

$$I_D = (15/2) \cdot 1,5 \cdot (5-1)^2 \approx 0,2 \text{ mA}$$

Las gráficas siguientes muestran las curvas características de los transistores MOS de tamaño mínimo ($ff = 1,5$) de la tecnología Ψ para diversos valores de la tensión de puerta; han sido obtenidas mediante simulación SPICE para $L = 1 \mu m$, $W = 1,5 \mu m$, $V_{TO,NMOS} = 1 \text{ V}$, $V_{TO,PMOS} = -1 \text{ V}$, $K_{p,NMOS} = 40 \mu A/V^2$ y $K_{p,PMOS} = 15 \mu A/V^2$.



Curvas características $I_D - V_{DS}$ del transistor NMOS para varios valores de V_{GS}



Curvas características $I_D - V_{DS}$ del transistor PMOS para varios valores de V_{GS}

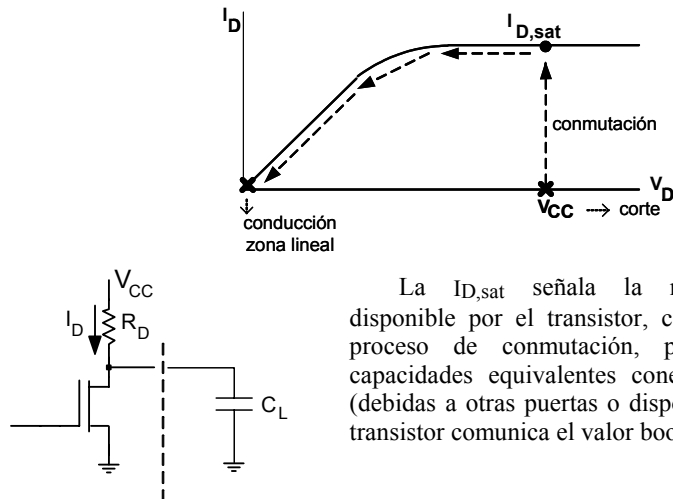
Comportamiento booleano

Las situaciones booleanas de un transistor MOS corresponden a:

- transistor en corte $V_{GS} < V_{TO}$ $I = 0$

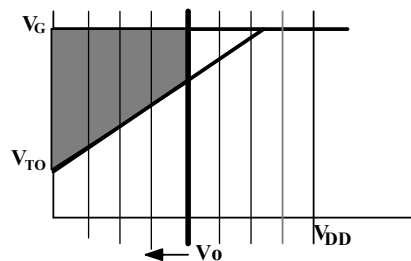
- transistor en zona lineal $V_{GS} > V_{TO}$ $V_{DS} = 0$
$$R_{eq} = \frac{1}{K_p \frac{W}{L} (V_G - V_{TO})}.$$

En la conmutación, en la puesta en conducción del transistor, éste recorre la curva característica correspondiente a $V_{GS} = V(1)$ desde la situación de canal saturado hasta alcanzar la zona lineal con $V_{DS} = 0$ V.



La $I_{D,sat}$ señala la máxima intensidad disponible por el transistor, con ella se inicia el proceso de conmutación, para descargar las capacidades equivalentes conectadas a su salida (debidas a otras puertas o dispositivos a los que el transistor comunica el valor booleano 0).

En condiciones funcionales la salida de una etapa se encontrará conectada a una o más etapas siguientes (o a otro tipo de carga), de forma que deberá cargar y descargar la capacidad equivalente que suponen las puertas de dichas etapas: C_L .



Al pasar la tensión de puerta V_G de 0 V a un valor $V(1)$ el transistor conduce y descarga la capacidad C_L que pasa de $V_o = V_{CC}$ (situación anterior, para $V_G = 0$ V) a $V_o = 0$ V: la intensidad, al principio, corresponde a la de canal saturado (área triangular en el diagrama de Memelink) y se reduce a medida que V_o disminuye (área trapezoidal), de forma que la intensidad decrece fuertemente al realizarse la descarga.

En la etapa representada en la figura, al pasar V_G de $V(1)$ a 0 V la capacidad C_L se carga a través de la resistencia R_D : proceso exponencial con constante de tiempo $R_D \cdot C_L$; resulta una situación sin interés respecto a las puertas lógicas CMOS, ya que tales puertas no se construyen con resistencias de carga.

T6.3. El transistor MOS real: efectos de segundo orden

Las principales diferencias de un transistor MOS real con el modelo desarrollado en los apartados anteriores son las siguientes:

1 La placa superior del «condensador» puerta-canal no es metálica, sino semiconductor, construida con silicio policristalino fuertemente dopado N^+ ; ello motiva que la distribución de carga en dicha placa no sea puramente superficial sino volumétrica: se distribuye en la zona superficial del polisilicio abarcando una cierta profundidad no nula. Tal distribución volumétrica de carga introduce un pequeño efecto resistivo, dando lugar a la correspondiente caída de tensión.

El alto dopado del polisilicio ($\sim 10^{20}$ ats/cm³, cien mil veces superior al sustrato, 10^{15}) reduce tal efecto volumétrico, permitiendo despreciarlo respecto al funcionamiento del transistor MOS. En cambio, deberá tenerse en cuenta que la resistividad de las conexiones realizadas en polisilicio es relativamente alta (unas 300 veces la de un metal) y su espesor es inferior al de las líneas de metal ($\sim 0,2$ frente a $0,5$ μm); por ello, la resistencia que presenta una línea de polisilicio es bastante superior (resistencia por cuadro: $30\Omega/\square$ el polisilicio, frente a $50\text{ m}\Omega/\square$ en las conexiones metálicas).

2 La tensión umbral V_{TO} se encuentra afectada por los siguientes efectos, todos ellos superficiales:

- la terminación abrupta de la red cristalina del sustrato,
- la existencia de iones positivos, principalmente de sodio, atrapados en el óxido de silicio durante los procesos de oxidación,
- la aparición de cargas fijas en las superficies de contacto entre el silicio y el óxido, que provienen, principalmente, de una oxidación incompleta del silicio.

Estos tres efectos se suman y pueden ser contabilizados conjuntamente en forma de una distribución de carga positiva $Q_{ox} = q \cdot N_{ox}$ que induce sobre el sustrato una carga negativa, es decir, crea una pequeña capa vacía inicial en ausencia de tensión exterior aplicada, reduciendo la tensión umbral V_{TO} (en el caso de un transistor de canal N; aumentándola, si es de canal P):

$$\Delta V_{TO} = - Q_{ox} / C_{ox} = - q \cdot N_{ox} / C_{ox}$$

La concentración equivalente de iones N_{ox} suele ser del orden de:

$$N_{ox} \approx 5 \times 10^{10} \text{ iones / cm}^2 = 500 \text{ iones / } \mu\text{m}^2 \quad Q_{ox} \sim 0,08 \text{ fC / } \mu\text{m}^2$$

$$\Delta V_{TO} \sim - 0,08 / 0,7 \approx -0,1 \text{ V.}$$

Un cuarto efecto superficial se debe a las interfases o potenciales de contacto polisilicio-óxido y óxido-sustrato y es consecuencia del diferente potencial o energía de extracción de los electrones en las capas en contacto. Tal potencial de contacto origina, asimismo, un desplazamiento de la tensión umbral

$$\Delta V_{TO} = \phi_{\text{sustrato}} - \phi_{\text{polisilicio}} \sim 0,8 \text{ V.}$$

con signo + para el sustrato N, canal P y con signo - para el sustrato P, canal N.

La suma de ambos desplazamientos (- 0,9 canal N y + 0,7 canal P) se traduce en la práctica cancelación de la tensión umbral V_{TO} en el caso del transistor N, mientras que para el transistor P dicha tensión umbral se reduce en forma importante (a 0,3 V).

Ahora bien, estos efectos que desplazan la tensión umbral V_{TO} expresan también la posibilidad de controlar tecnológicamente dicha tensión (en el proceso de fabricación de los transistores) mediante la implantación de iones en la superficie del sustrato.

La implantación de iones, a través del óxido de puerta en la zona semiconductora contigua al mismo, modifica la carga equivalente Q_{ox} y permite ajustar la tensión umbral de los transistores, situándola en valores adecuados que suelen estar en el entorno de 1 V en los transistores NMOS y en valores un poco inferiores (para compensar la menor movilidad de sus portadores) en los transistores PMOS.

Mediante la implantación de iones negativos (Boro⁻) se aumenta la tensión umbral del transistor N, acercándola a 1 V y, recíprocamente, la implantación de iones positivos (Fósforo⁺ o Arsenio⁺) permite situar la tensión umbral del transistor P en valores absolutos ligeramente superiores a 0,6 voltios. De esta forma, se utiliza la desigualdad $V_{TO} \text{ (PMOS)} < V_{TO} \text{ (NMOS)}$ para compensar un poco la asimetría entre ambos transistores, causada por la menor movilidad de los huecos.

Todos estos efectos superficiales inciden en el valor de la tensión umbral V_{TO} pero no afectan al modelo de transistor desarrollado anteriormente.

3 La reducción de las dimensiones del transistor MOS (longitud L o anchura W) afecta también a la tensión umbral V_{TO} , en forma de desplazamiento de la misma.

En las proximidades de las zonas de difusión (fuente o drenaje, ricas en portadores) el potencial de extracción de portadores es inferior al correspondiente al semiconductor que forma el sustrato. Por ello, cuando la longitud del canal es corta ($L \sim 1 \mu\text{m}$) la tensión umbral resulta afectada por la presencia de las difusiones (cercanas a todo el canal) y disminuye en forma apreciable, tanto más cuanto menor es la longitud del canal.

Por otra parte, la distribución de carga espacial de la zona vacía se extiende (en el sentido de la anchura del transistor) más allá de la longitud transversal definida por el óxido de puerta (más allá de la zona de puerta determinada por el polisilicio al cruzar a la difusión). El efecto de esta ampliación de la capa vacía es apreciable cuando el transistor es muy estrecho ($W < 1 \mu\text{m}$) y puede contabilizarse como una distribución de carga de vaciamiento Q_v superior a la considerada; como dicho término actúa en términos de resta, respecto a la tensión de puerta, se produce un aumento de la tensión umbral.

Estos dos efectos son de signo contrario y se compensan parcialmente entre sí, supuesto que ambas dimensiones L y W sean del mismo orden de tamaño. Sin embargo, en tecnologías «submicrónicas» ($< 1 \mu\text{m}$) el comportamiento real de los transistores difiere en gran medida del teórico (de las ecuaciones funcionales desarrolladas anteriormente), debido a este tipo de efectos relacionados con las dimensiones del transistor (influencia relativa de los bordes del transistor sobre su globalidad).

4 La longitud del transistor MOS no corresponde exactamente al tamaño transversal (anchura) del polisilicio que le cruza ya que la difusión penetra ligeramente debajo de la zona de puerta, por sus bordes, ocupando una franja de anchura L_D (difusión lateral); de forma que la longitud efectiva del canal será $L - 2L_D$.

Además, cuando el canal se encuentra saturado, se cierra antes de alcanzar al drenaje y su longitud efectiva es menor que la separación entre las dos difusiones (fuente y drenaje). Este efecto aumenta al hacerlo la tensión de drenaje y da lugar a que las curvas $I_D - V_{DS}$ no sean planas en la zona de canal saturado, sino que presenten una ligera pendiente; se conoce con el nombre de modulación de la longitud del canal y se contabiliza mediante un parámetro λ :

$$L_{\text{efectiva}} = (L - 2L_D) / (1 + \lambda V_{DS}).$$

En ocasiones, por analogía con los transistores bipolares, se utiliza como parámetro para expresar este efecto de aumento de I_D con V_D en la zona de canal saturado la denominada *tensión early* que corresponde a la tensión en que la prolongación de la recta I_D correspondiente a canal saturado corta al eje de tensiones.

5 La movilidad de los portadores μ_e μ_h y, por consiguiente, el parámetro de transconductancia K_p , depende fuertemente de la temperatura y de los campos eléctricos.

La temperatura disminuye la movilidad de los portadores y en el mismo sentido actúa el campo vertical producido por la tensión de puerta V_G ya que tiende a impulsar a los portadores hacia la superficie (interfase óxido-sustrato). En ambos casos aumenta el efecto de dispersión (los choques) que produce la red cristalina sobre los portadores (colisiones portadores - núcleos de la red); es el mismo efecto que sufren los materiales conductores al aumentar la temperatura y que da lugar a un aumento de su resistencia.

También afecta negativamente a la movilidad de los portadores el aumento del campo eléctrico horizontal producido por la diferencia de tensión entre fuente y drenaje; para valores de campo suficientemente intensas ($E > E_{\text{crítico}}$) se produce una saturación de la velocidad de los portadores: existe una velocidad máxima por encima de la cual la aceleración producida por el campo eléctrico no tiene efecto.

Por otra parte, el parámetro μ (movilidad de los portadores), además de ser sumamente sensible a todo tipo de magnitudes físicas, resulta de muy difícil medida; por ello, en muchas ocasiones, se utiliza directamente el parámetro de transconductancia $K_p = \mu \cdot C_{ox}$, cuya dependencia de la temperatura y de los campos eléctricos se establece mediante coeficientes de tipo experimental.

6 La distribución de tensión $V(x)$ a lo largo del canal (debida a las tensiones de fuente y drenaje) no solo actúa reduciendo la tensión efectiva de cada elemento Δx del condensador puerta-canal, $V_G - V_{TO} - V(x)$; también incrementa la carga de la capa vacía Q_v , reduciendo en igual cantidad la carga de la capa de inversión Q_i y su contribución a la corriente que circula por ella. Se denomina *efecto sustrato* al que producen las tensiones aplicadas horizontalmente (V_S, V_D) sobre la carga de la capa de vaciamiento.

Debido a la distribución de tensión que inducen V_S y V_D , la tensión en la superficie de un elemento $\Delta(x)$ será $\phi_B + V' > \phi_B$ y como la carga de la capa de vaciamiento es $Q_V = K_1 \sqrt{\phi_B + V'}$ (apartado T6.1.a), la carga de la capa de inversión será:

$$Q_I = Q - Q_V = c_{ox}(V_G - \phi_B - V') - K_1 \sqrt{\phi_B + V'} \\ = c_{ox}(V_G - \phi_B - V' - \gamma \sqrt{\phi_B + V'}) = c_{ox}(V_G - (\phi_B + V' + \gamma \sqrt{\phi_B + V'}))$$

Como $V_{TO} = \phi_B + \gamma \sqrt{\phi_B}$, $\phi_B = V_{TO} - \gamma \sqrt{\phi_B}$, podemos escribir

$$Q_I = c_{ox}(V_G - (\phi_B + V' + \gamma \sqrt{\phi_B + V'})) \\ = c_{ox}(V_G - (V_{TO} - \gamma \sqrt{\phi_B} + V' + \gamma \sqrt{\phi_B + V'})) \\ = c_{ox}(V_G - (V_{TO} + V' + \gamma \sqrt{\phi_B + V'} - \gamma \sqrt{\phi_B}))$$

y, con ello, la recta $V_{TO} + V$ del *diagrama de Memelink* se transforma en una curva de mayor pendiente: $V_{TO} + V + \gamma \sqrt{\phi_B + V} - \gamma \sqrt{\phi_B}$.

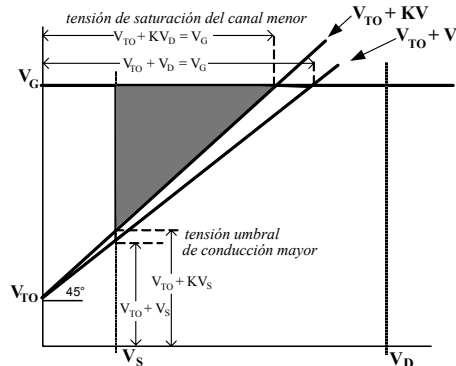
Resulta útil emplear una aproximación lineal de esta curva; para ello, en la expresión $Q_I = c_{ox}(V_G - \phi_B - V' - \gamma \sqrt{\phi_B + V'})$, el término de «efecto substrato» $\gamma \sqrt{\phi_B + V'}$ puede aproximarse mediante su desarrollo en serie de Taylor (primer término de la serie):

$$\gamma \sqrt{\phi_B + V'} = \gamma \sqrt{\phi_B} + aV' ; a = \frac{d}{dV'} (\gamma \sqrt{\phi_B + V'}) \Big|_{V'=0} = \frac{\gamma}{2\sqrt{\phi_B}}.$$

$$\gamma \sqrt{\phi_B + V'} - \gamma \sqrt{\phi_B} = aV'$$

$$Q_I = c_{ox}(V_G - (V_{TO} + V' + aV')) = c_{ox}(V_G - (V_{TO} + KV')) \quad \text{con } K = 1 + a \approx 1,15$$

$I = \mu \cdot c_{ox} \cdot \frac{W}{L} \cdot \int_{V_S}^{V_D} (V_G - (V_{TO} + K \cdot V(x))) \cdot dV$: la recta $V_{TO} + V$ del *diagrama de Memelink* pasa a $V_{TO} + K \cdot V$ ($K \sim 1,15 > 1$), es decir, aumenta su pendiente:



Habida cuenta de que la recta $V_{TO} + V$ actúa substractivamente, las consecuencias de su mayor pendiente son las siguientes:

➤ la intensidad que conduce el transistor es menor:

$$\begin{cases} I_D = K_p \frac{W}{L} \left(V_G - V_{TO} - K \frac{V_S + V_D}{2} \right) \cdot V_{DS} & \text{canal no saturado} \\ I_D = \frac{K_p}{2K} \frac{W}{L} (V_G - V_{TO} - KV_S)^2 & \text{canal saturado} \end{cases}$$

➤ cuando la tensión de fuente V_S es mayor que 0 V la tensión umbral aumenta:

para $V_S > 0$ V el transistor no conducirá hasta que

$$V_G \geq V_{TO} + K \cdot V_S = V_{TO} + V_S + a \cdot V_S ; V_{GS} \geq V_T = V_{TO} + a \cdot V_S;$$

siendo la nueva tensión umbral $V_T = V_{TO} + a \cdot V_S$ ($a \sim 0,15$).

Este efecto tiene importancia cuando la fuente del transistor no se encuentra unida al terminal de polarización (por ejemplo, en cuando los transistores se encuentran en serie) y reduce en forma apreciable la conductividad del transistor.

➤ la saturación del canal se produce para tensiones de drenaje inferiores:

la saturación se produce cuando $V_{TO} + K \cdot V_D = V_G$

o sea, para una tensión de drenaje $V_D = (V_G - V_{TO})/K$

$$V_D \sim 3.5 \text{ V para } V_G = 5 \text{ V y } V_{TO} = 1 \text{ V}$$

y dicho valor limita la tensión máxima de salida de un transistor de paso (como se verá en su momento).

7 Al analizar el comportamiento del transistor MOS se han detallado dos situaciones excluyentes: ausencia/presencia de canal (capa de inversión). Las condiciones establecidas para que exista canal ($V_G > V_T$) corresponden a una «inversión fuerte»: concentración de portadores en el canal análoga al dopado del substrato ($n \sim N_A$).

Existen otras dos situaciones posibles: inversión débil $n \ll N_A$ e inversión moderada $n < N_A$ que es preciso tener en cuenta cuando interesa el comportamiento del transistor para tensiones de puerta en el entorno de la tensión umbral V_T ; en principio, tales situaciones no afectan al caso digital pero han de ser tenidas en cuenta en etapas de tipo analógico.

T6.4. Modelos SPICE

El *modelo de nivel 1* (modelo de Shichman – Hodges) corresponde a las funciones desarrolladas en el apartado T6.1:

$$\begin{cases} I_D = K_p \frac{W}{L} \left(V_{GS} - V_{TO} - \frac{V_{DS}}{2} \right) V_{DS} & \text{canal no saturado} \\ I_D = \frac{K_p}{2} (V_{GS} - V_{TO})^2 & \text{canal saturado} \end{cases}$$

En este modelo, la descripción de un transistor MOS requiere tres parámetros:

KP	K_p parámetro de transconductancia
VTO	V_{TO} tensión umbral del transistor
TOX	t_{ox} espesor del óxido de puerta

TOX es necesario para tener en cuenta la capacidad de puerta; caso de no utilizarlo, se supone capacidad de puerta nula.

En lugar de KP puede utilizarse como parámetro la movilidad de los portadores μ , calculándose KP a partir del valor de dicha movilidad UO :

$$UO \quad K_p = \mu c_{ox} = \mu \frac{\epsilon_{SiO_2}}{t_{ox}}; \quad \mu = UO$$

En todo caso, es necesario indicar la dimensión de los transistores **L**, **W**, bien dentro del modelo o en la declaración de cada transistor; por defecto se asigna a L y W el valor de 1 metro, con lo cual se configuran transistores enormes, de 1 m² (que son tan grandes que dan lugar a resultados de simulación erróneos).

El modelo de nivel 1 admite los siguientes efectos de segundo orden:

- disminución de la longitud del canal debida a la difusión lateral **LD**: $L' = L - 2LD$
- modulación de la longitud de canal saturado **LAMBDA**: $L_{efectiva} = L / (1 + \lambda V_{DS})$
- efecto sustrato: la recta $V_{TO} + V$ pasa a ser $V_{TO} + V + \gamma \sqrt{\phi_B + V} - \gamma \sqrt{\phi_B}$
que requiere dos parámetros

GAMMA	γ coeficiente de efecto sustrato
PHI	ϕ_B potencial superficial para inversión fuerte,

o, en su defecto: **NSUB** N_A dopado del sustrato:

$$\phi_B = 2 \frac{KT}{q} \ln \frac{N_A}{N_i}$$

Ejemplo de modelo de transistores MOS (nivel 1):

- a) MODEL "nombre" NMOS LEVEL=1
+ KP=40U VTO=1 TOX=50E-9 L=1U W=1.5U
+ LD=0.05E-6 LAMBDA=0.01 GAMMA=0.25 NSUB=1E15
- b) MODEL "nombre" PMOS LEVEL=1
+ KP=15U VTO=-1 TOX=50E-9 L=1U W=1.5U
+ LD=0.05E-6 LAMBDA=0.01 GAMMA=0.25 NSUB=5E15

Además, pueden incluirse los efectos resistivos y capacitivos de las regiones físicas que componen el transistor mediante los correspondientes parámetros: **RSH** ($R_{\square}$ resistencia por cuadro –cuadrado– de la difusión), **CJ** (capacidad de la difusión respecto al sustrato), **CJSW** (capacidad perimetral de la difusión respecto al sustrato), **CGSO** y **CGDO** (capacidades de solapamiento puerta-fuente y puerta-drenaje), **RS**, **RD** y **RG** (resistencias óhmicas de los contactos con fuente, sustrato y drenaje), etc.

El *modelo de nivel 2* permite incluir todos los efectos de segundo orden comentados en el apartado T6.3 y, en particular, los relativos a los transistores submicrónicos; utiliza, en principio, los mismos parámetros que el de nivel 1, a los que añade parámetros complementarios para contabilizar los efectos sobre la movilidad de los portadores μ (y, por tanto, sobre K_p), sobre la tensión umbral V_T , etc. y para incluir situaciones de inversión débil. Los diversos efectos de segundo orden se tienen en cuenta o no según se asignen o no valores a los correspondientes parámetros.

El *modelo de nivel 3* corresponde a un ajuste semiempírico de coeficientes a través de aproximaciones de las características del transistor obtenidas a través de la medida de las mismas. Ha sido desarrollado principalmente para aproximar los resultados de la simulación a los valores experimentales en los transistores de canal ultracorto.

Además de los anteriores, existen otros modelos SPICE de los transistores MOS: versiones avanzadas de SPICE suelen incluir una docena de tales modelos. Sin embargo, en muchas ocasiones, en particular en la simulación de circuitos digitales con transistores no muy pequeños ($L \geq 1\mu m$), basta el modelo de nivel 1 para conseguir resultados muy aceptables.

T7 ESTUDIO EN DETALLE DEL INVERSOR CMOS

- T7.1. El inversor CMOS
- T7.2. Tiempo de propagación y mejora del mismo
- T7.3. Disipación de potencia
- T7.4. Puertas CMOS

Interesa estudiar en detalle el comportamiento de las puertas CMOS y ese detalle se refiere a tensiones, intensidades, tiempos y consumos: su función de transferencia (tensión de salida en relación con la tensión de entrada), la intensidad suministrable por la salida, los tiempos de conmutación, la disipación de potencia,... El inversor, como puerta más simple, de una sola entrada, resulta sumamente adecuado para desarrollar este estudio.

En el capítulo anterior, se introdujeron los «diagramas de Memelink» como forma de representar gráficamente la distribución de los portadores de carga en el canal de los transistores MOS y de facilitar los cálculos relativos a tensiones e intensidades. En el caso de un inversor, los «diagramas de Memelink» de los dos transistores que lo forman pueden representarse conjuntamente (en un mismo diagrama), de manera tal que puede calcularse la tensión de salida y la intensidad a través del inversor, para cualquier valor de su tensión de entrada.

De esta forma, en el presente capítulo se calcula la tensión de conmutación del inversor y la intensidad que consume en tal situación y se obtienen, también, las expresiones algebraicas que corresponden a los diversos «tramos» de la función de transferencia y, a partir de ella, los márgenes de ruido.

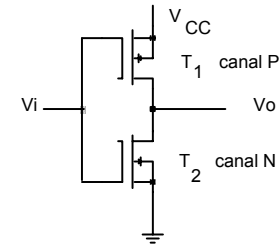
Asimismo es de gran interés comprender los factores que afectan a los tiempos de propagación y cómo dichos tiempos pueden mejorarse mucho, si la carga capacitiva es alta, utilizando una serie de inversores suplementarios, «en escalera» (cada uno de ellos de anchura superior al anterior).

La disipación de potencia en una puerta lógica, su consumo, tiene importancia en dos aspectos complementarios: la fuente de alimentación que ha de proporcionar suficiente intensidad y el calentamiento del circuito que aumenta sus tiempos de propagación y, en ocasiones, necesita adecuada refrigeración.

Por último, se recuerdan las características funcionales de las puertas CMOS y se destaca la limitación del número de entradas de las mismas, para evitar que un alto número de transistores en serie empeore en gran medida esas características.

T7.1. Inversor CMOS

El inversor CMOS utiliza un transistor de canal N y otro de canal P:



Supuestos $V_{CC} = 5 \text{ V}$ y $V_{TO,NMOS} = 1 \text{ V}$ y $V_{TO,PMOS} = -1 \text{ V}$
 Para $V_i < 1 \text{ V}$ T_1 conduce y T_2 no: $V_o = V_{CC} = V(1)$
 Para $V_i > 4 \text{ V}$ T_1 no conduce y T_2 sí: $V_o = 0 \text{ V} = V(0)$

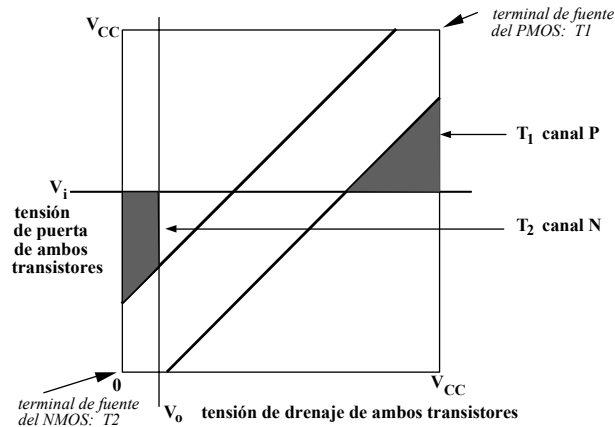
Las propias tensiones umbrales de los transistores aseguran las tensiones de salida booleanas $V(0)$ y $V(1)$ al mantener uno de los dos transistores en corte; en ambas situaciones booleanas, el consumo es nulo pues el transistor cortado impide el paso de corriente a través de ambos (que se encuentran en serie).

En la figura se ha representado también la polarización de los substratos: en el diseño físico han de configurarse las conexiones del substrato P (del transistor NMOS) a 0 V y del substrato N (pozo) a V_{CC} . Tales conexiones de polarización aseguran que todas las uniones PN que forman los transistores con el substrato se encuentran polarizadas inversamente, aislando a cada transistor respecto de su substrato.

El transistor NMOS tiene su terminal de fuente a 0 V y las tensiones de entrada V_{GS} y de salida V_{DS} son positivas y su referencia es 0 V. En cambio, el transistor PMOS tiene su terminal de fuente a V_{CC} (se encuentra colocado «boca abajo») y las tensiones de entrada V_{GS} y de salida V_{DS} son negativas, ambas con referencia a V_{CC} .

Para analizar la función de transferencia del inversor ($V_o - V_i$) puede utilizarse un diagrama de Memelink conjunto de ambos transistores. En dicho diagrama el origen de coordenadas del transistor NMOS (punto 0,0) no coincide con el del transistor PMOS (punto V_{CC}, V_{CC}) ya que la tensión de referencia, tensión de fuente, de ambos transistores es diferente. En cambio la tensión de entrada V_i (tensión de puerta) de ambos es común y actúa sobre el eje Y y también lo es su tensión de salida V_o (tensión de drenaje) que actúa sobre el eje X.

En una situación estática, la intensidad que circula por ambos transistores es la misma $I_{T1} = I_{T2}$ ($I_P = I_N$) ya que ambos están en serie; para lo cual tiene que cumplirse que $\alpha_P \cdot A_P = \alpha_N \cdot A_N$, siendo $\alpha = \mu \cdot C_{OX} \cdot W/L$ el parámetro que incluye los aspectos tecnológicos ($\mu \cdot C_{OX} = K_P$) y geométricos ($W/L = ff$) y A el efecto de las tensiones aplicadas sobre el transistor, que viene dado (en el diagrama de Memelink) por el área limitada por las cuatro rectas $x = V_S$, $x = V_D = V_o$, $y = V_G = V_i$ e $y = V_{TO} + V$.



$$\frac{A_N}{A_P} = \frac{\alpha_P}{\alpha_N} = \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} \frac{W_P/L_P}{W_N/L_N}; \quad \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} \approx \frac{1}{3}; \quad \frac{A_2}{A_1} \approx \frac{1}{3} \frac{ff_P}{ff_N} = \frac{1}{3} \beta$$

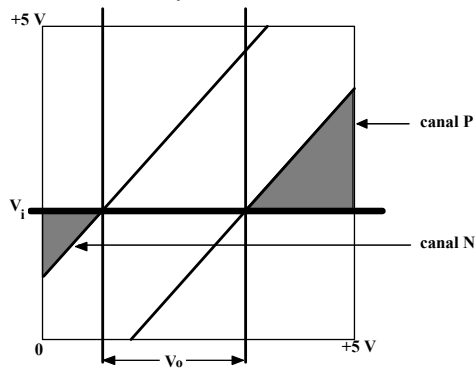
$\beta = (W_1/L_1) / (W_2/L_2) = ff_1 / ff_2$ es la relación de geometrías entre ambos transistores.

Para cualquier tensión de entrada V_i la tensión de salida V_o es la que corresponde a la siguiente relación entre áreas en el diagrama de Memelink: $A_N = (\beta/3) \cdot A_P$.

Tensión de conmutación

Supongamos ambos transistores iguales en dimensiones $ff_P/ff_N = \beta = 1$, por ejemplo, ambos de tamaño mínimo, ¿cuál será la tensión de conmutación del inversor?:

Para tensión de alimentación 5 V y tensión umbral de ambos transistores igual a 1 V:



Se aprecia en la figura que para la tensión de entrada V_i la tensión de salida V_o no es un valor único, sino un amplio intervalo de tensiones; dicho intervalo corresponde, precisamente, a la conmutación de la salida.

La tensión de entrada V_i a la cual se produce la conmutación es aquella para la cual las áreas de los dos triángulos rectángulos e isósceles (formados por V_i con $V_{TO} + V$) cumplen la relación anterior, ya que para dicha tensión de entrada existe todo un intervalo de tensiones de salida posibles, intervalo que corresponde al salto brusco de la conmutación.

$$\left. \begin{aligned} A_P &= \frac{1}{2} (4 - V_i)^2 \\ A_N &= \frac{1}{2} (V_i - 1)^2 \end{aligned} \right\} A_N = \frac{\beta}{3} A_P = \frac{A_P}{3}$$

$$\rightarrow \sqrt{3}(V_i - 1) = (4 - V_i) \rightarrow V_{i, \text{conmutación}} \approx 2.1 \text{ V}$$

La función de transferencia no es simétrica ($V_{i, \text{conmutación}} \neq 2.5 \text{ V}$) sino que la conmutación está desplazada hacia valores inferiores de V_i , debido a la menor movilidad de los huecos que repercute en una menor conductividad del transistor PMOS.

En la conmutación, la salida puede adoptar cualquier tensión dentro del intervalo señalado en la figura que viene delimitado por los vértices de ambos triángulos. Teniendo en cuenta que la recta $V_{TO} + V$ tiene una pendiente de 45° , los dos catetos de dichos triángulos son iguales y su valor es respectivamente:

NMOS: $V_{\text{conmutación}} - V_{TO, \text{NMOS}} = 1.1 \text{ V}$; $V_o(\text{vértice del triángulo}) = 1.1 \text{ V}$

PMOS: $V_{CC} - [V_{TO, \text{PMOS}} + V_{\text{conmutación}}] = 1.9 \text{ V}$; $V_o(\text{vértice del triángulo}) = 5 - 1.9 = 3.1 \text{ V}$

Es decir, la conmutación se produce con una variación vertical de la tensión de salida entre 1.1 y 3.1 V: $V_o = [1.1 - 3.1] \text{ V}$.

La intensidad que circula por el inversor en dicha situación de conmutación, supuestos transistores de tamaño mínimo ($ff = W/L = 1.5$), será:

$$I = K_p \cdot \frac{W}{L} \cdot A_N = K_p \cdot 1.5 \cdot \frac{1}{2} (V_i - 1)^2 = \frac{3K_p}{4} (V_{i, \text{conmutación}} - 1)^2.$$

Para la tecnología Ψ :

$$K_p (\text{NMOS}) \sim 40 \mu\text{A/V}^2; \quad I = \frac{120 \mu\text{A}}{4 \text{ V}^2} \cdot (2.1 - 1)^2 = 36 \mu\text{A}$$

Es posible conseguir una función de transferencia simétrica, compensando, a través de β , la relación entre las movilidades: si $\beta = 3$, resulta $A_P = A_N$ y $V_i = 2.5 \text{ V}$; para ello es necesario hacer el transistor PMOS de anchura triple, de manera que ocupará mayor superficie de integración y, además, aumentarán los tiempos de propagación de la puerta anterior: los transistores PMOS dejan de ser mínimos y aumentan sus dimensiones y su capacidad de puerta. No es, por ello, una buena solución y, en la práctica, no suele hacerse este «equilibrado» de los transistores PMOS.

Habida cuenta de que la precisión del proceso de fabricación es limitada, cabe preguntarse en qué medida una desviación en las dimensiones de los transistores afecta a la tensión de conmutación. Supongamos que, por alguna razón, se modifican las dimensiones de los transistores y no resultan exactamente iguales; expresando la tensión de conmutación V_i en función de la relación de geometrías β :

$$\frac{1}{2}(V_i - 1)^2 = \frac{\beta}{3} \frac{1}{2}(4 - V_i)^2; \quad (V_i - 1) = \sqrt{\beta/3}(4 - V_i); \quad V_i = \frac{4\sqrt{\beta/3} + 1}{\sqrt{\beta/3} + 1} = 4 - \frac{3}{\sqrt{\beta/3} + 1}$$

Si β en lugar de 1 pasa a valor 2 (modificación muy amplia, del 100 %, que equivale a un factor de forma del transistor PMOS doble) entonces $V_i = 2,35$ V (desviación en la tensión de conmutación de 0,25 V < 12 %) y, en cambio, si β pasa a 0,5 (factor de forma del transistor NMOS doble) entonces $V_i = 1,87$ (desviación 0,23 V < 11 %).

Es decir, el efecto de una desviación en las dimensiones de los transistores sobre la tensión de conmutación del inversor es muy pequeño.

Función de transferencia

El diagrama de Memelink del inversor (dos página atrás) permite obtener la curva de transferencia ($V_o - V_i$) punto a punto: dado un valor de V_i , calcular el correspondiente de V_o ; puede efectuarse un análisis por tramos, obteniendo las expresiones algebraicas que relacionan V_o con V_i en cada tramo. Los tramos diferenciados son los siguientes:

- | | |
|----------------------------------|--|
| I $V_i < V_{TO,NMOS}$ | el transistor N no conduce y $V_o = V_{CC}$ |
| II $V_i < V_{conmutación}$ | el área de conducción del transistor N es triangular y, en cambio, el área correspondiente al P es trapezoidal |
| III $V_i = V_{conmutación}$ | el área de conducción del transistor N es triangular y el área correspondiente al P también es triangular |
| IV $V_i > V_{conmutación}$ | el área de conducción del transistor N es trapezoidal y, en cambio, el área correspondiente al P es triangular |
| V $V_i > V_{CC} - V_{TO,PMOS} $ | el transistor P no conduce y $V_o = 0$ |

Supuesto un factor de forma 1 (ambos transistores iguales), con tensiones umbrales $V_{TO} = 1$ V y tensión de alimentación $V_{CC} = 5$ V, la tensión de conmutación será de 2,1 V (según los cálculos anteriores) y la de salida en los cuatro tramos antedichos se ajustará a las siguientes expresiones:

- | | |
|--------------------------|---|
| I $V_i < 1$ V | $V_o = 5$ V |
| II 1 V < V_i < 2,1 V | $V_o = V_i + 1 + \sqrt{13 - 2V_i - 2V_i^2}$ |
| III $V_i = 2,1$ V | $V_o = [1,1 ; 3,1]$ |
| IV $2,1$ V > V_i < 4 V | $V_o = V_i - 1 - \sqrt{(2V_i^2 + 2V_i - 13) / 3}$ |
| V $V_i > 4$ V | $V_o = 0$ V |

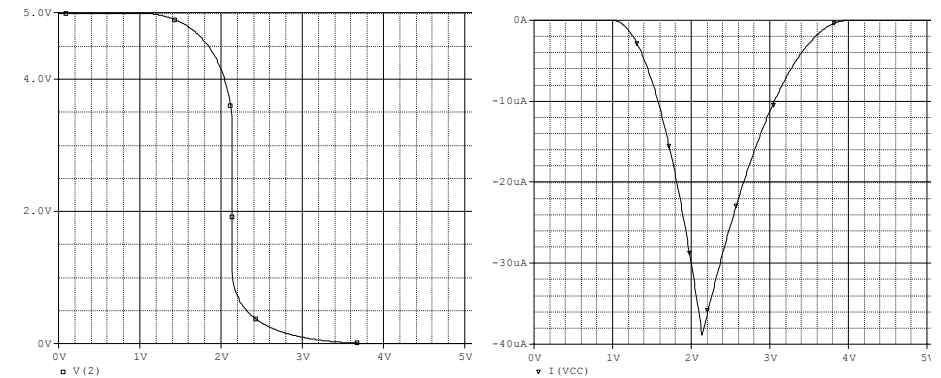
Para la tensión de conmutación $V_{conmutación} = 2,1$ V, las expresiones de los tramos II y IV proporcionan, respectivamente, los valores de 3,1 V y 1,1 V entre los cuales conmuta el inversor en dicha tensión:

$$V_i = V_{conmutación}$$

$$V_o = [3,1 ; 1,1] \text{ V}$$

También puede obtenerse la función de transferencia mediante simulación SPICE: las siguientes gráficas muestran, respectivamente, la función de transferencia $V_o - V_i$ y el consumo de intensidad de un inversor CMOS de la tecnología Ψ :

$L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$, $|V_{TO}| = 1$ V, $K_{p,NMOS} = 40 \mu\text{A/V}^2$ y $K_{p,PMOS} = 15 \mu\text{A/V}^2$.

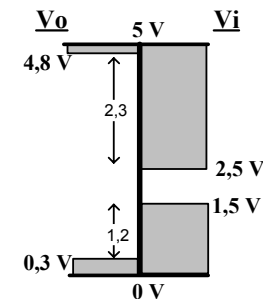


Función de transferencia $V_o - V_i$ CMOS

Consumo de intensidad I_{CC} del inversor

En la función de transferencia se aprecia que, en relación con el «ruido», es razonable extender la tensión de entrada correspondiente al **0** hasta 1,5 V (para dicho intervalo, $V_o(\mathbf{0}) > 4,8$ V) y la tensión de entrada para valor **1** puede tomarse desde 2,5 V (intervalo para el cual $V_o(\mathbf{0}) < 0,3$ V):

para $V_i < 1,5$ V, $V_o > 4,8$ V y para $V_i > 2,5$ V, $V_o < 0,3$ V.



Márgenes de ruido en tensión:

$$\Delta V(\mathbf{0}) = V_{i,m\acute{a}x}(\mathbf{0}) - V_{o,m\acute{a}x}(\mathbf{0}) = 1,5 - 0,3 = 1,2 \text{ V}$$

$$\Delta V(\mathbf{1}) = V_{o,m\acute{i}n}(\mathbf{1}) - V_{i,m\acute{i}n}(\mathbf{1}) = 4,8 - 2,5 = 2,3 \text{ V}$$

Es mayor el margen correspondiente al **1** a causa del desplazamiento de la función de transferencia hacia tensiones bajas, debido a la menor movilidad de los huecos.

Margen de ruido en potencia: $\Delta P = (\Delta V)^2 / R_o$. El margen de ruido en potencia (o sea, en energía por unidad de tiempo) es más significativo que el margen en tensión, ya que el ruido actúa como energía perturbativa sobre el circuito digital.

Resistencia de salida

La resistencia de salida es un parámetro indicativo del comportamiento de las puertas lógicas (una referencia de calidad de las mismas), en cuanto a intensidad disponible en la salida, inmunidad frente al ruido y tiempos de propagación, pues afecta fuertemente a estos tres aspectos: en principio, cuanto menor sea la resistencia de salida mayores serán la intensidad suministrable por la misma, el margen frente al ruido y la velocidad de trabajo. También interesa resistencia de salida baja en relación con el acoplo en tensión, pero dicho acoplo ya viene garantizado por el altísimo valor de la resistencia de entrada (que es cuasi-infinita).

Las situaciones booleanas corresponden a un transistor en zona lineal: para salida 0 el transistor NMOS se encontrará conduciendo en su zona lineal, mientras que para salida 1 será el transistor PMOS el que se encuentre en zona óhmica; de forma que $R_o(0)$ y $R_o(1)$ corresponden, respectivamente, a las resistencias que presentan los transistores NMOS y PMOS en zona lineal.

Valores de las resistencias de salida (tecnología Ψ):

Para transistores de tamaño mínimo ($W=1,5L$; $ff=1,5$) con tensión umbral de 1 V y tensión de puerta de 5 V los valores de la resistencia equivalente en zona lineal serán:

NMOS	PMOS
$K_p \approx 40 \mu A/V^2$	$K_p \approx 15 \mu A/V^2$
$R_{eq} = 1 / (40 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 4 \text{ k}\Omega$	$R_{eq} = 1 / (15 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 11 \text{ k}\Omega$
De manera que las resistencias de salida en situación booleana serán:	
$R_o(0) \sim 4 \text{ K}\Omega$	$R_o(1) \sim 11 \text{ K}\Omega$

Los valores de las resistencias de salida de los inversores de tamaño mínimo son del orden de varios $K\Omega$, $\sim 10^3$ ohmios; no son valores pequeños, pero son adecuados para el comportamiento de las puertas booleanas en el interior del circuito integrado. Téngase en cuenta que las capacidades de entrada de dichas puertas son pequeñas (del orden del fentofaradio, 10^{-15} F) y, por ello, también lo son las intensidades que se requieren y las constantes de tiempo que se generan ($10^3 \cdot 10^{-15} \sim ps$); asimismo, habida cuenta de que las dimensiones de las conexiones son muy reducidas, el efecto del ruido es muy pequeño.

En los terminales de salida de un circuito integrado interesan resistencias de salida inferiores a 100 ohmios (pues en este caso, se requieren intensidades del orden de 10 mA, las capacidades de carga son del orden de 10 pF y las longitudes de las conexiones van en centímetros); en cambio, en las puertas interiores son adecuadas resistencias del orden de 10 K (por las razones antedichas: intensidades del orden de 0,1 mA, capacidades de 1 fF y longitud de las conexiones en micras).

En todo caso, puede disminuirse el valor de la resistencia de salida aumentando en igual medida la anchura **W** del correspondiente transistor, el PMOS para $R_o(1)$ y el NMOS para $R_o(0)$; esto es lo que se hace, precisamente, en los adaptadores de las salidas del circuito integrado.

La intensidad suministrable por el inversor, en cada uno de sus dos estados, depende de la caída de tensión que admitamos en la salida: $I_o = \Delta V_o / R_o$. Esta intensidad es relativamente pequeña y, en ocasiones, no es suficiente; por ejemplo, en los citados adaptadores de las salidas, que precisan de una capacidad de intensidad (hacia el exterior del circuito integrado) del orden de 10 mA. En tales casos, basta aumentar adecuadamente el factor de forma de los transistores ($ff = W / L$), es decir, es necesario hacer $W \gg L$.

Para la tecnología Ψ:

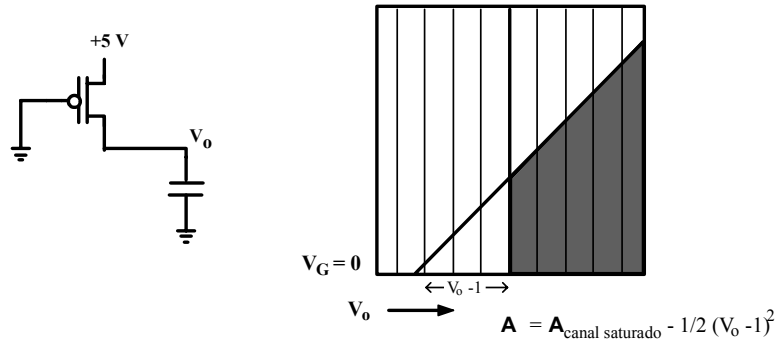
NMOS	PMOS
$R_o(0) \sim 4 \text{ K}\Omega$	$R_o(1) \sim 11 \text{ K}\Omega$
Intensidad suministrable por el inversor: $I_o = \Delta V_o / R_o$	
Admitiendo $\Delta V(0) = 0,2 \text{ V}$ y $\Delta V(1) = 0,6 \text{ V}$ que son valores aceptables (cuya diferencia va en relación a la asimetría de la función de transferencia y a la diferencia de márgenes de ruido):	
$I_o(0) \sim 0,2 / 4K \approx 50 \mu A$	$I_o(1) \sim 0,6 / 11K \approx 50 \mu A$
Haciendo $W > L$ se consiguen mayores intensidades de salida:	
para $W = 15L$	$I_o \approx 0,5 \text{ mA}$
para $W = 50L$	$I_o \approx 1,5 \text{ mA}$
para $W = 300L$	$I_o \approx 10 \text{ mA}$
Margen de ruido en potencia del inversor mínimo:	
$\Delta P(0) \sim 1,2^2 / 4 \text{ K} \approx 0,36 \text{ mW}$	$\Delta P(1) \sim 2,3^2 / 11 \text{ K} \approx 0,48 \text{ mW}$

El ruido actúa efectivamente como potencia: energía perturbativa por unidad de tiempo. En los terminales de un circuito integrado se requieren márgenes de ruido superiores de 20 mW para ambientes «normales» (viviendas o lugares públicos) y a 100 mW en ambientes «industriales»; en el interior de los circuitos integrados (en donde la captación y efecto del ruido es muy inferior, ya que las conexiones son «milimétricas»), son suficientes márgenes del orden de 0,1 mW.

T7.2. Tiempo de propagación y mejora del mismo

T7.2.1. Tiempo de conmutación de una puerta CMOS

La salida de una puerta booleana se encontrará conectada a la entrada de otra u otras puertas (u otros componentes del circuito global), cuyo efecto equivalente es el de una capacidad que ha de cargarse y descargarse en la conmutación, originando unos tiempos de retraso hasta que el proceso de carga o descarga alcanza los valores de tensión apropiados para el 0 y 1 booleanos.



Proceso de carga a través de un transmisor PMOS

Estudiemos el proceso de carga, expresando los tiempos de retraso en función de su intensidad con canal saturado $I_{SAT} = I$ y de la capacidad de carga $C_L = C$ que soporta:

- $I = I_{SAT} = \alpha \cdot A_{\text{canal saturado}}$, siendo A el área del triángulo de conducción en el diagrama de Memelink;
- supuestos $V_{CC} = 5 \text{ V}$ y $V_{TO} = 1 \text{ V}$: $I = \alpha \cdot 4^2 / 2$; $\alpha = I / 8$

En el proceso de carga, existen dos tiempos diferenciados:

t_1 : V_o de 0 a 1 V canal saturado $I_1 = I$, $\Delta Q = I \cdot t$, $t = C \cdot \Delta V / I$, $t_1 = C / I$.

t_2 : V_o de 1 a 5 V canal no saturado $I_2 < I$ y decrece al aumentar V_o , $I(t) dt = C dV$:

$$I(t) = I - \alpha \cdot (V - 1)^2 / 2 = I - (I/16) (V - 1)^2 = (I/16)(16 - (V - 1)^2)$$

$$(I/16) (16 - (V - 1)^2) dt = C dV; \quad \frac{I}{16C} dt = \frac{d(V - 1)}{4^2 - (V - 1)^2}$$

Integrando ambos miembros de la ecuación diferencial anterior:

$$\frac{I}{16C} t = \frac{1}{2.4} \ln \frac{4 + (V - 1)}{4 - (V - 1)}; \quad \frac{4 + (V - 1)}{4 - (V - 1)} = e^{\frac{I}{2C} t}$$

$$V(t) = 1 + 4 \frac{e^{\frac{I}{2C} t} - 1}{e^{\frac{I}{2C} t} + 1} = 1 + 4 \operatorname{tgh} \frac{I}{4C} t$$

La función tangente hiperbólica **tgh(x)** es fuertemente creciente (del tipo $1 - e^{-x}$, con pendiente aún mayor), de forma que tiende a 1 asintóticamente y alcanza el valor 0.76 (>75 %) para $x = 1$.

La conmutación de la salida hacia 5 V alcanzará 4 V en un tiempo aproximado al que corresponde a $x = 1$:

$$(I/4C) t_2 = 1 \quad t_2 = 4C / I \quad t_{\uparrow} = t_1 + t_2 = 5C / I$$

donde I es la intensidad de canal saturado del transistor PMOS.

La determinación del tiempo de descarga a través del transistor NMOS, hasta disminuir a 1 V, es análoga y conduce a la misma expresión $t_{\downarrow} = t_1 + t_2 = 5C / I$:

$$t_{\uparrow} = 5C / I(P) \quad t_{\downarrow} = 5C / I(N)$$

Para la tecnología Ψ :

y para $V_{CC} = 5 \text{ V}$ y $V_{TO} = 1 \text{ V}$:

$$L_{\text{mín}} = 1 \mu\text{m}, \quad W = 1,5 \mu\text{m}, \quad C_i \sim 2 \text{ fF}$$

$$I(P) = I_{\text{sat}} (\text{canal P}) \approx 0,2 \text{ mA}$$

$$I(N) = I_{\text{sat}} (\text{canal N}) \approx 0,5 \text{ mA}$$

para $\text{fan-out} = 1$ (una sola entrada conectada a la salida de un inversor mínimo):

$$t_{\uparrow} \approx 10 \text{ fF} / 0,2 \text{ mA} \sim 0,05 \text{ ns}$$

$$t_{\downarrow} \approx 10 \text{ fF} / 0,5 \text{ mA} \sim 0,02 \text{ ns}$$

El tiempo de subida es superior al de bajada como consecuencia de la menor movilidad de los huecos que conforman el canal del transistor PMOS; ello podría evitarse dimensionando adecuadamente dicho transistor pero implicaría mayor área de integración y, a la vez, mayor capacidad de carga de cada entrada lo cual redundaría en empeorar los tiempos de conmutación de la puerta anterior. Por ello, no interesa dimensionar las puertas lógicas para conseguir su simetría funcional.

Nos encontramos con tiempos de conmutación inferiores a un nanosegundo, que dependen linealmente de la capacidad de carga y, por ello, del *fan-out* y de las dimensiones de los transistores:

- al aumentar el *fan-out*, los tiempos de conmutación crecen en la misma proporción
- al disminuir las dimensiones de la tecnología los tiempos de conmutación se reducen cuadráticamente (C_i disminuye con L y con W).

Para una tecnología de 0,5 micras:

$$L_{\text{mín}} = 0,5 \mu\text{m}, \quad W = 0,75 \mu\text{m}, \quad C_i \sim 0,5 \text{ fF}$$

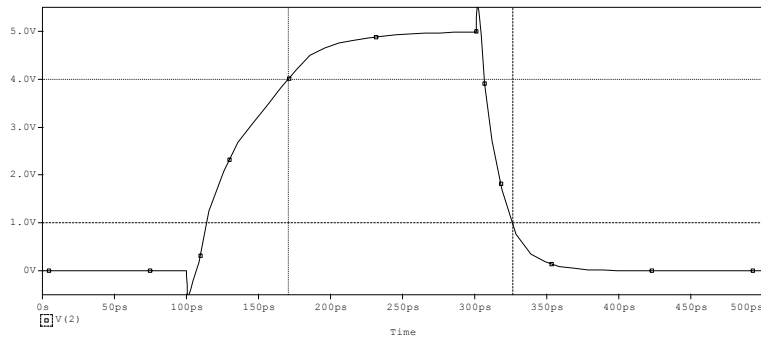
para $\text{fan-out} = 1$ (una sola entrada conectada a la salida de un inversor mínimo):

$$t_{\uparrow} \approx 2,5 \text{ fF} / 0,2 \text{ mA} \sim 0,015 \text{ ns}$$

$$t_{\downarrow} \approx 2,5 \text{ fF} / 0,5 \text{ mA} \sim 0,005 \text{ ns}$$

La siguiente gráfica muestra la conmutación de un inversor con un *fan-out* de 1, es decir, a su salida se encuentra conectado otro inversor análogo; la simulación SPICE se ha efectuado con los datos de la tecnología Ψ y transistores de tamaño mínimo:

$L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$, $|V_{TO}| = 1 \text{ V}$, $K_{p,NMOS} = 40 \mu\text{A/V}^2$ y $K_{p,PMOS} = 15 \mu\text{A/V}^2$.



Onda de conmutación de un inversor: $V(2) = V_o$

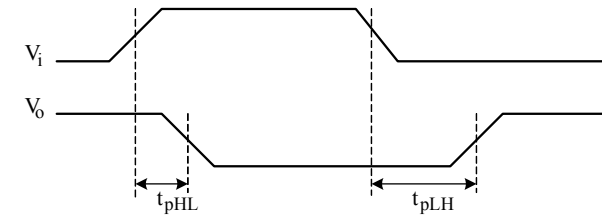
Los tiempos medidos en esta gráfica son: $t_{\uparrow} \approx 0,071 \text{ ns}$ y $t_{\downarrow} \approx 0,026 \text{ ns}$.

Los tiempos de conmutación dependen también fuertemente de la temperatura, ya que al aumentar ésta disminuye fuertemente la movilidad de los portadores (aumentan sus choques con los núcleos de la red cristalina) y, en consecuencia, se reduce K_p y la intensidad de saturación de los transistores. De ahí, el alto interés de evitar el calentamiento de los circuitos integrados, disipando adecuadamente el calor generado por la potencia consumida en ellos, si se desea aprovechar la velocidad máxima de trabajo.

El retardo que introduce un inversor es algo superior al tiempo de conmutación calculado en este apartado: dicho tiempo de conmutación ha sido determinado en relación con la capacidad de carga C_L (*fan-out*) y, en la práctica, influyen también otros efectos capacitivos internos del propio inversor.

Al conectar varios inversores seguidos, el tiempo de propagación del conjunto es mayor que los tiempos de conmutación individuales pero es inferior a la suma de ellos, ya que existe solapamiento entre ellos. Es decir, los tiempos de conmutación no son linealmente acumulativos ya que cada puerta inicia su conmutación antes de que la anterior complete la suya.

Para facilitar la suma de tiempos, cada tiempo de propagación suele medirse por el retraso entre el punto medio de la conmutación (tensión $V_{cc}/2$) de la onda de entrada y el punto medio de la conmutación de la señal de salida.



Sin ser exactos, las formulas y cálculos anteriores ofrecen una buena estimación del orden de magnitud y una buena aproximación al valor real de los tiempos de propagación. La simulación SPICE de los mismos puede proporcionar aún mayor precisión.

Además, habrá que tener en cuenta que cada conexión entre la salida de una puerta y la entrada de la siguiente introduce una línea conductora con sendos efectos resistivo y capacitivo y, con ello, un nuevo sumando al retardo de propagación de las señales; en tecnologías submicrónicas no es extraño que el retardo introducido por las líneas de conexión sea superior al tiempo de propagación de la propia puerta booleana.

T7.2.2. Mejora de los tiempos de propagación

En ocasiones las capacidades de carga son altas; por ejemplo, en los adaptadores de las salidas de los circuitos integrados (que han de ser capaces de soportar su conexión a cargas equivalentes de decenas de picofaradios) o, también, en puertas que deban soportar a su salida un *fan-out* alto (por ejemplo, las que transmiten a los diversos biestables la señal de reloj) o largas longitudes de polisilicio. En tales casos, los tiempos de conmutación se elevan en demasía y es preciso reducirlos mediante adaptadores de intensidad:

Para la tecnología Ψ :

Para una capacidad de carga de 10 pF (5.000 veces superior a C_i -*fan-out* 1-):

$$t_{\uparrow} = 5C / I(P) \approx 50 \text{ pF} / 0,2 \text{ mA} \approx 0,3 \mu\text{s}$$

$$t_{\downarrow} = 5C / I(N) \approx 50 \text{ pF} / 0,5 \text{ mA} \approx 0,1 \mu\text{s}$$

tiempos que resultan relativamente altos y limitan la velocidad a unos pocos MHz.

Para disminuir los tiempos de propagación causados por altas capacidades de carga o, lo que es lo mismo, por la necesidad de intensidades de salida altas, se utilizan esquemas de amplificación en cascada con inversores cuya anchura de transistor es progresivamente creciente.

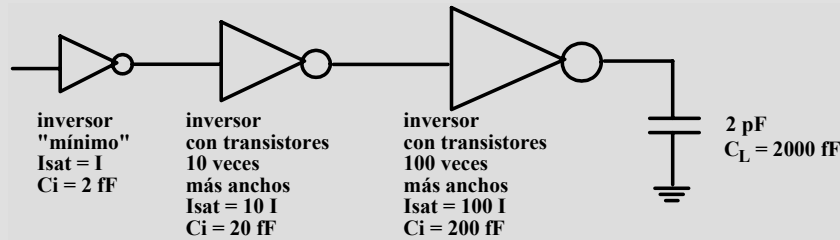
Ejemplo desarrollado con la tecnología Ψ :

Supongamos una carga de 2 pF que resulta ser unas 1.000 mayor que C_i (capacidad de carga que corresponde a *fan-out* 1):

$$t_{\uparrow} = 5C / I(P) \approx 10 \text{ pF} / 0,2 \text{ mA} \approx 50 \text{ ns}$$

$$t_{\downarrow} = 5C / I(N) \approx 10 \text{ pF} / 0,5 \text{ mA} \approx 20 \text{ ns}$$

Añadiendo un par de inversores cuya anchura de transistores sea, en cada uno de ellos, 10 veces la del anterior:



Como puede apreciarse en la figura, en cada inversor se produce un «salto» entre su capacidad de carga y su capacidad de entrada de 10; en total, un salto de 1000 que es la relación existente entre los 2 pF y C_i (capacidad de entrada del inversor mínimo).

$$t_{1\uparrow} = 5 \times 20 / 0,2 \approx 0,5 \text{ ns} \quad t_{2\uparrow} = 5 \times 200 / 2 \approx 0,5 \text{ ns} \quad t_{3\uparrow} = 5 \times 2000 / 200 \approx 0,5 \text{ ns}$$

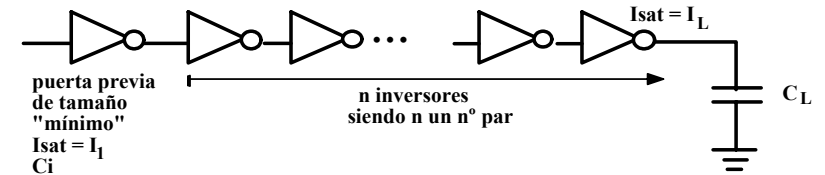
$$t_{1\downarrow} \approx 5 \times 20 / 0,5 \approx 0,2 \text{ ns} \quad t_{2\downarrow} \approx 5 \times 200 / 5 \approx 0,2 \text{ ns} \quad t_{3\downarrow} \approx 5 \times 2000 / 500 \approx 0,2 \text{ ns}$$

$$t_{\uparrow} \sim t_{1\uparrow} + t_{2\uparrow} + t_{3\uparrow} \approx 1,2 \text{ ns} \quad t_{\downarrow} \sim t_{1\downarrow} + t_{2\downarrow} + t_{3\downarrow} \approx 0,9 \text{ ns}$$

Ambos tiempos son muy inferiores a los que presenta el primer inversor en solitario: 50 ns y 20 ns, respectivamente.

Este esquema (en que se incluyen parejas de inversores «en escalera», es decir, con anchura de transistores creciente) resulta sumamente útil para configurar adaptadores de intensidad (*buffers*), tanto para los terminales de salida del circuito integrado como para aquellas conexiones internas que han de soportar un alto *fan-out* o alta carga capacitiva y, por tanto, han de proporcionar alta intensidad.

Un adaptador de intensidad puede incluir más de una pareja de inversores «en escalera» (siempre un número par para que no se produzca una inversión suplementaria), de forma que la relación de intensidad entre el último de ellos y la puerta «mínima» inicial se distribuya en múltiples «saltos» pequeños.



Se puede calcular el número óptimo de inversores para conseguir el menor tiempo de propagación global:

- sea K el cociente entre la capacidad de carga que debe soportar el conjunto y la correspondiente a una entrada mínima (primera puerta del conjunto): $K = C_L / C_i$
- si el necesario aumento del *fan-out* se distribuye de forma homogénea siendo ∂ el coeficiente incremental $C_{i+1} = \partial \cdot C_i$, será necesario que las anchuras W de transistores de dos inversores sucesivos se incrementen en dicho factor ∂ : $W_{i+1} = \partial \cdot W_i$
- la relación de anchuras repercute en proporción directa en la relación entre capacidades de entrada C_i y entre intensidades de saturación I_{sat} ; se verificará que

$$W_{i+1} / W_i = C_{i+1} / C_i = I_{sat,i+1} / I_{sat,i} = \partial$$

- el factor incremental global K corresponde a $n+1$ incrementos ∂ sucesivos:

$$K = C_L / C_{in} = \partial^{n+1}, \quad \partial = \sqrt[n+1]{K} \quad \text{y el incremento en intensidad será: } I_L / I_1 = \partial^n$$

Los tiempos de propagación de los inversores serán iguales, ya que la relación entre intensidad de saturación y capacidad de carga es constante, y dichos tiempos serán proporcionales a ∂ ya que dicho factor representa la carga relativa a su tamaño que cada inversor soporta. Por tanto, el tiempo de propagación del conjunto será proporcional a $(n+1) \cdot \partial$ y la situación de tiempo mínimo de propagación corresponde a:

$$\frac{d[(n+1) \cdot \partial]}{dn} = 0; \quad \partial = \sqrt[n+1]{K}$$

$$\frac{d((n+1) \cdot \sqrt[n+1]{K})}{dn} = 0; \quad \sqrt[n+1]{K} + (n+1) \cdot \sqrt[n+1]{K} \cdot \ln K \cdot \left[-\frac{1}{(n+1)^2} \right] = 0$$

$$1 - \ln K \cdot \frac{1}{(n+1)} = 0; \quad n+1 = \ln K; \quad K = e^{n+1};$$

$$\partial = \sqrt[n+1]{K} = e = 2,7$$

La adaptación óptima de la «escalera de inversores», para conseguir tiempos mínimos de propagación, se consigue cuando el número de inversores es $n + 1 = \ln K$ y, en este caso, el coeficiente incremental θ coincide con el número e :

$$W_{i+1} = 2,7 \cdot W_i \quad C_{i+1} = 2,7 \cdot C_i \quad I_{sat,i+1} = 2,7 \cdot I_{sat,i}$$

El tiempo de propagación de cada inversor será $\theta = e = 2,7$ veces el correspondiente a un inversor mínimo conectado a otro inversor análogo.

En el ejemplo desarrollado con la tecnología Ψ :

$$C_L = 2 \text{ pF}; C_{in} = 2 \text{ fF}; K = 1000; n+1 = \ln(1000) \approx 7$$

El mínimo tiempo de propagación se obtiene con 6 inversores intermedios, cuya anchura de transistores sea, en cada uno de ellos, 2,7 veces mayor que en el anterior.

Los tiempos de propagación de cada uno de los inversores son los siguientes:

$$t_{\uparrow} = 2,7 \times 5 \times 2 \text{ fF} / 0,2 \text{ mA} \approx 0,15 \text{ ns} \quad t_{\downarrow} = 2,7 \times 5 \times 2 \text{ pF} / 0,5 \text{ mA} \approx 0,05 \text{ ns}$$

De forma que, los tiempos de propagación para el conjunto total serán:

$$t_{\uparrow} \sim 4t_{\uparrow} + 3t_{\downarrow} \approx 0,8 \text{ ns} \quad t_{\downarrow} < 4t_{\downarrow} + 3t_{\uparrow} \approx 0,7 \text{ ns.}$$

Tales son los tiempos mínimos de propagación que pueden conseguirse, claramente inferiores a los que presentaría el primer inversor actuando en solitario sobre la capacidad de carga 2 pF (60 ns y 20 ns, respectivamente) y solamente un poco inferiores a los que se obtienen con una sola pareja de inversores en escalera (1,5 ns y 1 ns).

Como puede apreciarse añadir una primera pareja de inversores (adecuadamente dimensionados) tiene un gran efecto tiene respecto a la reducción de tiempos; cuando se añaden más parejas de inversores (ajustando en cada caso su anchura W), la inclusión de cada una de ellas tiene un efecto sucesivamente menor que la anterior. Por ello, debe valorarse la reducción de tiempos que se desea conseguir en relación con el área de silicio que se va a necesitar.

T7.3. Disipación de potencia

La potencia disipada por un inversor CMOS en situación estática es prácticamente nula, pues no existe ninguna línea de conducción directa entre alimentaciones (siempre uno de los dos transistores correspondientes a cada entrada se encuentra en corte). Ahora bien, en cada transistor existen uniones PN polarizadas inversamente (uniones difusión-substrato) que conducen una minúscula intensidad inversa, cuyo orden de magnitud es del picoamperio ($\text{mA} / 10^6$).

Ahora bien, al conmutar el inversor han de cargarse o descargarse las capacidades propias de sus transistores, en particular, su capacidad de puerta, lo cual determina un consumo dinámico proporcional a la frecuencia de conmutación:

- energía necesaria para cargar o descargar un condensador: $1/2 C \cdot V^2$
- potencia disipada al cargarlo y descargarlo con una onda de frecuencia f (en cada período hay dos conmutaciones, habrá que contabilizar una carga y una descarga): $(1/2 C \cdot V^2) \cdot 2f = C \cdot V^2 \cdot f$

Esta potencia (energía por unidad de tiempo) representa un consumo de intensidad desde la alimentación y una disipación de calor en el propio circuito. Son dos aspectos complementarios, relativos al consumo de intensidad, que requieren la correspondiente atención en el diseño y utilización del circuito integrado: una fuente de alimentación con suficiente capacidad de suministro de corriente y una disipación de calor adecuada. Teniendo en cuenta, además, que los tiempos de propagación aumentan con la temperatura y, en consecuencia, disminuye la velocidad de trabajo.

Conforme a la anterior expresión, la potencia consumida y disipada depende de:

- la tecnología, cuyas dimensiones determinan la capacidad C ; dicha capacidad equivalente de una puerta a efectos de consumo de intensidad tiene varios componentes, entre los cuales predomina la capacidad de entrada (las otras capacidades internas de los transistores son de valores muy inferiores) que es proporcional a la superficie de las zonas de puerta, es decir, al producto $L \cdot W$ y disminuye cuadráticamente al disminuir L ($W = 1,5L$);
- la tensión de alimentación, que también afecta cuadráticamente V^2 ; existe, por ello, una evolución continuada de la microelectrónica hacia tensiones de alimentación más bajas (el paso de 5 V a 3 V reduce el consumo a la tercera parte), siendo cada vez más frecuentes dispositivos de 2,5 V y de 1,8 V;
- la frecuencia de trabajo f , aumentando linealmente con ella (lo cual aconseja no trabajar a frecuencia más alta de la estrictamente necesaria para el sistema digital)

Al evaluar el consumo de un circuito integrado, debe tenerse en cuenta que no todas las puertas del mismo conmutan cada vez, sino que solamente lo hace una pequeña parte de ellas; por ejemplo, la escritura de un dato sobre una memoria de 1 Mega solamente afecta a uno de sus 1048576 registros (y al decodificador de direcciones y control de escritura).

Disipación de potencia (tecnología Ψ):

Para transistores de tamaño mínimo ($L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$) y espesor de óxido de puerta t_{ox} de 50 nm, el valor de la capacidad de puerta será:

$$C_{ox} = 0,7 \text{ fF} / \mu\text{m}^2; \quad C_G = C_{ox} \cdot W \cdot L = 0,7 \text{ fF} / \mu\text{m}^2 \cdot 1,5 \mu\text{m} \cdot 1 \mu\text{m} \approx \mathbf{1 \text{ fF}}.$$

Como la entrada del inversor se encuentra conectada a dos transistores, uno de canal N y otro de canal P, la capacidad de entrada será: $C_i \approx \mathbf{2 \text{ fF}}$

Para la tensión de alimentación habitual de 5 V ($V_{CC} = 5 \text{ V}$)

$$\mathbf{P = C \cdot V^2 \cdot f = 2 \text{ fF} \cdot 25 \text{ V}^2 \cdot f = 50 \cdot 10^{-15} \cdot f = \mathbf{0,05 \cdot f \text{ pW}}}$$

$$\text{Para } f = 1 \text{ MHz} = 10^6: \quad \mathbf{P = 50 \text{ nW} / \text{MHz};} \quad \mathbf{I_{CC} = 10 \text{ nA} / \text{MHz}}$$

Al aumentar la frecuencia aumenta la potencia disipada en cada entrada en igual proporción.

Es posible hacer una estimación de la potencia disipada en 1 mm^2 ocupado por puertas CMOS, supuesto que todos los transistores conmuten (lo cual no es una situación normal pues en cualquier bloque digital sólo conmuta una pequeña fracción de las puertas contenidas en él). El área activa ocupada por los transistores en cuanto a su zona de puerta ($L \cdot W$) suele ser inferior al 5 % (ya que la mayor parte de la superficie es necesaria para las zonas de fuente y drenaje, conexiones, separaciones,...):

$$1 \text{ mm}^2 = 1000000 \mu\text{m}^2; \quad 5 \% (1 \text{ mm}^2) = 50000 \mu\text{m}^2$$

$$C_{\text{transistores}} = C_{ox} \cdot \text{superficie activa} = 0,7 \cdot 50000 = 35 \text{ pF}$$

$$P = C \cdot V^2 \cdot f = 875 \text{ pW} \cdot f \sim 1 \text{ nW} \cdot f$$

Para $f = 1 \text{ MHz}$, $P \sim 1 \text{ mW} / \text{MHz}$, es decir, supuesto que todos los transistores contenidos en 1 mm^2 de silicio conmutasen, dicho milímetro cuadrado de silicio debería disipar en forma de calor una potencia del orden de 1 mW/MHz ; esta potencia no depende de las dimensiones de la tecnología sino del porcentaje de ocupación: superficie activa/superficie total (entendiendo por superficie activa la ocupada por las puertas de los transistores).

Además de la disipación de potencia de tipo capacitivo (principalmente la capacidad de entrada de los transistores MOS) existe otro efecto dinámico debido a que en la conmutación, durante un breve instante de tiempo, conducen ambos transistores PMOS y NMOS, dando lugar a un estrecho «pico de intensidad»; tal efecto resulta despreciable frente al anterior siempre que la conmutación sea adecuadamente rápida (tiempos de conmutación inferiores a 1 ns).

Sin embargo, dichos «picos de intensidad» deben ser tenidos en cuenta en relación con el «ruido», ya que al ser variaciones muy rápidas de intensidad generarán «picos de tensión» sobre las líneas de alimentación (por efecto inductivo).

El orden de magnitud del «pico de intensidad» en la conmutación fue calculado en el apartado anterior, por medio del área de conducción en el *diagrama de Memelink*; en situación de conmutación ($V_i = 2,1 \text{ V}$), la intensidad que circula por el inversor, supuestos transistores de tamaño mínimo ($ff = W/L = 1,5$), será:

$$I = \frac{3K_p}{4} (V_i - 1)^2.$$

Para la tecnología Ψ :

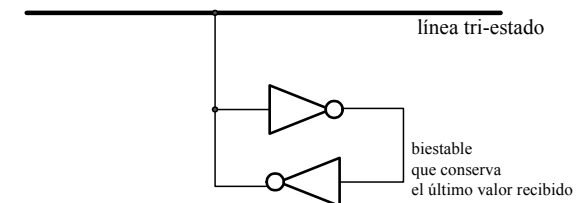
$$I = \frac{340 \mu\text{A}}{4 V^2} (2,1 - 1)^2 = 36 \mu\text{A}$$

$$\mathbf{P = 180 \mu\text{W}}$$

Un inversor, cuya entrada se encuentre en situación de alta independencia (tri-estado) o con un valor de tensión intermedio ($\sim V_{\text{conmutación}}$) genera una disipación de potencia del orden de $0,2 \text{ mW}$ y el correspondiente calentamiento del circuito (un valor unas cuatro mil veces superior al generado en la conmutación de dicho inversor a 1 MHz).

Por otra parte, existe una situación peligrosa en que este pico de intensidad (debido a la conducción «momentánea» de ambos transistores en la conmutación) puede ser estable y originar un fuerte calentamiento (e incluso la destrucción) del circuito integrado: si una entrada queda en alta impedancia tiende a adoptar un valor de tensión intermedia y ambos transistores conducen.

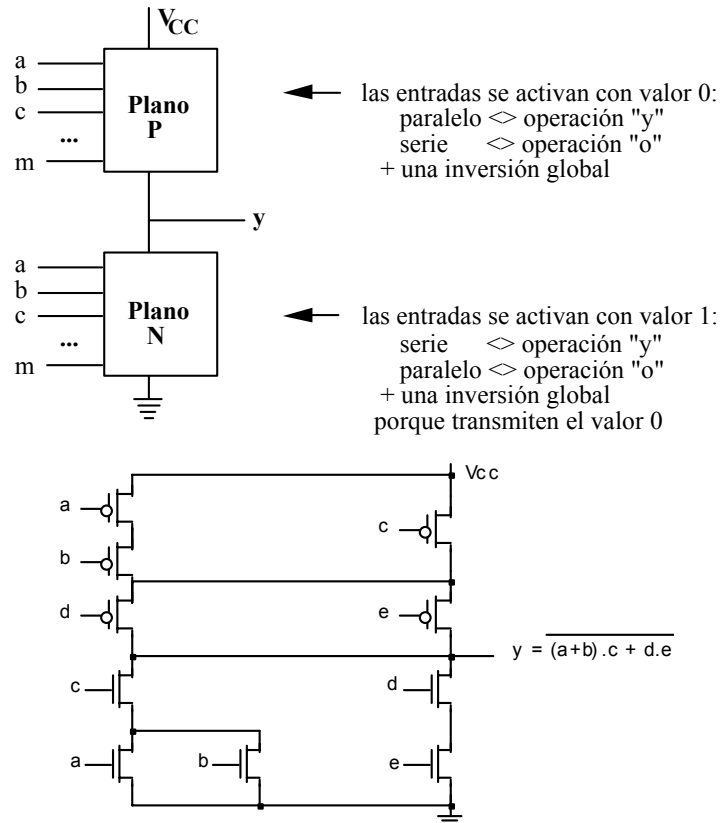
Esta situación puede darse en caso de utilizar buses (o adaptadores tri-estado) internos que puedan quedar en alta impedancia, posibilidad que puede evitarse conectando en cada línea una resistencia de alto valor, que la referencie a 0 V (*pull-down*), o bien un pequeño biestable (un par de inversores) que mantenga el último valor booleano establecido en ella (ver figura).



Conjuntos de circuitos auxiliares de este tipo, cuya finalidad es que las líneas de buses no se queden en estado de alta impedancia, se encuentran también disponibles en los catálogos de circuitos integrados.

T7.4. Puertas CMOS

Haciendo uso del álgebra de conmutadores, las puertas lógicas CMOS se construyen mediante dos planos «duales» de transistores: **plano N** y **plano P**; a cada entrada le corresponden sendos transistores, uno en el plano N y otro en el plano P, conectados de acuerdo con la dualidad serie-paralelo.



- en su plano N, la configuración de transistores corresponde (de acuerdo con el álgebra de conmutadores) a la función $(a + b) \cdot c + d \cdot e$, a la cual debe añadirse una negación global ya que dicho plano N transmite el valor 0; de forma que la función conformada por el plano N de la figura anterior es: $y = (a + b) \cdot c + d \cdot e$
- en cuanto al plano P, transmite el valor 1 pero sus transistores conducen cuando su entrada es 0, es decir, corresponden a variables negadas ($\bar{a}$, $\bar{b}$, $\bar{c}$, $\bar{d}$, $\bar{e}$) y su configuración (de acuerdo con el álgebra de conmutadores) conforma la función $(\bar{a} \cdot \bar{b} + \bar{c}) \cdot (\bar{d} + \bar{e})$; de manera que la función conformada por el plano P de la figura es la misma que la del plano N: $y = (\bar{a} \cdot \bar{b} + \bar{c}) \cdot (\bar{d} + \bar{e}) = (a + b) \cdot c + d \cdot e$

Características físicas de las puertas complementarias

Permiten una amplia diversidad de puertas

El juego de conexiones serie-paralelo permite configurar funciones complejas y muy diversas en una misma puerta, con la limitación de que la expresión algebraica de la función ha de presentar una negación global sobre el conjunto de operaciones booleanas (son puertas inversoras) y de que tal negación sea la única que aparezca: cada negación da lugar a una puerta adicional.

Los transistores P y N presentan diferente resistividad (debida a la menor movilidad de los huecos): la conexión PMOS en paralelo, y sus correspondientes NMOS en serie, contribuye a compensar la asimetría, mientras que la conexión de PMOS en serie acentúa dicha asimetría y su efecto resistivo; por ello, son preferibles las puertas "**y-negada**" (*Nand*) a las puertas "**o-negada**" (*Nor*).

Las entradas son de tipo capacitivo

Los transistores presentan una capacidad de puerta, que es preciso cargar o descargar en la conmutación cuando cambia el valor booleano presente en la correspondiente entrada; tal transitorio de carga o descarga:

- da lugar a unos tiempos de conmutación que limitan su velocidad de trabajo;
- limita, asimismo, el *fan-out* de la puerta anterior, es decir, el número de entradas de otras puertas que pueden conectarse sobre una salida (pues tal número condiciona la velocidad de trabajo);
- requiere un aporte puntual de intensidad durante la conmutación, que da lugar a un consumo dinámico proporcional a la frecuencia de conmutaciones;
- genera «ruido» sobre las líneas de alimentación como consecuencia del «pulso» de intensidad necesario para la conmutación.

La salida de cada puerta es resistiva

Cada plano de transistores, cuando conduce, presenta una resistencia relativa a la zona lineal u óhmica de sus transistores; tal resistencia depende de las dimensiones de los transistores (disminuyendo en proporción inversa a la anchura de estos) y afecta a:

- la intensidad suministrable por la puerta;
- los procesos de conmutación (carga y descarga de las capacidades de entrada de las puertas siguientes) y, en consecuencia, los tiempos de propagación y la velocidad;
- la inmunidad frente al «ruido» en términos de potencia.

Se presentan varios transistores en serie

La presencia de transistores en serie supone un aumento de la resistencia de salida de la puerta, que repercute, en concordancia con el apartado anterior, en la intensidad suministrable por la puerta, en los tiempos de propagación y la velocidad de trabajo y en la inmunidad frente al «ruido» en términos de potencia.

En cuanto a intensidades en la salida de puertas con más de una entrada, ha de tenerse en cuenta aquella situación booleana en que conducen varios transistores en serie, en cuyo caso la intensidad suministrable ha de dividirse, al menos, por el número de ellos (ya que se suman sus resistencias equivalentes) o alternatively ha de aumentarse en igual proporción la anchura de tales transistores.

También el margen de ruido en potencia disminuye por la suma de resistencias de transistores en serie: $\Delta P = (\Delta V)^2 / R_O$.

En los tres aspectos considerados (intensidad suministrable, tiempos de propagación e inmunidad en potencia) interviene la resistencia de salida: $R_O = \Sigma R_O$ (transistores en serie). En una primera aproximación, para **n** transistores se multiplica por **n** la resistencia de salida correspondiente a uno sólo de ellos; pero, además, se produce un efecto de «desplazamiento de la tensión de fuente» que aumenta progresivamente la resistencia de los diversos transistores.

Consideremos el caso de transistores NMOS en serie, todos ellos con tensión de entrada $V_G = V(1)$, solamente el inferior de ellos tiene su fuente conectada a 0 V y su tensión $V_{GS} = V(1)$. Mientras que la tensión de salida sea $V_o > 0$ (lo cual ocurre siempre que $I_o \neq 0$ y, en particular, durante la conmutación), la tensión puerta-fuente de los demás transistores es menor $V_{GS} < V(1)$ pues su terminal de fuente no está conectado a 0 V directamente, sino a través de los transistores que se encuentran debajo; por ello, dicha tensión V_{GS} disminuye al ascender en la serie y la resistencia efectiva que presenta cada transistor es mayor cuanto más alejado se encuentra de la conexión a 0 V.

Por ello, puertas con muchas entradas (muchos transistores en serie) presentan malas características funcionales: es altamente aconsejable limitar el número de entradas de las puertas CMOS, de forma que no aparezcan más de 6 transistores en serie.

El consumo estático es siempre nulo

Para cada valor booleano en una de las entradas, uno de sus transistores se encontrará en corte y el otro conducirá: todo camino de conducción entre los dos terminales de alimentación (V_{CC} y 0 V) incluye siempre un transistor en corte, por lo cual el consumo en reposo es nulo.

En cambio, sí que hay consumo dinámico originado por la carga o descarga de las diversas capacidades propias de los transistores en la conmutación y dicho consumo es proporcional a la frecuencia de conmutación.

Tiempos de conmutación

Al pasar de un inversor a una puerta booleana de dos o más entradas, los tiempos de propagación aumentan en aquellos procesos (carga o descarga) en que la conducción se produce a través de varios transistores en serie; es un efecto de suma de sus resistencias (se suma la longitud de sus transistores).

Para la tecnología Ψ:		
ns	• NAND de 2 entradas:	$t_{\uparrow} = 5C / I(P) \approx 10 \text{ fF} / 0,2 \text{ mA} \approx 0,05 \text{ ns},$
		$t_{\downarrow} = 2 \times 5C / I(N) \approx 2 \times 10 \text{ fF} / 0,5 \text{ mA} \approx 0,04$
		2 transistores en serie
	• NOR de 2 entradas:	$t_{\uparrow} = 2 \times 5C / I(P) \approx 2 \times 10 \text{ fF} / 0,2 \text{ mA} \approx 0,1 \text{ ns},$
		2 transistores en serie
		$t_{\downarrow} = 5C / I(N) \approx 10 \text{ fF} / 0,5 \text{ mA} \approx 0,02 \text{ ns}$

Debido a la diferente resistividad de los transistores PMOS y NMOS, interesa más la utilización de puertas "y-negada" (*Nand*) pues en ellas se equilibran un poco los dos tiempos de conmutación (aumenta el menor de ambos), mientras que para las puertas "o-negada" (*Nor*) se aumenta aún más el tiempo de subida (que es, ya de por sí, el mayor).

Ahora bien, cuando se desea obtener estimaciones cuantitativas de los tiempos de propagación de una puerta o de un conjunto de puertas booleanas es necesario acudir a la simulación eléctrica (SPICE u otros). Por un lado, los efectos relativos a transistores en serie (comentados en la página anterior) y, de otro, el solapamiento entre los tiempos de conmutación de puertas sucesivas (ya que cada puerta inicia su conmutación antes de haber finalizado la conmutación de la puerta anterior) limita, en gran medida, los resultados obtenidos a través de modelos simplificados o de razonamientos cualitativos.

T7 ESTUDIO EN DETALLE DEL INVERSOR CMOS

- T7.1. El inversor CMOS
- T7.2. Tiempo de propagación y mejora del mismo
- T7.3. Disipación de potencia
- T7.4. Puertas CMOS

Interesa estudiar en detalle el comportamiento de las puertas CMOS y ese detalle se refiere a tensiones, intensidades, tiempos y consumos: su función de transferencia (tensión de salida en relación con la tensión de entrada), la intensidad suministrable por la salida, los tiempos de conmutación, la disipación de potencia,... El inversor, como puerta más simple, de una sola entrada, resulta sumamente adecuado para desarrollar este estudio.

En el capítulo anterior, se introdujeron los «diagramas de Memelink» como forma de representar gráficamente la distribución de los portadores de carga en el canal de los transistores MOS y de facilitar los cálculos relativos a tensiones e intensidades. En el caso de un inversor, los «diagramas de Memelink» de los dos transistores que lo forman pueden representarse conjuntamente (en un mismo diagrama), de manera tal que puede calcularse la tensión de salida y la intensidad a través del inversor, para cualquier valor de su tensión de entrada.

De esta forma, en el presente capítulo se calcula la tensión de conmutación del inversor y la intensidad que consume en tal situación y se obtienen, también, las expresiones algebraicas que corresponden a los diversos «tramos» de la función de transferencia y, a partir de ella, los márgenes de ruido.

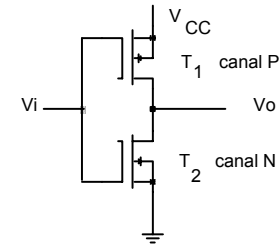
Asimismo es de gran interés comprender los factores que afectan a los tiempos de propagación y cómo dichos tiempos pueden mejorarse mucho, si la carga capacitiva es alta, utilizando una serie de inversores suplementarios, «en escalera» (cada uno de ellos de anchura superior al anterior).

La disipación de potencia en una puerta lógica, su consumo, tiene importancia en dos aspectos complementarios: la fuente de alimentación que ha de proporcionar suficiente intensidad y el calentamiento del circuito que aumenta sus tiempos de propagación y, en ocasiones, necesita adecuada refrigeración.

Por último, se recuerdan las características funcionales de las puertas CMOS y se destaca la limitación del número de entradas de las mismas, para evitar que un alto número de transistores en serie empeore en gran medida esas características.

T7.1. Inversor CMOS

El inversor CMOS utiliza un transistor de canal N y otro de canal P:



Supuestos $V_{CC} = 5\text{ V}$ y $V_{TO,NMOS} = 1\text{ V}$ y $V_{TO,PMOS} = -1\text{ V}$
 Para $V_i < 1\text{ V}$ T_1 conduce y T_2 no: $V_o = V_{CC} = V(1)$
 Para $V_i > 4\text{ V}$ T_1 no conduce y T_2 sí: $V_o = 0\text{ V} = V(0)$

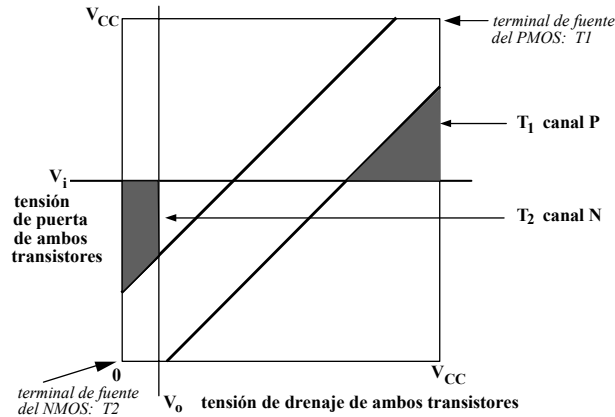
Las propias tensiones umbrales de los transistores aseguran las tensiones de salida booleanas $V(0)$ y $V(1)$ al mantener uno de los dos transistores en corte; en ambas situaciones booleanas, el consumo es nulo pues el transistor cortado impide el paso de corriente a través de ambos (que se encuentran en serie).

En la figura se ha representado también la polarización de los substratos: en el diseño físico han de configurarse las conexiones del substrato P (del transistor NMOS) a 0 V y del substrato N (pozo) a V_{CC} . Tales conexiones de polarización aseguran que todas las uniones PN que forman los transistores con el substrato se encuentran polarizadas inversamente, aislando a cada transistor respecto de su substrato.

El transistor NMOS tiene su terminal de fuente a 0 V y las tensiones de entrada V_{GS} y de salida V_{DS} son positivas y su referencia es 0 V. En cambio, el transistor PMOS tiene su terminal de fuente a V_{CC} (se encuentra colocado «boca abajo») y las tensiones de entrada V_{GS} y de salida V_{DS} son negativas, ambas con referencia a V_{CC} .

Para analizar la función de transferencia del inversor ($V_o - V_i$) puede utilizarse un diagrama de Memelink conjunto de ambos transistores. En dicho diagrama el origen de coordenadas del transistor NMOS (punto 0,0) no coincide con el del transistor PMOS (punto V_{CC}, V_{CC}) ya que la tensión de referencia, tensión de fuente, de ambos transistores es diferente. En cambio, la tensión de entrada V_i (tensión de puerta) de ambos es común y actúa sobre el eje Y , y también lo es su tensión de salida V_o (tensión de drenaje) que actúa sobre el eje X .

En una situación estática, la intensidad que circula por ambos transistores es la misma $I_{T1} = I_{T2}$ ($I_P = I_N$) ya que ambos están en serie; para lo cual tiene que cumplirse que $\alpha_P A_P = \alpha_N A_N$, siendo $\alpha = \mu_{COX} W/L$ el parámetro que incluye los aspectos tecnológicos ($\mu_{COX} = K_p$) y geométricos ($W/L = ff$) y A el efecto de las tensiones aplicadas sobre el transistor, que viene dado (en el diagrama de Memelink) por el área limitada por las cuatro rectas $x = V_S$, $x = V_D = V_o$, $y = V_G = V_i$ e $y = V_{TO} + V$.



$$I_{NMOS} = I_{PMOS} \quad \alpha_N \cdot A_N = \alpha_P \cdot A_P \quad \mu_{electrones} \cdot C_{ox} \cdot W_N / L_N \cdot A_N = \mu_{huecos} \cdot C_{ox} \cdot W_P / L_P \cdot A_P$$

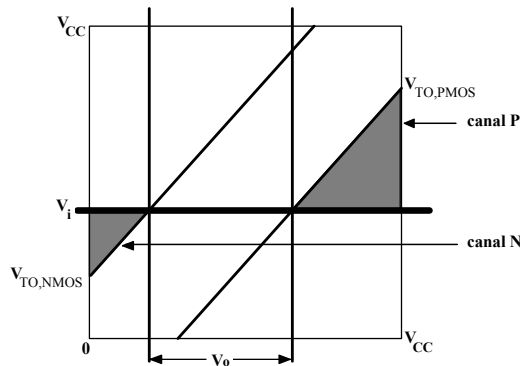
$$\frac{A_N}{A_P} = \frac{\alpha_P}{\alpha_N} = \frac{\mu_{huecos}}{\mu_{electrones}} \frac{W_P / L_P}{W_N / L_N}; \quad \frac{\mu_{huecos}}{\mu_{electrones}} = \mu'; \quad \frac{A_N}{A_P} = \mu' \frac{ff_P}{ff_N} = \mu' \cdot \beta$$

donde $\mu' = \mu_{huecos} / \mu_{electrones}$, relación entre las movilidades de huecos (transistor PMOS) y electrones (NMOS), es del orden de 1/3 y $\beta = (W_1/L_1) / (W_2/L_2) = ff_1 / ff_2$ es la relación de geometrías entre ambos transistores.

Para cualquier tensión de entrada V_i la tensión de salida V_o es la que corresponde a la siguiente relación entre áreas en el diagrama de Memelink: $A_N = \mu' \cdot \beta \cdot A_P$.

Tensión de conmutación

La tensión de conmutación corresponde a aquella tensión de entrada V_i en la que las áreas de conducción de ambos transistores en el diagrama de Memelink son triangulares: en tal situación, según se aprecia en la figura, la tensión de salida V_o no es un valor único, sino un amplio intervalo de tensiones; dicho intervalo corresponde, precisamente, a la conmutación de la salida.



La tensión de entrada V_i a la cual se produce la conmutación es aquella para la cual las áreas de los dos triángulos rectángulos e isosceles (formados por V_i con $V_{TO} + V$) cumplen la relación anterior, ya que para dicha tensión de entrada existe todo un intervalo de tensiones de salida posibles, intervalo que corresponde al salto brusco de la conmutación.

$$\left. \begin{aligned} A_P &= \frac{1}{2} (V_{CC} - |V_{TO,PMOS}| - V_i)^2 \\ A_N &= \frac{1}{2} (V_i - V_{TO,NMOS})^2 \end{aligned} \right\} A_N = \mu' \cdot \beta \cdot A_P$$

Sea $V_{uP} = |V_{TO,PMOS}|$ el valor absoluto de la tensión umbral del transistor PMOS (dicha tensión es negativa), $V_{uN} = V_{TO,NMOS}$ la tensión umbral del transistor NMOS, V_{conm} la tensión de conmutación del inversor y $k' = \sqrt{\mu' \cdot \beta}$.

$$\left. \begin{aligned} A_P &= \frac{1}{2} (V_{CC} - V_{uP} - V_{conm})^2 \\ A_N &= \frac{1}{2} (V_{conm} - V_{uN})^2 \end{aligned} \right\} A_N = \mu' \cdot \beta \cdot A_P$$

$$\rightarrow (V_{conm} - V_{uN}) = k' (V_{CC} - V_{uP} - V_{conm}) \rightarrow V_{conm} = \frac{V_{uN} + k' (V_{CC} - V_{uP})}{1 + k'}$$

La tensión de conmutación del inversor puede calcularse mediante la expresión

$$V_{conm} = \frac{V_{uN} + k' (V_{CC} - V_{uP})}{1 + k'} \quad \text{donde } k' = \sqrt{\mu' \cdot \beta}.$$

Si las tensiones umbrales son iguales en valor absoluto ($V_{uN} = V_{uP}$) y los transistores son iguales en dimensiones (relación de geometrías $\beta = 1$), la función de transferencia no es simétrica ($V_{conm} \neq V_{CC}/2$) sino que la conmutación está desplazada hacia valores inferiores de V_i , debido a la menor movilidad de los huecos que repercute en una menor conductividad del transistor PMOS.

En la conmutación, la salida puede adoptar cualquier tensión dentro del intervalo señalado en la figura que viene delimitado por los vértices de ambos triángulos. Teniendo en cuenta que la recta $V_{TO} + V$ tiene una pendiente de 45° , los dos catetos de dichos triángulos son iguales y su valor es respectivamente:

$$\text{NMOS: } V_{conm} - V_{uN};$$

$$V_o(\text{vértice del triángulo}) = V_{conm} - V_{uN}$$

$$\text{PMOS: } V_{CC} - V_{uP} - V_{conm};$$

$$V_o(\text{vértice del triángulo}) = V_{CC} - (V_{CC} - V_{uP} - V_{conm}) = V_{conm} + V_{uP}$$

Es decir, la conmutación se produce con una variación vertical de la tensión de salida entre $V_{conm} - V_{uN}$ y $V_{conm} + V_{uP}$.

La intensidad que circula por el inversor en la situación de conmutación será:

$$I_{\text{conm}} = \alpha \cdot A_N = K_p \cdot \frac{W}{L} \cdot A_N = K_p \cdot \frac{W}{L} \cdot \frac{1}{2} (V_{\text{conm}} - V_{uN})^2.$$

Para la tecnología Ψ :

$V_{CC} = 5 \text{ V}$; $V_{uN} = 1 \text{ V}$; $V_{uP} = 1 \text{ V}$;

$\mu_{\text{electrones}} = 600 \text{ cm}^2/\text{Vs}$; $\mu_{\text{huecos}} = 200 \text{ cm}^2/\text{Vs}$; $\mu' = 1/3$;

supuestos ambos transistores iguales en dimensiones,

por ejemplo, ambos de tamaño mínimo: $ff_P/ff_N = \beta = 1$

$$k' = \sqrt{\mu' \cdot \beta} = \sqrt{1/3} \approx 0,58$$

la tensión de conmutación será:

$$V_{\text{conm}} = \frac{V_{uN} + k' \cdot (V_{CC} - V_{uP})}{1 + k'} = \frac{1 + 0,58 \cdot (5 - 1)}{1 + 0,58} = 2,1 \text{ V}$$

La tensión de salida conmutará entre los siguientes valores:

NMOS: $V_O(\text{vértice del triángulo}) = V_{\text{conm}} - V_{uN} = 1,1 \text{ V}$

PMOS: $V_O(\text{vértice del triángulo}) = V_{\text{conm}} + V_{uP} = 3,1 \text{ V}$

y la intensidad en dicha situación es:

$K_p \text{ (NMOS)} \sim 40 \mu\text{A}/\text{V}^2$; $W/L = 1,5$

$$I_{\text{conm}} = K_p \cdot \frac{W}{L} \cdot \frac{1}{2} (V_{\text{conm}} - V_{uN})^2 = 40 \frac{\mu\text{A}}{\text{V}^2} \cdot 1,5 \cdot 0,5 \cdot (2,1 - 1)^2 \text{ V}^2 = 36 \mu\text{A}.$$

La función de transferencia no es simétrica ($V_{\text{conm}} = 2,1 \text{ V} \neq 2,5 \text{ V}$) sino que la conmutación está desplazada hacia valores inferiores de V_i , debido a la menor movilidad de los huecos que repercute en una menor conductividad del transistor PMOS.

Es posible conseguir una función de transferencia simétrica, compensando, a través de β , la relación entre las movilidades de los dos transistores: en este caso en que ambas tensiones umbrales son iguales y $\mu' = 1/3$, para $\beta = 3$ resulta $k' = \sqrt{\mu' \cdot \beta} = 1$, $A_P = A_N$ y $V_i = 2,5 \text{ V}$.

Para ello es necesario hacer el transistor PMOS de anchura triple, de manera que ocupará mayor superficie de integración y, además, aumentarán los tiempos de propagación de la puerta anterior: los transistores PMOS dejan de ser mínimos y aumentan sus dimensiones y su capacidad de puerta. No es, por ello, una buena solución y, en la práctica, no suele hacerse este «equilibrado» de los transistores PMOS.

Habida cuenta de que la precisión del proceso de fabricación es limitada, cabe preguntarse en qué medida una desviación en las dimensiones de los transistores afecta a la tensión de conmutación.

Supongamos que, por alguna razón, se modifican las dimensiones de los transistores y no resultan exactamente iguales; expresando la tensión de conmutación V_{conm} en función de la relación de geometrías β :

$$V_{\text{conm}} = \frac{V_{uN} + k' \cdot (V_{CC} - V_{uP})}{1 + k'} = \frac{V_{uN} + \sqrt{\mu' \cdot \beta} \cdot (V_{CC} - V_{uP})}{1 + \sqrt{\mu' \cdot \beta}}$$

Para la tecnología Ψ :

$$V_{\text{conm}} = \frac{V_{uN} + \sqrt{\mu' \cdot \beta} \cdot (V_{CC} - V_{uP})}{1 + \sqrt{\mu' \cdot \beta}} = \frac{1 + 0,58 \cdot \sqrt{\beta} \cdot (5 - 1)}{1 + 0,58 \cdot \sqrt{\beta}} = \frac{1 + 2,3 \cdot \sqrt{\beta}}{1 + 0,58 \cdot \sqrt{\beta}}$$

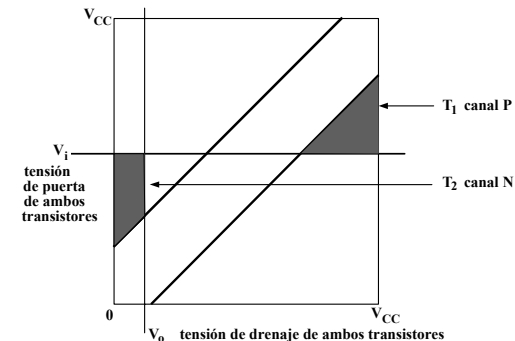
- si β en lugar de 1 pasa a valor 2 (modificación muy amplia, del 100 %, que equivale a un factor de forma del transistor PMOS doble) entonces $V_i = 2,34 \text{ V}$ (desviación en la tensión de conmutación de $0,24 \text{ V} < 12 \%$);

- en cambio, si β pasa a 0,5 (factor de forma del NMOS doble), $V_i = 1,86$ (desviación análoga a la anterior, $0,24 \text{ V} < 12 \%$).

Es decir, el efecto de una desviación en las dimensiones de los transistores sobre la tensión de conmutación del inversor es muy pequeño.

Función de transferencia

El *diagrama de Memelink* del inversor permite obtener la curva de transferencia ($V_O - V_i$) punto a punto: dado un valor de V_i , calcular el correspondiente de V_O ;



La relación entre la tensión de salida y la de entrada se obtiene a partir de la igualdad de intensidades en ambos transistores, $A_N = \mu' \cdot \beta \cdot A_P$, expresando el área de conducción de cada transistor en función de las tensiones de entrada V_i y de salida V_O .

Puede efectuarse un análisis por tramos, obteniendo las expresiones algebraicas que relacionan V_O con V_i en cada tramo.

Los tramos diferenciados son los siguientes:

- | | |
|----------------------------------|--|
| I $V_i < V_{TO,NMOS}$ | el transistor N no conduce y $V_o = V_{CC}$ |
| II $V_i < V_{conmutación}$ | el área de conducción del transistor N es triangular y, en cambio, el área correspondiente al P es trapezoidal |
| III $V_i = V_{conmutación}$ | el área de conducción del transistor N es triangular y el área correspondiente al P también es triangular |
| IV $V_i > V_{conmutación}$ | el área de conducción del transistor N es trapezoidal y, en cambio, el área correspondiente al P es triangular |
| V $V_i > V_{CC} - V_{TO,PMOS} $ | el transistor P no conduce y $V_o = 0$ |

Para la tecnología Ψ :

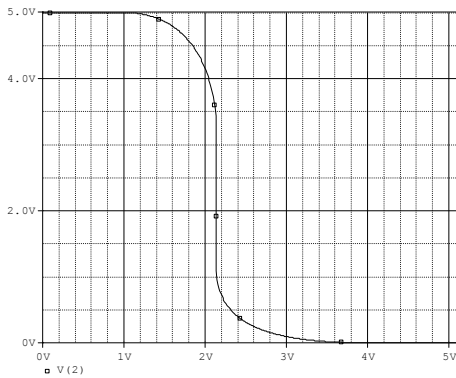
- | | |
|--|---|
| I $V_i < 1 \text{ V}$ | $V_o = 5 \text{ V}$ |
| II $1 \text{ V} < V_i < 2,1 \text{ V}$ | $V_o = V_i + 1 + \sqrt{13 - 2V_i - 2V_i^2}$ |
| III $V_i = 2,1 \text{ V}$ | $V_o = [1,1 ; 3,1]$ |
| IV $2,1 \text{ V} > V_i < 4 \text{ V}$ | $V_o = V_i - 1 - \sqrt{(2V_i^2 + 2V_i - 13) / 3}$ |
| V $V_i > 4 \text{ V}$ | $V_o = 0 \text{ V}$ |

En conmutación, $V_{conm} = 2,1 \text{ V}$, las expresiones de los tramos II y IV proporcionan los valores de 3,1 V y 1,1 V entre los cuales conmuta el inversor:

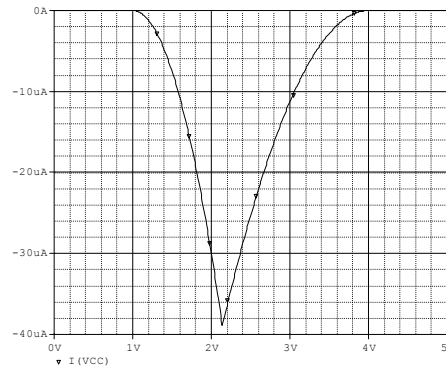
$$V_i = V_{conm} \quad V_o = [3,1 ; 1,1] \text{ V}$$

También puede obtenerse la función de transferencia mediante simulación SPICE: las siguientes gráficas muestran, respectivamente, la función de transferencia $V_o - V_i$ y el consumo de intensidad de un inversor CMOS de la tecnología Ψ :

$L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$, $|V_{TO}| = 1 \text{ V}$, $K_{p,NMOS} = 40 \mu\text{A/V}^2$ y $K_{p,PMOS} = 15 \mu\text{A/V}^2$.



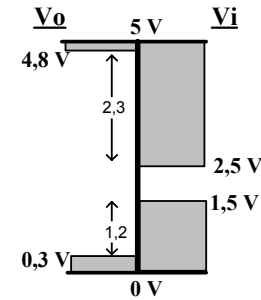
Función de transferencia $V_o - V_i$ CMOS



Consumo de intensidad I_{CC} del inversor

En la anterior función de transferencia se aprecia que, en relación con el «ruido» (para determinar mayores márgenes respecto al ruido), es razonable extender la tensión de entrada correspondiente al **0** hasta 1,5 V (para dicho intervalo, $V_o(\mathbf{1}) > 4,8 \text{ V}$) y la tensión de entrada para valor **1** puede tomarse desde 2,5 V (intervalo para el cual $V_o(\mathbf{0}) < 0,3 \text{ V}$):

para $V_i < 1,5 \text{ V}$, $V_o > 4,8 \text{ V}$ y para $V_i > 2,5 \text{ V}$, $V_o < 0,3 \text{ V}$.



Márgenes de ruido en tensión:

$$\Delta V(\mathbf{0}) = V_{i,\text{máx}}(\mathbf{0}) - V_{o,\text{máx}}(\mathbf{0}) = 1,5 - 0,3 = 1,2 \text{ V}$$

$$\Delta V(\mathbf{1}) = V_{o,\text{mín}}(\mathbf{1}) - V_{i,\text{mín}}(\mathbf{1}) = 4,8 - 2,5 = 2,3 \text{ V}$$

Es mayor el margen correspondiente al **1** a causa del desplazamiento de la función de transferencia hacia tensiones bajas, debido a la menor movilidad de los huecos.

Margen de ruido en potencia: $\Delta P = (\Delta V)^2 / R_o$. El margen de ruido en potencia (o sea, en energía por unidad de tiempo) es más significativo que el margen en tensión, ya que el ruido actúa como energía perturbativa sobre el circuito digital.

Resistencia de salida

La resistencia de salida es un parámetro indicativo del comportamiento de las puertas lógicas (una referencia de calidad de las mismas), en cuanto a intensidad disponible en la salida, inmunidad frente al ruido y tiempos de propagación, pues afecta fuertemente a estos tres aspectos: en principio, cuanto menor sea la resistencia de salida mayores serán la intensidad suministrable por la misma, el margen frente al ruido y la velocidad de trabajo.

También interesa resistencia de salida baja en relación con el acoplo en tensión, pero dicho acoplo ya viene garantizado por el altísimo valor de la resistencia de entrada (que es cuasi-infinita).

Las situaciones booleanas corresponden a un transistor en zona lineal: para salida **0** el transistor NMOS se encontrará conduciendo en su zona lineal, mientras que para salida **1** será el transistor PMOS el que se encuentre en zona óhmica; de forma que $R_o(\mathbf{0})$ y $R_o(\mathbf{1})$ corresponden, respectivamente, a las resistencias que presentan los transistores NMOS y PMOS en zona lineal.

Valores de las resistencias de salida (tecnología Ψ):

Para transistores de tamaño mínimo (W=1,5L; ff=1,5) con tensión umbral de 1 V y tensión de puerta de 5 V los valores de la resistencia equivalente en zona lineal serán:

NMOS	PMOS
$K_p \approx 40 \mu A/V^2$	$K_p \approx 15 \mu A/V^2$
$R_{eq} = \frac{1}{K_p \cdot W/L \cdot (V_G - V_{TO})}$	
$R_{eq} = 1 / (40 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 4 \text{ k}\Omega$	$R_{eq} = 1 / (15 \cdot 10^{-6} \cdot 1,5 \cdot (5-1)) \approx 11 \text{ k}\Omega$
De manera que las resistencias de salida en situación booleana serán:	
$R_o(0) \sim 4 \text{ K}\Omega$	$R_o(1) \sim 11 \text{ K}\Omega$

Los valores de las resistencias de salida de los inversores de tamaño mínimo son del orden de varios KΩ, ~10³ ohmios; no son valores pequeños, pero son adecuados para el comportamiento de las puertas booleanas en el interior del circuito integrado. Téngase en cuenta que las capacidades de entrada de dichas puertas son pequeñas (del orden del fentofaradio, 10⁻¹⁵ F) y, por ello, también lo son las intensidades que se requieren y las constantes de tiempo que se generan (10³.10⁻¹⁵ ~ ps); asimismo, habida cuenta de que las dimensiones de las conexiones son muy reducidas, el efecto del ruido es muy pequeño.

En los terminales de salida de un circuito integrado interesan resistencias de salida inferiores a 100 ohmios (pues en este caso, se requieren intensidades del orden de 10 mA, las capacidades de carga son del orden de 10 pF y las longitudes de las conexiones van en centímetros); en cambio, en las puertas interiores son adecuadas resistencias del orden de 10 K (por las razones antedichas: intensidades del orden de 0,1 mA, capacidades de 1 fF y longitud de las conexiones en micras).

En todo caso, puede disminuirse el valor de la resistencia de salida aumentando en igual medida la anchura **W** del correspondiente transistor, el PMOS para $R_o(1)$ y el NMOS para $R_o(0)$; esto es lo que se hace, precisamente, en los adaptadores de las salidas del circuito integrado.

La intensidad suministrable por el inversor, en cada uno de sus dos estados, depende de la caída de tensión que admitamos en la salida: $I_o = \Delta V_o / R_o$. Esta intensidad es relativamente pequeña y, en ocasiones, no es suficiente la correspondiente a transistores de tamaño mínimo. Por ejemplo, en los citados adaptadores de las salidas, que precisan de una capacidad de intensidad (hacia el exterior del circuito integrado) del orden de 10 mA. En tales casos, basta aumentar adecuadamente el factor de forma de los transistores ($ff = W / L$), es decir, es necesario hacer $W \gg L$.

Para la tecnología Ψ:

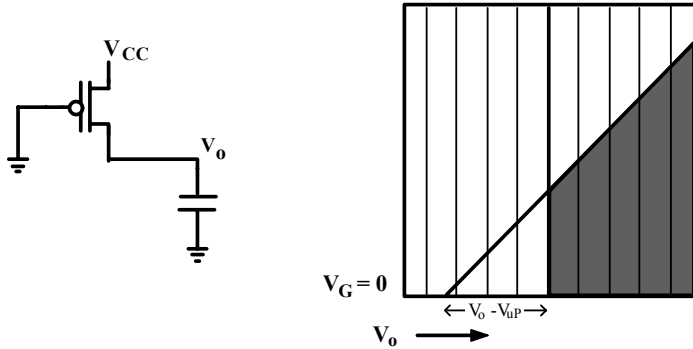
NMOS	PMOS
$R_o(0) \sim 4 \text{ K}\Omega$	$R_o(1) \sim 11 \text{ K}\Omega$
Intensidad suministrable por el inversor: $I_o = \Delta V_o / R_o$	
Admitiendo $\Delta V(0) = 0,2 \text{ V}$ y $\Delta V(1) = 0,6 \text{ V}$ que son valores aceptables (cuya diferencia va en relación a la asimetría de la función de transferencia y a la diferencia de márgenes de ruido):	
$I_o(0) \sim 0,2 / 4K \approx 50 \mu A$	$I_o(1) \sim 0,6 / 11K \approx 50 \mu A$.
Haciendo $W > L$ se consiguen mayores intensidades de salida:	
para $W = 15L$	$I_o \approx 0,5 \text{ mA}$
para $W = 50L$	$I_o \approx 1,5 \text{ mA}$
para $W = 300L$	$I_o \approx 10 \text{ mA}$.
Margen de ruido en potencia del inversor con transistores de tamaño mínimo:	
$\Delta P(0) \sim 1,2^2 / 4 \text{ K} \approx 0,36 \text{ mW}$	$\Delta P(1) \sim 2,3^2 / 11 \text{ K} \approx 0,48 \text{ mW}$.

El ruido actúa efectivamente como potencia: energía perturbativa por unidad de tiempo. En los terminales de un circuito integrado se requieren márgenes de ruido superiores de 20 mW para ambientes «normales» (viviendas o lugares públicos) y a 100 mW en ambientes «industriales»; en el interior de los circuitos integrados (en donde la captación y efecto del ruido es muy inferior, ya que las conexiones son «milimétricas»), son suficientes márgenes del orden de 0,1 mW.

T7.2. Tiempo de propagación y mejora del mismo

T7.2.1. Tiempo de conmutación de una puerta CMOS

La salida de una puerta booleana se encontrará conectada a la entrada de otra u otras puertas (u otros componentes del circuito global), cuyo efecto equivalente es el de una capacidad que ha de cargarse y descargarse en la conmutación, originando unos tiempos de retraso hasta que el proceso de carga o descarga alcanza los valores de tensión apropiados para el **0** y **1** booleanos.



Proceso de carga a través de un transistor PMOS: conmutación de la salida de 0 a 1

Estudiemos el proceso de carga, expresando los tiempos de retraso en función de la intensidad con canal saturado $I_{SAT} = I$ y de la capacidad de carga $C_L = C$ que soporta:

$I = I_{SAT} = \alpha \cdot A_{\text{canal saturado}}$, siendo A el área del triángulo de conducción en el diagrama de Memelink;

En el proceso de carga, existen dos tiempos diferenciados:

t_1 : V_O de 0 a $V_{uP} = |V_{TO, PMOS}|$ canal saturado intensidad constante $I_1 = I$,

$$\Delta Q = I \cdot t = C \cdot \Delta V \quad \Delta V = V_{uP} \quad t_1 = C \cdot V_{uP} / I$$

t_2 : V_O de V_{uP} a V_{CC} canal no saturado $I_2 < I$ y decrece al aumentar V_O ,

$$dQ = I(t) dt = C dV$$

el área de conducción corresponde ahora a la diferencia entre dos triángulos: el propio de la saturación de canal del transistor y el que determina la tensión de salida:

$$A = A_{\text{canal saturado}} - 1/2 \cdot (V - V_{uP})^2 \quad I(t) = I - \alpha \cdot (V - V_{uP})^2 / 2$$

expresando α en función de I : $I = I_{SAT} = \alpha \cdot A_{\text{canal saturado}} = \alpha \cdot 1/2 \cdot (V_{CC} - V_{uP})^2$;

$$\alpha = 2 \cdot I / (V_{CC} - V_{uP})^2 \quad I(t) = I - I \cdot (V - V_{uP})^2 / (V_{CC} - V_{uP})^2$$

$$I(t) dt = C dV \quad (I - I \cdot (V - V_{uP})^2 / (V_{CC} - V_{uP})^2) dt = C dV$$

$$\frac{I(V_{CC} - V_{uP})^2 - I(V - V_{uP})^2}{(V_{CC} - V_{uP})^2} dt = C dV$$

$$\frac{I}{C \cdot (V_{CC} - V_{uP})^2} dt = \frac{dV}{(V_{CC} - V_{uP})^2 - (V - V_{uP})^2}$$

Integrando ambos miembros de la ecuación diferencial anterior:

$$\frac{I}{C \cdot (V_{CC} - V_{uP})^2} t = \frac{1}{(V_{CC} - V_{uP})} \text{arc tgh} \frac{(V - V_{uP})}{(V_{CC} - V_{uP})}$$

$$\frac{I}{C \cdot (V_{CC} - V_{uP})} t = \text{arc tgh} \frac{(V - V_{uP})}{(V_{CC} - V_{uP})}$$

$$V(t) = V_{uP} + (V_{CC} - V_{uP}) \cdot \text{tgh} \frac{I}{(V_{CC} - V_{uP}) \cdot C} t$$

La función tangente hiperbólica **tgh(x)** es fuertemente creciente (del tipo $1 - e^{-x}$, con pendiente aún mayor): tiende a 1 asintóticamente y alcanza el valor 0.76 para $x = 1$. Para dicho valor ($x = 1$), la salida habrá recorrido, aproximadamente, el 80% de la tensión de alimentación:

$$V_0 = V_{uP} + (V_{CC} - V_{uP}) \cdot 0,76 = 0,76 \cdot V_{CC} + 0,24 \cdot V_{uP} \approx 0,8 V_{CC}$$

(habida cuenta de que V_{uP} se encontrará, generalmente, entre $0,2 \cdot V_{CC}$ y $0,4 \cdot V_{CC}$).

La conmutación de la salida hacia V_{CC} alcanzará $0,8V_{CC}$ en un tiempo aproximado:

$$\frac{I}{(V_{CC} - V_{uP}) \cdot C} t_2 = 1 \quad t_2 = \frac{(V_{CC} - V_{uP}) \cdot C}{I}$$

$$t = t_1 + t_2 = \frac{V_{uP} \cdot C}{I} + \frac{(V_{CC} - V_{uP}) \cdot C}{I} = \frac{V_{CC} \cdot C}{I} \quad t_{pLH} = t_1 + t_2 = V_{CC} \cdot C / I$$

donde I es la intensidad de canal saturado del transistor PMOS.

La determinación del tiempo de descarga a través del transistor NMOS hasta disminuir a $0,2V_{CC}$ es análoga y conduce a la misma expresión $t_{pHL} = V_{CC} \cdot C / I$:

$$t_{pLH} = V_{CC} \cdot C / I(\text{PMOS})$$

$$t_{pHL} = V_{CC} \cdot C / I(\text{NMOS}).$$

La intensidad de canal saturado del transistor PMOS $I(\text{PMOS})$ suele ser menor que la del NMOS $I(\text{NMOS})$ como consecuencia de la menor movilidad de los huecos que conforman el canal del transistor PMOS y, consiguientemente, $t_{pLH} > t_{pHL}$: el tiempo de subida es superior al de bajada. Esta desigualdad entre ambos tiempos de propagación podría evitarse dimensionando adecuadamente el transistor PMOS, pero implicaría mayor área de integración y, a la vez, mayor capacidad de carga de cada entrada lo cual redundaría en empeorar los tiempos de conmutación de la puerta anterior. Por ello, no interesa dimensionar las puertas lógicas para conseguir su simetría funcional.

Las expresiones de los tiempos de propagación parecen indicar que dichos tiempos disminuyen al hacerlo la tensión de alimentación V_{CC} , pero sucede al revés ya que la intensidad de canal saturado también depende de la tensión de alimentación y disminuye fuertemente con ella: $I_{SAT} = \alpha \cdot A_{\text{canal saturado}} = K_p \cdot W/L \cdot 1/2 \cdot (V_{CC} - V_{uP})^2$.

Para la tecnología Ψ :

Para transistores de tamaño mínimo ($L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$) el valor de la capacidad de puerta será: $C_G = C_{ox} \cdot W \cdot L = 0,7 \text{ fF} / \mu\text{m}^2 \cdot 1,5 \mu\text{m} \cdot 1 \mu\text{m} \approx 1 \text{ fF}$ y como cada entrada se encuentra conectada a dos transistores, NMOS y PMOS, la capacidad de entrada será: $C_i \approx 2 \text{ fF}$.

Para $V_{CC} = 5 \text{ V}$ y $V_{TO} = 1 \text{ V}$:
 $I(P) = I_{sat}(\text{canal P}) \approx 0,2 \text{ mA}$
 $I(N) = I_{sat}(\text{canal N}) \approx 0,5 \text{ mA}$

y con un *fan-out* = 1 (una sola entrada conectada a la salida de un inversor mínimo):

$$t_{pLH} \approx 5 \text{ V} \cdot 2 \text{ fF} / 0,2 \text{ mA} \sim 0,05 \text{ ns} \quad t_{pHL} \approx 5 \text{ V} \cdot 2 \text{ fF} / 0,5 \text{ mA} \sim 0,02 \text{ ns}$$

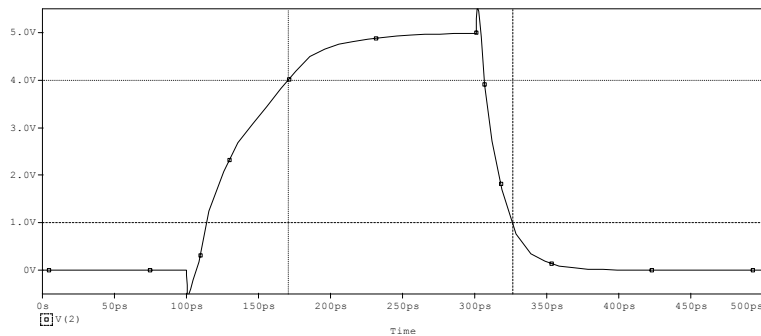
Nos encontramos con tiempos de conmutación por debajo del nanosegundo, que dependen linealmente de la capacidad de carga y, por ello, del *fan-out* y de las dimensiones de los transistores:

- al aumentar el *fan-out*, los tiempos de conmutación crecen en la misma proporción
- al disminuir las dimensiones de la tecnología los tiempos de conmutación se reducen cuadráticamente (C_i disminuye con L y con W).

Para una tecnología de 0,5 micras: $L_{\min} = 0,5 \mu\text{m}$, $W = 0,75 \mu\text{m}$, $C_i \sim 0,5 \text{ fF}$ con la misma alimentación $V_{CC} = 5 \text{ V}$, tensión umbral $V_{TO} = 1 \text{ V}$ y *fan-out* = 1:

$$t_{pLH} \approx 5 \text{ V} \cdot 0,5 \text{ fF} / 0,2 \text{ mA} \sim 0,015 \text{ ns} \quad t_{pHL} \approx 5 \text{ V} \cdot 0,5 \text{ fF} / 0,5 \text{ mA} \sim 0,005 \text{ ns}$$

La siguiente gráfica muestra la conmutación de un inversor con un *fan-out* de 1, es decir, a su salida se encuentra conectado otro inversor análogo; la simulación SPICE se ha efectuado con los datos de la tecnología Ψ y transistores de tamaño mínimo: $L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$, $|V_{TO}| = 1 \text{ V}$, $K_{p,NMOS} = 40 \mu\text{A/V}^2$ y $K_{p,PMOS} = 15 \mu\text{A/V}^2$.



Onda de conmutación de un inversor: $V(2) = V_o$

Los tiempos medidos en esta gráfica son: $t_{pLH} \approx 0,071 \text{ ns}$ y $t_{pHL} \approx 0,026 \text{ ns}$.

El retardo que introduce un inversor es algo superior al tiempo de conmutación calculado en este apartado: dicho tiempo de conmutación ha sido determinado en relación con la capacidad de carga C_L (*fan-out*) y, en la práctica, influyen también otros efectos capacitivos internos del propio inversor.

Sin ser exactos, las formulas y cálculos anteriores ofrecen una buena estimación del orden de magnitud y una buena aproximación al valor real de los tiempos de propagación. La simulación SPICE de los mismos puede proporcionar aún mayor precisión.

Además, habrá que tener en cuenta que cada conexión entre la salida de una puerta y la entrada de la siguiente introduce una línea conductora con sendos efectos resistivo y capacitivo y, con ello, un nuevo sumando al retardo de propagación de las señales; en tecnologías submicrónicas no es extraño que el retardo introducido por las líneas de conexión sea superior al tiempo de propagación de la propia puerta booleana.

Por otra parte, los tiempos de conmutación dependen fuertemente de la temperatura, ya que al aumentar ésta disminuye fuertemente la movilidad de los portadores (aumentan sus choques con los núcleos de la red cristalina) y, en consecuencia, se reduce K_p y la intensidad de saturación de los transistores. De ahí, el alto interés de evitar el calentamiento de los circuitos integrados, disipando adecuadamente el calor generado por la potencia consumida en ellos, si se desea aprovechar la velocidad máxima de trabajo.

«Equilibrado» de transistores (tecnología Ψ):

Si «equilibramos» ambos transistores para que su transconductancia K_p sea la misma:

$$W_{PMOS} = 3 \cdot W_{\min}$$

las intensidades de canal saturado se igualan: $I(P) = I(N) \approx 0,5 \text{ mA}$

la capacidad de entrada se duplica:

$$C_i = C_{ox} \cdot (W_{NMOS} \cdot L_{NMOS} + W_{PMOS} \cdot L_{PMOS}) = C_{ox} \cdot (4 \cdot W_{\min} \cdot L_{\min}) \approx 4 \text{ fF}.$$

y también se igualan los tiempos de propagación:

$$t_{pLH} \approx 5 \text{ V} \cdot 4 \text{ fF} / 0,5 \text{ mA} \sim 0,04 \text{ ns} \quad t_{pHL} \approx 5 \text{ V} \cdot 4 \text{ fF} / 0,5 \text{ mA} \sim 0,04 \text{ ns}.$$

Pero, calculando el retardo que introducen dos inversores seguidos, resultaría:

$$t_2 \text{ inversores} = t_{pLH} + t_{pHL} \sim 0,08 \text{ ns}$$

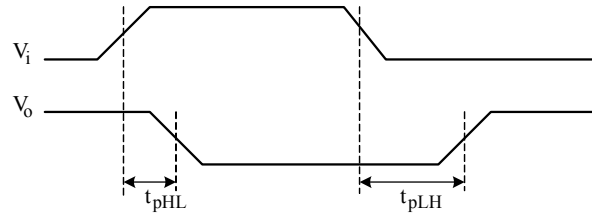
un tiempo de propagación superior al que resulta con dos inversores de tamaño mínimo:

$$t_2 \text{ inversores mínimos} = t_{pLH} + t_{pHL} \sim 0,05 + 0,02 \text{ ns} = 0,07 \text{ ns}.$$

O sea que, «equilibrando» el transistor PMOS de forma que conduzca igual que el NMOS y sus tiempos de propagación sean iguales, ocupamos mayor área de silicio y la velocidad de trabajo (considerada en conjuntos de puertas sucesivas) resulta inferior: en principio, no es una buena solución y, en la práctica, no suele hacerse.

Al conectar varios inversores seguidos, el tiempo de propagación del conjunto es mayor que los tiempos de conmutación individuales pero es inferior a la suma de ellos, ya que existe solapamiento entre ellos. Es decir, los tiempos de conmutación no son linealmente acumulativos ya que cada puerta inicia su conmutación antes de que la anterior complete la suya.

Para facilitar la suma de tiempos, cada tiempo de propagación suele medirse por el retraso entre el punto medio de la conmutación (tensión $V_{cc}/2$) de la onda de entrada y el punto medio de la conmutación de la señal de salida.



T7.2.2. Mejora de los tiempos de propagación

En ocasiones las capacidades de carga son altas; por ejemplo, en los adaptadores de las salidas de los circuitos integrados (que han de ser capaces de soportar su conexión a cargas equivalentes de decenas de picofaradios) o en puertas que deban soportar a su salida un *fan-out* alto (por ejemplo, las que transmiten a los diversos biestables la señal de reloj) o largas longitudes de polisilicio. En tales casos, los tiempos de propagación se elevan en demasía y es preciso reducirlos mediante adaptadores de intensidad.

Para la tecnología Ψ :

Para una capacidad de carga de 50 pF (25.000 veces superior a C_i -*fan-out* 1-):

$$t_{pLH} = V_{CC} \cdot C / I(P) \approx 5 \text{ V} \cdot 50 \text{ pF} / 0,2 \text{ mA} \approx 1,5 \mu\text{s}$$

$$t_{pHL} = V_{CC} \cdot C / I(N) \approx 5 \text{ V} \cdot 50 \text{ pF} / 0,5 \text{ mA} \approx 0,5 \mu\text{s}$$

tiempos que resultan relativamente altos y limitan la velocidad por debajo del MHz.

Para disminuir los tiempos de propagación causados por altas capacidades de carga o, lo que es lo mismo, por la necesidad de intensidades de salida altas, se utilizan esquemas de amplificación en cascada con inversores cuya anchura de transistor es progresivamente creciente.

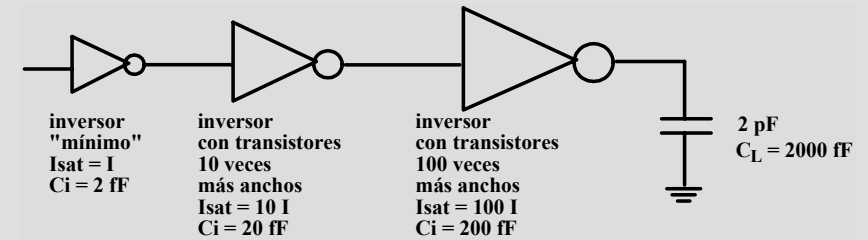
Ejemplo desarrollado con la tecnología Ψ :

Supongamos una carga de 2 pF que resulta ser unas 1.000 mayor que C_i (capacidad de carga que corresponde a *fan-out* 1):

$$t_{pLH} = V_{CC} \cdot C / I(P) \approx 5 \text{ V} \cdot 2 \text{ pF} / 0,2 \text{ mA} \approx 50 \text{ ns}$$

$$t_{pHL} = V_{CC} \cdot C / I(N) \approx 5 \text{ V} \cdot 2 \text{ pF} / 0,5 \text{ mA} \approx 20 \text{ ns}$$

Añadiendo un par de inversores cuya anchura de transistores sea, en cada uno de ellos, 10 veces la del anterior:



Como puede apreciarse en la figura, en cada inversor se produce un «salto» entre su capacidad de carga y su capacidad de entrada de 10; en total, un salto de 1000 que es la relación existente entre los 2 pF y C_i (capacidad de entrada del inversor mínimo).

$$t_{1pLH} \approx 5 \cdot 20 / 0,2 \approx 0,5 \text{ ns} \quad t_{2pLH} \approx 5 \cdot 200 / 2 \approx 0,5 \text{ ns} \quad t_{3pLH} \approx 5 \cdot 2000 / 20 \approx 0,5 \text{ ns}$$

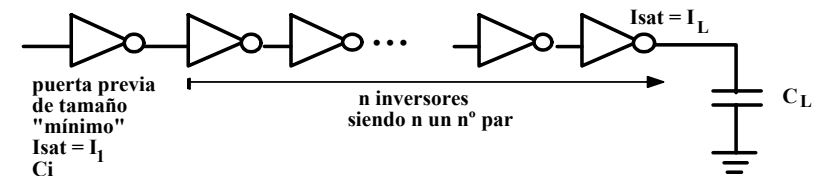
$$t_{1pHL} \approx 5 \cdot 20 / 0,5 \approx 0,2 \text{ ns} \quad t_{2pHL} \approx 5 \cdot 200 / 5 \approx 0,2 \text{ ns} \quad t_{3pHL} \approx 5 \cdot 2000 / 50 \approx 0,2 \text{ ns}$$

$$t_{pLH} \sim t_{1pLH} + t_{2pLH} + t_{3pLH} \approx 1,2 \text{ ns} \quad t_{pHL} \sim t_{1pHL} + t_{2pHL} + t_{3pHL} \approx 0,9 \text{ ns}$$

Ambos tiempos son muy inferiores a los que presenta el primer inversor en solitario: 50 ns y 20 ns, respectivamente.

Este esquema (en que se incluyen parejas de inversores «en escalera», es decir, con anchura de transistores creciente) resulta sumamente útil para configurar adaptadores de intensidad (*buffers*), tanto para los terminales de salida del circuito integrado como para aquellas conexiones internas que han de soportar un alto *fan-out* o alta carga capacitiva y, por tanto, han de proporcionar alta intensidad.

Un adaptador de intensidad puede incluir más de una pareja de inversores «en escalera» (siempre un número par para que no se produzca una inversión suplementaria), de forma que la relación de intensidad entre el último de ellos y la puerta «mínima» inicial se distribuya en múltiples «saltos» pequeños.



Se puede calcular el número óptimo de inversores para conseguir el menor tiempo de propagación global:

- sea **K** el cociente entre la capacidad de carga que debe soportar el conjunto y la correspondiente a una entrada mínima (primera puerta del conjunto): $K = C_L / C_i$
- si el necesario aumento del *fan-out* se distribuye de forma homogénea siendo ∂ el coeficiente incremental $C_{i+1} = \partial \cdot C_i$, será necesario que las anchuras **W** de transistores de dos inversores sucesivos se incrementen en dicho factor ∂ : $W_{i+1} = \partial \cdot W_i$
- la relación de anchuras repercute en proporción directa en la relación entre capacidades de entrada **C_i** y entre intensidades de saturación **I_{sat}**; se verificará que $W_{i+1} / W_i = C_{i+1} / C_i = I_{sat,i+1} / I_{sat,i} = \partial$
- el factor incremental global **K** corresponde a **n+1** incrementos ∂ sucesivos:
 $K = C_L / C_{in} = \partial^{n+1}$, $\partial = \sqrt[n+1]{K}$ y el incremento en intensidad será: $I_L / I_1 = \partial^n$

Los tiempos de propagación de los inversores serán iguales, ya que la relación entre intensidad de saturación y capacidad de carga es constante, y dichos tiempos serán proporcionales a ∂ ya que dicho factor representa la carga relativa a su tamaño que cada inversor soporta. Por tanto, el tiempo de propagación del conjunto será proporcional a $(n+1) \cdot \partial$ y la situación de tiempo mínimo de propagación corresponde a:

$$\frac{d[(n+1) \cdot \partial]}{dn} = 0; \quad \partial = \sqrt[n+1]{K}$$

$$\frac{d((n+1) \cdot \sqrt[n+1]{K})}{dn} = 0; \quad \sqrt[n+1]{K} + (n+1) \cdot \sqrt[n+1]{K} \cdot \ln K \cdot \left[-\frac{1}{(n+1)^2} \right] = 0$$

$$1 - \ln K \cdot \frac{1}{(n+1)} = 0; \quad n+1 = \ln K; \quad K = e^{n+1};$$

$$\partial = \sqrt[n+1]{K} = e = 2,7$$

La adaptación óptima de la «escalera de inversores», para conseguir tiempos mínimos de propagación, se consigue cuando el número de inversores es $n+1 = \ln K$ y, en este caso, el coeficiente incremental ∂ coincide con el número **e**:

$$W_{i+1} = 2,7 \cdot W_i$$

$$C_{i+1} = 2,7 \cdot C_i$$

$$I_{sat,i+1} = 2,7 \cdot I_{sat,i}$$

El tiempo de propagación de cada inversor será $\partial = e = 2,7$ veces el correspondiente a un inversor mínimo conectado a otro inversor análogo.

En el ejemplo desarrollado con la tecnología Ψ :

$$C_L = 2 \text{ pF}; \quad C_{in} = 2 \text{ fF}; \quad K = 1000; \quad n+1 = \ln(1000) \approx 7$$

El mínimo tiempo de propagación se obtiene con 6 inversores intermedios, cuya anchura de transistores sea, en cada uno de ellos, 2,7 veces mayor que en el anterior.

Los tiempos de propagación de cada uno de los inversores son los siguientes:

$$t_{pLH} = 2,7 \cdot 5 \text{ V} \cdot 2 \text{ fF} / 0,2 \text{ mA} \approx 0,15 \text{ ns} \quad t_{pHL} = 2,7 \cdot 5 \text{ V} \cdot 2 \text{ fF} / 0,5 \text{ mA} \approx 0,05 \text{ ns}$$

De forma que, los tiempos de propagación para el conjunto total serán:

$$t_{pLH} \sim 4t_{pLH} + 3t_{pHL} \approx 0,75 \text{ ns} \quad t_{pHL} \sim 4t_{pHL} + 3t_{pLH} \approx 0,65 \text{ ns}.$$

Tales son los tiempos mínimos de propagación que pueden conseguirse, claramente inferiores a los que presentaría el primer inversor actuando en solitario sobre la capacidad de carga 2 pF (50 ns y 20 ns, respectivamente) y solamente un poco inferiores a los que se obtienen con una sola pareja de inversores en escalera (1,2 ns y 0,9 ns).

Como puede apreciarse añadir una primera pareja de inversores (adecuadamente dimensionados) tiene un gran efecto respecto a la reducción de tiempos; cuando se añaden más parejas (ajustando en cada caso su anchura **W**), la inclusión de cada una de ellas tiene un efecto menor que la anterior. Por ello, debe valorarse la reducción de tiempos que se desea conseguir en relación con el área de silicio que se va a necesitar.

T7.3. Disipación de potencia

La potencia disipada por el inversor en situación estática es prácticamente nula: no existe ninguna línea de conducción directa entre alimentaciones (uno de los dos transistores de cada entrada se encuentra en corte). Ahora bien, en cada transistor existen uniones PN (difusión-substrato) polarizadas inversamente que conducen una minúscula intensidad inversa, de orden de magnitud del picoamperio (mA / 10⁶).

Ahora bien, al conmutar el inversor han de cargarse o descargarse las capacidades propias de sus transistores, en particular, su capacidad de puerta, lo cual determina un consumo dinámico proporcional a la frecuencia de conmutación:

- energía necesaria para cargar o descargar un condensador: $1/2 C \cdot V^2$
- potencia disipada al cargarlo y descargarlo con una onda de frecuencia **f** (en cada período hay dos conmutaciones, habrá que contabilizar una carga y una descarga): $(1/2 C \cdot V^2) \cdot 2f = C \cdot V^2 \cdot f$

Esta potencia (energía por unidad de tiempo) representa un consumo de intensidad desde la alimentación y una disipación de calor en el propio circuito. Son dos aspectos complementarios, relativos al consumo de intensidad, que requieren la correspondiente atención en el diseño y utilización del circuito integrado: una fuente de alimentación con suficiente capacidad de suministro de corriente y una disipación de calor adecuada. Teniendo en cuenta, además, que los tiempos de propagación aumentan con la temperatura y, en consecuencia, disminuye la velocidad de trabajo.

Conforme a la anterior expresión, la potencia consumida y disipada depende de:

- la tecnología, cuyas dimensiones determinan la capacidad **C**; dicha capacidad equivalente de una puerta a efectos de consumo de intensidad tiene varios componentes, entre los cuales predomina la capacidad de entrada (las otras capacidades internas de los transistores son de valores muy inferiores) que es proporcional a la superficie de las zonas de puerta, es decir, al producto **L.W** y disminuye cuadráticamente al disminuir **L** ($W = 1,5L$);
- la tensión de alimentación, que también afecta cuadráticamente **V²**; existe, por ello, una evolución continuada de la microelectrónica hacia tensiones de alimentación más bajas (el paso de 5 V a 3 V reduce el consumo a la tercera parte), siendo cada vez más frecuentes dispositivos de 2,5 V y de 1,8 V;
- la frecuencia de trabajo **f**, aumentando linealmente con ella (lo cual aconseja no trabajar a frecuencia más alta de la estrictamente necesaria para el sistema digital)

Al evaluar el consumo de un circuito integrado, debe tenerse en cuenta que no todas sus puertas conmutan cada vez, sino que solamente lo hace una pequeña parte de ellas; por ejemplo, la escritura de un dato sobre una memoria de 1 Mega solamente afecta a uno de sus 1048576 registros (y al decodificador de direcciones y control de escritura).

Disipación de potencia (tecnología Ψ):

Para transistores de tamaño mínimo el valor de la capacidad de puerta será:
 $C_{ox} = 0,7 \text{ fF} / \mu\text{m}^2$; $C_G = C_{ox} \cdot W \cdot L = 0,7 \text{ fF} / \mu\text{m}^2 \cdot 1,5 \mu\text{m} \cdot 1 \mu\text{m} \approx 1 \text{ fF}$.
 y el de la capacidad de entrada: $C_i = 2 \cdot C_G = \approx 2 \text{ fF}$

Para la tensión de alimentación habitual de 5 V ($V_{CC} = 5 \text{ V}$)

$$P = C \cdot V^2 \cdot f = 2 \text{ fF} \cdot 25 \text{ V}^2 \cdot f = 50 \cdot 10^{-15} \cdot f = 0,05 \cdot f \text{ pW}$$

$$\text{Para } f = 1 \text{ MHz} = 10^6: \quad P = 50 \text{ nW} / \text{MHz}; \quad I_{CC} = 10 \text{ nA} / \text{MHz}$$

La potencia disipada en cada entrada aumenta linealmente con la frecuencia.

Es posible hacer una estimación de la potencia disipada en 1 mm^2 ocupado por puertas CMOS, supuesto que todos los transistores conmuten (situación no habitual pues en cualquier bloque digital sólo conmuta una pequeña fracción de sus puertas).

El área activa ocupada por los transistores en cuanto a su zona de puerta (**L.W**) suele ser inferior al 5 % (ya que la mayor parte de la superficie es necesaria para las zonas de fuente y drenaje, conexiones, separaciones,...):

$$C_{\text{transistores}} = C_{ox} \cdot \text{superficie activa} = 0,7 \text{ fF}/\mu\text{m}^2 \cdot 1 \text{ mm}^2 \cdot 5\% = 35 \text{ pF}$$

$$P = C \cdot V^2 \cdot f = 875 \text{ pW} \cdot f \sim 1 \text{ nW} \cdot f$$

Para $f = 1 \text{ MHz}$, $P \sim 1 \text{ mW} / \text{MHz}$: supuesto que todos los transistores contenidos en 1 mm^2 de silicio conmutasen, dispararían en forma de calor una potencia del orden de 1 mW/MHz ; potencia que no depende de las dimensiones de la tecnología sino del porcentaje de ocupación: superficie activa/superficie total.

Además de la disipación de potencia de tipo capacitivo (principalmente la capacidad de entrada de los transistores MOS) existe otro efecto dinámico debido a que en la conmutación, durante un breve instante de tiempo, conducen ambos transistores PMOS y NMOS, dando lugar a un estrecho «pico de intensidad»; tal efecto resulta despreciable frente al anterior siempre que la conmutación sea adecuadamente rápida (tiempos de conmutación inferiores a 1 ns).

Sin embargo, dichos «picos de intensidad» deben ser tenidos en cuenta en relación con el «ruido», ya que al ser variaciones muy rápidas de intensidad generarán «picos de tensión» sobre las líneas de alimentación (por efecto inductivo).

El orden de magnitud del «pico de intensidad» en la conmutación fue calculado en el apartado anterior, por medio del área de conducción en el *diagrama de Memelink*; en situación de conmutación ($V_i = V_{\text{conm}}$), la intensidad que circula por el inversor, será:

$$I_{\text{conm}} = K_p \cdot \frac{W}{L} \cdot A_N = K_p \cdot \frac{W}{L} \cdot \frac{1}{2} (V_{\text{conm}} - V_{uN})^2.$$

Para la tecnología Ψ:

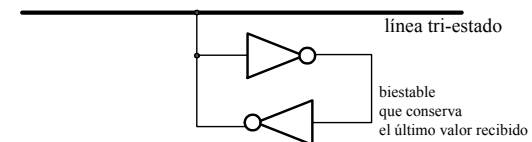
$$I_{\text{conm}} = 40 \frac{\mu\text{A}}{\text{V}^2} \cdot 1,5 \cdot 0,5 \cdot (2,1 - 1)^2 \text{ V}^2 = 36 \mu\text{A}$$

$$P = 180 \mu\text{W}$$

Un inversor, cuya entrada se encuentre en situación de alta independencia (tri-estado) o con un valor de tensión intermedio ($\sim V_{\text{conmutación}}$) genera una disipación de potencia del orden de 0,2 mW y el correspondiente calentamiento del circuito (un valor unas cuatro mil veces superior al generado en la conmutación de dicho inversor a 1MHz).

Por otra parte, existe una situación peligrosa en que este pico de intensidad (debido a la conducción «momentánea» de ambos transistores en la conmutación) puede ser estable y originar un fuerte calentamiento (e incluso la destrucción) del circuito integrado: si una entrada queda en alta impedancia tiende a adoptar un valor de tensión intermedia y ambos transistores conducen.

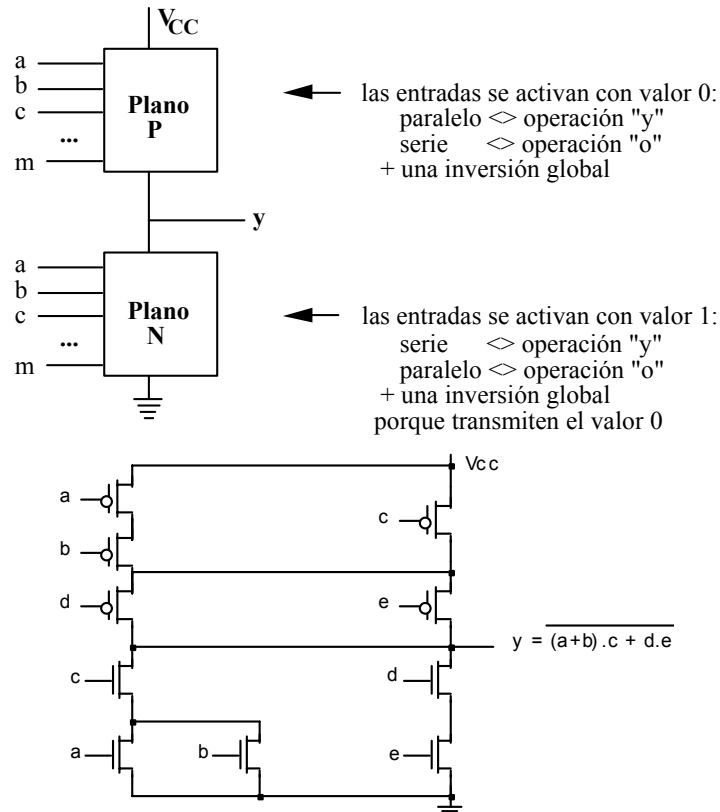
Esta situación puede darse en caso de utilizar buses (o adaptadores tri-estado) internos que puedan quedar en alta impedancia, posibilidad que puede evitarse conectando en cada línea una resistencia de alto valor, que la referencie a 0 V (*pull-down*), o bien un pequeño biestable (un par de inversores) que mantenga el último valor booleano establecido en ella (ver figura).



Conjuntos de circuitos auxiliares de este tipo, cuya finalidad es que las líneas de buses no se queden en estado de alta impedancia, se encuentran también disponibles en los catálogos de circuitos integrados.

T7.4. Puertas CMOS

Haciendo uso del álgebra de conmutadores, las puertas lógicas CMOS se construyen mediante dos planos «duales» de transistores: **plano N** y **plano P**; a cada entrada le corresponden sendos transistores, uno en el plano N y otro en el plano P, conectados de acuerdo con la dualidad serie-paralelo.



- en su plano N, la configuración de transistores corresponde (de acuerdo con el álgebra de conmutadores) a la función $(a + b) \cdot c + d \cdot e$, a la cual debe añadirse una negación global ya que dicho plano N transmite el valor 0; de forma que la función conformada por el plano N de la figura anterior es: $y = (a + b) \cdot c + d \cdot e$
- en cuanto al plano P, transmite el valor 1 pero sus transistores conducen cuando su entrada es 0, es decir, corresponden a variables negadas ($\bar{a}$, $\bar{b}$, $\bar{c}$, $\bar{d}$, $\bar{e}$) y su configuración (de acuerdo con el álgebra de conmutadores) conforma la función $(\bar{a} \cdot \bar{b} + \bar{c}) \cdot (\bar{d} + \bar{e})$; de manera que la función conformada por el plano P de la figura es la misma que la del plano N: $y = (\bar{a} \cdot \bar{b} + \bar{c}) \cdot (\bar{d} + \bar{e}) = (a + b) \cdot c + d \cdot e$

Características físicas de las puertas complementarias

Permiten una amplia diversidad de puertas

El juego de conexiones serie-paralelo permite configurar funciones complejas y muy diversas en una misma puerta, con la limitación de que la expresión algebraica de la función ha de presentar una negación global sobre el conjunto de operaciones booleanas (son puertas inversoras) y de que tal negación sea la única que aparezca: cada negación da lugar a una puerta adicional.

Los transistores P y N presentan diferente resistividad (debida a la menor movilidad de los huecos): la conexión PMOS en paralelo, y sus correspondientes NMOS en serie, contribuye a compensar la asimetría, mientras que la conexión de PMOS en serie acentúa dicha asimetría y su efecto resistivo; por ello, son preferibles las puertas "**y-negada**" (*Nand*) a las puertas "**o-negada**" (*Nor*).

Las entradas son de tipo capacitivo

Los transistores presentan una capacidad de puerta, que es preciso cargar o descargar en la conmutación cuando cambia el valor booleano presente en la correspondiente entrada; tal transitorio de carga o descarga:

- da lugar a unos tiempos de conmutación que limitan su velocidad de trabajo;
- limita, asimismo, el *fan-out* de la puerta anterior, es decir, el número de entradas de otras puertas que pueden conectarse sobre una salida (pues tal número condiciona la velocidad de trabajo);
- requiere un aporte puntual de intensidad durante la conmutación, que da lugar a un consumo dinámico proporcional a la frecuencia de conmutaciones;
- genera «ruido» sobre las líneas de alimentación como consecuencia del «pulso» de intensidad necesario para la conmutación.

La salida de cada puerta es resistiva

Cada plano de transistores, cuando conduce, presenta una resistencia relativa a la zona lineal u óhmica de sus transistores; tal resistencia depende de las dimensiones de los transistores (disminuyendo en proporción inversa a la anchura de estos) y afecta a:

- la intensidad suministrable por la puerta;
- los procesos de conmutación (carga y descarga de las capacidades de entrada de las puertas siguientes) y, en consecuencia, los tiempos de propagación y la velocidad;
- la inmunidad frente al «ruido» en términos de potencia.

Se presentan varios transistores en serie

La presencia de transistores en serie supone un aumento de la resistencia de salida de la puerta, que repercute, en concordancia con el apartado anterior, en la intensidad suministrable por la puerta, en los tiempos de propagación y la velocidad de trabajo y en la inmunidad frente al «ruido» en términos de potencia.

En cuanto a intensidades en la salida de puertas con más de una entrada, ha de tenerse en cuenta aquella situación booleana en que conducen varios transistores en serie, en cuyo caso la intensidad suministrable ha de dividirse, al menos, por el número de ellos (ya que se suman sus resistencias equivalentes) o alternativamente ha de aumentarse en igual proporción la anchura de tales transistores.

También el margen de ruido en potencia disminuye por la suma de resistencias de transistores en serie: $\Delta P = (\Delta V)^2 / R_O$.

En los tres aspectos considerados (intensidad suministrable, tiempos de propagación e inmunidad en potencia) interviene la resistencia de salida: $R_O = \Sigma R_O$ (transistores en serie). En una primera aproximación, para n transistores se multiplica por n la resistencia de salida correspondiente a uno sólo de ellos; pero, además, se produce un efecto de «desplazamiento de la tensión de fuente» que aumenta progresivamente la resistencia de los diversos transistores.

Consideremos el caso de transistores NMOS en serie, todos ellos con tensión de entrada $V_G = V(1)$, solamente el inferior de ellos tiene su fuente conectada a 0 V y su tensión $V_{GS} = V(1)$. Mientras que la tensión de salida sea $V_o > 0$ (lo cual ocurre siempre que $I_o \neq 0$ y, en particular, durante la conmutación), la tensión puerta-fuente de los demás transistores es menor $V_{GS} < V(1)$ pues su terminal de fuente no está conectado a 0 V directamente, sino a través de los transistores que se encuentran debajo; por ello, dicha tensión V_{GS} disminuye al ascender en la serie y la resistencia efectiva que presenta cada transistor es mayor cuanto más alejado se encuentra de la conexión a 0 V.

Por ello, puertas con muchas entradas (muchos transistores en serie) presentan malas características funcionales: es altamente aconsejable limitar el número de entradas de las puertas CMOS, de forma que no aparezcan más de 6 transistores en serie.

El consumo estático es siempre nulo

Para cada valor booleano en una de las entradas, uno de sus transistores se encontrará en corte y el otro conducirá: todo camino de conducción entre los dos terminales de alimentación (V_{CC} y 0 V) incluye siempre un transistor en corte, por lo cual el consumo en reposo es nulo. En cambio, sí que hay consumo dinámico originado por la carga o descarga de las diversas capacidades propias de los transistores en la conmutación y dicho consumo es proporcional a la frecuencia de conmutación.

Tiempos de conmutación

Al pasar de un inversor a una puerta booleana de dos o más entradas, los tiempos de propagación aumentan en aquellos procesos (carga o descarga) en que la conducción se produce a través de varios transistores en serie; es un efecto de suma de sus resistencias.

Para la tecnología Ψ :

- NAND de 2 entradas: $t_{pLH} = V_{CC} \cdot C / I(P) \approx 5 \text{ V} \cdot 2 \text{ fF} / 0,2 \text{ mA} \approx 0,05 \text{ ns}$,
 $t_{pHL} = 2 \times V_{CC} \cdot C / I(N) \approx 2 \cdot 5 \text{ V} \cdot 2 \text{ fF} / 0,5 \text{ mA} \approx 0,04 \text{ ns}$
- NOR de 2 entradas: $t_{pLH} = 2 \times V_{CC} \cdot C / I(P) \approx 2 \cdot 5 \text{ V} \cdot 2 \text{ fF} / 0,2 \text{ mA} \approx 0,1 \text{ ns}$,
 $t_{pHL} = V_{CC} \cdot C / I(N) \approx 5 \text{ V} \cdot 2 \text{ fF} / 0,5 \text{ mA} \approx 0,02 \text{ ns}$

Debido a la diferente resistividad de los transistores PMOS y NMOS, interesa más la utilización de puertas "**y-negada**" (*Nand*) pues en ellas se equilibran un poco los dos tiempos de conmutación (aumenta el menor de ambos), mientras que para las puertas "**o-negada**" (*Nor*) se aumenta aún más el de subida (que es, ya de por sí, el mayor).

Ahora bien, cuando se desea obtener estimaciones cuantitativas de los tiempos de propagación de una puerta o de un conjunto de puertas booleanas es necesario acudir a la simulación eléctrica (SPICE u otros). Por un lado, los efectos relativos a transistores en serie (comentados en la página anterior) y, de otro, el solapamiento entre los tiempos de conmutación de puertas sucesivas (ya que cada puerta inicia su conmutación antes de haber finalizado la conmutación de la puerta anterior) limita, en gran medida, los resultados obtenidos a través de modelos simplificados o de razonamientos cualitativos.

T8 PUERTAS SEUDO-NMOS Y PUERTAS DE TRANSMISIÓN

T8.1. Puertas pseudoNMOS

T8.2. Transistor de paso y puerta de transmisión

T8.3. Lógica dinámica

La tecnología **CMOS**, además de las puertas inversoras configuradas con dos planos de transistores complementarios (puertas «complementarias»), ofrece otros dos tipos de puertas que son muy útiles en determinados casos.

Las puertas **NOR pseudoNMOS**, en las cuales el plano P queda reducido a un único transistor PMOS que actúa como resistencia de polarización, permiten un alto número de entradas, sin que ello repercuta en alto número de transistores en serie (lo cual limitaría gravemente la respuesta de la puerta en cuanto a intensidad, velocidad y ruido). Estas puertas pseudoNMOS son muy apropiadas para configurar las estructuras reticulares: decodificadores, multiplexores y demultiplexores, y matriciales: ROM, PAL y PLA; también lo son para los dispositivos programables derivados de tales estructuras, con la importante ventaja de que solamente es preciso programar el plano N.

Por contra, las puertas pseudoNMOS pierden la característica de consumo estático nulo y requieren un dimensionamiento adecuado de sus transistores para asegurar la tensión de salida correspondiente al 0: $V(0) \approx 0 \text{ V}$.

Los transistores MOS individuales, utilizados como conmutadores (transistores de paso), presentan limitaciones a la hora de transmitir uno de los valores booleanos (el NMOS transmite mal el 1 y el PMOS el 0). Pero el conjunto NMOS - PMOS en paralelo resuelve esa limitación y conforma una «puerta de transmisión» que es útil para configurar multiplexores y biestables; tales bloques, contruidos con puertas de transmisión, requieren muchos menos transistores que si se configuran con puertas inversoras, ocupan menor área de integración y ofrecen menores tiempos de propagación. Asimismo, la puerta de transmisión es un adaptador tri-estado, pues, cuando no conduce, su salida se encuentra en situación de desconexión (alta impedancia).

Además, este capítulo introduce la «lógica dinámica» que, utilizando como «elementos de memoria» las capacidades de entrada de las propias puertas lógicas, permite reducir aún más el área de integración de los «biestables» y, con ello, aumentar la densidad de integración de los circuitos secuenciales. Este almacenamiento de tensiones booleanas sobre pequeños condensadores tiene el inconveniente de la descarga de los mismos a lo largo del tiempo pero resulta útil si dichos valores se actualizan o se refrescan con una frecuencia adecuada (superior a un cierto valor mínimo, $\sim 1 \text{ KHz}$).

Por otra parte, la «lógica dinámica» aplicada a las puertas pseudoNMOS, permite su funcionamiento con consumo estático nulo.

T8.1. Puertas pseudoNMOS

Cuando se necesitan puertas con un amplio número de entradas (en particular en las estructuras matriciales en que tal número suele ser muy alto) se utiliza la configuración «pseudoNMOS» y se emplean únicamente puertas "**o-negada**" (Nor):

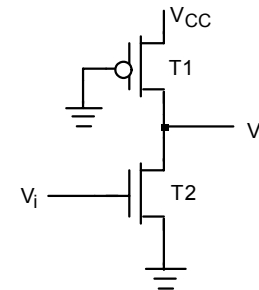
- la lógica «pseudoNMOS» conforma solamente el plano N y utiliza un solo transistor PMOS permanentemente habilitado (conectado a 0 V), que se comportará como una simple carga resistiva;
- las puertas "**o-negada**" (Nor) presentan sus transistores en paralelo en el plano N; no existe problema de incluir en dicho plano muchos transistores.

El calificativo «pseudoNMOS» proviene de que este tipo de puertas, con un transistor superior (que conduce siempre) como resistencia de polarización, es propio de las tecnologías NMOS: la lógica pseudoNMOS copia directamente el diseño NMOS utilizando un transistor de polarización de canal P.

La configuración "**o-negada**" (Nor) pseudoNMOS proporciona una alternativa eficaz para puertas de alto número de entradas y, además, reduce considerablemente el número de transistores, el área de integración y la complejidad del diseño físico; pero, en cambio presenta dos serios inconvenientes:

- los niveles de tensión de salida no quedan garantizados por la tensión umbral de los transistores
- y su consumo no es nulo, ya que para $V_0 = 0$ conducen el plano N y el transistor P.

Diseño de un inversor pseudoNMOS:



Sea $V_{CC} = 5 \text{ V}$ y $V_{TO} = 1 \text{ V}$,

$V_i < 1 \text{ V}$ T_2 no conduce y T_1 sí

$$\Rightarrow V_0 = V_{CC} = 5 \text{ V}$$

$V_i \gg 1 \text{ V}$ T_2 conduce y T_1 también

$$V_0 = V_{CC} \cdot R_2 / (R_1 + R_2)$$

siendo R_1 = resistencia PMOS

y R_2 = resistencia NMOS:

Interesa V_0 pequeño; para ello, $R_2 \ll R_1$.

En un inversor pseudoNMOS la salida 1 se produce de igual forma que en un inversor CMOS: conduce el transistor PMOS y no lo hace el NMOS; pero para salida 0 conducen ambos transistores y para asegurar una tensión de salida próxima a 0 V se requiere una «relación de geometrías» entre ambos transistores adecuada.

En el inversor CMOS cada valor booleano está garantizado por la tensión umbral del transistor que transmite dicho valor; en el pseudoNMOS hay un solo transistor activo, el NMOS, que garantiza solamente su situación de corte, salida **1**, mientras que para salida **0** aparece un divisor de tensión entre las resistencias que presentan ambos transistores y es preciso conseguir, por diseño geométrico, que la resistencia que presenta el transistor N sea muy inferior a la presentada por el P: $R_{NMOS} \ll R_{PMOS}$.

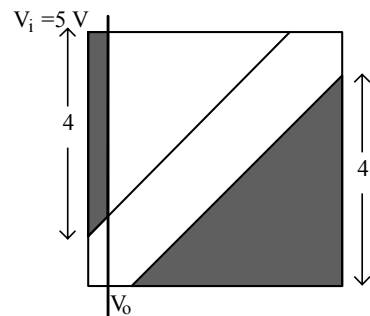
Habida cuenta de que la resistencia de paso de un transistor MOS es inversamente proporcional a su factor de forma $ff = W/L$, se requiere una relación adecuada entre los tamaños de los dos transistores que configuran el inversor, de manera que sus factores de forma verifiquen la desigualdad: $W_N/L_N \gg W_P/L_P$.

El diagrama de Memelink permite un análisis cuantitativo:

Sea $V_{CC} = 5\text{ V}$, $V_{TO} = 1\text{ V}$ y β la relación entre factores de forma: $\beta = W_P/L_P / W_N/L_N$

En situación estática por ambos transistores circula la misma intensidad:

$$\frac{A_N}{A_P} = \frac{\alpha_P}{\alpha_N} = \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} \frac{W_P/L_P}{W_N/L_N}, \quad \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} \approx \frac{1}{3}, \quad \frac{A_2}{A_1} \approx \frac{1}{3} \frac{ff_P}{ff_N} = \frac{1}{3} \beta$$



$$A_N = (\beta/3) \cdot A_P$$

$$4 V_o = (\beta/3) 4^2/2$$

$$V_{oL} = (2/3)\beta = 0,66 \beta$$

Para lo cual, interesa β pequeño: $\beta \ll 1$

$$\beta = \frac{ff_P}{ff_N} = \frac{W_P/L_P}{W_N/L_N} \ll 1$$

La tensión de salida para el **0** no será exactamente 0 V sino algo inferior a $\beta : 0,66 \beta$

$$\beta = 1/4 = 0,25$$

$$V_o = 0,17\text{ V}$$

$$\beta = 1/5 = 0,2$$

$$V_o = 0,13\text{ V}$$

$$\beta = 1/7 = 0,15$$

$$V_o = 0,1\text{ V}$$

Interesan valores de β bajos; además, la función de transferencia del inversor pseudoNMOS presenta mejores características de conmutación (se hace más abrupta), cuanto menor es β .

Para conseguir $\beta \ll 1$, con la menor ocupación posible de área de integración, se aumenta adecuadamente la longitud del transistor PMOS de polarización (habida cuenta de que en una puerta hay un solo transistor PMOS y muchos NMOS): para $\beta = 1/4$, $L_{PMOS} = 4 L_{\text{mín}}$.

Al utilizar lógica pseudoNMOS se pierde la característica de consumo nulo propia de la tecnología CMOS; cuando la salida es **0** conducen ambos transistores y, por tanto, circula una intensidad no nula:

$$I = K_p \cdot \frac{W}{L} \cdot A_N = K_p \cdot 1,5 \cdot (V_i - 1) \cdot V_o = K_p \cdot 1,5 \cdot 4 \cdot 0,66 \cdot \beta = 4 \cdot K_p \cdot \beta.$$

Para la tecnología Ψ :

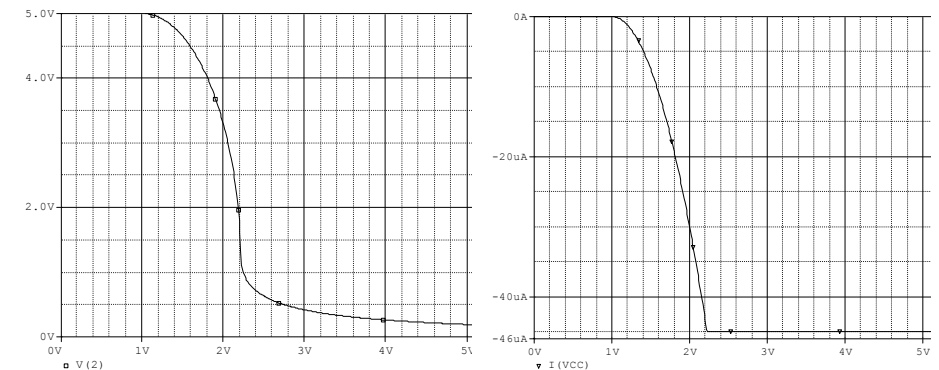
$$K_p (\text{NMOS}) \sim 40 \mu\text{A/V}^2;$$

$$I = 160 \beta \mu\text{A}$$

$$\text{para } \beta = 1/4 = 0,25$$

$$I = 160 \beta = 40 \mu\text{A}$$

Las gráficas siguientes presentan la función de transferencia $V_o - V_i$ y el consumo de intensidad de un inversor pseudoNMOS de la tecnología Ψ , obtenidas mediante simulación SPICE; el transistor NMOS es de tamaño mínimo y la relación de geometrías es $\beta = 1/4$ $L_{NMOS} = 1 \mu\text{m}$, $W_{NMOS} = 1,5 \mu\text{m}$, $V_{TO, NMOS} = 1\text{ V}$, $K_{p, NMOS} = 40 \mu\text{A/V}^2$ $L_{PMOS} = 4 \mu\text{m}$, $W_{PMOS} = 1,5 \mu\text{m}$, $V_{TO, PMOS} = -1\text{ V}$, $K_{p, PMOS} = 15 \mu\text{A/V}^2$.



Función de transferencia $V_o - V_i$ pseudoNMOS Consumo de intensidad I_{CC} del inversor

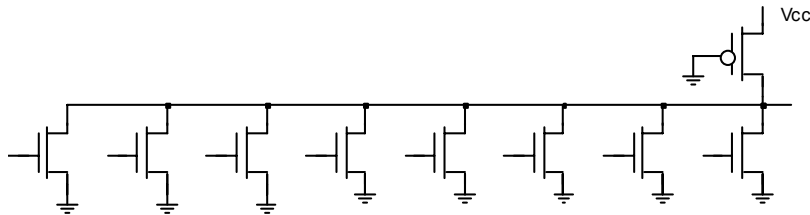
La lógica pseudoNMOS evita la necesidad de duplicar el Plano N en una combinación dual de transistores PMOS y la necesidad de conectar entre sí cada pareja de transistores; con ello se reduce considerablemente el área de integración y la complejidad del diseño geométrico. Pero, en cambio, presenta dos inconvenientes relativos, ambos, a la salida **0**:

1. en su diseño ha de tenerse en cuenta la relación de geometrías de los transistores: el cociente entre factores de forma $\beta = W_P/L_P / W_N/L_N$ para asegurar que $V_o(0) \approx 0\text{ V}$;
2. el consumo no es nulo: $I_{CC}(0) \neq 0$, pues conducen a la vez el plano N y el transistor P.

El primero de los inconvenientes repercute sólo en consideraciones geométricas en la etapa de diseño y la correspondiente ocupación de área.

El segundo, el consumo no nulo, no es grave si el número de estas puertas en un circuito integrado es reducido y su efecto puede disminuirse conectando la puerta del transistor PMOS a una señal de habilitación $\bar{E}$, normalmente a **1** y que pase a **0** solamente cuando interese la actuación de la correspondiente puerta lógica (caso de que el cálculo booleano que realiza dicha puerta no se requiera constantemente).

La configuración pseudoNMOS utiliza solamente puertas **"o-negada"** (Nor), ya que en ellas los transistores se encuentran en paralelo:



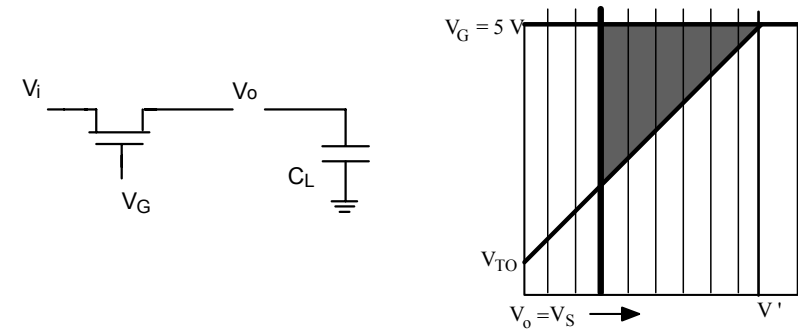
Estas puertas **"o-negada"** (Nor) pseudoNMOS resultan muy útiles para construir las estructuras reticulares de muchas entradas (decodificadores, multiplexores, demux., ROM, PLA, PAL,...); también son muy apropiadas para dispositivos programables, tipo PROM, PLA, PAL, PLS o CPLDs. Ello es debido a que:

- permiten configurar términos producto de un alto número de entradas, gracias a la conexión en paralelo de sus transistores
- y presentan un solo plano activo de forma que, en cuanto a dispositivos programables, solamente será necesario efectuar la programación sobre el plano NMOS y tal programación consistirá en «desconectar» transistores que se encuentran en paralelo.

Si se utilizan puertas CMOS, con los dos planos de transistores completos, además del problema de tener un alto número de transistores en serie, la programación de ambos planos sería sumamente compleja, pues, en aquel en que los transistores se encuentren en serie, su programación consistirá en «puentearlos» (lo cual es mucho más difícil que «desconectarlos»).

T8.2. Transistor de paso y puerta de transmisión

Otro importante complemento de las puertas booleanas CMOS, conformadas por sendos planos de transistores P y N, lo constituye el transistor de paso, equivalente a un simple interruptor:



El transistor conduce cuando $V_G = V_{CC} = \mathbf{1}$ y no lo hace para $V_G = 0 \text{ V} = \mathbf{0}$.

- A) Supongamos $V_G = \mathbf{1} = 5 \text{ V}$, para $V_i = 5 \text{ V}$ el transistor conduce y configura un camino de carga hacia la capacidad de entrada de la puerta o puertas a las que está conectado (C_L).

En este proceso de carga el terminal de entrada V_i actúa como drenaje y el terminal de salida V_o como fuente; al ir aumentando $V_o = V_S$ la intensidad se hace más pequeña, finalizando el proceso de carga cuando $V_o = V_S = V'$ (es decir, cuando $V_{TO} + V'$ alcanza el valor V_G), en cuyo caso ya no existe canal y el transistor no conduce.

La tensión de salida máxima sobre C_L es V' :

$$V_{TO} + V' = V_G; \quad V' = V_G - V_{TO} \quad (4 \text{ V para } V_G = 5 \text{ V y } V_{TO} = 1 \text{ V})$$

Este desplazamiento a la baja de la tensión de salida **1** es aún mayor debido al «efecto substrato» [ver apartado T6.3.6]:

$$V_G = V_{TO} + K V_D \quad V_D = (V_G - V_{TO}) / K \quad (\sim 3,5 \text{ V para } K = 1,15).$$

La salida no llega a alcanzar la tensión correspondiente al **1** booleano $V(\mathbf{1})$ sino que se queda por debajo. Con ello la puerta siguiente a un transistor de paso no tiene asegurado el corte del transistor PMOS; se pierde entonces la condición de consumo nulo y disminuye en gran medida el margen de ruido de dicha puerta.

Ahora bien, puede evitarse dicha situación (consumo no nulo de la puerta que sigue a un transistor de paso) si se integran transistores PMOS cuya tensión umbral V_{TO} sea mayor que la tensión umbral del transistor NMOS de paso; para $V_{TO,PMOS} = -1,8 \text{ V}$, la tensión de salida **1** del transistor de paso ($\sim 3,5 \text{ V}$) es suficiente para mantener en corte los transistores PMOS.

El desplazamiento de $V(1)$ no es acumulativo (no es un efecto resistivo sino de cierre de canal), de forma que si se conectan varios transistores de paso seguidos (en serie), la tensión de salida **1** del conjunto seguirá siendo $V_D = (V_G - V_{TO})/K$.

B) Para $V_i = 0$ V, si la capacidad C_L se encontraba previamente cargada (tensión de salida $V(1)$), se produce su descarga hasta alcanzar 0 V: el paso del **0** booleano se efectúa sin desplazamiento de tensión.

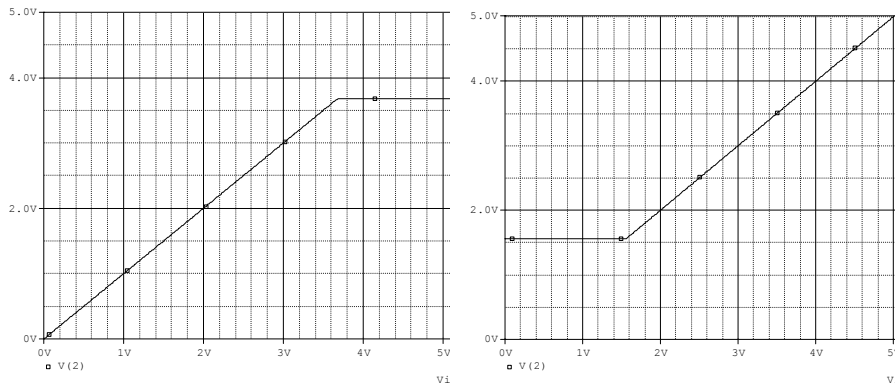
Téngase en cuenta que el transistor MOS es simétrico:

- cuando $V_i = 5$ V dicho terminal actúa como drenaje y V_o como fuente,
- pero para $V_i = 0$ V los papeles de drenaje y fuente se intercambian ya que la intensidad en el transistor circula en sentido contrario.

En conclusión, como transistor de paso el NMOS transmite bien el valor booleano **0** pero no el valor **1** (cuya tensión queda reducida al pasar a través del transistor); por ello, en las puertas CMOS los transistores NMOS forman el plano inferior que transmite el valor **0**. Por simetría, el transistor PMOS transmite bien el valor **1** pero no el **0** (que se desplaza hacia tensiones positivas al pasar a través del transistor); en las puertas CMOS los PMOS forman el plano superior que transmite el valor **1**.

Funciones de transferencia de transistores de paso NMOS y PMOS:

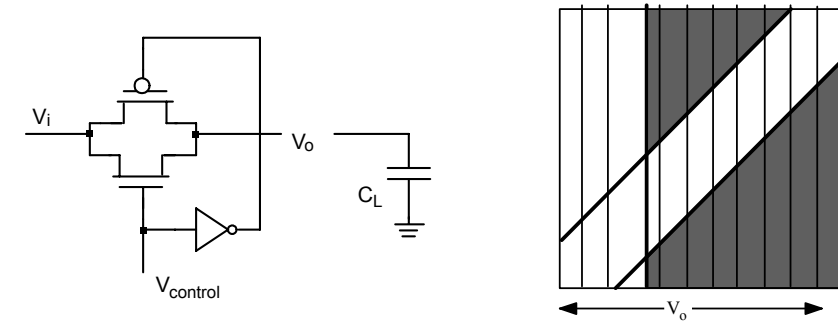
Tecnología: $V_{TO,NMOS} = 1$ V, $V_{TO,PMOS} = -1$ V, $GAMMA = 0,25$, $NSUB = 10^{15}$.



Transistor de paso NMOS

Transistor de paso PMOS

Añadiendo al transistor de paso canal N un transistor complementario canal P, en paralelo (y con las puertas de ambos conectadas a través de un inversor para que se encuentren en conducción y en corte a la vez) se obtiene una puerta de transmisión que evita el desplazamiento en tensión del **1** booleano.

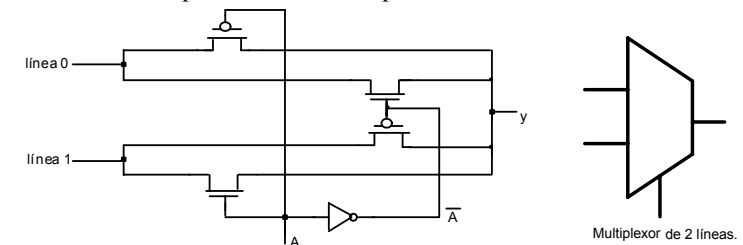


En una puerta de transmisión la salida (sobre C_L) llega a alcanzar $V(1)$ (caso de $V_i = V(1)$) pues el transistor PMOS conduce hasta ese valor y la descarga llega igualmente a 0 V (cuando $V_i = 0$) a través del transistor NMOS. El transistor PMOS asegura la salida **1** sin desplazamiento de tensión y el transistor NMOS hace lo propio para la salida **0**; por contra, son precisos 4 transistores (dos en la puerta de transmisión y otros dos del inversor).

El cálculo de los tiempos de conmutación de una puerta de transmisión es análogo al desarrollado en el capítulo T7 para un inversor, de forma que los tiempos resultantes son del mismo orden e incluso inferiores a los allí obtenidos: la carga se realiza básicamente por un transistor PMOS ayudado parcialmente por el NMOS que conlleva en paralelo y viceversa (y tal colaboración parcial de ambos transistores reduce los tiempos).

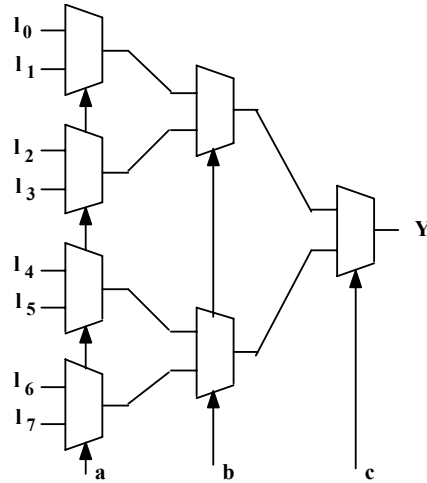
Multiplexores

La puerta de transmisión tiene el inconveniente de que la variable de control ha de actuar a través de dos entradas en forma invertida (con un inversor intermedio), de manera que a los dos transistores que configuran la puerta han de agregarse otros dos para el correspondiente inversor. Pero este inconveniente no es tal en aquellas estructuras que siempre utilizan las entradas en forma afirmada y negada; en tal sentido las puertas de transmisión resulta útiles para construir multiplexores:



Multiplexor de 2 líneas.

Un multiplexor de 2 entradas es directamente ampliable conectando módulos iguales en cascada (para n líneas serán necesarios $n-1$ módulos conectados en cascada):



El esquema anterior contiene 14 puertas de transmisión y 3 inversores, en total 34 transistores, mientras que un multiplexor análogo realizado con puertas "y-negada" (*Nand*) necesita 8 puertas de 4 entradas, una de 8 y 3 inversores, es decir, 86 transistores.

De esta forma, la conexión en cascada de puertas de transmisión proporciona una configuración muy simple y modular para los multiplexores, el número de transistores necesarios es claramente inferior a los que se requieren en una configuración típica CMOS (con puertas *Nand* o *Nor*) y el tiempo de propagación es algo menor.

La reducción en el número de transistores se refleja sobre el área de integración necesaria y, también, sobre los tiempos de propagación: habrá menor número de transistores en serie, con lo cual las constantes de tiempo que limitan la velocidad del circuito serán menores. Por ejemplo, en el multiplexor anterior, la transmisión de un **0** implica a 3 transistores NMOS en serie, mientras que con puertas "y-negada" (*Nand*) habría 8 transistores NMOS en serie en la puerta de salida.

Habida cuenta de que las puertas de transmisión son simétricas, un multiplexor construido con ellas también lo es y puede ser utilizado como demultiplexor; en tal caso, presenta el problema de que las líneas de salida no seleccionadas quedan en alta impedancia (no en salida **0**, como correspondería a un demultiplexor digital), lo cual puede solventarse añadiendo resistencias que referencien a 0 V dichas líneas.

Incluso, en muchas ocasiones, es útil el estado de alta impedancia de las salidas del demultiplexor pues permite conservar sobre cada línea el último de los valores transmitidos a la misma (para lo cual será necesario que exista una capacidad adecuada conectada en cada línea de salida).

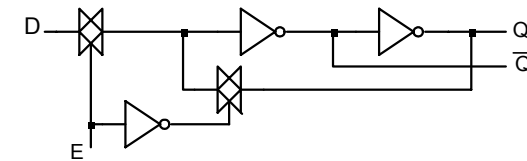
Las puertas de transmisión son, también, buenos interruptores analógicos (*analog switches*) que controlan el paso de señales en el intervalo $[0, V_{CC}]$ determinado por las tensiones de control (de puerta) de sus transistores; caso de que la señal de control utilice como niveles $-V_1$ y $+V_2$, la puerta de transmisión admitirá señales analógicas entre tales valores $[-V_1 ; +V_2]$.

De esta forma, las puertas de transmisión sirven para muestrear señales analógicas y para realizar su multiplexado o demultiplexado: un multiplexor construido con puertas de transmisión puede transmitir cualquier tensión dentro de los límites fijados por sus tensiones de control $[-V_1 ; +V_2]$; de esta forma se dispone de multiplexores analógicos que permite seleccionar por su número una de entre n tensiones de entrada (por ejemplo, para medir diversas tensiones con un mismo conversor analógico-digital).

El mismo multiplexor, utilizado en sentido inverso (una entrada, n salidas) es un demultiplexor analógico, cuyas líneas de salida adoptan el estado de alta impedancia cuando no son seleccionadas (y pueden conservar, sobre capacidades, el último valor de tensión transmitido a cada línea).

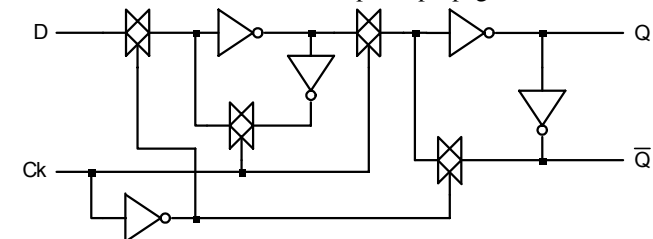
Biestables

Los biestables en tecnologías CMOS suelen construirse con puertas de transmisión:



Biestable D habilitado por niveles

Esta configuración requiere 10 transistores, mientras que la configuración típica con puertas "y-negada" (*Nand*) necesita 18; la reducción del número de transistores implica, también en este caso, una disminución del tiempo de propagación del biestable.



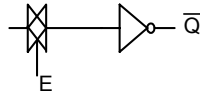
Biestable D síncrono: habilitado por flancos

Para construir un biestable D síncrono con puertas de transmisión se utilizan 18 transistores, frente a los 36 necesarios con puertas "y-negada" (*Nand*). Además de la reducción del número de transistores y del área de integración, también se reducen en forma importante los tiempos funcionales y, en particular, el tiempo de propagación.

T8.3. Lógica dinámica

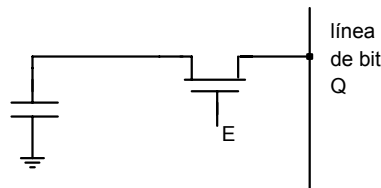
Biestables dinámicos

Los registros dinámicos utilizan biestables aún más simples que los representados en la página anterior:



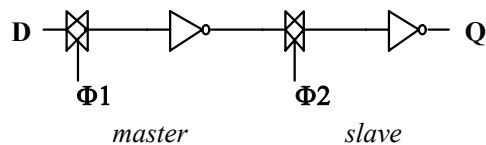
En este «biestable» la información se almacena sobre la capacidad de entrada del inversor (la cual sufre, cuando queda aislada, un proceso de descarga que limita el tiempo de almacenamiento a unos pocos milisegundos).

En muchas ocasiones, por ejemplo en las memorias dinámicas DRAM, el biestable se reduce solamente a un transistor y un condensador:

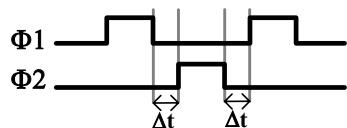


El ahorro en número de transistores (y por tanto, en área de integración) es superior a 2/3 respecto a los biestables estáticos; tal reducción de tamaño resulta muy atractiva y por ello, cuando el tiempo de permanencia de la información es reducido (inferior a 10 milisegundos) o cuando resulta fácil «refrescar» la información, se utilizan «registros dinámicos» configurados con estos biestables.

Particularmente interesante (por la reducción de tamaño) es la utilización de «biestables dinámicos» en el diseño síncrono (con reloj):



Este biestable síncrono dinámico requiere solamente 8 transistores mientras que su configuración estática (con puertas de transmisión) necesita 18 (y 36 si se configura con puertas *Nand*); los «biestables dinámicos síncronos» necesitan dos fases de reloj no solapadas:



Cuando $\Phi 1 = 1$ el *master* recoge la información de su entrada: «fase de captura» y en $\Phi 2 = 1$ el *slave* la presenta en su salida: «fase de salida».

La separación entre ambas fases (entre la «captura» y la «salida» de la información) viene exigida porque el más mínimo solapamiento entre ellas lleva directamente a la salida **Q** el valor de la entrada **D**. Es un requisito de seguridad de funcionamiento (véase el apartado 15.4, segundo volumen) que evita las restricciones relativas al tiempo de mantenimiento y a la verticalidad y simultaneidad en la señal de reloj; estas restricciones son particularmente fuertes en este caso, debido al reducido tiempo de propagación que presentan estos biestables tan simples.

La conexión en serie de «biestables dinámicos síncronos» da lugar a registros de desplazamiento dinámicos, con una ocupación de superficie muy reducida, y muy útiles cuando el desplazamiento se ha de producir con frecuencias de reloj superiores a 1 KHz (por debajo de esa frecuencia el tiempo de permanencia del dato en las capacidades que actúan como elementos de memoria, ~1 ms, no es suficiente).

En forma análoga es posible construir contadores dinámicos (para frecuencias de reloj superiores al KHz) sin más que añadir a las entradas D_i la lógica correspondiente al conteo. Por ejemplo, para un contador década con entrada de habilitación **E**:

$$D_0 = \overline{q_0} \cdot E + q_0 \cdot \overline{E} = q_0 \oplus E$$

$$D_1 = \overline{q_1} \cdot q_0 \cdot \overline{q_3} \cdot E + q_1 \cdot (\overline{q_0} + \overline{E})$$

$$D_2 = \overline{q_2} \cdot q_1 \cdot q_0 \cdot E + q_2 \cdot (\overline{q_1} + \overline{q_0} + \overline{E})$$

$$D_3 = \overline{q_3} \cdot q_2 \cdot q_1 \cdot q_0 \cdot E + q_3 \cdot (\overline{q_0} + \overline{E})$$

De igual forma, cualquier sistema secuencial síncrono puede diseñarse con biestables dinámicos; en general, tales biestables permiten una fuerte reducción en el área de integración, siempre que el reloj del sistema síncrono actúe a frecuencias no bajas.

Tiempo de permanencia de un dato en un registro dinámico

En los registros dinámicos (conformados por una puerta de transmisión y un inversor) el dato es almacenado en la capacidad de entrada del inversor (capacidad de puerta de los dos transistores que lo conforman); cuando la puerta de transmisión es deshabilitada la conservación de carga en dicha capacidad asegura la permanencia del valor booleano en el inversor durante un cierto tiempo.

Ahora bien, en la situación de deshabilitación las propias uniones difusión-sustrato de la puerta de transmisión configuran un camino de descarga para la capacidad de puerta que mantiene el valor booleano; éstas uniones se encontrarán en polarización inversa, lo cual supone una intensidad de corriente sumamente pequeña pero no nula. Al cabo del tiempo dicha capacidad se descargará y el valor booleano se perderá: precisamente por ello estos registros son dinámicos, necesitan ser refrescados cada cierto tiempo (presentan una frecuencia mínima de trabajo).

Podemos definir el tiempo de permanencia como aquel en que la variación de tensión de la capacidad de puerta es inferior a la tensión umbral de los transistores V_{TO} , ya que a partir de ese valor de tensión la intensidad estática por el inversor deja de ser nula (se pierde la condición de consumo estático nulo).

Una estimación del orden de magnitud puede ser la siguiente:

intensidad de descarga ~ 1 pA

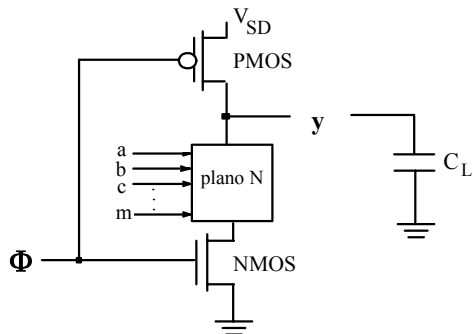
capacidad de puerta $\sim 1 \text{ fF}$

$$\Delta V = V_{TO} \sim 1 \text{ V} \quad t = C \cdot \Delta V / I \approx 10^{-15} / 10^{-12} = 10^{-3} \text{ s} = 1 \text{ ms.}$$

Lógicas pseudoNMOS dinámicas.

Para evitar el problema del consumo estático no nulo en las estructuras pseudoNMOS, se utiliza una «lógica dinámica» con reloj, en la cual se distinguen dos fases en cada unidad de tiempo (pulso de reloj):

- | | |
|--|------------|
| - fase de precarga (de la salida de la puerta) | $\Phi = 0$ |
| - fase de evaluación (de la función que realiza) | $\Phi = 1$ |



Se añade a la puerta un transistor auxiliar NMOS y los dos transistores extremos, el PMOS de polarización y el NMOS auxiliar, se conectan al reloj Φ de forma que, en ninguna de las dos fases de Φ , conducen a la vez ambos transistores: el consumo de la puerta es nulo.

En cada pulso de reloj, la salida es llevada inicialmente a 1:

```
precarga       $\Phi = 0$ ,   conduce el transistor PMOS,   salida  $y = 1$ 
```

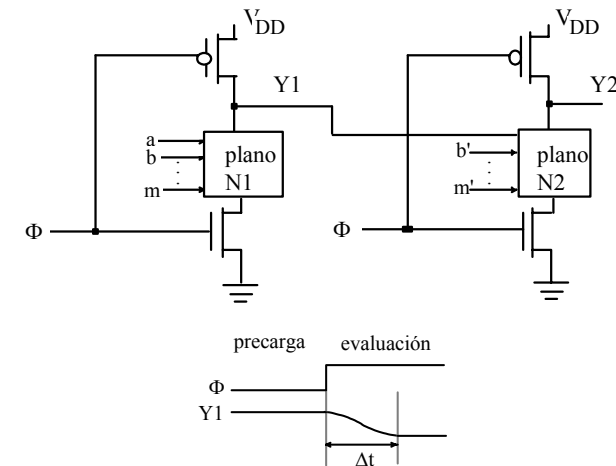
y, posteriormente, el plano N calcula la función:

evaluación $\Phi = 1$, caso de valor **0** descarga la salida, salida $y = f(X)$.

Durante la fase de precarga la salida es llevada a valor booleano **1** a través del transistor PMOS de carga (que conduce cuando $\Phi = 0$). En la fase de evaluación, si la salida (conforme a la función que calcula la puerta) debe ser **0**, tal valor es comunicado a través del plano N y del transistor NMOS auxiliar; pero cuando deba ser **1**, dicho valor ha de conservarse (sobre la capacidad de entrada de la puerta siguiente) a partir de la tensión almacenada en la precarga.

La duración de la fase de evaluación no puede ser indefinida, pues existirá un proceso de descarga que deteriorará el valor booleano almacenado: estas puertas son «dinámicas» y han de actuar por encima de una cierta frecuencia de reloj. Serán puertas de tipo "**o-negada**" (*Nor*) para admitir un amplio número de entradas (que es la aplicación propia de las puertas pseudoNMOS).

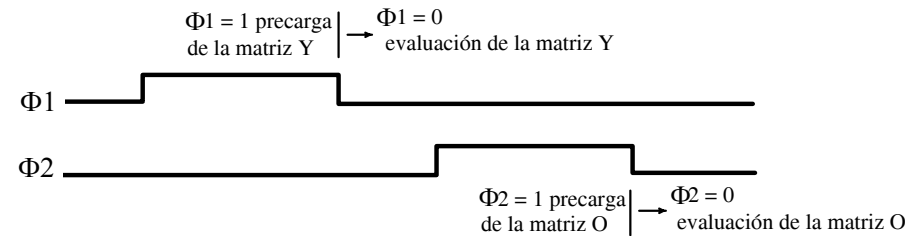
Además, este tipo de puertas (pseudoNMOS dinámicas) no pueden conectarse directamente entre sí, pues es preciso que los correspondientes valores de sus entradas (en particular, las entradas cuyo valor sea 0) estén presentes antes de iniciarse la fase de evaluación.



Durante la fase de precarga ambas salidas se ponen a **1**; al iniciarse la evaluación la salida Y1 tardará un cierto tiempo Δt en pasar a **0** (supuesto que tal sea su valor), durante el cual (Δt) Y1 provoca la descarga de Y2 (puertas *Nor*, transistores N en paralelo), de forma que, si Δt es suficientemente amplio para que Y2 deje de valer **1**, ya no podrá adoptar tal valor booleano (Δt anula el efecto de la precarga).

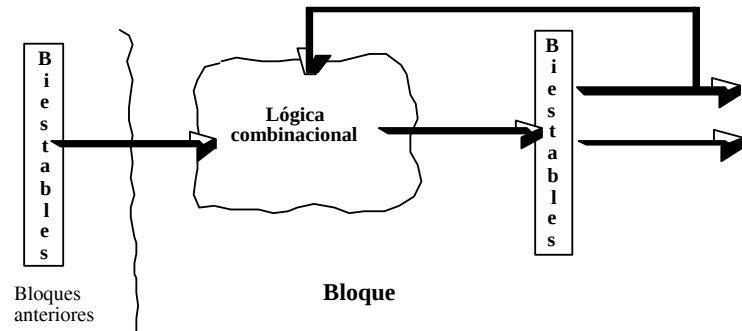
Por ello, estas puertas pseudoNMOS dinámicas son de utilización individual. Para conseguir la conectividad directa entre puertas pseudoNMOS dinámicas se han diseñado soluciones sumamente «complejas», «ingeniosas» y variadas: lógicas con 4 fases de reloj, lógicas dominó, C²MOS, CSVL,..., pero tales soluciones son más «académicas» que efectivas.

En las estructuras matriciales tipo ROM, PLA, PAL, ..., que utilizan dos conjuntos sucesivos de puertas (matriz Y - matriz O), es posible conectar tales puertas mediante el empleo de dos fases de reloj Φ_1 , Φ_2 no solapadas:



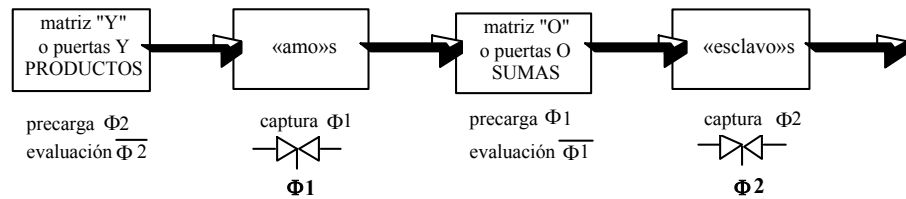
De esta forma, el primer conjunto de puertas (matriz Y) evalúa sus funciones booleanas mucho antes de iniciarse la evaluación del segundo conjunto (matriz O).

En general, los bloques constitutivos de un sistema digital son del tipo siguiente



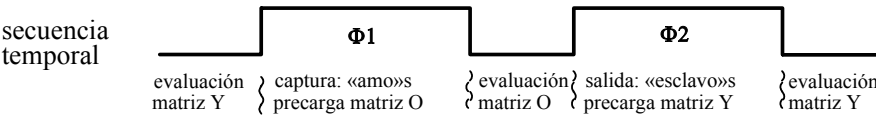
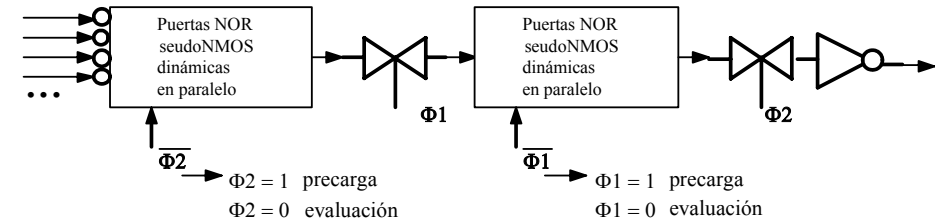
es decir, la lógica combinacional se encuentra entre biestables.

Las funciones booleanas «intermedias» (la lógica combinacional) pueden ser construidas en forma matricial (suma de productos = PAL) y los biestables (los de salida del propio bloque) pueden ser descompuestos en sus dos partes «amo-esclavo» e intercaladas en la lógica:



Cada semibi estable (*master/slave*) puede ser construido, en configuración dinámica, mediante una simple puerta de transmisión: el valor booleano queda almacenado en la capacidad de entrada de la puerta siguiente (o puertas siguientes).

Habida cuenta de que tanto la matriz Y como la matriz O han de ser realizadas mediante puertas "o-negada" (*Nor*), el bloque considerado quedaría en la forma siguiente:



T8 PUERTAS SEUDO-NMOS Y PUERTAS DE TRANSMISIÓN

T8.1. Puertas pseudoNMOS

T8.2. Transistor de paso y puerta de transmisión

T8.3. Lógica dinámica

La tecnología CMOS, además de las puertas inversoras configuradas con dos planos de transistores complementarios (puertas «complementarias»), ofrece otros dos tipos de puertas que son muy útiles en determinados casos.

Las puertas NOR pseudoNMOS, en las cuales el plano P queda reducido a un único transistor PMOS que actúa como resistencia de polarización, permiten un alto número de entradas, sin que ello repercuta en alto número de transistores en serie (lo cual limitaría gravemente la respuesta de la puerta en cuanto a intensidad, velocidad y ruido). Estas puertas pseudoNMOS son muy apropiadas para configurar las estructuras reticulares: decodificadores, multiplexores y demultiplexores, y matriciales: ROM, PAL y PLA; también lo son para los dispositivos programables derivados de tales estructuras, con la importante ventaja de que solamente es preciso programar el plano N.

Por contra, las puertas pseudoNMOS pierden la característica de consumo estático nulo y requieren un dimensionamiento adecuado de sus transistores para asegurar la tensión de salida correspondiente al 0: $V(0) \approx 0 \text{ V}$.

Los transistores MOS individuales, utilizados como conmutadores (transistores de paso), presentan limitaciones a la hora de transmitir uno de los valores booleanos (el NMOS transmite mal el 1 y el PMOS el 0). Pero el conjunto NMOS - PMOS en paralelo resuelve esa limitación y conforma una «puerta de transmisión» que es útil para configurar multiplexores y biestables; tales bloques, contruidos con puertas de transmisión, requieren muchos menos transistores que si se configuran con puertas inversoras, ocupan menor área de integración y ofrecen menores tiempos de propagación. Asimismo, la puerta de transmisión es un adaptador tri-estado, pues, cuando no conduce, su salida se encuentra en situación de desconexión (alta impedancia).

Además, este capítulo introduce la «lógica dinámica» que, utilizando como «elementos de memoria» las capacidades de entrada de las propias puertas lógicas, permite reducir aún más el área de integración de los «biestables» y, con ello, aumentar la densidad de integración de los circuitos secuenciales. Este almacenamiento de tensiones booleanas sobre pequeños condensadores tiene el inconveniente de la descarga de los mismos a lo largo del tiempo pero resulta útil si dichos valores se actualizan o se refrescan con una frecuencia adecuada (superior a un cierto valor mínimo, $\sim 1 \text{ KHz}$).

Por otra parte, la «lógica dinámica» aplicada a las puertas pseudoNMOS, permite su funcionamiento con consumo estático nulo.

T8.1. Puertas pseudoNMOS

Cuando se necesitan puertas con un amplio número de entradas (en particular en las estructuras matriciales en que tal número suele ser muy alto) se utiliza la configuración «pseudoNMOS» y se emplean únicamente puertas "o-negada" (Nor):

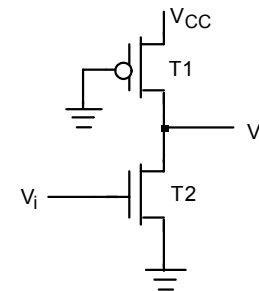
- la lógica «pseudoNMOS» conforma solamente el plano N y utiliza un solo transistor PMOS permanentemente habilitado (conectado a 0 V), que se comportará como una simple carga resistiva;
- las puertas "o-negada" (Nor) presentan sus transistores en paralelo en el plano N; no existe problema de incluir en dicho plano muchos transistores.

El calificativo «pseudoNMOS» proviene de que este tipo de puertas, con un transistor superior (que conduce siempre) como resistencia de polarización, es propio de las tecnologías NMOS: la lógica pseudoNMOS copia directamente el diseño NMOS utilizando un transistor de polarización de canal P.

La configuración "o-negada" (Nor) pseudoNMOS proporciona una alternativa eficaz para puertas de alto número de entradas y, además, reduce considerablemente el número de transistores, el área de integración y la complejidad del diseño físico; pero, en cambio presenta dos serios inconvenientes:

- los niveles de tensión de salida no quedan garantizados por la tensión umbral de los transistores
- y su consumo no es nulo, ya que para $V_0 = 0$ conducen el plano N y el transistor P.

Diseño de un inversor pseudoNMOS:



Sea $V_{CC} = 5 \text{ V}$ y $V_{TO} = 1 \text{ V}$,

$V_i < 1 \text{ V}$ T_2 no conduce y T_1 sí

$$\Rightarrow V_0 = V_{CC} = 5 \text{ V}$$

$V_i \gg 1 \text{ V}$ T_2 conduce y T_1 también

$$V_0 = V_{CC} \cdot R_2 / (R_1 + R_2)$$

siendo R_1 = resistencia PMOS

y R_2 = resistencia NMOS:

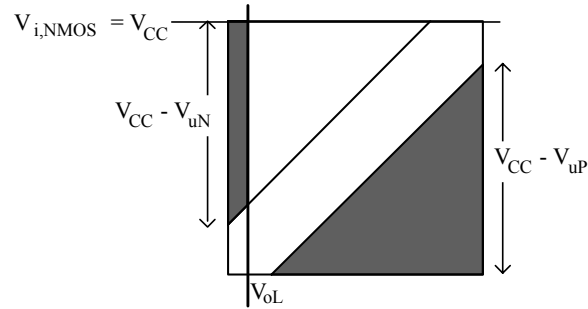
Interesa V_0 pequeño; para ello, $R_2 \ll R_1$.

En un inversor pseudoNMOS la salida 1 se produce de igual forma que en un inversor CMOS: conduce el transistor PMOS y no lo hace el NMOS; pero para salida 0 conducen ambos transistores y para asegurar una tensión de salida próxima a 0 V se requiere una «relación de geometrías» entre ambos transistores adecuada.

En el inversor CMOS cada valor booleano está garantizado por la tensión umbral del transistor que transmite dicho valor; en el pseudoNMOS hay un solo transistor activo, el NMOS, que garantiza solamente su situación de corte, salida **1**, mientras que para salida **0** aparece un divisor de tensión entre las resistencias que presentan ambos transistores y es preciso conseguir, por diseño geométrico, que la resistencia que presenta el transistor N sea muy inferior a la presentada por el P: $R_{NMOS} \ll R_{PMOS}$.

Habida cuenta de que la resistencia de paso de un transistor MOS es inversamente proporcional a su factor de forma $ff = W/L$, se requiere una relación adecuada entre los tamaños de los dos transistores que configuran el inversor, de manera que sus factores de forma verifiquen la desigualdad: $W_N/L_N \gg W_P/L_P$.

El diagrama de Memelink permite un análisis cuantitativo; para $V_o = 0$:



En situación estática por ambos transistores circula la misma intensidad

$$I_{NMOS} = I_{PMOS} \quad \alpha_N \cdot A_N = \alpha_P \cdot A_P \quad \mu_{\text{electrones}} \cdot C_{ox} \cdot W_N/L_N \cdot A_N = \mu_{\text{huecos}} \cdot C_{ox} \cdot W_P/L_P \cdot A_P$$

$$\frac{A_N}{A_P} = \frac{\alpha_P}{\alpha_N} = \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} \cdot \frac{W_P/L_P}{W_N/L_N}; \quad \frac{\mu_{\text{huecos}}}{\mu_{\text{electrones}}} = \mu'; \quad \frac{A_N}{A_P} = \mu' \cdot \frac{ff_P}{ff_N} = \mu' \cdot \beta$$

$$A_N = \mu' \cdot \beta \cdot A_P$$

$$A_P = \frac{1}{2} (V_{CC} - V_{uP})^2$$

supuesto V_{oL} pequeño

$$A_N = (V_{CC} - V_{uN}) \cdot V_{oL}$$

$$(V_{CC} - V_{uN}) V_{oL} = \mu' \cdot \beta \cdot \frac{1}{2} (V_{CC} - V_{uP})^2$$

La tensión de salida para el **0** booleano no será exactamente 0 V sino algo superior:

$$V_{oL} = \frac{(V_{CC} - V_{uP})^2}{2(V_{CC} - V_{uN})} \mu' \cdot \beta$$

y para que dicha tensión sea próxima a 0 V, interesa β pequeño: $\beta \ll 1$

$$\beta = \frac{ff_P}{ff_N} = \frac{W_P/L_P}{W_N/L_N} \ll 1$$

Para la tecnología Ψ :

$$V_{CC} = 5 \text{ V}; \quad V_{uN} = 1 \text{ V}; \quad V_{uP} = 1 \text{ V}; \quad \mu' = 1/3;$$

la tensión de salida para el **0** será:

$$V_{oL} = \frac{(V_{CC} - V_{uP})^2}{2(V_{CC} - V_{uN})} \mu' \cdot \beta = \frac{(5-1)^2}{2(5-1)} \cdot \frac{1}{3} \beta = \frac{2}{3} \beta$$

que no será exactamente 0 V, sino algo inferior a β : $0,66 \beta$

$$\beta = 1/4 = 0,25 \quad V_o = 0,17 \text{ V}$$

$$\beta = 1/5 = 0,2 \quad V_o = 0,13 \text{ V}$$

$$\beta = 1/7 = 0,15 \quad V_o = 0,1 \text{ V}.$$

Interesan valores de β bajos; además, la función de transferencia del inversor pseudoNMOS presenta mejores características de conmutación (se hace más abrupta), cuanto menor es β .

Para conseguir $\beta \ll 1$, con la menor ocupación posible de área de integración, se aumenta adecuadamente la longitud del transistor PMOS de polarización (en una puerta hay un solo transistor PMOS y muchos NMOS): para $\beta = 1/4$, $L_{PMOS} = 4 L_{mín}$.

Al utilizar lógica pseudoNMOS se pierde la característica de consumo nulo propia de la tecnología CMOS; cuando la salida es **0** conducen ambos transistores y, por tanto, circula una intensidad no nula:

$$I = K_p \cdot \frac{W}{L} \cdot A_N = K_p \cdot \frac{W}{L} \cdot (V_{CC} - V_{uN}) \cdot V_{oL} \quad \left\{ \begin{array}{l} I = K_p \cdot \frac{W}{L} \cdot \frac{(V_{CC} - V_{uP})^2}{2} \mu' \cdot \beta \\ V_{oL} = \frac{(V_{CC} - V_{uP})^2}{2(V_{CC} - V_{uN})} \mu' \cdot \beta \end{array} \right.$$

Para la tecnología Ψ :

$$K_p \text{ (NMOS)} \sim 40 \mu\text{A/V}^2; \quad W/L = 1,5$$

$$I = K_p \cdot \frac{W}{L} \cdot \frac{(V_{CC} - V_{uP})^2}{2} \mu' \cdot \beta = 160 \cdot \beta \mu\text{A}$$

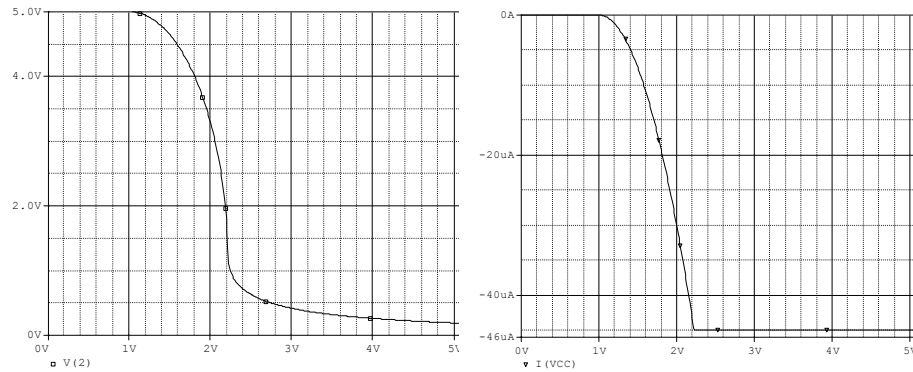
$$\text{para } \beta = 1/4 = 0,25 \quad V_o = 0,17 \text{ V} \quad I = 160 \beta = 40 \mu\text{A}$$

$$\beta = 1/5 = 0,2 \quad V_o = 0,13 \text{ V} \quad I = 160 \beta = 32 \mu\text{A}$$

$$\beta = 1/7 = 0,15 \quad V_o = 0,1 \text{ V} \quad I = 160 \beta = 23 \mu\text{A}$$

Esta intensidad es del mismo orden de magnitud que la que consume el inversor CMOS en la situación de conmutación (36 μA).

Las gráficas siguientes presentan la función de transferencia $V_o - V_i$ y el consumo de intensidad de un inversor pseudoNMOS de la tecnología Ψ , obtenidas mediante simulación SPICE; el transistor NMOS es de tamaño mínimo y la relación de geometrías es $\beta = 1/4$ $L_{NMOS} = 1 \mu m$, $W_{NMOS} = 1,5 \mu m$, $V_{TO,NMOS} = 1 V$, $K_{p,NMOS} = 40 \mu A/V^2$ $L_{PMOS} = 4 \mu m$, $W_{PMOS} = 1,5 \mu m$, $V_{TO,PMOS} = -1 V$, $K_{p,PMOS} = 15 \mu A/V^2$.



Función de transferencia $V_o - V_i$ pseudoNMOS Consumo de intensidad I_{CC} del inversor

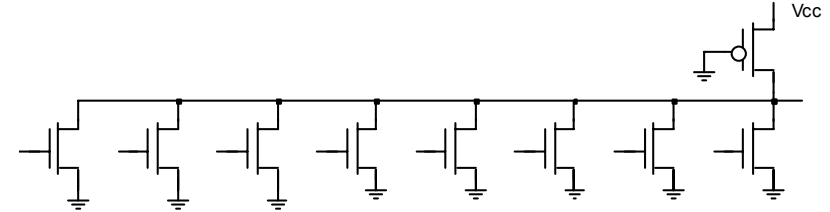
La lógica pseudoNMOS evita la necesidad de duplicar el Plano N en una combinación dual de transistores PMOS y la necesidad de conectar entre sí cada pareja de transistores; con ello se reduce considerablemente el área de integración y la complejidad del diseño geométrico. Pero, en cambio, presenta dos inconvenientes relativos, ambos, a la salida **0**:

1. en su diseño ha de tenerse en cuenta la relación de geometrías de los transistores: el cociente entre factores de forma $\beta = W_p/L_p / W_n/L_n$ para asegurar que $V_o(\mathbf{0}) \approx 0 V$;
2. el consumo no es nulo: $I_{CC}(\mathbf{0}) \neq 0$, pues conducen a la vez el plano N y el transistor P.

El primero de los inconvenientes repercute sólo en consideraciones geométricas en la etapa de diseño y la correspondiente ocupación de área.

El segundo, el consumo no nulo, no es grave si el número de estas puertas en un circuito integrado es reducido y su efecto puede disminuirse conectando la puerta del transistor PMOS a una señal de habilitación $\bar{E}$, normalmente a **1** y que pase a **0** solamente cuando interese la actuación de la correspondiente puerta lógica (caso de que el cálculo booleano que realiza dicha puerta no se requiera constantemente).

La configuración pseudoNMOS utiliza solamente puertas "**o-negada**" (Nor), ya que en ellas los transistores se encuentran en paralelo:



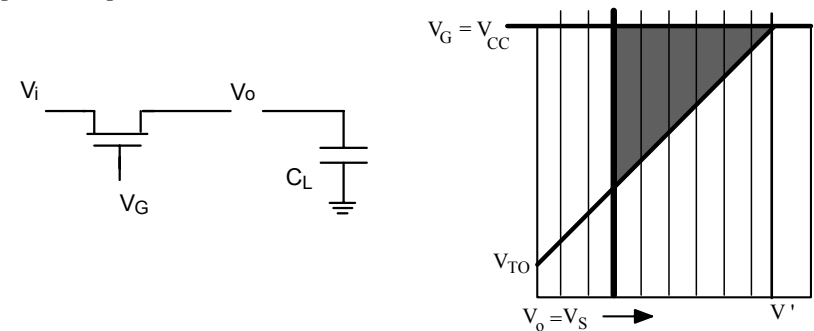
Estas puertas "**o-negada**" (Nor) pseudoNMOS resultan muy útiles para construir las estructuras reticulares de muchas entradas (decodificadores, multiplexores, demux., ROM, PLA, PAL,...); también son muy apropiadas para dispositivos programables, tipo PROM, PLA, PAL, PLS o CPLDs. Ello es debido a que:

- permiten configurar términos producto de un alto número de entradas, gracias a la conexión en paralelo de sus transistores
- y presentan un solo plano activo de forma que, en cuanto a dispositivos programables, solamente será necesario efectuar la programación sobre el plano NMOS y tal programación consistirá en «desconectar» transistores que se encuentran en paralelo.

Si se utilizan puertas CMOS, con los dos planos de transistores completos, además del problema de tener un alto número de transistores en serie, la programación de ambos planos sería sumamente compleja, pues, en aquel en que los transistores se encuentren en serie, su programación consistirá en «puentearlos» (lo cual es mucho más difícil que «desconectarlos»).

T8.2. Transistor de paso y puerta de transmisión

Otro importante complemento de las puertas booleanas CMOS, conformadas por sendos planos de transistores P y N, lo constituye el transistor de paso, equivalente a un simple interruptor:



El transistor conduce cuando $V_G = V_{CC} = \mathbf{1}$ y no lo hace para $V_G = 0 V = \mathbf{0}$.

A) Supongamos $V_G = 1 = V_{CC}$, para $V_i = V_{CC}$ el transistor conduce y configura un camino de carga hacia la capacidad de entrada de la puerta o puertas a las que está conectado (C_L).

En este proceso de carga el terminal de entrada V_i actúa como drenaje y el terminal de salida V_o como fuente; al ir aumentando $V_o = V_S$ la intensidad se hace más pequeña, finalizando el proceso de carga cuando $V_o = V_S = V'$ (es decir, cuando $V_{TO} + V'$ alcanza el valor V_G), en cuyo caso ya no existe canal y el transistor no conduce.

La tensión de salida máxima sobre C_L es V' :

$$V_{TO} + V' = V_G; \quad V' = V_G - V_{TO} \quad (4 \text{ V para } V_i = V_G = 5 \text{ V y } V_{TO} = 1 \text{ V})$$

Este desplazamiento a la baja de la tensión de salida **1** es aún mayor debido al «efecto sustrato» [ver apartado T6.3.6]:

$$V_G = V_{TO} + K \cdot V' \quad V' = (V_G - V_{TO}) / K \quad (\sim 3,5 \text{ V para } K = 1,15).$$

La salida no llega a alcanzar la tensión correspondiente al **1** booleano $V(1)$ sino que se queda por debajo. Con ello la puerta siguiente a un transistor de paso no tiene asegurado el corte del transistor PMOS; se pierde entonces la condición de consumo nulo y disminuye en gran medida el margen de ruido de dicha puerta.

Ahora bien, puede evitarse dicha situación (consumo no nulo de la puerta que sigue a un transistor de paso) si se integran transistores PMOS cuya tensión umbral V_{TO} sea mayor que la tensión umbral del transistor NMOS de paso (para $V_{TO,PMOS} = -1,8 \text{ V}$, la tensión de salida **1** del transistor de paso, $\sim 3,5 \text{ V}$, es suficiente para mantener en corte los transistores PMOS).

El desplazamiento de $V(1)$ no es acumulativo (no es un efecto resistivo sino de cierre de canal), de forma que si se conectan varios transistores de paso seguidos (en serie), la tensión de salida **1** del conjunto seguirá siendo $V' = (V_G - V_{TO}) / K$.

B) Para $V_i = 0 \text{ V}$, si la capacidad C_L se encontraba previamente cargada (tensión de salida $V(1)$), se produce su descarga hasta alcanzar 0 V : el paso del **0** booleano se efectúa sin desplazamiento de tensión.

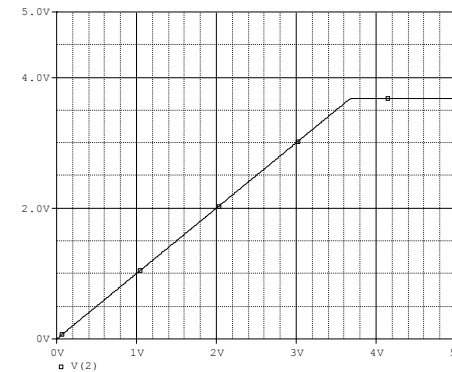
Téngase en cuenta que el transistor MOS es simétrico:

- cuando $V_i = V_{CC}$ dicho terminal actúa como drenaje y V_o como fuente,
- pero para $V_i = 0 \text{ V}$ los papeles de drenaje y fuente se intercambian ya que la intensidad en el transistor circula en sentido contrario.

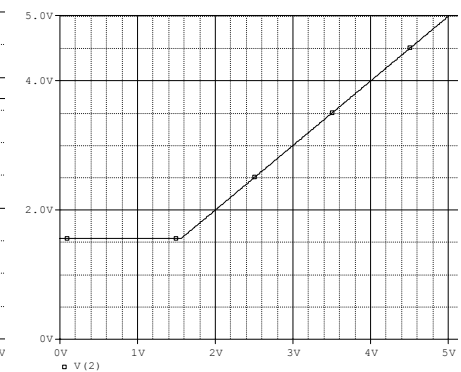
En conclusión, como transistor de paso el NMOS transmite bien el valor booleano **0** pero no el valor **1** (cuya tensión queda reducida al pasar a través del transistor); por ello, en las puertas CMOS los transistores NMOS forman el plano inferior que transmite el valor **0**. Por simetría, el transistor PMOS transmite bien el valor **1** pero no el **0** (que se desplaza hacia tensiones positivas al pasar a través del transistor); en las puertas CMOS los PMOS forman el plano superior que transmite el valor **1**.

Funciones de transferencia $V_o - V_i$ de transistores de paso NMOS y PMOS:

Tecnología: $V_{TO,NMOS} = 1 \text{ V}$, $V_{TO,PMOS} = -1 \text{ V}$, $\text{GAMMA} = 0,25$, $\text{NSUB} = 10^{15}$.

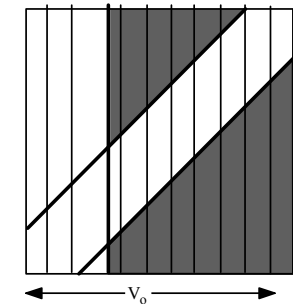
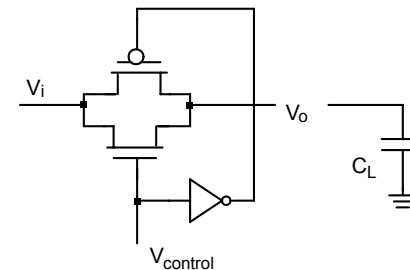


Transistor de paso NMOS



Transistor de paso PMOS

Añadiendo al transistor de paso canal N un transistor complementario canal P, en paralelo (y con las puertas de ambos conectadas a través de un inversor para que se encuentren en la misma situación de conducción o de corte), se obtiene una puerta de transmisión que evita el desplazamiento en tensión del **1** booleano.

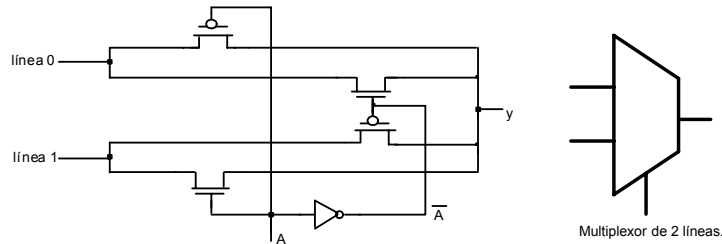


En una puerta de transmisión la salida (sobre C_L) llega a alcanzar $V(1)$ (caso de $V_i = V(1)$) pues el transistor PMOS conduce hasta ese valor y la descarga llega igualmente a 0 V (cuando $V_i = 0 \text{ V}$) a través del transistor NMOS. El transistor PMOS asegura la salida **1** sin desplazamiento de tensión y el transistor NMOS hace lo propio para la salida **0**; por contra, son precisos 4 transistores (dos en la puerta de transmisión y otros dos del inversor).

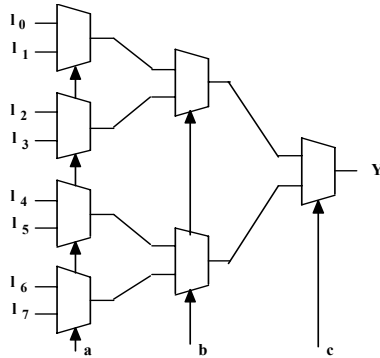
El cálculo de los tiempos de conmutación de una puerta de transmisión es análogo al desarrollado en el capítulo T7 para un inversor, de forma que los tiempos resultantes son del mismo orden e incluso inferiores a los allí obtenidos: la carga se realiza básicamente por un transistor PMOS ayudado parcialmente por el NMOS que conlleva en paralelo y viceversa (y tal colaboración parcial de ambos transistores reduce los tiempos).

Multiplexores

La puerta de transmisión tiene el inconveniente de que la variable de control ha de actuar a través de dos entradas en forma invertida (con un inversor intermedio), de manera que a los dos transistores que configuran la puerta han de agregarse otros dos para el correspondiente inversor. Pero este inconveniente no es tal en aquellas estructuras que siempre utilizan las entradas en forma afirmada y negada; en tal sentido las puertas de transmisión resulta útiles para construir multiplexores:



Un multiplexor de 2 entradas es directamente ampliable conectando módulos iguales en cascada (para n líneas serán necesarios $n-1$ módulos conectados en cascada):



El esquema anterior contiene 14 puertas de transmisión y 3 inversores, en total 34 transistores, mientras que un multiplexor análogo realizado con puertas "y-negada" (Nand) necesita 8 puertas de 4 entradas, una de 8 y 3 inversores, es decir, 86 transistores.

De esta forma, la conexión en cascada de puertas de transmisión proporciona una configuración muy simple y modular para los multiplexores, el número de transistores necesarios es claramente inferior a los que se requieren en una configuración típica CMOS (con puertas Nand o Nor) y el tiempo de propagación es algo menor.

La reducción en el número de transistores se refleja sobre el área de integración necesaria y, también, sobre los tiempos de propagación: habrá menor número de transistores en serie, con lo cual las constantes de tiempo que limitan la velocidad del circuito serán menores. Por ejemplo, en el multiplexor anterior, la transmisión de un 0 implica a 3 transistores NMOS en serie, mientras que con puertas "y-negada" (Nand) habría 8 transistores NMOS en serie en la puerta de salida.

Habida cuenta de que las puertas de transmisión son simétricas, un multiplexor construido con ellas también lo es y puede ser utilizado como demultiplexor; en tal caso, presenta el problema de que las líneas de salida no seleccionadas quedan en alta impedancia (no en salida 0, como correspondería a un demultiplexor digital), lo cual puede solventarse añadiendo resistencias que referencien a 0 V dichas líneas.

Incluso, en muchas ocasiones, es útil el estado de alta impedancia de las salidas del demultiplexor pues permite conservar sobre cada línea el último de los valores transmitidos a la misma (para lo cual será necesario que exista una capacidad adecuada conectada en cada línea de salida).

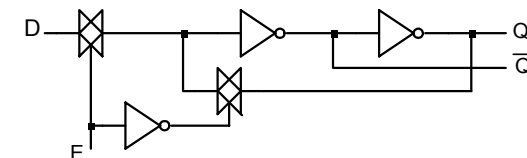
Las puertas de transmisión son, también, buenos interruptores analógicos (*analog switches*) que controlan el paso de señales en el intervalo $[0, V_{CC}]$ determinado por las tensiones de control (de puerta) de sus transistores; caso de que la señal de control utilice como niveles $-V_1$ y $+V_2$, la puerta de transmisión admitirá señales analógicas entre tales valores $[-V_1 ; +V_2]$.

De esta forma, las puertas de transmisión sirven para muestrear señales analógicas y para realizar su multiplexado o demultiplexado: un multiplexor construido con puertas de transmisión puede transmitir cualquier tensión dentro de los límites fijados por sus tensiones de control $[-V_1 ; +V_2]$; de esta forma se dispone de multiplexores analógicos que permite seleccionar por su número una de entre n tensiones de entrada (por ejemplo, para medir diversas tensiones con un mismo conversor analógico-digital).

El mismo multiplexor, utilizado en sentido inverso (una entrada, n salidas) es un demultiplexor analógico, cuyas líneas de salida adoptan el estado de alta impedancia cuando no son seleccionadas (y pueden conservar, sobre capacidades, el último valor de tensión transmitido a cada línea).

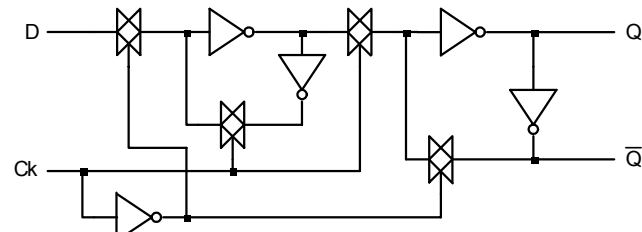
Biestables

Los biestables en tecnologías CMOS suelen construirse con puertas de transmisión:



Biestable D habilitado por niveles

Esta configuración requiere 10 transistores, mientras que la configuración típica con puertas "y-negada" (Nand) necesita 18; la reducción del número de transistores implica, también en este caso, una disminución del tiempo de propagación del biestable.



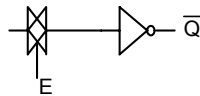
Biestable D síncrono: habilitado por flancos

Para construir un biestable D síncrono con puertas de transmisión se utilizan 18 transistores, frente a los 36 necesarios con puertas "y-negada" (Nand). Además de la reducción del número de transistores y del área de integración, también se reducen en forma importante los tiempos funcionales y, en particular, el tiempo de propagación.

T8.3. Lógica dinámica

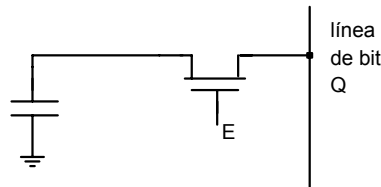
Biestables dinámicos

Los registros dinámicos utilizan biestables aún más simples que los representados en la página anterior:



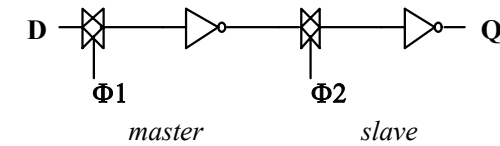
En este «biestable» la información se almacena sobre la capacidad de entrada del inversor (la cual sufre, cuando queda aislada, un proceso de descarga que limita el tiempo de almacenamiento a unos pocos milisegundos).

En muchas ocasiones, por ejemplo en las memorias dinámicas DRAM, el biestable se reduce solamente a un transistor y un condensador:

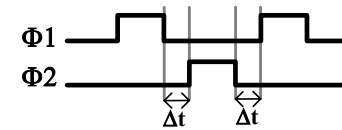


El ahorro en número de transistores (y por tanto, en área de integración) es superior a 2/3 respecto a los biestables estáticos; tal reducción de tamaño resulta muy atractiva y por ello, cuando el tiempo de permanencia de la información es reducido (inferior a 10 milisegundos) o cuando resulta fácil «refrescar» la información, se utilizan «registros dinámicos» configurados con estos biestables.

Particularmente interesante (por la reducción de tamaño) es la utilización de «biestables dinámicos» en el diseño síncrono (con reloj):



Este biestable síncrono dinámico requiere solamente 8 transistores mientras que su configuración estática (con puertas de transmisión) necesita 18 (y 36 si se configura con puertas Nand); los «biestables dinámicos síncronos» necesitan dos fases de reloj no solapadas:



Cuando $\Phi 1 = 1$ el *master* recoge la información de su entrada: «fase de captura» y en $\Phi 2 = 1$ el *slave* la presenta en su salida: «fase de salida».

La separación entre ambas fases (entre la «captura» y la «salida» de la información) viene exigida porque el más mínimo solapamiento entre ellas lleva directamente a la salida *Q* el valor de la entrada *D*. Es un requisito de seguridad de funcionamiento (véase el apartado 15.4, segundo volumen) que evita las restricciones relativas al tiempo de mantenimiento y a la verticalidad y simultaneidad en la señal de reloj; estas restricciones son particularmente fuertes en este caso, debido al reducido tiempo de propagación que presentan estos biestables tan simples.

La conexión en serie de «biestables dinámicos síncronos» da lugar a registros de desplazamiento dinámicos, con una ocupación de superficie muy reducida, y muy útiles cuando el desplazamiento se ha de producir con frecuencias de reloj superiores a 1 KHz (por debajo de esa frecuencia el tiempo de permanencia del dato en las capacidades que actúan como elementos de memoria, ~1 ms, no es suficiente).

En forma análoga es posible construir contadores dinámicos (para frecuencias de reloj superiores al KHz) sin más que añadir a las entradas *D_i* la lógica correspondiente al conteo. Por ejemplo, para un contador década con entrada de habilitación *E*:

$$D_0 = \overline{q_0} \cdot E + q_0 \cdot \overline{E} = q_0 \oplus E$$

$$D_1 = \overline{q_1} \cdot q_0 \cdot \overline{q_3} \cdot E + q_1 \cdot (\overline{q_0} + \overline{E})$$

$$D_2 = \overline{q_2} \cdot q_1 \cdot q_0 \cdot E + q_2 \cdot (\overline{q_1} + \overline{q_0} + \overline{E})$$

$$D_3 = \overline{q_3} \cdot q_2 \cdot q_1 \cdot q_0 \cdot E + q_3 \cdot (\overline{q_0} + \overline{E})$$

De igual forma, cualquier sistema secuencial síncrono puede diseñarse con biestables dinámicos; en general, tales biestables permiten una fuerte reducción en el área de integración, siempre que el reloj del sistema síncrono actúe a frecuencias no bajas.

Tiempo de permanencia de un dato en un registro dinámico

En los registros dinámicos (conformados por una puerta de transmisión y un inversor) el dato es almacenado en la capacidad de entrada del inversor (capacidad de puerta de los dos transistores que lo conforman); cuando la puerta de transmisión es deshabilitada la conservación de carga en dicha capacidad asegura la permanencia del valor booleano en el inversor durante un cierto tiempo.

Ahora bien, en la situación de deshabilitación las propias uniones difusión-sustrato de la puerta de transmisión configuran un camino de descarga para la capacidad de puerta que mantiene el valor booleano; éstas uniones se encontrarán en polarización inversa, lo cual supone una intensidad de corriente sumamente pequeña pero no nula. Al cabo del tiempo dicha capacidad se descargará y el valor booleano se perderá: precisamente por ello estos registros son dinámicos, necesitan ser refrescados cada cierto tiempo (presentan una frecuencia mínima de trabajo).

Podemos definir el tiempo de permanencia como aquel en que la variación de tensión de la capacidad de puerta es inferior a la tensión umbral de los transistores V_{TO} , ya que a partir de ese valor de tensión la intensidad estática por el inversor deja de ser nula (se pierde la condición de consumo estático nulo).

Una estimación del orden de magnitud puede ser la siguiente:

intensidad de descarga ~ 1 pA

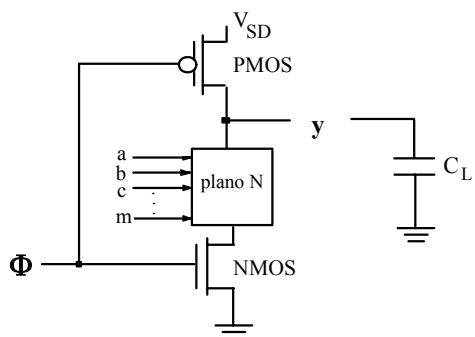
capacidad de puerta ~ 1 fF

$$\Delta V = V_{TO} \sim 1 \text{ V} \quad t = C \cdot \Delta V / I \approx 10^{-15} / 10^{-12} = 10^{-3} \text{ s} = 1 \text{ ms.}$$

Lógicas pseudoNMOS dinámicas.

Para evitar el problema del consumo estático no nulo en las estructuras pseudoNMOS, se utiliza una «lógica dinámica» con reloj, en la cual se distinguen dos fases en cada unidad de tiempo (pulso de reloj):

- fase de precarga (de la salida de la puerta) $\Phi = 0$
- fase de evaluación (de la función que realiza) $\Phi = 1$



Se añade a la puerta un transistor auxiliar NMOS y los dos transistores extremos, el PMOS de polarización y el NMOS auxiliar, se conectan al reloj Φ de forma que, en ninguna de las dos fases de Φ , conducen a la vez ambos transistores: el consumo de la puerta es nulo.

En cada pulso de reloj, la salida es llevada inicialmente a 1:

```

precarga       $\Phi = 0$ ,   conduce el transistor PMOS,   salida y = 1

```

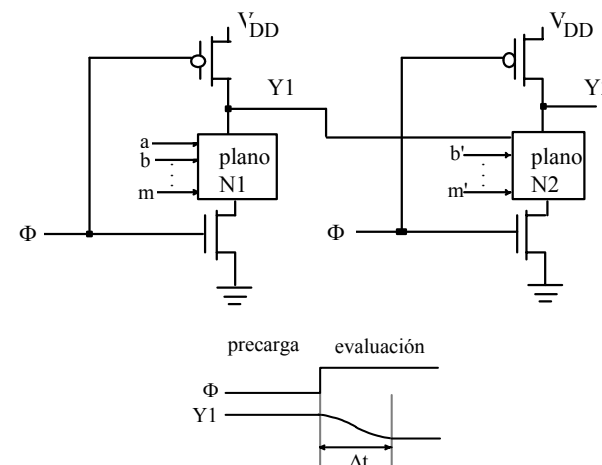
y, posteriormente, el plano N calcula la función:

evaluación $\Phi = 1$, caso de valor **0** descarga la salida, salida $y = f(X)$.

Durante la fase de precarga la salida es llevada a valor booleano **1** a través del transistor PMOS de carga (que conduce cuando $\Phi = 0$). En la fase de evaluación, si la salida (conforme a la función que calcula la puerta) debe ser **0**, tal valor es comunicado a través del plano N y del transistor NMOS auxiliar; pero cuando deba ser **1**, dicho valor ha de conservarse (sobre la capacidad de entrada de la puerta siguiente) a partir de la tensión almacenada en la precarga.

La duración de la fase de evaluación no puede ser indefinida, pues existirá un proceso de descarga que deteriorará el valor booleano almacenado: estas puertas son «dinámicas» y han de actuar por encima de una cierta frecuencia de reloj. Serán puertas de tipo "**o-negada**" (*Nor*) para admitir un amplio número de entradas (que es la aplicación propia de las puertas pseudoN MOS).

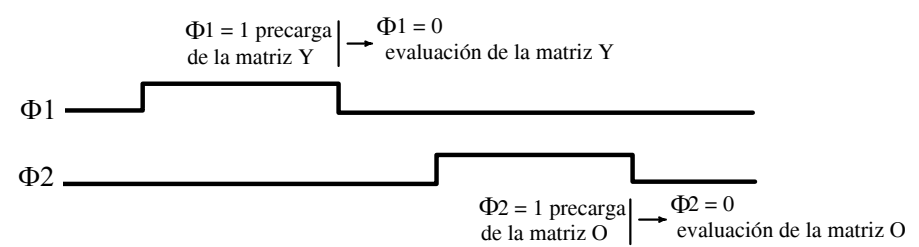
Además, este tipo de puertas (seudoNMOS dinámicas) no pueden conectarse directamente entre sí, pues es preciso que los correspondientes valores de sus entradas (en particular, las entradas cuyo valor sea 0) estén presentes antes de iniciarse la fase de evaluación.



Durante la fase de precarga ambas salidas se ponen a **1**; al iniciarse la evaluación la salida Y1 tardará un cierto tiempo Δt en pasar a **0** (supuesto que tal sea su valor), durante el cual (Δt) Y1 provoca la descarga de Y2 (puertas *Nor*, transistores N en paralelo), de forma que, si Δt es suficientemente amplio para que Y2 deje de valer **1**, ya no podrá adoptar tal valor booleano (Δt anula el efecto de la precarga).

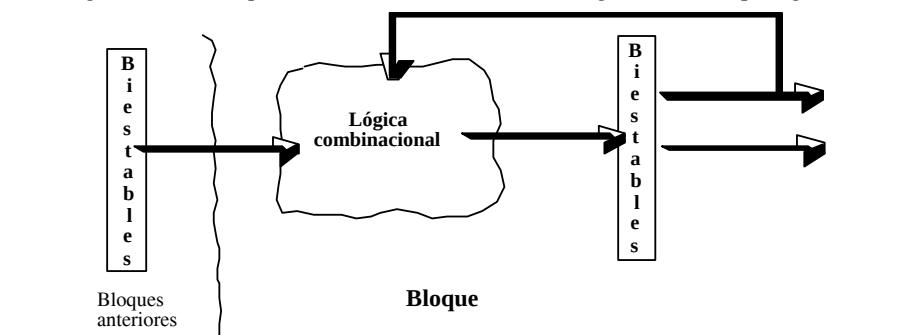
Por ello, estas puertas pseudoNMOS dinámicas son de utilización individual. Para conseguir la conectividad directa entre puertas pseudoNMOS dinámicas se han diseñado soluciones sumamente «complejas», «ingeniosas» y variadas: lógicas con 4 fases de reloj, lógicas dominó, C²MOS, CSVL,..., pero tales soluciones son más «académicas» que efectivas.

En las estructuras matriciales tipo ROM, PLA, PAL, ..., que utilizan dos conjuntos sucesivos de puertas (matriz Y - matriz O), es posible conectar tales puertas mediante el empleo de dos fases de reloj Φ_1 , Φ_2 no solapadas:



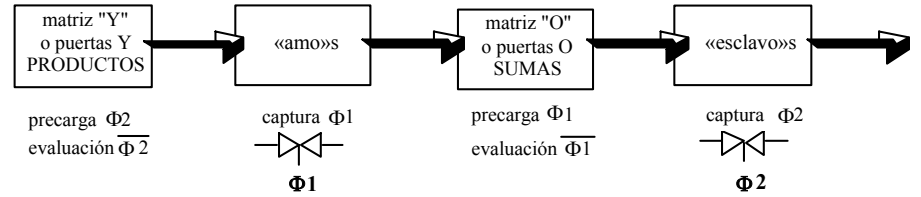
De esta forma, el primer conjunto de puertas (matriz Y) evalúa sus funciones booleanas mucho antes de iniciarse la evaluación del segundo conjunto (matriz O).

En general, los bloques constitutivos de un sistema digital son del tipo siguiente



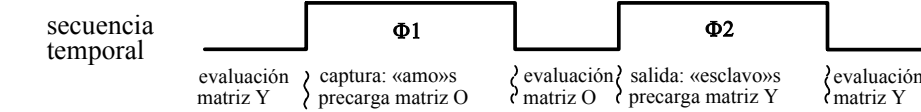
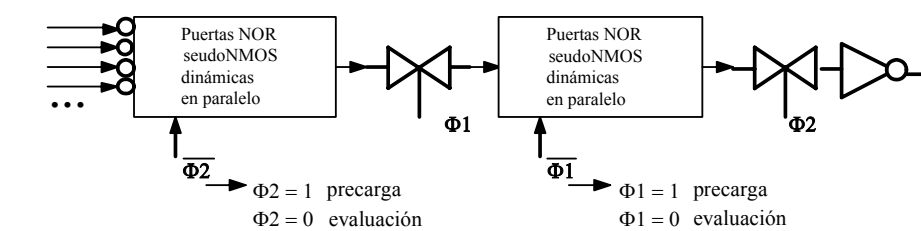
es decir, la lógica combinacional se encuentra entre biestables.

Las funciones booleanas «intermedias» (la lógica combinacional) pueden ser construidas en forma matricial (suma de productos = PAL) y los biestables (los de salida del propio bloque) pueden ser descompuestos en sus dos partes «amo-esclavo» e intercaladas en la lógica:



Cada semibiestable (*master/slave*) puede ser construido, en configuración dinámica, mediante una simple puerta de transmisión: el valor booleano queda almacenado en la capacidad de entrada de la puerta siguiente (o puertas siguientes).

Habida cuenta de que tanto la matriz Y como la matriz O han de ser realizadas mediante puertas «o-negada» (*Nor*), el bloque considerado quedaría en la forma siguiente:



T9 EL CIRCUITO INTEGRADO COMPLETO: CUESTIONES DIVERSAS

T9.1. Terminales del circuito integrado: entradas y salidas

T9.2. Densidad de integración

T9.3. Consideraciones respecto al diseño físico: reglas de diseño

Existe una serie de cuestiones a tener en cuenta respecto al circuito integrado completo, que no han sido consideradas en los capítulos anteriores dedicados expresamente a modelar el comportamiento de los transistores MOS y de las puertas booleanas.

*En primer lugar, además de las funciones booleanas que configuran el circuito, éste precisa comunicarse con el exterior por medio de entradas, salidas y líneas de alimentación (por las cuales recibe la energía necesaria para su funcionamiento). Esta comunicación se establece a través de los terminales (*pins* o *patillas*) del encapsulado del circuito integrado que conectan, mediante pequeños hilos metálicos soldados, con los PADs (cuadrados de aluminio) del «dado» de silicio (*chip*) resultante de la integración; internamente, se requieren adaptadores de entrada, de salida o simples líneas metálicas de conexión (en el caso de las alimentaciones).*

*Los adaptadores de entradas sirven para proteger, mediante recortadores y filtros, a las entradas del circuito booleano; en algunos casos es necesario que tales entradas tengan histéresis (entradas *Schmitt*). Las salidas precisan de adaptadores capaces de suministrar intensidades relativamente altas y, en ocasiones, capacidad de alta impedancia (tri-estado) o de bidireccionalidad (posibilidad de actuar como salida y como entrada).*

Se estudia, también, en este capítulo la relación entre las dimensiones de la tecnología y la densidad de integración (número de puertas o de transistores por mm^2) y, también, con la velocidad de trabajo (relación que se establece a través de la capacidad de puerta de los transistores). Ambos parámetros, densidad de integración y velocidad de trabajo, aumentan cuadráticamente al disminuir la anchura de las líneas de polisilicio ($L_{\min}$ = longitud mínima del canal, dimensión que caracteriza el proceso de integración).

*Asimismo se introducen y justifican las «reglas de diseño» (reglas que se refieren a las dimensiones físicas o gráficas) que expresan las limitaciones de la tecnología en cuanto a anchuras mínimas, separaciones, prolongaciones, solapamientos,... Tales reglas han de ser tenidas en cuenta en el diseño de las puertas a nivel físico, es decir, en el dibujo de las regiones semiconductoras y de las conexiones entre ellas, en el *layout* o «máscaras» que son el resultado último del diseño y el molde primigenio de la fabricación.*

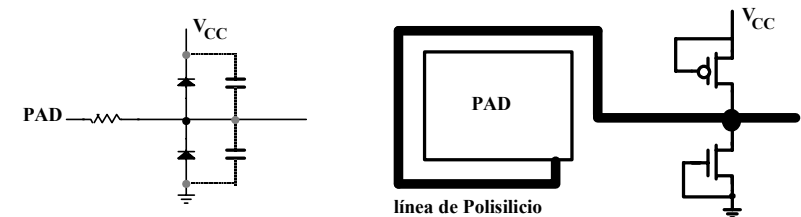
*Como parte de las reglas de diseño (y como requisito de particular importancia en el mismo) se reitera la necesidad de polarizar los substratos y el problema del *latch-up* que obliga a polarizarlos «intensivamente».*

T9.1. Terminales del circuito integrado: entradas y salidas

El óxido de pasivación del circuito integrado recubre todo el circuito (todo el *chip*) salvo una serie de cuadrados de metal periféricos (PADs) previstos para realizar la conexión con las patillas o terminales del circuito integrado encapsulado (*pins*); la conexión se realiza a través de finos hilos soldados por un extremo a las patillas y por el otro a dichos cuadrados de metal (PADs), que conectan internamente con las entradas y salidas del circuito integrado.

Entradas

Las entradas exteriores son recibidas en el circuito integrado a través de un circuito de protección frente a posibles picos de tensión, transitorios bruscos o carga estática; dicho circuito suele estar formado por una resistencia de entrada, dos diodos en polarización inversa (conectados a las dos líneas de polarización V_{CC} y 0 V) y dos pequeños condensadores en paralelo con los diodos. Los diodos recortan los picos de tensión que desbordan la alimentación y la componente capacitiva RC realiza un ligero filtrado frente a transitorios bruscos.



La resistencia se construye mediante una larga pista de polisilicio que habitualmente rodea al PAD.

Los diodos se forman aprovechando la unión PN entre drenaje y substrato de dos transistores NMOS y PMOS (con sus terminales de puerta y fuente en cortocircuito). Tales transistores se construyen relativamente gruesos (área de drenaje amplia), pues su unión drenaje-substrato pasa a polarización directa cuando la tensión de entrada se encuentra fuera del intervalo de alimentación [0 V ; V_{CC}] y, en tal situación, deben soportar intensidades relativamente altas; asimismo, se rodean de un «anillo de guarda» para asegurar la polarización del substrato y, también, para evitar fenómenos de *latch-up* (descritos en T2.1., primer volumen).

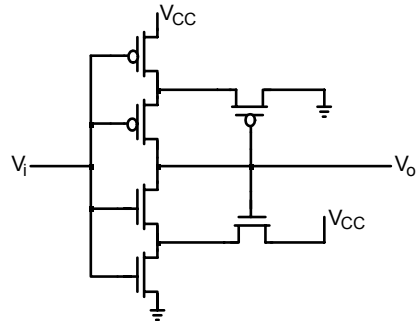
La componente capacitiva es aportada por las propias uniones PN de ambos diodos, polarizadas en inverso, y por la capacidad drenaje-puerta de los transistores.

La intensidad máxima que soportan las uniones drenaje-substrato, actuando como diodos de protección, se indica en los catálogos como I_K (*input clamp current*, intensidad de enclavamiento).

Otros tipos de entradas

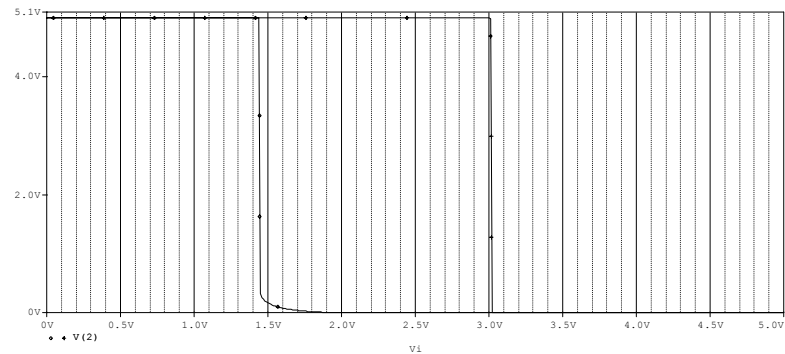
En muchas ocasiones resultan de interés las entradas con histéresis, «entrada Schmitt», que presentan dos tensiones de conmutación V_a y V_b ; tales entradas son útiles para aumentar la verticalidad de los flancos de la señal de entrada y para evitar «rebotes» cuando la entrada se encuentra en el entorno de la tensión de conmutación.

La histéresis siempre requiere realimentación positiva; en tecnología CMOS se consigue una entrada inversora con histéresis mediante tres parejas de transistores, según la figura siguiente: sendos transistores duplicados en serie, que reciben la entrada, y una tercera pareja de transistores que realimentan la salida del inversor.



Los transistores «horizontales», conectados a la tensión de salida, introducen la realimentación que da lugar a la histéresis: con salida 1 los dos NMOS inferiores forman un divisor de tensión cuyo efecto es desplazar hacia arriba la tensión de conmutación, mientras que para salida 1 el divisor de tensión de los dos PMOS superiores desplaza hacia abajo la otra tensión de conmutación.

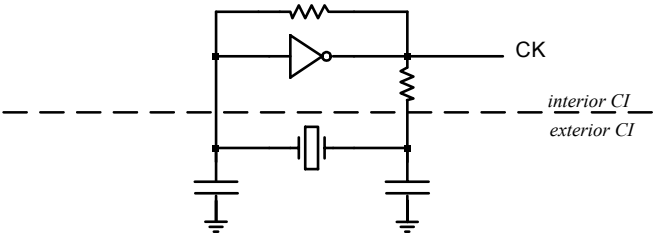
Función de transferencia de un inversor con histéresis (entrada Schmitt):
Tecnología Ψ , con transistores de tamaño mínimo:
 $L = 1 \mu\text{m}$, $W = 1,5 \mu\text{m}$, $|V_{TO}| = 1 \text{ V}$, $K_{p,\text{NMOS}} = 40 \mu\text{A/V}^2$ y $K_{p,\text{PMOS}} = 15 \mu\text{A/V}^2$.



Transición de 0 a 5 V: $V_a = 1,45 \text{ V}$

Transición de 5 a 0 V: $V_b = 3,05 \text{ V}$

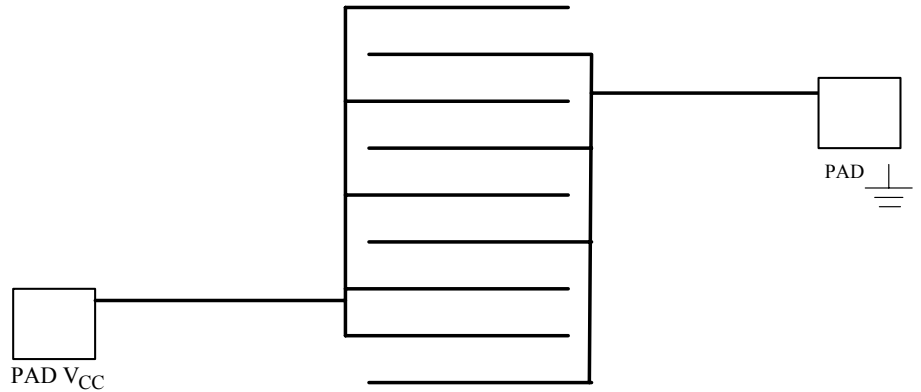
La integración de un simple inversor conectado a sendos terminales exteriores y con una resistencia de polarización que lo sitúe en la zona de conmutación ($V_i = V_o$) configura un amplificador inversor; realimentándolo externamente con un cristal de cuarzo o resonador piezoeléctrico se obtiene un oscilador en onda cuadrada, que puede ser utilizado como reloj del sistema digital o como frecuencia fija de referencia (véase el apartado dedicado a osciladores con cristal de cuarzo, T4.5., segundo volumen).



Alimentaciones

Las líneas de alimentación (V_{CC} y 0 V) han de ser conectadas al circuito integrado a través de entradas especiales que pasan directamente del correspondiente cuadrado metalizado de conexión (PAD) a líneas de aluminio. La alimentación se distribuye a las diversas puertas y biestables a través de líneas de metal, relativamente gruesas, a fin de que su resistencia sea mínima (y, con ella, las pérdidas de tensión); dichas líneas se hacen de la menor longitud posible para reducir tanto la resistencia como la autoinducción que presentan, la cual afecta a la producción de ruido en las conmutaciones.

En la conexión de la alimentación con las puertas y biestables, resulta útil una distribución alternada de ambas líneas V_{CC} y 0 V, en forma de «peine»:



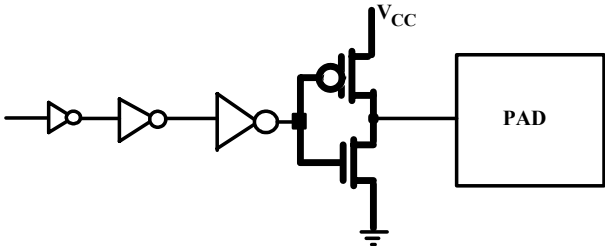
La anterior configuración de las líneas de alimentación «en peine» hace que las puertas deban alternarse en orientación invertida («cabeza arriba» / «cabeza abajo», es decir, con la tensión V_{CC} arriba / abajo).

En circuitos integrados complejos se incluyen varias entradas de alimentación, a fin de reducir el «ruido» debido a la conmutación simultánea de múltiples salidas (que será comentado un poco más adelante al tratar de los adaptadores de salida).

Salidas

Las salidas del circuito integrado se comunican al exterior a través de inversores con transistores muy anchos (W grande) para que puedan proporcionar intensidades de varios miliamperios (un centenar de veces más altas que las utilizadas en el interior del CI). Asimismo debe tenerse en cuenta que el metal que conduzca tales intensidades (desde la alimentación a los transistores y de éstos a las salidas) ha de ser convenientemente ancho: ~ 1µm / 1mA para el aluminio en tecnologías actuales.

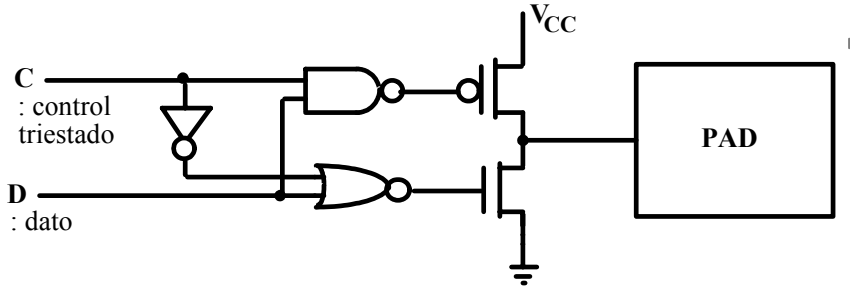
Para evitar los correspondientes retrasos en la propagación de las señales (debidos a capacidades de carga CL muy altas, tanto las capacidades exteriores como las propias de la entrada CG de los gruesos transistores de salida) ha de añadirse una serie de inversores en adecuada progresión de anchura W: parejas de «inversores en escalera» (ver T7.3.). Resulta conveniente rodear los transistores de salida con sendos «anillos de guarda» que eviten los fenómenos de latch-up, habida cuenta de que tales transistores pueden conducir intensidades altas.



Intensidad disponible en la salida para la tecnología Ψ:

NMOS	PMOS
$R_o(0) \sim 4\text{ K}\Omega$	$R_o(1) \sim 11\text{ K}\Omega$
Intensidad suministrable por el inversor: $I_o = \Delta V_o / R_o$	
Admitiendo $\Delta V(0) = 0,2\text{ V}$ y $\Delta V(1) = 0,6\text{ V}$	
$I_o(0) \sim 0,2 / 4\text{K} \approx 50\text{ }\mu\text{A}$	$I_o(1) \sim 0,6 / 11\text{K} \approx 50\text{ }\mu\text{A}.$
Para obtener $I_o \sim 10\text{ mA}$ es necesario aumentar el factor de forma $10 / 0,05 = 200$ veces y, para ello, hacer los transistores 200 veces más anchos: $W = 200\text{ W}_{\text{mín.}}$	
Con $W = 300\text{ }\mu\text{m}$ y $L = 1\text{ }\mu\text{m}$, $ff = 300$	
$R_o(0) \sim 20\text{ }\Omega$	$R_o(1) \sim 60\text{ }\Omega$
$I_o(0) \sim 0,2 / 20 \approx 10\text{ mA}$	$I_o(1) \sim 0,6 / 60 \approx 10\text{ mA}.$

Si la salida debe contar con la posibilidad de alta impedancia (tri-estado) se configura mediante la siguiente lógica:



C = 0	PMOS no conduce	NMOS tampoco
C = 1 D = 1	PMOS conduce	NMOS no
	habilitación PMOS = $\overline{C} \cdot D$	
C = 1 D = 0	PMOS no conduce	NMOS si
	habilitación NMOS = $C \cdot \overline{D} = \overline{\overline{C} + D}$	

Para configurar un terminal bidireccional se incluyen sendos adaptadores, uno de entrada y otro de salida tri-estado.

En ocasiones, se añade a la salida (sea ésta de tipo normal o tri-estado) una resistencia de valor alto conectada a 0 V (resistencia de *push-up*) o a VCC (*pull-down*) para asegurar que en la inicialización (o en situaciones de alta impedancia) tales salidas quedan referenciadas a 0 o a 1, respectivamente.

Asimismo, puede interesar prescindir del transistor PMOS en la salida, integrando solamente el NMOS: salidas de «drenaje abierto», análogas a las de «colector abierto», que requieren una resistencia exterior para conectar con la alimentación VCC y poder formar el valor booleano 1 de salida; dimensionando adecuadamente el transistor NMOS, estas salidas admiten tensiones e intensidades superiores a las de las salidas normales. Las salidas de «drenaje abierto» permiten lógica "y" cableada: la conexión de varias salidas de este tipo sobre una misma resistencia conectada a la alimentación VCC da lugar a una operación "y" entre dichas salidas.

Ruido debido a la conmutación de las salidas

En la conmutación de las puertas booleanas se producen variaciones en la intensidad, las cuales son considerablemente grandes en los adaptadores de intensidad (*buffers*) correspondientes a las salidas del circuito integrado. Tales variaciones no sólo se deben a los cambios de intensidad en las salidas sino también a los efectos capacitivos y transitorios de conmutación (conducción simultánea de PMOS y NMOS). Los cambios de valor de la intensidad relativos a tales adaptadores son importantes, habida cuenta de que la anchura de sus transistores W es grande y, además, son sumamente rápidos.

Estos cambios de intensidad originan picos de tensión (ruido) sobre las líneas de alimentación, que pueden producir errores en el funcionamiento del circuito; tales variaciones de tensión se producen por efecto inductivo en relación con la autoinducción L que toda línea de conexión presenta. Las tensiones inducidas dependen del valor de la autoinducción y de la magnitud de la variación de intensidad; más en concreto, de la pendiente que presenta el cambio de intensidad: $\Delta V = -L \, dI/dt$.

La autoinducción en las líneas de alimentación es particularmente significativa en el hilo de conexión entre los terminales del encapsulado (*pines*) y el propio circuito integrado y su efecto inductivo es importante porque las variaciones de intensidad en la conmutación de las salidas, además de ser relativamente altas, presentan pendientes fuertes ($\Delta t \sim 1 \text{ ns} = 10^{-9} \text{ s}$, que supone multiplicar por 10^9 la variación ΔI de intensidad).

Resulta necesario prestar atención a las salidas que conmutan o que pueden hacerlo a la vez, pues sus efectos inductivos se suman: conmutación simultánea de múltiples salidas. En tal caso conviene establecer varios terminales de alimentación (varios *pines*) en el circuito integrado y distribuir adecuadamente la alimentación, de forma que salidas que conmuten a la vez reciban su alimentación desde terminales diferentes.

T9.2. Densidad de integración

Densidad de integración en el núcleo booleano (*core*)

La ocupación de la superficie del silicio viene determinada por

- las dimensiones de los transistores MOS en su zona de puerta (L , W)
- las dimensiones de las zonas de difusión (fuente y drenaje)
- las conexiones entre transistores y de éstos con la alimentación
- las separaciones obligadas entre ellos y entre sus conexiones.

Además, los transistores PMOS requieren un sustrato N (pozo) que impone mayores separaciones: separación entre el límite del pozo y difusiones P (de los transistores PMOS) internas al mismo y separación entre pozo y difusiones N (de los NMOS) fuera del pozo.

Las «reglas de diseño» (ver apartado siguiente) establecen un puente entre la tecnología y el diseño, sintetizando en forma resumida y parametrizada las posibilidades y las limitaciones de la tecnología. Tales «reglas» señalan las anchuras, separaciones y solapamientos mínimos que aseguran una correcta fabricación con una tecnología concreta, expresando tales dimensiones mínimas en números enteros a partir de un parámetro λ tomado como unidad de medida.

La dimensión mínima de una tecnología corresponde a la anchura del polisilicio, o lo que es lo mismo a la longitud mínima de los transistores $L_{\min}$, y la anchura mínima de las difusiones, o sea, la anchura mínima de los transistores $W_{\min}$, suele ser vez y media la longitud mínima $L_{\min}$: $W_{\min} / L_{\min} = 3/2$.

L = anchura del polisilicio = longitud del transistor

$L_{\min}$ = dimensión mínima del proceso
(por ello $L_{\min}$ identifica a la tecnología)

W = anchura de la difusión = anchura del transistor

Para utilizar números enteros en las medidas se adopta como unidad el parámetro λ equivalente a la mitad de la longitud mínima: $\lambda = L_{\min} / 2$ y, de esta forma, todas las dimensiones resultan múltiplos enteros de λ :

en transistores mínimos, $L = 2\lambda$ $W = 3\lambda$.

Tecnología de 1μ : $L = 1\mu\text{m}$ $W = 1,5\mu\text{m}$ $\lambda = 0,5\mu\text{m}$

El área de la zona activa de un transistor mínimo (zona de puerta) será: $W \cdot L = 6\lambda^2$; su capacidad de puerta será $C_{ox} \cdot L \cdot W = C_{ox} \cdot 6\lambda^2$. Para $C_{ox} = 0,7 \text{ fF}/\mu\text{m}^2$ (que corresponde a un espesor de óxido de puerta $t_{ox} = 50 \text{ nm}$):

$$C_{ox} \cdot W \cdot L = 0,7 \cdot 6\lambda^2 \approx 4\lambda^2 \text{ fF}$$

y como cada entrada se encuentra conectada a dos transistores, uno de canal N y otro de canal P, la capacidad de entrada será:

$$C_i \approx 8\lambda^2 \text{ fF}.$$

Para la tecnología Ψ :	$L_{\min} = 1\mu\text{m}$, $W_{\min} = 1,5\mu\text{m}$
$\lambda = 0,5\mu\text{m}$, $C_{ox} = 0,7 \text{ fF}/\mu\text{m}^2$	
para transistores mínimos:	$C_G = 6\lambda^2 \cdot C_{ox} \approx 1 \text{ fF}$
	$C_i = 2 C_G \approx 2 \text{ fF}.$

Dicha capacidad de entrada es un buen indicador de la velocidad de trabajo de las puertas booleanas, pues los tiempos de conmutación son proporcionales a la capacidad de carga: $t_p \sim 5C/I_{\text{sat}}$; C depende cuadráticamente de las dimensiones de la tecnología (L , W) mientras que I_{sat} depende del factor de forma ($ff = L/W = 1,5$) y no de las dimensiones.

La siguiente tabla expresa el área de los transistores mínimos (en cuanto a su zona de puerta) y la capacidad de una entrada (con dichos transistores mínimos), en función de las dimensiones de la tecnología:

Tecnología	λ	área T mín.	C_i
2 μm	1	6 μm^2	8 fF
1 μm	0,5	1,5 μm^2	2 fF
0,8 μm	0,4	1 μm^2	1,5 fF
0,5 μm	0,25	0,4 μm^2	0,5 fF
0,2 μm	0,1	0,06 μm^2	0,1fF

Al disminuir la dimensión lineal de los transistores, su área y su capacidad disminuyen cuadráticamente y, en igual medida, aumenta la densidad de integración y aumenta su velocidad de trabajo (disminuyen los tiempos de propagación).

De ahí el gran interés en disminuir la longitud mínima de la tecnología: una pequeña disminución de un d % en $L_{\text{mín}}$ se traduce en un aumento cercano al $2d$ % en la densidad de integración y en la velocidad de trabajo alcanzable:

$$[\lambda \cdot (1 - d/100)]^2 = \lambda^2 \cdot (1 - 2d/100 + d^2/10000) \approx \lambda^2 \cdot (1 - 2d/100) .$$

Para estimar la ocupación de área efectiva, teniendo en cuenta las conexiones y separaciones así como la estructura de las puertas booleanas (que implican existencia del pozo, sustrato N de los transistores P, y de las zonas de fuente y drenaje), consideremos la densidad de integración en términos de puertas “**y-negada**” (*Nand*) de 2 entradas:

- a partir de «reglas de diseño» típicas (ver siguiente apartado) puede estimarse que una puerta *Nand* de 2 entradas ocupa, al menos, un rectángulo cuyos lados son del orden de 30 a 40 λ y cuya área será, por tanto, de unas $1000\lambda^2$, mientras que sus cuatro transistores, en cuanto a zona de puerta (cruce de polisilicio-difusión que conforma la zona activa del transistor), ocupan solamente $4 \times 6\lambda^2 = 24\lambda^2$;
- existe, pues, un factor del orden de 40 entre la superficie ocupada por los transistores (en sus zonas de puertas) y la superficie total necesaria para configurar el circuito (es decir, más del 95 % de la superficie se destina a conexiones, separaciones y conformación del circuito, ocupando los transistores, en cuanto a su zona activa, poco más del 2 %);
- en tal supuesto, la densidad de integración en términos de puertas *Nand* de 2 entradas será: $(1 \text{ mm}^2) / (1000 \lambda^2 \mu\text{m}^2) = 10^6 / 10^3 \lambda^2 = 1000 / \lambda^2$;
- en término de transistores la densidad será cuatro veces superior.

La tabla siguiente expresa la densidad de integración en términos de puertas *Nand* (puertas por mm^2) y en número de transistores por mm^2 , en función de las dimensiones de la tecnología:

Tecnología	λ	d_I (puertas <i>Nand</i> 2) puertas / mm^2	d_I (Transistores) transistores / mm^2
2 μm	1	1000	4000 = $4 \cdot 10^3$
1 μm	0,5	4000	16000 = $16 \cdot 10^3$
0,8 μm	0,4	6000	24000 = $24 \cdot 10^3$
0,5 μm	0,25	16000	64000 = $64 \cdot 10^3$
0,2 μm	0,1	100000	400000 = $0,4 \cdot 10^6$

Tecnologías de múltiples metales y de varios polisilicios permiten aumentar aún más la densidad de integración, al reducir el área necesaria para las conexiones.

Quede claro que estas densidades de integración se refieren al interior del circuito integrado o zona booleanamente activa, en la que se configuran las funciones y los subsistemas booleanos: núcleo booleano o *core*; los transistores de dicha zona suelen ser de tamaño mínimo.

Área necesaria para los elementos de contorno

Además de la superficie «booleanamente activa» (*core*), todo circuito integrado requiere una amplia superficie periférica (contorno) para los adaptadores de entrada y salida que permiten conectar el circuito con el exterior, así como para poder separar el circuito de los adyacentes en la oblea de integración.

A partir del borde extremo del circuito integrado (*chip*) es preciso reservar una franja todo alrededor para poder cortar e individualizar los diversos circuitos formados en la misma oblea: línea de corte (*scribe line*). Junto a esta línea de corte, rodeando también a todo el circuito, se incluye en la integración un anillo de metal (rectangular o cuadrado según la forma del *chip*) muy ancho, que proporciona un apantallamiento lateral al circuito: anillo de guarda.

Dentro del anillo de guarda y distribuidos a lo largo de su contorno han de ir los puntos de conexión externa, terminales del circuito integrado (*PADs*: entradas, salidas y alimentación), que permitan conectarlo, a través de hilos unidos por soldaduras, a los terminales (*pines*) de la carcasa que conforma la pastilla integrada definitiva; dichos puntos de soldadura (*PADs*) suelen ser cuadrados de unos 100 μm de lado y han de tener una mínima separación, tanto del anillo de guarda como de la superficie activa del silicio.

Además, las entradas y salidas deben llevar unos circuitos de adaptación (*buffers*):

- en las entradas tales adaptadores se reducen a una simple red Resistencia-Diodos (línea prolongada de polisilicio y transistores utilizados como diodos en polarización inversa),

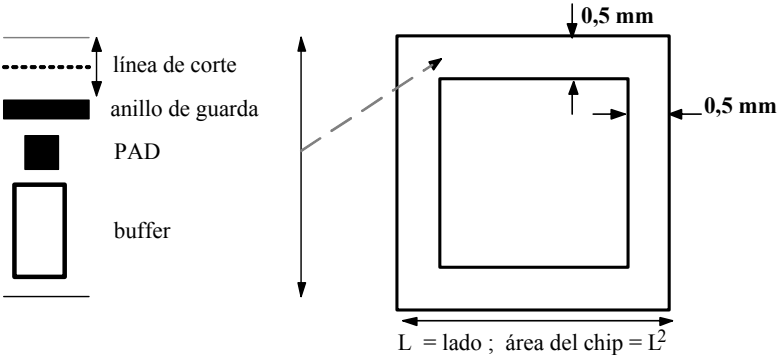
- pero en el caso de las salidas son precisos amplificadores de intensidad con amplios transistores que proporcionen intensidades de varios miliamperios e inversores «en escalera» para evitar altos tiempos de propagación; los adaptadores de salida se complican, aún más, en los casos de terminales bidireccionales (entrada-salida) y tri-estado (tercer estado de desconexión por alta impedancia).

Para apreciar la ocupación de área debida a todos estos elementos periféricos, podemos utilizar los siguientes valores estimativos de la anchura de los diversos elementos de contorno:

- línea de corte [~ 200 μm = 0,2 mm]
que al ser compartida con los circuitos contiguos, solamente la mitad repercute en cada circuito: ~ 100 μm
 - anillo de guarda ~ 50 μm = 0,05 mm
 - PADs: la anchura de la zona metálica para soldar el hilo de conexión a los *pines* suele ser [~ 100 x 100 μm]
- pero, además, ha de haber varias separaciones:
- separación anillo de guarda / PAD ~ 50 μm
 - anchura del PAD ~ 100 μm
 - separación de la zona activa ~ 50 μm
 - adaptadores (*buffers*): el peor caso corresponde a los adaptadores tri-estado y bidireccionales ~ 150 μm

Anchura total : ~ 100 + 50 + 50 + 100 + 50 + 150 μm ≈ 0,5 mm

Los elementos de contorno ocupan una franja de aproximadamente medio milímetro a lo largo de todo el contorno del circuito integrado.



El área de este contorno será la correspondiente a 4 trapecios cuyas bases difieren en 1 mm y cuya altura es de 0,5 mm:

$$\text{área} = 4 \cdot \frac{L + L - 1}{2} \cdot 0,5 = 2L - 1 \text{ mm}^2$$

siendo L la medida (en mm) del lado del circuito integrado (*chip*); el «aprovechamiento» de la superficie de silicio, en cuanto a zona booleana (*core*) será:

$$\text{superficie útil} = L^2 - (2L - 1) = L^2 - 2L + 1 = (L - 1)^2$$

$$\frac{\text{superficie útil}}{\text{superficie total}} \cdot 100 = \frac{(L - 1)^2}{L^2} \cdot 100 = 100 \cdot \left[1 - \frac{1}{L} \right]^2 \%$$

con L expresado en milímetros.

La tabla siguiente expresa el área de contorno y el aprovechamiento en cuanto área útil a efectos booleanos (*core*) para diversos circuitos integrados cuadrados según la longitud L del lado de los mismos:

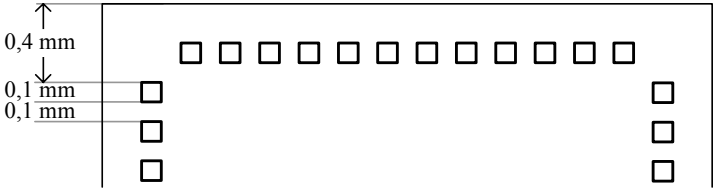
Área total	L	área de contorno	área útil	aprovechamiento
10 mm²	3,16 mm	5,2 mm²	4,8 mm²	48 %
25 mm²	5 mm	9 mm²	16 mm²	64 %
1 cm²	10 mm	19 mm²	81 mm²	81 %
10 cm²	3,16 cm	62 mm²	938 mm²	94 %

Obviamente el aprovechamiento de la superficie del circuito integrado aumenta al hacerlo la superficie del mismo, dado que la anchura ocupada por los elementos de contorno es prácticamente constante.

Número de PADs en el contorno

Una estimación del número de PADs (o sea, del número de entradas y salidas) que pueden situarse en el contorno de un circuito integrado puede ser la siguiente:

- anchura del PAD ~ 100 μm
- separación entre PADs ~ 100 μm
- separación entre el primer (y el último) PAD y el borde del *chip* ~ 400 μm.



Obviamente las anteriores dimensiones dependen de la tecnología de integración pero no pueden ser muy inferiores a las indicadas ya que los PADs son «puntos de soldadura», en los que han de fijarse los hilos que conectan con los terminales del encapsulado y requieren unas dimensiones y una separación mínimas para que la soldadura sea posible.

En un lado **L** del cristal de silicio caben **n** PADs; expresando **L** en milímetros:

$$L \geq 0,4 + n \cdot 0,1 + (n-1) \cdot 0,1 + 0,4; \quad n = \text{Entero} \left[\frac{L-0,7}{0,2} \right] = \text{Entero}[5 \cdot (L-0,7)].$$

La siguiente tabla indica el número de PADs en cada lado y el número total de los mismos para circuitos integrados cuadrados según su longitud **L** de lado:

Área total	L	PADs en cada lado	nº total de terminales (<i>pins</i>)
10 mm²	3,16 mm	12	48
25 mm²	5 mm	21	84
1 cm²	10 mm	46	184
10 cm²	3,16 cm	154	616

Obviamente, si es necesario un mayor número de entradas/salidas, pueden añadirse más PADs (con sus correspondientes adaptadores) en la zona interior del circuito integrado (lo que normalmente es zona booleana, *core*).

T9.3. Consideraciones respecto al diseño físico: reglas de diseño

La relación entre el proceso de diseño y el de fabricación de un circuito integrado se establece a través de las máscaras del circuito; las máscaras son el «resultado final» del diseño y el «molde» con el cual se efectúa la fabricación.

En un diseño físico completo, el diseñador ha de recorrer diversos niveles en la descripción del circuito, desde sus requisitos funcionales iniciales (especificaciones) hasta el diseño físico de las regiones que lo configuran, pasando por:

- definir la arquitectura (diagrama de bloques) que puede dar respuesta adecuada a las especificaciones,
- desglosar los bloques en celdas básicas (funciones booleanas y biestables),
- diseñar cada celda en términos de los transistores que la configuran,
- dimensionar tales transistores y las conexiones entre ellos realizando el dibujo físico de cada celda
- y, finalmente, ubicar adecuadamente cada celda en relación con las demás y establecer las conexiones entre ellas (convenientemente dimensionadas).

El último paso de dicho proceso, el paso de la configuración «electrónica» del circuito en términos de transistores a su conformación «física» en zonas superficiales diferenciadas, es una ardua labor de dibujo lineal, para la que se cuenta con eficaces ayudas informáticas CAD.

En caso de diseño con librería de celdas (*standard cell* o *gate array*), el diseñador del circuito digital no llega al nivel geométrico pero, previamente, quien ha diseñado las celdas (estándar) ha tenido que hacerlo a dicho nivel.

Al abordar el dibujo de los transistores (de sus zonas de fuente, drenaje y puerta) y de las conexiones entre ellos (y de las polarizaciones de los substratos) es necesario:

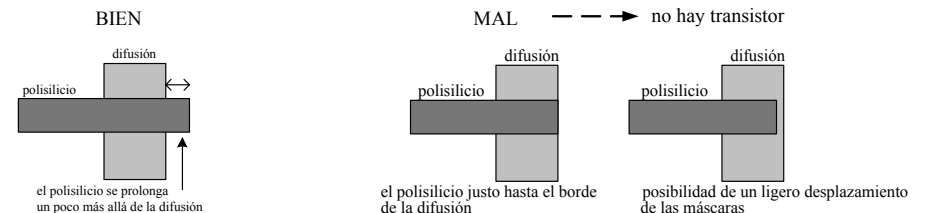
- dotar a cada zona y a cada línea de conexión de dimensiones adecuadas,
- establecer unas separaciones mínimas entre zonas,
- añadir prolongaciones (extensiones más allá del borde) entre las zonas que se cruzan (en particular, el polisilicio y la difusión al formar un transistor),
- asegurar un recubrimiento mínimo de los contactos y de las vías por las zonas entre las que establecen la conexión (y del pozo sobre las difusiones que contiene).

Aparte del adecuado dimensionamiento de los transistores (en conformidad con el diseño), los requisitos referentes a tamaños mínimos, separaciones, prolongaciones y recubrimientos vienen exigidos por las limitaciones de la tecnología de integración, para asegurar la correcta formación de las zonas activas. El conjunto de dimensiones mínimas que han de respetarse se denomina *reglas de diseño* y suelen expresarse en unidades enteras referidas al parámetro λ de la tecnología: $\lambda = L_{\text{mínima}}/2$.

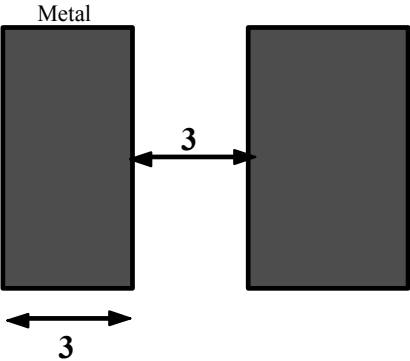
Para facilitar la ejecución del «dibujo físico» (*layout*) del circuito integrado se dispone de potentes herramientas de CAD que incorporan verificadores de las reglas de diseño (DRC: *design rule check*).

Algunos ejemplos de reglas de diseño y su relación con el proceso de fabricación pueden ser los siguientes:

- Anchuras mínimas por debajo de las cuales la tecnología disponible no asegura la continuidad o la fabricación correcta de la correspondiente región, sea ésta una difusión, una línea de metal o la zona de puerta,...
- Separaciones entre zonas del mismo tipo (por ejemplo, difusiones o líneas de metal), que, si estuvieran más cercanas, corren el peligro de que se establezca continuidad entre ellas y queden conectadas;
- El polisilicio al cruzarse con la correspondiente difusión forma un transistor y debe «rebasar» a dicha difusión en una longitud mínima, ya que, de no hacerlo así, pequeñas desviaciones de las máscaras en el proceso de integración pueden dar lugar a que el polisilicio no cruce por completo a la difusión y ésta mantenga continuidad en un extremo de manera que el «transistor» conduce siempre:

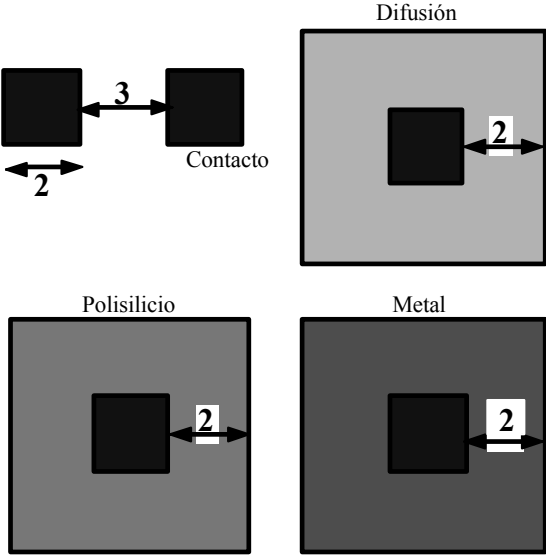


Metal



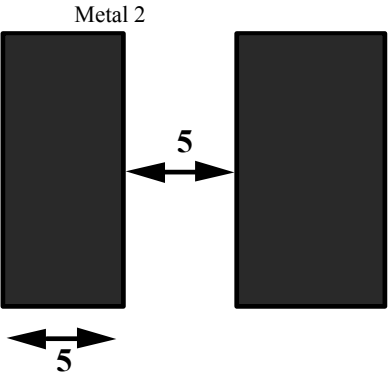
Anchura mínima del metal: 3
Separación entre metales: 3

Contactos



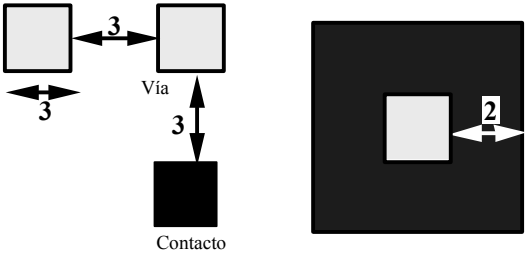
Anchura del contacto: 2
Separación entre contactos: 3
Recubrimiento de metal: 2
Recubrimiento de polisilicio: 2
Recubrimiento de difusión : 2

Metal2



Anchura del metal2: 5
Separación entre metales2: 5

Vías



Anchura de la vía: 3
Separación entre vías: 3
Separación entre vía y contacto: 3
Recubrimiento de metal: 2
Recubrimiento de metal2: 2

Polarización de los substratos

para aislar los transistores y para evitar el problema del «latch-up»

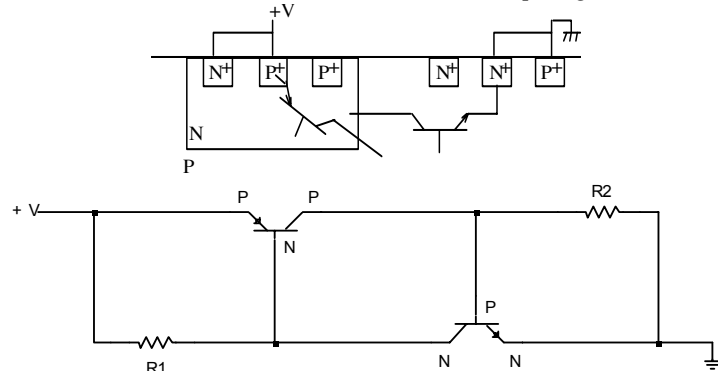
Los transistores NMOS están formados por regiones N⁺ (fuente - canal - drenaje) en el seno de un substrato de tipo P; para aislar los transistores del substrato y los diversos transistores entre sí se requiere polarizar el substrato P con la tensión de alimentación más negativa, de forma que todas las uniones NP con el substrato se encuentren polarizadas inversamente.

De igual forma, para aislar los transistores PMOS, formados por regiones P^+ en un sustrato de tipo N (pozo), se requiere polarizar el sustrato N (todos los pozos presentes en el integrado) con la tensión de alimentación más positiva, para que todas las uniones PN se encuentren polarizadas en inverso.

En sistemas digitales se utiliza, generalmente, una sola fuente de alimentación con dos terminales 0 V y V_{CC} :

- habrá que conectar el sustrato P general que forma la oblea a 0 V, mediante contactos de polarización de tipo P^+ (que se forman a la vez que las difusiones de los transistores PMOS)
- y, asimismo, habrá que conectar los pozos N (que son el sustrato para los transistores PMOS) a V_{CC} , mediante contactos de polarización de tipo N^+ (formados a la vez que las difusiones de los transistores NMOS).

Este requisito de polarización de los sustratos (para aislar los transistores) se encuentra reforzado por el hecho de que las regiones N y P propias de las estructuras CMOS se encuentran intercaladas entre sí y permiten la configuración de estructuras parásitas tipo tiristor (PNPN), con características destructivas al ser posible el disparo de una conducción de intensidad incontrolada. Este fenómeno autodestructivo debido al autodisparo de las estructuras PNPN se conoce como *latch-up* (enganche).



Si la caída de tensión en alguna de las resistencias transversales R_1 y R_2 consigue acercarse a 0,5 V, ambos transistores conducen sin limitación de intensidad (de hecho se encuentran realimentados positivamente: el aumento de la I_C de uno de ellos provoca el de la I_B del otro, con el consiguiente aumento de su I_C y, por tanto, de la I_B del primero), configurando un tiristor en situación de disparo y sin limitación de intensidad.

Ambas resistencias se forman a través de los sustratos, de manera que interesa que la resistividad de ambos sustratos (el P de la oblea y el N de los pozos) no sea muy alta y que la tensión de polarización alcance eficazmente a toda la masa del sustrato (para que las diferencias de tensión entre dos puntos del sustrato sean mínimas).

En tal sentido, es fundamental asegurar una polarización intensiva de toda la superficie de los sustratos: para evitar la posibilidad de *latch-up* es preciso polarizar cada

sustrato con múltiples contactos, de manera que cualquier punto de su superficie se encuentre cercano a un contacto de polarización.

El *latch-up* impone la exigencia de que la polarización de los sustratos (necesaria para aislar los transistores) se haga en forma múltiple e intensiva; este requisito constituye, también, una regla de diseño:

distancia máxima entre contactos de polarización de los sustratos: 50 μm .



Electrónica

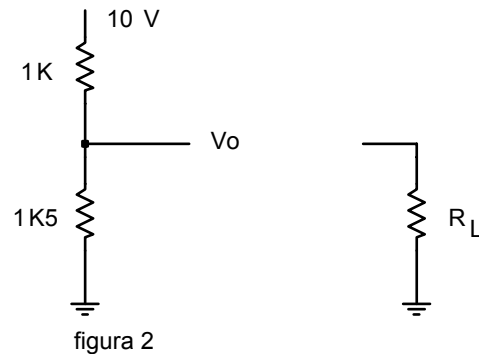
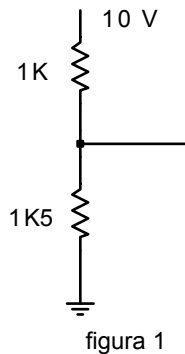
Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

1.- Calcular las potencias de 2 desde 2^0 hasta 2^{20} y aprender las 12 primeras de ellas.

2.- a) Hallar el equivalente thevenin del circuito de la figura 1.

b) ¿Cuál será la tensión de salida V_o de este circuito?

c) ¿Cuánto deberá valer una resistencia de carga R_L para que, al conectarla a la salida del circuito, dicha tensión de salida apenas se modifique (es decir, que su variación sea despreciable)? (figura 2).

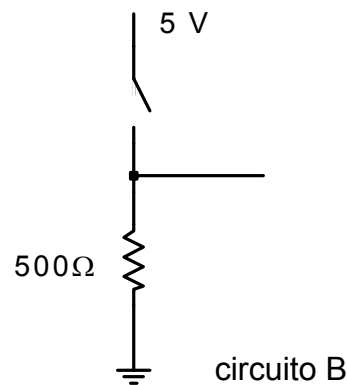
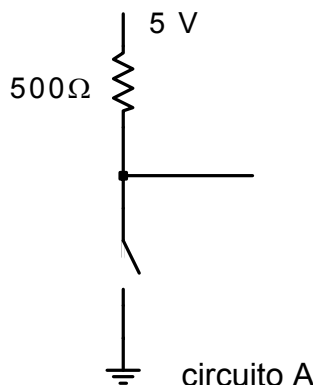


3.- a) Dibujar el equivalente thevenin del circuito A en las dos situaciones posibles:

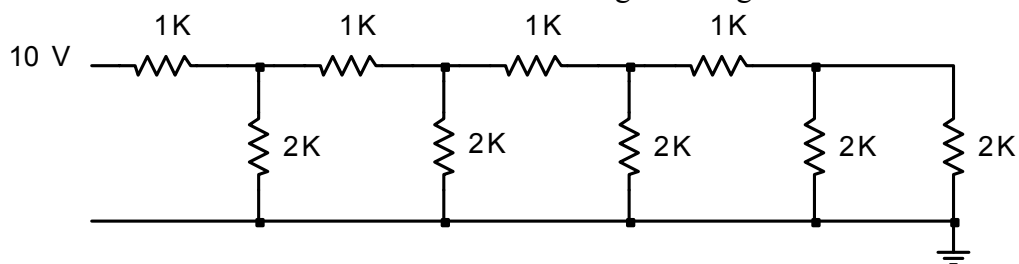
i) interruptor abierto;

ii) interruptor cerrado.

b) Hacer lo mismo con el circuito B.

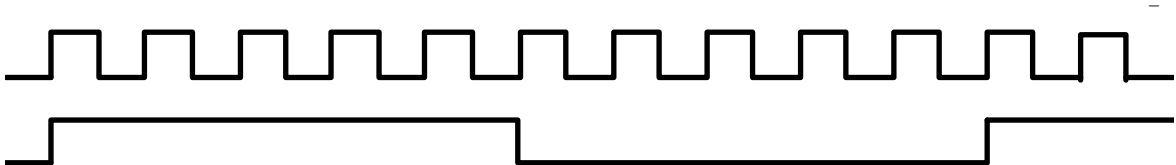


4.- Calcular las tensiones en los 4 nudos del circuito de la siguiente figura.



5.- a) Al aplicar la ley de Ohm, ¿en que unidades resulta la intensidad cuando la tensión se toma en voltios y la resistencia en kilo-ohmios?.

b) Al calcular la constante de tiempo de un circuito RC, ¿en que unidades se obtiene si la resistencia se aplica en kilo-ohmios y el condensador en nanofaradios?.

1. Demostrar los teoremas de absorción.
2. Demostrar en términos booleanos que si $a \Rightarrow b$, sucede también que $\bar{b} \Rightarrow \bar{a}$.
3. Demostrar que la operación Nand ($a * b = \overline{a \cdot b}$) no es asociativa.
4. a) Pasar a binarios los números siguientes: **9675, 2¹², 1024, 500**.
b) Pasar a decimal los n°s binarios: **100101110011, 100000, 111111, 10101010**.
5. ¿Cuántos millares de registros adquirimos al comprar una memoria de 64 Megs?.
6. Sean dos números **A** y **B** y las siguientes proposiciones (lógica proposicional):
 $p \equiv$ primera cifra mayor en **A** que en **B**; $q \equiv$ segunda cifra mayor en **A** que en **B**;
 $r \equiv$ primera cifra mayor en **B** que en **A**; $s \equiv$ segunda cifra mayor en **B** que en **A**;
Utilizando dichas proposiciones **p, q, r** y **s**, escribir en lógica proposicional cuando
a) **A** es igual a **B**; b) **A** es mayor que **B**; c) **A** no es menor que **B**.
7. En un consejo de administración participan su presidente con dos votos el secretario también con dos votos y tres vocales con un voto cada uno; ahora bien, cuando los tres vocales votan en contra el acuerdo queda vetado. Razonando en lógica proposicional (sin utilizar tabla de verdad), expresar booleanamente cuando se aprueba un acuerdo.
8. Ampliar a tres los interruptores conmutados para encender/apagar una lámpara.
9. ¿Cual será la onda de salida de:
a) una puerta "o"; b) una puerta "y"; c) una puerta "o-exclusiva";
si sus dos ondas de entrada son las siguientes:

10. Escribir la tabla de la función booleana: $y = \bar{a} + b \cdot \bar{c}$.
11. Un circuito digital con dos entradas **A** y **C** recibe por **A** una onda cuadrada.
a) ¿Qué combinación de puertas booleanas permite que cuando **C** = 1 dicha onda pase a la salida y no pase (**y** = 0) cuando **C** = 0?.
b) ¿Qué combinación de puertas booleanas permite que la salida sea igual a la entrada **A** cuando **C** = 0 y sea la onda **A** invertida cuando **C** = 1?.

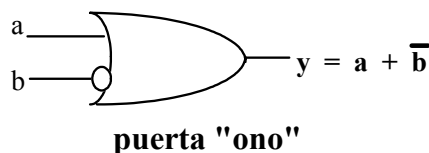
1.- Escribir con puertas Nand de 2 entradas e inversores las siguientes funciones:

- a) $y = a + b$
- b) $y = a + b.c$
- c) $y = a.b + c.d.e$
- d) $y = d.b + \bar{d}.c + c.\bar{a} + \bar{c}.b.a$
- e) $y = d.\bar{c} + d.\bar{a} + \bar{c}.\bar{b} + b.a + \bar{b}.\bar{a}$

2.- Escribir con sólo puertas Nor de 2 entradas (sin inversores ni equivalentes):

- a) $y = a + b.c$
- b) $y = d.c + d.a + c.b + b.a$

3.- Demostrar que la puerta representada en la figura, a la que denominaremos puerta "ono" es de tipo universal, es decir, con ella puede realizarse todo el álgebra booleana. ¿Qué inconveniente tiene esta puerta?



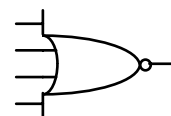
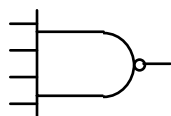
4.- La función “ser numero primo” para números de 4 dígitos puede escribirse en las dos formas siguientes:

$$y = \bar{d}.(a + \bar{c}.b) + a.(\bar{c}.b + c.\bar{b})$$

$$y = (a + \bar{d}.\bar{c}.b).[\bar{d} + (c + b).(\bar{c} + \bar{b})]$$

- a) ¿Cuál de estas dos expresiones resulta más adecuada para puertas *Nand*?. Escribir su expresión algebraica utilizando dichas puertas.
- b) ¿Cuál de estas dos expresiones resulta más adecuada para puertas *Nor*?. Escribirla.

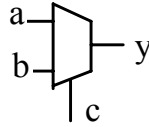
5.- Consideremos una puerta Nand y una puerta Nor de 4 entradas:



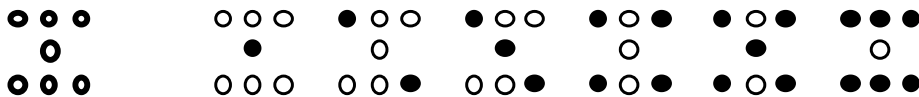
¿Cómo pueden construirse ambas puertas? (dibujar el circuito resultante)

- 1º) utilizando solamente puertas Nand de 2 entradas e inversores
- 2º) utilizando solamente puertas Nor de 2 entradas e inversores.

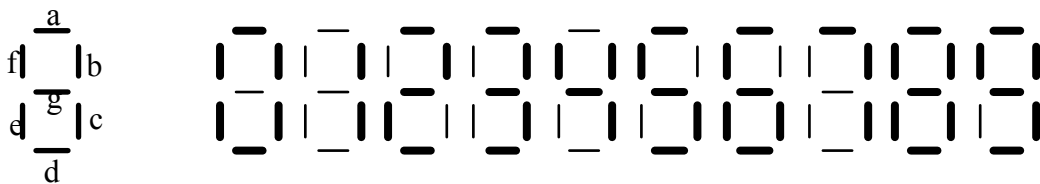
1.- La figura representa un bloque digital "multiplexor", cuya salida es $y = a$ si $c = 0$ e $y = b$ cuando $c = 1$. Escribir las formas canónica y canónica dual de esta función y simplificarlas. Construir la función con sólo puertas "Nand" y con sólo puertas "Nor".



2.- Un circuito digital proporciona números aleatorios del 1 al 6 en binario y con ellos se desea controlar 7 diodos luminiscentes LEDs para que simulen el comportamiento de un dado. Deducir las funciones necesarias.



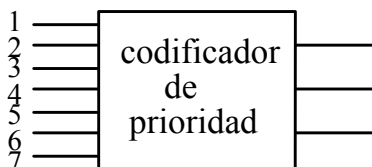
3.- Construir, simplificar y expresar en términos de puertas "Nand" las 7 funciones necesarias para activar un visualizador de 7 segmentos de forma que represente las 10 cifras decimales a partir de su valor en binario (BCD).



4.- Simplificar el siguiente diagrama de Karnaugh de 6 variables:

1 0 0 1	1 1 0 1
0 0 1 1	1 1 0 0
0 0 1 1	1 1 0 0
1 0 0 1	1 1 0 1
1 1 0 1	1 1 1 1
1 0 1 1	1 1 0 0
1 0 1 1	1 1 0 0
1 1 0 1	1 0 1 1

5.- Diseñar un "codificador de prioridad" que recibe 7 líneas de entrada, numeradas de 1 a 7, y proporciona como "salida" el número de la línea de entrada que se encuentra activada; caso de que haya varias líneas activadas indica la de mayor número y si ninguna línea está activada la salida es 0.



6.- Considérense los 12 meses del año numerados del 0 al 11 (en binario) y la función que detecta si uno de los meses tiene 31 días o no. Escribir la forma algebraica simplificada de dicha función en forma de suma de productos y, también en forma de producto de sumas.

1.- Expresar como suma de términos producto, en la forma más simplificada posible, la función:

$$y = (d + \bar{b} + \bar{a}).(\bar{d} + \bar{c} + a).(c + \bar{a}).(\bar{c} + b + a)$$

b) Construir dicha función con puertas *Nand* e inversores.

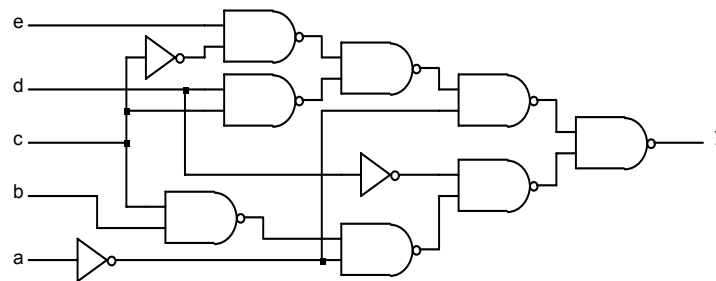
2.- Sea la función booleana

$$y = e.d.c.\bar{b}.a + e.d.\bar{c}.b + e.\bar{c}.\bar{b}.\bar{a} + \bar{e}.d.a + \bar{e}.\bar{d}.\bar{b}.\bar{a} + \bar{d}.c.\bar{a} + \bar{d}.b.\bar{a}$$

a) Expresar la función como producto de términos suma en la forma más simple (sacar luego sumandos comunes para abreviar la expresión).

b) Escribir la expresión algebraica de dicha función con operaciones *Nand*, utilizando el menor número posible de ellas.

3.- Configurar con puertas *Nor*, en la forma más reducida posible, la función booleana representada en la figura siguiente.



4.- Escribir la función como suma de productos en forma más reducida posible:

$$y_1 = (f + d + b + a).(\bar{f} + d + \bar{b} + a).(d + c + a).(f + \bar{e} + \bar{b}).(d + \bar{c} + \bar{b}).$$

$$.(e + d + \bar{c} + a).(\bar{f} + e + d + \bar{c} + b + a)$$

b) Escribir igualmente como suma de productos la función inversa: $y_2 = \bar{y}_1$.

5.- Sea el conjunto de números binarios enteros de 6 cifras **fedcba**; de dicho conjunto se eliminan los siguientes números: **1, 3, 5, 8, 9, 17, 19, 21, 24, 25, 28, 29; 35; 41; 44; 45; 51; 57; 60**; con los números restantes se forma el subconjunto **A**. Construir la función booleana que informa sobre la pertenencia o no al subconjunto **A**: expresar dicha función como suma de productos y como producto de sumas.

6.- Sea $y = f.\bar{d}.b + f.b.\bar{a} + \bar{f}.e.d.\bar{b}.\bar{a} + d.c.\bar{b}.\bar{a} + \bar{d}.\bar{c}.b + \bar{d}.b.\bar{a}$

obtener la expresión simplificada de la función inversa $\bar{y}$ como suma de productos.

1.- Simplificar el siguiente diagrama de Karnaugh de 6 variables:

1	0	0	1	1	1	0	1
0	0	1	1	1	1	0	0
0	0	1	1	1	1	0	0
1	0	0	1	1	1	0	1
1	1	0	1	1	1	1	1
1	0	1	1	1	1	0	0
1	0	1	1	1	1	0	0
1	1	0	1	1	0	1	1

2.- Sea el conjunto de números binarios enteros de 6 cifras **fedcba**; de dicho conjunto se eliminan los siguientes números: **1, 3, 5, 8, 9, 17, 19, 21, 24, 25, 28, 29; 35; 41; 44; 45; 51; 57; 60**; con los números restantes se forma el subconjunto **A**. Construir la función booleana que informa sobre la pertenencia o no al subconjunto **A**: expresar dicha función como suma de productos y como producto de sumas.

3.- Escribir la función como suma de productos en forma más reducida posible:

$$y_1 = (f + d + b + a).(\bar{f} + d + \bar{b} + a).(d + c + a).(f + \bar{e} + \bar{b}).(d + \bar{c} + \bar{b}).$$

$$.(e + d + \bar{c} + a).(\bar{f} + e + d + \bar{c} + b + a)$$

4.- Una apuesta de la lotería primitiva consiste en seleccionar 6 números del 1 al 49. Un apostante ha marcado los siguientes: **4, 11, 20, 27, 36 y 43**.

- a) ¿Cuál es la función booleana que expresa dicha apuesta?. Escribir dicha función, en forma simplificada, como suma de productos y como producto de sumas.
b) ¿Cuál de estas formas puede construirse con menos transistores NMOS?

5.- ¿Cuál de las siguientes funciones corresponde al mapa de Karnaugh expresado abajo?. Razonar adecuadamente la respuesta.

$$y = d.\bar{b}.(\bar{e} + \bar{a}) + e.\bar{a}.(c + b) + \bar{d}.a.(\bar{c} + b) \text{ y } y = (d + \bar{c} + e.\bar{a}).(c + a + d.\bar{b}).(\bar{d} + \bar{a} + \bar{e}.\bar{b})$$

e d	c b a								
	X	X	1	X		X	X	0	0
	1	1	0	0		X	0	1	1
	1	0	0	X		1	0	X	1
	0	1	X	X		1	X	0	X

¿Cuántos transistores son necesarios para construir, en tecnología NMOS, cada función (tal como están expresadas en el apartado anterior)?. Razonar la respuesta.

1.- Sea un restador de números de 6 dígitos especial: tiene una entrada de control x , tal que si $x = 0$ efectúa la resta $A-B$ y para $x = 1$ la operación que realiza es $B-A$; se desea realizar su diseño modularmente, utilizando 6 celdas restadoras de 1 bit.

a) Dibujar la conexión de dichas 6 celdas para configurar el restador, e indicar cómo puede utilizarse este restador para calcular $|A-B|$ (valor absoluto).

b) Escribir las ecuaciones booleanas necesarias para construir una de las celdas, en forma de suma de productos.

2.- Escribir la función booleana que calcula $A > B$, siendo A y B números de 2 bits. Simplificar dicha función, sacar factores comunes y generalizarla para n°s de 6 bits. ¿Cuántos términos producto tiene esta función?

3.- Utilizando puertas "y" y sumadores de 4 bits, construir un multiplicador de 4 bits?. ¿Cuántas puertas "y" y cuantos sumadores de 4 bits serían necesarios para construir en la misma forma un multiplicador de 8 bits?.

4.- Con multiplicadores y sumadores de 4 bits, construir un multiplicador de 8 bits.

5.- Interesa disponer de una unidad lógica capaz de realizar las operaciones:

S2 S1 S0		S2 S1 S0		S2 S1 S0 :
0 0 0	$a.b$	1 0 0	$a * b$	entradas
0 0 1	$a + b$	1 0 1	$a \Delta b$	de selección
0 1 0	$\bar{a}.b$	1 1 0	$a + \bar{b}$	de operación
0 1 1	$a \oplus b$	1 1 1	$a \otimes b$	

Diseñar dicha unidad lógica en las dos formas siguientes:

- utilizando un multiplexor de 4 líneas y puertas booleanas de 2 entradas (se admite también la puerta o-exclusiva);
- utilizando solamente puertas booleanas de 2 entradas (incluida la o-exclusiva).

6.- Sea una palabra de 8 dígitos; se nos pide diseñar un circuito capaz de contar el número de “unos” que contiene dicha palabra y para ello disponemos de celdas sumadoras de 1 bit (entradas a, b y c’). Dibujar el diagrama de bloques del circuito. ¿Cuántas celdas sumadoras serían necesarias si la palabra fuera de 15 bits?.

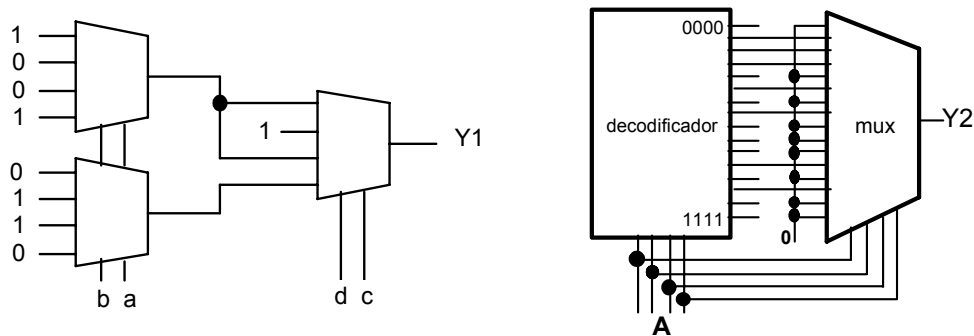
7.- Demostrar que una celda sumadora puede ser utilizada como puerta “o” de dos entradas y, también, puede ser utilizada como puerta “y” de 2 entradas; ¿qué otras puertas de 2 entradas pueden ser “reemplazadas” por una celda sumadora y cómo?.

1.- Disponemos de una fila de 30 diodos LED, numerados del 1 al 30, y se desea controlarlos por su número en binario, de forma que, en cada momento, podamos encender uno de los diodos poniendo su número en las entradas del circuito de control; diseñar dicho circuito, utilizando solamente decodificadores de 3 entradas y puertas booleanas básicas.

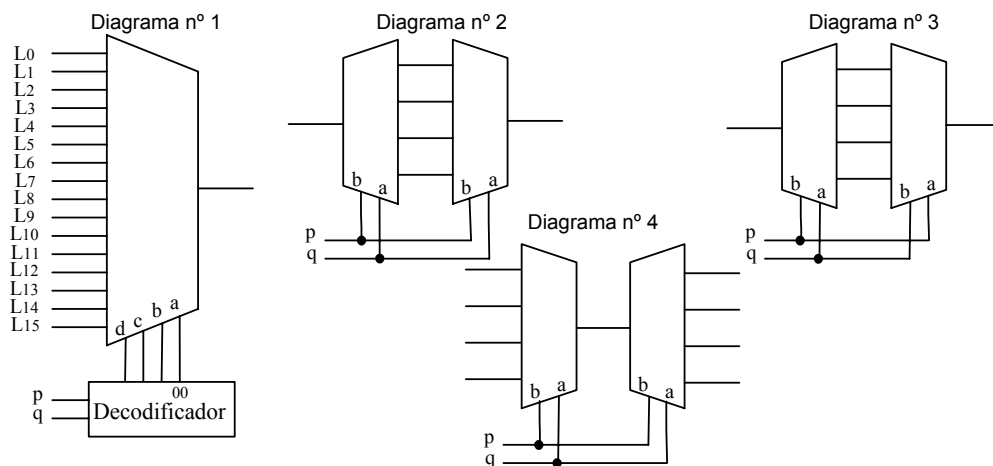
2.- Utilizando solamente un multiplexor de 2 líneas de entrada (1 entrada de control), construir las siguientes operaciones: $a + b$; $a.b$; $\bar{a}.b$; $\bar{a} + b$. ¿Cómo podría construirse la operación o-exclusiva añadiendo sólo un inversor?.

3.- Mediante un multiplexor de 8 líneas de entrada (3 entradas de control) construir la siguiente función: $y = (\bar{d} + c).(c + b).(\bar{b} + a)$.

4.- Encontrar las funciones booleanas Y1 e Y2 (A es un nº de 4 dígitos):



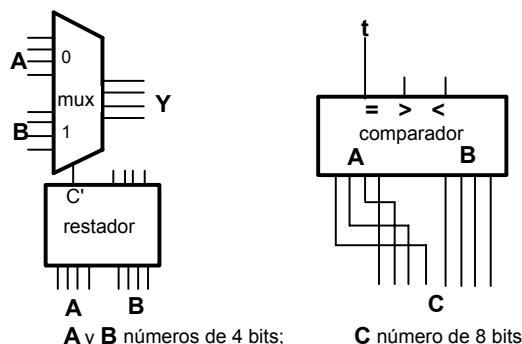
5.- Los 4 diagramas siguientes son claramente simplificables en cuanto a la función que realizan; ¿a qué equivale cada uno de estos diagramas?. Dibuja el resultado.



6.- Un decodificador tiene entrada BCD y 10 líneas de salida. Dibujar el esquema circuital correspondiente a su configuración interna, construido con puertas **NOR**.

1.- Con decodificadores de 4 líneas (sin entradas de habilitación) y puertas básicas (o, y), construir un decodificador, un multiplexor y un demultiplexor de 16 líneas.

2.- ¿Qué función realiza cada uno de los dos sistemas digitales representados en la siguiente figura, siendo **A** y **B** números de 4 bits y **C** un número de 8 bits?.



3.- Representar dos cifras BCD sobre sendos visualizadores de 7 segmentos de cátodo común, utilizando solamente un conversor de BCD a 7 segmentos; dibujar el diagrama de bloques combinacionales necesario.

4.- Diseñar con bloques combinacionales un sistema que acepte como entradas dos números de 4 bits y presente el mayor de ellos en su salida. Añadir una entrada **c** que permita seleccionar el mayor (**c = 1**) o el menor (**c = 0**) de dichos números.

5.- Los circuitos integrados 7485 (comparador) y 74157 (multiplexor de 2 buses) han sido utilizados en prácticas de laboratorio; dibujar con ellos el diagrama de bloques “completo” de un sistema digital que recibe 3 números binarios de 4 dígitos y selecciona (“deja pasar”) el menor de ellos.

6.- a) Un circuito recibe 3 números enteros de 8 bits (en complemento a 2); dibujar el esquema de bloques de un circuito que “deje pasar el mayor” de ellos.

b) Repetir el diseño anterior utilizando 3 adaptadores triestado.

c) Dibujar el diagrama de bloques para “dejar pasar” el número “intermedio” de los tres (el que no es el mayor ni el menor).

7.- Sea una palabra binaria de 4 bits ¿qué combinación de puertas booleanas es necesaria para efectuar cada una de las siguientes tareas?.

- a) indicar que todos los bits de la palabra son 0;
- b) indicar que los bits son todos 1;
- c) indicar que todos los bits de la palabra son iguales;
- d) indicar que el número de bits a 1 es impar.

1.- Sea un bloque matricial programable de 12 entradas y 8 salidas. ¿Cuántas conexiones fijas y cuántas programables tendrá en cada uno de los siguientes casos:

a) **PROM** b) **PLA**, con 40 térm. producto y c) **PAL**, con 6 términos en cada salida.

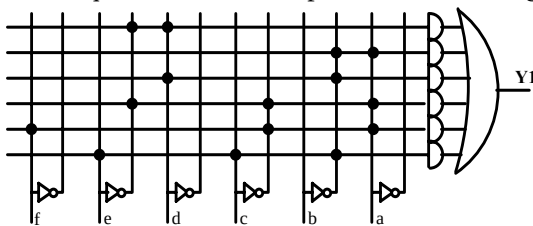
¿Cuántos transistores de cada tipo son necesarios para construir esos bloques?

2.- Un circuito dispone de 9 salidas **L9 . . L1**, conectadas a un visualizador formado por una línea de diodos LEDs que representa un "**código de barra**" de 10 niveles (nivel 0: **000000000**; nivel 1: **000000001**; ... nivel 8: **011111111**; nivel 9: **111111111**) Se desea diseñar un conversor "**código de barra**" a **BCD**: el diseño ha de ser realizado sobre dispositivos programables y si la entrada no corresponde a un nivel del "**código de barras**" la respuesta debe ser 0.

a) Para programarlo sobre una **PLA**, ¿cuáles serán los términos producto a construir y a qué salidas hay que conectar cada uno? ¿cuántas conexiones habría que eliminar?

c) Escribir las funciones necesarias para una **PAL**. (¿conexiones a eliminar?)

3.- Alguien ha programado una función **Y1** en un bloque **PAL** según la figura, pero se ha equivocado: la función que necesitaba era precisamente la negada de esta.



a) ¿Cuál será la expresión booleana necesaria para programar $\overline{Y1}$?

b) ¿Cómo se programaría $\overline{Y1}$ en una **PROM** de 6 entradas? ¿cuáles serían las conexiones ("fusibles") que habría que eliminar?

4.- Sean teclado numérico con 10 teclas para las cifras decimales (**j i h g f e d c b a**, la línea **a** corresponde a la tecla del **0** y así sucesivamente); se necesita un codificador tal que, cuando se pulsa una sola de dichas teclas, se genere el número Gray correspondiente (si se pulsaran dos teclas a la vez o, también, si no se pulsa ninguna la salida del codificador es **1111** - número Gray que corresponde al decimal 10 -).

a) Escribir la tabla funcional "resumida" de dicho codificador.

b) Disponemos de una **PLA** de 10 entradas, 4 salidas y 32 términos producto y disponemos también de inversores, ¿cuáles serán los términos producto a programar?

c) Modificando el enunciado, de forma que si hay varias teclas pulsadas se obtenga el valor correspondiente a la menor de ellas (y si no hay ninguna pulsada el valor **1111**), obtener las funciones que permiten su programación sobre un bloque **PAL**.

5.- Sea la función $y = A \leq B$ con **A**, **B** números binarios de 6 bits. Si programamos dicha función en el interior de un circuito integrado programable de tipo **PAL** de 16 entradas, 8 salidas y con 10 términos producto para cada salida; ¿cuál será la expresión booleana correspondiente a tal programación?

Más ejercicios de bloques programables

A.- Sea un bloque matricial programable de 20 entradas y 10 salidas. ¿Cuántas conexiones fijas y cuántas programables tendrá en cada uno de los siguientes casos:

a) **PROM** b) **PLA**, con 64 térm. producto y c) **PAL**, con 16 términos en cada salida.

B.- Supongamos un codificador de 8 entradas y 8 salidas, muy simple: lo único que hace es invertir las entradas, es decir, el código de salida es directamente el negado (bit a bit) del de entrada. Si queremos programar tal codificador en un bloque programable de 8 entradas y 8 salidas, ¿cuál será el número de conexiones a eliminar (el número de fusibles a fundir) en cada uno de los siguientes casos?

- a) al programar el codificador en una **PROM** de 8 entradas y 8 salidas
- b) en una **PAL** de 8 entradas y 8 salidas con 4 términos producto para cada una
- c) para una **PLA** de 8 entradas, 8 salidas y 24 términos producto a compartir.

C.- Sean 10 funciones de 8 entradas, cuya tabla funcional contiene el triple número de "unos" que de "ceros" y cuyas expresiones algebraicas están formadas, todas ellas, por 7 términos producto de 5 variables cada uno; se sabe también que en simplificación multifunción de las 10 funciones se obtienen 28 términos producto, todos ellos de 6 variables, y de los cuales cada función utiliza sólo 16 términos. Calcular el número de "fusibles" que es necesario fundir para programar dichas funciones en:

- a) una **PROM** de 8 entradas x 10 salidas;
- b) una **PLA** de 8 entradas x 40 términos producto x 10 salidas;
- c) una **PAL** de 10 módulos de 8 entradas x 12 términos producto.

D.- Un codificador de prioridad recibe 10 entradas procedentes de 10 teclas decimales numeradas de 0 a 9 y debe expresar en su salida en BCD cual es la tecla de mayor número que se encuentra activada; caso de no haber ninguna tecla pulsada, las salidas del codificador de prioridad se situarán todas ellas a "1".

a) Obtener las funciones booleanas que permiten la programación de este codificador sobre un bloque **PAL**; simplificar en lo posible dichas funciones.

b) ¿Qué términos producto sería razonable utilizar para programarlo sobre una **PLA**?

E.- Sean 2 números de 4 cifras, ¿cómo programar la función $A > B$ en una **PAL** de 10 entradas y 6 términos producto en cada uno de sus módulos?

Para programar esta función en **EPROM** ¿cuántas conexiones habría que eliminar?

F.-
$$y = [a + (f + d).(e + b).(d + c)]. [\bar{b} + (f + \bar{e}).(d + \bar{c}).(d + \bar{e})]$$

a) Caso de que esta función vaya a ser programada sobre un integrado **PROM** de 6 entradas, ¿qué conexiones programables ("fusibles") sería preciso eliminar?

b) Si la programación se realiza sobre un módulo **PAL** de 6 entradas y 6 términos producto ¿cual será la expresión a programar?

1.- Con una longitud de palabra de 64 bits, ¿cual será el rango de números enteros que puede manejarse con dicha longitud?: a) en sistema binario b) en BCD.

2.- Un sistema digital opera con números enteros codificados sobre 8 bits en complemento a 2; efectuar en binario y en decimal el resultado de las operaciones:

a) $01110100 + 11010100$; b) $00111100 + 11011101$;

c) $10011100 + 11100110$ d) $01110100 - 11010100$;

e) $00111100 - 11011101$; f) $10011100 - 11100110$.

3.- Un sistema utiliza números con longitud de palabra de 10 bits, coma fija de 3 bits; para las siguientes operaciones, expresar en binario los operandos y el resultado:

a) $(+42,5) + (+38,4) =$ b) $(+42,5) + (-38,4) =$

c) $(-33,7) - (+27,625) =$ d) $(-33,7) - (-27,625) =$

4.- a) Transformar en binario en coma fija (16 bits: 11 parte entera y 5 parte decimal)

A: 12,75 ; B: 23,432 ; C: + 868,85 ; D: -155,16.

b) ¿Cuanto valen exactamente el mayor número positivo y el menor número positivo (distinto del cero) que puede escribirse en la mencionada coma fija?.

c) Idem el mayor número negativo y el menor de ellos.

d) Efectuar en binario las siguientes operaciones y, una vez obtenido el resultado, transformar este en valor decimal (base 10): A + B; A - B; B - A

5.- Entenderemos por BC6 un sistema de numeración en base 6, cuyas cifras se codifican en binario; diseñar el circuito necesario para efectuar la suma en BC6, si se dispone de sumadores binarios de 4 bits.

6.- a) ¿Cuál será la palabra hexadecimal que corresponde al numero **52.483**?

b) ¿A qué numero decimal corresponde el hexadecimal **DECA**?. ¿Cuál será el número gray correspondiente (expresarlo en hexadecimal)?.

c) ¿Cuál será (en hexadecimal) el código Hamming para la palabra **CAFE**?

d) ¿Cuál será la palabra original si su código Hamming es **CAFE**?

7.- Sea una palabra binaria de 54 bits, calcular razonadamente cuantos bits se necesitan para transmitirla en código Hamming de distancia 4. Hacer lo mismo para otra palabra binaria con la mitad de dígitos, 27 bits.

8.- Un sistema digital envía a otro números binarios (enteros y positivos) de 6 bits en código Hamming; el segundo sistema, después de reconocer los números iniciales y corregirlos si es necesario, los suma y los devuelve codificados en la misma forma: ¿qué número remitirá si los recibidos han sido **01101110011** y **01011111100**?

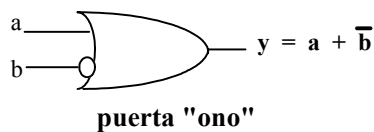
1.- En las siguientes funciones y sin necesidad de dibujar las puertas, contar el número de interruptores (o transistores NMOS) necesario para construir directamente cada una de las dos expresiones de ellas.

a) $y = \bar{c}.\bar{b}.\bar{a} + c.b.a = \overline{(c + b + a).(c.b.a)}$

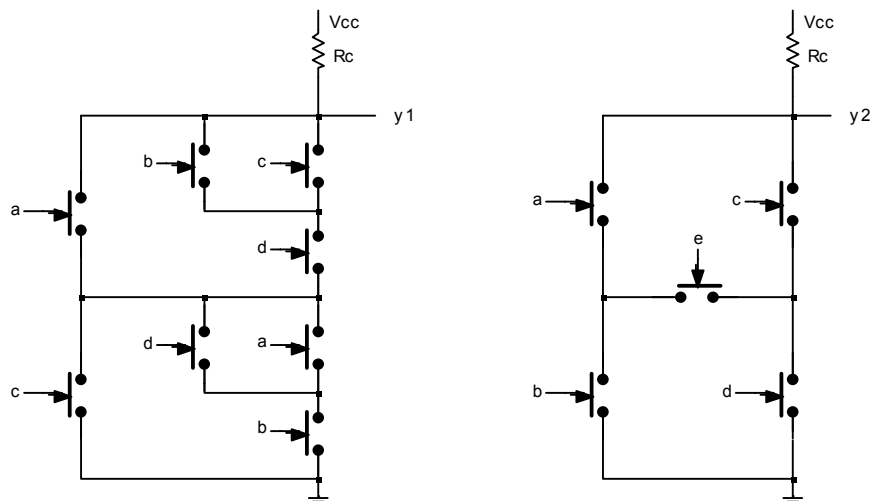
b) $y = (a + b).(c + d + e) = \overline{\overline{a.b} + \overline{c.d.e}}$

c) $y = (\bar{d} + \bar{b}).(\bar{c} + \bar{a}) + b.a = \overline{(d.b + c.a).(b.a)}$

2.- Construir con interruptores la puerta representada en la figura.



3.- Considérense los circuitos de la figuras siguientes. ¿Qué función booleana calcula cada uno de ellos?. Simplificar el primero de ellos: dibujarlo de nuevo, reduciendo en lo posible el número de interruptores.



4.- Dibujar las siguientes puertas booleanas con transistores NMOS, procurando utilizar el menor número posible de ellos:

a) $y = (\bar{c} + \bar{b} + \bar{a}).(c + b + a)$

b) $y = d.\bar{c} + d.\bar{a} + \bar{d}.c.b.a$

c) $y = (\bar{e} + \bar{a}).(\bar{d} + \bar{a}) + c.b$

d) $y = d.\bar{c} + d.\bar{a} + \bar{c}.\bar{b} + b.a + \bar{b}.\bar{a}$

1.- Sean las funciones booleanas siguientes; construir las en configuración CMOS, procurando utilizar el menor número de transistores posible.

$$y_1 = a + \bar{b}$$

$$y_2 = a + \bar{c}.b$$

$$y_3 = d.c + d.a + c.b + b.a$$

$$y_4 = c.(b \oplus a) + b \Delta a$$

$$y_5 = (c + \bar{b}).(\bar{c} + a).(\bar{b} + a).(b + \bar{a})$$

$$y_6 = \bar{d}.c + c.\bar{b} + \bar{c}.a + \bar{b}.a$$

$$y_7 = \bar{d}.c.a + \bar{d}.b.a + \bar{c}.b.\bar{a}$$

$$y_8 = (d + \bar{a}).(\bar{e} + \bar{d} + b).(\bar{d} + c + \bar{a}).(\bar{e} + \bar{d} + c + \bar{b} + a).(e + \bar{d} + \bar{c} + b + \bar{a})$$

2.- Completar la puerta lógica CMOS cuyo plano P es el representado en la figura 1. Simplificar dicha puerta, reduciendo en lo posible el número de transistores.

3.- El plano PMOS de una puerta lógica CMOS es el representado en la figura 2. ¿Qué función booleana realiza?. Dibújese el correspondiente plano NMOS.

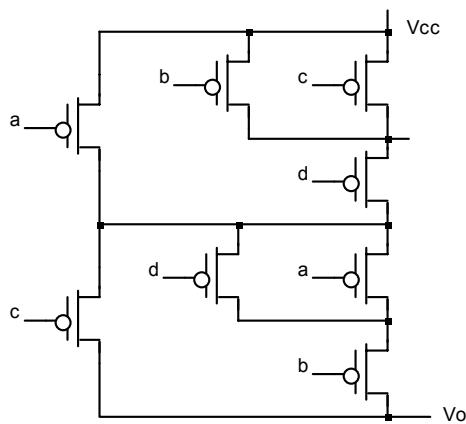


Figura 1

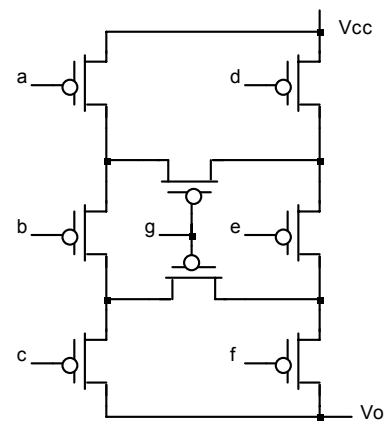


Figura 2

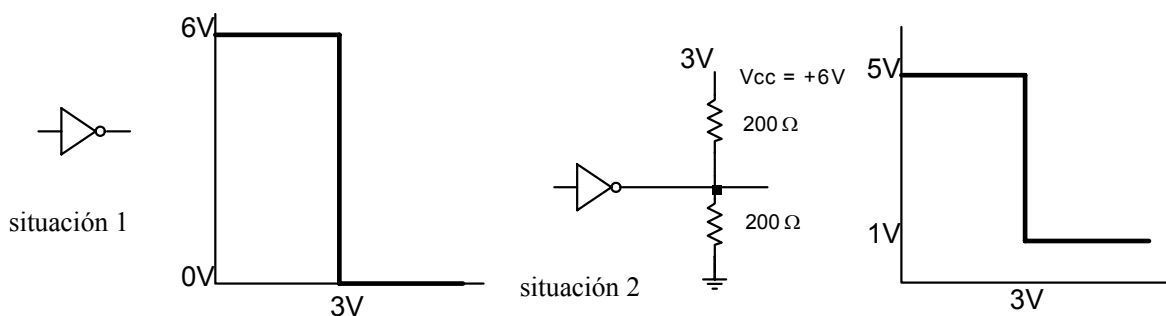
1.- Dos inversores de familias diferentes, cuyos datos de tensión e intensidad son:

inversor A: $V_{iH\min} = 2 \text{ V}$ $V_{iL\max} = 0,7 \text{ V}$ $I_i < 50 \mu\text{A}$
 para $I_o = 1 \text{ mA}$: $V_{oH} = 3,5 \text{ V}$ $V_{oL} = 0,2 \text{ V}$
 para $I_o = 6 \text{ mA}$: $V_{oH} = 3 \text{ V}$ $V_{oL} = 0,3 \text{ V}$

inversor B: $V_a = 1,5 \text{ v.}$ $V_b = 2 \text{ V}$ $I_i < 20 \mu\text{A}$
(es de tipo Schmitt) para $I_o = 0,1 \text{ mA}$: $V_{oH} = 5 \text{ V}$ $V_{oL} = 0 \text{ V}$
 para $I_o = 2 \text{ mA}$: $V_{oH} = 3 \text{ V}$ $V_{oL} = 1 \text{ V}$

¿Cual de ellos presenta mayor inmunidad frente al ruido?; ¿qué margen de seguridad frente al ruido tienen?; ¿qué "fan-out" puede deducirse de los datos anteriores?.

2.- Para medir la resistencia de salida de un inversor CMOS se han obtenido las funciones de transferencia representadas en la figura siguiente; ¿cuál es el valor de dicha resistencia?, ¿cuál será (en la situación 1) el margen de ruido en potencia?.



3.- Los montajes de la figura corresponden a un inversor CMOS, alimentado a 5 V; ¿cuánto valdrá la resistencia de salida del inversor en cada caso?, ¿a qué puede deberse el que ambos valores sean diferentes?, ¿cuál de las dos resistencias es más favorable y por qué?, ¿cuál es la intensidad máxima que podemos obtener de la salida del inversor si queremos que el margen de ruido no sea inferior a 1,5 V?.



4.- Un hipotético circuito integrado se alimenta a 12 voltios y contiene 4 puertas Nor de 2 entradas, sus datos de consumo según catálogo son los siguientes: $I_{CCH} = 0,4 \text{ mA}$; $I_{CCL} = 0,8 \text{ mA}$; $C_{pd} = 75 \text{ pF}$;

Si medimos el consumo de dicho integrado, conectando todas sus entradas a 0V menos una de ellas que se conecta a una señal cuadrada de 20 MHz, ¿cuál será el valor que indique el polímetro al efectuar esta medida?.

5.- Sea un C.I. de 6 inversores que se alimenta a 10 V; al medir el consumo de los 6 inversores conmutando con una onda cuadrada resultan los siguientes valores: 24 mA si la onda es de 4 MHz y 36 mA si es de 8 MHz; ¿qué podemos decir de su consumo estático?, calcular el parámetro C_{pd} de cada inversor.

ELECTRÓNICA DIGITAL [1ºQ 14]

En los guiones de prácticas se incluyen las hojas de características de la serie HC; compruébese que se sabe localizar sobre ellas:

- Tensiones e intensidades de entrada y de salida para ambos estados 1/0:
(V_{OH} , I_{OH}), (V_{OL} , I_{OL}); $V_{iHmín}$, $V_{iLmáx}$, I_{iH} , I_{iL} .
- Márgenes de ruido en tensión y en potencia.
- Fan-out.
- Consumo en cada estado 1/0. Consumo dinámico.
- Tiempos de propagación. Velocidad de trabajo estimada para un biestable.
- Intervalo de tensiones de alimentación.
- Intervalo de temperaturas.

Se sugiere, asimismo, estudiar-repasar la forma de medir estas magnitudes utilizada en la práctica nº 5.

Cuestiones referentes a laboratorio:

- ¿Cómo puede obtenerse, con un polímetro, la función de transferencia de una puerta?.
- ¿Cómo se medirá, con un polímetro, su resistencia de salida?.
- ¿Cómo puede medirse, con un osciloscopio, la resistencia de salida?.
- ¿Cómo se puede comprobar que el consumo estático de una puerta es nulo?.
- ¿Cómo se puede comprobar que el consumo dinámico aumenta con la frecuencia y que dicho aumento es lineal?.
- Si el consumo estático no fuera nulo, ¿cómo se mediría el consumo dinámico?.
- ¿Cómo puede medirse con un polímetro el margen de ruido en tensión?.
- ¿Cómo puede medirse con un polímetro el margen de ruido en potencia?.
- ¿Cómo podemos medir con un polímetro las dos tensiones de conmutación de una puerta con entrada Schmitt?.
- ¿Cómo puede medirse con un osciloscopio el tiempo de propagación de una puerta?.
- ¿Cómo puede medirse el tiempo de propagación con un polímetro?.

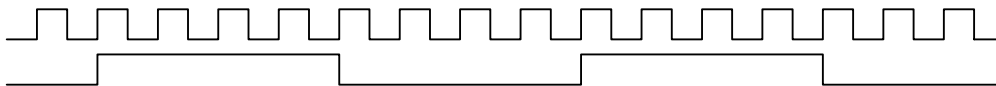
ELECTRÓNICA DIGITAL Práctica nº 1: Puertas Booleanas.

Objetivos:

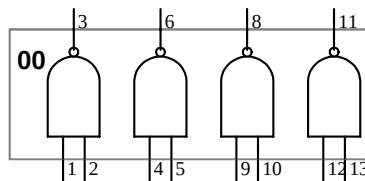
- “Tocar” y utilizar los circuitos integrados digitales, comenzando por los correspondientes a puertas lógicas.
- Montar en laboratorio y comprobar la operación de inversores y puertas "o", "y", "nand", "nor" y "o-exclusiva"
- Comprobar la inversión de ondas por parte del inversor, el "control de paso" por parte de una puerta "y" y el "control de inversión" por parte de una puerta "o-exclusiva".
- Verificar que las operaciones "nand" y "nor" son de tipo unitario, construyendo con ellas las operaciones básicas.
- Comprobar que la operación "nand" es útil para construir sumas de productos y la operación "nor" lo es para construir productos de sumas.

Preparación de la práctica:

1. Tablas de las operaciones "o", "y", "nand", "nor", "o-exclusiva"
2. Dibujar el esquema de conexiones necesario para ver en el osciloscopio, a la vez, la entrada y salida de un inversor que recibe una onda cuadrada; dibujar, también, (una debajo de otra) dichas ondas de entrada y salida.
3. Dibujar las ondas de salida de las puertas "o", "y", "nand", "nor" y "o-exclusiva" de dos entradas, siendo las señales conectadas a dichas entradas las siguientes:



4. Una representación de las 4 puertas "nand" en el circuito integrado 7400 es la siguiente



en esta figura las entradas se han dibujado a un lado y las salidas al otro; esta representación no se corresponde con la situación real de los terminales de dicho circuito integrado, dibuja el circuito integrado tal como sus terminales se encuentran situados en realidad.

5. Todos los circuitos integrados que utilizamos en esta práctica son de 14 terminles y se alimentan a 5 V por los terminales 14 (a +5 V) y 7 (referencia o «tierra» a 0 V); dibuja el circuito integrado junto con su fuente de alimentación.
6. Dibujar la forma de obtener con puertas "nand" las operaciones "o", "y", "nor", "o-exclusiva"; ¿cómo se puede obtener esta última operación con solamente 4 puertas "nand"?
7. Dibujar la forma de obtener con puertas "nor" las operaciones "o", "y", "nand", "o-exclusiva"
8. Dibujar la forma de obtener con puertas "nand" la función $y = a.b + c.d$
Escribir la tabla de dicha función
9. Dibujar la forma de obtener con puertas "nor" la función $y = (a + b) . (c + d)$
Escribir la tabla de dicha función
10. Empleamos diodos LED para observar el estado de las salidas de las puertas, dichos diodos deben llevar una resistencia en serie para limitar la intensidad que pasa por ellos; dibuja el circuito: inversor – diodo LED a su salida,

¿cuándo debe valer la resistencia si la tensión del diodo LED cuando conduce es de 2 V y la intensidad que se requiere para verlo bien es de unos 15 mA?

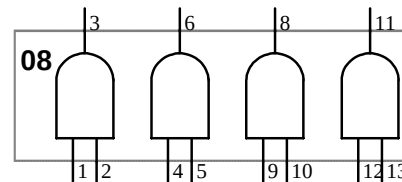
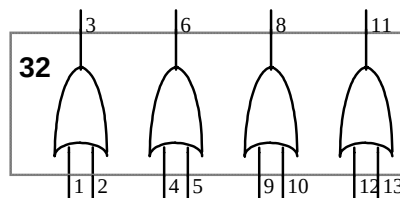
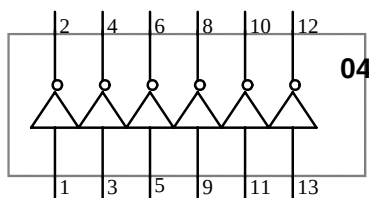
Desarrollo de la práctica:

1.- Puertas básicas. Inversor, puerta "o" y puerta "y":

- Comprobar la tabla de operación de un inversor (7404), de una puerta "o" (7432) y de una puerta "y" (7408), visualizando su salida en uno de los diodos LED.

Nota: Es necesario alimentar a 5 V estos circuitos integrados digitales.

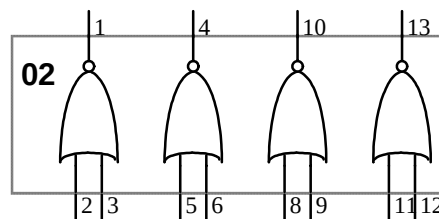
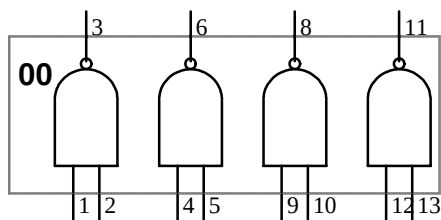
- Conectando a la entrada de un inversor una onda cuadrada de 10 kHz, observar en el osciloscopio la inversión de la onda, comparando la entrada y la salida del inversor.
- Conectando una onda cuadrada de 100 kHz a una de las entradas de una puerta "y", observar su comportamiento como "puerta de paso" controlada por la otra entrada ("1"= pasan los pulsos; "0"= no pasan).
- Configurar una puerta o-exclusiva y comprobar su funcionamiento.
- Conectando una onda cuadrada de 1 kHz a una de las entradas de dicha puerta o-exclusiva, observar su comportamiento como "puerta inversora" controlada por la otra entrada ("1"= se invierten los pulsos; "0"= no se invierten).



Terminales de alimentación: +5V 14 ; 0V 7

2.- Puertas "unitarias". Puerta Nand y puerta Nor:

- Comprobar la tabla de operación de una puerta Nand (7400) y de una puerta Nor (7402), visualizando su salida en uno de los diodos LED.
- Configurar con puertas Nand un inversor, una puerta "o" y una puerta "y" y comprobar su funcionamiento.
- Configurar con puertas Nor un inversor, una puerta "o" y una puerta "y" y comprobar su funcionamiento.
- Configurar con cuatro puertas Nand una puerta "o-exclusiva" y comprobarla.
- Construir con puertas Nand la función " $y = a.b + c.d$ " y comprobar su tabla funcional.
- Construir con puertas Nor la función " $y = (a + b) . (c + d)$ " y comprobar su tabla funcional.



Terminales de alimentación: +5V 14 ; 0V 7

ELECTRÓNICA DIGITAL Práctica nº 2: Funciones Booleanas.

Objetivos:

- “Tocar” y utilizar los circuitos integrados digitales para construir funciones booleanas.
- Montar en laboratorio y comprobar diversas funciones booleanas.
- Entrar en contacto con la suma y la comparación aritméticas, construidas como funciones booleanas.
- Comprobar la posibilidad de construir una función booleana con un multiplexor o con un decodificador.

Preparación de la práctica:

1. Obtener la expresión algebraica de la función “ser número primo” para números binarios de 3 dígitos y pasar dicha expresión a puertas "nand" e inversores
2. Sea el siguiente conjunto de números: {0, 1, 5, 8, 9, 10, 11, 13, 15}
y la función que indica si un número binario de 4 bits pertenece o no a dicho conjunto;
escribir la tabla de esta función y obtener su expresión algebraica con puertas "nor" e inversores
3. Sean dos números binarios de un solo dígito $A = a$, $B = b$,
obtener las funciones $A > B$ y $B > A$ y configurarlas con puertas "nand" e inversores;
teniendo en cuenta que la función $A = B$ es igual a $(\overline{A > B}) \cdot (\overline{A < B})$,
obtener la función $A = B$ aprovechando las puertas de las dos funciones anteriores;
de esta forma, dibujar las tres funciones utilizando solamente las puertas
de dos circuitos integrados: un 7400 y otro 7404
4. Una celda sumadora tiene tres entradas a , b y c (arrastre), y dos salidas s y c' (salida de arrastre)
escribir la tabla funcional de la celda sumadora
y obtener las expresiones algebraicas de las dos funciones de salida;
dibujar dichas dos funciones utilizando solamente puertas "o-exclusiva" y puertas "nand"
5. Escribir la tabla de la función $y = b \cdot a + \overline{b} \cdot (c \oplus a)$
y, a partir de dicha tabla, dibujar la forma de configurar esa función
con un multiplexor de 3 entradas: **c b a**
6. Escribir la tabla de la función “ser número primo” para números de 4 dígitos
y dibujar la forma de configurar esa función con un multiplexor de 3 entradas,
utilizando como entradas del multiplexor **c b a**
(para ello dividir la tabla en dos mitades, una para $d = 0$ y otra para $d = 1$);
repetir el dibujo utilizando como entradas del multiplexor **d c b**
(para ello dividir la tabla en dos mitades, una para $a = 0$ y otra para $a = 1$)
7. A partir de la tabla de la función $y = b \cdot a + \overline{b} \cdot (c \oplus a)$,
dibujar la forma de construirla con un decodificador de 3 entradas: **c b a**
8. En un mapa de Karnaugh de 4 variables, situar 4 “ceros” formando un cuadrado
y, luego, añadir un rectángulo de 2 “ceros” (que no se solapen, ni formen “línea” con los anteriores “ceros”);
escribir en forma lineal, la tabla de esta función de cuatro entradas:
dibujarla con puertas *Nand* e inversores y, también, con puertas *Nor* e inversores.

NOTAS:

1. El multiplexor 74151 tiene una entrada EN de habilitación, que ha de estar a **0** para que el multiplexor funcione (dicha entrada está negada y, por tanto, se activa con valor **0**). Tiene, asimismo, una salida suplementaria W, que no es sino la negada de la salida normal Y (es decir, de Y a W hay simplemente un inversor).
2. El decodificador 74138 presenta sus salidas negadas (normalmente están a **1** y, cuando una salida se activa, se pone a **0**). Tiene tres entradas de condicionamiento (o habilitación) G1, G2A y G2B que deben conectarse G1 a **1** y G2, ambas, a **0** (son entradas negadas).

Desarrollo de la práctica:

1.- Discriminador de números primos.

- Diseñar un circuito digital cuya salida sea "1" cuando la entrada sea un número binario de tres dígitos y dicho número sea primo; configurar dicho circuito con puertas *Nand* (74HC00) e inversores (74HC04), montarlo utilizando como entradas los conmutadores **P6 P5 P4** y comprobar su funcionamiento, observando la salida sobre uno de los diodos luminiscentes LED y la entrada sobre un visualizador de 7 segmentos.
- Ampliar el circuito digital anterior a números de 4 dígitos pero de forma que la entrada sea siempre una cifra decimal (número BCD de 0 a 9); entradas, los conmutadores **P7 P6 P5 P4** y observar la salida sobre un LED y la entrada sobre un visualizador de 7 segmentos.

2.- Otra función de 4 variables.

- Sea el siguiente conjunto de números: {0, 1, 5, 8, 9, 10, 11, 13, 15} y la función que indica si un número binario de 4 dígitos pertenece o no a dicho conjunto; construir la función, con puertas *Nor* e inversores. Comprobar su tabla funcional; entradas, los conmutadores **P7 P6 P5 P4** y salida sobre un LED.

3.- Comparador de un bit.

- Construir un comparador de dos números binarios de una sola cifra, generando las tres funciones ($=$, $>$, $<$) con puertas *Nand* (utilizando solamente un circuito integrado 74HC00) e inversores (un circuito integrado 74HC04), representando sus tres salidas (igual, mayor, menor) en los diodos LED.

4.- Célula sumadora.

- Configurar una celda sumadora (con sus tres entradas correspondientes a los dígitos de los dos operandos y al acarreo), utilizando puertas *Nand* y puertas o-exclusiva; visualizar sus salidas (resultado y arrastre) en los diodos LED y comprobar su funcionamiento.

5.- Funciones con multiplexores.

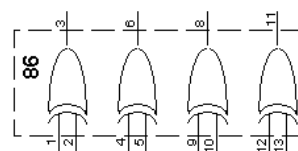
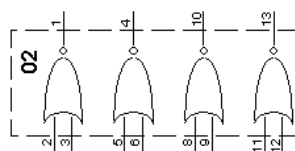
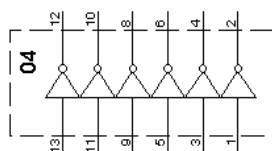
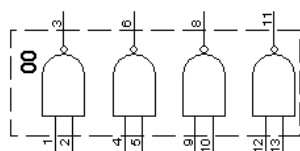
- Configurar y comprobar con el multiplexor de 8 canales (74151) la función: $y = b.a + \bar{b}.(c \oplus a)$.
- Configurar con dicho multiplexor (de 3 entradas) la función "ser número primo" para números binarios de 4 dígitos y comprobarla, observando la salida sobre un diodo LED y el vector de entrada sobre un visualizador de 7 segmentos.

6.- Funciones con decodificadores.

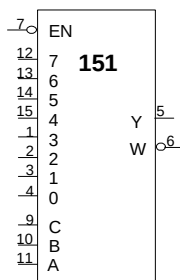
- Configurar con el decodificador de 8 líneas (74138, cuyas salidas están negadas) y una puerta *Nand* de 4 entradas la función: $y = b.a + \bar{b}.(c \oplus a)$ y comprobar su tabla funcional.

7.- Una función particular (si da tiempo).

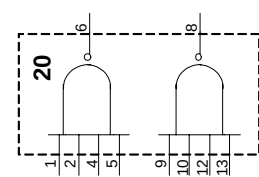
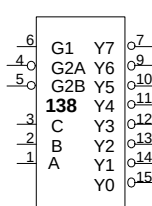
- En un mapa de Karnaugh de 4 variables, situar 4 "ceros" formando un cuadrado y, luego, añadir un rectángulo de 2 "ceros" (que no se solapen, ni formen "línea" con los anteriores "ceros"); llenar las casillas restantes de "unos". Escribir, en forma lineal, la tabla de esta función de cuatro entradas y construirla con puertas *Nand* e inversores; repetir asimismo la construcción con puertas *Nor* e inversores. Comprobar en ambos casos la tabla de la función.



Tensiones de alimentación: +5 V 14
0 V 7



Tensiones de alimentación: +5 V 16
0 V 8



alimentación: +5V 14
0V 7

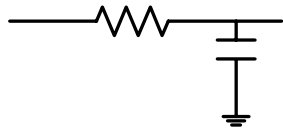
ELECTRÓNICA DIGITAL Práctica nº 3: Puertas lógicas con diodos y con interruptores.

Objetivos:

- “Tocar” y utilizar mínimamente diodos y transistores MOS para formar puertas lógicas.
- Montar en laboratorio y comprobar la operación de puertas "o" e "y" con diodos.
- Montar y comprobar la operación de inversores y puertas "nand", "nor" y "o-exclusiva" con transistores MOS.
- Comprobar la inversión de ondas por parte del inversor NMOS y estudiar su comportamiento en frecuencia.
- Medir el consumo de un inversor y compararlo con su cálculo circuital.
- Observar en la tensión de alimentación la producción de «ruido» causado por la conmutación.

Preparación de la práctica:

- 1.** Dibujar sendas puertas "o" e "y" con diodos
- 2.** Dibujar un inversor NMOS e indicar sobre el mismo los terminales del transistor: fuente, drenaje y puerta. (resistencia de polarización de 3K9 y tensión de alimentación de 10 V); calcular la intensidad que consume este inversor cuando su salida es **0** y cuando es **1**
- 3.** Dibujar (una debajo de otra) la entrada y salida de un inversor que recibe una onda cuadrada
- 4.** Dibujar la onda de salida del circuito de la figura, si la entrada es una onda cuadrada cuyos niveles son 0 y 10 V



dibujar cómo cambia dicha onda de salida si cambiamos la resistencia por otra 10 veces menor

- 5.** Calcular la intensidad que consume un condensador de 10 pF al cargarse y descargarse a una tensión de 10 V si lo hace a través de una resistencia con una onda cuadrada de: 500 KHz, 1 MHz, 1,5 MHz, 2 MHz
- 6.** Dibujar puertas NMOS de tipo "nand", "nor" y "o-exclusiva" y escribir al lado de las mismas la tabla de operación de cada puerta

NOTAS:

- 1. Compruébese, por favor, el valor de cada componente al cogerlo y al dejarlo en su cajita. Se solicita la colaboración de todos para que ningún componente esté en una caja que no le corresponda. Aunque ello suponga un cierto esfuerzo y una mayor dedicación de tiempo, agradeceremos mucho el esfuerzo por reordenar lo que otros han dejado en mal lugar: es la única forma de que el laboratorio pueda ser utilizado con eficacia.*
- 2. Los transistores MOS son muy "frágiles" frente a las cargas eléctricas. Procurar no tocarlos con las manos. Es probable que algunos de ellos se encuentren deteriorados: en caso de no conmutar adecuadamente, pruébese el circuito con otros ejemplares.*
- 3. Montar los transistores MOS insertando sus tres terminales en tres líneas seguidas de la placa de montaje, sin doblar ni deformar los terminales de los transistores; complétese adecuadamente el circuito con cablecitos cortos.*
- 4. Caso de que las ondas de salida presenten oscilaciones apreciables o ruido, filtrar la alimentación mediante un condensador de 470 nF (condensadores de terminales axiales que se insertan fácilmente en las líneas de alimentación a ambos lados de la placa de montaje).*
- 5. La medida de intensidades se efectúa por una entrada específica del polímetro (mA); después de efectuar tal medida conviene no dejar el terminal del polímetro conectado a dicha entrada, porque la medida de tensiones a través de ella puede ser destructiva (intensidad infinita).*

Desarrollo de la práctica:

Para la realización de esta práctica, utilícese una tensión de alimentación de 10 V y como niveles de tensión booleanos 0 y 10 V.

Las salidas se observarán sobre el osciloscopio.

[No conectar las salidas a los diodos LED] .

Diodos 1N4148

Transistor MOS: BS170 NMOS

Leer nota 1

1.- Puertas con diodos.

- Montar y comprobar el funcionamiento de una puerta "o" y de otra "y", construidas con diodos (resistencia de polarización de 10K).

2.- El transistor MOS como interruptor (inversor). Ver notas 2, 3 y 4

- Comprobar el funcionamiento como interruptor de un transistor NMOS en un inversor, con una resistencia de polarización de 3K9. Dibujar el circuito que corresponde a este inversor.
- Con una onda binaria de 1 KHz de frecuencia (utilizar la salida TTL del generador de señales, que proporciona una onda cuadrada entre 0 y 4 V), observar su comportamiento como inversor.
- Observar la conmutación para frecuencias altas: aumentar sucesivamente la frecuencia a 100 KHz, 500 KHz, 1 MHz y 2 MHz.
- Disminuir la resistencia de polarización del primer inversor (a 390 Ω) y observar de nuevo la conmutación para frecuencias altas.
- Observar (con la resistencia de 390 Ω y una señal de 10 KHz) el «ruido» producido por la conmutación del inversor sobre la tensión de alimentación; para ello, conectar el osciloscopio sobre la tensión de alimentación, sin condensador de filtrado, eliminar la componente continua y ampliar convenientemente. Medir la amplitud de los «picos de ruido». Colocar un condensador de 47 nF (axial) en paralelo con la alimentación y observar su efecto sobre el « ruido».

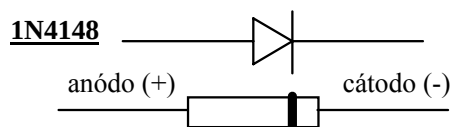
3.- Consumo del inversor. Ver nota 5

Effectuar estas medidas en el circuito anterior, con la resistencia de polarización de 3K9, situando el miliamperímetro entre la alimentación y la resistencia de polarización. [El osciloscopio no debe estar conectado al circuito.]. Colocar un condensador de 470 nF en paralelo con la alimentación, entre +10 V (después del miliamperímetro, junto a la resistencia de polarización) y 0V (la fuente del transistor).

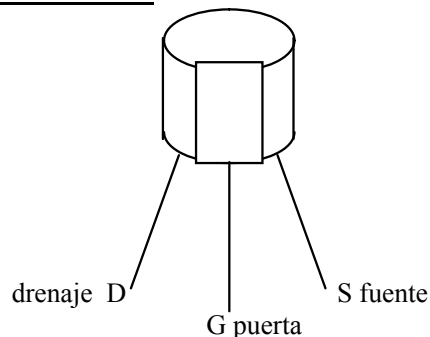
- Medir, con el polímetro, el consumo de este inversor (medir la intensidad que proporciona la fuente de alimentación) para cada uno de los dos valores booleanos: 0 y 1. Calcular el consumo de intensidad en el circuito correspondiente para cada valor booleano y comparar el resultado con el valor medido .
- Medir también el consumo en la entrada del inversor para cada uno de los dos valores booleanos. Comparar la medida con el resultado de calcular la intensidad de entrada en el circuito correspondiente.
- Medir el consumo dinámico de este inversor para frecuencias altas (500 KHz, 1 MHz, 1,5 MHz y 2 MHz). Para obtener el consumo dinámico del inversor, es preciso restar al consumo total el consumo estático promedio (dicho consumo estático puede medirse directamente mediante una señal de frecuencia baja, por ejemplo 1 KHz, en cuyo caso el consumo dinámico es despreciable).

4.- Puertas con interruptores: Lógica NMOS.

- Montar con transistores NMOS una puerta Nand de dos entradas (resistencia de polarización de 15K) y comprobar su tabla de operación.
- Hacer lo mismo con una puerta Nor de dos entradas .
- Montar la función o-exclusiva con 5 transistores y comprobar su "tabla de verdad" (función "ser diferentes").



BS170 vista frontal



ELECTRÓNICA DIGITAL Práctica nº 4: Bloques Combinacionales.

Objetivos:

- “Tocar” y utilizar los bloques integrados digitales.
- Montar en laboratorio y comprobar el funcionamiento de los bloques integrados más usuales.
- Montar y comprobar un pequeño sistema digital de “dejar pasar” el mayor / el menor de dos números.

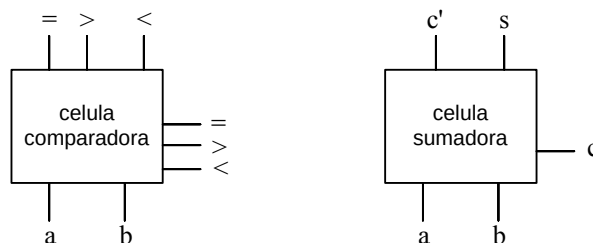
Preparación de la práctica:

Siempre es bueno y conveniente y saludable (más aún en caso de dudas)

consultar las hojas de características del circuito integrado

[un resumen útil de las mismas se encuentra al final de este cuadernillo]

- 1.** Escribir la tabla funcional de un decodificador de 4 entradas
entradas **d c b a** → salidas **Y0Y15**;
repetir dicha tabla para un decodificador de 10 salidas (BCD) con las salidas negadas:
entradas **D C B A** → salidas **Y0 Y1 Y2 Y3 Y4 Y5 Y6 Y7 Y8 Y9**
- 2.** Escribir la tabla funcional de un multiplexor de 8 líneas (3 entradas de control)
entradas **c b a** → salida **Y**
- 3.** Escribir la tabla funcional de un conversor de BCD a 7 segmentos de ánodo común:
entradas **D C B A** → salidas **a b c d e f g**;
el conversor tiene una entrada de “test de lámparas” LT (se activa con valor **0**),
¿qué valor tendrán las salidas cuando se activa esa entrada?
- 4.** El conversor BCD a 7 segmentos tiene una entrada RBI y una salida RBO
para anular la iluminación de “ceros” no significativos (ambas activas con valor **0**);
dibuja 4 conversores seguidos, para representar un número de 4 dígitos decimales,
de forma que los “ceros” no significativos no se “enciendan”
(no debe representarse 0065 sino 65; tampoco debe representarse 0000 sino 0)
- 5.** Un comparador de 4 dígitos está compuesto por 4 celdas comparadoras como la de la figura siguiente
dibuja la configuración interna (la conexión de celdas) del comparador de 4 bits;
¿a qué valores es necesario conectar las entradas = > < de la primera celda y por qué?



- 6.** Un sumador de 4 dígitos está compuesto por 4 celdas sumadoras como la de la figura anterior
dibuja la configuración interna (la conexión de celdas) del sumador de 4 bits;
¿a qué valor es necesario conectar la entrada de arrastre de la primera celda y por qué?
- 7.** Dibujar un circuito digital que “deje pasar” el mayor de dos números binarios de cuatro dígitos;
ampliar ese circuito, añadiéndole una entrada C,
de forma que cuando C = **0** “deje pasar” el mayor de los dos números
y cuando C = **1** “deje pasar” el menor de ellos

Desarrollo de la práctica:

1.- Decodificadores.

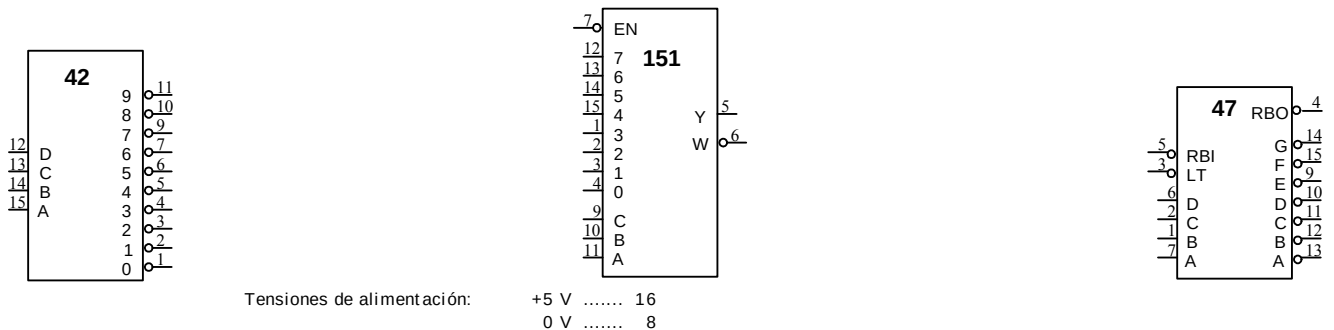
- Verificar el funcionamiento del decodificador de 10 líneas (7442), visualizando sus salidas sobre diodos LED (además de los 8 diodos LED disponibles en el entrenador será necesario montar otros 2, cada uno de ellos con una resistencia de 180Ω en serie).

2.- Multiplexores.

- Verificar el funcionamiento del multiplexor de 8 canales (74151), representando su salida sobre un diodo LED; conectar en sus entradas la palabra binaria correspondiente al número 210.

3.-Codificadores: Conversor BCD a 7 segmentos.

- Comprobar el funcionamiento del conversor BCD a 7 segmentos (7447), conectando sus salidas a un visualizador de 7 segmentos de ánodo común (limitar mediante resistencias la intensidad de cada segmento a un máximo de 12 mA, sabiendo que la tensión LED es aproximadamente 2 V y la tensión ON de salida del integrado es del orden de 0,5 V).
- En el conversor anterior, verificar el funcionamiento de las entradas de test de lamparas.
- Verificar, también, la anulación de ceros no significativos (entrada RBI) y observar el funcionamiento de la salida correspondiente a dicha anulación (RBO).



4.- Bloques aritméticos.

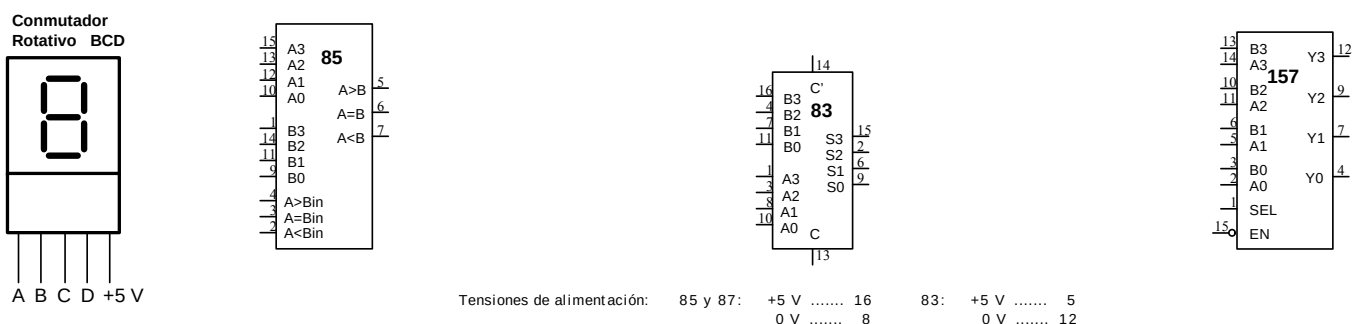
En los siguientes apartados se van a utilizar como entradas dos números de 4 bits; uno de dichos números se fijará con los conmutadores **P7 P6 P5 P4** y el otro con un conmutador rotativo BCD (dicho conmutador se alimentará a +5 V y se añadirán 4 resistencias de $1K5$ para la referencia a 0 V de sus 4 salidas); las salidas de los bloques se representarán sobre diodos LED.

Comparador integrado de números de cuatro bits:

- Comprobar el funcionamiento del comparador de números binarios de cuatro dígitos (7485).

Sumador integrado de números de cuatro bits:

- Verificar el funcionamiento del sumador de números binarios de cuatro dígitos (7483).



5.- Multiplexor de dos buses de cuatro líneas:

- Comprobar el funcionamiento del multiplexor de buses de cuatro líneas (74157).
- Configurar un circuito que “deje pasar” el mayor de dos números de cuatro bits.
- Ampliar este último circuito, añadiéndole una entrada C de forma que cuando $C = 0$ “deje pasar” el mayor de dos números de cuatro bits y cuando $C = 1$ “deje pasar” el menor de los dos números.

ELECTRÓNICA DIGITAL Práctica nº 5: Familia Lógica CMOS.

La primera clase del mes de enero se dedicará a comentar y razonar esta práctica.

Objetivos:

- Experimentar con detenimiento las características de la familia **CMOS 74HC**.
- Representar en osciloscopio la función de transferencia de un inversor.
- De dicha función de transferencia obtener los datos de tensiones y los márgenes de ruido.
- Medir las resistencias de salida de un inversor y comparar su valor con el que se obtiene de los datos del fabricante.
- Calcular la intensidad de salida que puede proporcionar y medirla .
- Medir, en forma aproximada, el tiempo de propagación.
- Medir los consumos estáticos y el consumo dinámico y comparar esos resultados con los teóricos.
- Observar las series HCT y SCHMITT.

Preparación de la práctica:

- 1.** Dibuja la función de transferencia de un inversor CMOS, supuesta que la conmutación se produce entre los 2,2 y los 2,6 V (alimentación de 5 V)
- 2.** Dibuja el circuito necesario para representar dicha función de transferencia en el osciloscopio
- 3.** De la función de transferencia, extraer los valores de V_{OL} , V_{OH} , $V_{iLmáx}$, $V_{iHmín}$, $\Delta V(0)$ y $\Delta V(1)$
- 4.** Dibuja los circuitos necesarios para medir las resistencias de salida; ¿cuál será el valor de dichas resistencias si para $I_o = 10$ mA las tensiones de salida son 0,4 V y 4,4 V?, ¿cuánto valdrá, en ese caso, $\Delta P(0)$ y $\Delta P(1)$?
- 5.** Con el valor de las resistencias de salida del apartado anterior, ¿qué intensidad podrá proporcionar la salida a **0** si la tensión de salida debe ser $V_{OL} < 1$ V?, ¿qué intensidad podrá proporcionar la salida a **1** si la tensión de salida debe ser $V_{OH} > 4$ V?
- 6.** Dibuja el circuito adecuado para medir el tiempo de propagación de un inversor (74HC04), ¿de qué orden de magnitud será dicho tiempo de propagación?
- 7.** Dibuja el circuito adecuado para medir los consumos estáticos de un inversor (74HC04), ¿qué valor debe darnos esta medida?
- 8.** Dibuja el circuito adecuado para medir el consumo dinámico de un inversor (74HC04); supuesto $C_{pd} = 20$ pF, ¿qué valor debe darnos la medida del consumo diámico a las siguientes frecuencias: 500 KHz, 1MHz, 1,5 MHz, 2 MHz?
- 9.** Dibuja la función de transferencia de un inversor con entrada SCHMITT cuyas tensiones de conmutación sean 2,5 V y 3,5V

Desarrollo de la práctica:

Se adjuntan hojas para representar las funciones de transferencia y anotar los resultados de las medidas.

1.- Funciones de transferencia.

Se utiliza el C. I. **74HC04** que contiene 6 inversores y para eliminar el "ruido" generado en la conmutación, se filtra la tensión de alimentación con un condensador de 47 nF (axial) en paralelo con ella y la entrada con otro de 10 nF (también en paralelo); ambos lo más próximos posible a los terminales del circuito integrado.

- Fijar en el generador de señales una onda triangular de 200 Hz, unipolar (es decir, toda ella positiva) que recorra el intervalo de tensiones de 0 a 5 V; excitando con dicha onda de entrada uno de los 6 inversores del circuito integrado **74HC04** y, utilizando la representación XY del osciloscopio, observar y dibujar la función de transferencia, **Vo-Vi**.

- A partir de la gráfica anterior expresar en el diagrama salida-entrada las tensiones **VoL**, **VoH**, **ViLmáx**, **ViHmín** y calcular los márgenes de ruido en tensión.

2.- Resistencia de salida y margen de ruido en potencia.

*[Quitar los dos condensadores de filtrado la entrada del inversor para desarrollar los apartados 2 al 5].
[A partir de aquí no se utiliza la representación XY del osciloscopio, ni la señal triangular de entrada].*

- Medir la resistencia de salida para cada uno de los dos estados booleanos, conectando para ello una resistencia de 220Ω en la salida ($R_o = \Delta V_o / I_o$), y calcular los márgenes de ruido en potencia.

Nota: La resistencia de salida no se mide sobre la función de transferencia sino conectando la entrada al correspondiente valor booleano y midiendo con el osciloscopio en la situación habitual V-t.

3.- Intensidad máxima de salida I_o .

- Con la entrada del inversor a "0" y conectando como resistencia de carga R_L un potenciómetro de 1K, observar la variación de la tensión de salida al variar la resistencia de carga; medir la máxima intensidad que suministra dicha puerta manteniendo $V_o(1) > 4 \text{ V}$ ($\Delta V_o < 1 \text{ V}$).

- Con la entrada del inversor a "1" y conectando la resistencia de carga R_L entre la salida y la tensión de alimentación, repetir la observación anterior y medir la máxima intensidad para $V_o(0) < 1 \text{ V}$.

4.- Retardo de propagación.

- Conectando en serie los 6 inversores del circuito integrado (uno detrás de otro), excitando el conjunto con la señal TTL del generador, a 1 MHz de frecuencia y utilizando la amplificación x10 de la escala horizontal del osciloscopio (en la división temporal de 0,5 μs), observar las ondas de entrada y de salida y (midiendo el retraso entre puntos medios de dichas ondas) deducir el orden de magnitud de los tiempos de propagación.

5.- Consumo.

- Con los 6 inversores en serie (uno detrás de otro, igual que en el apartado anterior) comprobar que el consumo estático es nulo; para ello, medir el consumo (en la alimentación del integrado) conectando a la entrada del primer inversor una señal TTL de baja frecuencia (por ejemplo, 100 HZ).

- Conectando dicha entrada (del primer inversor, con el resto en serie) a una señal de alta frecuencia, medir el consumo dinámico de las puertas para frecuencias de 500 KHz, 1MHz y 2 MHz. *Cuidar, en este caso, de que el osciloscopio no esté conectado al circuito (ni tampoco ningún otro condensador).*

6.- Serie 74HCT (compatible con TTL).

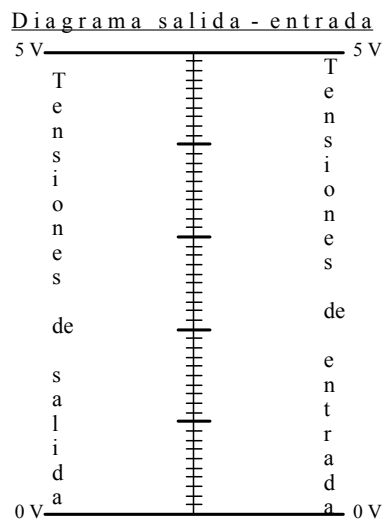
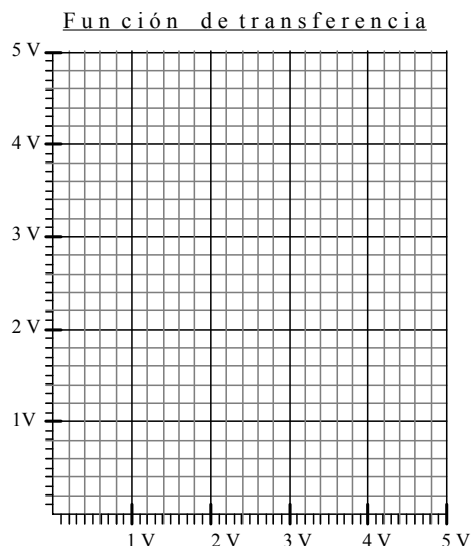
- Utilizando de nuevo la representación XY del osciloscopio, con el circuito integrado **74HCT04**, observar y dibujar la función de transferencia, **Vo-Vi**; expresar en el diagrama salida-entrada las tensiones **VoL**, **VoH**, **ViLmáx**, **ViHmín** y calcular los márgenes de ruido en tensión.

5.- Entrada Schmitt.

- Representar la función de transferencia del inversor con entrada Schmitt **74HC14** y medir las dos tensiones de conmutación de dicha entrada; cumplimentar, asimismo, el diagrama salida-entrada.

Familia Lógica: CMOS **Serie:** 74HC .

Función de transferencia.



Márgenes de ruido.

Márgenes de ruido en tensión:

$$V_{oL} = \quad V \qquad V_{oH} = \quad V$$

$$V_{iLm\acute{a}x} = \quad V \qquad V_{iHm\acute{i}n} = \quad V$$

$\Delta V(0) = \quad V$	$\Delta V(1) = \quad V$
-------------------------	-------------------------

Resistencia de salida:

$$R_o(0) = \quad \Omega \qquad R_o(1) = \quad \Omega$$

Márgenes de ruido en potencia:

$\Delta P(0) = \quad mW$	$\Delta P(1) = \quad mW$
--------------------------	--------------------------

Intensidad máxima de salida I_o .

$I_{om\acute{a}x}(0) = \quad mA$	$I_{om\acute{a}x}(1) = \quad mA$
----------------------------------	----------------------------------

Retardo de propagación: de los 6 inversores:

$$t_p \approx \quad ns$$

para 1 inversor $t_p \approx$	ns
-------------------------------	----

Consumo.

Consumo estático (consumo promedio a baja frecuencia: $I_{cc} = \quad mA$

Consumo dinámico:

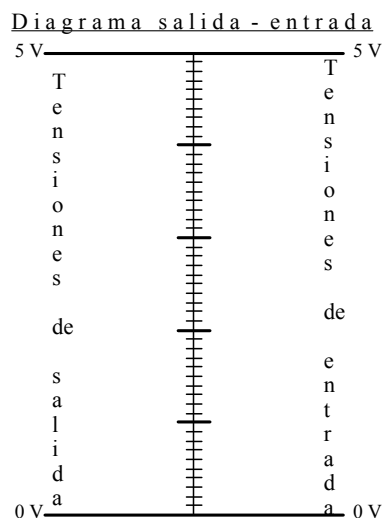
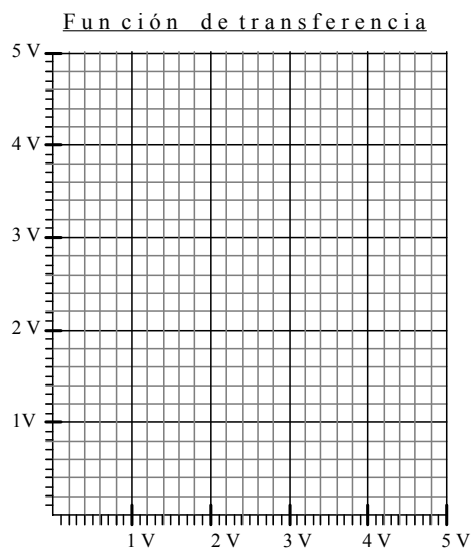
$I_{din\acute{a}mica}(500KHz) =$	mA
----------------------------------	----

$I_{din\acute{a}mica}(1\text{ MHz}) =$	mA
--	----

$I_{din\acute{a}mica}(2\text{ MHz}) =$	mA
--	----

Familia Lógica: CMOS Serie: 74HCT

Función de transferencia.



Márgenes de ruido.

Márgenes de ruido en tensión:

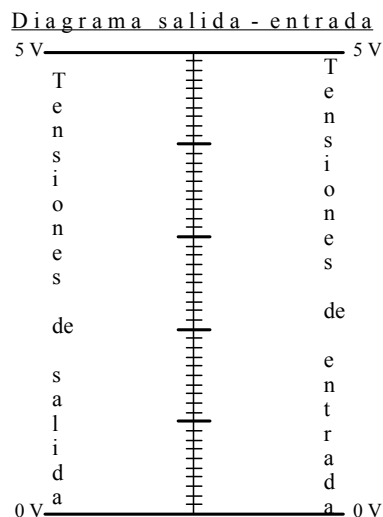
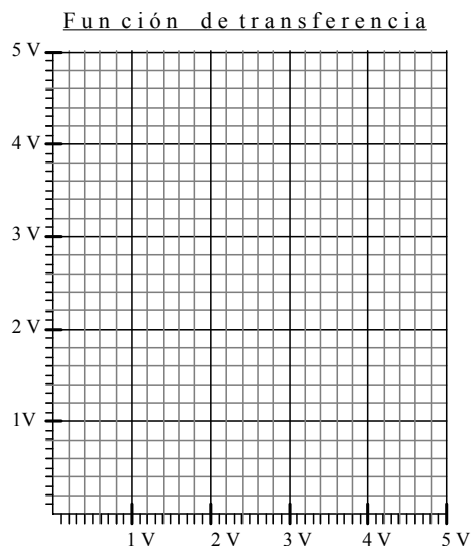
$$V_{oL} = \quad V \qquad V_{oH} = \quad V$$

$$V_{iLmáx} = \quad V \qquad V_{iHmín} = \quad V$$

$\Delta V(0) = \quad V$	$\Delta V(1) = \quad V$
-------------------------	-------------------------

Familia Lógica: CMOS Serie: 74HC entrada Schmitt

Función de transferencia.



Márgenes de ruido.

Márgenes de ruido en tensión:

$$V_{oL} = \quad V \qquad V_{oH} = \quad V$$

$$V_{iLmáx} = \quad V \qquad V_{iHmín} = \quad V$$

$\Delta V(0) = \quad V$	$\Delta V(1) = \quad V$
-------------------------	-------------------------



Electrónica

Sistemas Combinacionales
Sistemas Secuenciales
Microelectrónica

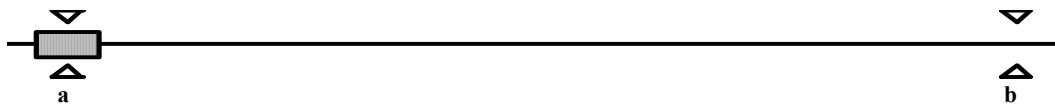
Respecto a los siguientes sistemas secuenciales se pide dibujar el grafo de estados del correspondiente sistema de control, codificar los estados y escribir las funciones de activación de las salidas [*algunos de ellos están en los apuntes; no es cuestión de copiarlos, sino de pensar el grafo que uno mismo propondría*]:

- 1.- Un montacargas es controlado con tres pulsadores: subida **S**, bajada **B** y parada **P**; para modificar el sentido de la marcha es preciso activar previamente el pulsador de paro y no responde cuando se activan simultáneamente varios pulsadores. Sendos topes fin de carrera **Ti** y **Ts** le impiden continuar cuando alcanza los extremos del recorrido.
- 2.- Un motor es controlado mediante dos pulsadores **A** y **B** de forma que con **A** gira hacia la izquierda y continua girando al ser soltado y con **B** gira hacia la derecha y lo sigue haciendo después de soltarlo; el motor se detiene al pulsar conjuntamente **A** y **B**. Si mientras esta girando en un sentido se activa el pulsador correspondiente al giro en sentido contrario, el motor se para y cambia el sentido del giro al soltar dicho pulsador.
- 3.- Una sirena se activa con un pulsador **A** y se desconecta con el mismo pulsador: al pulsar por primera vez **A**, la sirena comienza a sonar cuando se deja de pulsar; en cambio, cuando la sirena esta sonando basta pulsar sobre **A** para hacerla callar.
- 4.- Un sistema electrónico dispone de tres pulsadores **a**, **b** y **c**, tales que si se presiona primero el **a**, luego el **b** y finalmente el **c**, se enciende una lampara; en cambio si se pulsan dos a la vez, o en orden diferente al indicado, la lampara no se activa. La lampara permanece encendida hasta que se presionan a la vez los tres pulsadores.
- 5.- Un deposito de agua dispone de dos bombas **A** y **B** para su llenado y de tres detectores de nivel **MAX**imo, **MED**io y **MIN**imo. Cuando el nivel cae por debajo del nivel medio entra en funcionamiento una de las bombas hasta que alcanza el máximo, por debajo del nivel mínimo actúan ambas bombas hasta el nivel medio y luego una sola de ellas hasta el máximo; para equilibrar el desgaste de las bombas, cuando funciona una sola, lo hacen alternativamente (es decir, que si la vez pasada lo ha hecho la bomba **A**, entrará en funcionamiento la **B** y viceversa).
- 6.- Un cronómetro utiliza un contador con entradas de **HAB**ilitación y de **BOR**rado. Se desea añadirle un circuito de control con dos pulsadores **P** y **Q** tal que el conteje se inicie al activar **P** y se detenga al pulsar **Q** y un segundo pulso de **Q** borra el contador; en cambio, una vez detenido el conteje, si se pulsa **P** se reanuda el mismo.
- 7.- Una planta fabrica dos tipos de piezas de longitud **L+a** y **L-a**, respectivamente, las cuales pasan por una cinta transportadora delante de dos células fotoeléctricas con una separación **L** entre ambas; el circuito que permita distinguir entre los dos tipos de piezas debe tener dos salidas, que se activarán al detectar una u otra de las piezas.
- 8.- Cierta mecanismo **M** se desplaza por un riel circular a partir de la posición **A**, en la cual existe un sensor que detecta la presencia del mismo. Cuando se activa un pulsador **P**, **M** inicia su movimiento al soltar **P** y da una vuelta en el sentido de las agujas del reloj y otra en sentido contrario, tras lo cual se detiene en **A**; si durante el movimiento se pulsa **P**, **M** se para y, al soltar, completa la vuelta que esta dando y se detiene en **A**.

Respecto a los siguientes sistemas secuenciales se pide dibujar el grafo de estados correspondiente como autómatas de Moore (un solo vector de salida en cada estado) y, después, simplificar en lo posible los estados y dibujar el grafo de estados como autómatas de Mealy (con el menor número de estados posible); escribir, en ambos casos, las funciones de activación de las salidas:

1.- Un determinado mecanismo se mueve a lo largo de un riel entre dos posiciones A y B que se detectan mediante sendos sensores **a** y **b**; el movimiento se controla mediante un pulsador **P** de la siguiente manera:

- cuando el mecanismo se encuentra en A y se activa **P**, se inicia el movimiento hacia B al soltar el pulsador **P**
- de la misma forma, cuando se encuentra en B se inicia el movimiento hacia A al dejar libre el pulsador **P** (una vez activado)
- cuando el mecanismo se encuentra entre A y B, si se pulsa **P** el móvil se detiene y al soltar **P** continua su movimiento anterior.

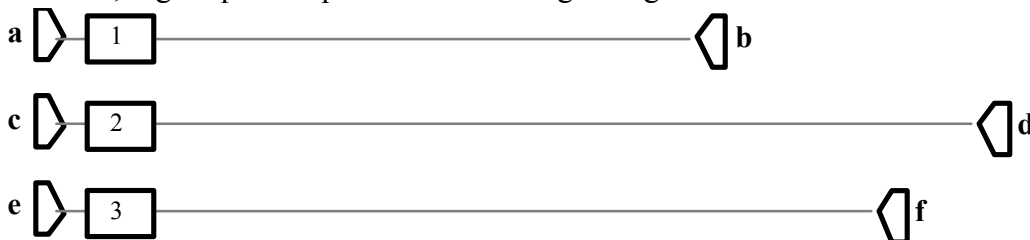


2.- Un polvorín tiene una entrada de seguridad con dos puertas sucesivas **a** y **b** de tipo persiana (cada puerta cuenta con los topes fin de carrera superiores **Supa Supb** e inferiores **Infra Infb** y dispone de un motor que puede ser accionado como subida **Sa Sb** y como bajada **Ba Bb**); el vigilante dispone de dos pulsadores, uno de ellos **Ent** para la operación de entrada y el otro **Sal** para la de salida. En la entrada la puerta **a** sube hasta alcanzar el tope superior e inmediatamente baja y, cuando dicha puerta **a** llega abajo, se abre la puerta **b** hasta alcanzar el tope superior y, también, baja inmediatamente; en la salida el ciclo de operación es análogo pero primero se abre la puerta **b** y luego la puerta **a**.

3.- Un motor puede girar en ambas direcciones y su movimiento será controlado por dos pulsadores **a** y **b** en la forma siguiente:

- al pulsar **a** se detiene el motor (si es que estaba en movimiento) y, al soltar dicho pulsador, el motor se pone a girar en el sentido de las agujas del reloj;
- lo mismo sucede al pulsar **b**, pero al soltarlo, el motor girará en sentido contrario;
- la forma de detener el movimiento del motor es pulsar **a** y **b** a la vez, en cuyo caso el motor se para y, al soltar los pulsadores continuará parado.

4.- Consideremos 3 mecanismos preparados para realizar un movimiento lineal de ida y vuelta entre dos extremos, en los cuales existen sendos topes fin de carrera que detectan la presencia del mecanismo en ellos, según queda representado en la figura siguiente:



a,b,c,d,e,f: sensores "fin de carrera "

El movimiento de estos mecanismos debe ser el siguiente:

- al pulsar un interruptor **P**, el primero se mueve hacia **b** y, cuando llega al extremo, el segundo se desplaza hacia **d** y, luego, el tercero va hacia **f**
- al encontrarse todos en el extremo derecho se inicia el movimiento de vuelta, uno a uno; primero vuelve el tercero, luego el segundo y, por último, el primero
- si en la ida se pulsa **P** no sucede nada pero si en la vuelta se pulsa **P** se pasa al movimiento de ida (que se realiza en el orden descrito anteriormente); cuando los tres se encuentran en el extremo derecho, la vuelta no se inicia si está **P** pulsado.

Se trata de volver a los ejercicios de las dos hojas anteriores para completar su diseño:

Hoja de [2ºQ 1] Diseño secuencial

- 1.- Obtener las funciones de marcado y de borrado de los biestables a partir del propio grafo de estados.*
- 2.- Al igual que en el anterior, obtener las funciones de marcado y de borrado de los biestables a partir del propio grafo de estados.*
- 3.- Escribir la tabla de evolución de los estados y, a partir de ella obtener las funciones de evolución de las variables de estado sin utilizar biestables y, luego, obtener también las funciones de marcado y de borrado de los biestables a partir de dicha tabla; dibujar los dos circuitos resultantes.*
- 4.- Codificar el grafo de estados en código gray y obtener las funciones de marcado y de borrado de los biestables a partir del grafo.*
- 5.- Al igual que en el anterior, obtener las funciones de marcado y de borrado de los biestables a partir del propio grafo de estados.*
- 6.- Escribir la tabla de evolución de los estados y, a partir de ella obtener las funciones de marcado y de borrado de los biestables; dibujar el circuito resultante.*
- 7.- Igual que en el anterior, obtener las funciones de marcado y de borrado de los biestables a partir de la tabla de evolución de los estados y dibujar el circuito.*
- 8.- Puede resolverse con u grafo de 6 estados (una vez simplificado); obtener las funciones de marcado y de borrado de los biestables a partir de dicho grafo.*

Hoja de [2ºQ 2] Simplificación de estados

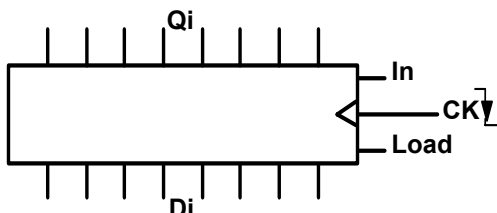
- 1, 2, 3 y 4.-** *Efectuar el diseño de los circuitos secuenciales correspondientes sobre el grafo de Mealy (con el menor numero de estados posible):*
- *obtener las funciones de activación de las salidas*
 - *y las funciones de evolución del estado (marcado y borrado de los biestables).*

- 1.- El biestable **AB** es de tipo síncrono y actúa con los flancos de bajada del reloj CK:
- cuando **B** = "1" se comporta como un biestable **D**, con entrada de dato por **A**
 - cuando **B** = "0" equivale a un biestable **T'** (el biestable conmuta –cambia de estado- en cada pulso de reloj cuando **A** = "1" y no cambia cuando **A** = "0").

Diseñar este biestable síncrono AB utilizando para ello puertas *Nor* e inversores.

- 2.- Un registro síncrono dispone de una entrada de habilitación **E** y de una entrada de borrado **B** (el borrado también es síncrono); diseñar con puertas *Nand* los biestables necesarios para construirlo.

- 3.- La figura muestra un registro de desplazamiento, con carga paralela síncrona, es decir, cuando **Load** = 1 el registro carga (en forma síncrona) el dato presente en sus entradas **Di** y cuando **Load** = 0 actúa como registro de desplazamiento con entrada **In**. Diseñar uno de los biestables de este registro en tecnología CMOS (utilizando, en todo lo posible, puertas de transmisión).



- 4.- Un registro de desplazamiento de 5 bits **Q4 .. Q0** se encuentra inicialmente a 0 y la entrada de dicho registro se conecta a la salida de una puerta *Nor* cuyas entradas son **Q4** y **Q1**; ¿cuántos pulsos de reloj han de transcurrir para que el registro vuelva a su situación inicial? ¿cual será la secuencia repetitiva de valores booleanos que se obtiene en **Q4** al conectar el reloj a una onda cuadrada.

- 5.- Un registro de desplazamiento está construido con biestables cuyo tiempo de anticipación **t_s** es 50 ns y el de mantenimiento **t_h** 20 ns ¿cual es el valor mínimo del tiempo de propagación **t_p** que deben tener?; en caso de que **t_p** = 75 ns ¿cual es la máxima frecuencia de reloj que podrá alcanzar dicho registro?.

1.- Un sistema de azar simula el volteo de una moneda que conmuta alternativamente entre cara y cruz a la velocidad de 10.000 veces por segundo y dispone de dos pulsadores para efectuar apuestas: **A** cara y **B** cruz. Al activar uno de los pulsadores se detiene la moneda y, si su posición coincide con el pulsador, señala "premio"; en caso contrario, señala "fallo" y si se activan ambos pulsadores se inhiben mutuamente.

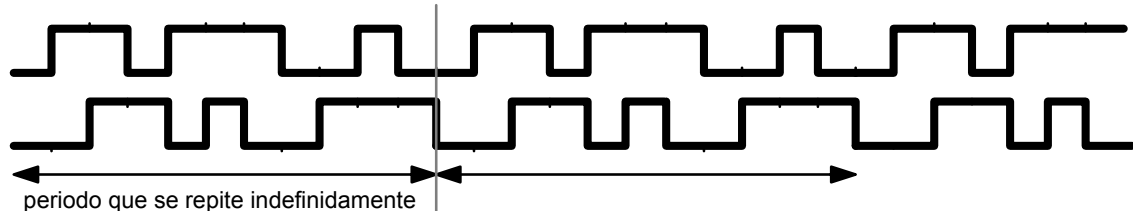
A) Dibujar el correspondiente grafo de estados, en la forma de autómeta de Moore.

B) Simplificar, en lo posible, dicho grafo y diseñar el sistema, utilizando biestables síncronos tipo **D**.

2.- Un sistema secuencial recibe números de dos bits y debe activar su única salida **Y** cuando ha recibido la secuencia **1 2 3** (cada paso de ella puede durar varias unidades de tiempo); la salida debe mantenerse en **1** hasta que se presente el número **0** en la entrada. Codificar el grafo de estados en código gray (exceptuando de la codificación gray las transiciones que llevan al reposo) y diseñar el correspondiente circuito secuencial, utilizando biestables síncronos tipo **D**.

3.- Sea un circuito síncrono con una sola entrada que recibe valores booleanos sincronizados con el reloj y con dos salidas; el circuito debe ser capaz de detectar y distinguir dos secuencias distintas **010** y **100**, señalando cada secuencia a través de una de sus salidas; se admite la posibilidad de solape. Dibujar solamente el grafo de estados y escribir las funciones de activación de las salidas

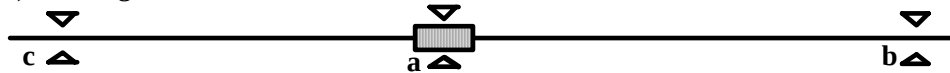
4.- Diseñar un sistema síncrono con dos de sus salidas que proporcionen repetitivamente las dos ondas de la figura, sincronizadas entre sí y con el reloj del sistema; efectuar el diseño de forma que pueda ser programado en un bloque **PLS** (PAL + biestables D).



5.- Diseñar, con biestables síncronos tipo **D**, un contador módulo 8, con una entrada de selección **A**, tal que si **A = 0** cuenta en binario y si **A = 1** cuenta en código Gray.

ELECTRONICA DIGITAL [2ºQ 6] Diseño secuencial síncrono III

1.- Diseñar el circuito de control necesario para un determinado mecanismo que se mueve a partir de una situación de reposo **A** y entre dos extremos **B** y **C** (posiciones **A**, **B** y **C** que se detectan mediante sendos sensores **a**, **b** y **c**) de la siguiente manera:



- cuando el mecanismo se encuentra en **A** y se activa **P**, se inicia el movimiento al soltar el pulsador y el mecanismo avanza hasta **B** y luego se dirige a **C** y vuelve a **A**
- si cuando el mecanismo está en movimiento se pulsa **P**, se dirige inmediatamente (antes de soltar **P**) hacia **A** por el camino más corto y se detiene
- puede suceder que llegue a **A** antes de haber soltado **P** y, en tal caso, debe evitarse que inicie un nuevo ciclo normal (**A** → **B** → **C** → **A**).

2.- Cierta mecanismo **M** se desplaza por un riel circular desde la posición **A**, en la que existe un sensor que detecta su presencia: cuando se activa un pulsador **P**, al soltarlo, **M** da una vuelta completa, pero cada vez da la vuelta en el sentido contrario a la anterior. Si mientras está dando la vuelta se activa **P**, se detiene y, al soltar, completa la vuelta en sentido contrario al que llevaba. Diseñar el circuito de control.

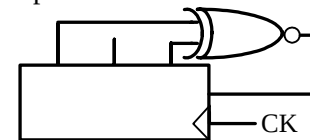
3.- Un determinado timbre se activa con un pulsador **P**, pero para evitar llamadas inoportunas, solamente suena al pulsar **P** tres veces (la tercera vez suena mientras se mantiene **P** pulsado). Diseñar el circuito de activación de dicho timbre.

4.- Se desea diseñar un detector de flancos especial que haga lo siguiente: cada vez que la señal de entrada presenta un flanco ascendente (subida) proporciona un pulso de 3 unidades de tiempo de reloj (**111**) y, si el flanco es descendente (bajada), genera dos pulsos de salida de 1 unidad de tiempo separados por 1 unidad de tiempo a **0** (**101**).

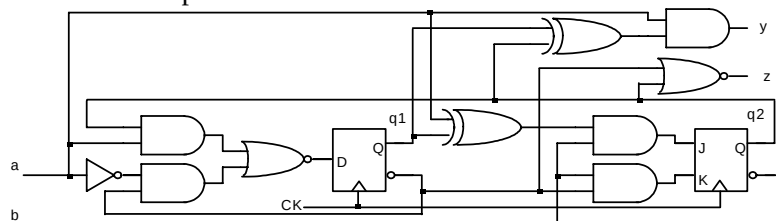
Los pulsos de salida (**111** y **101**) no se deben solapar ni juntar: si antes de completarse se recibe un nuevo flanco, es ignorado (no se genera la salida que le corresponde).

5.- a) Dibujar el grafo de estados de un registro de desplazamiento de 3 bits (desplazamiento hacia la izquierda).

b) Utilizando dicho registro de desplazamiento en la forma de la figura, dibujar la onda que se producirá en la salida más significativa del registro, si la frecuencia de la señal de reloj es de 140 KHz.



6.- Obtener el grafo de estados del circuito representado en la figura. ¿De qué tipo es: Moore o Mealy?. Dibujar el grafo que resultaría al cambiar de tipo de autómatas.



1.- Un registro de desplazamiento está construido con biestables cuyo tiempo de anticipación es de 50 ns y su tiempo de mantenimiento de 20 ns ¿cual es el valor mínimo del tiempo de propagación que deben tener tales biestables?; en caso de que su tiempo de propagación sea de 75 ns ¿cual es la máxima frecuencia de reloj que podrá alcanzar dicho registro?. ¿Qué tipo de herramienta detecta violaciones en los tiempos anteriores y cómo trabaja tal herramienta?.

2.- Los biestables de un registro de desplazamiento cuyos tiempos son: $t_s = 55$ ns, $t_p = 25$ ns y $t_h = 20$ ns, ¿cual es la máxima frecuencia de trabajo que podría alcanzar dicho registro?; ¿que pasará si cambiamos los biestables por otros con tiempo de mantenimiento de 35 ns?.

3.- Con biestables D síncronos, con tiempos de respuesta: $t_s = 50$ ns, $t_h = 25$ ns y $t_p = 15$ ns ¿cómo podría construirse un registro de desplazamiento de 6 bits que funcione correctamente?. ¿Cual será su velocidad máxima de trabajo?.

4.- Supongamos que los biestables D de nuestra librería de celdas tienen los siguientes tiempos: $t_p = 20$ ns, $t_s = 60$ ns, $t_h = 40$ ns.

a) Tales datos imponen una limitación inicial en la velocidad de trabajo de nuestros diseños secuenciales, ¿cual será el tope máximo de velocidad que se deduce de ellos?

b) ¿Qué otro aspecto temporal importante se deduce de estos datos?.

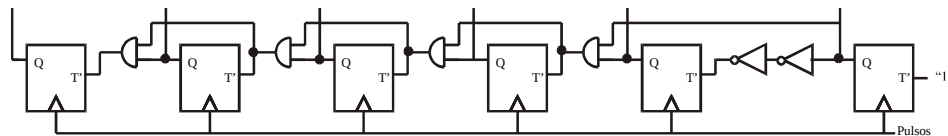
c) En caso de diseñar circuitos de muy alta velocidad es fácil que al resolver violaciones de permanencia (hold) se produzcan violaciones de anticipación (setup), ¿por qué? ¿qué será necesario hacer en tal caso?.

5.- Sea el circuito de la figura, cuyos componentes presentan los siguientes tiempos:

inversores $t_p = 2$ ns;

puertas "y" $t_p = 3$ ns;

biestables $t_p = 5$ ns, $t_s = 3$ ns, $t_h = 6$ ns.



a) ¿Cuál será la máxima frecuencia de pulsos que puede contar este contador?

b) Utilizando los mismos biestables, ¿cómo debe cambiarse el diseño para que pueda funcionar a mayores frecuencias?

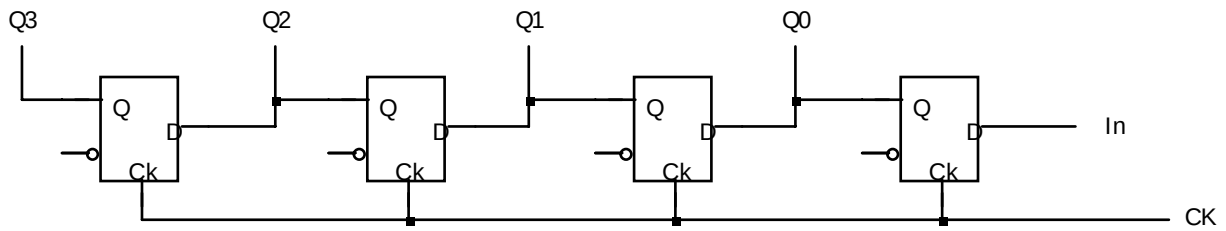
c) Dos inversores seguidos equivalen a un simple cable; ¿qué papel juegan los inversores en este circuito? ¿qué pasaría si los quitásemos?

d) ¿Cuántos caminos hay en el circuito?.

BIESTABLES: $t_s = 55 \text{ ns}$ $t_h = 20 \text{ ns}$ $t_p = 25 \text{ ns}$

Registro de desplazamiento

Calcular frecuencia máxima de reloj.



¿que pasaría con biestables cuyo tiempo de mantenimiento sea 35 ns?.

Contador década

tiempos de propagación:

inversor = 10 ns

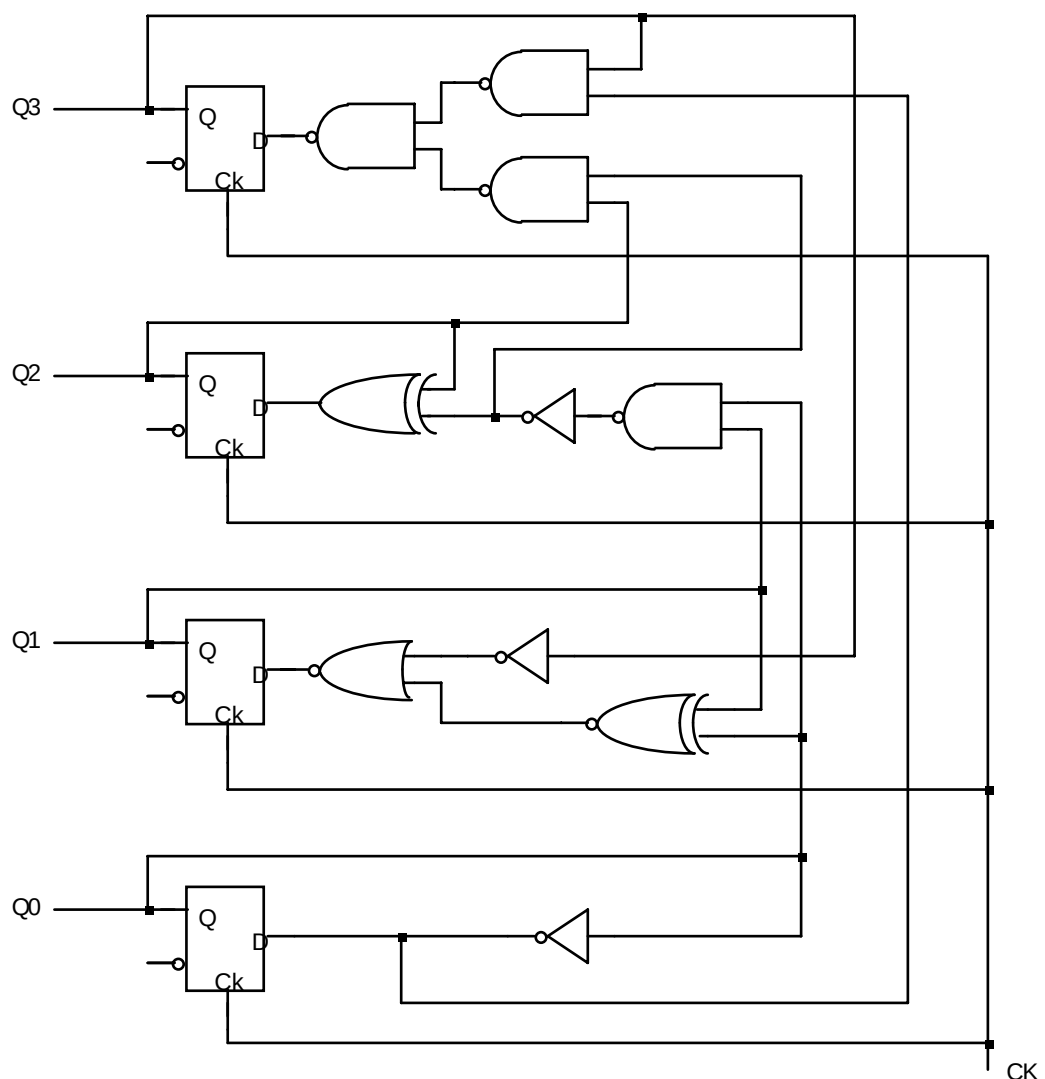
nand = 20 ns

o-exclusiva = 25 ns

nor = 15 ns

y-inclusiva = 30 ns

Calcular frecuencia máxima de reloj.



1.- Diseñar en esquema de bloques un cronometro para un tiempo máximo de 1 hora, con resolución de décimas de segundo y con las siguientes entradas de control: **B** = pulsador de borrado o puesta a cero; **M** = pulsador de puesta en marcha; **P** = pulsador de parada y **D** = pulsador de detención de la visualización (mientras se pulsa **D** el visualizador retiene la medida pero el cronometro continua su cuenta de tiempos; al dejar de pulsar **D** el visualizador representa directamente el conteaje).

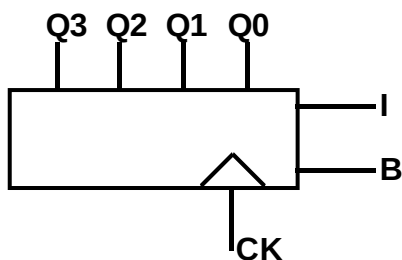
2.- Un circuito posee una única línea de entrada que recibe pulsos de diferente duración y presenta una sola línea de salida que debe situarse a **1** si la anchura del último pulso recibido es igual o mayor que 20 μ s y menor de 24 μ s, con una precisión de 0,5 μ s: la salida se actualiza cuando finaliza el pulso (flanco de bajada). Diseñar dicho circuito.

3.- Diseñar un cambio de escala que permita multiplicar por 5/12 el número de pulsos que se reciben (no utilizar ningún oscilador adicional).

4.- Una entrada suministra pulsos de 16 MHz y nos interesa disponer, además, de las frecuencias de 250 KHz y de 10 KHz. ¿Cómo obtenerlas si se dispone de contadores síncronos módulo 16 con entradas de habilitación y de borrado (síncrono)?.

5.- Utilizando contadores síncronos de 4 bits del tipo indicado en la figura, con entradas de inhibición de conteaje **I** y de borrado **B** (tal entrada de borrado actúa en forma síncrona con el reloj) y solamente las 4 salidas de sus biestables, diseñar un contador síncrono módulo 100 en sus dos versiones:

- que cuente en BCD
- que cuente en binario.



6.- Sea un contador binario módulo 13 con una entrada de habilitación del conteaje **E**.

- Si la señal en la entrada de pulsos es una onda cuadrada de 52 KHz de frecuencia, ¿cual será la frecuencia de las ondas que se observan en cada salida?.
- Con la misma onda cuadrada, ¿qué porcentaje de tiempo está en valor 1 cada una de las salidas del contador?.
- Caso de utilizar este contador en un sistema digital síncrono cuyo reloj sea de 10 MHz para contar los pulsos de una señal irregular de baja frecuencia, ¿cual será la forma de conectar dicha señal al contador?.

1.- Un sistema digital recibe pulsos de diferente duración, separados unos de otros, al menos, 0,1 ms y convierte cada pulso que recibe en otro de duración fija de 40 μ s. Diseñar el circuito necesario, de forma que sea síncrono, con un reloj de 1 MHz. Los pulsos de salida se producen a la vez que el de entrada y, para evitar espurios, no se tienen en cuenta los pulsos de entrada cuya duración sea inferior a 2 pulsos de reloj.

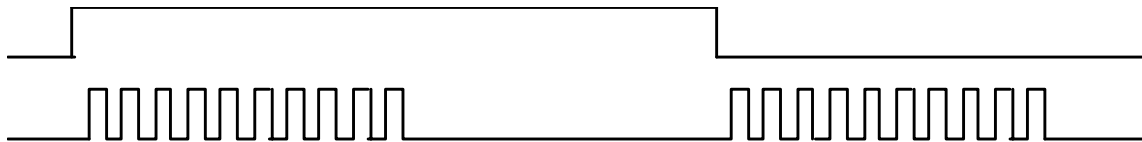
2.- Interesa disponer de un circuito con una entrada por la que recibirá pulsos de duración variable y una salida que debe avisar cuando los dos últimos pulsos recibidos sean iguales. El circuito será síncrono con un reloj de 4 MHz y el aviso (salida a 1) se producirá cuando finalice cada pulso hasta que llegue el nuevo pulso; pulsos de duración igual o superior a 50 μ s no son tenidos en cuenta. Dibujar el diagrama de bloques y explicar brevemente su funcionamiento.

3.- Un circuito digital tiene como misión duplicar (multiplicar por 2) la duración de los pulsos que recibe, pero solamente cuando la anchura del pulso se encuentra entre 20 y 80 μ s (pulsos inferiores a 20 μ s o superiores a 80 μ s son ignorados). La separación entre pulsos es siempre superior a 0,5 ms y el pulso de duración doble se produce al finalizar el pulso recibido; el circuito será síncrono con reloj de 2 MHz.

a) Dibujar el esquema de bloques necesario y explicar brevemente su funcionamiento.

c) ¿Cómo habría que modificar el diseño anterior si lo que se desea es que el circuito reduzca a la mitad (divida por 2) la duración de los pulsos que recibe? (al igual que antes, solamente de aquellos pulsos cuya anchura se encuentra entre 20 y 80 μ s).

4.- Diseñar un sistema síncrono con un reloj de 100 KHz para una señal de entrada de frecuencia variable cuyos semiperíodos son siempre de duración superior a 10 ms; en cada flanco de la señal de entrada la salida del sistema debe producir 10 pulsos de señal cuadrada de 2 KHz, según la figura.



5.- Un sistema síncrono con un reloj de 1 MHz recibe pulsos de diferente duración, separados, al menos, 50 μ s; el sistema mide la anchura del pulso recibido (n μ s) y, si n es inferior a 25 μ s, genera n pulsos (tantos como la propia duración del pulso de entrada) de 1 μ s, separados 1 μ s cada uno del siguiente; diseñar dicho sistema digital, en forma de diagrama de bloques.

1.- Diseñar en forma esquemática un frecuencímetro que actualice su visualizador cada 15 segundos, representando la frecuencia en pulsos por segundo. ¿Cómo se puede modificar para que el resultado apareciese en pulsos por minuto?.

2.- Un motor debe girar con un mínimo de 480 r.p.m. hasta un máximo de 540 (que no debe ser alcanzado); diseñar un circuito que vigile la velocidad del motor de forma que proporcione una señal de alarma cuando se encuentre fuera de dicho intervalo.

a) Diseñarlo para una alarma visual (señal luminosa) con tiempo de medida de 10 s.

b) Repetir el diseño: el circuito debe transmitir la señal de alarma a un microP. y el tiempo de medida de la velocidad debe ser lo menor posible.

3.- Una placa térmica dispone de un control de potencia on/off: la consigna de potencia es fijada por un mando circular con 10 niveles codificados en código Gray (0: **0000**, 1: **0001**, ..., 8: **1100** y 9: **1000**); el salto de potencia entre niveles debe ser del 10 %, salvo para el nivel 9 que debe proporcionar el máximo de potencia 100%.

a) Diseñar el circuito de control de potencia (ciclo on/off de 10 segundos).

b) Sea un sistema análogo para un horno, cuya temperatura se regula mediante un sensor lineal cuya salida va de 0 a 10 V y se prefija mediante un mando circular potenciométrico. El ciclo on/off consta de 64 unidades de 0,1 s y, al comienzo de cada ciclo, si la temperatura es inferior a la fijada se aumenta en una unidad el tiempo de "on" y cuando es superior se disminuye en una unidad. Diseñar dicho circuito.

6.- Sea un circuito digital que proporcione pulsos de amplitud modulada en la forma siguiente: un pulso cada milisegundo, el primero de ellos de 10 μ s y los siguientes con anchura creciente de 20 μ s, 30 μ s, ... (cada uno 10 μ s más largo que el anterior), hasta alcanzar los 800 μ s; luego los pulsos decrecerán (de 10 en 10 μ s) hasta reducirse a 10 μ s, momento en el cual volverán a crecer; y así sucesivamente.

a) Dibujar el diagrama de bloques suponiendo que los cálculos internos se efectúan en BCD y que cada bloque maneja, a lo sumo, una cifra BCD (4 bits).

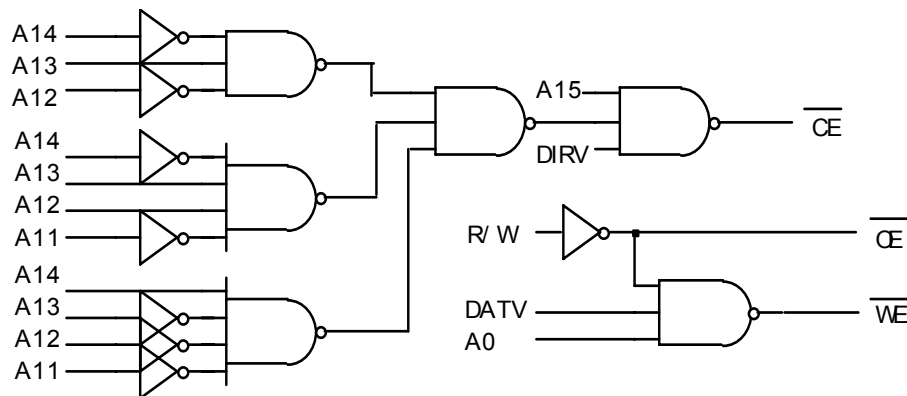
b) Si a la salida de este circuito se conecta un filtro pasa-baja (cuya frecuencia de corte sea del orden de 100 Hz), ¿cómo será la forma de la onda en la salida del filtro?

5.- Un amplio salón dispone de un sistema calefactor eléctrico y se desea realizar un control del mismo que permita controlar linealmente 100 niveles de potencia (que se representarán en un visualizador de 2 cifras decimales) con dos pulsadores, uno de subida **S** y otro de bajada **B**; al mantener presionado **S** la consigna de nivel de potencia debe aumentar a un ritmo de una unidad cada 0,2 s, mientras que el pulsador **B** la hace disminuir en la misma forma.

a) Dibujar el esquema de bloques, incluyendo los pulsadores y el visualizador.

b) ¿Cómo puede modificarse el diseño para que el circuito responda, no a los pulsadores **S** y **B**, sino a un termostato el cual dispone de un sensor de temperatura y de un divisor de tensión para fijar (en términos de tensión) la temperatura deseada?.

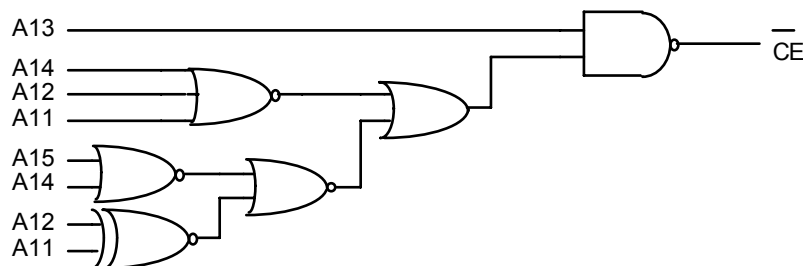
1.- ¿Qué posiciones de memoria ocupa un circuito integrado **RAM** de **8K** registros cuyas entradas de control sean las de la figura? ¿qué particularidad tiene respecto a las operaciones de lectura y de escritura? ¿cómo deben conectarse los terminales de direccionamiento **a_i** del circuito integrado?.



2.- Un mapa de memoria de 18 líneas de direccionamiento está ocupado inicialmente por una memoria RAM de 8K, situada al comienzo del mapa (desde la posición 0); a continuación de ella se desea colocar una segunda memoria RAM de 32K.

- ¿Cuál es el tamaño global del mapa de memoria y cuantas posiciones de memoria (registros) caben efectivamente en el mismo? .
- ¿Cuáles serán las direcciones del mapa que corresponden a la memoria de 32K?.
- Escribir la función booleana necesaria para la habilitación de dicha memoria.
- ¿Cómo deben conectarse sus líneas de direccionamiento si se desea conservar en el mapa el orden correlativo de los registros de la memoria? (es decir, el primer registro de la memoria sea el primero en aparecer en el mapa, el segundo sea el segundo, ...).

3.- ¿Qué posiciones del mapa de memoria ocupa un integrado **RAM** de **16K x 8** bits, cuya habilitación es la representada en la figura?. Detallar cómo deberán conectarse cada uno de los terminales de dicho circuito integrado. Para aprovechar la velocidad máxima de trabajo ¿qué condición tiene que cumplir el circuito de la figura?.



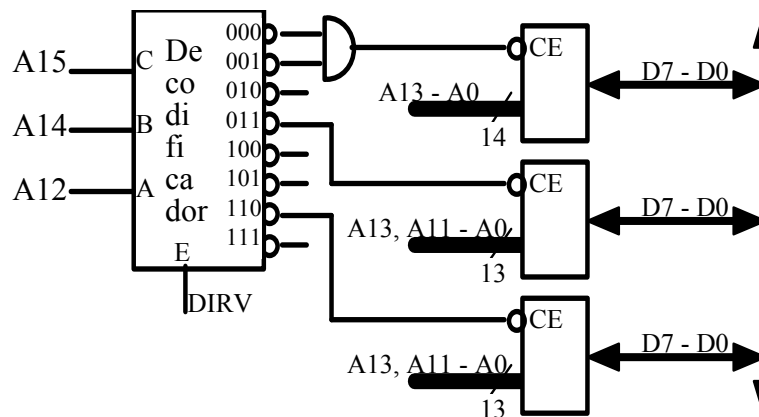
4.- Un bloque integrado **RAM** de **8K x 8** bits debe ser situado en las posiciones iniciales de un mapa de memoria correspondiente a un bus de direcciones de 16 líneas; sin embargo, las posiciones de memoria situadas entre la **0C00** y la **0FFF** se encuentran ya ocupadas por diversos dispositivos y también lo están las que van de la **1400** a la **1FFF**, de forma deberán "saltarse" (no ocupar) dichos sectores de memoria. Diseñar el circuito necesario para colocar el bloque en las zonas libres mas bajas posibles del mapa de memoria; indicar cómo deben conectarse todos sus terminales.

1.- Formando parte del espacio de memoria direccionado por un bus de 14 bits, se desea configurar un segmento de 12 Kbytes compuesto por 2K **RAM** bipolar, 2K **RAM** CMOS y 8K **ROM**, comenzando en la dirección **1000** y en el orden indicado. Se dispone para ello de circuitos integrados **RAM** bipolares de 1K, **RAM** CMOS de 2K y **ROM** de 4K. Especificar el mapa detallado de memoria de las distintas zonas y el circuito necesario para situar los integrados en dichas direcciones.

2.- Sea un procesador con 20 líneas en su bus de direcciones y una placa de memoria rápida **RAM** de **16K** construida con circuitos integrados de 2K **2018AP-45**. Diseñar la placa de memoria de forma que ocupe las posiciones **A8000 - ABFFF**.

3.- El circuito integrado **PIA** es un adaptador de periféricos para configurar 2 puertos paralelos; contiene 4 registros que suelen ocupar posiciones de memoria sucesivas. Se precisa utilizar 8 **PIAs** cuyo primer registro se encuentre, respectivamente en las siguientes posiciones de un mapa de memoria de 10 líneas: **030, 070, 0B0, 0F0, 230, 270, 2B0, 2F0**; diseñar el circuito necesario.

4.- Indicar el mapa de memoria que corresponde al circuito de la figura.

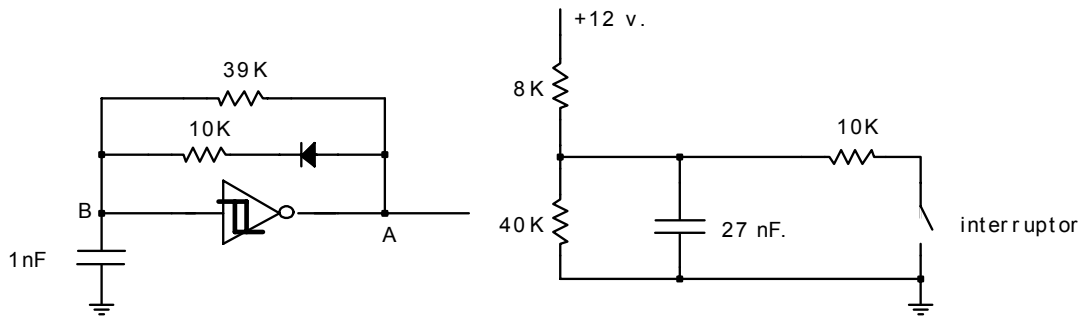


5.- Un procesador tiene 16 líneas en su bus de direcciones y el correspondiente mapa de memoria está ya ocupado en las 2K primeras posiciones y en las 4K últimas posiciones (debido a memoria RAM y ROM internas al procesador). Queremos ampliar la memoria disponible por el procesador colocando 2 circuitos RAM adicionales de 4K y uno ROM de 8 K, añadiendo también un adaptador de periféricos que contiene 32 registros. En nuestro diseño no van a ir en el mapa de memoria más pastillas que las antes indicadas.

- Dibujar el circuito necesario para colocar a los citados circuitos dentro del mapa de memoria, utilizando para posicionarlos un decodificador.
- Hacer una lista detallada del mapa de memoria indicando las posiciones que están ocupadas y quien las ocupa y las posiciones que quedan libres.

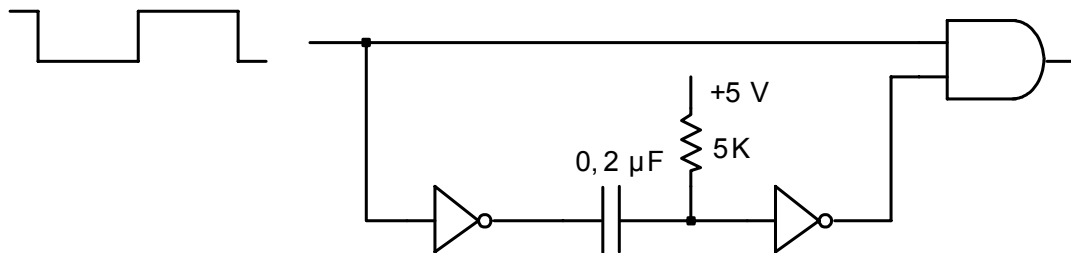
1.- En el primer circuito, dibujar las ondas en A y B y calcular sus semiperiodos.

Datos: $V_{OH} = 4\text{ V}$; $V_{iL} = 0$; $V_a = 1\text{ V}$; $V_b = 2\text{ V}$; el diodo puede suponerse ideal.



2.- En el segundo de los circuitos anteriores, supuesto que el interruptor se conmuta de forma que la tensión del condensador varíe entre 7 y 9 voltios. ¿Cuál será la frecuencia de la onda que actúa sobre dicho interruptor?.

3.- El circuito de la figura siguiente recibe pulsos de anchura variable, bastante separados entre sí. ¿Qué función realiza este circuito? ¿Cuál es la separación mínima entre pulsos necesaria para que el circuito realice dicha función sin errores?.

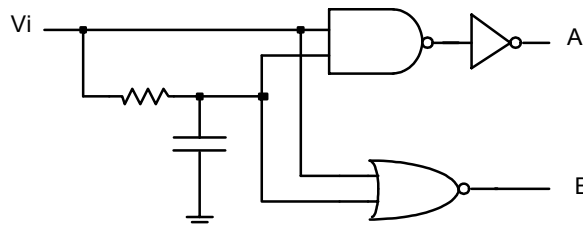


Datos: $V_{OH} = 5\text{ V}$; $V_{OL} = 0$; $V_i(\text{conmutación}) = 2\text{ V}$.

4.- En el circuito siguiente la señal de entrada V_i es cuadrada de frecuencia 1 KHz; dibújese correlativamente dicha onda de entrada y las de salida A y B, expresando numéricamente los intervalos de tiempo entre ellas.

Datos: $R = 5\text{ K}$; $C = 15\text{ nF}$; $V_{OH} = 5\text{ V}$; $V_{OL} = 0$;

las entradas de tipo Schmitt con tensiones de disparo 1 y 2,75 V.



5.- Un circuito recibe pulsos rectangulares (niveles +12 y -12 V), relativamente lentos (semiperiodos superiores a 1 ms). Se pide construir un "detector de flancos", de forma que cada vez que se recibe uno de los flancos ($\uparrow$ y $\downarrow$) genere un pulso digital de 35 μs . Diseñar el circuito utilizando puertas CMOS (tensión de conmutación $V_c = 2,5\text{ V}$).

ELECTRÓNICA DIGITAL Práctica nº 10: Aplicaciones de los contadores

Objetivos:

- Utilizar contadores integrados síncronos en aplicaciones de utilidad.
- Montar en laboratorio y comprobar la operación de un generador de palabras.
- Montar y comprobar un modulador de anchura de pulso: PWM.
- Utilizar el PWM anterior como selector de número de pulsos.
- Montar y comprobar la visualización dinámica de dos cifras decimales (base 10).
- Montar y comprobar un detector de flancos para contaje de pulsos diferentes a los del reloj.

Preparación de la práctica:

- 1.** Dibujar, en forma de esquema de bloques, el circuito de un generador de palabras de 8 bits
- 2.** Dibujar el esquema de conexiones necesario para utilizar DOBLECONTA como contador módulo 9
- 3.** Dibujar, en forma de esquema de bloques, el circuito de un modulador de anchura de pulsos PWM con diez niveles (de 0 a 9) y ciclo de 9 unidades
- 4.** Modificar el PWM anterior para convertirlo en un selector de pulsos que, de cada 10 pulsos que reciba, "deje pasar" m pulsos siendo m el número de consigna (de 0 a 9); dibujar el nuevo esquema de bloques resultante
- 5.** Dibujar el esquema de conexiones necesario para utilizar DOBLECONTA como contador BCD módulo 100
- 6.** Dibujar, en forma de esquema de bloques, el circuito necesario para la visualización dinámica de las dos cifras del contador anterior módulo 100
- 7.** Dibujar el circuito correspondiente a un detector de flancos de bajada

Nota: antes de comenzar esta práctica, es preferible completar primero la práctica nº 8 (caso de no haber hecho todos los apartados de ella en la sesión anterior) .

Desarrollo de la práctica:

Toda esta práctica se desarrolla con el control "up/down" C a 0 (contaje ascendente); para evitar perturbaciones que pudieran dar lugar a un contejo erróneo, filtrar levemente la entrada de pulsos del contador con un condensador de 100 pf, situado lo más cerca posible de la entrada de reloj y "tierra" del integrado.

1.- Generador de palabras.

- Conectando las salidas de uno de los dos contadores del circuito integrado **DOBLECONTA** a las entradas de control del multiplexor 74151, configurar un generador de palabras de 8 bits y comprobar su funcionamiento generando la siguiente forma de onda: **11010010**; comprobarlo en primer lugar pulso a pulso sobre un diodo LED y luego con un reloj de 10 KHz, observando la onda generada sobre el osciloscopio.

2.- Control on/off

- Con uno de los contadores del circuito integrado **DOBLECONTA** configurar directamente un contador módulo 9 y comprobar su funcionamiento: *Un contador módulo 9 divide el ciclo en nueve intervalos, numerados de 0 a 8; la duración de cada intervalo es $1/9 \approx 11\%$ del ciclo.*

- Añadiendo al contador anterior un comparador 7485 y utilizando como selector de niveles un conmutador rotativo BCD, montar un control on/off lineal de 10 niveles (es decir, 9 intervalos; a cada intervalo le corresponderá un porcentaje igual a $100/9 \approx 11\%$); utilizar como señal de reloj la salida TTL del generador de señales a 1 KHz. Comprobar su funcionamiento para cada uno de los niveles y medir en cada uno de ellos el cociente T_{on}/T_{ciclo} .

- Disminuyendo la frecuencia de reloj a 10 Hz y a 1 Hz observar los tiempos on/off sobre un diodo LED de la placa de montajes; observar el control de la iluminación del diodo LED utilizando como frecuencia de reloj 100 Hz.

b) Selectores de número de pulsos ("rate multipliers")

- En el mismo circuito anterior, modificar el contador de forma que sea módulo 10 y añadir a la salida del circuito una puerta "y" que controle el paso de los pulsos del propio reloj; comprobar el funcionamiento de este circuito como selector de número de pulsos: de cada 10 pulsos "deja pasar" el número indicado en el conmutador rotativo BCD. Para sincronizar la onda de los pulsos de salida, tomar con la otra sonda del osciloscopio la salida **máx** del contador y sincronizar con dicha señal.

3.- Visualización dinámica.

- Unir los dos contadores del circuito integrado **DOBLECONTA** configurando un contador BCD módulo 100; observando sus salidas sobre diodos LED, con pulsos manuales del pulsador P_0 comprobar su funcionamiento.

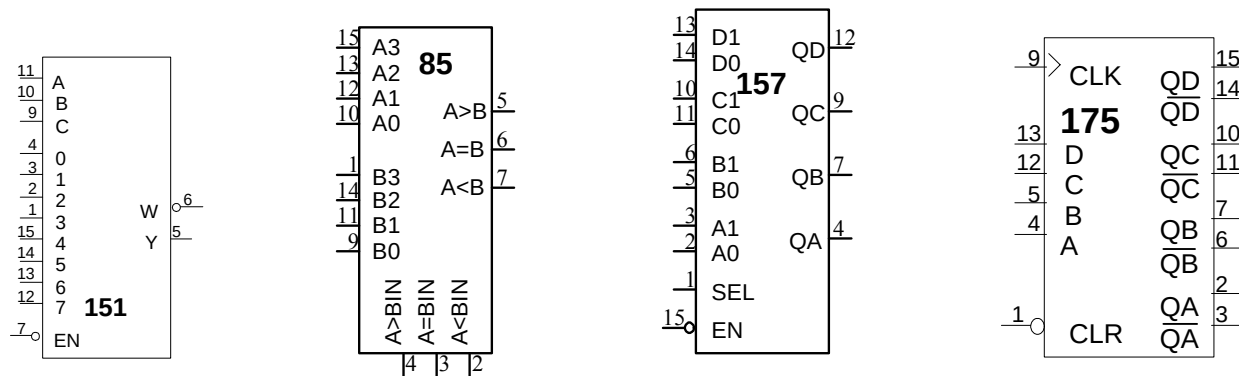
- Utilizando el multiplexor de 2 buses de 4 líneas 74157 y como onda de barrido la señal TTL del generador de señales a 100 Hz de frecuencia, representar la salida de dicho contador módulo 100 en los dos visualizadores de 7 segmentos de la placa de montaje; para controlar el cátodo común de cada uno de ellos con una intensidad relativamente alta, utilizar dos inversores TTL de la serie estandar **7404**. Comprobar la representación de los 100 primeros números decimales de dos cifras.

- Disminuyendo la frecuencia hasta 1 Hz observar la conmutación de las cifras propia de la representación dinámica.

b) Contaje de pulsos diferentes del reloj

- Añadiendo delante de la habilitación un detector de flancos y utilizando un reloj de 50 Hz (la misma señal de reloj para los contadores CK y como onda de barrido SCAN), comprobar el contejo de pulsos manuales generados con el pulsador P_7 .

- Con el mismo reloj de 50 Hz, conectar, después, la entrada de habilitación directamente al pulsador P_7 (desconectando el detector de flancos) para observar el contejo que se produce cuando se habilita el contador con los pulsos (P_7) que se desea contar.



ELECTRÓNICA DIGITAL Práctica nº 11 (opcional): Aplicaciones de los contadores II

Esta práctica opcional contiene circuitos de aplicación de los contadores, en forma de sistemas de control interesantes, útiles y de una cierta entidad. Resulta sumamente provechoso comprender, montar y comprobar estos circuitos como culminación de la asignatura de Electrónica Digital.

No queda tiempo para programar las correspondientes semanas de práctica; por ello, se establecerán unos días opcionales de prácticas para quienes tengan interés y tiempo (y, también, se ofrecerá la posibilidad de hacer esta práctica en las primeras semanas de la asignatura de Microelectrónica de tercer curso).

Objetivos:

- Utilizar contadores integrados síncronos en aplicaciones de utilidad y de una cierta entidad.
- Montar en laboratorio y comprobar la operación de un temporizador de luz de escalera.
- Montar y comprobar un generador de pulsos de anchura progresivamente creciente y decreciente.
- Montar y comprobar un detector-avisador de frecuencia máxima.
- Utilizar las salidas tri-estado para visualización dinámica.

Preparación de la práctica:

- 1.** Estudiar y comprender las especificaciones detalladas en los 4 enunciados del desarrollo de la práctica
- 2.** Estudiar en detalle y comprender el funcionamiento de los circuitos de las figuras

Está disponible en reprografía un pequeño cuadernillo de apuntes de aplicaciones de los contadores, cuyos ejemplos 1, 3 y 6 se corresponden, respectivamente, con los apartados 1, 2 y 4 de esta práctica.

Desarrollo de la práctica:

Para evitar perturbaciones que pudieran dar lugar a un conteo erróneo, filtrar levemente la entrada de pulsos del contador con un condensador de 100 pf, situado lo más cerca posible de las entrada de reloj y "tierra" del integrado.

1.- Control de iluminación de escalera

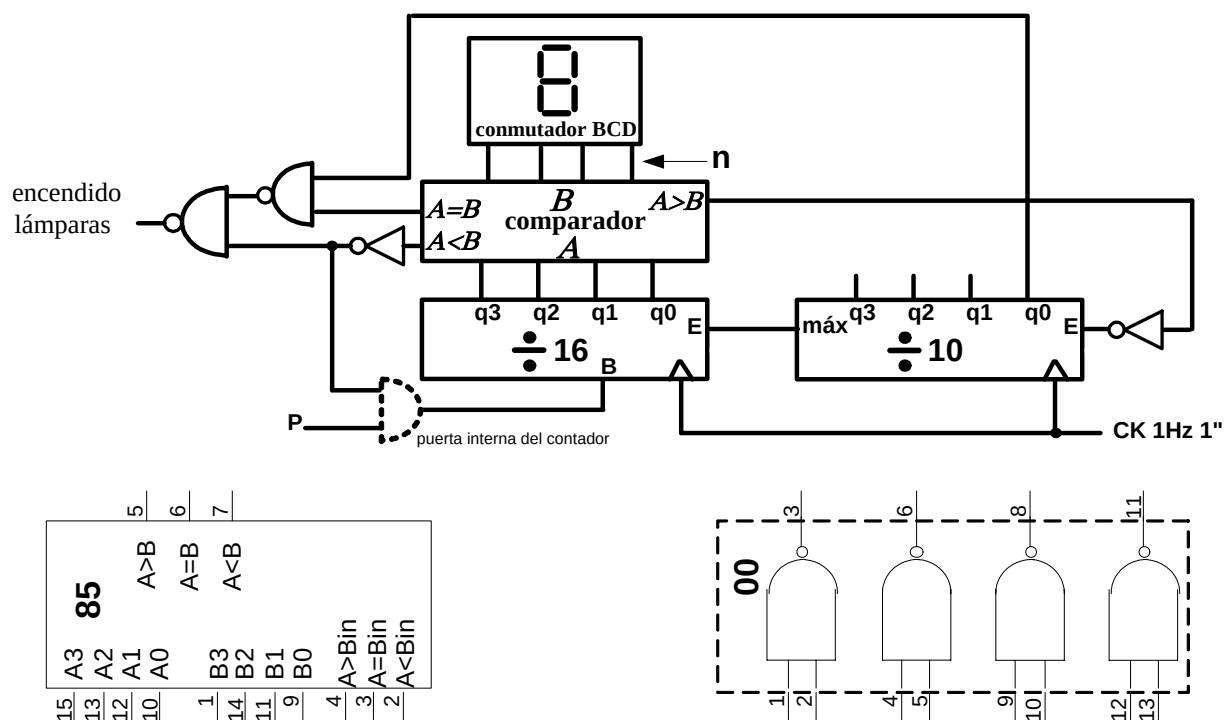
Un conmutador BCD controla las lámparas de iluminación de la escalera de una comunidad de vecinos; de forma que, si el número fijado en el conmutador BCD es n , deben mantenerse encendidas durante $n \cdot 10$ segundos cada vez que se pulsa un interruptor P ; durante los 10 segundos siguientes, deben apagarse y encenderse 5 veces sucesivas, con intervalos de 1 segundo. Cuando las lámparas están encendidas, si se pulsa P no hace nada, pero al hacerlo durante el intervalo de intermitencia, o posteriormente al mismo, se reinicia el ciclo de encendido.

A partir de un reloj de 1 Hz (1 segundo de período), un contador década determina ciclos de 10" y un segundo contador realiza el conteo de dichos ciclos y la comparación con el número **n** fijado en el conmutador BCD.

Mientras este segundo se encuentra por debajo de **n** (**n** estados, de 0 a n-1) las lámparas se mantienen encendidas; cuando ambos números son iguales, se produce la intermitencia (10") y, al superar el contador el valor **n** se detiene el conteo para evitar que «se dé la vuelta» (vuelva a 0) e inicie un nuevo ciclo de iluminación.

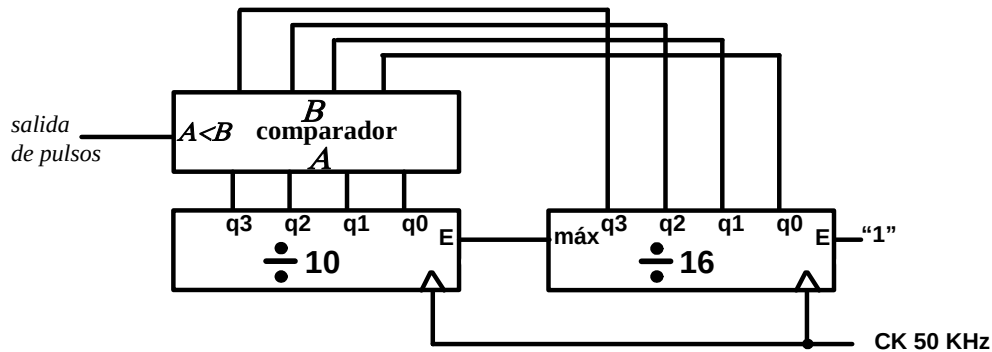
Para la intermitencia se aprovecha la primera salida (unidades) del primer contador, cuyo semiperíodo (tiempo en 1) es 1 segundo. El pulsador borra los dos contadores para comenzar una nueva temporización, pero no lo hace si las lámparas se encuentran en encendido fijo.

- Con el circuito integrado **DOBLECONTA**, configurar, en primer lugar, los dos contadores de la figura (simplemente los dos contadores módulos 10 y 16 seguidos) y comprobar el conteo mediante una señal de reloj de 5 Hz (salida TTL del generador de señales), visualizando las salidas de los contadores en los diodos LED.
- Montar, luego, el circuito completo; a fin de no tener que utilizar más de 3 circuitos integrados, utilizar puertas Nand para los inversores (un circuito integrado 74HC00 para las 2 puertas Nand y los 2 inversores). Usar como pulsador P el conmutador P₇ poniéndolo sucesivamente a **1** y a **0** y ver el conteo del segundo contador (módulo 16) en un visualizador de 7 segmentos.
- Comprobar el funcionamiento del circuito completo; el reloj debería ser de 1 Hz, pero para no tener que esperar decenas de segundos, puede aumentarse la frecuencia a 5 Hz.



2.- Generación de pulsos de anchura progresivamente creciente.

Modificando ligeramente el montaje anterior, según la figura, puede obtenerse un sencillo circuito cuya salida son pulsos de anchura creciente en la proporción de 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, ...



El contador módulo 16 genera el ciclo de modulación de anchura de pulso (ciclo on/off) y el otro contador (módulo 16) establece la referencia (numero de unidades de reloj en que la salida está en valor 1, en cada ciclo).

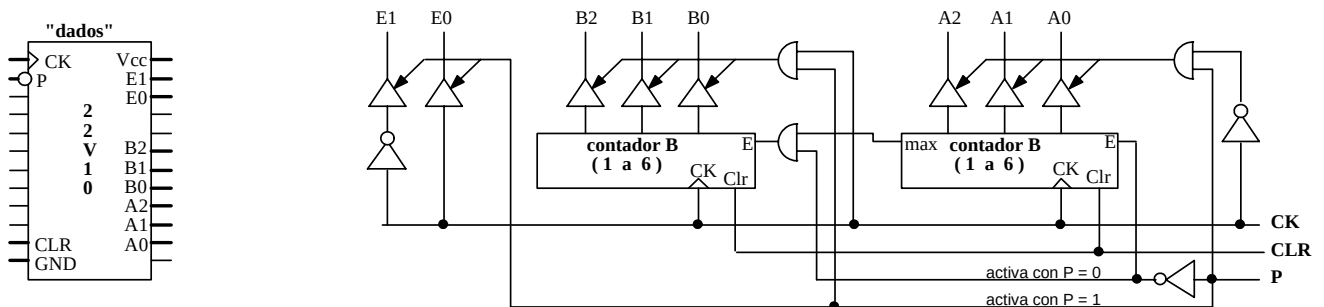
El segundo contador aumenta una unidad al final de cada ciclo, de forma que la anchura de los pulsos aumenta sucesivamente desde 0: 1, 2, ..., hasta llegar a 9, después del valor 9, el segundo contador pasa nuevamente a 0 y se repite la sucesión de pulsos de anchura creciente.

- Montar y comprobar el funcionamiento de este circuito, utilizando como señal de reloj la salida TTL del generador de señales a 50 KHz; observar los pulsos de salida en el osciloscopio y tomar en la otra sonda la salida **máx** del segundo contador, para poder sincronizar bien los pulsos de salida, cuya amplitud es variable.

3.- Visualización dinámica mediante adaptadores tri-estado: "dados de backgammon".

En otro circuito integrado programable 22V10 se ha programado el esquema de la figura que corresponde a un par de contadores sucesivos que cuentan de 1 a 6 cuando se activa el pulsador **P** (cuando **P = 0**):

- sus salidas pasan por adaptadores triestado (habilitados alternativamente con 0 y 1 del reloj) para facilitar su representación dinámica;
- otras dos salidas adicionales **E0 E1** sirven para activar adecuadamente el cátodo común de los visualizadores.
- Montar y comprobar el funcionamiento de este circuito programado en el dispositivo **22V10**, utilizando el pulsador **P₁** y un reloj de 1 KHz.



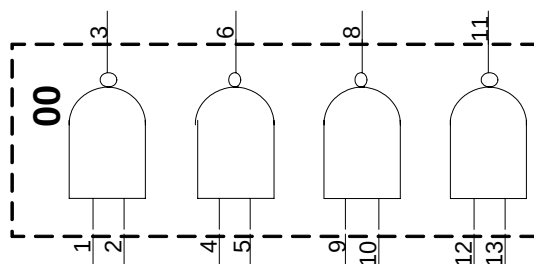
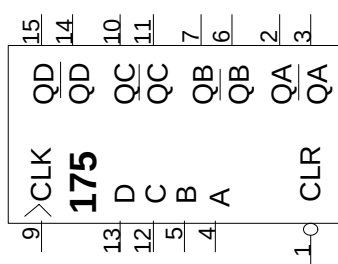
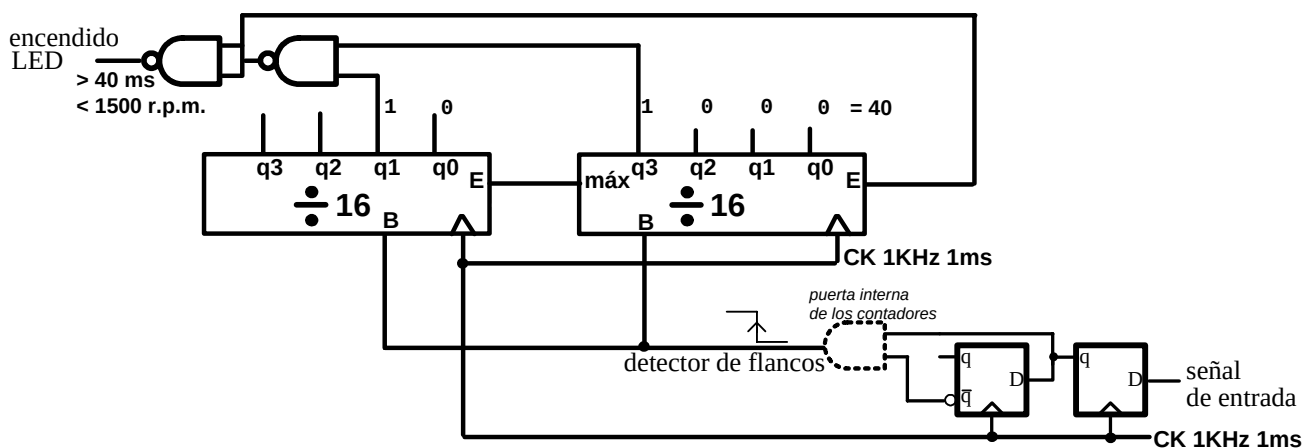
4.- Detección de frecuencia máxima.

Un circuito debe indicar si la frecuencia de una señal de entrada (por ejemplo la proveniente de un sensor de la velocidad de un motor – tacómetro –) está por debajo o por encima de 1500 r.p.m (25 Hz); para ello se mide el periodo entre dos pulsos consecutivos de la señal de entrada.

$1/1500 \text{ r.p.m.} = 60 / 1500 = 40 \text{ ms}$; $40 = \mathbf{10100}$ (reloj de 1 KHz, período: 1 ms)

El circuito corresponde a contar milisegundos entre dos pulsos sucesivos de la señal de entrada: un detector de flancos de subida borra el conteo cada vez que llega un pulso de entrada. Se detecta cuando el contador de milisegundos llega a 40 ms, en cuyo caso se detiene el conteo. El LED que visualiza la salida se enciende cuando el conteo llega o supera los 40 ms, es decir cuando la frecuencia se encuentra por debajo de las 1500 r.p.m.

- Con el circuito integrado **DOBLECONTA**, configurar y comprobar el conteo de los dos contadores de la figura.
- Montar y comprobar el funcionamiento de los dos biestables que conforman el detector de flancos, utilizando dos generadores de señal (salida TTL del generador para el reloj a 1 KHz y el generador de funciones del propio entrenador como onda de entrada a 200 Hz); comprobar que la salida del segundo biestable se encuentra retrasada un período de reloj respecto de la del primer biestable.
- Montar y comprobar el funcionamiento del circuito completo, utilizando dos generadores de señal: la salida TTL del generador de señales para el reloj a 1 KHz y la onda cuadrada del generador de funciones del propio entrenador como onda de entrada (probando frecuencias superiores e inferiores a 25 Hz).



ELECTRÓNICA DIGITAL Práctica nº 6: Biestables y circuitos secuenciales asíncronos

Objetivos:

- “Tocar” y utilizar biestables RS, biestables D y registros de retención.
- Montar y comprobar la operación de los biestables RS con los dos tipos de puertas, "nand" y "nor".
- Montar y comprobar la operación de biestables D y su agrupación para formar un registro de retención.
- Configurar sistemas secuenciales a partir de su grafo de estados.
- Montar y comprobar el funcionamiento de sendos sistemas secuenciales, uno de tipo MOORE y el otro de tipo MEALY.

Preparación de la práctica:

- 1.** Dibuja un biestable RS con puertas "nand" y escribe al lado su tabla de operación; repite dibujo y tabla, debajo, para un biestable RS con puertas "nor"
- 2.** Dibuja un biestable D construido con un RS y puertas booleanas; escribe la tabla funcional del biestable D
- 3.** Dibuja un registro de retención (*latch-memory*) construido con 4 biestables D; describe cómo actúa dicho registro
- 4.** Dibujar el grafo de estados y diseñar el siguiente circuito secuencial:
Semáforo de aviso en un cruce de vía única bidireccional con un camino: a ambos lados del cruce y a una distancia adecuada hay sendos detectores de paso de tren a y b; los trenes circulan en ambas direcciones y el semáforo debe señalar presencia de tren desde que este alcanza el primer sensor en su dirección de marcha hasta que pasa por el segundo sensor tras abandonar el cruce
Comparar las funciones obtenidas con las indicadas en el desarrollo de la práctica;
dibujar el circuito resultante utilizando puertas "nand" de 2 entradas para los biestables, una "nand" de 4 entradas e inversores para su marcado y borrado y la otra "nand" de 4 entr. para la salida; indicar en cada entrada y salida de las puertas el número del "pin" que se va a utilizar
- 5.** Indicar la secuencia de vectores de entrada que permite comprobar el funcionamiento del circuito anterior, expresando en cada caso, cual debe ser el estado siguiente y el correspondiente vector de salida
- 6.** Dibujar el grafo de estados como autómata de Mealy y diseñar el siguiente circuito secuencial:
Deposito para mezcla de tres líquidos: dispone de 3 electroválvulas A, B, C que controlan la salida de los líquidos y 4 detectores de nivel, siendo n1 el inferior y n4 el de llenado máximo; cuando el nivel del deposito desciende por debajo del mínimo n1 se produce un ciclo de llenado: primero con el liquido A hasta el nivel n2, luego B hasta el nivel n3 y, finalmente, C hasta completar el deposito
Dibujar el circuito resultante utilizando puertas "nor" de 2 entradas para los biestables y puertas "y" de 2 entradas e inversores para las funciones;
indicar en cada entrada y salida de las puertas el número del "pin" que se va a utilizar
- 7.** Indicar la secuencia de vectores de entrada que permite comprobar el funcionamiento del circuito anterior, expresando en cada caso, cual debe ser el estado siguiente y el correspondiente vector de salida
- 8.** Repetir el diseño del apartado **6.**, con el grafo de estados en forma de autómata de Moore; dibujar el circuito resultante utilizando solamente puertas "nor" de 2 entradas e inversores; indicar en cada entrada y salida de las puertas el número del "pin" que se va a .

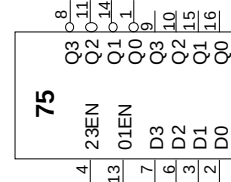
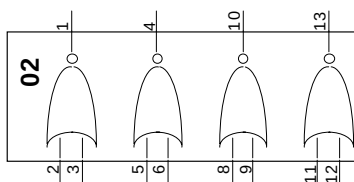
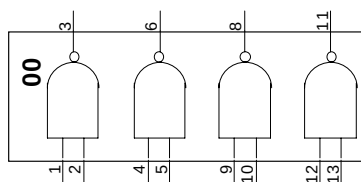
NOTA:

1. La finalidad de los apartados 4 a 8 de esta página, que corresponden a los apartados 3, 4 y opcional del desarrollo de la práctica es diseñar, montar y comprobar circuitos secuenciales de tipos Moore y Mealy; los enunciados de los mismos pueden ser cambiados por otros, caso de que algún grupo de prácticas desee diseñar y montar circuitos secuenciales diferentes de los aquí propuestos.

Desarrollo de la práctica:

1.- Biestable R-S.

- Configurar un biestable RS con puertas *Nor* (7402) y otro con puertas *Nand* (7400); comprobar su funcionamiento, utilizando como entradas los conmutadores P₄-P₅, P₆-P₇ y visualizando sus salidas en diodos LED. Verificar las diferencias entre estas dos configuraciones del biestable RS.



Tensiones de alimentación: 00 y 02 : +5 V 14
0 V 7

75 : +5 V 5
0 V 12

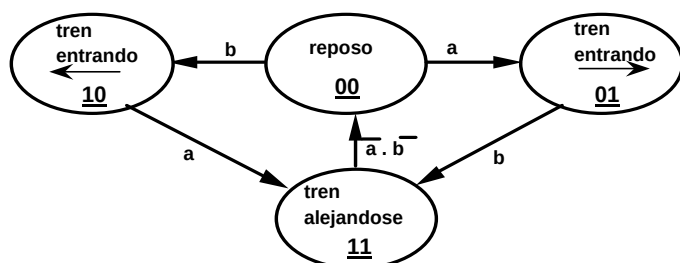
2.- Biestable D y registro de retención ("latch memory").

- Comprobar el funcionamiento de uno de los biestables D contenidos en el circuito integrado 7475, comprobando la adquisición de dato cuando E = 1 y la retención del mismo cuando E = 0.
- Utilizando dicho circuito integrado 7475 como registro de retención de 4 bits (las dos entradas de habilitación juntas), comprobar el proceso de transferencia de datos de las entradas D a las correspondientes salidas Q y la retención de los mismos durante el intervalo en que la habilitación se encuentra anulada; puede emplearse como pulso de habilitación el proporcionado por el pulsador P₁ (téngase en cuenta que es un pulsador "normalmente en 1") y los conmutadores P₇ - P₄ para los datos de entrada, visualizando las salidas en diodos LEDs.
- Observar lo que sucede modificando los datos de las entradas con la entrada de habilitación a "0" y con ella a "1".

3.- Diseño secuencial: grafo de estados. (Quien desee montar otro sistema secuencial diferente puede hacerlo).

Semáforo de aviso de paso de tren en un cruce de vía única bidireccional con un camino: la vía posee, a ambos lados del cruce y a una distancia adecuadamente grande, sendos detectores de paso de tren a y b; los trenes circulan por ella en ambas direcciones y se desea que el semáforo señale presencia de tren desde que este alcanza el primer sensor en su dirección de marcha hasta que pasa por el segundo sensor tras abandonar el cruce.

Supondremos, para simplificar el diseño, que ambos sensores a y b no se activan nunca a la vez, lo cual se corresponde bien con la situación real; en tal caso, el grafo de estados y las funciones de "marcado" y "borrado" de las variables de estado y de activación de la salida pueden ser las siguientes:



$$q_1 : S_1 = a$$

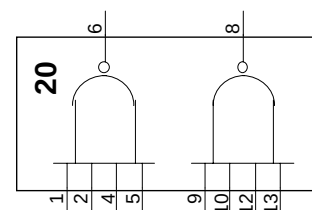
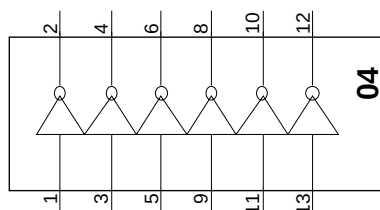
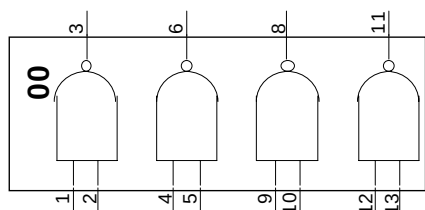
$$R_1 = \bar{a} \cdot \bar{b} \cdot q_2 \cdot q_1$$

$$q_2 : S_2 = b$$

$$R_2 = \bar{a} \cdot \bar{b} \cdot q_2 \cdot q_1$$

$$\text{semáforo} = q_2 + q_1$$

- Para construir el circuito electrónico correspondiente a este sistema secuencial realizaremos los siguientes pasos:
 - montar y comprobar dos biestables RS con puertas *Nand* (uno con las dos puertas de la parte inferior del circuito integrado y el otro con las dos puertas de la parte superior)
 - con dichos biestables, configurar el circuito correspondiente a este grafo de estados, utilizando para las funciones de marcado y borrado solamente dos circuitos integrados: 7420 (puertas *Nand* de 4 entradas) y 7404 (inversores)
 - configurar la función de salida (semáforo), aprovechando las puertas sobrantes de los anteriores circuitos integrados.
- Comprobar que el funcionamiento del circuito secuencial resultante se ajusta al enunciado y al grafo, visualizando, tanto la salida del circuito, como las variables de estado, sobre diodos LED.



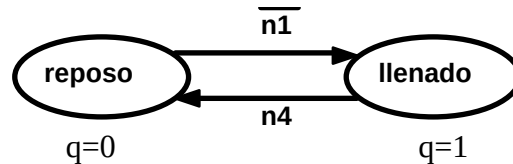
Tensiones de alimentación: +5V 14
0V 7

(continúa en la siguiente página)

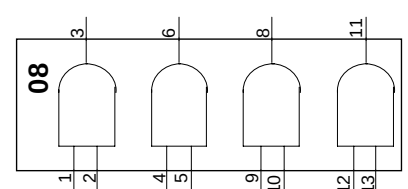
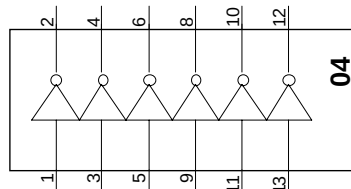
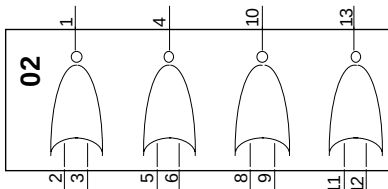
4.- Otro grafo de estados: tipo MEALY (Quien desee montar otro diferente puede hacerlo).

Un deposito se llena con una mezcla de tres líquidos diferentes, para lo cual dispone de tres electroválvulas **A, B, C** que controlan la salida de dichos líquidos y de cuatro detectores de nivel **n1, n2, n3, n4** siendo **n1** el inferior y **n4** el de llenado máximo. Solamente cuando el nivel del deposito desciende por debajo del mínimo **n1** se produce un ciclo de llenado: primero con el liquido A hasta el nivel **n2**, luego el liquido B hasta el nivel **n3** y, finalmente, C hasta completar el deposito **n4**.

Utilizaremos para los detectores de nivel los cuatro conmutadores $P_7 - P_4$ y para visualizar las electroválvulas tres diodos LED; el grafo de estados, como autómata de Mealy, será el siguiente:



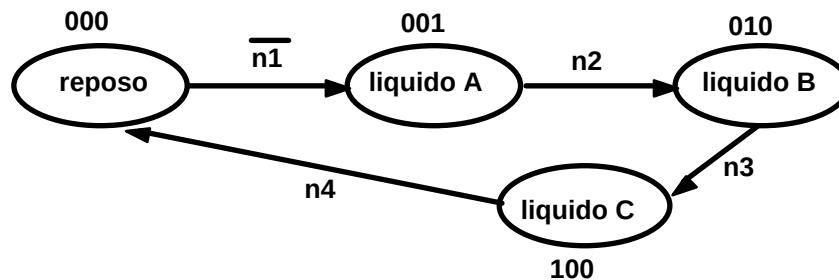
- Para construir el circuito electrónico correspondiente a este sistema secuencial realizaremos los siguientes pasos:
 - montar y comprobar el biestables RS con puertas *Nor*
 - con dicho biestable, configurar el circuito correspondiente al anterior grafo de estados.
- Construir las funciones de salida , utilizando puertas "y" de 2 entradas (7408) e inversores (7404).
- Comprobar que el funcionamiento del circuito resultante.



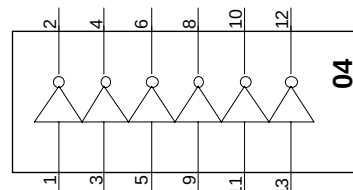
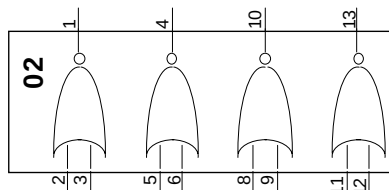
Tensiones de alimentación: +5V 14
0V 7

OPCIONAL: Se incluye una segunda versión del sistema secuencial anterior y se recomienda montarlo, aprovechado si es preciso parte de la siguiente sesión de prácticas.

El sistema anterior puede configurarse como autómata de Moore, en cuyo caso el grafo de estados será el siguiente



- Construir el circuito electrónico correspondiente, utilizando solamente puertas *Nor* de 2 entradas e inversores.
- Llevando las variables de estado (como salidas) a diodos LED, comprobar que su funcionamiento.



Tensiones de alimentación: +5V 14
0V 7

ELECTRÓNICA DIGITAL Práctica nº 7: Biestables y registros síncronos

Objetivos:

- “Tocar” y utilizar biestables y registros síncronos.
- Montar y comprobar la operación de biestables D síncronos y su borrado asíncrono.
- Agrupar 4 biestables D como registro síncrono y como registro de desplazamiento .
- Montar y comprobar un "detector de flancos".
- Montar y comprobar el funcionamiento de un registro de desplazamiento bidireccional, con carga paralelo.
- Verificar la capacidad de conversión serie-paralelo y paralelo-serie del registro de desplazamiento.
- Montar y comprobar la operación de biestables tipo JK y de biestables tipo T.

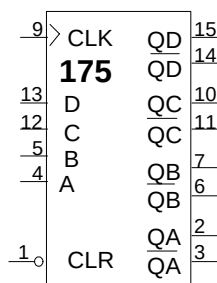
Preparación de la práctica:

- 1.** Dibujar un biestable D síncrono construido a partir de biestables D asíncronos; escribir la tabla de operación del biestable D síncrono
- 2.** Convertir el biestable D en un biestable tipo T y escribir al lado su tabla de operación
- 3.** Dibujar un registro síncrono construido con 4 biestables D; dibujar, luego, la forma de convertirlo en un registro de desplazamiento.
- 4.** Con un registro de desplazamiento de 2 bits y una puerta "o-exclusiva", configurar un "detector de flancos"; dibujar la onda de reloj, la onda de entrada (de frecuencia 10 veces inferior) y la onda de salida de ese "detector de flancos"
- 5.** Describir, gráficamente, como se puede utilizar un registro de desplazamiento como conversor serie-paralelo; idem, paralelo-serie
- 6.** Un contador Thompson o contador de anillo es aquel cuya secuencia de conteo es 0001, 0010, 0100, 1000; dibujar la forma de utilizar un registro de desplazamiento como contador de anillo
- 7.** Un contador de doble anillo cuenta 0000, 0001, 0011, 0111, 1111, 1110, 1100, 1000 ; dibujar la forma de utilizar un registro de desplazamiento como contador de doble anillo
- 8.** Con un registro de desplazamiento, ¿cómo se puede generar una onda cuyo semiperiodo en 1 sea tres veces más largo que el semiperiodo en 0?
- 9.** Dibujar un biestable JK síncrono construido a partir de un biestable D síncrono y escribir, al lado, su tabla de operación
- 10.** Dibujar 4 biestables tipo T conectados en cadena (la salida de cada uno a la entrada de pulsos del siguiente) y escribir la secuencia de estados por la que pasan al recibir pulsos en la entrada del primer biestable.

Desarrollo de la práctica:

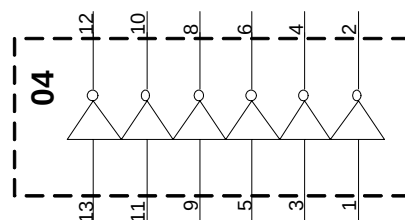
1.- Biestable D y registro.

- Comprobar el funcionamiento de un biestable síncrono D (74175), utilizando como reloj el pulso que proporciona el pulsador P_1 y visualizando su salida en uno de los diodos LED; comprobar asimismo su borrado a través de la correspondiente entrada CLR. Determinar cual es el flanco activo de la señal de reloj.
- Conectando la salida $\overline{Q}$ a la entrada del biestable, observar su comportamiento como biestable T; utilizar como reloj, en este apartado y en los siguientes, el pulsador P_0 (para evitar posibles "rebotes" al pulsar).
- Utilizando este circuito integrado como registro síncrono de 4 bits, comprobar el proceso de transferencia de datos de las entradas D a las correspondientes salidas Q y la retención de los mismos entre dos flancos del reloj (pulsador P_0); pueden emplearse los conmutadores $P_7 - P_4$ para los datos de entrada, visualizando las salidas en diodos LEDs.
- Conectando los 4 biestables de la pastilla en serie, observar su comportamiento como registro de desplazamiento.
- Con dos de los biestables y una puerta o-exclusiva, construir un detector de flancos, que proporcione un pulso de 1 μ s de duración, cada vez que una señal de entrada cambia de "0" a "1" o de "1" a "0"; utilizar como reloj (de 1 MHz) la salida TTL del generador de señales y como entrada la onda cuadrada de la placa de montajes a una frecuencia inferior.



Tensiones de alimentación:

+5 V 16
0 V 8



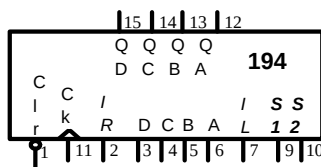
Tensiones de alimentación: +5 V 14
0 V 7

2.- Registro de desplazamiento ("shift register").

- En un registro de desplazamiento bidireccional de 4 bits (74194), comprobar su actuación como tal observando el desplazamiento en ambos sentidos; utilizar el conmutador P_0 como señal de reloj pulso a pulso y los diodos LEDs para visualizar las salidas. Pueden utilizarse los conmutadores P_2 y P_3 para las entradas de control S_1 y S_2 .
- Comprobar el borrado del registro y el proceso de carga paralelo; utilizar los conmutadores $P_7 - P_4$ para las entradas de datos en paralelo.
- Utilizar el registro como conversor serie-paralelo y paralelo-serie para la palabra binaria 1101.

Terminales de alimentación:

+5V 16
0V 8



$S_2 = 0$; $S_1 = 0$:: retención de la palabra almacenada

$S_2 = 1$; $S_1 = 0$:: desplazamiento hacia la izquierda

$S_2 = 0$; $S_1 = 1$:: desplazamiento hacia la derecha

$S_2 = 1$; $S_1 = 1$:: activación de la carga paralelo.

IR : entrada serie para el desplazamiento hacia la derecha

IL : entrada serie para el desplazamiento hacia la izquierda.

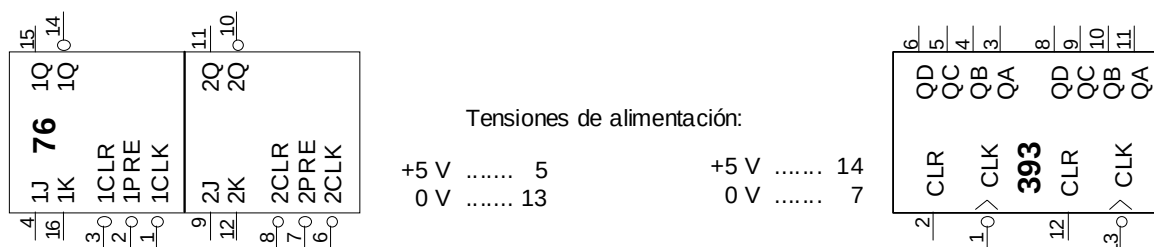
(continúa en la siguiente página)

Diversas aplicaciones del registro de desplazamiento anterior:

- Cargar la palabra 0001 y realimentando la salida del biestable más significativo a la entrada serie del registro, verificar su funcionamiento como contador en anillo módulo 4 (equivale a contar con 4 dedos, señalando cada vez uno de ellos).
- Con el registro inicialmente a 0 y realimentando la salida del biestable más significativo a la entrada serie a través de un inversor, verificar su comportamiento como contador en doble anillo módulo 8.
- Generar y observar en el osciloscopio una onda de 1KHz cuyo tiempo en "1" sea el triple de largo que el de "0"; generar una segunda onda en la que el tiempo de tiempo en "0" sea el triple de largo que en "1".

3.- Biestable J-K.

- Comprobar el funcionamiento de un biestable JK(7476), utilizando como reloj el pulso que proporciona el pulsador P_0 y visualizando su salida en uno de los diodos LED; comprobar asimismo su borrado y su puesta a "1" por las entradas asíncronas de borrado ("clear", CLR) y de marcado ("preset", PR), respectivamente.
- Conectando la salida J y K a "1", observar su comportamiento como biestable T.



4.- Biestables T y contador binario.

- Utilizar solamente uno de los dos contadores de la pastilla 74393, visualizando sus cuatro salidas sobre diodos LED; comprobar la secuencia de contejo pulso a pulso (pulsador P_0).
- Excitando dicho contador con onda cuadrada de 100 KHz, observar sobre el osciloscopio las formas de onda en sus salidas. Dibujar en papel cuadriculado las sucesivas señales de las salidas del contador (de manera que en el diagrama de ondas resultante dichas señales se correspondan verticalmente entre si: para ello, comenzar con la señal de reloj y la onda de menor frecuencia, iniciando el dibujo en la bajada de dicha onda, y observar siempre en el osciloscopio la onda anteriormente dibujada y la siguiente de menor frecuencia).

ELECTRÓNICA DIGITAL Práctica nº 8: Monostables y Astables

Objetivos:

- “Tocar” y utilizar los circuitos osciladores en onda cuadrada y temporizadores.
- Montar en laboratorio y comprobar la frecuencia de oscilación de astables RC.
- Montar en laboratorio y comprobar la frecuencia de oscilación de astables con cristal de cuarzo.
- Montar y comprobar el funcionamiento de circuitos integrados monostables.
- Comprobar la diferencia entre monostables redisparables y no-redisparables.

Preparación de la práctica:

- 1.** Dibujar el circuito de un astable RC con un inversor de entrada "Schmitt";
a partir de la formula de intervalos de tiempo de carga o de descarga
$$\Delta t = R.C . \ln \frac{V_{\infty} - V_{ini}}{V_{\infty} - V_{fin}}$$
 página 233 de “Electrónica Digital (II. Sistemas Secuenciales)”
deducir la formula del periodo de oscilación de ese circuito,
sabiendo que las tensiones de disparo "schmitt" son 2 y 3 V, respectivamente.
- 2.** Calcular los valores de R y de C para que el anterior astable oscile a 200 KHz:
1º fijar un valor de resistencia entre 500 Ω y 50 K Ω y calcular el valor del condensador
2º en el laboratorio solamente existen valores de 1, 2, 2 y 4,7 desde 10 pF a 4,7 mF,
tomar un valor de C de los disponibles en laboratorio próximo al calculado
y calcular el valor de la resistencia necesario
- 3.** Repetir los cálculos anteriores para las siguientes frecuencias: 200 KHz, 10 KHz, 500 Hz y 1 Hz
- 4.** Dibujar el circuito de un astable RC con dos inversores normales
y, a partir de la formula de intervalos de tiempo de carga o de descarga,
deducir la formula del periodo de oscilación de ese circuito,
sabiendo que la tensiones de conmutación de los inversores es la mitad de la de alimentación.
- 5.** Calcular los valores de R y de C para que el anterior astable oscile a 40 KHz
- 6.** Dibujar el circuito de un astable con dos inversores y un cristal de cuarzo:
oscilador en onda cuadrada en resonancia serie;
el primer inversor con una resistencia de realimentación (entre salida y entrada) de 10 K
- 7.** Dibujar el circuito de un astable con un inversor y un cristal de cuarzo:
oscilador en onda cuadrada en resonancia paralelo;
el inversor con una resistencia de realimentación (entre salida y entrada) de 220 K
y un circuito RC de desfase de 10 K y 10 pF
- 8.** El monostable redisparable 74HC123 tiene la siguiente formula para calcular su anchura de pulso:
$$anchura\ de\ pulso = 0,45 . R . C \quad con \quad 2K < R < 1M \quad y \quad C > 1\ nF$$

calcular los valores de R y C necesarios para anchuras de pulso de 1 μs y de 1 ms;
calcular los valores necesarios para una anchura de pulso de 5 segundos
- 9.** El monostable redisparable 74LS221 tiene la siguiente formula para calcular su anchura de pulso:
$$anchura\ de\ pulso = 0,7 . R . C \quad con \quad 2K < R < 100K \quad y \quad 100\ pF < C < 1\ mF$$

calcular los valores de R y C necesarios para una anchura de pulso de 200 μs

Desarrollo de la práctica:

1.- Astables: Osciladores con circuito RC.

- Utilizando un inversor con entrada Schmitt **74HC14** y realimentando la salida sobre la entrada a través de una red RC, comprobar su comportamiento como oscilador en onda cuadrada; montar este oscilador para las siguientes frecuencias: 200 KHz, 10 KHz y 500 Hz. Observar las ondas en la salida y en la entrada del inversor.
- Montar el circuito anterior de forma que produzca una onda cuadrada cuyo periodo sea 1 segundo y visualizar su salida sobre un diodo LED.
- Montar y comprobar un astable con dos inversores normales **7HC404** y el correspondiente circuito RC para 40 KHz.

2.- Astables: Osciladores con cristal de cuarzo.

- Montar y comprobar el funcionamiento de sendos osciladores en onda cuadrada con cristal de cuarzo: uno de ellos con resonancia serie, sin desfase en el lazo de realimentación (amplificador no inversor) y el otro con resonancia en paralelo, con un desfase de 180° en el lazo de realimentación (un circuito RC contribuye a conseguir dicho desfase).

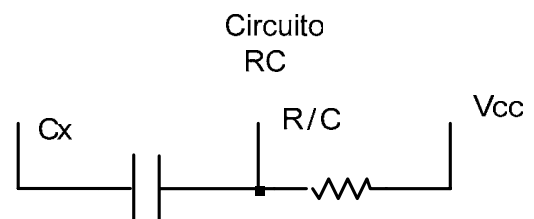
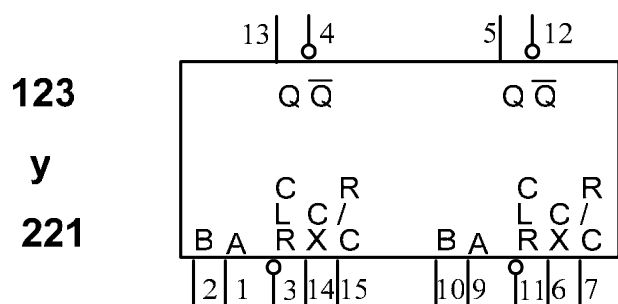
3.- Monostables.

3.1 Monostables redisparables.

- Calcular, montar y comprobar el comportamiento del monostable redisparable **74HC123** para anchuras de pulso de 1 μ s y 1 ms, utilizando para dispararlo la salida TTL del generador de señales y observando ambas ondas (señal de disparo y pulsos de salida) en el osciloscopio.
- Observar la diferencia existente al utilizar para el disparo la entrada A o la entrada B; comprobar el redisparo.
- Montar y comprobar un monostable para una anchura de pulso de 5 segundos, con disparo manual por la entrada B y visualizando la salida en un diodo LED. Utilizar para el disparo manual un pulsador externo conectado a 5 V, junto con una resistencia de 1K5 para situar la entrada a 0 V por defecto.

3.2 Monostables no redisparables.

- Verificar el funcionamiento del monostable no redisparable **74LS221** para un pulso de 200 μ s y comprobar que no admite nuevo disparo cuando este se produce dentro del mismo pulso.



Condiciones de disparo: $A = 0$ y $B = \uparrow$; $A = \downarrow$ y $B = 1$.

ELECTRÓNICA DIGITAL Práctica nº 9: Contadores

Objetivos:

- “Tocar” y utilizar contadores integrados síncronos.
- Observar las ondas de salida correspondientes al conteo y contrastarlas con las secuencias de conteo.
- Montar en laboratorio y comprobar la operación de contadores módulo m para diversos valores de m.
- Montar y comprobar contadores década para el conteo en BCD.
- Verificar la división de frecuencia que realizan los contadores.
- Utilizar un contador para control de número de pulsos (o, lo que es lo mismo, de número de unidades).

Preparación de la práctica:

- 1.** Estudiar con detenimiento y llegar a comprender el funcionamiento del circuito integrado DOBLECONTA (las características de este circuito integrado están descritas en la página anterior de este cuadernillo), en sus dos modos de funcionamiento: conteo ascendente binario y conteo descendente BCD
- 2.** Dibujar el esquema de conexiones necesario para utilizar DOBLECONTA como contador módulo 256; escribir la secuencia de conteo de los cinco primeros biestables (módulo 32) y dibujar, una debajo de otra y correlativas, las formas de onda en esos cinco biestables
- 3.** Dibujar el esquema de conexiones necesario para utilizar DOBLECONTA como contador módulo 10; escribir su secuencia de conteo y dibujar las formas de onda de ese conteo
- 4.** Dibujar el esquema de conexiones para utilizar DOBLECONTA como contador década bidireccional
- 5.** Un contador bi-quinario es un contador formado por un contador módulo 5 y otro módulo 2 sucesivos, de forma que el contador divide la frecuencia por 10 y, además, su onda de salida es cuadrada; dibujar el esquema de conexiones para utilizar DOBLECONTA como contador módulo 10 biquinario, escribir su secuencia de conteo y dibujar las formas de onda de ese conteo
- 6.** Dibujar el esquema de conexiones para utilizar DOBLECONTA como divisor de frecuencia por 12
- 7.** Dibujar el esquema de conexiones para utilizar DOBLECONTA como divisor de frecuencia por 7
- 8.** Dibujar el esquema de conexiones para utilizar DOBLECONTA como divisor de frecuencia por 9 pero de forma que "deje pasar" precisamente el noveno de cada nueve pulsos que recibe
- 9.** Dibujar el esquema de conexiones para un circuito de control de pulsos, que "deje pasar" cuatro pulsos cada vez que se active su entrada mediante un pulso P

Desarrollo de la práctica:

Para evitar perturbaciones que pudieran dar lugar a un conteo erróneo, filtrar levemente la entrada de pulsos del contador con un condensador de 100 pf, situado lo más cerca posible de las entrada de reloj y "tierra" del integrado.

1.- Contador binario.

- Unir los dos contadores módulo 16 del circuito integrado **DOBLECONTA**, contador síncrono módulo 256, anular el borrado de ambos contadores y conectar la habilitación a P₄, el "reset" a P₅ y el control de dirección "up/down" a P₇ (y este conmutador en posición de 0 V); visualizando las ocho salidas del contador resultante sobre diodos LED, comprobar la secuencia de conteo con una señal de 2 Hz (salida TTL del generador de señales).
- Excitando dicho contador con onda cuadrada de 200 KHz, observar sobre el osciloscopio las formas de onda en sus salidas (desde la de menor valor significativo hasta la de mayor valor).
- Comprobar que la salida de máximo de cada uno de los contadores es una onda de temporización (su duración corresponde a un solo ciclo de reloj), cuyo periodo es, respectivamente, 16 y 256 veces el del reloj.

2.- Décadas.

- Con uno de los dos contadores del circuito integrado **DOBLECONTA** realizar un contador módulo 10 y comprobar su funcionamiento con una onda cuadrada de frecuencia 2 Hz (salida TTL del generador de señales), visualizando sus salidas sobre diodos LED.
- Observar, también, el conteo representando sus salidas sobre un visualizador de 7 segmentos.

b) Contador bidireccional

- Variando la posición del conmutador P₇ que controla la dirección "up/down" del conteo, observar (sobre el visualizador de 7 segmentos) el conteo descendente (C = 1) y el conteo ascendente (C = 0), ambos módulos 10.
- Observar, asimismo, sobre un diodo LED la señal max/min en ambos sentidos del conteo.

c) Formas de onda contador década

- Dejando el contador módulo 10 en situación normal (C = 1: conteo ascendente), observar sobre el osciloscopio y dibujar en papel cuadriculado las ondas en las cuatro salidas del contador década, excitando su entrada con onda cuadrada de 20 KHz (cuidando que el diagrama de ondas sea correcto en cuanto a la relación de las 4 ondas entre sí, una debajo de otra). Justificar dicho diagrama de ondas.
- Observar sobre el osciloscopio y comprobar que la salida de máximo es una onda de temporización, con periodo igual a 10 veces el del reloj.

d) Contador bi-quinario

- Un contador bi-quinario es un contador módulo 10 formado por dos contadores módulo 5 y módulo 2 sucesivos; de esta forma el contador divide la frecuencia por 10 y, además, su onda de salida es cuadrada; realizar un contador de este tipo con el circuito integrado **DOBLECONTA** y comprobar su funcionamiento con una señal de reloj de 2 KHz.

3.- División de frecuencia.

- Aprovechando las tres entradas de borrado de uno de los contadores del circuito integrado **DOBLECONTA**, configurar un circuito que divida por 12 el número de pulsos que le llegan y comprobar su funcionamiento (señal de entrada de 100 KHz, observando ambas ondas de entrada y de salida en el osciloscopio).
- Configurar un segundo circuito que divida por 7 el número de pulsos que le llegan y comprobar su funcionamiento.
- Configurar otro circuito que divida por 9 el número de pulsos, pero de forma que deje pasar precisamente el noveno de dichos pulsos; comprobar su funcionamiento.

4.- Control de número de pulsos.

- Utilizando como reloj la señal TTL del generador de señales a 0,5 Hz de frecuencia, diseñar, montar y comprobar un circuito que deje pasar 4 pulsos de dicha frecuencia, cada vez que se active su entrada mediante un pulso manual P₀; visualizar la salida sobre un diodo LED.

